

IBM Datacap
Version 9

Application Development Guide



IBM Datacap
Version 9

Application Development Guide



Note

Before using this information and the product it supports, read the information in “Notices” on page 1021.

This edition applies to Version 8 Release 1 of Datacap (product number 5725-C15) and to all subsequent releases and modifications until otherwise indicated in new editions.

© **Copyright IBM Corporation 2014.**

US Government Users Restricted Rights – Use, duplication or disclosure restricted by GSA ADP Schedule Contract with IBM Corp.

Contents

About this guide xvii

Required hardware and software.	xvii
Prerequisite knowledge	xvii
ibm.com and related resources	xviii
How to send your comments.	xviii
Contacting IBM.	xix

Datacap application development. 1

Business Requirements and Application Architecture	1
Business requirements development	2
General Datacap application architecture	2
TravelDocs: Business requirements	3
Document types and page types.	4
Required document structure.	7
Fields for each page type	8
Permissible field values.	9
Business validation rules	10
Data export format	11
Datacap Studio	12
Quick tour of the user interface.	12
Starting Datacap Server	12
Opening a sample Datacap application	12
Panel organization within Datacap Studio	13
The Rulemanager tab	13
The Zones tab	14
The Test tab	15
TravelDocs: Start the TravelDocs application	15
The application framework	16
Connecting to the application	16
Document hierarchy	17
Document structure	17
Identification of page types from documents	17
Relation of the document hierarchy to the runtime batch hierarchy	18
Page type versions	18
TravelDocs: Create the document hierarchy.	19
Default document hierarchy	19
Creating document types	19
Creating page types	20
Specifying the structure of documents and pages within the batch	20
Creating data fields.	22
Specifying the structure of fields on each page	23
Sharing field definitions across the document hierarchy	23
The Datacap workflow	24
Understanding the Datacap workflow	24
Workflows, jobs, and tasks	24
Task profiles and rulesets.	26
Rulesets, rules, and actions	26
Document input	27
Electronic document input (virtual scanning)	27
Document conversion	28
Hardcopy document scans	28
Local scanning	29

Remote scanning	29
TravelDocs: Batch creation with VScan	29
Scanning the sample documents from the application images folder	29
Modifying the VScan ruleset.	30
Running VScan to generate a batch	30
Examining the files in the runtime batch folder	30
Local scanner setup (optional)	31
Creating the scan task in the Datacap Web Client	32
Creating a shortcut for the new scan task	32
Running the scan task.	33
Page Identification	33
Page identification methods	34
Fingerprint matching	34
Structure-based page identification	36
Text matching	37
Manual page identification	37
Image Enhancement	37
Goal of image enhancement	37
When to complete image enhancement	38
TravelDocs: Fingerprint library creation	38
Changing the fingerprint creation method	38
Fingerprint creation for known page types	39
Creating fingerprint classes	39
Adding individual fingerprints	40
TravelDocs: Sample fingerprint image enhancement	40
Determining appropriate image-processing settings.	40
Applying new image-processing settings to enhance the fingerprint images	41
TravelDocs: Run a batch through the workflow	42
Processing a batch	42
Runtime batch folder contents	43
Checking the confidence levels on the runtime pages	43
Rule Execution	44
Association of rules with objects	44
Example 1: Batch-level rule execution.	45
Example 2: Page-level rule execution	45
Order of rule execution	46
Example 1: Page identification rules	48
Example 2: Validation rules	48
Summary of order of rule execution	48
TravelDocs: Stepping a batch through the PageID task profile	49
Document assembly	49
Structured documents	50
Hierarchy-based documents	50
Assembling documents	51
Creation of the page data files	51
Document integrity.	52
CheckAllIntegrity action	53
Document integrity problem management	54

TravelDocs: Document creation and page file setup	55	Data Validation	79
Running a batch through the workflow	55	Validate the data	79
Contents of the runtime batch folder	55	Check data format validity	79
Page data files	56	Validate calculated fields	80
TravelDocs: Document integrity management	56	Show validation failures to an operator	82
Configuring branching	56	Use external data sources during validation	83
Running a batch with document integrity problems	57	Manage validation errors	84
Data recognition	58	TravelDocs: Update the application to complete validation	84
Page data recognition	58	Validate the currency fields	84
Identifying recognition zones by using fingerprints	58	Creating the Validate Currency Field rule	85
Recognition zone information storage	58	Adding the Validate Currency Field rule to the document hierarchy	85
Reading data from the page	59	Validate the flight cost	86
Dynamic locale support	60	Creating the Validate Flight Cost rule	86
Setting locale values	61	Adding the Flight Cost rule to the document hierarchy	86
Recognition language settings	62	Use a lookup database to validate the car type	87
Supported language codes	63	Creating the lookup database table	87
Check box options management	65	Creating the Validate Car Type rule	87
Check box recognition methods	65	Adding the Validate Car Type rule to the document hierarchy	88
Establishing parent fields	66	Creating a dictionary of valid car types	88
Setting the required variables on the parent field	67	Creating the dictionary	89
Implementing the OCR/A check box recognition method	67	Attaching the dictionary to the Car_Type field	89
Using the pixel threshold evaluation method	68	Running a batch through the workflow	89
Recognizing medical claim forms by using Autofield	69	Examination of page and field status values	90
TravelDocs: Specification of recognition zones	70	Creating recognition zones for the remaining fingerprints	92
Creating the text zones on the Rental_Agreement page	70	Running a batch through the workflow	92
Creating the OMR zones on the Rental_Agreement page	71	Page and field status codes in the TravelDocs application	93
Creating the zones for the other page types	71	Data verification	94
TravelDocs: Assignment of default rules to the document hierarchy	72	Field data verification	94
Assigning the default page level rules to new pages	72	Options for data verification	94
Assigning the default field level rules to new fields	72	Confidence levels and the page status	95
Updating the Recognize Page rule	73	Confidence levels	95
Running a batch through the workflow	73	Page status	95
TravelDocs: Updating the application to manage check box options	74	Overriding the default confidence value on specific fields	96
Setting the required variables on the Options and Insurance fields	74	Overriding validation failures	96
Specifying the check mark type	74	Skipping a verification task	97
Creating a rule to recognize the OMR fields	75	TravelDocs: Batch verification	98
Adding the Recognize OMR Fields rule to the document hierarchy	75	Setting the Car Type field to prevent overriding	99
Running a batch through the workflow	76	Batch verification with Datacap Desktop	99
TravelDocs: Using pixel threshold check box recognition (optional)	76	Creating dictionaries for check box options	99
Updating the Recognize OMR Fields rule to use RecogOMRThreshold	76	Preparing a batch for verification	100
Determining appropriate threshold and background settings	76	Opening the batch in Datacap Desktop	100
Checking the option values and obtaining the density string values	77	Reviewing the batch in Datacap Desktop	100
Interpreting the density string values	77	Submitting the batch	101
		Verifying batches with Datacap Web Client	101
		Data export	103
		Exporting data	103
		Export to a text file	103
		Configure text export for IBM Content Manager OnDemand	104
		Export to a database	104
		Export to an XML file	105
		Datacap Connector actions	105

Verifying the installation	106	Creating the recognition rules for the line items	156
Content repository authentication	106	Creating the recognition rule for the grid total	157
Integrating Connector actions into applications	107	Attaching the rules to the document hierarchy	157
Connector actions configuration	108	Running a batch through the workflow.	158
IBM Content Manager Connector actions	108	Creating rules to remove the non-line items	158
FileNet P8 Connector actions	113	TravelDocs: Validating line item grid data	159
SharePoint Connector actions	117	Validating the line item totals	159
FileNet Image Services Connector		Creating the validation rule	160
Connecting actions	123	Attaching the validation rule to the document hierarchy	160
Email Connector actions.	127	Validating the grid total	160
Fax Connector actions	133	Creating the validation rule	161
Connector actions log files	136	Attaching the rule to the document hierarchy	161
Viewing action details	137	Running a batch through the workflow.	161
TravelDocs: Exporting data to a database	137	TravelDocs: Verifying the line item grid pages	162
Configuring the export database	137	Verifying pages by using Datacap Desktop	162
Creating the ExportDB ruleset.	138	TravelDocs: Exporting line item grid data to a database	163
Adding theExportDB ruleset to the Export task profile	138	Exporting to a database	163
Attaching the Export Rental Agreement Data rule to the rental agreement page.	139	Creating the export database table	163
Running a batch through the workflow.	139	Adding rules to the ExportDB ruleset	163
TravelDocs: Exporting data to an XML file.	140	Attaching the Export Other rules to the document hierarchy	165
Creating the ExportXML ruleset	140	Running a batch through the workflow	166
Adding theExport XML ruleset to the Export task profile	141	Smart parameters	166
Attaching the Export XML rules to the document hierarchy	141	General structure of a smart parameter.	167
Running a batch through the workflow.	142	Special variables to access application configuration settings.	168
Application Debugging	143	Determining the correct key name	169
Datacap log files	143	Storing passwords, connection strings, and other parameters in the .app file	169
Enable logging for Datacap Web Client tasks	143	Reference passwords, connection strings, and other parameters from your actions	171
Rulerunner Service (RRS) log files	144	Access to the runtime hierarchy	171
Task log files	145	Examples of using special variables to access the runtime hierarchy	172
Debug your application from the Datacap Studio Test tab	145	Summary of special variables for accessing the runtime hierarchy	172
Using breakpoints	145	Use navigation elements to access the runtime hierarchy	173
Breakpoint types	146	Use other special variables	174
Setting breakpoints	146	Access job and task information	174
Disable and clear breakpoints	146	Access other information	174
Set generic breakpoints	147	TravelDocs: Exporting line item grid data to an XML file	175
Single-stepping through your code	147	Adding rules to the ExportXML ruleset.	175
Examining log files from the Test tab	148	Attaching the Export Other XML rules to the document hierarchy	177
Handling line item grids	148	Running a batch through the workflow.	177
Defining the document hierarchy for line item grids	149	Text matching	178
Rules to recognize line items	149	Identify pages with text matching	178
Text matching to locate fields	150	Locate data with text matching	179
Removing non-line items from the page data file	151	Locate simple strings.	179
Exporting data from a line item grid	152	Use regular expressions	180
TravelDocs: Adding new pages that contain line item grids	152	Text matching with keyword lists.	180
Updating the document hierarchy	152	Locate the field data	181
Adding pages to the document hierarchy	153		
Creating data fields	153		
Attaching the existing page rules to the new pages	155		
Creating the page fingerprints.	155		
Defining the recognition zones	155		
TravelDocs: Recognizing line item grid data	156		

Update the runtime data file with the recognized text	182	Analyze the Rulerunner log	209
Text matching for data recognition limitations	183	Disabling Rulerunner logging	209
TravelDocs: Update the application to use text matching	183	TravelDocs: Handle document integrity failures	210
Identifying unrecognized pages by using text matching	183	Moving document creation and integrity checking into the PageID task profile	210
Recognizing data with text matching	184	Creating the CreateDocs task	210
Attaching the rules to the document hierarchy	186	Configuring Rulerunner to run CreateDocs	211
Running a batch through the workflow.	186	Running a batch through the workflow.	211
Pattern Matching	187	TravelDocs: Identify pages manually	212
Pattern matching overview	188	Adding a function for manual page identification	213
Considerations for using pattern matching	189	Updating the Recognize Page ruleset	213
Auto registration with the FindFingerprint action	189	Adding the conditional branch to the PageID task	215
Anchor objects setup	190	Creating the ManualPageID job and task	215
Confidence level setup for pattern matching	190	Configuring branching and creating a shortcut	216
Geometric pattern matching	191	Configuring the Routing ruleset to handle manually identified pages	216
How the PatternMatch_Identify action works	191	Running a batch through the workflow.	217
Multiple anchor objects	192	Recognizing the data on the unidentified page	218
pat_RegisterZones action to adjust the positions of individual fields	193	TravelDocs: Generating fingerprints automatically	218
Text-based pattern matching	194	Creating the AutoFingerprint ruleset	219
How the pat_RecogMatch_Id action works	195	Assigning the rule to each page type	220
Determine the runtime field positions by using anchor offsets	196	Adding the ruleset to the Verify task profile	220
Field adjustment that is based on multiple anchors	196	Enabling logging for Datacap Web Client	220
TravelDocs: Use geometric pattern matching to identify pages	196	Running a batch through the workflow.	220
Setting up the pattern match anchor objects	196	Reviewing the RRS log file	221
Updating the PageID rule to use pattern matching	197	TravelDocs: Splitting a document from the main batch	222
Running a batch through the workflow.	198	Updating the Routing ruleset to split the batch	222
Reviewing the runtime batch files	198	Assigning the Batch Splitting rule to the Close element of the batch	223
Workflow automation, routing, and automatic fingerprint generation	199	Routing the split document to a supervisor	223
Use Rulerunner to automate background tasks	200	Creating the supervisor job.	223
Rulerunner overview	200	Configuring the job router	224
Rulerunner configuration	200	Configuring the supervisor shortcuts	224
Rulerunner operation.	201	Running a batch through the workflow.	225
Rulerunner logging	201	Datacap Web Client and remote scanning	226
Conditional branching and splitting to route documents	201	Moving the workflow to Datacap Web Client	226
Branching versus splitting	202	Scanning images remotely	227
Condition flags.	202	Configuring the remote scanning client.	227
Defining a condition and the associated action	203	Implementing a start panel.	228
Jobs to handle special conditions	204	Populating drop-down lists on a start panel	229
Creating a job and task	205	Running validation rules	229
Automatic fingerprint generation.	205	Remote virtual scanning.	229
TravelDocs: Automated background processing with Rulerunner	206	Verification by using the VeriFine web client	230
Defining background tasks in Datacap Application Manager	207	Restructure the batch by using the batch tree view (VeriFine)	230
Setting up background tasks in Rulerunner Manager	207	Configuring the VeriFine client	230
Enabling Rulerunner logging	207	Configuring additional VeriFine settings	232
Setting up the Job Monitor	208	Creating custom pages	233
Running a batch through the workflow.	208	Verification, page identification, and registration by using AIndex	234
		Restructure the batch by using the batch tree view (AIndex)	235
		AIndex client configuration.	235

Verifying in multiple passes	235
Storing multiple values in the runtime page data file	236
Actions that support multi-pass verification	237
Settings that support multi-pass verification	237
Example of two-pass data entry	238
Example of double-blind data entry	239
Manual page identification and registration	240
Enabling manual page registration (manual anchoring)	241
Registering a page by using manual anchoring	241
Verification by using the AVerify web client	242
Creating and using custom (static) panels	243
Exporting the default panel layout	243
Customizing the panel layout	244
Specifying the custom panels to use in a task	244
Verification by using the ImgEnter web client	245
Manual page identification and batch restructuring with ProtoId	245
ProtoId web client configuration	246
Administering an application	248
Job monitoring	248
TravelDocs: Scanning from Datacap Web Client	249
Creating a remote scan task	249
Configuring the remote scanning client.	250
Configuring the Upload task	250
Scanning and uploading a batch	251
Creating the web Job CreateDocs task	251
Configuring Rulerunner to run web jobs	252
Modifying the Verify shortcut	253
Opening the batch for verification	253
TravelDocs: Using AIndex for manual page identification and registration	253
Making a copy of the application.	253
Updating the application	254
Updating ManualPageID	255
Ignored field statuses.	255
Done field statuses	256
Done page statuses	256
Validation statuses.	256
Editing the ManualPageID settings	256
Creating the ManualIDValidate rule	257
Running a batch through the workflow.	258
Testing the ManualIDValidate rule	259
Filter batches by group in the Job Monitor	259
Defining group names for filtering batches	260
Assigning a group to a batch for filtering	261
Fingerprint Management	261
Review of basic fingerprint functionality	262
Create fingerprint files	262
Add fingerprints to the fingerprint library	262
Define field zones	263
The Fingerprint database	263
Using fingerprint XML files	264
The fingerprint XML file.	264
Enable FPXML	265

Adding fingerprints using the Datacap Studio Zones tab	265
Add fingerprints using actions	266
Exporting existing position information from the document hierarchy	266
Setting up the Fingerprint Maintenance Tool for your application	266
Exporting the position information	267
TravelDocs: Updating auto fingerprinting to use FPXML	267
Updating the AutoFingerprint ruleset	267
Updating the Recognize Page rule	268
Preparations for running a batch through the workflow.	268
Running a batch through the workflow.	268

Upgrading software and migrating applications 271

Migrating Datacap applications from 8.0.1 to 8.1.0. 273

Converting customized panels to Datacap Desktop 275

Generating the layout XML file	275
The layout XML file	275
Creating the Datacap Desktop in Microsoft Visual Studio.	276

Smart Parameter Special Variable Reference 277

Special variables for accessing the application configuration file	277
@APPPATH(<key_path>)	277
@APPVAR(<key_path>)	278
Special variables for accessing the runtime hierarchy.	279
@BATCHID	279
@ID	280
@STATUS	280
@VALUE	280
@VAR(<variable_name>)	281
@P\<field_name>[.<variable_name>]	281
@F\<field_name>[.<variable_name>]	282
@B\<field_name>[.<variable_name>]	282
@D\<field_name>[.<variable_name>]	282
@P.<variable_name>	283
@F.<variable_name>	283
Special variables for accessing job and task information	283
@JOBID	284
@JOBNAME.	284
@OPERATOR	284
@STATION	284
@TASKID	285
@TASKNAME	285
Miscellaneous special variables	285
@CHR(<unicode_value>)	286
@DATE(<format>).	286

@DCO(<property_name>)	286
@DICT_VALUE(<field>)	287
@DICT_WORD(<field>)	287
@DICT_VINDEX(<csv_string>)	287
@DICT_WINDEX(<i>csv_string</i>)	288
@EMPTY	288
@PATH(<key>)	288
@PILOT(<property_name>)	288
@PROJECTDIR	289
@PROCESSDIR	290
@STRING(<string_value>)	290
@TIME(<format>)	290
@TYPE	290

Standard Variable Reference 291

Variables that are used on all object types	291
MAX_TYPES	291
MESSAGE	291
MIN_TYPES	292
rules	292
STATUS	292
TYPE	294
hr_locale	294
Batch variables	294
LAST_RR_PROFILE	294
Document variables	295
DD	295
Page variables	295
Confidence	295
DATAFILE	295
Fingerprint Created	296
Image_Offset	296
IMAGEFILE	296
PatternConfidence	296
PD	297
ScanSrcPath	297
TEMPLATE IMAGE	297
TemplateID	297
Field variables	298
DataType	298
DensityString	299
DICT	299
Index	299
Label	299
Lookup	300
LookupEx	301
MaxLength	301
METRIC	302
MultiLine	302
MultiPunch	302
PatternMatch	303
PictureString	303
Pos<templateID>	304
Position	304
ReadOnly	305
RecogStatus	305
RecogType	305
ReqConf	305
SELECT	306
ShowChar	306
Sticky	307

Text	307
Zone_Offset	307

Application-specific variable reference 309

Medical Claims 5010 form configuration parameters	309
5010 Institutional form configuration variables	309
5010 Professional form configuration variables	320

Action library summaries 333

Global actions	333
Autodoc actions	334
BlankPagesIDBySize	335
CalculateOffset	336
CreateFingerprint	336
DeleteFingerprint	337
FindBlackFingerprint	337
FindFingerprint	338
FindTemplate	339
MergeCCOs_ByType	339
SetApplicationID	340
SetFilter_HostName	341
SetFilter_PageType	341
SetFingerprint	342
SetFingerprintDir	342
SetFingerprintFailureThreshold	343
SetFingerprintSearchArea	344
SetFingerprintWebServiceURL	345
SetMaxOffset	345
SetProblemValue	346
SetSearchArea	347
SetTemplateDir	347
UpdateFingerprintStats	348
Barcode_P actions	348
Get2DCodeBP	348
GetAllBarcodesBP	349
GetBarcodeBP	350
GetDataMatrixCodeBP	352
IdentifyByBarcodesBP	352
MatchBarcodeBP	353
MatchBarcodePrefixBP	354
ReadBarCodeBP	355
SetMinimumConfidenceBP	356
Barcode_X actions	356
GetBarCode	356
MatchBarcode	358
ReadBarCode	358
CC actions	359
FindFingerprintCC	359
SetKnowledgeBaseCC	360
SetLanguageCC	360
SetListenerURLCC	361
SetProblemValueCC	362
UpdateKnowledgeBaseCC	363
Cco2cco actions	364
NormalizeCCO	364
SetMaxCharacterHeightAVG	365
SetMaxCharacterHeightTMM	366
CMISClient actions	366
CMISCreateFolder	367

CMISDeleteFile	368	PDFImageCompression	412
CMISDeleteFolder	369	PDFImageFileExtension	414
CMISDoesFileExist	369	PDFImageFileResolution.	414
CMISDoesFolderExist	370	PDFImageUseFastBinarization.	415
CMISDownloadFile	371	Rtf actions	416
CMISLogDocumentTypes	372	RtfPrintQuality	416
CMISLogin	373	RtfTiffCompression	417
CMISRefreshClientCache	374	RtfToImage	417
CMISSetDocUploadProperty	375	Tiff actions	418
CMISSetDocUploadType	376	SplitMultipageTiff	419
CMISSetVersion	377	SplitTIFFCompression	419
CMISUploadFile	378	Txt actions	420
CMISUploadPage	379	TxtFontName	420
ColorToBW actions	380	TxtFontSize	421
C2BW_Convert.	380	TxtPrintQuality.	422
C2BW_SetAttributes	381	TxtTiffCompression	422
Convert actions.	382	TxtToImage	423
Common actions	383	Word actions	424
ExceptionSetFileTypes	383	WordDocumentToImage.	424
ExceptionSetHandler	383	WordPrintQuality	425
ExceptionSetVariableName	384	WordTiffCompression	426
ExceptionSetTaskCondition.	385	Zip actions	426
SetNamePattern	385	ZipOverwrite	427
Excel actions	386	ZipPassword	427
ExcelAutoFitColumns	387	ZipUnPack	428
ExcelAutoFitRows.	387	Dcclip actions	429
ExcelOrientationToLandscape	388	dci_clipfield	429
ExcelOrientationToPortrait	389	DCImageFix actions	430
ExcelPrintBlankPage	389	ImageEnhance	430
ExcelPrintGridlines	390	LoadSettings	431
ExcelPrintQuality	391	LoadSettings_FingerprintID	432
ExcelScalingFactor.	392	DCO actions.	433
ExcelTiffCompression.	392	ChkConfidence.	434
ExcelWorkbookToImage	393	ChkDCOStatus	434
Html actions	394	ChkDCOType	435
HtmlPrintQuality	394	ChkIntegrity.	435
HtmlTiffCompression.	395	ChkLastDCOType	436
HtmlToImage	395	ClearAltText.	437
Images actions	396	ClearDCO	437
ImageDefaultDPI	397	CopyPD2DD	438
ImageFileTypesToConvert	397	CountPagesToDocumentVar	438
ImageMonoThreshold	398	CreateDocuments	439
ImageMonoType	399	CreateFields	440
ImageToTIFF	400	DeleteFields	440
Outlook actions	401	IsDocumentCountMoreThan	441
OutlookMessageToAttachmentOnly	401	IsFirstDocumentInBatch	442
OutlookMessageToImageAndAttachment	402	JoinPreviousDocument	442
OutlookPrintQuality	403	PropagateToAltText	442
OutlookTiffCompression.	403	RemoveDocumentStructure.	443
Pdf actions	404	SetDCOStatus	444
PDFBitDepth	404	SetDCOType	444
PDFCompression	405	SetDocStatus	445
PDFConversionMethod	406	SetDocumentType	445
PDFDocumentToImage	407	SetFldConfidence	446
PDFGrayscale	407	SetPageFingerprintID.	447
PDFHorizontalResolution	408	SetPageStatus	448
PDFQuality	409	SetPageTemplateID	449
PDFVerticalResolution	409	SetPageType.	449
PdfFRE actions	410	dcpdf actions	450
PDFConversionMode.	410	dcpdf_CreateTiffFromPDF	450
PDFDocumentToImage	411	dcpdf_CreateTiffFromPDF_CreateDocs	451

dcpdf_MakePDFDoc	452	LineItem_ClearElements	494
dcpdf_MaxSizeToReconvert.	454	LineItem_ExportElements	494
dcpdf_SetApplication.	455	LineItem_SmartParameter	495
dcpdf_SetAuthor	455	NewLine	496
dcpdf_SetImageBitcount	456	PageVariable_ExportValue	496
dcpdf_SetImageCompression	457	ResetFieldVariables	497
dcpdf_SetImageGrayscale	458	SaveFilePathAsVariable	497
dcpdf_SetImageQuality	458	SetCSV	498
dcpdf_SetImageResolution	459	SetElementSeparator	499
dcpdf_SetKeywords	460	SetExportPath	499
dcpdf_SetProducer	460	SetExtensionName.	500
dcpdf_SetSubject	461	SetFileName.	501
dcpdf_SetTitle	462	SetFill	501
dcpdf_UseAltConversionMethod	462	SetFixedLength	502
Email actions	463	SetIgnoreFieldStatus	502
SendEmail	463	SetJustified	503
SetAttachment	464	SetOMR_Separator	504
SetBlindCarbonCopyRcpts	464	SetSpaceFill	504
SetCarbonCopyRcpts	465	SetZeroFill	505
SetEmailBody	465	Text	505
SetMailServer	466	Variable_ExportValue.	506
SetRecipients	467	Variable_IsValue	506
SetSender.	467	ExportDB actions	507
SetSubject	468	AddRecord	507
Equalize actions	468	ExportBatchIDToColumn	508
EqualizeUnbalancedImage	468	ExportCloseConnection	509
Ewsml actions	469	ExportFieldToColumn	510
ex_abort_time	470	ExportNodeXMLToColumn.	511
ex_done_folder.	470	ExportOpenConnection	512
ex_EMLOption	471	ExportPropertyToColumn	513
ex_ews_version.	472	ExportSmartParamToColumn	514
ex_HTTP_timeout	473	ExportToColumn	515
ex_load_properties_option	473	SetTableName	516
ex_login	474	ExportXML actions	517
ex_logout.	475	xml_CommitNode.	517
ex_max_docs	476	xml_NewNode	518
ex_problem_folder.	477	xml_SaveFile	518
ex_scan	477	xml_SetAttributeValue	519
ex_types	479	xml_SetExportPath	520
ex_wait_time	480	xml_SetFileName	520
Export actions	481	xml_SetNodeValue	521
BatchVariable_ExportValue	482	FileIO actions	521
BlankFields	482	CheckFreeDiskSpace	522
BlankLines	483	CopyDirectory	523
BPilot	483	CopyFile	524
CloseExportFile.	484	DeleteDirectory.	525
DCOPProperty	485	DeleteFile	526
DocumentVariable_ExportValue	485	GetFileSize	527
ExportAllFields.	486	GetProfileString	527
ExportField Value	487	IsDirectoryPresent	529
ExportMYValue	487	IsFilePresent.	530
ExportSmartParameter	488	IsFileReadOnly	530
ExportToBatchDir	488	IsProfilePresent.	531
Filler	489	RenameFile	532
FixedLenLJ	489	SetFileReadOnly	533
FixedLenRJ	490	SetProfileString.	534
GetDATE.	490	SplitFileName	535
GetProfileString	491	FileNetfIDM actions	536
GetTime	492	AddAllImagesToDocument.	537
LineItem_AddElement	493	AddFileToDocument	537
LineItem_BlankFields.	493	AddPDFImageToDocument.	538

AddTIFFImageToDocument	539	IBMCM_AddPages	578
CreateFolder.	539	IBMCM_CreateFolder	579
FileNetDB_ADOConnect	540	IBMCM_CreateItem	580
FileNETDocID_SaveAsSmartParameter	540	IBMCM_DeletePages	581
FileNETDocID_SetValue.	541	IBMCM_Logon.	582
GetDocuments	542	IBMCM_ReplacePage.	583
GetTopFolders	542	IBMCM_SearchItem	584
IndexProperty_ID_Component	543	IBMCM_SetAttributeValue	585
IndexProperty_ID_DateComponent	543	IBMCM_SetMimeType	586
IndexProperty_ID_Value.	544	IBMCM_SetDestinationFolder	587
IndexProperty_LeftJUSTIFY	545	IBMCM_StoreItemIDinDCO	588
IndexProperty_RightJUSTIFY	545	IBMCM_UploadDCO_DOC.	589
IndexProperty_SmartParameter	546	IBMCM_UploadDCO_Page.	590
Library_DMA_Initialize	547	ICR_C actions	591
Library_DS_Initialize	547	EnableLoggingICR_C.	591
Library_IS_Initialize	548	RecognizeFieldICR_C.	591
Library_LogIn	549	RecognizeFieldVoteICR_C	592
Library_LogOff.	549	RecognizePageFields2CCO_ICR_C	593
NewDocument	550	RecognizePageFieldsICR_C.	594
SaveDocToFolder	551	RecognizePageFieldsICR_CEx	594
Upload	551	RecognizePageICR_C.	595
Upload_SetNumAttempts	552	RecognizePageToPDFICR_C	596
UseIndexes_OFF	552	ICR_P actions	597
UseIndexes_ON	553	AddWord	597
FileNet P8 actions	554	DeleteWord	597
FNP8_CreateFolder	554	ImportCSF	598
FNP8_Login.	555	LoadFromFile	599
FNP8_MultiPageDocs	555	NewDictionary	599
FNP8_SetDestinationFolder.	556	RecognizeFieldsICR_P	600
FNP8_SetDocClassId	557	SaveToFile	601
FNP8_SetDocTitle	557	SetPostalDBPathICR_P	602
FNP8_SetFileType	558	ImageConvert actions	602
FNP8_SetKeyProperty	558	AppendAllImages	603
FNP8_SetLocale	559	AppendAllImages_ByType	603
FNP8_SetMultiValueProperty	560	AppendImage	604
FNP8_SetProperty	560	AppendImage_StartAsNew.	605
FNP8_SetPropertyEx	561	ConvertToJPEG.	606
FNP8_SetRetry	562	ConvertToTIFF	606
FNP8_SetTargetClassID	563	SetChrominanceFactor	607
FNP8_SetTargetObjectID.	563	SetDeleteOriginal	608
FNP8_SetTimeout	564	SetGrayScale	608
FNP8_SetUploadMode	564	SetLuminanceFactor	609
FNP8_SetURL	565	SetTIFFCompression	610
FNP8_UpdateProperties	565	ImageFix actions	610
FNP8_Upload	566	Imail actions.	610
FNP8_UploadDir	567	im_abort_time	611
FingerprintMaintenance actions	568	im_AcceptMixedAttachments	612
CloseDatabase	568	im_AcceptNoAttachments	612
DeleteFingerprint	569	im_done_folder.	613
DeleteFingerprints.	569	im_login	614
OpenDatabase	570	im_logout	615
SetFingerprintFolder	571	im_max_docs	615
FPXML actions	571	im_problem_folder	616
ReadZonesFPX	572	im_scan	617
SetDetailsAndLineitemPairFPX	573	im_SortByDate	618
SetDirectoryFPX	574	im_StoreEML	619
WriteZoneFPX	575	im_types	620
WriteZonesFPX	575	im_UseSSL	621
Grayscale actions	576	im_wait_time	621
ConvertGraytoBW.	577	Imprint actions	622
IBMCM actions.	577	AnnotateImage.	622

ImPrint	623	SetToDocIDMPTIFF	658
Redact.	624	SwapImages	658
RedactByRegEx.	624	SwitchMMDD	659
RedactParameters	625	UpdateFPStats	659
SetAdjustedWidth	626	ValidateVendor	660
SetFontName	627	WriteErrorMessage	660
SetFontSize	628	IOOverlay actions	661
SetOpaque	628	Overlay	661
Intellocate actions	629	SetBackgroundImage	662
iloc_AdjustZones	629	SetDitheringBackground	663
iloc_AssignPageType	630	SetHaloBackground	663
iloc_SetDetailZones	631	Locate actions	664
iloc_SetZones	631	AddKeyList	665
IsPageDataMissing	632	AggregateKeyList	666
Invoice actions	632	DefaultValue	667
AddToDetailErrorMsg	634	FilterIt.	667
AddToErrorMsg	634	FindDBList	668
AllMixedCase	634	FindDBList_InZone	668
AllowOnlyChars	635	FindKeyList	669
AlterDatebyDay	636	FindKeyList_InZone	670
CalculateNotesZone	636	FindLastKeyList	671
CaptureOpInfo	637	FindLastKeyList_InZone.	672
CheckAndFixDecimal	637	FindLastRegEx	673
CheckForSticky	638	FindLastRegEx_InZone	674
CheckFreeDiskSpace	638	FindLastRegExList.	674
ClearErrorMsg	639	FindLastRegExList_InZone	675
CreateFingerprint	640	FindLastWord	676
DetailFix	640	FindLastWord_InZone	677
DoMsgbox	641	FindNextDBList	677
ExecuteSQLBind	641	FindNextDBList_InZone.	678
FindExportImage	641	FindNextKeyList	679
FPXMLUsed.	642	FindNextKeyList_InZone	680
GenerateDetails	643	FindNextRegExList	681
iloc_SetDetailSimple	643	FindNextRegExList_InZone.	682
IncrementBatchVar	644	FindRegExList	683
IsChildFieldBlank	644	FindRegExList_InZone	684
IsChildFieldValue	645	GoAboveWord	685
IsCurrentObjValue.	645	GoBelowWord	686
IsCurrentObjVariable	646	GoDownLine	686
IsFingerPrintClass	646	GoFirstLine	687
IsInINI	647	GoFirstWord.	687
IsInList	647	GoLastLine	688
IsMultipageDocument	648	GoLastWord.	689
IsSinglePageDocument	648	GoLeftWord	689
IsStationIDSuffix	649	GoRightWord	690
IsTaskName	649	GoUpLine	691
Is_InCharSet.	650	GroupWords	691
Is_JobName	651	GroupWordsLEFT	692
Is_JobNamePrefix	651	GroupWordsRIGHT	693
LoadCCOFromField	652	IsAlpha	693
MovePDF	652	IsCurrency	694
OpenConnection	653	IsDateValue	695
ParseImageName	653	IsNumber	696
PopulateZNLineItemFieldDynamic	654	IsValue	696
ReadFPXMLZones.	655	IsValue_RegEx	697
SaveObjectVariable	655	MaxLength	698
ScanLineItemDynamic	655	MergeWordLF	698
SendOutlookNotification	656	MergeWordRT	699
SetDynamicDetailZones	656	MinLength	700
SetPicChar	657	RegExFind	700
SetStickyNo	657	RegExFind_InZone	701

RegExFindNext	702	mvscan actions	741
RegExFindNext_InZone	702	scan	742
ScanRT	703	set_abort_time	743
SelectSnippet	704	set_copy_folder	744
SetRect	705	set_delete_empty_folders	744
UpdateDCOField	705	set_folder	745
UpdateField	706	set_image_validation	746
ValueInField	707	set_max_docs	746
ValueInField_Fuzzy	707	set_metadata_types	747
ValueInField_RegEx	708	set_min_age	749
WordFind	708	set_move_wait_time	750
WordFind_InZone	709	set_multipage_burst	750
WordFindNext	710	set_problem_folder	751
WordFindNext_InZone	711	set_sort_method	752
WordFind_Offset	711	set_tree_mode	753
Lookup actions	712	set_types	753
ClearLookupResults	712	set_wait_time	754
CloseConnection	713	Maintenance Manager actions	755
ExecuteSQL	714	Application setup actions	755
OpenConnection	714	SetAdminDB	756
PopulateWithResult	715	SetApplication	756
SmartSQL	716	SetEngineDB	757
MC_Identify	717	SetPassword	758
AutoField	717	SetServer	759
FindFields	718	SetStation	759
ReadDCOSetup	719	SetupDisconnectAll	760
ReadPageSetup	719	SetupOpenApplication	761
SetFormType	720	SetupOpenApplicationEx	762
SetMaxTolerantDistance	721	SetUser	763
MC_Validation	721	Query setup actions	764
AddCenturyTo2YearDigit	722	QueryClear	764
AddToDetailErrorMsg	723	QuerySetAge	765
AddToErrorMsg	723	QuerySetBatchRange	766
CalculateHCFALineCharges	724	QuerySetBranch	766
CalculateUBLineCharges	724	QuerySetDateFormat	767
CheckDocID	725	QuerySetDateRange	770
ClearErrorMsg	725	QuerySetDateTimeFormat	771
CommonParseAddress	726	QuerySetGeneric	773
CommonValAddress	727	QuerySetJobID	774
ConvertHyphen	727	QuerySetOperator	775
FilterPID	728	QuerySetPriority	775
FormatFieldLengths	729	QuerySetSeparator	776
InheritSnippets	729	QuerySetStation	777
MC_ReadZones	730	QuerySetStatus	777
Parse31aPhSig	730	QuerySetTaskID	778
Parse58ainsnm	731	Batch processing actions	779
Parse58binsnm	731	ProcessChangeBatchStatus	780
Parse58cinsnm	732	ProcessChangeBatchStatusOrder	780
ParseConditionCodes	732	ProcessChangeBatchStatusTaskOrder	781
ParseEPSDT	733	ProcessClearAuditTable	782
ParseLastFirstIniNames	734	ProcessClearDebugTable	782
ParseNDC	735	ProcessDeleteBatches	783
PopulateFromField	735	ProcessDeleteBatchesEx	784
SetConf	736	ProcessInjectBatches	784
SetOriginalTIF	736	ProcessMoveBatches	785
StripTrailingAlpha	737	ProcessMoveBatchesEx	786
TransformLI	737	ProcessMoveDBRecords	787
UpdateCredentialList	739	ProcessResetPendingOrNotify	788
ValidateNPI	740	ProcessRunSqlQuery	790
ValProcedureCode	740	Logging actions	791
ValRequiredCode	741	LogClear	791

LogConfigure	791	SetNumberOfRetries	843
LogSendEmail	793	SetPollingInterval	844
LogWriteEventLog	794	SetProcessedFaxesFolder	844
LogWriteRecordSet	795	SetProtocol	845
LogWriteSQLQuery	795	SetRetryTimeout	846
Reporting actions	796	SetServerName	847
ReportQueryTMUsage	796	SetUserID	847
ReportSetReportingTable	797	SetUserPassword	848
ReportSetUsageDBTable	798	SetWindowsAuthentication	849
OCR_A actions	799	PatternMatch actions	850
EnableEngineLogsOCR_A	800	MatchPattern	850
OCRA_ConvertImage2BW	800	pat_RecogMatch_Id	851
RecognizeBarcodeOCR_A	801	pat_RegisterZones	852
RecognizeFieldOCR_A	801	pat_ReleasePageAnchors	853
RecognizeFieldVoteOCR_A	802	PatternMatch_Fingerprint	854
RecognizePageFieldsOCR_A	803	PatternMatch_Identify	855
RecognizePageOCR_A	804	PatternMatch_PageType	856
RecognizeToALTOOCR_A	804	SetMatchConfidence	857
RecognizeToPDFOCR_A	806	Picture actions	857
ReleaseEngineOCR_A	808	PIC_ApplyPictureString	858
RotateImageOCR_A	808	PIC_FilterFields	858
SetAutoRotationOCR_A	809	PIC_FormatFields	859
SetConfCalculationParamsOCR_A	810	PIC_ReplaceBlankField	861
SetFastModeOCR_A	810	PIC_SetPictureCharacter	862
OCR_N actions	811	PIC_ValidateField	863
RecognizePageFieldsOCR_N	811	POLR actions	864
RecognizePageOCR_N	812	CallPOLR	864
OCR_S actions	813	Recog_Shared actions	865
RecognizeDocToPDF	813	AnalyzeImage	865
RecognizeFieldOCR_S	814	CCONormalization_OFF	866
RecognizeFieldVoteOCR_S	815	CreateTextFile	867
RecognizePageFields2CCO_OCR_S	816	IsBlankPage	868
RecognizePageFieldsOCR_S	817	RecogContinueOnFailure	869
RecognizePageOCR_S	817	RecogOMRThresh	870
RecognizePageOCR_S_2TextFile	818	RecogOMRThreshold	870
RecognizeToFile_OCR_S	819	RegisterPageFields	872
RecognizeToPDF	821	ReleaseImage	873
RotateImage	822	RotateTio	873
SetEngineTimeout	823	SetAdjustFieldToChars	874
SetFastTradeOffOCR_S	824	SetFingerprintRecogPriority	874
SetLegacyDecompositionOCR_S	825	SetFullPageRecogArea	875
OCR_SR actions	826	SetOutOfProcessRecogTimeout	876
RecognizeFieldOCR_S	826	SetRecogFailureRetryDelay	877
RecognizeFieldVoteOCR_S	826	SnapCCOtoDCO	878
RecognizePageFieldsOCR_S	827	SnapDCOtoCCO	879
RecognizePageOCR_S	828	SnapFieldtoChars	879
RecognizeToFileOCR_S	829	UseOutOfProcessRecog	880
RecognizeToPDFOCR_S	831	rrunner actions	881
RotateImageOCR_S	832	AbortOnError	882
SetEngineTimeoutOCR_S	833	CheckAllIntegrity	882
OpenTextFaxServer actions	834	CheckDocCount	883
Connect	834	CheckPageCount	883
ContinueOnConnectionError	835	DebugMode_OFF	884
ContinueOnFaxImportError	836	DebugMode_ON	884
Disconnect	837	GoToNextFunction	885
ImportFaxes	837	PilotMessage_Clear	885
SendAsFax	839	PilotMessage_Set	886
SetAbortTimeout	839	ProcessChildren	886
SetFaxRemovalAfterImport	840	rr_AbortBatch	887
SetInputFolder	841	rr_Get	887
SetMaxNumberOfFaxes	842	rr_WriteNode	888

rrAppend	889	CopyFieldToField	930
rrCompare	889	DateStampField	930
rrCompareCase	890	DeleteAllAlpha	931
rrCompareCaseLength	891	DeleteAllMiscChars	931
rrCompareNot	892	DeleteAllNumeric	932
rrCompareNotCase	893	DeleteAllPunct	932
rrCompareNotCaseLength	894	DeleteAllSysChars	933
rrCopy	895	DeleteChildType	933
rrPrepend	896	DeleteLCSpaces	934
rrSet	897	DeleteParentObj	934
SetBatchPriority	898	DeleteSelectedChars	935
SetOperatorID	898	EmptyFieldValue	935
SetReturnValue	899	FailRuleSet	936
SetStationID	899	FieldContainsValue	936
SetTaskStatus	900	FilterFieldSelectedChars	937
SkipChildren	900	FormatNumberToLocale	937
Status_Preserve_OFF	901	GetJobID	938
Status_Preserve_ON	901	HasChildOfType	939
Task_NumberOfSplits	902	InsertChars	939
Task_RaiseCondition	903	InsertDecimalPoint	940
SPExport actions	903	IsFieldCurrency	940
SP_CreateFolder	904	IsFieldDate	941
SP_Login	904	IsFieldDateEqualOrAfter	942
SP_SetContentType	905	IsFieldDateEqualOrBefore	942
SP_SetFileType	905	IsFieldDateUpToToday	942
SP_SetProperty	906	IsFieldDateWithinRange	943
SP_SetUploadMode	907	IsFieldDateWithinXDays	944
SP_SetUrl	907	IsFieldDateWithReformat	944
SP_Upload	908	IsFieldEmpty	945
SP_UploadDir	909	IsFieldFilled	945
Split actions	909	IsFieldGreaterOrEqual	946
SplitBatch	909	IsFieldHidden	946
TifMerge actions	911	IsFieldLengthMax	947
TifMerge_CheckStatus	911	IsFieldLengthMin	947
TifMerge_ExportToBatchDir	912	IsFieldLessOrEqual	948
TifMerge_MergeImages	913	IsFieldMatching	948
TifMerge_MyImage	914	IsFieldPercentAlpha	949
TifMerge_PreserveCompression	915	IsFieldPercentNonNumeric	949
TifMerge_SetFileName	915	IsFieldPercentNumeric	950
TifMerge_SetFilePath	916	IsMatchingJobID	951
TM524 actions	917	IsMaxOMRChecked	951
Validations actions	917	IsMinOMRChecked	951
AddLeadingZeros	919	IsPatternInField	952
AddPaddingToEnd	919	IsSupportedImageFile	953
AddPaddingToLeft	920	IsThisFieldEmpty	953
AddPaddingToRight	920	IsThisFieldFilled	954
AddPaddingToStart	921	IsVariableEmpty	954
AddTrailingZeros	921	IsVariableFilled	955
AllowOnlyChars	922	LeftTruncate	955
AppendFromField	922	MessageBox	955
AppendToField	923	ParseMultilineAddress	956
AssignFieldDefault	923	ParseName	957
Calculate	924	ReadCurrentObjVariable	958
CalculateDateDifference	924	ReadFieldValue	958
CalculateFields	925	ReadPageVariableValue	959
CheckSubFields	926	ReplaceChars	960
CompareFields	927	ReplaceValueAtPosition	960
ConvertFieldToCurrency	928	ResetField	961
ConvertToLowerCase	928	RightTruncate	961
ConvertToUpperCase	929	SaveAsCurrentObjVariable	961
CopyField	929	SaveAsPageVariable	962

SetIsOverrideable	962	LoadBlockCCO	995
SplitFieldValueLeft	963	LoadZones	995
SplitFieldValuePreserveEnd	963	MCCOPositionAdjust	996
SplitFieldValuePreserveStart	964	MergeZones	997
SplitFieldValueRight	964	PadZone	997
SumFields	965	PopulateZNField	998
TimeStampField	965	PopulateZNLineItemField	998
TrimSpaces	966	ReadZones	999
TruncateFromEnd	966	RegisterPage	999
TruncateFromStart	967	ScanDetails	1000
Vote actions	967	ScanDetailsByLines	1001
VoteFld	968	ScanDetailsByVSpace	1001
Vscan actions	968	ScanLineItem	1002
AddDocument	969	SetEOL	1002
CopyFile	969	SetEOL_CRLF	1003
DeleteImageFile	970	ZoneBOTTOM_ImageBottom	1003
MoveImageFileToDirectory	971	ZoneBOTTOM_LowerBound	1004
Scan	972	ZoneBOTTOM_UpperBound	1005
SearchInSubdirectory	973	ZoneImage_SaveAs	1005
SetAlternateImageNames	973	ZoneLEFT_ImageLeft	1006
SetFastMode	974	ZoneLEFT_LeftBound	1007
SetImageType	974	ZoneLEFT_RightBound	1007
SetMailSourceFolder	975	ZoneRIGHT_ImageRight	1008
SetMaxImageFiles	976	ZoneRIGHT_LeftBound	1008
SetMultiPageTiff	977	ZoneRIGHT_RightBound	1009
SetSortOrder	977	ZoneTOP_ImageTop	1009
SetSourceDirectory	978	ZoneTOP_LowerBound	1010
Web Services actions	979	ZoneTOP_UpperBound	1010
WsCompare	979	Application specific actions	1011
WsDownloadFile	980	Medical Claims actions	1011
WsExecute	980	4010Common	1011
WsGetLineItems	981	4010Institutional	1012
WsGetValue	982	4010Professional	1012
WsMessageLineItemPropertyAdd	982	5010Common	1012
WsMessageLineItemPropertySet	983	5010Institutional	1012
WsSetHeaderValue	983	5010Professional	1012
WsSetMessageLineItemProperty	984	MC_Identify	1013
WsSetMessageProperty	984	MC_Validation	1013
WsSetMessageTemplate	985	Datacap Accounts Payable actions	1014
WsSetResponseNameSpace	986	APT_Localization	1015
WsUploadFile	986	APTCustom	1016
WsUrlReplaceValue	987	ConcatLineValues	1017
WsUrlSet	987	Documents	1017
Zones actions	988	FlexID	1017
AdjustZonesToImageOffset	989	Intellocate_Learning	1017
AnchorPage	989	PageID	1018
CalculateLocalOffset	990	PreVerifySetup	1018
CreateBlockCCO	990	Redaction	1018
FindBlocks_WhiteSpace	991	Notices 1021	
FindDataBlocks	991	Trademarks	1023
FindLineItems	992	Privacy policy considerations	1023
FindRegExBlocks	992	Index 1025	
FindZoneLineItems	993		
GetZoneText	994		
InheritParentPosition	994		

About this guide

These topics will guide you through the steps involved in developing a simple Datacap application. The information combines concepts with practice - describing general techniques used in the various stages of the Datacap workflow, and then provides hands-on instructions so you can implement some of those techniques using the Datacap application development tools. If you follow the topics in sequence, you will have built a complete Datacap application that:

- Inputs a collection of page image files
- Identifies the individual pages
- Creates a set of structured documents
- Captures the data from specific fields on each page
- Runs rules to ensure the validity of the data
- Displays pages to an operator for verification
- Exports the captured data to a database or other repository

Subsequent topics explore some more advanced subjects.

In the practice sections, you create an application called TravelDocs that processes travel documents - car rental agreements, hotel receipts, and flight tickets. To follow the instructions and build the application, you must download the sample image files that are provided. These include several samples of each page type from different vendors, so you will see how to capture and verify data in a way that is independent of the location of the data on the page. You can download the sample images from the same location where you downloaded this information.

“Required hardware and software”

“Prerequisite knowledge”

“ibm.com and related resources” on page xviii

Required hardware and software

If you want to follow the steps in the hands-on practice sections, you will need the following:

- A computer with a complete installation of Datacap 8.0 or higher.
- The sample image files, as described above.
- Optional: A scanner with an ISIS (R) driver (for the section on setting up a local scanner) or a TWAIN driver (for the section on remote scanning) -- you can complete all of the other sections without a scanner.

If Datacap is not already installed, refer to the *IBM Datacap Installation and Configuration Guide* for instructions.

Prerequisite knowledge

Familiarity with the following is helpful but not required:

- Basic structured and object oriented programming concepts
- XML

ibm.com and related resources

Product support and documentation are available from [ibm.com](http://www.ibm.com)[®].

Support and assistance

Product support is available on the Web. Click Support from the product Web site at:

Datacap

http://www.ibm.com/support/entry/portal/Software/Enterprise_Content_Management/Datacap_Taskmaster_Capture

Knowledge Center

You can view the product documentation on [ibm.com](http://www.ibm.com). See Knowledge Center at http://www.ibm.com/support/knowledgecenter/SSZRWW_9.0.0/.

PDF publications

You can view the PDF files online using the Adobe Acrobat Reader for your operating system. If you do not have the Acrobat Reader installed, you can download it from the Adobe Web site at <http://www.adobe.com>.

See the following PDF publications Web site:

Product	Web site
Datacap	http://www.ibm.com/support/docview.wss?uid=swg27035774

“How to send your comments”

“Contacting IBM” on page xix

How to send your comments

Your feedback is important in helping to provide the most accurate and highest quality information.

You can use any of the following methods to provide comments:

- Add comments by using the Comments pane at the bottom of every topic in Knowledge Center.
- Send your comments by clicking the **Feedback** link at the bottom of any topic in Knowledge Center.
- Send your comments by using the online readers' comment form at <http://www.ibm.com/software/data/rcf/>.
- Send your comments by e-mail to comments@us.ibm.com. Include the name of the product, the version number of the product, and the name and publication number of the information (if applicable). If you are commenting on specific text, please include the location of the text (for example, a title, a table number, or a page number).

Consumability survey

Tell us how you feel about the value of quality content by taking the Importance of High Quality Technical Information survey at the following link:

<http://www.ibm.com/survey/oid/wsb.dll/s/ag2c1>. If you want to help IBM

make this product easier to install and use, take the Consumability Survey at the following link: <http://www.ibm.com/software/data/info/consumability-survey/>.

Contacting IBM

To contact IBM customer service in the United States or Canada, call 1-800-IBM-SERV (1-800-426-7378).

To learn about available service options, call one of the following numbers:

- In the United States: 1-888-426-4343
- In Canada: 1-800-465-9600

For more information about how to contact IBM, see the Contact IBM Web site at <http://www.ibm.com/contact/us/>.

Datacap application development

This tutorial introduces you to the concepts and tasks that help you to develop your Datacap applications. Throughout the tutorial, you develop an application to process travel documents.

“Business Requirements and Application Architecture”

“Datacap Studio” on page 12

“Document hierarchy” on page 17

“The Datacap workflow” on page 24

“Document input” on page 27

“Page Identification” on page 33

“Rule Execution” on page 44

“Document assembly” on page 49

“Data recognition” on page 58

“Data Validation” on page 79

“Data verification” on page 94

“Data export” on page 103

“Application Debugging” on page 143

“Handling line item grids” on page 148

“Smart parameters” on page 166

“Text matching” on page 178

“Pattern Matching” on page 187

“Workflow automation, routing, and automatic fingerprint generation” on page 199

“Datacap Web Client and remote scanning” on page 226

“Filter batches by group in the Job Monitor” on page 259

“Fingerprint Management” on page 261

Business Requirements and Application Architecture

The first step in developing any Datacap application is to define the business requirements.

The process of defining business requirements includes these steps.

- Identifying the types of documents the application processes
- Identifying the page types that are associated with each document type
- Deciding what data you want to capture from each page
- Specifying the business rules that determine whether the captured data is valid or not
- Determining how to manage documents that have problems, including invalid structures, unrecognizable pages, nonconforming data, or low-confidence character recognition
- Deciding how you want to export or release the data at the end of the workflow

The following topics show how to develop the business requirements for a Datacap application. They show the general Datacap application architecture so that you can begin mapping the business requirements to the application model.

“Business requirements development”

“General Datacap application architecture”

“TravelDocs: Business requirements” on page 3

Business requirements development

Before you start implementation, you need to define the business requirements through collaboration with the various stakeholders. Defining the business requirement involves examining the documents that you want to process, determining which fields to capture, and deciding what to do with captured data.

Datacap applications vary in their scale and complexity. But they all seek to capture data from structured documents, which are also known as Forms. The documents can be printed pages or electronic images, but the data on the page must be first located and then interpreted with maximum accuracy.

If you are processing various document types, you must decide whether the documents are pre-sorted or processed as a mixed batch. If they are presorted, you can simplify implementation by processing each type independently, either with a separate application or a separate workflow for each type. However, if they are processed as mixed batches, you need a more sophisticated system of page identification and document assembly.

Although the goal is to create a fully automated system, there are inevitably points at which manual intervention is required. The business requirements must specify how to determine whether the information is accurate and what to do when there is a problem. After you defined the business requirements, you can design the application.

The tutorial does not provide a detailed procedure for determining business requirements. Instead, the tutorial presents the general Datacap application architecture and then examines the documents to process as you develop the TravelDocs application. This sample application is designed to demonstrate basic techniques that implement the main steps in the Datacap application workflow.

General Datacap application architecture

Datacap applications are designed to scan, process, and verify the data in your documents.

Although each Datacap application is different, most include seven basic steps.

Table 1. Flow chart of seven basic steps of an application, from page input to data export

Application step	Description
Page input	Scan a batch of hardcopy pages or import electronic documents into your application. The output from this stage is a <i>batch</i> of individual TIFF image files. Each page is initially assigned the page type <i>Other</i> .

Table 1. Flow chart of seven basic steps of an application, from page input to data export (continued)

Application step	Description
Page identification	Perform image enhancement to improve the image quality. Then, determine each page type, automatically or by displaying it to an operator for manual identification if necessary. The goal is to identify the page type, but not a variant (for example, an airline ticket, but not a ticket from a specific airline).
Document assembly	Organize the individual page files into a document according to predefined document definitions (for example, a form might have two required pages and an optional attachment). Run document integrity confirmation to ensure that each document satisfies the rules for that document type.
Data recognition	On each page, locate the data fields for that page type (for example, an airline ticket contains a passenger name, a departure airport). Then, use a Datacap recognition engine to obtain the character data for each field. The recognition engine indicates the degree of confidence for each character.
Data validation	Check the validity of specific fields. For example, you can check for valid dates, valid field formats, and correct totals. You can also complete searches to ensure that a state abbreviation is valid, or a purchase order number matches an item in a purchase order database.
Data verification	Display low-confidence data and fields that failed validation to an operator for verification, correction, and exception handling. When the operator submits the batch, the application runs the validation rules again to ensure that all data satisfies the validation criteria.
Data export	Export the data or document images to a text file, an XML file, a database, a Document Management system, or the next stage in a workflow.

TravelDocs: Business requirements

Before you develop the application, review the documents and pages that the application processes, identify the fields to capture, and determine the other business requirements.

Throughout the tutorial, you are developing an application to process travel documents. The tutorial demonstrates the general techniques for implementing each of the basic steps in the application workflow (document input, page identification, validation, export).

“Document types and page types” on page 4

- “Required document structure” on page 7
- “Fields for each page type” on page 8
- “Permissible field values” on page 9
- “Business validation rules” on page 10
- “Data export format” on page 11

Document types and page types

The documents that you use in TravelDocs are simplified versions of typical travel-related documents that might be submitted with an employee expense report.

These documents include car rental receipts, hotel receipts, and air tickets. The document types and page types are summarized in the table.

Document type	Page types
Car Rental	Rental Agreement
	Optional Insurance
Hotel	Room Receipt
	Meals
	Other Charges
Flight	Air Ticket

To consider each document type, you need to look at the sample images that are installed in \datacap\traveldocs\images folder.

Car Rental

The car rental documents have one required page and one optional page. Initially, the application supports documents from three car rental companies: Car Rental #1, Car Rental #2, and Car Rental #3.

The three sample rental agreement pages in the \datacap\traveldocs\images folder are Car1.tiff, Car3.tiff, and Car5.tiff. The fields include the data that you want to extract. These fields are common to all pages, although the position of each field is different for each page.

Vendor: Car Rental #1
 Pickup Date: Mon, Oct 4, 2010
 Pickup Location: New York (JFK)
 Return Date: Fri, Oct 8, 2010
 Return Location: New York (JFK)
 Car Type: Full size
 GPS ▪ Child Seat ☐ Fuel Service ▪
 Total Cost: \$582.77

Vendor: Car Rental #2
 Pickup Date: Sun, Aug 1, 2010
 Pickup Location: Los Angeles (LAX)
 Return Date: Fri, Aug 6, 2010
 Return Location: Los Angeles (LAX)
 Car Type: Luxury
 GPS ▪ Child Seat ▪ Fuel Service ▪
 Total Cost: \$503.39

Vendor: Car Rental #3
 Pickup Date: Sun, Oct 24, 2010
 Pickup Location: Chicago (ORD)

Return Date: Fri, Fri, Oct 29, 2010
Return Location: Chicago (ORD)
Car Type: Compact
GPS ☐ Child Seat ☐ Fuel Service ☐
Total Cost: \$535.18

The three sample optional insurance pages in the \datacap\traveldocs\images folder are Car2.tif, Car4.tif, and Car6.tif. The fields, for which you want to extract the data, are shown in these examples

Vendor: Car Rental #1
CDW: ■
PAI: ☐
PEP: ☐
ELP: ☐
Total Cost: \$104.95

Vendor: Car Rental #2
CDW: ☐
PAI: ☐
PEP: ☐
ELP: ☐
Total Cost: \$0.00

Vendor: Car Rental #3
CDW: ■
PAI: ☐
PEP: ☐
ELP: ☐
Total Cost: \$137.94

As with the rental agreement pages, the fields are common to all pages, but the position of each field is different for each variant.

Hotel

The hotel documents have one required page and two optional pages. Initially, the application supports documents from three hotel chains: Hotel #1, Hotel #2, and Hotel #3.

The three sample room receipts in the \datacap\traveldocs\images folder are Hotel1.tif, Hotel2.tif, and Hotel3.tif. As with the car rental pages, these fields are common to all pages, although the positions of the fields are different for each page.

Vendor: Hotel #1
Arrival Date: Sept 24, 2010
Departure Date: Sept 26, 2010
Total Cost: \$215.33

Vendor: Hotel #2
Arrival Date: Oct 14, 2010
Departure Date: Oct 16, 2010
Total Cost: \$282.51

Vendor: Hotel #3
Arrival Date: Sun, Oct 24, 2010
Departure Date: Tues, Oct 26, 2010
Total Cost: \$256.83

The following samples are the optional hotel pages Hotel4.tif and Hotel5.tif.

Vendor: Hotel #3
Item
Date: 10-24-10
Description: Dinner
Cost: \$48.81


```

Item
    Date: 10-25-10
    Description: Breakfast
    Cost: $12.28
Item
    Date: 10-25-10
    Description: Dinner
    Cost: $46.41
Item
    Date: 10-26-10
    Description: Breakfast
    Cost: $12.28
Total Cost: $119.78
Vendor: Hotel #3
Item
    Date: 10-24-10
    Description: Internet
    Cost: $5.95
Item
    Date: 10-25-10
    Description: Laundry
    Cost: $14.00
Item
    Date: 10-25-10
    Description: Internet
    Cost: $5.95
Item
    Date: 10-26-10
    Description: Parking
    Cost: $52.35
Total Cost: $78.25

```

Flight

Flight documents have one required page and no optional pages. Initially, the application supports documents from three airlines: Airline #1, Airline #2, and Airline #3.

The three sample air ticket pages in the \datacap\traveldocs\images folder are Flight1.tif, Flight2.tif, and Flight3.tif. As with the other pages, these fields are common to all pages, although the positions of the fields are different for each page.

```

Vendor: Airline #1
Outbound From: New York/Newark (EWR)
Outbound To: San Francisco (SFO)
Outbound Date: 24JUL10
Return From: San Francisco (SFO)
Return To: New York/Newark (EWR)
Return Date: 28JUL10
Airfare: 760.27
Taxes: 64.56
Total Cost: 824.83

Vendor: Airline #2
Outbound From: Chicago (ORD)
Outbound To: Atlanta (ATL)
Outbound Date: MON OCT 25, 2010
Return From: Atlanta (ATL)
Return To: Chicago (ORD)
Return Date: WED OCT 27, 2010
Airfare: $385.27
Taxes: $44.76
Total Cost: $430.03

```

Vendor: Airline #3
Outbound From: ORD Chicago
Outbound To: BOS Boston
Outbound Date: OCT 26, 2010
Return From: BOS Boston
Return To: ORD Chicago
Return Date: OCT 29, 2010
Airfare: 233.00 USD
Taxes: 21.40 USD
Total Cost: 254.40 USD

Required document structure

When you examined each travel document, you identified pages that are required and optional.

For example, in car rental documents:

- The rental agreement page is required.
- The insurance page is optional.

For travel documents with multiple pages, there might be requirements for the number or order of pages of each document type. This table summarizes the structure of each travel document type.

Document Type	Page Type	Number	Required?	Order
Car Rental		Any number per batch	No	Any position within batch
	Rental Agreement	One per document	Yes	Must be first in document
	Optional Insurance	One per document	No	Must be second in document
Hotel		Any number per batch	No	Any position within batch
	Room_Receipt	One per document	Yes	Must be first in document
	Meals	Any number per document	No	Cannot be first in document
	Other_Charges	Any number per document	No	Cannot be first in document
Flight		Any number per batch	No	Any position within batch
	Air_Ticket	One per document	Yes	Must be first in document

This structural information is an important element of the design requirements that you use when you implement the application's document hierarchy. When you implement the document assembly stage of the workflow, you use this information to determine whether the pages in the batch meet the structural requirements.

The assumption for the sample application is that you are entering batches of mixed travel documents with multiple, consecutive pages that are in the correct order. For example, a batch might include any number of car rental documents, flight documents, and hotel documents. Also, the pages within each document are consecutive and in the correct order. If the batch meets the structural requirements, then the application assembles the documents automatically. However, if the batch contains orphan pages or pages that do not meet the rules for document integrity, then operator intervention is required.

In the following example, the batch does not contain any errors, and no operator intervention is required.

Page type	Page type	Page type	Page type	Page type	Page type	Page type	Page type	Page type
Rental Agreement	Optional Insurance	Air Ticket	Room Receipt	Room Receipt	Meals	Rental Agreement	Optional Insurance	Air Ticket

In this second example, the batch contains three errors and requires operator intervention.

Page type	Page type	Page type	Page type	Page type	Page type	Page type	Page type	Page type
Optional Insurance (1)	Room Receipt	Room Receipt	Air Ticket	Meals (2)	Rental Agreement	Optional Insurance (3)	Optional Insurance (3)	Air Ticket

1. Orphaned optional insurance page must follow a rental agreement page.
2. Orphaned meals page must follow a room receipt page.
3. Two optional insurance pages are not allowed in a Car Rental document.

Fields for each page type

When you examined each sample page, you identified the fields of interest.

You noted that each variant of a page type includes all of these fields, but the position of each field is different for each variant. This list summarizes the fields that you need to capture for each page type.

Car Rental document:

- Rental Agreement page type fields:
 - Vendor
 - Pickup_Date
 - Pickup_Location
 - Return_Date
 - Return_Location
 - Car_Type
 - Options
 - Nav_System
 - Child_Seat
 - Fuel_Service
 - Total_Cost
- Optional Insurance page type fields:
 - Vendor
 - Collision Damage Waiver
 - CDW_Option
 - Personal Accident Insurance
 - PAI_Option
 - Personal Effects Protection
 - PEP_Option
 - Extended Liability Protection
 - ELP_Option
 - Total_Cost

Hotel document:

- Room Receipt page type fields:

- Vendor
- Arrival_Date
- Departure_Date
- Total_Cost
- Meals page type fields:
 - [Item]
 - Date
 - Description
 - Cost
 - Total_Cost
- Other Charges page type fields:
 - [Item]
 - Date
 - Category
 - Cost
 - Total_Cost

Flight document:

- Air Ticket page type fields:
 - Vendor
 - Outbound_From, Outbound_To, Outbound_Date
 - Return_From, Return_To, Return_Date
 - Airfare
 - Taxes
 - Total_Cost

The two car rental pages both include check box options. There is a requirement in Datacap that each check box option is the child of a parent container field. On the Rental Agreement page, the three options are each a child of the same parent field. On the Optional Insurance page, each option has its own parent. The implementation is a little different depending on which method is used. So, this tutorial uses one of each method to demonstrate both techniques when you complete the implementation. The choice is more an implementation decision than a business decision, although it does affect the format of the export data.

Second, the optional hotel pages include repeating line items, each with the same structure. You do not know in advance how many items might be on a page. Datacap includes functionality for handling line item grids that are introduced in the topic “Handling line item grids” on page 148.

Permissible field values

You can specify field values for your business requirements.

In addition to specifying the fields, the business requirements might specify permissible formats and values for each field.

Page	Field	Permissible values
Rental Agreement	Vendor	Any text
	Pickup_Date	Any valid date format
	Pickup_Location	Any text

Page	Field	Permissible values
	Return_Date	Any valid date format
	Return_Location	Any text
	Car_Type	Compact, Standard, Full size, SUV, or Other
	Options	Checkbox fields - selected or not selected
	Total_Cost	Any currency format (\$999.99, 999.99, and 999.99 USD are valid)

Business validation rules

First, you define the structure of each document type and the fields that you want to capture from each page. Then, you define how you want to validate the captured data to determine whether the data meets the business requirements.

For simplicity purposes in the sample application, you validate some of the fields only. You selected these fields specifically to demonstrate a few generic but commonly used techniques when you implement the data validation stage of the application workflow.

Table 2. Validation rules for sample application fields

Page	Field	Validation rule
Rental Agreement	Total Cost	Is the field value in a valid currency format? Specifically, is the field numeric with a two-digit decimal portion?
Optional Insurance	Total Cost	Is the field value in a valid currency format? Specifically, is the field numeric with a two-digit decimal portion?
Room Receipt	Total Cost	Is the field value in a valid currency format? Specifically, is the field numeric with a two-digit decimal portion?
Meals	Total Cost	Is the field value in a valid currency format? Specifically, is the field numeric with a two-digit decimal portion?
Other Charges	Total Cost	Is the field value in a valid currency format? Specifically, is the field numeric with a two-digit decimal portion?
Air Ticket	Air Fare	Is the field value in a valid currency format? Specifically, is the field numeric with a two-digit decimal portion?
	Taxes	
	Total Cost	
Rental Agreement	Car Type	Is the field value one of the following values: Compact, Standard, Full size, SUV, or Other?

Table 2. Validation rules for sample application fields (continued)

Page	Field	Validation rule
Air Ticket	Air Fare	Does the value of the Air Fare field plus the value of the Taxes field equal the value of the Total Cost field?
	Taxes	
	Total Cost	

A validation failure does not necessarily mean that the original page contains invalid data. It might mean that the recognition engine failed to recognize one or more characters correctly. Whatever the reason for the error, the application developer can set the page status to ensure that the page is displayed to an operator for verification.

Data export format

The last stage in developing the business requirements for the TravelDocs application is to specify the format of the captured data for export.

Datacap can export data to a text file, an XML file, a database, a Document Management system, or the input stage of another business application.

This example use case, which exports data only and does not export images, is not typical, and is used for simplicity. Almost all Datacap applications export images and documents together with captured data. In most Document Management systems, the captured data is stored in metadata or index fields that are associated with each document.

For TravelDocs, you specify that data is to be exported to a Microsoft Access database and also saved in XML format. To simplify the implementation, you export only the rental agreement page data initially:

- For the database export, the application must export the data from each rental agreement page as a single record.
- For the XML export, all rental agreement pages in the same batch are written to a single XML file.

```
<?xml version='1.0' ?>
<Rental_Agreements>
  <TM000001>
    <Pickup_Date>Tues, Dec 7, 2010</Pickup_Date>
    <Pickup_Location>Boston (BOS)</Pickup_Location>
    <Return_Date>Fri, Dec 10, 2010</Return_Date>
    <Return_Location>Boston (BOS)</Return_Location>
    <Car_Type>Compact</Car_Type>
    <Options>Fuel Service</Options>
    <Total_Cost>$345.70</Total_Cost>
  </TM000001>
  <TM000003>
    <Pickup_Date>Mon, Dec 6, 2010</Pickup_Date>
    <Pickup_Location>San Francisco (SF0)</Pickup_Location>
    <Return_Date>Fri, Dec 10, 2010</Return_Date>
    <Return_Location>San Francisco (SF0)</Return_Location>
    <Car_Type>SUV</Car_Type>
    <Options>Child Seat</Options>
    <Total_Cost>$489.31</Total_Cost>
  </TM000003>
  <TM000004>
    <Pickup_Date>Mon, Dec 13, 2010</Pickup_Date>
    <Pickup_Location>Newark (EWR)</Pickup_Location>
    <Return_Date>Thur, Dec 16, 2010</Return_Date>
```

```
<Return_Location>Newark (EWR)</Return_Location>
<Car_Type>Other</Car_Type>
<Options>Navigation System Child Seat Fuel Service</Options>
<Total_Cost>$387.40</Total_Cost>
</TM000004>
</Rental_Agreements>
```

In future tasks, you can export some of the line item grid data too.

Datacap Studio

Datacap Studio is the Datacap application development environment that provides the tools that you need to develop and test your application.

Datacap Studio contains three main tabs: **Rulemanager**, **Zones**, and **Test**. In addition, Datacap Studio consists of an application wizard that you use to generate an application framework, which includes the supporting folder structure and control files.

“Quick tour of the user interface”

“TravelDocs: Start the TravelDocs application” on page 15

Quick tour of the user interface

Before you develop the TravelDocs application, you can review the Datacap Studio interface by opening one of the sample applications that is installed with Datacap.

You use Datacap Studio extensively to develop the sample TravelDocs application.

“Starting Datacap Server”

“Opening a sample Datacap application”

“Panel organization within Datacap Studio” on page 13

“The Rulemanager tab” on page 13

“The Zones tab” on page 14

“The Test tab” on page 15

Starting Datacap Server

Depending on how your system is configured, the Datacap Server can start automatically or you might need to start it manually.

Datacap applications use required services (such as authentication, batch creation, and assignment) that are provided by the Datacap Server, which runs in the background as a Windows service. If the server is not running, you cannot log on to Datacap.

To start the Datacap Server:

1. In the **Start** menu click **IBM Datacap Services > Datacap Server Manager**.
2. On the **Service** tab in Datacap Server Manager, verify that the status is Running.
3. If the status is not Running, click the **Start** button.
4. Confirm that the status is Running and then click **Close**. The server is now running in the background.

Opening a sample Datacap application

After you confirm that the Datacap Server is running, you can start Datacap Studio and open any of the sample applications.

To open one of the sample applications in Datacap Studio:

1. In the **Start** menu click **IBM Datacap Developer ToolsDatacap Studio**.
2. In the Select Application window, select one of the existing sample applications, and click **Next**. For example, one of the existing sample applications is TravelDocs.
3. In the Datacap Login window, ensure that the **NT authentication** check box is not selected.
4. Enter these values for the fields as shown.
 - **User ID:**admin
 - **Password:**admin
 - **Station ID:**1
5. Click **Finish**.

Panel organization within Datacap Studio

Datacap Studio contains three main tabs, including the **Rulemanager** tab, the **Zones** tab, and the **Test** tab.

Tab	Description
Rulemanager	This tab is the primary application development area.
Zones	This tab is where you create page fingerprints and configure recognition zones.
Test	This tab provides integrated execution and debugging tools for testing your application.

Each main tab contains more tabs and panes.

You can customize the workplace by reorganizing the panes, removing panes, and adding panes.

To move a pane:

1. Use the mouse to drag the pane's tab from its current location. You see a set of insertion points that are located around the window. You can move the pane to the left, right, top, or bottom of another pane, or to the left, right, top, or bottom of the window. In the center, you can combine tabs. As you move the pointer over an insertion point, you see a shaded area that indicates the corresponding location.
2. Drop the pane on an insertion point to move the pane.

To remove a pane, right-click the pane's tab and choose **Close**.

To add a pane, right-click any tab and choose **Show tabs**. Then, choose from the available panes. After you add a pane, you can move the new pane.

The Rulemanager tab

The **Rulemanager** tab contains five panels where you define document structures, rulesets, rules, functions, and task profiles.

The **Rulemanager** tab includes these panels:

Table 3. Rulemanager tab panels

Panel	Description
Document hierarchy	Defines the structure of the documents you are processing and how each element within the structure is processed (see “Document hierarchy” on page 17).
Rulesets	Defines the rules, functions, and actions that make up each ruleset (see “Rulesets, rules, and actions” on page 26).
Task profiles	Defines the rulesets that are run by each task profile (see “Task profiles and rulesets” on page 26).
Actions library	Provides access to the complete library of pre-built actions and, in some cases, custom developed actions. To get help on an action, select the action and click the Information icon.
Properties	Displays the properties for the selected document hierarchy or ruleset object. If the corresponding pane is locked for editing, you can also modify existing properties, including specifying action parameters.

The Zones tab

The **Zones** tab contains four panels where you can add fingerprints and view properties of selected objects.

The **Zones** tab includes these panels:

Table 4. Zones tab panels

Panel	Description
Fingerprints	Displays the application's fingerprint library from which you can add fingerprints for new page types (see “Fingerprint matching” on page 34).
Document hierarchy	Defines the structure of the documents that you are processing and how each element within the structure is processed. (See “Document hierarchy” on page 17.)
Properties	Displays the properties for the selected document hierarchy object. If the document hierarchy is locked for editing, you can also modify existing properties. In the Properties panel, you can specify recognition options for the selected object. Datacap supports multiple recognition engines. The Properties panel displays the ICR/C, BAR/P, and OCR/S tabs by default. You can access other tabs by right-clicking within the Properties panel and selecting Show tabs .

Table 4. **Zones** tab panels (continued)

Panel	Description
Image View	Displays the selected fingerprint image and any recognition zones. Also, you can draw new recognition zones in the Image View panel (see “Identifying recognition zones by using fingerprints” on page 58). If you created the fingerprints by using full page recognition, you can view the recognition results in the Text tab.

The Test tab

The **Test** tab contains eight panels in which you can view information and properties of batches, jobs, documents, and rulesets.

The **Test** tab includes these panels:

Table 5. **Test** tab panels

Panel	Description
Workflow	Displays the job types and tasks that are defined in the Administrator tab. Also, you can run a batch through the workflow in the Workflow panel.
Runtime batch hierarchy	When a batch is running, this panel displays the runtime batch hierarchy, including any data values. If you select a page object, the page is displayed in the Image panel.
Document hierarchy	Displays the structure of the documents that you are processing, and shows how each element within the structure is processed.
Rulesets	Displays the rules, functions, and actions that make up each ruleset. As you step through the workflow, you can see the current execution point.
Image/Text	Displays the selected page in the runtime batch hierarchy.
Batch data	Displays batch level information for the batch that is running.
Properties	Displays the properties for the selected document hierarchy or ruleset object (read only).
Breakpoints/Runtime state/Call stack	A breakpoint stops processing at a predetermined ruleset, rule, or action. For more information, see “Using breakpoints” on page 145.

TravelDocs: Start the TravelDocs application

To start the application, you first need to create the application framework with the application wizard, and then connect to the application through Datacap Studio.

“The application framework” on page 16

“Connecting to the application”

The application framework

You can create an application, copy an application, or convert an application format from a previous version by using the Datacap application wizard in Datacap Studio.

You can create or copy an application, including a CMIS-based application, when you run the application wizard in Datacap Studio. You can also convert an 8.0.1 application to a 9.0 format. You do not need to convert an 8.1 application to a 9.0 format.

Select the **Forms** or **Learning** application template.

- Select the **Forms** application template for structured images. When you know the types of data that you want to capture and where that data is on each image, select the Forms application template. For example, a 1040EZ tax form and the types of data on the form, such as name and address, are in the same location on every 1040EZ form. The Forms application template sets up a workflow that you can match against your fingerprints.
- Select the **Learning** application template for unstructured images. When you know the types of data that you want to capture but you do not know where that data is contained in the image because the location of the data is different on each image, select the Learning application template. For example, if you want to capture the date, amount, and tax for expenses from different hotels, the receipt images from each hotel are unique. The location of the data you want to capture differs for each hotel receipt image so the data cannot be identified with Datacap fingerprints. The Learning application template sets up a workflow where you can add rules, such as Locate rules, for Datacap to learn the different hotel receipt formats as they are encountered.

For images where the data is not found, the verifier is prompted to click the image and identify where the data is located. This Click N Key process populates the data into the data set so that the Learning application can automatically find the data the next time that type of image is encountered. After the unstructured hotel bill is processed, the zones are saved to capture data directly. Then, each time an unstructured image with the same format is encountered, the data is captured directly in the same way that data is captured from structured images with Forms applications.

Connecting to the application

When you create a new application by using the application wizard, the application is added to the list of applications in the Datacap Application Manager.

Before you can work with the application in Datacap Studio, you must connect to the application.

To connect to the application from Datacap Studio:

1. In Datacap Studio, click the **Connection Wizard** button.
2. Select the **MyTravelDocs** application and click **Next**.
3. Log in by using **User ID: admin**, **Password: admin**, and **Station: 1**.
4. Click **Finish**. Datacap Studio displays the new project.

Document hierarchy

Document hierarchy defines the structure of the documents that you are processing and how Datacap processes each element within the structure. Document hierarchy is also referred to as the Setup DCO.

“Document structure”

“Identification of page types from documents”

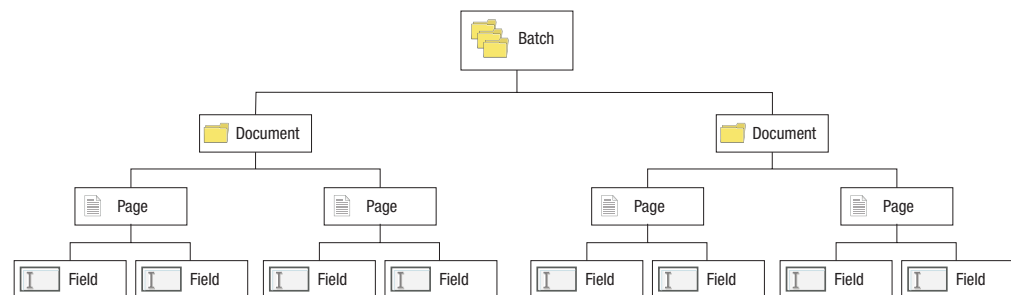
“Relation of the document hierarchy to the runtime batch hierarchy” on page 18

“Page type versions” on page 18

“TravelDocs: Create the document hierarchy” on page 19

Document structure

The document hierarchy describes the structure of the documents that your application is designed to process. The levels within the hierarchy are **batch**, **document**, **page**, and **field**.



At the top of the document hierarchy is the batch, which refers to all pages of all document types. Beneath the batch level, the document hierarchy defines:

The document types your application can process

An application can process only one document type, or multiple document types. For example, the TravelDocs sample application can process car rental documents, hotel expense documents, and flight documents.

The page types within each document type

Each document can contain only one page type or multiple page types. For example, the TravelDocs car rental document includes the rental agreement page and the optional insurance page, while the flight document has only an air ticket page.

The number and order of pages within each document type

Pages can be required or optional. For example, a car rental document has two pages at most. The rental agreement page is required and must come first; and the insurance coverage page is optional.

The data fields within each page type

Data fields can be required or optional. For example, the hotel document's **Other Charges** page has fields for expense category, number of items, unit cost, and total cost.

Identification of page types from documents

There are several techniques to identify individual page types, but the most common technique is called *fingerprint matching*.

In a typical Datacap application, documents start as a batch of unidentified image files with one image per page. A single batch might contain a mix of document types, and each document might contain a number of different page types. There is nothing within the page image that identifies the page type or any of the data on the page. In other words, the page images do not contain any structured content.

Before Datacap can begin to extract data, it must identify the individual page types. Datacap then maps pages to documents, and fields to pages, by using the information in the document hierarchy. After Datacap identifies the fields and their locations within each page, it extracts and stores the data in a structured format. The structured format is known as the *runtime batch hierarchy*.

Relation of the document hierarchy to the runtime batch hierarchy

The document hierarchy describes the general structure of the documents that your application supports in terms of document types, page types, and fields. By contrast, a runtime batch describes specific documents that contain specific pages and specific data.

The document hierarchy and the runtime batch can be described in object-oriented terms:

- The document hierarchy defines the document, page, and field **classes**.
- The runtime batch describes a set of **objects** that is built from those classes. Each object has a set of **variables** that is derived from the parent class, and each variable has a **value**.

While the document hierarchy describes a single, generalized version of each document and page type, a runtime batch can have any number of documents and pages.

In the TravelDocs application, the document hierarchy defines the three document types that include Car_Rental, Hotel, and Flight. The runtime batch might include two car rental documents, two hotel documents, and two flight documents. Each runtime document has one or more pages. Each page has the number of fields that are defined in the document hierarchy for that page type.

Page type versions

Although individual pages within a runtime batch might be of the same type, the pages might look different.

The TravelDocs runtime batch hierarchy includes two car rental documents, two hotel documents, and two flight documents. The car rental documents might be from different car rental companies. The hotel documents might be from different hotel chains, and the flight documents might be from different airlines.

For example, the TravelDocs runtime batch has two pages of type Rental_Agreement. (Review the files Car1.tif and Car3.tif in \Datacap\TravelDocs\images.) Structurally, the pages contain the same data.

Occasionally, the location of data is not in the same position on different pages. To identify the location of data, you can create a fingerprint for each variant and store the field location for each variant in the document hierarchy.

TravelDocs: Create the document hierarchy

The document hierarchy enables the Datacap application to convert a collection of unstructured images into a structured runtime batch hierarchy that contains the relevant business data.

The goal of this part of the tutorial is to create generalized definitions (classes) for the document types, page types, and fields that the application supports.

“Default document hierarchy”

“Creating document types”

“Creating page types” on page 20

“Specifying the structure of documents and pages within the batch” on page 20

“Creating data fields” on page 22

“Specifying the structure of fields on each page” on page 23

“Sharing field definitions across the document hierarchy” on page 23

Default document hierarchy

The Datacap Studio application wizard creates a default document hierarchy that you can use as a starting point.

The default hierarchy includes these objects:

- A batch node that has the same name as the application
- A page type **Other**, which is the default type that Datacap assigns to all pages before page identification
- A default document type called **Document**
- A default page type called **Page**
- One default field that is called **Field** and is associated with the page type **Page**

In the document hierarchy, the **Open** and **Close** nodes define the rules that are assigned to each element within the hierarchy. For example, the **Open** node beneath the page type **Other** defines the rules and actions that Datacap starts when it begins processing a page of type **Other**.

Creating document types

The business requirements specification for the TravelDocs application defines three document types that include Car rental, Hotel, and Flight.

You begin by adding these document types to the hierarchy.

To add these document types to the hierarchy:

1. In the Document Hierarchy pane, click **Lock DCO for editing** to lock the document hierarchy for editing.

Tip: The terms *DCO* and *document hierarchy* are used interchangeably.

2. Expand the tree so that you can see the default document and page types.
3. Select and single-click the **Document** node to edit the name.
4. Change the name from **Document** to **Car_Rental** and press **Enter**.

Important: You cannot include spaces in any of the document hierarchy node names.

5. Right-click the **TravelDocs** batch node and choose **Add multiple > > Documents**. Then, type 2 in the box and press **Enter**.

6. Rename the new documents from Document1 and Document2 to Flight and Hotel.
7. Click **Save**.

Creating page types

You need to create at least one page type for each of the three different document types that are contained in the TravelDocs document hierarchy.

The business requirements specification defines the following page types for each document type:

Table 6. Page types for each document type

Document types	Page types
Car_Rental	Rental_Agreement ✓
	Optional_Insurance ✓
Hotel	Room_Receipt ✓
	Meals x
	Other_Charges x
Flight	Air_Ticket ✓

To simplify the application, you can skip the Meals and Other_Charges pages.

To create new page types:

1. Confirm that the document hierarchy is still locked for editing.
2. Beneath the **Car_Rental** document node, select the default **Page** node and change the name from Page to Rental_Agreement.
3. Right-click on the **Car_Rental** document node and choose **Add > Page**. Then, change the name of the page from Page1 to Optional_Insurance.
4. Right-click the **Flight** document node and choose **Add > Page**. Then, expand the **Flight** node and change the name of the page from Page1 to Air_Ticket.
5. Right-click the **Hotel** document node and choose **Add > Page**. Then, expand the **Hotel** node and change the name of the page from Page1 to Room_Receipt.
6. Click **Save**.

Specifying the structure of documents and pages within the batch

In addition to creating page types for each document type, you need to configure rules and variables for the pages and documents.

Car Rental document

- Rental Agreement page
- Optional Insurance page

Flight document

- Air_Ticket page

Hotel document

- Room_Receipt page

The business requirements specify the following rules for the structure of each document type:

Table 7. Structural rules for each document type

Document type	Number	Required?	Order
Car Rental	Any number per batch	No	Any position within batch
Rental Agreement	One per document	Yes	Must be first in document
Optional Insurance	One per document	No	Cannot be first in document
Flight	Any number per batch	No	Any position within batch
Air_Ticket	One per document	Yes	Must be first in document
Hotel	Any number per batch	No	Any position within batch
Room_Receipt	One per document	Yes	Must be first in document

Within the document hierarchy, the following variables define the structure of the batch. By using these variables, you can define the structure of the batch.

Table 8. Variables defining the structure of the batch

Variable	Description
<i>Max</i>	Maximum number of objects of this type for each parent object. 0 means no maximum; 1 means Datacap creates a new document each time it encounters a page of this type, and so forth.
<i>Min</i>	Minimum number of objects of this type for each parent object. 0 means no minimum; 1 means there must be at least one, and so forth.
<i>Order</i>	Position of this object relative to other child objects of the same parent. 0 means any position.

Table 9. Batch structure variable values for each document type

Document type	<i>Max</i>	<i>Min</i>	<i>Order</i>
Car Rental	0	0	0
Rental Agreement	1	1	1
Optional Insurance	1	0	2
Flight	0	0	0
Air_Ticket	1	1	1
Hotel	0	0	0
Room_Receipt	1	1	1

To specify the structure of documents and pages within the batch:

1. Confirm that the document hierarchy is still locked for editing.
2. Right-click the **Car_Rental** document node and choose **Manage variables**.

3. Set the **Max**, **Min**, and **Order** values (The Car_Rental document is 0, 0, 0.), and click **Done**.
4. Right-click the **Rental_Agreement** page node and choose **Manage variables**.
5. Enter the **Max**, **Min**, and **Order** values. (The Rental_Agreement page is 1, 1, 1.), and click **Done**.
6. Repeat for each of the remaining document and page types.
7. Click **Save**.

Creating data fields

Each page type requires multiple field definitions.

The business requirements specification defines the following fields for each page type:

Table 10. Fields for each page type

Rental_Agreement	Optional_Insurance	Air_Ticket	Room_Receipt
Vendor x	Vendor x	Vendor x	Vendor x
Pickup_Date ✓	CDW ✓	Outbound_From ✓	Arrival_Date ✓
Pickup_Location ✓	CDW_Option ✓	Outbound_To ✓	Departure_Date ✓
Return_Date ✓	PAI ✓	Outbound_Date ✓	Total_Cost ✓
Return_Location ✓	PAI_Option ✓	Return_From ✓	
Car_Type ✓	PEP ✓	Return_To ✓	
Options ✓	PEP_Option ✓	Return_Date ✓	
Nav_System ✓	ELP ✓	Airfare ✓	
Child_Seat ✓	ELP_Option ✓	Taxes ✓	
Fuel_Service ✓	Total_Cost x	Total_Cost ✓	
Total_Cost ✓			

To simplify the application slightly, you can skip the fields marked x.

To create data fields:

1. Confirm that the document hierarchy is still locked for editing.
2. Expand the **Rental_Agreement** page, select the default **Field** node, and change the name from **Field** to **Pickup_Date**.
3. Right-click the **Rental_Agreement** page and choose **Add multiple > Fields**.
4. Type 6 in the box and press **Enter**.
5. Rename the new fields **Pickup_Location**, **Return_Date**, **Return_Location**, **Car_Type**, **Options**, and **Total_Cost**.
6. Right-click the **Options** field and choose **Add multiple > Fields**.
7. Type 3 in the box and press **Enter**.
8. Expand the **Options** and rename the new fields **Nav_System**, **Child_Seat**, and **Fuel_Service**.
9. Click **Save**.
10. Use the same procedure to add the fields to the **Optional_Insurance** page. The **Optional_Insurance** page has four fields, each of which has one subfield.
11. Click **Save**. The **Rental_Agreement**, **Room_Receipt**, and **Air_Ticket** pages all have a field that is called **Total Cost**. When you add this field to the **Room_Receipt** and **Air_Ticket** pages, Datacap Studio displays a message that

prompts you to reference the existing object. Click **Yes**. You see the same message when you add the **Return_Date** field to the **Air_Ticket** page. Click **Yes** again. For an explanation, see “Sharing field definitions across the document hierarchy.”

12. Repeat these steps for the **Air_Ticket** and **Room_Receipt** pages to add the fields marked ✓ in the table. The **Air_Ticket** page has nine fields and the **Room_Receipt** page has three fields.
13. Click **Save** after each page.
14. Click **Save**. The complete document hierarchy for TravelDocs includes three document types, each of which contain at least one page type and multiple fields.

Specifying the structure of fields on each page

The business requirements for the TravelDocs application specify that there must be only one instance of each field on each page. The order of the fields within the page is not important, but each field needs to be configured with **Max=1**, **Min=1**, and **Order=0**. The default values for fields are **Max=0**, **Min=0**, and **Order=0**.

There is no requirement to specify the structure of fields on each page because most applications create only one field of each type. Also, structure verification is only applied at the document and page level.

This procedure is optional. Specifying the values for all of the fields in the TravelDocs application is not required for this tutorial because all of the sample pages comply.

To specify the structure of fields within each page:

1. Right-click each field node and choose **Manage variables**.
2. Set **Max=1**, **Min=1**, and **Order=0** and then click **Done**.
3. After you complete the previous steps for all of the fields in the document hierarchy, including the subfields, click **Save**.
4. Click **Unlock DCO**.

Sharing field definitions across the document hierarchy

The document information, page information, and field information are stored in a file referred to as the document hierarchy or the setup DCO.

You specify the document, page, and field information in the document hierarchy panel. Datacap Studio saves this information in the `C:\Datacap\application_name\dco_application_name\application_name.xml` file.

The document hierarchy for the TravelDocs application is `C:\Datacap\TravelDocs\dco_TravelDocs\TravelDocs.xml`. This file defines the structure of the batch, and the structure of each document type, page type, and field within the batch. Although the batch structure is hierarchical, the structure of the file is flat, and the name of each document, page, and field object must be unique.

Within the document hierarchy file, the object definition specifies the child objects that are referenced by each parent object. For example, the Room Receipt definition specifies that a room receipt page has three child fields:

```

<P type="Room_Receipt">
  <V n="rules"></V>
  <F type="Arrival_Date" pos="0" min="0" max="0"/>
  <F type="Departure_Date" pos="0" min="0" max="0"/>
  <F type="Total_Cost" pos="0" min="0" max="0"/>
</P>

```

This structure allows multiple parent objects to reference the same child object.

In the TravelDocs application, the Rental Agreement, Air Ticket, and Room Receipt pages all have a **Total_Cost** field. The first time that you add the **Total_Cost** field to a page, Datacap adds the field to the document hierarchy. Later, when you add the field to the other page types, Datacap displays a message dialog that prompts you to use the existing reference. If you select **Yes**, all rules and properties are inherited.

The Datacap workflow

During the data capture process, documents go through a workflow that consists of several tasks, including page identification, character recognition, field validation, verification, and export. Some tasks require operator intervention, while other tasks run automatically.

These topics examine how the Datacap queuing mechanism moves batches of documents through the workflow and how tasks are implemented programmatically in terms of rulesets, rules, and actions.

“Understanding the Datacap workflow”

Understanding the Datacap workflow

A workflow contains jobs and tasks. Furthermore, tasks are associated with task profiles that contain rules and actions that are applied by the tasks while a job is processing a batch.

“Workflows, jobs, and tasks”

“Task profiles and rulesets” on page 26

“Rulesets, rules, and actions” on page 26

Workflows, jobs, and tasks

A workflow consists of a series of tasks, defines a way to process documents, and is associated with only one DCO.

Although Datacap applications can include multiple workflows, this tutorial focuses on single workflow applications. The standard workflow generated by the Application Wizard includes three job types:

- **Main Job:** This is the standard workflow for processing documents that takes a batch of documents through each of the processing steps that are previously identified, such as input documents, identify pages, and so on.
- **Fixup Job:** This job is used only when there are document integrity problems and displays the batch to an operator for corrective action. For more information, see the “Document integrity problem management” on page 54 topic.
- **Web Job:** This job is like the Main Job, but it defines the workflow for jobs that are initiated exclusively from the Datacap Web Client. It supports remote scanning and allows users to upload new batches to the server.

A job consists of one or more tasks. To process a batch of documents, you must run the batch through each task in the selected job. Some tasks (for example, Export) run without operator intervention, whereas others (for example, Verify) require an operator.

The tasks in the workflow are determined by the job type you select. You can see the tasks associated with each job type by looking in the Workflow pane on the Datacap Studio **Test** tab. The workflow for **Main Job** includes five tasks: VScan, PageID, Profiler, Verify, and Export. Each task is linked to a task profile.

Descriptions of each task are provided.

Table 11. Main Job task descriptions

Task Profile	Description
VScan	A virtual scanning profile that inserts pages into your application by copying images files from a specified location.
Upload	Used with remote scanning and virtual scanning through the Datacap Web Client interface, the Upload task is required for uploading images from remote scanning stations to the batch folder on the Datacap server.
PageID	Identifies the incoming pages by comparing them to known page types using fingerprint matching. Depending on the identification method used, this profile may perform full page OCR. It may also perform image cleanup.
Profiler	Organizes pages into documents, locates the fields defined for that page type, and performs OCR to recognize the field data (or obtains the data from the full page OCR results). Also runs validation rules to ensure that the data is valid.
Verify	Runs during the verification stage, when pages are displayed to an operator to ensure that recognition was accurate and to handle any validation errors.
Export	Exports the structured document data to an output file, a document management system, a database, or an external business process (can also include the original image).

In addition to the task profiles that run as part of the Main Job workflow, there are two other important task profiles the Application Wizard generates: FingerprintAdd and ImageFix.

Table 12. Additional task profiles

Task Profile	Description
FingerprintAdd	Generates the fingerprint files when you add new page types to the application from the Datacap Studio Zones tab.

Table 12. Additional task profiles (continued)

Task Profile	Description
ImageFix	Runs when you enhance a fingerprint image using the Image Processing window from the Zones tab.

Task profiles and rulesets

Each task is linked to a task profile that includes one or more rulesets. The default rulesets generated by the Application Wizard are displayed in the Task Profiles pane on the Datacap Studio **Rulemanager** tab.

The default **Main Job** workflow uses all of these task profiles, in the order as shown.

- VScan
- PageID
- Profiler
- Verify
- Export
- FingerprintAdd
- ImageFix

The **FingerprintAdd** profile runs when you add a new fingerprint to the application from the **Zones** tab.

The **Imagefix** profile runs when you enhance a fingerprint image by using the Image Processing window from the **Zones** tab.

Each ruleset defines one or more rules that you can run on specific documents, pages, or fields, or on the entire batch. The task profile specifies only that certain rulesets are associated with that profile. Nothing runs until you actually associate a specific rule with specific document, page, or field, or with the batch, as described in “Rule Execution” on page 44.

Within each task profile, rulesets run in the order, although a ruleset will not do anything if the rules in it are not associated with any objects in the document hierarchy.

Attention: The order of the rulesets within the task profile is important, because it defines the order in which Datacap runs rules. For example, you cannot check the integrity of a document before you create the document. So, the CreateDocs ruleset must come before the Document Integrity ruleset.

Multiple task profiles can reference the same ruleset. For example, the Profiler and Verify profiles both reference the Validate ruleset because you typically run validation rules after data recognition, and run the same rules again after verification by the operator.

Rulesets, rules, and actions

A ruleset consists of one or more rules. The rule itself is defined by the programmed functions and actions within it.

The default PageID ruleset has two rules, which are PageID and Set Fingerprint parameters. You can see the rules that are associated with each ruleset in the **Rulesets** panel on the Datacap Studio **Rulemanager** tab.

Rules are assigned to process specific objects in the document hierarchy (for example, to analyze and identify each page).

The default PageID rule consists of one function and two actions. The PageID function first launches the **AnalyzeImage** action. If **AnalyzeImage** is successful (returns True), the function launches the **FindFingerprint**. If **AnalyzeImage** fails (returns False), the function fails and Datacap launches the next function within the rule. In this case, there is not another function, but you could add an exception handling function to handle the error. See “Rule Execution” on page 44.

When you login to an application, Datacap Studio searches for and imports the rulesets from DLLs that are not in the `collection.xml` file yet. It searches the Rules folder for the application first, then it searches the central RRS folder. During this operation, the Update Status bar indicates that the current file is being processed with a rotating spinner. When the spinner is gone, the operation is finished.

Document input

Datacap works primarily with TIFF image files. So, the first activity in any Datacap workflow is to convert the documents to TIFF format and insert the documents into an input repository.

Documents can be hardcopy or electronic. If the documents are hardcopy, you must scan them and move the resulting files to the application repository. Electronic documents can come from various sources in various formats.

This tutorial examines different ways to place documents into your application for processing. You can set up a scanner for use with Datacap, which supports both ISIS and TWAIN scanners. For purposes of demonstration, it is assumed that you can set up a scanner with an ISIS driver attached to the computer that you are using. However, you can skip this requirement if you do not have a scanner with an ISIS driver. In that case, you can follow the virtual scanning examples.

For details about remote scanning with a TWAIN scanner, see “Remote scanning” on page 29.

“Electronic document input (virtual scanning)”

“Hardcopy document scans” on page 28

“TravelDocs: Batch creation with VScan” on page 29

“Local scanner setup (optional)” on page 31

Electronic document input (virtual scanning)

If your application is processing documents that are already available in electronic format, you can use *virtual scanning* to input the documents.

Datacap can manage a wide range of document types, including PDF files, fax files, and Microsoft Office documents. In addition, Datacap can ingest documents from various sources, including email and fax.

To scan image files from a shared folder on a network, or from a local folder, configure your application to use VScan actions. For details about VScan actions, see the Datacap Studio online help for the VScan action library.

The Datacap Studio application wizard generates an application framework that includes a virtual scanning task that copies files from the specified folder to the runtime batch folder. The virtual scan action is useful for application development and testing.

The Scan action copies the documents to the target location, and maintains the original files in the images folder.

“Document conversion”

Related concepts:

“Exporting data” on page 103

“Datacap Connector actions” on page 105

Document conversion

If your documents are not already in single-page TIFF format, you must convert them during the first stage of the processing workflow.

The action categories in the **Convert** library can process various file types.

- Excel
- HTML
- Image files (JPEG, BMP, PNG, and GIF)
- Outlook
- PDF
- RTF
- Multi-page TIFF
- TXT
- Word
- ZIP - extracts image files that can then be converted by other conversion actions

Hardcopy document scans

Datacap supports local scanning and remote scanning.

- Local scanning uses a scanner that is attached to and controlled from the Datacap Desktop component. Datacap Desktop supports ISIS and TWAIN scanners.
- Remote scanning uses the Datacap Web Client to scan and then upload the documents. The Datacap Web Client supports TWAIN scanners only.
- FastDoc scanning uses the FastDoc interface.

Confirm that your scanner driver is installed and that the scanner is functioning properly before you attempt to configure Datacap to use the scanner.

“Local scanning” on page 29

“Remote scanning” on page 29

Related information:

 Starting Fastdocs

Local scanning

When you scan from Datacap Desktop by using a local scanner, the scanned image files are delivered directly to the application's runtime batch folder. The scan task is responsible for creating the runtime batch files.

The application framework that is generated by the Datacap application wizard does not include a scan task. So, you need to create a scan task before you can scan locally. To create a scan task:

- Remove the existing VScan task from the **Main Job** workflow or create a new workflow for scanning (because a job can have only one batch creation task).
- Add a scan task to the workflow.

Tip: As an alternative to removing VScan and adding a scan task to the workflow, you can configure VScan to do physical scanning.

- Configure the scanner settings.
- Create a shortcut for the new scan task.

Detailed instructions are provided in “Local scanner setup (optional)” on page 31. The instructions are specific to the TravelDocs application, but you can generalize them for any Datacap application.

Remote scanning

You can scan documents into a Datacap application by using the Datacap Web Client.

Remote scanning is typically a two-step process:

- Use a **web scan** task to scan the pages and save the image files locally.
- Use an upload task to upload the image files and runtime batch files to the application's batches folder.

The default application framework includes a web scan task. So, you do not need to create one. For more information, see “Datacap Web Client and remote scanning” on page 226.

TravelDocs: Batch creation with VScan

Because sample electronic images are already installed, you do not need to scan paper documents. Instead, you can complete a virtual scan to create a batch.

“Scanning the sample documents from the application images folder”

“Modifying the VScan ruleset” on page 30

“Running VScan to generate a batch” on page 30

“Examining the files in the runtime batch folder” on page 30

Scanning the sample documents from the application images folder

The VScan (virtual scanning) ruleset copies files from the images folder into the runtime batch folder of the application.

The Datacap installation includes sample document images in the images folder of the application.

To work with the TravelDocs application, use the sample images that are provided, as summarized in the table.

Rental_Agreement	Optional_Insurance	Air_Ticket	Room_Receipt
Images_Page_01.tif	Images_Page_02.tif	Images_Page_06.tif	Images_Page_09.tif
Images_Page_03.tif	Images_Page_05.tif	Images_Page_07.tif	Images_Page_10.tif
Images_Page_04.tif		Images_Page_08.tif	Images_Page_11.tif

Modifying the VScan ruleset

Because the default VScan ruleset copies only the first four files, you must modify the VScan ruleset to copy up to 20 files.

1. On the Datacap Studio **Rulemanager** tab, Rulesets pane, select the **VScan** ruleset and click **Lock/Unlock ruleset** to lock the ruleset for editing.
2. Expand the **VScan** ruleset.
3. Select the **SetMaxImageFiles** action.
4. In the Properties pane, under **Parameters**, change the **StrParam** value from **4** to **20**.
5. In the Rulesets pane, click **Save**, and then click **Lock/Unlock ruleset** and select **Publish ruleset**.

Running VScan to generate a batch

You can generate a batch in Datacap Studio by running the VScan task.

1. Click the Datacap Studio **Test** tab.
2. In the **Workflow** pane, select the **VScan** task profile under **Main Job**.
3. Click **New** to start a new batch.
4. Click **Process rules for target object** on the main **Test** tab toolbar.
5. When you are prompted to release the batch, click **Advance**. The **Advance** command moves the batch to the next step in the workflow, which in this case is PageID.
6. If the runtime batch hierarchy is not already visible, click the **Runtime batch hierarchy** tab. 11 pages are denoted as type **Other**, which is the default type that is assigned to all pages before page identification.
Attention: If no pages are visible, ensure that you copied the sample image files as described in “Scanning the sample documents from the application images folder” on page 29.
7. Right-click the running batch icon in the **Workflow** pane and select **Cancel**. This step cancels the running of the PageID task profile because you did not define the rules for page identification.

Examining the files in the runtime batch folder

When you start a new batch, Datacap creates a runtime batch folder within the application **batches** folder.

The name of the folder matches the numeric batch identifier that Datacap generates automatically. In this example, 20100332.001 is the runtime batch folder.
C:\Datacap\TravelDocs\batches\20100332.001

Datacap stores all of the files that are associated with this batch in the runtime batch folder.

1. Open the application's most recent batch folder (C:\Datacap\TravelDocs\batches\< batch_id>). The folder contains the following files:

File	Description
TM00000*.tif	A copy of each of the original sample image files (copied from the images folder).
VScan.script	A file to aid in debugging.
VScan.xml	The runtime document hierarchy that is generated by the VScan task profile.
Vscan_rrs.log	The log file that is generated by the VScan task profile. The log file contains detailed descriptions of all the actions that are started by the task profile and is useful for troubleshooting. For more information, see "Datacap log files" on page 143.
PageID.xml	A copy of the runtime document hierarchy ready for use by the next task profile in the workflow (PageID).

2. Open the VScan.xml file in any XML editor or text editor.

```
<?xml-stylesheet type="text/xsl" href="../../../dco.xsl"?>
<B id="20100332.001">
  <V n="TYPE">TravelDocs</V>
  <V n="LAST_RR_TPROFILE">VScan:m:eRun</V>
  <P id="TM000001">
    <V n="TYPE">Other</V>
    <V n="STATUS">49</V>
    <V n="IMAGEFILE">tm000001.tif</V>
    <V n="ScanSrcPath">c:\datacap\traveldocs\images\images_page_01.tif</V>
  </P>
  <P id="TM000002">
    <V n="TYPE">Other</V>
    <V n="STATUS">49</V>
    <V n="IMAGEFILE">tm000002.tif</V>
    <V n="ScanSrcPath">c:\datacap\traveldocs\images\images_page_02.tif</V>
  </P>
  etc.
```

The VScan.xml file, 20100332.001 contains the runtime batch ID. The file also indicates that a Page Type of Other is initially assigned to all pages. A STATUS of 49 indicates that the page scanned successfully.

3. Close the file.

Local scanner setup (optional)

The default application framework does not include a scan task for scanning hardcopy documents into your application. Therefore, you must create a scan task for the TravelDocs application.

If you do not have a scanner that is attached to your computer, you do not need to set up a local scanner. If you have a scanner, ensure that the scanner driver is installed and that the scanner is working before you proceed.

You cannot include both a scan task and a virtual scan task in the same job workflow. You can have only one batch creation task per workflow. Because this tutorial requires a VScan task, you must copy the Main Job workflow, delete the VScan task from the copy, and create the scan task.

"Creating the scan task in the Datacap Web Client" on page 32

“Creating a shortcut for the new scan task”

“Running the scan task” on page 33

Creating the scan task in the Datacap Web Client

You can use Datacap Web Client to create and configure the scan task.

To create the scan task follow this procedure.

1. Open the Datacap Web Client and log in to your application.
2. Click the **Administrator** tab and then click **Workflow**.
3. Select the **Main Job** workflow and click **Copy**.
4. Name the new job **Scan Job**, enter the **Description** such as ISIS scan or TWAIN scan, and click **Apply**.
5. Expand the new **Scan Job**, select the **VScan** task, click **Remove** and click **OK**.

Important: A job can have only one batch creation task. The VScan task and Scan tasks are both batch creation tasks so you must remove VScan.

6. Select **Scan Job** and click **New** to create a new task.
7. In the Selected task details section, enter or select these values for the new task.
 - **Name:** MyISISscan or MyTWAINscan
 - **Description:** MyISISscan or MyTWAINscan
 - **Mode:** Batch Creation
 - **Queue to:** None
 - **Store:** None
 - **Program:** Datacap Desktop

Attention: To complete remote scanning through the Datacap Web Client, select **Scancel.aspx**. You can select **Multiple** to configure both Datacap Desktop and Scancel.aspx.
8. Click **Create Setup**, and then click **Setup**.
9. Add another Datacap Desktop panel set of key and value fields. Enter the application name, such as TravelDocs, in the key field and in the value field, enter either DotScanPanel1.ISISscan for ISIS scanners or DotScanPanel1.TWAINscan for TWAIN scanners. Click **Save**.
10. Select the new task, such as **MyISISscan** or **MyTWAINscan**, and press **Ctrl+Up Arrow** to move the task to the top of the workflow.
11. Click **Apply**, and then click **OK**.

Creating a shortcut for the new scan task

To run the scan task by using the Scancel.aspx page, you must create a shortcut in Datacap Web Client. If you are using Datacap Desktop to run the scan task, a shortcut is not required.

To create a shortcut for the new scan task:

1. In the Datacap Web Client, click the **Administrator** tab.
2. Click the **Shortcuts** tab, and click **New** to create a new shortcut.
3. In the Selected shortcut details section, enter or select these values for the following fields:
 - a. **Name:** Scan
 - b. **Description:** Scan task
 - c. **Mode:** Manual for Hold .

- d. Under Permissions, clear the check boxes and click the **MyISISscan** check box under Scan Job.
4. Click **Save**.

Running the scan task

Depending on how the task is configured, you can run the scan task in either Datacap Web Client or Datacap Desktop.

1. Load a page into your scanner's feeder.
2. If you are using remote scanning (scanc1.aspx), complete the Scan task as follows:
 - a. On the **Operations** tab of the Datacap Web Client, click **Scan**.
 - b. After scanc1.aspx loads into the Datacap Web Client, click **Scan**.
 - c. After Datacap scans the page, click **OK** and **Done**.

Important: To upload a scanned image to the Datacap server, you also must complete the Upload task, which you can start on the **Operations** tab.

3. If you are using Datacap Desktop, complete the Scan task as follows:
 - a. Start the Datacap Desktop program. On Windows in the **Start** menu click **IBM Datacap Clients > Datacap Desktop**.
 - b. Enter TravelDocs for the **Application**. Enter admin for the **User** and **Password**, and enter 1 for the **Station**. Click **Login**.
 - c. Select **Scan** from the **Shortcut** menu, and click **Start**.
 - d. For the first time that you are using Datacap Desktop with a scanner, or if you want to change the scanner that you used previously, click **Select...**
 - e. Choose the scanner that you want to use, and configure the scanner by completing one of the following tasks:
 - Set the values for functions, such as scan resolution, paper source, color mode, and others.
 - Click **Configure** and set the options in the scanner driver user interface.
 - f. In the Datacap Desktop main window, click **Scan**.

Page Identification

Page identification is one of the first steps in any Datacap application. All incoming pages are initially assigned the default page type Other. Before Datacap can assemble those pages into documents and extract data from the pages, it must determine the correct type for each page.

Page identification methods include fingerprint recognition, structure-based identification, text matching, and manual page identification. Image enhancement is typically done before page identification to remove lines, shading, and other graphic elements that might interfere with the recognition process.

“Page identification methods” on page 34

“Image Enhancement” on page 37

“TravelDocs: Fingerprint library creation” on page 38

“TravelDocs: Sample fingerprint image enhancement” on page 40

“TravelDocs: Run a batch through the workflow” on page 42

Page identification methods

Datacap supports several methods for page identification, which is also known as classification.

Page identification includes the following methods.

- Fingerprint matching
- Structure-based identification
- Text matching
- Manual page identification

Additionally, if your application supports only a single-page type, you can assign a static page type to all incoming pages.

“Fingerprint matching”

“Structure-based page identification” on page 36

“Text matching” on page 37

“Manual page identification” on page 37

Fingerprint matching

With fingerprint matching, Datacap generates a fingerprint that describes each incoming page. The fingerprint can include information about the relative densities of different regions of the page or the location of text on the page.

After you generate the fingerprint, Datacap compares it to a library of fingerprints for known page types. When it finds a match, it assigns the corresponding page type.

For example, assume that the incoming page matches the Hotel #1 room receipt. Datacap assigns the page type called Room_Receipt. It then records the ID of the matching fingerprint in the runtime batch hierarchy. The match is not exact because the data on the page is most likely different. However, you are just looking for the best match possible.

Selecting the fingerprint creation mode

Datacap provides two primary methods for generating page fingerprints.

Image analysis

This method scans the page image to identify the composite blackness of different regions of the page. This method provides fast page identification, but it requires that you do recognition later.

Full page recognition

This method does optical character recognition to identify the locations of text within the page. This method takes longer, especially with pages that include handwritten text. However, it reduces the time from subsequent workflow tasks because the full page recognition results are available for use.

Both of these methods write the resulting information to a CC0 file that is stored with the original TIFF image file in the fingerprint folder for the application.

Remember: The method that you use for creating library fingerprints must be the same as the method that you use to generate runtime fingerprints during page identification.

For example, if you decide to use image analysis, you must use image analysis in both the FingerprintAdd and PageID rulesets.

Important: Do not try to combine these methods because the recognition results are probably not accurate.

Image analysis

Image analysis uses a pixel-based algorithm to generate a CCO fingerprint file that represents the relative blackness of different regions of the page.

The AnalyzeImage action in the Recog_Shared actions library does image analysis on an image file.

Library	Action	Description
Recog_Shared	AnalyzeImage	Converts the TIFF image file that represents the current page to a CCO fingerprint file.

Full page recognition

Full page recognition, as the name suggests, uses the text and location of text on the page to generate the CCO fingerprint file. Datacap includes three optical character recognition (OCR) engines, plus one intelligent character recognition (ICR) engine that you can use to do full page recognition:

OCR_A

ABBYY FineReader OCR engine.

OCR_S

Nuance (formerly ScanSoft) OmniPage OCR engine.

OCR_SR

Newer implementation of the Nuance OmniPage OCR engine.

ICR_C

Open Text RecoStar ICR engine.

Other ICR engines are available as options. As a rule, the OCR engines work well with machine-printed text, whereas the ICR engine works well with hand-printed and machine-printed text.

Datacap includes actions libraries for each recognition engine (OCR_A, OCR_S, OCR_SR, and ICR_C). Each library includes its own version of the full page recognition action.

Library	Action	Description
ocr_a	RecognizePageOCR_A	Recognizes all characters on the current page and populates CCO fingerprint file for the page with the recognition results.

Library	Action	Description
OCR_s	RecognizePageOCR_S	Recognizes all characters on the current page and populates the CCO fingerprint file for the page with the recognition results.
ocr_sr	RecognizePageOCR_S	Recognizes all characters on the current page and populates the CCO fingerprint file for the page with the recognition results.
icr_c	RecognizePageICR_C	Recognizes all characters on the current page and populates the CCO fingerprint file for the page with the recognition results.

Fingerprint matching

The action that is used for all fingerprint matching, regardless of the creation method, is called FindFingerprint.

Library	Action	Description
AutoDoc	FindFingerprint	Tries to match the current page fingerprint to a fingerprint in the application fingerprint library.

Structure-based page identification

Structure-based identification uses the position of a page within the batch to determine its type.

You can assign page types that are based on position when application manages only one page type, or when the document structure is consistent. For example, all documents are two pages with a main page and a trailing page. For structure-based identification, use the **Set Page Type** action.

Library	Action	Description
DCO	SetPageType	Assigns a page type to the current page.
DCO	SetPageStatus	Sets the status of the current page.

If a batch contains documents of varying length, you can use separator pages between documents. For an example that uses barcoded separators, look at the Datacap Accounts Payable (APT) foundation application that you can run with Datacap.

When you identify a page by using structure-based identification, the page is not matched to a fingerprint. Therefore, even though recognition zones are available for your application to locate data during recognition, the zones are not aligned to the scanned image. After you identify a page with structure-based methods, the application can be customized to call CreateFields. When this call is in place,

recognition zones are located wherever they were defined on the original fingerprint image for that page type. The zone locations are not adjusted for shifting of the scanned image as they would be if Fingerprint matching were used. However, this limitation can be mostly overcome in at least two ways. You can crop and de-skew the image during an image-processing step. You can use pattern-match anchors to align the zones.

Text matching

To complete page identification by using text matching, you must first complete a full page recognition. You can then search the recognition results for a string that is unique to each page type.

In the TravelDocs application, the first function attempts a full page recognition and searches for the string Pickup on the current page. If the function finds Pickup, it assigns the page type Rental_Agreement. If the function does not find Pickup, it fails, and the second function searches for the string Flight. If the second function finds Flight, it assigns the page type Air_Ticket. If it does not find Flight, the second function fails, and the third function searches for the string Room. If the third function finds Room, it assigns the page type Room_Receipt. If it does not find Room, the page remains with the page type Other.

As with the structure-based techniques, when you identify a page by using text matching, the page is not matched to a fingerprint. Therefore, even though recognition zones are available for your application to locate data during recognition, the zones are not aligned to the scanned image. After you identify a page with text-matching methods, you can customize the application to call CreateFields. This call locates the recognition zones where they were defined on the original fingerprint image for that page type. The zone locations are not adjusted for shifting of the scanned image in the same manner that Fingerprint matching can adjust locations. However, you can work around this limitation by using either of two methods: You can crop and de-skew the image during an image-processing step, or you can use pattern-match anchors to align the zones.

Manual page identification

Although many page identification techniques identify pages automatically, you can configure your application to display unrecognized pages to an operator for manual identification.

For more information, see “Adding a function for manual page identification” on page 213.

Image Enhancement

Image enhancement consists of cleaning up images and removing elements that might produce recognition errors. You must complete any required image enhancement before page identification.

“Goal of image enhancement”

“When to complete image enhancement” on page 38

Goal of image enhancement

The goal of image enhancement is to eliminate lines, shading, misalignment, and other artifacts that can interfere with the recognition process.

The Datacap Image Processing tool provides a set of image enhancement capabilities that you can configure to handle various problem types. However, finding the best combination of image enhancement settings can take time,

especially if your application must handle multiple page types. Image enhancement is typically done before the page type is known, in other words, before page identification. You must set up the image-processing properties in a way that works well for all page types.

Important: Some special cases are exceptions to this statement. Rules, or ImageFix Actions, can do multiple passes of image enhancement, which is also known as image processing, before or after page identification. The rules can use different settings for different page types or based on other criteria. The most common use case is a single pass before page identification. Any image enhancement that is completed before fingerprint matching must be identical to the image enhancement that was done on the fingerprint template image when the template was created.

The default image-processing properties are designed to work well with typical printed pages that use plain black text on a white background. Establishing settings that work well for the pages your application must handle requires experimentation. For more information, see “Determining appropriate image-processing settings” on page 40.

The settings that you establish are stored in the file `imagefix.ini` in the application's `dco_<application_name>` folder.

When to complete image enhancement

You complete image enhancement on fingerprint images when you are setting up the fingerprint library. You must complete it again on your document images after input but before page identification.

When you add fingerprints to the fingerprint library, Datacap queries whether to enhance the image. Typically, you experiment to find settings that work well for all page types. You can skip image enhancement initially, and then return and enhance the fingerprint images, after you determined appropriate settings. See “Applying new image-processing settings to enhance the fingerprint images” on page 41.

After document input, you use the ImageFix ruleset to apply the same image-processing settings for image enhancement. The default ImageFix ruleset includes two rules.

- The first rule (ImageFix Load Settings) reads the image-processing properties from the settings (`.ini`) file.
- The second rule (Enhance Image) completes image processing on each page by using those settings and creates a backup of the original with a `.tio` extension.

TravelDocs: Fingerprint library creation

To create the initial fingerprint library for TravelDocs, you must change the default fingerprint creation method and create fingerprints for known page types.

“Changing the fingerprint creation method”

“Fingerprint creation for known page types” on page 39

Changing the fingerprint creation method

The application framework that is generated by the Application wizard uses the image analysis method for fingerprint creation. All of the pages in the TravelDocs application are printed from a computer. So, you must convert the application to use full page recognition with the OCR_s engine.

To change the fingerprint creation method, you must edit two of the rulesets that are defined on the Datacap Studio **Rulemanager** tab. The **FingerprintAdd** ruleset runs whenever you add a fingerprint to the fingerprint library. **PageID** generates the runtime fingerprints and matches them to determine the type of each incoming page. You can modify these rulesets to complete full page recognition instead of image analysis.

To modify the **FingerprintAdd** and **Page ID** rulesets:

1. On the Datacap Studio **Rulemanager** tab, in the Rulesets pane, select the **FingerprintAdd** ruleset and click **Lock/Unlock ruleset** (padlock) to lock the ruleset for editing.
2. Expand the **FingerprintAdd** ruleset completely.
3. Right-click the **AnalyzeImage** action and choose **Remove**.
4. Click the **Actions library** tab.
5. Expand the **OCR_S** library and select **RecognizePageOCR_S**.
6. Make sure **FingerprintAdd: Other Function 1** is selected in the Rulesets pane.
7. Click **Add to function**.
8. In the Rulesets pane, click **Save**. Then, click **Lock/Unlock ruleset** and select **Publish ruleset**.
9. Select the **PageID** ruleset and click **Lock/Unlock ruleset** to lock the ruleset for editing. Then, expand the ruleset and the **PageID** rule.
10. Remove the **AnalyzeImage** action and replace it with the **RecognizePageOCR_S** action. If necessary, use **Up arrow** or **Down arrow** to move the action to the correct position within the function.
11. In the Rulesets pane, click **Save**. Then, click **Lock/Unlock ruleset** and select **Publish ruleset**.

Fingerprint creation for known page types

To create fingerprints for known page types, you must create fingerprint classes and add individual fingerprints.

“Creating fingerprint classes”

“Adding individual fingerprints” on page 40

Creating fingerprint classes:

By using classes, you can categorize fingerprints within your application.

The default framework includes two classes:

<Global>

This class includes the generic 555 fingerprint with page type **Other**. The generic fingerprint is useful because it enables application development without page fingerprints.

<New>

When you use the **FindFingerprint** action during page identification, you can create fingerprints automatically for unrecognized pages. If you call **FindFingerprint** with the parameter **True** and Datacap does not find a matching fingerprint, Datacap adds the runtime fingerprint to the **New** class.

Here, you create a class for each document type: **Car_Rental**, **Hotel**, and **Flight**. Categorization by document type is not required but provides a useful way to organize many fingerprints.

To create the fingerprint classes:

1. On the Datacap Studio **Zones** tab, in the **Fingerprints** pane, click **Add new item** and select **Add fingerprint class**.
2. Enter `Car_Rental` and click **OK**.
3. Repeat for `Flight` and `Hotel`.

Adding individual fingerprints:

After you create the fingerprint classes, you add them to the fingerprint library.

To add individual fingerprints:

1. In the Fingerprints pane, right-click the new **Car_Rental** class and choose **Add fingerprint**.
2. Browse to the folder where the TravelDocs fingerprint images are located.
3. Select `Car1.tif`, and click **Open**. When you are prompted to enhance the image, click **No** (you enhance the image later). Datacap requires a few minutes to add the new fingerprint.
4. Repeat to add `Car2.tif`, `Car3.tif`, `Car4.tif`, `Car5.tif`, and `Car6.tif`. Again, do not enhance the images.
5. In the Fingerprints pane, select the first car rental fingerprint and confirm that it is a rental agreement page. Then, click **Type** at the top of the pane and choose **Rental_Agreement**.
6. Repeat to assign page types to the remaining car rental fingerprints. Use **Rental_Agreement** for the rental agreement pages and **Optional_Insurance** for the optional insurance pages.
7. Add `Flight1.tif`, `Flight2.tif`, and `Flight3.tif` to the **Flight** class and assign the type **Air_Ticket**.
8. Add `Hotel1.tif`, `Hotel2.tif`, and `Hotel3.tif` to the **Hotel** class and assign the type **Room_Receipt**.

Attention: Do not add `Hotel4.tif` or `Hotel5.tif`.

TravelDocs: Sample fingerprint image enhancement

To enhance the sample fingerprint images, you must determine the image-processing settings and apply them to the sample fingerprint files.

“Determining appropriate image-processing settings”

“Applying new image-processing settings to enhance the fingerprint images”
on page 41

Determining appropriate image-processing settings

Because image enhancement is completed before page identification, you must set up the image-processing properties for all page types.

Most Datacap applications must manage multiple page types.

Important: Some special cases are exceptions to this statement. Rules complete multiple passes of image enhancement before or after page identification by using different settings for different page types or based on other criteria. The most common use case is a single pass before page identification. Any image

enhancement that is completed before fingerprint matching must be identical to the image enhancement completed on the fingerprint template image when the template was created.

The default image-processing properties are designed to work with typical printed pages that use plain black text on a white background. One of the sample air ticket pages contains white text on a black background, which is the most difficult page to process. This procedure addresses first the page with white text on a black background.

1. In the **Fingerprints** pane on the **Zones** tab, expand the **Flight** class and select the third fingerprint (Airline #3).
2. In the **Image View** pane, click **Open image processing settings** in the upper right.
3. Click **Run image processing** to apply the default image-processing properties, as defined in the **Properties** pane.
4. Click **Reset image** to revert to the original image.
5. In the **Properties** pane, change the settings as follows:

Category	Property	Default setting	New setting
Border Removal	Border Removal	True	False
Inverse Text Correction	Minimum Area Width	300	100
Line Removal	Minimum Length	50	30

6. Click **Save** and choose **Save settings**. Then, click **OK**.

Attention: When you save the settings, Datacap saves the new image enhancement properties in the file C:\Datacap\TravelDocs\dco_TravelDocs\imagefix.ini. Datacap uses the same settings file for the image processing that takes place before page identification (ImageFix).

7. Click **Run image processing** to apply the new image-processing properties. This time all of the vertical and horizontal lines disappear. The top of the page is not clipped, and the white text on a black background is converted to black text on a white background.
8. Close the Image Processing window without saving the enhanced image.
9. Next, in the **Fingerprints** pane, select the second air ticket fingerprint (Airline #2). There are problems on this page with the default settings, but you can try it with the new settings.
10. In the **Image View** pane, click **Open image processing settings** in the upper right.
11. Click **Run image processing** to apply the new image-processing properties. The horizontal lines are removed while everything else is intact.
12. Close the Image Processing window without saving the enhanced image.

Applying new image-processing settings to enhance the fingerprint images

After you determined appropriate image-processing settings, you can apply these settings to all of the sample fingerprint files.

To apply the appropriate processing settings to the sample fingerprint files:

1. In the **Fingerprints** pane, expand the **Car_Rental** class and select the first **Rental_Agreement** fingerprint.

2. In the Image View pane, click **Open image processing settings**.
3. Click **Run image processing** to apply the image-processing properties.
4. Click **Save**, choose **Save image**, and click **OK**. Then, click x to close the Image Processing window.
5. Repeat to apply the same image-processing properties to all of the other fingerprints. Make sure that you explicitly save each image after image processing.

TravelDocs: Run a batch through the workflow

After you create the initial fingerprint library and determine the appropriate image-processing settings, you can run a batch through the workflow.

In summary, you completed these tasks in developing your TravelDocs application.

- Created an application framework for the TravelDocs application by using the Datacap Studio Application wizard.
- Modified the default document hierarchy to include the document types and pages types the TravelDocs application supports.
- Specified the required structure for documents and pages within a batch according to the business requirements.
- Within the document hierarchy, defined the fields of interest for each page type.
- Created the initial fingerprint library by using one sample image for each known variant of each page type.

In terms of implementing the workflow. You did not attach any rules to the document hierarchy, though some default rules are attached to the default elements. However, you can run a batch through the PageID task to make sure that the application is handling page identification correctly.

“Processing a batch”

“Runtime batch folder contents” on page 43

“Checking the confidence levels on the runtime pages” on page 43

Processing a batch

For testing purposes, you can process a batch on the **Test** tab of Datacap Studio.

To process a batch:

1. Open Datacap Studio and click the **Test** tab.
2. In the **Workflow** pane, select the **VScan** task profile under **Main Job**.
3. Click **New** to start a new batch.
4. Click **Process rules for target object** on the main Test tab toolbar.
5. When asked if you want to release the batch, click **Advance**. The batch is moved to the next step in the workflow (PageID).
6. Click **Process rules for target object** on the main Test tab toolbar and wait while the task profile runs. It might take a few moments because Datacap must do full page OCR on all the images in the batch.
7. When asked if you want to release the batch, click **Advance**. The batch is moved to the next step in the workflow (Profiler).
8. On the **Runtime batch hierarchy** tab, scroll through the list to see the page types that are assigned to TM000001, TM000002, and so on.
9. Right-click the running batch button in the **Workflow** pane and choose **Cancel**. You do not run the Profiler task profile until you assign rules.

Runtime batch folder contents

The application's most recent batch folder is at C:\Datacap\TravelDocs\batches*<batch_identifier>*.

The runtime batch folder contains these files.

File	Description
TM00000*.tif	An image-enhanced version of each of the sample image files.
TM00000*.tio	A copy of each of the original image files.
TM00000*c.xml	The results of full page recognition for each image file.
TM00000*.cco	The fingerprint file for each of the image files.
PageID.xml	The runtime document hierarchy that is generated by the PageID task profile.
pageid_rrs.log	The log file that is generated by the PageID task profile.
VScan.xml	The runtime document hierarchy that is generated by the VScan task profile.
vscan_rrs.log	The log file that is generated by the VScan task profile.
Profiler.xml	A copy of the runtime document hierarchy ready for use by the next task profile in the workflow (Profiler).

Checking the confidence levels on the runtime pages

During page identification, Datacap assigns a confidence level to each page. This process indicates the degree of similarity between the runtime page and the fingerprint that matches the page most closely.

You can see the confidence level for each page in the runtime batch file (PageID.xml) that is generated by the PageID task profile.

To check the confidence levels on the runtime pages:

1. Open the application's most recent batch folder (C:\Datacap\TravelDocs\batches*<batch_identifier>*).
2. Open the file PageID.xml in an XML viewer or in Notepad. This file includes the confidence level that is assigned to each page in the batch, and the identifier of the matching fingerprint.

```
<?xml-stylesheet type="text/xsl" href="..\..\dco.xsl"?>
<B id="20100321.002">
  <V n="TYPE">TravelDocs</V>
  <V n="LAST_RR_TPROFILE">PageID:m:eRun</V>
  <P id="TM000001">
    <V n="TYPE">Rental_Agreement</V>
    <V n="STATUS">49</V>
    <V n="IMAGEFILE">tm000001.tif</V>
    <V n="ScanSrcPath">c:\datacap\traveldocs\images\images_page_01.tif</V>
    <V n="RecogStatus">0</V>
    <V n="Confidence">0.9727517</V>    <!-- Confidence level
    <V n="Image_Offset">0,0</V>
```

```

<V n="TemplateID">556</V>          <-- identifier of matching fingerprint
<V n="Fingerprint Created">No</V>
</P>
...

```

The default application framework uses a confidence threshold of 0.7, so anything above 0.7 is considered a match. In the example above, the confidence level is 0.97, so the page is a good match. If multiple fingerprints match with a confidence level above 0.7, Datacap selects the fingerprint with the highest confidence value.

Important: The SetProblemValue action in the PageID: Set Fingerprint Params rule specifies the minimum required confidence level.

Rule Execution

Rule execution refers to how rules are associated with specific objects in the document hierarchy and how Datacap processes a batch of documents.

The Datacap workflow moves batches through the workflow from task to task. Task profiles are implemented as rulesets, which are constructed by using rules and actions.

You can use the Datacap Studio debugging tools to step through the PageID task profile and see how Datacap runs the rules.

A ruleset consists of one or more rules that contain functions and actions. When you run a rule, you are running the functions and actions that are in them.

The following example describes the execution flow that allows actions to run by using an if, then, else programming model.

```

Run a rule:
for each function in the rule
  for each action in the function
    run the action
    if the action returns false, exit from this function
  next action in the function
  if the last action in the function returns true, exit from this rule
next function in the rule
if the last action that is run returns true, then the rule result is true,
else the rule result is false
For validation rules, true means that validation passed, false means that it failed

```

The actions that are frequently used to control rules execution flow are rrCompare, rrCompareNot, GoToNextFunction, and SetReturnValue.

“Association of rules with objects”

“Order of rule execution” on page 46

“TravelDocs: Stepping a batch through the PageID task profile” on page 49

Association of rules with objects

Rules run when they are assigned to specific objects in the document hierarchy, and only if the parent rulesets are included in the current task profile.

Datacap can run rules on batches, documents, pages, and fields.

“Example 1: Batch-level rule execution” on page 45

“Example 2: Page-level rule execution” on page 45

Example 1: Batch-level rule execution

When a rule assembles individual pages into different document types, the rule must run at the batch level.

In the default **Profiler** task profile, the **CreateDocs** ruleset includes a rule that is called **Create Docs**. This rule assembles individual pages into documents that are based on the structure in the document hierarchy. For example, the TravelDocs car rental document type has a rental agreement page and an insurance coverage page.

From the object general information, you can observe these details about the pages.

Tip: To see the object general information with the rules for each page type within a document, lock the Document Hierarchy, right-click the page type, and select **Manage variables**.

1. The rental agreement page is required (Min=1), and that it is always the first page (Order=1).
2. There can be only one rental agreement page within each car rental document (Max=1).
3. The insurance page is optional (Min=0).

The Create Docs rule must run at the batch level because it assembles multiple document types.

When the Create Docs rule encounters a page of type Rental_Agreement, it creates a new document in the runtime batch hierarchy. If a rental agreement page is followed immediately in the batch by an Optional_Insurance page, Datacap adds the insurance page to the same car rental document. Otherwise, Datacap creates a new document.

Example 2: Page-level rule execution

When a rule locates field zones on different pages, the rule must be run at the page level.

The default **Recognize** ruleset includes a rule that is called **Recognize Page**. This rule locates each field zone on the current page by using the positional information in the document hierarchy. The rule then uses OCR to get the data within each zone.

The rule must run at the page level because of these reasons:

- The fields are different for each page type (for example, the TravelDocs rental agreement page and the optional insurance page have different fields).
- The field positions are different for each variation of the page type. For example, the Car_Type field is at a different location on the page for each rental car company.

Tip: To see the variable information for any field, lock the Document Hierarchy, right-click the field, and choose **Manage variables**.

The position variables define the position of the field for each of the car rental agreement forms that are identified during fingerprinting. 678, 695, and 696 are the fingerprint IDs.

The Document Hierarchy shows that the **Recognize Page** rule runs at the page level for each page type.

The rule is not assigned to the page type Other because no fields are defined for Other.

Order of rule execution

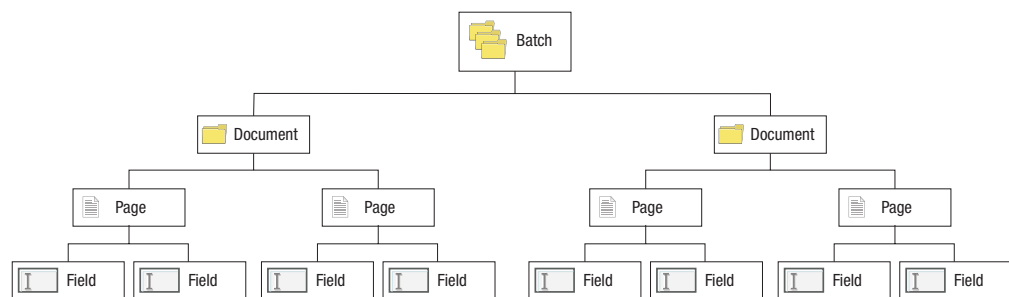
Rules are run according to the position of the ruleset within the task profile, and the position of the associated object within the document hierarchy.

Two conditions determine whether a rule is run. A rule runs only when it is associated with a specific object in the document hierarchy. Also, the rule runs only when the parent ruleset is included in the current task profile.

The position of a rule within its ruleset does not affect when the rule is run.

When you run a task profile that has more than one ruleset, Datacap runs the rulesets in the order that the Task Profiles pane displays them in Datacap Studio. For example, the PageID task profile, Datacap first runs the ImageFix ruleset, and next runs the PageID ruleset.

When Datacap runs a ruleset on a batch, it goes through the runtime batch hierarchy iteratively. (Page 1 of Document 1, including the two fields, is processed in its entirety before Datacap proceeds to Page 2 of Document 1.)



For each object, Datacap runs any rule in the current ruleset that is included in the object's *Open* element. For example, if you are running the Recognize ruleset on a page of type Rental_Agreement, Datacap runs the Recognize Page rule.

- Car_Rental
- Open
 - Rental_Agreement
 - Open
 - (global)
 - CreateDocs:Create fields
 - Recognize:Recognize Page

Restriction: You can include only one rule from a ruleset in an object's *Open* element.

The following example shows a portion of the document hierarchy for the TravelDocs application. Each object also has a *Close* element. Datacap runs rules in the object's *Close* element when it exits from the object. For objects at the lowest level of the hierarchy, *Close* rules run immediately after *Open* rules. For other objects, the *Close* rules do not run until Datacap processes the lower-level objects.

TravelDocs (Batch)

- Open
 - (global)
 - VScan: VScan
 - ImageFix: ImageFix Load Settings
 - PageID: Set Fingerprint Params
 - CreateDocs: Create Docs
 - Document Integrity: Batch Document
 - Export: Set Export Params
- Other (Page)
 - Open
 - (global)
 - ImageFix: Enhance Image
 - PageID: PageID
 - FingerprintAdd: FingerprintAdd
 - Close
- Car_Rental (Document)
 - Open
 - Rental Agreement (Page)
 - Open
 - (global)
 - CreateDocs: Create Fields
 - Recognize: Recognize Fields
 - Validate: Validate Page
 - Routing: Routing Rule 1
 - Export: Export Page Fields
 - Vendor (Field)
 - Pickup_Date (Field)
 - Open
 - Global
 - Validate: Validate Date
 - Close

The TravelDocs Batch rules run when batch processing begins, depending on the ruleset that you are running. For example, if you are running the VScan ruleset, Datacap runs the VScan rule.

Other page rules run each time Datacap begins processing a page of type Other, which is the default page type that is initially assigned to all pages. For example, if you are running the PageID ruleset, Datacap runs the PageID rule, which assigns the correct type to each page.

For the Car_Rental document, there are no document-level rules in this example.

The Rental_Agreement page rules run each time Datacap begins processing a page of type Rental_Agreement. For example, if you are running the CreateDocs ruleset, Datacap starts the Create Fields rule, which creates a page data file. Datacap uses the page data file to store the data that is captured later by the Recognize ruleset.

The Pickup_Date field rule runs when Datacap is processing a page of type Rental_Agreement and encounters a field of type Pickup_Date.

“Example 1: Page identification rules”

“Example 2: Validation rules”

“Summary of order of rule execution”

Example 1: Page identification rules

A page identification rule identifies each incoming page by comparing the page image to the known page types and by using fingerprint recognition.

The **PageID** task profile includes a ruleset that is called **PageID**, which includes a rule that is called **PageID**.

The **Sync DCO View with Ruleset View** button shows which documents, pages, or fields are associated with the selected rule. For example, if you select the PageID rule and click **Sync DCO View with Ruleset View**, Datacap Studio expands the document hierarchy to show you the objects that are associated with the PageID rule.

In TravelDocs, the **PageID** rule is associated only with the page type Other. In this case, the rule runs whenever Datacap processes a page of type Other while it runs the **PageID** task profile. Datacap assigns the Other page type to all incoming pages as a default because all pages are initially unknown. Datacap runs this rule on all pages during page identification and, assuming the page matches one of the known types, assigns the correct type (for example, Rental_Agreement or Air_Ticket).

Example 2: Validation rules

Validation rules are used to confirm that a field value conforms to a supported format.

The **Profiler** and **Verify** task profiles both include the **Validate** ruleset. In the TravelDocs application, the Validate ruleset includes a rule that is called **Validate Date**, which returns True if a field value conforms to one of the supported date formats.

If you select the **Validate Date** rule and click **Sync DCO View with Ruleset View**, Datacap Studio expands the document hierarchy to show you which objects are associated with the rule.

In the TravelDocs application, the **Validate Date** rule is associated with the Arrival_Date field and Departure_Date field on pages of type Room_Receipt. This rule runs whenever Datacap processes an Arrival_Date field or Departure_Date field on a page of type Room_Receipt while it runs the **Profiler** or **Verify** task profile.

Summary of order of rule execution

The order of rule execution closely follows the document hierarchy, starting with the batch, proceeding to the document, and continuing down to the field level.

The following pseudocode describes the order of rule execution.

```
for each ruleset in the current task profile
  run batch-level "Open" rules
  for each document type
    run document-level "Open" rules
    for each page type in the current document type
```

```

        run page-level "Open" rules
        for each field in the current page type
            run field-level "Open" rules
            run field-level "Close" rules
        next field
        run page-level "Close" rules
    next page
    run document-level "Close" rules
next document
run batch-level "Close" rules
next ruleset

```

TravelDocs: Stepping a batch through the PageID task profile

Even though you did not add any new functions to the application, you can run the VScan and PageID rulesets again and step through the actions.

1. Click the Datacap Studio **Test** tab.
2. In the Workflow pane, select the **VScan** task profile under **Main Job**.
3. Click **New** to start a new batch.
4. Click **Process rules for target object** on the main **Test** tab toolbar.
5. When asked if you want to release the batch, click **Advance**. The Advance command moves the batch to the next step in the workflow (PageID).
6. Click **Step in** on the main **Test** tab toolbar.
7. Click **Step in** a few times. Execution remains at the batch level. Because the ImageFix Load Settings rule is assigned at the batch level, Datacap expands the rule and prepares to run the LoadSettings action.
8. Continue clicking **Step in** to run the ImageFix Load Settings action and complete the rule. Because the Enhance Image rule is not assigned at the batch level, it does not run.
9. Continue clicking **Step in** until the runtime batch hierarchy indicates that first page is selected. Datacap is now ready to run page level rules on each page in the runtime hierarchy, starting with page TM000001.
10. Continue clicking **Step in**. Because the Enhance Image rule is assigned at the page level, Datacap expands this rule and prepares to run the ImageEnhance action.
11. In the center pane of Datacap Studio, click the **Image** tab if it is not already active. The **Image** tab displays the current runtime object.
12. Click **Step in** to run the ImageEnhance action. The Image pane displays the current page image after image enhancement. The check mark beside the action in the Rulesets pane indicates that the action returned True.
13. Right-click the batch in the **Workflow** pane and choose **Cancel**.

For more information, see “Single-stepping through your code” on page 147 and “Using breakpoints” on page 145.

Document assembly

Datacap identifies incoming pages and assigns the correct page type by using fingerprint matching or one of the other identification methods. The next step assembles the batch of individual pages into documents according to the rules that are defined within the document hierarchy.

In preparation for the recognition phase, you create a runtime data file for each page.

Document assembly also includes document integrity checking, which confirms that the documents conform to predefined rules. The process also takes corrective action if the documents do not conform.

“Structured documents”

“TravelDocs: Document creation and page file setup” on page 55

“TravelDocs: Document integrity management” on page 56

Structured documents

Structured documents are based on the document hierarchy and include data files. These documents conform to predefined integrity rules.

“Hierarchy-based documents”

“Creation of the page data files” on page 51

“Document integrity” on page 52

“Document integrity problem management” on page 54

Hierarchy-based documents

The document hierarchy defines both the document types that your application supports and the page types that are associated with each document type.

For example, the TravelDocs document hierarchy defines three document types, four associated page types, and the generic page type Other.

After page identification assigns the correct page type to each incoming page, your application uses information in the document hierarchy to determine the corresponding document. For example, a page of type Rental_Agreement is part of a car rental document. A page of type Air_Ticket is part of a flight document.

Datacap then uses the information in the document hierarchy to assemble individual pages into multi-page documents. Each page has three variables that define the structure of the parent document:

- *Max*: Maximum number of pages of this type for each document (0 means no maximum).
- *Min*: Minimum number of pages of this type for each document (0 means no minimum).
- *Order*: Position of a page relative to other pages in the same document (0 means any position).

For example, here are the variables that you specified earlier for each of the TravelDocs pages.

	Max	Min	Order	Description
Rental Agreement	1	1	1	One per document; required; must be first
Optional Insurance	1	0	2	One per document; optional; must be second
Air_Ticket	1	1	1	One per document; required; must be first

	Max	Min	Order	Description
Room_Receipt	1	1	1	One per document; required; must be first

The variables determine that each car rental document must contain one rental agreement page, and that the page might be followed by an optional insurance page. If Datacap identifies a rental agreement page that is immediately followed by an optional insurance page, it groups the two pages as a single document. The following example is a portion of the runtime data file (PageID.xml) that is generated after document creation.

```
<?xml-stylesheet type="text/xml" href="../../dco.xml"?>
<B id="20100321.011">
  <V n="TYPE">TravelDocs</V>
  <V n="LAST_RR_TPROFILE">Rulerunner:m:eRun</V>
  <D id="20100321.011.01">
    <V n="TYPE">Car_Rental</V>
    <V n="STATUS">0</V>
    <P id="TM000001">
      <V n="TYPE">Rental_Agreement</V>
      <V n="STATUS">49</V>
      <V n="IMAGEFILE">tm000001.tif</V>
      etc.
    </P>
    <P id="TM000002">
      <V n="TYPE">Optional_Insurance</V>
      <V n="STATUS">49</V>
      <V n="IMAGEFILE">tm000002.tif</V>
      etc.
    </P>
  </D>
  etc.
  "Assembling documents"
```

Assembling documents:

The Datacap Studio Actions library includes an action to assemble pages into documents.

Library	Action	Description
DCO	CreateDocuments	Assembles the pages of the current batch into documents that are based on the structure that is defined in the document hierarchy and the min , max , and order properties.

Important: You must use this action within a rule that runs at the batch level.

Creation of the page data files

After you arrange individual pages into documents, you must create a runtime data file for each page. The runtime data file is an XML file that includes an element for each field on the page.

The DCO actions library includes the following action to create a runtime data file for the current page.

Library	Action	Description
DCO	CreateFields	Creates a page data (XML) file for the current page. The file includes an element for each field that is defined in the document hierarchy for the current page type. Each field has an identifier and three properties (TYPE, Position, and Status) with default values.

Important: You must run this action within a rule that runs at the page level.

The data file is empty initially, but the CreateFields action uses the document hierarchy structure to create a shell with an element for each data field. The shell gets populated later during recognition.

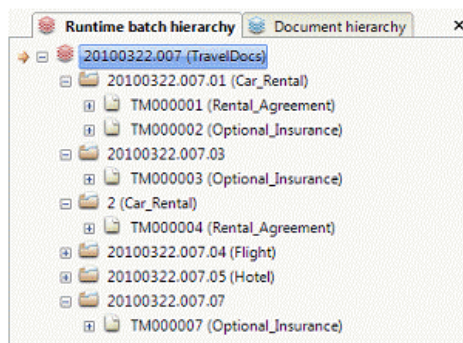
```
<?xml-stylesheet type="text/xsl" href="..\..\dco.xsl"?>
<P id="TM000001">                                <-- Page data file for first page in batch (type Rental_Agreement)
  <F id="Pickup_Date">                            <-- Pickup_Date field (no data)
    <V n="TYPE">Pickup_Date</V>
    <V n="Position">0,0,0,0</V>
    <V n="STATUS">0</V>
  </F>
  <F id="Pickup_Location">                        <-- Pickup_Location field (no data)
    <V n="TYPE">Pickup_Location</V>
    <V n="Position">0,0,0,0</V>
    <V n="STATUS">0</V>
  </F>
  etc.
```

Document integrity

Even though the document hierarchy defines the required structure of the batch, Datacap might encounter a batch that does not conform to the required structure.

For example, a batch in the TravelDocs application might include a rental agreement page that is followed by two optional insurance pages. Similarly, a batch might include an optional insurance page that is not preceded by a rental agreement page.

The following runtime batch has two structural integrity issues.



- In the first case, the CreateDocuments action grouped the first optional insurance page with the preceding rental agreement page. The action also placed the second insurance page in a separate document of a unidentified type.
- In the second case, the CreateDocuments action again placed the orphaned insurance page in a document of a unidentified type.

In both cases, the batch does not comply with the document integrity rules that are defined in the document hierarchy. However, the CreateDocuments action sets the document status to 0 (OK) and the page status to 49 (ScanOK).

```
<D id="20100322.007.03">
  <V n="TYPE"></V>
  <V n="STATUS">0</V>      <-- Document status is "OK" (0)
  <P id="TM000003">
    <V n="TYPE">Optional_Insurance</V>
    <V n="STATUS">49</V>    <-- Page status is "ScanOK" (49)
    etc.
  </P>
</D>
etc.
<D id="20100322.007.07">
  <V n="TYPE"></V>
  <V n="STATUS">0</V>      <-- Document status is "OK" (0)
  <P id="TM000007">
    <V n="TYPE">Optional_Insurance</V>
    <V n="STATUS">49</V>    <-- Page status is "ScanOK" (49)
    etc.
  </P>
</D>
```

“CheckAllIntegrity action”

CheckAllIntegrity action:

The Datacap Studio rrunner actions library includes the CheckAllIntegrity action that checks the structural integrity of a batch.

Library	Action	Description
rrunner	CheckAllIntegrity	Returns True if the current batch meets the requirements that are defined in the document hierarchy; returns False otherwise.

Important: You must run this action within a rule that runs at the batch level.

CheckAllIntegrity does not change the status variable on non-conforming documents. Instead, the action returns False if there is a document integrity issue, which you can use to trigger corrective action.

In the log file that is generated by the Rulerunner task profile (rulerunner_rrs.log), you can see the code that is returned by CheckAllIntegrity if the action returns False. If the batch includes multiple problems, the code represents the last problem. These codes represent different problems:

- 1 = Has more child objects than allowed by *max* attribute
- 2 = Has fewer child objects than required by *min* attribute
- 3 = A child object is not of a type that is supported by parent
- 4 = A child object is in the wrong position as specified by the *pos* attribute

Document integrity problem management

You can manage document integrity problems by routing a batch to a job that fixes the problems.

The Document Integrity ruleset demonstrates how to handle document integrity problems. The Batch Route To Fixup function is started only if CheckAllIntegrity returns a value of False.

A document integrity problem can be pages in the wrong order, or a missing required page. If CheckAllIntegrity identifies a document integrity issue, the application sends the batch to a Fixup job. An operator then fixes the problem and returns the batch to the main workflow. Moving a batch out of the current workflow in this way is known as branching.

Note: The Datacap Studio application wizard generates the Profiler task profile that includes recognition and validation. Therefore, Datacap completes recognition and validation before the batch branches to the Fixup job. So, some pages might have Status = 1, which indicates a problem. You cannot complete the Fixup job unless the Status on all pages is 0, which indicates that there is no problem. You can configure your application to use two separate task profiles so that Datacap branches to FixUp before recognition if there are document integrity problems. For details about using two task profiles, see the topic “Creating the CreateDocs task” on page 210.

The Batch Route To Fixup function uses these rrunner actions to branch to the Fixup job.

Library	Action	Description
rrunner	Task_NumberOfSplits	Specifies the number of jobs to which the batch is sent to before the branch returns to the main workflow (almost always 1).
rrunner	Task_RaiseCondition	Specifies the group index (almost always 0) and the index of the condition to raise, where 0 is the first condition. The index is including on a list on the Datacap Web Client, Administrator tab, Workflow page. To view the Workflow page, start the Datacap Web Client, log on to the TravelDocs application, click the Administrator tab, and click Workflow.

Before Datacap can branch to the FixUp job, you must configure the settings on the Document Integrity Failed condition. See the topic “Configuring branching” on page 56.

TravelDocs: Document creation and page file setup

To create documents and set up the page files, you must create a batch. Then, you can examine the runtime batch folder and review the page data files.

“Running a batch through the workflow”

“Contents of the runtime batch folder”

“Page data files” on page 56

Running a batch through the workflow

Because no functions are added to the CreateDocs ruleset yet, you can run the default version that is generated by the Datacap Studio application wizard.

1. Click the Datacap Studio **Test** tab.
2. In the **Workflow** pane, select the **VScan** task profile under **Main Job**.
3. Click **New** to start a new batch.
4. Click **Process rules for target object** on the main Test tab toolbar.
5. When you are prompted to release the batch, click **Advance**. The Advance command moves the batch to the next step in the workflow (PageID).
6. Click **Process rules for target object** on the main Test tab toolbar.
7. When you are prompted to release the batch, click **Advance**. The Advance command moves the batch to the next step in the workflow (Profiler).
8. Click **Process rules for target object** on the main Test tab toolbar.
9. When you are prompted to release the batch, click **Advance**. The Advance command moves the batch to the next step in the workflow (Verify).
10. On the **Runtime batch hierarchy** tab, expand each document node to see how the individual pages are now grouped into documents. The flight document and the hotel document all have only one page, but two of the car rental documents have multiple pages.
11. Right-click the running batch icon in the **Workflow** pane and choose **Cancel**.

Contents of the runtime batch folder

To examine the runtime batch folder, open the most recent batch folder (C:\Datacap\TravelDocs\batches\batch_identifier).

The runtime batch folder contains the following files:

File	Description
TM00000*.tif	An image-enhanced version of each of the sample image files.
TM00000*.tio	A copy of each of the original image files.
TM00000*.xml	The page data file for each image file.
TM00000*c.xml	The results of full page recognition for each image file.
TM00000*.cco	The fingerprint file for each of the image files.
Profiler.xml	The runtime document hierarchy that is generated by the Profiler task profile.
Profiler_rrs.log	The log file that is generated by the Profiler task profile

File	Description
PageID.xml	The runtime document hierarchy that is generated by the PageID task profile.
pageid_rrs.log	The log file that is generated by the PageID task profile.
VScan.xml	The runtime document hierarchy that is generated by the VScan task profile.
vscan_rrs.log	The log file that is generated by the VScan task profile.
Verify.xml	A copy of the runtime document hierarchy ready for use by the next task profile in the workflow (Verify).

Page data files

The CreateFields action in the Create Fields rule creates a page data file for the current page.

The data file includes all of the fields that are identified for the current page type in the document hierarchy. Each field has an identifier (ID) and three properties: TYPE, Position, and Status.

```
<?xml-stylesheet type="text/xsl" href="..\..\dco.xsl"?>
<P id="TM000001">      <!--Page data file for first page in batch (type Rental_Agreement)
<F id="Pickup_Date">      <!--Pickup_Date field (no data)
  <V n="TYPE">Pickup_Date</V>
  <V n="Position">0,0,0,0</V>
  <V n="STATUS">0</V>
</F>
<F id="Pickup_Location"> <!--Pickup_Location field (no data)
  <V n="TYPE">Pickup_Location</V>
  <V n="Position">0,0,0,0</V>
  <V n="STATUS">0</V>
</F>
etc.
```

Later, other actions can assign different values to these properties and add properties as needed.

TravelDocs: Document integrity management

To manage document integrity issues in the TravelDocs application, you need to configure branching and generate a batch.

“Configuring branching”

“Running a batch with document integrity problems” on page 57

Configuring branching

The Datacap Studio application wizard generates Document Integrity ruleset that is configured to identify document integrity problems and send the batch to the Datacap Desktop Fixup task. However, more setup steps are required in the Datacap Web Client **Administrator** tab to configure the required branching.

To configure branching in the TravelDocs application:

1. Open the **Administrator** tab in the Datacap Web Client for TravelDocs and click **Workflow**.

2. Expand **Main job** and then expand **Profiler**.
3. Select the **Document Integrity Failed** condition and, if necessary, configure the values as follows:
 - **Spawn type:** Branch
 - **Child Job:** Fixup Job
 - **Parent Status:** Pending
 - **Child status:** Pending
 - **Steps:** 1
4. Click **Save Condition** (if you had to change the default values). The application is configured to branch to the Fixup job when document integrity fails, and then return to the main job with a status of pending.
5. Leave the Datacap Web Client open.

Running a batch with document integrity problems

For demonstration purposes, an optional insurance page is added to the end of the batch. Because the page does not immediately follow a car rental agreement page, the **CheckAllIntegrity** action generates an error during document integrity checking.

1. Open C:\Datacap\TravelDocs\images.
2. Make a copy of Images_Page_02.tif and name the copy Images_Page_12.tif. Making a copy creates an orphaned insurance page at the end of the batch.
3. In the Datacap Web Client **Operations** tab, click **VScan**.
4. When the task is completed, click **Stop**.
5. On the **Operations** tab, click **Upload**.
6. When the Upload task displays a message to indicate that the task is complete, click **OK** and then click **Stop**.
7. Start the Datacap Desktop application. In the **Start** menu, select **IBM Datacap Clients > Datacap Desktop**. Log on to the TravelDocs application with the Admin account, and select **PageID** from the **Shortcut** menu.
8. Select **Profiler** from the **Shortcut** menu. When the task completes, click **Stop**.
9. In Datacap Web Client, select the **Monitor** tab to display the **Job Monitor** page. Notice that the batch has two entries in the queue.
 - The first entry indicates that the batch has a status of Pending for the Fixup task.
 - The second entry indicates that the batch has status of Waiting for the Verify task.
10. In Datacap Desktop, log on to the TravelDocs application with the Admin account, and select **Fixup** from the **Shortcut** menu. The batch opens in the Datacap Desktop Fixup window. The last document is selected and the **Comments** field indicates that the document has an invalid member (the orphaned insurance page).
11. Select the page TM000012, click **Delete**, and click **OK** to confirm.
12. Click **Finish** and then click **OK** in the Task Finished message.
13. In the Datacap Web Client **Job Monitor** page, press the F5 key to refresh the view. The Fixup task now has a status of Job done and the batch is now pending for the Verify task.
14. Open the C:\Datacap\TravelDocs\images folder and delete the file Images_Page_12.tif to avoid a recurrence of this problem.

Data recognition

Data recognition is the stage during which you locate the fields that you want to capture and then convert the fields into character-based data.

The data that is obtained from recognition is stored in the page data files that you set up in the document assembly stage.

There are several techniques that you can use to identify pages. The most widely used is fingerprint matching. If you used fingerprint matching for page identification, you most likely used the fingerprint images to define the recognition zones. These zones are the fields that you want to read on each page. If you use full page recognition, you can obtain the field data directly from the full page recognition results. Otherwise, you need to run the recognition engine on each field zone to capture the data.

The other recognition techniques do not use fingerprint zones to locate the field data. Instead, they use text matching or pattern matching to analyze the page and identify the fields.

“Page data recognition”

“Dynamic locale support” on page 60

“Check box options management” on page 65

“Recognizing medical claim forms by using Autofield” on page 69

“TravelDocs: Specification of recognition zones” on page 70

“TravelDocs: Assignment of default rules to the document hierarchy” on page 72

“TravelDocs: Updating the application to manage check box options” on page 74

“TravelDocs: Using pixel threshold check box recognition (optional)” on page 76

Page data recognition

The recognition of page data includes the using fingerprints to identify recognition zones, storing the recognition zone information, and reading data from the page.

“Identifying recognition zones by using fingerprints”

“Recognition zone information storage”

“Reading data from the page” on page 59

Identifying recognition zones by using fingerprints

Datacap uses fingerprints to identify incoming pages. For each incoming page, Datacap generates a fingerprint file that describes the page, and compares the new fingerprint to a fingerprint library for known page types. When Datacap finds a match, it assigns the corresponding page type

The fingerprint library has a second purpose, which is to identify the position of each field for each known page type. There can be many variants of each page type and the position of each field is different for each variant. So you must identify the recognition zones for each variant.

Recognition zone information storage

Datacap Studio stores the coordinates for each field zone as a variable in the document hierarchy.

The pickup date field on the Car Rental #1 page and Car Rental #2 page has the following zones:

- The Car Rental #1 fingerprint has ID 695, so Pos695 defines the position of the pickup date field for Car Rental #1.
- The Car Rental #2 fingerprint has ID 678, so Pos678 defines the position of the pickup date field for Car Rental #2.

You can see the position coordinates for a field by right-clicking the field name in the Document Hierarchy pane. Choose **Manage variables** (the document hierarchy must be locked). Alternatively, you can select the field and look in the Properties pane.

For every page that Datacap identifies a page as a Car Rental #2 rental agreement, the pickup date is at coordinates (579,353,943,415).

Important: The coordinates are relative to a reference point on the page. If the incoming page does not align precisely with the reference page, Datacap calculates an offset and uses it to determine the actual field positions.

Reading data from the page

The method that you use to read data from a page depends on the method that you used to generate the runtime fingerprints.

- If you used full page recognition to generate the runtime fingerprints, you can obtain the field data directly from the fingerprint (CCO) file.
- If you used **AnalyzeImage** to generate the runtime fingerprints, you must use recognition on each of the field zones to obtain the field data.

Populating the page data files using full page recognition results

The Datacap Studio Actions library include actions that take the character data from the fingerprint (CCO) file and apply it to the runtime batch hierarchy.

Library	Action	Description
Zones	ReadZones	Loads the zone position information for the current fingerprint.
Recog_Shared	SnapCCOtoDCO	Transfers the recognition results from the current page's fingerprint (CCO) file to the appropriate field objects in the runtime batch hierarchy.

You must run **ReadZones** before you can run **SnapCCOtoDCO**.

Populating the page data files using zone OCR

If you used **AnalyzeImage** to generate the runtime fingerprints, the following actions are available for zone-based recognition:

Library	Action	Description
Zones	ReadZones	Loads the zone position information for the current fingerprint.

Library	Action	Description
ocr_a	RecognizePageFieldsOCR_A	Recognizes the characters within each of the field zones using the position information in the current fingerprint.
OCR_s	RecognizePageFieldsOCR_S	Recognizes the characters within each of the field zones using the position information in the current fingerprint.
ocr_sr	RecognizePageFieldsOCR_S	Recognizes the characters within each of the field zones using the position information in the current fingerprint.
icr_c	RecognizePageFieldsICR_C	Recognizes the characters within each of the field zones using the position information in the current fingerprint.

Dynamic locale support

The Datacap Recognition Engines use locale variables to validate language and regional settings such as currency, numbers, and date data types. You can use dynamic locale support to set different locale variables on the nodes of your application that use different languages.

The value of the locale variable is inherited by to all of the child nodes of the batch object. Unless a different locale value is assigned to a child node. Then, all of the child nodes of that object inherit the new locale value. For example, if you add a locale value to a Document, that locale value is also assigned to the Pages and fields in that Document. If you then set a different locale value to a Page in the Document, that new value is also assigned to the fields in that Page.

You can set the locale variable for document processing by the application at the system level, the application level, or the node level in the application.

By default, the Windows Regional and Language locale is used system wide for all validations that are done during recognition. Unless the locale variable is set differently elsewhere.

For applications that use different languages, you can set the locale variable at the application level in the Datacap Application Manager. This setting is used on the batch object and is inherited down to of the all child nodes in the batch object. Setting the locale variable on the application level overrides the Windows Regional and Language setting.

If your applications run workflows, documents, pages, or fields that use different languages, you can set the locale variable at any node level in the DCO. Setting the locale variable on these levels overrides the locale value in Datacap Application Manager.

All of the standard actions, including the Recognition Engine, use the Locale property setting for the node to which the action is bound. The action property setting dynamically overrides the locale values that are set at the system, application, or batch object levels. For example, if the `rrSet(en-US,@P.hr_locale)` action is bound on the Page node. That node and its child nodes use the setting for English US (en-US) locale. Regardless of the locale that is set on the application level or in the Batch or Document level of the DCO.

“Setting locale values”

“Recognition language settings” on page 62

“Supported language codes” on page 63

Setting locale values

You set locale values on an application or batch object to validate the language and regional settings for that node. You can dynamically override the locale values that are inherited by the child nodes of the batch object. To override these locale values, assign a different locale value to the child node.

You can set locale variables at the following levels:

System-wide locale settings

Use the Windows Regional and Language settings.

Applications that use one language or multiple languages

Set the locale value at the application level in the Datacap Application Manager.

Applications with workflows, documents, pages, or fields that use different languages

Set the locale value at any node level in the DCO or on an action that is bound to a node.

To set the locale on an application or on the nodes in an application:

1. Assign a locale value to an application.
 - a. In the **Start** menu select **IBM Datacap Services Datacap Application Manager**.
 - b. In the **Applications** pane, select the application for which you want to set the locale.
 - c. Click the **Main** tab.
 - d. In the **Locale** field, select the language that you want and press **Enter**. You can select the **System** option from the **Locale** language list to reset the locale value. When you set the locale in the Datacap Application Manager, at run time the specified value is automatically created. The value is created in the `hr_locale` variable at the batch level of the runtime DCO.
 - e. Run the following steps. Set the locale property on the action to change this value at run time when you must change the current language during task execution.
2. Set the locale value on nodes in the DCO:
 - a. In Datacap Studio, click the **Document Hierarchy** tab.
 - b. Click the **Lock DCO for Editing** icon to lock the application for editing.
 - c. Select the node to which you want to set the locale.
 - d. Right-click and select **Manage Variables** and then click **New**.
 - e. Type `hr_locale` and press **Enter**.
 - f. Type the language code for the `hr_locale`.

- g. Click **Done**, then click **Save Changes**.
 - h. Select the node and click the **Properties** tab to verify that `hr_locale` was added.
 - i. Click **Unlock DCO**.
3. Set the locale property on the action:
 - a. In Datacap Studio, click the **Rulemanager** tab.
 - b. Select the ruleset for which you want to set the locale.
 - c. Click **Lock ruleset for Editing**. Locking the ruleset does not lock the DCO.
 - d. Add the `rrSet` action to the ruleset.

Important: The `rrSet` action must precede the action on the node to which you want to set the locale property.

- e. Using smart parameter syntax, configure the `rrSet` action to set the `hr_locale` variable to the locale value that you want. For example, use `rrSet(en-US,@P.hr_locale)` on a Page object or `rrSet(en-US,@F.hr_locale)` on a field object. The locale setting is used on all of the subsequent actions in the ruleset.
- f. Click **Save** to save the changes to the ruleset. The ruleset is published and unlocked.

Recognition language settings

You can process documents in different languages. You specify the language that is used by the recognition engines by setting the recognition language value on the node. You can set language values to recognize different languages on the documents, pages, and fields of your application.

A recognition engine can support the recognition of multiple languages. For these engines, you can set the language that is recognized for the entire page or for each field on the page. Setting recognition values on the Document and Page levels can recognize multiple language types within a single application. For example, you can configure Page A to be recognized in English and Page B to be recognized in French. Setting locale values at the field level enables multiple language recognition on a single page.

In addition to the locale specification, specific engine settings to configure the recognized language are also available by using Datacap Studio. On the **Zones** tab, you use the appropriate recognition engine tab, such as the **OCR/S** tab for the OCR/S engine. You can set the language settings in the **Language Environment**, **Recognition Setup**, and **Spell Check** sections of the **Properties** tab. If you set these values here, the recognition engine uses these values first. If these settings are not set here, the recognition engine uses the locale setting that is configured by the Datacap Application Manager. In this case, the Datacap Application controls the `hr_locale` variable.

The language value on the **Zones > Properties** tab for the recognition engine is English, even if a value was not set. To confirm that the language value was set to English, you can check the following DCO variables for the recognition engine:

- OCR/S: `s_lg`,
- OCR/A: `y_lg`
- ICR/C: `c_cr`.

The **Properties** tab of the recognition engine also contains a Use Locale setting. If the Use Locale value is set to Yes, the recognition engine must use the value of the

hr_locale variable. Even if the language settings for the engine are set. If you are using OCR/S and are recognizing simplified Chinese, you must set the OCR/S Module setting to Asian recognition. When you use the Asian recognition module, the Filter setting is not used.

Supported language codes

Datacap provides language support for many countries and regions around the world. You use language codes to set the locale that is associated with the language and regional settings on the documents that are processed by your application.

The Recognition Engine actions use the locale property to assign locales to the node to which the action is bound if the engine-specific recognition language settings are not set. For example, if the `rrSet(en-US,@D.hr_locale)` action is bound on a Document node, that node and its child nodes use the English US (en-US) locale. Regardless of the locale setting on the application level or in the DCO. Recognition engines do not necessarily support all of the languages that are specified in the following language tables.

Use the language codes in the following tables to set the locale on Datacap actions.

Eastern European and Russian language codes

Table 13. Supported Eastern European and Russian languages by country

Language	Code
Czech (Czech Republic)	cs-CZ
Croatian (Latin, Bosnia, and Herzegovina)	hr-BA
Croatian (Croatia)	hr-HR
Hungarian (Hungary)	hr-HU
Polish (Poland)	pl-PL
Romanian (Romania)	ro-RO
Russian (Russia)	ru-RU
Slovak (Slovakia)	sk-SK
Turkish (Turkey)	tr-TR

English language codes

Table 14. Supported English languages by country

Language	Code
English (Caribbean)	en-029
English (Australia)	en-AU
English (Belize)	en-BZ
English (Canada)	en-CA
English (Ireland)	en-IE
English (India)	en-IN
English (Jamaica)	en-JM
English (Malaysia)	en-MY
English (New Zealand)	en-NZ
English (Republic of the Philippines)	en-PH

Table 14. Supported English languages by country (continued)

Language	Code
English (United Kingdom)	en-UK
English (United States)	en-US
English (Zimbabwe)	en-ZW

French language codes

Table 15. Supported French languages by country

Language	Code
French (Belgium)	fr-BE
French (Canada)	fr-CA
French (Switzerland)	fr-CH
French (France)	fr-FR
French (Luxembourg)	fr-LU
French (Monaco)	fr-MC

German language codes

Table 16. Supported German languages by country

Language	Code
German (Austria)	de-AT
German (Switzerland)	de-CH
German (Germany)	de-DE
German (Liechtenstein)	de-LI
German (Luxembourg)	de-LU

Spanish language codes

Table 17. Supported Spanish languages by country

Language	Code
Spanish (Argentina)	es-AR
Spanish (Bolivia)	es-BO
Spanish (Chile)	es-CL
Spanish (Columbia)	es-CO
Spanish (Costa Rica)	es-CR
Spanish (Dominican Republic)	es-DO
Spanish (Ecuador)	es-EC
Spanish (Spain)	es-ES
Spanish (Guatemala)	es-GT
Spanish (Honduras)	es-HN
Spanish (Mexico)	es-MX
Spanish (Nicaragua)	es-NI
Spanish (Panama)	es-PA

Table 17. Supported Spanish languages by country (continued)

Language	Code
Spanish (Peru)	es-PE
Spanish (Puerto Rico)	es-PR
Spanish (Paraguay)	es-PY
Spanish (El Salvador)	es-SV
Spanish (United States)	es-US
Spanish (Uruguay)	es-UY
Spanish (Venezuela)	es-VE

Other language codes

Table 18. Other supported languages by country

Language	Language code
Chinese (simplified)	zh-Hans
Dutch (Belgium)	nl-BE
Dutch (Netherlands)	nl-NL
Italian (Italy)	it-IT
Italian (Switzerland)	it-CH
Portuguese (Brazil)	pt-BR
Portuguese (Portugal)	pt-T
Swedish (Finland)	sv-FI
Swedish (Sweden)	sv-SE

For detailed information about Datacap language support, see the Datacap Language Support techdoc at <http://www.ibm.com/support/docview.wss?&uid=swg27035841>

Check box options management

Managing check box options requires that you establish the parent fields and their required variables, and then use either OCR/A recognition or pixel threshold evaluation.

“Check box recognition methods”

“Establishing parent fields” on page 66

“Setting the required variables on the parent field” on page 67

“Implementing the OCR/A check box recognition method” on page 67

“Using the pixel threshold evaluation method” on page 68

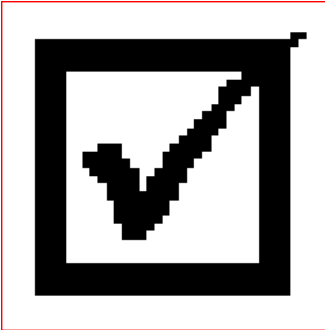
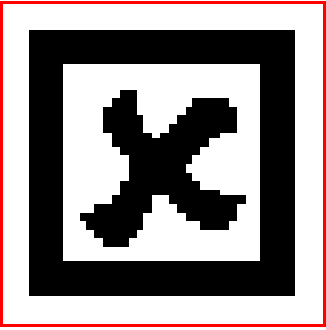
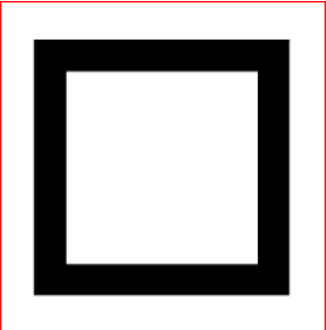
Check box recognition methods

Datacap employs optical mark recognition (OMR) to determine whether a check box option is selected.

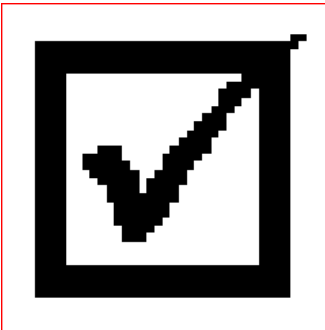
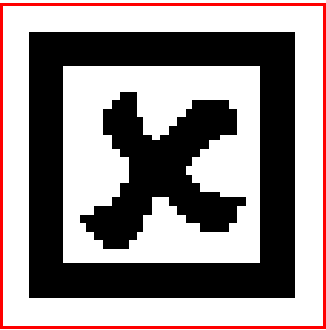
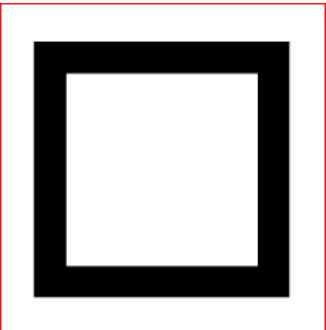
There are two basic OMR techniques.

- **OCR/A check box recognition method:** This method is easy to set up and works well with non-dropout check boxes (where the check box outline remains on the

page image). The method does not work as well with drop-out check box (where the outline drops out during scanning). The OCR/A recognition engine determines whether the specified region represents a selected check box (1) or a non-selected check box (0).

Selected	Selected	Not selected
		

- Pixel threshold evaluation method:** This method is more difficult to set up but is more reliable for drop-out check boxes. The method can also be used to read filled-in *bubbles* (O) on a response form. It calculates the percentage of black pixels within a specified zone and compares the result to a predetermined threshold value. For example, if the threshold is 20%, any OMR zone with more than 20% black pixels is considered selected (1). Any zone with 20% or less is considered not selected (0).

> 20% black	> 20% black	<= 20% black
		

The field setup requirements are the same for both setup check box recognition techniques.

Establishing parent fields

When you process pages with check box options, you must define the check box options as subfields of a parent field. You must also outline the subfields and the parent field when you are drawing the recognition zones.

You defined subfields for the options on the rental agreement page in the TravelDocs application. When you define the recognition zones, you need to define the positions of the parent fields and the subfields.

It might seem easier to draw the child field zone within the check box outline. However, recognition succeeds only if the check boxes on the runtime page align

perfectly with the fingerprint image. Even a slight misalignment can result in a false positive. The best approach is to draw the recognition zone around the check box.

You also defined parent fields and subfields on the optional insurance page, although in this case you created a parent field for each check box. When you define the zones, you need a zone for each parent and each subfield.

Setting the required variables on the parent field

To process check box options as OMR fields, you must set the **RecogType** variable on the parent field equal to 4. This setting instructs the Datacap recognition engine to use mark recognition rather than character recognition.

If the business requirements indicate that multiple options within the group can be selected, you must set the **MultiPunch** variable on the parent field to 1.

The **RecogType** and **MultiPunch** variables are not included by default, so you must add them manually to the parent object, as described later for the TravelDocs application see “Setting the required variables on the Options and Insurance fields” on page 74.

Implementing the OCR/A check box recognition method

The OCR/A check box recognition method uses the RecognizeFieldOCR_A action to determine whether the zone represents a selected check box or a non-selected check box. You must include this action in a rule that is bound to the parent zone. You must also configure the OCR/A settings for the parent zone (not the individual check box subfields).

You configure the OCR/A settings for specific zones by using the **OCR/A** tab in the Properties pane on the Datacap Studio **Zones** tab. The **OCR/A** tab is not displayed by default, but you can activate it by right-clicking any existing tab, choosing **Show tabs**, and selecting **OCR/A**.

Click the **OCR/A** tab to access the settings that the OCR/A recognition engine uses when it completes recognition on the selected field.

There are three OMR settings:

Table 19. OMR settings

Setting	Description
Check mark type	Select Square background to read non-dropout check box. This setting is stored in the document hierarchy by using the <i>OMRType</i> variable, where 0 is “Square background”. <V n="OMRType">0</V>
Length	The Length setting reflects the number of OMR subfields and is set automatically.
Multipunch	The Multipunch setting is the same as the <i>MultiPunch</i> variable you looked at earlier, where 1 is Yes. <V n="MultiPunch">1</V>

Using the pixel threshold evaluation method

The pixel threshold evaluation method uses the `RecogOMRThreshold` action in the `Recog_Shared` library.

Specifying the threshold and background levels

The `RecogOMRThreshold` action takes two parameters:

- **Threshold:** Specifies the percentage of black pixels over which the option is considered selected.
- **Background:** Used to determine the confidence level and specifies the percentage that can be attributed to the check box outline plus any scanner noise:
 - Any zone with a percentage of black pixels below this value is considered not selected with high confidence. Any zone with a percentage of black pixels between this value and the threshold value is considered not selected with low confidence.
 - Any zone with a percentage of black pixels over $(2 * \text{Threshold} - \text{Background})$ is considered selected with high confidence. Any zone with a percentage of black pixels between Threshold and $(2 * \text{Threshold} - \text{Background})$ is considered selected with low confidence.

However, if `MultiPunch=0` (or is not specified) then only the zone with the highest percentage is selected.

For example, if the threshold value is 20 and the background value is 15 then the high confidence threshold is $(2 * 20 - 15) = 25$. If you run `RecogOMRThreshold(20,15)` on an OMR group field with `MultiPunch=1`, then the following is true.

- Any zone with more than 25% black pixels is considered selected with high confidence
- Any zone with between 20% and 25% is considered selected with low confidence
- Any zone with between 15% and 20% is considered not selected with low confidence
- Any zone with 15% or less black pixels is considered not selected with high confidence

Determining the appropriate threshold and background values

To determine appropriate values for the threshold and background parameters, you must determine the percentage of pixels within the OMR zone that can be attributed to the check box outline plus any scanner noise. The easiest way to make this determination is to run a page that contains both checked and cleared option boxes through the workflow. Then get the pixel counts from the page data file.

When Datacap runs a `RecogOMRThreshold` action, it counts the number of black pixels within each OMR zone. Datacap then writes the resulting values to the page data file as a density string.

```
</F>
<F id="Options">
  <V n"Type">Options</V>
  <V n"Position">1171,327,1518,622</V>
  <V n"STATUS">0</V>
  <V n"DensityString">FBG</V>
```

```

<C cn="10" cr="1440,405,1490,418">49</C>
<C cn="10" cr="1440,475,1490,525">48</C>
<C cn="10" cr="1440,541,1490,591">49</C>
</F>

```

The `DensityString` has one character per OMR zone. In this code example, the **Options** field has three OMR zones and the `DensityString` value is FBG. Each of the characters corresponds to a percentage value according to the following formula.

Percentage black pixels = character's ASCII code value minus 48.

In this example:

- The ASCII code for each of the three characters is 70, 66, and 71 respectively.
- The percentage of black pixels for each of the three zones is 22% (70-48), 18% (66-48), and 23% (71-48), respectively.

After you obtain the percentage values, you can then refer to the original page image to see if the corresponding check box is selected. This example was obtained from a page where the first and third options were selected, and the second option was not selected.

Checkbox	Percentage filled
Check mark	22%
Empty Square	18%
Check mark	23%

Based on these three check boxes, you set the threshold and background values somewhere between 18 and 22. (Fractional values are permitted for the threshold and background parameters.) You can test the values scanning additional pages and checking their density strings before setting final values.

Implications of using `RecogOMRThreshold`

The `RecogOMRThreshold` action relies upon pixel counts within the OMR zone. So it is important that all OMR zones have the same dimensions, or as close as possible.

Drawing OMR zones on the Datacap Studio **Zones** tab can sometimes be difficult. You can establish approximate zone boundaries by drawing the bounding boxes on the **Image View** tab. Then edit the coordinates in the *Pos* variables field in the Properties pane.

The coordinates correspond to the upper left corner of the bounding box, such as the *x1*, *y1* coordinate, and the lower right corner (*x2*, *y2*). In this example, you enter *x1*, *y1*, *x2*, *y2* in the *Pos* variable field.

Recognizing medical claim forms by using Autofield

Datacap Medical Claims uses Autofield in addition to fingerprint matching to recognize medical claim pages and fields.

While Datacap compares fingerprint images to a library of fingerprints for known page types, Autofield works fitting pixels in an image to a particular zone. As a consequence, changing a zone by moving it even a few pixels makes scanned medical claims unrecognizable to Medical Claims.

Adding fields to pages in the DCO affects the accuracy of Autofield unless steps are taken so that Autofield knows how to handle them.

Adding a field to a page

In some cases, such as to store metadata, you need to add a field to a page. If you are creating a new field on a page, you need to specify that Autofield should not identify the field for zoning by adding the *RecogType* variable to the field and setting its value to 5.

In addition, when adding fields to a page, the new fields will show up in the DCO in the exact order you arrange them in the hierarchy. However, these fields will be added to the end of the *setup.xml* file in the order that they are created. After adding any new fields to a page, you need to manually edit the *setup.xml* to put the fields in the correct order.

Adding a field that is not under a page

When you add a field to the DCO that you do not want to be recognized as a field on any page, you need to add the *Autofield* variable to the new field and set its value to -1.

TravelDocs: Specification of recognition zones

You must define the positions of the various fields for one variant of each page type. As Datacap Studio locates field zones that are on the fingerprint images, it writes the position information for each field into the document hierarchy.

“Creating the text zones on the Rental_Agreement page”

“Creating the OMR zones on the Rental_Agreement page” on page 71

“Creating the zones for the other page types” on page 71

Creating the text zones on the Rental_Agreement page

Use Datacap Studio to create the text zones for the TravelDocs Rental_Agreement page.

To create the text zones on the Rental_Agreement page:

1. In the Fingerprints pane on the Datacap Studio **Zones** tab, select the first **Rental_Agreement** page (Car Rental #1).
2. In the Image View pane, click **Zoom** to enlarge the page so you can see the fields clearly.
3. In the Document hierarchy pane, click **Lock DCO for editing**.
4. In the Document hierarchy pane, expand the Rental_Agreement page and select the **Pickup_Date** field. Use the mouse to draw a bounding box around the pickup date on the page image.
5. Repeat for the **Pickup_Location**, **Return_Date**, **Return_Location**, **Car_Type**, and **Total_Cost** fields. Provide enough horizontal space around the field in case the text on the runtime page image is longer than the text on the fingerprint image.
6. In the Document hierarchy pane, click **Save** and then click **Unlock DCO**.
7. In the Document hierarchy pane, select any one of the fields you defined (for example, the **Car_Type** field). Then, look in the Properties pane at the lower left of the **Zones** tab. Make sure that the zone coordinates are saved in the field's **Pos<id>** variable, where *<id>* is the fingerprint you selected.

Creating the OMR zones on the Rental_Agreement page

You must create OMR zones to recognize check boxes on the page.

The Rental_Agreement page type includes an Options field with three subfields:

- Nav_System
- Child_Seat
- Fuel_Service

These options are check box options that you manage by using optical mark recognition (OMR). OMR zones must always be subfields of a parent field. There are two approaches you can use:

- You can create a single parent field that contains all of the OMR subfields. This tutorial uses this technique for the rental agreement page.
- You can create a separate parent field for each OMR subfield. This tutorial will use this technique when you do the optional insurance page.

To create the OMR zones on the rental agreement page:

1. In the Datacap Studio **Zones** tab Document Hierarchy pane, click **Lock DCO**.
2. In the Document Hierarchy pane, expand the **Rental_Agreement** page if necessary. Then, select the **Options** field and draw a bounding box around the Options region on the page image.
3. Expand the **Options** field node, select the **Nav_System** subfield, and draw a bounding box around the GPS Navigation check box. Then, repeat for the **Child_Seat** and **Fuel_Service** options. Try to make the bounding boxes as close to the same size as possible.
4. In the Document Hierarchy pane, click **Save**.

Creating the zones for the other page types

Use Datacap Studio to create the zones for the other page types in the TravelDocs application.

To create the zones for the other page type:

1. In the Fingerprints pane, select the first **Optional_Insurance** page (Car Rental #1).
2. In the Document hierarchy pane, expand the **Optional_Insurance** page. Select the **CDW** field, and draw a bounding box around the Collision Damage Waiver option on the page image.
3. Repeat for the **PAI**, **PEP**, and **ELP** fields. The size of the bounding boxes around the parent fields is not critical because it determines only what gets displayed to the operator during verification.
Attention: The parent field defines the region that is displayed during verification, so the dimensions are not critical.
4. In the Document hierarchy pane, expand the **CDW** field node. Select the **CDW_Option** subfield, and draw a bounding box around the **Collision Damage Waiver** check box on the page image. Try to make the bounding box the same size as the ones you drew on the rental agreement page.
5. Repeat for the **PAI_Option**, **PEP_Option**, and **ELP_Option** subfields. Try to make the bounding boxes around the check boxes the same size as the ones you drew for the options on the previous page.
6. In the Document hierarchy pane, click **Save**.
7. In the Fingerprints pane, select the first **Air_Ticket** page (Airline #1).

8. In the Document hierarchy pane, expand the **Air_Ticket** page. Then, select each of the fields in turn and draw a bounding box around the corresponding region on the page image. Then, click **Save**.
9. Repeat for each of the fields on the first **Room_Receipt** page (Hotel #1).
10. In the Document hierarchy pane, click **Save** and then click **Unlock DCO**.

TravelDocs: Assignment of default rules to the document hierarchy

When the Datacap Studio application wizard generates the application framework, it attaches only default rules to default elements. Therefore, you must attach rules to the new elements that you created.

“Assigning the default page level rules to new pages”

“Assigning the default field level rules to new fields”

“Updating the Recognize Page rule” on page 73

“Running a batch through the workflow” on page 73

Assigning the default page level rules to new pages

When you built the document hierarchy, you renamed the default page type from Page to Rental_Agreement, which therefore includes the default rules. However, you must assign rules for the other page types that you created.

To assign the default page level rules to the new pages:

1. On the Datacap Studio **Rulemanager** tab, in the Document Hierarchy pane, click **Lock DCO**.
2. Expand the document hierarchy so that you can see all the page nodes (Rental_Agreement, Optional_Insurance, Air_Ticket, and Room_Receipt).
3. In the Rulesets pane, expand each of the rulesets so you can see the rules that they contain (VScan, ImageFix Load Settings, Enhance Image, PageID, and so on).
4. In the CreateDocs ruleset, select the **Create Fields** rule.
5. In the Document Hierarchy pane, select the page node **Optional_Insurance**. Then, click **Add to DCO** on the side of the Rulesets pane. The Create Fields rule is added to the Optional_Insurance page's Open element.
6. Repeat for the Air_Ticket and Room_Receipt pages.
7. In the Recognize ruleset, select the **Recognize Page** rule. Then, add the rule to the Optional_Insurance, Air_Ticket, and Room_Receipt pages.
8. Repeat to add the Validate: Validate Page, Routing: Routing Rule 1, and Export: Export Page Fields rules to the same pages. Each of the pages now has an Open element.
9. Click **Save**.

Assigning the default field level rules to new fields

When you built the document hierarchy, you renamed the default field type from Field to Pickup_Date, which therefore includes the default rules. However, you must assign rules for the other fields that you created.

To assign the default field level rule to the new fields:

1. In the Document Hierarchy pane, make sure that the DCO is still locked for editing.
2. Expand the document hierarchy so you can see all the field nodes (Pickup_Date, Pickup_Location, and others.).

3. In the Clean ruleset, select the **Fields Clean** rule.
4. In the Document Hierarchy pane, select the field node **Pickup_Location**. Then, click **Add to DCO** on the side of the Rulesets pane. The Fields Clean rule is added to the Pickup_Location field's Open element.
5. Repeat for the remaining fields. But do not add the rule to the **Options** field and its subfields on the rental agreement page, or to the fields on the optional insurance page. They are the container groups for the check box options. Also, the **Total_Cost** and **Return_Date** field definitions are shared across multiple pages. So you are only able to add the rule to the first instance of each field.
6. When you are finished, click **Save** and then click **Unlock DCO**.

Updating the Recognize Page rule

The purpose of the Recognize Page rule is to locate the fields on each page and capture the data.

The default Recognize Page rule uses the information in the document hierarchy to locate the field zones. It then completes text recognition on those zones by using OCR_s to extract the data.

In the TravelDocs application, you completed full page OCR during fingerprint creation and page identification. So, it is not necessary to do OCR again on the individual fields. Instead, you can use the SnapCCOtoDCO action to take the recognition data from the runtime fingerprint CCO file and apply it to the runtime batch hierarchy.

To update the Recognize Page rule:

1. On the Datacap Studio **Rulemanager** tab, select the **Recognize** ruleset and click **Lock/Unlock ruleset** to lock the ruleset for editing.
2. Expand the **Recognize** ruleset completely.
3. Right-click the **RecognizePageFieldsOCR_S** action and choose **Remove**.
4. Click the **Actions** library tab.
5. Expand the **Recog_Shared** library and select **SnapCCOtoDCO**.
6. Make sure **Recognize: Page Function 1** is selected in the Rulesets pane.
7. Click **Add to function** at the left side of the Actions Library pane.
8. In the Rulesets pane, click **Save**. Then, click **Lock/Unlock ruleset** and choose **Publish Ruleset**.

Running a batch through the workflow

After you attach the default rules to the document hierarchy, you can run a batch through the workflow to view the progress of the application.

1. Click the Datacap Studio **Test** tab.
2. In the **Workflow** pane, select the **VScan** task profile under **Main Job**.
3. Click **New** to start a new batch.
4. Click **Process rules for target object** on the main Test tab toolbar.
5. When you are prompted to release the batch, click **Advance**. The batch is moved to the next step in the workflow, which is PageID.
6. Click **Process rules for target object** on the main Test tab toolbar and wait while the task profile launches. It might take a few moments as Datacap must run full page OCR on all the images in the images folder.
7. When asked if you want to release the batch, click **Advance**. The batch is moved to the next step in the workflow, which is Profiler.

8. Click **Process rules for target object** on the main Test tab toolbar and wait while the Profiler task profile runs.
9. When asked if you want to release the batch, click **Advance**. The batch is moved to the next step in the workflow, which is Verify.
10. On the **Runtime batch hierarchy** tab, expand the first car rental agreement page to see each of the defined fields and the associated data.
Attention: The **Pickup_Date** field in one of the sample images includes an intentional blurred character so that you can examine recognition confidence levels.
11. Because you have more work to do before you are ready to run the Verify task profile, right-click the batch in the **Workflow** pane and choose **Cancel**.

TravelDocs: Updating the application to manage check box options

You need to configure required variables, specify the check mark type, and create a rule to recognize the OMR fields to manage check box options.

“Setting the required variables on the Options and Insurance fields”

“Specifying the check mark type”

“Creating a rule to recognize the OMR fields” on page 75

“Adding the Recognize OMR Fields rule to the document hierarchy” on page 75

“Running a batch through the workflow” on page 76

Setting the required variables on the Options and Insurance fields

When you are updating the TravelDocs application, you must use Datacap Studio to set the required variables on the Options and Insurance fields.

To set variables on the Options and Insurance fields:

1. On the Datacap Studio **Rulemanager** tab in the **Document Hierarchy** pane, click **Lock DCO**.
2. Expand the **Car_Rental** document and the **Rental_Agreement** page.
3. Right-click the **Options** field and choose **Manage variables**.
4. Click **New**, type **RecogType**, and press **Enter**. Then, click **New**, type **MultiPunch**, and press **Enter**.
Attention: Variables are case-sensitive, so make sure that you capitalize **RecogType** and **MultiPunch** as shown here.
5. Enter the values **RecogType=4** and **MultiPunch=1**. Then, click **Done**.
6. Expand the **Optional_Insurance** page.
7. Right-click the **CDW** field and choose **Manage variables**.
8. Click **New**, type **RecogType**, and press **Enter**. **MultiPunch** is not required because the parent field has only one subfield.
9. Enter the value **RecogType=4** and click **Done**.
10. Repeat for the other parent fields (**PAI**, **PEP**, and **ELP**).
11. In the **Document Hierarchy** pane, click **Save** and then click **Unlock DCO**.

Specifying the check mark type

When you use the OCR_A engine for check mark recognition, you must specify whether the check marks have bounding boxes.

The default setting for check marks is for no bounding box (Clear Background). Because all of the check marks in the sample images have square bounding boxes, you must change the **Checkmark Type** setting in on the **OCR/A** settings tab. The settings tabs for the various recognition engines are displayed along the bottom of the Properties pane on the Datacap Studio **Zones** tab.

1. Click the Datacap Studio **Zones** tab.
2. In the **Document Hierarchy** pane, click **Lock DCO**.
3. Expand the **Rental_Agreement** page and select the **Options** field.
4. Look at the bottom of the Properties pane. If the **OCR/A** tab is not visible, right-click any existing tab and choose **Show tabs** and select the **OCR/A** option.
5. Click the **OCR/A** tab.
6. In the Properties pane, scroll down to the **OMR** section and set the **Checkmark type** to **Square background**.
7. In the Document Hierarchy pane, click **Save**.
8. Expand the **Optional_Insurance** page and do the same for the **CDW**, **PAI**, **PEP**, and **ELP** fields.
9. In the **Document Hierarchy** pane, click **Save** and then click **Unlock DCO**.

Creating a rule to recognize the OMR fields

You must use Datacap Studio to create the rule to recognize OMR fields.

1. In the Rulesets pane, select the **Recognize** ruleset and click **Lock/Unlock ruleset** (padlock) to lock the ruleset for editing.
2. Right-click the **Recognize** ruleset and choose **Add Rule**.
3. Rename the new rule from **Rule1** to **Recognize OMR Fields**.
4. Rename the default function from **Function1** to **Recognition: OMR**.
5. Click the **Actions library** tab.
6. Expand the **ocr_a** library and select **RecognizeFieldOCR_A**.
7. Make sure the **Recognition:OMR** function is selected in the Rulesets pane.
8. Click **Add to function** at the left side of the Actions Library pane.
9. In the Rulesets pane, click **Save**. Then, click **Lock/Unlock ruleset** and choose **Publish Ruleset**.

Adding the Recognize OMR Fields rule to the document hierarchy

You must add the Recognize OMR fields rule to the document hierarchy so that Datacap can recognize check marks.

To add the Recognize OMR Fields rule to the document hierarchy:

1. In the Document Hierarchy pane, click **Lock DCO**.
2. Expand the document hierarchy so the **Options** field on the **Rental_Agreement** page and **CDW**, **PAI**, **PEP**, and **ELP** parent fields on the **Optional_Insurance** page are visible.
3. In the Recognize rule set, select the **Recognize OMR Fields** rule.
4. In the Document Hierarchy pane, select the **Options** field node. Then, click **Add to DCO** on the Rulesets pane. The Recognize OMR Fields rule is added to the Open element of the **Options** field.
5. Repeat to add the rule to each of the parent fields on the **Optional_Insurance** page.
6. In the Document Hierarchy pane, click **Save** and then click **Unlock DCO**.

Running a batch through the workflow

You can run a batch through the workflow in Datacap Studio to determine if the application recognizes check marks.

1. Run a batch through the VScan, Page ID, and Profiler task profiles as described earlier (see “Running a batch through the workflow” on page 73).
2. When the Profiler task completes and you advance the batch to the Verify task, expand the first car rental page on the **Runtime batch hierarchy** tab. You can see the fields and the associated data. The value of the Options field is 001, which indicates that Datacap interpreted the first option as Not selected, the second option as Not selected, and the third option as Selected.
3. Expand the first optional insurance page and confirm that the values are correct.
4. Right-click the batch in the **Workflow** pane and choose **Cancel**.

TravelDocs: Using pixel threshold check box recognition (optional)

For demonstration purposes, you can go through the pixel threshold method.

“Updating the Recognize OMR Fields rule to use RecognOMRThreshold”

“Determining appropriate threshold and background settings”

Updating the Recognize OMR Fields rule to use RecognOMRThreshold

You must use Datacap Studio to update any rule, including the Recognize OMR Fields rule.

To update the Recognize OMR Fields rule to use RecognOMRThreshold:

1. In the **Rulesets** pane, select the **Recognize** ruleset and click **Lock/Unlock ruleset** to lock the ruleset for editing.
2. Expand the **Recognize** ruleset, the **Recognize OMR Fields** rule, and the **Recognition: OMR** function.
3. Right-click the **RecognizeFieldOCR_A** action and choose **Remove**.
4. Click the **Actions library** tab.
5. Expand the **Recog_Shared** library and select **RecognOMRThreshold**.
6. Make sure the **Recognition:OMR** function is selected in the Rulesets pane.
7. Click **Add to function** at the left side of the Actions Library pane.
8. Select the **RecognOMRThreshold** action and set the parameters in the **Properties** pane as follows:
 - Threshold = 20
 - Background = 20

Attention: These values are the starting values, which you can later recalculate with more accurate values.

9. In the Rulesets pane, click **Save**. Then, click **Lock/Unlock ruleset** and choose **Publish Ruleset**.

Determining appropriate threshold and background settings

The parameters on the **RecognOMRThreshold** action include a pair of numerical values. The first number is the threshold parameter and the second number is the

background parameter. Obtaining optimum values requires that you run multiple sample pages through the workflow and then check the density strings and confidence levels.

“Checking the option values and obtaining the density string values”

“Interpreting the density string values”

Checking the option values and obtaining the density string values:

To check the option values and obtain the density string values, you must run a batch in Datacap Studio. Then, open the page data file in the runtime batch folder.

1. Run a batch through the VScan, Page ID, and Profiler task profiles as described in the “Running a batch through the workflow” on page 89 topic.
2. When the Profiler task is completed and you advance the batch to the Verify task, expand the first car rental page on the **Runtime batch hierarchy** tab. You see the fields and the associated data. The value of the Options field is 001, indicating that Datacap interpreted the first option as not selected. Datacap interpreted the second option as not selected, and the third option as selected.
3. Select the **Options** field in the runtime batch hierarchy. Then, click the **Image** tab in the center pane of Datacap Studio if it is not already visible. The Options zone is highlighted. The first and second options are not selected, and the third is selected. In this case Datacap determined the values correctly. However, depending on the amount of space around the check boxes, your version might not work. Even if the 20,20 values worked, you must continue to determine optimum parameter values.
4. Open the most recent runtime batch folder in C:\Datacap\TravelDocs\batches.
5. Open the file tm000001.xml and scroll almost to the bottom to see the **Options** field data.
6. Make a note of the DensityString value (ACG in this example) and then close the file.

Important: Note the three lines that follow the density string line. These lines represent the three check box options. The values 48 and 49 are the ASCII values for 0 (not selected) and 1 (selected). The cn attributes represent the confidence level, where 10 is high confidence and 5 is low confidence.

7. Repeat the steps to check the values for the Insurance options and make a note of the DensityString value in tm000002.xml.

From tm000002.xml:

```
CDW: <V n="DensityString">C</V>
PAI: <V n="DensityString">C</V>
PEP: <V n="DensityString">B</V>
ELP: <V n="DensityString">G</V>
```

Interpreting the density string values:

By using the density string characters and the corresponding formula, you can calculate the percentage of black pixels for each OMR zone.

The previous examples produce these results.

Parent field	Subfield	Checkbox	Density value	Percentage filled
Options	Nav_System	Cleared	A	17%

Parent field	Subfield	Checkbox	Density value	Percentage filled
	Child_Seat	Cleared	C	19%
	Fuel_Service	Checked	G	23%
Insurance	CDW	Cleared	C	19%
	PAI	Cleared	C	19%
	PEP	Cleared	B	18%
	ELP	Checked	G	23%

By using the data in first table, you can determine that threshold = 20.5 and background = 20 represent good values. The $(2 * \text{threshold} - \text{background})$ formula sets the high-confidence threshold at 21, but you can scan more pages and check their density strings before you set final values.

Parent field	Subfield	Checkbox	Percentage filled	Result in runtime hierarchy
Options	Nav_System	Cleared	17%	Option: Selected Confidence: High
	Child_Seat	Cleared	19%	Option: Not selected Confidence: High
	Fuel_Service	Checked	23%	Option: Selected Confidence: High
Insurance	CDW	Cleared	19%	Option: Selected Confidence: High
	PAI	Cleared	19%	Option: Not selected Confidence: High
	PEP	Cleared	18%	Option: Not selected Confidence: High
	ELP	Checked	23%	Option: Not selected Confidence: High

Because the OMR zones that you drew are probably different from the ones that generated the data in second table, your values are different. Use the density characters that you obtained to determine appropriate values for the threshold and background parameters, and update the Recognize OMR Fields rule. Then, run a new batch and review the results, including the confidence values in the page data files.

Data Validation

Data validation determines whether captured data complies with the data integrity rules that are defined in your business requirements.

For example, when you established the business rules for the TravelDocs application, you tested if the cost fields are in a valid currency format. If the car type on the rental agreement page is one of a set of predefined values. And if the total cost on the air ticket page equals the air fare plus taxes.

A validation failure does not necessarily mean that the original page contains invalid data. It might mean that the recognition engine failed to recognize one or more characters correctly. You can change the status of the page, which contains the data that failed validation. This change ensures that the page is displayed to an operator for verification.

“Validate the data”

“TravelDocs: Update the application to complete validation” on page 84

Validate the data

Datacap completes validation by using rules that you create and attach to specific items in the document hierarchy.

The purpose of validation is to determine whether captured data conforms to specified business rules. For example:

- Does an expense lie within permitted limits?
- Are dates valid and within a permitted range?
- Is the total cost calculated correctly?
- Does the vendor information match the information that is stored in a database of approved vendors?
- Does a field value match one of a set of permitted values?

To check whether an expense lies within permitted limits, you might first create a rule that does the following actions.

- Ensures that the expense field contains numbers in a valid currency format
- Determines whether the value is less than or equal to the maximum permitted limit
- Does exception handling if the value is invalid or above the permitted limit

You can then attach the rule to the expense field in the document hierarchy.

“Check data format validity”

“Validate calculated fields” on page 80

“Show validation failures to an operator” on page 82

“Use external data sources during validation” on page 83

“Manage validation errors” on page 84

Check data format validity

Before, you can apply specific business rules to a field, you must confirm that the data format is valid.

For example, you cannot test whether an expense lies within allowed limits until you determine that the field contains a valid currency value.

The business requirements of the application specify valid formats for all of the fields that your application is testing. The following examples describe the values of three acceptable currencies as defined by the business requirements for the TravelDocs application.

- \$477.82
- 824.83
- 254.40 USD

The Validations actions library includes several actions that test the format of a field, including.

Library	Action	Description
Validations	IsFieldCurrency	Returns True if the field is a number and includes a two-digit decimal amount; returns False otherwise. The action ignores any leading currency symbol (for example, \$).
Validations	IsFieldDate	Returns True if the field is in one of the supported date formats; returns False otherwise.
Validations	IsFieldLengthMax	Returns True if the field contains no more than the specified number of characters; returns False otherwise.
Validations	IsFieldLengthMin	Returns True if the field contains at least the specified number of characters; returns False otherwise.
Validations	IsFieldPercentAlpha	Returns True if the field contains no numbers or special characters; returns False otherwise.
Validations	IsFieldPercentNumeric	Returns True if the field contains no letters or special characters; returns False otherwise.

For detailed information on these and other actions in the Validations library, select the action on the **Actions Library** tab and click **Display information**.

Validate calculated fields

You can run validation to ensure that a calculated value is correct.

A calculated field obtains its value from one or more independent fields. For example, on the TravelDocs air ticket, the total cost equals the airfare plus taxes and fees.

In the air ticket example, you can use validation to ensure that the total cost is correct, based on the airfare and tax fields. An error does not necessarily mean that the value was calculated incorrectly on the original page. It might mean that one of

the values was recognized incorrectly. You can change the status of the page, which contains the data that failed validation. This change ensures that the page is displayed to an operator for verification.

The Validations actions library includes a **CalculateFields** action completes arithmetic operations on number field values.

Library	Action	Description
Validations	CalculateFields	Returns True if the arithmetic expression is valid; returns False otherwise.

The following example returns True if the total cost equals the airfare plus taxes.

```
CalculateFields("'Total_Cost' = 'Airfare' + 'Taxes'")
```

The fields that you reference must contain number data. Otherwise, the action fails.

Because a calculation involves multiple field values, and because the CalculateFields action operates only on child objects, you cannot complete the calculation at the field level. Instead, you must complete the calculation at the page level.

Important: You can complete calculations on values from different pages or different documents within the batch. To do so, you must use variables.

When to do validation on calculated fields

The **CalculateFields** action works only on number fields. If your Validation rule set includes rules to validate and possibly correct the format of individual fields (for example, removing a “USD” suffix from a currency field). You must run the page-level validation after you completed all field-level validations. For example, an airline ticket has an associated series of fields. You can validate this associated series of fields in sequence before a full page-level validation by using the Close element of the page.

- Air_Ticket
- Open
 - Item Cost
 - Open
 - (global)
 - Validate: Validate Currency field
 - Close
 - Taxes
 - Open
 - (global)
 - Validate: Validate Currency field
 - Close
 - Total Cost
 - Open
 - (global)
 - Validate: Validate Currency field
 - Close

- Close
- (global)
 - Validate: Validate Total Cost

The `Validate Total Cost` rule runs after Datacap finishes processing all of the child fields on the page.

Show validation failures to an operator

To determine the pages that are to be displayed to an operator, Datacap maintains a *Status* variable for each object in the runtime batch hierarchy. For example, a status of 0 indicates that operations on the object were successful while a status code of 1 indicates a problem or potential problem.

Datacap updates the status when rules are started.

Datacap uses other status codes, such as 49, which indicates that a page was scanned successfully. By default, Datacap displays all pages, but you can configure the TravelDocs application to display only pages with a status of 1. For more information, see the topic “Page status” on page 95.

The following problems result in a status code of 1.

- Unrecognized or low confidence characters: By default, if a page has any low confidence characters, Datacap sets the page status to 1. Fields with low confidence characters are displayed in yellow in the verify panel.
- Validation failures: If a field fails validation, Datacap sets the field status to 1 and the page status to 1 and displays the field in red in the verify panel.

Important: When there is a validation failure, Datacap sets all parent objects’ status to 1, including the batch object. For example, if a subfield fails, Datacap sets the parent field to 1. The parent of the field (the page) is set to 1. The parent document of the page is set to 1. The parent batch object status is set to 1. There is only one batch object in a batch.

Use of the **Status_Preserve_OFF** action in the **runner** library to set the status correctly. This action and the related **Status_Preserve_ON** action determine whether validation rules can update an object's status.

Library	Action	Description
runner	Status_Preserve_OFF	Turns the Status Preserve setting of a page and its child fields to OFF, meaning validation rules can update an object's status if a validation fails.
runner	Status_Preserve_ON	Turns the Status Preserve setting of a page and its child fields to ON, meaning validation rules cannot change an object's status.

In most situations, you want to make sure **Status Preserve** is turned OFF at the start of validation.

The default page-level rule **Validate Page** in the **Validate** ruleset that is generated by the Application wizard sets **Status Preserve** to OFF for you.

The rule is attached to the default page type, but you must attach it manually to any new pages that you create.

Reading the status variable

You cannot check the status variable for a page or field from within Datacap Studio. So, you must read the runtime batch data files:

- You can get the status of each page from the task profile's data file (for example, `Profiler.xml`).
- You can get the status of each field from the page data files (for example, `tm000001.xml`).

Profiler.xml (page status)	tm000001.xml (field status)
<pre><P id="TM000001"> <V n="TYPE">Rental_Agreement</V> <V n="STATUS">1</V> <V n="IMAGEFILE">tm000001.tif</V> etc. </P></pre>	<pre><F id="Pickup_Date"> <V n="TYPE">Pickup_Date</V> <V n="Position">194,402,563,458</V> <V n="STATUS">0</V> <C cn="10" cr="203,416,225,438">77</C> <C cn="10" cr="230,423,245,438">111</C> etc. </F></pre>

In `Profiler.xml`, `<P id="TM000001">` is the page definition. In `tm000001.xml`, `<F id="Pickup_Date">` is the field definition.

Use external data sources during validation

You might need to compare runtime field values to an external data source to determine whether the values on a page are valid. For example, you might need to determine whether the vendor information matches the information that is stored in a database of approved vendors.

The Lookup library includes actions for connecting to external data sources and running SQL statements. The available actions include the following.

Library	Action	Description
Lookup	OpenConnection	Uses a data source name or connection string to open a connection to a database.
Lookup	ExecuteSQL	Runs a SQL statement. Returns True if the SQL statement runs successfully and any SELECT statement returns a value.
Lookup	CloseConnection	Closes an open database connection.

For an example of how to use an external data source to complete validation, see the “Use a lookup database to validate the car type” on page 87 topic.

Manage validation errors

Validation actions set the Status variable of the object if there is an error. The Status variable determines which pages Datacap displays to the operator. However, you might need to complete extra error handling within the application.

The following rule has two functions and each function has two actions. Suppose that Function1 contains validation actions and Function2 is there to handle validation errors.

```
Rule: Validation rule
  Function1: Perform validation
    Action1
    Action2
  Function2: Handle validation errors
    Action3
    Action4
```

Using the rules for execution of functions and actions within a rule:

- If Action1 returns False, Datacap skips Action2 and runs Function2.
- If Action1 returns True and Action2 returns False, Datacap runs Function2.
- If Action1 and Action2 both return True, Datacap does not run Function2.

TravelDocs: Update the application to complete validation

You can update your application to complete validation by using actions, external data sources, or dictionaries.

After you create a batch with the updated application, you can examine status values, and create more recognition zones.

“Validate the currency fields”

“Validate the flight cost” on page 86

“Use a lookup database to validate the car type” on page 87

“Creating a dictionary of valid car types” on page 88

“Running a batch through the workflow” on page 89

“Examination of page and field status values” on page 90

“Creating recognition zones for the remaining fingerprints” on page 92

“Running a batch through the workflow” on page 92

“Page and field status codes in the TravelDocs application” on page 93

Validate the currency fields

To validate currency fields, you need to create a rule with actions that delete letters and spaces, and confirm that the field contains a currency value. Then, the rule must be assigned to all of the currency fields in the document hierarchy.

The currency field validation rule that you need to create uses the following actions.

Action	Description
DeleteAllAlpha	Deletes all of the letters from the field. Use this action to remove the USD suffix that are used by one of the airlines.
DeleteSelectedChars	Deletes the specified characters from the field. Use this action to remove any spaces.

Action	Description
IsFieldCurrency	Returns True if the field is a number and includes a two-digit decimal amount; returns False otherwise. The action ignores any leading currency symbol (for example, \$).

“Creating the Validate Currency Field rule”

“Adding the Validate Currency Field rule to the document hierarchy”

Creating the Validate Currency Field rule:

You create the Validate Currency Field rule in Datacap Studio.

To create the Validate Currency Field rule:

1. Open Datacap Studio and click the **Rulemanager** tab.
2. In the **Rulesets** pane, select the **Validate** ruleset and click **Lock/Unlock ruleset** (padlock) to lock the ruleset for editing.
3. Right-click the **Validate** ruleset and choose **Add Rule**.
4. Rename the new rule from **Rule1** to **Validate Currency Field**.
5. Rename the default function from **Function1** to **Validation: Currency**.
6. Click the **Actions library** tab.
7. Expand the **Validations** library and select the **DeleteAllAlpha** action.
8. Make sure the **Validation: Currency** function is selected in the **Rulesets** pane.
9. Click **Add to function** at the left side of the **Actions library** pane.
10. On the **Actions library** tab, select the **DeleteSelectedChars** action and click **Add to Function**.
11. On the **Actions library** tab, select the **IsFieldCurrency** action and click **Add to Function**.
12. Select the **DeleteSelectedChars** action and set the **strParam** parameter to ' ' (a single space) in the **Properties** pane.
13. In the **Rulesets** pane, click **Save**.

Adding the Validate Currency Field rule to the document hierarchy:

To add the Validate Currency Field rule to the document hierarchy, you must use Datacap Studio

To add the Validate Currency Field rule to the document hierarchy:

1. In the Document Hierarchy pane, click **Lock DCO**.
2. Expand the **Car_Rental > Rental_Agreement** page so that the fields are visible.
3. In the **Validate** ruleset, select the **Validate Currency Field** rule.
4. In the Document Hierarchy pane, select the **Total_Cost** field node. Then, click **Add to DCO** on the left side of the **Rulesets** pane. The Validate Currency Field rule is added to the Open element in the **Total Cost** field.
5. Expand the **Flight > Air_Ticket** page so the fields are visible.
6. Select the **Airfare** field and click **Add to DCO** to add the Validate Currency Rule to the Open element of the **Airfare** field.
7. Select the **Taxes** field and click **Add to DCO** to add the Validate Currency Rule to the Open element of the **Taxes** field. Because the **Total_Cost** field definition

is shared across pages, the Validate Currency Rule is already included in the **Total_Cost** field on the Air_Ticket and Room_Receipt pages.

8. In the Document Hierarchy pane, click **Save**.

Validate the flight cost

To validate the flight cost, you need to create a rule that calculates the cost field. Then, you need to assign the rule to the Air_Ticket page Close element in the document hierarchy.

The rule that you create to validate the total cost field uses this action.

Action	Description
CalculateFields	Returns True if the arithmetic expression is valid; returns False otherwise.

“Creating the Validate Flight Cost rule”

“Adding the Flight Cost rule to the document hierarchy”

Creating the Validate Flight Cost rule:

You create the Validate Flight Cost rule in Datacap Studio.

To create the Validate Flight Cost rule:

1. Make sure the **Validate** ruleset is locked for editing.
2. Right-click the **Validate** ruleset and choose **Add Rule**.
3. Rename the new rule from **Rule1** to **Validate Flight Cost**.
4. Rename the default function from **Function1** to **Validation: Flight Cost**.
5. On the **Actions library** tab, select the **CalculateFields** action (also in the Validations library).
6. Make sure the **Validation: Flight Cost** function is selected in the **Rulesets** pane.
7. Click **Add to function** at the side of the **Actions Library** pane.
8. Select the **CalculateFields** action and in the **Properties** pane set the **strParam** parameter to 'Airfare' + 'Taxes' = 'Total_Cost'. These parameters are the names of the fields as specified in the document hierarchy.
9. In the **Rulesets** pane, click **Save**.

Adding the Flight Cost rule to the document hierarchy:

To add the Flight Cost rule to the document hierarchy, you must use Datacap Studio.

To add the Flight Cost rule to the document hierarchy

1. In the **Document Hierarchy** pane, make sure that the document hierarchy is locked for editing.
2. Select the **Close** element at the end of the **Air_Ticket** page definition.
3. In the **Validate** ruleset, select the **Validate Flight Cost** rule.
4. Click **Add to DCO** to add the Validate Flight Cost rule to the Close element of the Air_Ticket page.
5. In the **Document Hierarchy** pane, click **Save**.

Use a lookup database to validate the car type

The rental agreement page includes the `car_type` field, which you can test to determine whether the field contains a permitted value.

The list of permitted car types includes these types.

- Compact
- Standard
- Full size
- SUV
- Other

For each car type field, you test the field value and set the field status to 1. There is a problem if the car type does not match one of the permitted types.

To use the lookup database, use these actions.

Library	Action	Description
Lookup	OpenConnection	Uses a data source name or connection string to open a connection to a database.
Lookup	ExecuteSQL	Runs a SQL statement. Returns True if the SQL statement runs successfully and any SELECT statement returns a value.
Lookup	CloseConnection	Closes an open database connection.

“Creating the lookup database table”

“Creating the Validate Car Type rule”

“Adding the Validate Car Type rule to the document hierarchy” on page 88

Creating the lookup database table:

The Datacap installation includes a Microsoft Access lookup database, `TravelDocsLook.mdb`, which is in the `C:\Datacap\TravelDocs` folder. You create a table that is named `Car Types` in the lookup database.

To create the lookup database table:

1. Open the file `C:\Datacap\TravelDocs\TravelDocsLook.mdb` in Microsoft Access.
2. Create a table that is named `Car Types`.
3. Create a field that is named `Car_Type` of type Text.
4. Enter the permitted values: Compact, Standard, Full size, SUV, and Other.
5. Save the new table.

Creating the Validate Car Type rule:

You create the Validate Car Type rule in Datacap Studio.

To create the Validate Car Type rule:

1. In the Rulesets pane, make sure that the Validate ruleset is locked for editing.
2. Right-click the Validate ruleset and choose **Add Rule**.

3. Rename the new rule from Rule1 to Validate Car Type.
4. Rename the default function from Function1 to Validation: Car Type.
5. On the **Actions library** tab, expand the **Lookup** library and select the **OpenConnection** action.
6. Make sure the **Validation: Car Type** function is selected in the Rulesets pane.
7. Click **Add to function** at the left side of the Actions Library pane.
8. Select the **ExecuteSQL** action and click **Add to Function**.
9. Select the **CloseConnection** action and click **Add to Function**.
10. Select the **OpenConnection** action and in the **Properties** pane set the **strParam** parameter to @APPVAR(*/*lookupdb:cs).
This parameter is a Datacap smart parameter that obtains the connection string for the application's lookup database from the application configuration file.
11. Select the **ExecuteSQL** action and in the **Properties** pane set the **sStringIn** parameter to "SELECT Car_Type FROM Car_Types WHERE Car_Type=' %s';",Car_Type.
Attention: You must use the syntax exactly as it is used here. You can copy and paste from here if necessary.
12. In the **Rulesets** pane, click **Save**. Then, click **Lock ruleset** and choose **Publish ruleset**.

Adding the Validate Car Type rule to the document hierarchy:

To add the Validate Car Type rule to the document hierarchy, you must use Datacap Studio.

To add the Validate Car Type rule to the document hierarchy:

1. In the Document Hierarchy pane, make sure that the document hierarchy is locked for editing.
2. Expand the **Car_Rental > Rental_Agreement** page so the fields are visible.
3. In the **Validate** ruleset, select the **Validate Car Type** rule.
4. In the Document Hierarchy pane, select the **Car_Type** field node. Then, click **Add to DCO** on the left side of the Rulesets pane. The Validate Car Type rule is added to the Open element of the **Car_Type** field.
5. In the Document Hierarchy pane, click **Save**.

Creating a dictionary of valid car types

If there is a problem with the **car_type** field in the verification panel, Datacap can present a list of valid car types from which the operator can select a valid type.

The Datacap Desktop and Datacap Web Client verification interfaces enable the population of a drop-down list directly from the database by using an SQL statement that is embedded in the *SELECT* variable of the field. You need to create the variable in Datacap Studio by first unlocking the document hierarchy, right-clicking the **Car_Type** field, and choosing **Manage Variables**. You can then add the *SELECT* variable and set it to the following value.

```
<SQL flist='Car_Type' dsn="*/lookupdb:cs">SELECT Car_Type FROM Car_Types</SQL>
```

An SQL query (SELECT <column> FROM <table>) gets the list of valid car types from the application's lookup database (dsn="*/lookupdb:cs"). It then creates a drop-down list in the specified field (flist='<field>') that contains the returned values.

Another variable, *Lookup*, is functionally similar, except that it displays the list of available choices in a window instead of a drop-down list.

To create a selection list that works for all verification interfaces, you can create a dictionary that contains the same valid car types (Compact, Standard, Full size, SUV, and Other). You can then attach the library to the **Car_Type** field.

“Creating the dictionary”

“Attaching the dictionary to the Car_Type field”

Creating the dictionary:

You create the dictionary in Datacap Studio.

To create the dictionary:

1. Confirm that the document hierarchy is locked for editing.
2. Click **Dictionaries** at the top of the Document Hierarchy pane.
3. Click **Edit dictionary** and choose **Add dictionary**.
4. Change the dictionary name from <new_dictionary> to **Car_Types**.
5. Right-click the new dictionary and choose **Add word**.
6. Change the name from <new word> to **Compact** and the value from **value** to **Compact**.
7. Repeat to add **Standard**, **Full size**, **SUV**, and **Other** to the dictionary.
8. Click **Save**.

Attaching the dictionary to the Car_Type field:

After you create the dictionary, you need to attach it to the **Car_Type** field in Datacap Studio.

To attach the dictionary to the **Car_Type** field:

1. Make sure that the document hierarchy is locked for editing.
2. Expand the **Car_Rental > Rental_Agreement** page so the fields are visible.
3. Right-click the **Car_Type** field and select **Manage variables**.
4. Click **New**, type **DICT**, and press **Enter**.

Important: Variables are case-sensitive. Ensure that you capitalize **DICT**.

5. Enter the value **Car_Types**. Then, click **Done**.
6. In the Document Hierarchy pane, click **Save** and then click **Unlock DCO**.

Running a batch through the workflow

After you create the dictionary and attach it to the **Car_Type** field, you can run a batch through the workflow to see how the application is progressing.

1. Click the Datacap Studio **Test** tab.
2. In the **Workflow** pane, select the **VScan** task profile under **Main Job**.
3. Click **New** to start a new batch.
4. Click **Process rules for target object** on the main Test tab toolbar. When you are prompted to release the batch, click **Advance**. The batch is moved to the next step in the workflow, which is **PageID**.
5. Click **Process rules for target object** on the main Test tab toolbar. When you are prompted to release the batch, click **Advance**. The batch is moved to the next step in the workflow, which is **Profiler**.

- Click **Process rules for target object** on the main Test tab toolbar and wait while the task profile launches. When you are prompted to release the batch, click **Advance**. The batch is moved to the next step in the workflow, which is Verify.
- Because you are not ready yet to run the Verify task profile, right-click the batch in the **Workflow** pane and choose **Cancel**.

Examination of page and field status values

The validation rules affect the status that Datacap assigns to the status variable for each page and field. To see the page status, open Profiler.xml in the application's most recent batch folder.

The Profiler.xml file includes the status of each page in the batch.

```
<D id="20100323.007.01">
  <V n="TYPE">Car_Rental</V>
  <V n="STATUS">0</V>
  <P id="TM000001">
    <V n="TYPE">Rental_Agreement</V>
    <V n="STATUS">1</V>
    etc.
  </P>
  <P id="TM000002">
    <V n="TYPE">Optional_Insurance</V>
    <V n="STATUS">0</V>
    etc.
  </P>
</D>
<D id="20100323.007.02">
  <V n="TYPE">Car_Rental</V>
  <V n="STATUS">1</V>
  <P id="TM000003">
    <V n="TYPE">Rental_Agreement</V>
    <V n="STATUS">1</V>
    etc.
  </P>
</D>
etc.
<D id="20100323.007.04">
  <V n="TYPE">Flight</V>
  <V n="STATUS">1</V>
  <P id="TM000006">
    <V n="TYPE">Air_Ticket</V>
    <V n="STATUS">1</V>
    etc.
  </P>
</D>
etc.
```

A status of 0 indicates that there are no problems, and a status of 1 indicates that a problem exists. The three problem pages that are shown in the preceding have Status = 1 for different reasons. To see the nature of the problems, review the individual page files: tm000001.xml, tm000003.xml, and tm000006.xml.

TM000001

The following example shows a portion of the tm000001.xml page file:

```
<?xml-stylesheet type="text/xsl" href="..\..\dco.xsl"?>
<P id="TM000001">
  <F id="Pickup_Date">
    <V n="TYPE">Pickup_Date</V>
    <V n="Position">189,403,567,465</V>
    <V n="STATUS">0</V>
```

```

<C cn="7" cr="200,416,220,440">84</C>
<C cn="4" cr="218,415,226,430">114</C>
<C cn="10" cr="218,423,230,438">117</C>
etc.
</F>
<F id="Pickup_Location">
  <V n="TYPE">Pickup_Location</V>
  <V n="Position">195,537,558,592</V>
  <V n="STATUS">0</V>
  <C cn="10" cr="203,549,216,570">66</C>
  <C cn="10" cr="219,555,234,570">111</C>
  etc.
</F>
<F id="Return_Date">
  <V n="TYPE">Return_Date</V>
  <V n="Position">580,403,942,465</V>
  <V n="STATUS">0</V>
  <C cn="6" cr="593,416,604,438">70</C>
  <C cn="6" cr="606,423,615,438">114</C>
  <C cn="7" cr="619,416,621,438">105</C>
  <C cn="10" cr="625,434,630,441">44</C>
  <C cn="10" cr="690,416,691,438">32</C>
  etc.
</F>
etc.

```

All of the fields in TM000001 have Status = 0 (OK), but the pickup date and return date fields have low confidence characters. By default, any character with a confidence level below 8 is considered low confidence and is displayed to an operator for verification.

TM000003

The following example shows a portion of the tm000003.xml page file:

```

<?xml-stYLESHEET type="text/xml" href="..\..\dco.xml"?>
<P id="TM000003">
  <F id="Pickup_Date">
    <V n="TYPE">Pickup_Date</V>
    <V n="Position">0,0,0,0</V>
    <V n="STATUS">0</V>
  <F id="Pickup_Location">
    <V n="TYPE">Pickup_Location</V>
    <V n="Position">0,0,0,0</V>
    <V n="STATUS">0</V>
  </F>

```

Because you only defined recognition zones for the first fingerprint of each page type, TM000003 has no data that is associated with any of the fields. Page TM000003 is the rental agreement page for Car Rental #2 and has no recognition zones. Fix this problem and then run the batch again.

TM000006

The following example shows a portion of the tm000006.xml page file:

```

<?xml-stYLESHEET type="text/xml" href="..\..\dco.xml"?>
<P id="TM000006">
  etc.
  <F id="Airfare">
    <V n="TYPE">Airfare</V>
    <V n="Position">359,805,527,854</V>
    <V n="STATUS">1</V>
    <V n="MESSAGE">Failed By Calculate Action On Field 'TM000006'</V>
    etc.
  </F>

```

```

</F>
<F id="Taxes">
  <V n="TYPE">Taxes</V>
  <V n="Position">359,861,525,905</V>
  <V n="STATUS">1</V>
  <V n="MESSAGE">Failed By Calculate Action On Field &apos;TM000006&apos;.</V>
  etc.
</F>
<F id="Total_Cost">
  <V n="TYPE">Total_Cost</V>
  <V n="Position">361,912,527,961</V>
  <V n="STATUS">1</V>
  <V n="MESSAGE">Failed By Calculate Action On Field &apos;TM000006&apos;.</V> etc.
</F>
</P>

```

In TM000006, the Calculate('Airfare' + 'Taxes' = 'Total_Cost') validation action failed. Since Datacap cannot know which of the field values is incorrect, it flags all fields.

Creating recognition zones for the remaining fingerprints

After you review the page status and field status and confirm that the application is working properly, create the recognition zones for the remaining fingerprints.

Refer to “TravelDocs: Specification of recognition zones” on page 70 for instructions on how to create the recognition zones for the different page types.

You need to create recognition zones for each of the following fingerprints.

- Rental_Agreement (Car Rental #2)
- Optional_Insurance (Car Rental #2)
- Rental_Agreement (Car Rental #3)
- Optional_Insurance (Car Rental #3)
- Room_Receipt (Hotel #2)
- Room_Receipt (Hotel #3)
- Air_Ticket (Airline #2)
- Air_Ticket (Airline #3)

Important: As you draw the zones, click **Save** in the Document Hierarchy pane often.

Drawing the check box recognition zones

To get accurate recognition on the check box options, it is important that all the check box recognition zones on all fingerprints be as close to the same size as possible. You might find it difficult to make the zones the same size when you draw zones on the **Zones** tab. To establish approximate zone boundaries, draw the bounding boxes on the **Image View** tab, and then edit the coordinates in the *Pos* variables manually in the Properties pane. For more information, see the section “Implications of using RecogOMRThreshold” in the topic “Using the pixel threshold evaluation method” on page 68 for more information.

Running a batch through the workflow

After you define all of the required recognition zones, you can run a batch through the workflow.

1. In Datacap Studio, click the **Test** tab.
2. In the **Workflow** pane, select the **VScan** task profile under **Main Job**.
3. Click **New** to start a new batch.

4. Click **Process rules for target object** on the main **Test** tab toolbar. When you are prompted to release the batch, click **Advance**. The batch is moved to the next step in the workflow, which is PageID.
5. Click the **Process rules for target object** button on the main Test tab toolbar. When you are prompted to release the batch, click **Advance**. The batch is moved to the next step in the workflow, which is Profiler.
6. Click **Process rules for target object** on the main **Test** tab toolbar and wait while the task profile runs. When you are prompted to release the batch, click **Advance**. The batch is moved to the next step in the workflow, which is Verify.
7. Review each of the pages in the Runtime batch hierarchy pane to ensure that recognition was successful. Then, review the batch and page XML files in the runtime batch folder.

Page and field status codes in the TravelDocs application

After you run a batch through the workflow, review the status codes for each of the fields that you validated and for the pages in the runtime batch.

The following table describes how to interpret the status codes.

Field	STATUS = 0	STATUS = 1
Car_Rental		
Rental_Agreement	Page OK	Page contains unrecognized or low confidence characters, or a field with Status = 1
Car_Type	Field OK	Field value is not one of the valid values
Total_Cost	Field OK	Field value is not currency
Optional_Insurance	Page OK	Page contains unrecognized or low confidence characters, or a field with Status = 1
Total_Cost	Field OK	Field value is not currency
Hotel		
Room_Receipt	Page OK	Page contains unrecognized or low confidence characters, or a field with Status = 1
Total_Cost	Field OK	Field value is not currency
Flight		
Air_Ticket	Page OK	Page contains unrecognized or low confidence characters, or a field with Status = 1
Airfare	Field and all calculated fields OK	Field value is invalid or calculated fields do not add correctly
Taxes	Field and all calculated fields OK	Field value is invalid or calculated fields do not add correctly
Total_Cost	Field and all calculated fields OK	Field value is invalid or calculated fields do not add correctly

Data verification

During verification, Datacap displays pages to an operator for manual checking and possible correction.

There are three primary reasons to display pages to an operator:

- The batch failed document integrity checking.
- A page contains one or more characters or OMR fields that were marked low confidence by the recognition engine.
- A page does not pass a validation rule because there is a problem with the integrity of the data.

“Field data verification”

“Skipping a verification task” on page 97

“TravelDocs: Batch verification” on page 98

Field data verification

During verification, an operator confirms that data is accurate or, if necessary, corrects problem fields.

Problem fields can include various issues:

- Character fields with one or more low confidence characters
- OMR fields with low confidence values
- Fields with validation errors

“Options for data verification”

“Confidence levels and the page status” on page 95

“Overriding validation failures” on page 96

Options for data verification

Datacap Desktop and Datacap Web Client are two user interface options for verification.

Datacap applications can support any or all verification options simultaneously. All verification clients access the same job queue. The clients also provide similar functions, such as identifying and correcting problems, and submitting the batch to the next stage in the workflow.

Datacap Desktop

Datacap Desktop panels are .NET forms. The default field-at-a-time interface is generated automatically from the application's document hierarchy.

You can also create custom panels by using the Datacap Desktop panel builder, which is distributed as a Microsoft Visual Studio project. Custom panels typically display all of a page's fields simultaneously.

Datacap Web Client

Datacap Web Client generates verification panels automatically from the document hierarchy. However, it is also possible to create static layouts and add other custom functions. The web page for the Verifine verification client includes various components:

- An image pane that displays the current page

- A data entry panel that displays image snippets and controls for checking and correcting the data fields
- A batch tree view for restructuring the batch

Datacap Web Client is functionally similar to Datacap Desktop in that the operator must review each problem page, make any necessary corrections, and submit the batch when complete.

Related information:

 [Datacap Desktop panel customization](#)

Confidence levels and the page status

You can configure your application so that the confidence levels of the fields or characters within a page determine the status for that page.

“Confidence levels”

“Page status”

“Overriding the default confidence value on specific fields” on page 96

Confidence levels:

During recognition, Datacap assigns a confidence level to each character and OMR field. Confidence levels range from 1 (lowest confidence) to 10 (highest confidence).

You can see the confidence level for each character or OMR field in the **cn** attribute of the object in the page data file.

```
<F id="Pickup_Date">
  <V n="TYPE">Pickup_Date</V>
  <V n="Position">189,403,567,465</V>
  <V n="STATUS">0</V>
  <C cn="7" cr="205,414,219,439">83</C>      <-- ASCII 'T' [low confidence]
  <C cn="4" cr="205,414,219,439">83</C>      <-- ASCII 'r' [low confidence]
  <C cn="10" cr="224,423,236,438">117</C>     <-- ASCII 'u' [high confidence]
  <C cn="10" cr="241,423,255,438">101</C>     <-- ASCII 'e' [high confidence]
  <C cn="10" cr="256,423,266,438">115</C>     <-- ASCII 's' [high confidence]
  <C cn="10" cr="270,434,275,441">44</C>      <-- ASCII ',' [high confidence]
  <C cn="10" cr="334,416,335,438">32</C>      <-- ASCII ' ' [high confidence]
  <C cn="10" cr="288,416,304,438">68</C>      <-- ASCII 'D' [high confidence]
  <C cn="10" cr="308,423,320,438">101</C>     <-- ASCII 'e' [high confidence]
  etc.
</F>
```

The confidence level determines how Datacap displays the character and the parent field within the verification panel:

- The two verification clients display fields that contain low confidence characters in yellow, where low confidence in this case is anything less than 10.
- Within the field, the Datacap Web Client displays the low confidence characters in red, while Datacap Desktop highlights the problem characters in yellow within the image snippet. Low confidence in this case is anything less than 10 or the field's ReqConf value. (See “Overriding the default confidence value on specific fields” on page 96.)

Page status:

The confidence level does not directly affect the page status. For example, a page might have every character with a confidence level of 1 (lowest confidence) but the

page status is 0 (good). To set the page status that is based on the confidence level of the characters on the page, use the ChkConfidence action.

Library	Action	Description
DCO	ChkConfidence	Checks the confidence level of all characters. If the confidence level on any character is less than the value specified in parameter 1, the action assigns the status value in parameter 2 to the page.

The Datacap Studio application wizard generates a default Routing ruleset the uses this action to set the page status.

Routing Rule 1 is assigned to each page in the document hierarchy and works as follows:

- ChkDCOStatus checks the page status and returns True if the page status is 1. A status of 1 typically signifies that there is an error on the page. If the action returns True, function 2 does not run.
- ChkDCOStatus returns False if the page status is 0 (or any other value other than 1). A status of 0 typically signifies that the page contains no errors. If the action returns False, function 2 runs.
- ChkConfidence examines the characters on the current page and sets the page status to 1 if any character has a confidence level of less than 8 (or the ReqConf value of the field).

Following execution of Routing Rule 1, any page that contains a validation error or a character with a confidence level less than 8 has Status = 1. You can configure Datacap to display only pages with Status = 1 as described in “Show validation failures to an operator” on page 82.

Overriding the default confidence value on specific fields:

To determine which characters to display in red, the Datacap Web Client uses a confidence level of 10, unless the field has its own *ReqConf* value.

Similarly, the ChkConfidence action uses the confidence value that is specified in parameter 1, unless the field has its own *ReqConf* value. For example, if you specify 8 as the parameter value but a field has ReqConf=6, ChkConfidence uses the value 6 for that field.

To set the confidence level on a specific field:

1. In the Document Hierarchy pane, lock the document hierarchy for editing.
2. Right-click the field and choose **Manage variables**.
3. If the field has a *ReqConf* variable, assign the appropriate value; otherwise click **New**, type ReqConf, press **Enter**, and then assign the value.
4. Click **Done** and then click **Save** in the Document Hierarchy pane.

Overriding validation failures

By default, all validations can be overridden, which means that the operator can submit a batch that contains validation errors by selecting to override them

Depending on the business requirements, overriding validation failures is appropriate. For example, if the validation error stems from a calculation error on the original page, the operator does not modify the field values. Instead, the operator must override the error and submit the batch. The application sends the batch to an exception handling task. When the operator overrides a validation error, Datacap sets the page status to 73 and the document status to 142.

In other situations, you can prevent the operator from overriding validation errors by using the **SetIsOverrideable** action.

Library	Action	Description
Validations	SetIsOverrideable	If set to False, specifies that if validation on the current object fails, the operator cannot override the error. If set to True, the operator can override the error.

For example, to prevent the operator from overriding an error in the Validate Car Type rule, you can insert **SetIsOverrideable("False")**.

```
Validate Car Type
  Validation: Car Type
    SetIsOverrideable("False")
    OpenConnection("@APPVAR(*.lookupdb:cs)")
    ExecuteSQL("SELECT Car_Type FROM Car_Types WHERE Car_Type='%s';",Car_Type")
    CloseConnection()
```

In this case, the operator must select a valid car type from a drop-down menu that Datacap Desktop displays next to the field that failed validation.

If the operator attempts to submit a page that failed this validation, an error message is shown.

Skipping a verification task

You can configure your application to skip the verification task and proceed directly to exporting the data when every page passes validation.

You need to set the mode to **Router** for the task that precedes the task that you want to skip. In this case, the Profiler task precedes the verification task. Also, you must create a ruleset that includes a condition to cause the skip. Finally, you must add the ruleset at the end of the Profiler task in Datacap Studio.

1. Start Datacap Web Client and log in to the application that contains that task to be skipped.
2. Click the **Administrator**, expand the job that contains the verification task, and select the task before the verification task. In most applications, the Profiler task (the task that applies validation rules) precedes the verification.
3. In the Selected task details section, select **Router** from the menu for the **Mode** field.
4. Under Parameters, enter a value, such as Skip for the **Returned Conditions** key.
5. Select the condition (**Skip**), and in the Selected condition details window, select **Jump** from the menu for the **Spawn type** field, and enter 1 for the

Steps field. A value of 1 configures the workflow so that one task is skipped when the condition is encountered. In this case, the condition is every page passed validation.

6. Start Datacap Studio and log in to the same application that you configured in Datacap Web Client in steps 1 - 5.
7. Click the **Rulesets** tab, lock the rulesets for editing, and add a ruleset with a name such as Skip Verify.
8. Rename Rule 1 to a name such as Check Confidence.
9. Click **Function1**, select an action from the corresponding action library, and click **Add to Function** to add the action to **Function1**. Refer to the table to determine the action library from which to select an action and the parameter values to enter.

Important: You must add the actions in the order shown.

Action Library	Action	Values to enter	Description
DCO	SetPageStatus	0	0 indicates that the page passed validation.
DCO	ChkConfidence	8,1	The first parameter sets a high-confidence level, and 1 is the code to return for low confidence fields on the page.
RRunner	Task_NumberOfSplits	1	1 indicates that there is one subbatch.
RRunner	Task_RaiseCondition	0,0	The first value in the pair indicates the subbatch index, where 0 is the first subbatch. The second value raises the first Child Job Condition (in this case, Skip Verify) for the subbatch entry.

10. Save and publish the ruleset.
11. Click the **Task Profiles** tab, select the **Profiler** task profile, and lock it for editing,
12. In the Rulemanager tab, select **Skip Verify** (or the ruleset that you created in step 7), and click **Add ruleset to profile** to add the ruleset to the Profiler task profile.
13. Click **Save** in the Task Profiles tab, and unlock the Profiler task profile.

TravelDocs: Batch verification

You can use Datacap Desktop or the Datacap Web Client for verification.

“Setting the **Car Type** field to prevent overriding” on page 99

“Batch verification with Datacap Desktop” on page 99

“Verifying batches with Datacap Web Client” on page 101

Setting the Car Type field to prevent overriding

Before you run a batch through the workflow, set the **SetIsOverrideable** property on the **Car Type** field to **False** to prevent the operator from overriding an invalid car type.

1. On the Datacap Studio **Rulemanager** tab, Rulesets pane, select the **Validate** ruleset and click **Lock/Unlock ruleset**. The ruleset is locked for editing.
2. Expand the **Validate Car Type** rule completely.
3. Select the **Validation: Car Type** function.
4. Expand the **Validations** library and select **SetIsOverrideable**.
5. Click **Add to function** at the left side of the Actions Library pane. The action is added to the Validate ruleset.
6. In the **Properties** pane, set **StrParam** to **False**.
7. Use the **Up Arrow** button to move the new action to the beginning of the function.
8. In the Rulesets pane, click **Save**, click **Lock/Unlock ruleset**, and choose **Publish Ruleset**.

Batch verification with Datacap Desktop

Datacap Desktop is a thick client that you can customize and use for verification.

“Creating dictionaries for check box options”

“Preparing a batch for verification” on page 100

“Opening the batch in Datacap Desktop” on page 100

“Reviewing the batch in Datacap Desktop” on page 100

“Submitting the batch” on page 101

Creating dictionaries for check box options:

You can use Datacap Desktop for verification, but you must create dictionaries for the check box options, and attach the dictionaries to the check box fields.

1. Create the dictionaries for the check box options.
 - a. On the Datacap Studio **Rulemanager** tab, Document Hierarchy pane, click **Lock DCO** to lock the document hierarchy for editing.
 - b. Click **Dictionaries** in the Document Hierarchy pane.
 - c. Click **Edit dictionary** and select **Add dictionary**.
 - d. Change the dictionary name from <new_dictionary> to **Options**.
 - e. Right-click the new dictionary and choose **Add word**.
 - f. Change the name from <new word> to **Navigation System** and the value from value to **Navigation System**.
 - g. Repeat to add **Child Seat** and **Fuel Service** to the dictionary.
 - h. Click **Edit dictionary** and select **Add dictionary**.
 - i. Create another dictionary that is called **Checkbox** and add one word with the name **Selected** and a value **Selected**.
 - j. Click **Save** in the dialog, and then click **Save** in the Document Hierarchy pane.
2. Attach the dictionaries to the check box fields.
 - a. Confirm that the document hierarchy is still locked for editing.
 - b. Expand the **Car_Rental > Rental_Agreement** page so that the fields are visible.
 - c. Right-click the **Options** field and choose **Manage variables**.

- d. Click **New**, enter **DICT**, and press the Enter key.
- e. Enter the value **Options**, and click **Done**.
- f. Expand the **Car_Rental >Optional_Insurance** page so that the fields are visible.
- g. Right-click the **CDW** field and choose **Manage variables**.
- h. Click **New**, enter **DICT**, and press the Enter key.
- i. Enter the value **Checkbox**, and click **Done**.
- j. Repeat for the **PAI**, **PEP**, and **ELP** fields.
- k. In the Document Hierarchy pane, click **Save**, and then click **Unlock DCO**.

Preparing a batch for verification:

For demonstration purposes, run a batch in the Datacap Web Client through the Profiler task so that the batch is pending verification.

To prepare a batch for verification:

1. Start the Datacap Web Client (if it is not already running).
 - a. Select the **TravelDocs** application.
 - b. Log in using User ID: **admin**, Password: **admin**, and Station: **1**.
2. Select the **Administrator** tab and then click **Operations**.
3. Click **VScan** and wait for the task to complete. Then, click **Stop**.
4. On the **Operations** page, click **PageID** and wait for the task to complete. Then, click **Stop**.
5. On the **Operations** page, click **Profiler** and wait for the task to complete. Then, click **Stop**. The batch is now pending verification.

Opening the batch in Datacap Desktop:

Use Datacap Desktop to open batches that are pending verification.

To open the batch in Datacap Desktop:

1. In the **Start** menu click **IBM Datacap Clients Datacap Desktop**.
2. Enter these values in the login panel.
 - **Application:** **TravelDocs**
 - **User ID:** **admin**
 - **Password:** **admin**
 - **Station:** **1**
3. Click **Login**.
4. In the **Shortcut** field, select **Verify**, and click **Start**.
 - If there are no older batches on hold, click **OK** to run the next pending batch.
 - If there are older batches on hold, click **Run Pending!** to run the pending batch.

Reviewing the batch in Datacap Desktop:

When you start Datacap Desktop and open a batch that is pending verification, Datacap Desktop defaults to the field-at-a-time interface and displays the first problem field.

Within Datacap Desktop, you can go directly to the next problem by clicking **Next Problem**.

You can display any page in the batch by selecting it in the Batch View pane.

You cannot modify the default field-at-a-time interface. Instead, if you want to modify the Datacap Desktop interface you must create a custom panel for each page.

Related information:

 Datacap Desktop panel customization

Submitting the batch:

After you review each problem page in a batch and make any necessary corrections, you can submit the batch.

To submit the batch:

1. Review each problem page and complete these steps.
 - a. Correct any low confidence fields.
 - b. Correct the Car Type validation failure on the third car rental page by selecting **Other**.
 - c. Ignore the other fields with validation failures because the errors are on the original images.
2. Click **Submit** advance to the next problem page. When you are prompted to the override validation failures, click **OK**.
3. When you reach the end, click **OK** to finish the batch. The batch is now marked as pending for export.
4. Close Datacap Desktop.

Verifying batches with Datacap Web Client

Before you can use Datacap Web Client for verification, you must configure the Datacap Web Client server.

This task provides an outline of how to set up the Datacap Web Client server by using Microsoft Information Systems (IIS) 7.5 and Microsoft Windows 7 Professional.

Follow this procedure to use the Datacap Web Client for verification.

1. Set up the Datacap Web Client server:
 - a. In the **Start** menu, select **IBM Datacap Web > Datacap Web Server Configuration Tool**. The message box notifies you that the IIS components are installed.

```
Internet Information Services (IIS) 7.5:Found
IIS Component - IIS Management Console:Found
IIS Component - ASP.NET:Found
IIS Component - ASP:Found
IIS Component - Static Content:Found
```
 - b. If the message indicates that you are missing components, open the Programs and Features window from Windows Control Panel, click **Turn Windows features on or off**, and install the missing components.
 - The Management Console is under **Internet Information Services > Web Management Tools**.

- The ASP and ASP.NET components are located under **Internet Information Services > World Wide Web Services > Application Development Features**.
 - The Static Content component is located under **Internet Information Services > World Wide Web Services > Common HTTP Features**.
- c. After you install the missing components, run the Datacap Web Client Server Configuration tool again. Then, click **OK** to continue.
 - d. In the Datacap Web Client Server Configuration window, click **Configure** to set up the Datacap Web Client application in IIS.
 - e. Start the Internet Information Services (IIS) Manager or type inetmgr).
 - f. Expand **Sites > Default Web Site** and confirm that the TaskRun folder shortcut and the **tmweb.net** application are visible.
2. Set up the Datacap Web Client:
 - a. In the **Start** menu click **IBM Datacap Web Datacap Web Server Configuration Tools**
 - b. Click **Configure** to set up the security options in Internet Explorer.
 3. Create dictionaries for the check box options:
 - a. If you did not complete the section on using Datacap Desktop for verification, complete the procedures in “Creating dictionaries for check box options” on page 99 to create and attach the dictionaries to the check box fields.
 4. Prepare a batch for verification:
 - a. Start the Datacap Web Client, if it is not already running.
 - Select the **TravelDocs** application and click **OK**.
 - Log in using **User ID:** admin, **Password:** admin, and **Station:** 1.
 - b. Select the **Administrator** tab and then click **Operations**.
 - c. Click **VScan** and wait for the task to complete. Then, click **Stop**.
 - d. On the **Operations** page, click **PageID** and wait for the task to complete. Then, click **Stop**.
 - e. On the **Operations** page, click **Profiler** and wait for the task to complete. Then, click **Stop**. The batch is now pending verification.
 5. On the **Operations** page, click **Verify**.
 6. Review and submit the batch: Datacap Web Client displays the page with the fields and image snippets automatically.
 - a. Review each problem page and complete these steps:
 - Correct any low confidence fields.
 - Correct the Car Type validation failure on the third car rental page by selecting **Other**.
 - Do not correct the other fields with validation failures because the errors are on the original images.
 - b. Click **Submit** at the top of the verification panel to advance to the next problem page. When you are prompted to the override validation failures, click **OK**

Tip: If you are unable to get past the page with the validation error after you click **OK**, click **Hold** to put the batch on hold. Then, click the **Administrator** tab, select the **Verify** task in the **Workflow** page, and click **Setup** in the Selected task details pane. In the Webpage Dialog **Navigation** panel, enter a value of 0,2 for the **Done Page Status** field. After you complete the revision and save the setup file, reopen the batch

from the Datacap Web Client **Monitor** tab by clicking the **QID** field. For information about the Done Page Status and other settings, see “Configuring the VeriFine client” on page 230.

- c. When you reach the end of the batch, click **OK** to finish the batch. The batch is now marked as pending for export.

For more information about configuring the web verification client, see “Verification by using the VeriFine web client” on page 230.

Related information:

 Datacap Web Client installation and configuration

Data export

Datacap can export data to a text file, an XML file, a database, a Document Management system, or a custom business process. The default output format is a text file, but you can use some actions to export data to a database and an XML file.

“Exporting data”

“TravelDocs: Exporting data to a database” on page 137

“TravelDocs: Exporting data to an XML file” on page 140

Exporting data

You can export data to different file types or to specific repositories. To export to a Content Manager OnDemand repository, you must consider other configuration options. Also, you can use Datacap connector actions to automate capture and indexing processes.

“Export to a text file”

“Configure text export for IBM Content Manager OnDemand” on page 104

“Export to a database” on page 104

“Export to an XML file” on page 105

“Datacap Connector actions” on page 105

Export to a text file

The Application Wizard generates a framework that exports all captured data to a text file in the application's export folder.

The default Export ruleset includes two rules:

- Set Export Params: This rule is attached to the batch's Open element. It sets the export path and file name, and writes the header information to the file.
- Export Page Fields: This rule is attached to each page and writes all fields values from the current page to the export file.

The resulting file looks like the following example, where each line represents one page and the fields are separated by commas.

```
*****
Export for batch #20100334.019,12/01/2010,08:47:58 <--- Header information
,Tues, Dec 7, 2010,Boston (BOS),Fri, Dec 10, 2010,Boston (BOS),Compact,001,$345.70
,0,0,0,1
,Mon, Dec 6, 2010,San Francisco (SFO),Fri, Dec 10, 2010,San Francisco (SFO),SUV,010,$489.31
,Boston (BOS),Pittsburgh (PIT),17NOV10,Pittsburgh (PIT),Boston (BOS),21NOV10,313.17,64.56,477.73
,Newark, NJ (EWR),Charlotte, NC (CLT),MON NOV 15, 2010,Charlotte, NC (CLT),Newark, NJ (EWR),WED NOV 17, 2010,$524.76,$53.23,$577.99
,Dec 21, 2010,Dec 24, 2010,$293.03
,Nov 30, 2010,Dec 2, 2010,$243.07
```

The **Export** actions library includes actions that are typically used for exporting captured data to a text file. A few of the key export actions are outlined in the following table.

Library	Action	Description
Export	SetExportPath	Specifies the path to the export file's location. Typically you reference the export path in the application's configuration file by using the @APPPATH(export) smart parameter.
Export	SetFileName	Specifies the name for the export file (do not include the file extension).
Export	SetExtensionName	Specifies the extension for the export file.
Export	ExportAllFields	Writes all field values on the current page to the export file.
Export	ExportFieldValue	Writes the specified field's value to the export file, for example, <code>ExportFieldValue(Return_Date)</code> .
Export	CloseExportFile	Closes the export file.

Configure text export for IBM Content Manager OnDemand

You can configure Datacap to export index data and files into Content Manager OnDemand.

The Content Manager OnDemand tool contains the ARSLOAD component. ARSLOAD can enter a flat index file that contains index data and locations of files that can be uploaded with the index data. You can use the generic Export library action to create output index files in a format that is required by ARSLOAD.

For detailed information on ARSLOAD file formats and how to configure your Datacap system to export data into Content Manager OnDemand, see <http://www-01.ibm.com/support/docview.wss?uid=swg21502807>.

Export to a database

Datacap can export data to any DB2®, Microsoft Access, Microsoft SQL Server, or Oracle database by using the actions in the ExportDB library

Commonly used export actions are outlined in the following table.

Library	Action	Description
ExportDB	ExportOpenConnection	Opens a connection to the specified export database.
ExportDB	SetTableName	Specifies the name of the table to which data is to be exported.

Library	Action	Description
ExportDB	ExportFieldToColumn	Gets the value of the specified field on the current page and adds it to specified column in the internal data record. You build the record in memory before you commit it to the database by using AddRecord.
ExportDB	AddRecord	Inserts the assembled data record into the export table that is specified by the previous SetTableName action.
ExportDB	ExportCloseConnection	Closes an open export database connection.

For a complete example, see “Creating the ExportDB ruleset” on page 138.

Export to an XML file

Datacap can export data to an XML file by using the actions in the ExportXML action library.

The **ExportXML** library includes actions that you can use to write data to an XML file. Some of the export actions are described in the following table.

Library	Action	Description
ExportXML	xml_SetExportPath	Specifies the path to the XML file storage location.
ExportXML	xml_SetFileName	Specifies the name for the XML file (do not include the .xml extension).
ExportXML	xml_NewNode	Creates a child node under the specified parent node, creating the parent node if necessary.
ExportXML	xml_SetNodeValue	Sets the value of the specified node.
ExportXML	xml_SaveFile	Commits all unsaved nodes and saves the XML file to disk.

For a complete example, see “Creating the ExportXML ruleset” on page 140.

Datacap Connector actions

Datacap provides actions that you can use to integrate Datacap applications with supported content repositories. You can automate data capture, index documents, and process forms as a front end that stores document images and associated index values in the repository.

Datacap Connector actions are associated with objects in the Document Hierarchy at the batch, document, page, or field level through rulesets. For example, to

export objects from a Datacap application to IBM® Content Manager. You configure rules on the IBMCM Export ruleset to upload Datacap scanned images to IBM Content Manager.

You can use the IMail or EWSMail input actions to scan incoming email messages for attachments that can then be imported into document batches. You can use the Email output actions to send email messages. The Fax actions can import the content of incoming faxes from a specified fax source into document batches. You can process email and fax-based batches by using the Datacap standard Recognition or Verify tasks.

You configure these actions in a Datacap 8.0 or later Windows environment.

“Verifying the installation”

“Content repository authentication”

“Integrating Connector actions into applications” on page 107

“Connector actions configuration” on page 108

“IBM Content Manager Connector actions” on page 108

“FileNet P8 Connector actions” on page 113

“SharePoint Connector actions” on page 117

“FileNet Image Services Connector Connecting actions” on page 123

“Email Connector actions” on page 127

“Fax Connector actions” on page 133

“Connector actions log files” on page 136

“Viewing action details” on page 137

Verifying the installation:

Before you start to configure these actions, verify the Datacap Connector actions were installed as part of a complete Datacap product installation.

To verify the installation:

1. In Datacap Studio, click the **Rulemanager** tab.
2. Select the **Actions library** tab.
3. Scroll to **Global Actions** and verify that the actions file for your repository is listed.
 - For IBM FileNet® Content Engine, look for FileNetP8
 - For IBM Content Manager, look for Datacap.Libraries.IBMCM
 - For SharePoint, look for SPExport
 - For IBM FileNet Image Services, look for FileNetIDM
 - For eMail and eDoc Connector, look for IMail, EWSMail, and EMail
 - For Fax Connector, look for OpenTextFaxServer

The name of the action file might not match the name of the *connector_name.rxx* file. If the action file is not listed, the actions must be installed before you can continue.

Content repository authentication:

You must have write access to a folder on the repository and privileges to create and view documents in that folder. Then, you can use the Datacap Connector Actions to upload documents into the repository.

Access control is handled differently by each of the repositories and their connectors:

- For Datacap Connector for IBM Content Manager, access is controlled through the IBM Content Manager authentication.
- For Datacap Connector for FileNet Content Manager, access is controlled through the IBM FileNet Content Manager authentication.
- For Datacap Connector for Microsoft SharePoint, authentication is done by using the Login action with user credentials that are managed by SharePoint.
- For Datacap Connector for FileNet Image Services, authentication is done by the library into which you are importing the documents.
- For Datacap Connector for eMail and Electronic Documents, authentication is done by the Microsoft Exchange Server from which you are importing the attachments.
- For Datacap Connector for Fax, authentication is done by the Fax Server from which you are importing the faxes.

For more information about rule sets and tasks, see “Task profiles and rulesets” on page 26.

Integrating Connector actions into applications:

To integrate Datacap Connector actions into Datacap applications, you add the actions that you want to use to functions in your rulesets.

The Datacap applications are unique, so you can do some general steps to incorporate connector actions into an application. You must connect the rules to the appropriate levels in the DCO.

To integrate connector actions into an application:

1. In Datacap Studio, click the **Rulemanager > Actions** tab.
2. Select the ruleset to which you want to add the connector action and click **Lock ruleset for editing**. For example, you might select the Export To P8 ruleset.
3. Click **Sync DCO view with Ruleset view** to expand the Document Hierarchy.
4. Highlight the objects to which the ruleset is bound and note the object names and their object levels, such as Connect or Upload.
5. Select the function into which you want to incorporate the connector action in the **Rulesets** tab. For example, select **Logon**.
6. Select the **Page** or **Fieldlevel** action on the **Actions Library** tab and click **Add to Function** to add the action to the function. If you selected the Logon function, add the Logon action for your content repository. For example, for IBM Content Manager, you add the IBMCM_Logon action.
7. If needed, move the action by clicking **Move Up** or **Move Down** then change the action Properties as needed.
8. Click **Save** to save the changes to the ruleset.
9. Display the Connector Settings by clicking the **Zones** tab, then clicking the **Connector** tab.
10. On the **Document Hierarchy** tab, click **Lock DCO for editing** and select the objects that you highlighted in a previous step.
11. Change the appropriate Connector settings for the selected objects on the **Connector** tab,

12. On the **Document Hierarchy** tab, click **Save Changes** and then click **Unlock DCO**.
13. Test your changes, then click the **Rulesets** tab and click **Publish ruleset**.
“Storing passwords in the .app file”

Storing passwords in the .app file:

To pass passwords as action parameters, use smart parameters that retrieve credentials from the .app file where the passwords are stored as encoded strings.

You can use smart parameters in a key path to access the passwords for the Datacap Connector actions. See “Reference passwords, connection strings, and other parameters from your actions” on page 171 for information about storing action parameters in the .app file.

To store passwords in the .app file:

1. In the **Start** menu click **IBM Datacap Services Datacap Application Manager**.
2. Click the **Custom values** tab and select your application from the list in the left pane.
3. Under the **Advanced values** field, press **Add new**.
4. Enter the password name in the **Value name** field. Create a logical password name for your system, such as *FileNet P8 password*.
5. Enter the password in the **Value** field.
6. Close the Application Manager.
7. Access the password in the action by using the key path for the password, @APPVAR(values/adv/<value name>). For example, if the value name of the password is FileNet P8 password, the key path is @APPVAR(values/adv/<FileNet P8 password>).

Related information:

 [Application Manager](#)

Connector actions configuration:

To export documents and index files into Content repositories and libraries, you must add the Connector actions to the appropriate Export rulesets.

Open Datacap Studio and use the Export ruleset for the repository or library into which you want to export documents. For example, to export documents into an IBM Content Manager, you might use a ruleset named Export To CM. You configure the ruleset with rules that log on to Content Manager and upload a document into the repository. These rules might be named Connect to CM and AddDocument.

You then add functions like Login and AddPage to these rules. You then configure the functions with IBM Content Manager Actions that define how to Connect To CM and Add a Document.

IBM Content Manager Connector actions:

The IBM Content Manager Connector actions integrate Datacap applications with the IBM Content Manager repository.

Use these actions to upload documents and index fields into an IBM Content Manager repository.

You can configure the IBM Content Manager Connector actions for the following tasks:

- Log in to the IBM Content Manager server
- Search for an item in the IBM Content Manager repository based on an attribute and value or item ID that is provided.
- Create a IBM Content Manager document that is based on the type in the Document Hierarchy as a document or a page
- Add, delete, or replace pages in the IBM Content Manager document as needed
- Set the attribute value on the IBM Content Manager document
- Set the MIME type for the IBM Content Manager document that you are uploading
- Create an IBM Content Manager folder in the parent folder where you can upload documents
- Set the attribute value on the IBM Content Manager folder
- Set the path to the IBM Content Manager folder where you are uploading documents
- Upload the document, page, or directory to the IBM Content Manager server
- Store the item ID of the uploaded IBM Content Manager document or page in the DCO
- Store the ID of the most recently created IBM Content Manager folder into a variable of the Document Hierarchy
 - “IBM Content Manager Connector prerequisites”
 - “IBM Content Manager Connector settings” on page 110
 - “Configuring IBM Content Manager Connector actions” on page 111
 - “IBM Content Manager Connector upload examples” on page 111

IBM Content Manager Connector prerequisites:

To configure and run IBM Content Manager Connector actions, your environment must meet the hardware and software requirements for Datacap, Version 9.0.

The following components must be installed and running on your system before you can use IBM Content Manager Connector actions to upload images into a IBM Content Manager repository.

- Datacap Version 9.0 installed and running on either a single computer or a client/server installation
- Network access to an IBM Content Manager, Version 8.4

The following repository clients must be installed on each Datacap computer that runs the export ruleset. Export actions are run on Rulerunner in production. Export actions can also be run in Datacap Studio or Datacap Desktop for development or test purposes. Computers that run rules must have the appropriate clients, such as Rulerunner and Datacap Studio, installed on them.

- One of the following IBM DB2 Client options:
 - IBM Data Server Client 9.7 for Windows on 32-bit AMD and Intel systems (x86) (CZ1ALML)
 - IBM Data Server Client 9.7 for Windows on AMD64 and Intel EM64T systems (x64) (CZ1AMML)

- IBM Content Manager Enterprise Edition v8.4 Client for Windows Multilingual (C183VML)
- IBM Information Integrator for Content v8.4.2 (CZLB1ML)

Related information:

 [Hardware and Software Requirements for IBM Datacap Version 9.0](#)

IBM Content Manager Connector settings:

Record the system settings that you want to use to configure the IBM Content Manager Connector actions and have these values available during the configuration process.

This table describes the parameters that are required for Datacap Connector for IBM Content Manager actions.

Table 20. Required IBM Content Manager parameter settings

Action	Description
Logon	Server name, user ID, password
Search Item	The attribute name and value or the item ID of the item for which you want to search in IBM Content Manager. The item that is found is set as the current item for the actions that follow this action in the application.
Create Item	The IBM Content Manager item type, such as document or page.
New page	The pages to add to the existing IBM Content Manager document
Existing page	The existing IBM Content Manager page to delete or replace.
Set Attribute Value	A valid IBM Content Manager item type equivalent to a Document Class such as NOINDEX or a predefined Smart Parameter that contains a valid item type.
Create Folder	An IBM Content Manager folder in the parent folder. This new folder is based on the item type and the parent folder ID.
Create Folder Attribute Value	The attribute name and value of the IBM Content Manager folder or a predefined Smart Parameter that contains a valid attribute name and value.
Set Destination Folder	A valid IBM Content Manager destination folder ID based on the parent folder ID.
Upload Document	None
Upload Page	None
Store Item In DCO	The item ID of the document or page that you are uploading.
Store Folder ID In DCO	The folder ID of the most recently created IBM Content Manager ID.

Configuring IBM Content Manager Connector actions:

You must create an Export ruleset and configure its rules and functions with IBM Content Manager Connector actions to upload documents from Datacap applications into IBM Content Manager.

To configure IBM Content Manager Connector actions:

1. Install the IBM Content Manager Runtime Environment. For more information, see the IBM Content Manager client installation instructions.
2. Restart the Datacap Client Station.
3. Add the IBM Content Manager Connector actions (IBMCM.RXX) to the Export rulesets.

The following example describes an Export To Content Manager ruleset that logs on to the IBM Content Manager server and uploads a single page document into IBM Content Manager.

This ruleset contains the Connect and Upload rules. The Connect rule contains the Logon function and action. The Upload rule contains the AddPage function with actions that create the page, set attribute values for the page, upload, and store the page.

Export To Content Manager ruleset

- Connect rule
 - Logon function
 - IBMCM_Logon("ibmcmsrv,userid,password")
- Upload rule
 - AddPage function
 - IBMCM_CreateItem("APT")
 - IBMCM_SetAttributeValue("APT_Title,Page from CM8ItemDCO")
 - IBMCM_SetAttributeValue("APT_Date,@P.VerifyTime")
 - IBMCM_SetAttributeValue("APT_Vendor,@P.Vendor")
 - IBMCM_CreateFolder("APT_Folder", "123456789")
 - IBMCM_SetFolderAttributeValue("Name", "APT_Folder")
 - IBMCM_SetDestinationFolder("123456789")
 - IBMCM_UploadDCO_Page()
 - IBMCM_StoreItemIDinDCO("CM8ItemDCO")
 - IBMCM_StoreFolderIDinDCO("APT_Folder")

IBM Content Manager Connector upload examples:

The Datacap Connector for IBM Content Manager Upload actions configure the connection between the Datacap application and the IBM Content Manager repository.

You use these actions to upload a single page file or an image that contains multiple pages and their associated index values from Datacap into IBM Content Manager.

These actions are based on the IBM Content Manager Java™ API. If you use the IBM Content Manager Java APIs, you must install the IBM Information Integrator for Content connector for Content Manager on the computers where you want to run these actions.

The examples in the following tables show the sequence in which you must add the actions to the Export To Content Manager ruleset for the upload scenarios.

Upload a single page file

Table 21. The sequence of actions for uploading a single page file into IBM Content Manager

Action	Description
IBMCM_Logon("ibmcmsrv,userid,password")	Log the application on to the IBM Content Manager server.
IBMCM_CreateItem("NOINDEX")	Create an IBM Content Manager document.
IBMCM_SetAttributeValue("USERID,@OPERATOR")	Set an attribute value on the IBM Content Manager document.
IBMCM_SetMimeType("application/msword")	Set the MIME type for the IBM Content Manager document you are uploading.
IBMCM_CreateFolder("NOINDEX","123456789")	Create an IBM Content Manager folder that is based on the item type and parent folder ID.
IBMCM_SetFolderAttributeValue("Name","MyFolder")	Set an attribute value on the IBM Content Manager folder.
IBMCM_SetDestinationFolder("\APT")	Identify the folder into which the uploaded is IBM Content Manager.
IBMCM_UploadDCO_Page()	Upload the images that are associated with the current Page object of the document hierarchy to IBM Content Manager.

Upload a multiple page file

Table 22. The sequence of actions for uploading a multiple page file into IBM Content Manager

Action	Description
IBMCM_Logon("ibmcmsrv,userid,password")	Log the application on to the IBM Content Manager server.
IBMCM_CreateItem("NOINDEX")	Create a IBM Content Manager document.
IBMCM_SetAttributeValue("USERID, @OPERATOR")	Set an attribute value on the IBM Content Manager document.
IBMCM_SetMimeType("application/msword")	Set the MIME type for the IBM Content Manager document you are uploading.
IBMCM_CreateFolder("NOINDEX","123456789")	Create an IBM Content Manager folder that is based on the item type and parent folder ID.

Table 22. The sequence of actions for uploading a multiple page file into IBM Content Manager (continued)

Action	Description
IBMCM_SetFolderAttributeValue("Name","MyFolder")	Set an attribute value on the IBM Content Manager folder.
IBMCM_SetDestinationFolder ("\\APT")	Identify the folder into which the uploaded is IBM Content Manager.
IBMCM_UploadDCO_DOC()	Upload the images that are associated with the current Document object of the document hierarchy to IBM Content Manager.

FileNet P8 Connector actions:

The FileNet P8 Connector actions integrate Datacap applications with IBM FileNet Content Engine.

You can use FileNet P8 Connector actions to upload documents and index fields into a Content Engine repository.

To use the Secure Socket Layer to encrypt communications between Datacap and the IBM FileNet P8 repository, you must setup an SSL-encrypted connection in the FileNet P8 client.

This list describes the main function of the FileNet P8 Connector actions.

- Set up the URL of the Content Engine repository
- Log in to the Content Engine
- Set the class ID of the target location on Content Engine as ObjectStore or FileStore
- Set a locale that is accepted by the IBM FileNet P8 web service
- Set the object ID for the Object Store on Content Engine
- Set the path to the IBM FileNet P8 folder where you are uploading documents
- Specify the content type that defines the fields within a document library for the uploaded documents, such as an Invoice
- Create a folder in Content Engine where you can upload documents
- Upload the document, page, or directory into Content Engine
 - “FileNet P8 Connector prerequisites”
 - “FileNet P8 Connector settings” on page 114
 - “Configuring FileNet P8 Connector actions” on page 115
 - “FileNet P8 Connector upload examples” on page 116

FileNet P8 Connector prerequisites:

To configure and run FileNet P8 Connector actions, your environment must meet the hardware and software requirements for Datacap, Version 8.0, 8.0.1, 8.1, and 9.0.

The following repository clients must be installed on each Datacap computer that runs the export rule set. Export actions are run on Rulerunner in production. Export actions can also be run in Datacap Studio and Datacap Desktop for

development or test purposes. Computers that run rules must have the appropriate clients, such as Rulerunner and Datacap Studio, installed on them.

- Datacap Version 8.0, 8.0.1, 8.1, or 9.0 installed and running on either a single computer or a client/server installation
- Network access to an IBM FileNet Content Engine, Version 4.5.0, 4.5.1, 5.0, 5.1, or 5.2, where IBM FileNet Content Manager is supported
- Network access to an IBM FileNet P8 Content Server Library through the IBM FileNet P8 XML Web Service V3.5 or V4.0

The following repository clients must be installed on the Datacap Web Client that runs the Datacap export process. These clients are run on Microsoft Windows operating systems.

- Microsoft Web Service Enhancements 3.0 Runtime
- Access to the Content Engine software package so that you can run the Content Engine installation to download the CE.NET client

Related information:

 [Hardware and Software Requirements for IBM Datacap Version 9.0](#)

FileNet P8 Connector settings:

Record the system settings that you want to use to configure the FileNet P8 Connector actions and have these values available during the configuration process.

This table describes the parameters that are required for Datacap Connector for FileNet Content Manager actions.

Table 23. Required IBM FileNet P8 parameter settings

Connector action	Description
Set URL	The URL for the FileNet P8 web service.
Logon	FileNet P8 user ID and password.
Set Target Class ID	The value of the Class ID. Use either ObjectStore or FileStore. The default value is ObjectStore.
Set Locale	The locale value that is accepted by the FileNet P8 web service. Represented by a two-letter language code and a two-letter country code, for example, en_US or de_DE.
Set Target Object ID	The Object ID value to assign to the object store.
Set Destination Folder	The path to the FileNet P8 folder in the object store where you are uploading the documents, for example \TravelDocs\.
Create Folder	The name of the folder to create for the target class and object.
Set Doc Class ID	The value of the Document Class ID. The default value is Document.

Table 23. Required IBM FileNet P8 parameter settings (continued)

Connector action	Description
Set Doc Title	The value of a Document Title or a predefined special variable. The default value is Title.
Set Property	The value of the Property ID and the value or predefined special variable to assign to the property.
Set Multiple Page Documents	The parameter that specifies whether the upload actions create a single page or a multiple page document.
Upload Document	None
Upload Page	None
Upload Dir	The full path of the folder that contains the images you want to upload, for example, C:\images,True. Use True to delete the images from the folder after they are uploaded. Use False to leave the images in the folder after they are uploaded.

Configuring FileNet P8 Connector actions:

You must create an Export ruleset and configure its rules and functions with FileNet P8 Connector actions to upload documents from Datacap applications into Content Engine.

Datacap Connector for FileNet Content Manager actions can upload images from a Datacap batch to the IBM FileNet Content Server library by using the IBM FileNet P8 XML web service.

To configure FileNet P8 Connector actions:

1. Install the IBM FileNet P8 Runtime Environment and its prerequisites. For more information, see the IBM FileNet P8 installation instructions.
2. Install the IBM FileNet Content Engine Client files that are provided with the Content Engine Server installation program. The version of the Content Engine Client you install must match the version of the Content Engine Server. Run the installation program that matches the version of the installed Content Engine Server.

Version	Part Number	Installation program
IBM Content Manager 5.1	CI1NIML	5.1.0-P8CE-Win.exe
IBM Content Manager 5.0	CZS02ML	5.0.0-P8CE-Win.exe

3. Verify the URL and the version of the FileNet P8 Server. For example, <http://myp8server:9080/wsi/FNCEWS40MTOM>
4. Add the FileNet P8 Connector actions (FileNetP8.RRX) to the Export rulesets.

The following example describes an Export To P8 ruleset that logs on to the Content Engine server. Then, it uploads a single page document into the Content Engine repository.

The ruleset contains the Connect to CE and AddDocument rules. The Connect to CE rule contains the Logon function and actions you must run to make the connection to Content Engine. The AddDocument rule contains the AddPage function with actions that define the title and format of the page, and upload the page.

Export To P8 ruleset

- Connect to CE rule
 - Logon function
 - FNP8_SetURL("http://MyServer:9080/wsd/FNCEWS40MTOM")
 - FNP8_Login("P8Admin UserID,P8Admin Password")
 - FNP8_SetLocale("en_US")
 - FNP8_SetTargetClassID("ObjectStore")
 - FNP8_SetTargetObjectID("ObjectStoreName")
 - FNP8_SetDestinationFolder("/mydestfolder")
- AddDocument rule
 - AddPage function
 - FNP8_SetDocTitle("@ID")
 - FNP8_SetDocType("TIF")
 - FNP8_Upload()

FileNet P8 Connector upload examples:

The Datacap Connector for IBM Content Manager Upload actions configure the connection between the Datacap application and the IBM Content Manager repository.

You use these actions to upload a single page file or an image that contains multiple pages and their associated index values from Datacap into Content Engine.

The examples in the following tables describe the sequence in which you must add the actions to the Export To P8 ruleset for the upload scenarios.

Upload a single page file

Table 24. The sequence of actions for uploading a single page file into IBM FileNet Content Engine.

Action	Description
FNP8_SetURL("http://MyServer:9080/wsd/FNCEWS40MTOM")	Establish the URL for the FileNet P8 web service.
FNP8_Login("admin,password")	Provide the Content Engine user login credentials: admin and password.
FNP8_SetTargetClassID("ObjectStore")	Set the top-level repository type on Content Engine to ObjectStore.
FNP8_SetLocale("en_US")	Specify en_us as the locale used by the IBM FileNet P8 web service.

Table 24. The sequence of actions for uploading a single page file into IBM FileNet Content Engine. (continued)

Action	Description
FNp8_SetTargetObjectID("AP_ObjectStore")	Specify the identifier for the object store in which to store the file as AP_ObjectStore.
FNp8_SetDestinationFolder("\TravelDocs")	Identify the folder into which the document is uploaded in Content Engine as \TravelDocs.
FNp8_SetDocTitle("@ID")	Set the title property for the page to @ID.
FNp8_SetDocType("TIF")	Set the type property for the page to TIF.
FNp8_Upload()	Upload the image file for the page to the previously specified destination folder on Content Engine.

Upload a multiple page file

Table 25. The sequence of actions for uploading a multiple page file into IBM FileNet Content Engine.

Action	Description
FNp8_SetURL("http://MyServer:9080/wsd/FNCEWS40MTOM")	Establish the URL for the FileNet P8 web service.
FNp8_Login("admin,password")	Provide the Content Engine user login credentials: admin and password.
FNp8_SetTargetClassID("ObjectStore")	Set the top-level repository type on Content Engine to ObjectStore.
FNp8_SetLocale("en_US")	Specify the language to use on the IBM FileNet P8 Web Service. For example, enter en_US if you are using US English on the user interface.
FNp8_SetTargetObjectID("AP_ObjectStore")	Specify the identifier for the object store in which to store the file as AP_ObjectStore.
FNp8_SetDestinationFolder("\TravelDocs")	Identify the folder into which the document is uploaded in Content Engine as \TravelDocs.
FNp8_SetDocClassID("Document")	Set the FileNet Document Class ID for the page to Document.
FNp8_SetProperty("DocumentTitle")	Set the property for this document to the Datacap title of the current document.
FNp8_Upload()	Upload the multiple page image file for this document to the specified destination folder on Content Engine.

SharePoint Connector actions:

The Datacap Connector for Microsoft SharePoint actions integrate Datacap applications with Microsoft Office SharePoint Services (MOSS) for Microsoft SharePoint 2007 and 2010.

You then use SharePoint Connector actions to upload documents and index fields into a SharePoint library.

The following list describes the main functions of the SharePoint Connector actions.

- Log in to the SharePoint library
- Identify and set up the URL of the SharePoint library
- Specify the content type that defines the fields within a document library for the uploaded documents, such as an Invoice
- Set the format in which to release documents to the SharePoint library, such as TIF or PDF
- Create a folder in the SharePoint into which you upload documents
- Set the column properties (index values) in SharePoint for the documents you want to upload
- Upload the indexed documents into the SharePoint library
 - “SharePoint Connector prerequisites”
 - “SharePoint Connector settings”
 - “SharePoint and Datacap” on page 119
 - “Configuring SharePoint Connector actions” on page 121
 - “SharePoint Connector upload examples” on page 121

SharePoint Connector prerequisites:

To configure and run SharePoint Connector actions, your environment must meet the hardware and software requirements for Datacap, Version 8.0, 8.0.1, and 9.0.

The repository clients must be installed on each Datacap computer that runs the export ruleset. Export actions are run on Rulerunner in production. Export actions can also be run in Datacap Studio or Datacap Desktop for development or test purposes. Computers that run rules must have the appropriate clients, such as Rulerunner and Datacap Studio, installed on them.

You must meet the following prerequisites to export images to a SharePoint library:

- Datacap Version 8.0, 8.0.1, 9.0, or 9.0 installed and running on either a single computer or a client/server installation
- Network access to a SharePoint 2010 Server or a SharePoint 2007 Server with Microsoft Office SharePoint Services (MOSS) 3.0 installed
- SharePoint URL or HTTP address of every library to which you are releasing images
- Details about the columns in the library where you are exporting images; such as the static names of the columns, the column types, and column restraints
- Valid Content Types for each library and the exact spelling of these Content Types.
- The user ID that logs in to SharePoint is different than the user ID on the computer where the Export task runs. Then, you need the SharePoint login credentials.

Related information:

 [Hardware and Software Requirements for IBM Datacap Version 9.0](#)

SharePoint Connector settings:

Record the system settings that you want to use to configure the SharePoint Connector actions and have these values available during the configuration process.

This table describes the parameters that are required for Datacap Connector for Microsoft SharePoint actions.

Table 26. Required SharePoint Connector parameter settings

Connector action	Description
Create Folder	The folder in the SharePoint library into which you import your documents.
Set URL	The URL address of the SharePoint library.
Login	User ID, password, optional SharePoint domain.
Set Content Type	The name of the Content Type that defines the fields within a document library for the uploaded documents, such as an Invoice
Set File Type	The format in which to upload the document to the SharePoint library, for example TIF or PDF.
Set Property	The column property in SharePoint for the documents you want to upload.
Upload Batch	None
Upload Document	None
Upload Page	None
Upload Dir	The full path of the folder that contains the images you want to upload, for example, C:\images,True. Use True to delete the images from the folder after they are uploaded. Use False to leave the images in the folder after they are uploaded.

SharePoint and Datacap:

You create SharePoint columns at the library level, not at the folder level. Datacap passes Datacap index values to these columns.

The following table identifies the relationships between SharePoint columns and the index values passed to SharePoint columns by Datacap.

SharePoint Column Type	Datacap Index Value
Column constraints	Construct your Datacap application so it produces and exports index values that are valid according to the column constraints of SharePoint. If the index field value passed to SharePoint does not fit within the SharePoint constraints, the upload to SharePoint fails. SharePoint messages are logged in the SPExport_rrs.log file in the batch folder.

SharePoint Column Type	Datacap Index Value
When a column is defined as required in SharePoint	Each required column in a SharePoint library must be set up with a default value defined. Ensure that the index value for a required SharePoint column is always exported by the Datacap application. If during Datacap processing the Operator overrides a required field and an empty field is passed to SharePoint. The upload to SharePoint fails if no default value is defined for the column. SharePoint messages are logged in the SPExport_rrs.log file in the batch folder.
Single line of text columns (index fields) Might or might not have the Maximum number of characters	The index value must be a text string. The index value for this type of column can contain special characters, for example, !@#\$%^&*()_< >. Ensure that the exported index value does not contain more than maximum number of characters that are allowed for SharePoint columns.
Multiple lines of text	Same as single line of text.
Choice (multi-value list)	Define the default value to use when Allow Fill-In Choices is set to No .
Number (integer, float)	
Currency	
Yes/No columns	The exported index value must be one of the following values: • 0 for No • 1 for Yes
Date or Data and Time	The exported index value must be one of the following values: • YYYY-MM-DD • YYYY-MM-DDTHH:MM:SSZ (T and Z must enclose the time stamp)
Lookup	Not supported.
Calculated	Not supported
Business data	Not supported
Hyperlink or picture	Ensure that the exported index value is a valid URL address
Document Library Versioning Settings - Document Version History must be set to No versioning	Does not support version history.
Person or Group	Cannot be exported, automatically assigned by SharePoint.

Configuring SharePoint Connector actions:

Create an Export ruleset and configure its rules and functions with SharePoint Connector actions. Then, you can upload documents from Datacap applications into a SharePoint library.

You can export documents from an Datacap batch to a SharePoint library by adding the SharePoint Connector actions to the Export rulesets.

To configure SharePoint Connector actions:

1. Verify the URL of the SharePoint library.
2. Add the SharePoint connector actions (SPExport.RRX) to the Export rulesets.

The following example describes an Export To SP ruleset that logs on to SharePoint and uploads a single page document into the SharePoint library.

The ruleset contains the Connect to SP and AddDocument rules. The Connect to SP rule contains the Logon function and actions you must run to make the connection to the SharePoint library. The AddDocument rule contains the AddPage function with actions that define the title and format of the page, and upload the page.

Export To SP ruleset

- Connect to SP rule
 - Logon function
 - SP_Login("userID,password,domain")
 - SP_SetURL("http://blue/Docs/Documents/+BatchID+/+@ID")
 - SP_CreateFolder("http://blue/Docs/Documents/Test")
 - SP_Property("Date,@Value")
- AddDocument rule
 - AddPage function
 - SP_SetContentType("Invoice")
 - SP_SetFileType("jpg")
 - SP_Upload()

SharePoint Connector upload examples:

The SharePoint Connector Upload actions configure the connection between the Datacap application and the SharePoint library.

The Datacap Connector for Microsoft SharePoint Upload actions configure the connection between the Datacap application and the SharePoint library. You use these actions to upload a single page file or an image that contains multiple pages and their associated index values from Datacap into SharePoint.

The examples in the following tables describe the sequence in which you must add the actions to the Export To SharePoint ruleset for the upload scenarios.

Upload a single scanned image file

Table 27. The sequence of actions for uploading a single scanned image file into SharePoint.

Action	Description
SP_SetURL("http://full.url.com")	Establish the URL for the SharePoint library.

Table 27. The sequence of actions for uploading a single scanned image file into SharePoint. (continued)

Action	Description
SP_Login("admin,password")	Provide the SharePoint user login credentials: admin and password .
SP_Upload()	Upload the image file for the page to the specified URL location on the SharePoint library.

Upload a batch of scanned images

Table 28. The sequence of actions for uploading a batch of scanned images into a SharePoint library.

Action	Description
SP_SetURL("http://full.url.com")	Establish the URL for the SharePoint library.
TifMerge_MergeImages("all")	Optional: During processing, merge all the images of the pages that are associated with the current document.
SP_SetContentType("tiff")	Assign the Content Type to be TIFF for the page that you are uploading to the repository.
SP_Login("admin,password")	Provide the SharePoint user login credentials: admin and password .
SP_Upload()	Upload the image file for this batch to the specified URL location on the SharePoint library.

Upload pre-scanned images

Table 29. The sequence of actions for uploading pre-scanned images into a SharePoint library.

Action	Description
SP_SetURL("http://full.url.com")	Establish the URL for the SharePoint library.
SP_SetProperty("Date,@Value")	Set an index value for the Date column in SharePoint.
SP_Login("admin,password")	Optional: Provide the SharePoint user login credentials: admin and password .
SP_UploadDIR("/MyImages")	Upload the image files in this directory to the specified URL location on the SharePoint library. Specify whether the files are uploaded or deleted.

Collect field data and populate SharePoint columns

Table 30. The sequence of actions for collecting field data and populating SharePoint columns.

Action	Description
SP_SetURL("http://full.url.com")	Establish the URL for the SharePoint library.

Table 30. The sequence of actions for collecting field data and populating SharePoint columns. (continued)

Action	Description
TifMerge_MergeImages("all")	Optional: During processing, merge all the images of the pages that are associated with the current document.
SP_SetProperty("Date,@Value")	Set an index value for the Date column in SharePoint.
SP_SetContentType("tiff")	Assign the Content Type to be TIFF for the page that you are uploading to the repository.
SP_Login("admin,password")	Provide the SharePoint user login credentials: admin and password .
SP_Upload()	Upload the image file for this batch to the specified URL location on the SharePoint library.

FileNet Image Services Connector Connecting actions:

You use Datacap Connector for FileNet Image Services actions to upload documents and commit images to an IBM FileNet Image Services library.

The Rulerunner Service task that applies FileNet Image Services Connector rules and remembers the images that were previously committed to FileNet. When the Rulerunner Service task runs FileNet Image Services Connector Upload procedures, these previously uploaded images are not recommitted. The Rulerunner Service task generates a separate and unique Page file (<upload>.xml) every time it uploads a FileNet document. When the actions to upload documents and commit images to an IBM FileNet Image Services components initialize, the task polls the active batch folder for this Page file. If it does not find the file, it creates a new Page file.

The main function of the FileNet Image Services Connector actions to upload documents and commit images to an IBM FileNet Image Services library:

- Access and open an IBM FileNet Image Services library
 - Create a FileNet document to upload into the library
 - Define an Index Map that links FileNet properties to values that are associated with objects of the Document Hierarchy
 - Associate images with FileNet documents
 - Upload indexed documents and images for commitment to the library
- “FileNet Image Services Connector prerequisites”
“FileNet Image Services Connector settings” on page 124
“Configuring FileNet Image Services Connector actions” on page 125
“FileNet Image Services Connector upload examples” on page 126

FileNet Image Services Connector prerequisites:

To you configure and run FileNet Image Services Connector actions, your environment must meet the hardware and software requirements for Datacap, Version 8.0, 8.0.1, and 9.0.

The following repository clients must be installed on each Datacap computer that runs the export ruleset. Export actions are run on Rulerunner in production. Export

actions can also be run in Datacap Studio or Datacap Desktop for development or test purposes. Computers that run rules must have the appropriate clients, such as Rulerunner and Datacap Studio, installed on them.

- Datacap Version 8.0, 8.0.1, 9.0, or 9.0 installed and running on either a single computer or a client/server installation
- IBM FileNet IDM Desktop client
- Network access to the IBM FileNet Image Services library

Related information:

 [Hardware and Software Requirements for IBM Datacap Version 9.0](#)

FileNet Image Services Connector settings:

Record the system settings that you want to use to configure the FileNet Image Services Connector actions and have these values available during the configuration process.

This table describes the parameters that are required for Datacap Connector for FileNet Image Services actions.

Table 31. Required FileNet Image Services Connector parameter settings

Action	Description
Library Initialize (IS)	Elements of a previously defined library name by using the syntax <code><DefaultIMS>:<domain>:<organization></code> For example, ISLibrary:Datacap:FileNet
Library Logon	User ID, password
FileNet Database ADO Connect	None
New Document	The name of a previously defined Document Class
Add All Images to Document	None
Create Folder	The name of the new folder, for example Taxes2011
Get Top Folders	None
Save Doc To Folder	The folder name that is preceded by a forward slash, for example /Taxes2011
Add TIF Image To Folder	None
Add PDF Image To Folder	None
Add File To Document	The path name and file name of the file to add to the document, for example C:\Datacap\MQSW\Process\FNLog.log
FileNet Doc ID Set Value	The name of the child field object to which you want to assign the FileNet Document ID.
Use Indexes ON	None
Use Indexes OFF	None

Table 31. Required FileNet Image Services Connector parameter settings (continued)

Action	Description
Index Property ID Date Component	The following 4 values: 1. The name of the Date property 2. Name of a Document Hierarchy object with a Date property 3. Format of the Date when supplied to the FileNet document 4. Format value of the Date value that is added to the processing index of the task For example, IndexProperty_ID_Date_Component (FNStart,1040EZ,mmddy,yyyymmdd)
Index Property Left Justify	Name of the FileNet document property to left align and the maximum size of the value. For example, FNfldData,256
Index Property Right Justify	Name of the FileNet document property to right align and the maximum size of the value. For example, FNfldData,256
Upload	None

Configuring FileNet Image Services Connector actions:

You must create an Export ruleset and configure its rules and functions with FileNet Image Services Connector actions to upload documents from Datacap applications into a FileNet Image Services library.

You can export documents from an Datacap batch to an IBM FileNet Image Services library by adding the FileNet Image Services Connector actions to the Export rulesets.

To configure FileNet Image Services Connector actions:

1. Verify the URL of the library into which you want to export documents.
2. Add the FileNet Image Services Connector actions (FileNetIDM.RRX) to the Export rulesets.

The following example describes an Export To IS ruleset that logs on to FileNet Image Services and uploads a single page document into the library.

The ruleset contains the Connect to IS, CreateDocument, and AddDocument rules. The Connect to IS rule contains the Logon function and actions you must run to make the connection to the FileNet Image Services library. The AddDocument rule contains the AddPage function with actions that define the title and format of the page, and upload the page.

Export To IS ruleset

- Connect to IS rule
 - Logon function
 - Library_IS_Initialize(ISLibrary:Datacap:FileNet)
 - Library_Login("userID,password")

- AddDocument rule
 - AddPage function
 - NewDocument("1040EZtwo")
 - AddFileToDocument(C:\Datacap\MSQW\Process\FNLog.log)
 - Upload()

FileNet Image Services Connector upload examples:

The FileNet Image Services Connector Upload actions configure the connection between the Datacap application and the FileNet Image Services library.

Upload a single page file

Use these actions to upload a single page file or a document that contains multiple pages from Datacap into the FileNet Image Services library.

The examples in the following tables show the sequence in which you must add the FileNet Image Services Connector actions to the Export ruleset for the upload scenarios.

Table 32. The sequence of actions for uploading a single page file into an FileNet Image Services library.

Action	Description
Library_IS_Initialize (ISLibrary:Datacap:FileNet)	Initialize the previously defined FileNet Image Services library.
Library_Login("userid,password")	Log in to FileNet Image Services library.
FileNetDB_ADOConnect()	Establish an Active X Data Connection object (ADO) with the specified FileNet database.
NewDocument(1040EZtwo)	Set up a new FileNet document and specify the FileNet Document Class to assign to the new document.
CreateFolder(IncomeTaxes_2011)	Create a top-level FileNet folder in the FileNet Image Services library.
Upload()	Import the document into the FileNet Image Services library.
SaveDocToFolder(IncomeTaxes_2011)	Put the document in the specified folder in the FileNet Image Services library.

Upload a multiple page file

Table 33. The sequence of actions for uploading a multiple page file into an FileNet Image Services library.

Action	Description
Library_IS_Initialize (ISLibrary:Datacap:FileNet)	Initialize the previously defined FileNet Image Services library.
Library_Login("userid,password")	Log in to FileNet Image Services library.
FileNetDB_ADOConnect()	Establish an Active X Data Connection object (ADO) with the specified FileNet database.
NewDocument(1040EZtwo)	Set up a new FileNet document and specify the FileNet Document Class to assign to the new document.

Table 33. The sequence of actions for uploading a multiple page file into an FileNet Image Services library. (continued)

Action	Description
AddAllImagesToDocument()	Assigns all of the images that are associated within the Document object of the Document Hierarchy to the new document.
CreateFolder(IncomeTaxes_2011)	Create a top-level FileNet folder in the FileNet Image Services library.
Upload()	Import the document into the FileNet Image Services library.
SaveDocToFolder(IncomeTaxes_2011)	Put the document in the specified folder in the FileNet Image Services library.

Email Connector actions:

Email Connector actions create Datacap batches from the documents that you receive as email attachments. You can also send email notification messages when specific events occur.

The Email Connector actions option contains the following actions libraries:

- IMAP Email Input Actions (IMail.RRX)
- Exchange Web Service Email Input Actions (EWSMail.RRX)
- Email Sending Actions (EMail.RRX)
 - “Email Input actions”
 - “Email Send actions” on page 128
 - “Email Connector prerequisites” on page 128
 - “Email Connector settings” on page 129
 - “Configuring Email Connector actions” on page 131
 - “Email Connector import examples” on page 132

Email Input actions:

Email Input actions scan an email inbox for incoming mail messages and place selected messages and attachments into a new Datacap batch for processing.

Datacap supports two methods of accessing a mail server to obtain the image attachments.

- The actions in IMail.RRX use the Internet Message Access Protocol (IMAP) for Microsoft Exchange Server, Novell GroupWise, and other mail servers that provide support for IMAP.
- The actions in EWSMail.RRX use Exchange Web Services (EWS) for Microsoft Exchange Server. This service is a SOAP-based method of communication.

The Input actions are typically assigned to a Datacap task that is run by an unattended Rulerunner Service station. These Input actions scan one or more inboxes and defines the type of attachments to include in the batch. For example, you can specify only TIFF images or only PDF files, which helps eliminate input of improper files.

The batch is created when the maximum number of documents is received from the email servers or a specified time interval elapsed. At the batch level, the

EmailCount is captured. Each document in the batch is associated with one email message. The document contains a page for each attachment file and variables with the email Subject, From, To, DateSent, Priority, and Body. In addition, the IMail actions capture User, and the EWSMail actions capture DateReceived.

When an email is successfully processed and the attachment is input into the Datacap batch. The email is moved from the inbox to a specified email done folder. If the attachment is not one of the expected types such as a TIFF file, or there is a processing problem. The email, and attachment does not become part of the batch and are moved to a specified email problem folder.

Attention: To further process of the body of an email, or an attachment other than TIF after it was added to a Datacap batch by using the Email actions. You might have to construct more rules and to license other Datacap options.

To capture MS Office document attachments and turn them into images that can be processed by using the Datacap standard Recognition or Verify tasks. You can license the eDocument Conversion actions and put them in the processing rules of your application.

If you capture non-image files by using the Email Input actions, the pages that are created have non-image files that are attached to them. If you capture only the email body, the page might not have any files that are attached. In either of these cases, your application must handle these documents and pages. Processing a batch that contains pages with no images attached by using the standard recognition actions or the verify task, might not work as you expect.

Email Send actions:

You configure the Email Send actions for a Datacap task to compose and send informational email messages under the conditions that you define.

You can send notification emails directly to multiple recipients or use the CC and BCC options. The subject line can be specified and the email can be sent with or without attachments. The Send Action is useful when the other actions encounter exceptions and must notify a user. For example, if an error occurs during data verification, you can alert an administrator to take the appropriate action. The items that caused the failure can be attached to the email. If export rules encounter a problem and a batch is not exported successfully, an email can be composed and sent that contains the details.

Email Connector prerequisites:

To configure and run Email Connector actions, your environment must meet the hardware and software requirements for Datacap, Version 8.0, 8.0.1, and 9.0.

The following components must be installed and running on your system before you can use Email Connector actions:

- Datacap Version 8.0, 8.0.1, 9.0, or 9.0 installed and running on either a single computer or a client/server installation

For a list of the hardware and software requirements, see <http://www-01.ibm.com/support/docview.wss?uid=swg27020397>.

Email Input actions prerequisites

To use the Datacap Email Input actions, you must have access to a mail server with one of the following servers:

- Exchange Web Service (EWS) enabled on Microsoft Exchange. Datacap recommends that SSL is configured. Use the input actions in `EWSMail.RRX`.
- Email server that supports IMAP protocol and the mail server and firewall (if any) have IMAP access that is enabled. SSL is not supported now. Use the input actions in `IMail.RRX`.

You must have an email account that you can use for which you know the server URL, and login user name and password.

The dedicated email account contains an Inbox folder, a *Done* folder for messages that are successfully imported, and a *Problem* folder for messages that encountered errors. The names that you assign to these folders can be specified by using the `im_done_folder` and `im_problem_folder`, or `ex_done_folder` and `ex_problem_folder` actions.

Optional: set up an inbox that is dedicated to the emails that you intend to process. If there are preexisting messages in the Inbox, the oldest messages are processed first.

Email Send actions prerequisites

To use the Datacap Email Sending actions in `IMail.RRX` ensure:

- You have access to an SMTP server that can relay the emails that are created by the Datacap application.
- The Email Sending actions are run on a computer on which the Windows CDOSYS object or Microsoft Outlook object are registered. The Email Sending actions use the CDOSYS object by default to send email. If the CDOSYS object is not available, the actions can use the Outlook object. The CDOSYS object is available under Windows XP, Windows 2000, Windows 2003 Server or later.
- The Email Sending actions are run on a Workstation and under a user account that has permission to relay or send emails.

Related information:

 [Hardware and Software Requirements for IBM Datacap Version 9.0](#)

Email Connector settings:

Record the system settings that you want to use to configure the Email Connector actions and have these values available during the configuration process.

Each email message that contains one or more of the wanted attachment types becomes a new Datacap document. Message headers and body, and pages are created for each of the attachments.

IMail and EWSMail set an EmailCount at the Batch level and at the following Document level variables for each email message accepted (Done):
TYPE="Document", Message ID, Subject, From, To, DateSent, Priority, Body= User.

IMail also sets the following Page level variables for each attachment:
TYPE="Other", IMAGEFILE=*attachment filename*.

When you do not use the ex_EMLOption action, EWSMail sets the following Page level variables for each attachment: TYPE="Other", IMAGEFILE=*attachment filename in batch* ATTACHNAME=*original attachment filename*.

When you use the ex_EMLOption action for EWSMail, pages for attachments are not created, and variables for those attachments are not set.

Email Input actions

Use the actions in IMail.RRX when your mail server uses the Internet Message Access Protocol (IMAP). These include Microsoft Exchange Server, Novell GroupWise, and others.

Use the EWSMail Input actions when your mail server is an Microsoft Exchange mail server, which is configured to allow Exchange Web Service access.

Before you configure these actions, record the appropriate values for your system and have them available during the configuration process.

Table 34. Required Email Input actions parameter settings

Action	IMail.RRX	EWSMail.RRX
EWS Version		Version of Microsoft Exchange to use for EWSMail Input actions
Logon	URL of mail server, username, and password of the mail account	URL of mail server, username, and password of the mail account
Scan	None	None
Logout	None	None
Types	List of image files extensions to import	List of image files extensions to import
Wait Time	Maximum number of seconds to wait for input emails for a single batch	Maximum number of seconds to wait for input emails for a single batch
Abort Time	Number of seconds to wait before returning an abort	Number of seconds to wait before returning an abort
Max Docs	Maximum number of emails in each batch	Maximum number of emails in each batch
Done Folder	Name of folder into which successfully imported emails are stored	Name of folder into which successfully imported emails are stored
Problem Folder	Name of folder into which unsuccessfully imported emails are stored	Name of folder into which unsuccessfully imported emails are stored
EML Option		Optional for EWSMail Input actions: Create a one page document that contains the email and attachment in an .eml file. No attachment pages are created.

Email Sending actions

This table describes the parameters that are required for Email Sending actions. Before you configure these actions, record the appropriate values for your system and have them available during the configuration process.

Table 35. Required Email Sending actions parameter settings

Action	Description
Send Email	None
Set Attachment	Pathname and file name of the file you want to attach to the current email. Smart parameters are supported.
Set Blind Carbon Copy Receipts	Email addresses that receive a copy of the email as a blind carbon copy. You can enter multiple email addresses separated by commas.
Set Carbon Copy Receipts	Email addresses that receive a copy of the email as a carbon copy. You can enter multiple email addresses separated by commas.
Set Mail Body	Email message text. Smart parameters are supported
Set Mail Server	IP or DNS address of the outgoing mail (SMTP) server
Set Recipients	Email addresses of the recipients of the email
Set Sender	Email address of the sender of the email
Set Subject	Subject line of the email. Smart parameters are supported.

Configuring Email Connector actions:

Email Connector can actions to scan Email servers that support IMAP protocol or Exchange Web Service (EWS) for incoming email messages with attachments. You specify and export the attachments into a batch.

To configure Email Connector actions:

1. For EWS Email servers, specify the version of the mail server you want to scan for attachments.
2. Add the Email actions for your Email server (IMail.RRX or EWSMail.RRX) to the Export rulesets.

The following example describes a Use EWS ruleset that selects the version of the EWS Email server to use. The ruleset logs on to the mail server, scans the server for incoming mail with attachments and imports them into the batch.

The ruleset contains the Connect to EWS and Find Attachment rules. The Connect to EWS contains the Version and Logon functions and actions you must make the connection to the wanted version of the Email server. The Find Attachment rule contains the Scan function with actions that locate the specified attachments and import them into the batch.

Export EWS ruleset

- Connect to EWS rule
 - Version function
 - ex_ews_version("1")
 - Logon function
 - ex_login("hostname", "username", "password")
- Find Attachment rule
 - Scan function
 - ex_types("tiff", "pdf")
 - ex_scan()

Email Connector import examples:

The Email Connector actions connect to the Email servers that support IMAP protocol or Exchange Web Service (EWS). These actions scan incoming email messages for attachments that contain the types of attachments that you specify and import the attachments into a batch.

Import from an IMAP Email server

The examples in the following tables show the sequence in which you must add the actions to the Export ruleset for the different Email servers.

Table 36. The sequence of actions to use for an IMAP Email server

Action	Description
im_login("hostname","userid,password")	Log in to Email server for the specified host name.
im_types("tif", "pdf")	Specify the email attachment types that you want to import.
im_done_folder("folder_name")	Specify the destination IMAP folder for successfully imported email messages. If this Action is not called, the default folder named Done is used.
im_problem_folder("folder_name")	Specify the destination IMAP folder for unsuccessfully imported email messages. If this Action is not called, the default folder named Problem is used.
im_scan()	Scan the email messages in the Inbox for the specified types. Imports the selected emails and attachments into the batch.
im_logout	Disconnect from the Email server.

Import from an EWS Email server

Table 37. The sequence of actions to use for an EWS Email server

Action	Description
<code>ex_ews_version("1")</code>	Specify the version of the EWS Email server to use: • Type 1 for Exchange 2007 SP1 • Type 2 for Exchange 2010 • If called by any other parameter, the latest known library is used. Currently .NET 3.5 library on Exchange 2010 If the action is not called at all, defaults to the latest version.
<code>ex_login("hostname","userid,password")</code>	Log in to Email server for the specified host name.
<code>ex_types("tif", "pdf")</code>	Specify the email attachment types that you want to import.
<code>ex_done_folder("folder_name")</code>	Specify the destination EWS folder for successfully imported email messages. If this Action is not called, the default folder named Done is used.
<code>ex_problem_folder("folder_name")</code>	Specify the destination EWS folder for unsuccessfully imported email messages. If this Action is not called, the default folder named Problem is used.
<code>ex_scan()</code>	Scan the email messages in the Inbox for the specified types. Imports the selected emails and attachments into the batch.
<code>ex_logout</code>	Disconnect from the Email server.

Fax Connector actions:

You can use Fax Connector actions to create Datacap document batches from incoming faxes. You can also send the contents of a document to a specified fax number.

The Datacap Connector for Fax actions do the following steps to create document batches from information that you receive in faxes.

- Set the name of the Fax Server and the user ID and Password to use to connect to the OpenTextFaxServer
 - Specify the protocol to use to connect to the OpenTextFaxServer
 - Define the amount time before you stop running a batch, polling interval time, and the server authentication method.
 - Connect to the OpenTextFaxServer
 - Configure the maximum number of faxes in a batch and whether to remove processed faxes from the server
 - Import the faxes into the document batch from the OpenTextFaxServer
- “Fax Connector prerequisites” on page 134

“Fax Connector settings”

“Configuring Fax Connector actions” on page 135

“Fax Connector import examples” on page 135

Fax Connector prerequisites:

To configure and run Fax Connector actions, your environment must meet the hardware and software requirements for Datacap, Version 8.0, 8.0.1, and 9.0.

The following components must be installed and running on your system before you can use the Fax Connector actions:

- Datacap Version 8.0, 8.0.1, 9.0, or 9.0 installed and running on either a single computer or a client/server installation

Related information:

 [Hardware and Software Requirements for IBM Datacap Version 9.0](#)

Fax Connector settings:

Record the system settings that you want to use to configure the Fax Connector actions and have these values available during the configuration process.

This table describes the parameters that are required for the Datacap Connector for Fax actions.

Table 38. Required Fax Connector actions parameter settings

Action	Description
Set Server Name	Name of the OpenTextFaxServer
Set User ID	User ID used to log in to the OpenTextFaxServer
Set Password	Password that is used to log in to the OpenTextFaxServer
Set Windows Authentication	Whether to use Windows Authentication to connect to the OpenTextFaxServer
Set Protocol	Protocol that is used to connect to the OpenTextFaxServer
Set Polling Interval	Number of milliseconds to wait before fax polling from the OpenTextFaxServer resumes
Set Abort Timeout	Number of seconds to wait before a batch run is stopped
Set Max Number of Faxes	Maximum number of faxes in each batch
Set Fax Removal After Import	Whether to remove processed faxes from the Fax Server, must be set to true so that new faxes are imported each time. If this action is not called or set to false, the Import Faxes action imports the same faxes over and over.
Import Faxes	Import the faxes from the OpenTextFaxServer into the document batch
Connect	Connect to the OpenTextFaxServer
Send Faxes	Fax the contents of the document or page to the specified Fax number

Table 38. Required Fax Connector actions parameter settings (continued)

Action	Description
Disconnect	Disconnect from the OpenTextFaxServer

Configuring Fax Connector actions:

You must create an Export ruleset and configure its rules and functions with Fax Connector actions to import incoming faxes into a document batch.

To configure Fax Connector actions:

1. Specify the name of the Fax Server from which you want import faxes.
2. Add the OpenTextFaxServer.RRX actions to the Export rulesets.

The following example describes an Export Fax ruleset that selects the OpenTextFaxServer to use. The ruleset logs on to the server, and imports the faxes from the server into the batch.

The ruleset contains the Connect to Fax Server and Import Fax rules. The Connect to Fax rule contains the Server name, Logon, Protocol, and Connect functions and the actions that make the connection to the Fax server. The Import Fax rule contains the Import function with actions that locate the specified attachments and import them into the batch.

Export Fax ruleset

- Connect to Fax Server rule
 - Server name function
 - SetServerName("myserver")
 - Logon function
 - SetUserID("myuserID")
 - SetUserPassword("myPassword")
 - Protocol function
 - SetProtocol("4")
 - Connect function
 - Connect()
- Import Fax
 - Import function
 - ImportFaxes()

Fax Connector import examples:

The Fax Connector Actions connect to the OpenTextFaxServer to import incoming faxes into document batches.

Import faxes from a Fax server

The examples in the following tables show the sequence in which you must add the actions to the Export ruleset.

Table 39. The sequence of actions to import faxes

Action	Description
SetServerName("hostname")	Set the name of the OpenTextFaxServer to which you want to connect.
SetUserID("userid")	Set the user ID to use to log in to the OpenTextFaxServer.
SetUserPassword("Password")	Set the Password for the user ID to use to log in to the OpenTextFaxServer.
SetProtocol("4")	Specify the protocol to use to connect to the OpenTextFaxServer. The default value is "4" for TCP/IP. The other valid values are. • "1" - Named Pipes • "2" - IPXOS2 • "3" - SPX • "5" - IPX • "6" - SecTCPIP • "7" - SecSpx
SetPollingInterval("2000")	Set the amount of time, in milliseconds, to wait before you poll the OpenTextFaxServer for faxes again. The default value is "2000" (2 seconds).
Connect()	Connect to the Fax Server.
SetFaxRemovalAfterImport(True)	Removes faxes after they are imported to enable the new faxes to be imported when they are ready.
ImportFaxes()	Import the faxes from the OpenTextFaxServer and store them in a document in a batch.
Disconnect()	Close the connection to the OpenTextFaxServer.

Connector actions log files:

A log file contains the results of calling the action and explains why a document was not created when you upload documents into a repository. The name of the log file is based on the name of the task, for example `export_rss.log`.

After the documents are uploaded into a repository, the following file for that repository is created in the batch directory. This file contains the names of the files that were uploaded.

Repository	File name
IBM CMIS server	CMIS_Uploaded.xml
IBM Content Manager	IBMCM_Uploaded.xml
IBM FileNet Content Engine	FNP8_Uploaded.xml
SharePoint library	SP_Uploaded.xml
FileNet Image Services	FileNetIDM_Uploaded.xml
Email Input	EWEmail_Uploaded.xml

Repository	File name
Email Send	Email_Uploaded.xml
Open Text Fax Server	OpenTextFaxServer_Uploaded.xml

Viewing action details:

Datacap Studio provides help topics with detailed information for all of the connector actions. The topics include the action library name, description, parameters, DCO level, returns, and examples for each action.

You can refer to these descriptions when you configure the parameters of the Connector actions. These actions are specific to the entity into which you are uploading files. When you configure actions for an application, you must set the actions for the entity in the sequence in which they are presented in the examples.

To view connector action descriptions:

1. In Datacap Studio, click the **New Actions** tab.
2. Select the **Action library**.

Table 40. Action library names by connector

Connector	Action Library
IBM Content Manager Connector	IBMCM
FileNet P8 Connector	FileNetP8
SharePoint Connector	SPExport
FileNet Image Services Connector	FileNetIDM
eMail and eDoc Connector	IMail, EWSMail, EMail
Fax Connector	OpenTextFaxServer

3. Right-click on the action for which you want detailed information and select **Information**.

TravelDocs: Exporting data to a database

You can update the TravelDocs application to export data from each rental agreement page to an export database.

“Configuring the export database”

“Creating the ExportDB ruleset” on page 138

“Adding the **ExportDB** ruleset to the Export task profile” on page 138

“Attaching the Export Rental Agreement Data rule to the rental agreement page” on page 139

“Running a batch through the workflow” on page 139

Configuring the export database

You must use Datacap Application Manager to configure the export database.

To configure the export database:

1. In the **Start** menu click **IBM Datacap Services > Datacap > Datacap Datacap Application Manager**.
2. Select the **TravelDocs** application from the list on the left.
3. Click **Browse [...]** beside the **Export database** field.

4. In the **Database type** field, select **Microsoft Access**. Then, in the **Database** field, select the file C:\Datacap\TravelDocs\TravelDocsExport.mdb. Database authentication is not needed.
5. Click **OK** and then close the Datacap Application Manager window.

Related information:

 Application Manager

Creating the ExportDB ruleset

You use Datacap Studio to create the ExportDB ruleset. Also, you access the Datacap Studio actions library to add functions to the ruleset.

To create the ExportDB ruleset:

1. In the Rulesets pane, right-click the **TravelDocs** node and choose **Add Ruleset**.
2. Rename the new ruleset from Ruleset1 to ExportDB.
3. Rename the default rule from Rule1 to Export Rental Agreement Data.
4. Rename the default function from Function1 to ExportDB.
5. Click the **Actions library** tab and expand the **ExportDB** library.
6. Select and add each of the following actions that are shown in the following table to the **Export Data** function by clicking **Add to function**. Then, set the action parameters as shown in the following table.

Action	Parameter
ExportOpenConnection	@APPVAR(* /exportdb:cs)
SetTableName	Rental_Agreement
ExportBatchIDToColumn	BatchID
ExportFieldToColumn	Pickup_Date,Pickup_Date
ExportFieldToColumn	Pickup_Location,Pickup_Location
ExportFieldToColumn	Return_Date,Return_Date
ExportFieldToColumn	Return_Location,Return_Location
ExportFieldToColumn	Car_Type,Car_Type
ExportFieldToColumn	Options,Options
ExportFieldToColumn	Total_Cost>Total_Cost
AddRecord	
ExportCloseConnection	

7. In the Rulesets pane, click **Save**. Then, click **Lock/Unlock ruleset** and choose **Publish ruleset**.

Adding theExportDB ruleset to the Export task profile

After you create the **ExportDB** ruleset, you must add it to the Export task profile.

To add the **ExportDB** ruleset to the Export task profile:

1. In the Rulesets pane, select the **ExportDB** ruleset.
2. Click the **Task profiles** tab and click **Lock/Unlock task profiles**.
3. Select the **Export** task profile and click the **Add ruleset to profile** button in the Task profiles pane.
4. Expand the **Export** task profile and make sure the **ExportDB** ruleset is listed.
5. In the Task profiles pane, click **Save**. Then, click **Lock/Unlock task profiles**.

Attaching the Export Rental Agreement Data rule to the rental agreement page

You use Datacap Studio to attach the Export Rental Agreement Data rule to the rental agreement page.

To attach the Export Rental Agreement Data rule to the rental agreement page:

1. In the Document hierarchy pane, click **Lock DCO for editing**.
2. Expand the document hierarchy so the **Rental_Agreement** page is visible. Then, select the **Rental_Agreement** page.
3. In the Rulesets pane, select the **Export Rental Agreement Data** rule and click **Add to DCO**.
4. With the Export Rental Agreement Data rule still highlighted, click **Sync DCO view with Ruleset view**. Make sure that the new rule is now included in the **Open** element of the rental agreement page.
5. In the Document hierarchy pane, click **Save**, then click **Unlock DCO**.

Running a batch through the workflow

After you create and configure the ExportDB ruleset and attach the data rule to the rental agreement page. You can run a batch and confirm that the export task is operational.

To run a batch through the workflow:

1. Use the Connection wizard to reopen the TravelDocs application. Opening the Connection wizard, forces Datacap Studio to reload the information from the application configuration (.app) file. If you do not open the wizard, the export database connection string might not be in the cached copy of the .app file. The new ruleset might fail.
2. In Datacap Studio, click the **Test** tab.
3. In the **Rulesets** pane, expand the **ExportDB** ruleset. Then, right-click the **Export Rental Agreement Data** rule and choose **Set breakpoint**. The action stops when Datacap reaches the rule.
4. In the **Workflow** pane, select the **VScan** task profile under **Main Job**.
5. Click **New** to start a new batch.
6. Click **Process rules for target object** and **Advance** to move the batch through the **VScan**, **PageID**, **Profiler**, **Verify**, and **Export** tasks. The action stops at the breakpoint.
7. Click **Step in** to single-step into the function and start running the actions. As each line completes, ensure that there is a check mark beside the action, which indicates that the action returned True.

Tip: If ExportOpenConnection fails (as indicated by a ! beside the action), ensure that you set up the export database correctly. And that you added the connection string to the Datacap Application Manager. Then, use the Connection wizard to reopen the TravelDocs application.

8. Click **Process rules for target object** to resume normal execution. You must click **Process rules for target object** each time you press the Export Rental Agreement Data rule (for each rental agreement page). Then, click **Advance**.
9. Open the file C:\Datacap\TravelDocs\TravelDocsExport.mdb and review the exported data in the Rental_Agreement table.

TravelDocs: Exporting data to an XML file

You can update the TravelDocs application to export data from each rental agreement page to an XML file. If you want to export data from the other pages, you must have a separate rule for each page type.

“Creating the ExportXML ruleset”

“Adding the **Export XML** ruleset to the Export task profile” on page 141

“Attaching the Export XML rules to the document hierarchy” on page 141

“Running a batch through the workflow” on page 142

Creating the ExportXML ruleset

You use Datacap Studio to create the ExportXML ruleset. Also, the ruleset requires three rules.

Three separate rules are required.

- One rule that is attached to the Open element of the batch to set the XML export path and file name.
- One rule that is attached to the rental agreement page that writes the data for the current page.
- One rule that is attached to the Close element of the batch to save the XML file.

To create the ExportXML ruleset:

1. In the Rulesets pane, right-click the **TravelDocs** node and choose **Add Ruleset**.
2. Rename the new ruleset from Ruleset1 to Export XML.
3. Rename the default rule from Rule1 to Open XML File.
4. Rename the default function from Function1 to Open XML.
5. Click the **Actions library** tab and expand the **Export XML** library.
6. Select and add each of the following actions that are shown in the following table to the **Open XML** function by clicking **Add to function**. Then, set the action parameters as shown in the following table.

Action	Parameter
xml_SetExportPath	@APPPATH(export)
xml_SetFileName	@BatchID

Attention: @APPPATH(export) is a smart parameter that gets the export path from the application configuration file. @BatchID is a smart parameter that returns the current batch ID.

7. Right-click the **ExportXML** ruleset and choose **Add Rule**.
8. Rename the new rule from Rule1 to Export Rental Agreement XML.
9. Rename the default function from Function1 to Export XML.
10. Select and add each of the following actions that are shown in the following table to the Export XML function by clicking **Add to function**. Then, set the action parameters as shown in the following table.

Action	Parameter
xml_NewNode	@ID,Rental_Agreements
xml_NewNode	Pickup_Date,@ID
xml_SetNodeValue	Pickup_Date, @P\Pickup_Date

Action	Parameter
xml_NewNode	Pickup_Location,@ID
xml_SetNodeValue	Pickup_Location, @P\Pickup_Location
xml_NewNode	Return_Date,@ID
xml_SetNodeValue	Return_Date, @P\Return_Date
xml_NewNode	Return_Location,@ID
xml_SetNodeValue	Return_Location, @P\Return_Location
xml_NewNode	Car_Type,@ID
xml_SetNodeValue	Car_Type, @P\Car_Type
xml_NewNode	Options,@ID
xml_SetNodeValue	Options, @P\Options
xml_NewNode	Total_Cost,@ID
xml_SetNodeValue	Total_Cost, @P\Total_Cost

Attention: @ID gets the ID of the current object. @P\ gets the value of the specified field on the current page.

- Right-click the ExportXML ruleset and choose **Add Rule**.
- Rename the new rule from Rule1 to Close XML File.
- Rename the default function from Function1 to Close XML.
- Select and add the action that is shown in the following table to the Close XML function by clicking **Add to function**. This action has no parameter.

Action	Parameter
xml_SaveFile	

- In the Rulesets pane, click **Save**. Then, click **Lock/Unlock ruleset** and choose **Publish ruleset**. The finished ruleset looks like the following example.

Adding the Export XML ruleset to the Export task profile

After you create the **Export XML** ruleset, you must add it to the Export task profile.

To add the **Export XML** ruleset to the Export task profile:

- In the Rulesets pane, select the **Export XML** ruleset.
- Click the **Task profiles** tab and click **Lock/Unlock task profiles**.
- Select the **Export** task profile and click **Add ruleset to profile** in the Task profiles pane.
- Expand the **Export** task profile and ensure that the **Export XML** ruleset is displayed.
- In the Task profiles pane, click **Save**. Then, click **Lock/Unlock task profiles**.

Attaching the Export XML rules to the document hierarchy

After you add the ruleset to the task profile, you must attach the rules to the document hierarchy.

To attach the Export XML rules to the document hierarchy:

- In the Document hierarchy pane, click **Lock DCO for editing**.
- Expand the batch and select the **Open** element of the batch.

3. In the Rulesets pane, select the **Open XML File** rule and click **Add to DCO**.
4. Select the **Close** element of the batch.
5. In the Rulesets pane, select the **Close XML File** rule and click **Add to DCO**.
6. In the Document hierarchy pane, expand the **Car_Rental** document node and select the **Rental_Agreement** page.
7. In the Rulesets pane, select the **Export Rental Agreement XML** rule and click **Add to DCO**.
8. Select the **Open XML File** rule, click **Sync DCO view with Ruleset view**. Make sure that the rule is now included in the **Open** element of the batch.
9. Repeat for the **Export Rental Agreement XML** and **Close XML File** rules.
10. In the Document hierarchy pane, click **Save**, then click **Unlock DCO**.

Running a batch through the workflow

After you create and configure the ExportXML ruleset, and attach the required rules to the document hierarchy. You can run a batch and confirm that the export task is operational.

To run a batch through the workflow:

1. Click the Datacap Studio **Test** tab.
2. In the **Breakpoints** pane, click **Remove all breakpoints**.
3. In the **Workflow** pane, select the **VScan** task profile under **Main Job**.
4. Click **New** to start a new batch.
5. Click **Process rules for target object** and **Advance** to move the batch through the entire workflow
6. Open the file C:\Datacap\TravelDocs\export\batch_identifier.xml and review the exported XML data.

```
<?xml version='1.0' ?>
<Rental_Agreements>
<TM000001>
<Pickup_Date>Tues, Dec 7, 2010</Pickup_Date>
<Pickup_Location>Boston (BOS)</Pickup_Location>
<Return_Date>Fri, Dec 10, 2010</Return_Date>
<Return_Location>Boston (BOS)</Return_Location>
<Car_Type>Compact</Car_Type>
<Options>Fuel Service</Options>
<Total_Cost>345.70</Total_Cost>
</TM000001>
<TM000003>
<Pickup_Date>Mon, Dec 6, 2010</Pickup_Date>
<Pickup_Location>San Francisco (SF0)</Pickup_Location>
<Return_Date>Fri, Dec 10, 2010</Return_Date>
<Return_Location>San Francisco (SF0)</Return_Location>
<Car_Type>SUV</Car_Type>
<Options>Child Seat</Options>
<Total_Cost>489.31</Total_Cost>
</TM000003>
<TM000004>
<Pickup_Date>Mon, Dec 13, 2010</Pickup_Date>
<Pickup_Location>Newark (EWR)</Pickup_Location>
<Return_Date>Thur, Dec 16, 2010</Return_Date>
<Return_Location>Newark (EWR)</Return_Location>
<Car_Type>Luxury</Car_Type>
<Options>Navigation System Child Seat Fuel Service</Options>
<Total_Cost>387.40</Total_Cost>
</TM000004>
</Rental_Agreements>
```

Application Debugging

Application debugging requires that you review two runtime log files, which are the Rulerunner Service (RRS) log and the task log. The RRS log provides detailed information about each action and is most helpful to application developers. The task log documents internal calls and is used mostly by IBM software support.

Datacap Studio includes integrated debugging functionality through which you can control the execution environment and monitor your application at runtime.

“Datacap log files”

“Debug your application from the Datacap Studio Test tab” on page 145

Datacap log files

Datacap generates two types of log files during task execution.

- Rulerunner Service (RRS) log files include detailed information about each action as it runs.
- Task log files document mostly internal calls and is most helpful to IBM Software support.

Additionally, Report Viewer and Rulerunner can generate their own log files. Rulerunner logging is discussed in the topic “Rulerunner logging” on page 201.

“Enable logging for Datacap Web Client tasks”

“Rulerunner Service (RRS) log files” on page 144

“Task log files” on page 145

Enable logging for Datacap Web Client tasks

To enable logging for a web task, you must configure that task in the Datacap Web Client.

To enable logging from web client tasks:

1. In the Datacap Web Client, click the **Administrator** tab.
2. On the **Administrator** tab, click **Workflow**.
3. Expand the job that contains the task for which you want to enable logging, and select the task.
4. Click **Setup** in the Selected task details pane.
5. In the **Rulerunner service log** field, enter one of the values, as required.

Tip: RRS logging is only useful for tasks that run rules. If your web client is not associated with a task profile, an RRS log file is not generated.

Rulerunner Manager Service log setting	Result
0 or 1	No RRS log file
2	RRS log file with action logging but no action parameters displayed
3 or 4	RRS log file with action logging and action parameters displayed
5 or higher	RRS log file with action logging and complete DCO navigation

In most situations, a setting of 3 provides enough information to help you debug rule-related issues.

Rulerunner Service (RRS) log files

As Rulerunner runs each action, it writes detailed logging information to a Rulerunner Service (RRS) log file (*task_rrs.log*). Rulerunner also generates an RRS log file whenever you run a task from Datacap Studio.

If you want to generate an RRS log file for tasks that you run from the Datacap Web Client or for Datacap Desktop tasks, complete the following steps.

1. Start Datacap Rulerunner Manager.
2. Click the **Logging** tab.
3. Click the **RRS log** tab and select the logging options that you want.

In the Datacap Web Client, each task generates its own Rulerunner Service log file. The most recent TravelDocs batches folder contains a log file for each of the task profiles in the Main Job workflow.

Each log file contains detailed descriptions of the actions that are run by the task profile and is useful for application troubleshooting.

Example 1

Here is the *vscan_rrs.log* entry that shows execution of the **SetSourceDirectory** action in the VScan rule set:

```
[1] action SetSourceDirectory(bool=false,bool=true,str="@APPPATH(vscanimagedir)")
[2] 1 Smart Parameter element found
[3] Parsing Smart Parameter element {0} value: "@APPPATH(vscanimagedir)"
[4] @APPPATH key root value: 'vscanimagedir'
[5] @APPPATH looking for workflow key: '*/dco_TravelDocs/vscanimagedir'
[6] @APPPATH workflow key found: 'C:\Datacap\TravelDocs\images'
[7] Smart Parameter return value: 'C:\Datacap\TravelDocs\images'
[8] looking for:C:\Datacap\TravelDocs\images
[9] Action changes: Directory with source images: C:\Datacap\TravelDocs\images
[10] result 0[x0] = true
[11] action returned true
[12] execute statement On Action True
[13]   executing code:
[14]     Call OnActionEnd()
[15] /execute statement On Action True
[16] /action
[17] execute statement On Action Start
[18]   executing code:
[19]     Call OnActionStart()
[20] /execute statement On Action Start
```

By looking through the Rulerunner Service log file, you can see precisely how Rulerunner interprets and runs each action. In the **SetSourceDirectory** Example 1, Rulerunner:

- Identifies the @APPPATH(vscanimagedir) parameter as a smart parameter [line 2]
- Identifies the key value as **vscanimagedir** [line 4]
- Looks up the specified key value in the application configuration [line 5]
- Retrieves the value C:\Datacap\TravelDocs\images [line 6]
- Sets the image source directory to the specified location [line 9]

Example 2

In the previous example, the action that is executed successfully and returned true [line 11]. In this next example, you will introduce an invalid key name in the action parameter:

```
SetSourceDir("@APPPATH(imagedir)") <--"imageddir" is not a valid key
```

This time the batch aborts. If you run the task from the Datacap Studio, you see an error message.

In this case, you can look at the end of the log file to determine the cause of the error [line 6].

```
[1] action SetSourceDirectory (bool=false,bool=false,str="@APPPATH(imagedir)")
[2] 1 Smart Parameter element found
[3] Parsing Smart Parameter element {0} value: "@APPPATH(imagedir)"
[4] @APPPATH key root value: 'imagedir'
[5] @APPPATH looking for workflow key: '*/dco_TravelDocs/imagedir'
[6] @APPPATH workflow key not found. <--Key not found
[7] @APPPATH looking for appname key: '*/dco_TravelDocs/imagedir'
[8] @APPPATH appname key not found.
[9] @APPPATH looking for general key: '*/imagedir'
[10] @APPPATH general key not found.
[11] Smart Parameter return value: ''
[12] looking for:@APPPATH(imagedir)
[13] Error: Folder '@APPPATH(imagedir)' does not exist <--Folder does not exist
[14] Error (385875969=hex:17000001). In CIMainAlgorithm::execute4DC0: Aborting: Action [SetSourceDirectory]
    requested abort [api source:]
[15] EXCEPTION: code="385875969" msg="Aborting: Action [SetSourceDirectory]
    requested abort" loc="CIMainAlgorithm::execute4DC0" API=""
[16] execute statement On Process End
[17] executing code:
[18] Quit()
```

Task log files

Logging is enabled by default for all tasks except VScan and tasks that you start from Datacap Studio.

If you want to generate a task log for Datacap Web Client tasks, you must set the severity level to 1 or higher (on the scale of 0 to 9). The severity level and the options in the Datacap Web Client Task setup window determine how much information is written to the log file. (See “Enable logging for Datacap Web Client tasks” on page 143).

The task log file is saved in the batch folder.

The task log provides information that is typically most helpful to IBM support because it documents mostly internal calls.

Debug your application from the Datacap Studio Test tab

You can run an application from Datacap Studio to monitor the application during execution to determine whether the rules are running as you expect. The Datacap Studio Test tab includes debugging features.

“Using breakpoints”

“Single-stepping through your code” on page 147

“Examining log files from the Test tab” on page 148

Using breakpoints

A breakpoint stops a task at a predetermined rule set, rule, or action, or stops the task when a task starts processing a specific document, page, or field.

“Breakpoint types” on page 146

“Setting breakpoints” on page 146

“Disable and clear breakpoints”

“Set generic breakpoints” on page 147

Breakpoint types:

There are two types of breakpoints, both of which halt execution when Rulerunner encounters the specified element.

- Breakpoints: Halts execution when the Rulerunner execution manager encounters the specified element, regardless of context.
- Full Breakpoints: Halts execution when the Rulerunner execution manager encounters the specified element within the same context.

To understand the difference between a breakpoint and a full breakpoint, consider that the TravelDocs application includes two calls to `ExportCloseConnection()` (one call in the Export Rental Agreement Data rule and one call in the Export Other Close Database rule).

- If you set a breakpoint on the first instance of `ExportCloseConnection`, execution stops whenever `ExportCloseConnection` is called from anywhere in the application.
- If you set a full breakpoint on the first instance of `ExportCloseConnection`, execution stops only when `ExportCloseConnection` is called from the Export Data function in the Export Rental Agreement Data rule in the ExportDB rule set.

You can see the difference in the way the breakpoints are defined by selecting the Breakpoints tab, which displays all of the defined breakpoints.

Setting breakpoints:

You can set breakpoints for a rule set, rule, function, action, document, page, or field.

- For rule sets and rules, you can set only breakpoints, not full breakpoints.
- The document, page, or field must exist in the runtime hierarchy before you can set a breakpoint on it.

To set breakpoints:

1. Set a breakpoint on a rule set, rule, function, or action:
 - a. Go to the Rulesets pane on the Datacap Studio **Test** tab.
 - b. Right-click the item and select **Set breakpoint** or **Set full breakpoint**.
2. Set a breakpoint on a document, page, or field:
 - a. Go to the **Runtime batch hierarchy** pane on the Datacap Studio **Test** tab.
 - b. Right-click the item and select **Set breakpoint** or **Set full breakpoint**.

Disable and clear breakpoints:

You can enable or disable individual breakpoints by selecting or clearing check boxes. The buttons on the left of the Breakpoints pane are options to enable, disable, or remove breakpoints.

The Breakpoints pane displays all of the defined breakpoints.

The check box to the left of each breakpoint indicates whether the breakpoint is enabled or disabled. By default, breakpoints are enabled when you add them.

Table 41. Breakpoints pane checkboxes

Button	Description
Enable all breakpoints button	Enables all the breakpoints that are displayed in the Breakpoints pane.
Disable all breakpoints button	Disables all the breakpoints that are displayed in the Breakpoints pane.
Remove selected breakpoints button	Removes the highlighted breakpoints. To select multiple breakpoints, hold down the Ctrl key.
Remove all breakpoints button	Removes all breakpoints from the Breakpoints pane.

Set generic breakpoints:

The Datacap Studio **Test** tab includes two controls that you can select to halt processing when any rule or action fails

Control	Command	Description
A!	Stop on a failed action	Click this button to add a generic breakpoint that halts processing whenever an action fails.
R!	Stop on a failed rule	Click this button to add a generic breakpoint that halts processing whenever a rule fails.

Single-stepping through your code

Single-stepping is useful to determine whether the functions and actions within a rule are operating as intended. As you step through each line, you can see the actions that are returned as True (check mark) and False. (exclamation point).

If an action returns false, you can look in the batch log to see why the action returned false. (See “Examining log files from the Test tab” on page 148.)

Remember:

- If all actions within a function return True, Datacap skips any remaining functions in the current rule.
- If an action returns False, Datacap skips any remaining actions within that function and starts the next function (if there is one).

The **Test** tab provides UI controls (enabled with tooltips) for stepping through code:

UI control	Description
Step in	Steps into the next line of code. If the next line calls a rule or function, Step in opens the rule or function and halts inside it. If the next line is an action, Step in opens the action and you must click it again to close the action. For example, if a process is halted at a function, use this control to step into a function and start each of its actions.

UI control	Description
Step/Step over	Starts the next line of code and any lower-level functions and actions, and then stops. If the next line is an action, Step over works like Step in and opens the action. For example, if a process is halted at a function, use this control to start the function, including all of its actions.
Step out	Steps through the next line of code. If the next line is a rule or function, Step out works like Step over and starts any lower-level functions and actions. If the next line is an action, Step out starts and closes the action.

Examining log files from the Test tab

You can access the Output tab in Datacap Studio to view the output that is written to different log files.

1. Click the **Output** tab in the center pane of the **Test** tab.
2. If **Batch log** is not already selected, click the down-arrow and select it from the list of available logs.

The Output pane refreshes automatically whenever you stop at a breakpoint or single-step through a line of code, or when the current task profile completes, although you need to scroll to the bottom each time to see the latest messages. For example, if you select a **Stop on a failed action** (`a[@res="false"]`) you can see the action that returned False.

Handling line item grids

The techniques that you implemented rely upon data at predictable locations on the page. When you receive an invoice, you do not know how many items the invoice might contain. There might be just one item, or there might be a hundred items, possibly spanning multiple pages. Datacap includes actions to handle line items grids. You define the region on the page that might contain line items and define the structure of one line item. Datacap can then scan the region and locate all of the individual line items.

Later, you can update the TravelDocs application to demonstrate various techniques that are related to line item grids, including recognition, validation, verification, and export.

- “Defining the document hierarchy for line item grids” on page 149
- “Rules to recognize line items” on page 149
- “Text matching to locate fields” on page 150
- “Removing non-line items from the page data file” on page 151
- “Exporting data from a line item grid” on page 152
- “TravelDocs: Adding new pages that contain line item grids” on page 152
- “TravelDocs: Recognizing line item grid data” on page 156
- “TravelDocs: Validating line item grid data” on page 159
- “TravelDocs: Verifying the line item grid pages” on page 162
- “TravelDocs: Exporting line item grid data to a database” on page 163

Defining the document hierarchy for line item grids

A line item grid is a structure of repeating items, each of which typically contains several fields. To set up the document hierarchy, you must define the region of the page that can include line items. In addition, you must define the structure of one line item in terms of its fields.

You can define only one line item in the document hierarchy. At run time, Datacap expands the runtime hierarchy as needed to accommodate however many line items it finds.

```
<P id="TM000001">          <--Page
<F id="Grid_region">      <--Grid region
  <F id="Line_item0">      <--First line item
    <F id="Item"> etc. </F>      <--Item field
    <F id="Description"> etc. </F>      <--Description field
    <F id="Cost"> etc. </F>      <--Cost field
  </F>
  <F id="Line_item1">      <--Second line item
    <F id="Item"> etc. </F>      <--Item field
    <F id="Description"> etc. </F>      <--Description field
    <F id="Cost"> etc. </F>      <--Cost field
  </F>
  <F id="Line_item2">      <--Third line item
    <F id="Item"> etc. </F>      <--Item field
    <F id="Description"> etc. </F>      <--Description field
    <F id="Cost"> etc. </F>      <--Cost field
  </F>
  etc.
```

Rules to recognize line items

Datacap includes actions that simplify the processing of an undefined number of line items.

Table 42. Actions that simplify line item processing

Library	Action	Description
Zones	ScanDetails	Searches a line item grid object and looks for line items. Assign this action to the grid region in the document hierarchy.
Zones	ScanLineItem	Searches a line item object and looks for fields. Assign this action to each line item in the document hierarchy.
Zones	PopulateZNLineItemField	Populates the page data file with the recognized value in the zone for the current line item child field. Assign this action to each line item child field in the document hierarchy.

These three actions automate the process of reading line item grids. When you are assigning rules, the key is to make sure that the actions operate at the correct level in the document hierarchy.

Datacap runs each rule, first on the grid and then on each line item and field in the grid. As Datacap runs, it builds a data structure in memory. The data gets written to the page data file upon completion of the current task.

Table 43. Resulting data structures for selected actions

ScanDetails()	ScanLineItem()	PopulateZNLineItemField()
Runs once only (at the grid level)	Runs once for each line item	Runs once for each field
Resulting data structure (in memory):	Resulting data structure (in memory):	Resulting data structure (in memory):
Grid_region	Grid_region	Grid_region
• Line_item0	• Line_item0	• Line_item0
• Line_item1	– Item	– Item = 1176
	– Description	– Description = Widget
	– Cost	– Cost = \$6.95
	• Line_item1	• Line_item1
	– Item	– Item = 9122
	– Description	– Description = Widget
	– Cost	– Cost = \$8.25

In this way the three actions can read all the items in a grid of arbitrary length.

Text matching to locate fields

Frequently, a line item grid includes a total at the bottom. There might be other fields, like sales tax and shipping costs. Because you do not know in advance how many line items are in the grid, you cannot know where the other fields are located. Because the location of the other fields is unpredictable, you cannot use positional information to read these fields. Instead, you can use text matching to locate an adjacent label and then read the text beside the label.

Datacap provides many actions for locating text on a page. You can consider these actions in more detail in the topics about text matching. Typically, when you work with line item grids, you are searching for the last instance of a word or phrase like *Total* or *Sales Tax*. The **Locate** actions library has several useful actions, including the actions in the following table.

Library	Action	Description
Locate	FindLastRegEx	Locates the last occurrence of a word or phrase on the current page.
Locate	FindLastKeyList	Locates the last occurrence of a word or phrase that is contained in the specified keyword file. A keyword file is a text file with a .key extension that contains a list of similar words and phrases; for example, <i>Sales tax</i> and <i>Tax</i> .
Locate	GoRightWord	Moves <i>n</i> words to the right of the location of a previously found word.

Library	Action	Description
Locate	UpdateField	Updates the current field in the page data file with the value of the located word.

For detailed information on these and other actions in the Locate library, select the action on the **Actions Library** tab and click **Display information**.

The The Recognize Grid Total Field rule in the TravelDocs application locates the last instance of the word *Total* on the current page. The rule then moves one word to the right and confirms that the value is a currency value. Finally, the rule updates the current field in the page data file. The rule must be attached to the Total field in the document hierarchy.

Removing non-line items from the page data file

The region that is defined for the line item grid might include fields that are not line items. For example, the Total field that you used for text matching is not a line item. Therefore, Datacap might create line items for items that are not line items.

In the following invoice example, although there are only two line items on the invoice, Datacap created a third line item for the Total field.

Item	Descript	Cost
1176	Widget	\$6.95
9122	Widget	\$8.25
	Total	\$15.20

The following is the XML code for the invoice example.

```
<P id="TM000001">
<F id="Grid_region">
  <F id="Line_item0">    <!--First line item
    <F id="Item"> etc.</F>    <!--Has the value "1176"
    <F id="Description"> etc.</F>    <!--Has the value "Widget"
    <F id="Cost"> etc.</F>    <!--Has the value "6.95"
  </F>
  <F id="Line_item1">    <!--Second line item
    <F id="Item"> etc.</F>    <!--Has the value "9122"
    <F id="Description"> etc.</F>    <!--Has the value "Widget"
    <F id="Cost"> etc.</F>    <!--Has the value "8.25"
  </F>
  <F id="Line_item2">    <!--Third line item (not a line item)
    <F id="Item"> etc.</F>    <!--No data
    <F id="Description"> etc.</F>    <!--Has the value "Total"
    <F id="Cost"> etc.</F>    <!--Has the value "15.20"
  </F>
  etc.
```

1. If the page you are processing has non-line item fields within the grid region, you might need to identify those non-line items and remove them. Datacap includes an action to identify and remove these items for you.

Library	Action	Description
Validations	CheckSubFields	Confirms that values exist in child fields of specified parent field. Deletes the parent field if any of the specified child fields have no values.

2. You must attach the rule to the line item grid object in the document hierarchy and ensure that the rule runs after recognition is complete. Use one of the following methods to attach the rule:
 - Include the rule in the Recognize ruleset and attach it to the Close element of the line item grid object.
 - Include the rule within a separate task profile that runs after recognition but before validation; for example, the Clean ruleset. Attach the rule to the Open element of the line item grid object.

When you use either method, Datacap recognizes that the Item field in the third line item has no value. Therefore, Datacap deletes the field, leaving only the two real line items.

Exporting data from a line item grid

You can export line item grid data by using the export actions. However, since the data exists at different levels in the runtime hierarchy, you need different rules for each level.

For example, to export the data from an invoice, you typically need:

- A rule to set up the export file or open the database
- A rule to export the header information
- A rule to export each of the line items
- A rule to export any trailing items (for example, the invoice total field)
- A rule to close the file or database

An example of exporting data to a database is provided in the topic “Exporting to a database” on page 163.

For more information, see “Smart parameters” on page 166.

TravelDocs: Adding new pages that contain line item grids

In this example, you update the TravelDocs application to process the Meals and Other Charges pages of the Hotel document type. Both of these pages include line item grids of undefined length.

“Updating the document hierarchy”

“Attaching the existing page rules to the new pages” on page 155

“Creating the page fingerprints” on page 155

“Defining the recognition zones” on page 155

Updating the document hierarchy

When you set up the document hierarchy earlier, you skipped the Meals and Other_Charges pages. The business requirements specify the rules for the structure of the new page types.

The following rules are specified for the structure of the new page types:

	Number	Required?	Order
Hotel			
Meals	Any number per document	No	Any position in the document
Other_Charges	Any number per document	No	Any position in the document

Within the document hierarchy, the following variables define the structure of the pages within the document.

	Maximum	Minimum	Order
Hotel			
Meals	0	0	0
Other_Charges	0	0	0

“Adding pages to the document hierarchy”

“Creating data fields”

Adding pages to the document hierarchy:

You need to add new pages that contain the line item grids to the DCO.

To add pages to the document hierarchy:

1. In the Document Hierarchy pane, click **Lock DCO for editing** to lock the document hierarchy for editing.
2. Expand the tree so you can see the document types.
3. Right-click the **Hotel** document node and choose **Add multiple > Pages**. Then, type 2 in the box and press **Enter**.
4. Rename the new pages from Page 1 and Page 2 to Meals and Other_Charges.
5. Click **Save**.
6. Right-click the **Meals** page node and choose **Manage variables**.
7. Make sure the **Max**, **Min**, and **Order** values are as specified in the preceding table (the Meals page is 0, 0, 0). Then, click **Done**.
8. Right-click on the **Other_Charges** page node and choose **Manage variables**. Enter the **Max**, **Min**, and **Order** values as specified in the preceding table (the Other_Charges page is also 0, 0, 0) and click **Done**.
9. Click **Save**.

Creating data fields:

The business requirements specification defines these fields for each new page type.

Meals	Other Charges
Meals_Grid	Other_Charges_Grid
• Meals_Line_Item	• Other_Charges_Line_Item
– Date	– Date
– Description	– Category
– Cost	– Quantity
Meals_Total	– Unit_Cost
	– Total
	Other_Charges_Total

1. Confirm that the document hierarchy is still locked for editing.
2. Right-click the **Meals** page and choose **Add multiple > Fields**. Then, type 2 in the box and press **Enter** on your keyboard.
3. Rename the new fields Meals_Grid and Meals_Total.
4. Right-click the **Meals_Grid** field and choose **Add > Field**.
5. Rename the new field Meals_Line_Item.
6. Right-click the **Meals_Line_Item** field and choose Add multiple > Fields. Then, type 3 in the box and press **Enter**.
7. Rename the new fields Date, Description, and Cost.
8. Repeat to add the fields and subfields for the **Other_Charges** page, as shown in the table. When you add the **Date** field, click **Yes** to inherit all rules and properties.
9. Click **Save**. The following example shows the document hierarchy for the new Hotel pages.

Meals

Open

- Meals_Grid
- Open
 - Meals_Line_Item
 - Open
 - Date
 - Description
 - Cost
 - Close
- Close
- Meals_Total

Close

Other_Charges

Open

- Other_Charges_Grid
- Open
 - Other_Charges_Line_Item
 - Open
 - Date
 - Category
 - Quantity

- Unit_Cost
- Total
- Close
- Close
- Other_Charges_Total
- Close

Attaching the existing page rules to the new pages

You need to add the same rules you used on the other pages in the TravelDocs application to the new pages.

To attach the existing page rules to the new pages:

1. Make sure that the document hierarchy is still locked for editing.
2. Expand the document hierarchy so the Meals page and Other_Charges page are visible and then select the Meals page.
3. In the Rulesets pane, expand the **CreateDocs** ruleset, select the **Create Fields** rule, and click **Add to DCO**. Datacap adds the rule to the Open element of the Meals page.
4. Select the Other_Charges page and click **Add to DCO**. Datacap adds the rule to the Open element of the Other_Charges page.
5. Repeat to add the following rules to each page: Recognize: Recognize Page, Validate: Validate Page, Routing: Routing Rule 1, and Export: Export Page. Each page now has an Open element.
6. Click **Save** and then click **Unlock DCO**.

Creating the page fingerprints

The next step is to add fingerprints for the new page types to the fingerprint library.

To create the page fingerprints:

1. Click the Datacap Studio **Zones** tab.
2. In the Fingerprints pane, right-click the **Hotel** class and choose **Add fingerprint**.
3. Browse to the folder where the TravelDocs fingerprint images are located.
4. Select **Hotel4.tif**, and click **Open**. When asked if you want to enhance the image, click **Yes**.
5. In the Image Processing window, click **Run image processing** to apply the application's image-processing properties. Make sure that the lines disappear from the processed page.
6. Click **Save**, choose **Save image**, and click **OK**. Then, click x to close the Image Processing window.
7. With the new fingerprint selected, click the **Type** field and choose **Meals**.
8. Repeat to add **Hotel5.tif** and enhance the page image in the same way, but set the type to **Other_Charges**.

Defining the recognition zones

Next, you need to define the recognition zones for each of the new page types.

1. Define the zones on the Meals page.
 - a. In the **Fingerprints** pane, select the **Meals** page.
 - b. In the **Image View** pane, click **Zoom** to enlarge the page so you can see the fields clearly.

- c. In the **Document hierarchy** pane, click **Lock DCO for editing**.
- d. In the **Document hierarchy** pane, expand the **Meals** page so you can see all of the fields and subfields.
- e. Select the **Meals_Total** field and draw a bounding box around the total cost beneath the line item grid.

Tip: If you create the **Meals_Details** zone first, you cannot draw the **Meals_Total** zone inside it. Therefore, you are drawing the **Meals_Total** zone first.

- f. Select the **Meals_Grid** field. Then, draw a bounding box around the grid items. Because you do not know how many line items there might be on the actual page images, extend the bounding box to the bottom of the page.
 - g. Select the **Meals_Line_Item** field and draw a bounding box around the first line item.
 - h. Select the **Meals_Line_Item >Date** field and draw a box around the date in the first line item.
 - i. Repeat to create zones for the **Description** and **Cost** fields.
 - j. In the **Document hierarchy** pane, click **Save**.
2. Define the zones on the **Other_Charges** page.
 - a. In the **Fingerprints** pane, select the **Other_Charges** page.
 - b. Make sure that the document hierarchy is still locked for editing. Then, expand the **Other_Charges** page so that you can see all of the fields and subfields.
 - c. Select the **Other_Charges_Total** field and draw a bounding box around the total cost beneath the line item grid.
 - d. Select the **Other_Charges_Grid** field and draw a bounding box around the grid items. Extend the bounding box to the bottom of the page as you did for the **Meals** page.
 - e. Select the **Other_Charges_Line_Item** field and draw a bounding box around the first line item.
 - f. Select each of the subfields in turn and draw a bounding box around each field in the first line item.
 - g. In the **Document hierarchy** pane, click **Save** and then click **Unlock DCO**.

TravelDocs: Recognizing line item grid data

Recognizing the data with a line item grid requires these rules. Each rule is attached to a different object in the document hierarchy.

- A rule that is attached to the line item grid object to scan all line items
- A rule that is attached to the line item object to scan each line item
- A rule that is attached to each field within the line item to read each field
- A rule that is attached to the grid total field to locate and read the total cost

“Creating the recognition rules for the line items”

“Creating the recognition rule for the grid total” on page 157

“Attaching the rules to the document hierarchy” on page 157

“Running a batch through the workflow” on page 158

“Creating rules to remove the non-line items” on page 158

Creating the recognition rules for the line items

You need to create new recognition rules for the line items in the grid by using Rulemanager.

To create the recognition rules for the line items:

1. Click the Datacap Studio **Rulemanager** tab.
2. In the Rulesets pane, select the **Recognize** ruleset and click **Lock/Unlock ruleset**.
3. Right-click the **Recognize** ruleset and choose **Add Rule**. Rename the new rule **Recognize Line Item Grid**.
4. Right-click the **Recognize** ruleset and choose **Add Rule**. Rename the new rule **Recognize Line Item**.
5. Right-click the **Recognize** ruleset and choose **Add Rule**. Rename the new rule **Recognize Line Item Field**.
6. Under the **Recognize Line Item Grid** rule, select **Function 1**. Then, click the **Actions library** tab, expand the **Zones** library, select the **ScanDetails** action, and click **Add to function**.
7. Under the **Recognize Line Item** rule, select **Function 1**. Then, select the **Scan LineItem** action and click **Add to function**.
8. Under the **Recognize Line Item Field** rule, select **Function 1**. Then, select the **PopulateZNLItemField** action and click the **Add to function**.
9. Click **Save**.

Creating the recognition rule for the grid total

You need to create a recognition rule for the total amount that is listed as the sum of all the line item totals. You can then compare this total with the sum you get for the line item totals.

To create the recognition rule for the grid total:

1. Right-click the **Recognize** ruleset and choose **Add Rule**. Rename the new rule **Recognize Grid Total Field**.
2. Under the **Recognize Line Item Grid** rule, select **Function 1**.
3. On the **Actions library** tab, expand the **Locate** library, select the **FindLastRegEx** action, and click **Add to function**.
4. In the **Locate** library, select the **GoRightWord** action and click **Add to function**.
5. In the **Locate** library, select the **UpdateField** action and click **Add to function**.
6. In the Rulesets pane, select the **FindLastRegEx** action. Then, in the Properties pane, set **strParam** to **Total**.
7. In the Rulesets pane, select the **GoRightWord** action. Then, in the Properties pane, set **strParam** to **1**.
8. Click **Save** and then click **Lock/Unlock ruleset** and choose **Publish ruleset**.

Attaching the rules to the document hierarchy

After you create the rules, you will need to add them to the appropriate fields in the document hierarchy.

To attach the rules to the document hierarchy:

- Attach the **Recognize Line Item Grid** rule to the **Meals_Grid** and **Other_Charges_Grid** fields.
- Attach the **Recognize Line Item** rule to the **Meals_Line_Item** and **Other_Charges_Line_Item** field.
- Attach the **Recognize Line Item Field** rule to each field.
- Attach the **Recognize Grid Total Field** rule to the **Meals_Total** and **Other_Charges_Total** fields.

To attach the rules to the document hierarchy:

1. In the Document hierarchy pane, click **Lock DCO for editing**.
2. Expand the document hierarchy so you can see all of the fields and subfields on the Meals and Other_Charges pages.
3. Select the **Meals_Grid** field. Then, in the Rulesets pane, select the **Recognize Line Item Grid** rule and click **Add to DCO**. Datacap adds the rule to the Open element of the field.
4. Select the **Other_Charges_Grid** field and click **Add to DCO**.
5. Select the **Meals_Line_Item** field. Then, in the Rulesets pane, select the **Recognize Line Item** rule and click **Add to DCO**.
6. Select the **Other_Charges_Line_Item** field and click **Add to DCO**.
7. Select the **Date** field under the Meals_Line_Item. Then, in the Rulesets pane, select the **Recognize Line Item Field** rule and click **Add to DCO**.
8. Repeat to attach the **Recognize Line Item Field** rule to the **Description** and **Cost** fields under the Meals_Line_Item and the **Category**, **Quantity**, **Unit_Cost**, and **Total** fields under the Other_Charges_Line_Item.
9. Select the **Meals_Total** field. Then, in the Rulesets pane, select the **Recognize Grid Total Field** rule and click **Add to DCO**.
10. Select the **Other_Charges_Total** field and click **Add to DCO**.
11. In the Document hierarchy pane, click **Save**, then click **Unlock DCO**.

Running a batch through the workflow

You can run a batch through the workflow to test the document hierarchy and the recognition rules you created.

1. Click the Datacap Studio **Test** tab.
2. In the **Workflow** pane, select the **VScan** task profile under **Main Job**.
3. Click **New** to start a new batch.
4. Click **Process rules for target object** and **Advance** to move the batch through to the **Verify** task (do not run the **Verify** task).
5. In the **Runtime batch hierarchy** pane, expand the last hotel document and confirm that pages TM000012 and TM000013 were identified correctly.
6. Expand the **Meals** page and then expand each of the line items to confirm that each line item contains data.

When you expand the last line item, you see a value for the Cost field only. This field is the Total field at the bottom of the grid. Because this field is not a real line item, you must create a rule to eliminate non-line items. For more information, see “Creating rules to remove the non-line items.”
7. Expand the **Other_Charges** page and then expand each line item to confirm that each line item contains data. When you expand the last line item, you see a value for the Total field only. This field is the Total field at the bottom of the grid that must also be eliminated.
8. In the **Workflow** pane, right-click the batch and choose **Cancel**.

Creating rules to remove the non-line items

You need to create rules that remove non-line item values such as dates and descriptions.

To create rules to remove the non-line items:

1. Click the Datacap Studio **Rulemanager** tab.
2. In the Rulesets pane, select the **Clean** ruleset and click **Lock/Unlock ruleset**.

3. Right-click the **Clean** ruleset and choose **Add Rule**. Rename the new rule **Remove Non Line Items (Meals)**.
4. Under the **Remove Non Line Items (Meals)** rule, select **Function 1**.
5. On the **Actions library** tab, expand the **Validations** library. Select the **CheckSubFields** action and then click **Add to function**.
6. In the Rulesets pane, select the **CheckSubFields** action. Then, in the Properties pane, set **strParam** as follows:
'Date' AND 'Description' AND 'Cost'
7. Right-click the **Clean** ruleset and choose **Add Rule**. Rename the new rule **Remove Non Line Items (Other Charges)**.
8. Under the **Remove Non Line Items (Other Charges)** rule, select **Function 1**.
9. Select the **CheckSubFields** action, and click **Add to function**.
10. In the Rulesets pane, select the new CheckSubFields action and set **strParam** as follows:
'Date' AND 'Category' AND 'Quantity' AND 'Unit_Cost' AND 'Total'
11. Click **Save**. Then, click **Lock/Unlock ruleset** and choose **Publish ruleset**.
12. In the Document hierarchy pane, click **Lock DCO for editing**.
13. Expand the **Meals** and **Other_Charges** pages.
14. Select the **Meals_Grid** field. Then, in the Rulesets pane, select the **Remove Non Line Items (Meals)** rule and click **Add to DCO**. Datacap adds the rule to the Open element of the field.
15. Select the **Other_Charges_Grid** field. Then, in the Rulesets pane, select the **Remove Non Line Items (Other Charges)** rule and click **Add to DCO**.
16. Click **Save** and then click **Unlock DCO**.
17. Run another batch through the workflow as described on the previous page and make sure that the non-line items are no longer included. Cancel the batch when you are done.

TravelDocs: Validating line item grid data

In the TravelDocs application, you add validations to the Other Charges page to confirm that the calculations on them are correct.

The grid on the Other Charges page includes calculated fields.

- The first **Total** field represents the Quantity that is multiplied by the Unit Cost.
- The second **Total** field represents the sum of all the line item totals.

“Validating the line item totals”

“Validating the grid total” on page 160

“Running a batch through the workflow” on page 161

Validating the line item totals

Due to the way Datacap processes line item grids, you cannot attach calculations to line item objects. Instead, what you can do is create a field for calculation purposes.

You used the CalculateFields action earlier when you checked that the total cost of the air ticket equals the airfare plus taxes.

Library	Action	Description
---------	--------	-------------

Validations	CalculateFields	Returns True if the arithmetic expression is valid; returns False otherwise.
-------------	-----------------	--

Previously, you used the action to complete the calculation at the page level because the page was the next level up in the document hierarchy. In this case, the next level up in the hierarchy is the line item.

Because of the manner in which Datacap processes line item grids, you cannot attach calculations to line item objects. Instead, you can create a field for calculation purposes, for example, Validation, within the line item and attach the rule to this field.

“Creating the validation rule”

“Attaching the validation rule to the document hierarchy”

Creating the validation rule:

You need to create a rule to check that the calculations on the Other charges page are correct.

1. On the Datacap Studio **Rulemanager** tab, in the **Rulesets** pane, select the **Validate** ruleset and click **Lock/Unlock ruleset**.
2. Right-click the **Validate** ruleset and choose **Add Rule**. Rename the new rule **Validate Other Charge**.
3. Under the **Validate Other Charge** rule, select **Function 1**.
4. On the **Actions library** tab, expand the **Validations** library. Select the **CalculateFields** action and click **Add to function**.
5. In the **Rulesets** pane, select the **CalculateFields** action. Then, in the **Properties** pane, set **strParam** as follows:
'Quantity' * 'Unit_Cost' = 'Total'
6. Click **Save**. Then, click **Lock/Unlock ruleset** and choose **Publish ruleset**.

Attaching the validation rule to the document hierarchy:

You need to create the field that you need to do validation and attach the new rule to it. In the document hierarchy, this field is a subfield of the Other_Charges_Line_Item field.

1. In the **Document hierarchy** pane, click **Lock DCO for editing**.
2. Expand the **Other_Charges** page completely, so you can see all the individual fields.
3. Right-click the **Other_Charges_Line_Item** field and choose **Add > Field**. Then, rename the new field **Validation**.
4. Select the **Validation** field. Then, in the **Rulesets** pane, select the **Validate Other Charge** rule and click **Add to DCO**. Datacap adds the rule to the field's “Open” element.
5. Click **Save**.

Validating the grid total

This validation ensures that the grid total (**Other_Charges_Total**) equals the sum of the line item totals (**Total**)

There are different methods to do this calculation. You can attach the following rule to the page's **Close** element:

The validation action looks a little unusual:

```
CalculateFields("'Total' = 'Other_Charges_Total'")
```

The reference to **Total** sums all of the child fields that are labeled **Total**. The action then compares the sum to the **Other_Charges_Total** field.

“Creating the validation rule”

“Attaching the rule to the document hierarchy”

Creating the validation rule:

You can create a validation rule to ensure that the grid total (**Other_Charges_Total**) equals the sum of the line item totals (**Total**).

1. On the Datacap Studio Rulemanager tab, in the Rulesets pane, select the **Validate** ruleset and click **Lock/Unlock ruleset**.
2. Right-click the **Validate** ruleset and choose **Add Rule**. Rename the new rule **Validate Other Charges Total**.
3. Under the **Validate Other Charges Total** rule, select **Function 1**.
4. On the Actions library tab, expand the Validations library. Select the **CalculateFields** action and click **Add to function**.
5. In the Rulesets pane, select the **CalculateFields** action. Then, in the Properties pane, set **strParam** as follows:

```
'Total' = 'Other_Charges_Total'
```
6. Click **Save**, then click **Lock/Unlock ruleset** and choose **Publish ruleset**. Here is an example of the **Validate Other Charges** rule:
 - **Validate Other Charges Total**
 - **Function1**
 - **CalculateFields**("'Total' = 'Other_Charges_Total'")

Attaching the rule to the document hierarchy:

You must attach the new validation rule to the **Other_Charges** page's **Close** element.

1. Make sure that the document hierarchy is still locked for editing.
2. If necessary, expand the **Other_Charges** page so you can see the page's **Close** element.
3. Select the **Close** element. Then, in the Rulesets pane, select the **Validate Other Charges Total** rule and click **Add to DCO**. Datacap adds the rule to the **Close** element.
4. Click **Save** and then click **Unlock DCO**.

Running a batch through the workflow

The **Other Charges** page includes calculation errors to trigger validation failures. When you run the batch through the workflow, you can see the validation failures.

1. Click the Datacap Studio **Test** tab.
2. In the **Workflow** pane, select the **VScan** task profile under **Main Job**.
3. Click **New** to start a new batch.
4. Click **Process rules for target object** and **Advance** to move the batch through to the **Verify** task (do not run the **Verify** task).
5. Open the application's most recent batch folder `C:\Datacap\TravelDocs\batches\batch_id`. Then, open the file `Ruleruner.xml` and scroll down to the data for page **TM0000013**.


```

<P id="TM000013">
<V n="TYPE">Other_Charges</V>
<V n="STATUS">1</V>      <-- Page status is '1'
etc.
<V n="MESSAGE">Failed Calculation:FormatNumber(138.75 ,8,0,0) <--Calc failure
FormatNumber( 238.75,8,0,0)</V>
<V n="DATAFILE">tm000013.xml</V>
</P>

```

The page status is '1' (indicating a problem) and the message indicates that the calculation for the Total_Charges field failed.

6. Open the file tm000013.xml. Notice that there is a calculation failure in the first line item and that Datacap flags all the fields that are involved in the calculation.

```

<F id="Other_Charges_Line_Item0">
<V n="TYPE">Other_Charges_Line_Item</V>
etc.
<F id="Quantity">
etc.
<V n="STATUS">1</V>
<V n="MESSAGE">Failed By Calculate Action On Field <-- Quantity field
&apos;Validation&apos;.</V>
etc.
</F>
<F id="Unit_Cost">
etc.
<V n="STATUS">1</V>
<V n="MESSAGE">Failed By Calculate Action On Field <-- Unit_Cost field
&apos;Validation&apos;.</V>
etc.
</F>
<F id="Total">
etc.
<V n="STATUS">1</V>
<V n="MESSAGE">Failed By Calculate Action On Field <-- Total field
&apos;Validation&apos;.</V>
etc.
</F>
<F id="Validation">
etc.
<V n="STATUS">1</V>
<V n="MESSAGE">Failed Calculation:FormatNumber <-- Validation field
(1 * 4.95 ,8,0,0)=FormatNumber( 9.9,8,0,0)</V>
</F>
</F>

```

7. In Datacap Studio, cancel the batch when you are done.

TravelDocs: Verifying the line item grid pages

For demonstration purposes, use Datacap Desktop to verify the line item grid pages.

“Verifying pages by using Datacap Desktop”

Verifying pages by using Datacap Desktop

To verify the line item grid pages, use Datacap Desktop, which defaults to the field-at-a-time interface.

This procedure requires that you prepare a batch and place the batch on hold. For details, see “Preparing a batch for verification” on page 100.

1. In the **Start** menu, click **IBM Datacap Clients** and select **Datacap Desktop**.
2. In the **Application** field, type TravelDocs.

3. Enter User ID: admin, Password: admin, and Station: 1 and click **Login**.
4. In the **Shortcut** field, select **Verify** and click **Start**. Then, open the most recent batch by double-clicking it.
5. In the Batch View pane, expand the third Hotel document and select the Other_Charges page. Then, select **Other_Charges_Line_Item0** in the grid. Datacap Desktop displays the corresponding line item in the snippet view beneath the grid.
6. Review the fields with validation errors that are marked in red.
7. Because the application is configured so that operators can override validation failures, click **Submit** and then click **OK** to override the errors.

Important: If you want to modify the Datacap Desktop interface, you must create a custom panel for each page. See *Datacap Desktop panel customization*.

8. When prompted to continue from the start of the batch, click **No** and then close Datacap Desktop.

Related information:

 Datacap Desktop panel customization

TravelDocs: Exporting line item grid data to a database

You can update the application to export line item grid data from the Other Charges page to a table in the export database.

“Exporting to a database”

Exporting to a database

To export the line item grid data to a database, you need to create an export database table, add rules to the ExportDB ruleset, and attach the rules to the document hierarchy.

“Creating the export database table”

“Adding rules to the ExportDB ruleset”

“Attaching the Export Other rules to the document hierarchy” on page 165

“Running a batch through the workflow” on page 166

Creating the export database table:

You must use Microsoft Access to add the export database table to the TravelDocsExport.mdb file. You can skip this procedure if you previously used the sample Access file because the file that you copied includes the required table.

1. Open the file C:\Datacap\TravelDocs\TravelDocsExport.mdb in Microsoft Access.
2. Create a table that is called Other_Charges.
3. Create a field for BatchID and for each of the fields that are defined for the Other Charges page. Make all fields of type Text.
4. Save the new table.

Adding rules to the ExportDB ruleset:

To successfully export line item grid data, you must add rules to the ExportDB ruleset and then add actions to the functions in these rules.

1. In the Datacap Studio **Rulesets** pane, select the **ExportDB** ruleset and click **Lock/Unlock ruleset for editing**.

2. Right-click the **ExportDB** ruleset and choose **Add Rule**. Rename the new rule **Export Other Open Database**.
3. Repeat to create three more rules:
 - Export Other Line Item
 - Export Other Total
 - Export Other Close Database
4. Click the **Actions library** tab and expand the **ExportDB** library.
5. Expand the **Export Other Open Database** rule and select **Function1**.
6. Select and add each of the actions to **Function1** by clicking **Add to function**, and set the action parameters as shown in the table.

Action	Parameter
ExportOpenConnection	@APPVAR(* /exportdb:cs)
SetTableName	Other_Charges

7. Expand the **Export Other Line Item** rule and select **Function1**.
8. Select and add each of the actions to **Function1** by clicking **Add to function**, and then set the action parameters as shown in the table.

Action	Parameter
ExportBatchIDToColumn	BatchID
ExportFieldToColumn	Date,Charge_Date
ExportFieldToColumn	Category,Category
ExportFieldToColumn	Quantity,Quantity
ExportFieldToColumn	Unit_Cost,Unit_Cost
ExportFieldToColumn	Total>Total
AddRecord	

9. Expand the **Export Other Total** rule and select **Function1**.
10. Select and add each of the actions to **Function1** by clicking **Add to function**, and set the action parameters as shown in the table.

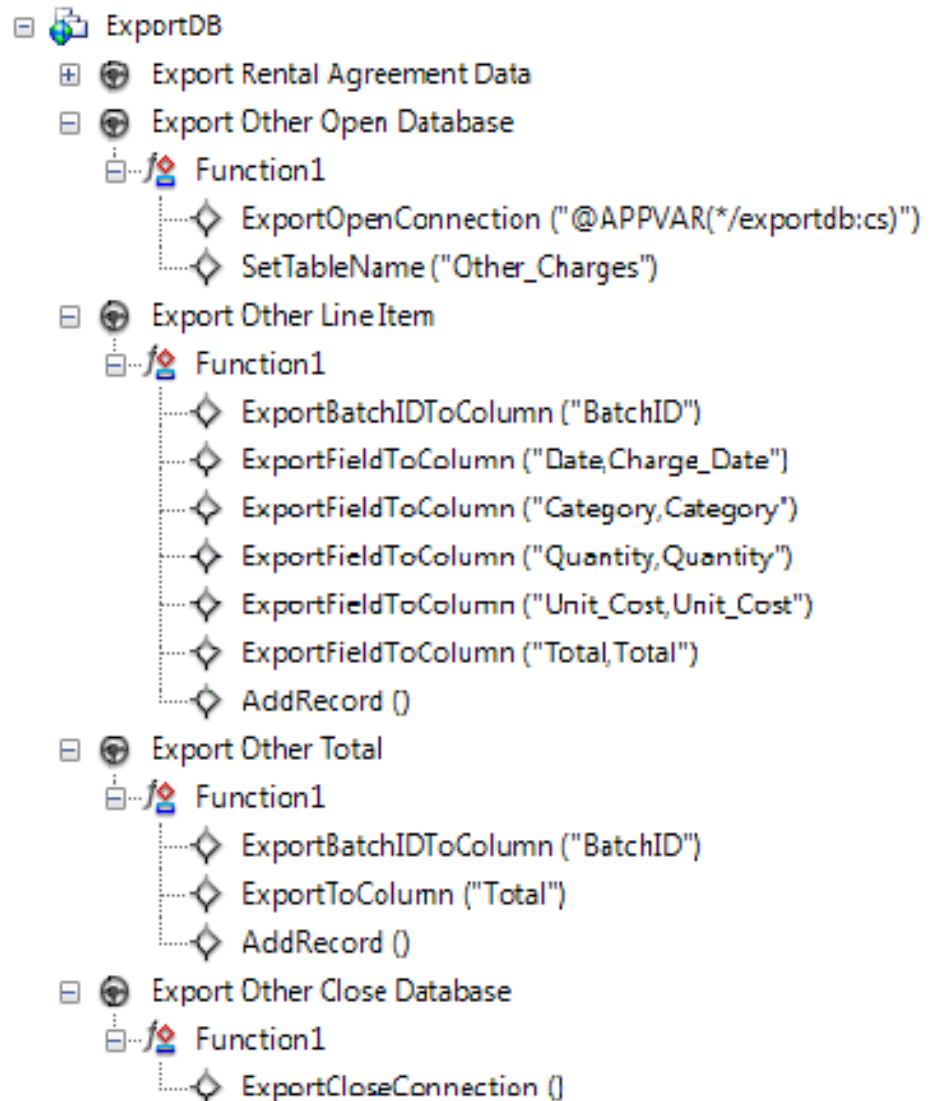
Important: The second action is **ExportToColumn**.

Action	Parameter
ExportBatchIDToColumn	BatchID
ExportToColumn	Total
AddRecord	

11. Expand the **Export Other Close Database** rule and select **Function1**.
12. Select and add the action to **Function1** by clicking **Add to function**.

Action	Parameter
ExportCloseConnection	

13. In the Rulesets pane, click **Save**. Then, click **Lock/Unlock ruleset** and choose **Publish ruleset**. The following image shows the finished ruleset:



Attaching the Export Other rules to the document hierarchy:

After you add the rules to the ExportDB ruleset, you must attach them to the document hierarchy.

1. In the Document hierarchy pane, click **Lock DCO for editing**.
2. Expand the document hierarchy so all of the fields in the Other_Charges page are visible.
3. Select the Other_Charges page. Then, select the Export Other Open Database rule and click the **Add to DCO**.
4. Select the Other_Charges_Line_Item field. Then, select the Export Other Line Item rule and click **Add to DCO**.
5. Select the Other_Charges_Total field. Then, select the Export Other Total rule and click **Add to DCO**.
6. Select the Other_Charges page's Close element. Then, select the Export Other Close Database rule and click **Add to DCO**.
7. In the Document hierarchy pane, click **Save** and then click **Unlock DCO**. The rules are linked to the document hierarchy.

Running a batch through the workflow:

After you run a batch through the workflow, you can open the `TravelDocsExport.mdb` file and confirm that the line item grid data was exported to the `Other_Charges` table.

1. Click the Datacap Studio Test tab.
2. In the Workflow pane, select the **VScan** task profile under **Main Job**.
3. Click **New** to start a new batch.
4. Click **Process rules for target object** and **Advance** to move the batch through the **VScan**, **PageID**, **Profiler**, **Verify**, and **Export** tasks.
5. Open the file `C:\Datacap\TravelDocs\TravelDocsExport.mdb` and review the exported data in the `Other_Charges` table.

Smart parameters

Smart parameters are action arguments that get evaluated at run time.

You used a few smart parameters already in the TravelDocs application. For example, when you created the ExportXML ruleset you used `@BatchID` to set the export file name equal to the ID of the current batch:

```
xml_SetFileName("@BatchID")
```

Since the value of `@BatchID` is different each time you run a batch through the workflow, you can create a unique export file for each batch.

Datacap provides various special variables (`@<variable_name>`) that you can use within smart parameters to access dynamic information at run time. You can use smart parameters to do the following tasks:

- Get information from the application configuration file
- Get (or set) the value of an object in the document hierarchy (typically a variable or a field value)
- Get job information such as the task name, ID of the operator who is running the batch
- Get system information such as the current date, time

Smart parameters can also include strings, navigation elements, and combinations of strings, navigation elements, and special variables.

You are going to look at smart parameters in more detail. You can update the TravelDocs application to export the line item grid data to an XML file. This update requires the use of various smart parameters.

“General structure of a smart parameter” on page 167

“Special variables to access application configuration settings” on page 168

“Access to the runtime hierarchy” on page 171

“Use other special variables” on page 174

“TravelDocs: Exporting line item grid data to an XML file” on page 175

Related information:



Application Manager

General structure of a smart parameter

A smart parameter can include any number of elements that are combined at run time to produce a single action argument.

Elements might include special variables, string constants, and navigation elements.

Restriction: Smart parameters do not work with all actions. Check the Action help in Datacap Studio for compatibility information.

Example 1

The following example shows an action with a single smart parameter argument that includes three smart parameter elements.

```
SetSourceDirectory("@Appath(vscandimagedir)+\+Input")
```

Smart parameter element	Description
@APPPATH(vscanimagedir)	Special variable that gets a setting from the application configuration (.app) file. See the "Special variables to access application configuration settings" on page 168 topic.
\	String constant
Input	String constant

Elements are combined by using the '+' sign. At run time, Datacap first evaluates any special variables and then concatenates the elements to create a single string that becomes the action argument.

```
07:13:25.53 3 Smart Parameter elements found
07:13:25.53 Parsing Smart Parameter element {0} value: "@APPPATH(vscanimagedir)"
07:13:25.54 @APPPATH key root value: 'vscanimagedir'
07:13:25.54 @APPPATH looking for workflow key: '*/dco_TravelDocs/vscanimagedir'
07:13:25.54 workflow key found: 'C:\Datacap\TravelDocs\images'
07:13:25.54 Parsing Smart Parameter element {1} value: "\"
07:13:25.54 Parsing Smart Parameter element {2} value: "Input"
07:13:25.54 Smart Parameter return value: 'C:\Datacap\TravelDocs\images'
07:13:25.54 looking for:C:\Datacap\TravelDocs\images\Input
07:13:25.55 Action changes: Directory with source images: C:\Datacap\TravelDocs\images\Input
```

It is incorrect to specify the argument as @APPPATH(vscanimagedir)+\Input. The '\' sign when followed by a string represents a navigation element. See the "Use navigation elements to access the runtime hierarchy" on page 173 topic. In this example, you do not want to specify a navigation element. Instead, you want to concatenate the result of @APPPATH(vscanimagedir) with the string \Input by using +\+Input.

Example 2

The next example shows an action with two smart parameter arguments. Each argument includes one smart parameter element:

```
rrSet ("..\Pickup Location","@B.FieldValue")
```

Smart parameter argument	Description
..\Pickup_Location	Navigation element that references another field on the same page within the runtime hierarchy
@B.FieldValue	Special variable that references a batch level custom variable within the runtime hierarchy

The portion of the following log file shows how Datacap evaluates the first argument by recognizing it as a navigation element. It then goes to the referenced element within the runtime hierarchy and retrieves the value of the field. In this example, the action is bound to a field, so `..\Pickup_Location` references another field at the same level on the same page.

```
08:17:30.892          action rrSet (str="..\Pickup_Location",str="@B.FieldValue")
08:17:30.892          execute statement On Action Start
08:17:30.892          executing code:
08:17:30.892          Call OnActionStart()
08:17:30.892          /execute statement On Action Start
08:17:30.892 1 Smart Parameter element found
08:17:30.892 Parsing Smart Parameter element {0} value: "..\Pickup_Location"
08:17:30.892 DCO Parent Navigation key match (starts with '\' or '..\'). Calling
DCONavGetValue(..\Pickup_Location)
08:17:30.892 Finding Child 'Pickup_Location' -->
08:17:30.893 Found child 'Pickup_Location'
08:17:30.895 Finding Dictionary assigned to DCO Node:'Pickup_Location'
08:17:30.895 This DCO does not have an assigned Dictionary or is not an OMR type Field.
08:17:30.895 Smart Parameter return value: 'Orlando (MCO)'
08:17:30.896 Setting '20110054.002.FieldValue' value to 'Orlando (MCO)'.
```

Special variables to access application configuration settings

The application configuration file, or `.app` file, stores the paths, connection strings, and other settings of the application. You can use special variables to access the application configuration settings.

Do not attempt to modify this file directly, use the Datacap Application Manager. You used the Datacap Application Manager to configure the export database. For more information, see “Configuring the export database” on page 137.

The `.app` file is stored in the root of the application folder. For example, the configuration file of the TravelDocs application is `C:\Datacap\TravelDocs\TravelDocs.app`:

```
<app name="TravelDocs" ver="45" modder="localadm.YODA647.DC14.DATACAP"
dt="03/09/12.753 11:41:06.753 " src_ver="1">
  <k name="tmservers">
    <k name="tms" ip="127.0.0.1" port="2402" retry="3"/>
  </k>
  <k name="runtime" v="batches"/>
  <k name="tmengine" cs="<encoded_connection_string>"/>          <-- encoded
  <k name="tadmin" cs="<encoded_connection_string>"/>          <-- encoded
  <k name="dco_TravelDocs">
    <k name="setupdco" v="TravelDocs.xml"/>
    <k name="rules" v="rules"/>
    <k name="imagefix" v="imagefix.ini"/>
    <k name="UseFPXML" v="False"/>
    <k name="fingerprintconn" cs="<encoded_connection_string>"/>  <-- encoded
    <k name="vscanimagedir" v="C:\Datacap\TravelDocs\images"/>    <-- encoded
    <k name="exportdb" cs="<encoded_connection_string>"/>        <-- encoded
  </k>
  <k name="fingerprint" v="fingerprint"/>
  <k name="export" v="export"/>
</app>
```

Connection strings might contain user names and passwords, so they are encoded when they are written to the `.app` file. Datacap encodes and decodes user names and passwords automatically, so no special handling is required when you access them from your application by using smart parameters. For information about storing other action parameters in the `.app` file as encoded strings, see “Storing passwords, connection strings, and other parameters in the `.app` file” on page 169.

Applications can access settings in the configuration file by using the following special variables:

Smart Parameter	Description
@APPPATH	Retrieves the path to a file or folder from the application configuration file.
@APPVAR	Retrieves a connection string, value, or other attribute from the application configuration file.

For detailed information about these and other special variables, see the “Smart Parameter Special Variable Reference” on page 277.

For each special variable, you specify a key that represents the field that you want to get from the configuration file. When you used the **APPPATH** parameter, you specified export as the key (**@APPPATH(export)**).

“Determining the correct key name”

“Storing passwords, connection strings, and other parameters in the .app file”

“Reference passwords, connection strings, and other parameters from your actions” on page 171

Determining the correct key name

You can obtain the correct special variable key name from the Datacap Application Manager.

To determining the correct key name:

1. Start the Datacap Application Manager. From the Windows **Start** menu, select **IBM Datacap Services > Datacap > Datacap Datacap Application Manager**.
2. Move the mouse pointer over the field. The smart parameter and key name are displayed in the tooltip.

The tooltip shows the path to the images folder of the application. The dco_*[1] prefix is required if the application has multiple workflows. Substitute *[1] with the workflow name, for example:

```
@APPPATH(dco_Workflow2/vscanimagedir)
```

If there is only one instance, you can use * instead, for example:

```
@APPPATH(*vscanimagedir)
```

Keys for connection strings are more complicated because the connection string is stored in the cs attribute rather than the v attribute. The v attribute is the default attribute, so you do not need to specify the attribute name. To obtain the value of a different attribute, you must specify the attribute name by using :<attribute_name>.

You used the following syntax earlier to obtain the connection string for the lookup database of the application:

```
@APPVAR(*lookupdb:cs)
```

Details of these special variables and a listing of key names are provided in the “Special variables to access application configuration settings” on page 168 topic.

Storing passwords, connection strings, and other parameters in the .app file

The sample .app file illustrates how Datacap encodes the standard Datacap database connection strings (engine, admin, fingerprint, lookup, and export) before it writes them to the .app file.

The function that is described in here is available in Datacap 8.0.1 or higher.

You can use the .app file to store other action parameters as encoded strings. You can then use smart parameters to access the strings from your actions. You do not have to specify sensitive information like passwords as action parameters.

- Instead of: `ex_login("svr/exch.asmx","user@company.com","secret")`
- Use: `ex_login("svr/exch.asmx","user@company.com",@APPVAR(values/adv/pwd))`

You can also use the .app file to store other action parameters that might not be sensitive. Action parameters that you do not want to hardcode into your actions. For example, you might choose to store a machine-specific path as a custom value. Then, you can change it easily if you move the application to a different computer.

To store passwords, connection strings, and other parameters in the .app file:

1. In the **Start** menu, select **IBM Datacap ServicesDatacap Application Manager**.
2. Click the **Custom values** tab and select your application from the list on the left.
3. Click **Add new value/CS name** beneath the field you want to use:

Field	Description
General string values	Use this field for action parameters you do not want to hardcode in your actions. Instead of specifying a machine-specific path as an action parameter, enter the path here and reference it from your actions as described in the next section. Datacap encodes the values when it saves them to the .app file. Do not use this field for passwords as the strings are visible to anyone who is using the Datacap Application Manager. Use the Advanced values that are listed in this table.
Data source connection string values	Use this field to store data source connection strings that are not Datacap connection strings. Type or paste your connection string into this field and reference it from your actions as described in the next section.
Datacap data source connection string values	Use this field to store Datacap data source connection strings. Click the[...] to create the connection string by using Datacap supported providers and reference it from your actions as described in the next section.
Advanced values	Use this field to store passwords or other strings you do not want to reveal through the Datacap Application Manager. Values that you type here are masked. Reference the value from your actions as described in the next section.

4. Enter the value name and the value. Advanced values are masked whereas other values are not.
5. Close the Datacap Application Manager window.
Attention: If you change any of the settings in the application configuration file while Datacap Studio is open, click **Connection Wizard** to reopen your application. Then, you can run tasks from the Datacap Studio Test tab. Reconnecting to the application forces Datacap Studio to reload the information from the application configuration (.app) file.

Related information:

 Application Manager

Reference passwords, connection strings, and other parameters from your actions

To reference the custom values from your actions, you must know the key path of the actions.

You can get the key path from the help text on the **Custom values** tab in the Datacap Application Manager.

The text at the beginning of each section shows how to reference the value from an action. For example, for values that are defined in the Advanced values section, use @APPVAR(values/adv/<value_name>). You can reference the value as:
@APPVAR(values/adv/MyPassword1)

The following table shows how to reference the values for each field type.

Field	Description
General string values	@APPVAR(values/gen/<value_name> Example: @APPVAR(values/gen/MyParameter1)
Data source connection string values	@APPVAR(values/dsn/<value_name>:cs) Example: @APPVAR(values/dsn/MyDatabase1:cs)
Datacap data source connection string values	@APPVAR(values/tmdsn/<value_name>:cs) Example: @APPVAR(values/tmdsn/MyTMDatabase1:cs)
Advanced values	@APPVAR(values/adv/<value_name> Example: @APPVAR(values/adv/MyPassword1)

Attention: The :cs suffix is required to access connection strings that are defined in the **Data source connection string** and **Datacap data source connection string** fields.

Access to the runtime hierarchy

You can access the runtime batch hierarchy indirectly through the Datacap Studio Test tab and directly by opening the runtime XML files.

The runtime XML file maps to the **Runtime batch hierarchy** in the Datacap Studio **Test** tab. For example the XML elements B id=, D id=, and P id=, all map to the Batch , Document, and Page in the batch hierarchy.

You access the information in the runtime hierarchy by using smart parameters.

“Examples of using special variables to access the runtime hierarchy” on page 172

“Summary of special variables for accessing the runtime hierarchy” on page 172

“Use navigation elements to access the runtime hierarchy” on page 173

Examples of using special variables to access the runtime hierarchy

The TravelDocs application used special variables to access data in the runtime hierarchy. You can also use the ExportXML ruleset.

The @BatchID example and the @ID example describe how to access data in the following sample runtime batch hierarchy XML from the ExportXML ruleset or with special variables.

```
<B id="20110003.001">
  <V n="TYPE">TravelDocs</V>
  <D id="2011003.001.01">
    <V m="TYPE">Car_Rental</V>
    <P id="TM000001">
      etc.
```

Use @BatchID to get the current batch ID

xml_SetFileName("@BatchID") returns xml_SetFileName("20110003.001")

Use @ID to get the ID of the current page

xml_NewNode("@ID,Rental_Agreements") returns
xml_NewNode("@TM000001,Rental_Agreements") .

Use @P\<field_name> to get the value of a field on the current page

The @P variable retrieves the value of a field on a current page, as shown in this sample runtime page data XML file.

```
<P id=TM00001>
<F id="Pickup_Date">
  <V n="TYPE">Pickup_Date</V>
  <V n="Position">179,384,543,462</V>
  <V n="STATUS">0</V>
  <C cn="7" cr="200,416,220,430>84</C>      <!-- T -->
  <C cn="10" cr="226,425,240,440>117</C>    <!-- u -->
  <C cn="10" cr="245,425,258,440>101</C>    <!-- e -->
  <C cn="10" cr="260,425,270,440>115</C>    <!-- s -->
  <C cn="10" cr="273,435,278,444>44</C>     <!-- , -->
  <C cn="10" cr="336,419,337,440>32</C>     <!-- -->
  <C cn="10" cr="290,419,306,440>68</C>     <!-- D -->
  <C cn="10" cr="310,425,324,440>101</C>    <!-- e -->
  <C cn="10" cr="325,425,336,440>99</C>     <!-- c -->
  <C cn="10" cr="370,419,371,444>32</C>     <!-- -->
  <C cn="10" cr="349,419,363,440>55</C>     <!-- 7 -->
  <C cn="10" cr="365,435,370,444>44</C>     <!-- , -->
  <C cn="10" cr="445,419,446,440>32</C>     <!-- -->
  <C cn="10" cr="381,419,395,440>50</C>     <!-- 2 -->
  <C cn="10" cr="396,419,411,440>48</C>     <!-- 0 -->
  <C cn="10" cr="415,419,428,440>49</C>     <!-- 1 -->
  <C cn="10" cr="430,419,445,440>48</C>     <!-- 0 -->
```

Using the sample XML, xml_SetModeValue("Pickup_Date,@P\Pickup_Date") results in xml_SetModeValue("Pickup_Date,Tues, Dec 7, 2010").

Summary of special variables for accessing the runtime hierarchy

You can use @ID and @B/ @D/ @P<variable_name> special variables to get most of the batch, document, and page information from the runtime batch file.

For a full listing of special variables for accessing the runtime hierarchy, see “Special variables for accessing the runtime hierarchy” on page 279. The following examples illustrate how to use special variables to access the runtime hierarchy.

Table 44. Special variables for accessing the runtime hierarchy

Access this part of the runtime hierarchy...	...by using this special variable	Hierarchy level
<?xml-stylesheet type="text/xsl" href="..\..\dco.xsl"?>	Not applicable	BATCH
<B id="appdevguide_20110003.001">	@ID	BATCH
<V n="TYPE">TravelDocs</V>	@B.TYPE	BATCH
<V n="LAST_RR_TPROFILE">Rulerunner:m:eRun</V>	@B.LAST_RR_PROFILE	BATCH
<V n="STATUS">1</V>	@B.STATUS	BATCH
<D id="appdevguide_20110003.001.01">	@ID	DOCUMENT
<V n="TYPE">Car_Rental</V>	@D.TYPE	DOCUMENT
<V n="STATUS">0</V>	@D.STATUS	DOCUMENT
<P id="appdevguide_TM000001">	@ID	PAGE
<V n="TYPE">Rental_Agreement</V>	@P.TYPE	PAGE
<V n="STATUS">1</V>	@P.STATUS	PAGE
<V n="IMAGEFILE">tm000001.tif</V>	@P.IMAGEFILE	PAGE
<V n="ScanSrcPath">c:\...\images\page_01.tif</V>	@P.ScanSrcPath	PAGE
<V n="RecogStatus">0</V>	@P.RecogStatus	PAGE
<V n="Confidence">0.9660463</V>	@P.Confidence	PAGE
<V n="TemplateID">556</V>	@P.TemplateID	PAGE
<V n="DATAFILE">tm000001.xml</V>	@P.DATAFILE	PAGE
</P>	Not applicable	PAGE

Similarly, you can get most of the field information from the runtime page data file by using the @ID, @F.<variable_name>, and @P\<field_name> special variables, for example.

Table 45. Special variables for accessing field data in the runtime hierarchy

Access this part of the runtime hierarchy...	...by using this special variable	Hierarchy level
<F id="appdevguide_Pickup_Date">	@ID	FIELD
<V n="TYPE">Pickup_Date</V>	@F.TYPE	FIELD
<V n="Position">179,394,543,462</V>	@F.Position	FIELD
<V n="STATUS">0</V>	@F.STATUS	FIELD
<C cn="7" cr="200,416,220,440">84</C>	@P\Pickup_Date	FIELD
<C cn="10" cr="226,425,240,440">117</C>	@P\Pickup_Date	FIELD
<C cn="10" cr="245,425,258,440">101</C>	@P\Pickup_Date	FIELD
<C cn="10" cr="260,425,270,440">115</C>	@P\Pickup_Date	FIELD
</F>	Not applicable	FIELD

Use navigation elements to access the runtime hierarchy

In addition to using special variables, you can also use navigation elements to reference fields values and variables.

Smart parameters support the following navigation elements:

Table 46. Smart parameter navigation elements

Example	Description
<code>\< field_name></code>	References a field that is 1 level beneath the current object
<code>..\< field_name></code>	References a field at the same level as the current object

When you reference a field by specifying just the name of the field, Datacap retrieves the text value of the field. You can obtain the value of a variable that is associated with the field by appending `.<variable_name>`, for example: `..\Car_Type.TYPE`.

You cannot use this syntax to access variables on parent objects.

Examples

In these examples, the `rr_Get` action is bound to a field.

- In the first example, the smart parameter returns the text of the **Car_Type** field on the current page.
- In the second example, the smart parameter returns the value of the *TYPE* variable of the **Car_Type** field.

```
Action: rr_Get("..\\Car_Type")      <F id="Car_Type">
Return value: SUV                  <V n="TYPE">Car_Type</V>
<C cn="10" cr="588,748,600,769">83</C>  <-- ASCII 'S'
Action: rr_Get("..\\Car_Type.TYPE")
<C cn="10" cr="605,748,620,769">85</C>  <-- ASCII 'U'
Return value: Car_Type
<C cn="10" cr="625,748,643,769">86</C>  <-- ASCII 'V'
</F>
```

Use other special variables

You can use other special variables to access task information, job information, and other information.

“Access job and task information”

“Access other information”

Access job and task information

Datacap includes several special variables for accessing job and task information.

The special variables can be used to access the following task and job information.

- The ID and name of the current job
- The ID and name of the current task
- The user name and station that is associated with the job

A listing of these special variables is provided in “Special variables for accessing job and task information” on page 283.

Access other information

You can use special variables to access other information such as date, time, DCO and Pilot objects information, and application settings.

Datacap includes more special variables for accessing the following information.

- The current date and time
- Information from the DCO and Pilot objects
- Settings that are defined in the `Paths.ini` file of the application

A listing of these special variables is provided in “Miscellaneous special variables” on page 285.

TravelDocs: Exporting line item grid data to an XML file

You can update the application to export data to an XML file by using smart parameters to access the runtime document hierarchy.

You added functions to the TravelDocs application to export the Other Charge grid data to a database. For more information, see “Export to a database” on page 104. You also set up a custom variable to store data within the document hierarchy. Now, you can export that same data to an XML file.

“Adding rules to the ExportXML ruleset”

“Attaching the Export Other XML rules to the document hierarchy” on page 177

“Running a batch through the workflow” on page 177

Adding rules to the ExportXML ruleset

To export line item grid data to an XML file, you must add the appropriate rules to the ExportXML ruleset.

Adding rules to the ExportXML ruleset:

1. In the Datacap Studio Rulesets pane, select the **ExportXML** ruleset and click **Lock/Unlock ruleset for editing**.
2. Right-click the **ExportXML** ruleset and choose **Add Rule**. Rename the new rule **Export Other XML Page Node**.
3. Right-click the **ExportXML** ruleset and choose **Add Rule**. Rename the new rule **Export Other XML Line Item**.
4. Right-click the **ExportXML** ruleset and choose **Add Rule**. Rename the new rule **Export Other XML Total Cost**. The **ExportXML** ruleset includes the following rules:
 - Open XML File
 - Export Rental Agreement XML
 - Export Other XML Page Node
 - Export Other XML Line Item
 - Export Other XML Total Cost
 - Close XML File
5. Expand the **Open XML File** rule and the **Open XML** function.
6. Click the **Actions library** tab and expand the **Export XML** library.
7. Select and add each of the following actions that are shown in the following table to the end of the **Open XML** function by clicking **Add to function**. Then, set the action parameters as shown in the second table.

Action	Parameter
xml_NewNode	Car_Rentals,BatchID_+@BatchID
xml_NewNode	Rental_Agreements,Car_Rentals

Action	Parameter
xml_NewNode	Flights,BatchID_+@BatchID
xml_NewNode	Hotels,BatchID_+@BatchID
xml_NewNode	Other_Charges,Hotels

8. Expand the **Export Other XML Page Node** rule and select **Function1**.
9. Select and add each of the actions that are shown in the following table to **Function1** by clicking **Add to function**. Then, set the action parameters as shown in the table.

Library	Action	Parameter
ExportXML	xml_NewNode	@ID,Other_Charges
rrunner	rrSet	varSource = @ID
	(do not use rr_Set)	varTarget = @P.ID

Important: xml_NewNode("@ID,Other_Charges") creates a new XML node by using the ID of the current page (for example, <TM000013>) beneath the <Other_Charges> node.

rrset("@ID","@P.ID") stores the ID of the current page in a variable that is called *ID* within the runtime hierarchy (for example, <V n="ID">TM000013</V>).

10. Expand the **Export Other XML Line Item** rule and select **Function1**.
11. Select and add each of the actions that are shown in the following table to **Function1** by clicking **Add to function**. Then, set the action parameters as shown in the table.

Library	Action	Parameter
ExportXML	xml_NewNode	Item,@P.ID
ExportXML	xml_SetAttributeValue	Item,Category,@F\Category
ExportXML	xml_SetAttributeValue	Item,Cost,@F\Total

Important: xml_NewNode("Item,@P.ID") creates a new node <Item> that is a child of the node that you created in the **Export Other XML Page Node** rule. The @P.ID parameter identifies the parent node by using the *ID* variable of the page that you saved earlier by using rrset. You cannot reference the ID of a parent object by using a smart parameter.

xml_SetAttributeValue("Item,Category,@F\Category") creates a Category attribute on the current <Item> node and sets the value to the value of the current line item's Category field (for example, <Item Category=Internet/>).

The xml_SetAttributeValue("Item,Cost,@F\Total") parameter is the same except that the attribute is Cost and the value is the current line item's Total field (for example, <Item Cost="9.90"/>).

12. Expand the **Export Other XML Total** rule and select **Function1**.
13. Select and add each of the actions in the following table to **Function1** by clicking **Add to function**. Then, set the action parameters as shown in the table.

Library	Action	Parameter
ExportXML	xml_NewNode	Total_Cost,@P.ID
ExportXML	xml_SetNodeValue	Total_Cost, @P\Other_Charges_Total

Important: `xml_NewNode("Total_Cost,@P.ID")` creates a new XML node that is called `<Total__Cost>` beneath the page node that you created in the Export Other XML Page Node rule. The `xml_SetNodeValue` action sets the value of this node to the value of the current page's `Other_Charges_Total` field (for example, `<Total_Cost>$238.75</Total_Cost>`).

14. In the Rulesets pane, click **Save**. Then, click **Lock/Unlock ruleset** and choose **Publish ruleset**.

Attaching the Export Other XML rules to the document hierarchy

After you add the appropriate rules to the ExportXML ruleset, you must attach the Export Other XML rules to the document hierarchy.

To attaching the Export Other XML rules to the document hierarchy:

1. In the **Document hierarchy** pane, click **Lock DCO for editing**.
2. Expand the document hierarchy so that the fields in the **Other_Charges** page are visible.
3. Select the **Other_Charges** page. Then, select the **Export Other XML Page Node** rule and click **Add to DCO**.
4. Select the **Other_Charges_Line_Item** field. Then, select the **Export Other XML Line Item** rule and click **Add to DCO**.
5. Select the **Other_Charges_Line_Total** field. Then, select the **Export Other XML Total Cost** rule and click **Add to DCO**.
6. In the **Document hierarchy** pane, click **Save**. Then, click **Unlock DCO**. The rules are linked to the document hierarchy.

Running a batch through the workflow

To run a batch through the workflow:

1. Click the Datacap Studio **Test** tab.
2. In the **Workflow** pane, select the **VScan** task profile under **Main Job**.
3. Click **New** to start a new batch.
4. Click **Process rules for target object**.
5. Click **Advance** to move the batch through the entire workflow.
6. Open the file `C:\Datacap\TravelDocs\export\<batch_id>.xml` and review the exported XML data.

```
<?xml version='1.0' ?>
<BatchID_20100351.006>
  <Flights/>
  <Car_Rentals>
    <Rental_Agreements>
      <TM000001>
        <Pickup_Date>Trues, Dec 7, 2010</Pickup_Date>
        etc.
      </TM000001>
      <TM000003>
        <Pickup_Date>Mon, Dec 6, 2010</Pickup_Date>
        etc.
      </TM000003>
      <TM000004>
```



```

                                <Pickup_Date>Mon, Dec 13, 2010</Pickup_Date>
                                etc.
                                </TM000004>
                                </Rental_Agreements>
</Car_Rentals>
<Hotels>
    <Other_Charges>
        <TM000013>
            <Item Category="Internet" Cost="$9.90"/>
            <Item Category="Laundry" Cost="$18.00"/>
            <Item Category="Internet" Cost="$4.95"/>
            <Item Category="Newspaper" Cost="$2.00"/>
            <Item Category="Mini bar" Cost="$8.00"/>
            <Item Category="Internet" Cost="$4.95"/>
            <Item Category="Newspaper" Cost="$2.00"/>
            <Item Category="Mini bar" Cost="$8.00"/>
            <Item Category="Internet" Cost="$4.95"/>
            <Item Category="Newspaper" Cost="$2.00"/>
            <Item Category="Parking" Cost="$74.00"/>
            <Total_Cost>$238.75</Total_Cost>
        </TM000013>
    </Other_Charges>
</Hotels>
</BatchID_20100351.006>

```

Text matching

You can add flexibility to your applications by using text matching to identify pages and locate data.

You used fingerprints to identify pages and recognition zones to locate data on those pages. The one exception was when you used the **RegExFind** action to locate the **Total Cost** field when you process line item grids. You saw in that example how you can use text matching to locate data that is not in a predictable location on the page.

To use text matching, you first do full page OCR on the incoming page. You can then search the recognition results for specific text. For example, if a page contains the words Car rental agreement, then the chances are high that it is a car rental agreement page. Similarly, if you locate a string Pickup date, then the chances are high that beside or below the string is the actual pickup date.

You can identify pages and locate data by using text matching. You can update the TravelDocs application to identify and process a new car rental agreement page by using text matching.

“Identify pages with text matching”

“Locate data with text matching” on page 179

“TravelDocs: Update the application to use text matching” on page 183

Identify pages with text matching

Text matching uses the full page recognition results to identify pages.

You can identify pages by searching the recognition results for a string that is unique to each page type.

If all of the actions in a function return True, Datacap does not run any other functions in the current rule. For example, if you get a match on the word Car and can set the page type successfully. The PageID rule exits without running any of the other tests

Text matching uses the full page recognition results, so you must do full page OCR (or ICR) before you run any of the text matching actions. You can then use the WordFind action to determine whether a specific string is present and the SetPageType action to set the page type.

Library	Action	Description
Locate	WordFind	Locates the first (or next) occurrence of the specified word or phrase on the current page.
DCO	SetPageType	Assigns a page type to the current page in the runtime hierarchy.

The WordFind action is case-sensitive. Additionally, if different variants of a page type have different unique identifiers, you might need to use a more flexible matching technique. Such as regular expressions or keyword lists. For more information, see “Use regular expressions” on page 180 and “Text matching with keyword lists” on page 180.

Related information:

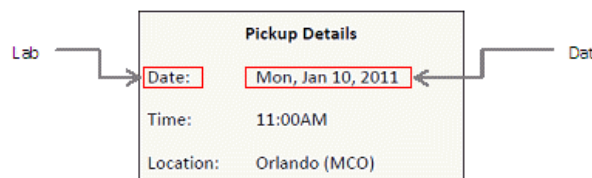
➡ Using regular expressions

➡ Using keyword lists

Locate data with text matching

You can locate data without using recognition zones. You search the full page recognition results for some static text that is next to the field you want to read.

Forms typically have a label beside each field. When you locate the label, you can locate the adjacent data and then update the runtime hierarchy.



In this example, you locate the word Date. Then, you go to the right to obtain the matching data, and then write the information to the runtime hierarchy.

“Locate simple strings”

“Use regular expressions” on page 180

“Text matching with keyword lists” on page 180

“Locate the field data” on page 181

“Update the runtime data file with the recognized text” on page 182

“Text matching for data recognition limitations” on page 183

Locate simple strings

The Locate library includes actions that you can use to locate specific text strings on the current page.

The Locate library includes the following actions to locate specific text strings on a page.

Library	Action	Description
Locate	WordFind	Locates the first (or next) occurrence of the specified word or phrase on the current page.
Locate	FindLastWord	Locates the last occurrence of the specified word or phrase on the current page.

For more information on all of the actions in the Locate library, select the action on the **Actions Library** tab and click **Display information**.

The WordFind action is used in the previous page identification example. You can use WordFind or FindLastWord to locate any word or phrase on the current page. These actions require an exact match. If you need more flexible matching, you can use regular expressions or keyword lists.

Use regular expressions

If you use text matching on pages with various labels, you might be unable to locate a label by using a simple text string. In this case, you can use a regular expression.

For example, one car rental company might use the label Date, another might use the label Pickup date, while another might use Pickup Date. In this example, you can use the Locate library regular expression actions, including the following actions.

Library	Action	Description
Locate	RegExFind	Same as WordFind, except that it supports regular expressions.
Locate	FindLastRegEx	Same as FindLastWord, except that it supports regular expressions.

One way you can search for any of the three pickup date labels in the previous example is by using a single regular expression.

```
RegExFind( "(Date)|(Pickup [Dd]ate) ")
```

The Locate library regular expression actions use the VBScript regular expression rules. Some of this usage is described in detail in the following MSDN article

<http://msdn.microsoft.com/en-us/library/1400241x%28v=vs.85%29.aspx>

Text matching with keyword lists

If the page on which you are text matching has too many variations on the page, regular expressions might be unwieldy. A keyword list might be a better option.

Some labels might have more variability than you can address by using regular expressions. For example, if you are processing invoices from different vendors, one company might use the label Total Cost, another might use Amount Due, and another might use Invoice total.

The Locate library provides actions to do text matching by using either a keyword text file or a database.

Library	Action	Description
Locate	FindKeyList	Locates the first (or next) occurrence of a word or phrase that matches one of the entries in a keyword file.
Locate	FindLastKeyList	Locates the last occurrence of a word or phrase that matches one of the entries in a keyword file.
Locate	FindDBList	Locates a word that matches one of a list of words that are obtained from a SQL query.

In the invoice example, you can include all three labels and any others in a keyword text file. You can then use FindRegExList to locate the matching field. The keyword file must be a plain text file with the extension `.key`. The file must be in the application `dco_application_name` folder, unless you specify the full path to an alternative location. An example file might be named `TotalCost.key` and include variations on the theme such as:

- Total Cost
- Total cost
- TOTAL COST
- Amount Due
- Amount due
- AMOUNT DUE
- Total Amount
- Total amount
- TOTAL AMOUNT
- Total amount due
- Total Amount Due
- TOTAL AMOUNT DUE
- Invoice total
- Invoice Total
- INVOICE TOTAL

The locate library action is **FindRegExList("TotalCost.key")**

Tip: You can also include regular expressions in the keyword list.

Locate the field data

After you locate the label, you must locate the adjacent field data.

The field data is usually to the right of the label, but it might also be above or below the label. Additionally, you might need to group words together if the data you are searching for includes spaces.

The full page recognition engine organizes the recognition results in the CCO file as a coordinate-based grid of lines and words. Each word is assigned a different position in the grid.

	1	2	3	4	5	6	7	8	9	10
1	Car	Rental	#4							
2	Pickup	Details	Return	Details						
3	Date:	Mon,	Jan	10,	2011	Date:	Fri,	Jan	14,	2011
4	Time:	11:00AM	Time:	04:00PM						
5	Location:	Orlando	(MCO)	Location:	Orlando	(MCO)				

This structure allows you to move around the recognition results by using the Locate library's navigation actions. The library also includes actions for grouping words together.

Library	Action	Description
Locate	GoRightWord	Moves the specified number of words to the right of the previously found word or phrase.
Locate	GoDown(Up)Line	Moves down (up) the specified number of lines from the previously found word or phrase and selects the first word.
Locate	GroupWordsRIGHT(LEFT)	Groups words to the right (left) of the previously found word if they are no more than the specified number of character widths apart.
Locate	GroupWords	Groups words to the left and right of the previously found word if they are no more than the specified number of character widths apart.

The TravelDocs Recognize ruleset contains a rule that searches for the word Date and then goes one word to the right to obtain the data. The **GroupWordsRIGHT** action is required because, without it, you would get only the first word (Mon, in the Car Rental #4 example). The parameter **2** instructs the rule to group words that are two or fewer character widths apart.

Update the runtime data file with the recognized text

After you locate the data field, you must write the data to the runtime hierarchy.

You write data to the runtime hierarchy by using the UpdateField action from the Locate library. The UpdateField action updates the page data file with the recognized value and position of the located word.

The CreateFields action is responsible for setting up each field in the runtime hierarchy. Initially both the field position and the field data are empty, as shown in the left column on the following example. The right column shows the field after it is populated by using the UpdateField action. The position information is used later to display the corresponding image snippet to the operator during verification.

After CreateFields():	After UpdateField():
<pre><F id="Pickup_Date"> <V n="TYPE">Pickup_Date</V> <V n="Position">0,0,0,0</V> <V n="STATUS">0</V> </F></pre>	<pre><F id="Pickup_Date"> <V n="TYPE">Pickup_Date</V> <V n="Position">539,419,789,452</V> <V n="STATUS">0</V> <C cn="10" cr="543,423,565,444">77</C> M <C cn="10" cr="570,429,585,444">111</C> o <C cn="10" cr="588,429,600,444">110</C> n <C cn="9" cr="605,440,610,448">44</C> , <C cn="9" cr="0,0,0,0">32</C> <C cn="9" cr="620,423,628,444">74</C> J <C cn="10" cr="630,429,643,444">97</C> a <C cn="10" cr="648,429,660,444">110</C> n <C cn="9" cr="0,0,0,0">32</C> <C cn="10" cr="674,423,685,444">49</C> 1 <C cn="10" cr="689,423,704,444">48</C> 0 <C cn="9" cr="705,440,710,448">44</C> , <C cn="9" cr="0,0,0,0">32</C> <C cn="10" cr="723,423,735,444">50</C> 2 <C cn="10" cr="739,423,754,444">48</C> 0 <C cn="10" cr="756,423,769,444">49</C> 1 <C cn="10" cr="774,423,785,444">49</C> 1 </F></pre>

Text matching for data recognition limitations

While text matching provides a quick way to read data from a non-fingerprinted page, it does have limitations.

The most notable limitations of text matching for data recognition are processing pages that include check box options or line item grids.

TravelDocs: Update the application to use text matching

You can update the TravelDocs application to use text matching for data recognition.

To update the TravelDoc application to use text matching, you must determine which pages to process. Then, you can run data recognition and add the rules to the document hierarchy.

“Identifying unrecognized pages by using text matching”

“Recognizing data with text matching” on page 184

“Attaching the rules to the document hierarchy” on page 186

“Running a batch through the workflow” on page 186

Identifying unrecognized pages by using text matching

You can identify unrecognized pages by creating a ruleset with the Identify using Text Match function. You use the rule to recognize car rental pages by looking for text that is unique to that page type.

To identify unrecognized pages by using text matching:

1. In the Datacap Studio **Rulesets** pane, select the **PageID** ruleset and click **Lock/Unlock ruleset for editing**.
2. Expand the **PageID** rule.
3. Change the name of the existing function from PageID: Other Function 1 to Identify using Fingerprint. Then, change the parameter on the FindFingerprint action to False.

Attention: Setting the parameter to False ensures that Datacap does not automatically generate a fingerprint file for unrecognized pages. If the current page does not match one of the existing fingerprints, this action fails and Datacap starts the next function, if there is one.

4. Right-click the **PageID** rule and choose **Add Function**. Then, rename the new function to Identify using Text Match.
5. Click the **Actions library** tab and add the actions that are shown in the following table to the Identify using Text Match function by clicking **Add to function**. Then, set the action parameters as shown.

Library	Action	Parameter
Locate	RegExFind	Car
Locate	RegExFind	Pickup
DCO	SetPageType	Rental_Agreement
runner	rrSet	varSource = Text varTarget = @P.MatchType

Important: The Car and Pickup parameters are tested to avoid mismatches on insurance pages. The rrSet action sets up a page variable that you must later identify which pages to process by using text matching.

Attention: If the Identify using Fingerprint function succeeds in identifying the page, the Identify using Text Match function does not start.

6. In the **Rulesets** pane, click **Save**. Then, click **Lock/Unlock ruleset** and choose **Publish Ruleset**.

Recognizing data with text matching

To recognize data by using text matching, add rules to the Recognize ruleset. These rules locate the data on the rental agreement pages that you identified by using text matching.

Avoid running the rules on pages that were identified by using fingerprint matching. You can use the variable that you set up earlier to determine which pages use fingerprint matching.

To recognize data by using text matching:

1. In the Datacap Studio **Rulesets** pane, select the **Recognize** rule set and click **Lock/Unlock ruleset for editing**.
2. Right-click the **Recognize** rule set and choose **Add Rule**. Repeat this step to add a total of six new rules. Rename the rules as follows:
 - Recognize Pickup Date
 - Recognize Pickup Location
 - Recognize Return Date
 - Recognize Return Location
 - Recognize Car Type
 - Recognize Total Cost
3. Add the actions from the following table to **Recognize Pickup Date > Function1** and set the action parameters as shown.

Library	Action	Parameter
runner	rrCompare	object1 = @P.MatchType object2 = Text
Locate	RegExFind	Date
Locate	GoRightWord	1
Locate	GroupWordsRIGHT	2
Locate	UpdateField	

4. Add the actions from the following table to **Recognize Pickup Location > Function1** and set the parameters as shown.

Library	Action	Parameter
runner	rrCompare	object1 = @P.MatchType object2 = Text
Locate	RegExFind	Location
Locate	GoRightWord	1
Locate	GroupWordsRIGHT	2
Locate	UpdateField	

5. Add the actions from the following table to **Recognize Return Date > Function1** and set the parameters as shown.

Library	Action	Parameter
runner	rrCompare	object1 = @P.MatchType object2 = Text
Locate	FindLastRegEx	Date
Locate	GoRightWord	1
Locate	GroupWordsRIGHT	2
Locate	UpdateField	

6. Add the actions from the following table to **Recognize Return Location > Function1** and set the parameters as shown.

Library	Action	Parameter
runner	rrCompare	object1 = @P.MatchType object2 = Text
Locate	FindLastRegEx	Location
Locate	GoRightWord	1
Locate	GroupWordsRIGHT	2
Locate	UpdateField	

7. Add the actions from the following table to **Recognize Car Type > Function1** and set the parameters as shown.

Library	Action	Parameter
runner	rrCompare	object1 = @P.MatchType object2 = Text
Locate	RegExFind	Car Type
Locate	GoRightWord	1
Locate	GroupWordsRIGHT	2
Locate	UpdateField	

8. Add the actions from the following table to **Recognize Total Cost > Function1** and set the parameters as shown.

Library	Action	Parameter
runner	rrCompare	object1 = @P.MatchType object2 = Text
Locate	RegExFind	Total Cost
Locate	GoRightWord	1
Locate	UpdateField	

9. In the **Rulesets** pane, click **Save**. Then, click **Lock/Unlock ruleset** and choose **Publish Ruleset**.

Attention: The use of text matching to locate and recognize **OMR** check box fields is beyond the scope of this activity. When you run the rule set, it leaves the OMR options in their default state ('0').

Attaching the rules to the document hierarchy

You must attach each of the new Recognize rules to the corresponding field on the **Rental_Agreement** page definition.

To attaching the rules to the document hierarchy:

1. In the Document hierarchy pane, click **Lock DCO for editing**.
2. Expand the document hierarchy so you can see the fields on the **Rental_Agreement** page.
3. Select the **Pickup_Date** field. In the Rulesets pane, select the **Recognize Pickup Date** rule and click **Add to DCO**.
4. Select the **Pickup_Location** field. In the Rulesets pane, select **Recognize Pickup Location** and the **Add to DCO**.
5. Select the **Return_Date** field. In the Rulesets pane, select **Recognize Return Date** and click **Add to DCO**.
6. Select the **Return_Location** field. In the Rulesets pane, select **Recognize Return Location** and click **Add to DCO**.
7. Select the **Car_Type** field. In the Rulesets pane, select **Recognize Car Type** and click **Add to DCO**.
8. Select the **Total_Cost** field. In the Rulesets pane, select **Recognize Total Cost** and click **Add to DCO**.
9. In the Document hierarchy pane, click **Save**, then click **Unlock DCO**.

Running a batch through the workflow

Datacap provides an image file for a new rental agreement page that you can run through the workflow without fingerprinting.

To run a batch through the workflow:

1. On the Datacap Studio **Test** tab, select the **VScan** task profile and click **New**.
2. Click **Process rules for target object** and click **Advance** to move the batch through the **VScan** and **PageID** task profiles.
3. On the **Runtime batch hierarchy** tab, select the first page (TM000001) and make sure the Car Rental #4 page is displayed on the **Image** tab. Confirm that the page type is **Rental_Agreement**.
4. Click **Process rules for target object** and click **Advance** to move the batch through the **Rulerunner** task profile.
5. On the **Runtime batch hierarchy** tab, expand the first **Rental_Agreement** page to confirm that the data was recognized correctly (except for the options).
6. In the **Workflow** pane, right-click the batch (it must be in the **Verify** task) and choose **Hold**.
7. Start the Datacap Web Client, select the **TravelDocs** application, and log on.
8. In the Datacap Web Client window, select the **Monitor** tab. Locate the most recent batch at the top of the list with a status of **Hold**.
9. Click the batch's row number at the left of the row and choose **Yes** to start the selected batch.
10. Review the data on the Car Rental #4 page. Then, quit the task and click **OK** to put the batch back on hold.
11. In Datacap Studio, in the **Workflow** pane, right-click the batch and choose **Pick** to change its status back to **Running**. Then, click **Process rules for target object** and click **Advance** to move the batch through the remainder of the workflow.

Pattern Matching

You can use Datacap pattern matching to identify pages and adjust misaligned or distorted images.

Standard fingerprint matching can compensate for minor page misalignment, but the offsets it can handle are small. If an incoming page is misaligned relative to the fingerprint, Datacap might not be able to identify the page. If Datacap does identify the page successfully, recognition might fail if the fields are not registered accurately. This issue is most problematic if the page contains OMR or hand print boxed data. But it can happen with any page, especially if the image is distorted during copying, scanning, or faxing.

Pattern matching actions use reference patterns, or anchor objects, that you define on the page fingerprints. The actions try to match those patterns to regions on the runtime pages. Pattern matching techniques that are available can use:

- Geometric patterns like page registration marks or vendor logos
- Text-based patterns

Pattern matching can be used to update the TravelDocs application to handle misaligned pages.

“Pattern matching overview” on page 188

“Anchor objects setup” on page 190

“Geometric pattern matching” on page 191

“Text-based pattern matching” on page 194

“TravelDocs: Use geometric pattern matching to identify pages” on page 196

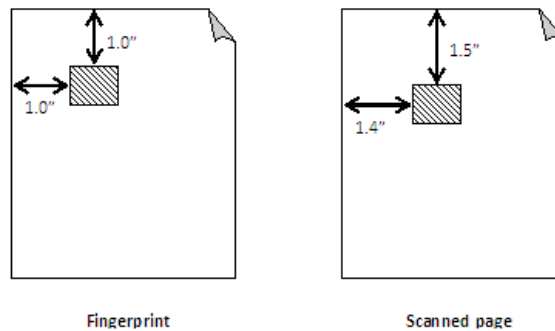
Pattern matching overview

Pattern matching uses anchor objects that you define on the page fingerprints. These anchor objects can be geometric patterns, like a page registration marks or vendor logos, or text-based patterns.

The anchor objects can act both as identification markers used during page identification and as reference points used during registration or realignment of the image.

The Datacap pattern matching actions analyze the runtime pages and look for geometric or text-based patterns that match the patterns in the page fingerprints. It is not unlike standard fingerprint identification, except that it uses only a selected region of the fingerprint image. However, you can use the difference between the location of the pattern on the fingerprint and its location on the runtime page to correct registration problems.

Whether you use geometric pattern matching or text-based pattern matching, the basic concept is the same and is illustrated in the following example. Here, the cross hatched region is the anchor object. In the fingerprint, the anchor is located 1.0 inches from the top and left edge of the page. In the scanned page, the anchor pattern is 1.5 inches from the top and 1.4 inches from the left edge of the page.



A misalignment of this magnitude almost certainly causes a fingerprint match to fail if you are using one of the standard techniques to match fingerprints. Pattern matching attempts to locate the anchor object and, if successful, computes the offsets that are required to bring the page back into alignment. As such, it can handle much larger registration problems. In the previous example, the image must be moved 0.4 inches to the left and 0.5 inches up, so the required image offset values are **-80, -100**.

Attention: Datacap processes pages at an effective resolution of 200 x 200 pixels per inch, so 0.4 inches is equivalent to 80 pixels and 0.5 inches is equivalent to 100 pixels.

When you are use geometric pattern matching with multiple anchor objects, Datacap can do interpolate realignment. The field positions are adjusted based on their proximity to each of the anchor objects.

Although geometric pattern matching and text-based pattern matching are conceptually the same, the implementations are slightly different and use different actions.

“Considerations for using pattern matching”

“Auto registration with the FindFingerprint action”

Considerations for using pattern matching

Some of the things you might consider for pattern matching include when to use for page identification and good anchor patterns.

When is pattern matching a good choice for page identification?

If your application must handle various page types that are similar, standard fingerprint identification (FindFingerprint) might generate mismatches. Pattern matching uses a smaller defined region of the page. You can select an area that is unique to each page type and thus avoid mismatches.

What makes a good anchor pattern?

For pattern matching to work, you must have a good anchor pattern. If you are using geometric pattern matching, the image must be composed of simple, solid regions. Images that contain different degrees of shading do not work well for geometric pattern matching.

For text-based pattern matching, the only requirement is that the text pattern is unique to the page type.

What types of pages typically require pattern matching for image registration?

The standard Fingerprint Matching action, FindFingerprint, can tolerate some minor page misalignment relative to the fingerprint image. For more information, see “Auto registration with the FindFingerprint action.” Pages that have more serious alignment problems or have inconsistently proportioned areas require pattern matching for registration. Faxed images are especially vulnerable because the sending and receiving fax machines might pull the paper through at different speeds, resulting in longer, or shorter images. Additionally, pages with OMR fields require accurate registration, especially if the boxes are closely spaced.

Is there a way to register pages manually?

In certain situations, you might want to do image registration manually. You can register images by using the AIndex web client. For details, see “Manual page identification and registration” on page 240.

Auto registration with the FindFingerprint action

When FindFingerprint detects a match, it automatically computes the offsets that are required to correct the image registration. It stores these offsets in the *Image_Offset* variable in the page. The ReadZones action uses the offsets when it sets the runtime field positions.

The following example shows the runtime data for a page that Datacap identified correctly and registered automatically by using FindFingerprint.

```
<P id="TM000003">
  <V n="TYPE">Air_Ticket</V>
  <V n="STATUS">49</V>
  <V n="IMAGEFILE">tm000003.tif</V>
  <V n="ScanSrcPath">c:\datacap\traveldocs\images\problem_images_page_3.tif</V>
  <V n="RecogStatus">0</V>
  <V n="Confidence">0.8689438</V>      <-- Confidence level high enough for a match
```

```

<V n="Image_Offset">-24,-20</V>          <!-- Offsets calculated automatically by FindFingerprint
<V n="TemplateID">566</V>                <!-- Matching fingerprint ID
<V n="Fingerprint_Created">No</V>
</P>

```

FindFingerprint calculates the offsets without using anchor objects. However, the offsets it can handle are small. The CalculateOffset action in the Autodoc library can increase the size that the offsets it can handle. Increasing the size that offsets handle can slow down the matching process.

Library	Action	Description
Autodoc	CalculateOffset	Sets the maximum offset supported when you are matching pages to fingerprints.

Anchor objects setup

An anchor object is a region of a fingerprint image that is used during pattern matching. If you use pattern matching to identify pages, you must specify a pattern that is unique to each fingerprint.

The process for setting up anchor objects is the same for a geometric pattern or a text-based pattern.

The coordinates of the anchor object and other information are stored in the document hierarchy. You must create a field-level object to store the details of the anchor object. Aside from the coordinates of the anchor object, the other key element is the **PatternMatch** variable, which identifies the object as a pattern match anchor. Additionally, you typically set the **STATUS** variable of the anchor object to **-1** so the field is not displayed during verification.

The following example shows the XML definition of an anchor field. The anchor is created as follows:

- In Datacap Studio Document Hierarchy pane, you added a field object that is called **Anchor_Object_1** to each page definition.
- The **PatternMatch** of the anchor object variable is set to **1** to identify the object as an anchor object.
- On the **Zones** tab, a recognition zone is drawn around the anchor on each of the three fingerprints in this example.

```

<F type="Anchor_Object_1">
  <V n="ID">0</V>
  <V n="TYPE">Field</V>
  <V n="STATUS">-1</V>          <!--STATUS = -1 keeps the anchor field hidden-->
  <V n="Position">0,0,0,0</V>
  <V n="MIN_TYPES">0</V>
  <V n="MAX_TYPES">0</V>
  <V n="ReqConf">8</V>
  <V n="rules"></V>
  <V n="PatternMatch">1</V>    <!--PatternMatch = 1 defines the object as an anchor-->
  <V n="Pos565">183,195,276,282</V> <!--Anchor object coordinates for fingerprint 565-->
  <V n="Pos566">636,185,730,284</V> <!--Anchor object coordinates for fingerprint 566-->
  <V n="Pos567">1094,185,1181,279</V> <!--Anchor object coordinates for fingerprint 567-->
</F>

```

“Confidence level setup for pattern matching”

Confidence level setup for pattern matching

The default confidence level for pattern matching is 8. You can set this value for geometric pattern matching by using the **PatternMatch_*** actions. Or you can set the value for text-based pattern matching by using the **pat_RecogMatch_Id** action.

Geometric pattern matching

You set required confidence level for geometric pattern matching in the **ReqConf** variable of the anchor object. You can change the default confidence level by right-clicking the anchor object in the Document Hierarchy pane and choosing **Manage Variables**.

Alternatively, you can change the **ReqConf** variable through the Properties pane in the **Zone** tab.

Text-based pattern matching

The **pat_RecogMatch_Id** action does not use the **ReqConf** variable. Instead, it uses the confidence level that is established by using the **SetMatchConfidence** action.

Library	Action	Description
PatternMatch	SetMatchConfidence	Sets the confidence threshold for pattern matching.

Geometric pattern matching

Geometric pattern matching uses graphical images, like page registration marks or vendor logos. You can use geometric pattern matching to identify pages and correct registration problems.

The following table identifies some of the key actions in the PatternMatch library that are used for geometric pattern matching.

Library	Action	Description
PatternMatch	PatternMatch_Identify	Identifies a page that uses geometric pattern matching, sets the page type and image offsets, and creates the page data file.
PatternMatch	pat_RegisterZones	Use after you run PatternMatch_Identify if you have multiple anchors on the page. This action adjusts the positions of all fields that are based on the positions of multiple anchor fields.

“How the PatternMatch_Identify action works”

“Multiple anchor objects” on page 192

“pat_RegisterZones action to adjust the positions of individual fields” on page 193

How the PatternMatch_Identify action works

When Datacap starts the PatternMatch_Identify action, it gathers all of the anchor objects from all of the fingerprints, then looks for a match on the current page.

The Datacap system does not search the entire page. Instead, it searches a region 200 pixels greater in each direction than the zone defined for the anchor object. If it finds a match that meets the required confidence level, it sets the page type and computes the offset values.

Tip: You can change the size of the search region by setting the anchor field's METRIC variable. For example, METRIC=200,300 increases the width by 200 pixels in each direction and the height by 300 pixels in each direction.

The following RRS log entries, which are taken from pageid_rrs.log, illustrate how PatternMatch_Identify works.

```
Created PatternMatch Object
Acquired PM lock
Loading Patterns...
  Vendor_Logo : Pattern Found for '565' with zone (171,194,563,302)
  Vendor_Logo : Pattern Found for '566' with zone (678,191,1044,296) 1
  Vendor_Logo : Pattern Found for '567' with zone (1187,206,1524,286)
Opening 'provider=microsoft.jet.oledb.4.0;data
source=C:\Datacap\TravelDocs\TravelDocs\Fingerprint.mdb;persist security info=false'
Fingerprint/Rules Database connection established.
Search Image: 'c:\datacap\traveldocs\batches\20110018.013\tm000001.tif'
Matching ID# 566 Conf: 10. 2 X: 240 Y: 271. Search Area: 478,0,1244,496 3 Field
ReqConf:8 --> TRUE
  Calculated offset is: (40,80) 4
Releasing PM lock
```

RRS log entry	Description
1	The PatternMatch_Identify action finds a Vendor_Logo pattern that is defined in each of three fingerprints: 565, 566, and 567.
2	It matches the pattern on fingerprint 566 to a region on the current page with a confidence level of 10 (the highest possible).
3	The search area for fingerprint 566 is 200 pixels greater in each direction than the zone defined for the anchor object.
4	It calculates the x and y offsets between the position of the anchor object and the position of the matching pattern on the page.

Multiple anchor objects

To improve pattern matching accuracy, you can specify multiple anchor objects. For example, by defining anchor objects at the upper left and lower right of the page, you can improve the resulting registration.

PatternMatch_Identify does only a simple averaging of the offsets. The pat_registerZones action can do interpolate registration, where field positions are adjusted based on their proximity to each of the anchor objects.

In the following example, Datacap located an anchor object at the upper left and the lower right of the page. Then, it calculated an offset value for each anchor. It averaged these values to determine the offset required to bring the page image into best alignment. It wrote these values to the *Image_Offset* variable of the page.

RRS log

```
Matching ID# 570 Conf: 10. X: 280 Y: 274. Search Area: 0,0,564,513 Field ReqConf:8 --> TRUE
Calculated offset is: (100,100) <-- Offset for first anchor object (top left)
Matching ID# 570 Conf: 10. X: 301 Y: 262. Search Area: 1158,1640,1700,2162 Field ReqConf:8 --> TRUE
Calculated offset is: (101,62) <-- Offset for second anchor object (bottom right)
```

Runtime page data

```
<P id="TM000018">
  <V n="TYPE">Test</V>
  <V n="STATUS">49</V>
  <V n="IMAGEFILE">tm000018.tif</V>
  <V n="ScanSrcPath">c:\datacap\traveldocs\images\refstopbottom.tif</V>
  <V n="RecogStatus">0</V>
  <V n="LC_Confidence">8.571231E-02</V>
  <V n="LC_Image_Offset">-16,0</V>
  <V n="LC_TemplateID">562</V>
  <V n="Fingerprint Created">No</V>
  <V n="Confidence">8.571231E-02</V>
  <V n="TemplateID">570</V>
  <V n="PatternConfidence">10</V>
  <V n="Image_Offset">-100,-81</V> <-- Average required offset
  <V n="DATAFILE">tm000018.xml</V>
</P>
```

In addition to storing the page image offset, PatternMatch_Identify also creates a page data file. It also stores the offset of the zone for each anchor in the *Zone_Offset* variable of the field.

```
<V n="Zone_Offset">100,100</V> <-- Offset for first anchor
```

pat_RegisterZones action to adjust the positions of individual fields

If you are using multiple anchors, you can use the pat_RegisterZones action to compute the optimum offsets for individual fields.

PatternMatch_Identify computes the offset for the entire page and stores the value in the *Image_Offset* variable of the page.

```
<V n="Image_Offset">-100,-81</V>
```

It also creates the page data file and stores the offset for each anchor in the *Zone_Offset* variable of the field.

```
<V n="Zone_Offset">100,10</V> <-- Offset for first anchor
```

Using the pat_RegisterZones action to compute handles situations where the difference between the fingerprint and the runtime image varies across the page.

Library	Action	Description
PatternMatch	pat_RegisterZones	Adjusts the positions of all fields on the current page that is based on the positions of the anchor fields of the page.

Important: The pat_RegisterZones action is an alternative to the ReadZones action. Do not use both on the same page.

The TravelDocs application runs the pat_RegisterZones action immediately after it runs the PatternMatch_Identify action.

The following RRS log entries, which are taken from `pageid_rrs.log`, illustrate how **pat_RegisterZones** works.

```
Created PatternMatch Object
Acquired PM lock
Anchor Anchor_1 found.          (1) Looking for offset...
Expected 1384,313,1529,392
Image_Offset
Zone_Offset 81,100              (2)
Anchor Anchor_2 found.          (3) Looking for offset...
Expected 697,1988,1006,2074
Image_Offset
Zone_Offset 102,101             (4)
Register using 2 anchors
Set Arrival_Date from Position to 411,405,713,484 to 507,506,812,585
Set Departure_Date from Position to 1154,412,1450,488 to 1246,513,1532,589 (5)
Set Total_Cost from Position to 1150,781,1331,860 to 1242,882,1417,961
```

(1) The `pat_RegisterZones` action locates an anchor object (`Anchor_1`) on the current page.

(2) It retrieves the zone offset for this field that was computed earlier by `PatternMatch_Identify`.

(3) It locates a second anchor object (`Anchor_2`) on the current page.

(4) It retrieves the zone offset for this field that was computed earlier by `PatternMatch_Identify`.

(5) It uses the two zone offsets to compute the new positions for each of the three data fields on the current page. It uses an algorithm that takes into account the proximity of each field to each anchor zone.

Text-based pattern matching

Text-based pattern matching works much like geometric pattern matching except that the anchor objects are text strings. You set up anchor fields in the document hierarchy and then define the anchor zones on each fingerprint.

You can view the properties for any of the text anchor zones on the Properties tab.

The following action supports page identification and image realignment by using text-based pattern matching:

Library	Action	Description
PatternMatch	pat_RecogMatch_Id	Identifies a page by using text_based pattern matching, and sets the page type and image offsets. This action uses the patterns (anchor objects) from all fingerprints in the fingerprint library.

“How the `pat_RecogMatch_Id` action works” on page 195

“Determine the runtime field positions by using anchor offsets” on page 196

“Field adjustment that is based on multiple anchors” on page 196

How the pat_RecogMatch_Id action works

When Datacap runs the pat_RecogMatch_Id action, it gathers all the anchor objects from the fingerprint library and looks for a match on the current page.

For each anchor object, Datacap searches the current page in a region 400 pixels greater in each direction than the text zone defined in the fingerprint. If it finds a match that meets the required confidence level, it sets the page type and computes the offset values.

Restriction: The METRIC variable does not change the size of the search region that is used by pat_RecogMatch_Id.

The following RRS log entries illustrate how pat_RecogMatch_Id works.

```
Created PatternMatch Object
Aquired PM lock
Opening 'provider=microsoft.jet.oledb.4.0;
    data source=C:\Datacap\TravelDocs\TravelDocsFingerprint.mdb;persist security info=false'
Fingerprint/Rules Database connection established.
#572, path:'C:\Datacap\TravelDocs\fingerprint\572.cco'
FPZone:'1384,313,1529,392' TXTZone:'1408,334,1498,359' Value:'Room' 1
FPZone:'697,1988,1006,2074' TXTZone:'758,2018,811,2036' Value:'Hotel #3'
-----
ANCHOR TEXT:'Room'--->'R[o00][o00]m' METRIC:'400,400' 2
SEARCH AREA:'Hotel #3
Room
Check out Wed Nov 24 2010
speed internet microwave fridge 3
$109 95
$329 85'
Matched Value >>Room<< 4
Check FingerPrintID# 572 Match Confidence: 9. Search Area: 1008,0,1700,759
Offset(-80,-100) 5
-----
ANCHOR TEXT:'Hotel #3' --->
'[Z2][o00][\(\)iIt11][o00][\ ]*H[\(\)iIt11][\(\)iIt11][\(\)iIt11][\(\)iIt11][o00]p[\ ]*H[o00]
[\(\)iIt11]e[\(\)iIt11]s' METRIC:'400,400'
SEARCH AREA:'Hotel #3' 6
Matched Value >>Hotel[SPACE CHARACTER]#3<<
Check FingerPrintID# 572 Match Confidence: 9. Search Area: 358,1618,1211,2200
Offset(-100,-101)
RecogMatch FingerPrint#:572 PAGETYPE:Room_Receipt 7
```

RRS log entry	Description
1	The action finds two anchor zones that are defined in fingerprint 572: Room and Hotel #3.
2	It computes a bounding region 400 pixels greater in each direction than the region (TXTZone) defined in the fingerprint CCO file for the first anchor value (Room).
3	It identifies all the text within that bounding region on the current page.
4	It locates the anchor value Room within the search region.
5	It computes the offset by comparing the word's position on the page to the position in the fingerprint.
6	It repeats the process for the second anchor value.
7	It sets the page's template ID and type because at least one of the zones matched.

Determine the runtime field positions by using anchor offsets

The **pat_RecogMatch_Id** uses the anchor offsets to determine the **Image_Offset** value it writes to the runtime page file.

The following example shows how the field positions are determined.

```
<V n="Image_Offset">-100,-101</V>
```

The value here (**-100, -101**) is from the same Hotel #3 page in “How the **pat_RecogMatch_Id** action works” on page 195. In this example, you can see that **pat_RecogMatch_Id** used the offset value from **Text_Anchor_2**.

Important: The **pat_RecogMatch_Id** action does not create a page data file and does not store the individual offset for each anchor field.

Later on, when Datacap runs the **ReadZones** action, it uses the **Image_Offset** value to compute the position of each runtime field, for example:

```
<F id="Arrival_Date">
  <V n="TYPE">Arrival_Date</V>
  <V n="Position">511,506,813,585</V>
  <V n="STATUS">0</V>
  etc.
```

Compare the field positions that are defined in the fingerprint, which you can see in the **Properties** pane, with the field positions in the runtime page file. You can see how Datacap used the offset value (**-100, -101**) to compute the position of each data field.

Field	Fingerprint 572	Runtime page
Arrival_Date	411,405,713,484	511,506,813,585
Departure_Date	1154,412,1450,488	1254,513,1550,589
Total_Cost	1150,781,1331,860	1250,882,1431,961

Field adjustment that is based on multiple anchors

The **pat_RegisterZones** action does not work on pages that were identified by using **pat_RecogMatch_Id**. The **pat_RecogMatch_Id** action does not save the individual anchor field offsets.

TravelDocs: Use geometric pattern matching to identify pages

You can update the TravelDocs application to use geometric pattern matching to identify pages and correct registration problems.

To update the TravelDoc application to use geometric pattern matching, you set up the pattern match and anchor objects and update the PageID rule. Then, you can run a batch through the workflow.

“Setting up the pattern match anchor objects”

“Updating the PageID rule to use pattern matching” on page 197

“Running a batch through the workflow” on page 198

“Reviewing the runtime batch files” on page 198

Setting up the pattern match anchor objects

You can create pattern matching zones for each of the air ticket pages. Use the vendor logo at the top of each page as the anchor object you are trying to match.

Before, you can define the zones, you must add a field to the document hierarchy for the anchor object.

Setting up the pattern match and anchor objects:

1. Click the Datacap Studio **Zones** tab.
2. In the Document hierarchy pane, click **Lock DCO for editing** and expand the **Flight** document.
3. Right-click the Air_Ticket page and choose **Add > Field**.
4. Rename the new field **Vendor_Logo** and then use **Up Arrow** to move it to the top of the list.
5. Right-click the **Vendor_Logo** field and choose **Anchor field**. The field is identified as an anchor object by setting the *PatternMatch* variable to 1.
6. In the Properties pane, set the *STATUS* variable to **-1**.
7. In the Document hierarchy pane, click **Save**. Leave the document hierarchy locked for editing.
8. In the Fingerprints pane, expand the **Flight** class and select the first air ticket page (Airline #1).
9. In the Document hierarchy pane, select the **Vendor_Logo** field. Use the mouse to draw a bounding box around the vendor logo on the page image.
10. Repeat for the remaining air ticket pages (Airline #2 and Airline #3).
11. In the Document hierarchy pane, click **Save** and then click **Unlock DCO**.

Updating the PageID rule to use pattern matching

You must update the PageID rule to identify unrecognized pages by using pattern matching.

You put the pattern matching function at the end so it runs only if standard fingerprint matching and text matching both fail. You are going to use a new sample image that has offsets large enough to make standard fingerprint matching fail. Most applications use one page identification method, this exercise is for illustration purposes only.

To update the PageID rule to use pattern matching:

1. Click the Datacap Studio **Rulemanager** tab.
2. In the Rulesets pane, select the **PageID** ruleset and click **Lock/Unlock ruleset** to lock the ruleset for editing.
3. Expand the **PageID** ruleset. Then, right-click the **PageID** rule and choose **Add Function**.
4. Rename the new function **Identify using Pattern Match**.
5. Click the **Actions library** tab.
6. Select and add each of the actions that are shown in the following table to the **Identify using Pattern Match** function by clicking **Add to function**. Then, set the action parameters as shown in the table.

Library	Action	Parameter
PatternMatch	PatternMatch_Identify	
rrunner	rrSet	varSource = GeometricPattern
		varTarget = @P.MatchType

Important: rrSet ("GeometricPattern", "@P.MatchType") stores the string GeometricPattern in a page variable that is called MatchType within the runtime hierarchy. You can see which pages were identified by which PageID function.

7. Add the rrSet action to the **Identify using Fingerprint** function and set the action parameter as shown.

Library	Action	Parameter
runner	rrSet	varSource = Fingerprint
		varTarget = @P.MatchType

8. In the Rulesets pane, click **Save**. Then, click **Lock/Unlock ruleset** and choose **Publish Ruleset**.

Running a batch through the workflow

You can test the pattern matching function. Datacap provided an extra page image file for an Airline #2 air ticket where the image is offset enough to cause a standard fingerprint match to fail.

To run a batch through the workflow:

1. Copy the file OffsetAirTicket.tif into the TravelDocs application images folder. Open the page and compare it to the original Airline #2 ticket. You see that the image is offset by 0.5 inches in both the x and y directions.
2. Click the Datacap Studio **Test** tab.
3. In the **Workflow** pane, select the **VScan** task profile under **Main Job**
4. Click **New** to start a new batch.
5. Click **Process rules for target object**. Wait for the VScan task profile to run, click **Advance** when it completes.
6. Click **Process rules for target object**. Wait for the Page ID task profile to run, click **Advance** when it completes.
7. On **Runtime batch hierarchy** tab, scroll to the bottom and make sure that the last page is identified with type **Air_Ticket**. Then, select it and confirm that the new Airline #2 ticket is displayed on the Image tab.
8. Click **Process rules for target object** and click **Advance** to move the batch through the remainder of the workflow.

Reviewing the runtime batch files

After you run the batch through the workflow, you can review the runtime batch files to ensure that they were processed correctly.

To review the runtime batch files:

1. Open the most recent batches folder of the TravelDocs application.
2. Open the file PageID.xml and scroll till end of the file to view the details for the last page.

```
<P id="TM000015">
  <V n="IMAGEFILE">tm000015.tif</V>
  <V n="ScanSrcPath">c:\datacap\traveldocs\images\offsetairticket.tif</V>
  <V n="RecogStatus">0</V>
  <V n="LC_Confidence">0.4774427</V>      <!--Confidence level from fingerprint matching
  <V n="LC_Image_Offset">-24,0</V>
  <V n="LC_TemplateID">569</V>          <!--Closest matching fingerprint
  <V n="Fingerprint Created">No</V>
  <V n="Confidence">0.4774427</V>
  <V n="TemplateID">566</V>             <!--Matching fingerprint based on pattern matching
  <V n="PatternConfidence">10</V>      <!--Confidence level from pattern matching
```

```

<V n="Image_Offset">-100,-100</V>          <!--Offset relative to fingerprint 566
<V n="MatchType">GeometricPattern</V>      <!--Page identified using geometric pattern matching
<V n="DATAFILE">tm000015.xml</V>
</P>

```

In this example, the closest matching fingerprint is fingerprint 569, one of the hotel pages. But the confidence level is only 0.477, which is not enough for a match. This result caused the FindFingerprint action to fail.

After the FindFingerprint action failed, Datacap ran the Identify using Text Match function. However, this function failed because the Airline #2 ticket does not include the word "Car." Datacap then ran the Identify using Pattern Match function and found an exact match with the fingerprint 566, the original Airline #2 fingerprint. The match is exact because the vendor logo, not the data, is identical on both pages. Open the export text file (C:\TravelDocs\export\batch_id.txt). You can see that Datacap located all of the data fields by using the offsets that it calculated during pattern matching and recognized the data successfully.

```

,U Airline #2,Newark, NJ (EWR),Dayton, OH (DAY),MON JAN 10, 2011,
Dayton, OH (DAY),Newark, NJ (EWR),THUR JAN 13, 2011,360.56,33.23,393.79

```

Workflow automation, routing, and automatic fingerprint generation

You can configure Rulerunner to monitor the job queue and run background tasks like PageID, Profiler, and Export automatically whenever batches are pending.

The manual workflow processing was linear so far. You moved each batch from task to task in the same sequence, VScan > PageID > Profiler, Export. The one exception was when a batch was diverted out of the standard workflow and into the FixUp task to address document integrity problems. You can also do conditional routing, which can route a batch to a specific job if it requires special handling. The example that is used is manual page identification, which works if the automated identification methods fail to recognize a page.

The last new task you to cover is automatic fingerprint generation. Automatic fingerprint generation is useful when you want operators to add new page types to the fingerprint library automatically, instead of defining them by using Datacap Studio. This way the fingerprint and recognition zones are saved and can be used next time that you need to process a page of the same type.

Then, you can configure the TravelDocs application to use Rulerunner for background task processing. You can also update the application to handle unidentified pages by routing the batch to a new web client interface called ProtoId for manual page identification. You add a function to automatically generate fingerprints for the pages.

“Use Rulerunner to automate background tasks” on page 200

“Conditional branching and splitting to route documents” on page 201

“Automatic fingerprint generation” on page 205

“TravelDocs: Automated background processing with Rulerunner” on page 206

“TravelDocs: Handle document integrity failures” on page 210

“TravelDocs: Identify pages manually” on page 212

“TravelDocs: Generating fingerprints automatically” on page 218

“TravelDocs: Splitting a document from the main batch” on page 222

Use Rulerunner to automate background tasks

You can use Rulerunner to automate the processing of background tasks for pending batches.

Up to now, you initiated each workflow task manually, typically by starting the task from the Datacap Web Client or from the Datacap Studio **Test** tab.

Production environments want to automate batch processing to the greatest extent possible. To achieve this goal, means identifying background tasks and running them automatically.

Background tasks are tasks that do not require operator intervention. The TravelDocs application has three background tasks:

- PageID - identifies each page and assigns a page type
- Profiler - combines pages into documents, and does recognition and validation tasks
- Export - writes structured data to a data repository

Each of these tasks runs a set of rules, updates the runtime batch hierarchy, and marks the task as complete. The Datacap queuing engine then readies the batch for the next task in the workflow. If the rules detect any problems, for example, the batch failed document integrity checking, the task can raise an error condition. You can divert the task out of the normal processing workflow for special handling.

Routing and exception handling are addressed later. For now, focus on moving a batch through the standard workflow with as little operator intervention as possible.

“Rulerunner overview”

“Rulerunner configuration”

“Rulerunner operation” on page 201

“Rulerunner logging” on page 201

Rulerunner overview

Rulerunner is the Datacap background processing engine.

You can configure Rulerunner to monitor the job queue and start designated background tasks automatically whenever batches are pending.

For example, in the TravelDocs application, the VScan task and the Verify task are manual tasks that require an operator. The other three tasks do not require any manual input, so you can run them in the background under the control of Rulerunner.

Rulerunner configuration

You must configure Rulerunner to be able to run background tasks without user intervention.

Configuring Rulerunner is a two-step process:

- Define the tasks that you want to run as background tasks in the Datacap Application Manager.
- Configure the background tasks in the Rulerunner Manager.

You do these steps for the TravelDocs application in the “Defining background tasks in Datacap Application Manager” on page 207 and “Setting up background tasks in Rulerunner Manager” on page 207 topics.

Rulerunner operation

Rulerunner is configured to monitor job queues for jobs to run in the background.

Rulerunner runs in the background as a Windows service. Although you can configure Rulerunner to work with multiple Datacap applications, consider the case where it is configured for one application.

Rulerunner monitors the job queue of the application that is looking for jobs it is configured to run. The Datacap Web Client **Monitor** tab is used to monitor the queue. But the actual job queue is maintained in the queue table in the Datacap engine database. By default, the engine database is a Microsoft Access database in the root of the application folder. For example, the engine database of the TravelDocs application is C:\Datacap\TravelDocs\TravelDocsEng.mdb.

Attention: Datacap supports other database types, including DB2, Microsoft SQL Server and Oracle.

Rulerunner polls the job queue every 10 seconds and looks for pending jobs. When it finds a pending job, Rulerunner processes the batch. Upon completion, the batch is ready for the next task in the workflow. In this way, you can move batches through any number of sequential background tasks automatically.

Rulerunner logging

Rulerunner provides extensive logging options that you can configure by using the Rulerunner Manager.

The Rulerunner log is needed when you use Rulerunner with the TravelDocs application. For more information, see “Enabling Rulerunner logging” on page 207 and “Analyze the Rulerunner log” on page 209.

Conditional branching and splitting to route documents

You can use conditional branching and splitting to route a batch or part of a batch to a separate job.

So far, most of the batch processing was done by going through the workflow linearly.

- VScan > PageID > Profiler > Verify > Export

The one exception was when you used a branch to divert a batch out of the standard workflow, between the Profiler task and the Verify task. Then, into the FixUp task to address document integrity problems.

You can also consider conditional branching and splitting and ways to route a batch or a portion of a batch to a separate job.

Remember: You cannot raise condition flags from tasks that are running under the Datacap Web Client. The tasks exit normally.

“Branching versus splitting” on page 202

“Condition flags” on page 202

“Defining a condition and the associated action” on page 203

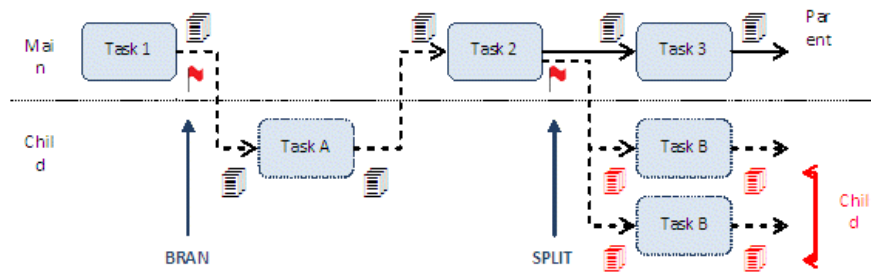
“Jobs to handle special conditions” on page 204

Branching versus splitting

The two basic methods for workflow routing are branches and splits.

When you use branching to route workflows, the entire batch is sent from the main job to a child job. When the child job completes, the batch returns to the main job.

When you use splitting to route workflows, documents in the batch are split off from the parent batch and placed into one or more child batches. The child batches are sent to a child job for processing and do not return to the main job.



In this diagram, Task 1 in the main job raises a branch condition and sends the entire batch to Task A in the child job. Task A then returns the batch to Task 2 in the main job. Task 2 in the main job raises a split condition and creates two child batches, which are sent to Task B in the child job. The parent batch continues to Task 3 in the main job.

Condition flags

Branching and splitting are initiated by using a condition flag that is raised during starting of a rule.

For branching, use the **Task_RaiseCondition** action to raise the condition flag of the task.

For splitting, use the **SplitBatch** action to raise the condition flag of the task implicitly.

Branching

In the TravelDocs application, the Batch Document Integrity Check ruleset, under the Document Integrity rule, contains the Batch Route To Fixup function and two related actions, **Task_NumberOfSplits** and **Task_RaiseCondition**. In “Document assembly” on page 49, you used the **Task_RaiseCondition** action to raise a condition flag that determines what happens when the task profile completes. For example, a Batch Route To Fixup function starts only if CheckAllIntegrity returns false.

- The **Task_NumberOfSplits** action is required and specifies the number of jobs to which the batch is sent before it returns to the main workflow (almost always 1).
- The **Task_RaiseCondition** action specifies the group index (almost always 0) and condition's index value. For example, the Profiler task can have one condition so the index for this condition is 0.

A batch can be diverted to the Fixup job so that an operator can fix the document integrity problem.

Remember: The branch does not occur until the current task profile is completed.

You update the TravelDocs application to branch when pages require manual identifications.

Splitting

The **SplitBatch** action implements batch splitting and also raises the condition flag.

Library	Action	Description
Split	SplitBatch	Creates one or more child batches that are based on the value of the specified document-level variable.

Attention: You must run the **SplitBatch** action at the batch level.

Unlike the branching action, the **SplitBatch** action does not require a condition index. The implied condition index is always 0 (the first condition). For example, a profile task can have three defined conditions, and the **SplitBatch** action always raises the first condition, which is Split Condition.

The **SplitBatch** action uses the document-level variable that is specified in the action parameter. This action determines whether a document is split off to a child batch or remains in the parent batch. In this example, you are using a document variable called *Split*.

```
SplitBatch("@D.Split")
```

This example means any document that has a variable *Split* with any value assigned is split off into a child batch. Furthermore, the value of the *Split* variable determines into which child batch the document goes. So, documents with `<V n="Split">1</V>` go into child batch 1 while documents with `<V n="Split">2</V>` go into child batch 2.

Attention: The values do not need to be numbers. Also, if the value of the variable is the same for all documents, then you get a single child batch.

You implement splitting for the TravelDocs application to split off documents that contain pages that were not recognized during page identification. For more information, see “Updating the Routing ruleset to split the batch” on page 222.

Defining a condition and the associated action

A task can have any number of conditions that are associated with it, although splitting uses only the first condition. Branching actions can use any of the conditions.

To define a condition and the associated action:

1. Start the Datacap Web Client, click the **Administrator** tab, and click **Workflow**.
2. Expand the job that contains the task, which includes the condition and associated action, and select the task.

3. In the **Selected task details** section, select **Router** from the **Mode** drop-down menu.
4. In the **Parameters** section, enter Return Conditions under the **Key** column, and enter the condition name under the **Value** column.
5. Click **Apply**.
6. Click the task that you created and in the **Selected condition details** section, and select or enter values for the corresponding fields, as needed:

Field	Description
Spawn type:	The following values are available: • Branch – sends the current batch to the specified job, then returns to the main workflow • Jump – skips the next <steps> tasks in the main workflow • Split – used with the SplitBatch action to send a child batch to the specified job • Stop – stops processing the batch (status = “batch stopped”)
Child Job:	Determines where the batch is sent. Used for Branch or Split only. For example, the batch is sent to Fixup Job .
Parent status:	The batch status when the batch returns. Can be Pending or Hold .
Child status:	Can be Pending or Hold .
Steps:	• When used with Jump: The number of workflow tasks to jump • When used with Branch: The return point after branching is completed (0 = same task, 1 = next task, and so on.)

7. Click **Save condition**.

For more detailed instructions, see “Adding the conditional branch to the PageID task” on page 215.

Jobs to handle special conditions

When you used branching, you sent batches with document integrity problems to the FixUp job. The FixUp job is generated automatically by the Datacap Application Wizard, so all you had to do was configure branching to use the existing job.

However, you might need to create a new job to handle a specific condition. For example, if you update the TravelDocs application to identify pages manually. Since there is no existing job to identify pages manually, you must create a job to do it. You can then branch to the new job if the batch contains unidentified pages and then return to the main job when the pages are identified.

A new job requires at least these items:

- A job definition with at least one task
 - A program that is associated with each task
- “Creating a job and task” on page 205

Creating a job and task:

You can create jobs and tasks to run on your workflow.

To define jobs and tasks:

1. On the Datacap Web Client, click the **Administrator** tab.
2. Open the **Workflow** page.
3. Create a job.
 - a. Select the workflow for which you want to create a job. Typically, there is only one workflow with the same name as the application.
 - b. Click **New** and enter the job name.
4. Create a task.
 - a. Click the parent job and click **New**.
 - b. Define the task by entering values for the following fields:

Field	Description
Name:	The task name
Description:	Description of the task. For example, a Verification task might have a description of Verify with Rule Validation.
Mode	The following modes are available: • Batch creation: Typically for Scan or virtual scan (VScan) tasks • Router: Enables job routing (conditional branching) for the corresponding task • Normal: Typically for other tasks where conditions are inapplicable, including such tasks as page identification, verification, and export.
Queue by:	Defines which user and station can process a batch that completes this task. The default is None , which means that there are no restrictions.
Store:	Determines whether to save the user ID or Station ID that completed the task.
Program:	From the drop down menu, select a program, such as Datacap Desktop , Rulerunner , or select a web page (.aspx file) for the task.

- c. Click **Setup** to specify more options if necessary.
- d. Or click **Create Setup** to create a default task setup.

Automatic fingerprint generation

You can add a function to your application to generate fingerprints automatically from unrecognized pages.

You can add automatic fingerprint generation function to the Verify task so that an operator can complete the task. However, you can route a document with unidentified pages to a supervisor for fingerprint generation. By either method, the fingerprint and recognition zones are saved and are used the next time that you encounter a page of the same type.

The following are the basic steps for generating fingerprints automatically:

- Identify the page type, either manually or by using a text-based identification technique.
- Display the page to an operator and have the operator define the recognition zones.
- Use the **CreateFingerprint** action to create a new fingerprint file from the current page image.
- Use the **SetFingerprint** action to set the class and type for the new fingerprint.
- Use the **iloc_SetZones** action to add the recognition zone position information to the document hierarchy.

The following are the new actions:

Library	Action	Description
AutoDoc	CreateFingerprint	Creates a fingerprint from the current page. The fingerprint consists of two files: the image (TIFF) file and the fingerprint processing (CCO) file.
AutoDoc	SetFingerprint	Sets the new fingerprint's class and type.
Intellocate	iloc_SetZones	Writes the recognition zone coordinates from the current page data file to the Position properties of the corresponding field objects in the document hierarchy XML file.

For details about configuring the TravelDocs application to generate fingerprints automatically, see the topic “Creating the AutoFingerprint ruleset” on page 219.

TravelDocs: Automated background processing with Rulerunner

You can update the TravelDocs application to run automated background processing by using Rulerunner.

To update the TravelDoc application to use automated background processing, you must define the background tasks and set up these tasks in Rulerunner Manager. Then, you can enable logging, set up the Job Monitor and run a batch through the workflow.

“Defining background tasks in Datacap Application Manager” on page 207

“Setting up background tasks in Rulerunner Manager” on page 207

“Enabling Rulerunner logging” on page 207

“Setting up the Job Monitor” on page 208

“Running a batch through the workflow” on page 208

“Analyze the Rulerunner log” on page 209

“Disabling Rulerunner logging” on page 209

Defining background tasks in Datacap Application Manager

To automate background processing for TravelDocs, you must first define the background processing tasks in Datacap Application Manager.

To define background tasks in Datacap Application Manager:

1. In the **Start** menu click **IBM Datacap Services> Datacap Application Manager**.
2. Select the **TravelDocs** application and click the **Rulerunner** tab.
3. Use the **Add new task** button to create three task profiles - one for each background task:

Task	Task profile
PageID	PageID
Profiler	Profiler
Export	Export

4. Close the Datacap Application Manager.

Setting up background tasks in Rulerunner Manager

After you define the background processing tasks, you must first set up these tasks in Rulerunner Manager.

To set up background tasks in Rulerunner Manager:

1. In the **Start** menu click **IBM Datacap Services> Rulerunner Manager**.
2. Click the **Rulerunner Login** tab.
3. Select **Datacap Authentication** and enter **User ID: admin**, **Password: admin**, and **Station ID: 1**.
4. Click **Connect**. If you receive a message that indicates the configuration file does not exist, click **Yes** to create the configuration file.
5. Click the **Workflow: Job: Task** tab and select the top-level **TravelDocs** item. Then, under **Main Job**, select **PageID**, **Profiler**, and **Export**.
6. Right-click inside the empty right pane and choose **Threads > Add Thread**.
7. Drag the TravelDocs application **Main Job** from the left pane onto **<thread0>** in the right pane.
8. Click **Save** and then click **Yes** to create the configuration file.

Enabling Rulerunner logging

You can enable Rulerunner logging to create log files from which you can monitor processing and troubleshoot problems.

To enable Rulerunner logging:

1. In the Rulerunner Manager window, click the **Logging** tab and then click the **Rulerunner Log** tab at the bottom of the window.
2. Select the **Output to** option and leave the target folder set to **C:\Datacap**.
3. Click **Save**.
4. Click the **Rulerunner Login** tab and click **Disconnect**.

Important: You must be connected to Datacap Server to configure Rulerunner. When you finish configuring Rulerunner, disconnect from the Datacap Server before you start the Rulerunner service.

Setting up the Job Monitor

You set up the Job Monitor to monitor the status of your batch as the automated background processing tasks run on Rulerunner.

To set up the Job Monitor:

1. If the Datacap Web Client is not running, start your web browser and enter the URL `http://localhost/tmweb.net`.
2. Log in to the Datacap Web Client.
 - a. Select the **TravelDocs** application.
 - b. In the **User ID** and **Password** fields, enter `admin`.
 - c. In the **Station** field, enter `1`.
3. Click the **Monitor** tab and select **Job Monitor**.
4. In the **Refresh rate** field, right-click and select **10 seconds**.

Running a batch through the workflow

You can run the batch through the workflow to test the automated background processing results. Use the Job Monitor and the Rulerunner log file to review the results and troubleshoot any problems.

To run a batch through the workflow:

1. Start Datacap Desktop by clicking **IBM Datacap Clients > Datacap Desktop**.
2. Log in to Datacap Desktop:
 - a. Enter `TravelDocs` for the application.
 - b. In the **User ID** and **Password** fields, enter `admin`.
 - c. In the **Station** field, enter `1` and click **Login**.
 - d. In the **Shortcut** field, select **VScan** from the menu.
3. In Datacap Desktop, browse to the image files in `../TravelDocs/images`, select a file, and click **Scan**.
4. When the Datacap Desktop VScan task is complete, click **Done**.
5. Confirm that the Datacap Web Client **Job Monitor** and Rulerunner Manager are still running.
6. Arrange the **Job Monitor** and Rulerunner Manager so that both are visible on your desktop.
7. In Rulerunner Manager, click the **Rulerunner** tab and click **Start**.
8. Watch the status of the batch in the **Job Monitor**. The **Task** and **Status** fields go through the following sequence. You might not see each step because of the 10-second refresh interval and the Rulerunner 10-second polling interval.

Sequence	First	Second	Third
Task	PageID (Status pending > running)	Rulerunner (Status pending > running)	Verify (Status pending)

The batch is now ready for verification. Because verification is a manual step, you must complete this task before Rulerunner can run the Export task.

9. Verify the batch as you did before by using Datacap Desktop or the Datacap Web Client.
 - When you reach the page with the invalid car type, set the type to **Other**.
 - If a validation failure message is displayed, click **OK** to override and continue.
 - When you reach the end, click **OK** to finish the batch.

- Switch to the **Job Monitor** and watch the status of the batch as Rulerunner runs the Export task.

Task:	Export
Status:	pending > running > Job done

Analyze the Rulerunner log

After you run the batch through the workflow, you can analyze the Rulerunner log file to review the results and troubleshoot any problems.

Rulerunner generates a separate log file for each active thread. The base product licensing allows single threading only and the log file is C:\Datacap\rulerunner0.log. These excerpts from the Rulerunner log illustrate the sequence of events.

```
ExecuteCode: Grabbed Job[Main Job]:Task[PageID] Queue Id:[42]. <--- Grab batch for PageID
RunGrabbedQNRelease: BatchID [C:\Datacap\TravelDocs\batches\20110067.005].
RunGrabbedQNRelease: Project Path set
to:[C:\Datacap\TravelDocs\dco_TravelDocs\assemble.set.xml]
RunGrabbedQNRelease: Selected pagefile [rrsvscan.xml]. <--- Read input DCO file
RunGrabbedQNRelease: Read page file
[C:\Datacap\TravelDocs\batches\20110067.005\PageID.xml]. <--- Create output DCO file
RunGrabbedQNRelease: RRS run successful.
RunGrabbedQNRelease: ProcessedPages in Batch:[15]
RunGrabbedQNRelease: Job[Main Job]:Task[PageID] queue id: [42] ran batch[20110067.005]
and the status is [finished]. <--- PageID complete
ReleaseTheQ: Released batch, status is now [pending]. <--- Batch pending for Rulerunner
```

```
ExecuteCode: Grabbed Job[Main Job]:Task[Profiler] Queue Id:[42]. <--- Grab batch for Profiler
RunGrabbedQNRelease: BatchID [C:\Datacap\TravelDocs\batches\20110067.005].
RunGrabbedQNRelease: Project Path set
to:[C:\Datacap\TravelDocs\dco_TravelDocs\Profiler.set.xml]
RunGrabbedQNRelease: Selected pagefile [PageID.xml]. <--- Read input DCO file
RunGrabbedQNRelease: Read page file
[C:\Datacap\TravelDocs\batches\20110067.005\Rulerunner.xml]. <--- Create output DCO file
RunGrabbedQNRelease: RRS run successful.
RunGrabbedQNRelease: ProcessedPages in Batch:[15]
RunGrabbedQNRelease: Job[Main Job]:Task[Profiler] queue id: [42] ran
batch[20110067.005] and the status is [finished]. <--- Rulerunner complete
ReleaseTheQ: Released batch, status is now [pending]. <--- Batch pending for Verify
```

```
ExecuteCode: No batches to process, sleeping for [10] seconds.
ExecuteCode: No batches to process, sleeping for [10] seconds. <--- Monitoring queue during
ExecuteCode: No batches to process, sleeping for [10] seconds. manual Verify task
ExecuteCode: No batches to process, sleeping for [10] seconds.
```

```
ExecuteCode: Grabbed Job[Main Job]:Task[Export] Queue Id:[42]. <--- Grab batch for Export
RunGrabbedQNRelease: BatchID [C:\Datacap\TravelDocs\batches\20110067.005].
RunGrabbedQNRelease: Project Path set
to:[C:\Datacap\TravelDocs\dco_TravelDocs\Export.set.xml]
RunGrabbedQNRelease: Selected pagefile [Verify.xml]. <--- Read input DCO file
RunGrabbedQNRelease: Read page file
[C:\Datacap\TravelDocs\batches\20110067.005\Export.xml]. <--- Create output DCO file
RunGrabbedQNRelease: RRS run successful.
RunGrabbedQNRelease: ProcessedPages in Batch:[15]
RunGrabbedQNRelease: Job[Main Job]:Task[Export] queue id: [42] ran batch[20110067.005]
and the status is [finished]. <--- Export complete
ReleaseTheQ: Released batch, status is now [Job done]. <--- Batch at end of workflow
```

Disabling Rulerunner logging

Rulerunner writes to the log file every 10 seconds while it is running. You need to enable logging only at specific times for troubleshooting.

To disable Rulerunner logging:

- In the Rulerunner Manager window, click the **Rulerunner** tab and click **Stop**. Then, wait for the Rulerunner Service to stop. If you get a timeout message, click **OK**.
- Click the **Rulerunner Login** tab and click **Connect**.

3. Click the **Logging** tab and click the **Rulerunner Log** tab at the bottom of the window.
4. Clear the **Output to folder** option and click **Save**.
5. Click the **Rulerunner Login** tab and click **Disconnect**.
6. Click the **Rulerunner** tab and click **Start** to restart the Rulerunner Service.

TravelDocs: Handle document integrity failures

After you implement recognition and validation, you must run the document creation and integrity checking tasks in their own task profile to handle document integrity failures.

In the Document Assembly topics, the TravelDocs application completes recognition and validation before it branches to the FixUp job. This routing was not a problem because you did not implement recognition and validation. Now that recognition and validation are implemented, some pages have **Status = 1**, which indicates low confidence values or validation errors. The Datacap Desktop FixUp task does not finish a job when there are pages with **Status = 1**.

You must move the document creation task and the integrity checking task out of the Profiler task profile and into their own task profile. Now, you can correct any batch integrity problems before the recognition and validation tasks are run.

“Moving document creation and integrity checking into the PageID task profile”

“Creating the CreateDocs task”

“Configuring Rulerunner to run CreateDocs” on page 211

“Running a batch through the workflow” on page 211

Moving document creation and integrity checking into the PageID task profile

You can move the document creation and integrity checking tasks into the PageID task profile to handle document integrity failures before recognition and validation are run.

To move document creation and integrity checking into the PageID task profile:

1. Start Datacap Studio and open the TravelDocs application.
2. On the **Rulemanager** tab, in the **Task Profiles** pane, click **Unlock task profiles**.
3. Expand the **Profiler** task profile and delete the **CreateDocs** and **Document Integrity** rulesets by clicking **Remove**.
4. Click **Add a new task profile**, select **Custom**, enter CreateDocs, and click **OK**.
5. Select the new **CreateDocs** task profile and add the **CreateDocs** and **Document Integrity** rulesets by selecting them in the **Rulesets** pane and click **Add ruleset to profile** at the left of the **Task Profiles** pane.
6. Click **Save** and then click **Lock task profiles**.

Creating the CreateDocs task

The existing Profiler task contains the job routing function that you need for the CreateDocs task, so you must copy and modify the existing task.

To create the CreateDocs task:

1. In the Datacap Web Client, click the **Administrator** tab, and click **Workflow**.
2. Expand **Main Job**, select **Profiler**, and click **Copy**.
3. Change the name of **Copy of Profiler** to CreateDocs.

4. Select the new **CreateDocs** task and click the up arrow to move the task between the **PageID** task and the **Profiler** task.
5. With the **CreateDocs** task selected:
 - a. Change the **Description** field to Create documents.
 - b. Select **Normal** as the **Mode**.
 - c. Leave **Queue by** and **Store** set to **None**.
 - d. Under **Parameters**, select **Rulerunner** for the **Program** and clear any values for any additional parameters if they are present.
 - e. Click **Create Setup**.
 - f. Click **Apply**.
6. Click the **Shortcuts** tab and click **New** to create a new shortcut.
7. In the **Name** field, enter CreateDocs, and Create Documents for the **Description**.
8. For the **Mode**, select **Manual for Hold**.
9. Click **Done**.
10. Click the **Permissions** check box. The **CreateDocs** task is now available on the Datacap Web Client **Operations** tab.
11. Click **Save**.

Configuring Rulerunner to run CreateDocs

CreateDocs is a background task, so you must configure Rulerunner to run it automatically whenever a batch is pending.

To configure Rulerunner to run CreateDocs:

1. Open the Rulerunner Manager window.
2. If Rulerunner is running, click **Stop** and wait for the service to stop.
3. Click the **Rulerunner Login** tab and click **Connect**.
4. Click the **Workflow: Job: Task** tab and select the **TravelDocs** application.
5. Under **Main Job**, select the **CreateDocs** task and drag it to **<thread0>**.
6. Click **Save**.
7. Click the **Rulerunner Login** tab and click **Disconnect**.
8. Click the **Rulerunner** tab and click **Start** to restart the service with the new settings.

Running a batch through the workflow

To introduce a document integrity problem, add an optional insurance page to the end of the batch. This orphan page causes the document integrity process to raise an error condition and Datacap routes the batch to the FixUp task.

After you fix the document integrity problem by deleting the page, processing can continue as normal.

Remember: The goal here is to demonstrate the routing capabilities of Datacap. So, deleting the problem page is acceptable. Typically, the corrective action is specified in the business requirements.

To run a batch through the workflow:

1. Open C:\Datacap\TravelDocs\images.
2. Delete the files CarRental.tif and OffsetAirTicket.tif (the files that you used for text and pattern matching).

3. Make a copy of Images_Page_02.tif (the first optional insurance page) and name the copy Images_Page_14.tif to create an orphaned insurance page.
4. In the Rulerunner Manager window, confirm that the Rulerunner Service service is running.
5. Start and log in to the Datacap Web Client, and click **VScan** on the **Operations** tab.
6. In the VScan window, click **Browse**, go to C:\Datacap\TravelDocs\images. Then, select a file and click **Open**, then click **Scan**.
7. When the VScan task displays a message to indicate that the scanning is complete, click **OK**, click **Done** then click **OK** and **Stop**.
8. On the Datacap Web Client **Operations** tab, click **Upload**.
9. When the Upload task displays a message to indicate that the task is complete, click **OK** and then click **Stop**.
10. In the Datacap Web Client, click the **Monitor** tab to display the Job Monitor page, click **Job Monitor** to display the job queue.
11. Wait until the Profiler task picks up and finishes the pending batch, and passes the batch to the CreateDocs task.
The CreateDocs task raises a condition flag and routes the batch to the FixUp task (a manual task that Rulerunner cannot process).
12. Click the **Operations** tab and click **Fixup**.
13. Select the batch with the FixUp task that is pending, and click **Yes** in the dialog to confirm that you want to start the selected batch. The batch opens in the Datacap Web Client FixUp window. The last document is selected and the **Comments** field indicates that the document has an invalid member (the orphaned insurance page).
14. Select the page **TM00001** and click **Delete**. Then, click **OK** to confirm. Datacap deletes the page and the parent document.
15. Click **Finish** and then click **OK** in the Task Finished message box. The FixUp task now has status of Job done and the batch is now pending for the Profiler task.
16. Wait until the Profiler task picks up the pending batch and processes the batch. When Profiler is completed, the batch is pending for the Verify task.
17. Open the pending batch in Datacap Desktop or Datacap Web Client and submit the batch as before. Then, switch to the Job Monitor window and watch the status of the batch as Rulerunner completes the Export task.
18. Delete the file Images_Page_14.tif from the images folder so you do not encounter this problem again.

TravelDocs: Identify pages manually

You can implement another conditional branch to handle the situation where you must identify pages manually.

Currently, the application implements three page identification techniques:

- Fingerprint matching
- Text matching
- Pattern matching

Here, you add another function to the PageID ruleset to manage pages that are still unidentified. If a batch includes unidentified pages, the PageID ruleset raises a condition flag. It sends the batch to the ManualPageID task, where the operator can set the page type manually.

- “Adding a function for manual page identification”
- “Updating the Recognize Page ruleset”
- “Adding the conditional branch to the PageID task” on page 215
- “Creating the ManualPageID job and task” on page 215
- “Configuring branching and creating a shortcut” on page 216
- “Configuring the Routing ruleset to handle manually identified pages” on page 216
- “Running a batch through the workflow” on page 217
- “Recognizing the data on the unidentified page” on page 218

Adding a function for manual page identification

You must add the function Identify Manually to configure TravelDocs to do manual page identification.

To add a function for manual page identification:

1. In the Datacap Studio Rulesets pane, select the **PageID** ruleset and click **Lock/Unlock ruleset**. Then, expand the PageID ruleset to view the two rules.
2. Right-click the PageID rule and choose **Add Function**. Rename the new function **Identify Manually**.
3. Add the actions and parameters that are shown in the following table to the PageID > Identify Manually function.

Library	Action	Parameter
runner	rrSet	varSource = Manual
		varTarget = @P.MatchType
DCO	SetPageStatus	1
runner	Task_NumberOfSplits	1
runner	Task_RaiseCondition	0,0

Attention: Task_RaiseCondition(0,0) references the first condition (index = 0) in the PageID task. You add the condition when you update the Recognize Page ruleset.

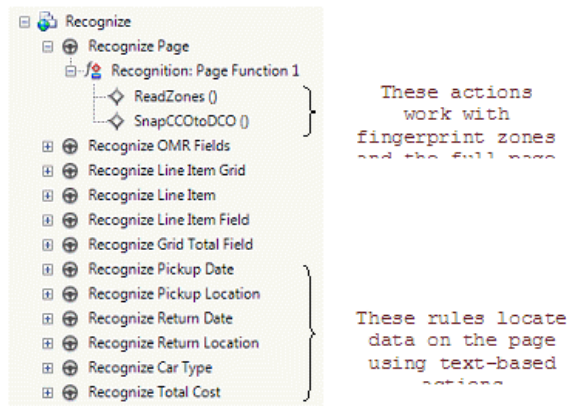
4. In the Rulesets pane, click **Save**. Then, click **Lock/Unlock ruleset** and choose **Publish Ruleset**.

Updating the Recognize Page ruleset

You must update the Recognize ruleset to handle pages that are identified manually.

Currently, the ruleset works for pages that are identified by using fingerprint matching, text matching, or pattern matching.

- For pages that are identified by using fingerprint matching or pattern matching, you use **ReadZone** and **SnapCCOtoDCO** to write the recognition data into the runtime hierarchy.
- For rental agreement pages that are identified by using text matching. You can use various text-based matching actions to locate the recognition data and write it to the runtime hierarchy.



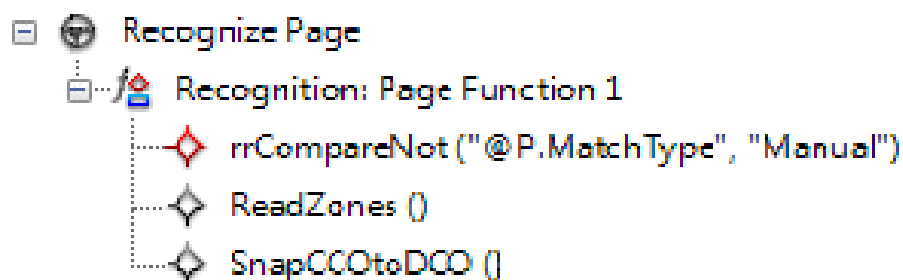
Datacap starts the Recognize Page rule on all page types, even though the rule works only when the current page has a valid matching fingerprint. With manual page identification, there is no matching fingerprint. When fingerprint matching fails, Datacap writes the ID of the closest match into the **LC_TemplateID** variable of the page. The **ReadZones** action uses the recognition zones from this low confidence fingerprint even though they are not valid. To work around this problem, you add an action to the Recognize Page rule to enable the rule to exit if the identification method is manual. Later, when you open the page for verification, you define the recognition zones manually.

To update the Recognize Page ruleset:

1. In the **Rulesets** pane, select the **Recognize** ruleset and click **Lock/Unlock ruleset**. Then, expand the **Recognize** ruleset and the **Recognize Page** rule.
2. Add the following action and parameters to beginning of **Recognize Page > Recognition: Page Function 1**.

Library	Action	Parameter
runner	rrCompareNot	object1 = @P.MatchType
		object2 = Manual

The finished rule is shown in the following example:



If a page has a match type of **Manual**, the function exits without starting **ReadZones**.

3. In the **Rulesets** pane, click **Save**.
4. Click **Lock/Unlock ruleset** and choose **Publish Ruleset**.

Adding the conditional branch to the PageID task

When the rules to handle manual page identification are complete, you can configure the PageID task. Configure the task to branch to the manual page identification task if there are unidentified pages.

To add the conditional branch to the PageID task:

1. Start the Datacap Web Client, select the **TravelDocs** application, and log in with the Admin account.
2. In the Datacap Web Client, click the **Administrator** tab and click **Workflow**.
3. Expand **Main job** and select the **PageID** task.
4. In the **Selected task details** section, select **Router** from the menu for the **Mode**.
5. In the **Parameters** section, enter Return Conditions under the **Key** column, and enter Page Identification Failed under the **Value** column.
6. Click **Apply**.

Tip: To see the new **Page Identification Failed** condition, you might need to refresh the page by clicking another tab, and returning to the Workflow page. You only created the condition node for the PageID task. Before, you can configure the branch, must create the job to use for manual page identification. Complete the steps that are identified in “Jobs to handle special conditions” on page 204.

Creating the ManualPageID job and task

You must create a job and task to be run on the workflow for manual page identification.

To create the ManualPageID job and task:

1. Start the Datacap Web Client, select the **TravelDocs** application, and log in with the Admin account.
2. In the Datacap Web Client, click the **Administrator** tab and click **Workflow**.
3. Select the main **TravelDocs** node and click **New** to create a new job node.
4. Enter or select values in the fields for the new job as shown in this table:

Field	Value
Name:	ManualPageID Job
Description:	Manual Page Identification Job
Priority:	1
Parameters:	Leave this field empty.

5. Click **Apply**.
6. Select the **ManualPageID Job** and click **New** to create a new task.
7. Enter or select values in the fields for the new task as shown in this table:

Field	Value
Name:	ManualPageID
Description:	Manual Page Identification task
Mode:	Normal
Queue by:	None
Store:	None

Field	Value
Program:	ProtoID.aspx

8. Click **Apply**.

Configuring branching and creating a shortcut

You configure branching to route batches to do manual page identification.

To configure branching and create a shortcut:

1. Start the Datacap Web Client, select the **TravelDocs** application, and log in with the Admin account.
2. In the Datacap Web Client, click the **Administrator** tab and click **Workflow**.
3. Expand **Main job** and expand the **PageID** task.
4. Click the **Page Identification Failed** condition and in the **Selected condition details** section, enter the following values for the corresponding fields:

Field	Description
Spawn type:	Branch
Child Job:	ManualPageID Job
Parent status:	Pending
Child status:	Pending
Steps:	1

5. Click **Save condition**.
6. Datacap Web Client **Administrator** tab, click **Shortcuts** and then click **New**.
7. In the **Selected shortcut details** section:
 - a. Enter these values for the following fields:

Field	Description
Name:	ManualPageID
Description:	Manual Page Identification
Mode:	Manual for Hold

- b. Under **Permissions**, click the **ManualPageID** check box. Confirm that the other check boxes are cleared.
- c. Click **Save**.

Configuring the Routing ruleset to handle manually identified pages

You must configure the Routing ruleset to handle manually identified pages.

You configured the application to display only pages with Status = 1 to the operator. Since manually identified pages have no recognition data, there are no low confidence characters to set the page status to 1. Depending on the way your validation rules are constructed, you might also have no validation errors.

To ensure that manually identified pages are displayed to an operator, you force the page status for the pages to 1. You force the status in the Routing ruleset, which runs immediately after validation.

To configure the Routing ruleset to handle manually identified pages:

1. In the Datacap Studio Rulesets pane, select the **Routing** ruleset and click **Lock/Unlock ruleset**. Then, expand the ruleset to view the rule.
2. Right-click the **Routing Rule 1** rule and choose **Add Function**. Rename the new function Set Manual Page Status.
3. Use **Up Arrow** to move the new function to the beginning of the rule.
4. Add the following action and parameters to the Routing Rule 1 > Set Manual Page Status function.

Library	Action	Parameter
runner	rrCompare	object1 = @P.MatchType
		object2 = Manual
DCO	SetPageStatus	1

Important: Make sure that Set Manual Page Status is the first function in the rule.

5. In the Rulesets pane, click **Save**.
6. Click **Lock/Unlock ruleset** and select **Publish Ruleset**.

Running a batch through the workflow

Datacap includes an image file for a new air ticket page that you can use to test manual page identification.

Because the image file page fails fingerprint matching, text matching, and pattern matching, Datacap starts the Identify Manually function. This function causes the batch to branch to the Manual Page ID task. After you manually identify the page, processing continues as normal.

To run a batch through the workflow:

1. Copy the file NewAirline.tif from C:\Datacap\TravelDocs\images\New to C:\Datacap\TravelDocs\images\ folder.
2. Confirm that the file Images_Page_14.tif is no longer in the images folder. Delete Images_Page_14.tif, if necessary.
3. Confirm that Rulerunner Service is running. If necessary, click **Start** on the Rulerunner tab in Rulerunner Manager.
4. In the Datacap Web Client, click **VScan** in the **Operations** tab. When the task completes, click **OK** and then click **Stop**.
5. On the Datacap Web Client **Operations** tab, click **Upload**.
6. When the Upload task displays a message to indicate that the task is complete, click **OK** and then click **Stop**.
7. Click the **Monitor** tab to display the **Job Monitor** page.
8. Wait until Rulerunner processes the pending batch. The PageID task raises a condition flag and routes the batch to the ManualPageID task.
9. On the Datacap Web Client **Operations** tab, click the **ManualPageID** shortcut and wait for the page images to load. Then, scroll till end of the page.
For information on other features in the ProtoId web client, see “Manual page identification and batch restructuring with ProtoId” on page 245.
10. Click the drop-down list beneath the last page and choose **Air_Ticket**.
11. Click **Done** and then click **OK > Stop**.
12. Return to the **Job Monitor** page. The ManualPageID task now has a status of Job done and the batch is now pending for the CreateDocs task.

13. Wait until Rulerunner runs the pending batch through the CreateDocs task and Profiler task. When Rulerunner completes the tasks, the batch is pending for the Verify task.

Recognizing the data on the unidentified page

Airline #4 page is not associated with a valid fingerprint and you did not create any text-based rules for air ticket pages. The new page does not have any recognition data. You recognize the data during verification.

You use the Datacap Web Client to recognize the data. But you cannot use Datacap Desktop.

To recognize the data on the unidentified page:

1. If necessary, start Datacap Web Client and log in to the **TravelDocs** application.
2. Click the **Verify** shortcut to open the pending batch.
3. Go through the batch as you did previously until you reach the Airline #4 page.
4. Click the **Outbound_From** field, and then, use the mouse to draw a box around the field in the image pane.
When you release the mouse button, the web client inserts the recognition data into the grid.
5. Repeat for the other fields on the page.
6. Click **Verify**. Datacap displays a message to indicate that the validations passed.
7. Click **Submit** to submit the page, and then click **OK** to finish the batch.
8. Click **OK** and then click **Stop**.
9. Click the **Monitor** tab and open the **Job Monitor** page to watch the status of the batch as Rulerunner runs the Export task.
10. When the export task completes, open the most recent text (.txt) file in the C:\Datacap\TravelDocs\export folder to see the information for the Airline #4 page.
,,Okron/Canton, OH (CAK),Washington, DC (DCA),14MAR11,
Washington, DC (DCA),Okron/Canton, OH (CAK),17MAR11,313.17,64.56,377.73

TravelDocs: Generating fingerprints automatically

You configured the TravelDocs application for manual page identification and drew bounding zones around each field to obtain the recognition data. But you did not create a new fingerprint or save the recognition zones.

Now you can update the application to generate fingerprints for unrecognized pages. When you are finished, run the same batch through the workflow. But this time the application adds a fingerprint to the fingerprint library and the recognition zones to the document hierarchy. The next time that the application encounters an Airline #4 page, the application can use the new fingerprint. Automatic page identification and field recognition is completed by using the new fingerprint.

“Creating the AutoFingerprint ruleset” on page 219

“Assigning the rule to each page type” on page 220

“Adding the ruleset to the Verify task profile” on page 220

“Enabling logging for Datacap Web Client” on page 220

“Running a batch through the workflow” on page 220

“Reviewing the RRS log file” on page 221

Creating the AutoFingerprint ruleset

You must create the AutoFingerprint ruleset with the actions that enable automatic fingerprint generation.

To create the AutoFingerprint ruleset:

1. In the Datacap Studio Rulesets pane, right-click the **TravelDocs** application and choose **Add Ruleset**.
2. Rename the new ruleset AutoFingerprint and rename the default rule from **Rule1** to Create New Fingerprint.
3. Select **Function1** and add the actions and parameters in the following table.

Library	Action	Parameter
rrunner	rrCompare	object1 = @P.MatchType
		object2 = Manual
AutoDoc	SetFingerprintDir	@APPPATH(fingerprint)
AutoDoc	CreateFingerprint	
AutoDoc	SetFingerprint	@D.TYPE,@P.TYPE
Intellocate	iloc_SetZones	

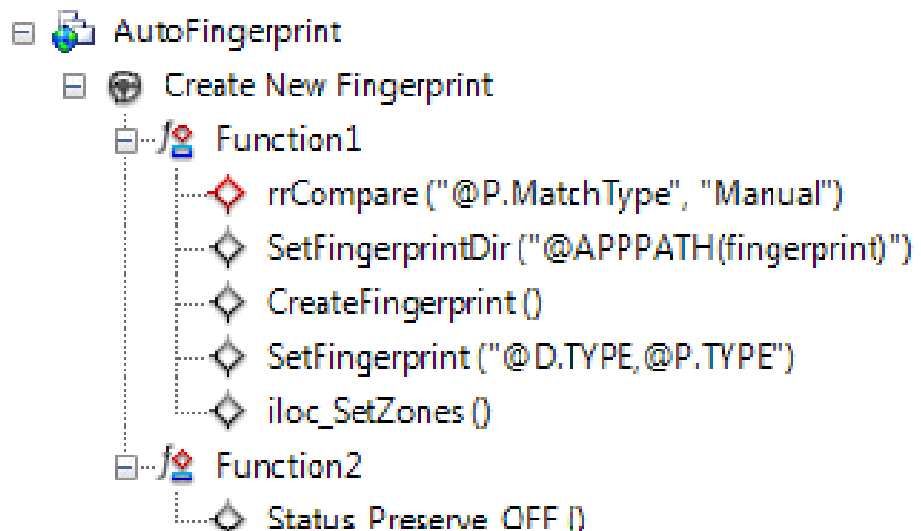
Attention: The parameter on the SetFingerprint action sets the fingerprint class to the current document type and the fingerprint type to the current page type. Make sure that TYPE is uppercase.

4. Right-click the **Create New Fingerprint** rule and choose **Add Function**.
5. Select **Function2** and add the following action.

Library	Action	Parameter
runner	Status_Preserve_OFF	

The purpose of this function is to ensure that the rule returns True and thus avoid triggering a validation error.

6. In the Rulesets pane, click **Save**. Then, click **Lock/Unlock ruleset** and choose **Publish Ruleset**.



Assigning the rule to each page type

You must assign the Create New Fingerprint rule to each of the page types.

To assign the rule to each page type:

1. In the Document Hierarchy pane, click **Lock DCO for editing**.
2. Expand the document hierarchy so you can see all page types.
3. Select the **Rental_Agreement** page type.
4. In the Rulesets pane, select the **Create New Fingerprint** rule and click **Add to DCO**.
5. In the Document Hierarchy pane, select the **Optional_Insurance** page type and then click **Add to DCO**.
6. Repeat to add the Create New Fingerprint rule to the Air_Ticket, Room_Receipt, Meals, and Other_Charges pages.
7. In the Document Hierarchy pane, click **Save** and then click **Unlock DCO**.

Adding the ruleset to the Verify task profile

You must add the AutoFingerprint ruleset to the Verify task profile.

To add the ruleset to the Verify task profile:

1. In the Rulesets pane, select the **AutoFingerprint** ruleset.
2. Click the **Task profiles** tab and then click **Lock/Unlock task profiles**.
3. Select the **Verify** task profile and then click **Add ruleset to profile**.
4. Expand the Verify task profile and confirm that it contains the **Validate** ruleset and the **AutoFingerprint** ruleset.
5. Click **Save** and then click **Lock/Unlock task profiles**.

Enabling logging for Datacap Web Client

Before you can run a batch through the workflow, you must enable logging for the Verify task.

You must enable the RRS logging in the Verify task Setup dialog in the Datacap Web Client. You are running the Verify task rules from the Datacap Web Client.

Attention: When you run the Verify task, the log file is saved as `verify_rrs.log` in the current batch folder.

To enable logging for Datacap Web Client:

1. Open the Datacap Web Client, select the **TravelDocs** application, and login with the Admin account.
2. Click the **Administrator** tab, and then click **Workflow**.
3. Expand **Main Job**, and click the **Verify** task.
4. In the Selected task details section, click **Setup**.
5. In the Verify.task.xml - Web Dialog window, scroll down to the Rulerunner Settings section.
6. In the **Rulerunner service log:** field, enter 3.
7. Scroll down and click **Save**.

Running a batch through the workflow

You can run a batch through the workflow to test automatic fingerprint generation.

To run a batch through the workflow:

1. Run a batch through the workflow (see “Running a batch through the workflow” on page 217), but stop when the batch is pending for verification.
2. Start the Datacap Web Client/Datacap client and log in to the **TravelDocs** application with the Admin account.
3. Click the **Verify** shortcut in the **Operations** tab to open the pending batch.
4. Go through the batch as before until you reach the Airline #4 page.
5. Click the **Outbound_From** field. Then, use the mouse to draw a bounding box around the field in the image pane. When you release the mouse, the web client inserts the recognition data into the grid.
6. Repeat for the other fields on the page.
7. Click **Submit**. In the background, Datacap runs the AutoFingerprint ruleset to create the new fingerprint file and add the zone information to the document hierarchy. Then, click **OK > Stop**.
8. In Datacap Studio, click the **Zones** tab and then click **Refresh**.
9. Expand the first **Flight** class (the SetFingerprint action creates a new class even though there is already a class called **Flight**) and select the new fingerprint.
10. Unlock the document hierarchy and select one of the **Air_Ticket** fields to activate the zones in the **Image View** pane.
11. After you review the zones in the **Image View** pane, lock the document hierarchy.

Reviewing the RRS log file

If your application did not create the new fingerprint with the zone information, you can check the RRS log file to troubleshoot the problems.

To review the RRS log file:

1. Open the current batch folder.
2. Open the file `verify_rrs.log`.
3. Scroll to till end of the page of the file to see the log entries for the AutoFingerprint ruleset.

```
ruleset name="AutoFingerprint" id="14" target object="P" target id="TM000014"
target type="Air_Ticket"
dco open tag="P" id="TM000014" type="Air_Ticket"
rule "Create New Fingerprint"
  func "Function1"
    action rrCompare ("@P.MatchType","Manual")
    action returned true    <--Match type is "Manual"
  /action
  action SetFingerprintDir (false,false,"@APPPATH(fingerprint)")
  load rrx code: "c:\datacap\rrs\autodoc.rrx"
  /load
  action returned true    <--Fingerprint directory established
  /action
  action CreateFingerprint (false,false)
  action returned true    <--Successfully created the new fingerprint
  /action
  action SetFingerprint (false,false,"@D.TYPE,@P.TYPE")
  action returned true    <--Set the fingerprint class and page type
  /action
  action iloc_SetZones (false,false)
  load rrx code: "c:\datacap\rrs\intellocate.rrx"
  /load
  action returned true    <--Successfully saved the new zone information
  /action
  func result: "true"
/func
```

```

        rule result: "true"
    /rule
  /ruleset
  c:\datacap\RRS\Logs\wrrs
end log to batch

```

TravelDocs: Splitting a document from the main batch

You can split manually identified pages from the main batch and send them to a supervisor for fingerprint creation.

“Updating the Routing ruleset to split the batch”

“Assigning the Batch Splitting rule to the Close element of the batch” on page 223

“Routing the split document to a supervisor” on page 223

“Running a batch through the workflow” on page 225

Updating the Routing ruleset to split the batch

You use the existing Routing ruleset to split the batch by using a document-level variable that you created for this purpose.

You must set this variable for any document that contains a manually identified page. The Routing ruleset runs at the end of the Profiler task profile after recognition and validation tasks are completed.

To update the Routing ruleset to split the batch:

1. In the Datacap Studio Rulesets pane, select the **Routing** ruleset and click **Lock/Unlock ruleset**.
2. Expand the **Routing** ruleset, **Routing Rule 1**, and the **Set Manual Page Status** function.
3. Select the **Set Manual Page Status** function and add the following action and parameters to the end of the function.

Library	Action	Parameter
runner	rrSet	varSource = Yes
		varTarget = @D.Split

This function assigns the value Yes to a document-level variable called Split, and creates it if necessary.

4. Right-click the **Routing** ruleset and choose **Add Rule**. Rename the new rule **Batch Splitting**.
5. Expand the **Batch Splitting** rule, select **Function1**, and add the following action and parameter.

Library	Action	Parameter
Split	SplitBatch	@D.Split

This action raises the condition flag and creates a child batch that contains any documents with the document-level variable Split. In this case, the only possible value of the variable is Yes. So, all documents are sent to the same child batch.

6. In the Rulesets pane, click **Save**. Then, click **Lock/Unlock ruleset** and choose **Publish Ruleset**.

Assigning the Batch Splitting rule to the Close element of the batch

The SplitBatch action must run at the batch level. But it depends on the status of the *Split* variable that in this case is created by a page-level rule in the same ruleset.

In an earlier section that describes the order of rule execution, you learned that batch-level rules typically run before page-level rules. (See “Order of rule execution” on page 46.) However, you also saw how you can attach a rule to the Close element. The rule runs after Datacap finished processing all lower-level objects.

To assign the Batch Splitting rule to the Close element of the batch:

1. In the Document Hierarchy pane, click **Lock DCO for editing**.
2. Expand the document hierarchy so you can see the Close element of the batch.
3. Select the **Close** element of the batch.
4. In the Rulesets pane, select the **Batch Splitting** rule and click **Add to DCO**.
5. In the Document Hierarchy pane, click **Save** and then click **Unlock DCO**.

Routing the split document to a supervisor

Before, you can configure the split condition, you must to create the supervisor job to handle the child batch. Then, you can configure the job router and create the shortcuts for the supervisor job.

“Creating the supervisor job”

“Configuring the job router” on page 224

“Configuring the supervisor shortcuts” on page 224

Creating the supervisor job:

You must create a supervisor job to handle the child batch that you want to split from the main batch.

Creating the supervisor job:

1. Start the Datacap Web Client, and log in to the **TravelDocs** application with the Admin credentials.
2. Click the **Administrator** tab, and then click **Workflow**.
3. Select the **TravelDocs** workflow and click **New** to create a new job node.
4. Name the new job node **Supervisor Job** and enter a description.
5. Select a **Priority** of **1** and click **Apply**.
6. Select **Supervisor Job** and click **New**.
7. Name the new task node **Verify**. Datacap populates the other fields in the Selected task details section.
8. Click **Apply**.
9. Select **Supervisor Job** again and click **New**.
10. Name the new task node **Export**. Datacap populates the other fields in the Selected task details section.
11. Click **Apply**.

Configuring the job router:

After you create the supervisor job, you must configure the job router that is used to send the documents to the supervisor.

To configure the job router:

1. Open the Datacap Web Client, log in to the **TravelDocs** application with the Admin credentials, and click the **Administrator** tab.
2. Click **Workflow**, expand **Main Job**, and expand the **Profiler** task.
3. Select the existing **Document Integrity Failed** condition and click **Remove**.
This function was moved to the CreateDocs task earlier, so it is no longer needed here.

Tip: If you want to refresh the page, click another tab and return to the Workflow page to confirm that the condition is removed.

4. Select the **Profiler** task, and under Parameters in the Selected task details section, enter Split Condition in the **Value** column for **Return Conditions**.
5. Click **Apply**.
6. Refresh the page, expand the **Profiler** task, and select **Split Condition**.
7. In the Selected condition details section, configure the values as follows:

Field	Value
Spawn type	Split
Parent Status	Pending
Steps	1
Child Job	Supervisor Job
Child Status	Pending

8. Click **Save condition**.

Configuring the supervisor shortcuts:

After you create the supervisor and configure the job router to send the documents to the supervisor, you can configure shortcuts for the supervisor.

To configure the supervisor shortcuts:

1. In the Datacap Web Client, click the **Administrator** tab, and then click **Shortcuts**.
2. Click **New** to create a new shortcut.
3. In the Selected shortcut details section, enter or select these values for the following fields:
 - a. **Name:** Supervisor Verify
 - b. **Description:** Supervisor verification
 - c. **Mode:** **Manual for Hold**.
 - d. Under Permissions, clear the check boxes and click the **Verify** check box under Supervisor Job.
4. Click **Save** in the Selected shortcut details section.
5. Click **New** to create a new shortcut.
6. In the Selected shortcut details section, enter or select these values for the following fields:

- a. **Name:** Supervisor Export
 - b. **Description:** Supervisor export data
 - c. **Mode:** **Manual for Hold** .
 - d. Under Permissions, clear the check boxes and click the **Export** check box under Supervisor Job.
7. Click **Save** in the Selected shortcut details section.

Running a batch through the workflow

You can run a batch through the workflow to test the document split from the main branch.

Before, you run a batch through the workflow, you need to delete the new Airline #4 fingerprint. So you can process it again as an unrecognized page.

To run a batch through the workflow:

1. In Datacap Studio, click the **Zones** tab.
2. Expand the first **Flight** class and select the **Airline #4** fingerprint.
3. Check the Image View pane to confirm that the Airline #4 fingerprint is selected, and then click **Remove selected**.
4. Start the Datacap Web Client, and log in to the **TravelDocs** application with the Admin credentials.
5. Click **VScan** on the **Operations** tab, browse to the location of the image files, click open and click **Scan**. When the task completes, click **OK** and **Done**. Then, click **OK** and **Stop** to return to the **Operations** tab.
6. Click **Upload**. When the task completes, click **OK** and **Stop** to return to the **Operations** tab.
7. Start Datacap Desktop, log in to TravelDocs, and select the **PageID** shortcut. When the task completes, click **OK**, and exit from Datacap Desktop.
8. Click **ManualPageID** on the Datacap Web Client **Operations** tab and wait for the page images to load.
9. Scroll to the bottom and set the page type for the last page to **Air_Ticket**.
10. Click **Done**, click **OK** and then click **Stop**.
11. Click **CreateDocs** on the **Operations** tab.
12. When **CreateDocs** is complete, click **Stop**.
13. Start Datacap Desktop, log in to TravelDocs, and select the **Profiler** shortcut. When the task completes, click **OK**, and exit from Datacap Desktop.
14. Click the **Monitor** tab and check the Job Monitor to see the result of the split. The child job is pending for the Supervisor Verify task and the main job is pending for the Main Verify task.
15. Click **Supervisor Verify** on the **Operations** tab to open the pending batch. The Airline #4 page is displayed.
16. Define the zone for each field. For more information, see “Running a batch through the workflow” on page 220.
17. Click **Submit** and then click **OK**. In the background, Datacap runs the AutoFingerprint ruleset to create the new fingerprint file and add the zone information to the document hierarchy.
18. Click **OK** and **Stop**.
19. Click the **Monitor** tab and check the Job Monitor. See the child job that is pending for the Supervisor Export task and the main job still pending for the Main Verify task.

Attention: If you run another batch through the workflow, the batch runs from end to end with no branching or splitting. The Airline #4 page is now in the fingerprint library. If you want to run another batch with branching and splitting, delete the Airline #4 fingerprint from the Datacap Studio **Zones** tab.

Datacap Web Client and remote scanning

You can now update your application by using Datacap Web Client Administrator and run a batch through the entire workflow by using a combination of web components and Rulerunner.

The Datacap Web verification client is discussed in other topics and the manual page identification client topics. These topics describe some of the other Datacap Web Client components, including the remote scanning client, other verification clients, and the Datacap Web Client administration interface.

“Moving the workflow to Datacap Web Client”

“Scanning images remotely” on page 227

“Remote virtual scanning” on page 229

“Verification by using the VeriFine web client” on page 230

“Verification, page identification, and registration by using AIndex” on page 234

“Verification by using the AVerify web client” on page 242

“Verification by using the ImgEnter web client” on page 245

“Manual page identification and batch restructuring with ProtoId” on page 245

“Administering an application” on page 248

“Job monitoring” on page 248

“TravelDocs: Scanning from Datacap Web Client” on page 249

“TravelDocs: Using AIndex for manual page identification and registration” on page 253

Moving the workflow to Datacap Web Client

The batch processing workflow that you developed so far is a mix of Datacap Desktop tasks, web-based tasks, and Rulerunner background tasks.

Datacap includes web components that you can use to administer most of the workflow from a web browser, including the functions that are listed in the following table.

Function	Web page
Remote scanning and image upload	scancl.aspx and uplbfc.aspx
Virtual scanning and image upload	Vscancl.aspx and uplbfc.aspx
Verification	verfine.aspx
	averify.aspx
	imgEnter.aspx
Verification, manual page identification, and manual registration	aindex.aspx
Manual page identification and fixup	ProtoID.aspx
Application administration	Standard tmweb.net interface
Job monitoring	Standard tmweb.net interface

Scanning images remotely

The Datacap Web Client remote scanning client (`scanc1.aspx`) and the related upload client (`uplbfcl.aspx`) allow operators to scan and upload batches from remote web clients. When the remote batch is queued on the server, Datacap processes the batch in the same manner that it processes any other job.

Attention: The Datacap Web Client remote scanning clients support TWAIN scanners only.

Datacap supports two remote scanning options:

- Scan the images from the web client directly into the application *batch* folder. This option requires that you share the *batch* folder and give write permission to all client machines.
- Scan the images to a local folder on the web client and then upload them to the application *batch* folder. Although Datacap Web Client initially stores the image files locally, it creates the runtime batch file in the server *batch* folder. Datacap Web Client can create the file without requiring you to enable sharing on the *batch* folder.

You specify which option to use in the task setup dialog in the Datacap Web Client, as described in “Configuring the remote scanning client.” You configure the remote scan task and the upload task later in the “Creating a remote scan task” on page 249 and “Configuring the Upload task” on page 250 topics.

“Configuring the remote scanning client”

“Implementing a start panel” on page 228

Configuring the remote scanning client

To configure the remote scanning client, `scanc1.aspx`, you need to use the Datacap Web Client. By default, the scan task is configured to save the image files locally in `C:\Datacap\scan` and to use the Upload task. If you want to scan directly to the batches folder of the application, you must share the folder and provide write access to all remote clients.

To configure `scanc1.aspx`:

1. In the Datacap Web Client, click the **Administrator** tab, and then click **Workflow**.
2. Expand **Scan Job** and then select **MyISScan**.
3. In the Selected task details section, click **Setup**.
4. In the setup dialog (MyISScan.set.xml - Webpage Dialog):
 - a. Scroll down to the Scan section.
 - b. Enter a value for the **Scan into directory** field. The default is `c:\datacap\scan`.
 - c. To enable direct scanning into the batches folder of the application without using the Upload task, select the **Local processing** check box. If you do not select this option, Datacap scans the images to a local folder on the web client and then uses the Upload task to upload the images to the application batches folder.
 - d. Click **Save**.

Implementing a start panel

A start panel prompts the operator to enter data before the remote scan page is displayed. You can use a start panel to capture any information specific to the batch that you want to collect (for example, date, and operator name).

To enable a start panel:

1. Start Datacap Web Client, click the **Administrator** tab, click **Workflow**, and select the task for which you want to enable the start panel.
2. In the Selected task details pane, click **Setup**.
3. In the *task.set.xml* - Webpage Dialog window, scroll to the **Scan or Disk Scan (VScan)** section, and select the **Show the Start Batch Panel** check box, if it is not already selected. (Clearing the check box disables Start Batch Panel.)
4. Scroll down and click **Save**. The start panel displays a data entry field for each batch level field that is defined in the document hierarchy. A batch level field is generally at the same level of documents that are defined in the batch hierarchy. To capture the name of the person who is completing a scan, you can create a batch level field that is called **Name** in the document hierarchy. In the TravelDocs application, the **Name** field is at the same level as the **Car_Rental**, **Flight**, and **Hotel** document types. At the beginning of a scanning task, Datacap displays a dialog that prompts the operator to enter a name. In the following runtime batch hierarchy XML sample, Datacap stored the data in a batch level field (<F id="Name">). The ASCII code values in the character elements spell Henderson, which is the name that the operator entered.

```
<B id="20110154.008">
  <V n="TYPE">TravelDocs</V>
  <V n="STATUS">73</V>
  <V n="ScanOperator">admin</V>
  <V n="ScanStation">1</V>
  <D id="20110154.008.01">
    <V n="TYPE"></V>
    <V n="STATUS">0</V>
    <P id="TM000001">
      <V n="imagePath">c:\datacap\scan\20110154.008\tm000001.tif</V>
      <V n="TYPE">Other</V>
      <V n="STATUS">49</V>
      <V n="ScanSrcPath">C:\Datacap\TravelDocs\images\Images_Page_01.tif</V>
    </P>
    etc.
  </D>
  <F id="Name">
    <V n="TYPE">Name</V>
    <V n="Position">0,0,0,0</V>
    <V n="STATUS">0</V>
    <C cn="10" cr="0,0,0,0">72</C>
    <C cn="10" cr="0,0,0,0">101</C>
    <C cn="10" cr="0,0,0,0">110</C>
    <C cn="10" cr="0,0,0,0">100</C>
    <C cn="10" cr="0,0,0,0">101</C>
    <C cn="10" cr="0,0,0,0">114</C>
    <C cn="10" cr="0,0,0,0">115</C>
    <C cn="10" cr="0,0,0,0">111</C>
    <C cn="10" cr="0,0,0,0">110</C>
  </F>
</B>
```

"Populating drop-down lists on a start panel" on page 229

"Running validation rules" on page 229

Populating drop-down lists on a start panel:

As with verification panels, you can use dictionaries or SELECT variables to populate start panel fields.

- For example, if you create a dictionary of operator names and associate it with the batch level field, the operator can select his or her name from a drop-down list:
- Alternatively, you can use the field's SELECT variable to populate the drop-down list from a database. : For example, the following SELECT value gets a list of operator names from the application's lookup database:
<SQL flist='Name' dsn="*/lookupdb:cs">SELECT Operator FROM Operators</SQL>

Running validation rules:

You can run validation rules on start panel fields.

To run validation rules on start panel fields, enter the name of the validation task profile in the `set.xml` file of the remote scan task client:

1. Start Datacap Web Client, click the **Administrator** tab, click **Workflow**, and select the remote scan task for which you want to specify the task profile.
2. In the Selected task details pane, click **Setup**.
3. In the `task.set.xml` - Webpage Dialog window, scroll to the **Rulerunner settings** section.
4. In the **Main task profile** field, enter a name of the validation task profile, such as `ValidateStartPanel`. Datacap starts the `ValidateStartPanel` task profile when the user clicks **Submit** in the start panel.
5. Scroll down and click **Save**.
6. Implement validation rules in Datacap Studio.
 - a. Create a task profile with an associated ruleset.
 - b. Create a validation rule for each start panel field you want to validate.
 - c. Bind each rule to the associated batch level field. For example, the following rule validates the operator name by using a database **lookup** command.
 - Validate Start Panel ruleset
 - Validate Operator rule
 - Validate Operator function
 - `SetIsOverrideable("False")`
 - `Status_Preserve_OFF()`
 - `OpenConnection("@APPVAR(*/lookupdb:cs)")`
 - `ExecuteSQL("SELECT Operator FROM Operators WHERE Operator='%s';",Name)`
 - `CloseConnection()`

Remote virtual scanning

The Datacap Web Client virtual scanning client (`vscanc1.aspx`) is similar to the remote scanning clients. However, instead of scanning files, the virtual scanning client imports them from a local image folder. You then upload the batches to integrate them into the regular workflow.

As with the remote scanning client, the remote virtual scanning client provides two upload options:

- Scan the images from the web client directly into the application's batches folder (**Local Processing** check box is checked in the task's Setup dialog).
- Scan the images to a local folder on the web client and then upload them to the application's batches folder (**Local Processing** check box is not checked in the task's Setup dialog).

You configure the **Local Processing** setting by selecting the task in the Workflow page on the **Administrator** tab of the Datacap Web Client.

Verification by using the VeriFine web client

The VeriFine web verification client (VeriFine.aspx) generates verification pages that are automatically based on the document hierarchy.

It can also generate custom verification pages by using predefined static layouts (see “Configuring the VeriFine client”).

In addition to the image view and the data entry panel, a VeriFine page includes a toolbar and a batch tree view that you can use to split or join documents, reorder pages, and mark documents or pages for deletion.

“Restructure the batch by using the batch tree view (VeriFine)”

“Configuring the VeriFine client”

“Configuring additional VeriFine settings” on page 232

“Creating custom pages” on page 233

Restructure the batch by using the batch tree view (VeriFine)

The batch tree view displays the type and status of each document and page within the batch. You can use controls to restructure the batch.

UI Control	Description
Check mark (✓) on a folder (Split document icon)	Splits the current document so the selected page becomes the first page in a new document.
Two pages on a folder (Join documents icon)	Joins the current document and the previous document.
Mark for deletion icon (X)	Marks the selected document or page for deletion. Documents are assigned the first Ignored Page Statuses (IPS) value; pages are assigned the second IPS value. (See the section <i>Configuring the page and field status settings</i> in “Configuring the VeriFine client.”)
Up arrow icon (↑)	Moves up the selected page within the current document.
Down arrow icon (↓)	Moves the selected page down within the current document.

Configuring the VeriFine client

You can configure the VeriFine client in the **Workflow** page that you access through the **Administrator** tab in the Datacap Web Client.

More settings are described in “Configuring additional VeriFine settings” on page 232.

To configure VeriFine settings:

1. Start the Datacap Web Client.
2. Log in to the application that contains the job and task that use the VeriFine client.
3. Click the **Administrator** tab, and click **Workflow**.
4. Expand the job that contains the task that uses VeriFine.aspx, and select the task.
5. Click **Setup** in the Selected task details section.
6. Scroll to the relevant section in the *task.set.xml* Webpage Dialog, and enter or select values for the corresponding parameters.

Configuring the page layouts

By default, VeriFine uses a generic two-column layout that is based on the document hierarchy for the current page and is generated automatically.

You can use the VeriFine section to assign **Image View**, **Data Entry Panel**, and the **Batch View** to a specific location on the page.

For information on creating custom static pages for use with VeriFine.aspx, see “Creating custom pages” on page 233.

Configuring the page and field status settings

The page and field status settings in the Navigation sections (Ignored Page Status(es), Done Page Statuses, and others) control how VeriFine manages pages and fields with the specified status values. See the *STATUS* variable in the “Standard Variable Reference” on page 291 for a list of commonly used status values.

Setting	Description
Ignored Page Status(es)	Determines which pages to ignore. Datacap does not display a page if it has one of the specified status values. Additionally, the first value is assigned to any document you mark for deletion in the VeriFine tree view pane. The second value is assigned to any page you mark for deletion. Do not include any Validation Statuses or Done Page Statues value in this list. For example, if you specify 72,74, Datacap does not display pages with STATUS=72 or 74. Additionally, Datacap assigns STATUS=72 to documents marked for deletion and STATUS=74 to pages marked for deletion.

Setting	Description
Done Page Statuses	Determines when a batch is complete. When all pages have one of the specified values, Datacap displays All documents are complete. Click OK to finish the batch. Otherwise, you can put the batch on hold only. For example, if you specify Done Page Statuses=0,2, the batch is complete when all pages have STATUS=0 (OK) or STATUS=2 (validation failure overridden by the operator).
Ignored Field Statuses	Determines which fields to hide. Datacap does not display a field if it has one of the specified status values. For example, if you specify Ignored Field Statuses =-1, Datacap does not display fields with STATUS=-1 (hidden).
Done Field Statuses	Determines which fields to hide when the Problems only check box is selected. Datacap does not display any field with one of the specified status values. For example, if you specify Done Field Status=0 and the Problems only check box is selected, Datacap does not display fields with STATUS=0 (OK).
Validation statuses	Specifies the status value that is assigned to the current page after validation: • The first value is assigned when validation passes (Done) • The second value is assigned when the operator overrides a validation error (Override) • The third value is assigned when validation fails and override is not used (Fail) For example, Validation statuses=0,2,1 specifies Done status = 0; Override status = 2; Fail status = 1.

Configuring additional VeriFine settings

You can configure additional VeriFine settings.

1. Start the Datacap Web Client.
2. Log in to the application that contains the job and task that use the VeriFine client.
3. Click the **Administrator** tab, and click **Workflow**.
4. Expand the job that contains the task that uses VeriFine.aspx, and select the task.
5. Click **Setup** in the Selected task details section.

6. Scroll to the relevant section in the *task.set.xml* Webpage Dialog, and enter or select values for the corresponding parameters.
 - a. To load pages, click the **Load all documents** check box in the Document startup section.

Tip: This option loads all page XML files for the current document into a temporary folder. Use this option for cross-page validations that must access multiple pages within a document. However, this option adversely affects system performance.

- b. To load images, click the **Load all images** check box in the Document startup section.

Tip: This option is enabled by default, and loads all images in a document when you open that document. Disable this option when you have large documents that require much time to load all of the images before the first page is displayed. Enabling this option delays the display of subsequent pages.

- c. To configure the batch tree view, enter values in the DCO Tree View section for **Display variables**.

Tip: By default Datacap displays the type and status variables for each document and page. To add a variable, click the plus sign (+), or to remove a variable, click the minus sign (-).

- d. To customize background field colors, scroll to the Page Processing section, and in the Background field colors subsection, enter color values for the following field types:

Field	Default color
Low Confidence fields	yellow
Invalid fields	lightpink
Normal fields	white

Tip: X11 color names are supported.

- e. To enable multi-pass verification, enter values in the Alternative or Blind texts section for **DCO field alt text index** and **Blind confirm index**.
Datacap applications can display the same page to multiple operators to ensure that accurate data entry and verification. The VeriFine client provides limited support for multi-pass verification that includes two-pass verification, whereas the AIndex client provides full support for multi-pass verification, including double-blind. For details, see “Verifying in multiple passes” on page 235.

Creating custom pages

The VeriFine client generates a default verification panel for each page type automatically. It maps each of the fields in the document hierarchy into a two-column table. You can use VeriFine to replace the standard verification panel with a custom (static) panel.

To create and use custom pages:

1. Create a custom panel layout with the Custom Layout Generator.
 - a. Open the Datacap Web Client Custom Layout Generator (<http://<server>/tmweb.net/task/support/dragresize.htm>).

- b. Create a layout by browsing to and selecting an .xml file, or edit an existing layout by browsing to and selecting an .ascx file.
 - c. If you are creating a new layout, select a page in the **Page** menu, and click **Generate**.
 - d. Move or resize any labels or fields, as wanted.
 - e. Click **Save Panel**.
2. Locate the panel layout file (.ascx) that you saved in Step 1.
The file is in one of the following folders, depending on the version of Windows that you are using.
 - C:\ (the root of the C: drive)
 - C:\Users\< username>\AppData\Local\VirtualStore
3. Rename the file to match the page type. For example, you might name the file Rental_Agreement.aspx.
4. Move the .ascx file into the C:\Datacap\tmweb.net\Task folder.

Important: You must repeat these steps for each custom page layout.

5. Specify the custom panel in the task's setup.xml file.
 - a. Start the Datacap Web Client and log in the **TravelDocs** application with the admin account.
 - b. Click the **Administrator** tab, and then click **Workflow**.
 - c. Select the Verify task, and click **Setup**.
 - d. In the Verify.set.xml - Webpage dialog window, scroll to the Custom web panels section and click the **Use custom web panels** check box.
 - e. Enter a page type and the corresponding .ascx file name, for each of the custom pages that you created.

Tip: You can mix custom page layouts with default page layouts. For example, if your application has 10 page types but you create only two custom layouts, specify just the two custom layouts. VeriFine uses the default layout for the remaining page types.

- f. Click **Save** and close the window.

Tip: To revert to the default panels at any time, clear the **Use custom web panels** check box and click **Save**.

6. Open a batch in the Verify task and confirm that the task is using the new custom panels.

Verification, page identification, and registration by using AIndex

The AIndex web client (aindex.aspx) is similar to VeriFine, but includes full support for multipass verification and manual image registration. As with the other web clients, you enable the client by specifying the name of the .aspx file in the Selected task details, Setup Web Dialog. You access this dialog box through the Datacap Web Client **Administrator** tab.

For more information, see “Verifying in multiple passes” on page 235 and “Manual page identification and registration” on page 240.

AIndex generates verification pages automatically that are based on the document hierarchy, and includes a toolbar, an image view, a data entry panel, and a batch tree view. The web page is similar to VeriFine, but AIndex does not display image

snippets in the data entry panel. If you do not require multipass verification, VeriFine is a better choice for web verification.

“Restructure the batch by using the batch tree view (AIndex)”

“AIndex client configuration”

“Verifying in multiple passes”

“Manual page identification and registration” on page 240

Restructure the batch by using the batch tree view (AIndex)

The batch tree view displays the type of each page within the batch. You can use this view to change the document or page type and restructure the batch.

Table 47. Batch tree view controls

Control	Description
Start Doc check box	A check mark indicates that the current page is the first page in a document: • If the check box is cleared, selecting it splits the current document so the page becomes the first page in a new document. • If the check box is selected, clearing it joins the current document and the previous document.
• Hotel – Room_Receipt	Displays the document type and page type for the current page. You can change the page type. If the page is the first page in a document, you can also change the document type.
Move up button	Moves the selected page up.
Move down button	Moves the selected page down.
Problem fields only Check box	Select to display only problem fields in the data entry panel.
Anchors button	See “Manual page identification and registration” on page 240.

AIndex client configuration

You configure the AIndex client in the Selected task details, Setup Web Dialog that you access through Datacap Web Client **Administrator** tab.

The page and field status settings are the same as for the VeriFine client. (See the “Configuring the page and field status settings” section in the “Configuring the VeriFine client” on page 230 topic.)

In addition, AIndex supports the following settings that you can use to implement “Verifying in multiple passes”:

- **DCO field alt text index:** Specifies the DCO field alternate text index.
- **Blind confirm index:** Enables automatic double-blind checking that requires matching another alternate value (specified index) or retyping same value twice.
- **Show other alt texts:** Specifies whether to display other alternatives as hyperlinks.

Verifying in multiple passes

Datacap applications can display the same page to multiple operators to ensure that accurate data entry and verification. Datacap supports two main

implementations of multi-pass verification: *two pass* and *double blind*. Other implementations are possible, but you focus on these two implementations.

- In two-pass verification:

1. An operator (or a recognition engine) enters the initial value for each field.
2. Datacap displays the page to a second operator but hides the initial values. The operator enters a new value for each field.

Tip: If you are using a recognition engine to implement the first pass, you might choose to display only low confidence fields to the operator.

3. For each field, Datacap compares the new value to the initial value. If they match, Datacap accepts the value; otherwise, the operator must reenter the value. Only when the operator enters the same value twice in a row does Datacap accept the value.

- In double-blind verification:

1. An operator (or a recognition engine) enters the initial data values.
2. Datacap displays the page to a second operator but hides the initial values. The operator enters a new value for each field and Datacap saves all the values (no comparing).
3. Datacap displays the page to a third operator. Using a feature of AIndex that displays multiple values, the operator can see both the initial value and the second value.
4. For fields where the initial value and the second value are different, the operator must determine which value is correct, or enter a new value:
 - Clicking the initial value (or pressing Alt+Shift+A when the field has the focus) moves the initial value into the data entry field.
 - To enter a new value, the operator must enter the same value twice in a row.

“Storing multiple values in the runtime page data file”

“Actions that support multi-pass verification” on page 237

“Settings that support multi-pass verification” on page 237

“Example of two-pass data entry” on page 238

“Example of double-blind data entry” on page 239

Storing multiple values in the runtime page data file:

Your application can store values from multiple data entry passes in the runtime page data file.

Datacap can store multiple values in the runtime page data file for any specified field:

```
<F id="Vendor">
  <V n="TYPE">Vendor</V>
  <V n="Position">0,0,0,0</V>
  <V n="STATUS">1</V>
  <C n="6,8,10" cr="0,0,0,0">68,49,83</C>
  <C n="6,8,10" cr="0,0,0,0">97,50,112</C>
  <C n="6,8,10" cr="0,0,0,0">116,51,105</C>
</F>
```

In the task example, the **Vendor** field has three 3-character values that are represented by the ASCII characters shown. The first value (**AltText[0] = ASCII 68,97,116 = “Data”**) is the primary data value.

Your application can use this structure to store values from multiple data entry passes. However, because data is always captured in **AltText[0]**, you need to move the data.

For example, to implement double blind:

1. Capture the initial data in **AltText[0]**.
2. Move the data into **AltText[1]**.
3. Display the page to Operator 2 and save the new data in **AltText[0]**.
4. Copy the **AltText[0]** data into **AltText[2]**.
5. Display the page so Operator 3 can review **AltText[1]** and **AltText[2]** and accept either version or enter new data in **AltText[0]**.

Actions that support multi-pass verification:

The actions that are required to move and copy data are shown in this table.

Library	Action	Description
DCO	PropagateToAltText	Copies the character values from AltText[0] to the specified position.
DCO	ClearAltText	Clears the character values from the specified position in the field's array.

To implement a move, do a copy (**PropagateToAltText**) and then delete the original (**ClearAltText**).

The action that is used to compare AltText[0] and AltText[1] values is VoteFld.

Library	Action	Description
Vote	VoteFld	Sets the confidence level on each character to high and the field status to 0 (OK) if the AltText[0] and AltText[1] values match. Sets the confidence levels to low and the field status to 1 (problem) if the values do not match.

To implement double blind verification, use the VoteFld action before the page is displayed to the last operator. If the initial and second values do not match, the action sets the field status to 1 and the field is displayed in red.



Settings that support multi-pass verification:

You can specify these settings in the Datacap Web Client **Administrator** tab, clicking **Workflow**, and selecting the relevant task.

In the *task.set.xml* Webpage Dialog, scroll to the **Alternative or blind text** section, and enter the values for the corresponding fields, as needed.

Field	Value	Description
DCO field alt text index	0, 1 or 2:	Determines which "alt text" is the primary text (defaults to AltText[0]). AIndex displays the primary text in the data entry field.
Show other alt texts	3:	AIndex displays AltText[0] in the data entry field and AltText[1] and AltText[2] beside it.
	2:	AIndex displays AltText[0] in the data entry field and AltText[1] beside it.
	1:	AIndex displays the AltText[0] value initially, but you can toggle back and forth between AltText[1] and AltText[0] using the Alt+Shift+A hot key.
	0 or -1:	AIndex displays the AltText[0] value in the data entry field. AltText[1] is hidden but is used for comparisons.
Blind confirm index	1:	AIndex compares the new value to the initial value and forces the operator to enter a new value twice.
	-1:	The operator can enter any new value. There is no comparison and no need to enter the value twice.

Example of two-pass data entry:

In this example, *Operator 1* can be a person or a recognition engine. If you use a recognition engine for the initial data entry, it runs as a background process so there is no user interface shown.

Table 48. Example of two-pass data entry

Person or recognition engine	User Interface	AltText [0]	AltText [1]	AltText [2]	Task settings in set.xml file
Operator 1 (initial data entry)	1	1			Show other alt texts is set to -1. (hide alt text) Blind confirm index is set to -1. (do not compare)
Propagate ToAltText ("1")			1 (Moved from AltText [0])		
ClearAltText("0")					
Operator 2 (initial state)			1		Show other alt texts is set to -1. (hide alt text) Blind confirm index is set to 1. (compare)

Table 48. Example of two-pass data entry (continued)

Person or recognition engine	User Interface	AltText [0]	AltText [1]	AltText [2]	Task settings in set.xml file
Operator 2 (after data entry)	2	2	1		Show other alt texts is set to -1. (hide alt text) Blind confirm index is set to 1. (compare)

The data for Operator 2 data ('2') is now stored in **AltText[0]** and the data for Operator 1 ('1') is in **AltText[1]**. In this case, the compare fails, and the operator must enter the same value twice to override the initial value.

Important: You specify the **Show other alt texts** value and the **Blind confirm index** value in the set.xml Webpage dialog for the task in Datacap Web Client.

Example of double-blind data entry:

In this double blind example, *Operator 1* can be a person or a recognition engine. If you use a recognition engine for the initial data entry, the recognition engine runs as a background process, in which case there is no user interface.

Table 49. Double blind example of data verification

	User Interface	AltText [0]	AltText [1]	AltText [2]	Alternative or blind text settings
Operator 1 (initial data entry)	1	1			Show other alt texts = -1 (hide alt text) Blind confirm index = -1 (do not compare)
PropagateToAltText("1") ClearAltText("0")		1			
Operator 2 (initial state)			1		Show other alt texts = -1 (hide alt text) Blind confirm index = -1 (do not compare)
Operator 2 (after data entry)	2	2	1		Show other alt texts = -1 (hide alt text) Blind confirm index = -1 (do not compare)
PropagateToAltText("2")		2	1		
VoteFld()		2	1	2	
Operator 3 (initial state)	2 (1)	2	1	2	Show other alt texts = 2 (show alt text) Blind confirm index = 1 (compare)

Important: You specify the **Show other alt texts** value and the **Blind confirm index** value in the set.xml Webpage dialog for the task in Datacap Web Client.

In the preceding example, **PropagateToAltText** moves the initial data entry value (1) from AltText[0] to AltText[1], and **ClearAltText** removes the value from AltText[0]. Operator 2 enters a value of 2 and then **PropagateToAltText** copies the value of 2 from AltText[0] to AltText[3]. The **VoteFld** action sets the field status to '1' (problem) because AltText[0] and AltText[1] do not match, and AIndex highlights the field. Operator 3 can now do one of three tasks:

- Accept the current AltText[0] value ('2').
- Swap in and accept the AltText[1] value ('1').
- Enter a new value twice. The new value becomes the new AltText[0] value.

The following table shows the result for each of these three actions.

Table 50. Results of the blind example actions

		AltText [0]	AltText [1]	AltText [2]	Alternative or blind text settings
Operator 3 (accept AltText[0])	2 (1)	2	1	2	Show other alt texts = 2 (show alt text) Blind confirm index = 1 (compare)
Operator 3 (use AltText[1])	1 (2)	1	1	2	Show other alt texts = 2 (show alt text) Blind confirm index = 1 (compare)
Operator 3 (enter new data)	3 (1)	3	1	2	Show other alt texts = 2 (show alt text) Blind confirm index = 1 (compare)

Manual page identification and registration

You can use the AIndex web client to complete manual page identification and manual registration.

- You first set the document and page type by using the drop-down menu in the batch tree view pane.
- You then click **Anchors** at the end of the batch tree view to select the matching fingerprint and complete manual page registration.

When you register the page, Datacap displays a floating image of the anchor object of the page that you align with the actual anchor image on the current page. When you place the anchor object, Datacap computes the required page offsets that are used to calculate the positions of the data fields on the page.

Datacap can usually handle page identification and registration for you automatically, even when the offsets relative to the fingerprint are large (see “Pattern Matching” on page 187). Using manual page identification and registration might be useful to you in special situations. However, if you are using AIndex for manual page identification, you must run the manual page identification task after the CreateDocs task because AIndex requires a structured batch. You also need to confirm that each unidentified page is a separate document. These latter steps are discussed when you update the TravelDocs application to use AIndex.

“Enabling manual page registration (manual anchoring)” on page 241

“Registering a page by using manual anchoring” on page 241

Enabling manual page registration (manual anchoring):

You can enable manual anchoring for a specific page type.

1. Define an anchor field for the page type and specify the zone position on each of the corresponding page fingerprints. See “Setting up the pattern match anchor objects” on page 196. Confirm that the field's *PatternMatch* variable is set to 1.
2. Add a variable that is called *Required* to the anchor field and set its value to 1.
3. For the task in which you want to perform manual anchoring, specify the location of the application's fingerprint folder. For example, you might enter C:\Datacap\TravelDocs\fingerprint) in the task's Setup dialog in the Datacap Web Client

Registering a page by using manual anchoring:

Manual anchoring is only enabled for pages that include an anchor field with the variable *Required*=1 and where the anchor field's position in the runtime page data file is undefined or set to 0,0,0,0.

Unidentified pages (pages of type *Other*) typically do not have a runtime page data file. So you must first assign a page type. When you assign a page type, AIndex creates a page data file and sets all of the fields to 0,0,0,0. If the page type you assign has an anchor field with *Required*=1, AIndex enables manual anchoring.

AIndex checks document integrity when you submit the batch. So your documents and pages must be structured correctly. Additionally, you need to have appropriate **Done Page Statuses**, **Validation Statuses**, **Done Field Statuses**, and **Ignored Field Statuses** values in the task's .set.xml file (see “Configuring the VeriFine client” on page 230). You do this task later when you update the TravelDocs application to use AIndex (see “Updating ManualPageID” on page 255).

To identify or register a page by using manual anchoring:

1. In the AIndex batch tree view pane, select the page that requires identification or manual registration.
2. If the page is unidentified, use the drop-down lists to set the document type and page type. Depending on the position of the page within the batch, you might need to select the **Start Doc** option before you can set the document type.
3. If manual anchoring is enabled for the page type that you selected, Datacap displays a message that you must set the anchor position. Click **OK** to close the message box.
4. Click **Anchors** in the batch tree view pane.
5. If the page is unidentified, Datacap displays thumbnails of all fingerprints with a required anchor (**Required**=1). You are asked to double-click the matching fingerprint. Click **OK** to close the message box and then double-click the matching fingerprint. Datacap reads the position of the anchor object for the fingerprint that you selected from the document hierarchy, and floats a red version of the anchor image over the current page image.
6. Use the mouse to align the anchor object with the page image.
7. When you complete the batch, Datacap writes the anchor position to the runtime page data file and writes the offset values to the runtime batch hierarchy. The following table provides an example.

Runtime page data file	Runtime batch hierarchy file
<pre><P id="appdevguide_TM000006"> <F id="appdevguide_Vendor_Logo"> <V n="TYPE">Vendor_Logo</V> <V n="Position">221,229,608,332</V> <V n="STATUS">-1</V> </F> etc.</pre>	<pre><P id="appdevguide_TM000006"> <V n="IMAGEFILE">tm000006.tif</V> <V n="TYPE">Air_Ticket</V> <V n="STATUS">0</V> <V n="Image_Offset">-45,-29</V> <V n="TemplateID">562</V> etc.</pre>

Verification by using the AVerify web client

The AVerify client is functionally similar to the VeriFine client, but lacks the batch restructuring features of VeriFine. Unlike VeriFine, AVerify completes most of the processing on the client, rather than on the Datacap Web Client server.

AVerify client can manage processing tasks such as generating screens, loading data, and saving data. Therefore, AVerify is a good option if you are processing high volumes by using multiple web clients that are attached to the same web server. The server is started only for validations and for saving the XML files. Aside from this feature, VeriFine is usually a better choice for web verification. The following table describes the UI controls (and the corresponding tooltip, if accessible) that are available in AVerify.

Next LC	Highlights the next low confidence character on the current page.	Show snippet	Displays a window with an enlarged view of the current field.
◀ (left arrow, Previous page tooltip)	Displays the previous page in the batch.	Plus sign (+) on a magnifying glass (Zoom in tooltip)	Enlarges the page image view.
Exclamation point (!) on an arrow that points left (Previous problem tooltip).	Submits the current page and displays the previous problem page.	Minus sign (–) on magnifying glass (Zoom out icon)	Reduces the size of the page image view.
Two vertical bars (Put batch on hold tooltip)	Puts the batch on hold.	Magnifying glass on page (Zoom quarter tooltip)	Displays one quarter of the page.
Exclamation point (!) on an arrow that points right (Previous problem tooltip).	Submits the current page and displays the next problem page.	Letter T on page graphic (Show words tooltip)	Outlines all of the words on the page.
▶ (Right arrow, Next page tooltip)	Displays the next page in the batch.	Lines on a page graphic (Show lines tooltip)	Outlines all lines on the page image.
Hold	Puts the batch on hold.	Letter T on page (Show fields tooltip)	Outlines all recognition fields on the page image.
Submit	Submits the current page and displays the next problem page.	Override check box	Select to override a validation failure.
?	Runs the Verify task profile (validation rules).	Batch view...	Displays the data from the runtime batch hierarchy.

To use the AVerify client, select **averify.aspx** as the value for **Program** in the Verify task's setup dialog in the Datacap Web Client.

Creating and implementing custom (static) panels is covered in other topics.

AVerify uses the same page and field status settings and Rulerunner settings as the VeriFine client. For more information, see the *Configuring the page and field status settings* section in the “Configuring the VeriFine client” on page 230 topic.

“Creating and using custom (static) panels”

Creating and using custom (static) panels

AVerify generates a default verification panel for each page type automatically. It maps each of the fields in the document hierarchy into a two-column table.

To create and use custom static panels:

1. Export the default panel layout.
2. Customize the panel layout.
3. Specify the panel layout in the task's Setup dialog in the Datacap Web Client.

If you specify that AVerify is to use static panels but do not define a static panel for each page type, you receive a runtime error. AVerify is unable to display a verification panel.

Therefore, if you choose to use static panels, you must define a static panel for each page type.

“Exporting the default panel layout”

“Customizing the panel layout” on page 244

“Specifying the custom panels to use in a task” on page 244

Exporting the default panel layout:

A custom panel is referred to as a static panel. To create a static panel, you use Datacap Web Client Custom Layout Generator.

To export the default panel layout:

1. Open the Datacap Web Client Custom Layout Generator (<http://<server>/tmweb.net/task/support/dragresize.htm>).
2. Browse to and select an .ascx file that corresponds to the panel layout that you want to export.
3. Move or resize any labels or fields, as wanted.
4. Click **Save Panel**.

The .ascx file is in one of these folders, depending on the version of Windows that you are using:

- C:\ (the root of the C: drive)
- C:\Users\< username>\AppData\Local\VirtualStore

5. Rename the file to match the page type. For example, you might name the file Rental_Agreement.aspx.
6. Move the .ascx file into the C:\Datacap\tmweb.net\Task folder.
7. Specify the custom panel in the task's setup.xml file.
 - a. Start the Datacap Web Client and log in the **TravelDocs** application with the admin account.
 - b. Click the **Administrator** tab, and then click **Workflow**.
 - c. Select the **Verify** task, and click **Setup**.

- d. In the Verify.set.xml - Webpage dialog window, scroll to the Custom web panels section and click the **Use custom web panels** check box.
- e. Enter a page type and the corresponding .ascx file name for the default panel that you want to export.
- f. Click **Save** and close the window.

Tip: To revert to the previous panel, clear the **Use custom web panels** check box and click **Save**.

8. Prepare a batch for verification.
9. Open the batch in AVerify and display at least one instance of each page type. For each page you open, Datacap creates an HTML file with the layout of the standard two-column verification panel for that page. The files are named *static page_type>.htm*, for example, *staticRental_Agreement.htm*. The files are saved in one of the following locations:
 - C:\ (the root of the C: drive)
 - C:\Users\<username>\AppData\Local\VirtualStore

Customizing the panel layout:

The HTML file that AVerify exports for each page type defines each of the fields within a standard two-column table layout. Each cell represents one field and contains a label, an image snippet, and an edit control.

The HTML within the image snippet and edit control cells contains code that cannot be modified.

Image snippet	Edit control
<pre><OBJECT style="WIDTH: 100%; HEIGHT: 30px" id=dcim_Pickup_Date title=Pickup_Date tabIndex=-1 codeBase="dcim.cab" classid=clsid:BA893287-8932-11D3-A0DB-58B204C16365 width=191 height=30> . . . </OBJECT>
</pre>	<pre><OBJECT onblur=reSetValue(me) style="BORDER-BOTTOM: gray 1px solid; BORDER-LEFT: gray 1px solid; WIDTH: 99%; FONT-FAMILY: Arial; FONT- SIZE: 12pt; BORDER-TOP: gray 1px solid; BORDER-RIGHT: gray 1px solid" onkeydown=hkPress() id=txt_Pickup_Date language=vbscript class=AFlatEdit onfocus=Remember(me) title=Pickup_Date name=dcedit codeBase="dcim.cab" classid=clsid:D20D94B4-B85C-466C-B29B-19B2ADAF60EC height=23> . . . </OBJECT></TD></pre>

However, you can change the labels, rearrange the cells, remove the image snippets, and so on.

To customize the default panel layout:

1. Open the HTML file in an HTML editor.
2. Make the required modifications to the labels, layout, and so on.
3. Save the file.

Specifying the custom panels to use in a task:

You control the use of custom (static) panels through the task setup in the Datacap Web Client.

1. Open the Datacap Web Client and log on to the TravelDocs application.

2. Click the **Administrator** tab, and click **Workflow**.
3. Expand **Web Job**, and click the **Verify** task.
4. In the Selected task details pane, click **Setup**.
5. In the Verify.set.xml - Webpage dialog window, scroll to the **Custom web panels** section and click the **Use custom web panels** check box.
6. Under **Bind page to ascx panel**:
 - a. In the **Panel for** field, replace Page_Type by entering Rental_Agreement and replace panel.htm by entering staticRental_Agreement.htm.
 - b. Click the + sign to add a second **Panel for** field.
 - c. In the second **Panel for** field, replace Key2 by entering Optional_Insurance and replace Value2 by entering staticOptional_Insurance.htm.
7. Click **Save**.
8. Open a batch in AVerify and confirm that AVerify is using the new custom panels.
9. Optional: To revert to the default panels at any time:
 - a. Repeat steps 1 - 5 and click **Use custom web panels** to clear the check box.
 - b. Under **Bind page to ascx panel**:
 - Click the - sign to remove Optional_Insurance.
 - Replace Rental_Agreement by entering Page_Type.
 - Replace staticRental_Agreement.htm by entering panel.htm.
 - c. Click **Save**.

Verification by using the ImgEnter web client

The ImgEnter (imgenter.aspx) web client is different from the other web verification clients in that you enter data through the page image view.

ImgEnter displays a gray border around each data field. When you click within a field, the web client displays a data entry edit field. The data entry field displays the recognized data that you can change. Fields with low confidence characters are displayed in yellow and fields with validation errors are displayed in red. To use the ImgEnter client, select ImgEnter.aspx as the **Value for Program** key in the Verify task's details pane in the Datacap Web Client. Click the **Administrator** tab, click **Workflow**, and then select **Verify** in the **Web Job**.

ImgEnter uses the same page and field status settings and RRS settings as the VeriFine client. For more information, see the *Configuring the page and field status setting* section in the "Configuring the VeriFine client" on page 230 topic.

Manual page identification and batch restructuring with Protold

You use the ProtoId web client (ProtoId.aspx) to do manual page identification. You can use the list beneath each thumbnail to change the current page type. Additionally, the small toolbar above each thumbnail image provides batch restructuring functions.

The following table lists the batch restructuring controls.

Control	Description	Control	Description
Plus sign (+)	Enlarges all thumbnails	Left arrow (<)	Moves the page to the left (ALT+U)

Control	Description	Control	Description
Minus sign (-)	Shrinks all thumbnails	Check mark (✓)	Indicates that the page was copied to the clipboard
keys	Displays hot key list	Copy	Copies the page to the clipboard (CTRL+C)
Question mark (?)	Runs document integrity check	Paste	Inserts the page from the clipboard* (CTRL+V)
		Curved arrow	Rotates the page thumbnail by 90 degrees (CTRL+G)
		right arrow (>)	Moves the page to the right (ALT+N)

When you copy a page, Datacap adds a page-level variable to the runtime hierarchy with the ID of the source page. For example, if you copy page 1, Datacap adds `<V n="Copy">TM000001</V>` to the cloned version.

Tip: You can move between thumbnails by using TAB/SHIFT+TAB. You can display a full page image by clicking the thumbnail or pressing ENTER. For more information, see “ProtoID web client configuration.”

“ProtoID web client configuration”

ProtoID web client configuration

You can insert new pages, modify the available page types, disable document integrity checks, and run rules with the ProtoID web client.

To use the ProtoId client:

1. Start the Datacap Web Client, click the **Administrator** tab, and click **Workflow**.
2. Create or select a task for which you want to use the ProtoID client.
3. Under the Parameters heading in the Selected task details section, select **ProtoID.aspx** as the **Value** for the **Program**.
4. Click **Save**.

Inserting pages

The **CTRL+M** hot key inserts a new page before the selected page. To specify the type for the new page:

1. In the Datacap Web Client, click the **Administrator** tab, and click **Workflow**.
2. Select the task that uses ProtoID.aspx.
3. Click **Setup** in the Selected task details section.
4. In the *task.set.xml* Webpage Dialog, scroll down to the Page ID section and enter the name of an existing page type in the **Insert type:** field.

For example, if you enter `Separator_Page` here, and later during subsequent processing you press **CTRL+M**, ProtoId inserts a page of type `Separator_Page` and assigns the image file `blank.tif`. If you specify an invalid type, ProtoId assigns the type of page that is selected when you press **CTRL+M**.

5. Click **Save**.

Controlling the list of available page types

By default, ProtoId lists all available page types in the drop-down list below each thumbnail image.

If you want to limit the available page types or display aliases, create a dictionary of available page types in the application's document hierarchy XML file. The dictionary must have the name `PageNames`, for example:

```
<DICT n="PageNames">
  <W v="Page_Type_1">Page_Type_1</W>
  <W v="Page_Type_2">Page_Type_2</W>
  <W v="Page_Type_3">Page_Type_3</W>
</DICT>
```

Disabling document integrity checking

By default, ProtoId checks document integrity automatically when you click **Done**. You cannot complete the batch if there are integrity problems.

To disable automatic document integrity checking:

1. Start the Datacap Web Client, click the **Administrator** tab, and click **Workflow**.
2. Select the task that uses `ProtoID.aspx`.
3. Click **Setup** in the Selected task details section.
4. In the `task.set.xml` Webpage Dialog, scroll down to the Page ID section and clear the **Document Integrity** check box.
5. Click **Save**.

Running rules from ProtoID

ProtoId uses the same RRS settings as the VeriFine verification client (see “Verification by using the VeriFine web client” on page 230). The specified task profile runs immediately before document integrity checking.

Using “super variables”

The **ALT+S** hot key assigns a super variable to the selected page in the runtime batch hierarchy, for example:

```
<P id="TM000001">
  <V n="Super">Three</V>    <-- Super variable assigned a value of "Three"
  <V n="STATUS">0</V>
  <V n="TYPE">Rental_Agreement</V>
  etc.
```

The value is displayed above the page thumbnail image.

To specify the available super variable values, add the values to the **Special Variable values** field in the task's Setup dialog in the Datacap Web Client.

For example:

Special Variable values: One,Two,Three

Each time that you press **ALT+S**, ProtoId assigns the next value in the series. In this example, the first time you press **ALT+S**, it assigns a super variable value of One. The second time, it assigns Two; the third time, it assigns Three; the fourth time, it removes the Super variable.

You can use super variables to perform additional integrity checking by specifying a page type with each value, for example:

```
SuperVars=One|Rental_Agreement,Two|Air_Ticket,Three|Room_Receipt
```

ProtoId uses these *<value>|<page_type>* combinations during document integrity checking. In this example:

- Integrity checking fails if you assign Super=One to a page that is not of type Rental_Agreement.
- Integrity checking fails if you assign Super=Two to a page that is not of type Air_Ticket.
- Integrity checking fails if you assign Super=Three to a page that is not of type Room_Receipt.

In addition, you can use the tilde (~) character to indicate values that are not valid for a specific type, for example:

```
[PageID]  
SuperVars=One|~Rental_Agreement
```

In this example, integrity checking fails if you assign Super=One to a page that is of type Rental_Agreement.

Administering an application

You use the **Administrator** tab in the Datacap Web Client for all administration tasks. You use this tab to configure your application from any machine on the network.

The **Administrator** tab gives you access to a setup window where you can configure task settings.

To configure a task:

1. Start the Datacap Web Client and click the **Administrator** tab.
2. On the **Workflow** page, expand the job that contains the task that you want to configure.
3. Select the task that you want to configure.
4. In the Selected task details pane, you can:
 - a. Specify the **Mode**, such as **Batch Creation** or **Router**), and any queuing or storing options, by selecting a value from the corresponding menu.
 - b. Specify the program that the task uses, such as Rulerunner, Datacap Desktop, or one of several .aspx web pages.
 - c. Click **Setup** to open the task.set.xml - Webpage dialog window where you can select or configure more settings.

Important: The options that are available vary, depending on the program that you specify for the task. You must click **Save** in the Webpage dialog to save the additional configuration settings.

5. Click **Apply** in the Selected task details pane.

Job monitoring

You can use **Monitor** tab in the Datacap Web Client to monitor the status of the job queue.

The labels at the top of the **Monitor** tab are active.

- The **Items per page** label controls how many jobs are displayed.
- The **Delete batches** label deletes all displayed batches. Use the Batch, Job, Task, or Status fields to control which jobs are displayed, or use the **Filter** button.

Tip: To delete an individual batch, click the batch number and then click **Delete**.

- The **Filter** label provides finer control over which jobs are displayed
- The **Refresh** label refreshes the job list (or set the rate to refresh automatically)
- The **Default** label returns to the default view (all jobs)

You can also use the web Monitor to perform the following tasks:

- Click the QID to run a batch
- Click the batch number to view the batch details, change the batch status, and optionally delete the batch.

TravelDocs: Scanning from Datacap Web Client

You can create, configure, and run tasks that are related to scanning batches remotely by using Datacap Web Client.

“Creating a remote scan task”

“Configuring the remote scanning client” on page 250

“Configuring the Upload task” on page 250

“Scanning and uploading a batch” on page 251

“Creating the web Job CreateDocs task” on page 251

“Configuring Rulerunner to run web jobs” on page 252

“Modifying the Verify shortcut” on page 253

“Opening the batch for verification” on page 253

Creating a remote scan task

You are going to need a TWAIN scanner to create a remote scan task. Before you proceed, make sure that the scanner is connected and functioning.

Attention: If you want to run Datacap Web Client from a different machine, run the WebClientConfig utility on that machine to configure the required Internet Explorer security settings. You also need to add the Datacap Web Client (tmweb.net) server as a trusted site.

To create a remote scan task:

1. Start Internet Explorer and open the Datacap Web Client home page:
 - If the server is running on the same machine: <http://localhost/tmweb.net>
 - If the server is running on a different machine: http://tmweb_server/tmweb.net
2. Log in to the **TravelDocs** application and click the **Administrator** tab.
3. Click **Workflow**, expand **TravelDocs**, and then expand **Web Job**.
4. Select the iVscan task and click **Remove**. Then, click **OK**.
5. Select **Web Job** and click **New**.
6. Enter the task details as follows:

Name: Scan	Description: Scan from Datacap Web Client	Mode: Batch Creation
Queue by: None	Store: None	Program: Scanc1.aspx

7. Click **Apply**.
8. Select the new **Scan** task and click the **Up Arrow** to move the task to the top of the web Job task list.
Attention: If the new task disappears, expand any job with conditions and you can see the new task again.
9. Click **Shortcuts** and click **New**.
10. Enter the shortcut details as follows:

Name: RScan	Description: Remote Scanning
Mode: Auto	

11. Scroll down and under **Web Job**, check **Scan**. Then, click **Save**.

Important: If the web page stops responding, open the Datacap Server Manager, then stop and restart the server.

Related information:



Add tmweb.net address as a trusted site

Configuring the remote scanning client

You confirm that the remote scanning client is configured to use the two-step scan-upload process.

To configure the remote scanning client:

1. In the Datacap Web Client, click the **Administrator** tab, and then click **Workflow**.
2. Expand **Web Job** and then select **Scan**.
3. In the Selected task details section, click **Setup**.
4. In the setup window (Scan.set.xml - Webpage Dialog):
 - a. Scroll down to the Scan section.
 - b. Enter c:\datacap\scan for the **Scan into directory** field.
 - c. Confirm that the **Local processing** option is not selected Datacap scans the images to a local folder on the web client and then uses the Upload task to upload the images to the application batches folder.
 - d. Click **Save**.

Configuring the Upload task

The default application framework includes an Upload task, but you need to create a shortcut.

To configure the Upload task:

1. Start the Datacap Web Client, and log in to the **TravelDocs** application with the Admin credentials.
2. Click the **Administrator** tab and then click **Shortcuts**.
3. Click **New**.
4. Enter the shortcut details as follows:

Option	Description
Name:	Upload
Description:	Upload from Datacap Web Client
Mode:	Auto

5. Scroll to the **Web Job** option and select **Upload**. Then, click **Save**. If the web page is not responding after you save the shortcut, open the Datacap Server Manager window, and stop and restart the server.

Scanning and uploading a batch

After you create and configure the scan and upload tasks, you can run them on your batches.

To scan and upload a batch:

1. Print the file ScanPage.pdf that is included in the Sample Documents download.
2. In the Datacap Web Client, click the **Operations** tab and confirm that the two shortcuts that you created are present. If you do not see the shortcuts, click **Logout** and then log back on.
3. Click the **RScan** shortcut. Datacap Web Client displays the remote scanning page (vscancel.aspx).
4. Open a second web browser and open tmweb.net. Then, click the **Monitor** tab and set the **Refresh rate** to **10 sec**. Arrange the web browser windows so that you can see them both.

The **Monitor** tab indicates that Datacap created the batch folder and created a new entry in the job queue.

5. In the remote scanning page, confirm that your scanner is selected and that the page is in the scanner's feeder or on the flatbed. Then, click **Scan**.

Attention: If your scanner does not have a feeder, you might see information messages about unsupported features. Click **OK** to continue.

6. When the scan completes and the page thumbnail is displayed, click **Done**. Then, click **OK** and **Stop**. In the Monitor tab, you can see the job as pending for the Upload task. (You might need to wait a moment.)
7. On the **Operations** tab, click the **Upload** shortcut and wait for the file to upload. When it completes, click **OK**. You can see the job as pending for the PageID task.

Attention: Rulerunner does not process the batch automatically because it is not configured to process web jobs. This processing is done later.

8. Open the most recent batches folder. (If you are using two machines, this folder is on the Datacap Server machine). You can see the image file and the two runtime batch files:

- scan.xml is the file that is generated by the RScan task
- upload.xml is the file that is generated by the Upload task

Creating the web Job CreateDocs task

When you created the CreateDocs task, you created it for Main Jobs only. You must also create the task for web Jobs.

1. In Datacap web, click the **Administrator** tab.
2. On the **Workflow** subtab, expand the **TravelDocs** workflow and expand **Web Job**.
3. Select the **Web Job** node and click **New** to create a new task.
4. Enter the task details as follows:

Option	Description
Name:	CreateDocs

Option	Description
Description:	Create documents
Mode:	Normal
Queue by:	None
Store:	None

Attention: Since a task with the same name exists in the Main Job workflow, the fields auto-populate after you enter the name.

- Click **Apply**.
- Select the new **CreateDocs** task and click **Up Arrow** to move the task between **PageID** and **Profiler**.

Tip: If the new task disappears, expand any job with conditions to see the new task again.

Configuring Rulerunner to run web jobs

You can configure Rulerunner to run your web jobs remotely.

To configure Rulerunner to run web jobs:

- In the **Start** menu click **IBM Datacap Services > Rulerunner Manager** to open the Datacap Rulerunner Manager window.
- On the **Rulerunner** tab, click **Start** to start the Datacap Rulerunner Service.
- Click the **Rulerunner Login** tab and click **Connect**.
- Click the **Workflow: Job: Task** tab and select **TravelDocs** in the left pane.
- Under **TravelDocs > Web Job**, select **PageID**, **CreateDocs**, **Profiler**, and **Export**.
- Drag **Web Job** to **<thread0>**.
 <thread0> must be like this example:
 - <thread0>
 - TravelDocs
 - tms
 - <db>
 - Main Job
 - PageID
 - Profiler
 - Export
 - CreateDocs
 - Web Job
 - PageID
 - CreateDocs
 - Profiler
 - Export
- Click **Save**.
- Click the **Rulerunner Login** tab and click **Disconnect**.
- Click the **Rulerunner** tab and click **Start**.
- Open the Datacap Web Client and click the **Monitor** tab to watch Rulerunner process the web job through the PageID, CreateDocs, and Profiler tasks. Rulerunner stops when the web job is pending for the Verify task.

Modifying the Verify shortcut

The default Verify shortcut is only configured for Main Jobs, not web jobs. Before you can open the batch for verification, you must modify the shortcut.

1. In the Datacap Web Client, click the **Administrator** tab and then click **Shortcuts**.
2. Select the **Verify** shortcut.
3. Under the **Permissions** section in the Selected shortcut details pane, scroll to **Web Job** and click the **Verify** check box. Then, click **Save**.

Opening the batch for verification

You can verify the batch on the Datacap Web Client **Operations** tab.

1. Click the Datacap Web Client **Operations** tab, and then click **Verify**

Important: If the All documents are complete message is displayed, click **Cancel** so that you can review the page.

2. Scroll the second panel to view the check box options and the listbox control. You can see that the check box outlines are clipped, but the recognition engine is still able to identify the selected **Fuel Service** option.

Tip: To fix the clipped check box outlines, you might need to modify the image-processing settings based on a scanned page. Due to scanner differences, your scanned page might be different from the example that is described here. If the **Fuel Service** option is not selected, select **Fuel Service** before proceeding.

3. Click **Submit** and **OK** to finish the batch. Then, click **OK** and **Stop**.
4. Check the Datacap Web Client **Monitor** tab and observe Rulerunner complete the Export task.

TravelDocs: Using AIndex for manual page identification and registration

You must be running Datacap 8.0.1 Fix Pack 1 or later to complete this section.

“Making a copy of the application”

“Updating the application” on page 254

“Updating ManualPageID” on page 255

“Creating the ManualIDValidate rule” on page 257

“Running a batch through the workflow” on page 258

“Testing the ManualIDValidate rule” on page 259

Making a copy of the application

When you finish reviewing AIndex, you need to roll back the changes to the application. You can use the Datacap Studio Application Wizard to make a copy of the application and then work on the copy.

1. Start **Datacap Studio** but do not connect to any application. Instead, click **Close** in the Application window.
2. In the Datacap Studio window, click the **Datacap Application Wizard** button at the upper right.
3. In the Datacap Application Wizard dialog, click **Next**.
4. Select **Copy an existing RRS application** and click **Next**.
5. In the top field, select the **TravelDocs** application.
6. Leave the default values for the **Root folder** and **Datacap Web Client folder** fields.

7. Select **Rename copy** and enter **TravelDocs2** in the **New Name** field.
8. Click **Next** and then click **Finish**. Then, wait for the copy to complete and click **Close**.

Updating the application

You configured the PageID ruleset to use multiple page identification techniques that are implemented in a cascade fashion. To demonstrate manual page identification by using AIndex, you are going to remove the text matching and pattern matching functions and send any batch with unidentified pages to AIndex.

However, if you are using AIndex for manual page identification you must run the manual page identification task after you create a structured batch. You need to move the branching function out of the PageID task and into the CreateDocs task. You also need to make sure that each unidentified page is a separate document, which requires an update to the document hierarchy.

Updating the PageID ruleset

1. In the Datacap Studio window, click **Connection Wizard** at the upper right.
2. Select the **TravelDocs2** application, click **Next**, enter the admin password, and click **Finish**.
3. Select the **PageID** ruleset and click **Lock/Unlock rulset for editing**.
4. Expand the **PageID** ruleset and the **PageID** rule.
5. Remove the **Identify using Text Match** function and the **Identify using Pattern Match** function.
6. Expand the **Identify Manually** function and remove the **Task_NumberOfSplits** action and the **Task_RaiseCondition** action.
7. Change the parameters on the rrSet action as shown in the following table.

Library	Action	Parameter
runner	rrSet	varSource = Yes
		varTarget = @B.ManualID

8. Click **Save**. Then, click **Lock/Unlock ruleset** and choose **Publish ruleset**. The PageID rule should look like the following example.

Updating the Document Integrity ruleset

The CreateDocs task profile includes two rulesets: CreateDocs and Document Integrity. Because you need to branch to AIndex after you create the structure batch, you add the branching function to the Document Integrity ruleset.

Previously, you used the Document Integrity ruleset to raise a branch condition if the batch had structural problems that required manual fixup. You are going to modify this ruleset to raise the branch condition if the batch contains any pages that require manual identification.

1. Select the **Document Integrity** ruleset and click **Lock/Unlock rulset for editing**.
2. Expand the **Document Integrity** ruleset, the **Batch Document Integrity Check** rule, and both functions.
3. Remove the **CheckAllIntegrity** action from the first function and replace it with the following action.

Library	Action	Parameter
runner	rrCompareNot	object1 = @B.ManualID object2 = Yes

Attention: This action returns `False` if the batch variable **ManualID** is `Yes`, which causes the rule to start the **Batch Route To Fixup** function. The **PageID** rule configures this variable to `Yes` if the batch includes pages that failed fingerprint matching.

4. Click **Save**. Then, click **Lock/Unlock ruleset** and choose **Publish ruleset**.

Updating the Document Hierarchy

1. In the document hierarchy pane, click **Lock DCO for editing**.
2. Expand the Document Hierarchy so that you can see the **Other** page and the **Air Ticket** page.
3. Right-click the **Other** page and choose **Manage variables**. Then, set the **Order** value to `-1` and click **Done**.

Attention: Setting the **Order** value to `-1` causes the **CreateDocs** action to create a new document for each page of type `Other`.

4. Expand the **Air_Ticket** page.
5. Right-click the **Vendor_Logo** field and choose **Manage variables**.
6. Click **New**, type `Required`, and press **Enter**.
7. Set the value of the **Required** variable to `1` and click **Done**.
8. Click **Save changes** and then click **Unlock DCO**.

Changing the branch condition

1. Open the Datacap Web Client and click the **Administrator** tab.
2. Click **Workflow** tab, expand **Main Job**, expand **CreateDocs**, and select the **Document Integrity Failed** condition.
3. Click the down-arrow beside the **Child Job** field and choose **ManalPageID Job**.
4. Click **Apply** and then click **Done**.

Updating ManualPageID

You must configure the **ManualPageID** task to use `aindex.aspx`. You also must add the required page and field status settings and set the template path to point to the fingerprint folder of the application. Additionally, you must create a rule that runs if a user changes an existing page status.

The page and field status settings for **AIndex** are the same as those settings for **VeriFine** (see “Configuring the VeriFine client” on page 230).

“Ignored field statuses”

“Done field statuses” on page 256

“Done page statuses” on page 256

“Validation statuses” on page 256

“Editing the ManualPageID settings” on page 256

Ignored field statuses:

The **Ignored Field Statuses** field determines which fields to hide, meaning **AIndex** does not display any field that has one of the specified status values.

Because you are using AIndex for manual page identification, there is no field data and it is not necessary to display a page of empty fields. Therefore, you can hide all of the fields by entering 0, -1 for the **Ignored Field Statuses**.

- 0 is the default status of each field when the page data file is created initially
- -1 is typically assigned to anchor fields so that they can be hidden

Done field statuses:

The **Done Field Statuses** field determines which fields to hide when the **Problem fields only** check box is selected. Because you are hiding all of the fields, it does not really matter which value you enter. However, you can enter 0 (the standard setting) for the **Done Field Statuses** field.

Done page statuses:

The **Done Page Statuses** field is used to determine when a batch is complete. When all of the pages in a batch have one of the specified values, you can complete the batch. Otherwise, you can put the batch on hold only.

By default, the VScan task assigns a status of 49 on all pages. In the PageID task, you assign a status of 1 to the batch if it includes pages that failed fingerprint matching.

When you manually assign a page type in AIndex, AIndex assigns the status that is specified as the first Validation status value (for example, 0,2,1, where Done status = 0; Override status = 2; Fail status = 1). You can assign a status of 0, and so you need to set a value of 0,49 for the **Done Page Statuses** field. In this manner, the user can complete the batch only when all of the pages are identified.

Validation statuses:

The **Validation Statuses** field specifies three page status values.

- The first value is the done status that is assigned when you set the page type for an unidentified page. When you set up the **Done Page Status** values, you assigned a value of 0.
- The second value is used for validation overrides. These values do not apply in this situation, but you can assign the standard override value, which is 2.
- The third value is assigned if you change an existing page type. However, it is also the failed status. AIndex does not complete the batch if a page has this status, regardless of what you put in the **Done Page Status** field. In most situations users do not change an existing page status, but they might mistakenly, or for some other reason, do so. You can manage this situation by using a rule (see “Creating the ManualIDValidate rule” on page 257). You can assign the value 99 initially and then use the rule to assign the done status.

To set the done status to 0, the override status to 2, and the failed status to 99, enter a value of 0,2,99 in the **Validation Statuses** field.

Editing the ManualPageID settings:

The ManualPageID settings control how AIndex manages pages and fields with the specified status values.

To edit the ManualPageID settings:

1. Open the Datacap Web Client, log on to the TravelDocs2 application, and click the **Administrator** tab.
2. On the Workflow page, expand **ManualPageID Job**, and select the **ManualPageID** task.
3. In the Selected task details pane, select **aIndex.aspx** in the **Program** field under the Parameters section.
4. Click **Setup**.
5. In the ManualPageID.set.xml -Webpage dialog window, enter these values for the corresponding fields:

Option	Description
Done Page Statuses	0,49
Validation Statuses	0,2,99
Done Field Statuses	0
Ignored Field Statuses	-1,0
Main Task Profile	ManualIDValidate
RuleRunner Service Log	3
Batch Log	1
RRS Type	LocalRRS
WRRS URL	http://127.0.0.1/RRS/
Template Folder	C:\Datacap\TravelDocs2\fingerpint

6. Click **Save**.

Creating the ManualIDValidate rule

AIndex assigns the failed (DOF) page status if you change an existing page type. You cannot complete the batch if any page has a failed status, so you need a rule to change the status to done.

The user must click **Submit** for each changed pageto run the rule. The rule does not do anything except return True, but this value is enough to cause AIndex to assign the done status to the page.

1. In Datacap Studio, in the Rulesets pane, right-click **TravelDocs2** and choose **Add Ruleset**.
2. Change the name of the new ruleset to ManualIDValidate and then select **Function1**.
3. Click the **Actions library** tab and expand the **rrunner** library.
4. Select the **Status_Preserve_OFF** action and click **Add to function** to add the action to Function1.
5. In the Rulesets pane, click **Save**. Then click **Lock/Unlock ruleset** and choose **Publish ruleset**.
6. Click the **Task profiles** tab and then click **Lock/Unlock task profiles**.
7. Click **Add a new task profile**, select **Custom**, type ManualIDValidate, and click **OK**.
8. Select the new **ManualIDValidate** profile, select the **ManualIDValidate** ruleset, and click **Add ruleset to profile** at the left of the Task Profiles pane.
9. Click **Save** and then click **Lock task profiles**.

Running a batch through the workflow

Because Rulerunner is not configured to run the TravelDocs2 application, you need to use Datacap Web Client to run the batch through the background tasks.

1. Using Windows Explorer, open C:\Datacap\TravelDocs2\images.
2. Delete the page NewAirline.tif.
3. Copy the file OffsetAirTicket.tif from the sample documents download into the C:\Datacap\TravelDocs2\images folder. (This file is the same file that is used earlier in the “Pattern Matching” on page 187 section.)

Important: The \images folder must contain the files Images_Page_01.tif through Images_Page_13.tif and OffsetAirTicket.tif.

4. Open the Datacap Web Client and log on to the TravelDocs2 application.
5. In the **Operations** tab:
 - a. Click **VScan**, browse to the location of the image files, click open and click **Scan**. When the task completes, click **OK** and **Done**. Then, click **OK** and **Stop**.
 - b. Click **Upload**. When the task completes, click **OK** and **Stop**.
 - c. Click **CreateDocs** and wait for the task to complete. Then, click **Stop**.
6. Start Datacap Desktop, log in to TravelDocs, and select the **PageID** shortcut. When the task completes, click **OK**, and exit from Datacap Desktop.
7. In the Datacap Web Client **Operations** tab, click **CreateDocs** and wait for the task to complete. Then, click **Stop**.
8. In the Datacap Web Client, click the **Monitor** tab to open the Job Monitor page, which shows the batch that is pending for the ManualPageID task.
9. On the **Operations** tab, click **ManualPageID**. The AIndex web client displays the last page of the batch. Although the PageID task assigned the page type **Other** to this unidentified page, AIndex assigns the first page type, **Rental Agreement**, and forces you to change it because it has STATUS=1.
10. Use the menu at the top of the batch tree view pane to set the document type to **Flight** and the page type to **Air_Ticket**.
11. When you select the Air_Ticket page type, Datacap activates the **Anchors** button, which prompts you to set anchors for the page. Click **OK** to close the message box.
12. Click **Anchors**. Datacap prompts you to select a thumbnail image. Click **OK** to close the message box.

Attention: Datacap displays all fingerprints that have an anchor field with the same name as the anchor field defined for the selected page type. In this case, the **Vendor_Logo** field is defined only for the **Air_Ticket** page. So, only the air ticket fingerprint thumbnails are displayed.
13. Double-click the **Airline #2** fingerprint thumbnail (the second one). Then, use the mouse to align the red anchor object over the Airline #2 vendor logo
14. Click **Done** to complete the batch. Then, click **OK** and **Stop**.
15. Since the batch is now pending for the Profiler task, Start Datacap Desktop, log in to TravelDocs, and select the **Profiler** shortcut. When the task completes, click **OK**, and exit from Datacap Desktop.
16. In the Datacap Web Client **Operations** tab, click **Verify** to open the batch in the AIndex web client for verification.
17. Complete the batch by clicking **Submit** for each page. On page TM000004, set the car type to **Other**. On pages TM000006 and TM000013, select the yellow checkbox at the top of the page to override the validation failures.

Tip: You might need to scroll till end of the data entry panel to see the **Submit** button. Also, you cannot select multiple options for the **Options** groups on the Rental_Agreement pages because the AIndex web client does not support the MultiPunch variable.

Testing the ManualIDValidate rule

Run another batch through the workflow but this time change the page status on one of the pages to start the ManualIDValidate profile.

1. Start and log in to the Datacap Web Client, and click **VScan** on the **Operations** tab.
2. In the VScan window, click **Browse**, go to C:\Datacap\TravelDocs\images. Select a file and click **Open**, then click **Scan**.
3. When the VScan task displays a message to indicate that the scanning is complete, click **OK**, click **Done** then click **OK** and **Stop**.
4. On the Datacap Web Client **Operations** tab, click **Upload**.
5. When the Upload task displays a message to indicate that the task is complete, click **OK** and then click **Stop** to return to the **Operations** tab.
6. Start Datacap Desktop, log in to TravelDocs, and select the **PageID** shortcut. When the task completes, click **OK**.
7. In the Datacap Web Client **Operations** tab, click the **CreateDocs** icon and wait for the task to complete. Then, click **Stop**.
8. On the Datacap Web Client **Operations** tab, click the **ManualPageID** shortcut.
9. For the last page, set the document type to **Flight** and the page type to **Air_Ticket**. Then, click **Anchors** to assign the matching fingerprint and register the page, as you did before.
10. In the batch tree view pane, scroll up to the page list and select the first **Rental_Agreement** page.
11. Use the dropdown list to change the document type to **Hotel**. Then, change it back to **Car_Rental**. AIndex sets the page status to failed (99).
Attention: If you want to confirm this status, click **Hold** to put the batch on hold and then open the file manualpageid.xml in the current batch folder. You see `<V n="STATUS">99</V>` under page **TM000001**. Then, reopen the batch by clicking the batch ID on the **Datacap Web Monitor** tab.
12. With the Car Rental #1 page displayed, click **Submit**. AIndex runs the validation rule and changes the page status to 0.
13. Click **Done** to complete the batch. Then, click **OK** and **Stop**.

Filter batches by group in the Job Monitor

You can filter batches by groups in the Job Monitor based on your ADSI, LDAP, or LLLDAP group authentication.

The view of batches in Job Monitor can be restricted to batches that are assigned to a specific group. The application's rules determine the groups that can view the batch. Four steps must be completed to filter batches by group in the Job Monitor.

- You must set up your ADSI, LDAP, or LLLDAP group authentication in the Datacap Server Manager. If LLLDAP is used, the group-based authentication option must be used. For information on setting up LLLDAP group authentication, see *LLLDAP group authentication*.
- A custom column must be defined in the tmBatch table in the Datacap Engine database. For information on defining a custom column, see *Creating a custom column in the Job Monitor*.

- The group names must be set up for filtering purposes by defining a custom Job Monitor filter in the Application Manager. The two methods for filtering are with exclusive groups and additive groups.
 - You can define a maximum of 31 additive groups. Each user can be member of multiple groups and each batch can be assigned multiple groups. The user must belong to all of the groups that are assigned to the batch to view that batch.
 - You can define up to 32,000 exclusive groups. Each user can be a member of only one exclusive group and a batch can be assigned to one filter group. You can create a supervisor group whose members can view all batches with exclusive filters.
- You must assign one or more groups to each batch to be filtered by adding a rule in Datacap Studio. When groups are assigned to a batch by this rule, the batch can be viewed in Job Monitor only by members of the assigned group.

The batches that you run by selecting **Datacap Web Client > Operations > Run Shortcut** can be limited by using additive group filters in the same way that those batches are limited in Job Monitor. You cannot use exclusive group filters to limit batches that are run when you select **Run Shortcut**.

“Defining group names for filtering batches”

“Assigning a group to a batch for filtering” on page 261

Related information:



LLLDAP group authentication



Creating a custom column in the Job Monitor

Defining group names for filtering batches

You can define group names in the Application Manager for filtering batches by group.

Add each ADSI, LDAP, or LLLDAP filtering group to the **Custom Values** tab of the Application Manager.

You can define a maximum of 31 additive groups. Each user can be member of multiple groups. Each batch can be assigned multiple groups and the user must belong to all of the groups that are assigned to the batch.

You can define up to 32,000 exclusive groups. Each user can be a member of only one exclusive group and a batch can be assigned to one filter group. You can create a supervisor group whose members can view all batches with exclusive filters.

1. Open the Application Manager and select the application that you want from the list of applications.
2. Select the **Custom values** tab.
3. In the **General string values** field, add each filter group **Value name** and **Value**.

The group **Value Name** consists of the word Group followed by a number.

- For additive groups, the number is 0 - 31
- For exclusive groups, the number is 1 - 32767.

The **Value** of each group is the name of the ADSI, LDAP, or LLLDAP user group to associate with the internal group number. The group name value is not case-sensitive but it must include the domain, if any, such as NYOffice.SomeDomain.

Assigning a group to a batch for filtering

You can assign one or more groups to each batch to be filtered by defining a custom Job Monitor filter and adding a rule in Datacap Studio. When the groups are assigned to a batch, the batch can be viewed in Job Monitor only by members of the assigned groups.

1. Define the custom Job Monitor filter by adding the CustomJMFilter and CustomMultiGroup values to the Application Manager.
 - a. Open the Application Manager and select the application that you want from the list of applications.
 - b. Select the **Custom values** tab.
 - c. In the **General string values** field, add the custom Job Monitor filter **Value name**, CustomMultiGroup. Set the value to **0** for exclusive group filtering or set the value to 1 for additive filters. The default value is 1.
 - d. In the **General string values** field, add the custom Job Monitor filter **Value name**, CustomJMFilter. The appropriate value depends on your database type and filter type. If the custom column in Job Monitor is named, pb_allowed, define the additive or exclusive filter as follows.
 - For additive filtering with Oracle, enter `bitand(CAST(pb_allowed As Integer),{0}) = CAST(pb_allowed As Integer)`
 - For additive filtering with SQL Server, enter `CAST(pb_allowed AS INT) & {0} = CAST(pb_allowed AS INT)`
 - For exclusive filtering with Oracle, enter `CAST(pb_allowed As Integer) = {0}`
 - For exclusive filtering with SQL Server, enter `CAST(pb_allowed AS INT) = {0}`
 - For exclusive filtering where **Group1** is the supervisor group with SQL Server, enter `(CAST(pb_allowed AS INT) = {0} OR 1 = {0})`
 - For exclusive filtering with MS-Access, enter `CInt(pb_allowed) = {0}`
2. Add a rule in Datacap Studio to assign one or more groups to each batch.
 - You can assign groups immediately when the batch is scanned. Add the rule to a validation task profile for an interactive start batch panel that is based on the scan operator, job type, or other metadata that is defined in the start batch panel.
 - You can assign groups later in the workflow as part of background processing rules that are based on any metadata in the batch.

Your rule must call a custom action to set the custom Job Monitor column named pb_allowed. Set the value to group number for exclusive groups or the sum of additive group values.

Fingerprint Management

Fingerprints are used both for page identification and for specifying recognition zones. The following topics review basic fingerprint functions, provide more details about the fingerprint database, and examine an alternative method for storing zone position information with fingerprint XML (FPXML) files. Later, you can update the TravelDocs application to use FPXML.

“Review of basic fingerprint functionality” on page 262

“The Fingerprint database” on page 263

“Using fingerprint XML files” on page 264

“TravelDocs: Updating auto fingerprinting to use FPXML” on page 267

Review of basic fingerprint functionality

In Datacap applications, fingerprints have two basic functions.

- You can use them during page identification to determine if the incoming page matches a known page:
- You can use them to identify the field positions for each variant of each known page type:

“Create fingerprint files”

“Add fingerprints to the fingerprint library”

“Define field zones” on page 263

Create fingerprint files

Datacap provides two methods for creating fingerprint files, image analysis and full page recognition.

- Image analysis: This method scans the page image to identify the composite blackness of different regions of the page. It does fast page identification, but requires that you perform recognition later.
- Full page recognition: This method performs optical character recognition to identify the locations of text within the page. The full page recognition method takes longer, especially with pages that include handwritten text. However, this method reduces time from subsequent workflow tasks because the full page recognition results are available for use.

For both methods, Datacap creates two files each time you generate a new fingerprint.

- A TIFF file with an image of the page.
- A CCO file with the fingerprint information.

If you are using full page recognition, Datacap creates a temporary XML file that is generated during recognition.

Attention: The method that you use for creating library fingerprints must be the same as the method you use to generate runtime fingerprints during page identification.

- An XML file (*<fingerprint_id>c.xml*) with the recognition results.

Add fingerprints to the fingerprint library

You can add fingerprints to the fingerprint library from the Datacap Studio **Zones** tab. Each time that you add a fingerprint, Datacap starts the *FingerprintAdd* rule set. In the fingerprint library, you can define the fingerprint generation method, such as image analysis or full page recognition.

You can add new fingerprints and define the recognition zones using actions. You can also create fingerprints for manually identified pages. (See “Creating the AutoFingerprint ruleset” on page 219.)

Fingerprint files are stored in the application's fingerprint folder. The location of this folder is specified in the Datacap Application Manager and stored in the application configuration (.app) file.

In addition to saving the TIFF and CCO files, Datacap creates an entry for the new fingerprint in the fingerprint database. (See “The Fingerprint database” on page 263). The fingerprint database is also specified in the .app file.

Define field zones

The position of each field zone is defined by coordinates (x1, y1, x2, y2) that specify the upper left and lower right corners of the zone relative to the upper left corner of the page.

During recognition, Datacap uses the zone information to determine where the required information is on the page. If the runtime page image is not aligned precisely with the matched fingerprint, Datacap uses the calculated offset values to adjust the zone positions.

When you define the field zones in Datacap Studio, Datacap writes the position information into the document hierarchy. If you are defining the zones during verification, the **iloc_SetZones** action does the same. The document hierarchy XML file example shows the position of the **Pickup_Date** field for three different fingerprints:

```
<F type="Pickup_Date">
  <V n="ID">0</V>
  <V n="TYPE">Field</V>
  <V n="STATUS">0</V>
  <V n="POSITION">0,0,0,0</V>
  <V n="MIN_TYPES">0</V>
  <V n="MAX_TYPES">0</V>
  <V n="ReqConf">8</V>
  <V n="rules">&lt;in&gt;&lt;r id="1" rs="9" /> </V>
  <V n="Pos556">183,402,535,463</V>
  <V n="Pos558">568,331,967,389</V>
  <V n="Pos560">1199,389,1600,448</V>
</F>
```

The last three entries in the XML file indicate the Zone position for fingerprints 556, 558, and 560.

It is also possible to store the field position information for each fingerprint in a separate file. For more information, see “Using fingerprint XML files” on page 264. This is helpful if your application has many fingerprints.

The Fingerprint database

The information to manage the application fingerprint files is stored in the Fingerprint database of the application. By default, the Access database file (<app_name>Fingerprint.mdb) is stored in the root of the application folder.

The connection string for the Fingerprint database is defined through the Datacap Application Manager and stored in the application configuration file.

The fingerprint database includes three tables:

- **Host:** This table defines the name, host ID, and reference ID for each fingerprint class, for example:

Name	Host ID	Reference ID
<Global>	9	-1
Car_Rental	226	DC226
Flight	227	DC227
Hotel	228	DC228

Tip: You can see the host ID and reference ID by moving the mouse pointer over the class name in Datacap Studio.

- **PageType:** This table defines the name and ID for each page type, for example:

Page Type Name	Page Type ID
Other	1
Rental_Agreement	40
Optional_Insurance	41
Air_Ticket	42
Room_Receipt	43
Meals	44
Other_Charges	45

- **Template:** This table defines the ID, CCO file, TIFF file, host ID (class), and page type for each fingerprint, for example:

ID	CCO Path	Image Path	Host ID	Page Type
555	C:\Datacap\...\fingerprint\555.cco	C:\Datacap\...\fingerprint\555.tif	9	1
556	C:\Datacap\...\fingerprint\556.cco	C:\Datacap\...\fingerprint\556.tif	226	40
557	C:\Datacap\...\fingerprint\557.cco	C:\Datacap\...\fingerprint\557.tif	226	41
558	C:\Datacap\...\fingerprint\558.cco	C:\Datacap\...\fingerprint\558.tif	227	42
559	C:\Datacap\...\fingerprint\559.cco	C:\Datacap\...\fingerprint\559.tif	228	43

Using fingerprint XML files

A fingerprint typically consists of an entry in the fingerprint database, a TIFF image file, a CCO fingerprint file, and position information stored in the document hierarchy XML file.

This setup works well when the number of fingerprints is small, but as the number of fingerprints increases the document hierarchy file gets bigger and the time to locate the position information increases.

“The fingerprint XML file”

“Enable FPXML” on page 265

“Exporting existing position information from the document hierarchy” on page 266

The fingerprint XML file

You can move the zone position information of a fingerprint out of the document hierarchy and into a separate fingerprint XML (FPXML) file.

Document hierarchy XML:


```

<F type="Pickup_Date">
  <V n="ID">0</V>
  <V n="TYPE">Field</V>
  <V n="STATUS">0</V>
  etc.
  <V n="Pos556">183,402,535,463</V>
  <V n="Pos558">568,331,967,389</V>
  <V n="Pos560">1199,389,1600,448</V>
  </F>
<F type="Pickup_Location">
  <V n="ID">0</V>
  <V n="TYPE">Field</V>
  <V n="STATUS">0</V>
  etc.
  <V n="Pos556">180,528,532,589</V>
  <V n="Pos558">573,448,967,502</V>

```

Fingerprint XML file:

```

<S>
  <P type="Rental_Agreement">
    <V n="HostID">226</V>
    <V n="HostName">Car_Rental</V>
    <F type="Pickup_Date">
      <V n="Postion">183,402,535,463</V>
      </F>
    <F type="Pickup_Location">
      </V>
      <V n="Postion">180,528,532,589</V>
    </F>
    etc.

```

Although the total number of files is increased because each fingerprint now has a TIFF file, a CCO file, and an XML file, this approach has several benefits:

- The document hierarchy XML file remains small.
- Overall performance can increase.
- It eliminates possible contention if multiple users attempt to add fingerprints simultaneously.

Attention: These benefits apply to applications that use dynamic, action-based fingerprint generation, because Datacap Studio writes to the document hierarchy and the fingerprint XML file when FPXML is enabled.

Enable FPXML

You can add fingerprints using the Datacap Studio Zones tab or by using actions from a verification panel.

“Adding fingerprints using the Datacap Studio Zones tab”

“Add fingerprints using actions” on page 266

Adding fingerprints using the Datacap Studio Zones tab:

By default, Datacap Studio writes the zone position information to the application's document hierarchy XML file, but can write to a fingerprint XML file.

To configure Datacap Studio to write zone position information to a fingerprint XML file, select the **Enable FPXML** option in the Application Manager.

After you enable FPXML and restart Datacap Studio, each time you add a fingerprint or modify an existing fingerprint's zones, Datacap Studio creates or updates the fingerprint XML file.

Important: Datacap Studio also adds the information to the document hierarchy XML file since it uses this information to display the zone outlines on the Image tab.

Add fingerprints using actions:

The FPXML library includes actions for reading and writing zone information using fingerprint XML files.

Library	Action	Description
FPXML	SetDirectoryFPX	Sets the location for the fingerprint XML files.
FPXML	ReadZonesFPX	Loads the zone position information for the current fingerprint.
FPXML	WriteZonesFPX	Writes the position information for all fields on the current page.

For a demonstration of how to use these actions when you update the TravelDocs application, see the topic “Updating the AutoFingerprint ruleset” on page 267.

Exporting existing position information from the document hierarchy

The techniques that are described in the previous section work for fingerprints that are generated after you decided to use FPXML. For existing fingerprints, the zone position information is still in the document hierarchy.

With the Fingerprint Maintenance Tool, you can export existing position information from the document hierarchy and into individual XML files. The Fingerprint Maintenance Tool is included as an APT-specific utility. However, it is possible to use the tool with other Datacap applications by following the instructions that are provided.

Important: If you move existing zone position information out of the document hierarchy and into FPXML files, you must update the actions that your application uses to read the zone information. The **ReadZones** action reads information from the document hierarchy, whereas the **ReadZonesFPX** action reads information from FPXML files.

“Setting up the Fingerprint Maintenance Tool for your application”

“Exporting the position information” on page 267

Setting up the Fingerprint Maintenance Tool for your application:

You need to copy the Fingerprint Maintenance Tool files to your application directory and configure the tool for use with your application.

1. In the folder C:\Datacap\APT\dco_APT, locate the files Fingerprint Maintenance Tool.exe and Interop.DCAppleLib.dll.
2. Copy the two files from C:\Datacap\APT\dco_APT to C:\Datacap*<app_name>*\dco_*<app_name>*, where *<app_name>* is the name of your application)
3. Using a text editor, create a file that is called Settings.ini in the C:\Datacap*<app_name>*\dco_*<app_name>* folder and insert the following information (replace *<app_name>* with the name of your application):

```

[Database]
FingerprintDatabase=Provider=Microsoft.Jet.OLEDB.4.0;Data
Source=C:\Datacap\<app_name>\<app_name>Fingerprint.mdb;Persist Security
Info=False
[Paths]
FingerprintDirectory=C:\Datacap\<app_name>\fingerprint
FingerprintBackupDirectory=C:\Datacap\<app_name>\Fingerprint Backup
SetupDCO=C:\Datacap\<app_name>\dco_<app_name>\<app_name>.xml
[FMT]
FilteredSummary=Select
Template.tp_TemplateID,Template.tp_DateAdded,Template.tp_HitCount,Template.tp_L
astHit,Host.hs_RefName FROM Template,Host WHERE host.hs_HostID =
Template.tp_HostID

```

Exporting the position information:

This procedure removes the zone position information from the document hierarchy XML file (C:\Datacap\< app_name>\dco_<app_name>\<app_name>.xml). Make a backup copy of this file before you export.

1. Double-click Fingerprint Maintenance Tool.exe.
2. Confirm that the information displayed at the top of the FMT window is correct.
3. Click the **DCO to FPXML** button and click **OK** to create the FPXML files.

TravelDocs: Updating auto fingerprinting to use FPXML

You can update the AutoFingerprint ruleset to save new fingerprint position information in a separate fingerprint XML file. You can also update the Recognize Page rule to read the zone information from a fingerprint XML file, if one exists.

“Updating the AutoFingerprint ruleset”

“Updating the Recognize Page rule” on page 268

“Preparations for running a batch through the workflow” on page 268

“Running a batch through the workflow” on page 268

Updating the AutoFingerprint ruleset

You can update the AutoFingerprint ruleset to save new fingerprint position information in a separate fingerprint XML file.

To update the AutoFingerprint ruleset:

1. Start Datacap Studio and connect to the **TravelDocs** application.
2. On the **Rulemanager** tab, in the Rulesets pane, select the **AutoFingerprint** ruleset and click **Lock/Unlock ruleset**.
3. Expand the **AutoFingerprint** ruleset completely.
4. Right-click the **ilocSetZones** action and choose **Remove**.
5. Add the actions and parameters in the table below to the end of Function1.

Library	Action	Parameter
FPXML	SetDirectoryFPX	@APPPATH(fingerprint)
FPXML	WriteZonesFPX	@D.TYPE,@P.TYPE,@P.TYPE

Attention: The **WriteZonesFPX** action sets the fingerprint host name to the current document type, the fingerprint host ID to the current page type, and the fingerprint page type to the current page type.

6. Click **Save**. Then click **Lock/Unlock ruleset** and choose **Publish Ruleset**.

Updating the Recognize Page rule

You can update the Recognize Page rule to read the zone information from a fingerprint XML file.

1. On the Datacap Studio **Rulemanager** tab, in the **Rulesets** pane, select the **Recognize** ruleset and click **Lock/Unlock ruleset**.
2. Expand the **Recognize** ruleset and the **Recognize Page** rule.
3. Change the name of the function to **Recognition: Fingerprint-Non-FPXML**.
4. Remove the **rrCompareNot** action and replace it with the following action and parameters:

Library	Action	Parameter
runner	rrCompare	object1 = @P.MatchType object 2 = Fingerprint

5. Right-click the **Recognize Page** rule and choose **Add Function**.
6. Rename the new function to **Recognition: Fingerprint-FPXML** and use the **Up Arrow** button to move it to the beginning of the rule (before the other recognition function).
7. Add the actions and parameters below to the **Recognition: Fingerprint - FPXML** function.

Library	Action	Parameter
runner	rrCompare	object1 = @P.MatchType object 2 = Fingerprint
FPXML	SetDirectoryFPX	@APPPATH(fingerprint)
FPXML	ReadZonesFPX	
Recog_Shared	SnapCCOtoDCO	

8. Click **Save**. Then click **Lock/Unlock ruleset** and choose **Publish Ruleset**.
Attention: ReadZonesFPX fails if there is no matching FPXML file and Datacap runs the Non-FPXML function.

Preparations for running a batch through the workflow

Confirm that you have the correct files for this exercise before you run the batch through the workflow.

1. Start the Rulerunner Manager. If the Rulerunner service is running, click **Stop** and wait for the service to stop.
2. In Datacap Studio, click the **Zones** tab and then click the **Refresh** button.
3. Check to see if there is already a fingerprint for the Airline #4 ticket. If there is, select it and click the **Remove selected** button. The Airline #4 fingerprint, if you have one, is under its own Flight class.
4. In Windows Explorer, open C:\Datacap\TravelDocs\images and confirm that you have the files Images_Page_01.tif through Images_Page_13.tif and NewAirline.tif.

Running a batch through the workflow

Now that you have completed updating auto fingerprinting to use FPXML, you can test it by running a batch through the workflow.

1. Start Datacap Web Client and log in to the **TravelDocs** application.

2. On the **Operations** tab, click **VScan**, browse to the location of the image files, click open and click **Scan**. When the task completes, click **OK** and **Done**. Then, click **OK** and **Stop** to return to the **Operations** tab.
3. Click **Upload**. When the task completes, click **OK** and **Stop** to return to the **Operations** tab.
4. Start Datacap Desktop, log in to TravelDocs, and select the **PageID** shortcut. When the task completes, click **OK** and exit from Datacap Desktop.
5. In the Datacap Web Client **Operations** tab, click the **ManualPageID** shortcut and wait for the page images to load. Then, scroll to the bottom and set the page type for the last page to **Air_Ticket**.
6. Click **Done** and then, click **OK** and **Stop**.
7. In the **Operations** tab, click the **CreateDocs** shortcut. When it completes, click **Stop**.
8. Start Datacap Desktop, log in to TravelDocs, and select the **Profiler** shortcut. When the task completes, click **OK** and exit from Datacap Desktop.
9. In the Datacap Web Client, check the Job Monitor. You can see the result of the split, where the child job is pending for the Supervisor Verify task and the main job is pending for the Main Verify task.
10. On the In the **Operations** tab, click the **Supervisor Verify** shortcut to open the pending batch. The Airline #4 page is displayed.
11. Define the zone for each field as you did earlier (see “Running a batch through the workflow” on page 220).
12. Click **Submit** and then, click **OK**. In the background, Datacap runs the AutoFingerprint ruleset to create the new fingerprint and the fingerprint XML file. Then, click **OK** and **Stop**.
13. In Windows Explorer, open **C:\Datacap\TravelDocs\fingerprint** and locate the most recent fingerprint. You can see a .cco, a .tif, and a .xml file for the new fingerprint.
14. Open the fingerprint XML file in a text editor. Here is an example:

```
<S>
<P type="Air_Ticket">
  <V n="HostID">Air Ticket</V>
  <V n="HostName">Flight</V>
  <F type="Outbound_From">
    <V n="Position">649,488,1046,537</V>
  </F>
  <F type="Outbound_To">
    <V n="Position">1052,486,1477,545</V>
  </F>
  <F type="Outbound_Date">
    <V n="Position">182,484,386,541</V>
  </F>
  <F type="Return_From">
    <V n="Position">646,619,1023,674</V>
  </F>
  <F type="Return_To">
    <V n="Position">1053,621,1475,671</V>
  </F>
  <F type="Return_Date">
    <V n="Position">188,617,360,675</V>
  </F>
  <F type="Airfare">
    <V n="Position">1374,884,1513,928</V>
  </F>
  <F type="Taxes">
    <V n="Position">1391,936,1509,981</V>
  </F>
</P>
```

```
<F type="Total_Cost">
  <V n="Position">1387,989,1522,1037</V>
</F>
</P>
</S>
```

Attention: If you run another batch through the workflow, it runs from end to end with no branching or splitting because the Airline #4 page is now in the fingerprint library. This time recognition task profile uses the zone position information from the fingerprint XML file.

Upgrading software and migrating applications

You use the Datacap installation program to upgrade Datacap 8.0.1 and 8.1.0 Datacap software to Datacap 9.0. Your customized code might not be retained from the previous version.

For hardware and software requirements, see System Requirements: Hardware and Software Requirements for IBM Datacap, Version 9.0.

In a production client/server environment, install the Datacap software on fresh computers. Complete the following general steps to upgrade and migrate your Datacap applications. Each step can require some extensive installation or customization tasks.

To upgrade software and migrate applications from a previous release:

1. Configure a Datacap 9.0 test environment:
 - a. In a nonproduction development location, install the new version of Datacap software to simulate your current production environment. Use computers on which no customized Datacap software is installed. You use this installation to deploy, configure, and test any modifications. You might also use new interfaces that are available on Datacap 9.0. For example, since 8.0.1 the Datacap Web Client and IBM Content Navigator administrative clients replaced the Datacap thick client. For installation details, see Installing, configuring, and running Datacap in a client/server environment.
 - b. Review the installation topics that describe the export and import of encryption keys. You must have FIPS encryption as of version 8.1.0 for password protection.
 - c. Run the upgrade to Datacap 9.0.
 - d. Manually modify your customized applications.
 - e. Open Datacap Studio and determine whether any existing rules or actions were eliminated or deprecated in the new release.
 - f. Modify your customized settings as needed. Deprecated elements are designated as Private and they cannot run in the **Test** tab in Datacap Studio. But they can run in your updated applications. In a subsequent release, remove these deprecated elements from the product.
 - g. Update the code and customized user interfaces that were not fully migrated to maintain your current level of function.
 - h. Apply any new features in the updated software version that you want to implement.
 - i. Test your updated applications and configuration in the development environment against your current production system. You run this parallel testing to validate features, performance, connectivity, and volume throughput. For example, ensure that program communication works as intended between servers and clients, web or otherwise. FIPS encryption is new as of Datacap 8.1.0 and is documented in the client/server installation topics. Encryption keys must match on all computers in the configuration to protect passwords for program-related accounts.
2. Configure a Version 9.0 production environment:

- a. Install and verify the new version of Datacap software on computers that are not running current Datacap software. For installation details, see *Installing, configuring, and running Datacap in a client/server environment*.
- b. Copy the migrated or newly customized applications from your test environment to the newly installed release. For details on migrating database information, see *Copying the application to the Datacap server*.
- c. Test and verify the functions of the new configuration before you deploy it into production.
- d. Ensure that users can connect to the newly updated production-ready configuration.
- e. Switch to the newly configured system for production use. For example, you might disable user activity permanently on the previous production system and ensure that new server IP addresses are correctly designated.

Migrating Datacap applications from 8.0.1 to 8.1.0

If you want to use applications that were created in the 8.0.1 release of Datacap in version 9.0, use the Application Wizard to convert these applications to work with the 8.1.0 release of Datacap.

To migrate an application from 8.0.1 to 8.1.0:

1. Copy the entire folder of the application that you want to convert to the \datacap folder on the Datacap 8.1.0 computer.
2. On the Datacap 8.1.0 computer, open datacap.xml and add the name of the application that you want to update. For example, if the application name is *MyApp*, copy the *MyApp* folder to \datacap.
3. If you are using DB2, Oracle, or Microsoft SQL Server, create a new database by using the creation scripts for each database type (administration, engine, fingerprint).
Attention: The creation scripts are in the *8.1.0 installation drive/datacap/support* folder.
4. If you are using a Microsoft Access database, complete these steps.
 - a. On the Datacap 8.1.0 computer, copy AppWizardAdm.mdb, AppWizardEng.mdb, and AppWizardFingerprint.mdb from /datacap/dstudio/appwizard to a temporary folder. The databases that you are copying are blank, but they contain the Datacap 8.1.0 database table formats. If the existing batches of documents are not needed in the converted application, you can use a blank Engine database. If you need the batches of documents, complete the batches before you convert the application.
 - b. Rename the three databases Adm, Eng, Fingerprint in the temporary folder to the name of the application that you are converting. For example, if the application name is *MyApp*, rename the three databases to MyAppAdm.mdb, MyAppEng.mdb, and MyAppFingerprint.mdb
 - c. If you customized any of the version 8.0.1 databases, you must add any custom columns and new columns from the 8.0.1 databases to the 8.1.0 databases.

Important: Ensure that the column names that you are adding to the 8.1.0 databases match the column names in the 8.0.1 databases.

5. Start Datacap Studio on your workstation by clicking **Start> IBM Datacap Developer Tools > Datacap Studio**.
6. In the Datacap Studio startup dialog that prompts you to select an application, click **Close**.
7. In Datacap Studio, click the **Datacap application wizard** icon.
8. In the Application Wizard Overview window, click **Next**.
9. Select **Convert an 8.0.1 application to 8.1.0 format**.
10. In the Source Application window, select the application that you want to convert.
11. In the 8.1.0 Target Database dialog, enter the database information that your updated application must use.
 - If you are using a Microsoft Access database, browse to the temporary folder where you copied the databases and select the three databases.

Important: You must enter information for each of the databases: Administration, Engine, and Fingerprint. Click the corresponding tab to enter database information.

12. Click **Next** and then **Finish**. The Application wizard displays a message to indicate a successful update, and whether you must review the Application wizard log for any errors or warnings. The log file is in *8.1.0 installation drive\datacap\application name\appwiz_convert.log*.

Important: During the conversion of your application, Datacap makes the following modifications.

- Tasks that were previously mapped to use `prelayout.aspx` are now mapped to use `verifine.aspx`.
 - Tasks that previously used project files are configured through the task setup window. You access task setup by selecting the task on the Workflow page of the **Administrator** tab in the Datacap Web Client, and clicking **Setup**.
 - The databases in the temporary folder are merged with the databases in the application folder. The databases in the temporary folder now contain the 8.0.1 application data in the 9.0.0 tables.
13. Copy the three databases in the temporary folder to the application folder and select **Yes, overwrite existing files** at the prompt. For example, the application folder `c:/datacap/MyApp`
 14. Select **Start> IBM Datacap Services> Datacap Application Manager** to start Datacap Application Manager and confirm that the three databases are pointing to the correct data source.

Tip: The Fingerprint database is configured on the **Main** tab, and the Administration and Engine databases are configured on the **Datacap** tab.

- For MS-Access databases:
 - The Fingerprint database data source must be set to *8.1.0 installation drive\datacap/MyApp/MyAppFingerprint.mdb*. Such as data source=`c:/datacap/MyApp/MyAppFingerprint.mdb`.
 - The Administration database DSN must be set to *8.1.0 installation drive\datacap/MyApp/MyAppAdm.mdb*. Such as DSN=`c:/datacap/MyApp/MyAppAdm.mdb`.
 - The Engine database DSN must be set to *8.1.0 installation drive\datacap/MyApp/MyAppEng.mdb*. Such as DSN=`c:/datacap/MyApp/MyAppEng.mdb`.
- 15. If necessary, change the path to the correct data source and restart the Datacap Server Manager on the **Service** tab of the Datacap Application Manager.
- 16. Restart Datacap Studio, log on to the application that you converted, and confirm that you can connect to the Administration and Engine databases.
- 17. Run the Datacap 9.0 installation program on the 8.1.0 computer to upgrade Datacap 8.1.0 to 9.0.

Converting customized panels to Datacap Desktop

If you used customized Batch Pilot or DotEdit panels in Datacap, you must convert that panel to Datacap Desktop on Datacap 9.0.

If you do not have Datacap 8.0.1 installed on a separate computer, you must convert the Batch Pilot panel before you can upgrade to Datacap Version 9.0.

“Generating the layout XML file”

“The layout XML file”

“Creating the Datacap Desktop in Microsoft Visual Studio” on page 276

Generating the layout XML file

You can convert your customized panels to Datacap Desktop panels. You must generate a layout XML file that contains the panel to convert and the name and location of the converted panel.

To generate the layout XML file:

1. Start the client for which you want to convert panels, Batch Pilot, or DotEdit.
2. Click **File > Open Project**.
3. Select the **Verify** project of the application and click **Open**. For example, `\Datacap\Survey\dco_Survey\verify.bpp`.
4. In the Batch View window, expand the application to the page level so that the pages are visible.
5. Right-click the page that you want to convert and choose **View Form**.
6. Click **Form > Run Script**.
7. Press **Shift+Alt+S** to run the script that exports the form information as a layout XML file.
8. Select the target location and specify the file name.
9. Click **Open** to generate the XML file.

The layout XML file

After you generate the layout XML file, you can open the file in a text editor like Notepad. You can review the control types that are listed in the file.

The layout XML file contains information about each of the standard Datacap controls that are used. The file includes the control type, control name, position, and other attributes. The script exports only the following control types.

- dcimage
- dcredit
- combobox
- label

If your form includes other control types, you must manually add them by using Microsoft Visual Studio.

Creating the Datacap Desktop in Microsoft Visual Studio

After you generate and review the layout XML file, open the layout XML file in Microsoft Visual Studio. Then, you can convert the customized panel to the Datacap Desktop panel.

To create the Datacap Desktop form in Microsoft Visual Studio:

1. In Visual Studio, open the Datacap Desktop project and press **Ctrl+F5**.
2. From the **Select User Control** list, select the **DotEditPanels.dotMaster**.
3. In the **DCO Setup** field, click **Browse** and select the application DCO XML file. For example, `C:\Datacap\MyApp\dco_MyApp\MyApp.xml`.
4. In the **Layout XML** field, click **Browse** and select the **XML** file that you generated.
5. In the **Page Type** field, select the page for which to create the custom panel.
6. In the **New name** field, enter the name for the new C# class.
7. Click **Create**. Datacap Desktop displays a message that indicates you must reload the project. You are prompted to do reload the project in the next step.
8. Click **OK** to close the message box and then close the 'dotmaster' UserControl TestContainer window.
9. Click **Reload**.
10. In the Solution Explorer, double-click the new `.cs` file to display the new custom panel that is generated from the XML file.

Smart Parameter Special Variable Reference

Special variables are for use with smart parameters. Smart parameters do not work with all actions.



Check the action help in Datacap Studio for compatibility information.

“Special variables for accessing the application configuration file”

“Special variables for accessing the runtime hierarchy” on page 279

“Special variables for accessing job and task information” on page 283

“Miscellaneous special variables” on page 285

Special variables for accessing the application configuration file

These variable return information that is related to the application configuration (.app) file.

“@APPPATH(<key_path>)”

“@APPVAR(<key_path>)” on page 278

@APPPATH(<key_path>)

Description

Retrieves the path to a file or folder from the application's configuration (.app) file. Use the Datacap Application Manager to modify the information that is contained in this file. Do not edit the file directly.

Syntax

@APPPATH(key_path)

Arguments

key_path Specifies the path through the XML hierarchy to field you want:

- If the field name is unique within the file, you can specify just the field name.
- If the field name is not unique within the file, you must specify the path to the instance you want. For example, you might have multiple workflows. If you do not specify the path, you get the first instance. To obtain the path, move the mouse pointer over the field in the Datacap Application Manager and read the path from the tooltip.

Examples

You can use @APPPATH to retrieve the following information from the application configuration file.

Field name	Key name	Example	Notes
Application fields			
Batches folder	runtime	@APPPATH(runtime)	(Field name is unique)

Field name	Key name	Example	Notes
Export folder	export	@APPPATH(export)	(Field name is unique)
Fingerprint folder	fingerprint	@APPPATH(fingerprint)	(Field name is unique)
Workflow fields			
Setup DCO	setupdco	@APPPATH(setupdco)	(assumes single workflow)
Rules folder	rules	@APPPATH(dco_Workflow1/rules)	(specifies Workflow 1)
VScan source folder	vscanimagedir	@APPPATH(vscanimagedir)	(assumes single workflow)
Imagefix INI	imagefix	@APPPATH(dco_Workflow2/imagefix)	(specifies Workflow 2)

@APPVAR(<key_path>)

Description

Retrieves a connection string, value, or other attribute from the application configuration (.app) file. Use the Datacap Application Manager to modify the information that is contained in this file. Do not edit the file directly.

Attention: Before Datacap 8.0, this parameter returns the value for the specified variable as defined in the [Variables] section in paths.ini. The .ini file is in the process folder of the application.

The Datacap Application Manager encrypts the connection strings and decryption is handled by the standard actions.

Syntax

@APPVAR(key_path[:attribute])

Arguments

key_path - Specifies the path to field you want. See @APPPATH for details.

attribute - Specifies the attribute name (optional):

- For custom values, use "v" (if you do not specify an attribute, "v" is assumed).
- For connection strings, use "cs".

The easiest way to get the path and attribute is to look in the Datacap Application Manager:

- On the **Application** and **Datacap** tabs, move the mouse pointer over the field and read the smart parameter value from the tooltip.
- On the **Custom Values** tab, read the smart parameter value that is displayed in each section.

Examples

You can use @APPVAR to retrieve the following information from the application configuration file.

Field name	Key name:attribute	Example	Notes®
Workflow fields			

Field name	Key name:attribute	Example	Notes®
Lookup database	lookupdb:cs	@APPVAR(lookupdb:cs)	(Workflow 1)
Fingerprint database	fingerprintconn:cs	@APPVAR(fingerprintconn:cs)	(one workflow)
Export database	exportdb:cs	@APPVAR(dco_Wkflw1/exportdb:cs)	(Workflow 1)
Datacap fields			
Engine database	tmengine:cs	@APPVAR(tmengine:cs)	(Field is unique)
Admin database	tadmin:cs	@APPVAR(tadmin:cs)	(Field is unique)
Custom values			
General string values	values/gen/<value_name>	@APPVAR(values/gen/Value1)	(Value1)
Connection strings	values/dsn/<value_name>:cs	@APPVAR(values/dsn/DB1:cs)	(CS for DB1)
Datacap connection strings	values/tmdsn/<value_name>:cs	@APPVAR(values/tmdsn/TMDB1:cs)	(CS for TMDB1)
Advanced values	values/adv/<value_name>	@APPVAR(values/adv/Value1)	(Value1)

Related tasks:

“Storing passwords, connection strings, and other parameters in the .app file” on page 169

Special variables for accessing the runtime hierarchy

These variables return information that is related to documents, batches, pages, and fields.

“@BATCHID”

“@ID” on page 280

“@STATUS” on page 280

“@VALUE” on page 280

“@VAR(<variable_name>)” on page 281

“@P\<field_name>[.<variable_name>]” on page 281

“@F\<field_name>[.<variable_name>]” on page 282

“@B\<field_name>[.<variable_name>]” on page 282

“@D\<field_name>[.<variable_name>]” on page 282

“@P.<variable_name>” on page 283

“@F.<variable_name>” on page 283

@BATCHID

Description

Returns the value of the id attribute for the current batch.

Example

In this example, the smart parameter returns the ID of the current batch.

Action and return value	XML example (if applicable)
Action: rr_Get("@BATCHID")	<B id="20110046.001">
Return value: 20110046.001	<V n="TYPE">TravelDocs</V>
	<V n="STATUS">1</V>

@ID

Description

Returns the value of the id attribute for the current object. For example, if the rule that contains this special variable is bound to a page, the current object is the current page.

Example

In this example, the rule that contains the action is bound to a page and the smart parameter returns the ID of the current page.

Action and return value	XML example (if applicable)
Action: rr_Get("@ID")	<P id="TM000001">
Return value: TM000001	<V n="TYPE">Rental_Agreement</V>
	<V n="STATUS">1</V>

@STATUS

Description

Returns the value of the STATUS variable for the current object. STATUS is also a setup property and might specify a special characteristic (for example, **-1** indicates a hidden field).

Example

In this example, the rule that contains the action is bound to a page and the smart parameter returns the status of the current page.

Action and return value	XML example (if applicable)
Action: rr_Get("@STATUS")	<P id="TM000001">
Return value: 1	<V n="TYPE">Rental_Agreement</V>
	<V n="STATUS">1</V>

@VALUE

Description

Returns the text of the current object (usually a field).

Example

In this example, the rule that contains the action is bound to a field and the smart parameter returns the text of the current field. If the current object is not a field,

@VALUE looks for a value in a variable *Text* of the calling object.

Action and return value	XML example (if applicable)
Action: rr_Get("@VALUE")	<F id="Car_Type">
Return value: SUV	<V n="TYPE">Car_Type</V>
	<C cn="10" cr="588,748,600,769">83</C> <-- ASCII 'S'
	<C cn="10" cr="605,748,620,769">85</C> <-- ASCII 'U'
	<C cn="10" cr="625,748,643,769">86</C> <-- ASCII 'V'
	</F>

@VAR(<variable_name>)

Description

Returns the value of the specified variable on the current object.

Example

In this example, the rule that contains the action is bound to a page. The smart parameter returns the value of the *TYPE* variable for the current page.

Action and return value	XML example (if applicable)
Action: rr_Get("@VAR(TYPE)")	<<P id="TM000001">
Return value: Rental_Agreement	<V n="TYPE">Rental_Agreement</V>
	<V n="STATUS">1</V>

@P\<field_name>[.<variable_name>]

Description

Finds the parent page of the calling DCO. Then, find the child with the *dco_id* or *dco_type* of the parent page. Then, find the variable of this name on the specified child of the parent page.

The character values used in the following examples are ASCII codes, but they might be Unicode as well.

Example

In the first example, the smart parameter returns the text of the **Car_Type** field on the current page.

In the second example, the smart parameter returns the value of the *TYPE* variable of the **Car_Type** field.

Action and return value	XML example (if applicable)
Action: rr_Get("@P\Car_Type")	<F id="Car_Type">
Return value: SUV	<V n="TYPE">Car_Type</V>
	<C cn="10" cr="588,748,600,769">83</C> <-- ASCII 'S'
	<C cn="10" cr="605,748,620,769">85</C> <-- ASCII 'U'
	<C cn="10" cr="625,748,643,769">86</C> <-- ASCII 'V'
	</F>
Action: rr_Get("@P\Car_Type.TYPE")	
Return value: Car_Type	

@F\<field_name>[.<variable_name>]

Description

Returns the text of the specified subfield of the current field, for example, a field within a line item. Or returns the value of the specified variable of the specified subfield.

Example

In these examples, the rule that contains the action is bound to a field with subfields. In the first example, the smart parameter returns the text of the Unit_Cost subfield of the current field.

In the second example, the smart parameter returns the value of the Unit_Cost subfield *TYPE* variable.

Action and return value	XML example (if applicable)
Action: rr_Get("@F\Unit_Cost") Return value: \$9.90	<pre><F id="Other_Charges_Line_Item0"> <V n="TYPE">Other_Charges_Line_Item</V> <F id="Unit_Cost"> <V n="TYPE">Unit_Cost</V> <C cn="10" cr="1290,511,1305,540">36</C> <-- ASCII '\$' <C cn="10" cr="1308,515,1321,536">57</C> <-- ASCII '9' <C cn="10" cr="1325,533,1329,536">46</C> <-- ASCII '.' <C cn="10" cr="1334,515,1348,536">57</C> <-- ASCII '9' <C cn="10" cr="1350,515,1365,536">48</C> <-- ASCII '0'</pre>
Action: rr_Get("@F\Unit_Cost.TYPE") Return value: Unit_Cost	<pre></F></pre>

@B\<field_name>[.<variable_name>]

Description

Returns the text of the specified field on the current batch, or the value of the specified variable of the specified field on the current batch.

Example

In this example, the smart parameter returns the value of the *TYPE* variable for the current batch.

Action and return value	XML example (if applicable)
Action: rr_Get("@B.TYPE") Return value: TravelDocs	<pre><B id="20110046.001"> <V n="TYPE">TravelDocs</V> <V n="STATUS">1</V></pre>

@D\<field_name>[.<variable_name>]

Description

Returns the text of the specified field on the current document, or the value of the specified variable of the specified field on the current document.

Example

In this example, the smart parameter returns the value of the *TYPE* variable for the current document.

Action and return value	XML example (if applicable)
Action: rr_Get("@D.TYPE")	<D id="20110046.001.01"> <V n="TYPE">Car_Rental</V>
Return value: Car_Rental	<V n="STATUS">0</V>

@P.<variable_name>

Description

Returns the value of the specified variable on the current page.

Example

In this example, the smart parameter returns the value of the *TemplateID* variable for the current page.

Action and return value	XML example (if applicable)
Action: rr_Get("@P.TemplateID")	<P id="TM000001"> <V n="TYPE">Rental_Agreement</V>
Return value: 556	<V n="TemplateID">555</V>

@F.<variable_name>

Description

Returns the value of the specified variable within the current field.

Example

In this example, the smart parameter returns the value of the *TYPE* variable for the current field.

Action and return value	XML example (if applicable)
Action: rr_Get("@F.TYPE")	<F id="Pickup_Date"> <V n="TYPE">Pickup_Date</V>
Return value: Pickup_Date	<V n="Position">183,402,535,463</V>

Special variables for accessing job and task information

These variables return metadata that is related to a particular job or task, such as the job name or operator.

"@JOBID" on page 284

"@JOBNAME" on page 284

"@OPERATOR" on page 284

"@STATION" on page 284

"@TASKID" on page 285

"@TASKNAME" on page 285

@JOBID

Description

Returns the ID of the current job. This ID is the value that is specified in the **ID** field on the Datacap Administrator **Workflow** tab.

Example

In this example, the smart parameter returns the ID of the current job.

Action and return value	XML example (if applicable)
Action: rr_Get("@JOBID")	
Return value: Main Job	

@JOBNAME

Description

Returns the name of the current job. This name is the value that is specified in the **Description** field on the Datacap Administrator **Workflow** tab.

Example

In this example, the smart parameter returns the name of the current job.

Action and return value	XML example (if applicable)
Action: rr_Get("@JOBNAME")	
Return value: Main Job	

@OPERATOR

Description

Returns the user name of the person who ran the job.

Example

In this example, the smart parameter returns the name of the person who ran the job.

Action and return value	XML example (if applicable)
Action: rr_Get("@OPERATOR")	
Return value: admin	

@STATION

Description

Returns the ID of the station that runs the job. This ID is the value that is specified in the **ID** field on the Datacap Administrator **Workflow** tab.

Example

In this example, the smart parameter returns the ID of the station that runs the current job.

Action and return value	XML example (if applicable)
Action: rr_Get("@STATION")	
Return value: 1	

@TASKID

Description

Returns the ID of the current task. This ID is the value that is specified in the **ID** field on the Datacap Administrator **Workflow** tab.

Example

In this example, the smart parameter returns the ID of the current task.

Action and return value	XML example (if applicable)
Action: rr_Get("@TASKID")	
Return value: Export	

@TASKNAME

Description

Returns the name of the current task. This name is the value that is specified in the **Description** field on the Datacap Administrator **Workflow** tab.

Example

In this example, the smart parameter returns the name of the current task.

Action and return value	XML example (if applicable)
Action: rr_Get("@TASKNAME")	
Return value: Export	

Miscellaneous special variables

Miscellaneous special variables can be used with Smart parameters.

"@CHR(<unicode_value>)" on page 286

"@DATE(<format>)" on page 286

"@DCO(<property_name>)" on page 286

"@DICT_VALUE(<field>)" on page 287

"@DICT_WORD(<field>)" on page 287

"@DICT_VINDEX(<csv_string>)" on page 287

"@DICT_WINDEX(*csv_string*)" on page 288

"@EMPTY" on page 288

"@PATH(<key>)" on page 288
 "@PILOT(<property_name>)" on page 288
 "@PROJECTDIR" on page 289
 "@PROCESSDIR" on page 290
 "@STRING(<string_value>)" on page 290
 "@TIME(<format>)" on page 290
 "@TYPE" on page 290

@CHR(<unicode_value>)

Description

Returns the character corresponding to the specified UNICODE.

Example

In this example, the smart parameter returns the character corresponding to UNICODE value 38.

Action and return value	XML example (if applicable)
Action: rr_Get("@CHR(38)")	
Return value: &	

@DATE(<format>)

Description

Returns the current date in the format specified (defaults to MM/DD/YYYY).

Example

In this example, the smart parameter returns the current date.

Action and return value	XML example (if applicable)
Action: rr_Get("@DATE(mm.dd.yyyy)")	
Return value: 12.31.2010	

@DCO(<property_name>)

Description

Returns the value of the specified DCO object property. The DCO object is an internal data structure that contains the current runtime batch hierarchy information. This information includes the batch ID (ID), batch type (TYPE), batch status (STATUS), and the full runtime batch hierarchy XML.

Example

In this example, the rule that contains the action is bound to a page. The example also contains the smart parameter returns the portion of the runtime batch hierarchy XML for the current page.

Action and return value	XML example (if applicable)
Action: rr_Get("@DCO(XML)")	
Return value: <?xml-stylesheet type="text/xsl" href="..\..\dco.xsl"?><P id="TM000001"><F id="Pickup_Date"><V n="TYPE">Pickup_Date</V><V n="Position">0,0,0,0</V><V n="STATUS">0</V></F>	

@DICT_VALUE(<field>)

Description

This special variable works with OMR fields (RecogType=4) that are bound to a dictionary. It returns the dictionary values corresponding to the items that are selected in the specified **OMR** field.

Example

In this example, the rule that contains the rr_Get action is bound to a page that contains an **OMR** field (Options). The **OMR** field has three subfields and is bound to the dictionary shown in the XML code. On the current page all three options are selected, so the return string contains all three dictionary values.

Action and return value	XML example (if applicable)
Action: rr_Get("@DICT_VALUE(Options)")	<DICT n="Options">
Return value: Navigation System Child Seat Fuel Service	<W v="Navigation System">Navigation System</W> <W v="Child Seat">Child Seat</W> <W v="Fuel Service">Fuel Service</W> </DICT>

@DICT_WORD(<field>)

Description

Same as @DICT_VALUE, except this special variable returns the dictionary words corresponding to the selected items.

@DICT_VINDEX(<csv_string>)

Description

This special variable works with actions bound to an **OMR** field (RecogType=4) where the **OMR** field is bound to a dictionary. The parameter returns a string of 1's and 0's corresponding to the dictionary values you specify as a comma-separated list.

Example

In this example, the rule that contains the rr_Get action is bound to an **OMR** field. The **OMR** field has three subfields and is bound to the dictionary shown on the right. The values that are specified correspond to the second and third items in the dictionary, so the return value is 011. The rr_Get sets the character values on the three OMR subfields to 0, 1, and 1.

Action and return value	XML example (if applicable)
Action: rr_Get("@DICT_VINDEX(Fuel Service,Child Seat)") Return value: 011	<DICT n="Options"> <W v="Nav System">Nav System</W> <W v="Child Seat">Child Seat</W> <W v="Fuel Service">Fuel Service</W> </DICT>

@DICT_VINDEX(csv_string)

Description

Same as @DICT_VINDEX, except the argument for this special variable uses the dictionary words.

@EMPTY

Description

This special variable represents an empty string.

Example

In this example, rrSet clears the custom page variable *MyVar*.

Action and return value	XML example (if applicable)
Action: rrSet("@EMPTY","@P.MyVar")	

@PATH(<key>)

Description

Returns the full path for the specified identifier as defined in the [PATHS] section of paths.ini, in the dco_<app_name> folder of the application. The function that is supported by this special variable was replaced by the Datacap Application Manager and the @APPPATH special variable.

Example

In this example, the smart parameter returns the path to the image folder of the APT application. This folder is defined in the paths.ini file that is shown on the right.

Action and return value	XML example (if applicable)
Action: rr_Get("@PATH(VscanImageDir)") Return value: C:\Datacap\APT\Images\Input	[Paths] VscanImageDir=..\Images\Input ProcessDir=..\dco_APT RRXDir=..\..\dco_APT\Rules FingerprintDir=..\Fingerprint ExportDir=..\Export

@PILOT(<property_name>)

Description

Returns the value of the specified Pilot object property. The Pilot object is an internal data structure that is configured by Datacap at the start of task execution.

It contains the information that is required to run the task. Such as the batch folder (BATCHDIR), the batch ID (BATCHID), the input DCO file (DCOFILE), the task priority (PRIORITY)

The PILOT object properties that you can use to get values from Datacap and provide them to actions are.

- BATCHID
- BATCHDIR
- OPERATOR
- STATION
- CHILDRENQUANTITY
- PRIORITY
- PROJECTPATH
- CAPTION
- PAGESINBATCH
- DOCSINBATCH
- EXPECTEDPAGES
- ADJUSTEDPAGES
- ADJUSTEDDOCS
- JOBNAME
- FORMPROFILE
- DCOFILE
- JOBID
- TASKID

Example

In this example, the smart parameter returns the input DCO file for the current task.

Action and return value	XML example (if applicable)
Action: rr_Get("PILOT(DCOFILE)")	
Return value: C:\Datacap\TravelDocs\ batches\20110004.004\Export.xml	

@PROJECTDIR

Description

Returns the path and file name for the setup (.xml) file of the current task. The path is relative to the *dco_folder of the application*.

Example

Action and return value	XML example (if applicable)
Action: rr_Get("@PROJECTDIR")	
Return value: \RRS_VScan.bpp	

@PROCESSDIR

Description

Returns the full path to the dco_<app_name> folder of the application.

Example

Action and return value	XML example (if applicable)
Action: rr_Get("@PROCESSDIR")	
Return value: C:\Datacap\TravelDocs\ dco_TravelDocs	

@STRING(<string_value>)

Description

Returns the value that is specified as a string.

Example

Action and return value	XML example (if applicable)
Action: rr_Get("@STRING(MyString)")	
Return value: MyString	

@TIME(<format>)

Description

Returns the current time in the format specified. If you do not specify a format like @TIME(), defaults to HH:MM:SS.

Example

Action and return value	XML example (if applicable)
Action: rr_Get("@TIME(HH:MM)")	
Return value: 10:45	

@TYPE

Description

Returns the type of the current object (Batch, Document, Page, or Field)

Example

Action and return value	XML example (if applicable)
Action: rr_Get("@TYPE")	
Return value: Page	

Standard Variable Reference

Standard variables can be used on batches, documents, pages, and fields.

“Variables that are used on all object types”

“Batch variables” on page 294

“Document variables” on page 295

“Page variables” on page 295

“Field variables” on page 298

Variables that are used on all object types

Some variables are available for all object types.

“MAX_TYPES”

“MESSAGE”

“MIN_TYPES” on page 292

“rules” on page 292

“STATUS” on page 292

“TYPE” on page 294

“hr_locale” on page 294

MAX_TYPES

Applies to

Applies	Does not apply
Setup DCO	Runtime DCO

Description

Specifies the maximum number of child object types that can be present at run time to meet document integrity requirements (0 = no maximum).

MESSAGE

Applies to

Applies	Does not apply
Runtime DCO	Setup DCO

Description

Used by many validation and look up actions to report errors.

Example

This example shows an error message that is written to the runtime hierarchy by a failed validation action.

```
<V n="MESSAGE">Failed Calculation:377.73=477.73</V>
```

MIN_TYPES

Applies to

Applies	Does not apply
Setup DCO	Runtime DCO

Description

Specifies the minimum number of child object types that must be present at runtime to meet document integrity requirements.

rules

Applies to

Applies to: Setup DCO **Does not apply:** Runtime DCO

Description

Stores the rule map of an object that is established in Datacap Studio.

For example, you can map one or more rules to an object in the Document hierarchy. When you select the rule in the Document hierarchy object to which you mapped the rule, Datacap Studio displays the **DCO.Rule Map** in the Properties tab. The values in the **DCO.Rule Map** include the **Ruleset ID** and the **Rule name**. The **Ruleset ID** references the position of the ruleset on the Rulesets tab in Datacap Studio, and the **Rule name** references the position of the rule in the ruleset. For example, if an object is mapped with a rule named **Vendor**, which is part of the **Validate** ruleset, then the **Rule name** value can be 4 and the **Ruleset ID** can be 6. These values mean that the **Validate** ruleset is in the sixth position in the Ruleset tab, and the **Vendor** rule is in the fourth position within the **Validate** ruleset. The corresponding line in the Setup DCO is `<r id="4" rs="6" />`. If the object is mapped with five different rules, then the Setup DCO can be similar to this sample:

```
<in>
<r id="4" rs="6" />
<r id="2" rs="13" />
<r id="9" rs="7" />
<r id="3" rs="14" />
<r id="6" rs="16" />
</in>
```

Example

This example shows how the XML in the sample is stored in the Setup DCO file.

```
<V n="rules"><in><r id="4"
rs="6" /><r id="2" rs="13" />
<r id="9" rs="7" /><r
id="3" rs="14" /><r
id="6" rs="16" /></in></V>
```

STATUS

Applies to

Setup DCO, Runtime DCO

Description

Specifies the status of the object. The status is initially zero or not present in the Setup DCO. The status is updated at run time depending on the object type and the rules and actions of the task. Some common status values are shown in this table with their conventional meanings. Specific applications can use other status values.

Value	Status	Applies to	Assigned by
49	ScanOK	Page objects	Scan tasks
0	OK	Batch, document, and page objects	Rulerunner tasks
1	Problem		
2	Overridden	Pages that fail validation but are overridden by the operator.	Verify tasks
48	RecogDoneOK	Page objects	Recognition tasks
0	OK	Field objects	Any task
1	Error		Can specify -1 in Setup DCO
-1	Ignore		

A few less common status values are shown in this table.

Value	Status
51	CannotFindAnchor
52	DontNeedVerification
70	RescanPage
71	VerificationFailed
72	PageOnHold
73	PageOverridden
74	NoData
75	DeletedPage
77	DeleteApproved
79	ReviewPage
128	DeletedDoc
145	ReviewDoc

Example

```
<V n="STATUS">1</V>
```

TYPE

Applies to

Applies	Does not apply
Setup DCO	
Runtime DCO	

Description

In the Setup DCO, TYPE specifies the object type (batch, document, page, or field), for example:

```
<V n="TYPE">Page</V>
```

TYPE is updated in the Runtime DCO to specify the object name, for example:

```
<V n="TYPE">Rental_Agreement</V>
```

hr_locale

The hr_locale variable specifies the locale value that is used by the recognition engines to validate currency, numerals, and date data types for localization.

Applies to

Global DCO

Description

The hr_locale variable uses the BCP 47 defined language values for language-region, language code ID, and locale description. For example, the hr_locale variable for the English language in the United States, reads en-US, 1033, English (United States).

Batch variables

These standard variables can be used only on batch object types.

"LAST_RR_PROFILE"

LAST_RR_PROFILE

Applies to

Does not apply: Setup DCO **Applies to:** Runtime DCO

Description

Specifies the name of the last task profile that ran.

Example

```
<V n="LAST_RR_TPROFILE">Ru1erunner:m:eRun</V>
```

Document variables

These standard variables can be used only on document object types.

“DD”

DD

Applies to

Runtime DCO

Description

Used by some scan tasks - DD contains the value that is imprinted on the first page of the document by a scanner, or an externally assigned document ID.

Page variables

These standard variables can be used only on page object types.

“Confidence”

“DATAFILE”

“Fingerprint Created” on page 296

“Image_Offset” on page 296

“IMAGEFILE” on page 296

“PatternConfidence” on page 296

“PD” on page 297

“ScanSrcPath” on page 297

“TEMPLATE IMAGE” on page 297

“TemplateID” on page 297

Confidence

Applies to

” Setup DCO p Runtime DCO

Description

Specifies the confidence level achieved during fingerprint matching.

DATAFILE

Applies to

Runtime DCO

Description

Specifies the name of the data XML file that is associated with the page. DATAFILE is initially blank in the Setup DCO and is assigned at run time (for example, TM000001.xml).

Fingerprint Created

Applies to

Runtime DCO

Description

Specifies whether or not Datacap added a fingerprint for the page to the fingerprint library, which happens when fingerprint matching fails and the argument to the FindFingerprint action is True.

Example

```
<V n="Fingerprint Created">No</V>
```

Image_Offset

Applies to

Runtime DCO

Description

Specifies the offset in pixels (x, y) between the runtime page image and the fingerprint image.

Example

```
<V n="Image_Offset">-100,-100</V>
```

IMAGEFILE

Applies to

- Setup DCO
- Runtime DCO

Description

Specifies the name of the associated runtime image file. IMAGEFILE is blank in the Setup DCO and is assigned at run time.

Example

```
<V n="IMAGEFILE">tm000010.tif</V>
```

PatternConfidence

Applies to

Applies	Does not apply
Runtime DCO	Setup DCO

Description

Specifies the confidence level achieved when pattern matching is used for page identification.

Example

```
<V n="PatternConfidence">10</V>
```

PD

Applies to

Applies	Does not apply
Runtime DCO	Setup DCO

Description

Used by isscan tasks only - specifies the page data string.

ScanSrcPath

Applies to

Applies	Does Not Apply
Runtime DCO	Setup DCO

Description

Used by scan tasks only - specifies the full path and file name to the original image file.

Example

```
<V n="ScanSrcPath">c:\datacap\apt\images\input\invoice_0001.tif</V>
```

TEMPLATE IMAGE

Applies to

Setup DCO and Runtime DCO

Description

Specifies the name of the matching fingerprint (CCO) file. The value is blank in the Setup DCO and is assigned at runtime.

TemplateID

Applies to

Applies	Does not apply
Runtime DCO	Setup DCO

Description

Specifies the ID of the matching fingerprint.

Example

```
<V n="TemplateID">567</V>
```

Field variables

These standard variables can be used only on field object types.

- "DataType"
- "DensityString" on page 299
- "DICT" on page 299
- "Index" on page 299
- "Label" on page 299
- "Lookup" on page 300
- "LookupEx" on page 301
- "MaxLength" on page 301
- "METRIC" on page 302
- "MultiLine" on page 302
- "MultiPunch" on page 302
- "PatternMatch" on page 303
- "PictureString" on page 303
- "Pos<templateID>" on page 304
- "Position" on page 304
- "ReadOnly" on page 305
- "RecogStatus" on page 305
- "RecogType" on page 305
- "ReqConf" on page 305
- "SELECT" on page 306
- "ShowChar" on page 306
- "Sticky" on page 307
- "Text" on page 307
- "Zone_Offset" on page 307

DataType

Applies to

Setup DCO

Verification panels that support this variable

Datacap Web Client (VeriFine.aspx)

Description

Specifies the type of characters the user can enter into the field in the Verify panel. Use the following codes to specify the allowed data type:

Value	Description
0	Alphanumeric
1	Integer
2	Float
3	Date
4	Time

Value	Description
5	Currency

DensityString

Applies to

Runtime DCO

Description

This variable is used for OMR zones, one character per zone, where the character represents the percentage of black pixels within the zone according to the following formula.

Character's ASCII code value = Percentage black pixels + 48

For example, if the zone has 20% black pixels, the result is ASCII code 68 = 'D'.

Example

This example represents a field with three OMR check boxes.

```
<V n="DensityString">@@D</V>
```

DICT

Applies to

Setup DCO

Verification panels that support this variable

Datacap Desktop

Datacap Web Client (VeriFine.aspx and aindex.aspx)

Description

Used with selection fields and specifies the name of a dictionary (within this Setup DCO) containing a list of possible values.

Index

Applies to

Runtime DCO

Description

Used in FormSpec to optionally specify the field's index.

Label

Applies to

Setup DCO

Verification panels that support this variable

- Datacap Desktop
- Datacap Web Client (VeriFine.aspx)
- Datacap Web Client (aindex.aspx)

Description

The value of this variable, if specified, defines the label that is displayed beside the field in verification panels (Datacap Web Client and Datacap Desktop only). If not specified, the type attribute of the field is used as the label.

Lookup

Applies to

Applies	Does not apply
Setup DCO	Runtime DCO

Verification panels that support this variable

Support this variable	Do not support this variable
Datacap Desktop	N/A
Datacap Web Client (VeriFine.aspx)	
Datacap Web Client (aindex.aspx)	

Description

Specifies a database lookup statement that gets run during verification when the user clicks the hyperlinked field label. A list of matching entries from the database that is specified by the **dsn** attribute is displayed. The selected entry is used to populate the fields that are specified in the **flist** attribute.

Example

The following sample Lookup value gets a list of car types from the lookup database that is specified in the application configuration (.app) file. The sample code then populates the Car_Type field with the selected value.

```
<SQL flist='Car_Type' dsn="*/lookupdb:cs">SELECT Car_Type FROM Car_Types</SQL>
```

The next example (from APT) gets a list of matching vendor names, ZIP codes, and vendor IDs from the application's lookup database. The list is then displayed to the user. The SQL statement uses the text in the **Vendor** field as the search string. The user can then, for example, enter the first letter and see a list of vendor names that start with that letter. Upon selecting a vendor from the list, it populates the Vendor, Remittance_Zip, and Vendor_Number fields with the information for the selected vendor.

Attention: The carriage return in the following example is only for readability; it is not technically required.

```
<SQL flist='Vendor,Remittance_Zip,Vendor_Number' dsn="*/lookupdb:cs">  
SELECT VendorName,VendorZip,VendorID FROM VendorTable WHERE VendorName LIKE '@@Vendor@%'</SQL>
```

Note in this example the special syntax that is required to reference a field value from the SQL statement: @@Vendor@%. Note also that **WHERE** <column> = '<value>' is not supported.

LookupEx

Applies to

Applies	Does not apply
Setup DCO	Runtime DCO

Verification panels that support this variable

Support this variable:	Do not support this variable:
Datacap Desktop	Datacap Web Client (aindex.aspx)
Datacap Web Client (VeriFine.aspx)	

Description

Specifies a database lookup statement that gets run during verification when the user leaves the field (for example, by clicking or moving to the next field). LookupEx is typically used to populate other fields that are based on current field's value. The structure of the lookup statement is similar to that of the Lookup variable.

Example

The sample LookupEx value looks up the vendor name that is based on the ID in the VendorID field and populates the VendorName field with the result.

Attention: The carriage return in the following example is only for readability; it is not technically required.

```
<SQL flist='VendorName' dsn="*/lookupdb:cs">  
SELECT Vendor FROM VendorTable WHERE VendorID LIKE '@@VendorID@%'</SQL>
```

For more information about the LookupEx variable, see the “Lookup” on page 300 variable.

MaxLength

Applies to

Applies	Does not apply
Setup DCO	Runtime DCO

Verification panels that support this variable

Support this variable	Do not support this variable
Datacap Web Client (VeriFine.aspx)	Datacap Desktop
	Datacap Web Client (aindex.aspx)

Description

Specifies the maximum number of characters the user can type into the field in the Verify panel.

METRIC

Applies to

Applies	Does not apply
Setup DCO	Runtime DCO

Description

Specifies the size of the search region that is used during geometric pattern matching. The dimensions that are specified are relative to the anchor field. For example, METRIC=200,300 creates a search region 200 pixels larger left and right and 300 pixels larger above and below.

MultiLine

Applies to

Applies	Does not apply
Setup DCO	Runtime DCO

Verification panels that support this variable

Support this variable	Do not support this variable
Datacap Desktop	Datacap Web Client (VeriFine.aspx) Datacap Web Client (aindex.aspx)

Description

When set to '1,' Multiline shows the field as a multiline edit field in the Datacap Desktop Verify pane.

MultiPunch

Applies to

Applies	Does not apply
Setup DCO	Runtime DCO

Verification panels that support this variable

Support this variable	Do not support this variable
Datacap Desktop Datacap Web Client (VeriFine.aspx)	Datacap Web Client (aindex.aspx)

Description

Used when the field contains multiple OMR (check box) options. When MultiPunch is set to '1,' multiple selections are allowed; otherwise only one selection is allowed.

PatternMatch

Applies to

Applies to	Does not apply
Setup DCO	Runtime DCO

Description

Some PatternMatch actions require anchor fields, which are used to adjust the image registration. You can identify one or more fields as an anchor field in the **Zones** on Datacap Studio.

To identify a field as an anchor field, lock the DCO, right-click the anchor field, and select **Manage Variables**. Create the variable PatternMatch, if it does not exist, and set the value to 1.

When the PatternMatch variable is set to 1, the field is an anchor field for pattern matching.

PictureString

Applies to

Applies	Does not apply
Setup DCO	Runtime DCO

Verification panels that support this variable

Support this variable	Do not support this variable
Datacap Web Client (VeriFine.aspx)	Datacap Desktop
	Datacap Web Client (aindex.aspx)

Description

Specifies which characters the user can type into the field according to this key.

Value	Characters allowed
A	Alphabetic or space
a	Alphabetic, punctuation, or space
D	(Date) numeric digit, minus sign, decimal point (period), forward slash
F	(Float) numeric digit, minus sign, or decimal point (period)
f	Numeric digit or punctuation
L	Lowercase alphabetic or space

Value	Characters allowed
l	Lowercase alphabetic, punctuation, or space
N	Numeric digit
n	Uppercase alphabetic character, numeric digit, or space.
P	Punctuation or space
T	(time) numeric digit, A, P, M, or colon
U	Uppercase alphabetic or space
u	Uppercase alphabetic, punctuation, or space
X	Alphabetic, numeric digit, or space
x	Alphabetic, numeric digit, punctuation, or space
Z	Any character
#	Numeric digit or minus sign

Example

- `PictureString="A"` - Any upper/lower case letter or space is allowed (no numbers or special characters)
- `PictureString=""` - All characters are allowed (default)

Tip: If you specify more than one-character set code, the first code applies to the first character that is typed by the user. The second code applies to the second character that is typed by the user, a third code applies to the third character that is typed, and so on. The last code applies to all remaining characters that are typed.

Pos<templateID>

Applies to

Applies	Does not apply
Setup DCO	Runtime DCO

Description

Specifies the position of the recognition zone for a specific fingerprint image (<**templateID**>). This variable uses the upper left and lower right corners of the zone (x1, y1, x2, y2) to specify the position.

Position

Applies to

Applies
Setup DCO
Runtime DCO

Description

Specifies the field's position by using the upper left and lower right corners (x1, y1, x2, y2). The value of Position is initially (0,0,0,0) in the Setup DCO and is populated at run time.

ReadOnly

Applies to

Applies	Does not apply
Setup DCO	Runtime DCO

Verification panels that support this variable

Support this variable	Do not support this variable
Datacap Web Client (VeriFine.aspx)	Datacap Desktop
	Datacap Web Client (aindex.aspx)

Description

When set to "1," the field is read-only in the Verify panel and the user cannot change the preset or recognized value.

RecogStatus

Applies to

Does not apply: Setup DCO **Applies to:** Runtime DCO

Description

Numeric code set by some recognition actions to indicate status of the operation.

RecogType

Applies to

Applies to: Setup DCO **Does not apply:** Runtime DCO

Description

Specifies the code for the recognition engine to use when reading data from this field. OMR (check box) fields require RecogType=4 and these fields are the only ones that typically require this variable.

ReqConf

Applies to

Applies to: Setup DCO **Does not apply:** Runtime DCO

Description

Specifies the field level recognition confidence that is required for all characters that are populated to the field from a recognition action. Used with validation

actions to flag the field for validation is any character in the recognized result is lower than this value.

SELECT

Applies to

Applies	Does Not Apply
Setup DCO	Runtime DCO

Verification panels that support this variable

Support this variable:	Do not support this variable:
Datacap Desktop	Datacap Web Client (aindex.aspx)
Datacap Web Client (VeriFine.aspx)	

Description

Specifies a database lookup statement that converts an edit field in a verification panel into a drop-down list with values from a database. A list of matching entries from the database that is specified by the **dsn** attribute is displayed. The selected entry is used to populate the fields that are specified in the **flist** attribute.

Example

This sample Lookup value retrieves a list of car types from the lookup database that is specified in the application configuration (.app) file. This sample then populates the list in the **Car_Type** field.

```
<SQL flist='Car_Type' dsn="*/lookupdb:cs">SELECT Car_Type FROM Car_Types</SQL>
```

Note that you can populate multiple fields simultaneously (see the Lookup variable for an example).

ShowChar

Applies to

Applies	Does not apply
Setup DCO	Runtime DCO

Verification panels that support this variable

Support this variable	Do not support this variable
Datacap Web Client (VeriFine.aspx)	Datacap Desktop
	Datacap Web Client (aindex.aspx)

Description

This variable works with the Datacap Web Client (VeriFine.aspx) panel only. When set to 1, the character zones are displayed in the image snippet of the field in the Verify panel.

Sticky

Applies to

Applies	Does not apply
Setup DCO	Runtime DCO

Verification panels that support this variable

Support this variable:	Do not support this variable:
Datacap Web Client (VeriFine.aspx)	Datacap Desktop
	Datacap Web Client (aindex.aspx)

Description

If Sticky is defined for any field (the value is not important), the Verify panel includes a check box to the left of the field's label. If you select the check box, enter a value, and submit the page, the field value populates across all pages of that type in the current batch.

Text

Applies to

Applies	Does not apply
Setup DCO	
Runtime DCO	

Verification panels that support this variable

Support this variable:	Do not support this variable:
Datacap Web Client (aindex.aspx)	Datacap Desktop
	Datacap Web Client (VeriFine.aspx)

Description

If defined for a field (the value must be empty), the field in the Verify panel becomes sticky. If the user enters a value and submits a page, the field value populates across all pages of that type in the current batch. The field must be initially empty for automatic population to work.

Zone_Offset

Applies to

Applies	Does not apply
Setup DCO	Runtime DCO

Description

Used with anchor fields during geometric pattern matching. Specifies the offset in pixels (x, y) between the anchor's position on the runtime page image and its position in the fingerprint image.

Application-specific variable reference

Some variables have been designed for use only with specific Datacap applications.
“Medical Claims 5010 form configuration parameters”

Medical Claims 5010 form configuration parameters

The configuration variables for the Medical Claims 5010 Institutional and Professional forms can be found in the configuration .ini files (5010_Inst.ini and 5010_Prof.ini).

“5010 Institutional form configuration variables”

“5010 Professional form configuration variables” on page 320

5010 Institutional form configuration variables

The following variables are configured in the 5010_Inst.ini configuration file.

Table 51. General

Variable	Default	Description
ImpliedDecimalON	TRUE	When set to TRUE, this boolean variable normalizes all currency data values to have a two-digit value for cents such as 0.00. The variable then formats output for 837 requirements of no leading or trailing zeros.
PreFilter	TRUE	This boolean variable filters character output to EDI elements when set to TRUE. The only characters allowed are Comma, Hyphen, Period, 0-9, and A-Z. Lower case a-z characters are capitalized.

Table 52. Log

Variable	Default	Description
SegmentLog	TRUE	When set to TRUE, this boolean variable adds a single log entry to the RRS log when a segment is counted for adding to the EDI export stream.
SegmentReport	FALSE	When set to TRUE, this variable saves all segment logs added to the export stream and outputs the stream to the RRS log file as a summary after the EDI is created. This variable does not affect output.

Table 53. X12N

Variable	Default	Description
Segmentseparator	*	Segment Separator character which indicates the start/end of 837 segment elements.
SegmentTerminator	~	Segment Terminator character which indicates the end of a single 837 segment.
RepetitionSeparator	^	Delimiter character used to separate repeated occurrences of a simple data element or composite data structure.
TerminatorPlusLineFeed	FALSE	When set to TRUE, this variable adds a line feed and carriage return to the end of each Segment Terminator character. Typically this option is disabled in a production environment, and is only used for debugging the output data. Attention: The program will by default also create a file named EDI_OutputText.txt which performs the same debugging function. This file is generated using the output file to ensure data integrity, and cannot be disabled.
AllUpperCase	FALSE	When set to TRUE, all output characters are converted to uppercase. Restriction: Use of the Prefilter option forces all characters to uppercase regardless of this option.
AppendToLastGSGE	FALSE	When merging individual claim EDI files, this option chooses whether the new claim is added as the first claim in a new GS/GE loop in the merged file. When this option is set to FALSE, each claim will be created in a new GS/GE and ST/SE loop in the merged file. When this option is set to TRUE, then each claim will be added to the existing GS/GE loop in the merged file. See the AppendToLastSTSE option for ST/SE information when added with this option set to TRUE.

Table 53. X12N (continued)

Variable	Default	Description
AppendToLastSTSE	FALSE	When merging individual claim EDI files, this option chooses whether the new claim is added as the first claim in a new ST/SE loop in the merged file. When this option is set to FALSE, each claim will be created in a new ST/SE loop within the existing GS/GE loop in the merged file. When this option is set to TRUE, then each claim will be added to the existing GS/GE loop in a new ST/SE loop in the merged file.
SuppressSVdateSpan	FALSE	When this variable is set to TRUE, there is no creation of a DTP RD8 date span in the service line even if Type of Bill is Inpatient type.

Table 54. ISA_Header

Variable	Default	Description
AuthorizationInformation		Sets value for ISA02. Limited to ten characters, empty by default. A value placed here will automatically set ISA01 from 00 (default-No authorization info present) to 03 (Additional Data Identification).
SecurityInformation		Sets value for ISA04. Limited to ten characters, empty by default. A value placed here will automatically set ISA03 from 00 (default-No Security Info present) to 01 (Password).
InterchangeSenderQualifier	ZZ	Sets value for ISA05. Code indicating the type of Sender ID. Value codes are: 01, 14, 20, 27, 28, 29, 30, 33 or ZZ. Standard setting is ZZ.
InterchangeSenderID	DATA CAP	Sets value for ISA06. Sender ID value, limited to 15 characters.
InterchangeReceiverQualifier	ZZ	Sets value for ISA07. Code indicating the type of Receiver ID. Value codes are: 01, 14, 20, 27, 28, 29, 30, 33 or ZZ. Standard setting is ZZ.

Table 54. ISA_Header (continued)

Variable	Default	Description
InterchangeReceiverID	00000	Sets value for ISA08. Receiver ID value, limited to 15 characters.
InterchangeControlNum	BATCH	Sets the value for ISA13. Valid values are BATCH or a fixed value. Use of BATCH will use the current batch ID - right-justified and minus decimal characters. Any other entry is treated as a fixed output value for this element. This value is limited to fifteen numeric characters. Attention: By default, if the Batch ID is a split batch alphadecimal value, then the new value will be the 3 digit Julian Day, followed by the three digit daily batch number, and then concluded with the integer conversion of the alphadecimal split batch value. The maximum length of the split batch number is limited to 3 digits. Alternatively, the value for ISA(13) also supports smart parameters, so that entries can generate or create their own unique values and include it in the export.
AcknowledgmentRequested	0	Sets value for ISA14. Valid values are 0 (No Acknowledgement Default) or 1 (Interchange Acknowledgement Requested)
TestIndicator	P	Sets value for ISA15. Valid values are P (production) or T (test).
ComponentElementSeparator	<	Sets value for ISA16 the sub-element separator value. Segment Elements have multiple values are required to separate the values with a predefined character. Recommended are : or <.

Table 55. GS_Header

Variable	Default	Description
SenderCode	DATACAP EXPORT	GS02 value limited to 15 characters.

Table 55. GS_Header (continued)

Variable	Default	Description
ReceiverCode	00000	GS03 value limited to 15 characters, Defaults to 00000.
EDIstandard	005010X222A1	GS08 EDI version identifier code.

Table 56. Header

Variable	Default	Description
SubmitterName	The Defaults NM103 setting.	Sets the Loop 1000A NM1(03) value. Limited to 60 characters.
SubmitterPhone	The Defaults PER04 setting.	Sets the Loop 1000A PER(03) value.
SubmitterID	The GS_Header SenderCode setting value. If SenderCode is blank this value defaults to the Defaults NM109 setting.	Sets the Loop 1000A NM1(09) value. The standard setting is usually blank which fills this value with the GS_Header SenderCode value.
ReceiverName	The Defaults NM103 setting.	Sets the Loop 1000B NM1(03) value.
ReceiverID	00000	
SequenceSerialNumber	The value found in ST(02)	Sets the BHT(03) Reference ID value. Limited to 50 characters.
TransactionSetControlSource	The current Document ID value	Sets ST(02) value to use a Batch parameter. Entry is the parameter name to use as the source for the ST(02) value.
PayerIDNumber	123456789	Sets Loop 2000B SBR(03) value if field 11 Plan and Policy are empty. See also the CheckJobID settings in this setting section. Also sets Loop 2010BB NM1(09) if field 11 Policy number is empty. Note this value is overridden if the setting 2010BB_BC PrimaryPayerIDNumber has a value.

Table 56. Header (continued)

Variable	Default	Description
CheckJobID	FALSE	When set to TRUE, this variable enables the ValueByJobID(Key) utility to retrieve values based on the JobID of the task. This allows the processing of claims for multiple entities, such as different payers. For example, the following function looks in the Header section for the number of different JobID's for which to check:. <pre>[Header] CheckJobID=TRUE/FALSE numJobID=2 numJobID[1]=JobName1 numJobID[2]=JobName2</pre> Use values for paramter 'Key' (PayerIDNumber in example) as follows: <pre>[JobName1] PayerIDNumber=1111111 [JobName2] PayerIDNumber=2222222</pre> The example shows ini entries numJobID, numJobID[n] and JobName[n] PayerIDNumber that must be created by the user.

Table 57. FileNameFormat

Variable	Default	Description
FileHeader	00501_	The default file name is the Page or Doc ID with decimals removed and spaces replaced with the underscore character. This is an option to place the specified text leading the default file name. For example if the Doc ID 2012004.03 is used and the FileHeader contains 00501_ then the file name becomes 00501_201200403. This can be used in conjunction with the FileTrailer option.

Table 57. FileNameFormat (continued)

Variable	Default	Description
FileTrailer		The default file name is the Page or Doc ID with decimals removed and spaces replaced with the underscore character. This is an option to place the specified text trailing the default file name. For example if the Doc ID 2012004.03 is used and the FileTrailer contains _0500 then the file name becomes 201200403_0500. This can be used in conjunction with the FileHeader option.
MergeFileHeader	5010_	If no File name is specified when calling to merge the 5010 file, the default file name becomes the batch Id. This is an option to place the specified text leading the default file name.
MergeFileTrailer		If no File name is specified when calling to merge the 5010 file, the default file name becomes the batch Id. This is an option to place the specified text trailing the default file name. The default value is blank.

Table 58. Defaults

Variable	Default	Description
NM103		General default for NM1(03) segment element values. Loops 1000A, 1000B, 2010BA, 2010CA, 2310A, 2310B, 2310C, 2330A, 2330B, 2420A.
NM104		General default for NM1(04) segment element values. Loops 2010BA, 2010CA, 2310A, 2310B, 2310CDF, 2330A, 2420A.
NM109		General default for NM1(09) segment element values. Loops 1000A, 1000B, 2010BA, 2310A, 2310CDF, 2330A, 2330B.
N301		General default for N3(01) segment element values. All instances.
N401		General default for N4(01) segment element values. All instances.

Table 58. Defaults (continued)

Variable	Default	Description
N402		General default for N4(02) segment element values. All instances.
N403		General default for N4(03) segment element values. All instances.
PER04		General default for PER(04) segment element values. Loop 1000A, 2010AA
DMG02		General default for DMG(02) segment element values. Loop 2000CA, 2010BA.
CLM07		Value for setting Loop 2300 CLM(07). Default is blank.
CLM08		Value for setting Loop 2300 CLM(08) if field 53 is not N or Y. Default is blank.
CLM09		Value for setting Loop 2300 CLM(09) if field 52 is not I or Y. Default is blank.
OccDT		Default Date Value. Loop 2300 DTP 434 & 345, also Loop 2300 default date if procedure date is not valid, and occurrence span dates, Loop 2400 DTP 472
HI_BK		Default principle diagnosis code if no diagnosis fields on the claim are filled. Default is blank. Loop 2300 HI BK.
SV201		Default Loop 2400 SV2(1) RevCode if captured value is blank or all zeros. Default is blank.

Table 59. ImpliedDecimalON

Variable	Default	Description
BuildLog	FALSE	TRUE/FALSE value. Adds RRS logging when building EDI if the ImpliedDecimal setting is enabled. See the General settings.

Table 60. 2000B

Variable	Default	Description
SubscriberIsPatient	FALSE	TRUE/FALSE. Setting for forcing EDI to output Subscriber is the Patient. Default is false.

Table 60. 2000B (continued)

Variable	Default	Description
UseCurrentPayer	See description	Value is a Regular expression that is tested against the value in field 50. If a match is found this is the Payer for this claim. Otherwise default is the last payer a, b or c on the completed form. Example: If payer is always Medicaid. Use of regular expression MED[iI11T]iI11TD will find the correct field 50 line to assign. Default is blank and uses last payer on form.
UsePreviousPayer	See description	Value is a Regular expression that is tested against the value in field 50. If a match is found the line before the match becomes the Payer info for this claim. Otherwise default is the last payer a, b or c on the completed form. Example: If payer is always listed before Medicaid. Use of regular expression MED[iI11T]CA[iI11T]D will find the prior field 50 line to assign. Default is blank and uses last payer on form. Attention: Both UseCurrentPayer and UsePreviousPayer can be used simultaneously. Use Previous takes priority if both expressions match.

Table 61. 2010BB_BC

Variable	Default	Description
PrimaryPayerName		Setting to use for all processed claims for Loop 2010BB NM1(03). Takes priority over captured value as well as default value set in Header PayerName section.
PrimaryPayerIDNumber		Setting to use for all processed claims for Loop 2010BB NM1(09). Takes priority over default value set in Header PayerIDNumber section.
PrimaryPayerAddress		Value to place in Loop 2010BB N3(01).

Table 61. 2010BB_BC (continued)

Variable	Default	Description
PrimaryPayerCity		Value to place in Loop 2010BB N4(01).
PrimaryPayerState		Value to place in Loop 2010BB N4(02).
PrimaryPayerZip		Value to place in Loop 2010BB N4(03).

Table 62. 2300

Variable	Default	Description
CalculatedTotal	TRUE	TRUE/FALSE. Option to calculate the lineitem total using service line data instead of the field 28 total charges field for Loop 2300 CLM(02).
DischargeStatus	01	Default value for Loop 2300 CL1(03) if field 17 is blank.
PatientReason		Default value for Patient Reason for admission if field 69 is blank. Outpatient Claims only, default is blank.

Table 63. 2310A

Variable	Default	Description
CreateOutpatient	FALSE	TRUE/FALSE. Setting to optionally always export Loop 2310A even if Type of Bill is outpatient.
ExportSecondaryID	FALSE	TRUE/FALSE. Optionally export a REF 1G segment in this loop with the value from field 76.

Table 64. 2320

Variable	Default	Description
OI03	Y	Value for Loop 2320 OI(03).
OI06	Y	Value for Loop 2320 OI(06).

Table 65. 2330A

Variable	Default	Description
UsePatientAddress	FALSE	TRUE/FALSE. Option to use always use field 9 address values for Loop 2330A N4 segment if patient is not the subscriber.

Table 65. 2330A (continued)

Variable	Default	Description
SubscriberCity		Default value for Loop 2330A N4(01) if patient is not the subscriber and UsePatientAddress is not enabled.
SubscriberState		Default value for Loop 2330A N4(02) if patient is not the subscriber and UsePatientAddress is not enabled
SubscriberZip		Default value for Loop 2330A N4(03) if patient is not the subscriber and UsePatientAddress is not enabled

Table 66. 2330B

Variable	Default	Description
PayerIDNumber		Value to place in loop 2330B NM1(09) if field 51 is blank.
PayerCity		Value to place in Loop 2330B N4(01).
PayerState		Value to place in Loop 2330B N4(02).
PayerZip		Value to place in Loop 2330B N4(03).

Table 67. N3

Variable	Default	Description
BuildLog	FALSE	TRUE/FALSE value. Adds more detailed RRS logging when creating N3 segments.
KeepTrailingSeparators	FALSE	TRUE/FALSE value. Option to not remove blank trailing elements from N3 segments.

Table 68. N4

Variable	Default	Description
BuildLog	FALSE	TRUE/FALSE value. Adds more detailed RRS logging when creating N4 segments.
KeepTrailingSeparators	FALSE	TRUE/FALSE value. Option to not remove blank trailing elements from N4 segments.

Table 69. REF

Variable	Default	Description
BuildLog	FALSE	TRUE/FALSE value. Adds more detailed RRS logging when creating REF segments.

5010 Professional form configuration variables

The following variables are configured in the 5010_Prof.ini configuration file.

Table 70. General

Variable	Default	Description
ImpliedDecimalON	TRUE	When set to TRUE, this boolean variable normalizes all currency data values to have a two-digit value for cents such as 0.00. The variable then formats output for 837 requirements of no leading or trailing zeros.
PreFilter	TRUE	This boolean variable filters character output to EDI elements when set to TRUE. The only characters allowed are Comma, Hyphen, Period, 0-9, and A-Z. Lower case a-z characters are capitalized.

Table 71. Log

Variable	Default	Description
SegmentLog	TRUE	When set to TRUE, this boolean variable adds a single log entry to the RRS log when a segment is counted for adding to the EDI export stream.
SegmentReport	FALSE	When set to TRUE, this variable saves all segment logs added to the export stream and outputs the stream to the RRS log file as a summary after the EDI is created. This variable does not affect output.

Table 72. X12N

Variable	Default	Description
Segmentseparator	*	Segment Separator character which indicates the start/end of 837 segment elements.
SegmentTerminator	~	Segment Terminator character which indicates the end of a single 837 segment.

Table 72. X12N (continued)

Variable	Default	Description
RepetitionSeparator	^	Delimiter character used to separate repeated occurrences of a simple data element or composite data structure.
TerminatorPlusLineFeed	FALSE	When set to TRUE, this variable adds a line feed and carriage return to the end of each Segment Terminator character. Typically this option is disabled in a production environment, and is only used for debugging the output data. Attention: The program will by default also create a file named EDI_OutputText.txt which performs the same debugging function. This file is generated using the output file to ensure data integrity, and cannot be disabled.
AllUpperCase	FALSE	When set to TRUE, all output characters are converted to uppercase. Restriction: Use of the Prefilter option forces all characters to uppercase regardless of this option.
AppendToLastGSGE	FALSE	When merging individual claim EDI files, this option chooses whether the new claim is added as the first claim in a new GS/GE loop in the merged file. When this option is set to FALSE, each claim will be created in a new GS/GE and ST/SE loop in the merged file. When this option is set to TRUE, then each claim will be added to the existing GS/GE loop in the merged file. See the AppendToLastSTSE option for ST/SE information when added with this option set to TRUE.

Table 72. X12N (continued)

Variable	Default	Description
AppendToLastSTSE	FALSE	When merging individual claim EDI files, this option chooses whether the new claim is added as the first claim in a new ST/SE loop in the merged file. When this option is set to FALSE, each claim will be created in a new ST/SE loop within the existing GS/GE loop in the merged file. When this option is set to TRUE, then each claim will be added to the existing GS/GE loop in a new ST/SE loop in the merged file.

Table 73. ISA Header

Variable	Default	Description
AuthorizationInformation		Sets the value for ISA02. Limited to ten characters, and blank by default. A value placed here will automatically set ISA01 from 00 (default-No authorization info present) to 03 (Additional Data Identification).
SecurityInformation		Sets the value for ISA04. Limited to ten characters, and blank by default. A value placed here will automatically set ISA03 from 00(default-No Security Info present) to 01 (Password).
InterchangeSenderQualifier	ZZ	Sets the value for ISA05. This is a code indicating the type of Sender ID. Value codes are: 01, 14, 20, 27, 28, 29, 30, 33 or ZZ.
InterchangeSenderID		Sets value for ISA06. Sender ID value, limited to 15 characters.
InterchangeReceiverQualifier	ZZ	Sets the value for ISA07. This is a code indicating the type of Receiver ID. Value codes are: 01, 14, 20, 27, 28, 29, 30, 33 or ZZ.
InterchangeReceiverID		Sets value for ISA08. Receiver ID value, limited to 15 characters.

Table 73. ISA Header (continued)

Variable	Default	Description
InterchangeControlNum		Sets the value for ISA13. Valid values are BATCH or a fixed value. Use of BATCH will use the current batch ID - right-justified and minus decimal characters. Any other entry is treated as a fixed output value for this element. This value is limited to fifteen numeric characters.
AcknowledgmentRequested		Sets the value for ISA14. Valid values are 0 (No Acknowledgement Default) or 1 (Interchange Acknowledgement Requested).
TestIndicator	P	Sets the value for ISA15. Valid values are P (production) or T (test).
ComponentElementSeparator	<	Sets the value for ISA16 the sub-element separator value. Segment elements that have multiple values are required to separate the values with a predefined character. Recommended values include : or <.

Table 74. GS_Header

Variable	Default	Description
SenderCode	DATA CAP EXPORT	GS02 value limited to 15 characters.
ReceiverCode	00000	GS03 value limited to 15 characters, Defaults to 00000.
EDIstandard	005010X222A1	GS08 EDI version identifier code.

Table 75. Header

Variable	Default	Description
SubmitterName	The Defaults NM103 setting.	Sets the Loop 1000A NM1(03) value. Limited to 60 characters.
SubmitterPhone	The Defaults PER04 setting.	Sets the Loop 1000A PER(03) value.
SubmitterID	The GS_Header SenderCode setting value. If SenderCode is blank this value defaults to the Defaults NM109 setting.	Sets the Loop 1000A NM1(09) value. The standard setting is usually blank which fills this value with the GS_Header SenderCode value.
ReceiverName	The Defaults NM103 setting.	Sets the Loop 1000B NM1(03) value.

Table 75. Header (continued)

Variable	Default	Description
ReceiverID	00000	
SequenceSerialNumber	The value found in ST(02)	Sets the BHT(03) Reference ID value. Limited to 50 characters.
TransactionSetControlSource	The current Document ID value	Sets ST(02) value to use a Batch parameter. Entry is the parameter name to use as the source for the ST(02) value.
PayerIDNumber	123456789	Sets Loop 2000B SBR(03) value if field 11 Plan and Policy are empty. See also the CheckJobID settings in this setting section. Also sets Loop 2010BB NM1(09) if field 11 Policy number is empty. Note this value is overridden if the setting 2010BB_BC PrimaryPayerIDNumber has a value.
CheckJobID	FALSE	When set to TRUE, this variable enables the ValueByJobID(Key) utility to retrieve values based on the JobID of the task. This allows the processing of claims for multiple entities, such as different payers. For example, the following function looks in the Header section for the number of different JobID's for which to check: <pre>[Header] CheckJobID=TRUE/FALSE numJobID=2 numJobID[1]=JobName1 numJobID[2]=JobName2</pre> Use values for parameter 'Key' (PayerIDNumber in example) as follows: <pre>[JobName1] PayerIDNumber=1111111 [JobName2] PayerIDNumber=2222222</pre> The example shows ini entries numJobID, numJobID[n] and JobName[n] PayerIDNumber that must be created by the user.

Table 76. FileNameFormat

Variable	Default	Description
FileHeader	00501_	The default file name is the Page or Doc ID with decimals removed and spaces replaced with the underscore character. This is an option to place the specified text leading the default file name. For example if the Doc ID 2012004.03 is used and the FileHeader contains 00501_ then the file name becomes 00501_201200403. This can be used in conjunction with the FileTrailer option.
FileTrailer		The default file name is the Page or Doc ID with decimals removed and spaces replaced with the underscore character. This is an option to place the specified text trailing the default file name. For example if the Doc ID 2012004.03 is used and the FileTrailer contains _0500 then the file name becomes 201200403_0500. This can be used in conjunction with the FileHeader option.
MergeFileHeader	5010_	If no File name is specified when calling to merge the 5010 file, the default file name becomes the batch Id. This is an option to place the specified text leading the default file name.
MergeFileTrailer		If no File name is specified when calling to merge the 5010 file, the default file name becomes the batch Id. This is an option to place the specified text trailing the default file name. The default value is blank.

Table 77. Defaults

Variable	Default	Description
NM103		General default for NM1(03) segment element values. Loops 1000A, 1000B, 2010BA, 2010CA, 2310A, 2310B, 2310C, 2330A, 2330B, 2420A.

Table 77. Defaults (continued)

Variable	Default	Description
NM104		General default for NM1(04) segment element values. Loops 2010BA, 2010CA, 2310A, 2310B, 2310CDF, 2330A, 2420A.
NM109		General default for NM1(09) segment element values. Loops 1000A, 1000B, 2010BA, 2310A, 2310CDF, 2330A, 2330B.
N301		General default for N3(01) segment element values. All instances.
N401		General default for N4(01) segment element values. All instances.
N402		General default for N4(02) segment element values. All instances.
N403		General default for N4(03) segment element values. All instances.
PER04		General default for PER(04) segment element values. Loop 1000A, 2010AA
DMG02		General default for DMG(02) segment element values. Loop 2000CA, 2010BA.

Table 78. ImpliedDecimalON

Variable	Default	Description
BuildLog	FALSE	TRUE/FALSE value. Adds RRS logging when building EDI if the ImpliedDecimal setting is enabled. See the General settings.

Table 79. 2000B

Variable	Default	Description
NO_SBR03	FALSE	When set to TRUE SBR03 will always map as empty. When this variable is set to TRUE and the SBR04 variable is used, then if the if 11cIP1an field is blank, the value for SBR03 will be mapped to the Header section's PayerIDNumber variable.

Table 79. 2000B (continued)

Variable	Default	Description
SubscriberIsPatient	FALSE	TRUE/FALSE. Setting for forcing EDI to output Subscriber is the Patient. Default is false.
SBR03		Sets a default value for SBR03 to be used if the 11IPolNo field is empty.
SBR04		Sets a default value for SBR04 to be used if the 11cIPlan field is empty.
SBR09	ZZ	Setting for Loop 2000B SBR(09) claim filing indicator code. Standard values are 11 to 17, AM, BL, CH, CI, DS, FI, HM, LM, MA, MB, MC, OF, TV, VA, WC, and ZZ. This setting is FindValue enabled, so if another value is placed in this setting on the current page, then the document will be searched for a field or variable with this value. The search order is: 1. Page Field 2. Page Variable 3. Document Variable If no DCO nodes with this id are found, the value is used.

Table 80. 2010AA

Variable	Default	Description
BOXPattern	TRUE	Setting this attribute to TRUE enables detection using a regular expression for PO Box values in the loop 2010AA street address field. If a PO Box is detected, the field value is set to the DefaultStreetAddress setting value by default.
DefaultStreetAddress		Value to replace in the street address field if a PO Box is detected in the address field.

Table 80. 2010AA (continued)

Variable	Default	Description
RemapBOX2PayTo	FALSE	When this variable is set to TRUE, if a POBox is detected in the field 32 address line, and the field 31 address also has a value, then the field 32 address is remapped to loop 2010AB and the field 31 address is mapped to loop 2010AA.

Table 81. 2010BB_BC

Variable	Default	Description
PrimaryPayerName		Setting to use for all processed claims for Loop 2010BB NM1(03). Takes priority over captured value as well as default value set in Header PayerName section.
PrimaryPayerIDNumber		Setting to use for all processed claims for Loop 2010BB NM1(09). Takes priority over default value set in the Header section's PayerIDNumber variable.
PrimaryPayerAddress		Value to place in Loop 2010BB N3(01).
PrimaryPayerCity		Value to place in Loop 2010BB N4(01).
PrimaryPayerState		Value to place in Loop 2010BB N4(02).
PrimaryPayerZip		Value to place in Loop 2010BB N4(03).

Table 82. 2300

Variable	Default	Description
CalculatedTotal	TRUE	TRUE/FALSE. Option to calculate the lineitem total using service line data instead of the field 28 total charges field for Loop 2300 CLM(02).

Table 82. 2300 (continued)

Variable	Default	Description
REFD9		Value for Loop 2300 Ref D9 segment. By default, this value is blank, and no Ref D9 is exported. This setting is FindValue enabled, so if another value is placed in this setting on the current page, then the document will be searched for a field or variable with this value. The search order is: 1. Page Field 2. Page Variable 3. Document Variable If no DCO nodes with this id are found, the value is used.
REFEA	The <i>dvDCN</i> document variable	Value for Loop 2300 Ref EA segment. This setting is FindValue enabled, so if another value is placed in this setting on the current page, then the document will be searched for a field or variable with this value. The search order is: 1. Page Field 2. Page Variable 3. Document Variable If no DCO nodes with this id are found, the value is used.

Table 83. 2310A

Variable	Default	Description
AlwaysInclude	FALSE	Setting this value to TRUE will always export Loop 2310A even if fields 23 or 19 do not have a captured value.

Table 84. 2320

Variable	Default	Description
SBR09	ZZ	Value for the Loop 2320 SPR(09) claim filing indicator code. Standard values are 11 to 17, AM, BL, CH, CI, DS, FI, HM, LM, MA, MB, MC, OF, TV, VA, WC, and ZZ. This setting is FindValue enabled, so if another value is placed in this setting on the current page, then the document will be searched for a field or variable with this value. The search order is: 1. Page Field 2. Page Variable 3. Document Variable If no DCO nodes with this id are found, the value is used.

Table 85. Loop2320

Variable	Default	Description
OI03	Y	Value for Loop 2320 OI(03).
OI04		Value for Loop 2320 OI(04).
OI06	Y	Value for Loop 2320 OI(06).

Table 86. 2330B

Variable	Default	Description
OtherPayerAddress		Value to place in Loop 2330B N3(01)
OtherPayerCity		Value to place in Loop 2330B N4(01).
OtherPayerState		Value to place in Loop 2330B N4(02).
OtherPayerZip		Value to place in Loop 2330B N4(03).

Table 87. 2400

Variable	Default	Description
AutoPopulateUnit	TRUE	When set to TRUE, this variable will auto populate SV1(03) with MJ and SV1(04) with the service line INFO field if the INFO field has a captured value. Otherwise SV1(03) uses UN and SV1(04) is filled with the Days field value.

Table 87. 2400 (continued)

Variable	Default	Description
RoundCharges	TRUE	When set to TRUE, this variable toggles rounding of SV1(04) to a whole number.

Table 88. N3

Variable	Default	Description
BuildLog	FALSE	When set to TRUE, this variable will add more detailed RRS logging when creating N3 segments.
KeepTrailingSeparators	FALSE	When set to TRUE, this variable will not remove blank trailing elements from N3 segments.
POBoxDetect	FALSE	When set to TRUE this variable will enable evaluation of all N3 segments for POBox values, and will remap the POBox to the second address line in the N3 segment if the second line is blank.

Table 89. N4

Variable	Default	Description
BuildLog	FALSE	TRUE/FALSE value. Adds more detailed RRS logging when creating N4 segments.
KeepTrailingSeparators	FALSE	TRUE/FALSE value. Option to not remove blank trailing elements from N4 segments.

Table 90. REF

Variable	Default	Description
BuildLog	FALSE	TRUE/FALSE value. Adds more detailed RRS logging when creating REF segments.

Action library summaries

Information is available for all actions from within Datacap Studio. To access the



embedded help, select an action on the **Actions Library** tab and click the button.

Descriptions of the available actions are provided in the sections that follow.

“Global actions”

“Application specific actions” on page 1011

Global actions

You can use the global actions with any of the Datacap applications.

“Autodoc actions” on page 334

“Barcode_P actions” on page 348

“Barcode_X actions” on page 356

“CC actions” on page 359

“Cco2cco actions” on page 364

“CMISClient actions” on page 366

“ColorToBW actions” on page 380

“Convert actions” on page 382

“Dcclip actions” on page 429

“DCImageFix actions” on page 430

“DCO actions” on page 433

“dcpdf actions” on page 450

“Email actions” on page 463

“Equalize actions” on page 468

“Ewsml actions” on page 469

“Export actions” on page 481

“ExportDB actions” on page 507

“ExportXML actions” on page 517

“FileIO actions” on page 521

“FileNetIDM actions” on page 536

“FileNet P8 actions” on page 554

“FingerprintMaintenance actions” on page 568

“FPXML actions” on page 571

“Grayscale actions” on page 576

“IBMCM actions” on page 577

“ICR_C actions” on page 591

“ICR_P actions” on page 597

“ImageConvert actions” on page 602

“ImageFix actions” on page 610

- "Email actions" on page 610
- "Imprint actions" on page 622
- "Intellocate actions" on page 629
- "Invoice actions" on page 632
- "IOverlay actions" on page 661
- "Locate actions" on page 664
- "Lookup actions" on page 712
- "MC_Identify" on page 717
- "MC_Validation" on page 721
- "mvscan actions" on page 741
- "Maintenance Manager actions" on page 755
- "OCR_A actions" on page 799
- "OCR_N actions" on page 811
- "OCR_S actions" on page 813
- "OCR_SR actions" on page 826
- "OpenTextFaxServer actions" on page 834
- "PatternMatch actions" on page 850
- "Picture actions" on page 857
- "POLR actions" on page 864
- "Recog_Shared actions" on page 865
- "runner actions" on page 881
- "SPEXport actions" on page 903
- "Split actions" on page 909
- "TifMerge actions" on page 911
- "TM524 actions" on page 917
- "Validations actions" on page 917
- "Vote actions" on page 967
- "Vscan actions" on page 968
- "Web Services actions" on page 979
- "Zones actions" on page 988

Autodoc actions

Use the Autodoc actions to create fingerprints and match pages to fingerprints by using the local fingerprint service or a web fingerprint service.

The Autodoc actions connect you to a fingerprint service. You run these actions to specify how to create fingerprints, set up fingerprint matching processes, and update fingerprint statistics.

- "BlankPagesIDBySize" on page 335
- "CalculateOffset" on page 336
- "CreateFingerprint" on page 336
- "DeleteFingerprint" on page 337
- "FindBlackFingerprint" on page 337
- "FindFingerprint" on page 338
- "FindTemplate" on page 339
- "MergeCCOs_ByType" on page 339

“SetApplicationID” on page 340
“SetFilter_HostName” on page 341
“SetFilter_PageType” on page 341
“SetFingerprint” on page 342
“SetFingerprintDir” on page 342
“SetFingerprintFailureThreshold” on page 343
“SetFingerprintSearchArea” on page 344
“SetFingerprintWebServiceURL” on page 345
“SetMaxOffset” on page 345
“SetProblemValue” on page 346
“SetSearchArea” on page 347
“SetTemplateDir” on page 347
“UpdateFingerprintStats” on page 348

BlankPagesIDBySize

Uses the size of the Image file to determine if the file represents a blank page.

Syntax

```
bool BlankPagesIDBySize(StrParam)
```

Parameters

A three-part, comma-separated value consisting of:

1. Numeric value indicating the maximum size in bytes that qualifies a page as a blank page.
2. String value representing the Page Type of a blank page.
3. Numeric value (0, 1 or 2) to designate which pages in a multi-page Image file are to be evaluated.

The third parameter is optional.

- 0 = both sides of a two-page Image file.
- 1 = odd pages only.
- 2 = even images only.

Returns

False if any parameter is invalid, or the rule with this action is bound to a Field object of the Document Hierarchy. Otherwise, **True**.

Level

Batch, Document, or Page levels.

Details

Any page with an Image file smaller than the size parameter (in bytes) is assigned the Page Type value you enter as a parameter.

Example

```
BlankPagesIDBySize("1000,Blank_Page")
```

CalculateOffset

Sets the standard Offset value to be used when matching source pages to fingerprints.

Syntax

()

Returns

False if the action is not applied at the Page level or an error occurs. Otherwise, **True**.

Level

Page level only.

Details

Calculates the Confidence and Image_Offset variables (similarity with the Fingerprint and image shift compared with the Fingerprint image) for the page based on its relation to the page's fingerprint. The fingerprint ID must previously be set using Findfingerprint, SetFingerprint or some other method.

Example

```
CalculateOffset()
```

CreateFingerprint

Adds the image (TIF) file of the current page and the fingerprint (CCO) file to the fingerprint library of the application. The resulting fingerprint consists of the Image file (.tif) of the page and its Processing file (.cco).

Syntax

()

Parameters

None.

Returns

False if the rule with this action is not bound to a Page object of the Document Hierarchy; if the current page does not have an Image file; or if the fingerprint's two files cannot be created. Otherwise, **True**.

Level

Page level only.

Details

Creates a fingerprint for the current source page. The resulting fingerprint will consist of two files: the page's Image file (.tif) and its Processing file (.cco).

Attention: A SetFingerprintDir action must precede this action.

Example

```
SetFingerprintDir("C:\ParentDirectory\Application\fingerprint")  
CreateFingerprint()CreateFingerprint()
```

DeleteFingerprint

Removes the image (TIF) file of the current page and the fingerprint (CCO) file from the fingerprint library of the application.

Syntax

()

Parameters

None.

Returns

Always returns **True**. Under certain conditions the action will be unable to delete the fingerprint but will still return True. For example if the action is not applied at the Page level or if the fingerprint's Image file cannot be found. Please review the log file if DeleteFingerprint does not perform as expected.

Level

Page level only.

Details

Deletes the Image file (.tif) And Processing file (.cco) of the current page's fingerprint from the application's fingerprint directory, and its record from the Fingerprint database.

Attention: A SetFingerprintDir action must precede this action.

Example

```
SetFingerprintDir("C:\ParentDirectory\Application\fingerprint")  
DeleteFingerprint()
```

FindBlackFingerprint

Attempts to match black forms to fingerprints in the fingerprints directory of an application. If a match does not occur, the action responds according to the parameters that you enter.

Syntax

(StrParam)

Parameters

Two comma-separated values (the second is optional)

1. **True or False: True** If a fingerprint match is not found, a new fingerprint is created and the two fingerprint files (.tif and .cco) are placed in the fingerprint directory. **False**, if the task is to proceed without creating a new fingerprint.
2. **Optional:** The Page Type that is to be assigned to the newly created fingerprint if the first parameter is **True**. If you do not include this parameter, the action assigns the Page Type of the current page.

Returns

False, if the action is not applied at the Page level or if the first parameter is **False** and a fingerprint match does not occur. Otherwise, **True**.

Level

Page level only.

Details

This action attempts to match black forms to fingerprints in the fingerprints directory of an application.

If a match does not occur, the action responds according to the parameters you enter.

Example:

```
AnalyzeImage()  
SetSearchArea("0.5")  
SetProblemValue("0.7")  
SetFingerprintDir("ParentDirectory\Application\fingerprint")  
FindBlackFingerprint("True,PageType")
```

In this sequence, the FindBlackFingerprint action uses only the first 50% of the fingerprint to search for a match. It accepts a match of 0.7 or higher.

If no match is found, the sequence creates a new fingerprint and stores it in the location that is specified by the SetFingerprintDir action.

If the parameter is set to False, no fingerprint is created when a match is not found.

FindFingerprint

Attempts to match the current page to a fingerprint and creates a new fingerprint if a match does not occur.

Syntax

(StrParam)

Parameters

Two comma-separated values (the second is optional).

1. A **True** / **False** value: **True**, if a task is to create a new fingerprint and add it to the fingerprint directory when a match does not occur; **False**, if the task is to proceed without creating a new fingerprint.
2. Optional: The Page Type that is to be assigned to the newly created fingerprint. If omitted, the current Page Type of the current page is used.

Returns

False if the action is not applied at the Page level, or if the parameter is False and a fingerprint match does not occur. Otherwise, True. If a new Fingerprint cannot be added, the action still returns **True**.

Level

Page level only.

Details

This action attempts to match the current source page to a fingerprint, and creates a new fingerprint if a match does not occur. Include this action after a rule's SetSearchArea, SetProblemValue, and SetFingerprintDir actions.

Example

```
AnalyzeImage()  
RotateImage()  
RecognizePageOCR_S()  
SetSearchArea("0.5")  
SetProblemValue("0.7")  
SetFingerprintDir("\ParentDirectory\Application\Fingerprint")  
FindFingerprint("True,Invoice_Page")
```

In this example sequence, the FindFingerprint action uses only the first 50% of the current page to search for a match. However, it accepts a match of 0.7 or higher. If no match is found, the sequence creates a new fingerprint and stores it in the location that is specified by the SetFingerprintDir action. If the parameter is set to False, a new fingerprint is not created when there is no match.

FindTemplate

This action is replaced by FindFingerprint.

Syntax

(StrParam)

Parameters

String boolean value: **True** if the action is to create a new fingerprint if the current source page does not match the target fingerprint. **False** if the action is not to create a new fingerprint if there is not a match.

Returns

False if the action is not applied at the Page level. Otherwise, **True**.

Level

Page level only.

Details

This action should not be used, as it is scheduled to be removed in future versions. It has been replaced by FindFingerprint.

MergeCCOs_ByType

Merges the fingerprint files (.cco) that are associated with Page objects of the Document Hierarchy.

Syntax

(StrParam)

Parameters

Comma-separated String values that indicate the Page Types of the Document Hierarchy objects to be merged.

Returns

False, if a Fingerprint file (.cco) for one of the Page Types is not available. Otherwise, **True**.

Level

Document level only.

Details

Merges the Fingerprint files (.cco) associated with Page objects of the Document Hierarchy.

Example

```
MergeCCOs_ByType("Invoice_Page,Invoice_Cont")
```

The MergeCCOs_ByType action allows all values of the source pages to be assigned to a single, searchable Fingerprint Processing file (.cco).

SetApplicationID

Used with the Fingerprint Service to limit a fingerprint search to the specified application.

Syntax

(StrParam)

Parameters

String parameter that represents unique Application ID. Smart parameters are supported.

This value is used to retrieve a correct list of fingerprints that are loaded to the server.

Returns

False, if action SetFingerprintWebServiceURL() was not called. Otherwise, **True**.

Level

Batch, Document, or Page levels.

Details

Uses this action to specify unique application name, if multiple applications are using the same Fingerprint Service.

Example

```
SetFingerprintWebServiceURL(http://'FPSERVERNAME'/fpService/Service.asmx?WSDL)  
SetApplicationID("1040ez")
```

SetFilter_HostName

Limits a fingerprint match to the specified fingerprint class only.

Syntax

(StrParam)

Parameters

The String value of the Fingerprint Class you want to use as the filter.

Returns

Always **True**.

Level

Page level.

Details

Sets the Fingerprint Class filter for the identification (matching) algorithm. Smart Parameters are supported.

The filter forces the fingerprint-matching algorithm to use only fingerprints associated with the specified Fingerprint Class.

To disable this filter, call with an empty string as the parameter.

Example

```
SetFilter_HostName("MyFingerprintClass")
```

SetFilter_PageType

Limits a fingerprint match to the specified page type only.

Syntax

(StrParam)

Parameters

The String value of the Page Type you want to use as the filter.

Returns

Always **True**.

Level

All levels, but generally at the Page level.

Details

Sets the Page Type Filter For the identification (matching) algorithm. Smart Parameters are supported.

The filter will force the fingerprint-matching algorithm to use only fingerprints associated with that Page Type.

Example

```
SetFilter_PageType("Invoice_Page")
```

SetFingerprint

Sets the class and page type after you create a new fingerprint.

Syntax

(StrParam)

Parameters

A two-part, comma-separated value consisting of:

1. The Fingerprint Class value. Smart parameters are supported. Alternatively, the name of the Field on this page that specifies the Fingerprint Class.
2. (optional) The Fingerprint ID value. Smart parameters are supported. Alternatively, the name of the Field on this page that specifies the Fingerprint ID.

Returns

False if either parameter is invalid. Otherwise, **True**.

Level

Page level only.

Details

Sets a newly created fingerprint's Fingerprint Class and Fingerprint Class ID values.

After the mandatory Fingerprint Class value and optional Fingerprint Class ID have been manually assigned by a fingerprint creation task, this action places these values into the Host table of the application's Fingerprint database - as RefName and RefID values.

Example

```
SetFingerprint("@P\VendorName,@P\VendorID")
```

In this example, runtime values of the VendorName and VendorID Field objects will populate the Host table of the application's Fingerprint Database. Alternative method:

```
SetFingerprint("@VendorName,@VendorID")
```

SetFingerprintDir

Specifies the Fingerprint directory of your application.

Syntax

(StrParam)

Parameters

A String value that specifies the directory's name and path. Smart parameters are supported. A Drive ID, such as C:\, is optional.

Returns

Always **True**.

Level

All levels, but usually applied at the Page level.

Details

Sets the Fingerprint directory of your application. This directory contains the application's fingerprints.

Example

```
SetFingerprintDir("C:\ParentDirectory\Application\Fingerprint")
```

This action identifies the location and path of an application's Fingerprint directory.

SetFingerprintFailureThreshold

Specifies the percentage of fingerprint upload failures to ignore when you use the Fingerprint web service.

Syntax

(StrParam)

Parameters

Int parameter that represents percent threshold of fingerprint upload failures to ignore. Smart parameters are supported.

The batch aborts if the percentage of fingerprints that are failed to load exceeds this value.

Returns

Returns **False** when:

- The fingerprint service is not configured using action `SetFingerprintWebServiceURL()`.

Returns **False** and aborts the batch when:

- The input parameter is not a value from 0 to 100.
- If the percentage of fingerprints that failed to load is greater than the threshold.
- If no fingerprints have been loaded and at least one has failed to load.

Otherwise, **True**.

Level

Batch, Document, or Page levels.

Details

Uses this action to set the maximum number of fingerprint upload failures to ignore. If the application has been set using the action `SetApplicationID`, then the threshold will be set for that specific application only.

Example

```
FindFingerprint(True)
SetFingerprintFailureThreshold("10")
```

SetFingerprintSearchArea

Specifies the portion of the current page that is used during fingerprint matching.

Syntax

()

Parameters

string matchStart

string matchEnd

Parameters

Two parameters:

1. A decimal value from 0.01 (1%) to 1.00 (all) to indicate how much of the page is to be matched. If the second parameter is empty the first parameter represents the bottom of the area for matching, starting from the top of the page. If the second parameter is not empty, the first parameter is the top or start of the area for matching. For example, a single parameter of 0.5 indicates that fingerprint matching is limited to the first half of the page (0 - 50%). Decimal separators must be appropriate for the current locale. In applications that might be used in locales with different decimal separators, use percentage notation.
2. Optional: A decimal value from 0.01 (1%) to 1.0 (all) to indicate the end point on the page to be used for fingerprint matching. If this parameter is supplied, the first parameter is the starting point. For example: if the first parameter is 0.6, and the second parameter is 1.0, the last 40% of the page is used for fingerprint matching (60-100%).

Attention: In both cases, you can replace a decimal value with a percentage. To indicate that the value is a percentage, the action can use a number followed by p, such as 50p to represent 50%.

When you are using the percentage values, the number must be a whole number and must not contain a decimal separator. Decimal separators must be appropriate for the current locale. In applications that are used in locales with different decimal separators, use percentage notation.

Returns

False, if the first parameter is missing or is not numeric. Also returns **False**, if the second parameter is not numeric. Otherwise, **True**.

Level

All levels, but generally at the Page level.

Details

This action uses the numeric values that you supply to determine the portion of the current page that is used to find a matching fingerprint.

Example

```
SetFingerprintSearchArea("0.5","")
```

This example compares lines and words in the upper 50% of the current page to the lines and words in the same portion of each fingerprint. Notice that the second parameter is empty.

```
SetFingerprintSearchArea("0.5","1.0")
```

This example compares lines and words in the lower 50% of the current page to the lines and words in the same portion of each fingerprint. You can replace a parameter's decimal value with a percentage(p) or metric(m) number.

SetFingerprintWebServiceURL

Specifies the URL that links to the Fingerprint web service.

Syntax

(StrParam)

Parameters

String value specifying the URL that is the link to the Fingerprint Web Service. Smart parameters are supported.

Returns

Always **True**.

Level

All levels.

Details

Add SetFingerprintWebService URL with its single parameter to the function and move so it is prior to SetFingerprintDir action.

Attention: this action is not effective unless the Fingerprint Service has been installed and configured.

Example

```
SetFingerprintWebServiceURL("http://www.grandcorp.AR.com/fpservice/")  
SetFingerprintDir("\ParentDirectory\Application\Fingerprint")
```

SetMaxOffset

Sets the maximum offset value to use while matching source pages to fingerprints.

Syntax

(StrParam)

Parameters

Integer value between 1 and 255.

Returns

False, if the parameter is invalid. Otherwise, **True**.

Level

All levels.

Details

Sets the Maximum Offset value while matching source pages to fingerprints.

This action has no effect when used with the Fingerprint Service. The actual shift is four times the maximum offset value in pixels ($4 * \text{MaxOffset}$). Increasing this value improves but slows the matching process.

The default value is 6: $4 * 6 = 24$ pixels.

Example

```
SetMaxOffset("12")
```

This example results in a Maximum Offset value of 48 pixels, which is greater than the 24 pixel default.

SetProblemValue

Uses the decimal value that you supply as a parameter to set a minimum Matching Tolerance Rating.

Syntax

(StrParam)

Parameters

A decimal value from 0.00 (Lowest Tolerance) to 0.99 (Highest Tolerance). The decimal separator must be appropriate for the locale.

Returns

False, if the parameter is missing or the parameter is not numeric. Otherwise, **True**.

Level

All, but usually at the Page level.

Details

Uses the decimal value that you supply as a parameter to set a minimum Matching Tolerance Rating.

Important: A lower rating results in lower tolerance and a greater chance for a match, but also a greater chance for a **False** match.

Example

```
AnalyzeImage()  
CreateFields()  
RotateImage()  
RecognizePageOCR-S()  
SetSearchArea("0.5")  
SetProblemValue("0.70")  
SetFingerprintDir("\ParentDirectory\Application\Fingerprint")  
FindFingerprint("True")
```

In this sequence, the FindFingerprint action assigns a Matching Tolerance Rating that is not overly restrictive or unrealistically accepting. If the rule's conditions do not result in a match, and **True** is used as the parameter for the FindFingerprint action, a new fingerprint is added to the library.

SetSearchArea

This action is replaced by SetFingerprintSearchArea.

Syntax

(StrParam)

Parameters

Two comma-separated parameters. See SetFingerprintSearchArea for details.

Details

This action should not be used, as it is scheduled to be removed in future versions. It is replaced by SetFingerprintSearchArea.

SetTemplateDir

This action should not be used, it will be removed in future versions. This action is replaced by SetFingerprintDir.

Syntax

(StrParam)

Parameters

String value specifying the directory's name and path. A Drive ID (such as c:\) is optional.

Returns

Always **True**.

Level

All levels, but generally at the Page level.

Details

This action was replaced by SetFingerprintDir.

UpdateFingerprintStats

Updates the fingerprint statistics in the Fingerprint database.

Syntax

()

Parameters

None.

Returns

False if called from any level other than Page level, or the Fingerprint database is not accessible. Otherwise, **True**.

Level

Page level.

Details

Use this action to increment the count for the current page and update the fingerprint statistics in the fingerprint database.

Example

```
UpdateFingerprintStats()
```

Barcode_P actions

Use the Barcode_P actions to locate and read PDF-417 barcodes.

The Barcode_P actions recognizes various barcode types, searches the page for all barcodes and writes them to a list, and reads the value of the barcodes.

“Get2DCodeBP”

“GetAllBarcodesBP” on page 349

“GetBarcodeBP” on page 350

“GetDataMatrixCodeBP” on page 352

“IdentifyByBarcodesBP” on page 352

“MatchBarcodeBP” on page 353

“MatchBarcodePrefixBP” on page 354

“ReadBarCodeBP” on page 355

“SetMinimumConfidenceBP” on page 356

Get2DCodeBP

Recognizes PDF-417 codes.

Member of namespace

Barcode_P

Syntax

Get2DCodeBP ()

Parameters

None.

Returns

True if the action is called at the page level or field level. Otherwise, **False**.

In addition the calling object's value and variable *GetBarCode* is filled with the bar code value. This action also stores barcode information such as confidence, coordinates, code name, and size. If the object's barcode settings are set to read more than one barcode, and more than one barcode is found, barcodes are also stored in the variable *GetBarCodeX* where X is the index of the barcode found.

Level

Page or Field level only.

Details

Use this action if your page has PDF-417 codes. Any field must have a Position assigned when bar code reading is performed. For example the application would invoke the CreateFields action, and if the field has Position defined in the Document Hierarchy, it is ready for barcode reading. If anchors or fingerprint matching are used, ReadZones or other registration may be required to align the fields correctly.

Example:

```
Get2DCodeBP()
```

GetAllBarcodesBP

Searches the current page for all barcodes and writes them to the *GetBarCodeList* variable of the calling object.

Member of namespace

Barcode_P

Syntax

```
GetAllBarcodesBP (StrParam)
```

Parameters

The separator to use when storing multiple barcodes. The default separator is a comma.

Returns

True if the action is called at the page level or field level. Otherwise, **False**.

In addition the calling object's value and variable *GetBarCode* is filled with the bar code value. This action also stores barcode information such as confidence, coordinates, code name, and size. If the barcode settings of the object are set to read more than one barcode, and more than one barcode is found, barcodes are also stored in the variable *GetBarCodeX* where X is the index of the barcode found.

Level

Page and field level.

Details

Searches all the barcodes on the current page and stores them in the *GetBarcodeList* variable of the calling object. Each barcode value is separated using the string separator value entered as a parameter.

If the engine does not detect any barcodes, then the variable *GetBarcodeList* is not populated nor created.

Example:

```
GetAllBarcodesBP(",")
```

GetBarcodeBP

Recognizes arbitrary 1D or 2D codes.

Member of namespace

Barcode_P

Syntax

```
GetBarcodeBP ()
```

Parameters

None.

Returns

True, if the action is called at the page level or field level. Otherwise, **False**.

In addition, the calling object's value and variable *GetBarcode* is filled with the bar code value. This action also stores bar code information such as confidence, coordinates, code name, and size. If the object's bar code settings are set to read more than one bar code, and more than one bar code is found, bar codes are also stored in the variable *GetBarcodeX* where X is the index of the bar code found.

Level

Page or Field level only.

Details

Use this action if your page has 1D or 2D bar codes. Any field must have a Position assigned when bar code reading is performed. For example, the application starts the CreateFields action, and if the field has Position defined in the Document Hierarchy, it is ready for bar code reading. If anchors or fingerprint matching are used, ReadZones or other registration might be required to align the fields correctly.

To explicitly tell the engine which bar code types to be read, the field/page being recognized contains a variable or setup property called *bp_tp*. If the *bp_tp* variable is empty or does not exist, the engine defaults to Unknown. See the information for Unknown in the valid bar code types.

The value of this variable must be a combination of the bar code types. This variable is automatically set when you choose the bar code types through the **Zones** tab in Datacap Studio, under the BAR/P recognition settings.

Multiple types can be read by adding the values of their codes together.

When the bar code type is set to **Patch Code**, the following string values might be returned by the engine. The string value depends on the type of Patch Code that is found:

- Patch 1: **1100**
- Patch 2: **1001**
- Patch 3: **1010**
- Patch 4 / Toggle Patch: **0110**
- Patch 6: **0011**
- Patch T / Transfer patch: **0101**

Valid bar code types:

- 0 : Unknown. Detects all bar code types automatically, with the exception of PatchCode and PDF417. The PatchCode and PDF417 bar code types must be set explicitly to be detected.
- 1 : INDUSTRY 2 OF 5
- 2 : INTERLEAVED 2 OF 5
- 4 : IATA 2 OF 5
- 8 : DATALOGIC 2 OF 5
- 16 : INVERT 2 OF 5
- 32 : BCD MATRIX
- 64 : MATRIX 2 OF 5
- 128 : CODE 32
- 256 : CODE 39
- 512 : CODABAR 2
- 1024 : CODE 93
- 2048 : CODE 128
- 4096 : EAN-13
- 8192 : EAN-8
- 16384 : UPC-A
- 32768 : UPC-E
- 65536 : ADD 5
- 131072 : ADD 2
- 262144 : UCC128/EAN-128
- 524288 : Patch Code
- 1048576 : PostNet
- 2097152 : PDF417
- 4194304 : DataMatrix

- 8388608 : Code 39 Extended
- 16777216 : Code 93 Extended
- 33554432 : QRCode
- 67108864 : IntelligentMail
- 134217728 : Royal Mail (RM4SCC)
- 268435456 : Australian Post 4-State Code
- 536870912 : Aztec

Example:

```
GetBarcodeBP()
```

GetDataMatrixCodeBP

Recognizes Data Matrix codes

Member of namespace

Barcode_P

Syntax

```
GetDataMatrixCodeBP ()
```

Parameters

None.

Returns

True if the action is called at the page level or field level. Otherwise, **False**.

In addition the calling object's value and variable *GetBarCode* is filled with the bar code value. This action also stores barcode information such as confidence, coordinates, code name, and size. If the object's barcode settings are set to read more than one barcode, and more than one barcode is found, barcodes are also stored in the variable *GetBarCodeX* where X is the index of the barcode found.

Level

Page or Field level only.

Details

Use this action if your page or field has Data Matrix codes. Any field must have a Position assigned when bar code reading is performed. For example the application would invoke the CreateFields action, and if the field has Position defined in the Document Hierarchy, it is ready for barcode reading. If anchors or fingerprint matching are used, ReadZones or other registration may be required to align the fields correctly.

Example:

```
GetDataMatrixCodeBP()
```

IdentifyByBarcodesBP

Updates the current page type if a barcode match is found.

Syntax

```
(bool IdentifyByBarcodesBP(string barcodePageMappings,  
string mappingsDelim, string keyValueSep, bool caseSensitive)
```

Parameters

string **barcodePageMappings**

string **mappingsDelim**

string **keyValueSep**

bool **caseSensitive**

Parameters

barcodePageMappings: Delimited list of barcode value-page type mappings to be evaluated.

mappingsDelim: (Optional) Delimiter separating mappings. Default: comma.

keyValueSep: (Optional) Delimiter that is used to separate the barcode value and the page type in an individual mapping. Default: equals sign (=).

caseSensitive: (Optional) **True** enforces case sensitivity when you evaluate the list of barcodes that are found on the page. Default: **False**.

Returns

True, if a match is found. Otherwise, **False**.

Level

Page level

Details

This action searches all of the barcodes that are found on the current page for a match from the provided mappings by using the *GetBarcodeList* variable. If a barcode value is found, this action updates the type of the current page by using the corresponding input mapping.

The GetAllBarcodesBP action is called if the *GetBarcodeList* variable does not exist. If the GetAllBarcodesBP action was called earlier with a string parameter, use an identical symbol for the **mappingsDelim** parameter.

If multiple barcode values are found, the page type that is assigned is the first corresponding value from the *GetBarcodeList* variable.

Example

```
IdentifyByBarcodesBP(Separator=Separator_PageAttach=Attachment_Page,,)  
IdentifyByBarcodesBP(Separator=Separator_Page%Attach=Attachment_Page%,,,)  
IdentifyByBarcodesBP(Separator|Separator_Page%Attach|Attachment_Page%,|,True)
```

MatchBarcodeBP

Searches all the barcodes on the current page and checks if one of them matches the value that you entered as a parameter.

Member of namespace

Barcode_P

Syntax

MatchBarcodeBP (StrParam)

Parameters

The String value of the barcode.

Returns

True if the action is called at the page level or field level and one of the barcode values on the page matches the parameter value. The parameter value must not be empty. Otherwise, **False**.

In addition the calling object's value and variable *GetBarCode* is filled with the bar code value. This action also stores barcode information such as confidence, coordinates, code name, and size. If the object's barcode settings are set to read more than one barcode, and more than one barcode is found, barcodes are also stored in the variable *GetBarCodeX* where X is the index of the barcode found.

Level

Page level and field level.

Details

Searches all the barcodes on the current page and checks if one of the barcodes matches the parameter value. If a match occurs, the barcode's value is placed into a page level variable called *GetBarCode*. Refer to action *GetBarCode* for information regarding barcode configuration in Datacap Studio.

Example:

```
MatchBarcodeBP("2008")
```

MatchBarcodePrefixBP

Searches all the barcodes on the current page and checks if one of them matches the value that you entered as a parameter.

Member of namespace

Barcode_P

Syntax

MatchBarcodePrefixBP (StrParam)

Parameters

The String value of the barcode.

Returns

True if the action is called at the page level or field level and one of the barcode values on the page matches the parameter value. The parameter value must not be empty. Otherwise, **False**.

In addition the calling object's value and variable *GetBarCode* is filled with the barcode value. This action also stores barcode information such as confidence, coordinates, code name, and size. If the object's barcode settings are set to read more than one barcode, and more than one barcode is found, barcodes are also stored in the variable *GetBarCodeX* where X is the index of the barcode found.

Level

Page level and field level.

Details

Searches all the barcodes on the current page and checks if one of the barcodes starts with the parameter value. If a match occurs, the value of the barcode is placed into a page level variable called *GetBarCode*.

Refer to the *GetBarCode* action for information regarding barcode configuration in Datacap Studio.

Example:

```
MatchBarcodePrefixBP("ATM")
```

ReadBarCodeBP

Tests to determine if the first barcode on the current page contains the value that is specified by the parameter.

Member of namespace

Barcode_P

Syntax

```
ReadBarCodeBP (StrParam)
```

Parameters

A single string value of the barcode.

Returns

True if the first barcode on the page has a value that matches the parameter. Otherwise, **False**.

Level

Page level only.

Details

Checks if the current page contains a barcode with the value specified by the parameter. This action uses the first barcode it encounters. One possible use of this

action is to identify a document's Separator page. Refer to action `GetBarCode` for information regarding barcode configuration in Datacap Studio.

Example:

```
ReadBarCodeBP("Separator")
SetPageType("Separator")
```

This example looks for a barcode with the value "Separator". If found, the second action, a DCO action, establishes the page as a Separator page.

SetMinimumConfidenceBP

Sets the minimum confidence level that is required for barcodes that are read by the engine to be accepted.

Syntax

```
(bool SetMinimumConfidenceBP(StrParam)
```

Parameters

An integer value that represents the minimum confidence level that is required for barcodes that are read by the engine to be accepted. Valid values are in the range of 1-10. The default value, when this engine is not used, is 7.

Returns

False if the confidence value cannot be set either because it is invalid or if an error occurs or if the action is called at a level other than page or field. Otherwise, **True**.

Level

Page or Field level only

Details

Sets the minimum confidence level that is required for barcodes that are read by the engine to be accepted.

Example

```
SetMinimumConfidenceBP(5)
GetBarcodeBP()
```

Barcode_X actions

Use the Barcode_X actions to locate and read the first barcode on the page.

The Barcode_X actions retrieve the first barcode on a page, searches the page for that barcode, and reads the value of the barcode.

“GetBarCode”

“MatchBarcode” on page 358

“ReadBarCode” on page 358

GetBarCode

Gets the value of the first bar code on the current page or within the current field zone.

Member of namespace

Barcode_X

Syntax

GetBarcode ()

Parameters

None

Returns

Always **True**.

Level

Page or Field Level.

Details

Returns the value of any bar code within a zone. A rule set with this action can be applied to a Page object or Field object of the Document Hierarchy.

When applied to a Page object, the action reads the first bar code that it encounters on the page and creates a Page-level variable that is called *GetBarcode*, and places the bar code's value in the variable.

When applied to the Field object of a zoned field, the action assigns the bar code's value to the field.

- **Barcode_X Datacap Studio Information:** Use the Datacap Studio **Zones** tab to adjust the BAR/X properties of each bar code zone for optimal recognition.
- **Barcode Type:** It is important to set this property to the specific type of bar code that is expected. The default "All" searches for any 3of9, CODABAR, I2of5, Code 128, and Code 93 symbols, and returns spurious data in some cases.
- **Orientation:** Configures the expected orientation of the bar code. Horizontal is the normal orientation, where the bars are vertical within the overall symbol, which is wider than it is tall, and any printed text is below.
- **Quality:** This property is no longer used.
- **Height / Width:** This property is no longer used.
- **Search Up To:** The maximum number of bar codes that are expected within the field. Default is 1.
- **Remove Spaces:** If set to "Yes", any white-space characters are removed from the final recognized string.
- **Skip Recognition:** If set to "Yes", this field is skipped during recognition.

Example:

```
GetBarcode()  
rr_Compare_Not("@VALUE, @EMPTY")
```

If you need to test that a barcode was read, you can use the action `rr_Compare_Not()`. This example shows how to perform that test on the field level.

MatchBarcode

Searches all the barcodes on the current page and checks if one matches the value you enter as a parameter.

Member of namespace

Barcode_X

Syntax

MatchBarcode (StrParam)

Parameters

The String value of the barcode.

Returns

True if the action is called at the page level and one of the barcode values on the page matches the parameter value. Otherwise, **False**.

Level

Page level.

Details

Searches all the barcodes on the current page and checks if one of the barcodes matches the parameter value. If a match occurs, the barcode's value is placed into a page level variable called *GetBarCode*. Refer to action GetBarCode for information regarding barcode configuration in Datacap Studio.

Example

```
MatchBarCode("2008")
```

ReadBarCode

Tests to see if the first barcode on the current page contains the specified value.

Member of namespace

Barcode_X

Syntax

ReadBarCode (StrParam)

Parameters

A single string value of the barcode.

Returns

True if the first barcode on the page has a value that matches the parameter. Otherwise, **False**.

Level

Page level only.

Details

Checks if the current page contains a barcode with the value specified by the parameter. This action uses the first barcode it encounters.

One possible use of this action is to identify a document's Separator page.

Refer to action GetBarCode for information regarding barcode configuration in Datacap Studio.

Example

```
ReadBarCode("Separator")
SetPageType("Separator")
```

This example looks for a barcode with the value "Separator". If found, the second action, a DCO action, establishes the page as a Separator page.

CC actions

Use the CC actions to connect to IBM Content Classification.

The CC actions enable Datacap applications to use the IBM Content Classification Knowledge Base for fingerprint matching.

"FindFingerprintCC"

"SetKnowledgeBaseCC" on page 360

"SetLanguageCC" on page 360

"SetListenerURLCC" on page 361

"SetProblemValueCC" on page 362

"UpdateKnowledgeBaseCC" on page 363

FindFingerprintCC

Finds the closest matching fingerprint and assigns the page type to the page.

Member of namespace

CC

Syntax

FindFingerprintCC ()

Returns

True if the fingerprint is found. Otherwise, **False**.

Level

Page level.

Details

This action identifies a page using the IBM Content Classification technology. This technology analyzes the full text of pages and attempts to find match within the Knowledge Base specified in the SetKnowledgeBaseCC action. If a match is found, the Page type is populated with the ID of the category that was matched.

If a match is not found, the page type is set to **Other**.

Because the matching relies on a page's full text, a full page recognition action must be called prior to using the FindFingerprintCC action.

Example

```
RecognizePageOCR_S()  
FindFingerprintCC()
```

SetKnowledgeBaseCC

Sets the name of the CC Knowledge Base to use to classify documents.

Member of namespace

CC

Syntax

```
SetKnowledgeBaseCC ()
```

Parameters

KnowledgeBaseName

Type string

Knowledge Base name. Smart parameters are supported.

Parameters

The name of the IBM Content Classification Knowledge Base. This parameter is required and cannot be empty.

Returns

False, if the parameter is empty. Otherwise, **True**.

Level

All.

Details

Sets the name of the IBM Content Classification Knowledge Base to use to classify documents in an application.

This action must be called after the SetListenerURLCC action.

This action must be called before the actions SetProblemValueCC, FindFingerprintCC, and UpdateKnowledgeBaseCC.

Example

```
SetKnowledgeBaseCC("Mortgage")  
UpdateKnowledgeBaseCC()
```

SetLanguageCC

Sets the language used in the specified Knowledge Base.

Member of namespace

CC

Syntax

SetLanguageCC ()

Parameters

LanguageName

Type string

The name of the language used in the specified Knowledge Base

Returns

Always **True**.

Level

All.

Details

This action sets the language of the specified Knowledge Base.

This action must be called before the FindFingerprintCC action.

Example

```
SetListenerURLCC("http://localhost18087")
SetLanguageCC("English")
SetKnowledgeBaseCC("Mortgage")
SetProblemValueCC(0.9)
FindFingerprintCC()
```

SetListenerURLCC

Sets the URL of the CC Listener that is used for classification.

Member of namespace

CC

Syntax

SetListenerURLCC ()

Parameters

URL Type string

The URL of the CC Listener that will be used for classification. Smart parameters are supported.

Returns

True if the service URL is successfully set. Otherwise, **False**. This action does not test that the CC Listener exists and is running.

Level

All.

Details

This action configures the URL of the IBM Content Classification Listener that is used for classification, and opens the connection to the IBM Content Classification Listener.

This action must be called before the actions FindFingerprintCC, SetKnowledgeBaseCC, and UpdateKnowledgeBaseCC.

Example

```
SetListenerURLCC("http://localhost18087")
SetKnowledgeBaseCC("Mortgage")
SetProblemValueCC(0.9)
FindFingerprintCC()
```

SetProblemValueCC

Sets the minimum score for fingerprint matching.

Member of namespace

CC

Syntax

SetProblemValueCC ()

Parameters

MinScore

Type: double

Minimum score for fingerprint matching. Valid values are fractional values between zero and one (for example: 0.0 and 1.0)

Returns

True if the parameter value is between the valid range of zero to one (0.0 and 1.0). Otherwise, **False**.

Level

All.

Details

When FindFingerprintCC searches for a fingerprint match, a score between zero (no match) and one (a positive match) is calculated. This action sets the minimum score that a match must have to be considered a match. Any matches with a score less than the value specified is rejected. With this action, you can control the tolerance for documents matching an existing example.

When setting up the parameter in your application, use the decimal character from the system locale defined for the application in the Datacap Application Manager. For example, when the decimal character is a period, use a value from 0.0 to 1.0. When the decimal character is a comma, use a value from 0,0 to 1,0.

This action must be called before the FindFingerprintCC action.

Example

```
SetProblemValueCC(0.9)  
FindFingerprintCC()
```

UpdateKnowledgeBaseCC

Updates the CC Knowledge Base.

Member of namespace

CC

Syntax

UpdateKnowledgeBaseCC ()

Parameters

None.

Returns

True if the IBM Content Classification Knowledge Base is successfully updated. Otherwise, **False**.

Level

Page level.

Details

Provides classification feedback to the IBM Content Classification server.

Important: This method is to be used to provide feedback to a previously trained knowledge base, and not to be used as the primary method for training a knowledge base from scratch.

Although training via feedback is not recommended to build the knowledge base, it is recommended that it is used to submit feedback on classification data returned on a "trained" category. For instance, let us use a knowledge base that contains 5 categories, 4 of which have been trained. When FindFingerprintCC runs, it will either return a match or no match. When a match is found, it is recommended to provide feedback (using UpdateKnowledgeBaseCC) to the knowledge base to assert that the classification was correct.

When there is no match (no category returned) or there is a mismatch (wrong category returned), the recommended process is to export a copy of the document to a folder and train the knowledge base with these new documents via the IBM Content Classification Workbench. After the new documents have been added, the knowledge based should be exported, and documents processed in Datacap to provide feedback. The training should be done in a non Production system, then moved to Production after testing.

Although it is recommended that this procedure is used to submit feedback on trained categories, it should be done for a period of time, and not indefinitely, since this can cause the knowledge base to grow very large.

In conclusion, this action should be used only for documents where the classification result was correct, and should only be used during a period of time until the training on the knowledge base is complete.

Example

```
FindFingerprintCC()  
UpdateKnowledgeBaseCC()
```

Cco2cco actions

Use the Cco2cco actions to sort and filter the words and lines in a fingerprint CCO file.

This normalization is only required after full page recognition by an OCR or ICR action that does not automatically normalize the CCO. The OCR/S, OCR/A, and ICR/C actions normalize automatically.

“NormalizeCCO”

“SetMaxCharacterHeightAVG” on page 365

“SetMaxCharacterHeightTMM” on page 366

NormalizeCCO

The NormalizeCCO action sorts and filters the words and lines in a Fingerprint file (.cco) that are created by a Recognition engine, for use by navigation and pattern match actions. This action is only required after full page recognition by an OCR or ICR action that does not automatically normalize the CCO.

Member of namespace

Cco2cco

Syntax

NormalizeCCO ()

Returns

Always **True**.

Level

Page.

Details

This action sorts the words and lines in a Fingerprint file (.cco) created by a recognition engine, for use by navigation and pattern match actions. The action is called by full-page recognition actions for ICR/C, OCR/S, and OCR/A. This action must always be called before the Locate actions or the pat_RecogMatch_ID action are used to find recognized text on a page.

In this context, the fingerprint is calculated for a particular image in a batch, not in the Fingerprint database that contains fingerprints for various page types and layout variations that are defined for a particular application.

There are two types of Fingerprint files. One type is based on the image geometry. The second type is based on recognized text. The AnalyzeImage action creates a geometric fingerprint that contains lines and words that are based only on the

black pixels in the image. Full-page recognition actions, such as RecognizePageOCR_S, RecognizePageICR_C, RecognizePageOCR_A, create a fingerprint that is based on the results of recognition; that is, both geometry and text of the recognized characters, words and lines.

In Recognition-based fingerprints, the order of lines and words might appear to be arbitrary, especially if the page contains images, tables, stamps, or blocks of text with varying font sizes. This can cause unpredictable results from Locate actions that navigate geometrically. The word-matching and phrase-matching action **pat_RecogMatch_ID** also requires well-ordered text to work reliably.

The NormalizeCCO action reorders the words of text in a Recognition-based fingerprint into lines and words in standard reading order, from top to bottom and left to right.

Important: NormalizeCCO discards any "words" or blocks that contain characters taller than 1/4 inch, or the height set by SetMaxCharacterHeightTMM().

If the AnalyzeImage action is called before full-page recognition, the recognized text is placed into the geometry that is created by AnalyzeImage. This hybrid Fingerprint file is not always suitable for cco2cco. To force creation of a pure recognition-based fingerprint, call SetFingerprintRecogPriority(True) before full-page recognition. This guarantees that any existing geometric fingerprint is ignored, and it applies to OCR_S and ICR_C only.

The full page recognition actions from the ICR_C, OCR_A, and OCR_S libraries call NormalizeCCO() automatically unless the action CCONormalization_OFF (from the Recog_Shared library) is called before recognition. The full page recognition from the OCR_SR library, however, requires that NormalizeCCO() to be called manually post recognition.

Example:

```
SetFingerprintRecogPriority(True)
RecognizePageOCR_S()
NormalizeCCO()
pat_RecogMatch_ID()
```

SetMaxCharacterHeightAVG

Sets the maximum height of characters, by percentage over the average, that is allowed by the Cco2cco and NormalizeCCO actions.

Member of namespace

Cco2cco

Syntax

SetMaxCharacterHeightAVG (strParam)

Parameters

One or two digit **Integer** value sets the percent maximum height of characters over the average to permit in the CCO. Off by default. Parameter of zero (0) or less turns off this functionality.

Returns

Always **True**.

Level

Page.

Details

Example:

```
SetMaxCharacterHeightAVG(15)
NormalizeCCO()
```

SetMaxCharacterHeightTMM

Sets the maximum height of characters, by absolute value, that is allowed by the Cco2cco and NormalizeCCO actions.

Member of namespace

Cco2cco

Syntax

```
SetMaxCharacterHeightTMM (strParam)
```

Parameters

Integer: maximum height of characters to permit in the CCO, in 1/10 mm units. Default is 64, or approx 1/4 inch (50 pixels at 200 DPI).

Returns

Always **True**.

Level

Page.

Details

Example:

```
SetMaxCharacterHeightTMM(75)
NormalizeCCO()
```

CMISClient actions

IBM Content Management Interoperability Services (IBM CMIS) is an open standard that CMISClient actions use to enable communication between Datacap applications and content management systems over the internet.

The CMISClient actions configure the connection between Datacap applications and the IBM CMIS server. You run these actions to access the CMIS server, set up document attributes and folders on the server, and upload documents to the server for storage.

“CMISCreateFolder” on page 367

“CMISDeleteFile” on page 368

“CMISDeleteFolder” on page 369

“CMISDoesFileExist” on page 369

“CMISDoesFolderExist” on page 370

“CMISDownloadFile” on page 371

“CMISLogDocumentTypes” on page 372
“CMISLogin” on page 373
“CMISRefreshClientCache” on page 374
“CMISSetDocUploadProperty” on page 375
“CMISSetDocUploadType” on page 376
“CMISSetVersion” on page 377
“CMISUploadFile” on page 378
“CMISUploadPage” on page 379

CMISCreateFolder

Creates a folder on the CMIS server.

Member of namespace

CMISClient

Syntax

```
bool CMISCreateFolder(string cmisFolderPath, string cmisFolderName)
```

Parameters

cmisFolderPath

Type: string

cmisFolderName

Type: string

Parameters

- **cmisFolderPath** : The path of the parent folder.
- **cmisFolderName** : The name of the new folder.

Smart parameters are supported for all parameters.

Returns

True if the folder is created. Otherwise, **False**.

Level

All levels.

Details

Creates a folder on the configured CMIS server.

Note that objects created on the CMIS server may not be immediately accessible by the client. For example, the indexing service on your CMIS server may run on a configured interval, so your new object may not be available until the indexing service completes. If you are having trouble accessing newly created objects, check the settings and documentation of your CMIS server to determine when objects should become available.

Example:

```
CMISCreateFolder("/MyParent", "MyNewFolder")
```

Creates the folder MyNewFolder in the MyParent folder.

CMISDeleteFile

Deletes a file on the CMIS server.

Member of namespace

CMISClient

Syntax

```
bool CMISDeleteFile(string cmisFilePath, bool ignoreFailure)
```

Parameters

cmisFilePath

Type: string

ignoreFailure

Type: bool

Parameters

- **cmisFilePath** : The file to delete on the CMIS server. Smart parameters are supported.
- **ignoreFailure** : Set to **True** to ignore any failure so the action will always return **True**.

Returns

If **ignoreFailure** is **True**, the action always returns **True**, even if the file could not be deleted. If **ignoreFailure** is **False**, returns **True** if the file is deleted, otherwise **False** is returned.

Level:

All levels.

Details

Deletes the specified file in the CMIS repository.

Some, but not all, repositories have the notion of a "soft" delete, which will perform the final deletion at a later time. You should assume that once the object is deleted, that it is deleted forever. If an object is deleted, it can only be restored if the repository provides a mechanism to restore the object.

A delete failure can be ignored by using the **ignoreFailure** parameter. This allows actions to continue even if the delete should fail.

Example:

```
CMISDeleteFile("/MyFolder/MyDoc.txt", false)
```

This returns true if MyDoc.txt is deleted, otherwise it returns false.

```
CMISDeleteFile("/MyFolder/MyDoc.txt", true)
```

This always returns true, even if the file could not be deleted.

CMISDeleteFolder

Deletes a folder on the CMIS server.

Member of namespace

CMISClient

Syntax

```
bool CMISDeleteFolder(string cmisDirectoryPath, bool ignoreFailure)
```

Parameters

cmisDirectoryPath

Type: string

ignoreFailure

Type: bool

Parameters

- **cmisDirectoryPath** : The path of the folder to delete. Smart parameters are supported.
- **ignoreFailure** : Set to **True** to ignore any failure so the action will always return **True**.

Returns

If **ignoreFailure** is **True**, the action always returns **True**, even if the folder could not be deleted. If **ignoreFailure** is **False**, the action will return **True** if the directory is deleted, otherwise **False** is returned.

Level

All levels.

Details

The folder must be empty for the delete to be successful. Some, but not all, CMIS repositories have the notion of a "soft" delete where the deletion is performed at a later time. You should assume that once the object is deleted, that it is deleted forever. If an object is deleted, it can only be restored if the repository provides a mechanism to restore the object.

A delete failure can be ignored by using the **ignoreFailure** parameter. This allows actions to continue even if the delete should fail.

Example:

```
CMISDeleteFolder("/MyFolder/AnotherFolder", false)
```

This returns true if AnotherFolder is deleted, otherwise it returns false.

```
CMISDeleteFolder("/MyFolder/AnotherFolder", true)
```

This always returns true, even if the folder is not deleted.

CMISDoesFileExist

Tests that a file exists on the CMIS server.

Member of namespace

CMISClient

Syntax

```
bool CMISDoesFileExist(string cmisFilePath, bool existenceResult)
```

Parameters

cmisFilePath

Type: string

existenceResult

Type: bool

Parameters

- **cmisFilePath** : The document and path to test for existence. Smart parameters are supported.
- **existenceResult** : If **True** the action will return **True** when the file exists, otherwise **False** is returned. If **False** the action will return false if the file exists, otherwise **True** is returned.

Returns

existenceResult if **True**, then return **True** if file exists. If **False**, return **True** if file does not exist.

Level

All levels.

Details

This action tests that the provided file exists in the connected CMIS repository. This action can be configured to return true if the file exists or return true if the file does not exist.

Example:

```
CMISDoesFolderExist("/MyFolder/MyDoc.txt", True)
```

Returns true if MyDoc.txt exists in the repository and returns false if it does not exist.

```
CMISDoesFolderExist("/MyFolder/MyDoc.txt", False)
```

Returns false if MyDoc.txt exists in the repository and returns true if MyFolder does not exist.

CMISDoesFolderExist

Tests that a folder exists on the CMIS server.

Member of namespace

CMISClient

Syntax

```
bool CMISDoesFolderExist(string cmisDirectoryPath, bool existenceResult)
```

Parameters

cmisDirectoryPath

Type: string

existenceResult

Type: bool

Parameters

- **cmisDirectoryPath** : The folder and path to test for existence. Smart parameters are supported.
- **existenceResult** : If **True** the action will return **True** when the folder exists, otherwise **False** is returned. If **False** the action will return **False** if the folder exists, otherwise **True** is returned.

Returns

existenceResult if **True**, then returns **True** if folder exists. If **False**, returns **True** if directory does not exist.

Level

All levels.

Details

Tests that the provided directory path exists in the connected CMIS repository. This action can be configured to return true if the folder exists or return true if the folder does not exist.

Example:

```
CMISDoesFolderExist("/MyFolder/AnotherFolder", True)
```

Returns true if "AnotherFolder" exists in the repository and returns false if it does not exist.

```
CMISDoesFolderExist("/MyFolder/AnotherFolder", False)
```

Returns false if "AnotherFolder" exists in the repository and returns true if the folder does not exist.

CMISDownloadFile

Download a file that is on the CMIS server to a local hard disk.

Member of namespace

CMISClient

Syntax

```
bool CMISDownloadFile(string cmisFileToDownload, string fullFileNameTarget)
```

Parameters

cmisFileToDownload

Type: string

fullFileNameTarget

Type: string

Parameters

- `cmisFileToDownload` : The file, with the full path, to download from the CMIS repository.
- `fullFileNameTarget` : The path and file name for the downloaded file. Smart parameters are supported for all parameters.

Returns

Returns **True**, if the download is successful. Otherwise, **False**.

Level:

All levels.

Details

Downloads a file from the CMIS repository to a local hard disk. The file in the repository is not changed. If the target file exists, it is overwritten.

Example:

```
CMISDownloadFile("/MyCMISDir/MyCMISFile.txt", "c:\\MyLocalDir\\MyLocalFile.txt")
```

Downloads a copy of the file `MyCMISFile.txt` and names it `MyLocalFile.txt` on the local system.

CMISLogDocumentTypes

Logs the document types defined on the CMIS server.

Member of namespace

CMISClient

Syntax

```
bool CMISLogDocumentTypes()
```

Parameters

None.

Returns

Always **True**.

Level

All levels.

Details

This action will query the CMIS server and log all of the server defined document types to the RRS log, providing that logging is enabled. This is a diagnostic action and is not intended for use in a production environment, nor is it supported in a production environment. `CMISLogin` must be called prior to this action.

It may be useful for an developer to obtain this type information during application development and has no intended use other than this limited

diagnostic. Be aware that depending on the number of types and attributes defined on the CMIS server, this action may be slow and create a large log file.

Example:

```
LogDocumentTypes()
```

CMISLogin

Supply login credentials and connect to the CMIS server.

Member of namespace

CMISClient

Syntax

```
bool CMISLogin (string atomPubURL, string userID, string password,  
string repositoryID)
```

Parameters

atomPubURL

Type: string

userID

Type: string

password

Type: string

repositoryID

Type: string

Parameters

- atomPubURL : The AtomPub URL for your CMIS compatible repository.
- userID : The logon user ID.
- password : The password.
- repositoryID: Optionally specify the CMIS repository ID.

Smart parameters are supported for all parameters.

Returns

True if the login was successful. Otherwise, **False**.

Level

All levels.

Details

This action connects to a CMIS compatible repository by using an AtomPub URL. If the repository ID is blank, the connection is made to the first repository that is returned from the CMIS connection. This action must be called before you can use any of the other CMIS actions.

Example:

```
CMISLogin("http://localhost:8080/alfresco/service/api/cmisis",  
"MyUserID", "@APPVAR(values/adv/cmisispassword)")
```

The password is obtained from the application service and is set by using the Application Manager.

```
CMISLogin("http://localhost:8080/alfresco/service/api/cmisis",  
"MyUserID", "@APPVAR(values/adv/cmisispassword)", "MyRepositoryID")
```

This example passes the optional repository ID.

CMISRefreshClientCache

Refreshes the client-side cache.

Member of namespace

CMISClient

Syntax

```
bool CMISRefreshClientCache(string milliseconds)
```

Parameters

milliseconds

Type: string

Parameters

milliseconds : Refreshes the object local cache if it is older than the specified number of milliseconds. A value of 0 refreshes immediately. A value of **-1** disables this refresh feature and the CMIS interface manages the cache itself. By default, this refresh is disabled. Smart parameters are supported.

Returns

Always **True**.

Level

All levels.

Details

CMIS client-side caching first checks the session cache, if an object exists in the client cache. If the required object is found, that object is used without contacting the repository. It is possible that it might be a stale object.

This action refreshes the client-side cache, if it is older than the time specified in the parameter. The value of 0 refreshes the cache immediately. This action affects all subsequent CMIS actions, refreshing each object as they are used. Call this action with a value of **-1** to disable this automatic refresh.

Typically this action does not need to be called. It can be called when a specific application requires a cache refresh. If this action is not called, the CMIS interface manages the cache as appropriate.

Note: Objects that are created on the CMIS server might not be immediately accessible by the client. For example, the indexing service on your CMIS server might run on a configured interval, so a new object might not be available until the indexing service completes. This refresh setting does not affect components on

the CMIS server, such as the indexing service interval. If you have trouble accessing newly created objects, check the settings and documentation of your CMIS server to determine when objects become available.

Example:

```
CMISRefreshClientCache("1000")
```

Refreshes the client local cache if it is older than 1 second.

CMISSetDocUploadProperty

Sets the value of a property that belongs to the file that is uploaded.

Member of namespace

CMISClient

Syntax

```
bool CMISSetDocUploadProperty(string cmisDocPropertyName, string cmisDocPropertyValue,  
string valueType, bool isMulti)
```

Parameters

cmisDocPropertyName

Type: string

cmisDocPropertyValue

Type: string

valueType

Type: string

isMulti

Type: bool

Parameters

- **cmisDocPropertyName** : The name of the CMIS property to set.
- **cmisDocPropertyValue** : The value of the specified property.
- **valueType** : The CMIS type of the value. Must be string, integer, datetime, or boolean.
- **isMulti** : **True**, if the CMIS type is a multi-value type. **False**, if the CMIS type is single.

Smart parameters are supported for **cmisDocPropertyName**, **cmisDocPropertyValue** and **valueType**.

Returns

False, if the property cannot be set or the datetime parameter cannot not be processed. Otherwise, **True**.

Level

All levels.

Details

This action sets a value to the specified property for files or pages that are uploaded to the CMIS repository. Use of this action is optional. Standard CMIS properties are of the form CMIS:xxx. By convention, custom CMIS properties do not use the CMIS: prefix.

This action must be called before CMISUploadFile or CMISUploadPage. If multiple properties need to be set for a document upload, call this action multiple times to set multiple properties. The property must be defined for the document type within the CMIS repository. If the property or value is not valid for the document type, the error is not reported until the upload action is called. When the upload is complete, the property settings set with this action are discarded and this action needs to be called again to set any custom properties for another upload.

For a Boolean property type, the value must be either **True** or **False**.

The datetime type accepts an input string that specifies just the date or the date and time. The application attempts to parse the input date format based on the current locale. The action uses the locale set in the Application Service / *hr_locale* variable. If that is not set, the current OS locale is used to interpret the date/time. The action attempts to ignore unrecognized data, if possible. If the month, day, or year is missing, it attempts to use the values from the current date. If a time is not specified, then the time is set to 12:00 midnight. It is recommended to provide the full short date in the correct format for the current locale, and a full 4-digit year, to reduce the chance of the action guessing values incorrectly. Only Gregorian short date formats are supported.

Example input datetime strings in the en-US locale.

- 05/01/2009 14:57:32.8 becomes 5/1/2009 2:57:32 PM
- 2009-05-01 14:57:32.8 becomes 5/1/2009 2:57:32 PM
- 2009-05-01T14:57:32.8375298-04:00 becomes 5/1/2009 11:57:32 AM
- 5/01/2008 14:57:32.80 -07:00 becomes 5/1/2008 2:57:32 PM.

Example:

```
SetCMISDocUploadType("cmisbook:poem")
SetCMISDocUploadProperty("cmisbook:author", "Edgar Allan Poe", "string", False)
SetCMISDocUploadProperty("cmisbook:Title", "The Raven", "string", False)
CMISUploadPage("MyFile.txt", "/MyCMISDir", "text/plain")
```

This example uses a predefined custom document type and then sets the author and title properties of the uploaded file.

CMISSetDocUploadType

Sets the type of the file that will be uploaded.

Member of namespace

CMISClient

Syntax

```
bool CMISSetDocUploadType(string cmisDocType)
```

Parameters

cmisDocType

Type: string

Parameters

cmisDocType : The document type of the uploaded file. Smart parameters are supported.

Returns

Always returns **True**.

Level

All levels.

Details

This action sets the CMIS document type for the file that will be subsequently uploaded in a following upload action. Use of this action is optional. If it is not called, the value of cmis:document is used. To set the document type, this action must be called before CMISUploadFile or CMISUploadPage.

Standard CMIS properties are of the form CMIS:xxx. By convention, custom CMIS properties should not use the CMIS: prefix. Calling this action with an empty string, will reset the value back to the default cmis:document type. Once the upload is complete, the document type is reset to the default value and this action will need to be called again to set a custom value for another upload.

If the specified upload type is invalid or not defined on your CMIS server, an error will occur in the upload action.

Example:

```
SetCMISDocUploadType("cmisbook:poem")
SetCMISDocUploadProperty("cmisbook:author", "Edgar Allan Poe")
SetCMISDocUploadProperty("cmisbook:poemtitle", "The Raven")
CMISUploadPage("MyFile.txt", "/MyCMISDir", "text/plain")
```

This example uses a predefined custom document type and then sets the custom author and title properties of the uploaded file.

CMISSetVersion

Sets the version type of the file that is uploaded.

Member of namespace

CMISClient

Syntax

```
bool CMISSetVersion(string cmisVersion)
```

Parameters

cmisVersion

Type: string

Parameters

cmisSetVersion : The version of the uploaded document. It must be one of the following options: None, Major, Minor, CheckedOut. If this action is not called before the file upload, the None type is used.

Smart parameters are supported.

Returns

Always returns **True**. If the specified type is invalid or if a CMIS connection is not established, the version type defaults to None.

Level

All levels.

Details

Call this action before the CMISUploadFile action to set the version type of the uploaded file. The support for versions depends on the target repository. Check your repository documentation to determine which version types it supports when you are using the CMIS interface. The supported types might also vary based on the specific version configuration settings of your repository. This action is optional. If this action is not called, then the version type of None is used for uploaded files.

Example:

```
CMISSetVersion("Major")
CMISUploadFile("C:\MyDir\MyFile.txt", "MyFile.txt", "/MyCMISDir", "text/plain")
```

CMISUploadFile

Upload a file to the CMIS server.

Member of namespace

CMISClient

Syntax

```
bool CMISUploadFile(string fullyQualifiedFileName, string cmisUploadedName,
string cmisUploadDirectory, string mimeType)
```

Parameters

fullyQualifiedFileName

Type: string

cmisUploadedName

Type: string

cmisUploadDirectory

Type: string

mimeType

Type: string

Parameters

- **fullyQualifiedFileName** : The file name to upload with the full path specified.
- **cmisUploadedName** : The wanted final name of the file in the repository.

- **cmisUploadDirectory** : The full path for the folder within the repository where the file is stored.
- **contentType** : The mime type of the uploaded file.

Smart parameters are supported for all parameters.

Returns

True, if the file is successfully uploaded. Otherwise, **False**.

Level

All levels.

Details

This action uploads a file to the configured repository. You can optionally set properties of the uploaded file using the actions **CMISSetDocUploadProperty** and **CMISSetDocUploadType**.

Note: Objects that are created on the CMIS server might not be immediately accessible by the client. For example, the indexing service on your CMIS server might run on a configured interval, so your new object might not be available until the indexing service completes. If you are not able to access newly created objects, check the settings and documentation of your CMIS server to determine when objects are available.

A record of uploaded files is placed in the file **UploadRecord.xml** in the batch directory.

Example:

```
SetCMISDocUploadType("cmisbook:poem"
SetCMISDocUploadProperty("cmisbook:author", "Edgar Allan Poe")
SetCMISDocUploadProperty("cmisbook:Title", "The Raven")
CMISUploadFile("C:\MyDir\MyFile.txt", "MyFile.txt", "/MyCMISDir", "text/plain")
```

The uploaded file is placed into the **\MyCMISDir** folder and be given the mime type of **text/plain**.

CMISUploadPage

Upload the current DCO page to the CMIS server.

Member of namespace

CMISClient

Syntax

```
bool CMISUploadPage(string cmisUploadedName, string cmisUploadDirectory,
string mimeType)
```

Parameters

cmisUploadedName

Type: string

cmisUploadDirectory

Type: string

contentType

Type: string

Parameters

- **cmisUploadName** : The desired final name of the file in the repository.
- **cmisUploadDirectory** : The full path for the folder within the repository where the file will be stored.
- **contentType** : The mime type of the uploaded file.

Smart parameters are supported for all parameters.

Returns

True if the file is successfully uploaded. Otherwise, **False**.

Level

All levels.

Details

This action uploads the current DCO page to the configured repository.

Note: Objects created on the CMIS server may not be immediately accessible. For example, the indexing service on your CMIS server may run on a configured interval, so your new object may not be available until the indexing service completes. If you are having trouble accessing newly created objects, check the settings and documentation of your CMIS server to determine when objects should become available.

A record of uploaded files will be placed in the file UploadRecord.xml in the batch directory.

Example:

```
SetCMISDocUploadType("cmisbook:poem")
SetCMISDocUploadProperty("cmisbook:author", "Edgar Allan Poe")
SetCMISDocUploadProperty("cmisbook:poemtitle", "The Raven")
CMISUploadPage("MyFile.txt", "/MyCMISDir", "text/plain")
```

This set of actions sets property values of the page that will be uploaded, then uploads the page. The current page is placed into the \MyCMISDir folder and be given the mime type of text/plain.

ColorToBW actions

Use the ColorToBW actions to change the color depth of an image.

The ColorToBW actions can specify the color-to-BW conversion settings and change the color depth of an image according to these conversion settings.

“C2BW_Convert”

“C2BW_SetAttributes” on page 381

C2BW_Convert

Changes the color depth of an image according to the conversion settings specified by using C2BW_SetAttributes.

Syntax

(sParam)

Parameters

C2BW_Convert always produces a TIF image. If the source image has a TIF extension, use this parameter to provide an extension to use for a backup of the original file.

If not provided, the parameter defaults to **tio**. If the source image does not have a TIF extension, this parameter is ignored and use a TIF extension and the original image name will not be changed.

Smart parameters are supported.

Returns

Always **True**.

Level

All Levels.

Details

Changes the color depth of an image based on a set of defined attributes, creating a new TIF image.

If action C2BW_SetAttributes is not called first, a black and white image will be created.

If called at the batch level converts all images. If called at the document level, images within the document are converted. If called on the page, or field level the page is converted.

The supported input image formats are TIF (including LZW compression), PNG, BMP and JPG.

Example

```
C2BW_SetAttributes("1","0","3")
C2BW_Convert("tic")
```

C2BW_SetAttributes

Specifies the color-to-BW conversion settings.

Syntax

()

Parameters

string **BitsPerPixel**

string **Palette**

string **Dither**

Parameters

Requires 3 numeric values to configure the output image specifications:

1. Bits per pixel - 1, 4, 8, 24. A bit depth of 1 will produce a black and white image. A bit depth of 4, 8 or 24 will produce a grayscale or color image depending on the selected palette.
2. Palette - 0 Optimized, 1 Fixed, 2 Grayscale.
3. Dither - 0 None, 1 Floyd-Steinberg, 2 Ordered, 3 Optimized.

Smart parameters are supported.

Returns

Always **True**.

Level

All levels.

Details

Adjusts the output of the action C2BW_Convert. Optionally called before C2BW_Convert to configure the desired image output specifications. If this action is not called, these default values are used by C2BW_Convert:

- BitsPerPixel = 1 (Black and White)
- Palette = 0 (Optimized)
- Dither = 0 (None)

Example

```
C2BW_SetAttributes("1","0","3")
C2BW_Convert(tic)
```

Convert actions

Use the Convert actions to convert various electronic document files into TIFF image files.

The convert actions can convert files from these formats; Microsoft Excel. HTML, Microsoft Outlook, PDF, RTF, Text, Microsoft Word.

“Common actions” on page 383

“Excel actions” on page 386

“Html actions” on page 394

“Images actions” on page 396

“Outlook actions” on page 401

“Pdf actions” on page 404

“PdfFRE actions” on page 410

“Rtf actions” on page 416

“Tiff actions” on page 418

“Txt actions” on page 420

“Word actions” on page 424

“Zip actions” on page 426

Common actions

Use the Common actions to define the properties that are used by all of the conversion libraries for exception handling.

The default behavior is to abort the batch. But you can enable continuing to run the batch after a failure to increment a variable or to raise a task condition if workflow routing is configured.

Exception handling can be isolated to specific file types, with the unspecified file types aborted upon exception.

“ExceptionSetFileTypes”

“ExceptionSetHandler”

“ExceptionSetVariableName” on page 384

“ExceptionSetTaskCondition” on page 385

“SetNamePattern” on page 385

ExceptionSetFileTypes:

Sets the file types to be monitored for exception handling.

Member of namespace

Convert

Syntax

```
bool ExceptionSetFileTypes(string types)
```

Parameters

types Type: string

A comma delimited list of the file extensions to monitor.

Returns

Always **True**.

Level

Any level.

Details

Sets one or more file types to be monitored for special exception handling. Unspecified file types that encounter processing failures cause an abort without any further action. If this action is not called or the parameter is blank, then all file types are monitored.

Example:

```
ExceptionSetHandler(1)  
ExceptionSetFileTypes(.pdf,.doc,.docx,.xls,.xlsx,.txt)
```

ExceptionSetHandler:

Sets the type of exception handling to use during a batch processing failure.

Member of namespace

Convert

Syntax

```
bool ExceptionSetHandler(int handler)
```

Parameters

handler

Type: integer

The type of exception handler.

Parameters

- 0: Abort the batch (Default).
- 1: Increment a variable in the runtime hierarchy, do not abort the batch.
- 2: Raise a task condition, do not abort the batch.

Returns

True, if a valid parameter is specified. Otherwise, **False**.

Level

Any level.

Details

Use this action to change or reset the abort behavior that is caused by conversion failures. Avoiding aborts can be useful when an external workflow process is already in place.

Example:

```
ExceptionSetHandler(0)
```

ExceptionSetVariableName:

Sets the runtime document hierarchy variable to increment upon execution.

Member of namespace

Convert

Syntax

```
bool ExceptionSetVariableName(string varName)
```

Parameters

varName

Type: string

The variable to increment. Smart parameters are supported.

Returns

Always **True**.

Level

Any level.

Details

Sets the variable to increment upon execution.

The default value is *@B.ConvertExceptions* when the *ExceptionHandler* action is set to use the *var* variable value.

Example:

```
ExceptionHandler(1)
ExceptionSetVariableName(@B.PDFConvertExceptions)
```

ExceptionHandlerCondition:

Sets the task condition to be raised upon execution.

Member of namespace

Convert

Syntax

```
bool ExceptionSetTaskCondition(int taskCondition)
```

Parameters

taskCondition

Type: integer

A zero-based task condition index.

Returns

Always **True**.

Level

Any level.

Details

Sets the task condition, which is defined in workflow administration, to be raised upon execution.

Batch splitting is not supported.

Example:

```
ExceptionHandler(2)
ExceptionSetTaskCondition(0)
```

SetNamePattern:

Changes the default naming pattern for the converted files.

Member of namespace

Convert

Syntax

```
bool SetNamePattern(string PatternType)
```

Parameters

PatternType

Type: string

A single positive numeric value.

Parameters

Value of **1** or **2**.

- **1**: Uses the default AlphaDecimal pattern, which consists of 4 pairs of 2 characters from 01 to ZZ.
- **2**: Uses a TMxxxxxx pattern where xxxxxx is a number from 000001 to 999999.

Returns

True, if the setting is successful.

False, if the setting is rejected.

Level

Any level.

Details

Changes the default naming pattern for the converted files.

Example:

```
SetNamePattern(2)
```

Excel actions

Use the Excel actions to convert a Microsoft Excel document file into TIFF image files.

The Excel actions can resize columns or rows from a table, set page orientation, create blank pages, and print grid lines when you convert documents from Excel.

Restriction: The actions in the following table work with files from Excel 2000 or later.

“ExcelAutoFitColumns” on page 387

“ExcelAutoFitRows” on page 387

“ExcelOrientationToLandscape” on page 388

“ExcelOrientationToPortrait” on page 389

“ExcelPrintBlankPage” on page 389

“ExcelPrintGridlines” on page 390

“ExcelPrintQuality” on page 391

“ExcelScalingFactor” on page 392

“ExcelTiffCompression” on page 392

“ExcelWorkbookToImage” on page 393

ExcelAutoFitColumns:

Sets the automatic sizing of all columns for Excel Workbook converted to TIFF.

Member of namespace

Convert

Syntax

ExcelAutoFitColumns ()

Parameters

autoFitColumns

Type: bool

Parameters

autoFitColumns : A Boolean value that enables and disables the automatic sizing of columns in an Excel file.

True: The columns are set to automatically adjust the size based on the content.

False: The columns do not have their size adjusted.

Returns

Always **True**.

Level

Page level.

Details

This enables the auto sizing of columns to shrink or increase the size of the column to fit all of the data within the column. This action must be called before ExcelWorkbookToImage. If this action is not called, the setting that was saved in the original Excel file is used.

Example:

```
ExcelAutoFitColumns("False")  
ExcelWorkbookToImage()
```

ExcelAutoFitRows:

Sets the automatic sizing of all rows for Excel Workbook converted to TIFF.

Member of namespace

Convert

Syntax

ExcelAutoFitRows ()

Parameters

autoFitRows

Type: bool

Parameters

autoFitRows : A Boolean value that enables and disables the automatic sizing of rows in an Excel file.

True: The rows are set to automatically adjust the size based on the content.

False: The rows do not have their size adjusted.

Returns

Always **True**.

Level

Page level.

Details

This enables the auto sizing of rows to shrink or increase the size of the row to fit all of the data within the row. This action must be called before `ExcelWorkbookToImage`. If this action is not called, the setting that was saved in the original Excel file will be used.

Example:

```
ExcelAutoFitRows("False")
ExcelWorkbookToImage()
```

ExcelOrientationToLandscape:

Forces the orientation of Excel files to landscape for `ExcelWorkbookToImage`.

Member of namespace

Convert

Syntax

```
ExcelOrientationToLandscape ()
```

Parameters

None.

Returns

Always **True**.

Level

Page level.

Details

The TIFF files created from ExcelWorkbookToImage will all be created with a landscape orientation. This action must be called before ExcelWorkbookToImage. If this action is not called, the portrait / landscape setting that was saved in the original Excel file is used.

Example:

```
ExcelOrientationToLandscape()  
ExcelWorkbookToImage()
```

ExcelOrientationToPortrait:

Forces the orientation of Excel files to portrait for ExcelWorkbookToImage.

Member of namespace

Convert

Syntax

```
ExcelOrientationToPortrait ()
```

Parameters

None.

Returns

Always **True**.

Level

Page level.

Details

The TIFF files created from ExcelWorkbookToImage will all be created with a portrait orientation. This action must be called before ExcelWorkbookToImage. If this action is not called, the portrait / landscape setting that was saved in the original Excel file is used.

Example:

```
ExcelOrientationToPortrait()  
ExcelWorkbookToImage()
```

ExcelPrintBlankPage:

Determines if blank pages are created when converting Excel Workbook to TIFF.

Member of namespace

Convert

Syntax

```
ExcelPrintBlankPage ()
```

Parameters

blankPage

Type: bool

Parameters

blankPage : A Boolean value that enables or disables creation of blank TIFFs when there is a blank Excel page.

True: A blank TIFF is created if the page is blank.

False: A blank TIFF is not created if the page is blank.

Returns

Always **True**.

Level

Page level.

Details

When printing an Excel Workbook, it is typical to have overflow pages that sometimes have data and sometimes are blank. Setting the value to true creates a TIFF for the blank overflow pages in this process. If this action is not called before `ExcelWorkbookToImage`, then the default of False is used.

Example:

```
ExcelPrintBlankPage("False")
ExcelWorkbookToImage()
```

ExcelPrintGridlines:

Enables or disables gridlines when converting Excel files to TIFF.

Member of namespace

Convert

Syntax

```
ExcelPrintGridlines ()
```

Parameters

gridlines

Type: bool

Parameters

gridlines : A Boolean value that determines if gridlines are displayed in Excel files converted to TIFF.

True: Gridlines are included in the converted image.

False: Gridlines are not shown in the converted image.

Returns

Always **True**.

Level

Page level.

Details

The TIFF files created from ExcelWorkbookToImage will all use the specified grid setting when converted to TIFF. This action must be called before ExcelWorkbookToImage. If this action is not called, the grid line setting that was saved in the original Excel file is used.

Example:

```
ExcelPrintGridlines("False")
ExcelWorkbookToImage()
```

ExcelPrintQuality:

Adjusts the resolution of the image output by ExcelWorkbookToImage.

Member of namespace

Convert

Syntax

ExcelPrintQuality ()

Parameters

dpi Type: int

Parameters

dpi : A single positive numeric value for the dots per inch (dpi) of the output image.

Returns

False if the parameter is invalid. Otherwise, **True**.

Level

Page level.

Details

Sets the resolution of the output image for ExcelWorkbookToImage. If this action is not called, the default value of 200 dpi will be used. Typically, input documents for recognition are 200 dpi.

Example:

```
ExcelPrintQuality(200)
ExcelWorkbookToImage()
```

ExcelScalingFactor:

Forces the print scaling of Excel Workbook to a specific value for ExcelWorkbookToImage.

Member of namespace

Convert

Syntax

```
ExcelScalingFactor ()
```

Parameters

percent

Type: int

Parameters

percent: A positive integer that controls the scaling by percentage used when converting an Excel Workbook to TIFF. For example, a value of 100 means to print at 100%.

Returns

Always **True**.

Level

Page level.

Details

The TIFF files created from ExcelWorkbookToImage will all use the specified printing scale value when converted to TIFF. This action must be called before ExcelWorkbookToImage. If this action is not called, the scaling factor that was saved in the original Excel file is used.

Example:

```
ExcelScalingFactor(100)  
ExcelWorkbookToImage()
```

ExcelTiffCompression:

Sets the compression used in the TIFF output by ExcelWorkbookToImage.

Member of namespace

Convert

Syntax

```
ExcelTiffCompression ()
```

Parameters

tiffCompression

Type: string

Parameters

tiffCompression is one of the following values to set the TIFF compression:

- NONE : A color image with no compression.
- LZW : A color image using LZW compression.
- CCITT4 : A black and white image with fax CCITT4 compression.

Returns

Always **True**.

Level

Page level.

Details

Sets the compression of the output image from ExcelWorkbookToImage. If this action is not called, the default value of CCITT4 is used. Typically, input documents for recognition are black and white with fax compression.

Example:

```
ExcelTiffCompression("CCITT4")  
ExcelWorkbookToImage()
```

ExcelWorkbookToImage:

Converts a page with *.xls or *.xlsx file to a page or pages in TIFF format.

Member of namespace

Convert

Syntax

ExcelWorkbookToImage ()

Parameters

None.

Returns

True if the file is successfully converted to a TIFF document.

False if the current page is not an Excel Workbook or if there is a failure in the conversion.

If the number of input files/pages exceeds the maximum allowed or if there is a failure in the conversion, the batch is set to abort.

Level

Page level.

Details

If the current page is an Excel Workbook, the file is converted to multiple TIFF files, one TIFF file for each page within the Workbook, based on the settings of the other Excel actions that configure the conversion settings.

Each new TIFF also has a new page created within the application environment which can be processed by subsequent rules. The original file name from which the page was extracted is stored in the *ParentImage* variable, for possible future reference within your application.

If the configured output image format and compression only supports black and white, such as CCITT4, colored text is exported as black and background shading is set to white.

Example:

```
ExcelPrintQuality(200)
ExcelTiffCompression(CCITT4)
ExcelPrintBlankPage(False)
ExcelWorkbookToImage()
```

Html actions

Use the Html actions to convert an image file from HTML format to TIFF for recognition processing by Datacap.

The Html actions can specify output resolution and the compression algorithm to use when you convert documents from HTML to TIFF.

“HtmlPrintQuality”

“HtmlTiffCompression” on page 395

“HtmlToImage” on page 395

HtmlPrintQuality:

Adjusts the resolution of the image output by HtmlToImage.

Member of namespace

Convert

Syntax

HtmlPrintQuality ()

Parameters

dpi Type: int

A single positive numeric value for the dots per inch (dpi) of the output image.

Returns

False if the parameter is invalid. Otherwise, **True**.

Level

Page level.

Details

Sets the resolution of the output image for `HtmlToImage`. If this action is not called, the default value of 200 dpi is used. Typically, input documents for recognition are 200 dpi.

Example:

```
HtmlPrintQuality(200)  
HtmlDocumentToImage()
```

HtmlTiffCompression:

Sets the compression used in the TIFF output by `HtmlToImage`.

Member of namespace

Convert

Syntax

```
HtmlTiffCompression ()
```

Parameters

tiffCompression

Type: string

A parameter of one of the following values to set the TIFF compression:

Parameters

- NONE : A color image with no compression.
- LZW : A color image using LZW compression.
- CCITT4 : A black and white image with fax CCITT4 compression.

Returns

Always **True**.

Level

Page level.

Details

Sets the compression of the output image from `HtmlToImage`. If this action is not called, the default value of CCITT4 is used. Typically, input documents for recognition are black and white with fax compression.

Example:

```
HtmlTiffCompression(CCITT4)  
HtmlToImage()
```

HtmlToImage:

Converts a page with *.htm or *.html file to a page or pages in TIFF format.

Member of namespace

Convert

Syntax

HtmlToImage ()

Parameters

None.

Returns

True if the file is successfully converted to a TIFF document.

False if the current page is not an Html Document or if there is a failure in the conversion.

If the number of input files/pages exceeds the maximum allowed or if there is a failure in the conversion, the batch will be set to abort.

Level

Page level.

Details

If the current page is an Html Document, the file is converted to multiple TIFF files, one TIFF file for each page within the Document, based on the settings of the other Html actions that configure the conversion settings.

Each new TIFF also has a new page created within the application environment which can be processed by subsequent rules. The original file name from which the page was extracted is stored in the ParentImage variable, for possible future reference within your application.

If the configured output image format and compression only supports black and white, such as CCITT4, colored text is exported as black. Html table borders will not be rendered onto output images.

Example:

```
HtmlPrintQuality(200)
HtmlTiffCompression(CCITT4)
HtmlToImage()
```

Images actions

Use the Images actions to convert various image formats into a black and white TIFF image file for recognition processing by Datacap.

The Images actions can specify default DPI settings and image type file extensions for the images that you want to convert to TIFF.

“ImageDefaultDPI” on page 397

“ImageFileTypesToConvert” on page 397

“ImageMonoThreshold” on page 398

“ImageMonoType” on page 399

“ImageToTIFF” on page 400

ImageDefaultDPI:

Sets the default dpi (dots per inch) when converting images that do not have an embedded dpi value.

Member of namespace

Convert

Syntax

`ImageDefaultDPI ()`

Parameters

X Type: short

Horizontal dpi (X axis). A positive numeric value normally ranging from 96 to 300

Y Type: short

Vertical dpi (Y axis). A positive numeric value normally ranging from 96 to 300

Returns

False if there is a failure to set the default X or Y dpi value. Otherwise **True**.

Level

Page level.

Details

Sets the default dpi to use when converting color or grayscale images to TIFF when the source image does not have a dpi value.

Example:

```
ImageFileTypesToConvert(".jpg,.jpeg,gif,bmp")
ImageDefaultDPI("200","200")
ImageMonoType(4)
ImageMonoThreshold(9)
ImageToTIFF()
```

ImageFileTypesToConvert:

Sets the file extension values of image types to convert to tiff.

Member of namespace

Convert

Syntax

`ImageFileTypesToConvert ()`

Parameters

fileextensions

Type: string

A CSV string of file extensions that defines the image types that will be converted.

Parameters

fileextensions : A CSV string of file extensions that defines the image types that will be converted.

Returns

Always **True**.

Level

Page level.

Details

Sets the file extension values of image types to convert to TIFF. The file types of JPEG, BMP, PNG, TIFF and GIF are supported. A period prefixing the extension is allowed, but not required. Because TIFF is supported, it is possible to use this action to convert a color TIFF to a Black and White TIFF.

This action must be used prior to ImageToTIFF.

Example:

```
ImageFileTypesToConvert(".jpg,.jpeg,gif,bmp")  
ImageToTIFF()
```

ImageMonoThreshold:

Sets the threshold value when converting Image to 1 bit tiff using threshold type conversion.

Member of namespace

Convert

Syntax

```
ImageMonoThreshold ()
```

Parameters

thresh Type: short

A positive numeric value from 1 to 255.

Parameters

thresh: A positive numeric value from 1 to 255.

Returns

Always **True**.

Level

Page level.

Details

Sets the B/W conversion algorithm value to use when converting color or grey scale images to TIFF with the Threshold method option.

Example:

```
ImageFileTypesToConvert(".jpg,.jpeg,gif,bmp")
ImageMonoType(4)
ImageMonoThreshold(9)
ImageToTIFF()
```

ImageMonoType:

Sets the method to use when converting color images to black and white tiffs.

Member of namespace

Convert

Syntax

ImageMonoType ()

Parameters

Mono Type: int

Parameters

Mono : A positive numeric value of 1 to 4 for the type of conversion to use.

1. Convert image using Diffusion method.
2. Convert image using Halftone method.
3. Convert image using Bayer method.
4. Convert image using Threshold method with a threshold value. Value defaults to 10.

Returns

Always **True**.

Level

Page level.

Details

Sets the black and white conversion algorithm to use when converting color or grayscale images to TIFF. If you are using the threshold method, you must also call the ImageMonoThreshold action prior to converting the image with ImageToTIFF.

Error diffusion is a type of halftoning in which the quantization residual is distributed to neighboring pixels that have not yet been processed. Its main use is to convert a color image into a black and white image. Halftone is the reprographic technique that simulates continuous tone imagery through the use of dots, varying either in size or in spacing. Bayer method is an image dithering algorithm that is commonly used to maintain the characteristics of a photo image of higher colors

in an image of less color depth. The threshold method makes individual pixels in an image black if their value is greater than the threshold value and the remaining pixels white.

For best results, you may need to experiment with the different types of images that you are expecting to process and select the conversion method that gives you the best results. If the resulting TIFF images are going to be used in a subsequent recognition process, pick the technique that works best to output dark, solid characters with as little background noise, or dithering around the characters, as possible.

Example:

```
ImageFileTypesToConvert(".jpg,.jpeg,gif,bmp")
ImageMonoType(1)
ImageToTIFF()
```

ImageToTIFF:

Converts an Image file to TIFF format.

Member of namespace

Convert

Syntax

ImageToTIFF ()

Parameters

None.

Returns

True if the file is successfully converted to a TIFF document.

False if the current page is not a supported Image type or if there is a failure in the conversion.

If the number of input files/pages exceeds the maximum allowed or if there is a failure in the conversion, the batch is set to abort.

Level

Page level.

Details

If the current page is an Image, the file will be converted to a single TIFF file. ImageFileTypesToConvert must be used along with ImageToTIFF. Only the image types set by a previous call to ImageFileTypesToConvert is converted to a TIFF. No images are converted if ImageFileTypesToConvert has not been set.

Each TIFF will also have a new page created within the application environment which can be processed by subsequent rules. The original file name from which the page was extracted is stored in the *ParentImage* variable, for possible future reference within your application.

Example:

```
ImageFileTypesToConvert(".jpg,.jpeg,gif,bmp")  
ImageToTIFF()
```

Outlook actions

Use the Outlook actions to convert the text of a Microsoft Outlook message file to a TIFF image file and save any attachments to disk. Saved attachments can then be processed as needed by subsequent actions.

The Outlook actions can specify the image resolution and the compression algorithm that is used when you convert from Outlook to TIFF.

“OutlookMessageToAttachmentOnly”

“OutlookMessageToImageAndAttachment” on page 402

“OutlookPrintQuality” on page 403

“OutlookTiffCompression” on page 403

OutlookMessageToAttachmentOnly:

Extracts attachments from *.msg or *.eml to pages without converting the MSG to a TIFF.

Member of namespace

Convert

Syntax

OutlookMessageToAttachmentOnly ()

Parameters

None.

Returns

True, if the attachments are successfully extracted from the message, or if the message contains no attachments.

False, if the current page is not an Outlook Message or if there is a failure in the conversion.

If the number of input files/pages exceeds the maximum that is allowed or if there is a failure in the conversion, the batch is set to abort.

Level

Page level.

Details

If the current page is an Outlook Message, the attachments within the message are saved as separate files.

Each attachment or embedded image within the email is removed from the email and placed on disk. A new page entry is created for each attachment that can be

processed by subsequent rules. The original file name from which the page was extracted will be stored in the *ParentImage* variable, for possible future reference within your application.

Example:

```
OutlookMessageToAttachmentOnly()
```

OutlookMessageToImageAndAttachment:

Converts a *.msg or *.eml file to a page or pages in TIFF format.

Member of namespace

Convert

Syntax

```
OutlookMessageToImageAndAttachment ()
```

Parameters

None.

Returns

True if the file is successfully converted to a TIFF document.

False if the current page is not an Outlook Message or if there is a failure in the conversion.

If the number of input files/pages exceeds the maximum allowed or if there is a failure in the conversion, the batch is set to abort.

Level

Page level.

Details

If the current page is an Outlook Message, the file is converted to multiple TIFF files, one TIFF file for each page within the Message, based on the settings of the other Outlook actions that configure the conversion settings. A copy of the original MSG file without attachments is created and converted to TIFF as well.

Each new TIFF also has a new page created within the application environment which can be processed by subsequent rules. The original file name from which the page was extracted is stored in the *ParentImage* variable, for possible future reference within your application.

Each attachment or embedded image within the e-mail is removed from the e-mail and placed on disk. A new page entry will be created for each attachment that can be processed by rules.

If the configured output image format and compression only supports black and white, such as CCITT4, colored text is exported as black.

Example:

```
OutlookPrintQuality(200)
OutlookTiffCompression("CCITT4")
OutlookMessageToImageAndAttachment()
```

OutlookPrintQuality:

Adjusts the resolution of the image output by OutlookMessageToImageAndAttachment.

Member of namespace

Convert

Syntax

OutlookPrintQuality ()

Parameters

dpi Type: int

Parameters

dpi : A single positive numeric value for the dots per inch (dpi) of the output image.

Returns

False if the parameter is invalid. Otherwise, **True**.

Level

Page level.

Details

Sets the resolution of the output image for OutlookMessageToImageAndAttachment. If this action is not called, the default value of 200 dpi is used. Typically, input documents for recognition are 200 dpi.

Example:

```
OutlookPrintQuality(200)
OutlookMessageToImageAndAttachment()
```

OutlookTiffCompression:

Sets the compression used in the TIFF output by OutlookMessageToImageAndAttachment.

Member of namespace

Convert

Syntax

OutlookTiffCompression ()

Parameters

tiffCompression

Type: string

Parameters

tiffCompression : A parameter of one of the following values to set the TIFF compression:

- NONE : A color image with no compression.
- LZW : A color image using LZW compression.
- CCITT4 : A black and white image with fax CCITT4 compression.

Returns

Always **True**.

Level

Page level.

Details

Sets the compression of the output image from OutlookMessageToImageAndAttachment. If this action is not called, the default value of CCITT4 is used. Typically, input documents for recognition are black and white with fax compression.

Example:

```
OutlookTiffCompression("CCITT4")
OutlookMessageToImageAndAttachment()
```

Pdf actions

Use the Pdf actions to convert an image file from PDF format to TIFF for recognition processing by Datacap.

The Pdf actions can specify bit depth of the output image, as well as the compression algorithm and conversion method to use when you convert from PDF to TIFF.

"PDFBitDepth"

"PDFCompression" on page 405

"PDFConversionMethod" on page 406

"PDFDocumentToImage" on page 407

"PDFGrayscale" on page 407

"PDFHorizontalResolution" on page 408

"PDFQuality" on page 409

"PDFVerticalResolution" on page 409

PDFBitDepth:

Sets the bit depth of the image output by PDFDocumentToImage.

Member of namespace

Convert

Syntax

PDFBitDepth ()

Parameters

p_iVal Type: int

Parameters

One of the following bit depth values are allowed.

- 1 : black and white.
- 8 : 256 color.
- 24 : True color.

Returns

Always **True**.

Level

Page level.

Details

Sets the bit depth for the output TIFF. The number of bits determine the color capacity of an image. If this action is not called, a default value of 1 is used. This action must be called before PDFDocumentToImage.

Example:

```
PDFBitDepth(1)
PDFDocumentToImage()
```

PDFCompression:

Sets the compression method used in the TIFF output by PDFDocumentToImage.

Member of namespace

Convert

Syntax

PDFCompression ()

Parameters

p_iVal Type: int

Parameters

One of the following compression values are allowed.

- 1 : No compression.
- 2 : CCITT modified Huffman RLE.
- 3: CCITT Group 3 fax.
- 4: CCITT Group 4 fax.
- 5: LZW Lempel-Ziv and Welch

- 7 : JPEG

Returns

Always **True**.

Level

Page level.

Details

Sets the compression of the output image from `PDFDocumentToImage`. If this action is not called, the default value of CCITT4 will be used. Typically, input documents for recognition are black and white with fax compression.

Example:

```
PDFBitDepth(1)
PDFCompression(4)
PDFDocumentToImage()
```

PDFConversionMethod:

Sets the conversion method used in the TIFF output by `PDFDocumentToImage`.

Member of namespace

Convert

Syntax

```
PDFConversionMethod ( )
```

Parameters

p_iVal Type: int

Parameters

A numeric value of one of the following.

- 1 : Uses method 1 of converting PDF to TIFF.
- 2 : Uses method 2 of converting PDF to TIFF.

Returns

Always **True**.

Level

Page level.

Details

There are two methods that can be used to convert a PDF to TIFF. The first method is slightly faster. The second method gives better results. If this action is not called, `PDFDocumentToImage` uses the second method by default, which produces a more accurate output TIFF.

Example:

```
PDFBitDepth(1)
PDFConversionMethod(2)
PDFDocumentToImage()
```

PDFDocumentToImage:

Create a TIFF image for each page in a PDF file.

Member of namespace

Convert

Syntax

```
PDFDocumentToImage ()
```

Parameters

None.

Returns

True if the file is successfully converted to a TIFF document.

False if the current page is not a PDF or if there is a failure in the conversion.

If the number of input files/pages exceeds the maximum allowed or if there is a failure in the conversion, the batch is set to abort.

Level

Page level.

Details

If the current page is a PDF, the file will be converted to multiple TIFF files, one TIFF file for each page within the document.

Each new TIFF also has a new page created within the application environment which can be processed by subsequent rules. The original file name from which the page was extracted is stored in the *ParentImage* variable, for possible future reference within your application.

Example:

```
PDFBitDepth(1)
PDFDocumentToImage()
```

PDFGrayscale:

Sets the output by PDFDocumentToImage to be grayscale.

Member of namespace

Convert

Syntax

```
PDFGrayscale ()
```

Parameters

p_bVal

Type: bool

Parameters

A Boolean value to set grayscale mode.

True: The resulting TIFF is converted to grayscale.

False: The resulting TIFF is not converted to grayscale.

Returns

Always **True**.

Level

Page level.

Details

Sets the PDF conversion to TIFF so the output images are grayscale. This action must be called before `PDFDocumentToImage`. If this action is not called, a default value of **False** is used by `PDFDocumentToImage`.

Example:

```
PDFGrayscale(True)
PDFDocumentToImage()
```

PDFHorizontalResolution:

Sets the output horizontal resolution for `PDFDocumentToImage`.

Member of namespace

Convert

Syntax

```
PDFHorizontalResolution ()
```

Parameters

p_iVal Type: int

Parameters

p_iVal : A positive numeric value for the horizontal resolution in dots per inch (DPI).

Returns

Always **True**.

Level

Page level.

Details

Sets the horizontal resolution for PDF conversion to TIFF. It is recommended that the horizontal and vertical resolutions be kept the same, creating an isotropic image. If this action is not called, the default resolution of 200 is used. This action must be called before `PDFDocumentToImage`.

Example:

```
PDFBitDepth(1)
PDFHorizontalResolution(200)
PDFVerticalResolution(200)
PDFDocumentToImage()
```

PDFQuality:

Sets the conversion quality for `PDFDocumentToImage`.

Member of namespace

Convert

Syntax

`PDFQuality ()`

Parameters

p_iVal Type: int

Parameters

p_iVal : A positive numeric value between 0 and 100 that determines the image quality. 100 is the highest quality.

Returns

Always **True**.

Level

Page level.

Details

This determines the quality of the output TIFF. Choosing a higher value will give you a better output TIFF, but will increase processing time. This action must be called before `PDFDocumentToImage`. If `PDFQuality` is not called, the default value of 100 is used.

Example:

```
PDFBitDepth(1)
PDFQuality(100)
PDFDocumentToImage()
```

PDFVerticalResolution:

Sets the output vertical resolution for `PDFDocumentToImage`.

Member of namespace

Convert

Syntax

PDFVerticalResolution ()

Parameters

p_iVal Type: int

Parameters

p_iVal : A positive numeric value for the vertical resolution in dots per inch (DPI).

Returns

Always **True**.

Level

Page level.

Details

Sets the vertical resolution for PDF conversion to TIFF. It is recommended that the horizontal and vertical resolutions be kept the same, creating an isotropic image. If this action is not called, the default resolution of 200 is used. This action must be called before PDFDocumentToImage.

Example:

```
PDFBitDepth(1)
PDFHorizontalResolution(200)
PDFVerticalResolution(200)
PDFDocumentToImage()
```

PdfFRE actions

Use the PdfFRE actions can use the Abbyy FineReader Engine to convert an image file from PDF format to TIFF for recognition processing by Datacap.

The PdfFRE actions can specify bit depth of the output image, as well as the compression algorithm and conversion method to use when you convert from PDF to TIFF.

“PDFConversionMode”

“PDFDocumentToImage” on page 411

“PDFImageCompression” on page 412

“PDFImageFileExtension” on page 414

“PDFImageFileResolution” on page 414

“PDFImageUseFastBinarization” on page 415

PDFConversionMode:

Sets the default conversion mode to use for images.

Member of namespace

Convert

Syntax

```
bool PDFConversionMode(int mode)
```

Parameters

mode Type: int
The conversion mode.

Parameters

mode: a positive numeric value of 0 to 1 for the type of conversion to use.

0: Preserve the color in images. Default. The default Black and White, Color, and Gray image compression is used unless it is changed by using the `SetCompression()` method.

1: Convert all images to Black and White. The default Black and White image compression is used unless it is changed by using the `SetCompression()` method.

Returns

Always **True**.

Level

Any level.

Details

Sets the default image compression mode to use for black and white, gray, and color images.

Example:

```
PDFImageCompression(0,16)  
PDFImageCompression(1,33)  
PDFImageCompression(2,32)  
PDFConversionMode(1)  
PDFDocumentToImage()
```

PDFDocumentToImage:

Converts a PDF file to TIFF format.

Member of namespace

Convert

Syntax

```
bool PDFDocumentToImage()
```

Parameters

None

Returns

True if the file is successfully converted to a TIFF document.

False if the current page is not a supported image type or if there is a failure in the conversion.

If the number of input files/pages exceeds the maximum number that is allowed or if there is a failure in the conversion, the batch is set to abort.

Level

Page level.

Details

If the current page is a PDF, the file is converted to multiple TIFF files, one TIFF file for each page in the document.

A new page is created in the application environment for each new TIFF file, which can be processed by subsequent rules. The original file name from which the page was extracted is stored in the *ParentImage* variable for future reference in your application.

Example:

```
PDFDocumentToImage()
```

PDFImageCompression:

Sets the default image compression to use for black and white, gray, and color images.

Member of namespace

Convert

Syntax

```
bool PDFImageCompression(int ImageType, int compression)
```

Parameters

imageType

Type: int

The applicable image type.

compression

Type: int

The image compression.

Parameters

intimageType: A positive numeric value of 0 to 2 for the type of conversion to use.

- **0:** BW images. Default compression is IFF_TiffBwCcittGroup3
- **1:** Color images. Default compression is IFF_TiffColorLZW
- **2:** Gray images. Default compression is IFF_TiffGrayLZW

Compression: A positive numeric value from the following values:

- IFF_BmpBwUncompressed = 1
- IFF_BmpGrayUncompressed = 2
- IFF_BmpColorUncompressed = 3
- IFF_DcxBwPackbits = 4
- IFF_DcxGrayPackbits = 5
- IFF_DcxColorPackbits = 6
- IFF_JpegGrayJfif = 7
- IFF_JpegColorJfif = 8
- IFF_PcxBwPackbits = 9
- IFF_PcxGrayPackbits = 10
- IFF_PcxColorPackbits = 11
- IFF_PngBwPng = 12
- IFF_PngGrayPng = 13
- IFF_PngColorPng = 14
- IFF_TiffBwUncompressed = 15
- IFF_TiffBwCcittGroup3 = 16
- IFF_TiffBwCcittGroup4 = 18
- IFF_TiffBwPackBits = 19
- IFF_TiffGrayUncompressed = 20
- IFF_TiffGrayPackBits = 21
- IFF_TiffGrayJpegJfif = 22
- IFF_TiffColorUncompressed = 23
- IFF_TiffColorPackBits = 24
- IFF_TiffColorJpegJfif = 25
- IFF_Jpeg2kGray = 28
- IFF_Jpeg2kColor = 29
- IFF_TiffBwLZW = 31
- IFF_TiffGrayLZW = 32
- IFF_TiffColorLZW = 33
- IFF_TiffBwZip = 34
- IFF_TiffGrayZip = 35
- IFF_TiffColorZip = 36
- IFF_JBIG2 = 43

Returns

Always **True**.

Level

Any level.

Details

Sets the default image compression to use for black and white, gray, and color images.

Example:

```
PDFImageCompression(0,16)
PDFImageCompression(1,33)
PDFImageCompression(2,32)
PDFConversionMode(1)
PDFDocumentToImage()
```

PDFImageFileExtension:

Sets the default file extension to use for black and white, gray, and color images.

Member of namespace

Convert

Syntax

```
bool PDFImageFileExtension(int imageType, string extension)
```

Parameters**imageType**

Type: int

The applicable image type.

extension

Type: int

The image extension.

Parameters

imageType: A positive numeric value of 0 to 2 for the type of conversion to use.

- **0:** BW images. Default compression is IFF_TiffBwCcittGroup3
- **1:** Color images. Default compression is IFF_TiffColorLZW
- **2:** Gray images. Default compression is IFF_TiffGrayLZW

Compression: The file extension to use for the image type.

Returns

Always **True**.

Level

Any level.

Details

Sets the default file extension to use for black and white, gray, and color images.

Example:

```
PDFImageCompression(1,33)
PDFImageFileExtension(1, ".jpeg")
PDFDocumentToImage()
```

PDFImageFileResolution:

Sets the resolution to be used when you are extracting images.

Member of namespace

Convert

Syntax

```
bool PDFImageFileResolution(int resolution)
```

Parameters

resolution

Type: int

The resolution to use when you are extracting images.

Parameters

resolution: A positive numeric value of 50 to 3200 for the type of conversion to use. Default is 300.

Returns

Always **True**.

Level

Any level.

Details

Sets the default resolution to use for extracted images.

Example:

```
PDFImageCompression(0,16)  
PDFImageCompression(1,33)  
PDFImageCompression(2,32)  
PDFConversionMode(1)  
PDFImageFileResolution(400)  
PDFDocumentToImage()
```

PDFImageUseFastBinarization:

Triggers the engine to use algorithms for faster binarization when you save images from color or grayscale to black and white.

Member of namespace

Convert

Syntax

```
bool PDFImageUseFastBinarization(bool useFastBinarization)
```

Parameters

useFastBinarization

Type: bool

Turn Fast Binarization On and OFF.

Parameters

useFastBinarization: **True** or **False**. Default is **False**. When set to **True**, the engine uses algorithms to run binarization faster, may result in lower quality images.

Returns

Always **True**.

Level

Any level.

Details

Triggers the engine to use algorithms for faster binarization when you save images from color or grayscale to black and white.

Example:

```
PDFImageCompression(0,16)
PDFImageCompression(1,33)
PDFImageCompression(2,32)
PDFImageUseFastBinarization(true)
PDFConversionMode(1)
PDFDocumentToImage()
```

Rtf actions

Use the Rtf actions to convert an image file from RTF format to TIFF for recognition processing by Datacap.

The Rtf actions can specify the output resolution and compression algorithm that is used when you convert from RTF to TIFF.

“RtfPrintQuality”

“RtfTiffCompression” on page 417

“RtfToImage” on page 417

RtfPrintQuality:

Adjusts the resolution of the image output by RtfToImage.

Member of namespace

Convert

Syntax

RtfPrintQuality ()

Parameters

dpi Type: int

A single positive numeric value for the dots per inch (dpi) of the output image.

Returns

False if the parameter is invalid. Otherwise, **True**.

Level

Page level.

Details

Sets the resolution of the output image for RtfToImage. If this action is not called, the default value of 200 dpi is used. Typically, input documents for recognition are 200 dpi.

Example:

```
RtfPrintQuality(200)
RtfToImage()
```

RtfTiffCompression:

Sets the compression used in the TIFF output by RtfToImage.

Member of namespace

Convert

Syntax

```
RtfTiffCompression ()
```

Parameters

tiffCompression

Type: string

A parameter of one of the following values to set the TIFF compression:

Parameters

- NONE : A color image with no compression.
- LZW : A color image using LZW compression.
- CCITT4 : A black and white image with fax CCITT4 compression.

Returns

Always **True**.

Level

Page level.

Details

Sets the compression of the output image from RtfToImage. If this action is not called, the default value of CCITT4 is used. Typically, input documents for recognition are black and white with fax compression.

Example:

```
RtfTiffCompression(CCITT4)
RtfToImage()
```

RtfToImage:

Converts a page with *.rtf file to a page or pages in TIFF format.

Member of namespace

Convert

Syntax

RtfToImage ()

Parameters

None.

Returns

True if the file is successfully converted to a TIFF document.

False if the current page is not a Rtf Document or if there is a failure in the conversion.

If the number of input files/pages exceeds the maximum allowed or if there is a failure in the conversion, the batch is set to abort.

Level

Page level.

Details

If the current page is a Rtf Document, the file will be converted to multiple TIFF files, one TIFF file for each page within the Document, based on the settings of the other Rtf actions that configure the conversion settings.

Each new TIFF also has a new page created within the application environment which can be processed by subsequent rules. The original file name from which the page was extracted is stored in the *ParentImage* variable, for possible future reference in your application.

If the configured output image format and compression only supports black and white, such as CCITT4, colored text is exported as black.

Example:

```
RtfPrintQuality(200)
RtfTiffCompression(CCITT4)
RtfToImage()
```

Tiff actions

Use the Tiff action to convert a multi-page TIFF image file into multiple single page TIFF image files for recognition processing by Datacap.

The Tiff actions can split a multi-page TIFF image file into individual pages and set the compression method that is used by the SplitMultipageTiff action in the TIFF output.

“SplitMultipageTiff” on page 419

“SplitTIFFCompression” on page 419

SplitMultipageTiff:

Creates separate images for each page in a multipage Tiff file.

Member of namespace

Convert

Syntax

`SplitMultipageTiff ()`

Parameters

None.

Returns

True if the file is successfully converted to a TIFF document.

False if the current page is not a TIFF or if there is a failure in the conversion.

If the number of input files/pages exceeds the maximum allowed or if there is a failure in the conversion, the batch is set to abort.

Level

Page level.

Details

If the current page is a TIFF, the file is converted to multiple TIFF files, one TIFF file for each page within the document. If the input page was a single page TIFF file, a single page TIFF is still be output.

Each new TIFF also has a new page created within the application environment which can be processed by subsequent rules. The original file name from which the page was extracted is stored in the *ParentImage* variable, for possible future reference in your application.

If the input TIFF has a bit depth of 1 (black and white), then the output compression is set to group 4 fax. If the input TIFF has a bit depth greater than 1, then the output compression is uncompressed.

Example:

```
SplitMultipageTiff()
```

SplitTIFFCompression:

Sets the compression method used in the TIFF output by SplitMultipageTiff.

Member of namespace

Convert

Syntax

`SplitTIFFCompression ()`

Parameters

compressionTypeColor

Type: int

compressionTypeBW

Type: int

Parameters

One of the following compression values are allowed.

- 0 : Source Image compression.
- 1 : No compression.
- 2 : CCITT modified Huffman RLE. (BW only)
- 3 : CCITT Group 3 fax. (BW only)
- 4 : CCITT Group 4 fax. (BW only)
- 5 : LZW Lempel-Ziv and Welch

Returns

True if the compression setting was successful.

False if the compression setting used is not a supported type.

Level

Page level.

Details

Sets the compression of the output image from SplitMultipageTiff. If this action is not called, the default value of CCITT4 is used for black and white pages and No compression for color pages.

Example:

```
SplitTIFFCompression(4,1)
plitMultipageTiff()
```

Txt actions

Use the Tiff actions to convert an image file from TXT format to TIFF for recognition processing by Datacap.

The Tiff actions can specify the output resolution and the compression algorithm that is used when you convert from Text to TIFF.

“TxtFontName”

“TxtFontSize” on page 421

“TxtPrintQuality” on page 422

“TxtTiffCompression” on page 422

“TxtToImage” on page 423

TxtFontName:

Adjusts the font name of the text in the image that is output by the TxtToImage action.

Member of namespace

Convert

Syntax

```
bool TxtFontName(string fontName)
```

Parameters

fontName

Type: string

The font name to use in the output image.

Returns

False if the parameter is invalid. Otherwise, **True**.

Level

Page level.

Details

Sets the font name for the text in the output image for the TxtToImage action. If this action is not called, the default value of Times New Roman is used.

Example:

```
TxtFontName(Courier New)  
TxtToImage()
```

TxtFontSize:

Adjusts the font size of the text in the image that is output by the TxtToImage action.

Member of namespace

Convert

Syntax

```
bool TxtFontSize(int fontSize)
```

Parameters

fontSize

Type: int

The size of the font to use in the output image.

Returns

False if the parameter is invalid. Otherwise, **True**.

Level

Page level.

Details

Sets the font size for the text in the output image for the TxtToImage action. If this action is not called, the default value of 10 is used.

Example:

```
TxtFontSize(12)
TxtToImage()
```

TxtPrintQuality:

Adjusts the resolution of the image output by TxtToImage.

Member of namespace

Convert

Syntax

```
TxtPrintQuality ()
```

Parameters

dpi Type: int

A single positive numeric value for the dots per inch (dpi) of the output image.

Returns

False if the parameter is invalid. Otherwise, **True**.

Level

Page level.

Details

Sets the resolution of the output image for TxtToImage. If this action is not called, the default value of 200 dpi is used. Typically, input documents for recognition are 200 dpi.

Example:

```
TxtPrintQuality(200)
TxtToImage()
```

TxtTiffCompression:

Sets the compression used in the TIFF output by TxtToImage.

Member of namespace

Convert

Syntax

```
TxtTiffCompression ()
```

Parameters

tiffCompression

Type: string

A parameter of one of the following values to set the TIFF compression:

Parameters

- NONE : A color image with no compression.
- LZW : A color image using LZW compression.
- CCITT4 : A black and white image with fax CCITT4 compression.

Returns

Always **True**.

Level

Page level.

Details

Sets the compression of the output image from TxtToImage. If this action is not called, the default value of CCITT4 is used. Typically, input documents for recognition are black and white with fax compression.

Example:

```
TxtTiffCompression(CCITT4)
TxtToImage()
```

TxtToImage:

Converts a page with *.txt file to a page or pages in TIFF format.

Member of namespace

Convert

Syntax

TxtToImage ()

Parameters

None.

Returns

True if the file is successfully converted to a TIFF document.

False if the current page is not a Txt Document or if there is a failure in the conversion.

If the number of input files/pages exceeds the maximum allowed or if there is a failure in the conversion, the batch is set to abort.

Level

Page level.

Details

If the current page is a Txt Document, the file are converted to multiple TIFF files, one TIFF file for each page within the Document, based on the settings of the other Txt actions that configure the conversion settings.

Each new TIFF also has a new page created within the application environment which can be processed by subsequent rules. The original file name from which the page was extracted is stored in the *ParentImage* variable, for possible future reference within your application.

If the configured output image format and compression only supports black and white, such as CCITT4, colored text is exported as black.

Example:

```
TxtPrintQuality(200)
TxtTiffCompression(CCITT4)
TxtToImage()
```

Word actions

Use the Word actions to convert an image file from Microsoft Word format to TIFF for recognition processing by Datacap.

The Word actions can specify the output resolution and the compression algorithm that is used when you convert from Word to TIFF.

“WordDocumentToImage”

“WordPrintQuality” on page 425

“WordTiffCompression” on page 426

WordDocumentToImage:

Converts a page with *.doc or *.docx file to a page or pages in TIFF format.

Member of namespace

Convert

Syntax

```
WordDocumentToImage ()
```

Parameters

None.

Returns

True if the file is successfully converted to a TIFF document.

False if the current page is not a Word Document or if there is a failure in the conversion.

If the number of input files/pages exceeds the maximum allowed or if there is a failure in the conversion, the batch is set to abort.

Level

Page level.

Details

If the current page is a Word Document, the file is converted to multiple TIFF files, one TIFF file for each page within the Document, based on the settings of the other Word actions that configure the conversion settings.

Each new TIFF also has a new page created within the application environment which can be processed by subsequent rules. The original file name from which the page was extracted is stored in the *ParentImage* variable, for possible future reference in your application. If the configured output image format and compression only supports black and white, such as CCITT4, colored text is exported as black.

Example:

```
WordPrintQuality(200)
WordTiffCompression("CCITT4")
WordDocumentToImage()
```

WordPrintQuality:

Adjusts the resolution of the image output by WordDocumentToImage.

Member of namespace

Convert

Syntax

WordPrintQuality ()

Parameters

dpi Type: int

Parameters

dpi : A single positive numeric value for the dots per inch (dpi) of the output image.

Returns

False if the parameter is invalid. Otherwise, **True**.

Level

Page level.

Details

Sets the resolution of the output image for WordDocumentToImage. If this action is not called, the default value of 200 dpi is used. Typically, input documents for recognition are 200 dpi.

Example:

```
WordPrintQuality(200)
WordDocumentToImage()
```

WordTiffCompression:

Sets the compression used in the TIFF output by WordDocumentToImage.

Member of namespace

Convert

Syntax

```
WordTiffCompression ()
```

Parameters

tiffCompression

Type: string

Parameters

tiffCompression is one of the following values to set the TIFF compression:

- NONE : A color image with no compression.
- LZW : A color image using LZW compression.
- CCITT4 : A black and white image with fax CCITT4 compression.

Returns

Always **True**.

Level

Page level.

Details

Sets the compression of the output image from WordDocumentToImage. If this action is not called, the default value of CCITT4 is used. Typically, input documents for recognition are black and white with fax compression.

Example:

```
WordTiffCompression(CCITT4)
WordDocumentToImage()
```

Zip actions

Use the Zip actions to extract the images files in a compressed ZIP archive and convert them to separate TIFF files for recognition by Datacap.

The Zip actions can overwrite the files that exist when you extract from ZIP archives and sets the Password value that is used to extract files from password protected archives.

“ZipOverwrite”

“ZipPassword”

“ZipUnPack” on page 428

ZipOverwrite:

Controls overwriting files when extracting from ZIP archives.

Member of namespace

Convert

Syntax

ZipOverwrite ()

Parameters

overwrt

Type: bool

Parameters

A Boolean value that indicates if a subsequent ZipUnPack action should overwrite existing files.

True: Overwrite files that already exist.

False: Do not overwrite files that already exist.

Returns

Always **True**.

Level

Page level.

Details

This action must be called before ZipUnPack. If this action is not called, then ZipUnPack uses the default value of **True**.

Example:

```
ChkDCOStatus("49")
ZipOverwrite("True")
ZipUnPack()
SetDCOStatus("75")
```

ZipPassword:

Sets the password for Password protected archives that will be used by ZipUnPack.

Member of namespace

Convert

Syntax

ZipPassword ()

Parameters

pwd Type: string

Parameters

pwd : The password for the archive. Smart parameters are supported.

Returns

Always **True**.

Level

Page level.

Details

The password provided is used to extract password protected archives. This action must be called before ZipUnPack.

Example:

```
ChkDCOStatus("49")
ZipOverwrite("True")
ZipPassword("MySafePassword")
ZipUnPack()
SetDCOStatus("75")
```

ZipUnPack:

Extracts each file within a compressed file archive into separate files.

Member of namespace

Convert

Syntax

ZipUnPack ()

Parameters

None.

Returns

True, if the contents of the compressed file is successfully extracted.

False, if the current page is not a PDF or if there is a failure in the extraction.

If the number of input files/pages exceeds the maximum that is allowed or if there is a failure in the extraction, the batch is set to abort.

Level

Page level.

Details

If the current page is compressed, the files that are contained in the archive are placed into the current batch directory. Each new file also has a new page that is created within the application environment that can be processed by subsequent rules. The original file name from which the page was extracted is stored in the *ParentImage* variable, for future reference within your application.

Example:

```
ChkDCOStatus("49")
ZipUnPack()
SetDCOStatus("75")
```

In this example, the action checks if the current page is the type, Other, and tries to extract the page. If the extraction is successful, then the current page was a valid compressed file. The files are extracted from the compressed file and put in the batch directory. The next action sets the page status of the compressed file to Deleted. The action does not delete it from the batch, but stops further processing of the compressed file.

Dcclip actions

Use the Dcclip action to clip a portion of each page image and save it as a separate TIFF file.

The Dcclip action uses the recognition zone coordinates of the current field and clips the specified region of each page to a separate TIFF file.

“dci_clipfield”

dci_clipfield

During processing, clips the field on the Image file (.tif) of every source page represented by the bound Field object of the Document Hierarchy and generates a separate Image file (.tif) displaying the clipped field's contents.

Syntax

(strParam)

Parameters

Two comma-separated string values:

1. The Page Type that is to be assigned to the Image file containing the clipped field and its value.

Remember: The new Image file is represented by a new page in the current Page file. The Page Type value you assign will be used to identify pages with clipped fields.

2. The Page Status to be assigned to pages with clipped images of the bound Field object of the Document Hierarchy.

Attention: Be sure that the status you assign conforms to Page Status specifications used throughout your application.

Returns

False if either parameter is invalid. Otherwise **True**.

If the `dci_clipfield` action cannot locate the target field on a source page, the action will not generate an Image file for the clipped field and will not add a corresponding page to the current Page file (.xml).

Level

Field level only.

Details

During processing, clips the field on the Image file (.tif) of every source page represented by the bound Field object of the Document Hierarchy and generates a separate Image file (.tif) displaying the clipped field's contents.

The action also adds a page representing the new Image file to the current Page file.

Attention: If the Image ID assigned to the Image file representing the source page has this format: `tm000001.tif`. The Image ID of a clipped field's Image file adds one underscore character and a two digit index and has this format: `tm000001_01.tif`

The second pair in the batch will have these ID's: `tm000002.tif` and `tm000002_01.tif`. This assumes that a source page has only one clipped field.

Example

```
dci_clipfield(OfficePens_Page, 0)
```

DCImageFix actions

Use the DCImageFix actions to clean up and enhance page images.

The image INI settings indicate which processes are run by the ImageEnhance action. You configure these INI settings on the **Zones** tab in Datacap Studio.

“ImageEnhance”

“LoadSettings” on page 431

“LoadSettings_FingerprintID” on page 432

ImageEnhance

Runs image processing by using pre-configured image enhancement settings, typically from the `imagefix.ini` file.

Member of namespace

DCImageFix

Syntax

`ImageEnhance(BackupFileExtension)`

Parameters

String: **BackupFileExtension**

Parameters

The file extension that the action is to assign to the backup of the original Image file. For example: tio.

The extension must be at least one character. If the leading period is provided, at least one character must follow it. Long name file extensions are allowed.

Returns

False If the parameter is not 3 or 4 alphanumeric characters, or if an exception is encountered while the image is being enhanced. Otherwise, **True**.

Level

Page or Field Level.

Details

Initiates image processing to run a pre-configured set of image enhancements.

Include this action after a LoadSettings or LoadSettings_FingerprintID action.

Example:

```
LoadSettings(C:\ParentDir\Invoice\Process\ImageFix.ini)
ImageEnhance("tio")
```

In this example, the ImageFix settings that are specified in ImageFix.ini are applied to every page in the batch. An example of the copied file name is "TM000001.tio".

```
LoadSettings(C:\ParentDir\Invoice\Process\ImageFix.ini)
ImageEnhance("tio.tif")
```

This example uses a longer file extension, which preserves the original file type. This extension makes it easier to view the original file without renaming. An example of the copied file name is "TM000001.tio.tif".

LoadSettings

Loads the image enhancement settings that are used by the ImageEnhance action to run image processing.

Member of namespace

DCImageFix

Syntax

LoadSettings ()

Parameters

String: BackupFileExtensionThe value of the path to the ImageFix Settings file (.ini).

Attention: The action can also use Smart Parameter syntax, such as the '@PATH(string)' method, to specify the path.

Returns

False if the ImageFix Settings file that you specify as a parameter is not found. Otherwise, **True**.

Level

All.

Details

This action loads the settings that the ImageFix action uses to process all images in the current batch. The action's parameter includes the file's name and complete path to its location in the application's Process directory. As an alternative, the parameter can use a smart parameter such as @Path to designate the value of the path to the same Process directory.

Example:

```
LoadSettings(C:\ParentDir\Invoice\Process\ImageFix.ini)
ImageEnhance(tio)
```

Example:

This example loads the settings file using the path denoted by the ScanFixSettings key that is listed in the Paths.ini file. If the key points to a relative path, it would be converted to the appropriate full path and then use that path to find the settings:

```
LoadSettings(@PATH(ScanFixSettings))
ImageEnhance(tio)
```

LoadSettings_FingerprintID

Loads the specific ImageFix Settings file (.ini) that corresponds to the Fingerprint ID of the current page.

Member of namespace

DCImageFix

Syntax

```
LoadSettings_FingerprintID ()
```

Parameters

String: FingerprintsFolderPathThe value of the path to the fingerprint folder.

Attention: The action can also use Smart Parameter syntax, such as the '@PATH(string)' method to specify the path.

Returns

False if a fingerprint-specific Settings file does not exist. Otherwise, **True**.

Level

Page level only.

Details

The action searches the fingerprint folder of the application for a fingerprint-specific ImageFix Settings file. The settings for these files are assigned during the Image Enhancement phase of Fingerprint Definition, using the tools in Rule Manager's Image Processing Setup dialog. (Chapter 3 of the Rule Manager Reference shows you how to define a fingerprint-specific ImageFix Settings file.)

Important: The name of a fingerprint-specific ImageFix Settings file is limited to the Fingerprint ID with the .ini extension: 1044.ini, for example.

Example:

```
LoadSettings_FingerprintID()  
ImageEnhance(tio)
```

DCO actions

Use the DCO actions to set up and modify the runtime batch hierarchy (runtime DCO) information.

The DCO actions can create documents, set up the status and type properties of an object, check the status of an object, and create a page data file for an object.

- "ChkConfidence" on page 434
- "ChkDCOStatus" on page 434
- "ChkDCOType" on page 435
- "ChkIntegrity" on page 435
- "ChkLastDCOType" on page 436
- "ClearAltText" on page 437
- "ClearDCO" on page 437
- "CopyPD2DD" on page 438
- "CountPagesToDocumentVar" on page 438
- "CreateDocuments" on page 439
- "CreateFields" on page 440
- "DeleteFields" on page 440
- "IsDocumentCountMoreThan" on page 441
- "IsFirstDocumentInBatch" on page 442
- "JoinPreviousDocument" on page 442
- "PropagateToAltText" on page 442
- "RemoveDocumentStructure" on page 443
- "SetDCOStatus" on page 444
- "SetDCOType" on page 444
- "SetDocStatus" on page 445
- "SetDocumentType" on page 445
- "SetFldConfidence" on page 446
- "SetPageFingerprintID" on page 447
- "SetPageStatus" on page 448
- "SetPageTemplateID" on page 449
- "SetPageType" on page 449

ChkConfidence

Checks the confidence of all field data on child pages against a minimum acceptable confidence value (Parameter 1). If any fields in a page contain Low Confidence data, assigns the Page Status that is specified in Parameter 2 to the page.

Syntax

(StrParam)

Parameters

Two or three comma-separated values:

1. The Numeric value of the minimum confidence required. This value is superseded on a field-by-field basis, if the field's *ReqConf* variable is set.
2. The Numeric Page Status code to assign to any page that has one or more fields with Low Confidence data: if a field's *ConfidenceString* property contains a value lower than the first parameter. Subfields, line items, and so on, are included. Typically, "1" (Problem) is the value of this parameter. If only two parameters are specified, only pages with Status=0 are checked by this action.
3. If a third parameter is supplied, these parameters specify the list of Page Statuses to be checked.

Returns

True, if all fields in all source pages are High Confidence. **False**, if any field has Low Confidence data, or if the parameters are not Numeric.

Level

All levels. This action operates on the entire batch regardless of the level to which its rule is bound.

Details

Checks the confidence of all field data on selected pages, which are selected by Page Status, against a minimum acceptable confidence value (Parameter 1). If any fields contain Low Confidence data, the page is marked with the status specified as a parameter.

Optionally, checks only pages of the status that is specified as Parameter 3.

Example:

```
ChkConfidence(8,1)
```

ChkDCOStatus

Confirms that the status of the Document Hierarchy's current object is identical to the status entered as the parameter.

Syntax

(StrParam)

Parameters

The Numeric value of the status that you are checking.

Returns

True, if the DCO status matches the parameter value. Otherwise, **False**.

Level

All levels.

Details

Confirms that the status of the Document Hierarchy's current object is identical to the status entered as the parameter.

Example:

```
ChkDCOStatus(0)  
returns True, if the current object has a status equal to 0, and False, if it does not.
```

```
ChkDCOStatus(48)  
returns True, if the current object has a status equal to 48, and False, if it does not.
```

ChkDCOType

Confirms that the Type property of the Document Hierarchy's current object is identical to the type entered as the parameter.

Syntax

(StrParam)

Parameters

The String value of the Type property of the object you're checking.

Returns

True if the value of the DCO Type matches the parameter. Otherwise, **False**.

Level

All levels.

Details

Confirms that the Type property of the Document Hierarchy's current object is identical to the type entered as the parameter.

Example:

```
ChkDCOType(Invoice)  
SetPageStatus(1)
```

Applied at the Page level, the action returns **True** if the current object is an Invoices Page object (using the Invoices application as an example), and **False** if it is not.

This action will confirm the current DCO Type matches an expected type and take additional subsequent actions that follow this action. In this case, if the current DCO Type is Invoice, the page status is set to 1.

ChkIntegrity

Checks that the integrity of the batch, as detailed in the Page file of the current task, meets the integrity requirements set within the Document Hierarchy (Setup DCO).

Syntax

()

Parameters

None.

Returns

Returns **True** if no integrity problems are found. Otherwise, **False**.

Level

Batch and Document levels.

Details

Checks that the integrity of the batch, as detailed in the Page file of the current task, meets the integrity requirements set within the Document Hierarchy (Setup DCO).

Integrity refers to the correct types and numbers of pages within each document in the batch and the correct order of the pages in each document.

Example:

```
ChkIntegrity()
```

ChkLastDCOType

Checks that the Type property of the Document Hierarchy's previous object is identical to the type entered as the parameter.

Syntax

(StrParam)

Parameters

The previous object's DCO Type to compare.

Returns:

True, if the Type property of the Document Hierarchy's previous object matches the parameter. Otherwise, **False**.

Level

All.

Details

Checks that the Type property of the Document Hierarchy's previous object is identical to the type entered as the parameter. You can use this action to test the last DCO Type that is encountered so you can take specific subsequent steps that are based on that type.

The example is applied at the Page level and checks to see whether the previous Page object's type matches the parameter (Separator).

Example:

```
ChkLastDCOType(Separator)
SetPageType(Invoice)
SetPageStatus(1)
```

Applied at the Page level, this sequence checks to see whether the previous Page object was a Separator page. If so, the type of the current page is set to Invoice, and its status is set to "1". If the previous document type was not a separator, the subsequent actions do not execute.

ClearAltText

Clears character and confidence values from the Character Array position specified by the parameter.

Syntax

(strParam)

Parameters

The index in the Character Array where you want to clear the character and confidence values. 0 is the first index, followed by 1, etc.

Returns

Always **True**.

Level

Field level.

Details

Clears character and confidence values from the Character Array position specified by the parameter. When cleared, the confidence values are set to 10 (high confidence). (A field in the Data file can hold more than one representation of the field's value. Values other than the current, visible value are accessed via an index number. The current value is at index 0, the next value is at index 1. Each additional value also has corresponding character confidences.)

Note: Most actions only work with characters and confidence values located in the first position of the Character Array (position 0). The Cleartext action is used with User Application Web's Advanced Index task, for "Double Blind" data entry.

Example:

```
PropagateToAltText(1)
ClearAltText(0)
```

In this example, all characters at the first index (0) of the Character Array will be copied to the second index (1). The second action will then clear character and confidence values from the first index in the Character Array.

ClearDCO

Clears all objects of the Document Hierarchy which are children of the bound object, and their variables.

Syntax

()

Parameters

None.

Returns

True if all child objects and their variables are removed, otherwise **False**.

Level

All.

Details

Removes all DCO children and variable references from the bound object.

Example:

```
CreateFields()  
ClearDCO()
```

Applied at the Page level, the example will first add fields to the page and then remove them.

CopyPD2DD

Assigns the value in a Page object's PD (Page Data) variable to the Document object's DD (Doc Data) variable.

Syntax

()

Parameters

None.

Returns

False if the action is not at the Document level, or if the PD variable at page level has no value. Otherwise, **True**.

Level

Document level.

Details

Assigns the value in a Page object's PD (Page Data) variable to the Document object's DD (Doc Data) variable.

Example:

```
CopyPD2DD()
```

CountPagesToDocumentVar

Counts the number of pages in the document.

Syntax

```
bool CountPagesToDocumentVar(StrParam)
```

Parameters

None

Returns

Always **True**.

Level

Document level.

Details

This action counts the number of page objects in a document and writes the result in a document variable.

Example:

```
CountPagesToDocumentVar("MyVarName")
```

CreateDocuments

Arranges the contents of a task's Page file (for example, Recognition.xml) into documents based on the Document Integrity rules (min, max and order) specified in your application's Document Hierarchy, and assembles documents from the pages in the batch.

Syntax

()

Parameters

None.

Returns

True if successful. Otherwise, **False**.

Level

Batch level only.

Details

Arranges the contents of a task's Page file (for example, Recognition.xml) into documents based on the Document Integrity rules for min, max, and order properties specified in your application's Document Hierarchy, and assembles documents from the pages in the batch.

Batches containing existing document structures will cause this action to return **False**, with no affect to the existing document structure.

Note: During document creation, temporary IDs are assigned (with a different format than real Document IDs), and if the action fails, these temporary IDs remain.

Important: This action is applied at the Batch level, and generally in its own Ruleset (in a CreateDocs ruleset, for example.)

Example:

```
CreateDocuments()
```

CreateFields

Creates the Data file for a page in a batch. The Data file for the first page in the batch, tm000001, as an example, is tm000001.xml.

Syntax

```
()
```

Parameters

None.

Returns

True if successful. Otherwise, **False**.

Level

Page level.

Details

Creates the Data file for a page in a batch. The Data file for the first page in the batch, tm000001, as an example, is tm000001.xml.

This Data file lists all fields for the current page based on the fields listed in the setup Document Hierarchy. Each field has an ID (for an Invoices page, for example, the Date field's ID is Date), and three properties with default values: TYPE, Position, and Status.

Later, actions of various kinds (Locate, Zone, Validation, DCO, etc.) can assign other values to these properties. These actions can also add properties (variables) and values to the Data file, or remove properties and values.

Example:

```
AnalyzeImage()  
RotateImageRecognizePageOCR_S()  
SetSearchArea(0.5)  
SetProblemValue(0.3)  
SetTemplateDir(\ParentDirectory\Invoice\Template)  
FindFingerprint(True)  
CreateFields()
```

This Invoices application sequence sets up the current page for processing, recognizes the words on the page, associates the page with a fingerprint, and finally creates a Data file with blank fields for the page.

DeleteFields

Deletes all child fields and characters from calling object of the Document Hierarchy.

Syntax

```
()
```

Parameters

None.

Returns

True if successful, otherwise **False**.

Level

All.

Details

Deletes all fields and characters that are children of the bound object of the Document Hierarchy. This action will also remove the Data file (.xml) from the batch if called from a Page object.

Example:

```
DeleteFields()
```

IsDocumentCountMoreThan

Compares the number of documents in a batch with a parameter that you specify to let you manage the document size of your batches.

Syntax

()

Parameters

returnTrueIfMore: determines whether the action returns **True** or **False** based on the results of the document count comparison.

Smart parameters are supported.

Returns

When set to **True**, returns **True** if the document count exceeds the value of the specified parameter. Otherwise, **False**.

When set to **False**, returns **False** if the document count exceeds the value of the specified parameter. Otherwise, **True**.

Level

All levels.

Details

This action compares the current batch document with the parameter that is provided and returns **True** or **False** depending on whether the count is exceeded by the provided parameter or not.

Example:

```
IsDocumentCountMoreThan(1, true)
```

This example returns **True** if there is more than 1 document in the batch


```
sDocumentCountMoreThan(1, false)
```

This example returns **False** if there is more than 1 document in the batch

IsFirstDocumentInBatch

Checks to see if this document is the first document in a batch.

Syntax

```
bool IsFirstDocumentInBatch()
```

Parameters

None

Returns

True if the action is on the first document, or an object in the first document. Otherwise **False**.

Level

Document level.

Details

This action checks to see if this action is on an object in the first document in a batch.

JoinPreviousDocument

Specifies the document type to join with the current document.

Syntax

```
bool JoinPreviousDocument(StrParam)
```

Parameters

None

Returns

Always **True**.

Level

Document, Page, or Field level.

Details

This action copies the pages of the previously named document into the front of the current document.

Example:

```
JoinPreviousDocument("SeparatorDoc")
```

PropagateToAltText

Copies the character and confidence values from the first index of the Character Array (index 0) to the index specified by the parameter.

Syntax

(strParam)

Parameters

The index of the Character Array where you want to copy the character and confidence values. 0 is the first index, followed by 1, etc.

Returns

Always **True**.

Level

Field level.

Details

Copies the character and confidence values from the first index of the Character Array (index 0) to the index specified by the parameter. (The character "node" of a page's Data file is an array that can hold many recognized character values, and their corresponding confidence values.)

Note: Rules will only work with characters and confidence values located in the first position of the Character Array (index 0).

The PropagateToAltText action is used with the User Application Web's Advanced Index task, for "Double Blind" data entry.

Example:

```
PropagateToAltText(1)
ClearAltText(0)
```

All characters in the first index of the Character Array will be copied to the second index. Then, the second action will clear the character and confidence values from the first index.

RemoveDocumentStructure

Removes the document hierarchy from the batch.

Syntax

bool RemoveDocumentStructure()

Parameters

None

Returns

Always **True**.

Level

Batch level.

Details

This action flattens the document and page hierarchy from the batch. If the batch consists of multiple documents, each with a set of pages, the document level is removed and all of the pages become a flat structure. Once complete, there is no distinction of sets of pages within a document.

Example:

```
RemoveDocumentStructure()
```

SetDCOStatus

Assigns a value to the Status property of the current object of the Document Hierarchy.

Syntax

(StrParam)

Parameters

An Integer representing the new status.

Returns

Always **True**.

Level

All.

Details

Assigns a value to the Status property of the current object of the Document Hierarchy.

Example:

```
ChkDCOType(Invoice)  
SetDCOStatus(1)
```

This sequence checks to see if the current object of the Document Hierarchy is a Page object - in this example, an Invoices page. If so, the value of the Page object's Status property is set to "1".

SetDCOType

Assigns a value to the Type property of the current object of the Document Hierarchy.

Syntax

(StrParam)

Parameters

A String value you're assigning to the current object's Type property.

Returns

Always **True**.

Level

All.

Details

Assigns a value to the Type property of the current object of the Document Hierarchy.

Example:

```
ChkLastDCOType(Separator)
SetDCOType(Invoice)
```

This sequence checks to see if the previous object of the Invoices application's Document Hierarchy was a Page object - in this case, a Separator page. If so, it sets Invoice as the Type property of the current object.

SetDocStatus

Assigns a status to the current document.

Syntax

(StrParam)

Parameters

String value representing the status to be assigned to the current document.

Typically:

- "0" = Complete.
- "1" = Incomplete.

Returns

False if the ruleset is not bound to a Document object, or the current object is not a document. Otherwise, **True**.

Level

Document level.

Details

Assigns a status to the current document.

Example:

```
SetDocStatus(DocOK)
```

SetDocumentType

The action assigns a Document Type to the current Document object of the Document Hierarchy.

Syntax

(StrParam)

Parameters

The value you want to assign as the Document object's Type property.

You can also designate a field in a Page object's Data file, and use its text value to set the Document Type. Simply enter the name of a valid Field object and surround it with single quotes. For example: 'Number'.

Returns

False if there are no Document objects in the Data file, or if the parameter is invalid. Otherwise, **True**.

Level

Document, Page, and Field levels.

Details

Similar to the SetDCOType action but works at the Document, Page or Field level.

The action assigns the Document Type you enter as a parameter to the current Document object of the Document Hierarchy. You can also use a Field object's value to set the Document Type. (Refer to the Parameter section.)

Example:

```
SetDocumentType('Number')\  
or  
SetDocumentType(Invoice_Document)
```

SetFldConfidence

For a specific field, sets the confidence for all characters in the field to the same value.

Syntax

(StrParam)

Parameters

A comma-separated value that consists of:

1. The field name, or a Smart Parameter that designates the field.
2. The confidence value (1-10) to be assigned to the field's characters.

Returns

Always **True**. If an input parameter is invalid, no confidence levels are changed and a message is logged.

Level

Field level.

Details

This action unconditionally sets the confidence values for every character within a field to a specific level. This action can help change confidence levels when the preceding actions are all successful.

For example, a successful set of calculations that involves one or more fields might indicate that the confidence level of those fields' values ought to be high, regardless of the confidence set by the recognition engine. After the success of the calculation, you can call `SetFldConfidence` to unconditionally reset the confidence values of the fields.

The first parameter is the name of the field that has its confidence level set. The second parameter is the wanted confidence level, between 1 and 10. If the second parameter is not passed in, 10 is used as the default.

Note: This function supports Smart parameters for the field name. See the following examples.

Example:

```
Example 1:  
To set all characters in the field "GrossSalary" to a confidence of 9.  
SetFldConfidence(@P\GrossSalary,10)  
  
Example 2:  
To set all characters in the field "AdjustedPay" to a confidence of 1.  
SetFldConfidence(@P\AdjustedPay,1)  
  
Example 3:  
In the context of a test run before setting the confidence.  
  
Calculate("'1TotalWages' + '2TaxableInterest' + '3Unemployment' = '4AdjustedGross'")  
  
If this calculation works, the application can assume that all of the characters are read correctly  
and SetFldConfidence can adjust the fields to high confidence.  
  
Note, you might also want to add a check that the values are all non-zero to eliminate a bad read.  
SetFldConfidence("@P\1totalWages,10")  
SetFldConfidence("@P\2TaxableInterest,10")  
SetFldConfidence("@P\3Unemployment,10")  
SetFldConfidence("@P\4AdjustedGross,10")
```

SetPageFingerprintID

Assigns a value to the `FingerprintID` property of the selected Page object of the Document Hierarchy.

Syntax

(StrParam)

Parameters

The String value of the Fingerprint ID.

Returns

True if the rule is applied at the Page level. Otherwise, **False**.

Level

Page level.

Details

Assigns a value to the `FingerprintID` property of the selected Page object of the Document Hierarchy.

Important: The SetPageFingerprintID action will create a FingerprintID property of the current Page object if it does not already exist.

Example:

```
WordFind(MQSW)  
SetPageFingerprintID(1010)
```

In this sequence, if the WordFind action locates "MQSW" on the current page, the SetPageFingerprintID action assigns "1010" as the page's Fingerprint ID. This links the page to a fingerprint with a Fingerprint ID of "1010".

SetPageStatus

The action assigns a page status to an object of the Document Hierarchy.

Syntax

(StrParam)

Parameters

Numeric value that represents the status.

The Invoices application (as an example) employs three default Page Statuses:

- 49 = ScanOK
- 1 = Incomplete/Not validated
- 0 = Complete

You can define your own statuses by using the **Filter** tab of a task's **Task Settings** dialog.

Returns

Always **True**.

Level

Page or field level.

Details

The action assigns the page status that you enter to the page object of the Document Hierarchy. The current object can be the page or field. If the current object is a field object, it sets the page status for its parent page object.

Example:

A scan task might assign Other as the Page Type and 49 as the default Page Status for every successfully scanned image in the batch.

The following sequence is an example of a rule that converts Other pages to Invoices pages, and assigns a Page Status to each:

```
SetPageType(Invoice)  
SetPageStatus(1)
```

This combination establishes the page as an Invoices page, and gives it a status of 1. This means that the page is not validated and must be processed by a task that applies Validation rules (a Recognition or Verification task, for example).

SetPageTemplateID

Assigns a value to the FingerprintID property of the selected Page object of the Document Hierarchy.

Syntax

(StrParam)

Parameters

The String value of the Fingerprint ID.

Returns

True if the rule is applied at the Page level. Otherwise, **False**.

Level

Page level.

Details

The SetPageTemplateID action creates a FingerprintID property of the current Page object if one does not already exist.

Example:

```
WordFind(MQSW)  
SetPageTemplateID(1010)
```

In this sequence, if the WordFind action locates "MQSW" on the current page, the SetPageTemplateID action assigns "1010" as the page's Fingerprint ID. This links the page to a fingerprint with a Fingerprint ID of "1010".

SetPageType

The action assigns a Page Type to the current Page object of the Document Hierarchy.

Syntax

(StrParam)

Parameters

A String value that represents the Page Type.

You can also designate a field in a Page object's Data file, and use its text value to set the Page Type. Enter the name of a valid Field object and surround it with single quotation marks. For example: 'PageCode'.

Returns

False, if there are no Page objects in the Page file, or if the parameter is invalid. Otherwise, **True**.

Level

Page and Field levels.

Details

Similar to the SetDCOType action, but works at the Page or Field level.

The action assigns the Page Type you enter as a parameter to the current Page object of the Document Hierarchy. You can also use a Field object's value to set the Page Type. (See the Parameter section.)

Example:

The application's scan task typically assigns Other as the Page Type and 49 as the default Page Status for every successfully scanned image in the batch.

The following sequence is an example of a rule that converts Other pages to Invoices pages, and assigns a Page Status to each:

```
SetPageType(Invoice)
SetPageStatus(1)
```

This combination sets the page as an Invoices page, and gives it a status of 1. This means that the page is not validated and must be processed by a task that applies Validation rules (a Rulerunner Task, for example).

dcpdf actions

Use the dcpdf actions to convert PDF files to TIFF at the start of the workflow. You can also convert the TIFF files in a document into a PDF file.

The dcpdf actions can specify properties for PDF documents, set bits per pixel counts for images in PDF documents, and specify the compression method to use to convert PDF documents to TIFF.

- "dcpdf_CreateTiffFromPDF"
- "dcpdf_CreateTiffFromPDF_CreateDocs" on page 451
- "dcpdf_MakePDFDoc" on page 452
- "dcpdf_MaxSizeToReconvert" on page 454
- "dcpdf_SetApplication" on page 455
- "dcpdf_SetAuthor" on page 455
- "dcpdf_SetImageBitcount" on page 456
- "dcpdf_SetImageCompression" on page 457
- "dcpdf_SetImageGrayscale" on page 458
- "dcpdf_SetImageQuality" on page 458
- "dcpdf_SetImageResolution" on page 459
- "dcpdf_SetKeywords" on page 460
- "dcpdf_SetProducer" on page 460
- "dcpdf_SetSubject" on page 461
- "dcpdf_SetTitle" on page 462
- "dcpdf_UseAltConversionMethod" on page 462

dcpdf_CreateTiffFromPDF

Converts each page of a PDF file into a TIFF file.

Member of namespace

dcpdf

Syntax

dcpdf_CreateTiffFromPDF ()

Parameters

None.

Returns

False, if the action is not run at the Batch level; if there are no PDF files in the batch; or if an error occurs while TIFF files are being created. Otherwise, **True**.

Level

Batch only.

Details

This action looks for PDF files in the current batch; creates Image files (.tiff) for each page in the PDF file; and creates one document for the PDF file.

Attention: dcpdf_MaxSizeToReconvert can be used to control how this action creates a TIFF file. PDFs are converted to TIFF using a fast algorithm. With some occasional input documents, this may produce images that are large and may not recognize well, causing them to be automatically reconverted to TIFF using a cleaner but slightly slower algorithm. See the help for dcpdf_MaxSizeToReconvert for more information.

Example:

```
dcpdf_CreateTiffFromPDF()
```

When the action in this example encounters a PDF file in the batch, it creates a corresponding Image file (.tiff) and gives the Image file the PDF file's name.

dcpdf_CreateTiffFromPDF_CreateDocs

Converts each page of a PDF file into a TIFF file and creates a runtime hierarchy.

Member of namespace

dcpdf

Syntax

dcpdf_CreateTiffFromPDF_CreateDocs (strParam)

Parameters

The parameter can be True or False.

True: Causes a document hierarchy to be created when TIFF pages are created from a PDF file.

False: A document hierarchy will not be created, making this action operate exactly like `dcpdf_CreateTiffFromPDF`.

Returns

False, if the action is not run at the Batch level, or if the source PDF file does not have a minimum number of pages (0). Otherwise, **True**.

Level

Batch Level.

Details

This action converts the pages in a PDF file to TIFFs, like `dcpdf_CreateTiffFromPDF`. The difference is that if the parameter is **True**, a document hierarchy will be created for each PDF to group each of the pages from a single PDF into their own document.

Example:

```
dcpdf_CreateTiffFromPDF_CreateDocs("True")
```

In this example, the action uses a primary algorithm to establish a runtime Document for each source document in the PDF file, and assign pages to the runtime Documents according to their placement in the source PDF files.

dcpdf_MakePDFDoc

Creates a PDF document that contains one or more pages of the current document.

Member of namespace

dcpdf

Syntax

```
bool dcpdf_MakePDFDoc(strParam)
```

Parameters

Comma-separated parameters: **IncludeFieldText**, **PageTypes**.

The **IncludeFieldText** parameter can be **True** or **False**.

True: stores recognized field data that is associated with the TIFF image of each page in the PDF file to make a searchable PDF. The text data is not visible, but it is searchable.

False: inserts the image of a page into the PDF file, the recognized text is not inserted.

The **PageTypes** parameter is a list of one or more page types to include in the PDF output. The page types in the list are separated by commas.

If no parameters are provided, **IncludeFieldText** defaults to **False** and all document pages are included in the PDF.

If only the **IncludeFieldText** parameter is provided, all document pages are included in the PDF.

Smart parameters are supported.

Returns

False if the action is not run at the Document level; if there are no pages in the document; or if an error occurs while the PDF file is being created. Otherwise, **True**. If there are pages in the document but none of them match the specified page type, the action does not create a PDF file and still returns **True**.

Level

Document Level.

Details

Creates a PDF document that contains one or more pages of a document. The PDF format can contain only the original image or it can contain the image along with the recognized text to make the PDF searchable.

Only zoned field text is included if the input parameter is **True** and is searchable within the PDF. The searchable text might not accurately reflect the text position within the original input image. If you need the entire page text to be included as searchable text, or more accurate alignment, use one of the recognition actions to create the PDF, such as RecognizeDocToPDF from the OCR_S library.

To exclude specific page types, set the variable *typesToExclude* to a comma delimited list of page types to exclude from the PDF.

To include specific page types, set the variable *typesToInclude* to a comma delimited list of page types to include in the PDF.

To exclude specific page status, set the variable *statusToExclude* to a comma delimited list of page status to exclude from the PDF.

When more than one filter is specified, the following order of precedence takes place:

- *statusToExclude* overrides *typesToInclude*
- *typesToInclude* overrides *typesToExclude*

If you are calling the action at the Document level, the types and status filters apply to both the documents and their child pages.

If you are calling the action at the Page level, the types and status filters apply to the page only.

These variables must be set before you call the RecognizeToPDFOCR_A action.

Example:

```
rrSet("75", "@D.statusToExclude)
rrSet("Blank", "@D.typesToExclude)
dcpdf_SetTitle("MedicalClaims")
dcpdf_SetSubject("Validated")
```

```
dcpdf_SetAuthor("Steven Moffat")
dcpdf_SetProducer("Russell Davies")
dcpdf_SetApplication("MClaims")
dcpdf_MakePDFDoc("True")
```

This example sets several properties of the PDF file and creates a PDF that contains the recognized text along with all of the page images for the current document. Pages whose type is "Blank" and status is "75" are skipped.

```
dcfpdf_MakePDFDoc("False,Main_Page")
```

This example does not include the field text and exports only the page types of type Main_Page.

```
dcfpdf_MakePDFDoc("True,Main_Page,Trailing_Page")
```

This example includes the field text and exports only the page types of type Main_Page or Trailing_Page.

dcpdf_MaxSizeToReconvert

Causes the dcpdf_CreateTiffFromPDF action to use a different conversion algorithm if the file that results from the default algorithm exceeds the specified size.

Member of namespace

dcpdf

Syntax

```
dcpdf_MaxSizeToReconvert (strParam)
```

Parameters

A Numeric value representing the maximum image size in KB before attempting to convert with an alternate algorithm. If the value is 0, then the alternate algorithm will never be used.

Returns

False if the parameter is not Numeric. Otherwise, **True**.

Level

Batch or Document level.

Details

Causes dcpdf_CreateTiffFromPDF to use a different algorithm based on the configured file size.

The conversion of PDF to TIFF usually works fast and produces clean images. From time to time, the resulting TIFF may become unusually large. A reason for this has been due to the background becoming a dithered light gray background instead of a pure white. This leads to very large image sizes because compression is now less efficient. It also could lead to reduced recognition quality.

To compensate for these rare situations, this action can be used to produce a clean TIFF from a PDF that compresses well. When a TIFF is created from a PDF, the file size is checked to see if it exceeds the value set by this action. If the file size is smaller, then is considered a successful conversion. If the file size is larger, then this alternate method is performed.

This action must be called prior to `dcpdf_CreateTiffFromPDF`. If this action is not called, the default value of 2000KB is used. Calling `dcpdf_SetImageGrayscale(TRUE)` will cause `dcpdf_CreateTiffFromPDF(True)` to always use the alternate algorithm.

Example:

```
dcpdf_SetMaxImageSize("10000")  
dcpdf_CreateTiffFromPDF("True")
```

This example will cause the alternate conversion algorithm to be used if the initial conversion produces a TIFF image that is larger than 10000KB.

dcpdf_SetApplication

Specifies the Application ID property for PDF documents that are generated by the `dcpdf_MakePDFDoc` action.

Member of namespace

`dcpdf`

Syntax

```
dcpdf_SetApplication (strParam)
```

Parameters

String value of the Application ID.

Returns

Always **True**.

Level

Batch or Document level.

Details

This sets the Application ID property of a PDF document generated by a subsequent `dcpdf_MakePDFDoc` action.

If this action is not called, the value will default to "Application".

Example:

```
dcpdf_SetApplication("Invoices")  
dcpdf_SetAuthor("Harriet Jones")  
dcpdf_MakePDFDoc("True")
```

dcpdf_SetAuthor

Specifies the Author property for PDF documents that are generated by the `dcpdf_MakePDFDoc` action.

Member of namespace

dcpdf

Syntax

dcpdf_SetAuthor (strParam)

Parameters

String value of the Author.

If you do not call this action, the value will default to Should be a Task Name.

Returns

Always **True**.

Level

Batch or Document level.

Details

This action attaches the Author's name (or a related value) to a PDF page or document generated by a subsequent dcpdf_MakePDFDoc action.

Example:

```
dcpdf_SetAuthor("Harriet Jones")
dcpdf_SetProducer("Russell Davies")
dcpdf_SetApplication("Invoices")
dcpdf_MakePDFDoc("True")
```

dcpdf_SetImageBitcount

Sets the bit count, bits per pixel, for images in the PDF document that is generated by the dcpdf_MakePDFDoc action.

Member of namespace

dcpdf

Syntax

dcpdf_SetImageBitcount (strParam)

Parameters

Numeric value of the BitCount of any image in a PDF document. The action's acceptable values are:

- 1 for Black and White images (a typical value).
- 8 for Grayscale images.
- 24 for Color images.

Returns

False, if the parameter is not Numeric. Otherwise, **True**.

Level

Batch or Document level.

Details

Sets the BitCount of any image identified in a PDF file by an action such as CreateTiffFromPDF. The value of the parameter for the action typically reflects the nature of the images in the PDF file. The default value is 1 (Black and white images).

Attention: The BitCount is the standard number of bits per pixel throughout an image.

Example:

<code>dcpdf_SetImageBitCount("1")</code>	black and white images.
<code>dcpdf_SetImageBitCount("8")</code>	grayscale images.
<code>dcpdf_SetImageBitCount("24")</code>	color images.

dcpdf_SetImageCompression

Specifies the compression method to use when you convert a PDF file to TIFF.

Member of namespace

dcpdf

Syntax

`dcpdf_SetImageCompression (strParam)`

Parameters

A numeric value representing one the following Compressions:

- 1 = NONE (DUMP MODE)
- 2 = CCITTRLE (CCITT modified Huffman RLE)
- 3 = CCITTFAX3 (CCITT Group 3 fax encoding)
- 4 = CCITTFAX4 (CCITT Group 4 fax encoding)
- 5 = COMPRESSION_LZW (Lempel-Ziv and Welch)
- 7 = JPEG (%JPEG DCT compression)
- 32773 = PACKBITS (Macintosh RLE)

Returns

False, if the parameter is not *Numeric*. Otherwise, **True**.

Level

Batch or Document level.

Details

Sets the compression that will be used when converting a page from a PDF file to a TIFF file. Group 4 Fax is the most common compression used for text recognition. It produces a lossless compressed black and white image. The default compression value is 4 (CCITT Group 4 fax encoding).

Example:

```
dcpdf_SetImageCompression("7")
dcpdf_CreateTiffFromPDF("True")
```

dcpdf_SetImageGrayscale

Specifies how gray areas of a grayscale image are handled when you convert a PDF file to TIFF.

Member of namespace

dcpdf

Syntax

```
dcpdf_SetImageGrayscale (strParam)
```

Parameters

The parameter can be **True** or **False**.

- **True**: Gray areas of the image must not be dithered.
- **False**: Gray areas of the image must be dithered. This option is the default.

Returns

False, if the parameter is invalid. Otherwise, **True**.

Level

Batch or Document.

Details

Sets a Grayscale flag for images in a PDF file that is generated by an action such as `dcpdf_CreateTiffFromPDF`.

This action controls how gray areas of a grayscale image are handled when converted to black and white. For example, if you have a text document with a gray background, it is recommended to call this action and pass **True**. This option causes the gray area below a predefined tolerance to be converted to white, producing an image that can be recognized. If you have a grayscale image and this action is set to **False**, gray areas are dithered to simulate gray in the resulting TIFF image. The default value is **False**.

If this action is used, it must be called before `dcpdf_CreateTiffFromPDF` or `dcpdf_CreateTiffFromPDF_CreateDocs`.

Example:

```
dcpdf_SetImageGrayscale("True")
dcpdf_CreateTiffFromPDF("True")
```

dcpdf_SetImageQuality

Specifies the image quality to use when you convert a PDF file to TIFF.

Member of namespace

dcpdf

Syntax

`dcpdf_SetImageQuality (strParam)`

Parameters

Numeric value, between 1 and 100, of the Image Quality standard for images in the PDF file.

Returns

False, if the parameter is not Numeric. Otherwise, **True**.

Level

Batch or Document level.

Details

This action determines the resulting image quality when creating a TIFF from a PDF file. A higher number will produce a better looking image, but may require more processing time. The default value is 100.

Example:

```
dcpdf_SetImageQuality("75")
dcpdf_CreateTiffFromPDF("True")
```

dcpdf_SetImageResolution

Specifies the output resolution to use when you convert a PDF file to TIFF.

Member of namespace

dcpdf

Syntax

`dcpdf_SetImageResolution (strParam)`

Parameters

Two comma-separated Numeric values specifying X resolution and Y resolution (in that order). The values are expressed in Dots (pixels) Per Inch (DPI).

Returns

False, if the parameters are not Numeric. Otherwise, **True**.

Level

Batch or Document level.

Details

This action sets the X and Y resolution for the pages of a PDF that is converted to TIFF. If this action is not called, the default of 200 x 200 will be used. It is strongly recommended that the X and Y resolutions are always set the same to produce an isotropic image, which will allow for better fingerprinting and recognition.

This action must be called before the `dcpdf_CreateTiffFromPDF` or `dcpdf_CreateTiffFromPDF_CreateDocs` actions.

Example:

```
dcpdf_SetImageResolution("300,300")  
dcpdf_CreateTiffFromPDF()
```

dcpdf_SetKeywords

Specifies a keyword to assign to PDF documents that are generated by the `dcpdf_MakePDFDoc` action.

Member of namespace

`dcpdf`

Syntax

```
dcpdf_SetKeywords (strParam)
```

Parameters

String value of the Keyword.

If you do not call this action, the keyword value is left blank.

Returns

Always **True**.

Level

Batch or Document level.

Details

This action assigns a single Keyword to a PDF page or document that is generated by a subsequent `dcpdf_MakePDFDoc` action.

Use this action repeatedly within a rule to assign more Keywords.

Important: This action must precede the `dcpdf_MakePDFDoc` action.

Example:

```
dcpdf_SetKeywords("Invoices")  
dcpdf_MakePDFDoc("True")
```

dcpdf_SetProducer

Specifies the Producer property for PDF documents that are generated by the `dcpdf_MakePDFDoc` action.

Member of namespace

`dcpdf`

Syntax

```
dcpdf_SetProducer (strParam)
```

Parameters

String value of the Producer ID.

Returns

Always **True**.

Level

Batch or Document level.

Details

Sets the producer value of the PDF document. This action must be called prior to calling `dcpdf_MakePDFDoc`. If this action is not called, the value will default to Producer.

Example:

```
dcpdf_SetAuthor("Steven Moffat")
dcpdf_SetProducer("Russell Davies")
dcpdf_SetApplication("Invoices")
dcpdf_MakePDFDoc("True")
```

dcpdf_SetSubject

Specifies the Subject property for PDF documents that are generated by the `dcpdf_MakePDFDoc` action.

Member of namespace

dcpdf

Syntax

```
dcpdf_SetSubject (strParam)
```

Parameters

String value of the Subject.

Returns

Always **True**.

Details

This action sets the Subject property of PDF generated by a subsequent `dcpdf_MakePDFDoc` action.

The Subject is a searchable value for the page or document. If this action is not called, the value of the subject is blank in the resulting document.

Example:

```
dcpdf_SetSubject("HealthClaimDoc")
cpdf_SetAuthor("Harriet Jones")
dcpdf_MakePDFDoc("True")
```

dcpdf_SetTitle

Specifies the Title property for PDF documents that are generated by the dcpdf_MakePDFDoc action.

Member of namespace

dcpdf

Syntax

dcpdf_SetTitle (strParam)

Parameters

String value of the Title.

Returns

Always **True**.

Level

Batch or Document level.

Details

This action sets the Title property of a PDF document generated by a subsequent dcpdf_MakePDFDoc action.

The Title is a searchable value for the page or document. If this action is not called prior to dcpdf_MakePDFDoc, the value will default to Untitled.

Example:

```
dcpdf_SetTitle("NewInvoice")  
dcpdf_MakePDFDoc("True")
```

dcpdf_UseAltConversionMethod

Causes dcpdf_CreateTiffFromPDF to use an alternate conversion algorithm.

Member of namespace

dcpdf

Syntax

dcpdf_UseAltConversionMethod ()

Parameters

None.

Returns

Always **True**.

Level

Batch or Document level.

Details

There are two internal algorithms that are used to convert a PDF to a TIFF. This action enables the alternate algorithm. It is recommended that the alternate algorithm is used. Testing has shown that it produces cleaner TIFF images.

This action must be called prior to the `dcpdf_CreateTiffFromPDF` or `dcpdf_CreateTiffFromPDF_CreateDocs` actions.

Example:

```
dcpdf_UseAltConversionMethod()  
dcpdf_CreateTiffFromPDF()
```

Email actions

Use the email actions to compose and then send an email by using CDOSYS and an SMTP server. These actions also support Outlook, which requires the Outlook user to be logged on to the computer and security prompts might be displayed for each message.

The Email actions specify path and file name for attachments, email recipients, and email subject.

“SendEmail”

“SetAttachment” on page 464

“SetBlindCarbonCopyRcpts” on page 464

“SetCarbonCopyRcpts” on page 465

“SetEmailBody” on page 465

“SetMailServer” on page 466

“SetRecipients” on page 467

“SetSender” on page 467

“SetSubject” on page 468

SendEmail

Sends an email.

Syntax

()

Parameters

None.

Returns

False, if the rule does not include a previous `SetRecipients` action, or if the email cannot be sent. Otherwise, **True**. If the email cannot be sent, the batch is set to abort.

Level

All levels.

Details

Sends an email that is assembled by previous actions. Typically, this is the final action in an email rule set. At a minimum, the `SetSender` and `SetRecipients` actions must be called before sending an email.

After the email is sent, this action will discard the contents of the email in memory. Calls to the email actions after `SendEmail` causes the creation of a new email message.

Example:

```
SetSender("paul@adomain.com")
SetRecipients("lisa@adomain.com,beth@adomain.com")
SetSubject("Document Integrity")
SetEmailBody("Document Page Types and counts are accurate. Thanks for your help.")
SendEmail()
```

SetAttachment

Adds a file attachment to an email.

Syntax

(StrParamMW)

Parameters

The file's path, name and extension. Smart Parameters are supported.

Returns

False if the file does not exist or cannot be attached. Otherwise, **True**.

Level

All levels.

Details

Attaches the specified file to the current email.

Example:

```
SetAttachment("h:\MyDir\MQSW\export\+@BATCHID+.txt")
```

This example attaches the Export file of the current batch to the email.

SetBlindCarbonCopyRcpts

Sets Blind Carbon Copy recipients' email address(es).

Syntax

(StrParam)

Parameters

The email addresses to receive a copy of the email as a blind carbon copy. You can enter multiple email addresses separated by commas.

Returns

False if you do not enter an email addresses parameter, if the address is rejected by the mail system or if the email object cannot be initialized. Otherwise, **True**.

Invalid email addresses may not be reported until SendEmail is called.

Level

All levels.

Details

Adds addresses to the Bcc (Blind Carbon Copy) portion of the email's header.

Example:

```
SetRecipients("lisa@monarchy.com")  
SetBlindCarbonCopyRcpts("james@regency.com")
```

SetCarbonCopyRcpts

Set Carbon Copy recipients' email address(es).

Syntax

(StrParam)

Parameters

Email addresses to receive a copy of the email as a carbon copy. You can enter multiple email addresses separated by commas.

Returns

False if you do not enter an email addresses parameter, if the address is rejected by the mail system or if the email object cannot be initialized. Otherwise, **True**.

Invalid email addresses may not be reported until SendEmail is called.

Level

All levels.

Details

Adds addresses to the Cc (Carbon Copy) portion of the email's header.

Example:

```
SetRecipients("lisa@adomain.com")  
SetCarbonCopyRcpts("cindy@anotherdomain.org")
```

SetEmailBody

Sets the text of the email's body.

Syntax

(StrParamMW)

Parameters

The email message text. Smart Parameters are supported.

Returns

False if the mail object cannot be initialized. Otherwise, **True**.

Level

All levels.

Details

Sets the text of the email's body.

Example:

```
SetSubject("Document Integrity")  
SetEMailBody("Document Page Types and counts are accurate. Thanks for your help.")
```

SetMailServer

Configures the mail server to use for sending mail. Use this action only if you are sending emails with CDOSYS.

Syntax

(StrParam)

Parameters

The IP or DNS address of the outgoing mail (SMTP) server.

Returns

Always **True**.

Level

All levels.

Details

Sets the address of the outgoing mail (SMTP) server. This server might be the same mail server that you configure in your mail program. The server must be accessible from the computer that runs the email actions. This action must be the first action in an email rule, if the CDOSYS object is being used.

Use this action only if you are sending emails with CDOSYS. To use CDOSYS, this action must be called before any of the other email actions. If this action is not called before other email actions, these actions use Outlook for sending emails.

You can use Email actions to direct a task to compose and send emails that contain information and attachments. Email actions use the Windows CDOSYS library to send email by your preferred SMTP mail server. The CDOSYS object is included with Windows 2000 and higher versions. Alternatively, Email actions can use the Outlook object but it is not recommended.

One of these two libraries (CDOSYS or Outlook) must be registered on the computer that runs the rules that employ email actions.

Outlook is primarily useful for demonstration purposes because it is not suitable for unattended operation. It requires the Outlook user to be logged in to the computer, and security prompts might be displayed for each message sent.

Example:

```
SetMailServer("mail.YourISP.com")
SetSender("paul@adomain.com")
SetRecipients("lisa@adomain.com")
SetSubject("Document Integrity")
SetEMailBody("Document Page Types and counts are accurate. Thanks for your help.")
SendEMail()
```

SetRecipients

Sets email recipients' address(es).

Syntax

(StrParam)

Parameters

Email address(es) of recipient(s). You can either call this action multiple times to add multiple recipients, or you can enter multiple email addresses separated by commas.

Returns

False if you do not enter an email addresses parameter, if the address is rejected by the mail system or if the email object cannot be initialized. Otherwise, **True**.

Invalid email addresses may not be reported until SendEMail is called.

Level

All levels.

Details

The email address of the email's primary recipients.

Example:

```
SetRecipients("lisa@adomain.com,Joe@adomain.com")
```

SetSender

Sets the sending email address.

Syntax

(StrParam)

Parameters

The sender's email address.

Returns

False if the mail object cannot be initialized. Otherwise, **False**.

Invalid email addresses may not be reported until SendEMail is called.

Level

All levels.

Details

Sets the email address of the sender for the current email. When using the CDO SYS object, use this action. When using the Outlook object, the current email account is used as the sender.

Example:

```
SetRecipients("lisa@adomain.com")  
SetSender("paul@adomain.com")
```

SetSubject

Sets the text for the email's Subject field.

Syntax

(StrParamMW)

Parameters

The subject line of the email. Smart Parameters are supported.

Returns

False if the mail object cannot be initialized. Otherwise, **True**.

Level

All levels.

Details

Sets the text for the email Subject field. It is recommended that the subject line be no longer than 78 characters as this is a common subject line length limitation. Some systems may support even shorter lengths, truncating the subject. Our testing has been successful with lengths up to 255 characters. It is recommended to test your settings and use lengths appropriate for your systems.

Example:

```
SetSubject("Document Integrity")
```

Equalize actions

Use the Equalize action to equalize the x and y resolutions of an image.

The Equalize actions convert an image with different x and y resolutions to one with the same x and y resolutions.

“EqualizeUnbalancedImage”

EqualizeUnbalancedImage

Resolves differences in the dpi (dots per inch) resolutions along the horizontal (X) and vertical (Y) planes of one or more faxed images in a batch.

Syntax

(StrParam)

Parameters

Integer value which determines the cut-off point for the resolution which should be equalized:

`EqualizeUnbalancedImage(0)`, for example, specifies that there is no cut-off point: all images will be subject to equalization.

`EqualizeUnbalancedImage(20)` establishes a cut-off point of 200(x)/180(y). This means that the action will equalize all images with this resolution ratio and more (200/180, 200/160 etc.) - but will ignore all images with balance ratios less than 200/180 (in this example.)

Attention: Standard Mode fax resolution in Dots per Inch (DPI) is 204/98; Fine Mode fax resolution is 204/196.

Returns

False if the parameter is not numeric or if the rule containing the action is not bound to a Page object of the Document Hierarchy. Otherwise, **True**.

Level

Page level only.

Details

Resolves differences in the dpi (dots per inch) resolutions along the horizontal (X) and vertical (Y) planes of one or more faxed images in a batch.

The action selects images from the batch in response to the parameter you enter, and produces new images with 200 x 200 dpi.

Example

```
EqualizeUnbalanceImage(0)
```

Ewsmail actions

Use the Ewsmail actions to import image file attachments from an Exchange Server into the current batch by using Exchange Web Service (EWS).

The `ex_scan` action polls the Exchange Server and imports image attachments until the batch reaches the specified size (`ex_max_docs`) or until the wait time (`ex_wait_time`) expires.

“`ex_abort_time`” on page 470

“`ex_done_folder`” on page 470

“`ex_EMLOption`” on page 471

“`ex_ews_version`” on page 472

“`ex_HTTP_timeout`” on page 473

“`ex_load_properties_option`” on page 473

“`ex_login`” on page 474

“`ex_logout`” on page 475

“`ex_max_docs`” on page 476

“`ex_problem_folder`” on page 477

"ex_scan" on page 477

"ex_types" on page 479

"ex_wait_time" on page 480

ex_abort_time

Specifies how long to wait before you stop running the current batch if, for example, the Exchange Server is unavailable.

Member of namespace

Ewsmail

Syntax

`ex_abort_time ()`

Parameters

`nSecs` Type: int

Parameters

`nSecs` : The number of seconds to wait.

Returns

Always **True**.

Level

Batch level.

Details

The action will wait the specified time before returning when an abort occurs. This action can be useful to prevent a large number of aborted batches due to an abort condition. For example, if the email server should become unavailable for a time, the abort timeout will limit the amount of aborted batches until the mail server becomes available again.

If this action is not called, the default value of 30 seconds will be used.

Example:

```
ex_wait_time("20")
ex_abort_time("60")
ex_max_docs("200")
ex_scan
```

ex_done_folder

Specifies the mailbox subfolder to which email messages are moved after the attachment was imported.

Member of namespace

Ewsmail

Syntax

`ex_done_folder ()`

Parameters

folder Type: string

Destination folder for successfully imported emails

Parameters

folder : Destination folder for successfully imported emails

Returns

Always **True**.

Level

Batch level.

Details

Specifies the name of the email folder into which successfully imported emails are placed. This folder must be a subfolder of the email account's Inbox. When an email is processed and the attachment is imported, the email is moved to the folder name specified by this action.

If this action is not called, the default value of **Done** is used.

Example:

```
ex_done_folder("Imported")
ex_problem_folder("Failed")
```

ex_EMLOption

Creates a one page document per email containing an .eml file.

Member of namespace

Ewsmail

Syntax

ex_EMLOption ()

Parameters

folder Type: int

Optional - use a nonzero value to store one .eml file per email.

Parameters

folder : Optional - use a nonzero value to store one .eml file per email.

Returns

Always **True**.

Level

Batch level.

Details

If set, the `ex_scan` action creates a one page document containing the email and attachments in an `.eml` file; no attachment pages are created. When called with a non-zero parameter, the `ex_scan` function does not create pages for each attachment, instead one page is created per email document, containing an `.eml` file suitable for subsequent processing using eDocument Conversion actions.

When you use this action, pages for attachments are not created, and variables for those attachments are not set.

Example:

```
ex_EMLOption(1)
ex_scan()
```

ex_ews_version

Select the Exchange Server version.

Member of namespace

Ewsmail

Syntax

```
ex_ews_version ()
```

Parameters

version

Type: int

Parameters

The value 0, 1 or 2, indicating the following:

- 1 = Exchange 2007 SP1
- 2 = Exchange 2010
- 0 = latest version (default)

Returns

Always **True**.

Level

Batch level, Open event.

Details

Each version of Exchange uses a slightly different communication protocol. Use this action to set the expected version.

In order to connect successfully with:

- Exchange 2007 SP1, call this action with parameter **1** before `im_login`.
- Exchange 2010, call action with parameter **2** before `im_login`.
- The latest version known by the .NET library in use, which is .NET 3.5 and is currently Exchange 2010. This action must be called with parameter **0**.

If this action is not called at all it uses the default **0**, the latest version.

Example:

```
ex_ews_version(1)
ex_login("mymailserver/Exchange.asmx","Username@Org","password")
```

ex_HTTP_timeout

Specifies the maximum time to wait for an HTTP request or response from Exchange Server.

Member of namespace

Ewsmail

Syntax

```
ex_HTTP_timeout (int nSecs)
```

Parameters

String **nSecs**

Parameters

nSecs : The maximum number of seconds to wait.

Returns

Always **True**.

Level

Batch level Open event only.

Details

The maximum time to wait for an HTTP request or response from Exchange Server. For example, this may be used by the `ex_scan` action while trying to ingest a very large email that may otherwise result in import failing due to the operation timing out. This timeout is in addition to any server-side timeout configuration settings.

If this action is not called, the timeout defaults to 100 seconds.

Example:

```
ex_HTTP_timeout("60")
ex_scan()
```

This example causes the timeout to be set to 60 seconds. When ingesting emails, any response exceeding this value may fail.

ex_load_properties_option

Option to flag .

Member of namespace

Ewsmail

Syntax

`ex_load_properties_option ()`

Parameters

nOption

Type: int

Parameters

nOption : 0 to load base properties, 1 to load limited extended base properties, 2 to load extended properties

Returns

Always **True**.

Level

Batch level Open event only.

Details

Used to optionally load certain properties at once. If this action is not called, the default of 0 will be used to load base properties.

Example:

```
ex_load_properties_option("1")
ex_scan()
```

This example reduces the number of times you have to access the Exchange Server because the additional properties are loaded once.

ex_login

Specifies the Exchange Server and mail account

Member of namespace

Ewsmail

Syntax

`ex_login ()`

Parameters

hostname

Type: string

URL of Exchange Web Service, ends with /Exchange.asmx (Smart parameters are supported)

username

Type: string

Username@Org for mail account and organization (blank to use Windows Authentication, Smart parameters are supported)

password

Type: string

Password for mail account (blank if none or Windows Authentication, Smart parameters are supported)

Parameters

- hostname : URL of Exchange Web Service, ends with /Exchange.asmx
- username : Username@Org for mail account and organization (blank to use Windows Authentication)
- password : Password for mail account (blank if none or Windows Authentication)

All parameters support smart parameters.

Returns

True if the login succeeds. Otherwise, **False**.

Level

Batch level, open event.

Details

Connects to the mail server using the specified account information. Login credentials are for a Microsoft Exchange mail server.

The mail account must contain an Inbox folder and separate folders for messages imported (Done) and errors (Problem). The Done and Problem folders must be subfolders of the email account's Inbox. The names of these folders can be specified using the ex_done_folder and ex_problem_folder actions.

The Microsoft Exchange Email actions are designed to scan an email Inbox for incoming mail messages, and place selected messages into a new batch. It is possible to ignore all messages except those containing specific attachment types. The actions are typically assigned to a Task that is executed by an unattended Rulerunner station. Multiple Inboxes can be scanned by stringing together a set of login/scan/logout actions for each Inbox, or by assigning an Inbox to each input task.

Example:

```
ex_login("mymailserver/Exchange.asmx","Username@Org","password")
```

ex_logout

Disconnect from the mail server

Member of namespace

Ewsmail

Syntax

```
ex_logout ()
```

Parameters

None.

Returns

Always **True**.

Level

Batch level Open or Close event only.

Details

Closes the connection to the mail server. Call once for each batch after `ex_login` and `ex_scan` have completed.

Example:

```
ex_logout()
```

ex_max_docs

Specifies maximum number of emails to include in a single batch.

Member of namespace

Ewsmail

Syntax

```
ex_max_docs ()
```

Parameters

nDocs Type: int

The maximum number of emails in a batch.

Parameters

nDocs : The maximum number of emails in a batch.

Returns

Always **True**.

Level

Batch level Open event only.

Details

The import of emails into the batch will stop when this email limit is reached or when the maximum wait time has been reached. While waiting for new mail to arrive, the configured mailbox will be polled every two seconds to check for waiting mail.

If this action is not called, the default value of 100 is used. The actual amount of emails included in the batch could be less than this maximum.

Note: The value indicates the number of emails with expected attachments that will be processed. If a waiting email contains an unexpected attachment, it is not counted against the total maximum count.

It is also possible for an email to contain more than one valid attachment type. When this happens, it is possible for the number of items input to the batch can be greater than the number of configured emails. For example, if `ex_max_docs` is set to 5 and if there are 5 waiting emails and each email has 2 attachments, the total number of attachments that are included in the batch is 10.

Example:

```
ex_max_docs("50")
ex_scan()
```

This example causes the scan operation to limit the number of emails in a batch to a maximum of 50.

ex_problem_folder

Specifies folder for problem emails

Member of namespace

Ewsmail

Syntax

```
ex_problem_folder ()
```

Parameters

folder Type: string

Destination folder for problem email

Parameters

folder : Destination folder for problem email

Returns

Always **True**.

Level

Batch level.

Details

When an email is processed and the attachment is not one of the expected types, the email is moved to the folder name specified by this action.

This folder must be a subfolder of the email account's Inbox.

If this action is not called, the default value of Problem is used.

Example:

```
ex_done_folder("Imported")
ex_problem_folder("Failed")
```

ex_scan

Poll the specified mail server for incoming email with image attachments

Member of namespace

Ewsmail

Syntax

`ex_scan ()`

Parameters

None.

Returns

False if the operation fails, and the action will also pause before returning based on the configured abort time configured by `ex_abort_time`. Otherwise, **True**.

If no selected emails were available, the action returns **True** and also pauses before returning based on the wait time configured using `ex_wait_time`.

Level

Batch level Open event only.

Details

Scans emails in the Inbox for specified attachments, imports selected emails with attachments into the batch. Call once for each batch. A connection to the email server must have previously been established using the `ex_login` action.

Each input email (document) will contain the following variables set in the document hierarchy:

- **TYPE** : Always set to Document.
- **MessageID** : The email message ID.
- **Subject** : The email subject.
- **Body** : The text within the email.
- **DateSent** : The sent date stamp on the email.
- **From** : The email sender.
- **To** : The email recipients.
- **Priority** : The state of the email importance flag.
- **DateReceived** : The received date stamp on the email.

Each email attachment (page) will have the following variables set:

- **TYPE** : Always set to "Other".
- **IMAGEFILE** : The name of the attachment as saved on disk.
- **ATTACHNAME** : The name of the attachment.

The following batch level variable is created:

- **EmailCount** : The number of emails scanned into the batch.

Note: If the configured Done or Problem folders do not exist, the email will be moved to the Deleted Items folder.

Waiting emails are processed in blocks. The amount of emails processed in a block will be the number of waiting emails, or the number of emails remaining from the value set in `ex_max_docs`, up to a maximum of 50. After a block of emails are processed, the wait time is checked. If the wait time has not been reached and the maximum number of emails has not been reached, then another block of emails are processed. If the wait time has been reached, then no more emails are input to the batch.

It is recommended that the subject line be no longer than 78 characters as this is a common subject line length limitation. Some systems may support even shorter lengths, truncating the subject. Our testing has been successful with lengths up to 255 characters. It is recommended to test your settings and use lengths appropriate for your systems.

Use `ex_max_docs` to configure the number of emails processed per batch. The number of items included in the batch can be different than this amount, see `ex_max_docs` for more details.

Example:

```
ex_wait_time("20")
ex_abort_time("40")
ex_max_docs("50")
ex_types("jpg,tif")
ex_scan()
```

ex_types

Specifies valid image attachment extensions

Member of namespace

Ewsmail

Syntax

`ex_types ()`

Parameters

extensions

Type: string

Comma-separated list of file image file attachment extensions to import, with or without period. If a blank extension is supplied, `ex_scan` will import messages with no attachments.

Parameters

extensions : A comma-separated list of file image file attachment extensions to import, with or without period. If a blank extension is supplied, `ex_scan` will import messages with no attachments.

Returns

Always **True**.

Level

Batch level.

Details

This action specifies the allowable email attachment types.

If this action is not called, the default value of .pdf is used.

If this action is called and no attachment types are specified, all emails and any attachments are added to the batch.

If attachment types are specified and the email contains:

- Only the specified attachment types, the email and attachments are added to the batch
- Only unspecified attachment types, or if it contains a mix of specified and unspecified attachment types, the email is moved to the Problem folder
- No attachments, the email is moved to the Problem folder

Example:

```
ex_wait_time("20")
ex_abort_time("40")
ex_max_docs("200")
ex_types("jpg,tif")
```

ex_wait_time

Specifies the maximum time to wait for input emails for a single batch.

Member of namespace

Ewsmail

Syntax

ex_wait_time ()

Parameters

nSecs Type: int

Parameters

nSecs : The maximum number of seconds to wait.

Returns

Always **True**.

Level

Batch level Open event only.

Details:

The maximum time to wait for input files for a single batch. Used by the ex_scan action after the first email has been processed to determine how long to wait for the batch to fill up.

The import of emails into the batch will stop when the wait limit is reached or when the maximum emails per batch has been reached. While waiting for new mail to arrive, the mailbox will be polled every two seconds to check for waiting

mail. The action will continue to include new emails into the batch until this wait time is reached or the maximum number of emails per batch is reached.

If this action is not called, the default wait time of 5 seconds will be used.

Example:

```
ex_wait_time("20")  
ex_scan()
```

This example causes the scan operation to wait for 20 seconds for additional email, after processing the first email, before finishing the scan operation.

Export actions

Use the Export actions to set up and write information to the export text file.

The Export actions can setup lineitem values, batch, document and page variables, path locations, before exporting the information.

- "BatchVariable_ExportValue" on page 482
- "BlankFields" on page 482
- "BlankLines" on page 483
- "BPilot" on page 483
- "CloseExportFile" on page 484
- "DCOProperty" on page 485
- "DocumentVariable_ExportValue" on page 485
- "ExportAllFields" on page 486
- "ExportFieldValue" on page 487
- "ExportMYValue" on page 487
- "ExportSmartParameter" on page 488
- "ExportToBatchDir" on page 488
- "Filler" on page 489
- "FixedLenLJ" on page 489
- "FixedLenRJ" on page 490
- "GetDATE" on page 490
- "GetProfileString" on page 491
- "GetTime" on page 492
- "LineItem_AddElement" on page 493
- "LineItem_BlankFields" on page 493
- "LineItem_ClearElements" on page 494
- "LineItem_ExportElements" on page 494
- "LineItem_SmartParameter" on page 495
- "NewLine" on page 496
- "PageVariable_ExportValue" on page 496
- "ResetFieldVariables" on page 497
- "SaveFilePathAsVariable" on page 497
- "SetCSV" on page 498
- "SetElementSeparator" on page 499
- "SetExportPath" on page 499

"SetExtensionName" on page 500
"SetFileName" on page 501
"SetFill" on page 501
"SetFixedLength" on page 502
"SetIgnoreFieldStatus" on page 502
"SetJustified" on page 503
"SetOMR_Separator" on page 504
"SetSpaceFill" on page 504
"SetZeroFill" on page 505
"Text" on page 505
"Variable_ExportValue" on page 506
"Variable_IsValue" on page 506

BatchVariable_ExportValue

Exports the value contained in the specified batch-level variable.

Syntax

()

Parameters

The name of the Batch variable.

Returns

Always **True**.

Level

Any level.

Details

Exports the value contained in the specified batch-level variable.

Example

```
BatchVariable_ExportValue("ED")
```

This action will export the value located in the ED Batch variable to your Export file.

BlankFields

Inserts n blank fields into the Export file, adjacent to the current field.

Syntax

()

Parameters

A number indicating how many blank fields to add to the Export file.

Returns

Always **True**.

Level

All levels.

Details

The BlankFields action adds the number of fields you specify. The fields are blank; other actions direct the Export task to fill the fields.

Attention: Make sure you call SetCSV, and optionally SetElementSeparator, to set the separator values as desired for your export file. If either of these actions are not called before the BlankFields action, blank fields cannot be exported into the file because the default separator is set to no separator.

Example

```
SetCSV("TRUE")
BlankFields("12")
```

BlankLines

Inserts n blank lines into the Export file.

Syntax

()

Parameters

A number n that indicates how many blank lines to add after the current line.

Returns

Always **True**.

Level

All level.

Details

Inserts n blank lines into the Export file.

Example

```
BlankLines("4")
```

This action inserts four empty lines, leaving the insertion point for the next output on the following line. Additional output begins on the fifth line.

BPilot

Exports the value assigned to the Batch Pilot property designated as the parameter.

Syntax

()

Parameters

The name of the Batch Pilot Property whose value is to be included in the Export file.

- BatchDir: The name and location of the application's Batches directory.

- **BatchID:** The Batch Number of the current batch (20020072.003, for example.)
- **JobName:** The name of the current User Application job (Main, for example.)
- **Operator:** The User ID of the operator currently processing the batch.
- **PagesInBatch:** A count of all pages in the batch.
- **DocsInBatch:** A count of all documents in the batch. Remember: in most configurations, a Recognition task reorganizes a batch into a series of documents and their pages.
- **Priority:** The processing priority assigned to the current batch ("10" = Low, "1" = High, "5" = Default). A task selects batches from its queue first according to Priority.
- **Station:** The Station ID of the workstation currently processing the batch.
- **TaskName:** The name of the task with the batch in its queue.
- **XtraBatchFieldValue:** The value in a custom field you've added to the Job Monitor's Batch Information Table.

Returns

False if the parameter is not a Batch Pilot property. Otherwise, **True**.

Level

Any level.

Details

Exports the value assigned to the Batch Pilot property that designated as the parameter.

Example

```
NewLine()
Text("BatchID:")
BPilot ("BatchID"
```

This sequence adds "BatchID:" followed by the current Batch ID into the Export file. For example: BatchID: 20050019.001

CloseExportFile

Closes the currently opened Export file.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

Any level.

Details

Closes the currently opened Export file.

This action usually belongs to its own RuleSet (ExportClose, for example), and applies to the Batch object of the Document Hierarchy. However, it can be used at any level.

Example

```
CloseExportFile()
```

DCOProperty

Exports the value assigned to the DCO property that you designate as the parameter.

Syntax

()

Parameters

The name of the DCO Property whose value is to be included in the Export file.

- **ID:** The value of an object's ID property. For a Batch object, this might be 20020072.003. You can apply this action at any level(s).
- **ImageName:** The name and location of a Page object's Image file: for example, c:\ParentDirectory\Invoices\batches\20030145.001\TM000001.tif.
- **Status:** The value assigned to an object's Status property. You can apply this action at any level(s).
- **Type:** The value assigned to an object's Type property. You can apply this action at any level(s).

Returns

False if the parameter is not a valid DCO Property. Otherwise, **True**.

Level

All levels.

Details

Exports the value assigned to the DCO Property that you designate as the parameter.

Example

```
NewLine()  
Text("Document: ")  
DCOProperty("ID")
```

If this sequence is applied to a Document object, the Export file for document 01 in batch 20050219.057 will look like: Document:
20050219.057.01.

DocumentVariable_ExportValue

Exports the value that is contained in the specified Document-level variable.

Syntax

()

Parameters

Document Variable Name.

Returns

Always **True**.

Level

Document, Page or Field level.

Details

Exports the value that is contained in the specified Document-level variable.

Example

A number of techniques can add variables to a Document object of the Document Hierarchy. These variables are listed as properties of the object in the Document Hierarchy Setup window.

The following example reads the value that is assigned to the *PageCount Document* variable and places it in the Export file.

```
DocumentVariable_ExportValue("PageCount")
```

ExportAllFields

Exports all field values on the current page, including values of Line Item Detail sub-fields - with exceptions.

Syntax

()

Parameters

None.

Returns

False, if the action is not used at the Page level. Otherwise, **True**.

Level

Page level.

Details

This action exports all field values of the current page. However, the action does not export those values of Field objects of the Document Hierarchy with:

1. A setup *NOEXPORT* variable of "1", or
2. A value of a runtime *SetIgnoreStatus*, a field status equal to the Numeric field status defined by a previously-run *SetIgnoreFieldStatus* action. For details, see the description of that action.

Example

```
ExportAllFields()
```

ExportFieldValue

Exports the specified Field object's value to the Export file.

Syntax

(StrParam)

Parameters

The name of the Field object whose value you want to export.

Returns

False if the parameter is not a Field object's name. Otherwise, **True**.

Level

Page level.

Details

Exports the specified Field object's value to the Export file. Will only export the last Field if multiple fields of the same field type are found.

Example

```
ExportFieldValue("Date")  
ExportFieldValue("Number")  
ExportFieldValue("Total")
```

This sequence exports the current values stored in the Date, Number and Total fields.

ExportMYValue

Exports the current field value to the Export file.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

Field level.

Details

Exports the current field value to the Export file.

Example

```
ExportMYValue()
```

ExportSmartParameter

Exports an evaluated smart parameter value to the Export file.

Syntax

```
()
```

Parameters

The value to export expressed with smart parameter syntax.

Returns

Always **True**.

Level

Any level.

Details

Exports an evaluated smart parameter value to the Export file. If the input parameter is not a smart parameter, it will export an empty field.

Example

The example will export the value of a variable named *Expired* on field *DueDate* which is a child field on the parent page (@P) of the calling node.

```
ExportSmartParameter("@P\DueDate.Expired")
```

ExportToBatchDir

Specifies that the path for the current Export file (txt) is the Batch directory.

Syntax

```
()
```

Parameters

None.

Returns

False, if the Batch Directory is not accessible. Otherwise, **True**.

Level

All levels.

Details

This action sets the path for the Export file to the current Batch directory.

Attention: Usually, an Export file is placed in the application's Export folder instead of in a Batch folder of the Batch directory.

Example

```
ExportToBatchDir()
```

Filler

Adds a string of identical filler characters to the Export.

Syntax

```
()
```

Parameters

Two comma-separated values. The first is a number indicating the total length in characters of the filler. The second parameter is the filler's character.

The second parameter is optional: if you do not enter a value, the action will use the most recent Global character setting.

Returns

False if the first parameter is not numeric or if the second parameter is more than 1 character. Otherwise, **True**.

Level

All levels.

Details

Adds a single filler character to the Export, repeated by the number of times indicated in the first input parameter. The second parameter cannot be a space. The filler string is written regardless of the data in the current field. If you wish to use a space as a filler character, use `SetSpaceFill` and then call `Filler` without the optional second parameter.

Example

```
Filler("12,n")
```

The action in the example fills the current field with 12 instances of the character "n".

FixedLenLJ

Exports a specified number of characters from a field's left end (left-justified).

Syntax

```
()
```

Parameters

Two comma-separated values.

1. The field's name: The name of the corresponding Field object of the Document Hierarchy.
2. The number of characters that are to be exported, counting from the field's left end.

Returns

False, if either parameter is invalid or if the action is called at the wrong level. Otherwise, **True**.

Level

Page level.

Details

This action is similar to the FixedLenRJ. It exports a specified number of characters from a field's left end (left-aligned).

Example

```
FixedLenLJ("Volume,8")
```

FixedLenRJ

Exports a specified number of characters from a field's right end (right-justified).

Syntax

()

Parameters

Two comma-separated values.

1. The field's name: The name of the corresponding Field object of the Document Hierarchy.
2. The number of characters that are to be exported, counting from the field's right end.

Returns

False, if either parameter is invalid or if the action is called at the wrong level. Otherwise, **True**.

Level

Page level.

Details

Similar to the FixedLenLJ action, this action exports a specified number of characters from a field's right end (right-aligned).

Example

The following example exports 12 characters from the right end of the InvoiceDate field.

```
FixedLenRJ("InvoiceDate,12")
```

GetDATE

Exports today's Date in the format specified as the parameter.

Syntax

()

Parameters

The Date's format.

"*" stipulates the default mm/dd/yyyy construction. However, you can combine any of the following String values to define a different format:

-
- d = day of the month, 1-31
- dd = two-digit day, 01-31
- yyyy = four-digit year
- yy = two-digit year
- m = month, 1-12
- mm = two-digit month, 01-12
- ccyy = four-digit year
- y = Julian day of the year

"," and "/" are valid separators.

Returns

Always **True**.

Level

All levels.

Details

Exports today's Date in the format specified as the parameter.

Example

```
GetDate("*") inserts today's date into the Export file with this format:  
11/16/2005
```

GetProfileString

Accesses a Settings file (.ini) and adds a value in that file to your Export file.

Syntax

()

Parameters

1. The [Section] within the Settings file.
2. The Key entry within the section, with the value you want to retrieve.
3. The name of the Settings file.

Returns

False if the settings file cannot be found. Otherwise, **True**. If the settings file can be found but the key entry cannot be found within the file, this action will return **True**.

Level

All levels.

Details

Accesses a Settings file (.ini), locates the specified key and adds the value of the key to your Export file. If the key cannot be read from the settings file, an empty string will be written to the export. If the settings file cannot be found, nothing will be written to the export.

Important: The action assumes that the Settings file resides in the current batch directory. If you want the INI file to reside in the Batches directory, specify your file name with a relative path like this: `..\myfile.ini`.

Example

```
GetProfileString("General,MyValue,Batch.ini")
```

GetTime

Exports the current Time in the format specified as the parameter.

Syntax

()

Parameters

The parameter specifies the display format for the current time.

* = A single asterisk will use the HH:MM:SS time format. However, you can combine any of the following String values to define a different format:

-
- m = minute 1-59
- s = second 1-59
- h = 1-23
- mm, min, minute = two-digit minute, 01-59
- ss, sec, second = two-digit second, 01-59
- hh, hr, hour = two-digit hour, 01-23

"/" are the valid separators.

Returns

Always **True**.

Level

Any level.

Details

Exports the current Time in the format specified by the input parameter.

Example

```
GetTime("*")
inserts the current time into the Export file with this format: 07:08:16
```

```
GetTime("hh-mm-ss")
inserts the current time into the Export file with this format: 07-08-16
```

```
GetTime("ss:mm:hh")  
inserts the current time into the Export file with this format: 16:08:07
```

LineItem_AddElement

Includes the specified Line Item Field object as an element of a Line Item Array.

Syntax

()

Parameters

The name of the child Field object of the Document Hierarchy.

Returns

Always **True**.

Level

The parent field that contains the child Line Item field.

Details

This action includes the specified Line Item field object as an element of a Line Item Array.

A Line Item Array accumulates and organizes captured line item values that are retrieved from the Data file of a particular page.

Note: A rule that uses this action must be applied to the LINEITEM fields of the Document Hierarchy.

This action is used for exporting Line Item values.

Example

The following action expands the Line Item Array by one field: Price.

LineItem_ExportElements populates this element and other elements of the array with the captured values that it finds in a page's Data file before exporting them.

```
LineItem_AddElement("Price")  
LineItem_ExportElements()
```

LineItem_BlankFields

Includes the specified number of blank fields as elements of a Line Item Array.

Syntax

()

Parameters

The number of Blank fields to export as part of the Line Item Array.

Returns

Always **True**.

Level

The parent field that contains the child Line Item field.

Details

This action includes the specified number of Blank Line Item fields as elements of a Line Item Array.

A Line Item Array accumulates and organizes captured line item values that are retrieved from the Data file of a particular page.

Attention: A rule that uses this action must be applied to the LINEITEM fields of the Document Hierarchy.

This action is used for exporting Blank Line Item values.

Example

The following action expands the Line Item Array by six blank fields.

LineItem_ExportElements populates this element and other elements of the array with the captured values that it finds in a page's Data file before exporting them.

```
LineItem_BlankFields("6")
```

LineItem_ClearElements

Clears values in the Line Item Array.

Syntax

()

Parameters

None

Returns

Always **True**.

Level

The parent Field object of the Document Hierarchy that contains a child Line Item field, such as you may typically find in an invoice application.

Details

This is mainly a housekeeping function you can use to be sure the array does not contain leftover values.

Example

```
LineItem_ClearElements()
```

LineItem_ExportElements

Exports the captured values in a page's Line Item Array that have been populated with LineItem_AddElement actions.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

The parent field that contains the child Line Item sub-fields.

Details

Exports the captured values in a page's Line Item Array that have been populated with `LineItem_AddElement` actions.

Example

```
LineItem_AddElement("Price")
LineItem_AddElement("LineTotal")
LineItem_ExportElements()
NewLine()
```

This example exports the values included in the Line Item Array to your Export file.

LineItem_SmartParameter

Add a smart parameter algorithm as an element of a Line Item Array.

Syntax

()

Parameters

Smart Parameter to be evaluated during processing of the Line Item array.

Returns

Always **True**.

Level

The parent field that contains the child Line Item field.

Details

This action permits adding a smart parameter as an element of a Line Item Array to be evaluated during the Array processing.

A Line Item Array accumulates and organizes captured line item values that are retrieved from the Data file of a particular page.

Important: A rule that uses this action must be applied to the `LINEITEM` fields of the Document Hierarchy.

This action is used for exporting Blank Line Item values.

Example

The following action places a child field Price of the calling field node (@F) appended with the current time in format HH:MM:SS to the export file.

LineItem_ExportElements populates this element and other elements of the array with the captured values that it finds in a page's Data file before exporting them.

```
LineItem_SmartParameter("@F\Price+@TIME(HH:MM:SS)")  
LineItem_ExportElements()
```

NewLine

Starts a new line in your Export file.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

Any level.

Details

Starts a new line in your Export file.

Example

```
NewLine()  
Text("Export Output")
```

This sequence starts a new line and adds "Export Output" to the beginning of the line.

PageVariable_ExportValue

Exports runtime values assigned to a variable of the bound Page object of the Document Hierarchy.

Syntax

()

Parameters

String value of the variable's name.

Returns

Always **True**.

Level

Page level.

Details

Exports runtime values assigned to a variable of the bound Page object of the Document Hierarchy.

Example

```
PageVariable_ExportValue("TemplateID")
```

ResetFieldVariables

Resets the variables of the bound Field object of the Document Hierarchy.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

Any level.

Details

This action resets some of the export settings that can be independently set by other actions.

- The default field fill character is reset to a space.
- Field alignment is reset to left.
- The ignore field status is reset to not ignore any fields that are based on the status.
- Fields are no longer set to a specific length.

Example

```
ResetFieldVariables()
```

SaveFilePathAsVariable

Saves the path and name of your Export file to the variable specified by the parameter.

Syntax

()

Parameters

Variable name specifying where the Export file name and path will be stored.

Returns

Always **True**.

Level

All levels.

Details

Saves the path and name of your Export file to the variable specified by the parameter. If the variable does not exist, it will be created.

Example

```
SaveFilePathAsVariable("Export_File")
```

SetCSV

Ensures that all exported values are delimited by a comma separator.

Syntax

()

Parameters

TRUE or **ON**: Enables CSV formatted output using a comma separator.

FALSE: Uses a custom separator, as set by `SetElementSeparator()`, between output fields.

OFF: Causes no separator to be placed between output fields.

Smart parameters are supported.

Returns

Always **True**.

Level

All levels.

Details

Ensures that all exported values are delimited by a comma separator.

Example

```
SetCSV("TRUE")  
ExportFieldValue("Date")  
ExportFieldValue("Number")  
ExportFieldValue("Total")  
SetCSV("FALSE")
```

This sequence will export the captured values of the Date, Number, and Total Field objects into your Export file. A comma will be added after each value to separate the fields.

SetElementSeparator

Ensures that all exported values are delimited by a separator designated as the parameter.

Syntax

()

Parameters

The input parameter can be one of the following:

- A field separator character: Uses the provided custom character as the separator between fields. Smart parameters are supported.
- **ON** or **TRUE**: Sets the field element separator to a single space. This is the default value.
- **OFF** or **FALSE**: No separator will be placed between fields.

Smart parameters are supported.

Returns

Always **True**.

Level

All levels.

Details

Ensures that all exported values are delimited by a separator designated as the parameter.

Attention: If you wish to set your own custom separator, SetCSV(FALSE) must be called prior to exporting fields. If SetCSV(FALSE) is not called, then your custom element separator will not be used for export.

Example

```
SetCSV("FALSE")  
SetElementSeparator("|")
```

This action uses "|" to delimit the Export file's values.

```
SetElementSeparator("Off")
```

Turns off the action.

SetExportPath

Specifies the path to the Export file's location. Alternatively, you can use a Smart Parameter to identify a Paths.ini file that has a set of path parameters for your application.

Syntax

()

Parameters

The complete path to the application's Export folder. Smart parameters are supported.

Returns

True, if the path specified by the parameter exists. Otherwise, **False**.

Level

All

Details

The action's parameter specifies the path to the Export file's location, or uses a Smart Parameter to retrieve a path's value from the application's Paths.ini file.

Example

```
SetExportPath("c:\ParentDirectory\Invoice\Export")
```

```
SetExportPath("@APPPATH(export)")
```

In the second example, a smart parameter is used to obtain the location of the Export Directory from the value that is configured in the Application Manager.

SetExtensionName

Assigns an extension to the current Export file.

Syntax

()

Parameters

The file extension you want to use, including the leading period. If you do not want a file extension, do not pass any parameter to this action. Disallowed characters are * > < [] / \ : ? and control characters.

Smart parameters are supported.

Returns

Always **True**.

Level

Any level.

Details

Assigns an extension to the current Export file. If this action is not called, the default value of .TXT will be used.

Example

```
SetFileName("Export_+@BATCHID")  
SetExtensionName(".dat")
```

In this example, the Export file will have a .dat extension.

SetFileName

Assigns a name to the current Export file.

Syntax

()

Parameters

The file's name (without an extension).

Smart parameters are supported.

Returns

Always **True**.

Level

All levels.

Details

Assigns a name to the current Export file. If SetExtensionName is not called, the file extension defaults to .TXT. If you require a different file extension, use SetExtensionName.

Example

```
SetFileName("Export_+@BATCHID")
SetExtensionName(".txt")
```

This sequence establishes a series of Export files with names such as
Export_20021231.001.txt
Export_20021231.002.txt

In contrast,
SetFileName("@BATCHID")
SetExtensionName(".txt")

will establish a series of Export files with Batch IDs only:
20021231.001.txt
20021231.002.txt

SetFill

Sets the filler character to be used to expand the current value of a field in a flat file, if the field's allowable length is greater than the length of its current export value.

Syntax

()

Parameters

Single String character to be used as the filler value.

Returns

False, if more than one character is entered as a parameter. Otherwise, **True**.

Level

Any level.

Details

Sets the filler character to be used to expand the current value of a field in a flat file, if the field's allowable length is greater than the length of its current export value.

Attention: When using `SetFill`, the action `SetFixedLength`, `FixedLenRJ` or `FixedLenLJ` must also be used to set the maximum length of the field. You can use `SetSpaceFill` if you wish to make the filler character a space. Use `ResetFieldVariables` to clear this setting.

Example

```
SetFixedLength("10")  
SetFill("$")
```

This example sets the fill character to a \$.

SetFixedLength

Uses the Numeric value you enter as a parameter to establish a fixed length of a value exported from the current field.

Syntax

()

Parameters

Numeric value indicating the field's length.

Returns

False if the parameter is not Numeric. Otherwise, **True**.

Level

Any level.

Details

Uses the Numeric value you enter as a parameter to establish a fixed length of a value exported from the current field.

Use `ResetFieldVariables` to clear this setting.

Example

```
SetFixedLength("12")
```

SetIgnoreFieldStatus

Assigns a Numeric value to the application's `SetIgnoreStatus` variable. Any field with this status cannot export data to an Export file or database.

Syntax

()

Parameters

A Numeric value that represents the status of fields to be ignored by Export tasks.

Returns

False if the parameter is not Numeric. Otherwise, **True**.

Level

Any level.

Details

This action establishes the status that determines if an Export task will export a field's value. If the status of the field being exported matches this set value, the field will not be exported.

Use `ResetFieldVariables` to clear this setting.

Example

```
SetIgnoreFieldStatus("1")
```

This example ensures that runtime values for fields with a "1" status will not be added to an Export file or update an Export database. (Typically, "1" denotes a problem field.)

SetJustified

Right-justifies or left-justifies a field's exported values.

Syntax

()

Parameters

An uppercase R (right-align) or L (left-align).

Returns

False, if the parameter is not an "R" or "L". Otherwise, **True**.

Level

Any level.

Details

Right-justifies or left-justifies a field's exported values according to the parameter you enter.

Use `ResetFieldVariables` to clear this setting. `SetFixedLength` must also be used to set the maximum length of the field.

Example

```
SetFixedLength("10")  
SetJustified("R")
```

SetOMR_Separator

For multi-punch OMR fields, uses the parameter's value as the separator character.

Syntax

()

Parameters

The separator character you want to use.

Returns

Always **True**.

Level

All levels.

Details

For multi-punch OMR fields, uses the parameter's value as the separator character. When multi-punch fields are exported, you typically do not want to use the same separator that you are using for fields as these values are typically all within a single field. This action allows you to specify a custom separator to be used when exporting. If this action is not called, the default value is a space.

Example

```
SetOMR_Separator(";")
```

SetSpaceFill

Specifies the use of the ASCII 32 space as the global filler value to be used to expand the current value of a field in a flat file, if the field's allowable length is greater than the length of its current export value.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

Any level.

Details

Specifies the use of the ASCII 32 space as the global filler value to be used to expand the current value of a field in a flat file, if the field's allowable length is greater than the length of its current export value.

If you use `SetSpaceFill`, the action `SetFixedLength`, `FixedLenRJ` or `FixedLenLJ` must also be used to set the maximum length of the field. Use `ResetFieldVariables` to clear this setting.

Example

```
SetSpaceFill()
```

Note that the action specifies the use of the ASCII 32 "space" as the filler character.

SetZeroFill

Sets the ASCII 48 zero as the global filler value to be used to expand the current value of a field in a flat file, if the field's allowable length is greater than the length of its current export value.

Syntax

```
()
```

Parameters

None.

Returns

Always **True**.

Level

Any level.

Details

Sets the ASCII 48 zero as the global filler value to be used to expand the current value of a field in a flat file, if the field's allowable length is greater than the length of its current export value.

`SetFixedLength`, `FixedLenRJ` or `FixedLenLJ` must also be used to set the maximum length of the field. Use `ResetFieldVariables` to clear this setting.

Example

```
SetFixedLength("10")  
SetZeroFill()
```

Note that the action specifies the use of the ASCII 48 "zero" filler.

Text

Places a string into the Export file.

Syntax

```
()
```

Parameters

The string to write to the export file.

Returns

Always **True**.

Level

Any level.

Details

This action unconditionally adds a string to the export file. No character padding will be performed on this value.

Example

```
SetFileName("Export_+@BatchID")
SetExtensionName(".txt")
Text("This line will appear in the export file.")
```

Variable_ExportValue

Exports the value assigned to a variable of the current object of the Document Hierarchy.

Syntax

()

Parameters

The name of the variable with the value you want to export.

Returns

Always **True**.

Level

Any level.

Details

Exports the value assigned to a variable of the current object of the Document Hierarchy.

Example

```
Variable_ExportValue("ID")
```

This action exports the value assigned to the ID property of the current object of the Document Hierarchy.

Variable_IsValue

Checks to see if the parameter value matches the value assigned to a variable of the current object of the Document Hierarchy.

Syntax

()

Parameters

1. The name of the variable with the value you want to compare.
2. The value you want to match with the variable's value.

Returns

True if the variable's value matches the parameter's value. Otherwise, **False**.

Level

Any level.

Details

Checks to see if the parameter value matches the value assigned to a variable of the current object of the Document Hierarchy.

Example

```
Variable_IsValue("Invoice,Yes")
```

This action returns **True** if the value of the current Page object's Invoice variable is *Yes*.

ExportDB actions

Use the ExportDB actions to set up and write information to an export database. You build the record in memory before you commit it to the database by using `AddRecord`.

The ExportDB actions open a connection to the database, specify the table to which the data is exported, and write the batch values to the internal data record.

“AddRecord”

“ExportBatchIDToColumn” on page 508

“ExportCloseConnection” on page 509

“ExportFieldToColumn” on page 510

“ExportNodeXMLToColumn” on page 511

“ExportOpenConnection” on page 512

“ExportPropertyToColumn” on page 513

“ExportSmartParamToColumn” on page 514

“ExportToColumn” on page 515

“SetTableName” on page 516

AddRecord

Inserts assembled data into the database table specified by a previous `SetTableName` action.

Syntax

()

Parameters

None.

Returns

False if there is no connection to the database; if an error occurs when the action attempts to add the record to the database; or if a SetTableName action was not previously used. Otherwise, **True**.

Level

All, but generally used at the Page or Field level.

Details

Inserts assembled data into the database table specified by a previous SetTableName action.

Important: This action must be placed after earlier actions that gather data, open the database, and access the correct table.

Example:

```
SetTableName("Invoice")
ExportFieldToColumn("VendorID,db_Vendor")
ExportFieldToColumn("Number,db_Number")
ExportFieldToColumn("Total,db_Total")
AddRecord()
```

This ExportDB rule applies to a Page object of the Document Hierarchy.

The actions open the database and direct the rule's attention to the Invoice table. The rule then sets up a record with three values - Vendor ID, Invoice Number and Total. Afterwards, the AddRecord action updates the table with new information.

Note: A return of False means the record may not have been successfully exported, and needs to be followed by a failure check rule that typically would call SetTaskStatus(0) (or the rrunner action rr_AbortBatch) to ensure that the batch aborts.

ExportBatchIDToColumn

Exports the current Batch ID to the database column specified by the parameter.

Syntax

()

Parameters

Three comma-separated values:

1. The name of the database column that will hold the value.
2. The name of the field whose value will be stored in the column.
3. Optional: Can be set to the numerical value of 1, 2, or 3.

If this parameter is not specified, quotes will be placed around the value in the SQL query to update the table.

Setting the value to 1 causes this action to return False if the Table name is invalid; otherwise the error will not be discovered until the action that performs the DB insert call, such as an AddRecord action, is called. This value would typically only be used in a development environment as it will increase processing time.

Setting this value to 2 allows NULL column values and inserts no quotes around values (for numeric) in the SQL query to update the database.

Setting the value to 3 allows NULL and if the column value is not NULL, the column value will be surrounded in quotes in the SQL query to update the database.

Returns

False if the second parameter is set to "1" and the parameters do not identify a valid database column. Otherwise, **True**.

Level

All, but generally at the Page or Field level.

Details

Exports the current Batch ID to the database column specified by the parameter. The action also has an optional Style parameter with two values.

Example:

```
SetTableName("Export_Results")
ExportBatchIDToColumn("db_BatchID,1,2")
ExportFieldToColumn("Date,db_Date")
ExportFieldToColumn("Total,db_Total")
AddRecord()
```

ExportCloseConnection

Closes an open connection to your Export database.

Syntax

()

Parameters

None.

Returns

True, even if the connection is already closed.

Level

All, but generally used as part of a separate RuleSet at the Batch level.

Details

Closes an open connection to the previously opened Export database.

Usually, this action is placed in a RuleSet that is separate from the RuleSet that opens the connection and stores the data.

For example this action could be placed into a RuleSet called ExportDBCclose, and attached to a batch level close event which executes after all data has been exported from the batch to the specified database.

Example:

```
ExportCloseConnection()
```

This action closes the previously opened connection to the Export database.

This action is usually part of a separate RuleSet that prevents the need to repeatedly open the connection to the Export database. (You can open the connection once in the first RuleSet, export data from all documents and pages in the batch, then close the connection once in the second RuleSet.)

ExportFieldToColumn

A page-level action that extracts the captured value of a Field object from the Data file of the current page, and specifies its target location within a table of the Export database.

Syntax

()

Parameters

Three comma-separated values:

1. The name of the field whose value will be stored in the column.
2. The name of the database column that will hold the value.
3. Optional: Can be set to the numerical value of 1, 2, or 3.

If this parameter is not specified, quotes will be placed around the value in the SQL query to update the table.

Setting the value to 1 causes this action to return False if the Table name is invalid; otherwise the error will not be discovered until the action that performs the DB insert call, such as an AddRecord action, is called. This value would typically only be used in a development environment as it will increase processing time.

Setting this value to 2 allows NULL column values and inserts no quotes around values (for numeric) in the SQL query to update the database.

Setting the value to 3 allows NULL and if the column value is not NULL, the column value will be surrounded in quotes in the SQL query to update the database.

Returns

False if:

1. There is no connection to the database.
2. The database column specified as a parameter does not exist.
3. The Field object identified by the parameter does not exist.
4. A SetTableName action was not used previously.

Otherwise, **True**.

Level

Page level only.

Details

This is a page-level action that extracts the captured value of a Field object from the Data file of the current page, and specifies its target location within a table of the Export database.

Within a rule, this action should run before an AddRecord action, which commits the data to the database.

Example:

```
SetTableName("Export_Results")
ExportFieldToColumn("VendorID,db_Number,2")
ExportFieldToColumn("Date,db_Date")
ExportFieldToColumn("Total,db_Total")
AddRecord()
```

This action exports the captured value of three Field objects from the Data file of the current page, to corresponding columns of the Export_Results table, within the open Export database. It also provides the action with an optional Style value.

Important: Make sure you use the ExportOpenConnection action to establish a connection to your Export database. This is usually accomplished by a rule at the Batch level.

ExportNodeXMLToColumn

Exports the value of the XML property of the bound object (node) of the Document Hierarchy to a column of the Export database.

Syntax

()

Parameters

Three comma-separated values:

1. The smart parameter path to the bound object of the Document Hierarchy. This object's XML property will be added to a designated column of the Export database.
2. The String value of the name of the target column in the Export database. The action will add the value of the calling object's XML property to this column.
3. Optional: Can be set to the numerical value of 1 or 3. If this parameter is not specified, quotes will be placed around the value in the SQL query to update the table.

Setting the value to 1 causes this action to return False if the Table name is invalid; otherwise the error will not be discovered until the action that performs the DB insert call, such as an AddRecord action, is called. This value would typically only be used in a development environment as it will increase processing time.

Setting the value to 3 allows NULL and if the column value is not NULL, the column value will be surrounded in quotes in the SQL query to update the database.

Returns

False if:

1. There is no connection to the database.
2. The column identified by the parameter does not exist.
3. A SetTableName action was not previously used.
4. The smart parameter path does not point to a valid object of the Document Hierarchy.

Otherwise, **True**.

Level

All, but generally at the Page or Field level.

Details

This action exports the value of the XML property of the bound object (node) of the Document Hierarchy to a column of the Export database.

Example:

```
ExportNodeXMLToColumn("@P\MyField,MYDBCOLUMN")
```

ExportOpenConnection

Opens a connection to the database specified as the parameter.

Syntax

()

Parameters

The Connection String of the target export database. Use Smart parameters to avoid clear text passwords in your application.

Returns

True, if the connection opens. Otherwise, **False**.

Level

All, but most often used at the Batch level.

Details

Opens a connection to the database specified as the parameter.

A rule that contains this action can apply to any object of the Document Hierarchy, but it is most often used at the batch level.

Example:

```
ExportOpenConnection("@APPVAR(values/dsn/exportdb:cs)")
```

Note: This action must come before any other ExportDB actions. This example uses smart parameters to obtain the connection string from the application service where the custom connection string key exportdb was

created in the **Custom Values** tab. It provides a single location for the connection string and security for passwords.

Note: If the action is establishing a connection with an Oracle database, or a SQL Server database using SQL Server Authentication, be sure to expand the DSN parameter by adding the correct Provider, user ID and Password. For example:

- Oracle:

```
ExportOpenConnection("PROVIDER=ODBCORACLE;DSN=1040Look;CATALOG=;DBNTA=;  
UID=Admin;PWD=Admin;")
```

- SQL Server Authentication:

```
ExportOpenConnection("PROVIDER=ODBCMSSQL;DSN=1040Look;CATALOG=;DBNTA=;  
UID=Admin;PWD=Admin;")
```

Note: Although the Oracle and SQL Server examples show the connection string, it is recommended that a smart parameter is used to obtain the connection string from the application service, as shown in the first example, to provide password security and application portability.

ExportPropertyToColumn

Adds the value of a property (variable) of the selected object to a column of the Export database

Syntax

()

Parameters

Three comma-separated values:

1. The property name whose value will be stored in the column.
2. The name of the database column that will hold the value.
3. Optional: Can be set to the numerical value of 1, 2, or 3. If this parameter is not specified, quotes will be placed around the value in the SQL query to update the table.

Setting the value to 1 causes this action to return False if the Table name is invalid; otherwise the error will not be discovered until the action that performs the DB insert call, such as an AddRecord action, is called. This value would typically only be used in a development environment as it will increase processing time.

Setting this value to 2 allows NULL column values and inserts no quotes around values (for numeric) in the SQL query to update the database.

Setting the value to 3 allows NULL and if the column value is not NULL, the column value will be surrounded in quotes in the SQL query to update the database.

Returns

False if:

1. There is no connection to the Export database.
2. The column of the database does not exist.
3. The property (variable) identified by the parameter does not exist.
4. A SetTableName action was not used previously.

Otherwise, **True**.

Level

All, but generally used at the Page or Field level.

Details

Adds the value of a property (variable) of the selected object to a column of the Export database.

Example:

```
SetTableName("Export_Results")
ExportFieldToColumn("VendorID,db_Number")
ExportPropertyToColumn("Status,db_PageStatus,2")
ExportFieldToColumn("Total,db_Total")
AddRecord()
```

This sequence updates the db_PageStatus column of the Export_Results table with the value of the selected object's Status property. It also uses the optional third parameter to define a Style for the action.

ExportSmartParamToColumn

Adds the evaluated value of a smart parameter to a column of the Export database

Syntax

()

Parameters

Three comma-separated values:

1. A smart parameter that specifies the value to be stored into the column.
2. The name of the database column that will hold the value.
3. Optional: Can be set to the numerical value of 1, 2, or 3. If this parameter is not specified, quotes will be placed around the value in the SQL query to update the table.

Setting the value to 1 causes this action to return False if the Table name is invalid; otherwise the error will not be discovered until the action that performs the DB insert call, such as an AddRecord action, is called. This value would typically only be used in a development environment as it will increase processing time.

Setting this value to 2 allows NULL column values and inserts no quotes around values (for numeric) in the SQL query to update the database.

Setting the value to 3 allows NULL and if the column value is not NULL, the column value will be surrounded in quotes in the SQL query to update the database.

Returns

False if:

1. There is no connection to the database.
2. The column identified by the parameter does not exist.
3. A SetTableName action was not previously used.

Otherwise, **True**.

Level

All, but generally at the Page or Field level.

Details

Using the database opened by an earlier ExportDB action, this action will store a value into the specified column for the current row.

The action allows the input value to be specified with a smart parameter.

Example:

```
ExportSmartParamToColumn("@P\MyField.TYPE,EXPDBCOLUMN")
```

ExportToColumn

A field-level action that exports the captured value of the current Field object from the page's Data file to a target column within a previously designated table of an open Export database.

Syntax

()

Parameters

Two comma-separated values:

1. The name of the database column that will hold the value of the current field.
2. Optional: Can be set to the numerical value of 1, 2, or 3. If this parameter is not specified, quotes will be placed around the value in the SQL query to update the table.

Setting the value to 1 causes this action to return False if the Table name is invalid; otherwise the error will not be discovered until the action that performs the DB insert call, such as an AddRecord action, is called. This value would typically only be used in a development environment as it will increase processing time.

Setting this value to 2 allows NULL column values and inserts no quotes around values (for numeric) in the SQL query to update the database.

Setting the value to 3 allows NULL and if the column value is not NULL, the column value will be surrounded in quotes in the SQL query to update the database.

Returns

False if:

1. There is no connection to the database.
2. The column identified by the parameters does not exist.
3. A SetTableName action was not previously used.

Otherwise, **True**.

Level

Field level only.

Details

This field-level action exports the captured value of the current Field object from the page's Data file to a target column within a previously designated table of an open Export database. Optionally, it can assign a Style to the action.

Example:

```
Batch Level:  
ExportOpenConnection("@APPVAR(values/dsn/exportdb:cs")  
SetTableName("Export_Results")
```

```
Current field:  
ExportToColumn("db_Date,3")
```

```
Last Field:  
AddRecord()
```

This example exports the captured value of the Field object to which the rule applies, from the Data file of the current page to the db_Date column of the Export_Results table, within the Export database.

It also uses the second, optional parameter value to assign a Style to the action.

SetTableName

Sets the name of the table in your database to which the data is to be exported.

Syntax

()

Parameters

Two comma separated parameters:

1. The name of the database table where the exported data will be inserted.
2. Optional: If the numeric value of 1 is specified, the action will immediately check the database to confirm that the supplied table name is valid.

If the table name is not valid, SetTableName will return **False**. Note that this value causes an extra database call so it is typically specified only during development, not in a production system as it will increase the number of database calls and running time.

If the second parameter is not specified, this action will always return **True**.

If the specified table name is invalid, an error will be returned when the current row is inserted to the database on the next AddRecord call.

Returns

Returns **False** only if the first parameter contains an invalid table name and a value of 1 is specified as a second parameter. Otherwise this action always returns **True**.

Level

All.

Details

Sets the name of the table in your database to which the data is to be exported. This action needs to be used before the AddRecord action.

Example:

```
SetTableName(Export_Results,1)
ExportFieldToColumn(MyDate, db_Date)
AddRecord()
```

ExportXML actions

Use the ExportXML actions to set up and write information to an export XML file.

The ExportXML actions can create nodes, assign attribute values to nodes, and specify the path to the storage location and the name of the export XML file.

“xml_CommitNode”

“xml_NewNode” on page 518

“xml_SaveFile” on page 518

“xml_SetAttributeValue” on page 519

“xml_SetExportPath” on page 520

“xml_SetFileName” on page 520

“xml_SetNodeValue” on page 521

xml_CommitNode

Commits (closes) current xml node with the xml tag value of the parameter.

Syntax

()

Parameters

String value of the xml Tag. Smart parameters are supported.

Returns

Always **True**.

Level

All.

Details

This action closes the specified node, which allows a new node with the same tag to be created at the same hierarchical level in the output xml. This action can use Smart Parameters.

Example

```
xml_CommitNode("LineTotal")
```

Example commits (closes) the current xml node with a Tag Line Total.

xml_NewNode

Creates a child node under the specified parent node, creating the parent node if necessary.

Syntax

()

Parameters

Comma-separated String values of:

1. The NodeID (tag name) of a new child node.
2. The NodeID (tag name) of the parent node, if there is a parent.

Smart parameters are supported.

Returns

True, if parent node exists. **False**, if duplicate root node is declared, or parent NodeID does not exist.

Level

All.

Details

The new NodeID followed by the parent NodeID creates a new Node in the Export XML file. The action can use Smart Parameters.

The first xml_NewNode action in a rule set creates the root node of the XML file using the new child NodeID. The parent NodeID must be blank.

Example

```
xml_SetExportPath("C:\ParentDir\APT")
ml_SetFileName("@BATCHID")
xml_NewNode("B")
ml_SetAttributeValue("B,id,@BATCHID")
```

The example starts an export XML file that is named the same as the current batch ID. It has a root node B whose id is assigned the batch ID. It is expected that more rules continue to build the XML structure. The last action that was called must call xml_SaveFile to save the export file to disk.

```
xml_NewNode("ClaimsData,HCFA")
```

Duplicated NodeIDs cause the previous node of the same ID to commit to its specific parent node and is no longer available for modification. Adding a second child NodeID with the same Tag name of the root node causes the action to return **False**.

xml_SaveFile

Commits all unsaved nodes and saves the XML file to disk.

Syntax

()

Parameters

None.

Returns

True if the file is created successfully. Otherwise, **False**.

Level

Batch, Document or Page level.

Details

You must use this action to complete the creation of the export XML file.

Example

```
xml_SaveFile()
```

xml_SetAttributeValue

Assigns attributes to a specific node.

Syntax

(StrParam)

Parameters

Three comma-separated values:

1. The NodeID.
2. The attribute's name.
3. The value to be assigned to the attribute.

Smart parameters are supported.

Returns

False, if the node does not exist. Otherwise, **True**.

Level

All.

Details

Sets an attribute value within a specific node in the XML hierarchy.

Example

The following example assigns the current Page's Number field value to the Xpage node's Number attribute.

```
xml_SetAttributeValue("Xpage,Number,@P\Number")
```

The following example shows how creation of an XML for invoice line items might look.

```
xml_NewNode("LineItem,Invoice")
xml_SetAttributeValue("LineItem,id,@ID")
xml_NewNode("ItemID,LineItem")
```

```
xml_SetNodeValue("ItemID,@F\ItemID")
xml_NewNode("ItemDesc,LineItem")
xml_SetNodeValue("ItemDesc,@F\ItemDesc")
xml_NewNode("Qty,LineItem")
xml_SetNodeValue("Qty,@F\Qty")
xml_CommitNode("LineItem")
```

xml_SetExportPath

Specifies the full path to the directory that will store the export XML file.

Syntax

```
bool xml_SetExportPath(strParam)
```

Parameters

Specifies the full path to the directory that stores the export XML file. If this action is not called, the XML file is saved in the batch directory. Smart parameters are supported.

Returns

Always **True**.

Level

All.

Details

This action will only set the path. You will still need to use `xml_SetFileName` to set the name of the export file.

Example

```
xml_SetExportPath("C:\Invoice\ExportXML")
xml_SetFileName("@BatchID")
```

xml_SetFileName

Specifies the name for the export XML file, which does not include the .xml extension.

Syntax

```
bool xml_SetFileName(StrParam)
```

Parameters

The name of the XML export. Smart Parameters are supported.

Returns

Always **True**.

Level

All.

Details

This action is required to set the name of the XML Export file. This action can be called at any level, but it is usually called at the batch level. If the `xml_SetExportPath` action is not called, the file is saved in the batch folder.

Example

```
xml_SetExportPath("C:\Invoice\ExportXML")
xml_SetFileName("@BatchID")
```

xml_SetNodeValue

Sets the value of the specified node.

Syntax

()

Parameters

Two comma-separated values:

1. The NodeID.
2. The value to assign to the node.
3. The action defaults the current object's value.

Smart parameters are supported.

Returns

Always **True**.

Level

All.

Details

Assigns a value to a specific node. The action can use Smart Parameters.

Example

The following example shows how creation of an XML for invoice line items might look.

```
xml_NewNode("LineItem,Invoice")
xml_SetAttributeValue("LineItem,id,@ID")
xml_NewNode("ItemID,LineItem")
xml_SetNodeValue("ItemID,@F\ItemID")
xml_NewNode("ItemDesc,LineItem")
xml_SetNodeValue("ItemDesc,@F\ItemDesc")
xml_NewNode("Qty,LineItem")
xml_SetNodeValue("Qty,@F\Qty")
xml_CommitNode("LineItem")
```

FileIO actions

Use the FileIO actions to do various file system functions.

The FileIO actions can check for available disk space, copy or delete directories, and write file values to a specified variable.

“CheckFreeDiskSpace” on page 522

“CopyDirectory” on page 523
“CopyFile” on page 524
“DeleteDirectory” on page 525
“DeleteFile” on page 526
“GetFileSize” on page 527
“GetProfileString” on page 527
“IsDirectoryPresent” on page 529
“IsFilePresent” on page 530
“IsFileReadOnly” on page 530
“IsProfilePresent” on page 531
“RenameFile” on page 532
“SetFileReadOnly” on page 533
“SetProfileString” on page 534
“SplitFileName” on page 535

CheckFreeDiskSpace

Checks the size of the available disk space.

Member of namespace

FileIO

Syntax

```
CheckFreeDiskSpace(string DriveLetter, string Threshold, string TargetVariable)
```

Parameters

string **DriveLetter**

string **Threshold**

string **TargetVariable**

Parameters

DriveLetter: The drive letter to test for available disk space. UNC paths are not supported.

Threshold: Optional. The minimum amount of required disk space in bytes. If provided, this value must be a positive integer.

TargetVariable: Optional. The DCO variable to store the value of the available disk space.

Smart parameters are supported for each parameter.

Returns

False If the threshold is specified and the amount of free disk space is less than the specified value. If the target variable is provided but cannot be set or if the drive letter is invalid. Otherwise, **True**.

Level

Any level.

Details

Checks the available amount of free disk space in bytes.

Example

```
CheckFreeDiskSpace("D","1000","@P.FreeSpace")
```

Checks that the amount of available disk space on drive D is at least 1000 bytes. It also stores the amount of free space in the page level runtime DCO variable *FreeSpace*.

```
CheckFreeDiskSpace("D","@B.MyLimit","")
```

Only checks that the amount of available disk space on drive D is at least as much as the value specified in the batch level variable *MyLimit*.

```
CheckFreeDiskSpace("D","","@P.FreeSpace")
```

Stores only the amount of free space in the page level runtime DCO variable *FreeSpace*.

CopyDirectory

Copies a directory and its subdirectories

Member of namespace

FileIO

Syntax

```
CopyDirectory ()
```

Parameters

SourceDirectory

Type: string

DestDirectory

Type: string

Recursive

Type: bool

Parameters

- SourceDirectory : The path of the directory to copy.
- DestDirectory : The path to the destination directory.
- Recursive : **True** copies the directory files and subdirectories. **False** only copies the files in the initial directory.

Smart parameters are supported for the directory paths.

Returns

False if a failure occurs when copying a directory. If some of the files have been copied prior to the failure, those files remain. Otherwise, **True**.

Level

Any level.

Details

Copies the source directory to the target directory. If the target directory does not exist, it will be created, however the parent directory does need to exist.

Example

```
CopyDirectory("C:\MyDir1\MyDir2","C:\MyNewDir",True)
```

This example copies MyDir2 and all files and subdirectories to C:\MyNewDir.

CopyFile

Copies a file.

Member of namespace

FileIO

Syntax

```
CopyFile(string sourcefile, string targetfile, bool overwrite)
```

Parameters

string **sourcefile**

string **targetfile**

bool **overwrite**

Parameters

sourcefile: The name and path of the file to copy. Smart parameters are supported.

targetfile: The target path and file name. Smart parameters are supported.

overwrite: If the target file exists, this parameter determines whether it is overwritten. **True** overwrites the target file.

Returns

True if the file is successfully copied. Otherwise, **False**.

Level

All levels.

Details

The file is copied from one location to the specified location. If a file of the same target name exists in the target directory, the overwrite parameter determines

whether that file is overwritten by the new file. The output directory must exist for this action to succeed. You can use action `IsDirectoryPresent` to create the target directory.

If target file ends with a backslash, meaning only the target directory is specified, the same file name from the source string is used as the target file.

Because the target file name can be different from the source file, this action allows the copied file to be renamed in one step. If you want to do a "move" operation, call the `DeleteFile` action to remove the source file.

Smart parameters are supported in the source and target file name parameters. DOS * and ? wildcards are not supported.

Example

```
CopyFile("C:\MyFile.txt", "c:\temp\+@BATCHID+.txt", true)
```

This action copies `MyFile.txt` to the `temp` directory and give it a new name of the current batch ID.

```
CopyFile("@VAR(IMAGEFILE)", "c:\temp\", true)
```

This action copies the file that is specified by the variable `IMAGEFILE` and copies it to the `temp` directory.

DeleteDirectory

Deletes a directory and optionally deletes subdirectories

Member of namespace

FileIO

Syntax

```
DeleteDirectory(string Directory, bool Recursive, bool FailureReturnValue)
```

Parameters

string **Directory**

bool **Recursive**

bool **FailureReturnValue**

Parameters

Directory: The path of the directory to delete.

Recursive: True deletes the directory and all files and subdirectories. False deletes only the directory and requires that the directory is empty.

FailureReturnValue: True causes the action to always return True, even if the delete fails. False causes the action to return False if the directory delete fails.

Smart parameters are supported for the directory path.

Returns

Always returns **True** if the directory is deleted.

If a failure occurs when you delete a directory, the action returns the value of **FailureReturnValue**. This result lets you determine whether the action returns **True** or **False** on a directory failure. Depending on the application, it might make sense to ignore a delete failure so this action can be set to always return **True**.

Level

Any level.

Details

Deletes the specified directory and can optionally delete subdirectories.

Example

```
DeleteDirectory("C:\MyDir1\MyDir2",True,True)
```

This example deletes MyDir2 and all files and subdirectories. If the delete fails, the action still returns **True**.

DeleteFile

Deletes a file.

Member of namespace

FileIO

Syntax

```
DeleteFile ()
```

Parameters

filename

Type: string

Parameters:

filename: The name and path of the file to delete. Smart parameters are supported.

Returns

Always **True**.

Level

All levels.

Details

Deletes the specified file. DOS wildcards are permitted in the file name.

Example

The following example deletes the specific file.

```
DeleteFile("C:\Temp\DeleteThis.txt")
```

The following example deletes all of the files in the Temp directory.

```
DeleteFile("C:\Temp\*.*)" 
```

The following example deletes the file name that matches the current batch ID and has an extension of TXT.

```
DeleteFile("C:\Temp\@BATCHID+.txt")
```

GetFileSize

Obtains the size of a file and stores it in the specified variable.

Member of namespace

FileIO

Syntax

```
GetFileSize ()
```

Parameters

filename

Type: string

targetVariable

Type: string

Parameters

- **filename** : The name and path of the file.
- **targetVariable** : The name of the variable to store the file size. The variable level must be specified with a Smart Parameter.

Smart parameters are supported for each of the input parameters.

Returns

False if the target variable is blank. Otherwise, **True**.

Level

All levels.

Details

The size of the specified file will be stored into the specified variable. This can be helpful to test for problems, such as empty files. Subsequent actions can perform tests on the stored value and act upon it.

If the specified file is not found, the action will return **True** and a size of 0 will be stored into the variable. If any other kind of error occurs, the action will return **False** and the variable may or may not be set to 0.

Example

```
GetFileSize("C:\Temp\MyFile.txt", "@B.MyVariable")
```

This example sets the file size in the variable *MyVariable* at the batch level.

GetProfileString

Reads a key value from a settings file.

Member of namespace

FileIO

Syntax

```
GetProfileString ()
```

Parameters

filename

Type: string

section

Type: string

key

Type: string

targetVariable

Type: string

Parameters

- filename : The INI file name.
- section : The section within the file.
- key : The key within the section.
- targetVariable : The variable where to store the value. The variable level must be specified with a smart parameter.

Smart parameters are supported for each of the input parameters.

Returns

True if the settings file exists and the target variable is valid. Otherwise, **False**.

Level

All levels.

Details

Reads the value from a settings file, typically a .INI file, and stores it into the variable specified. If the variable does not exist, it will be created. If the key does not exist but the file does exist, the action will still return true and an empty string will be stored in the variable. If you need to test for a blank key value, you can use the IsProfilePresent action.

If you are reading Unicode characters from an INI file, it is required that the file is in a format that supports Unicode. If the file is not a Unicode format, the Unicode characters may appear incorrectly when read by GetProfileString or displayed in an editor. Most file editors will let you chose the type of file encoding. See the help for your specific editor.

Example

```
GetProfileString("C:\Settings.ini", "MySection", "MyKey", "@B.MyVariable")
```

The value for **MyKey** is placed into the variable *MyVariable*.

IsDirectoryPresent

Determines if the specified directory exists and optionally creates it.

Member of namespace

FileIO

Syntax

```
IsDirectoryPresent ()
```

Parameters

directoryName

Type: string

create Type: bool

testExistence

Type: bool

Parameters

- **directoryName** : The directory path to test. Smart parameters are allowed.
- **create** : Specifies if the directory should be created. **True** creates the directory, if it does not exist.
- **testExistence** : Determines if **True** should be returned if the directory exists or does not exist.

Returns

If **testExistence** is **True**, the action will return **True** if the directory exists or if the directory did not exist but was successfully created.

If **testExistence** is **False**, the action will return **True** if the directory does not exist. This allows you to perform negative tests that will return true when a directory does not exist.

If an error occurs, the action will return **False**.

Level

All levels.

Details

Checks for the existence of a directory. Depending on the input parameters, if the directory does not exist, the action creates the directory. The meaning of the return value can be changed using the **testExistence** parameter. This allows rules to continue if a directory exists or rules can continue if a directory does not exist.

Example

```
IsDirectoryPresent("c:\temp", true, true)
```

This example creates the directory C:\temp, if it does not exist, and returns **True** if the directory exists or was created successfully.

```
IsDirectoryPresent("c:\temp", true, false)
```


This example creates the directory C:\temp, if it does not exist, and returns **False** if the directory exists or was created successfully.

IsFilePresent

Determines if the specified file exists.

Member of namespace

FileIO

Syntax

```
IsFilePresent ()
```

Parameters

filename

Type: string

testExistence

Type: bool

Parameters

- filename : The file name and path. Smart parameters are allowed.
- testExistence : true tests that the file exists, false tests that the file does not exist.

Returns

If testExistence is **True**, the action will return **True** if the file exists. If testExistence is **False** the action will return **True** if the file does not exist. If an error occurs, the action will return **False**.

Level

All levels.

Details

Checks for the existence of a file. Depending on the testExistence parameter, the action can return true if a file exists or true if a file does not exist to provide flexibility when creating rules.

Example

```
IsFilePresent("C:\MyDir\MyFile.abc", false)
```

In this example, if the file does not exist, the action returns **True** and any subsequent actions are processed.

IsFileReadOnly

Tests the read only attribute of a file.

Member of namespace

FileIO

Syntax

```
IsFileReadOnly ()
```

Parameters

filename

Type: string

testForReadOnly

Type: bool

Parameters

- filename : The file name and path of the file to test. Smart parameters are allowed.
- testForReadOnly : Determines if the action should return true for read-only or read-write.

Returns

If testForReadOnly is **True**, the action returns **True** if the file is read only.

If testForReadOnly is **False**, the action returns **True** if the file is not read only.

If an error occurs, or if the file does not exist, the action returns **False**.

Level

All levels.

Details

This action will indicate if a file is set to read only.

Example

```
IsFileReadOnly("c:\mydir\myfile.txt", true)
```

if the file myfile.txt is set to read only, the action returns **True**.

```
IsFileReadOnly("c:\mydir\myfile.txt", false)
```

if the file myfile.txt is not set to read only, the action returns **True**.

IsProfilePresent

Tests that a profile exists and that a specific section and key exists within it.

Member of namespace

FileIO

Syntax

```
IsProfilePresent ()
```

Parameters

filename

Type: string

section

Type: string

key

Type: string

testExistence

Type: bool

Parameters

- filename : The INI file name.
- section : The section within the file.
- key : The key within the section.
- testExistence : true tests that the it exists, false tests that it does not exist.

The filename, section, and key parameters can accept smart parameters.

Returns

True if testExistence is true and the section and key is found within the profile and the key has an assigned value. **True** if testExistence is false and the section or key is not found within the profile or the key value is blank. Otherwise, **False**.

Level

All levels.

Details

Checks that a specific section and key exists within a settings file, typically an INI file. It does not test the value of the key, only that a value exists.

Example

```
IsProfilePresent("C:\MyDir\settings.ini", "mysection", "mykey", true)
```

In this example, if settings.ini exists and contains *mysection*, *mykey* and if *mykey* has a non-blank value, it returns **True**.

RenameFile

Renames or moves the specified file.

Member of namespace

FileIO

Syntax

```
RenameFile ()
```

Parameters

oldName

Type: string

newName

Type: string

overwrite

Type: bool

Parameters

- oldName : The source file name and path. Allows smart parameters.
- newName : The destination file name and path. Allows smart parameters.

- **overwrite** : Specifies if the destination file should be overwritten. **True** overwrites an existing file.

Returns

True if the file is successfully renamed or moved. Otherwise, **False**.

Level

All levels.

Details

Renames the specified file to the new name. If the directories for the original and new file names are different, the file will be moved to the new directory. If the **overwrite** parameter is true, the file will overwrite an existing file. If **overwrite** is false, an existing file will not be overwritten.

Example

```
RenameFile("C:\MyDir\File1.txt", "C:\MyDir\File2.txt", true)
```

This example renames the file within the same directory, overwriting any existing file called File2.txt.

```
RenameFile("C:\MyDir\File1.txt", "C:\New\File1.txt", true)
```

This example moves the file from MyDir to New, overwriting any existing file called File1.txt.

SetFileReadOnly

Sets or removes the read only attribute from a file or set of files.

Member of namespace

FileIO

Syntax

```
SetFileReadOnly ()
```

Parameters

readonly

Type: bool

filename

Type: string

Parameters

- **readonly** : true turns on the read only attribute, false turns it off.
- **filename** : The path and filename. Smart parameters and DOS wildcards are allowed.

Returns

Always **True**.

Level

All levels.

Details

Sets a the read only attribute of a file, or a group of files. The read only attribute can be set or cleared with this action. Standard DOS * and ? wildcards can be used to affect a change on multiple files at once.

Example

```
SetFileReadOnly(false, "C:\MyDir\*.*)"
```

This example sets all of the files in MyDir as read-write.

SetProfileString

Writes a value to a profile file, typically called an INI file.

Member of namespace

FileIO

Syntax

```
SetProfileString ()
```

Parameters

filename

Type: string

section

Type: string

key

Type: string

value

Type: string

Parameters

- **filename** : The profile file name.
- **section** : The section within the file.
- **key** : specifies if the destination file should be overwritten.
- **value** : The value to write. Smart parameters are supported for each of the input parameters.

Returns

True if the value is written to the profile. Otherwise, **False**.

Level

All levels.

Details

Writes a value to a settings profile, typically a file with an INI extension. The value is stored in the variable provided as a parameter. If the variable does not exist, it

will be created. If the variable does exist, the current value will be replaced with the new value. If the file does not exist, the file will be created.

If you are writing Unicode characters to the INI file, it is required that the file exist and that it is in Unicode format. If the file is not a Unicode format, the Unicode characters may appear incorrectly when read by `GetProfileString` or displayed in an editor. Most file editors will let you chose the type of file encoding. See the help for your specific editor.

Example

```
SetProfileString("C:\MyDir\config.ini", "MySection", "MyKey", "Batch+@BATCHID")
```

SplitFileName

Splits a file name into user specified variables.

Member of namespace

FileIO

Syntax

```
SplitFileName ()
```

Parameters

inputFilename

Type: string

rootPathVariable

Type: string

pathVariable

Type: string

fileVariable

Type: string

extVariable

Type: string

Parameters

- **inputFilename** : The input file name to break into its three logical parts.
- **rootPathVariable** : The name of the variable to store the root path.
- **pathVariable** : The name of the variable to store the file path.
- **fileVariable** : The name of the variable to store the file name without the extension.
- **extVariable** : The name of the variable to store the file extension.

Smart parameters are supported for each of the input parameters. For each of the parameters that accept a variable, the level must be specified with a Smart Parameter.

It is not necessary to specify a variable name for each part of the path that is split. Only destination variables for the desired values need to be specified. If you do not want to store a particular file name part, leave that parameter blank.

Returns

False if the structure of the file name or path is invalid. Otherwise, **True**. The file does not need to exist for this action to succeed.

Level

All levels.

Details

This action splits the input file name into its root path, path, file name and the extension, and place each of the parts into their own variables. The file does not need to exist.

If the file contains a file extension, the saved file extension will contain a preceding period.

Example

```
SplitFileName("C:\Dir1\Dir2\MyFile.abc", "@B.FROOT", "@B.FPATH", "@B.FNAME", "@B.FEXT")
This example splits the file and saves the information into 4 variables at the batch level:
FROOT = "C:\\"
FPATH = "C:\Dir1\Dir2\"
FNAME = "MyFile"
FEXT = ".abc"

SplitFileName("@VAR(IMAGEFILE)", "", "", "", "@B.EXT")
Obtains the value stored in the variable IMAGE file and only saves the file extension into a variable called EXT.
```

FileNetIDM actions

Use the FileNetIDM actions to upload documents into an FileNet Image Services library

The FileNetIDM actions integrate Datacap applications with the FileNet Image Services library. You run these actions to access the FileNet Image Services server, set up document attributes and folders on the server, and upload documents to the server for storage.

- "AddAllImagesToDocument" on page 537
- "AddFileToDocument" on page 537
- "AddPDFImageToDocument" on page 538
- "AddTIFImageToDocument" on page 539
- "CreateFolder" on page 539
- "FileNetDB_ADOConnect" on page 540
- "FileNETDocID_SaveAsSmartParameter" on page 540
- "FileNETDocID_SetValue" on page 541
- "GetDocuments" on page 542
- "GetTopFolders" on page 542
- "IndexProperty_ID_Component" on page 543
- "IndexProperty_ID_DateComponent" on page 543
- "IndexProperty_ID_Value" on page 544
- "IndexProperty_LeftJUSTIFY" on page 545
- "IndexProperty_RightJUSTIFY" on page 545
- "IndexProperty_SmartParameter" on page 546
- "Library_DMA_Initialize" on page 547
- "Library_DS_Initialize" on page 547

"Library_IS_Initialize" on page 548
"Library_LogIn" on page 549
"Library_LogOff" on page 549
"NewDocument" on page 550
"SaveDocToFolder" on page 551
"Upload" on page 551
"Upload_SetNumAttempts" on page 552
"UseIndexes_OFF" on page 552
"UseIndexes_ON" on page 553

AddAllImagesToDocument

Adds all Document Page object images to the Image Services document object.

Syntax

()

Parameters

None.

Returns

False if action is placed at the Batch level, if the current active FileNet document has already been committed to the Library, or if no documents exist in the batch. Otherwise, **True**.

Attention: If the action cannot access the batch's image files, the action directs the Rulerunner task to finish with a status of Aborted.

Level

Document, Page or Field levels.

Details

Assigns all images associated within a Document object (or parent Document object) of the Document Hierarchy to the new FileNet document.

This action is valid for IS libraries only. DS libraries only permit a single associated file. This action solicits information from the Rulerunner task's Page file (upload.xml, for example) as it assigns Image files representing pages linked to a Document object or child Page objects of the Document Hierarchy to the FileNet document.

The images are not yet committed to the library.

Example

```
NewDocument("1040EZtwo")  
AddAllImagesToDocument()
```

AddFileToDocument

Adds a file to the current FileNet document.

Syntax

()

Parameters

String value of the File name to add to the document and its path. Smart Parameters are supported.

Returns

False if the specified file is not found, or if the active FileNet document has already been committed to the library. Otherwise, **True**.

Attention: If the action cannot access the batch's image files, the action directs the Rulerunner task to finish with a status of Aborted.

Level

All levels.

Details

Adds any file you designate as a parameter to the current FileNet document.

If the parameter does not include a path to a folder, the action will use the path to the current Batches directory as the default. You can also designate a variable of the bound object of the Document Hierarchy as the source of path's value by using the # character followed by the variable's name. For example: *#FilePath*.

Example

```
NewDocument("1040EZtwo")  
AddTIFFImageToDocument()  
AddFileToDocument("C:\Datacap\MQSW\Process\FNLog.log")
```

This sequence assumes that Datacap logs its FileNet activities and that a resulting Log file is available for the document.

AddPDFImageToDocument

Adds a PDF image file to the new FileNet document.

Syntax

()

Parameters

None.

Returns

False if no Page component of the calling object is found, batch images cannot be accessed, or if the active FileNet document has already been committed to the library. Otherwise, **True**.

Attention: If the action cannot access the batch's image files, the action directs the Rulerunner task to finish with a status of Aborted.

Level

Field or Page level.

Details

Adds a PDF Image file associated with a Page object of the Document Hierarchy to the new FileNet document. This action works only if the PDF file is in the appropriate folder of the application's Batches directory – and has the same name as an associated page's corresponding Tiff Image file.

Example

```
NewDocument("1040EZtwo")  
AddPDFImageToDocument()
```

This sequence associates calling components bound to the PDF file with a new FileNet '1040EZtwo' document.

AddTIFImageToDocument

Adds a TIF image file to the new FileNet document.

Syntax

()

Parameters

None.

Returns

False, if no Page component of the calling object is found, batch images cannot be accessed, or if the active FileNet document is already committed to the library. Otherwise, **True**.

Attention: If the action cannot access the batch's image files, the action directs the Rulerunner task to finish with a status of Aborted.

Level

Field or Page level.

Details

Adds the Image file that is associated with a Page object of the Document Hierarchy to the new FileNet document.

If a rule that contains this action is bound to a Field object, it adds the Image file that is associated with the field's parent Page object. An Upload action must stay at any level lower than this action.

Example

```
NewDocument("1040EZtwo")  
AddTIFImageToDocument()
```

This sequence associates the calling component's bound Image file with a new FileNet 1040EZtwo document.

CreateFolder

Creates a top-level FileNet folder.

Syntax

()

Parameters

The name of the folder to create. Smart Parameters are supported.

Returns

False if the folder cannot be created. Otherwise, **True**.

Level

Document, Page or Field levels.

Details

Creates a top-level FileNet folder.

Example

```
CreateFolder("MQSW_Q601")
```

FileNetDB_ADOConnect

Establishes an ActiveX Data connection object with FileNet.

Syntax

()

Parameters

None.

Returns

False if the FileNet library is not initialized. Otherwise, **True**.

Attention: If the action encounters an error connecting to the specified database, the action directs the Rulerunner task to finish with a status of Aborted.

Level

All levels.

Details

Establishes an ActiveX Data Connection object (ADO connection) with the specified FileNet database.

Example

```
Library_IS_Initialize("DefaultIMS:Domain:FileNet")  
LibraryLogin("Admin,AdOK")  
FileNetDB_ADOConnect()
```

FileNETDocID_SaveAsSmartParameter

Assigns the ID of the FileNet document to a variable of the bound object in the Document Hierarchy.

Syntax

()

Parameters

None.

Returns

False, if no active FileNet document is found or if the active FileNet document has not been committed. Otherwise, **True**.

Level

All levels.

Details

Assigns the FileNet document's ID to the TEXT variable of the bound object of the Document Hierarchy.

Example

```
NewDocument("1040EZtwo")
AddAllImagesToDocument()
Upload()
FileNETDocID_SaveAsSmartParameter()
```

FileNETDocID_SetValue

Assigns the ID of the FileNet document to a child object of the bound object in the Document Hierarchy.

Syntax

()

Parameters

The name of the Field object. Smart Parameters are supported.

Returns

False if an active FileNet document is not found; if the active FileNet document has not been committed; or if the child Field object was not found. Otherwise, **True**.

Level

Document level, Page level or Field with child fields.

Details

Assigns the FileNet Document's ID to a child Field object of the bound Document, Page or parent Field object of the Document Hierarchy.

Example

```
NewDocument("1040EZtwo")
AddAllImagesToDocument()
Upload()
FileNETDocID_SetValue("DocID")
```

This sequence will set up a new FileNet document, commit it to the FileNet library, and assign its ID as the Text value of the specified child field value of the bound object.

GetDocuments

Logs the names of the documents in the FileNet collection.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

All levels.

Details

A utility action used to aid debugging, it logs the names of the documents in the collection. This action is used to help verify the FileNet connection and that documents have been created. It is recommended that this is used for debugging and not in a normal production environment.

Example

```
GetDocuments()
```

GetTopFolders

Lists the existing top-level folders in the log file of the task.

Syntax

()

Parameters

None.

Returns

False if the top-level folder collection cannot be located. Otherwise, **True**.

Level

All levels.

Details

Lists existing top-level FileNet folders in the current task's Log file.

Example

```
GetTopFolders()
```

IndexProperty_ID_Component

Links the property of the FileNet document to an object of the Document Hierarchy.

Syntax

()

Parameters

Comma-separated values of:

1. The name of a FileNet document's property;
2. The name(s) of one or more Document Hierarchy objects with values for the variable that will be transferred to the FileNet document's property;
3. The value of the maximum length of the subscript value.

Smart Parameters are supported.

Returns

False if the FileNet property specified is invalid, or the FileNet Property Collection does not exist. Otherwise, **True**.

Attention: If the action returns False, the action directs the Rulerunner task to finish with a status of Aborted.

Level

Page level or Field level with child Fields.

Details

This action links the property of the FileNet document to an object of the Document Hierarchy.

Example

```
IndexProperty_ID_Component("FNBatch,1040EZ,18")
IndexProperty_ID_Component("FNDoc,1040EZTwo,22")
IndexProperty_ID_Component("FNPageF,Front,12")
IndexProperty_ID_Component("FNPageB,Back,12")
IndexProperty_ID_Component("FNfldLast,Last,18")
IndexProperty_ID_Component("NFldFirst,First,18")
IndexProperty_ID_Component("NFldMI,MI,2")
```

This sequence defines seven elements of a processing index for a FileNet task. In each case, the first parameter is the name of a property of a FileNet document that has been previously assembled. The second parameter assigns the Name of a Document Hierarchy object to the FileNet document's property. The third parameter is the property's maximum length.

During task operations, runtime values for each object will become the FileNet document's Index values.

IndexProperty_ID_DateComponent

Sets up the Date element of the processing index of the FileNet task.

Syntax

()

Parameters

String consisting of four comma-separated values:

1. The name of the FileNet document's Date property
2. The name of a Document Hierarchy object with a Date property
3. The format of the Date when supplied to the FileNet document
4. The format of the Date value that is being added to the task's processing index.
See the example for acceptable Date values.

Smart Parameters are supported.

Returns

False if the FileNet property specified is invalid, or the FileNet Property Collection does not exist. Otherwise, **True**.

Attention: If the action returns **False**, the action directs the Rulerunner task to finish with a status of Aborted.

Level

Page level or Field level with child Fields.

Details

This element can supply Date information to a Date property of the FileNet document.

The parameters can use these Date formats: cc Century 20 yy Year 03 yyyy Year 2003 dd Day 29 mm Month 06 Julian Year/Day 03-145

Example

```
IndexProperty_ID_Date_Component("FNStart,1040EZ,mm/dd/yy,yyyy/mm/dd")
```

This example re-formats the value of the Batch object's Start Date and assigns it to the FNStart property of the FileNet document.

IndexProperty_ID_Value

Assigns a constant value to a particular property of a FileNet document.

Syntax

()

Parameters

String with two comma-separated values:

1. The name of the FileNet document's target property;
2. The constant's value.

Smart Parameters are supported.

Returns

False, if the FileNet property specified cannot be set. Otherwise, **True**.

Level

All levels.

Details

Assigns a constant value to a particular property of a FileNet document.

Example

The following action assigns "3" to the FNTaxQtr property of the FileNet document when the FileNet task processes the bound object of the Document Hierarchy, the object to which the action's rule applies.

```
IndexProperty_ID_Value(FNTaxQtr,3)
```

IndexProperty_LeftJUSTIFY

Left-justifies a value that is being assigned to a target property of the FileNet document.

Syntax

()

Parameters

String with two comma-separated values:

1. The name of the property of the FileNet document; and
2. The maximum size of a value.

Smart Parameters are supported.

Returns

False if the FileNet property cannot be set. Otherwise, **True**.

Level

All levels.

Details

Left-justifies a value that is being assigned to a target property of the FileNet document.

Example

```
IndexProperty_ID_Variable("FNfldData,Year+SSN+ Income+Deductions+Net,256")  
IndexProperty_LeftJUSTIFY("FNfldData,256")
```

This sequence provides the FNfldData property with a value, then formats the value before it is actually assigned to the active FileNet document.

IndexProperty_RightJUSTIFY

Right-justifies a value that is being assigned to a target property of the FileNet document.

Syntax

()

Parameters

String with two comma-separated values:

1. The name of the property of the FileNet document; and
2. The maximum size of a value.

Smart Parameters are supported.

Returns

False if the FileNet property cannot be set. Otherwise, **True**.

Level

All levels.

Details

Right-justifies a value that is being assigned to a target property of the FileNet document.

Example

```
IndexProperty_ID_Variable("FNfldData,Year+SSN+ Income+Deductions+Net,256")  
IndexProperty_RightJUSTIFY("FNfldData,256")
```

This sequence provides the FNfldData property with a value, then formats the value before it is actually assigned to the active FileNet document.

IndexProperty_SmartParameter

Assigns a constant value to a particular property of a FileNet document.

Syntax

()

Parameters

1. The name of the FileNet document's target property.
2. The value to assign to the property.
3. The length of the property value (space filled).

Smart Parameters are supported for all arguments.

Returns

False, if the FileNet property specified cannot be set. Otherwise, **True**.

Level

All levels.

Details

Assigns a constant value to a particular property of a FileNet document.

Example

The following action assigns the value of field Taxes to the FNTaxQtr property of the FileNet document.

```
IndexProperty_SmartParameter(FNTaxQtr,@P/Taxes,"")
```

The following action assigns the space filled value of page variable *LastName* to the FNNameLast property of the FileNet document, if the variable value exceeds 10 characters the value that is saved is right-truncated.

```
IndexProperty_SmartParameter(FNNameLast,@P.LastName,10)
```

Library_DMA_Initialize

Initializes but does not open the FileNet DMA library.

Syntax

()

Parameters

String value consisting of three colon-separated elements of the Library Name. Smart Parameters are supported.

Returns

False if there is a problem connecting to the FileNet DMA Library. Otherwise, **True**.

Note: If the action returns False, the action directs the Rulerunner task to finish with a status of Aborted.

Level

Any level, but the Batch level is recommended.

Details

Do not confuse the Library Name with the local FileNet Neighborhood label. In some cases, the formal three-part Library Name must be used to properly configure initialization. Please check your FileNet documentation on how to discern your formal library name.

Example

```
Library_DMA_Initialize("DMALibrary:Datacap:FileNet")
```

This action initializes but does not open the library – see Library_Login.

Library_DS_Initialize

Initializes a previously defined, active Document Services Library.

Syntax

()

Parameters

String value consisting of the three colon-separated elements of the Library Name. Smart Parameters are supported.

Returns

False if there is a problem connecting to the FileNet Document Services Library. Otherwise, **True**.

Note: If the action returns **False**, the action directs the Rulerunner task to finish with a status of Aborted.

Level

Any level, but the Batch level is recommended.

Details

Do not confuse the Library Name with the local FileNet Neighborhood label. In some cases, the formal three-part Library Name must be used to properly configure initialization. Please check your FileNet documentation for guidelines on designating a formal Library Name.

Example

```
Library_DS_Initialize("DSLibrary:Datacap:FileNet")
```

This action initializes but does not open the library – see `Library_Login`.

Library_IS_Initialize

Initializes a previously defined, active Image Services library.

Syntax

()

Parameters

String value containing the Library name. The Library name will commonly consist of three, colon-separated elements of the Library Name. In some cases, the short name can be used. Smart Parameters are supported.

Returns

False if there is a problem connecting to the FileNet Image Services Library. Otherwise, **True**.

Attention: If the action returns **False**, the action directs the Rulerunner task to finish with a status of Aborted.

Level

Any level, but the Batch level is recommended.

Details

Initializes a previously defined, active Image Services Library.

In some cases, the formal three-part Library Name should be used to properly configure initialization. However, there may be some cases where configuring the library through the IDM Configuration tool first and then passing the short name could be used. Please check your FileNet documentation for guidelines on designating a formal Library Name.

Example

```
Library_IS_Initialize("LibraryName")
```

This action initializes but does not open the library – see `Library_Login`.

Library_Login

Logs in to the Image Services library by using the user ID and Password parameter values.

Syntax

```
bool Library_Login(StrParam)
```

Parameters

String values of the User ID and Password, with a comma separator. Smart Parameters are supported.

Returns

False if an active library is not found, the parameter values are incorrect, or an error occurs while logging into the library. Otherwise, **True**.

Attention: If the action returns **False**, the action directs the Rulerunner task to finish with a status of Aborted.

Level

Any level, but the Batch level is recommended.

Details

Logs into the initialized FileNet library using the User ID and Password parameter values. You must include this action to access a library. Be sure the `Library_Login` action follows one of the `Library_Initialize` actions.

Example

```
Library_DS_Initialize("DefaultLib:Datacap:FileNet")
Library_Login("FileNet2,FN2")
```

```
Library_IS_Initialize("LibraryName")
Library_Login("@APPVAR(values/gen/ISUser),@APPVAR
(values/adv/ISPassword)")
```

```
Library_IS_Initialize("LibraryName")
Library_Login("@STRING(@APPVAR
(values/gen/ISUser),@APPVAR(values/adv/ISPassword))")
```

Library_LogOff

Closes the FileNet connection to the Image Services library.

Syntax

```
()
```

Parameters

None.

Returns

True, if the logoff was successful. Otherwise, **False**.

Level

Any level, but the Batch level is recommended.

Details

Closes the FileNet connection.

Example

```
Library_LogOff()
```

NewDocument

Sets up a new document and assigns a previously defined Document Class to it.

Syntax

()

Parameters

String value of a previously defined FileNet Document Class. Smart Parameters are supported.

Returns

False if a new document cannot be created. Otherwise, **True**.

Attention: If the action returns **False**, the action directs the Rulerunner task to finish with a status of Aborted.

Level

All Levels.

Details

Sets up a new FileNet document, and assigns the FileNet Document Class you specify as an argument to the new FileNet document.

After an Upload action commits the document to a FileNet library, FileNet immediately links a unique ID to the document.

Example

```
Library_DS_Initialize("DefaultIMS:Datacap:FileNet")  
Library_Login("FileNet2,FN2")  
NewDocument("1040EZtwo")
```

In this example, the NewDocument action instantiates a new FileNet document of Class:'1040EZtwo'. To populate the document, you'll probably follow this action with one of the AddImage actions.

SaveDocToFolder

Saves the document into an existing folder of the open FileNet library.

Syntax

()

Parameters

String value of the Folder ID, beginning with a forward slash (/) - see the example. Smart Parameters are supported.

Returns

False if there is no active FileNet document, no active FileNet library, invalid parameter, or if the active FileNet document has not been committed. Otherwise, **True**.

Attention: If the action cannot access the specified folder, the action directs the Rulerunner task to finish with a status of Aborted.

Level

All Levels.

Details

Places the committed FileNet document in an existing folder of the open FileNet library. Although the forward slash (/) character is a standard element of this action's parameter, the setup of your FileNet library may mean that the forward slash is not used. Under exceptional circumstances, this action will have this syntax – note that a forward slash does not precede the Folder ID:
SaveDocToFolder("1074a").

Example

```
Library_DS_Initialize("1040Docs")  
Library_Login("FileNet2,FN2")  
NewDocument("1040EZtwo")  
AddAllImagesToDocument()  
Upload()  
SaveDocToFolder("/1074a")
```

As the example shows, you can insert this action after adding images and successfully committing (uploading) the document to the FileNet library.

Upload

Uploads the active FileNet document to the open FileNet library.

Syntax

()

Parameters

None.

Returns

False if the Document object does not exist; if the library object is missing; if all pages were previously committed; if the active FileNet document has already been committed to the library; or the upload is unsuccessful. Otherwise, **True**.

Attention: If the active FileNet document has already been committed, or the action encounters an error, the action directs the Rulerunner task to finish with a status of Aborted.

Level

All levels.

Details

Commits the active FileNet document to the open FileNet library.

Example

```
Upload()
```

Upload_SetNumAttempts

Sets the number of attempts to complete a failed Upload action.

Syntax

```
()
```

Parameters

The number of times to retry the FileNet upload upon failure. Smart Parameters are supported.

Returns

Always **True**.

Level

All levels.

Details

If the upload action fails, it will be automatically retried. The number of retries can be controlled with this action. If this action is not called, the default value of 3 will be used.

Example

```
Upload_SetNumAttempts("5")
```

UseIndexes_OFF

Turns off the Rulerunner indexing feature.

Syntax

```
()
```

Parameters

None.

Returns

Always **True**.

Level

All levels.

Details

Turns Off a Rulerunner task's Indexing feature. Because the feature is On by default, the task will continue to generate and assign index values until a rule with this action turns it Off.

A rule with action must be applied before a new FileNet document is created.

Example

```
UserIndexes_OFF()
```

UseIndexes_ON

Allows the task to access the properties of the FileNet document and provide these values to the objects in the Document Hierarchy.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

All levels.

Details

This status allows the task to access the properties of the FileNet document, and to provide these properties with values of objects of the Document Hierarchy. **True** is the default value for using indexes.

Example

```
UserIndexes_ON()  
IndexProperty_ID_Component("FNDoc,1040EZTwo,12")
```

A task cannot define or populate indices until a rule with this action activates the Indexing feature. However, the On status is a default status, and is in effect unless a UseIndexes_OFF action turns it Off.

FileNet P8 actions

Use the FileNet P8 actions to export data to a FileNet P8 repository.

The FileNet P8 actions integrate Datacap applications with the IBM FileNet P8 repository. You run these actions to access the FileNet P8 server, set up document attributes and folders on the server, and upload documents to the server for storage.

- "FNP8_CreateFolder"
- "FNP8_Login" on page 555
- "FNP8_MultiPageDocs" on page 555
- "FNP8_SetDestinationFolder" on page 556
- "FNP8_SetDocClassId" on page 557
- "FNP8_SetDocTitle" on page 557
- "FNP8_SetFileType" on page 558
- "FNP8_SetKeyProperty" on page 558
- "FNP8_SetLocale" on page 559
- "FNP8_SetMultiValueProperty" on page 560
- "FNP8_SetProperty" on page 560
- "FNP8_SetPropertyEx" on page 561
- "FNP8_SetRetry" on page 562
- "FNP8_SetTargetClassID" on page 563
- "FNP8_SetTargetObjectID" on page 563
- "FNP8_SetTimeout" on page 564
- "FNP8_SetUploadMode" on page 564
- "FNP8_SetURL" on page 565
- "FNP8_UpdateProperties" on page 565
- "FNP8_Upload" on page 566
- "FNP8_UploadDir" on page 567

FNP8_CreateFolder

Creates a subfolder on a specified target class and object.

Syntax

```
bool FNP8_CreateFolder(StrParam)
```

Parameters

A string value or a predefined Smart Parameter variable which specifies the name of the folder to create.

The allowed predefined variables are: *@BATCHID*, *@ID*, *@STATUS*, *@TYPE*, *@VALUE*, *@JOBID*, *@JOBNAME*, *@OPERATOR*, *@STATION*, *@TASKID*, and *@TASKNAME*. Please refer to the smart parameter documentation for more information on these values.

Returns

False if the parameter cannot be parsed, the set up information is invalid, or the folder cannot be created. Otherwise, **True**.

Attention: If the action returns **False**, the action directs the Rulerunner task to finish with a status of Aborted.

Level

All levels.

Details

This action creates a subfolder on a specified target class and object. Like the Upload actions, this action must be preceded by SetURL, Login and SetTargetClassID actions.

Example

```
FNp8_SetDestinationFolder("/1040EZ/Export/")  
FNp8_CreateFolder("@BATCHID")
```

This example creates a subfolder under the \1040EZ\Export\ folder and change the destination folder to the newly created folder. If the folder is created successfully, the action adds a variable *Folder_ID* to the current DCO with the folder ID returned from FileNet Web Service.

FNp8_Login

Sets the user ID and password for login to the FileNet P8 system.

Syntax

```
bool FNp8_Login(StrParam)
```

Parameters

Two comma-separated smart parameter supported string values:

1. Login Name.
2. Password.

Returns

False, if DC_P8_Server.dll was not successfully installed on this computer, or either parameter value is missing or not a string. Otherwise, **True**.

Level

All, but generally at the Batch level.

Details

This action provides the User ID and Password to use when logging in to IBM FileNet P8.

Example

```
FNp8_Login("User1,Password1")
```

FNp8_MultiPageDocs

Sets the upload mode to create a multiple page FileNet P8 document.

Syntax

```
bool FNp8_MultiPageDocs(StrParam)
```

Parameters

Specifies whether the FileNet_MultiPageDocs action creates in IBM FileNet Content Manager a single content element for all the Datacap document pages or one content element per Datacap document page.

Returns

False if the parameter is blank or if the value of the parameter is invalid. Otherwise, **True**.

Level

All levels

Details

Set FNP8_MultiPageDocs to **True** to configure the upload actions to create one FileNet P8 document that contains multiple pages.

If the FNP8_MultiPageDocs action is set to **True**, the FNP8_Upload or FNP8_UploadDir actions place all of the pages that are listed the DCO object into a single FileNet P8 document. The default behavior for these actions is to create a FileNet P8 document for each individual page that is listed in the document DCO object.

Example:

```
FNP8_MultiPageDocs("True")  
FNP8_Upload()
```

FNP8_SetDestinationFolder

Sets the destination folder for the documents to be uploaded.

Syntax

```
bool FNP8_SetDestinationFolder(StrParam)
```

Parameters

The path to the destination FileNet P8 folder in the selected Object Store where documents are to be uploaded. For example: \1040EZ\. The default value is the root folder of the specified target object. Smart parameters are supported.

Attention: The trailing backslash is optional. Subfolders of the root folder do not need to have a trailing backslash after the name. In some cases, a main folder of the object store requires a trailing backslash, if the object store is configured for it.

Returns

False, if DC_P8_Server.dll was not successfully installed on this computer, or either parameter value is missing or not a string. Otherwise, **True**.

Level

All levels.

Details

Sets the destination folder for the documents to be uploaded. See also `FNp8_CreateFolder`.

Note: This setting can be changed by a subsequent `FNp8_CreateFolder` action. If you call it with the folder name `Unfiled Documents` that is not case-sensitive, it uploads to the FileNet special `Unfiled Documents` folder.

Example

```
FNp8_SetDestinationFolder("/1040EZ/")
```

FNp8_SetDocClassId

Sets the FileNet P8 document class for the uploaded files.

Syntax

```
bool FNp8_SetDocClassId(StrParam)
```

Parameters

String value of the Document Class ID.

Returns

False if `DC_P8_Server.dll` was not successfully installed on this computer, or if parameter is not a string. Otherwise, **True**.

Level

All levels.

Details

Sets the Document Class to be used in FileNet P8 for the documents being uploaded. If this action is not called, the default value of Document is used.

Example

```
FNp8_SetDocClassId("Document")
```

FNp8_SetDocTitle

Sets the document title for documents that you are uploading.

Syntax

```
bool FNp8_SetDocTitle(StrParam)
```

Parameters

String value of a Document Title or a predefined Smart Parameter variable. *Title* is an acceptable default parameter.

Smart parameters are supported.

Returns

False if `DC_P8_Server.dll` was not successfully installed on this computer, or if parameter is not a string. Otherwise, **True**.

Level

All levels.

Details

Sets the Document Title for documents being uploaded.

Example

```
FNp8_SetDocTitle("1040ez")  
or  
FNp8_SetDocTitle("@ID")
```

FNp8_SetFileType

Assigns the file type for the files that are uploaded.

Syntax

```
bool FNp8_SetFileType(StrParam)
```

Parameters

A string identifying the file type. If not provided, the value defaults to TIF. Valid types are tif, jpeg, jpg, jpe, gif, pdf, xls, doc, msg, docx, xlsx, and zip.

Returns

Always returns **True**.

Level

Batch or Document level.

Details

Use this action to identify the file type of the files that will be uploaded to FileNet P8.

Example

```
FNp8_SetFileType("jpg")  
FNp8_Upload()
```

FNp8_SetKeyProperty

Sets the update key to a FileNet document property and its corresponding property value.

Syntax

```
bool FNp8_SetKeyProperty(StrParam)
```

Parameters

Strings values identify a FileNet document class property and its corresponding property value. The document class property name and its value must exist on the destination FileNet P8 Object Store. The property value must also be unique because the call to the FNp8_UpdateProperties action returns only one value. Smart parameters are allowed for the second parameter.

Returns

False if either parameter is blank or if the value parameter is invalid. Otherwise, **True**.

Level

Batch, Document or Page level.

Details

This action sets a key that is used by the UpdateProperties action. The key is a FileNet document property id and its corresponding value. The UpdateProperties action uses the key to search for an existing FileNet document. If the document is found, the properties on the document are updated with the values specified in the SetProperty actions.

Example

```
FNp8_SetKeyProperty("DCKey,@DCKey")
FNp8_SetProperty("FNProperty,@SomeValue")
FNp8_SetKeyProperty("@FNProperty2,@SomeValue2")
FNp8_UpdateProperties()
```

This example uses the parameters of **FNp8_SetKeyProperty("DCKey,@DCKey")** to search for a FileNet document on an object store. If the FileNet document is found, it is assigned the values that are specified in the SetProperty action when the UpdateProperties action is called. The UpdateProperties action performs actions only on the first document that is returned. If more than one document matches the criteria that is specified in the SetKeyProperty action, only the first document is updated.

FNp8_SetLocale

Identifies the locale on the target FileNet P8 system.

Syntax

```
bool FNp8_SetLocale(StrParam)
```

Parameters

Locale value accepted by the FileNet P8 Web Service. The default value is en_US.

Locales are represented by 2-letter ISO 639 Language Codes and 2-letter ISO 3166 Country Codes separated by a _ character. For example: en_US or de_DE.

Please consult these ISO documents to determine your locale value.

Returns

False, if DC_P8_Server.dll was not successfully installed on this computer, or either parameter value is missing or not a string. Otherwise, **True**.

Level

All, but generally at the Batch level.

Details

Sets the Locale (the language and language conventions such as date format) used on the P8 server. This action is only required if the FileNet P8 repository uses a locale other than US English.

Example

```
FNPP8_SetLocale("en_US")
```

FNPP8_SetMultiValueProperty

Sets the values in a multi-value property.

Syntax

```
bool FNPP8_SetMultiValueProperty(StrParam)
```

Parameters

A comma separated string consisting of these three values:

1. Property ID
2. Property Value. Smart parameters are supported for this parameter only.
3. An optional Property Type. The default is a String. Refer to the FileNet P8 documentation for a list of Property Types.

Returns

False if the ID or Value parameters are missing or if the specified property is not a multi-value property. Otherwise **True**.

Level

All levels.

Details

This action sets the property of a FileNet P8 multi-value property. It can be called multiple times.

Example

```
FNPP8_SetDocClassId("MyFileNetClass")
FNPP8_SetDocTitle("MyFileNetClass Documents")
FNPP8_SetProperty("CustomerName,@D.CustomerName")
FNPP8_SetMultiValueProperty("InvoiceList,@D.InvoiceList")
FNPP8_SetProperty("ScanStation,@STATION")
FNPP8_SetProperty("ScanOperator,@OPERATOR")
FNPP8_SetFileType("pdf")
FNPP8_Upload()
```

FNPP8_SetProperty

Sets the designated FileNet P8 property to a specified value.

Syntax

```
bool FNPP8_SetProperty(StrParam)
```

Parameters

The following comma-separated string values:

1. Property ID: the name of an existing document property in the FileNet library (equivalent to a document index field).
2. Property Value: the value to assign to the associated Property ID.
3. Optional property type. If this parameter is not specified, the property type will default to a 'string'. Supported types are: Binary, Boolean, DateTime, Float, ID, Integer, Object and String.

Smart Parameters are supported for values.

Returns

False if either parameter is blank or if the value parameter is invalid. Otherwise **True**.

Attention: If the action returns **False**, the action directs the Rulerunner task to finish with a status of Abort. The task will also abort if more than one value is assigned to the Smart Parameter variable. If a ruleset calls the FNP8_SetProperty action more than once, using the same Property ID as the opening parameter, FileNet P8 assumes that the second parameter is multi-value and assigns that value to the property.

Level

All levels.

Details

Sets the designated FileNet property to a specified value. This is equivalent to setting an index value for a document in other document management systems.

Example

```

FNP8_SetProperty("DocumentTitle, @ID")
FNP8_Upload()

```

FNP8_SetPropertyEx

Sets the designated FileNet P8 property to a specified value.

Syntax

```
bool FNP8_SetPropertyEx(StrParam)
```

Parameters

The following comma-separated string values:

1. Property ID: the name of an existing document property in the FileNet P8 library (equivalent to a document index field).
2. Property Value: the value to assign to the associated Property ID.
3. Optional property type. If this parameter is not specified, the property type defaults to a string. Supported types are: Binary, Boolean, DateTime, Float, ID, Integer, Object and String.

Smart Parameters are supported for values.

Returns

False if either parameter is blank or if the value of the parameter is invalid. Otherwise **True**.

Attention: If the action returns **False**, the action directs the Rulerunner task to finish with a status of Abort. The task will also abort if more than one value is assigned to the Smart Parameter variable.

Important: Any ruleset that calls the FNP8_SetPropertyEx action more than once, using the same Property ID as the opening parameter, the property value will not create a multivalue property. Unlike the behavior of the FNP8_SetProperty action, it replaces the existing property value that was set up by any previous ruleset.

Level

All levels.

Details

Sets the designated FileNet property to a specified value. This is equivalent to setting an index value for a document in other document management systems.

Example

```
FNp8_SetPropertyEx("DocumentTitle, @ID")  
FNp8_Upload()
```

FNp8_SetRetry

Sets the number of automatic upload retries.

Syntax

```
bool FNp8_SetRetry(StrParam)
```

Parameters

A integer value identifies the amount of retries performed if the upload failed. Smart parameters are supported.

Returns

False if non-numeric parameter specified. Otherwise **True**.

Level

Batch, Document or Page level.

Details

Use this action to set the amount of retries for the FileNet P8 upload actions. If the upload to FileNet fails, the upload action will immediately be retried the number of times specified by the FNp8_SetRetry action.

If this action is not called prior to a FileNet P8 upload action, the default amount of retries is 0.

Example

```
 FNP8_SetRetry("3")
 NP8_Upload()
```

FNP8_SetRetry("3") causes the FNP8_Upload to initiate 3 upload attempts if the upload to FileNet fails.

FNP8_SetTargetClassID

Sets the FileNet P8 document class for uploaded documents.

Syntax

```
bool FNP8_SetTargetClassID(StrParam)
```

Parameters

Specifies the repository type. The valid values are ObjectStore and FileStore.

Returns

False, if DC_P8_Server.dll was not successfully installed on this computer, or either parameter value is missing or not a string. Otherwise, **True**.

Level

All levels.

Details

Sets the top-level repository type. If this action is not called, the default value of ObjectStore is used. FileStore is an alternative repository type.

Example

```
 FNP8_SetTargetClassID("ObjectStore")
```

FNP8_SetTargetObjectID

Sets the name of the Object Store in which documents are stored.

Syntax

```
bool FNP8_SetTargetObjectID(StrParam)
```

Parameters

String value of the Object ID. Smart parameters are supported.

Returns

False, if DC_P8_Server.dll was not successfully installed on this computer, or either parameter value is missing or not a string. Otherwise, **True**.

Level

All, but generally at the Batch level.

Details

Provides the name of the Object Store that you wish to store your documents in.

Example

```
FNFP8_SetTargetObjectID("AP_ObjectStore")
```

FNFP8_SetTimeout

Sets the timeout for the FileNet P8 web service in milliseconds.

Syntax

```
bool FNFP8_SetTimeout(StrParam)
```

Parameters

A single parameter identifying the timeout in milliseconds for the FileNet P8 Web Service. The default timeout is 600000 milliseconds (600 seconds).

Returns

False if DC_P8_Server.dll was not successfully installed on this computer, or if the parameter is not formatted as a valid integer value. Otherwise, **True**.

Level

All, but generally at the Batch level.

Details

This action sets the timeout in milliseconds for the FileNet P8 Web Service. This action should be called before the FNFP8_Upload action.

Example

```
FNFP8_SetTimeout(90000)
```

FNFP8_SetUploadMode

Sets the upload mode.

Syntax

```
bool FNFP8_SetUploadMode(StrParam)
```

Parameters

A string or Smart Parameter identifying the page level variable where file name is stored. If this action is not called the value defaults to blank and regular upload logic applied. Smart parameters are supported.

For example FNFP8_SetUploadMode("ParentImage") uploads a file with the name stored in *ParentImage* variable on the page level.

Returns

Always **True**.

Level

Batch, Document or Page level.

Details

Use this action to identify the files that will be uploaded to FileNet P8.

Example

```
FNFP8_SetUploadMode("ParentImage")
FNFP8_Upload()
```

FNFP8_SetURL

Sets the URL for the FileNet P8 Server that is used.

Syntax

```
bool FNFP8_SetURL(StrParam)
```

Parameters

A single parameter that identifies the URL for the IBM FileNet P8 Server. The IP port is typically :9080 (IBM WebSphere®) and the URL typically ends in /WSDL.

Returns

False, if DC_P8_Server.dll was not successfully installed on this computer, or if the parameter is not formatted as a valid URL. Otherwise, **True**.

Attention: This action returns **True** whether the URL is to an operating IBM FileNet P8 Server or not. If not, an error is reported when the Upload or UploadDir action runs. You can check that the URL is correct and operating by displaying it in any web browser.

Level

All, but generally at the Batch level.

Details

This action sets the URL for the IBM FileNet P8 IBM FileNet P8 Server. This action must be called before any other FileNet P8 upload actions.

Example

```
FNFP8_SetURL("http://myp8server:9080/wsi/FNCEWS40MTOM/")
```

Important: The address can be case-sensitive and the trailing forward-slash "/" must be present. To confirm that the address reaches a working IBM FileNet P8 Server, browse to the address after you add wsd1 to the end of the URL address, as in the following example, http://myp8server:9080/wsi/FNCEWS40MTOM/wsd1

FNFP8_UpdateProperties

Updates the properties of an existing FileNet P8 document by using the data that is passed into the FNFP8_SetProperty action.

Syntax

```
bool FNFP8_UpdateProperties()
```

Parameters

Uses the key specified in the FNP8_SetKeyProperty action to search for an existing document within a FileNet P8 Case Manager Object Store and updates the P8 document's properties as specified by the FNP8_SetProperty action.

Returns

True if the update is successful, **False** if the action is unable to update the document.

Level

Batch, Document or Page level.

Details

Use this action to update an existing FileNet P8 document's properties. The UpdateProperties action should only be used to update properties for an existing document. If a new document is created use the Upload action.

Example

```
FNP8_SetKeyProperty ("DCKey,@DCKey")
FNP8_SetProperty("AGIncome, @AGIncome")
FNP8_SetProperty("TaxYear,@TaxYear")
FNP8_UpdateProperties()
```

This example will first set the key document property name and value to search for a document, the next two actions indicate which properties and values that the FileNet P8 document should be updated with. The FNP8_UpdateProperties will invoke the actual update. If more than one document matches the criteria specified in the SetKeyProperty action, only the first document will be updated.

FNP8_Upload

Uploads the batch images to the FileNet P8 repository.

Syntax

```
bool FNP8_Upload()
```

Parameters

None.

Returns

False if the upload is not successful, or the action was applied to the Field level. Otherwise, **True**. If successful, each page uploaded will have a variable *Doc_ID* set to the FileNet document identifier.

Attention: The action directs the Rulerunner task to finish with a status of Aborted.

Level

Batch, Document or Page level.

Details

Uploads image files from the Datacap batch to the specified destination folder in the FileNet library.

When called at the Batch or Document level, attempts to upload a multipage TIF image file named ObjID.tif, where ObjID is the DCO Object ID of the Batch or Document. If such a file does not exist, tries to upload all page images in the Batch or Document.

When called at the Page level, uploads the image file for that page. It is important that any required properties are set before calling FNP8_Upload such as FNP8_SetDocTitle otherwise an upload error could occur.

Example

```
FNPF8_SetURL("http://server:port//wsi/FNCEWS40MTOM/")
FNPF8_Login("user,password")
FNPF8_SetLocale("en-us")
FNPF8_SetDocClassId("MyFileNetDocClass")
FNPF8_SetDocTitle("Document Title")
FNPF8_SetProperty("CustomerName,@D.CustomerName")
FNPF8_SetMultiValueProperty("InvoiceList,@D.InvoiceList")
FNPF8_SetProperty("ScanStation,@STATION")
FNPF8_SetProperty("ScanOperator,@OPERATOR")
FNPF8_SetFileType("pdf")
FNPF8_Upload()
```

FNPF8_UploadDir

Uploads all of the images in the folder to the specified destination folder.

Syntax

```
bool FNPF8_UploadDir(StrParam)
```

Parameters

Two comma-separated String values:

1. The full path of the source folder that contains the images to be uploaded. For example: C:\images.
2. A Boolean value to indicate whether images must be deleted from the source folder. **False**: images will not be deleted from the source folder after they are uploaded. **True**: images will be deleted from the source folder after they are uploaded.

Returns

False, if the upload is not successful. Otherwise, **True**.

Attention: If the action returns **False**, the action directs the Rulerunner task to finish with a status of Aborted.

Level

Batch or Document level.

Details

Uploads all images in the folder you enter as the first parameter to the specified destination folder.

This action is an alternative to FNP8_Upload if you want to upload images that are not within a Datacap batch.

Example

```
FNP8_UploadDir("C:\images,False")
```

This example leaves the images in the source folder after they are uploaded.

```
FNP8_UploadDir("C:\images,True")
```

This example deletes the images in the source folder after they are uploaded.

FingerprintMaintenance actions

Use the FingerprintMaintenance actions to delete fingerprints from the fingerprint library of the application.

The FingerprintMaintenance actions open and close connections to the fingerprint database, specify the folder that contains the fingerprints to be deleted, and deletes those fingerprints.

“CloseDatabase”

“DeleteFingerprint” on page 569

“DeleteFingerprints” on page 569

“OpenDatabase” on page 570

“SetFingerprintFolder” on page 571

CloseDatabase

Closes connection to the fingerprint database and saves the Setup DCO.

Member of namespace

FingerprintMaintenance

Syntax

```
CloseDatabase ()
```

Parameters

None.

Returns

Always **True**.

Level

Any level.

Details

Closes the open database connection and saves the Setup DCO if position information was deleted. The Setup DCO is not modified if FPXML files are used, or if no fingerprints were deleted.

Example:

```
CloseDatabase()
```

DeleteFingerprint

Deletes specified fingerprint.

Member of namespace

FingerprintMaintenance

Syntax

```
DeleteFingerprint ()
```

Parameters

ID Type: string
 ID of fingerprint to delete

Parameters

The ID of the fingerprint to delete.

Returns

Always **True**.

Level

Any level.

Details

Deletes the CCO, TIFF and FPXML files for the specified fingerprint. Also deletes the database record and position information from the DCO. A smart parameter may be used.

Example:

```
DeleteFingerprint("1002")
```

DeleteFingerprints

Deletes all fingerprints that are returned by the SQL statement in the parameter

Member of namespace

FingerprintMaintenance

Syntax

```
DeleteFingerprints ()
```

Parameters

SQL Type: string
 SQL statement to return a recordset of fingerprints to be deleted.

Parameters

SQL statement that is used to return a recordset of fingerprints that must be deleted.

Returns

Always **True**.

Level

Any level.

Details

Deletes all fingerprints that are returned by the query that is specified in the parameter. A smart parameter can be used.

Example:

```
Access - DeleteFingerprints("SELECT * FROM Template WHERE tp_LastHit < dateadd("d",-30,NOW)")
SQL - DeleteFingerprints("SELECT * FROM Template WHERE tp_LastHit < dateadd(d,-2,GETDATE())")
Oracle- DeleteFingerprints("SELECT * FROM Template WHERE tp_LastHit < TRUNC(SYSDATE) - 30")
```

OpenDatabase

Opens a connection to the fingerprint database.

Member of namespace

FingerprintMaintenance

Syntax

OpenDatabase ()

Parameters

ConnectionString

Type: string

Fingerprint database connection string.

Parameters

Fingerprint database connection string.

Returns

True, if the connection is established. **False**, if the connection is not established.

Level

Any level.

Details

Opens a connection to the fingerprint database by using the connection string provided in the parameter. A smart parameter can be used.

Example:

```
OpenDatabase("provider=microsoft.jet.oledb.4.0;
data source=c:\datacap\apt\APTFingerprint.mdb;
persist security info=false")
```

This example obtains the connection string set in the Application Manager.

```
OpenDatabase("@APPVAR(*\dco_*[1]/fingerprintconn:cs)")
```

SetFingerprintFolder

Specifies the folder containing the fingerprint files.

Member of namespace

FingerprintMaintenance

Syntax

```
SetFingerprintFolder ()
```

Parameters

Folder Type: string

Folder containing the fingerprints.

Parameters

Folder path where the fingerprints are located.

Returns

Always **True**.

Level

Any level.

Details

Specifies the folder containing the fingerprint files. A smart parameter may be used.

Example:

```
SetFingerprintFolder("C:\Datacap\APT\fingerprint")
```

This example obtains the path set in the Application Manager.

```
SetFingerprintFolder("@APPPATH(*\fingerprint)")
```

FPXML actions

Use the FPXML actions to store zone coordinates in an external XML file instead of the document hierarchy (setup DCO). These actions are useful for porting fingerprints between systems or to avoid making frequent modifications to the document hierarchy.

The FPXML actions can load zone information for a fingerprint, set the type of Details and LineItem fields, and set the location for the fingerprint XML files.

“ReadZonesFPX” on page 572

“SetDetailsAndLineitemPairFPX” on page 573

“SetDirectoryFPX” on page 574

“WriteZoneFPX” on page 575

“WriteZonesFPX” on page 575

ReadZonesFPX

Loads position information for current fingerprint

Member of namespace

FPXML

Syntax

ReadZonesFPX ()

Parameters

None.

Returns

False, if the fingerprint.xml cannot be loaded or if SetDirectoryFPX has not been called to set the location of the fingerprint XML files. Otherwise, **True**.

Level

Page or Field level.

Details

This action reads the field zone information from the fingerprint XML file associated with the fingerprint for the current page object. This action is similar to the ReadZones action in the Zones library. The primary difference is that ReadZones reads the zone information stored in the setup DCO and ReadZonesFPX reads the zone information from the associated fingerprint XML file. Typically this action is called after the CreateFields action but before field recognition. ReadZonesFPX loads position information for each node in the calling object and its children. Pre-adjusts these values based on offset information stored in the *Image_Offset* variable at any node level.

The offset value is applied to all child objects of a node where an *Image_Offset* variable is found; unless overwritten by a child node also having an **Image_Offset** value to apply.

Position information is based on the Fingerprint XML position for the parent page's fingerprint ID.

If **Enable FFXML** setting is checked in the application settings **Main** tab in the Datacap Application Manager, Datacap Studio places field zone information into a fingerprint XML file instead of placing the zone information into the applications setup DCO. One FFXML file is created per fingerprint. Depending on application needs, using fingerprint XML files might have advantages. Refer to Datacap documentation for more information about using fingerprint XML files.

If the application is using FFXML files, then use this action to load the zones.

Example:

```
SetDirectoryFPX("@APPPATH(* /fingerprint)")  
ReadZonesFPX()
```

SetDetailsAndLineitemPairFPX

Sets the type of the special Details and Lineitem fields.

Member of namespace

FPXML

Syntax

SetDetailsAndLineitemPairFPX (StrParam)

Parameters

Two comma separated parameters:

- **Detail field type:** type of the field that is the parent to the line item field, often called Details.
- **LineItem field type:** type of the line item field which is the parent of the fields to be captured, often called Lineitem.

Smart parameters are supported.

Returns

False if one of the parameters is missing. Otherwise **True**.

Level

Any level.

Details

Use this action if your page contains Details and Lineitems fields, as is common with an invoice page. The action sets the type of the special Details and Lineitem fields. These fields, unlike other field types, are handled by the ReadZonesFPX and WriteZonesFPX actions in a special way that is compatible with the Locate actions framework.

If your page has Details and Lineitems, then this action is required to identify the types so the position information is properly saved. Using SetDetailsAndLineitemPairFPX prior to calling WriteZonesFPX causes WriteZonesFPX to save line item field positions to the fingerprint file. The second parameter is the type of the line item field which is the parent of the fields to be captured and is often called Lineitem.

If your page contains Detail and Lineitem fields, this action also must be used prior to calling ReadZonesFPX.

Details are the parent object of the line item fields. Details are a collection of lineitems and each line item has a collection of fields. For example, in a typical invoice application a single lineitem can contain the fields: quantity, item number, description, unit price, and total.

Example:

```
SetDetailsAndLineitemPairFPX("Details,Lineitem")
```

```
ReadZonesFPX()
```

In this example, the details field is called "Details" and the line items are called "Lineitem".

An example DCO hierarchy may look something like this:

```
- Details
--- Lineitem
----- Qty
----- ItemID
----- ItemDesc
----- Price
----- LineTotal
```

SetDirectoryFPX

Sets the directory in which fingerprint xml files should be read and written to.

Member of namespace

FPXML

Syntax

```
SetDirectoryFPX (StrParam)
```

Parameters

A string identifying the Fingerprint directory. This is the directory that contains the FPXML files.

Smart parameters are supported.

Returns

False if the parameter is missing. Otherwise **True**.

Level

Any level.

Details

This action sets the directory from which fingerprint XML files will be written or read.

This action must be called to identify the FPXML directory before calling ReadZonesFPX or WriteZonesFPX.

Example:

```
SetDirectoryFPX("@APPPATH(* /fingerprint)")
ReadZonesFPX()
```

This example uses the @APPPATH smart parameter to obtain the directory from the application service.

```
SetDirectoryFPX("C:\Datacap\APT\fingerprint")
ReadZonesFPX()
```

This example hard codes the directory, which is fine but could make the application less portable.

WriteZoneFPX

Writes the positions of a field to Fingerprint XML

Member of namespace

FPXML

Syntax

WriteZoneFPX (StrParam)

Parameters

Field name (required only when the action is called at the page level)

Returns

False if there is there is not a fingerprint directory set, or the fingerprint ID cannot be determined, if the field in the parameter cannot be found, if called at the page level without a parameter, or if there is a problem opening or saving the fingerprint XML file. Otherwise **True**.

Level

Page and Field

Details

When called at the field level, writes position information for each node in the calling object and its children.

When called at the page level, writes position information the specified field and its sub fields.

WriteZonesFPX

Writes position information for all fields of the page

Member of namespace

FPXML

Syntax

WriteZonesFPX (StrParam)

Parameters

Three comma separated parameters:

- **Fingerprint host name:** This is the fingerprint class name and can be seen in the **Zones** tab of Datacap Studio. It is used to group or classify fingerprints. If this value is not provided, then a blank class name will be used.
- **Fingerprint host id:** This is an internal numeric value that is associated with the fingerprint class name. If a value is not provided, a blank value is used.

- **Fingerprint page type:** This is the page type that corresponds to the fingerprint for the page. If this value is not provided, it uses the page type of the current page.

Smart parameters are supported. All parameters are optional. If you provide only the second or third parameter, you must supply an empty value for the previous parameters.

Returns

False, if there is a problem saving to fingerprint XML file or if not called at the page level, if the page does not have a fingerprint assigned, or a fingerprint directory has not been set. Otherwise **True**.

Level

Page level only.

Details

This action writes out all of the field positions for the current page to the fingerprint XML file. Typically this action is used for applications that are using fingerprint XML files and new pages are dynamically zoned at runtime. After the zoning of the page has been completed, then this action is used to write out the zone information to the FPXML file, allowing subsequent pages of this type to be processed using the saved zone information.

If the fingerprint directory that had been previously specified using SetDirectoryFPX does not exist, it is created, providing that the parent directory already exists.

Fields with empty positions are not written to the fingerprint XML file.

Example:

```
SetDirectoryFPX("@APPPATH(* /fingerprint)")
WriteZonesFPX("MyFPClass,,MyPageType")
```

This example writes out the zones for the current page and sets the fingerprint class to "MyFPClass" and the page type to "MyPageType".

```
SetDirectoryFPX("@APPPATH(* /fingerprint)")
WriteZonesFPX("MyFPClass,,@P.MyType")
```

This example writes out the zones for the current page and sets the fingerprint class to "MyFPClass" and the page type to the value contained by the page variable "MyType".

Grayscale actions

Use the Grayscale action to convert grayscale TIFF images to black-and-white.

The ConvertGraytoBW action can convert color dropout forms, including medical claims, scanned in grayscale to black and white. The action categorizes each pixel in the image as either foreground (black), dropout (red / light gray), or background (white), to produce a final black and white image without the dropout color.

“ConvertGraytoBW” on page 577

ConvertGraytoBW

Converts Grayscale TIFF files to black and white TIFF files.

Syntax

()

Parameters

None.

Returns

False if a rule set with this action is bound to a Field object of the Document Hierarchy. Otherwise, **True**.

If the input image is not grayscale, the image will not be converted but the action will still return **True**.

Level

Batch, Document or Page level. If called at the batch level, all images will be converted.

If called at the document level all of the pages within the document will be converted.

If called at the page level, the single page will be converted.

Details

This action converts grayscale TIFF files into Black and White TIFF files.

The ConvertGraytoBW action is especially good at converting color dropout forms, including medical claims, scanned in grayscale to black and white. The action categorizes each pixel in the image as either foreground (black), dropout (red / light gray), or background (white), to produce a final black and white image without the dropout color.

Attention: The action renames the original grayscale image using the same base file name, but replaces the .tif filename extension with .tis.

Example

```
ConvertGraytoBW()
```

IBMCM actions

Use the IBMCM actions to upload documents into an IBM Content Manager Connector repository.

The IBM Content Manager Connector actions integrate Datacap applications with the IBM Content Manager Connector repository. You run these actions to access the IBM Content Manager Connector server, set up document attributes and folders on the server, and upload documents to the server for storage.

“IBMCM_AddPages” on page 578

“IBMCM_CreateFolder” on page 579

“IBMCM_CreateItem” on page 580

"IBMCM_DeletePages" on page 581
 "IBMCM_Logon" on page 582
 "IBMCM_ReplacePage" on page 583
 "IBMCM_SearchItem" on page 584
 "IBMCM_SetAttributeValue" on page 585
 "IBMCM_SetMimeType" on page 586
 "IBMCM_SetDestinationFolder" on page 587
 "IBMCM_StoreItemIDinDCO" on page 588
 "IBMCM_UploadDCO_DOC" on page 589
 "IBMCM_UploadDCO_Page" on page 590

IBMCM_AddPages

Adds new pages to an existing document in the IBM Content Manager repository. The new pages are appended to the existing pages in the IBM Content Manager document.

Syntax

```
bool IBMCM_AddPages(int newPage)
```

Parameters

Int **newPage**: the new page to add to the IBM Content Manager document.

Parameters

newpage is required when you call the action at the document level.

If **newpage**=0, all of the new pages in the runtime DCO are added to the existing IBM Content Manager document.

If **newpage**>0, only the new page that is specified by **newpage** is added.

Returns

True, if the new page or pages are successfully added. Otherwise, **False**.

Level

Doc and Page levels.

Details

You must retrieve the existing IBM Content Manager document from the IBM Content Manager repository before you call this action to add new pages to it.

You can use the IBMCM_SearchItem action to retrieve the existing IBM Content Manager document.

Example

```
IBMCM_SearchItem("", "A1001001A14B04B12546D00215")
IBMCM_AddPages(2)
```

This example searches the IBM Content Manager repository for the existing document with the unique ID of A1001001A14B04B12546D00215.

If the document is found, this action adds a second page (TM000002) in the runtime Document DCO to this document.

```
IBMCM_SearchItem("EmployeeID","14B04B12")  
IBMCM_AddPages(0)
```

This example searches the IBM Content Manager repository for the existing document with the unique ID of EmployeeID=14B04B12.

If the document is found, this action adds all of the pages in the runtime Document DCO object to this document.

IBMCM_CreateFolder

Creates an IBM Content Manager folder in the parent folder that is based on the specified item type.

Syntax

```
bool IBMCM_CreateFolder(string itemType, string attribute, string attributeValue,  
bool hasParent)
```

Parameters

String **itemtype**: the item type (classification) that is used to create the IBM Content Manager folder. This parameter is required.

String **attribute**: the attribute name.

String **attributeValue**: the unique attribute value or folder ID.

bool **hasParent**: if **True** the new folder has a parent folder.

Parameters

If **hasParent** is **False**, the new folder is created without a parent folder and the action ignores the **attribute** and **attributeValue** parameters.

If **hasParent** is **True**, it is expected that this folder is the child of an existing folder and that the **attribute** and **attributeValue** parameters specify the parent folder.

If **hasParent** is **True** and both **attribute** and **attributeValue** are not provided, the parent folder is set to the most recently created folder.

If the folder ID is used in the **attributeValue** parameter, leave the **attribute** parameter empty ("").

Smart parameters are supported for the string parameters.

Returns

True, if the folder is successfully created. Otherwise, **False**. The action fails if the a folder with the specified attribute or ID is not found.

Level

All levels.

Details

Creates a folder in the IBM Content Manager repository. The IBMCM_SetAttributeValue action can be called following this action to set the attributes of the newly created folder.

Example:

```
IBMCM_CreateFolder("NOINDEX","", "",False)
```

This example creates an IBM Content Manager folder that is based on the NOINDEX item type. The new folder has no parent and is placed in the root directory.

```
IBMCM_CreateFolder("NOINDEX","",  
"A1001001A14B04B12546D00215", True)
```

This example creates an IBM Content Manager folder that is based on the NOINDEX item type. The folder is placed inside the parent folder with the ID "A1001001A14B04B12546D00215".

```
IBMCM_CreateFolder(@MyItemType,@MyAttribute,  
@MyAttributeValue,True)  
IBMCM_SetAttributeValue("name,@BATCHID")
```

This example creates an IBM Content Manager folder that is based on the information that is stored in the smart parameter *@ MyItemType*. This folder is a child that is identified by the values from the batch level smart parameters *@ MyAttribute*, and *@MyAttributeValue*.

```
IBMCM_CreateFolder(@B.MyItemType,"","",False)  
IBMCM_SetAttributeValue("Name,MyFolder1")  
IBMCM_CreateFolder(@B.MyItemType,"","",True)  
IBMCM_SetAttributeValue("Name,MyFolder2")
```

This example creates the folders "MyFolder1" and "MyFolder2" based on the type that is identified by *@B.MyItemType*.

The "MyFolder1" folder has no parent folder. The parent folder of "MyFolder2" is the "MyFolder1" folder.

IBMCM_CreateItem

Creates an IBM Content Manager document that is based on a specified item type.

Member of namespace

ibmcm

Syntax

```
bool IBMCM_CreateItem(string itemtype)
```

Parameters

itemtype: Creates an IBM Content Manager document that is based on the item type.

Parameters

itemtype: a string value of a valid IBM Content Manager Item Type, for example NOINDEX. An IBM Content Manager Item Type is equivalent to a Document Class (Index Class).

Smart parameters are supported, such as *@BATCHID*, *@ID*, *@STATUS*, *@TYPE*, *@VALUE*, *@JOBID*, *@JOBNAME*, *@OPERATOR*, *@STATION*, *@TASKID*, and *@TASKNAME*. For more information, refer to the smart parameter documentation.

Returns

True If the document is successfully created. Otherwise, **False**.

Level

Document or Page level.

Details

Creates an IBM Content Manager based on the item type. This action must be called before the document and page upload actions.

Example:

```
IBMCM_CreateItem("NOINDEX")
IBMCM_SetMimeType("image/tiff")
IBMCM_UploadDC)_DOC()
```

This example creates an IBM Content Manager document that is based on the NOINDEX item type. It sets the mime type of the uploaded document, and then runs the upload action.

```
IBMCM_CreateItem(@P.name)
```

This example creates an IBM Content Manager document that is based on the value that is contained inside the Smart Parameter *@P.name* at the Page level.

Typically, there is one item for each processed document that is represented by a Document object of the Document Hierarchy, or for a processed page that is represented by a Page object.

IBMCM_DeletePages

Deletes pages from an existing document in the IBM Content Manager repository.

Syntax

```
bool IBMCM_DeletePages(int existingPage)
```

Parameters

Int **existingPage**: the existing page to delete from the document. This parameter is required.

Parameters

existingPage: required when you call this action at the Document level.

If **existingPage** = 0, all of the existing pages in the IBM Content Manager document are deleted. Otherwise, only the existing page that is specified by **existingPage** is deleted from the document.

Returns

True, if the page or pages are successfully deleted. Otherwise, **False**.

Level

All levels.

Details

You must retrieve the existing IBM Content Manager document from the IBM Content Manager repository before you call this action to delete pages from it.

The IBMCM_SearchItem action can be used to retrieve the existing IBM Content Manager document.

Important: Be careful with this destructive action. It should be used at the Batch level to make sure that it is run one time per batch. If you must use it at the Document or Page level, make sure that you set filters and conditions to keep this action from being called repeatedly.

Example

```
IBMCM_SearchItem("EmployeeID", "14B04B12")
IBMCM_DeletePages(2)
```

This example searches the IBM Content Manager repository for the existing IBM Content Manager document with the unique attribute EmployeeID=14B0B12.

If the existing IBM Content Manager document is found, it deletes the second page from the existing IBM Content Manager document.

```
IBMCM_SearchItem("EmployeeID", "14B04B12")
IBMCM_DeletePages(0)
```

This example searches the IBM Content Manager repository for the existing IBM Content Manager document with the unique attribute EmployeeID=14B0B12.

If the existing IBM Content Manager document is found, it deletes all of the pages from the existing IBM Content Manager document.

IBMCM_Logon

Logs on to an IBM Content Manager Server.

Member of namespace

ibmcm

Syntax

```
bool IBMCM_Logon(string connectioninfo)
```

Parameters

connectioninfo

Type: string

A comma-separated string consisting of three values:

1. The ID of the IBM Content Manager Server
2. A valid Content Manager User ID
3. The user's Password

Smart parameters are supported for each parameter.

Returns

True if the logon succeeds. **False** if the logon is unsuccessful. The logon is unsuccessful if the action cannot find the specified server, or if the user ID or password is invalid.

Level

Any level. Usually at the Batch level.

Details

Performs the logon to the IBM Content Manager system. This action must be called before the actions that communicate with the IBM Content Manager repository. Connectivity, based on the requirements of the IBM Content Manager Server, must be setup on the machine that runs the Datacap rules.

Example:

```
IBMCM_Logon("ibmcmsserver,user1,password")
IBMCM_Logon("ibmcmsserver,user1,+@APPVAR(values/adv/MyPassword)")
```

This example uses the smart parameter @APPVAR to get the password from the advanced value section of the Application Manager. The custom value name is "MyPassword".

IBMCM_ReplacePage

Replaces a page in an existing IBM Content Manager document with a new page in the runtime DCO hierarchy.

Syntax

```
bool IBMCM_ReplacePage(int existingPage, int newPage)
```

Parameters

Int **existingPage**: the existing page in the IBM Content Manager document.

Int **newPage**: the new page in the DCO hierarchy.

Parameters

When you call this action at the Document level, both **existingPage** and **newPage** parameters are required.

When you call this action at the Page level, the **newPage** parameter is ignored.

Returns

True, if the existing page in the IBM Content Manager document is replaced with the new page in the runtime DCO. Otherwise, **False**.

Level

Document and Page level.

Details

You must retrieve the existing IBM Content Manager document from the IBM Content Manager repository before you call this action.

The IBMCM_SearchItem action can be used to retrieve the existing IBM Content Manager document.

Example

```
IBMCM_SearchItem("EmployeeID", "14B04B12")  
IBMCM_ReplacePage(2,3)
```

This example searches the IBM Content Manager repository for the existing IBM Content Manager document with the unique attribute EmployeeID=14B0B12.

If the existing IBM Content Manager document is found, it replaces the second page in the existing IBM Content Manager document with the third page (TM000003) in the runtime Document DCO.

IBMCM_SearchItem

Searches for the existing item in the IBM Content Manager repository.

Syntax

```
bool IBMCM_SearchItem (string attribute, string attributeValue)
```

Parameters

String **attribute**: the attribute name.

String **attributeValue**: the unique attribute value or folder ID.

Parameters

Both **attribute** and **attributeValue** are required parameters. If folder ID is used in the second parameter, leave the first parameter empty ("").

Smart parameters are supported.

Returns

True, if the item is found. Otherwise, **False**.

Level

All levels.

Details

Searches for the existing item in the IBM Content Manager repository that matches the specified attribute and value. If an item is found, the current item is set to it. Otherwise the current item is set to NULL.

IBMCM_SearchItem is used to retrieve an existing IBM Content Manager item before calling other actions such as IBMCM_AddPages, IBMCM_DeletePages, IBMCM_ReplacePage, and IBMCM_SetAttributeValue to update the attributes of the item or the contents of the item.

Example

```
IBMCM_SearchItem("Department", "Human Resource")
IBMCM_SetAttributeValue("Department", "Operations")
```

This example searches for an item with the attribute name *Department* and the attribute value *Human Resources*. It then changes the attribute value to *Operations*.

```
IBMCM_SearchItem ("", "A1001001A14B04B12546D00215")
```

This example searches for an item with the ID equal to *A1001001A14B04B12546D00215*.

IBMCM_SetAttributeValue

Sets the attribute value on an IBM Content Manager document or folder.

Member of namespace

ibmcm

Syntax

```
IBMCM_SetAttributeValue(string attributesvalues)
```

Parameters

String **attributesvalues**: Sets the attribute value on an IBM Content Manager document or folder.

Parameters

attributesvalues is a string value of two comma separated variables:

Name: The name of the variable to set.

Value: The value of the variable to set.

A IBM Content Manager Item Type is equivalent to a Document Class (Index Class).

Smart parameters are supported, such as @BATCHID, @ID, @STATUS, @TYPE, @VALUE, @JOBID, @JOBNAME, @OPERATOR, @STATION, @TASKID, and @TASKNAME. Refer to the smart parameter documentation for more information and usage.

Returns

True if the document is successfully created. Otherwise, **False**.

Level

All levels.

Details

Sets the attribute value on a IBM Content Manager document or folder. The specified attribute must already be defined on the IBM Content Manager server. The attribute can be applied to a document or a folder. This action must follow a

previous action that created the new folder or that identifies a document. This action can be called multiple times to set multiple processes.

Example:

```
IBMCM_CreateFolder("MyItemType")
IBMCM_SetAttributeValue("Name,@BATCHID")
```

This example creates an IBM Content Manager folder and sets the *Name* property of the folder to the ID of the current batch.

```
IBMCM_CreateItem("@B.name")
IBMCM_SetAttributeValue("Author,@D.TheAuthor")
IBMCM_SetAttributeValue("Date,@DATE")
```

When running on the document level, this example creates a new document by using the batch level value that is stored in the DCO variable *Name* and sets the *Author* attribute to the value that is stored in the DCO document level variable named *TheAuthor*. Then it stores the current date in the *Date* property.

```
IBMCM_SearchItem("employeeID","3F1234D")
IBMCM_SetAttributeValue("FirstName,Thomas")
```

This example searches the IBM Content Manager repository for an existing IBM Content Manager document with the *EmployeeID* that is 3F1234D. If the existing IBM Content Manager document is found, then its attribute *FirstName* is set to *Thomas*.

IBMCM_SetMimeType

Sets the mime type for the upload document.

Syntax

```
bool IBMCM_SetMimeType(string mimeType)
```

Parameters

String **mimeType**: the mime type.

Parameters

Smart parameter is supported.

Returns

True, if the mime type is successfully set. Otherwise, **False**.

Level

All levels.

Details

Set the mime type for the upload document. This action is needed when you are uploading non-tiff files.

When you call this action at the batch or document level, the same mime type is applied to all of the files in the batch or document DCO.

This action must be called before the document and page upload actions.

Table 91. Supported mime types

MIME Types	File Extensions
application/msword	dod, dot, rtf
application/octet-stream	bin, dms, lha, lzh, exe, class
application/pdf	pdf
application/rtf	rtf
application/lotus-1-2-3	123, wk4, wk3, wk1, wks, wg1
application/lotus-freelance	prz, pre
application/vnd.ms-excel	xls, xlt, xlm, xld, xla, xlc, xlw, xll
application/vnd.ms-powerpoint	ppt, pot, ppa, pps, pwz
application/vnd.visio	vsd
audio/basic	au, snd, ulw
audio/mpeg	mpeg, mpg, m1s, m1a, mp2, mp3, mpm, mpa, mpga
audio/x-aiff	aif, aiff, aifc
audio/x-midi	midi, mid, smf, kar
audio/x-wav	wav
image/bmp	bmp, dib
image/gif	gif
image/jpeg	jpeg, jpg, jpe, jfif, pjpeg, pip
image/tiff	tiff, tif
text/html	html, htm, shtml, plg
text/plain	txt, text
text/xml	xml, dtd
video/mpeg	mpeg, mpg, mpe, m1s, m1v, m1a, m75, m15, mp2
video/quicktime	mov, qt

Example:

```
IBMCM_SetMimeType("image/tiff")
```

This example sets the mime type to "image/tiff".

IBMCM_SetDestinationFolder

Sets the destination folder for uploading images to the IBM Content Manager repository.

Syntax

```
bool IBMCM_SetDestinationFolder(string attribute, string attributeValue)
```

Parameters

String **attribute**: the attribute name.

String **attributeValue**: the unique attribute value or folder ID.

Parameters

If both parameters are not provided, the destination folder is set to the most recently created folder.

If the folder ID is used in the second parameter, leave the parameter empty ("").

Smart parameters are supported.

Returns

True, if the destination folder is successfully set. Otherwise, **False**. The action fails if a folder with the specified attribute or ID is not found.

Level

All levels.

Details

Sets the upload destination folder in the IBM Content Manager repository. To set the destination to a newly created folder, first create the folder by using the IBMCM_CreateFolder action. Then, call the IBMCM_SetDestinationFolder action with empty ("") parameters.

Example:

```
IBMCM_SetDestinationFolder("Department","Human Resource")
IBMCM_CreateItem("NOINDEX")
IBMCM_SetMimeType("image/tif")
IBMCM_UploadDCO_DOC()
```

This example sets the destination folder to the folder with the attribute name "Department" and the attribute value "Human Resource" to upload images to it.

```
IBMCM_SetDestinationFolder("", "A1001001A14B04B12546D00215")
IBMCM_CreateItem("NOINDEX")
IBMCM_SetMimeType("image/tif")
IBMCM_UploadDCO_DOC()
```

This example sets the destination folder to folder with the ID "A1001001A14B04B12546D00215" to upload images to it.

```
IBMCM_CreateFolder("NOINDEX","", "A1001001A14B04B12546D00215", true)
IBMCM_SetDestinationFolder("", "")
IBMCM_CreateItem("NOINDEX")
IBMCM_SetMimeType("image/tif")
IBMCM_UploadDCO_DOC()
```

This example sets the destination folder to the most currently created folder to upload images to it.

IBMCM_StoreItemIDinDCO

Stores the Item ID of the most recently created folder or the most recently uploaded IBM Content Manager item into a variable of the current object of the Document Hierarchy.

Member of namespace

ibmcm

Syntax

```
bool IBMCM_StoreItemIDinDCO(string itemID)
```

Parameters

String **itemID**: sets the attribute value on an IBM Content Manager document or folder.

Returns

True, if the Item ID is returned successfully. Otherwise, **False**.

Level

All levels.

Details

Stores the Item ID of the most recently created folder or the most recently uploaded IBM Content Manager document into a variable of the current object of the Document Hierarchy. If the variable does not exist, it is created on the current DCO Hierarchy object.

It might be useful to store the item ID if the object is referenced in the following action, such as setting the upload directory.

Example:

```
IBMCM_CreateFolder("NOINDEX","", "",False)
IBMCM_StoreItemIDinDCO("ItemID")
```

This example stores the ID of the new IBM Content Manager folder in a variable that is called *ItemID* in the current object of the DCO Hierarchy.

```
IBMCM_CreateItem("NOINDEX")
IBMCM_MimeType("image/tiff")
IBMCM_UploadDCO_DOC()
IBMCM_StoreItemIDinDCO("ItemID")
```

This example stores the ID of the new IBM Content Manager document in a variable that is called *ItemID* in the current object of the DCO Hierarchy.

IBMCM_UploadDCO_DOC

Uploads the set of Images files that are associated with the current document object of the Document Hierarchy to the IBM Content Manager Server.

Member of namespace

ibmcm

Syntax

```
bool IBMCM_UploadDCO_DOC ()
```

Parameters

None

Returns

True if the document is successfully created. Otherwise, **False**.

Level

Document level

Details

Uploads the Image files that are associated with current Document object of the Document Hierarchy to IBM Content Manager Server.

Example:

```
IBMCM_Logon("cmserver,adminPWD,adminUID")
IBMCM_CreateItem("NOINDEX")
IBMCM_SetAttributeValue("USERID, @OPERATOR")
IBMCM_SetMimeType("image/tiff")
IBMCM_UploadDCO_DOC()
```

This sequence uploads the set of Images files that are associated with the current document object of the Document Hierarchy to the IBM Content Manager server. Additionally, all attributes set by using IBMCM_SetAttributeValue action are also persisted.

IBMCM_UploadDCO_Page

Uploads image files that are associated with the current Page object of the Document Hierarchy to IBM Content Manager.

Member of namespace

ibmcm

Syntax

```
bool IBMCM_UploadDCO_Page ()
```

Returns

True if the action successfully sends the image to the server. **False** if the action is unable to save the image.

Level

Page level

Details

Uploads the Image file associated with current Page object of the Document Hierarchy to the IBM Content Manager server. Additionally, all attributes set using IBMCM_SetAttributeValue are also persisted.

Example:

```
IBMCM_Logon(cmserver,adminPWD,adminUID)
IBMCM_CreateItem(NOINDEX)
IBMCM_SetAttributeValue(USERID,@OPERATOR)
IBMCM_UploadDCO_Page()
```

This sequence uploads Image files associated with the current Page object of the Document Hierarchy to IBM Content Manager, and assigns the name of the object – the value of its Type property.

ICR_C actions

Use the ICR_C actions to recognize constrained (unconnected) hand or computer printed characters. These actions use the OpenText RecoStar engine.

The ICR_C actions can recognize characters in fields, page, and zones that are configured for IRC_C recognition and store the results in a PDF file.

“EnableLoggingICR_C”

“RecognizeFieldICR_C”

“RecognizeFieldVoteICR_C” on page 592

“RecognizePageFields2CCO_ICR_C” on page 593

“RecognizePageFieldsICR_C” on page 594

“RecognizePageFieldsICR_CEx” on page 594

“RecognizePageICR_C” on page 595

“RecognizePageToPDFICR_C” on page 596

EnableLoggingICR_C

Enables or disables event logging for the ICR/C engine.

Member of namespace

icr_c

Syntax

EnableLoggingICR_C (strParam)

Parameters

A boolean value. **True** enables ICRC logging and **False** disables ICRC logging. If no parameter is passed in, it uses the default value of **True**.

Returns

True if the logging state is successfully changed. Otherwise, **False**.

Level

All levels.

Details

This action enables or disables event logging for the ICR/C engine. Logs are written to the System event log under the entry Datacap.Recognition.Recostar. This action is intended for debugging ICRC recognition problems. If this action is never called, no logging will occur.

Example:

```
EnableLoggingICR_C("true")
```

RecognizeFieldICR_C

Performs character recognition for a specific field.

Member of namespace

icr_c

Syntax

```
RecognizeFieldICR_C ()
```

Parameters

None.

Returns

False, if called at the wrong level or if there is a failure in the recognition process. Otherwise, **True**.

Level

Field level.

Details

Recognition is performed on the current field. The field must have the proper settings in the **ICR/C** tab in Datacap Studio. This field-level action recognizes characters based on the settings in the **ICR/C** tab in Datacap Studio and the value is stored in the current field object.

Important: When recognizing an area that contains hand printed characters only, a country, not language, should be selected from the **Country** dropdown in the **ICR/C** tab of Datacap Studio for optimal recognition results. If the zoned area contains both hand printed and machine printed characters, then a language, and not a country, should be selected from the **Country** dropdown in the **ICR/C** tab of Datacap Studio for optimal recognition results.

Example:

```
RecognizeFieldICR_C()
```

RecognizeFieldVoteICR_C

Recognizes characters based on results from two recognition engines.

Member of namespace

icr_c

Syntax

```
RecognizeFieldVoteICR_C ()
```

Parameters

None.

Returns

False, if called at the wrong level or if there is a failure in the recognition process. Otherwise, **True**.

Level

Field level.

Details

Voting adjusts the assigned confidence of recognized characters based on the results from two different recognition engines. This action is expected to be called after field level recognition is performed on the same field by a different engine.

When this action stores the results of recognition, it first determines if the corresponding Field object of the Document Hierarchy already contains a value. If a value is present, indicating recognition had previously been performed on the field, the action compares the existing value with the field's new recognition results - character by character. If a character's values match the existing value, the Confidence Rating for the character is raised to the maximum level.

Note that when using voting actions, the recognition results are never assigned to the field. Instead, the action changes the Confidence Ratings on the basis of results provided by the first Recognition engine. However, if there are no previous recognition results in the field when this action is called, it will perform like the RecognizeFieldICR_C action.

Example:

```
RecognizeFieldOCR_S()  
RecognizeFieldVoteICR_C()
```

This example first uses the OCR/S engine to recognize the text and store it in the field object. Then it votes on the confidence of the character by comparing it to the ICR/C engine result.

RecognizePageFields2CCO_ICR_C

Performs recognition for all of the zoned fields on a page.

Member of namespace

icr_c

Syntax

```
RecognizePageFields2CCO_ICR_C ()
```

Parameters

None.

Returns

False if called at the wrong level or if the CCO does not already exist. Otherwise, **True**.

Level

Page level.

Details

This action recognizes all of the zoned fields on a page and puts the recognition results for each field in both the Datacap Object Hierarchy and in an already existing CCO. To create a CCO, call either a Full Page OCR action or the AnalyzeImage action before this action.

Example:

```
AnalyzeImage()  
RecognizePageFields2CCO_ICR_C()
```

RecognizePageFieldsICR_C

Performs recognition on all fields that have been configured for ICR/C in Datacap Studio.

Member of namespace

icr_c

Syntax

```
RecognizePageFieldsICR_C ()
```

Parameters

None.

Returns

False, if called at the wrong level or if there is a failure in the recognition process. Otherwise, **True**.

Level

Page level.

Details

This page-level action recognizes all fields on the page that have been configured for ICR/C recognition in Datacap Studio. Individual field-level recognition actions will overwrite the results from this page-level action. This action will not recognize a zoned field if the **Skip Recognition** checkbox is selected in the **ICR/C** tab in Datacap Studio.

Important: The recognition settings must be set individually for each field being recognized. When recognizing an area that contains hand printed characters only, a country, not language, should be selected from the **Country** dropdown in the **ICR/C** tab of Datacap Studio for optimal recognition results. If the zoned area contains both hand printed and machine printed characters, then a language, and not a country, should be selected from the **Country** dropdown in the **ICR/C** tab of Datacap Studio for optimal recognition results.

Example:

```
AnalyzeImage()  
RotateImage()  
RecognizePageFieldsICR_C()
```

RecognizePageFieldsICR_CEx

Recognizes all fields on the page that have been configured for ICR/C recognition.

Member of namespace

icr_c

Syntax

RecognizePageFieldsICR_CEx ()

Parameters

None.

Returns

False, if called at the wrong level or if there is a failure in the recognition process. Otherwise, **True**.

Level

Page level.

Details

This page-level action recognizes all fields on the page that have been configured for ICR/C recognition. Individual field-level recognition actions will overwrite the results from this page-level action. This action will not recognize a zoned field if the Skip Recognition checkbox is selected in the **ICR/C** tab in Datacap Studio.

Important: The recognition settings must be set individually for each field being recognized. When recognizing an area that contains hand printed characters only, a country, not language, should be selected from the **Country** dropdown in the **ICR/C** tab of Datacap Studio for optimal recognition results. If the zoned area contains both hand printed and machine printed characters, then a language, and not a country, should be selected from the **Country** dropdown in the **ICR/C** tab of Datacap Studio for optimal recognition results.

Example:

```
RecognizePageFieldsICR_CEx()
```

RecognizePageICR_C

Performs full page recognition using the ICR/C Engine.

Member of namespace

icr_c

Syntax

RecognizePageICR_C ()

Parameters

None.

Returns

False, if called at the wrong level or if there is a failure in the recognition process. Otherwise, **True**.

Level

Page level.

Details

Performs full page character recognition on the current page based on the settings in the **ICR/C** tab in Datacap Studio. The action will recognize all characters on the page, and populate the page's CCO file with the recognition results. If a CCO file does not exist at the time this action is called, the action will create one.

Example:

```
AnalyzeImage()  
RotateImage()  
RecognizePageICR_C()
```

This sequence creates a CCO file and checks to see if rotation of the image is needed. Full-page recognition then takes place based on the settings in the **ICR/C** tab in Datacap Studio. The recognition results are stored in the page's CCO file.

RecognizePageToPDFICR_C

Performs recognition on the current page and places the results in a PDF file.

Member of namespace

icr_c

Syntax

```
RecognizePageToPDFICR_C ( )
```

Parameters

None.

Returns

False, if called at the wrong level or if there is a failure in the recognition process. Otherwise, **True**.

Level

Page level.

Details

Performs full page character recognition on the current page based on the settings in the **ICR/C** tab in Datacap Studio. The action will create a PDF that contains the original page image and the recognized text.

Example:

```
AnalyzeImage()  
RotateImage()  
RecognizePageToPDFICR_C()
```

ICR_P actions

Use the ICR_P actions to recognize the contents within zoned fields that are configured for recognition. These actions use the Parascript FieldScript for IBM Datacap recognition engine.

The ICR_P actions can create a Parascript vocabulary file, load the vocabulary, and add and delete words from the vocabulary.

“AddWord”

“DeleteWord”

“ImportCSF” on page 598

“LoadFromFile” on page 599

“NewDictionary” on page 599

“RecognizeFieldsICR_P” on page 600

“SaveToFile” on page 601

“SetPostalDBPathICR_P” on page 602

AddWord

Add a word to an existing vocabulary.

Member of namespace

ICR_P

Syntax

AddWord (StrParam)

Parameters

Two parameters to add a set of characters (word), and a weight (or priority value). Smart parameters are supported.

1. The word to add to the vocabulary file.
2. The weight or priority value of the word. Valid values are 0 - 15.

Returns

False if the word cannot be added to the vocabulary. Can occur when the vocabulary object does not exist. Otherwise, **True**.

Level

Page or Document level.

Details

Adds a word to an existing vocabulary.

Example:

```
NewDictionary
AddWord(ExampleWord,8)
SaveToFile(\\ServerName\\ParentDir\\testvoc.voc)
```

DeleteWord

Deletes a word from an existing vocabulary.

Member of namespace

ICR_P

Syntax

DeleteWord (StrParam)

Parameters

One parameter that contains the set of characters (word) to be removed from the vocabulary file. Smart parameters are supported.

Returns

False if the word cannot be removed from the vocabulary. Can occur when the word or vocabulary object does not exist. Otherwise, **True**.

Level

Page or Document level.

Details

Removes a word from an existing vocabulary.

Example:

```
NewDictionary
LoadFromFile(\\ServerName\ParentDir\testvoc.voc)
DeleteWord(ExampleWord)
SaveToFile(\\ServerName\ParentDir\testvoc.voc)
```

ImportCSF

Imports data from a comma separated text file into a Parascript vocabulary file.

Member of namespace

ICR_P

Syntax

ImportCSF (StrParam)

Parameters

Provides the UNC path and filename of the comma separated file to import. Smart parameters are supported.

Returns

False if the Parascript object does not exist, if the comma separated file does not exist, or if the comma separated file is not in the correct format. Otherwise, **True**.

Level

Page or Document level.

Details

Imports data from a comma separated file. The comma separated file must contain text in word,priority_value where:

- word is the word to be added to the vocabulary and
- priority_value (or weight) is a value ranging from 0 - 15.

Do not include a space after the comma used to separate the word and priority value.

Example:

```
ImportCSF(\\ServerName\ParentDir\testtable.csv)
```

LoadFromFile

Loads vocabulary from an existing vocabulary (.voc) file.

Member of namespace

ICR_P

Syntax

LoadFromFile (StrParam)

Parameters

Provides the UNC path and filename of the vocabulary file to load. Smart parameters are supported.

Returns

False if cannot create the object. Returns false when the vocabulary file has an invalid format, is missing, or cannot be read. Otherwise, **True**.

Level

Page or Document level.

Details

Loads a vocabulary (dictionary) from a file. If the vocabulary object does not exist, this action creates a new one.

Example:

```
LoadFromFile(\\ServerName\ParentDir\testvoc.voc)
```

NewDictionary

Creates a new Parascript vocabulary file (also called a dictionary).

Member of namespace

ICR_P

Syntax

NewDictionary ()

Parameters

None.

Returns

False if the object cannot be created. Otherwise, **True**.

Level

Page or Document level.

Details

Creates a new Parascript vocabulary object. Use when a new vocabulary (dictionary) is needed.

Example:

```
NewDictionary()  
AddWord(ExampleWord,8)  
SaveToFile(\\ServerName\\ParentDir\\testvoc.voc)
```

RecognizeFieldsICR_P

Recognizes the contents within zoned fields configured for recognition using the Parascript recognition engine. Performs recognition on all of the zoned fields and saves the recognition results automatically.

Member of namespace

ICR_P

Syntax

RecognizeFieldsICR_P ()

Parameters

None.

Returns

False if the rule containing this action is not applied to a Batch, Document, Page, or Field, or if the specified Parascript recognition engine is unavailable. Otherwise, **True**.

Level

Batch, Document, Page, or Field level. If the action is set at a level above the Field level, Parascript recognizes all underlying Fields contained in the object.

Details

The action can be part of a rule that is bound to a Batch, Document, Page, or Field object of the Document Hierarchy. The Parascript FieldScript intelligent character recognition engine (ICR/P) provides recognition for English language, machine printed, cursive, unconstrained, and lightly constrained hand printed US postal addresses that contain a zip code. The Parascript recognition engine provides high

quality postal address recognition by using an up-to-date postal database. And, it provides support for building and using vocabularies to enhance recognition results.

This action uses the recognition settings you set up in Datacap Studio on zoned fields.

Open your application in Datacap Studio, click the **Zones** tab, lock the Document Hierarchy, and click the **ICR/P Properties** tab to display the settings.

For each zoned field, set the Field Type, then select the settings appropriate to that Field Type and the data you expect in the field. In general, the following processes need to happen in an application before this action is called:

- Document was scanned (using scan task, virtual scan, etc.)
- Document was identified (Page ID, fingerprint matching, etc.)
- Fields were created for each document type (CreateFields action)
- Zones were assigned (ReadZones action)

Example:

```
SetPostalDBPathICR_P(\\ServerName\ParentDir\icrp\PostalDB\  
RecognizeFieldsICR_P()
```

SaveToFile

Saves a Parascript vocabulary object to a file.

Member of namespace

ICR_P

Syntax

SaveToFile (StrParam)

Parameters

Provides the UNC path and filename of the file to which to save the vocabulary object. Smart parameters are supported.

Returns

False if cannot open the file specified or the Parascript vocabulary (dictionary) object does not exist. Otherwise, **True**.

Level

Page or Document level.

Details

Saves a Parascript vocabulary (dictionary) object to a file. If the file already exists, it is overwritten with new data. If the file does not exist, a new file is created.

Example:

```
NewDictionary  
AddWord(ExampleWord,8)  
SaveToFile(\\ServerName\ParentDir\testvoc.voc)
```


SetPostalDBPathICR_P

Provides the UNC path to the folder containing the Parascript postal database.

Member of namespace

ICR_P

Syntax

SetPostalDBPathICR_P (StrParam)

Parameters

Path to the folder containing the postal database. Smart parameters are supported.

Returns

False if the rule containing this action is not applied to a Batch, Document, Page or a Field, or if the specified Parascript recognition engine is unavailable. Otherwise, **True**.

Level

All, but generally used at the Batch level.

Details

Sets the path to the folder containing the postal database used during Parascript (ICR/P) recognition. If you use this action to set the path to the postal database at the Page or higher level, you do not have to set the Postal Database Path property for each field.

Example:

```
SetPostalDBPathICR_P(\\ServerName\ParentDir\icrp\PostalDB\)  
RecognizeFieldsICR_P()
```

ImageConvert actions

Use the ImageConvert actions to combine image files or to convert image files to JPEG or TIFF.

The ImageConvert actions can combine multiple pages into a single document, specify both the compression format for file conversion as well as the luminance and color of the output.

“AppendAllImages” on page 603

“AppendAllImages_ByType” on page 603

“AppendImage” on page 604

“AppendImage_StartAsNew” on page 605

“ConvertToJPEG” on page 606

“ConvertToTIFF” on page 606

“SetChrominanceFactor” on page 607

“SetDeleteOriginal” on page 608

“SetGrayScale” on page 608

“SetLuminanceFactor” on page 609

“SetTIFFCompression” on page 610

AppendAllImages

Appends all of the images in the document to the first page.

Syntax

()

Parameters

None.

Returns

True, if all of the TIFF images within the document have been appended to the end of the first TIFF image. Otherwise, **False**.

Level

Document level.

Details

This action will append (concatenate) all of the images within the document to the first image of the document, creating one long image. The result of the action will modify the first image in the document to become a single continuous image like this:

- Page 1
- Page 2
- Page 3

If images are appended prior to recognition, the entire page area will be recognized and click-n-key enabled.

It is highly recommended that this action be used only documents that have a small number of pages, such as two or three pages per document. The size of the final composite image can quickly become larger than can be handled by some image viewers and possibly subsequent actions. To keep memory usage to a minimum, it is also recommended that 1-bit Black and White images are used.

Note: This action only operates on TIFF images.

Example:

```
AppendAllImages()
```

AppendAllImages_ByType

Appends all of the images of a specific type within a document.

Syntax

(StrParam)

Parameters

A string that matches the *Type* variable of the page.

Returns

True, if all of the TIFF images that match the specified type within the document have been appended to the end of the first matching TIFF image. Otherwise, **False**.

Level

Document level.

Details

This action will append (concatenate) all of the images, that are of the type specified by the input parameter, that are all within the same document to the first image of that type, creating one long image. Images are not appended across documents. Assuming you have a document where the first three pages are all the same matching type, the result of the action will modify the first image in the document to become a single continuous image like this:

- Page 1
- Page 2
- Page 3

If images are appended prior to recognition, the entire page area will be recognized and click-n-key enabled.

It is highly recommended that this action be used only documents that have a small number of pages, such as two or three pages per document. The size of the final composite image can quickly become larger than can be handled by some image viewers and possibly by subsequent actions. To keep memory usage to a minimum, it is also recommended that 1-bit Black and White images are used.

Note: This action only operates on TIFF images.

Example:

```
AppendAllImages_ByType("P0")
```

This example will append all of the images within a document that have a page type of PO.

AppendImage

Concatenates the current image to the bottom of an existing image.

Syntax

()

Parameters

None.

Returns

True, if the current page is successfully concatenated with the previous page. Otherwise, **False**.

Level

Page level.

Details

This action concatenates the image for the current page to the bottom of the previous image that is processed by a call to `AppendImage_StartAsNew` or `AppendImage`. If `AppendImage_StartAsNew` is not called, then the first image that is encountered is treated as the starting image.

If images are appended before recognition, the entire page area is recognized and click-n-key enabled.

The image must be a TIFF. See `AppendImage_StartAsNew` for details.

Example:

```
AppendImage()
```

AppendImage_StartAsNew

Sets the current page as the first page for a concatenated file.

Syntax

()

Parameters

None.

Returns

True, if the image file exists for the current page and if it is a TIFF file. Otherwise, **False**.

Level

Page level.

Details

Sets the current page as the start of a new concatenated document. This action is used with `AppendImage` to create a single image that is created by concatenating images to the bottom of the first image. `AppendImage_StartAsNew` identifies the first page and `AppendImage` identifies all subsequent pages. The result of the action modifies the first image in the document to become a single continuous image like this:

- Page 1
- Page 2
- Page 3

If images are appended before recognition, the entire page area is recognized and click-n-key enabled.

It is recommended to use this action only for documents that have a few pages, such as two or three pages per document. The size of the final composite image can quickly become larger than can be handled by some image viewers and possibly subsequent actions. To keep memory usage to a minimum, you can use 1-bit Black and White images.

Attention: This action operates only on TIFF images.

Example:

```
AppendImage_StartAsNew()
```

ConvertToJPEG

Converts the current image to a JPEG.

Syntax

()

Parameters

None.

Returns

True, if the image is converted to a JPEG. **False**, if the input image is a JPEG or if a failure occurs during the conversion.

Level

Page level.

Details

Converts the current image to a JPEG. The output file name will have a JPG extension.

Supported input file formats: BMP (1, 4, 8, or 24-bit), GIF (1, 4 or 8-bit), PNG (1, 4, 8 and 24-bit), and TIFF (1, 4, 8 and 24-bit) with compression (RLE, Group 3 fax, and Group 4 fax, Pack Bits, LZW, JPEG).

Note: Not all actions that manipulate images support the same input file formats as this action.

Example:

```
SetLuminanceFactor("24")  
SetChrominanceFactor("10")  
SetGrayScale("True")  
ConvertToJPEG()
```

ConvertToTIFF

Converts the current image to a TIFF.

Syntax

()

Parameters

None.

Returns

True, if the image is converted to a TIFF. **False**, if the input image is a TIFF or if a failure occurs during the conversion.

Level

Page Level.

Details

Converts the current image to a TIFF. The output file name will have a TIF extension.

Supported input file formats: BMP (1, 4, 8, or 24-bit), GIF (1, 4 or 8-bit), JPG or JPEG (8 and 12-bit grayscale and 24-bit color), and PNG (1, 4, 8 and 24-bit).

Note: Not all actions that manipulate images support the same input file formats as this action.

Example:

```
SetTiffCompression("3")  
ConvertToTIFF()
```

SetChrominanceFactor

Set Compression for JPEG Output Files.

Syntax

(StrParam)

Parameters

A value of 0 to 255, with a value of 0 meaning minimum compression (best quality) and 255 meaning maximum image compression. Smart Parameters are supported.

Returns

Always **True**.

Level

All levels.

Details

JPEG images are stored in a compressed format. This action controls the amount of compression that is used when you are converting an image to a JPEG image using the ConvertToJPEG action.

Data loss is the result of JPEGs ability to achieve high compression ratios. Higher-quality settings result in less compression, while lower quality settings result in higher compression. Quality versus compression is a trade-off. Adjust image compression ratio by setting the SetLuminanceFactor and SetChrominanceFactor actions. The SetLuminanceFactor action adjusts the luminance or gray scale quality, while the SetChrominanceFactor action adjusts the chrominance or color quality. Lower settings for these properties result in higher-quality images with less compression. Higher settings for these properties result in lower quality images with more compression.

If this action is not called, a default value of 10 is used.

Example:

```
SetLuminanceFactor("24")
SetChrominanceFactor("10")
SetGrayScale("True")
ConvertToJPEG()
```

SetDeleteOriginal

Controls deletion of file after conversion.

Syntax

(StrParam)

Parameters

True: After the new image is created, the original will be deleted. **False:** After the new image is created, the original will remain.

Smart Parameters are supported.

Returns

Always **True**.

Level

All levels.

Details

Use this action to control the deletion of the source image file when using the actions ConvertToJPEG and ConvertToTIFF. If this action is not called, the default value of **False** will be used, the source image will not be deleted after conversion.

Example:

```
SetDeleteOriginal("True")
ConvertToJPEG()
```

SetGrayScale

Controls the grayscale output for JPEG images.

Syntax

(StrParam)

Parameters

True: Saves the image as a grayscale image. **False:** Saves the image as color.

Smart Parameters are supported.

Returns

Always **True**.

Level

All Levels.

Details

This action will determine if a JPEG image will be output as grayscale or color. If SetGrayScale is **True**, all images will be saved as 8 bit grayscale. If SetGrayScale is **False**, color images will be saved as 24 bit color and if the original image is 1 bit black and white, the new image will be 8 bit grayscale. If this action is not called, the value will default to **False** (color).

Example:

```
SetLuminanceFactor("24")
SetChrominanceFactor("10")
SetGrayScale("True")
ConvertToJPEG()
```

SetLuminanceFactor

Sets the image luminance or grayscale quality.

Syntax

(StrParam)

Parameters

A value of 0 to 255, with a value of 0 meaning minimum compression (best quality) and 255 meaning maximum image compression. Smart Parameters are supported.

Returns

Always **True**.

Level

All levels.

Details

JPEG images are stored in a compressed format. This action controls the amount of compression that is used when you are converting an image to a JPEG image using the ConvertToJPEG action.

Data loss is the result of JPEGs ability to achieve high compression ratios. Higher-quality settings result in less compression, while lower quality settings result in higher compression. Quality versus compression is a trade-off. Adjust image compression ratio by setting the SetLuminanceFactor and SetChrominanceFactor actions. The SetLuminanceFactor action adjusts the luminance or gray scale quality, while the SetChrominanceFactor action adjusts the chrominance or color quality. Lower settings for these properties result in higher-quality images with less compression. Higher settings for these properties result in lower quality images with more compression.

If this action is not called, a default value of 24 is used.

Example:

```
SetLuminanceFactor("24")
SetChrominanceFactor("10")
SetGrayScale("True")
ConvertToJPEG()
```


SetTIFFCompression

Controls the compression format when saving a TIFF.

Syntax

(StrParam)

Parameters

One of the following values:

- 0 : No compression.
- 1 : Run length encoding (RLE).
- 2 : CCITT Group 3 fax compression.
- 3 : CCITT Group 4 fax compression.
- 4 : LZW compression.
- 5 : Apple Macintosh PackBits compression.
- 6 : JPEG compression format.
- 7 : Lossless compression standard derived from LZ77.
- 8 : CCITT Group 3 two-dimensional standard fax compression.

Smart Parameters are supported.

Returns

Always **True**.

Level

All levels.

Details

This action sets the compression type of the TIFF that is output from the action `ConvertToTIFF`. Typically Group 4 compression is used for performing recognition on images.

All of these possible output formats may not be supported by other image processing actions. If this action is not called, Group 4 compression will be used.

Example:

```
SetTiffCompression("3")
ConvertToTIFF()
```

ImageFix actions

The ImageFix actions are older versions of the DCImageFix action. Use the DCImageFix actions instead.

Email actions

Use the Email actions to import image attachments from a mail server into the current batch by using IMAP.

The Email scan action polls the server and imports image attachments until the batch reaches the specified size or until the wait time expires.

“im_abort_time” on page 611

"im_AcceptMixedAttachments" on page 612
"im_AcceptNoAttachments" on page 612
"im_done_folder" on page 613
"im_login" on page 614
"im_logout" on page 615
"im_max_docs" on page 615
"im_problem_folder" on page 616
"im_scan" on page 617
"im_SortByDate" on page 618
"im_StoreEML" on page 619
"im_types" on page 620
"im_UseSSL" on page 621
"im_wait_time" on page 621

im_abort_time

Specifies the delay time before it returns when a batch stops running.

Member of namespace

Imail

Syntax

```
bool im_abort_time(int nSecs)
```

Parameters

nSecs Type: int

Parameters

nSecs: The number of seconds to wait.

Returns

Always **True**.

Level

Batch level.

Details

The action waits the specified time before it returns when an abort occurs. This action can be useful to prevent many aborted batches due to an abort condition. For example, if the email server can become unavailable for a time, the abort timeout limits the number of aborted batches until the mail server becomes available again.

If this action is not called, the default abort time value of 30 seconds is used.

Example:

```
im_wait_time("20")
im_abort_time("60")
im_max_docs("200")
im_types("jpg,tif")
```

im_AcceptMixedAttachments

Ingest emails with selected attachment types, even if non-selected attachment types are included

Member of namespace

Email

Syntax

```
bool im_AcceptMixedAttachments(bool bMixedOK)
```

Parameters**bMixedOK**

Type: bool

A boolean value that enables ingestion of emails with both allowed and disallowed attachment types.

Parameters

True An email message with an attachment type not specified by `im_types` is ingested when it contains a required attachment.

False An email message with any disallowed attachment is rejected, which is the default process if this action is never called.

Returns

Always **True**.

Level

Batch level.

Details

If this action is never called, the presence of any attachment that is not specified by `im_types` causes `im_scan` to treat that email message as a Problem.

If this action is called with parameter **True**, messages with disallowed attachment types are ingested when they contain at least one allowed attachment.

If `im_types` is called with a blank parameter before `im_scan`, all emails are ingested, so this action has no effect.

im_AcceptNoAttachments

Ingest emails without attachments, even if attachment types are specified

Member of namespace

Email

Syntax

```
bool im_AcceptNoAttachments(bool bNoAttachOK)
```

Parameters

bNoAttachOK

Type: bool

A boolean value that enables ingestion of emails with no attachments.

Parameters

True: An email message with no attachments is ingested by im_scan.

False: An email message no attachments are rejected by im_scan, unless im_types was called with a blank parameter, which is the default behavior if this action is never called.

Returns

Always **True**.

Level

Batch level.

Details

If this action is never called, the absence of an attachment causes im_scan to treat that email message as a Problem.

If this action is called with parameter **True**, a message with no attachments are ingested.

If im_types is called with a blank parameter before im_scan, all emails are ingested, so this action has no effect.

im_done_folder

Specifies the IMAP folder for successfully imported emails.

Member of namespace

Iemail

Syntax

```
bool im_done_folder(string folder)
```

Parameters

folder Type: string

Parameters

folder: Destination IMAP folder for successfully imported emails.

Returns

Always **True**.

Level

Batch level.

Details

When an email is processed and the attachment is imported, the email is moved to the folder name specified by this action. The folder is expected to be on the root level, the same level as the inbox folder in the mail account that is specified by the `im_login` action. If the folder exists in a subfolder, then use a forward slash to separate the folder names.

If this action is not called, the default value of Done is used.

Example:

```
im_done_folder("Imported")
im_problem_folder("Failed")
```

This example changes the default names of the Done and Problem folders and also requires the Imported and Failed folders to be on the same level as the Inbox folder.

```
im_done_folder("Inbox/Imported")
im_problem_folder("Inbox/Failed")
```

This example requires the folders to be a subfolder of the Inbox folder.

im_login

Specifies the IMAP mail server URL and login credentials.

Member of namespace

Imail

Syntax

```
bool im_login(string hostname, string username, string password)
```

Parameters

hostname

Type: string

username

Type: string

password

Type: string

Parameters

- hostname: URL of IMAP mail server.
- username: Username for mail account.
- password: Password for mail account.

Smart parameters are supported for all parameters.

Returns

True If the login succeeds. Otherwise, **False**.

Level

Batch level Open event only.

Details

Connects to the mail server by using the specified account information. Specifies the mail server URL and login credentials. Login credentials are for a mail server that supports the IMAP protocol (such as MS Exchange, and many others). Call one time for each batch.

To connect on an encrypted channel by using SSL / HTTPS protocol, call `im_useSSL(True)` before `im_login`.

Example:

```
im_login("mymailserver.com","theuser","password")
```

im_logout

Disconnect from the mail server.

Member of namespace

Imail

Syntax

```
bool im_logout ()
```

Parameters

None.

Returns

Always **True**.

Level

Batch level Open or Close event only.

Details

Closes the connection to the mail server. Call one time for each batch after `im_login` and `im_scan` are completed.

Example:

```
im_logout()
```

im_max_docs

Specifies maximum number of emails to include in a single batch.

Member of namespace

Imail

Syntax

```
bool im_max_docs(int nDocs)
```

Parameters

nDocs Type: int

Parameters

nDocs: The maximum number of emails in a batch.

Returns

Always **True**.

Level

Batch level.

Details

The import of emails into the batch stops when this email limit is reached or when the maximum wait time is reached. While it is waiting for new mail to arrive, the configured mailbox is polled every 2 seconds to check for waiting mail.

If this action is not called, the default value of 100 is used. The actual amount included in the batch might be less than this maximum.

Example:

```
im_wait_time("20")
im_abort_time("60")
im_max_docs("50")
im_types("jpg,tif")
```

This example causes the scan operation to limit the number of emails in a batch to a maximum of 50.

im_problem_folder

Specifies the IMAP folder for problem emails.

Member of namespace

Imail

Syntax

```
bool im_problem_folder(string folder)
```

Parameters

folder Type: string

Parameters

folder: Destination IMAP folder for problem emails.

Returns

Always **True**.

Level

Batch level.

Details

When an email is processed and the attachment is not one of the expected types, the email is moved to the folder specified by this action.

The folder is expected to be on the root level, the same level as the inbox folder of the mail account that is specified by the `im_login` action. If the folder exists in a subfolder, then use a forward slash to separate the folder names.

If this action is not called, the default value of Problem is used.

Example:

```
im_done_folder("Imported")
im_problem_folder("Failed")
```

This example changes the default name of the Problem folder to Failed and requires the Imported and Failed folders to be on the same level as the Inbox folder.

```
im_done_folder("Inbox/Imported")
im_problem_folder("Inbox/Failed")
```

This example specifies the folder, which is a subfolder of the Inbox folder.

im_scan

Poll the specified mail server for incoming emails with image attachments.

Member of namespace

Imail

Syntax

```
bool im_scan ()
```

Parameters

None.

Returns

False If the operation fails, and pauses before it returns. Otherwise, **True**.

If no selected emails were available, the action returns **True** and pauses before it returns.

Action returns when timeout is reached, or the requested number of emails are processed.

Level

Batch level Open event only.

Details

Scans emails in the Inbox for specified attachments, imports selected emails with attachments into the batch. Call one time for each batch. A connection to the email server must be previously established by using the `im_login` action.

Each input email contains the following variables set in the document hierarchy:

- **TYPE:** Always set to "Document".
- **Subject:** The email subject.
- **Body:** The text within the email.
- **DateSent:** The sent date stamp on the email.
- **From:** The email sender.
- **User:** The email user who is specified in the `im_login` action.
- **To:** The email recipients.
- **Priority:** The state of the email importance flag.

Attachments are ingested as individual pages within the email document, unless `im_StoreEML(true)` is called before `im_scan`. With individual attachments, each attachment page has the following variables set.

- **TYPE:** Always set to "Other".
- **IMAGEFILE:** The name of the attachment as saved on disk.

The following batch level variable is created:

- **EmailCount:** The number of emails that are scanned into the batch.

It is recommended that the subject line is no longer than 78 characters because this length is a common subject line length limitation. Some systems might support even shorter lengths, truncating the subject. Testing was successful with lengths up to 255 characters. It is recommended to test your settings and use lengths appropriate for your systems.

Example:

```
im_wait_time("20")
im_abort_time("40")
im_max_docs("200")
im_types("jpg,tif")
im_scan()
```

im_SortByDate

Sort emails by received date

Member of namespace

Email

Syntax

```
bool im_SortByDate(bSort)
```

Parameters

bSort Type: bool

A boolean value that enables or disables sorting of emails before ingestion.

Parameters

True: Causes `im_scan` to sort email messages by date they are received and ingested the oldest messages first. This sorting is the default behavior if `im_SortByDate` is never called.

False: Disables sorting of email messages. This sorting greatly increases performance of `im_scan` when many emails are pending in the inbox.

Returns

Always **True**.

Level

Batch level.

Details

When you enable sorting, all pending emails are sorted and the oldest is ingested first. This process can take a significant amount of time for each batch. When the inbox might contain many emails (for example, more than 100), call `im_SortByDate(False)` to speed up the process. In this case, messages are processed in the order that the email server presents them to Datacap.

`im_StoreEML`

Store one .eml file per message, rather than one file per attachment

Member of namespace

Imail

Syntax

```
bool im_StoreEML(bool bStoreEML)
```

Parameters

`bStoreEML`

Type: bool

A boolean value that enables storing each email message as an .eml file.

Parameters

True: Store each ingested email message, formatted together with all attachments, in a single .eml file and attach it as a page to the document.

False: Add each ingested email message as a document, with associated variables, and add each attachment as a page within the document. This process is the default if `im_StoreEML` is not called.

Returns

Always **True**.

Level

Batch level.

Details

If set, the `im_scan` action creates a one page document that contains the email and attachments in an `.eml` file. No attachment pages are created and no variables are set in the document.

`im_types`

Specifies valid file extensions for image attachments.

Member of namespace

Imail

Syntax

```
bool im_types(string extensions)
```

Parameters

extensions

Type: string

Parameters

extensions: Comma-separated list of file image file extensions to import, with or without period. If the parameter is blank, all emails with or without attachments are processed.

Returns

Always **True**.

Level

Batch level.

Details

This action specifies the allowable email attachment types.

If this action is not called, the default value of `.pdf` is used.

If this action is called and no attachment types are specified, all emails and any attachments are added to the batch. Emails that are processed without attachments result in documents without any pages.

If attachment types are specified and the email contains:

- Only the specified attachment types, the email, and attachments are added to the batch
- Only unspecified attachment types, or if it contains a mix of specified and unspecified attachment types, the email is moved to the Problem folder
- No attachments, the email is moved to the Problem folder

Example:

```
im_wait_time("20")
im_abort_time("40")
im_max_docs("200")
im_types("jpg,tif")
```

im_UseSSL

Connect to IMAP Server by using an SSL encrypted channel

Member of namespace

Imail

Syntax

```
bool im_UseSSL(bool bUseSSL)
```

Parameters**bUseSSL**

Type: bool

A boolean value that enables or disables SSL communication.

Parameters

True: Use SSL to encrypt communication with the mail server.

False: Communicate with the email server without encryption, which is the default method if `im_UseSSL` is not called.

Returns

Always **True**.

Level

Batch level.

Details

When SSL is enabled, subsequent `im_login` and `im_scan` actions communicate with the email server over IMAPS protocol by using SSL encryption on port 993. When SSL is not in use, these actions communicate with the email server over IMAP protocol on port 143.

im_wait_time

Specifies the maximum time to wait for input emails for a single batch.

Member of namespace

Imail

Syntax

```
bool im_wait_time(int nSecs)
```

Parameters

nSecs Type: int

Parameters

nSecs: The maximum number of seconds to wait.

Returns

Always **True**.

Level

Batch level.

Details

The maximum time to wait for input emails for a single batch, when there is at least one pending email. Used by the `im_scan` action after the first email is processed to determine how long to wait for the batch to fill up. If no emails are pending, the `im_scan` action does not wait regardless of the wait time value.

The import of emails into the batch stops when the wait limit is reached or when the maximum email per batch is reached. While it is waiting for new mail to arrive, the configured mailbox is polled every 2 seconds to check for waiting mail. The action continues to include new files into the batch until this wait time is reached or the maximum number of emails per batch is reached.

If this action is not called, the default wait time of 5 seconds is used.

Example:

```
im_wait_time("20")
im_abort_time("60")
im_max_docs("200")
im_types("jpg,tif")
```

Imprint actions

Use the Imprint actions to imprint text over an image, or for blackout or whiteout redactions.

The Imprint actions can specify the width, font style, and font size of the imprinted text as well as whether overlay rectangles are opaque or transparent.

“AnnotateImage”

“ImPrint” on page 623

“Redact” on page 624

“RedactByRegEx” on page 624

“RedactParameters” on page 625

“SetAdjustedWidth” on page 626

“SetFontName” on page 627

“SetFontSize” on page 628

“SetOpaque” on page 628

AnnotateImage

Imprints the text that you specify onto the current page image.

Syntax

()

Parameters

string **displayText**

string **xCoordinate**

string **yCoordinate**

Parameters

Three parameters:

- **displayText** smart parameter supported string to be placed onto the image. The **displayText** can combine plain text along with smart parameters.
- **xCoordinate** The X coordinate for the starting position of the text on the image. Smart parameter supported.
- **yCoordinate** The Y coordinate for the starting position of the text on the image. Smart parameter supported.

Returns

False, if parameters are missing, or the X or Y parameters are not Numeric. Otherwise, **True**.

Level

Page level only.

Details

Imprints the text you specify onto the current page's image file. By default, the text's font size is 12, and font's style Times New Roman, with an adjusted width of 30.

Example:

```
SetFontName("Arial")
SetFontSize("10")
SetAdjustedWidth("100")
AnnotateImage("@BATCHID+ Page:+@ID","0","0")
```

This example places the Batch ID followed by Page: and the calling object ID at the top of the image using the Arial font with a point size of 10 and a width of 100.

ImPrint

Use the AnnotateImage action because this action is deprecated and is scheduled to be removed in a future release.

Syntax

(StrParam)

Parameters

Three parts, separated by semicolons. See AnnotateImage for descriptions of the parameters.

Details

***** This Action Is Deprecated ***** This action has been deprecated and is scheduled to be removed in a future release. It is recommended that you no longer use this action. Instead, use the AnnotateImage action. See the action help for details.

Redact

Use the RedactParameters action because this action is deprecated and is scheduled to be removed in a future release.

Syntax

(StrParam)

Parameters

Six comma-separated parameters:

1. The fill color to use. Must be either White, or Black.
2. Optional - The text to include in the overlay.
3. The upper left X coordinate in pixels.
4. The upper left Y coordinate in pixels.
5. The lower right X coordinate in pixels.
6. The lower right Y coordinate in pixels.

Returns

True, if the area is redacted. Otherwise, **False**.

Level

Page and Field

Details

This Action Is Deprecated - This action is deprecated and is scheduled to be removed in a future release. It is recommended that you no longer use this action. Instead, use the RedactParameters action. See the action help for details.

Deprecated Description - This action overlays a black or white rectangle on the image. A default text value can optionally be applied to the overlay. If run on the field level, the entire field is redacted and the last four parameters are ignored. If run on the page level, the last four parameters to specify the location must be included.

Example:

```
Redact("white,")  
Applies a white overlay using the current field position  
  
Redact("black,,0,0,100,100")  
Applies a black square overlay on the top left of the image
```

RedactByRegEx

Searches the entire page and redacts all occurrences of the phrase that matches the RegEx expression.

Syntax

()

Parameters

string **RegEx**

string **VariableBase**

Parameters

- **RegEx** The expression to search for on the page. Smart parameters are supported.
- **VariableBase** A variable to create in the runtime DCO to track each redaction.

Returns

Always **True**.

Level

Page Level.

Details

This action searches the entire page and redacts all occurrences of the phrase that matches the **RegEx** expression. This action must be called at a field level, however the coordinates of the field will change during the operation and will remain on the last found occurrence of the found text, so it may be necessary to use a zoned but unused field for this action.

The **VariableBase** parameter specifies a variable name to use as a history of redacted words. For each redaction, a variable of the form "VariableBasen", where n is an incrementing number, is created in the runtime DCO with the coordinates of the redacted word. For example, if the parameter was "Hello", the variables created would be named: "Hello1", "Hello2", "Hello3", etc. for each redaction. An example value of the variable: "TM000001,159,1085,242,1115".

Example:

```
RedactByRegEx("[Hh]istory", "Location")
```

This example will redact all areas on the page where the word "History" or "history" exists. The redactions will be tracked in the runtime DCO in a variable called Location0, Location1, etc.

RedactParameters

Overlays a black or white rectangle on the image to redact the current field or the specified region of a page.

Syntax

()

Parameters

string

string **FillColor**

string **NewText**

string **TopLeftX**

string **TopLeftY**

string **BottomRightX**

string **BottomRightY**

Parameters

- **FillColor** The fill color to use. Must be either White, or Black.
- **NewText** Optional - The text to include in the overlay.
- **TopLeftX** The upper left X coordinate in pixels.
- **TopLeftY** The upper left Y coordinate in pixels.
- **BottomRightX** The lower right X coordinate in pixels.
- **BottomRightY** The lower right Y coordinate in pixels.

Smart parameters are supported for all parameters.

Returns

True, if the area is redacted. Otherwise, **False**.

Level

Page and Field level.

Details

This action overlays a black, or white rectangle on the image. A default text value might optionally be applied to the overlay.

If run on the field level, the entire field is redacted and the last four X/Y parameters are ignored.

If run on the page level the last four parameters to specify the location must be provided.

Example:

```
Redact("black","", "0", "0", "100", "100")
```

Called at the page level, this applies a black square overlay on the upper left of the image.

```
Redact("White", "Hello", "", "", "", "")
```

Called at the field level, this applies a white square overlay on the coordinates for the current field, and imprint the word "Hello".

SetAdjustedWidth

Specifies the width of the imprinted text.

Syntax

(StrParam)

Parameters

Numeric value for a length adjustment of the string to be imprinted. Smart parameters are supported.

Returns

False if the parameter is not numeric. Otherwise, **True**.

Level

Page level only.

Details

This actions adjusts the maximum width of the imprinted text. It is possible that the calculation of the string length may not allow for the entire string in the specified font and point size. This is an adjustment factor which will lengthen the area for the string to imprint on the image.

This action is optional. If the action is not used, the adjusted width will default to 30. If your text is being cut off, increase the parameter value. If you use this action it must be called prior to the AnnotateImage action.

Example:

```
SetFontName("Arial")
SetFontSize("10")
SetAdjustedWidth("100")
AnnotateImage("@BATCHID+ Page:+@ID","0","0")
```

SetFontName

Specifies the font style to use for the imprinted text.

Syntax

(StrParam)

Parameters

String value of the font's name. Smart parameters are supported.

Returns

False if the parameter is missing. Otherwise **True**.

Level

Page level only.

Details

Specifies the font style that is used.

This action is optional. If not used, the font style will default to Times New Roman. If you use this action, it must be called prior to the AnnotateImage action.

Example:

```
SetFontName("Arial")
SetFontSize("10")
SetAdjustedWidth("100")
AnnotateImage("@BATCHID+ Page:+@ID","0","0")
```

SetFontSize

Specifies the font size to use for the imprinted text.

Syntax

(StrParam)

Parameters

Numeric value of the font's size. Smart parameters are supported.

Returns

False if the parameter is not Numeric. Otherwise **True**.

Level

Page level only.

Details

Specifies the font size that will be used.

This action is optional. If you do not use the action, the font size will default to 12. If you use this action, it must be called prior to the AnnotateImage action.

Example:

```
SetFontName("Arial")
SetFontSize("10")
SetAdjustedWidth("100")
AnnotateImage("@BATCHID+ Page:+@ID","0","0")
```

SetOpaque

Specifies whether the transparency of the overlay rectangles are opaque (1) or transparent (0).

Syntax

(StrParam)

Parameters

- **Integer value:** 1 or 0.
- **SetOpaque(1)** indicates full opacity / black.
- **SetOpaque(0)** results in subsequent rectangles that are white.

Smart parameters are supported.

Returns

False if the parameter is not an Integer. Otherwise, **True**.

Level

All, but generally used at the Page level.

Details

Sets the transparency of the background of the imprint text. This is the background that is added behind the imprinted text. If you use this action it must be called prior to the AnnotateImage action.

If this action is not called, then the default value of 0, white, will be used for the background of the annotation text.

Example:

```
SetOpaque("1")
AnnotateImage("@BATCHID+ Page:+@ID","0","0")
```

Intellocate actions

Use the Intellocate actions to update the existing field position information in the document hierarchy (setup DCO) or to add position information for a new fingerprint.

The Intellocate actions can generate fingerprints automatically by using a page image and the recognition zones that are specified manually during verification.

“iloc_AdjustZones”

“iloc_AssignPageType” on page 630

“iloc_SetDetailZones” on page 631

“iloc_SetZones” on page 631

“IsPageDataMissing” on page 632

iloc_AdjustZones

Updates fingerprint-specific position coordinates for Field objects in the Document Hierarchy based on the locations listed for the current source page's Data file (.xml).

Member of namespace

Intellocate

Syntax

```
iloc_AdjustZones ()
```

Parameters

None.

Returns

False if a fingerprint match has not occurred and a Template ID value has not been assigned to the current page, or if the Document Hierarchy file cannot be saved. Otherwise, **True**.

Level

Page level only.

Details

Updates fingerprint-specific position coordinates for Field objects in the Document Hierarchy based on the locations listed for the current source page's Data file (.xml).

This action is designed to be used on existing fingerprints only. It will only update fields that do not have position information. Existing fields will not be updated, even if the position information for those fields has changed.

Example:

```
iloc_AdjustZones()
```

iloc_AssignPageType

Assigns a required Page Type value to a newly created fingerprint.

Member of namespace

Intellocate

Syntax

```
iloc_AssignPageType (strParam)
```

Parameters

The value of the page type to assign to the current page. The input parameter must be one of the following:

1. A numeric value that corresponds to the Page Type.
2. A string value of the page type name.

Returns

False if the numeric value cannot be retrieved from the fingerprint database or if there is no connection to the fingerprint database. Otherwise, **True**.

Level

Page level only.

Details

Assigns a required Page Type value to a newly created fingerprint. The assigned value corresponds to the values configured in the PageType table within the fingerprint database. These values are custom defined for each application. If a string value is passed as a parameter, this action will look up the corresponding numeric value within the database.

Example:

```
Iloc_AssignPageType("2")
```

This example assigns a page type of 2 to the new fingerprint.

```
Iloc_AssignPageType("PageSeparator")
```

This example assigns a page type of "PageSeparator" to the new fingerprint.

iloc_SetDetailZones

Writes the position coordinates of a new fingerprint's Detail Line fields from a page's Data file to the Pos properties of the corresponding Detail Line Field objects of the application's Document Hierarchy.

Member of namespace

Intellocate

Syntax

```
iloc_SetDetailZones ()
```

Parameters

None.

Returns

Always **True**.

Level

Page level only.

Details

Writes the position coordinates of a new fingerprint's Detail Line fields from a page's Data file to the Pos properties of the corresponding Detail Line Field objects of the application's Document Hierarchy.

Example:

```
iloc_SetDetailZones()
```

iloc_SetZones

Writes the position coordinates of a new fingerprint's zoned fields from a page's Data file to the Pos properties of the corresponding Field objects in the Document Hierarchy file (.xml).

Member of namespace

Intellocate

Syntax

```
iloc_SetZones ()
```

Parameters

None.

Returns

False if a fingerprint match has not occurred, or if the Document Hierarchy file (.xml) cannot be saved. Otherwise, **True**.

Level

Page level only.

Details

Writes the position coordinates of a new fingerprint's zoned fields from a page's Data file to the Pos properties of the corresponding Field objects in the Document Hierarchy file (.xml).

Example:

```
iloc_SetZones()
```

IsPageDataMissing

Checks to see that the current page data exists.

Member of namespace

Intellocate

Syntax

```
IsPageDataMissing ()
```

Parameters:

None.

Returns

True if the current page does not have page data. Otherwise, **False**.

Level

Page level only.

Details

Checks to see that the current page data exists. This action does not physically check for the existence of a data file. It confirms that a valid page data is loaded into memory.

Example:

```
IsPageDataMissing()
```

Invoice actions

Use the Invoice actions to process invoices by using the Datacap Accounts Payable application.

The Invoice actions can configure date, fingerprint, format, and vendor settings on invoice pages before export.

“AddToDetailErrorMsg” on page 634

“AddToErrorMsg” on page 634

“AllMixedCase” on page 634

“AllowOnlyChars” on page 635

“AlterDatebyDay” on page 636

"CalculateNotesZone" on page 636
"CaptureOpInfo" on page 637
"CheckAndFixDecimal" on page 637
"CheckForSticky" on page 638
"CheckFreeDiskSpace" on page 638
"ClearErrorMsg" on page 639
"CreateFingerprint" on page 640
"DetailFix" on page 640
"DoMsgbox" on page 641
"ExecuteSQLBind" on page 641
"FindExportImage" on page 641
"FPXMLUsed" on page 642
"GenerateDetails" on page 643
"iloc_SetDetailSimple" on page 643
"IncrementBatchVar" on page 644
"IsChildFieldBlank" on page 644
"IsChildFieldValue" on page 645
"IsCurrentObjValue" on page 645
"IsCurrentObjVariable" on page 646
"IsFingerPrintClass" on page 646
"IsInINI" on page 647
"IsInList" on page 647
"IsMultipageDocument" on page 648
"IsSinglePageDocument" on page 648
"IsStationIDSuffix" on page 649
"IsTaskName" on page 649
"Is_InCharSet" on page 650
"Is_JobName" on page 651
"Is_JobNamePrefix" on page 651
"LoadCCOFromField" on page 652
"MovePDF" on page 652
"OpenConnection" on page 653
"ParseImageName" on page 653
"PopulateZNLineItemFieldDynamic" on page 654
"ReadFPXMLZones" on page 655
"SaveObjectVariable" on page 655
"ScanLineItemDynamic" on page 655
"SendOutlookNotification" on page 656
"SetDynamicDetailZones" on page 656
"SetPicChar" on page 657
"SetStickyNo" on page 657
"SetToDocIDMPTIFF" on page 658
"SwapImages" on page 658
"SwitchMMDD" on page 659
"UpdateFPStats" on page 659

“ValidateVendor” on page 660

“WriteErrorMessage” on page 660

AddToDetailErrorMsg

Appends extra text to the existing error message.

Syntax

()

Parameters

The error message text.

Returns

Always **False**.

Level

Field level.

Details

Appends additional text to the existing error message.

Example

```
AddToDetailErrorMsg("Description cannot be blank.")
```

AddToErrorMsg

Appends the supplied text to any existing error message string.

Syntax

()

Parameters

The error message text.

Returns

Always **False**.

Level

Field level.

Details

Appends the supplied text to any existing error message string.

Example

```
AddToErrorMsg("Vendor Number cannot be blank.")
```

AllMixedCase

Changes the value of the current field to be Title case. The first letter of each word is capitalized.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

Field level.

Details

This action will change the value of the current field to be Title case. The first letter of each word will be capitalized.

Example

```
AllMixedCase()
```

If the field in this example is "hello, i MUST be going.", the text will be changed to "Hello, I Must Be Going."

AllowOnlyChars

Matches the field characters with the allowed set of characters. When the field contains characters that are not in the allowed set, the characters are deleted from the field. The comparison is case-sensitive.

Syntax

()

Parameters

A list of all of the characters that are allowed in the field.

Returns

False if this action is called at the wrong level. Otherwise, **True**.

Level

Field Level.

Details

This action will match the field characters with the allowed set of characters. If the field contains characters that are not in the allowed set, the characters will be deleted from the field. The comparison is case sensitive.

Example

```
CheckAnFixDecimal()  
Is_InCharSet("01234567890.-,$ ")  
AllowOnlyChars("0123456789.-")
```

AlterDatebyDay

Adds the specified number of days to the date contained in the current field. The original character confidence is not changed.

Syntax

()

Parameters

The number of days to add.

Returns

False if called at the wrong level or if the input is not numeric. Otherwise, **True**.

Level

Field level.

Details

This action will add the specified number of days to the date contained in the current field. The original character confidence is not changed.

Example

```
AlterDatebyDay("7")
```

This example adds one week to the date contained in the field. If the date crosses over a month or year boundary, they will be adjusted appropriately.

CalculateNotesZone

Creates a zone between detail lines to recognize the text between lines.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

Field level, on the detail field.

Details

This action creates a zone between detail lines, so the text between lines can be recognized.

Example

```
CalculateNotesZone()
```

CaptureOpInfo

Captures the Operator, Station ID, and current time and place them into variables. The variables are named based on the provided variable prefix, such as, *prefix Operator*, *prefix Station*, and *prefix Time*. When, a prefix is not provided the task name is used as the prefix by default.

Syntax

()

Parameters

An optional prefix to the variable name.

Returns

Always **True**.

Level

Any level.

Details

This action will capture the Operator, Station ID and current time and place them into variables. The variables are named based on the provided variable prefix, like this: *prefix Operator*, *prefix Station*, and *prefix Time*. If a prefix is not provided, the task name is used as the prefix by default.

Example

```
Status_Preserve_OFF()  
ClearErrorMsg()  
CaptureOpInfo("Production")  
rrCompare("@P\Routing_Instructions","None")
```

In this example, the variables created will be *Production Operator*, *Production Station*, and *Production Time*.

CheckAndFixDecimal

Replaces the space or comma with a decimal point. This action is used to fix errors where the decimal is not recognized and leaves a blank in that area.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

Field level.

Details

Replaces the space or , with . to fix errors where the decimal is not recognized and leaves a blank in that area. This action can also be used for conversion for European numbers that use a comma to separate dollars from cents so \$100,00 becomes \$100.00 or \$100 00 becomes \$100.00.

Example

```
CheckAndFixDecimal()  
Is_InCharSet("0123456789.-,$")  
AllowOnlyChars("0123456789.-")  
IsFieldCurrency()
```

CheckForSticky

For new fingerprints, checks whether there is another matching fingerprint within the same batch that was already verified. It can be used to obtain zone information.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

Page level.

Details

An example of this action is when a batch contains two similar invoices, which have never been processed by the system before. Once the first invoice is zoned during verify, the same new fingerprint can be used on the second invoice. This is only needed on new matching fingerprints within the same batch because a fingerprints zones are saved at export time and are available for subsequent batches.

Example

```
CheckForSticky()
```

CheckFreeDiskSpace

Reads the **LowDiskSpaceThreshold** setting from the [Notifications] section of the INI file that is specified in the first parameter.

Syntax

()

Parameters

Two comma separated parameters:

1. The path to .ini file that contains the **LowDiskSpaceThreshold** setting.

2. The letter drive of the drive to check. The letter drive must be accompanied by a colon, for example, C:.

Returns

False if called at the wrong level, if parameters are missing or if the settings.ini file cannot be found. Otherwise, **True**.

Level

Batch level.

Details

If the available disk space is less than the value specified in the .ini file, then a notification will be created in the .ini file. The notification is created under the [Notifications] section of the .ini file, under the **LowDiskNotification** setting. This setting will be set to **Yes**.

If the **LowDiskSpaceThreshold** is missing from the .ini file, the default value of 3000 bytes will be used.

If a notification has been previously created, and the disk space has been increased since the last time the action ran, then the notification will be removed from the .ini file and **LowDiskNotification** is set to **No**.

Example

```
CheckFreeDiskSpace("C:\Datacap\APT\dco_APT\settings.ini,C:")
Scan()
```

ClearErrorMsg

Use this action to clear the current error message.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

Any level.

Details

This action clears the current error message.

Example

```
Status_Preserve_OFF()
ClearErrorMsg()
CaptureOpInfo()
rrCompare("@P\Routing_Instructions","None")
```

CreateFingerprint

Creates a fingerprint for the current page, even when there is an existing fingerprint.

Syntax

()

Parameters

None.

Returns

True if a new fingerprint has been created. Otherwise, **False**.

Level

Page level.

Details

This action will create a fingerprint for the current page. Forces the creation of a new fingerprint using the current image, even if there is an existing fingerprint. A typical scenario is when there is a very similar fingerprint that incorrectly matches the current image, such as two invoices that are very similar. When the new invoice is received in the future, it should now match on this new fingerprint.

Example

```
IsChildFieldValue("Add_New_Fingerprint,YES")
SetFingerprintDir("@APPPATH(fingerprint)")
SetFingerprintRecogPriority("True")
RecognizePageOCR_S()
CreateFingerprint()
SetFingerprint("@P\Vendor")
iloc_SetDetailSimple("Details")
IncrementBatchVar("Verify requested Add Fingerprint")
```

DetailFix

Calculates the quantity, price, and line total when one of them is blank within a detail field. This calculation is done for all detail lines on the page.

Syntax

()

Parameters

None.

Returns

False if called at the wrong level.

False if any of following fields do not exist or are not numeric: Qty, Price, and LineTotal.

Otherwise, **True**.

Level

Field level.

Details

A typical scenario is a similar fingerprint incorrectly matches the current image, such as two similar invoices. When a new invoice is received in the future, it now matches this new fingerprint.

Example

```
DetailFix()
```

DoMsgbox

This action is deprecated, its functionality is no longer supported.

Syntax

()

Parameters

The message to display.

Returns

Always **True**.

Level

All levels.

Details

The message is displayed to the user and they must press **OK** to continue.

Attention: This action must not be used within a ruleset that is run unattended or processing will stop.

Example

```
CheckDCOStatus("75")  
DoMsgbox("The page was deleted.")
```

ExecuteSQLBind

This action was deprecated. It is scheduled to be removed in a future release.

Syntax

()

Details

This action should not be used, as it is scheduled to be removed in future versions.

FindExportImage

Searches the batch directory for a file that corresponds to the current page that contains the current field. The file extension must match the extension that is specified with the input parameter.

Syntax

()

Parameters

One of the following values:

1. Document Level TIFF
2. Document Level PDF
3. Page Level TIO
4. Page Level TIF

Returns

False if called at the wrong level or if the input parameter is invalid. **True** if the correctly named file with the specified extension exists in the batch directory.

Level

Field level.

Details

Example

```
FindExportImage("1")
```

FPXMLUsed

Indicates when FPXML format fingerprint files are being used. Call this action when you are using fingerprints from FPXML with Invoice.rrx.

Syntax

()

Parameters

None.

Returns

True if a FPXML fingerprint file exist for the current page object. Otherwise, **False**.

Level

Page level.

Details

Call this action if you are using fingerprints from FPXML with Invoice.rrx.

Example

```
FPXMLUsed()  
ZoneBOTTOM_ImageBottom()  
ScanDetails()
```

GenerateDetails

Sets up a field in APT and puts a detail subfield on every page. This extra detail subfield appears on all pages of a multi-page invoice, allowing every page to be viewed.

Syntax

()

Parameters

None.

Returns

False if called on the wrong level. Otherwise, **True**.

Level

Field level.

Details

This action sets up a field in APT and puts a detail subfield on every page. This extra detail subfield appears on all pages of a multi-page invoice, allowing every page to be viewed.

Example

```
GenerateDetails()
```

iloc_SetDetailSimple

Fills the Setup DCO from Runtime Data file.

Syntax

(sDetailName)

Parameters

The detail Datacap Object (DCO) Type.

Returns

False if called at the wrong level, if the DCO node cannot be found, if the fingerprint ID is not found, or if the setup DCO cannot be saved. Otherwise, **True**.

Level

Page level.

Details

This action fills the Setup DCO from Runtime Data file.

Example

```
SetFingerprint("@P\Vendor")
iloc_SetZones()
iloc_SetDetailSimple("Details")
IncrementBatchVar("Intellocate Fingerprint")
```

IncrementBatchVar

Increments a batch level variable by 1. When the variable does not exist, it is created. When the variable exists but the value is not a number, the variable is set to 1.

Syntax

()

Parameters

The name of the variable to increment.

Returns

Always **True**.

Level

Any level.

Details

This action increments a batch level variable by 1. If the variable does not already exist, it will be created. If the variable exists but the value is not numeric, the variable is set to 1.

Example

```
rrCompareNot("@P.RecogStatus","1")
SetDocStatus("128")
IncrementBatchVar("Recog - Deleted Document")
```

IsChildFieldBlank

This action is replaced by the rrCompare action in the rrunner library.

Syntax

()

Parameters

The name of a child field of the current object.

Returns

True if the child field is located and if it has an empty text field. Otherwise, **False**.

Level

Field Level.

Details

*** This Action Is Deprecated ***

This action has been deprecated and is scheduled to be removed in a future release. It is recommended that you no longer use this action. Instead, use the

runner action library. Use rrCompare with smart parameters. For example, rrCompare("@EMPTY", "@F\MyFieldToTest") See the action help for details.

*** Deprecated Help Description ***

Searches for a child field that matches the provided parameter. If it matches and if the text value is empty, then this action returns true, causing the next action in the ruleset to run.

Example

```
IsChildFieldBlank("Qty")
```

IsChildFieldValue

This action was deprecated. It is scheduled to be removed in a future release.

Syntax

()

Parameters

Two comma separated parameters:

1. The name of the variable to check.
2. The value to compare.

Returns

False if the field is not found, the number of parameters is incorrect or if it is called at the wrong level. Otherwise, **True**.

Level

Document Level

Details

*** This Action Is Deprecated ***

This action has been deprecated and is scheduled to be removed in a future release. It is recommended that you no longer use this action. Instead, use the runner action library. Use rrCompare with smart parameters. For example, rrCompare("Expected Value", "@F\MyFieldToTest") See the action help for details.

*** Deprecated Help Description ***

This action locates a child field and compares its value to the one passed in. This action can be used to perform actions if a field contains a specific value.

Example

```
IsChildFieldValue("Add_New_Fingerprint,YES")
```

IsCurrentObjValue

This action is replaced by the rrCompare action in the runner library.

Syntax

()

Details

This action should not be used, as it is scheduled to be removed in future versions. It has been replaced by rrCompare in runner. Use the @VALUE parameter to access the 'Text' variable of the current object and provide the comparison text string.

Example

```
rrCompare("@VALUE","Yes")
SetPageStatus("75")
SetDocStatus("128")
```

IsCurrentObjVariable

This action is replaced by rrCompare in the runner action library.

Syntax

()

Details

This action should not be used, as it is scheduled to be removed in future versions. It has been replaced by rrCompare in runner.

Example

```
rrCompare("@F.Sticky","No")
SkipChildren()
```

IsFingerPrintClass

Connects to the fingerprint database and verifies that the specified fingerprint class contains the fingerprint ID of the current page.

Syntax

()

Parameters

Two comma separated parameters:

1. The database connection string. Smart parameters are supported.
2. Fingerprint class.

Returns

False if called with the wrong number of parameters, on the wrong level, if the fingerprint ID does not exist in the specified class.

Level

Page Level.

Details

Example

```
ChkDCOStatus("75")
IsFingerPrintClass("@APPVAR(*fingerprintconn:cs)+,[New]")
DeleteFingerprint()
IncrementBatchVar("Deleted New Fingerprint")
```

A parameter of **[New]** checks to see if the fingerprint exists in the new class, and if it does, follow on actions can perform further processing.

IsInINI

Reads and returns to the action the value of the specified key in the INI file.

Syntax

()

Parameters

A comma separated string of:

1. The INI Filename.
2. The section within the INI file.
3. The keyword to find in the section.

Returns

True if the value of the current field is a substring within the string specified in the INI file. Otherwise, **False**.

Level

Field Level.

Details

When possible it is recommended to use the generic actions that read and write INI files in the FileIO action library. Reads and returns to the action the value of the specified key in the INI file. It compares the value of the current field with the with the string in the INI file. The field must be a substring of the string in the INI file.

Example

```
IsInINI("C:\MyDir\settings.ini", "mysection", "mykey")
```

IsInList

Validates that the value of a field is contained within a string.

Syntax

()

Parameters

A string that is a substring of the expected Text value.

Returns

True if the value of the *Text* variable of the current field is contained within the specified list.

Level

Field Level.

Details

Example

```
IsInList("Hello Larry")
```

In this example, if the field Text value is *Larry*, the action returns **True**.

IsMultipageDocument

Determines whether the current object is a document with multiple pages attached.

Syntax

```
()
```

Parameters

None.

Returns

False if called at the Batch level, the document object cannot be found, or if the document does not have more than 1 child. Otherwise, **True**.

Level

Document level.

Details

This action is used to determine if the current object is a document with multiple pages attached.

Example

```
IsMultipageDocument()
```

IsSinglePageDocument

Determines whether the current object is a document with only 1 page attached.

Syntax

```
()
```

Parameters

None.

Returns

False if not called on a document object or if the document does not contain exactly one page. **True** if the document contains one page.

Level

Document level.

Details

This action is used to determine if the current object is a document with only 1 page attached.

Example

```
IsSinglePageDocument()  
PopulateZNField()
```

IsStationIDSuffix

Tests the current station ID by checking that the specified parameter matches the rightmost portion of the station ID.

Syntax

()

Parameters

The expected Station ID suffix.

Returns

False if the suffix is longer than the current Station ID or if the specified suffix does not match the Station ID. Otherwise, **True**.

Level

Any level.

Details

This action checks that the specified parameter matches the right most portion of the station ID. This action is useful if you have stations with different suffixes and you want to control actions based on the station name.

Example

```
IsStationIDSuffix("-Test")  
CloseConnection()  
OpenConnection("@APPVAR(* /lookupdb.cs)")
```

In this example, If the station name is Validate-Test, the action returns **True** and continues executing the following actions.

IsTaskName

This action is deprecated and is scheduled to be removed in a future release. Use the rrCompare action in the runner action library.

Syntax

()

Parameters

The expected task name.

Returns

True if the specified name matches the currently running task. Otherwise, **False**.

Level

Any levels.

Details

*** This Action Is Deprecated ***

This action has been deprecated and is scheduled to be removed in a future release. It is recommended that you no longer use this action. Instead, use the runner action library. Use `rrCompare` to test using smart parameters. For example: `rrCompare("MyExpectedName", "@TASKNAME")`. See the action help for details.

*** Deprecated Help Description ***

This action checks the name of the currently running task to see if it matches the specified task name. If it matches, it will return **True**.

Example

```
IsTaskName("Verify")
LoadCCOFromField()
PopulateZNField()
```

Is_InCharSet

This action is deprecated and is scheduled to be removed in a future release. Use the actions in the Picture action library.

Syntax

()

Parameters

A list of valid characters.

Returns

False if called at the wrong level, if the *Text* value of the current object is empty or if a character in the *Text* variable of the current object does not exist in the specified character set. Otherwise, **True**.

Level

Field Level.

Details

*** This Action Is Deprecated ***

This action has been deprecated and is scheduled to be removed in a future release. It is recommended that you no longer use this action. Instead, use the Picture action library. First call the action `PIC_SetPictureCharacter` to set the list of valid characters with a custom picture string. Multiple lists can be defined as necessary. Then call `PIC_ApplyPictureString` to validate the field with the custom picture string. See the action help for details.

*** Deprecated Help Description ***

This action checks the *Text* value of the current field to confirm that each of the characters are contained within the specified set of expected characters. If one or more characters are not in the specified set, the action returns **False**.

Attention: The comparison is case sensitive.

Example

```
CheckAndFixDecimal()  
Is_InCharSet("0123456789.-,$ ")
```

Is_JobName

This action is deprecated and is scheduled to be removed in a future release. Use the rrCompare action in the runner action library.

Syntax

()

Parameters

The name of the job name you expect for the current job.

Returns

True if the current job matches the provided name. Otherwise, **False**.

Level

Any level.

Details

*** This Action Is Deprecated ***

This action has been deprecated and is scheduled to be removed in a future release. It is recommended that you no longer use this action. Instead, use the rrrunner action library. Use rrCompare to test the job name using smart parameters. For example, rrCompare("MyExpectedName", "@JOBNAME") See the action help for details.

*** Deprecated Help Description ***

This action will compare the provided job name with the name of the currently running job. This can be useful, if you wish to perform actions based on different job names. The comparison is case sensitive.

Example

```
Is_JobName("Demo-Multipage TIFF")  
PageIDByVariableChange("ScanSrcPath,Main_Page,Trailing_Page")
```

Is_JobNamePrefix

Tests the leftmost portion of the job name to determine if it matches the provided prefix.

Syntax

()

Parameters

The prefix you are expecting for the current job.

Returns

True if the left most portion matches. Otherwise, **False**.

Level

Any level.

Details

Example

```
Is_JobNamePrefix("Test")
```

If the current job name is TestRecognition, the action returns **True**.

LoadCCOFromField

Loads the DCO from a field object. The verify panels do not load the CCO into the scripting engine so this action accomplishes that task.

Syntax

()

Parameters

None.

Returns

False if this action is not called on a field or if the CCO file does not exist. Otherwise, **True**.

Level

Field Level.

Details

This action is required for any invoice action that uses the CCO.

Example

```
LoadCCOFromField()  
SetDynamicDetailZones("Zone Bottom,Notes")  
ZoneBOTTOM_ImageBottom()  
ScanDetails()
```

MovePDF

This action is deprecated and is scheduled to be removed in a future release. Use the actions in the FileIO action library.

Syntax

()

Parameters

The target directory for the PDF. Smart Parameters are supported.

Returns

False if the document cannot be found, if the source file does not exist or if the target file does exist. Otherwise, **True**.

Level

Document level.

Details

*** This Action Is Deprecated ***

This action has been deprecated and is scheduled to be removed in a future release. It is recommended that you no longer use this action. Instead, use the FileIO action library. Use RenameFile to move the file using smart parameters. For example: RenameFile("@DCO(BATCHDIR)+\+MyFile.pdf","@APPPATH(export)+\+MyFile.pdf",true). See the action help for details.

*** Deprecated Help Description ***

This action copies the PDF associated with the current DCO document to the specified directory. The PDF document name must match the name/ID of the current DCO document for the copy to take place.

Example

```
MovePDF("@APPPATH(export)")
```

OpenConnection

This action is deprecated and is scheduled to be removed in a future release. Use the OpenConnection action in the Lookup action library.

Syntax

()

Details

This action should not be used, as it is scheduled to be removed in future versions. It has been replaced by OpenConnection in Lookup.rrx.

ParselImageName

Extracts only the file name from the variable *ScanSrcPath* and places it into the *Text* variable of the current object.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

Document or Page level.

Details

*** This Action Is Deprecated ***

This action has been deprecated and is scheduled to be removed in a future release. It is recommended that you no longer use this action. Instead, use the FileIO action library. Use SplitFileName with smart parameters. For example: SplitFileName("@P.ScanSrcPath","", "", "@P\Fieldname.Text", ""). See the action help for details.

*** Deprecated Help Description ***

This action extracts only the file name from the variable *ScanSrcPath* and places it into the *Text* variable of the current object. It is expected that the file name contains an underscore and only the value up to the underscore is kept.

Example

```
ParseImageName()
```

PopulateZNLItemFieldDynamic

This action is like the PopulateZNLItemField except that it uses the CCO that was loaded into memory by LoadCCOFromField, instead of the global CCO.

Syntax

```
()
```

Parameters

None

Returns

True if there is no field position or if the data was found in a zone. **False** if there is a zone defined but it is blank.

Level

Page level.

Details

This action populates the data file of the Fingerprint with the recognized value contained inside the zone of a child Field object of a LINEITEM parent field. This action should only be used with sub-fields of the LINEITEM field - ItemID, ItemDesc, Quantity, Price in the Invoices application.

Example

```
FPXMLUsed()  
PopulateZNLItemFieldDynamic()
```

ReadFPXMLZones

Reads the zones from the FPXML into the objects for the page and stores the specified fields. When it creates detail lines, it knows the positions of the fields of these detail lines.

Syntax

()

Parameters

Comma-separated parameters:

1. The fingerprint directory. Smart parameters are supported.
2. A comma-separated list of the detail lines.

Returns

False if called at the wrong level or if the FPXML does not exist. Otherwise, **True**.

Level

Page level.

Details

Example

```
ReadFPXMLZones("@APPPATH(fingerprint),details,lineid,itemdesc,qty,price,linetotal")
rrSet("FPXML", "@P.ZoneRead")
IsMultipageDocument()
SetEOL("|")
```

SaveObjectVariable

This action is scheduled to be removed in a future release. Use the rrSet action in the runner action library.

Syntax

()

Details

This action should not be used, as it is scheduled to be removed in future versions. It has been replaced by rrSet in runner.

Example

```
rrSet("Normal", "@P.RecogType")
Is_JobName("Demo-Dot Matrix")
rrSet("s_fg", "7")
rrSet("s_ml", "3")
rrSet("Dot_Matrix", "@P.RecogType")
rrCompare("Normal", "@P.RecogType")
```

ScanLineItemDynamic

Scans the line items from the CCO that was loaded into the field. This action is like ScanLineItem except that it uses the CCO loaded for the field and reads position variables from the line item level.

Syntax

()

Parameters

A comma separated list of any fields that should be ignored.

Returns

Always **True**.

Level

Field Level.

Details

This action is required for Find Details functionality. It saves the positions of the line and of the fields at the detail level so all of the line items can be erased and recreated.

Example

```
FPXMLUsed()  
ScanLineItemDynamic("ZoneBottom,Notes")
```

SendOutlookNotification

Uses Outlook to send a notification to specified email addresses. The message within the email is determined by previous calls to actions with a set notification, such as CheckFreeDiskSpace.

Syntax

()

Parameters

The extension of the attachment. The attachment is expected to be named the same as the current document ID.

Returns

False if called at the wrong level, if the connection to Outlook could not be established, or if email addresses could not be found in Settings.ini.

Level

Document level.

Details

This action requires Outlook to be installed on the computer and logged in with an ID that has the appropriate permissions to send emails.

Example

```
SendOutlookNotification(".pdf")
```

SetDynamicDetailZones

Takes the position of the line items and builds the line coordinates. It sets the details zones from the first line to the end of the CCO.

Syntax

()

Parameters

The parameter is the Zone Bottom field. If more than one field is listed, Zone Bottom must be the first field and other fields are ignored.

Returns

False, if no children exist. Otherwise, **True**.

Level

Page level.

Details

Example

```
LoadCCOFromField()  
SetDynamicDetailZones("Zone Bottom,Notes")  
ZoneBOTTOM_ImageBottom()  
ScanDetails()
```

SetPicChar

This action is deprecated, it is scheduled to be removed in future versions. Use the PIC_SetPictureCharacter action in the Picture action library.

Syntax

()

Parameters

Details

This action should not be used, as it is scheduled to be removed in future versions. It is replaced by PIC_SetPictureCharacter in Picture.rrx.

SetStickyNo

Sets the Sticky indicator to No to indicate that there are no sticky fingerprints. Sticky fingerprints identify a page within a single verification session when another form of the same type appears after a previous form was zoned.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

Any level.

Details

Example

```
IsCurrentObjVariable("Sticky,Yes")  
SetStickyNo()
```

SetToDocIDMPTIFF

This action is deprecated and is scheduled to be removed in a future release. Use the rrSet action in the rrrunner action library.

Syntax

()

Parameters

The path to the exported tiff files.

Returns

Always **True**.

Level

Any level.

Details

*** This Action Is Deprecated ***

This action has been deprecated and is scheduled to be removed in a future release. It is recommended that you no longer use this action. Instead, use the rrrunner action library. Use rrSet with smart parameters. For example: rrSet("@APPPATH(export)+\+@ID+.tif","@F\FieldName.txt"). See the action help for details.

*** Deprecated Help Description ***

This action builds a full path to the a tiff file and sets the field text to this value. The path consists of the export path + DOCID + .TIF. The value is put into the field so it can be access later in the application, if necessary. It only builds the path. The action does not output the TIF file.

Example

```
SetToDocIDMPTIFF("@APPPATH(export)")
```

SwapImages

Interchanges the TIF for the current page with another TIF that has the same file name but a different extension.

Syntax

()

Parameters

Two extensions

1. The extension to be saved as TIF.
2. The extension that current tif is saved as.

Returns

False if called at the wrong level, if the wrong number of parameters are specified, or if the original file cannot be found. **True** if the file extensions are interchanged.

Level

Page level.

Details

One possible use for this action is if you had a backup of the TIF and need to restore it. The current TIF is named with the new extension while the other named TIF, is renamed to have a TIF extension.

Example

```
SwapImages("TIO,TIB")
```

SwitchMMDD

Switches US Month and Day date values. It swaps the first 2 characters of the field value with the 2 characters that follow the separator.

Syntax

()

Parameters

A list of separators.

Returns

True if two separators are found and the separators are swapped. Otherwise, **False**.

Level

Field level.

Details

Example

```
SwitchMMDD("/")
```

In this example, the value "03/09/10" becomes "09/03/10".

UpdateFPStats

Updates the fingerprint statistics in the fingerprint database. This action keeps track of the last accessed fingerprint and the number of times a fingerprint is accessed.

Syntax

()

Parameters

None.

Returns

False if called at the wrong level or if the fingerprint does not exist. Otherwise, **True**.

Level

Page level.

Details

Example

```
UpdateFPStats()
```

ValidateVendor

Checks whether the current vendor, vendor number, and postal code exist on the same record in the lookup database.

Syntax

```
()
```

Parameters

The connection string to the lookup database. Smart Parameters are supported.

Returns

False if the connection cannot be opened or if the settings file cannot be found.

False if these fields cannot be found: Vendor, Vendor_Number, or Remittance_Zip.

False if the vendor cannot be found in the database.

Otherwise, **True**.

Level

Page level.

Details

The fields that are validated are in the validate vendor check are Vendor, Vendor_Number, and Remittance_Zip.

Example

```
SetIsOverrideable("False")  
ValidateVendor("@APPVAR(*lookupdb:cs)")
```

WriteErrorMessage

Writes the message to field level variable message that appears in the status bar. The message is stored in the *MESSAGE* variable.

Syntax

```
()
```

Parameters

The error message.

Returns

Always **False**.

Level

All levels.

Details

Example

```
GetDCOStatus("75")  
WriteErrorMessage("The page is set to a deleted status.")
```

IOverlay actions

Use the IOverlay actions to combine the current page image with a background image. You can use this action to reapply a form background that dropped out during scanning.

The IOverlay actions enable dithering of a background image and haloing with white pixels around the black pixels on an image. These actions make the image easier to read.

“Overlay”

“SetBackgroundImage” on page 662

“SetDitheringBackground” on page 663

“SetHaloBackground” on page 663

Overlay

Combines the current image with the image file specified by the SetBackgroundImage action into a new image replacing the current image. This action is used to reinstate a form background that was dropped out during scanning.

Syntax

()

Parameters

None.

1. The parameter of the preceding SetBackgroundImage action can be a smart parameter that locates a file.
2. Looks for a field type called Image_Offset or IMAGEOVERLAYOFFSET containing a comma separated value containing the amount of X and Y pixel offset for the overlay image.

These fields are automatically generated by running FindFingerprint, CalculateOffset, WordFind_Offset, CalculateLocalOffset, Autofield(), and most of the PatternMatch.rxx actions.

Returns

False if the action is not applied to a Page object; if it cannot locate the Background Image file; or if the action encounters an error of a different kind. Otherwise, **True**.

Level

Page level only.

Details

Example:

```
SetBackgroundImage(c:\ParentDir\mclaims\process\hcfa\hcfa.tif)
Overlay()
```

In this example, the Overlay action uses the file name and path in the parameter of the SetBackgroundImage action to locate the Background Image file.

```
SetBackgroundImage(@APPPATH(formdir)+\ub04.tif)
Overlay()
```

Here, the SetBackgroundImage action uses a smart parameter to load the file name with the directory path coming from the application service.

Note: SetBackgroundImage must be called before Overlay. Other actions such as SetHaloBackground and SetDithering are optional and, if used, should precede the call to the Overlay action.

SetBackgroundImage

Designates the Image file that will overlay the image of the current page.

Syntax

(StrParam)

Parameters

Full path to the overlay Image file.

Returns

False if image does not exist, otherwise **True**.

Level

All.

Details

Designates the Image file that will overlay the image of the current page. Smart Parameters supported.

Example:

```
SetBackgroundImage(c:\ParentDir\mclaims\process\hcfa\hcfat.tif)
SetDitheringBackground(True)
SetHaloBackground(True)
Overlay()
```

SetDitheringBackground

Enables or disables dithering of the background image. Dithering makes the background image appear lighter than the information of the current image so that visually it appears less prominent.

Syntax

(StrParam)

Parameters

String value to enable dithering (**True**) or disable dithering (**False**).

Returns

Always **True**.

Level

All.

Details

Example:

```
SetBackgroundImage(c:\ParentDir\mclaims\process\hcfa\hcfat.tif)
SetDitheringBackground(True)SetHaloBackground(True)
Overlay()
```

SetHaloBackground

Enables or disables a halo of white pixels around any black pixels from the current image where they would otherwise touch pixels from the background. This makes the foreground information easier to read.

Syntax

(StrParam)

Parameters

String value to enable a halo (**True**) or prevent a halo (**False**).

Returns

Always **True**.

Level

All.

Details

Example:

```
SetBackgroundImage(c:\ParentDir\mclaims\process\hcfa\hcfat.tif)
SetDitheringBackground(True)
SetHaloBackground(True)
Overlay()
```

Locate actions

Use the Locate actions in combination with full text recognition to locate words or regular expressions on the page. And to move around the page by line or word.

The Locate library also includes a few format validation actions, such as IsCurrency and IsDateValue. Most of the validation actions are located in the Validate library.

- "AddKeyList" on page 665
- "AggregateKeyList" on page 666
- "DefaultValue" on page 667
- "FilterIt" on page 667
- "FindDBList" on page 668
- "FindDBList_InZone" on page 668
- "FindKeyList" on page 669
- "FindKeyList_InZone" on page 670
- "FindLastKeyList" on page 671
- "FindLastKeyList_InZone" on page 672
- "FindLastRegEx" on page 673
- "FindLastRegEx_InZone" on page 674
- "FindLastRegExList" on page 674
- "FindLastRegExList_InZone" on page 675
- "FindLastWord" on page 676
- "FindLastWord_InZone" on page 677
- "FindNextDBList" on page 677
- "FindNextDBList_InZone" on page 678
- "FindNextKeyList" on page 679
- "FindNextKeyList_InZone" on page 680
- "FindNextRegExList" on page 681
- "FindNextRegExList_InZone" on page 682
- "FindRegExList" on page 683
- "FindRegExList_InZone" on page 684
- "GoAboveWord" on page 685
- "GoBelowWord" on page 686
- "GoDownLine" on page 686
- "GoFirstLine" on page 687
- "GoFirstWord" on page 687
- "GoLastLine" on page 688
- "GoLastWord" on page 689
- "GoLeftWord" on page 689
- "GoRightWord" on page 690
- "GoUpLine" on page 691
- "GroupWords" on page 691
- "GroupWordsLEFT" on page 692
- "GroupWordsRIGHT" on page 693
- "IsAlpha" on page 693
- "IsCurrency" on page 694

"IsDateValue" on page 695
"IsNumber" on page 696
"IsValue" on page 696
"IsValue_RegEx" on page 697
"MaxLength" on page 698
"MergeWordLF" on page 698
"MergeWordRT" on page 699
"MinLength" on page 700
"RegExFind" on page 700
"RegExFind_InZone" on page 701
"RegExFindNext" on page 702
"RegExFindNext_InZone" on page 702
"ScanRT" on page 703
"SelectSnippet" on page 704
"SetRect" on page 705
"UpdateDCOField" on page 705
"UpdateField" on page 706
"ValueInField" on page 707
"ValueInField_Fuzzy" on page 707
"ValueInField_RegEx" on page 708
"WordFind" on page 708
"WordFind_InZone" on page 709
"WordFindNext" on page 710
"WordFindNext_InZone" on page 711
"WordFind_Offset" on page 711

AddKeyList

Adds a list of keywords or phrases that can be used for matching.

Syntax

()

Parameters

1. String value used as a reference name for other actions to call this list of keywords or phrases.
2. Up to twenty five keywords or phrases to be used for matching.

Returns

False if no keywords or phrases are entered. Otherwise, **True**.

Level

Any.

Details

Adds a list of keywords or phrases that can be used for matching.

This action complements the List search actions that can load a Keyword text file which contains a list of words or phrases, separated by new lines, that are used for matching.

AddKeyList adds a list of up to 25 words or phrases without having to read or edit a file.

Attention: You can not redefine or overwrite an existing key list

Example

```
AddKeyList("InvNum","Invoice Number","Inv. Num.,""Invoice #:")  
FindKeyList("InvNum")
```

This action searches the current page, from first word to the last word of the current page, for the first occurrence of all keywords in the Invoices Number Keyword list "InvNum".

If successful, the search stops and remembers the location of that word for subsequent actions.

If not successful, the action continues searching for the next keyword in the InvNum list, starting from first word of the current page, repeating this search pattern until a match is found, or until there are no more keywords.

AggregateKeyList

Toggles the Key list searches to merge all words or phrases in the list into a single query.

Syntax

()

Parameters

Toggles Key list searches to merge all words or phrases in the list into a single query.

Returns

Always **True**.

Level

Any.

Details

Use of this action changes the search behavior for all 'List' type key word search actions. Default behavior is to search each keyword in the list sequentially. This action with the parameter set to **True** changes the search behavior to check for all the keywords in a single aggregate keyword search. To change back to the default behavior use this action with the **False** parameter.

Example

```
AggregateKeyList(True)  
FindKeyList("InvNum")
```

This action searches the current page, from first word to the last word of the current page, for the first occurrence of all keywords in the Invoices

Number Keyword file (InvNum.key). If successful, the search stops and remembers the location of that word for subsequent actions; if not, the action continues searching for the next word in InvNum.key, starting from first word of the current page, repeating this search pattern until a match is found, or until there are no more keywords.

DefaultValue

Sets the text value of the current field in the page data file to the value that is specified.

Syntax

()

Parameters

The String value assigned to the bound Field object of the Document Hierarchy that represents the current field.

Returns

Always **True**.

Level

Field level only.

Details

Sets the captured value of the current field to the String value you enter as a parameter.

Example:

```
DefaultValue("Bill Paid")
DefaultValue("Past Due")
```

FilterIt

Removes all instances of the specified characters from the word that is located.

Syntax

()

Parameters

A string containing the characters to remove.

Important: This action removes every instance of the character or characters.

Returns

Always **True**.

Level

Field level only.

Details

Removes all instances of each character you enter as a parameter from the located word or phrase.

Example

```
FilterIt("-")
31-Dec-01 becomes 31Dec01
FilterIt("1")
31-Dec-01 becomes 3-Dec-0
FilterIt("1-")
31-Dec-01 becomes 3Dec0
```

FindDBList

Locates a word that matches one of a list of words that are obtained from a SQL query.

Syntax

()

Parameters

An SQL statement whose result returns a word or phrase, or a list of words or phrases.

Returns

True if the word or phrase is located on the page. Otherwise, **False**.

Level

Page or field level.

Details

This action issues an SQL query against the lookup database, returning a list of words or phrases that will be located on the current page. The search is performed from the first word of the current page. Initially, the first listed word or phrase is searched for on the page. If there is no match, it will search for the next word or phrase returned from the database. This continues until a match is found or none of the returned words are contained on the page. The location of the found word or phrase will be remembered so the result can be used by subsequent actions. The search is case sensitive.

To improve matching, this action automatically adjusts the search criteria to allow for common character substitutions. For example, if the list includes "will" and the recognition read "wi11", a match will still occur.

Common substitutions include characters: B8, Z2, S5, oO0 and i1t1.

Example

```
FindDBList("Select LastName from Providers")
```

This example gets a list of last names from a provider table and find the first matching occurrence on the page.

FindDBList_InZone

Locates a word that matches a word from the current field.

Syntax

()

Parameters

An SQL statement whose result returns a word or phrase, or a list of word or phrases.

Returns

True, if the word or phrase is on the page. Otherwise, **False**.

Level

Page or field level.

Details

This action generates an SQL query against the Lookup database, returning a list of words or phrases that are in the current field. The search is performed from the first word of the current field. Initially, the first listed word or phrase is searched for in the field. If there is no match, it will search for the next word or phrase that is returned from the database. The search continues until a match is found or none of the returned words are contained on the page. The location of the found word or phrase is remembered so the result can be used by subsequent actions. The search is case-sensitive.

To improve matching, this action automatically adjusts the search criteria to allow for common character substitutions. For example, if the list includes `w11` and the recognition that is read is `w11`, a match still occurs.

Common substitutions include characters: **B8**, **Z2**, **S5**, **oO0**, and **il1l1**.

Example

```
FindDBList("Select LastName from Providers")
```

This example obtains a `LastName` list from a provider table and finds the first matching occurrence on the page.

FindKeyList

Locates the first or next occurrence of a word or phrase that matches one of the entries in a keyword file.

Syntax

()

Parameters

The name of the Keyword text file. The file will contain a list of words or phrases, separated by new lines, that will be used for matching.

The file name can be provided in one of two ways:

1. A full path name of the file, including the `.key` extension.
2. The file name only, with no extension specified. The application will look in the process directory and the file must have a `.key` extension.

Returns

True if at least one word on the page matches any word or pattern in the Keyword file. Otherwise, **False**.

Level

Field level only.

Details

Opens the Keyword file you specify as a parameter, then checks the words or phrases on the current page against the keywords in the list to find a match. The location of the found word or phrase that matches an entry in the keyword file will be remembered so the result can be used by subsequent actions. Matching is case sensitive.

To improve matching, this action automatically adjusts the search criteria to allow for common character substitutions. For example, if the list includes "will" and the recognition read "wi11", a match will still occur.

Common substitutions include characters: B8, Z2, S5, oO0 and i1t1.

Example

```
FindKeyList("InvNum")
```

This action searches the current page, from first word to the last word of the current page, for the first keyword in the Invoices Number Keyword file (InvNum.key). If successful, the search stops and remembers the location of that word for subsequent actions; if not, the action continues searching for the next word in InvNum.key, starting from first word of the current page, repeating this search pattern until a match is found, or until there are no more keywords.

FindKeyList_InZone

Locates the first (or next) occurrence of a word or phrase that matches one of the entries in the current field.

Syntax

()

Parameters

The name of the Keyword text file. The file will contain a list of words or phrases, separated by new lines, that will be used for matching.

The file name can be provided in one of two ways:

1. A full path name of the file, including the .key extension.
2. The file name only, with no extension specified. The application will look in the process directory and the file must have a .key extension.

Returns

True if at least one word in the current bound zone matches any word or pattern in the Keyword file. Otherwise, **False**.

Level

Field level.

Details

Opens the Keyword file you specify as a parameter, then checks the words or phrases in the current field of the source page's .cco file against the keywords in the key file list to find a match. The location of the found word or phrase that matches an entry in the keyword file will be remembered so the result can be used by subsequent actions. Matching is case sensitive.

To improve matching, this action automatically adjusts the search criteria to allow for common character substitutions. For example, if the list includes "will" and the recognition read "wi11", a match will still occur.

Common substitutions include characters: B8, Z2, S5, oO0 and i!t1.

Example

```
FindKeyList_InZone("Line_Items")
```

This action searches the current bound zone for the first keyword in the Keyword file (Line_Items.key). If successful, the search stops and remembers the location of that word for subsequent actions; if not, the action continues searching for the next word in Line_Items.key, from the beginning of the current bound zone, repeating this search pattern until a match is found, or until there are no more keywords.

FindLastKeyList

Locates the last occurrence of a word or phrase that matches one of the entries in a keyword file.

Syntax

()

Parameters

The name of the Keyword text file. The file will contain a list of words or phrases, separated by new lines, that will be used for matching.

The file name can be provided in one of two ways:

1. A full path name of the file, including the .key extension.
2. The file name only, with no extension specified. The application will look in the process directory and the file must have a .key extension.

Returns

True if at least one word on the page matches any word or pattern in the Keyword file. Otherwise, **False**.

Level

Page level.

Details

Opens the Keyword file you specify as a parameter, then checks the words or phrases on the current page against the keywords in the list to find a match. The current page will be searched to find the last occurrence of word or phrase. The location of the last occurrence word or phrase that matches an entry in the keyword file will be remembered so it can be utilized by subsequent actions. Word matching is case sensitive.

To improve matching, this action automatically adjusts the search criteria to allow for common character substitutions. For example, if the list includes "will" and the recognition read "wi11", a match will still occur.

Common substitutions include characters: B8, Z2, S5, oO0 and iItl1.

Example

```
FindLastKeyList("InvNum")
```

This action searches the current page for the last instance of a match with the first keyword in the Keyword file (InvNum.key). If successful, the search stops and remembers the location of that word for subsequent actions; if not, the action continues searching using the next word in InvNum.key, matching the last occurrence of the word, repeating this search pattern until a match is found, or until there are no more keywords.

FindLastKeyList_InZone

Locates the last occurrence of a word or phrase that matches one of the current field.

Syntax

()

Parameters

The name of the Keyword text file. The file will contain a list of words or phrases, separated by new lines, that will be used for matching.

The file name can be provided in one of two ways:

1. A full path name of the file, including the .key extension.
2. The file name only, with no extension specified. The application will look in the process directory and the file must have a .key extension.

Returns

True if at least one word on the page matches any word or pattern in the Keyword file. Otherwise, **False**.

Level

Field level.

Details

Opens the Keyword file you specify as a parameter, then checks the words or phrases in the current bound zone against the keywords in the list to find a match.

The location of the found word or phrase will be remembered so the result can be used by subsequent actions. Word matching is case sensitive.

To improve matching, this action automatically adjusts the search criteria to allow for common character substitutions. For example, if the list includes "will" and the recognition read "wi11", a match will still occur.

Common substitutions include characters: B8, Z2, S5, oO0 and iIt11.

Example

```
FindLastKeyList_InZone("ClaimsData")
```

This action searches the current bound zone for the last instance of a match with the first keyword in the Keyword file (ClaimsData.key). If successful, the search stops and remembers the location of that word for subsequent actions; if not, the action continues searching using the next word in ClaimsData.key, matching the last occurrence of the word, repeating this search pattern until a match is found, or until there are no more keywords.

FindLastRegEx

Same as the FindLastWord action, except that it supports regular expressions.

Syntax

()

Parameters

A word or phrase to find on the current page. The parameter is expected to be a Regular Expression.

Returns

True if the word or phrase is located on the page. Otherwise, **False**.

Level

Page level.

Details

Locates the last occurrence of a word or phrase on the current page where the input search term is specified as a regular expression. The search is started from the first word of the current page. The location of the found word or phrase will be remembered so the result can be used by subsequent actions. The search is case sensitive.

Example

```
FindLastRegEx("Da?e")  
GoRightWord("1")  
IsDateValue()  
UpdateField()
```

FindLastRegEx will look for the last occurrence of a word that matches the regular expression. If it match is found, it will continue processing by moving one word right to the result of RegExFind and act on that word. In this example, if it is a date format, UpdateField will place the value into the current field.

FindLastRegEx_InZone

Same as the FindLastWord_InZone action, except that it supports regular expressions.

Syntax

()

Parameters

A word or phrase to find in the current field. The parameter is expected to be a Regular Expression.

Returns

True if the word or phrase is located in the field. Otherwise, **False**.

Level

Field level.

Details

Locates the last occurrence of a word or phrase in the current field where the input search term is specified as a regular expression. The search is started from the first word of the current field. The location of the found word or phrase will be remembered so the result can be used by subsequent actions. The search is case sensitive.

Example

```
FindLastRegEx_InZone("Da?e")
GoRightWord("1")
IsDateValue()
UpdateField()
```

FindLastRegEx will look for the last occurrence of a word that matches the regular expression. If it match is found, it will continue processing by moving one word right to the result of RegExFind and act on that word. In this example, if it is a date format, UpdateField will place the value into the current field.

FindLastRegExList

Same as the FindLastKeyList action, except that it supports regular expressions.

Syntax

()

Parameters

The name of the Keyword text file. The file will contain a list of words or phrases, separated by new lines, that will be used for matching. The entries in the keyword file are expected to be regular expressions.

The file name can be provided in one of two ways:

1. A full path name of the file, including the .key extension.
2. The file name only, with no extension specified. The application will look in the process directory and the file must have a .key extension.

Returns

True if at least one word on the page matches any word or pattern in the Keyword file. Otherwise, **False**.

Level

Page level.

Details

Opens the Keyword file you specify as a parameter, then checks the words or phrases on the current page against the keywords in the list to find a match. The entries in the keyword file are expected to be regular expressions. To perform the action, the search first looks for the last occurrence of the first word or phrase in the keyword file. If the word is found on the page, the search stops at the last occurrence of the word. If no match is found, the next line from the keyword file is read and again the entire page is searched. This process continues until a match is found or all of the lines in the keyword file have been read. The location of the last occurrence of a found word or phrase will be remembered so the result can be used by subsequent actions. Word matching is case sensitive.

Example

```
FindLastRegExList("InvoiceNum")
```

This action searches the current page, from the first word of the current page, for the last occurrence of the first keyword in the Keyword file (InvoiceNum.key). If successful, the search stops and remembers the location of that word for subsequent actions; if not, the action continues searching for the last occurrence of the next word in InvoiceNum.key, starting from starting of the current page, repeating this search pattern until a match is found, or until there are no more keywords.

FindLastRegExList_InZone

Same as the FindLastRegExList action, except that it searches the current field only.

Syntax

()

Parameters

The name of the Keyword text file. The file will contain a list of words or phrases, separated by new lines, that will be used for matching. The entries in the keyword file are expected to be regular expressions.

The file name can be provided in one of two ways:

1. A full path name of the file, including the .key extension.
2. The file name only, with no extension specified. The application will look in the process directory and the file must have a .key extension.

Returns

True if at least one word in the field matches any word or pattern in the Keyword file. Otherwise, **False**.

Level

Field level.

Details

Opens the Keyword file you specify as a parameter, then checks the words or phrases in the current field against the keywords in the list to find a match. The entries in the keyword file are expected to be regular expressions.

To perform the action, the search first looks for the last occurrence of the first word or phrase in the keyword file. If the word is found on the page, the search stops at the last occurrence of the word. If no match is found, the next line from the keyword file is read and again the entire field is searched. This process continues until a match is found or all of the lines in the keyword file have been read. The location of the last occurrence of a found word or phrase will be remembered so the result can be used by subsequent actions. Word matching is case sensitive.

Example

```
FindLastRegExList_InZone("InvoiceNum")
```

This action searches the current page, from the first word of the current page, for the last occurrence of the first keyword in the Keyword file (InvoiceNum.key). If successful, the search stops and remembers the location of that word for subsequent actions; if not, the action continues searching for the last occurrence of the next word in InvoiceNum.key, from starting of the current page, repeating this search pattern until a match is found, or until there are no more keywords.

FindLastWord

Locates the last occurrence of the specified word or phrase on the current page.

Syntax

()

Parameters

A word or phrase to find on the page.

Returns

True, if the word or phrase is on the page. Otherwise, **False**.

Level

Page level.

Details

The current page is searched to find the last occurrence of a word or phrase. The location of the last word or phrase that matches the parameter is remembered and it can be used by subsequent actions.

To improve matching, this action automatically adjusts the search criteria to allow for common character substitutions. For example, if the list includes will and the recognition that is read is will, a match still occurs.

Common substitutions include characters: B8, Z2, S5, oO0, and iItl1

Example

```
FindLastWord("Total")  
GoRightWord("1")  
IsCurrency()
```

In this example, the action finds the last instance of Total on the current page. Then, it moves right one word and checks to be sure that the word has a Currency value.

FindLastWord_InZone

Locates the last occurrence of the specified word or phrase on the current field.

Syntax

()

Parameters

The word or phrase to locate within the current zone.

Returns

True if the word or phrase is located in the field. Otherwise, **False**.

Level

Page level.

Details

The current field will be searched to find the word or phrase. The location of the last occurrence of the word or phrase that matches the parameter will be remembered so it can be utilized by subsequent actions. Word matching is case sensitive.

To improve matching, this action automatically adjusts the search criteria to allow for common character substitutions. For example, if the list includes "will" and the recognition read "wi11", a match will still occur.

Common substitutions include characters: B8, Z2, S5, oO0 and iItl1.

Example

```
FindLastWord_InZone("Total")  
GoRightWord("1")  
IsCurrency()
```

This sequence attempts to find the last occurrence of "Total" in the current zone. It then locates and validates a Currency amount for the Total field, assuming the currency amount is to the right of the word "Total".

FindNextDBList

Same as the FindDBList action, except that it locates the next instance.

Syntax

()

Parameters

An SQL statement whose result returns a word or phrase, or a list of word or phrases.

Returns

True, if the word or phrase is on the page. Otherwise, **False**.

Level

Page level.

Details

This action generates an SQL query against the Lookup database, returning a list of words or phrases that are on the current page. Starting from the location of a previously found word, the search is performed. Initially, the first listed word or phrase from the SQL query is searched for on the page. If there is no match, it will search for the next word or phrase that is returned from the database. The search continues until a match is found or none of the returned words are contained on the page. The location of the found word or phrase is remembered so the result can be used by subsequent actions. The search is case-sensitive.

To improve matching, this action automatically adjusts the search criteria to allow for common character substitutions. For example, if the list includes `will` and the recognition that is read is `wi1l`, a match still occurs.

Common substitutions include characters: **B8**, **Z2**, **S5**, **oO0**, and **iItl1**.

Example

```
WordFind("Hello")
FindNextDBList("Select LastName from Providers")
```

In this example, when the first word `Hello` is found, the `LastName` list is obtained from a provider table. Then, the first matching occurrence on the page that occurs after the word `Hello` is found.

FindNextDBList_InZone

Same as the `FindNextDBList` action, except that it searches the current field only.

Syntax

()

Parameters

An SQL statement whose result returns a word or phrase, or a list of word or phrases.

Returns

True, if the word or phrase is in the field. Otherwise, **False**.

Level

Field level.

Details

This action generates an SQL query against the lookup database, returning a list of words or phrases that are in the current field. Starting from the location of a previously found word, the search is performed. Initially, the first listed word or phrase from the SQL query is searched for on the page. If there is no match, it will search for the next word or phrase that is returned from the database. The search continues until a match is found or none of the returned words are contained on the page. The location of the found word or phrase is remembered so the result can be used by subsequent actions. The search is case-sensitive.

To improve matching, this action automatically adjusts the search criteria to allow for common character substitutions. For example, if the list includes `will` and the recognition that is read is `wi1l`, a match still occurs.

To improve matching, this action automatically adjusts the search criteria to allow for common character substitutions.

Common substitutions include characters: **B8**, **Z2**, **S5**, **oO0**, and **iItl1**

Example

```
WordFind_InZone("Hello")
FindNextDBList("Select LastName from Providers")
```

In this example, when the first word `Hello` is found in the current field, the `LastName` list is obtained from a provider table. Then, the first matching occurrence in the field that occurs after the word `Hello` is found.

FindNextKeyList

Same as the `FindKeyList` action, except that it locates the next instance.

Syntax

()

Parameters

The name of the Keyword text file. The file will contain a list of words or phrases, separated by new lines, that will be used for matching.

The file name can be provided in one of two ways:

1. A full path name of the file, including the `.key` extension.
2. The file name only, with no extension specified. The application will look in the process directory and the file must have a `.key` extension.

Returns

True if at least one word or phrase on the page, starting from the previously located word, matches any word or pattern in the Keyword file. Otherwise, **False**.

Level

Field level.

Details

Opens the Keyword file you specify as a parameter, then checks the words or phrases on the current page against the keywords in the list to find a match. The search starts from the last word that had been found using an action such as `FindKeyList` or `FindNextKeyList`. The location of the found word or phrase that matches an entry in the keyword file will be remembered so the result can be used by subsequent actions. Matching is case sensitive.

To improve matching, this action automatically adjusts the search criteria to allow for common character substitutions. For example, if the list includes "will" and the recognition read "wi11", a match will still occur.

Common substitutions include characters: B8, Z2, S5, oO0 and iItl1.

Example

```
FindKeyList("Tax")  
FindNextKeyList("IRS")
```

Starting from the word or phrase that was located in the `FindKeyList` action, this action searches the current page, starting from that previously found word, for the first word or phrase in the first line of the Keyword file (IRS.key). If successful, the search stops and remembers the location of that new word for subsequent actions; if not, the action continues searching using the next word or phrase in IRS.key, starting again from the location of the word previously found by `FindKeyList(Tax)`, repeating this search pattern until a match is found, or until there are no more keywords in the file.

FindNextKeyList_InZone

Same as the `FindNextKeyList` action, except that it searches the current field only.

Syntax

()

Parameters

The name of the Keyword text file. The file will contain a list of words or phrases, separated by new lines, that will be used for matching.

The file name can be provided in one of two ways:

1. A full path name of the file, including the .key extension.
2. The file name only, with no extension specified. The application will look in the process directory and the file must have a .key extension.

Returns

False if the Fingerprint file (.cco) of the current source page has not been loaded, or is empty. Otherwise, **True**.

Level

Field level.

Details

Opens the Keyword file you specify as a parameter, then checks the words or phrases in the current field against the keywords in the list to find a match. The search starts from the last word that had been found using an action such as FindKeyList_InZone or FindNextKeyList_InZone. The location of the found word or phrase that matches an entry in the keyword file will be remembered so the result can be used by subsequent actions. Matching is case sensitive.

To improve matching, this action automatically adjusts the search criteria to allow for common character substitutions. For example, if the list includes "will" and the recognition read "wi11", a match will still occur.

Common substitutions include characters: B8, Z2, S5, oO0 and iItl1.

Example

```
FindKeyList_InZone("Inventory")  
FindNextKeyList_InZone("Parts")
```

This action searches the current field, starting from the first word of the zone, for the first occurrence of a word or phrase in the Keyword file "Inventory.key". If a match is found, the application will then look for a word from within the Keyword file "Parts.key", starting from the previously found word from the FindKeyList_InZone action and using the first word or phrase in the "Parts.key" file. If successful, the search stops and remembers the location of the new word for subsequent actions; if not, the action continues searching using the next word in "Parts.key", starting again from the location of previously found word in the zone, repeating this search pattern until a match is found, or until there are no more keywords in the file.

FindNextRegExList

Same as the FindRegExList action, except that it locates the next instance.

Syntax

()

Parameters

The name of the Keyword text file. The file will contain a list of words or phrases, separated by new lines, that will be used for matching.

The file name can be provided in one of two ways:

1. A full path name of the file, including the .key extension.
2. The file name only, with no extension specified. The application will look in the process directory and the file must have a .key extension.

Returns

True if at least one word on the page, that follows the result of a previous find, matches any word or pattern in the Keyword file. Otherwise, **False**.

Level

Field level only.

Details

Opens the Keyword file you specify as a parameter, starting from the location of a previous find, this action checks the words or phrases on the current page against the keywords in the list to find a match. The entries in the keyword file are expected to be regular expressions.

To perform the action, the search first looks for the first word or phrase in the keyword file. Starting from the location of the last find, if the word is found, the search stops. If no match is found, the next line from the keyword file is read and again the search starts from the result of a previous find. This process continues until a match is found or all of the lines in the keyword file have been read. The location of the found word or phrase that matches an entry in the keyword file will be remembered so the result can be used by subsequent actions. Word matching is case sensitive.

Example

```
FindRegExList("InvoiceNum")  
FindNextRegExList("InvoiceNum")
```

This action searches the current page, starting from the result of the FindRegExList action, for the first keyword in the Keyword file (InvoiceNum.key). If successful, the search stops and remembers the location of that word for subsequent actions; if not, the action continues searching for the next word in InvoiceNum.key, repeating this search pattern until a match is found, or until there are no more keywords. Although this example shows FindNextRegExList using the same keyword file, it is allowed to use a different keyword file.

FindNextRegExList_InZone

Same as the FindNextRegExList action, except that it searches the current field only.

Syntax

()

Parameters

The name of the Keyword text file. The file contains a list of words or phrases, separated by new lines, that will be used for matching. The file name can be provided in one of two ways:

1. A full path name of the file, including the .key extension.
2. The file name only, with no extension specified. The application looks in the process directory and the file must have a .key extension.

Returns

True if at least one word in the field, that follows the result of a previous find, matches any word or pattern in the Keyword file. Otherwise, **False**.

Level

Field level only.

Details

Opens the Keyword file you specify as a parameter, starting from the location of a previous find, this action checks the words or phrases in the current field against the keywords in the list to find a match. The entries in the keyword file are expected to be regular expressions.

To perform the action, the search first looks for the first word or phrase in the keyword file. Starting from the location of the last find, if the word is found, the search stops. If no match is found, the next line from the keyword file is read and again the search starts from the result of a previous find.

This process continues until a match is found or all of the lines in the keyword file have been read. The location of the found word or phrase that matches an entry in the keyword file is remembered so the result can be used by subsequent actions. Word matching is case sensitive.

Example

```
FindRegExList_InZone("InvoiceNum")  
FindNextRegExList_InZone("InvoiceNum")
```

This action searches the current page, starting from the result of the FindRegExList_InZone action, for the first keyword in the Keyword file (InvoiceNum.key). If successful, the search stops and remembers the location of that word for subsequent actions; if not, the action continues searching for the next word in InvoiceNum.key, repeating this search pattern until a match is found, or until there are no more keywords. Although this example shows FindNextRegExList_InZone using the same keyword file, it is allowed to use a different keyword file.

FindRegExList

Same as the FindKeyList action, except that it supports regular expressions.

Syntax

()

Parameters

The name of the Keyword text file. The file will contain a list of words or phrases, separated by new lines, that will be used for matching.

The file name can be provided in one of two ways:

1. A full path name of the file, including the .key extension.
2. The file name only, with no extension specified. The application will look in the process directory and the file must have a .key extension.

Returns

True if at least one word on the page matches any word or pattern in the Keyword file. Otherwise, **False**.

Level

Field level only.

Details

Opens the Keyword file you specify as a parameter, then checks the words or phrases on the current page against the keywords in the list to find a match. The entries in the keyword file are expected to be regular expressions.

To perform the action, the search first looks for the first word or phrase in the keyword file. If the word is found on the page, the search stops. If no match is found, the next line from the keyword file is read and again the entire page is searched. This process continues until a match is found or all of the lines in the keyword file have been read. The location of the found word or phrase will be remembered so the result can be used by subsequent actions. Word matching is case sensitive.

Example

```
FindRegExList("InvoiceNum")
```

This action searches the current page, from the first word of the current page, for the first keyword in the Keyword file (InvoiceNum.key). If successful, the search stops and remembers the location of that word for subsequent actions; if not, the action continues searching for the next word in InvoiceNum.key, starting from first word of the current page, repeating this search pattern until a match is found, or until there are no more keywords.

FindRegExList_InZone

Same as the FindRegExList action, except that it searches the current field only.

Syntax

()

Parameters

The name of the Keyword text file. The file will contain a list of words or phrases, separated by new lines, that will be used for matching.

The file name can be provided in one of two ways:

1. A full path name of the file, including the .key extension.
2. The file name only, with no extension specified. The application will look in the process directory and the file must have a .key extension.

Returns

True if at least one word on the page matches any word or pattern in the Keyword file. Otherwise, **False**.

Level

Field level only.

Details

Opens the Keyword file you specify as a parameter, then checks the words or phrases on the current page against the keywords in the list to find a match. The entries in the keyword file are expected to be regular expressions.

To perform the action, the search first looks for the first word or phrase in the keyword file. If the word or phrase is found in the current field zone, the search stops. If no match is found, the next line from the keyword file is read and again the current field zone is searched. This process continues until a match is found or all of the lines in the keyword file have been read. The location of the found word or phrase that matches an entry in the keyword file will be remembered so the result can be used by subsequent actions. Word matching is case sensitive.

Example

```
FindRegExList_InZone("InvoiceNum")
```

This action searches the current page, from the first word of the current field, for the first keyword in the Keyword file (InvoiceNum.key). If successful, the search stops and remembers the location of that word for subsequent actions; if not, the action continues searching for the next word in InvoiceNum.key, starting from first word of the current field, repeating this search pattern until a match is found, or until there are no more keywords.

GoAboveWord

Moves up the specified number of lines from the previously found word or phrase.

Syntax

```
()
```

Parameters

An integer indicating the number of lines to move up.

Returns

True if a word is found. Otherwise, **False**.

Level

Page or field level.

Details

Starting from a word or phrase found by a previous Locate action, the location position is moved to a word directly above it by the number of lines indicated by the input parameter. Subsequent Locate actions can then perform additional operations on this located word. Regardless of being called at the page or field level, this action operates on the recognized text for the current page.

Attention: If the action is added with an empty or non-numeric parameter, the action will default the argument to '1'.

Example

```
WordFind("Total")  
GoAboveWord("1")  
GoRightWord("1")  
UpdateField()
```

This sequence finds the word which is the Tax field's entered value ("29.78") when a fingerprint has these fields and values:

```
Tax 29.78  
Total 234.70
```

GoBelowWord

Moves down the specified number of lines from the previously found word or phrase.

Syntax

()

Parameters

An integer indicating the number of lines to move down.

Returns

True if the word or phrase is found. Otherwise, **False**.

Level

Page or field level.

Details

Starting from a word or phrase found by a previous Locate action, the location position is moved to a word directly below it by the number of lines indicated by the input parameter. Subsequent Locate actions can then perform additional operations on this located word. Regardless of being called at the page or field level, this action operates on the recognized text for the current page.

Attention: If the action is added with an empty or non-numeric parameter, the action will default the argument to '1'. This action is always called after a search action has been used.

Example

```
WordFind("Tax")
GoBelowWord("1")
GoRightWord("1")
UpdateField()
```

This sequence finds the word which is the Total field's entered value ("234.70") when a fingerprint has these fields:

```
Tax 29.78
Total 234.70
```

GoDownLine

Moves down the specified number of lines from the previously found word or phrase and selects the first word.

Syntax

()

Parameters

An integer indicating the number of lines to move down below the current line.

Returns

True if a line is found. Otherwise, **False**.

Level

Page or field level.

Details

Starting from the current position, the location position is moved down by the number of lines indicated in the input parameter and is moved to the first word of that line.

Attention: If the action is added with an empty or non-numeric parameter, the action will default the argument to '1'. This action is always called after a search action has been used.

Example

```
FindKeyList("Invoice")
GoDownLine("1")
GoRightWord("1")
UpdateField()
```

This sequence finds the word which is the Number field's entered value ("10034") when a fingerprint has these fields:

INVOICE Number: 10034

GoFirstLine

Moves to the first line of the page when you are running full page recognition.

Syntax

()

Parameters

None.

Returns

False if the page's Fingerprint file(.cco) is unavailable or empty. Otherwise, **True**.

Level

Page or field level.

Details

Starting from the current position, the location position is moved to the first line of the current zone, or to the first line on the page if a zone is not present.

Subsequent Locate actions can then perform additional operations from this position. Regardless of being called at the page or field level, this action operates on the recognized text for the current page.

Example

```
GoFirstLine()
```

GoFirstWord

Moves to the first word on the current line.

Syntax

()

Parameters

None.

Returns

False if the page's Fingerprint file (.cco) is not available or if it is empty. Otherwise, **True**.

Level

Page or field level.

Details

Starting from a word or phrase found by a previous Locate action, the location position is moved to the first word on the current line. If the page's Fingerprint file (.cco) does not have a line position, the action defaults to the first word on the first line of the current zone, or on the first line of the current page if a zone is not present. Subsequent Locate actions can then perform additional operations on this located word. Regardless of being called at the page or field level, this action operates on the recognized text for the current page.

Example

```
GoFirstWord()
```

GoLastLine

Moves to the last line of the page when you are running full page recognition.

Syntax

()

Parameters

None.

Returns

False if the source page's Fingerprint file (.cco) is not available, or if it is empty. Otherwise, **True**.

Level

Page or field.

Details

Starting from the current position, the location position is moved to the last line and word of the current zone, or to the last line and word on the page if a zone is not present. Subsequent Locate actions can then perform additional operations from this position. Regardless of being called at the page or field level, this action operates on the recognized text for the current page.

Example

```
GoLastLine()
```

GoLastWord

Moves to the last word on the current line.

Syntax

```
()
```

Parameters

None.

Returns

False if the Fingerprint file (.cco) is not available, or if it is empty. Otherwise, **True**.

Level

Page or field level.

Details

Starting from a word or phrase found by a previous Locate action, the location position is moved to the last word on the current line. If the page's Fingerprint file (.cco) does not have a line position, the action defaults to the last word on the first line of the current zone, or on the first line of the current page if a zone is not present. Subsequent Locate actions can then perform additional operations on this located word. Regardless of being called at the page or field level, this action operates on the recognized text for the current page.

Example

```
GoLastWord()
```

GoLeftWord

Moves the specified number of words to the left of the previously found word or phrase.

Syntax

```
()
```

Parameters

An integer indicating the number of words to move to the left.

Returns

True if a word is found. Otherwise, **False**.

Level

Page or field level.

Details

Starting from a word or phrase found by a previous Locate action, the location position is moved to the left on the current line by the number of words indicated by the input parameter. Subsequent Locate actions can then perform additional operations on this located word. Regardless of being called at the page or field level, this action operates on the recognized text for the current page.

Attention: If the action is added with an empty or non-numeric parameter, the action will default the argument to '1'. This action is always called after a search action has been used.

Example

```
WordFind("Total")
GoLeftWord("1")
GoBelowWord("1")
UpdateField()
```

If the fingerprint has a table with columns such as Total and Tax, the actions above will locate the Tax amount below ("344.76"):

```
Tax Total
344.76 13,774.00
```

GoRightWord

Moves the specified number of words to the right of the previously found word or phrase.

Syntax

()

Parameters

An integer indicating the number of words to move to the right.

Returns

True if a word is found. Otherwise, **False**.

Level

Page or field level.

Details

Starting from a word or phrase found by a previous Locate action, the location position is moved to the right on the current line by the number of words indicated by the input parameter. Subsequent Locate actions can then perform additional operations on this located word. Regardless of being called at the page or field level, this action operates on the recognized text for the current page.

Attention: If the action is added with an empty or non-numeric parameter, the action will default the argument to '1'. This action is always called after a search action has been used.

Example

```
WordFind("Tax")GoRightWord("1")
GoBelowWord("1")
UpdateField()
```

If the fingerprint has a table with columns such as Total and Tax, the actions above will locate the Total amount below ("13,774.00"):

```
Tax Total 344.76 13,774.00
```

GoUpLine

Moves up the specified number of lines from the previously found word or phrase and selects the first word.

Syntax

()

Parameters

An integer indicating the number of lines to move up above the current line.

Returns

True if a line is found. Otherwise, **False**.

Level

Page or field level.

Details

Starting from the current position, the location position is moved up by the number of lines indicated in the input parameter and is moved to the first word of that line.

Attention: If the action is added with an empty or non-numeric parameter, the action will default the argument to '1'. This action is always called after a search action has been used.

Example

```
FindKeyList("Invoice")
GoUpLine("1")
GoRightWord("1")
UpdateField()
```

This sequence finds and extracts the entered value of the Date field ("2/13/03"), in a fingerprint with these words:

```
Date: 2/13/03
INVOICE
```

GroupWords

Groups words to the left and right of the previously found word if they are no more than the specified number of character widths apart.

Syntax

()

Parameters

Long value of the maximum character width separating words to the right and left of the current word.

Returns

Always **True**.

Level

Page or field level.

Details

Groups any words to the left and right of a located word if the target words are themselves separated by a character width equal to or less than the character width you specify as a parameter. Regardless of being called at the page or field level, this action operates on the recognized text for the current page.

Example

```
WordFind("Treasury")  
GroupWords("1.5")
```

If a line contains these words:
20 000 U S Treasury 7 33

the GroupWords action merges "20" with "000"; "U" with "S" and "7" with "33" to produce:

```
20 000 U S Treasury 7 33
```

GroupWordsLEFT

Groups words to the left of the previously found word if they are no more than the specified number of character widths apart.

Syntax

()

Parameters

Long value of the maximum character width separating words to the left of the current word.

Returns

Always **True**.

Level

Page or field level.

Details

Groups only words to the left of the located word if the target words are separated by a character width equal to or less than the character width you specify as a parameter. Regardless of being called at the page or field level, this action operates on the recognized text for the current page.

Example

```
WordFind("000")  
GroupWordsLeft("2")
```

If a line contains these words:
Found 20 000 US Treasury bills.

the GroupWordsLeft action merges "20" with "000" to produce:
20 000

GroupWordsRIGHT

Groups words to the right of the previously found word if they are no more than the specified number of character widths apart.

Syntax

()

Parameters

Long value of the maximum character width separating words to the right of the current word.

Returns

Always **True**.

Level

Page or field level.

Details

Groups words to the right of the located word if the target words are separated by a character width equal to or less than the character width you specify as a parameter. Regardless of being called at the page or field level, this action operates on the recognized text for the current page.

Example

```
WordFind("US")  
GroupWordsRight("2")
```

If a line contains these words:
Found 20 000 US Treasury bills.

the GroupWordsRight action merges "US" with "Treasury" to produce:
US Treasury

IsAlpha

Determines whether the specified percentage of characters in a located word is letters (defaults to 100%)

Syntax

()

Parameters

An integer (0-100) indicating the minimum percentage of characters that must be alphabetic. If no value is provided, the percentage defaults to 100; all characters must be alphabetic.

Returns

True if the minimum percentage of characters specified by the parameter is alphabetic. Otherwise, **False**.

Level

Page or field level.

Details

Using the current location of a previously located word or phrase, this action determines if the characters are alphabetic. By testing the type of characters recognized in the current word or phrase, it is possible for an application to determine it has located the type of data that is required, and then take subsequent actions based on the result of the test. Regardless of being called at the page or field level, this action operates on the recognized text for the current page.

Example

```
FindKey(Name)  
GoRightWord("1")  
IsAlpha("100")
```

If the located word's recognized value is: ABC1

IsAlpha("75") returns **True**

IsAlpha("80") returns **False**

IsCurrency

Determines whether the value of the located word is a currency value. The value contains numbers and includes a two-digit decimal amount.

Syntax

()

Parameters

None.

Returns

True if the located value is currency. Otherwise, **False**.

Level

Page or field level.

Details

Using the current location of a previously located word or phrase, this action determines if the characters are in a valid currency format and are a valid currency

value. By testing the type of characters recognized in the current word or phrase, it is possible for an application to determine it has located the type of data that is required, and then take subsequent actions based on the result of the test. Regardless of being called at the page or field level, this action operates on the recognized text for the current page.

Attention: This action is NOT dynamic locale aware and uses a simple regex and separator character test.

Example

If the recognized value of the word or phrase is: 12.00
IsCurrency() returns TRUE

If the recognized value of the word or phrase is: 12,00
IsCurrency() returns TRUE

If the recognized value of the word or phrase is: 1200
IsCurrency() returns FALSE

If the recognized value of the word or phrase is: 1,200.00
IsCurrency() returns TRUE

If the recognized value of the word or phrase is: \$12.00
IsCurrency() returns TRUE

IsDateValue

Determines whether the value of the located word is in one of the supported date formats.

Syntax

()

Parameters

None.

Returns

True if the located value is an acceptable Date. Otherwise, **False**.

Level

Page or field level.

Details

Using the current location of a previously located word or phrase, this action determines if the characters are in a valid date format and are a valid date. By testing the type of characters recognized in the current word or phrase, it is possible for an application to determine it has located the type of data that is required, and then take subsequent actions based on the result of the test. Regardless of being called at the page or field level, this action operates on the recognized text for the current page.

Attention: This action is dynamic locale aware.

Example

If the located word's recognized value is: 01/01/2003
IsDateValue() returns TRUE

If the located word's recognized value is: January 01,2003
IsDateValue() returns TRUE

If the located word's recognized value is: 13/13/2003
IsDateValue() returns FALSE

IsNumber

Determines whether the specified percentage of characters in a located word is numbers (defaults to 100%)

Syntax

()

Parameters

An integer (0-100) indicating the minimum percentage of characters that must be numeric. If no parameter is specified, the value defaults to 100 percent; all characters must be numeric.

Returns

True if the located value meets the parameter's requirement for an integer. Otherwise, **False**.

Level

Page or field level.

Details

Using the current location of a previously located word or phrase, this action determines if the characters are numeric. By testing the type of characters recognized in the current word or phrase, it is possible for an application to determine it has located the type of data that is required, and then take subsequent actions based on the result of the test. This action does not consider the decimal symbol, digit grouping symbol or a currency symbol to be numeric. Regardless of being called at the page or field level, this action operates on the recognized text for the current page.

Example

```
WordFind("Total")  
GoRightWord("1")  
IsNumber("100")
```

If the located word's recognized value is: #755

```
IsNumber("75") returns TRUE  
IsNumber("80") returns FALSE
```

IsValue

Determines whether the value of the located word matches the value that is specified.

Syntax

()

Parameters

The value to be compared to the object's recognized value.

Returns

True if the located value matches the parameter's value. Otherwise, **False**.

Level

Page or field level.

Details

Using the current location of a previously located word or phrase, this action determines if the value matches the value of the input parameter. By testing the value recognized in the current word or phrase, it is possible for an application to determine it has located the data that is required, and then take subsequent actions based on the result of the test. If you want to check the value of a subset of the word or phrase, use the `ValueInWord` action. Regardless of being called at the page or field level, this action operates on the recognized text for the current page.

Attention: Match test is not case sensitive and does not include leading and trailing spaces.

Example

```
WordFind("Houston")  
GoRightWord("2")  
IsValue("77770")
```

This sequence confirms that the current page's recognized value for Houston's ZIP code is "77770".

The action returns a Boolean value: **True** if the values are the same, **False** if they are not.

IsValue_RegEx

Determines whether the value of the located word matches the regular expression specified.

Syntax

()

Parameters

A Regular Expression that will be used for comparison with the recognized value of the word or phrase.

Returns

True if the located value matches the parameter's value. Otherwise, **False**.

Level

Page or field level.

Details

Using the current location of a previously located word or phrase, this action determines if the regular expression provided in the input parameter finds a match in the value. By testing the value recognized in the current word or phrase, it is possible for an application to determine it has located the data that is required, and then take subsequent actions based on the result of the test. Regardless of being called at the page or field level, this action operates on the recognized text for the current page.

Example

```
WordFind("Houston")
GoRightWord("2")
IsValue_RegEx("Total")
```

MaxLength

Determines whether the number of characters in the located word is greater than or equal to the number specified.

Syntax

()

Parameters

An integer specifying the maximum number of characters the word or phrase can contain.

Returns

False if the parameter is not Numeric, or if the actual number of characters exceeds the parameter. Otherwise, **True**.

Level

Page or field level.

Details

Compares the number of characters in the located word or phrase to a maximum number you supply as the parameter. Regardless of being called at the page or field level, this action operates on the recognized text for the current page.

Example

```
If the recognized value of the located word or phrase is: ANYTHING
MaxLength("14") returns TRUE
MaxLength("8") returns TRUE
MaxLength("3") returns FALSE
```

MergeWordLF

Merges the located word with one or more words to the left, on the same line.

Syntax

()

Parameters

An integer that indicates the number of words or phrases to the left of the previously found field that is to be placed into the current object field.

Returns

Always **True**.

Level

Page or field level.

Details

Merges the located word or phrase with one or more words to the left, on the same line. A "word" in this context is a string of characters that might include spaces. This action is used when the value searched for might have spaces in it. Regardless of being called at the page or field level, this action operates on the recognized text for the current page.

Example

```
WordFind("2000")  
MergeWordLF("1")UpdateField()
```

Given the following cco string:
Invoice Date: Jan 2000

FindWord("2000") locates the highlighted value:
Invoice Date: Jan 2000

MergeWordLF("1") consolidates it with the text "Jan":
Invoice Date: Jan 2000

so that the UpdateField action will save the entire value,
"Jan 2000" into the current object's field.

MergeWordRT

Merges the located word with one or more words to the right, on the same line.

Syntax

()

Parameters

An integer indicating the number of words to the right, starting from the previously found word or phrase, to be placed into a field.

Returns

Always **True**.

Level

Page or field level.

Details

Places the located word or phrase with one or more words to the right, on the same line, into the current object field. Regardless of being called at the page or field level, this action operates on the recognized text for the current page.

Example

```
WordFind("Jan")  
MergeWordRT("1")  
UpdateField()
```

For the cco string, Invoice Date: Jan 2000 , the result of WordFind("Jan") is to locate the highlighted word, Jan, in Invoice Date: Jan. MergeWordRt("1") consolidates that value with the year value to its right, which is Invoice Date: Jan 2000. UpdateField saves the entire value, Jan 2000, which is saved to the calling object field.

MinLength

Determines whether the number of characters in the located word is less than or equal to the number specified.

Syntax

()

Parameters

An integer specifying the minimum number of characters the word or phrase can contain.

Returns

False if the parameter is not Numeric, or if the actual number of characters is less than the parameter. Otherwise, **True**.

Level

Page or field level.

Details

Compares the number of characters in the located word or phrase to a minimum number you supply as the parameter. Regardless of being called at the page or field level, this action operates on the recognized text for the current page.

Example

```
If the recognized value of the word or phrase is: ABC1  
MinLength("4") returns TRUE  
MinLength("3") returns TRUE  
MinLength("6") returns FALSE
```

RegExFind

Same as the WordFind action, except that it supports regular expressions.

Syntax

()

Parameters

A word or phrase to find on the current page. The parameter is expected to be a Regular Expression.

Returns

True if the word or phrase is located on the page. Otherwise, **False**.

Level

Page level.

Details

Locates the first occurrence of a word or phrase on the current page where the input search term is specified as a regular expression. The search is started from the first word on the current page. The location of the found word or phrase will be remembered so the result can be used by subsequent actions. The search is case sensitive.

Example

```
RegExFind("Da?e")  
GoRightWord("1")  
IsDateValue()  
UpdateField()
```

RegExFind will look for the first occurrence of a word or phrase that matches the regular expression. If a match is found, it continues processing by moving one word to the right of the result of RegExFind and acts on that word. If it is a date format, UpdateField will place the value into the current field.

RegExFind_InZone

Same as the RegExFind action, except that it searches the current field only.

Syntax

()

Parameters

A word or phrase to find in the current field. The parameter is expected to be a Regular Expression.

Returns

True if the word or phrase is located in the current field. Otherwise, **False**.

Level

Field level.

Details

Locates the first occurrence of a word or phrase in the current field where the input search term is specified as a regular expression. The search is started from the first word of the current field. The location of the found word or phrase will be remembered so the result can be used by subsequent actions. The search is case sensitive.

Example

```
RegExFind_InZone("Da?e")GoRightWord("1")
IsDateValue()
UpdateField()
```

RegExFind_InZone will look for the first occurrence of a word or phrase that matches the regular expression. If a match is found, it will continue processing by moving one word right of the result of RegExFind and act on that word. If it is a date format, UpdateField places the value into the current field.

RegExFindNext

Same as the WordFindNext action, except that it supports regular expressions.

Syntax

()

Parameters

A word or phrase to find on the current page. The parameter is expected to be a Regular Expression.

Returns

True if the word or phrase is located on the page. Otherwise, **False**.

Level

Page level.

Details

Starting from the location of a previously found word or phrase, this action locates the first occurrence of a word or phrase on the current page where the input search term is specified as a regular expression. The search is started from the location of a previously found word or phrase. The location of the found word or phrase will now be remembered so the result can be used by subsequent actions. The search is case sensitive.

Example

```
RegExFind("ItemID")
RegExFindNext("Desc")
```

In this sequence, the first action looks for ItemID. If the search succeeded, RegExFindNext looks for the first occurrence of Desc that comes after ItemID.

RegExFindNext_InZone

Same as the RegExFindNext action, except that it searches the current field only.

Syntax

()

Parameters

A word or phrase to find in the current field. The parameter is expected to be a Regular Expression.

Returns

True if the word or phrase is located on the page. Otherwise, **False**.

Level

Page level.

Details

Starting from the location of a previously found word or phrase, this action locates the first occurrence of a word or phrase in the current field where the input search term is specified as a regular expression. The search is started from the location of a previously found word or phrase in the current field. The location of the found word or phrase will be remembered so the result can be used by subsequent actions. The search is case sensitive.

Example

```
RegexFind_InZone("ItemID")  
RegexFindNext_InZone("Desc")
```

In this sequence, the first action looks for ItemID. If the search succeeded, RegexFindNext looks for the first occurrence of Desc that comes after ItemID.

ScanRT

Moves the specified number of words to the right of the current word. Expands the search area up and down slightly in case the word is a little above or below the current word.

Syntax

()

Parameters

Numeric value of the number of words to be evaluated to the right of the current word or phrase.

Returns

True, if a word is found. Otherwise, **False**.

Level

Field level only.

Details

ScanRT (scan right) looks for a word in positions that are slightly above or below the line on which the current word or phrase is located.

Example

```
WordFind("Number")  
ScanRT("1")
```

In this example, the action finds the Number on the current page. Then, it moves one word to the right as it searches for a value.

To compensate for the possibility that this value might be printed slightly above or below the line on which *Number* was printed, the ScanRT action expands the area for the target value. When a page is recognized, then skewing or data layout might cause alignment problems. Words might not be recognized as being on the same line and might be recognized as being slightly above or below the current line.

This action compensates for alignment problems at the current word location. The action uses a calculation adjustment to determine which of the words to the right of the current word can be considered to be on the same line. It then remembers the location of the word that best fits this criteria. This new remembered location can then be used by subsequent actions.

SelectSnippet

Populates a snippet field with the recognized value of the located word. Used with directional actions.

Syntax

()

Parameters

1. The character that is to appear in the Snippet field if a value is not available. "~" is the default.
2. The size of the captured value's snippet image. 1" is 1x of the actual word area, and is the default. "2" is 2x of the actual word area, the zone is twice the size of the selected target word. ".5" is 1/2 the actual word area, not recommended as only a portion of the target word would display in the snippet at verify time.

Returns

Always **True**.

Level

Field level only.

Details

Used in conjunction with directional actions, this action will populate a Snippet field with the recognized value of the located word or phrase.

This action is usually the last rule of a Locate RuleSet, and is used when the probable area of the sought after value has been found.

Example

```
FindKeyInList("InvNum")
GoRightWord("1")
SelectSnippet("~",1")
```

This sequence first tries to locate an Invoices Number keyword in the current page. If successful, the next action attempts to lock onto a word or phrase to the right of the located word or phrase.

If that word is present, the SelectSnippet action will place the image of the word's recognized value into the Snippet of the applicable Field object. The

Data Entry operator can then determine if the Snippet contains the correct value and can enter the data into the accompanying Data Edit field in the Data Entry Panel.

SetRect

Sets the position and size of the current field in the page data file to the values specified.

Syntax

()

Parameters

Four comma separated coordinates designating the rectangle's size and location: X, Y, Width and Height.

Returns

Always **True**.

Level

Field level only.

Details

Updates the zone of the current field.

Example

```
SetRect("0,0,100,200")
```

This action changes the field's zone position to a rectangle at coordinates 0, 0, which is 100 wide and 200 high.

This action is useful if you wish to change the zone to a specific area of the image where you know your value resides.

UpdateDCOField

Updates the position coordinates for the specified field in the page data file with the position of the located word.

Syntax

()

Parameters

String value of the target field's name (Smart Parameter enabled), as a Field object of the Document Hierarchy.

Returns

False if the source page's Fingerprint file (.cco) is not available or is empty; if the target field cannot be found; or if there is no information about the target field's starting and ending sizes and locations. Otherwise, **True**.

Level

Page or Field level.

Details

This action updates the size and position coordinates of the Field object representing the field identified by the parameter. Typically, the action follows earlier actions and events which modify the field's width and height, or its precise placement on the current source page.

Do not confuse this action with the UpDateField action, which updates Text values. This action does not update a field's Text value. Instead, it modifies the size and location parameters of a field or zone.

Example

```
To find a sibling field 'Preparer_Name':  
UpdateDCOField("../Preparer_Name")
```

```
To find a child field 'Preparer_Name':  
UpdateDCOField("Preparer_Name")
```

UpdateField

Updates the current field in the page data file with the value and position of the located word.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

Field level only.

Details

Updates the appropriate field in the current page's Data file with the recognized (and possibly formatted) value of the located word or phrase.

Important: An entered value that the UpdateField action places in a Data file becomes a captured value, and can be processed by Validation and Export RuleSets.

Example

```
FindKeyList("Date")  
GoRightWord("1")  
IsDateValue()  
UpdateField()
```

The first action in the sequence finds a word or phrase that identifies the Date Field object of the current page. This is the field's static value - probably its title.

The next action moves right one word or phrase to locate the field's entered value, a recognized date such as 12/31/2002. The third action checks to be sure the value has an acceptable Date format.

The concluding UpdateField action takes place only if the others are successful. It adds the field's entered value to the current page's Data file, where it is a captured value awaiting the attention of upcoming rules with Validate and Export actions.

ValueInField

Determines whether any part of the located word matches the value that is specified.

Syntax

()

Parameters

The value that is to be matched with a portion of the value in the current field.

Returns

False if no match occurs. Otherwise, **True**.

Level

Field level.

Details

Checks if the parameter you enter is within the value of the current field represented by the bound Field object of the Document Hierarchy. Only a portion of the field's value needs to match the parameter. If the entire field must match, use IsValue. Case insensitive.

Example

```
ValueInField("Invoice")
```

ValueInField_Fuzzy

Uses fuzzy matching to determine whether any part of the located word matches the value that is specified.

Syntax

()

Parameters

String value to be matched to the current field's value, using fuzzy matching procedures.

Returns

False if no match occurs. Otherwise, **True**.

Level

Field level only.

Details

Checks if there is a "fuzzy" match of the parameter's value with the value in the current field. Only a portion of the field's value needs to match. The match is performed by allowing for common substitutions that can occur during recognition. These substitutions include characters: B8, Z2, S5, oO0 and iItl1. Case insensitive.

Example

```
ValueInField_Fuzzy("Invoice")
```

ValueInField_RegEx

Determines whether any character or series of characters in the located word matches the specified regular expression.

Syntax

()

Parameters

The portion of the value to find in the field. The parameter is expected to be expressed as a regular expression.

Returns

False if no match occurs. Otherwise **True**.

Level

Field level only.

Details

This action checks if the Regular Expression you specify as the parameter is equivalent to the value of the current field. Only a part of the field must match the parameter. To match the entire value of the field, use `IsValueRegEx`.

Example

```
ValueInField_RegEx("[^\b\s\n\r]Inv[o00][iItl1]ce[\b\s]*")
```

WordFind

Locates the first or next occurrence of the specified word or phrase on the current page.

Syntax

()

Parameters

A word or phrase to find on the page.

Returns

True, if the word or phrase is on the page. Otherwise, **False**.

Level

Field level only.

Details

The current page is searched to find the whole word or phrase. The location of the first word or phrase that matches the parameter is remembered and can be used by subsequent actions. Matching is case-sensitive.

To improve matching, this action automatically adjusts the search criteria to allow for common character substitutions. For example, if the list includes `will` and the recognition that is read is `wll`, a match still occurs.

This action matches whole word values and does not search for parts of words or characters within words.

Common substitutions include characters: **B8**, **Z2**, **S5**, **oO0**, and **iItl1**

Example

```
WordFind("Sales Tax")
GoRightWord("1")
IsCurrency()
UpdateField
```

In this example, the `WordFind` action looks for the first occurrence of `Sales Tax` within the current page, always starting at the first word of the page. If the phrase `Sales Tax` is found, the subsequent actions operate based on the location of the found phrase.

WordFind_InZone

Same as the `WordFind` action, except that it searches the current field only.

Syntax

()

Parameters

A word or phrase to find in the current zoned field.

Returns

True if the word or phrase is located in the field. Otherwise, **False**.

Level

Page or Field

Details

The current field will be searched to find the word or phrase. The location of the first word or phrase that matches the parameter will be remembered so it can be utilized by subsequent actions. Matching is case sensitive.

To improve matching, this action automatically adjusts the search criteria to allow for common character substitutions. For example, if the list includes "will" and the recognition read "wi11", a match will still occur.

Common substitutions include characters: B8, Z2, S5, oO0 and i1t1.

Example

```
WordFind_InZone("Sub Total")
GoRightWord("1")
IsCurrency()
```

In this example, the WordFind action looks for the first occurrence of Sub Total within the current field, always starting at the first word of the zone. If the phrase Sub Total is found, the subsequent actions will operate based on the location of the found phrase.

WordFindNext

Same as the WordFind action, except that it locates the next occurrence.

Syntax

()

Parameters

A word or phrase to find on the page, following a previously found word or phrase.

Returns

True if the parameter is located. Otherwise, **False**.

Level

Field level only.

Details

The current page will be searched to find the word or phrase, starting from the location remembered from the last search, such as from WordFind. The location of the first word or phrase that matches the parameter will now be remembered so it can be utilized by subsequent actions. Matching is case sensitive.

To improve matching, this action automatically adjusts the search criteria to allow for common character substitutions. For example, if the list includes "will" and the recognition read "wi11", a match will still occur.

Common substitutions include characters: B8, Z2, S5, oO0 and i1t1.

Example

```
WordFindNext("Total")
WordFindNext("Total")
GoRightWord("1")
IsCurrency()
UpdateField
```

In this sequence, the Locate rule locates the third instance of the word Total in the current page.

WordFindNext_InZone

Same as the WordFindNext action, except that it searches the current field only.

Syntax

()

Parameters

A word or phrase to find in the current field, following a previously found word or phrase.

Returns

True if the parameter is located. Otherwise, **False**.

Level

Field level.

Details

The current field will be searched to find the word or phrase, starting from the remembered location of a previous find. The new location of the first word or phrase in the field that matches the parameter will now be remembered so it can be utilized by subsequent actions. Matching is case sensitive.

To improve matching, this action automatically adjusts the search criteria to allow for common character substitutions. For example, if the list includes "will" and the recognition read "wi11", a match will still occur.

Common substitutions include characters: B8, Z2, S5, oO0 and i1t1.

Example

```
WordFind_InZone(ItemID)
WordFindNext_InZone(Desc)
```

In this example, the WordFind_InZone action looks for the first occurrence of ItemID within the current field, always starting at the first word of the zone. If the phrase ItemID is found, the word Desc will be looked for in the current field, starting after the location of ItemID. If Desc is found, any subsequent actions will operate based on the location of that found phrase.

WordFind_Offset

Sets the value of the page's Image_Offset variable. The variable is based on the difference in the position of the specified word on the current page and on the matched fingerprint image.

Syntax

()

Parameters

1. String value of a keyword that the action is to find on both the fingerprint and recognized image.
2. An optional parameter that specifies the offset threshold. If not specified, the default value is 100 pixels.

Returns

Always **True**.

Level

Page level.

Details

This action locates a word or phrase on both the recognized page and on the fingerprint. The positioning of both locations are compared to determine an offset value. The calculated difference is stored in the DCO of the current page in the variable `Image_Offset`. This value will be used by subsequent actions, such as `ReadZones`, to compensate for the difference so the field data is properly located.

For best results, the word or phrase should appear only once or the first instance of the word or phrase should always appear in the same location.

The threshold is the maximum distance between keywords found on the Live image and the fingerprint. The Default value is 100 pixels. If the keywords are more than 100 pixels apart no Offset is generated; preventing matches of keywords where there is more than one instance of the word to be found on an image. A successful matched pair will update the `Image_Offset` variable at the page level.

This action requires the FingerPrint CCO to have full page recognition results. Otherwise, there will be nothing to match against.

This action should not be located in the same ruleset where full page recognition is done. It needs to exist in a ruleset that is performed afterwards.

Example

```
WordFind_Offset("Invoice")
```

Lookup actions

Use the Lookup actions to validate field values by using database lookups and populate fields with lookup results.

The Lookup actions opens a connection to the Lookup database and populates the current field with a value returned by the previous `ExecuteSQL` or `LookupReturnValue` action.

“ClearLookupResults”

“CloseConnection” on page 713

“ExecuteSQL” on page 714

“OpenConnection” on page 714

“PopulateWithResult” on page 715

“SmartSQL” on page 716

ClearLookupResults

Clears the results that are returned by the previous Lookup action.

Syntax

```
()
```

Parameters

None.

Returns

Always **True**.

Level

All.

Details

This action clears the stored results returned from a previous Lookup action such as `PopulateWithResult`.

Example:

```
OpenConnection("@APPVAR(*/*lookupdb:cs)")
ExecuteSQL("SELECT NAME, ADDRESS FROM Vendor;")
PopulateWithResult("1")
ClearLookupResults()
```

Here, `ClearLookupResults()` clears the stored results of the Vendor Name and Address.

CloseConnection

Closes an open connection to the Lookup database.

Syntax

()

Parameters

None.

Returns

True, even if the connection is already closed.

Level

All, but generally used as part of a separate RuleSet at the Batch level.

Details

Closes an open connection to your Lookup database.

Usually, this action is placed in a RuleSet that is separate from the RuleSet that opens the connection and stores the data.

`LookupDBCclose` RuleSet, for example, is run at the Batch level after all data has been exported from the batch to the specified database.

Example:

```
CloseConnection()
```


This action closes the previously opened connection to the Lookup database. This action is usually part of a separate RuleSet that prevents the need to repeatedly open the connection to the database. You can open the connection once in the first RuleSet, use the database from all documents and pages in the batch, then close the connection once in the second RuleSet.

ExecuteSQL

Runs a SQL statement on the Lookup database. If a SELECT statement returns one or more values, these values are stored in an internal data record that you can access by using the PopulateWithResult action.

Syntax

()

Parameters

1. The SQL expression that you want to run surrounded by quotation marks (" "). '%s' or %s can be substituted in the SQL expression to represent a field value.
 - If %s represents a text field, it must be surrounded by ' '.
 - If %s represents a numeric field, it appears without surrounding apostrophe characters.
2. ,3+ Field names whose captured values you want to use in the SQL expression (see the example).

Returns

True, if the SQL statement runs successfully, select statements must also return a value. **False**, if it does not.

Level

All.

Details

Runs the SQL statement that you enter in the first parameter. Field values can be substituted for "%s" in the statement.

Example:

This sequence opens a connection to the InvoiceLook database. Next, it inserts values into the CompanyCode and Type columns of the Vendor table:

```
OpenConnection("@APPVAR(* /lookupdb:cs)")
ExecuteSQL("INSERT INTO Vendor (CompanyCode,Type) VALUES ('MQSW','New')")
```

Here, dbVendorID is a numeric field, while dbVendorname is a text field.

```
OpenConnection("@APPVAR(* /fingerprintconn:cs)")
ExecuteSQL("SELECT CompanyCode FROM Vendor WHERE dbVendorID = %s
AND dbVendorName = '%s';",VendorID,VendorName")
```

OpenConnection

Uses a data source name or connection string to open a connection to a Lookup database.

Syntax

()

Parameters

The Data Source Name or Connection String.

If the action is establishing a connection with an Oracle database, or a SQL Server database by using SQL Authentication, be sure to expand the DSN parameter by adding the correct Provider, user ID and Password.

Smart parameters are supported to prevent plain text credentials in the application rules.

Returns

True if the action results in a connection to the database. Otherwise, **False**.

Level

All.

Details

This action uses the Connection String that you provide as the parameter to open a connection to your LookUp database.

Example:

```
OpenConnection("@APPVAR(* /fingerprintconn:cs)")
```

This example opens the fingerprint database and obtains the connection information from the Application Service. It is recommended to use the application service so passwords are kept hidden.

Access:

```
OpenConnection("Provider=MSACCESS;DSN=C:\Datacap\1040EZ\1040ezLook.mdb;  
UID=;PWD=;")
```

Oracle:

```
OpenConnection("Provider=OraOLEDB.Oracle.1;Password=mypassword;  
Persist Security Info=True;User ID=myuserid;Data Source=TM2")
```

SQL Server Authentication:

```
OpenConnection("Provider=SQLOLEDB.1;Integrated Security=SSPI;  
Persist Security Info=False;Initial Catalog=mycatalog;Data Source=mysource")
```

PopulateWithResult

Populates the current field with a value returned by the previous ExecuteSQL or LookupReturnValue action. If the previous action returned multiple values, you can specify the value that you want to use.

Syntax

()

Parameters

1. A number that indicates the value in a record that is retrieved by an earlier ExecuteSQL or SmartSQL action is to be assigned to the current Field object (and added to the Data file of the current page.)

- "1" refers to the first column in a recordset, "2" refers to the second column, and so on.
2. **True** or **False**. **True** causes the action to fail, if the action returns a recordset with multiple lines. **False** permits the action to accept a record set with multiple lines but to use values in the first record of the record set.

Returns

True, if the second parameter is **True** and a previous ExecuteSQL or SmartSQL action finds a recordset with only one record.

True, if the second parameter is "False" and a previous ExecuteSQL (or SmartSQL) action finds a recordset with one or more records.

Otherwise, **False**.

Level

Field level only.

Details

Populates a Field object with a database value retrieved by a ExecuteSQL or SmartSQLAction.

This action allows multiple rules to populate multiple Field objects with data from a single database record (see the following example.)

Example:

```
(Field #1)
OpenConnection("@APPVAR(*lookupdb:cs)")
ExecuteSQL("Select * From Vendor Where VendorID = %s;", VendorID")
PopulateWithResult("1, FALSE")

(Field #2)
PopulateWithResult("2, FALSE")
```

In the example, the ExecuteSQL action of the RuleSet applied to Field #1 retrieves the recordset (if it exists).

The PopulateWithResult action places the value of the first record's first column into the field where the rule applied. The PopulateWithResult action of a rule that is applied to Field #2 populates the field with the value of the first record's second column.

False means that the action can accept a recordset with multiple records but extracts values from the first record only.

SmartSQL

Runs a SQL statement that supports smart parameters.

Syntax

()

Parameters

1. The SQL expression that you want to run. Smart parameters are supported within the expression.

2. True/False value to enable or suppress populating value of any record that is returned from the SQL expression.

Returns

True, if the SQL statement runs successfully, select statements must also return a value. **False**, if the SQL statement does not execute successfully.

Level

All.

Details

Runs the SQL statement that you enter in the first parameter.

Example:

This sequence opens a connection to the InvoiceLook database. Next, it inserts values into the CompanyCode and Type columns of the Vendor table:

```
OpenConnection("@APPVAR(* /lookupdb:cs)")
SmartSQL("INSERT INTO Vendor (CompanyCode,Type) VALUES ('MQSW','New')")
```

Here, dbVendorID is a numeric field, while dbVendorname is the calling text field.

```
OpenConnection("@APPVAR(* /lookupdb:cs)")
SmartSQL("SELECT CompanyCode FROM Vendor WHERE dbVendorID =+@P\VendorID+
AND dbVendorName = '+@F+';",YES)
```

MC_Identify

Use the MC_Identify actions to identify claim forms in a batch.

The MC_Identify actions identify the medical claim forms that are processing in the batch.

“AutoField”

“FindFields” on page 718

“ReadDCOSetup” on page 719

“ReadPageSetup” on page 719

“SetFormType” on page 720

“SetMaxTolerantDistance” on page 721

AutoField

Identifies red HCFA-1500 or red UB04 forms.

Member of namespace:

MC_Identify

Syntax:

```
bool AutoField()
```

Parameters

None

Returns

False, if the action is not applied at the page level. Otherwise, **True**.

Level

Page level.

Details

This action Identifies red HCFA-1500 or red UB04 forms.

This action must be placed in the rule after the `SetMaxToleranceDistance`, `SetFormType`, `ReadDCOSetup`, and `SetWritePosFile` actions.

Example:

```
SetMaxToleranceDistance(60)
SetFormType(0)
ReadDCOSetup(HFCA.xml,POS 1052)
AutoField()
```

FindFields

Sets up a data file for the current page and provides field position information to the data file.

Member of namespace:

MC_Identify

Syntax:

```
bool FindFields()
```

Parameters

None

Returns

False, if the rule with this action is not bound to a page object of the document hierarchy or if a source page that is represented by the page object is not available. Otherwise, **True**.

Level

Page level.

Details

This action sets up a data file (.xml) for the current page and supplies the data field with field position information.

This action is usually part of a Medical Claims ID_PageFix rule.

Example:

```
SetMaxToleranceDistance(60)
SetFormType(0)
ReadDCOSetup(HFCA.xml,POS 1052)
FindFields`()
```

ReadDCOSetup

Designates the file name of the document hierarchy.

Member of namespace:

MC_Identify

Syntax:

```
bool ReadDCOSetup (StrParam)
```

Parameters

A comma-separated string value that is made up of a smart parameter that designates the file name of the document hierarchy followed by a variable name that indicates the position of the fingerprint. The file name is usually the application ID with an .xml extension. For example, HCFA.xml is name of document hierarchy file of the HCFA application. **POS 1052** specifies a previously assembled HCFA-1500 fingerprint with details of the form's Field IDs, locations, and data types. For use with the Application Service, the syntax of the first parameter changes to include the smart parameter that points to the Setup DCO. For example, it uses @APPPATH(setupdco) instead of HCFA.xml.

Returns

False, if the document hierarchy is not found. Otherwise, **True**.

Level

All levels, but usually the Batch level.

Details

Example:

```
ReadDCOSetup(HCFA.xml,POS 1052)
```

For use with the Application Manager with the @APPPATH smart parameter to locate the Setup DCO:

```
ReadDCOSetup(@APPPATH(setupdco),POS 1052)
```

This action is used with almost every other MC_Identify action.

ReadPageSetup

Designates the file name and path of the document hierarchy.

Member of namespace:

MC_Identify

Syntax:

```
bool ReadPageSetup(string DCOSetupPath,string FPPosition,string PageType)
```

Parameters

string **DCOSetupPath**

string **FPPosition**

string **PageType**

Parameters

1. **PageType**: A smart parameter enabled argument that designates the file name and path of the document hierarchy.
2. **FPPosition**: The fingerprint position variable, usually **Pos** followed by the ID of the desired fingerprint.
3. **PageType**: The page type of the page to identify, usually PClaim or IClaim.

Returns

False, if the document hierarchy file is not found. Otherwise, **True**.

Level

All levels.

Details

Example:

```
ReadPageSetup(@APPPATH(setupdco),POS 1059,PClaim)
```

This action is used with almost every other MC_Identify action.

SetFormType

Set the value of the Form Type that is used by the AutoField action.

Member of namespace:

MC_Identify

Syntax:

```
bool SetFormType(StrParam)
```

Parameters

String value that indicates the Form Type:

- For HCFA-1500 or CMS-1500, use **0** or **hcfa**.
- For UB-04, use **2** or **ub04**.

Returns

False, if the parameter is invalid. Otherwise, **True**.

Level

All levels.

Details

This action sets the Form Type value that is used by the AutoField action.

Example:

```

SetMaxToleranceDistance(60)
SetFormType(0)
ReadDCOSetup(HFCA.xml,POS 1052)
SetWritePosFile(True)
AutoField()

```

SetMaxTolerantDistance

Set the tolerance level that the AutoField action uses to match HCFA-1500 or red UB04 forms.

Member of namespace:

MC_Identify

Syntax:

```
bool SetMaxTolerantDistance(StrParam)
```

Parameters

The **Maximum Tolerance Distance**: an integer from 1 for the lowest tolerance to 100 for the highest tolerance.

Returns

False, if the parameter is not an integer from 1 to 100. Otherwise, **True**.

Level

All levels.

Details

This action sets the tolerance level that is used by the AutoField action to match HCFA-1500 or red UB04 forms based on the form that is specified in the SetFormType action.

Example:

```

SetMaxToleranceDistance(60)
SetFormType(0)
ReadDCOSetup(HFCA.xml,POS 1052)
AutoField()

```

MC_Validation

Use the MC_Validation actions to validate medical claim form information.

The MC_Identify actions validates the information in the medical claim forms that are processing in the batch.

“AddCenturyTo2YearDigit” on page 722

“AddToDetailErrorMsg” on page 723

“AddToErrorMsg” on page 723

“CalculateHCFALineCharges” on page 724

“CalculateUBLLineCharges” on page 724

“CheckDocID” on page 725

“ClearErrorMsg” on page 725

- "CommonParseAddress" on page 726
- "CommonValAddress" on page 727
- "ConvertHyphen" on page 727
- "FilterPID" on page 728
- "FormatFieldLengths" on page 729
- "InheritSnippets" on page 729
- "MC_ReadZones" on page 730
- "Parse31aPhSig" on page 730
- "Parse58ainsnm" on page 731
- "Parse58binsnm" on page 731
- "Parse58cinsnm" on page 732
- "ParseConditionCodes" on page 732
- "ParseEPSDT" on page 733
- "ParseLastFirstIniNames" on page 734
- "ParseNDC" on page 735
- "PopulateFromField" on page 735
- "SetConf" on page 736
- "SetOriginalTIF" on page 736
- "StripTrailingAlpha" on page 737
- "TransformLI" on page 737
- "UpdateCredentialList" on page 739
- "ValidateNPI" on page 740
- "ValProcedureCode" on page 740
- "ValRequiredCode" on page 741

AddCenturyTo2YearDigit

Converts two-digit Year values to four-digit Year values.

Member of namespace:

MC_Validation

Syntax:

```
bool AddCenturyTo2YearDigit()
```

Parameters

None.

Returns

False, if the value is not a valid date in the mmddyy format or if the action is not applied at the Field level. Otherwise, **True**.

Level

Field level.

Details

This action converts two-digit Year values to four-digit Year values.

All dates are assumed to be before today's date with a format of mmddyy. If today is 150507 and this action is applied to a field with a value of 221095, the date is assumed to be 221095.

Example:

```
AddCenturyTo2YearDigit()
```

AddToDetailErrorMsg

Adds the specified value to the existing value for the page variable *ErrorMessage*.

Member of namespace:

MC_Validation

Syntax:

```
bool AddToDetailErrorMsg(StrParam)
```

Parameters

1. A smart parameter or regular string to add to the error message variable.
2. Optional comma separated second parameter to trigger the action to return **True**.

Returns

True, if the optional second comma separated parameter is used. Otherwise, **False**.

Level

Field level.

Details

This action adds the specified value to the existing value for the page variable *ErrorMessage*.

Example:

```
AddToDetailErrorMsg("Description cannot be blank")
```

AddToErrorMsg

Adds the specified value to the existing value for the page variable *ErrorMessage*.

Member of namespace:

MC_Validation

Syntax:

```
bool AddToErrorMsg(StrParam)
```

Parameters

1. A smart parameter or regular string to add to the error message variable.
2. Optional comma separated second parameter to trigger the action to return **True**.

Returns

True, if the optional second comma separated parameter is used. Otherwise, **False**.

Level

Field level.

Details

This action adds the specified value to the existing value for the page variable *ErrorMessage*.

Example:

```
AddToErrorMsg("Invoice Number must be 60% numeric with a  
minimum length of 2.")
```

CalculateHCFALineCharges

Calculates charges for HCFA service lines.

Member of namespace:

MC_Validation

Syntax:

```
bool CalculateHCFALineCharges()
```

Parameters

None.

Returns

True, if the line charges equal the charges field. Otherwise, **False**.

Level

Field level.

Details

This action calculates charges for HCFA service lines.

Example:

```
CalculateHCFALineCharges()
```

CalculateUBLLineCharges

Calculates charges for UB service lines.

Member of namespace:

MC_Validation

Syntax:

```
bool CalculateUBLLineCharges()
```

Parameters

None.

Returns

True, if the line charges equal the charges field. Otherwise, **False**.

Level

Field level.

Details

This action calculates charges for UB service lines.

Example:

```
CalculateUBLLineCharges()
```

CheckDocID

Checks the document IDs and updates them to the proper format.

Member of namespace:

MC_Validation

Syntax:

```
bool CheckDocID()
```

Parameters

None.

Returns

True, if the docid is formatted without an error. Otherwise, **False**.

Level

Document level.

Details

This action checks the document IDs and updates them to the proper format.

Example:

```
CheckDocID()
```

ClearErrorMsg

Clears the value of the page variable *ErrorMessage*.

Member of namespace:

MC_Validation

Syntax:

```
bool ClearErrorMsg()
```

Parameters

None.

Returns

Always **True**.

Level

Page level.

Details

This action clears the value of the page variable *ErrorMessage*.

Example:

```
ClearErrorMsg()
```

CommonParseAddress

Parses the addresses in the HCFA and UB92 fields into appropriate sub-fields.

Member of namespace:

MC_Validation

Syntax:

```
bool CommonParseAddress(StrParam)
```

Parameters

For HCFA, string value of:

1. HCField32Object: for parsing the Facility Address field (field 32).
2. HCField33Object: for parsing the Physician Address field (field 33).

For UB04, string value of:

1. UBField1Object: for parsing the Provider Address field (field 1).
2. UBField2Object: for parsing the Pay-To Address field (field 2).
3. UBField13Object: for parsing the Patient Address field (field 13).
4. UBField38Object: for parsing the Responsible Party Name and Address field (field 38).

Returns

False, if the parameter is invalid or if the action is not on the Page level.
Otherwise, **True**.

Level

Page level.

Details

This action parses the addresses in the following fields into appropriate sub-fields:

- HFCA - Facility Address (field 32) or Physician Address (field 33)

- UB92 - Patient Address (field 13), Facility Address (field 1), or Responsible Party Address (field 38).

Example:

```
CommonParseAddress(HCField320Object)
```

CommonValAddress

Validates the address values first name, last name, street, city, state, zip code, and phone number.

Member of namespace:

MC_Validation

Syntax:

```
bool CommonValAddress(StrString)
```

Parameters

Comma-delimited String that contains a list of name with addresses to be validated.

Returns

False, if the action is not run at the Page level. Otherwise, **True**.

Level

Page level.

Details

This action validates the following address values:

1. First Name: value can start with Ms, Mr, Miss, Dr salutations. The remaining values must be alphanumeric with no special characters. Punctuation is allowed only after the salutation.
2. Last Name: same requirements as the first name.
3. Street: alphanumeric, upper or lower case. Can include punctuation and the # character.
4. City: characters from A to Z, upper or lower case, comma, period, space, and the & character.
5. State: must be 2 alphanumeric characters.
6. Zip Code: must be between 5 and 9 characters. This value is checked against the State value above.
7. Phone Number: the area code is checked against the State and Zip Code values above.

Example:

```
CommonValAddress(Insured,4InsFNam,4InsLNam,7|AddStr,7|AddCity,
7|AddSta,7|AddZip)
or
CommonValAddress(Description,12plname,12pfname,13paddr1,13paddr2,
13padcit,13padsta,13padzip)
```

ConvertHyphen

Removes spaces, commas, hyphens, and invalid characters.

Member of namespace:

MC_Validation

Syntax:

```
bool ConvertHyphen()
```

Parameters

None.

Returns

False, if the parameter is not called at the Field level. Otherwise, **True**.

Level

Field level.

Details

This action removes spaces, commas, hyphens, and invalid characters.

"1,2,3,4" becomes "1234", "1-2-3®" becomes "123". Valid characters for this field are {1,2,3,4}.

Characters other than 1,2,3,4, space, commas, or hyphens will lower the field confidence level.

Example:

```
ConvertHyphen()
```

FilterPID

Filters the qualifier from the attending physician for UB04 claims.

Member of namespace:

MC_Validation

Syntax:

```
bool FilterPID(StrParam)
```

Parameters

The name of the field to filter.

Returns

False, if not called at the Field level. Otherwise, **True**.

Level

Field level.

Details

This action filters the qualifier from the attending physician for UB04 claims.

Example:

```
FilterPID(76apqual)
```

FormatFieldLengths

Truncates the length of the field and sets the last character of the field to low confidence.

Member of namespace:

MC_Validation

Syntax:

```
bool FormatFieldLengths()
```

Parameters

None.

Returns

Always **True**.

Level

Page level.

Details

This action truncates the length of the field and sets the last character of the field to low confidence.

Example:

```
FormatFieldLengths()
```

InheritSnippets

Assigns the snippet position information of the current Field object to the Field objects that are specified in the parameter.

Member of namespace:

MC_Identify

Syntax:

```
bool InheritSnippets(StrParam)
```

Parameters

The names of the fields that will inherit the same snippet information as the current Field object.

For example, 2paLname, 2PaFname, aPaMInit.

Returns

False, if the action is not called at the Field level or if a parameter is incorrect. Otherwise, **True**.

Level

Field level.

Details

This action assigns the snippet position information of the current Field object to the Field objects that are specified in the parameter.

Example:

```
InheritSnippets(2paLname, 2PaFname, aPaMInit)
```

MC_ReadZones

Adjusts the autofield that is based on the OMR field zone positions on the calling page.

Member of namespace:

MC_Validation

Syntax:

```
bool MC_ReadZones()
```

Parameters

None.

Returns

False, if the action is not called at the Page level. Otherwise, **True**.

Level

Page level.

Details

This action adjusts the autofield that is based on the OMR field zone positions on the calling page.

Important: This action handles Autofield-based OMR zone detection for Medical Claims application. This action is not compatible with standard rules-based OMR zone detection procedures.

Example:

```
MC_ReadZones()
```

Parse31aPhSig

Parses the 31aPhSig field of the HFCA application.

Member of namespace:

MC_Validation

Syntax:

```
bool Parse31aPhSig()
```

Parameters

None.

Returns

Always **True**.

Level

Field level.

Details

This action parses the 31aPhSig field of the HFCA application.

Example:

```
Parse31aPhSig()
```

Parse58ainsnm

Parses the 58ainsnm field of the UB04 application.

Member of namespace:

MC_Validation

Syntax:

```
bool Parse58ainsnm()
```

Parameters

None.

Returns

Always **True**.

Level

Field level.

Details

This action parses the 58ainsnm field of the UB04 application.

Example:

```
Parse58ainsnm()
```

Parse58binsnm

Parses the 58binsnm field of the UB04 application.

Member of namespace:

MC_Validation

Syntax:

```
bool Parse58binsnm()
```

Parameters

None.

Returns

Always **True**.

Level

Field level.

Details

This action parses the 58binsnm field of the UB04 application.

Example:

```
Parse58binsnm()
```

Parse58cinsnm

parses the 58cinsnm field of the UB04 application.

Member of namespace:

MC_Validation

Syntax:

```
bool Parse58cinsnm()
```

Parameters

None.

Returns

Always **True**.

Level

Field level.

Details

This action parses the 58cinsnm field of the UB04 application.

Example:

```
Parse58cinsnm()
```

ParseConditionCodes

Detects and parses Condition Code data elements from the calling field value.

Member of namespace:

MC_Validation

Syntax:

```
bool ParseConditionCodes()
```

Parameters

None.

Returns

False, if the action generates an error or an invalid code is detected. Otherwise, **True**.

Level

Field level that contains Condition Code data to detect and parse to fields CCode 1 through CCode4.

Details

This action detects and parses Condition Code data elements from the calling field value.

Valid codes are: AA, AB, AC, AD, AE, AF, AG, AH, AI, W2, W3, W4, or W5.

Example:

```
ParseConditionCodes()
```

ParseEPSDT

Detects and parses an EDSDT Reason Code from the calling field value.

Member of namespace:

MC_Validation

Syntax:

```
bool ParseEPSDT(string EPSDTCode)
```

Parameters

A Smart Parameter enabled value that represents the target field where the parsed data is to be saved.

Returns

False, if the action generates an error. Otherwise, **True**.

Level

Field level that contains the Reason Code data.

Details

This action detects and parses an EDSDT Reason Code from the calling field value. Parsing looks for "AV", "S2", "ST", or "NU" as a single word at the end of the field value.

Example:

```
ParseEPSDT("..\\EPSDTCODE")
```

ParseLastFirstIniNames

Parses the name information in the first line of an address superfield.

Member of namespace:

MC_Validation

Syntax:

```
bool ParseLastFirstIniNames(StrParam)
```

Parameters

These comma separated parameters:

1. The name of the Last Name Field object.
2. The name of the First Name Field object.
3. The name of the Middle Name or Middle Initial object.
4. The name of the Credential Field object.
5. The name of the Suffix Field object.

Returns

False, if the parameter values are invalid. Otherwise, **True**.

Level

Field level.

Details

This action parses the name information in the first line of an address superfield.

The action parses the value of the full name into the Last, First, and MiddleName/Initial fields that are specified by the parameter. In the absence of any explicit pattern, such as a punctuation mark or a middle initial, parsing defaults to First Middle Last. A parameter value of **-1** in the argument changes this default in the absence of any explicit pattern to Last First.

Example:

For form fields where the instructions specify First Middle Last:

```
ParseLastFirstIniNames(8plname,8pfname,8pminit)
```

For form fields where the instructions specify Last First Middle:

```
ParseLastFirstIniNames(17RelLNam,17RelFNam,17RelMini,  
17RelCred,17RelSufx,-1)
```

ParseNDC

Detects and parses NDC data elements from the calling field value.

Member of namespace:

MC_Validation

Syntax:

```
bool ParseNDC(string NDCField,string TypeField,string QuantityField)
```

Parameters

Three Smart Parameters that represent the target path from the calling object to the field's parsed data that is to be saved.

Returns

False, if the action generates an error. Otherwise, **True**.

Level

Field level that contains NDC data to detect and parse.

Details

This action detects and parses NDC data elements from the calling field value. NDC value parsing looks for "N4" followed by 11 numbers. The NDC Type and Quantity parameters look for "F2", "GR", "ML", or "UN" followed by 1 to 9 numbers.

Example:

```
ParseNDC("..\\NDC","..\\NDCType","..\\NDCQty")
```

PopulateFromField

Copies the value from field that is specified by the parameter into the current field.

Member of namespace:

MC_Validation

Syntax:

```
bool PopulateFromField(StrParam)
```

Parameters

The name of the field whose value is to be assigned to the current field

Returns

False, if the parameter is invalid or if the action is not applied at the Field level. Otherwise, **True**.

Level

Field level.

Details

This action copies the value from field that is specified by the parameter into the current field.

Example:

```
PopulateFromField(24aDtFr1)
```

SetConf

Set a confidence string for a field.

Member of namespace:

MC_Validation

Syntax:

```
bool SetConf(StrParam)
```

Parameters

The value of the new confidence for each character in the field, **0** to **9**.

Returns

Always **True**.

Level

Field level.

Details

This action sets a confidence string for a field.

Example:

```
SetConf(9)
```

This example sets the confidence for each character in the field to 9, which is the highest confidence.

SetOriginalTIF

Replaces the current TIF file with the original TIF image that was previously renamed with a T11 extension.

Member of namespace:

MC_Validation

Syntax:

```
bool SetOriginalTIF(StrParam)
```

Parameters

The extension of the original image file.

Returns

False, if the original image does not exist. Otherwise, **True**.

Level

Page level.

Details

This action replaces the current TIF file with the original TIF image that was previously renamed with a TIF extension. It is assumed that the original file name was copied to a file name that uses a different extension for safe keeping.

Example:

```
SetOriginalTIF(TIF)
```

This example replaces the current TIF file with the original TIF image that was previously renamed with a TIF extension.

StripTrailingAlpha

Removes all alpha characters from the captured value, except from the first character position.

Member of namespace:

MC_Validation

Syntax:

```
bool StripTrailingAlpha()
```

Parameters

None.

Returns

Always **True**.

Level

Field level.

Details

This action removes all alpha characters from the captured value, except from the first character position.

Example:

```
StripTrailingAlpha()
```

TransformLI

Assigns values to the fields in the lines of the Line Item Table.

Member of namespace:

MC_Validation

Syntax:

```
bool TransformLI()
```

Parameters

None.

Returns

True if the assignment of values is successful for all of the fields in all of the lines of the Line Item Table.

False, if the transfer of values for one or more of the fields fails.

Level

Document or Page level.

Details

Each line in the table is remapped into a repeating set of fields. These fields have a parent field with a unique ID. All of the data in a source field is retained in the new target field, with the exception of the new fields type and ID. The linear field structure is replaced with a structure that is based on a parent field in the claim page called DETAILS.

Each set of fields in the row of table data is then placed in a LINEITEM type field. The ID of the field is patterned on the index of its insertion in the DETAILS field. The first Line Item field is called LINEITEM1, the second is LINEITEM2, and so on. Each Line Item field contains an identical set of Field Type and ID as outlined in the following values list.

For example, the HCFA Line Item Table has 12 24nDtfr fields. During processing, the TransformLI action assigns recognized values for these fields to a Field object named DateFrom that is a child of the Line Item Table Details new parent Field object. So, 12 recognized values can be assigned to the Field object DateFrom.

Values that are associated with rows of the HCFA-1500 table are assigned to the Field types where n is value from 1 to 12:

- 24aDtfn = DateFrom
- 24aDtTon = DateThru
- 24bPlacn = PlaceOfService
- 24cTypen = TypeOfService
- 24cEMG_n = EMG_C
- 24dCPT_n = CPT_Code
- 24dModin = Modifiers
- 24eDiagn = DiagPointer
- 24fChgsn = Charges
- 24gDaysn = Days_Units
- 24hEPsDn = EPSD
- 24iQualn = Qualifier
- 24jReflDn = ReferenceId

- 24jEMG_n = EMG_I
- 24jCOBn = COB
- 24kLocn = LocalUse
- 24lInfon = Info

Values that are associated with the rows of a UB92/UB04 table are assigned to the child Field objects where n is a value from a to z:

- 42nrevcd = RevCode
- 43ndscrt = Description
- 44nhcpcs = HCPCS
- 44nMod = Modifiers
- 45nsrvdt = ServiceDate
- 46nsrvun = Units
- 47nttchg = Charges
- 48nncchg = NotCovered
- 49n = LocalUse

Note: This action converts all of the pages in a document if it called from a Document object. The expected page types are HCFA, 1500, UB92, and UB04_Page.

UpdateCredentialList

Updates the default list of abbreviations that are used by the parsing actions to extract credential strings from names.

Member of namespace:

MC_Validation

Syntax:

```
bool UpdateCredentialList(string sCredential,string AddRemove)
```

Parameters

string **sCredential**

string **AddRemove**

Parameters

A two parameter specification of a list of credential abbreviations and a Add or Remove from the list indicator.

sCredential is a comma separated list of credential abbreviations. Smart parameters are supported.

AddRemove indicator is an Add or Remove Condition from the list command. Defaults to Add. The keywords for setting the remove from the list mode are: "OFF", "REMOVE", "0", "FALSE", "-1", "NO", and "DELETE".

Returns

False, if the action generates an error. Otherwise, **True**.

Level

All levels.

Details

This action updates the default list of abbreviations that are used by the following name parsing actions to extract credential strings from names: ParseLastFirstIniNames(), Parse58ainsnm(), Parse58binsnm(), Parse58cinsnm(), Parse82name(), Parse83aname(), Parse83bname(), Parse31aPhSig(),

Example:

```
UpdateCredentialList(MD,Add)
```

ValidateNPI

Validates the NPI value by evaluating the 10 digits in the value that use a modified LUHN checkdigit algorithm.

Member of namespace:

MC_Validation

Syntax:

```
bool ValidateNPI()
```

Parameters

None.

Returns

True, if the field contains a valid NPI value. Otherwise, **False**.

Level

Field level.

Details

This action validates the NPI value by evaluating the 10 digits in the value that use a modified LUHN checkdigit algorithm.

Example:

```
ValidateNPI()
```

ValProcedureCode

Validates the Procedure Code fields in a HCFA-1500 form.

Member of namespace:

MC_Validation

Syntax:

```
bool ValProcedureCode()
```

Parameters

None.

Returns

False, if the action is not applied at the Field level or if the Procedure Code is invalid. Otherwise, **True**.

Level

Field level.

Details

This action validates the Procedure Code fields in a HCFA-1500 form.

Example:

```
ValProcedureCode()
```

ValRequiredCode

Checks that all of the fields in a designated group are filled with data.

Member of namespace:

MC_Validation

Syntax:

```
bool ValRequiredCode(StrParam)
```

Parameters

The names of fields in the group.

Returns

False, if the parameters are invalid or if any of the parameter fields do not contain data. Otherwise, **True**.

Level

Field level.

Details

This action checks that all of the fields in a designated group are filled with data.

Example:

```
ValRequiredCode(24aDtFr1,24aDtTo1,24adCPT_1,24fChg1,24gdays1)
```

mvscan actions

Use mvscan actions in place of the vscan actions to create batches from files that are stored on your local disk. The mvscan actions can handle large numbers of files that are stored in the input folder.

The mvscan actions can run simultaneously on multiple stations, in multiple threads. These actions can read from a single folder or multiple folders to provide

high availability and throughput. The mvscan actions can also read metadata and include it in the batch with the input files. Refer to the set_metadata_types action for details.

- "scan"
- "set_abort_time" on page 743
- "set_copy_folder" on page 744
- "set_delete_empty_folders" on page 744
- "set_folder" on page 745
- "set_image_validation" on page 746
- "set_max_docs" on page 746
- "set_metadata_types" on page 747
- "set_min_age" on page 749
- "set_move_wait_time" on page 750
- "set_multipage_burst" on page 750
- "set_problem_folder" on page 751
- "set_sort_method" on page 752
- "set_tree_mode" on page 753
- "set_types" on page 753
- "set_wait_time" on page 754

scan

Polls the folder that is specified by the set_folder action and scans the images into a batch folder.

Member of namespace

mvscan

Syntax

scan ()

Parameters

None.

Returns

False, if the operation fails, and pauses before the return. Otherwise, **True**.

Action returns when timeout is reached, or the requested number of files is ingested.

Level

Batch level Open event only.

Details

Scans the source folder for files with the extensions that you want, then ingests them into a batch. Call one time for each batch. All options must be set before this action is called. If no files are present for ingestion, the batch is set to pending

status, and the action returns immediately. The files in each batch folder are ingested by the order of the date the file was last modified.

Each input file generates one or more "pages" in the batch with the following page variables set:

- TYPE : Always set to "Other".
- IMAGEFILE : The file name within the batch, for example TM000002.tif.

Example:

```
set_folder("@APPATH(vscanimagedir)+@STRING(\mvscan folder)")
set_types("jpg,pdf,tif")
set_max_docs("2")
scan()
```

set_abort_time

Specifies the length of time in seconds to delay processing if the scan action encounters an unrecoverable error and sets the batch status to aborted.

Member of namespace

mvscan

Syntax

set_abort_time ()

Parameters

nSecs Type: int

Time to wait before continuing when a serious error occurs.

Parameters

nSecs : Time to wait before continuing when a serious error occurs.

Returns

Always **True**.

Level

Batch level.

Details

The scan action waits the specified time before returning if a fatal error is detected that would cause the batch to abort. This action can be useful to prevent a large number of aborted batches due to an abort condition. For example, if the source folder should become unavailable for some time, the abort timeout will limit the number of aborted batches until the folder becomes available again.

This delay restricts the number of aborted batches if the problem persists.

If this action is not called, the default abort time value of 5 seconds is used.

Example:

```
set_abort_time("60")
scan()
```

set_copy_folder

Optional, sets a folder to contain a copy of each ingested file.

Member of namespace

mvscan

Syntax

```
bool set_copy_folder (string folderpath)
```

Parameters

folderpath

Type: string

Path to the folder that contains a copy of each ingested file.

Parameters

folderpath : A string value that specifies the path of the folder where a copy of each input file is placed, in addition to the batch.

Smart parameters are supported.

Returns

Always **True**.

Level

Batch level.

Details

If the path is not specified or set to an empty string, input files are moved to the batch folder without making a copy.

This action must be called before scan() to take effect.

Example:

```
set_folder("@APPPATH(vscanimagedir)+@STRING(\input folder)")
set_copy_folder("@APPPATH(vscanimagedir)+@STRING(\copy folder)")
scan()
```

set_delete_empty_folders

Determines whether to delete the empty subdirectories.

Member of namespace

mvscan

Syntax

```
set_delete_empty_folders ()
```

Parameters

bParam

Type: bool

Parameters

bParam : A Boolean value that enables or disables deleting sub-folders if they are empty. The root folder specified for ingestion is never deleted. Subfolders are only deleted if both this setting and tree mode are enabled.

True: Delete subfolders of the main ingestion folder if they are empty. This is the default if the action is not called.

False: Leave empty subfolders intact.

Details

If tree mode is not enabled, this setting has no effect. For this action to take effect, it must be called before the Scan action.

Example:

```
set_types("tif")
set_min_age("10")
set_tree_mode(True)
set_delete_empty_folders(False)
scan()
```

set_folder

Specifies the path of the folder in which to search for the image files that you want to scan.

Member of namespace

mvscan

Syntax

set_folder ()

Parameters

folderpath

Type: string

Path to folder containing files to be ingested

Parameters

folderpath : A string value specifying the path of the folder which will contain input files to be placed into batches for processing.

Returns

Always **True**.

Level

Batch level.

Details

This action must be called before scan().

Example:

```
set_folder("@APPPATH(vscanimagedir)+@STRING(\input folder)")
scan()
```

set_image_validation

Causes the Scan action to validate that the specified input files are all valid image files. Invalid files are moved to the problem folder.

Member of namespace

mvscan

Syntax

```
set_image_validation ()
```

Parameters**bValidate**

Type: bool

Parameters

bValidate : A Boolean value that enables or disables validating image files during ingestion.

True: Check to confirm that each file ingested contains a valid TIFF or JPG image.

False: Copy input files regardless of type or contents. This is the default if the action is not called.

Returns

Always **True**.

Level

Batch level.

Details

Enables testing that each file ingested is a valid image file. If enabled, converts black and white TIF files to G4 compression. This action should not be called against invalid file types such as PDFs due to incompatibility.

For this action to take effect, it must be called before the Scan action.

Example:

```
set_types("tif")
set_image_validation("True")
scan()
```

set_max_docs

Specifies maximum number of pages in each batch when used with the set_types action.

Member of namespace

mvscan

Syntax

```
set_max_docs ()
```

Parameters

nDocs Type: int

Number of documents in a batch. Default 100.

Parameters

nDocs : An integer value which specifies the maximum number of input files to place into a batch.

If Metadata files are used, `set_metadata_types()` is called instead of `set_types()`, this specifies the maximum number of metadata files to ingest. All files referenced in each metadata file are included in the batch, there is no limit on the number of pages in the batch in this case. The default value is 100.

Returns

Always **True**.

Level

Batch level.

Details

For this action to take effect, it must be called before the Scan action.

Example:

```
set_tree_mode(false)
set_max_docs("1")
scan()
```

set_metadata_types

Specifies the input file name extensions of XML files that contain the names of the files to ingest and the metadata to include in the batch for each file.

Member of namespace

mvscan

Syntax

```
set_metadata_types ()
```

Parameters

extensions

Type: string

Metadata image file extensions. If specified, XML metadata files (trigger files) control ingestion of pages into batches, along with associated metadata. See more documentation for syntax and usage of metadata trigger files. If specified, these extensions override any prior call to `set_types()`.

Parameters

String value of extensions of input files that contain pointers to files to be ingested along with metadata to be included in the batch for each file. For multiple types, separate each one with a comma. Any files with extensions not in this list are ignored.

It is optional to include a period before each extension. The parameter is not case-sensitive.

Returns

Always **True**.

Level

Batch level Open event only.

Details

The format of the metadata file is documented here. This action overrides any previous call to `set_types()`, and forces the action to read properly formatted XML from the metadata trigger file. Then, the action determines which files get ingested, rather than ingesting individual files into the batch. Other settings such as `set_min_age()` are applied to the metadata files.

You must set up the XML file in the correct format to use the `set_metadata_type` action to scan metadata files into batches.

The following example illustrates a sample XML file:

```
<input>
  <item name="Invoice1" vendor_number="TS265329"
    vendor_name="Busy Car Repair" invoice_number="28100">
    <filename>C:\Datacap\APT\images\Input\
      Invoice_0001S1.tif</filename>
    <filename>C:\Datacap\APT\images\Input\
      Invoice_0002S1.tif</filename>
  </item>
  <item name="Invoice2" vendor_number="TS23785354"
    vendor_name="Trucking Co." invoice_number="876-3456">
    <filename>C:\Datacap\APT\images\Input\
      Invoice_0003S1.tif</filename>
    <filename>C:\Datacap\APT\images\Input\
      Invoice_0004S1.tif</filename>
  </item>
</input>
```

The XML file must follow this format.

1. The tag name of the XML root node must be `<input>`.
2. The `<item>` nodes represent a group of input files or images that share metadata.
3. The `<item>` nodes must contain at least one attribute, but can have a number of attributes. In this example, the `<item>` nodes contain the attributes `name=`, `vendor_number=`, `vendor_name=`, `invoice_number=`.
4. The `<item>` attribute names are entered in the item as text that follows the `=`.

5. These name values are prepended with Meta_Item_ when they are written to the DCO to define the metadata variable names that are placed on each page. The attribute values populate the variable values.
6. The inner most nodes must be <filename> and contain items that represent single input files.
7. A <filename> can be either a fully qualified path, for example \\server\folder\filename, or a partial path relative to the location of the metadata file such as folder\filename. The number of file names in each repeating item is only limited to the practical limit of how many files you want to ingest in a single batch.
8. Each file is ingested as a Page in the batch, regardless of the file type or contents.

All of the items in a metadata file are ingested into the same batch. For example, if a metadata file contains 1 node with 50 file names, 50 files are ingested for that single metadata file. If the metadata file contains 5 nodes with 50 file names in each of them, then 250 files are ingested. The external system that creates the metafile must create all of the files to be ingested before it writes the metafile that refers to them.

Example:

```
set_metadata_types("xml")
Scan()
```

This sequence scans for metadata trigger files that end in ".xml".

set_min_age

Specifies the minimum time in seconds to wait after the file was modified before ingesting the file.

Member of namespace

mvscan

Syntax

```
set_min_age ()
```

Parameters

nSecs Type: int

Number of seconds to wait after file modified before ingesting

Parameters

nSecs : An integer value, the number of seconds to wait after file modified before ingesting.

The default value is zero, meaning ingest files as soon as they are present.

Returns

Always **True**.

Level

Batch level.

Details

This setting can be used to prevent premature ingestion of a file which may be incomplete. For example, a network scanner or multifunction device might create the image file and write contents over a period of time, rather than all at once. In that case ingesting the file immediately could cause errors, whereas waiting for 5 or 10 seconds would provide sufficient time for the file to be completed. If required, the appropriate value must be determined experimentally.

For this action to take effect, it must be called before the Scan action.

Example:

```
set_types("tif")
set_min_age("10")
scan()
```

set_move_wait_time

Specifies the time to wait for a source file to be deleted after it is moved from to the batch folder.

Syntax

```
bool set_move_wait_timeFNP8_MultiPageDocs(int nSecs)
```

Parameters

int **nSecs** - Number of seconds to wait for the source file to be deleted before the action fails.

Returns

Always **True**.

Level

Batch level

Details

Over a slow network, and especially with large input files, moving files from the source folder to batch folder can take a significant length of time. The scan action checks that the source file was deleted after it was moved. If the source folder still exists, the scan action waits for a length of time periodically checking to see if the file was deleted.

If the folder still exists at the end of this time period, the copy in the batch folder is renamed with the extension `.failed` and it is not included in the batch. It is assumed that the original source file will be ingested by a subsequent scan task.

Example:

```
set_move_wait_time("120")
scan()
```

set_multipage_burst

Forces multipage TIFF images to be split into single pages during ingestion.

Member of namespace

mvscan

Syntax

`set_multipage_burst ()`

Parameters

bBurst

Type: bool

Nonzero to force bursting of TIFF image files. Default = 0, no bursting.

Parameters

bBurst : A Boolean value that enables or disables bursting multipage image files during ingestion.

True: Any multipage TIFF file is separated into multiple pages in the batch, one per image.

False: One page is output per input file. This is the default if the action is not called.

Returns

Always **True**.

Level

Batch level.

Details

Enables splitting or bursting multipage source image files into one image per page in the batch. For this action to take effect, it must be called before the Scan action.

This action requires that `set_types()` be called with only one extension of TIF, TIFF, JPG or JPEG.

Example:

```
set_types(".tif")
set_multipage_burst(1)
scan()
```

If the scan action in this sequence encounters a multipage .tif file, it reads each one into the current batch as a separate image, thereby bursting the multipage file into individual images.

set_problem_folder

Sets the folder into which the files that cannot be ingested are placed.

Member of namespace

mvscan

Syntax

`set_problem_folder ()`

Parameters

folderpath

Type: string

Path to folder to contain any files that cannot be ingested

Parameters

folderpath : A string value that specifies the path of the folder where files that cannot be ingested are placed.

Returns

Always **True**.

Level

Batch level.

Details

This action must be called before scan(). If this action is not called, and scan is unable to ingest a file, the batch is aborted. Files that are already ingested into the batch remain in the batch. The file that cannot be ingested might be in a locked state. Manual intervention is required.

Example:

```
set_folder("@APPPATH(vscanimagedir)+@STRING(\input folder)")
set_problem_folder("@APPPATH(vscanimagedir)+@STRING(\problem folder)")
scan()
```

set_sort_method

Selects method to use for sorting files for ingestion.

Member of namespace

mvscan

Syntax

```
bool set_sort_method(string method)
```

Parameters

method

Type: string

Sort method can be either DATE (default) or NAME. The parameter is not case sensitive.

Returns

Always **True**.

Level

Batch level Open event only.

Details

The scan action takes snapshots of the input folder, sorts the files by either date last modified (default) or by filename, and then tries to ingest the files in that order. This action allows you to override the default sort order.

Example:

```
set_sort_method("NAME")
Scan()
```

This sequence ingests files in alphabetical order.

set_tree_mode

Determines whether subdirectories are included in the scan for files to ingest.

Member of namespace

mvscan

Syntax

```
set_tree_mode ()
```

Parameters

bTreeFlag

Type: bool

Parameters

bTreeFlag : A Boolean value that enables or disables ingesting files from sub-folders within the top-level scan folder.

True: Search for files in subfolders to any depth under the scan folder. This is the default if the action is not called.

False: Ingest files that are located in the scan folder only. Ignore subfolders.

Details

If tree mode is enabled, each batch contains files from only one folder. Files from multiple folders are never ingested into the same batch.

For this action to take effect, it must be called before the Scan action.

Example:

```
set_types("tif")
set_tree_mode(True)
scan()
```

set_types

Specifies the extensions of the file types to ingest in a comma-separated list of file extensions.

Member of namespace

mvscan

Syntax

```
set_types ()
```

Parameters

extensions

Type: string

Comma-separated list of file image file extensions to import

Parameters

String value of extension(s) of input files that should be ingested. For multiple types, separate each one with a comma. Any files with extensions not in this list are ignored. A smart parameter that evaluates to a file extension or list of extensions is accepted.

It is optional to include a period before each extension. The parameter is not case-sensitive.

Returns

Always **True**.

Level

Batch level Open event only.

Details

Uses the value of a file extension(s) to specify the type of files the task will scan.

This is an optional action: the task will scan .tif files by default. For this action to take effect, it must be called before the scan action.

This action overrides any previous call to `set_metadata_types()`, and causes files to be ingested without metadata.

Example:

```
set_types("tif,tiff,pdf")
scan()
```

This sequence ingests files ending with .tif, .tiff, and .pdf.

set_wait_time

Specifies the interval of time to wait for additional files to ingest, before finishing a batch which has not reached the specified maximum size or time limits.

Member of namespace

mvscan

Syntax

```
set_wait_time ()
```

Parameters

nSecs Type: int

Time to wait for files to complete a batch.

Parameters

nSecs : The maximum number of seconds to wait for files to complete a batch.

Returns

Always **True**.

Level

Batch level.

Details

The maximum time to wait for input files to arrive, if at least one file was ingested, no more files are available, and the batch has not reached capacity. Ingestion of files into the batch stops when the wait limit is reached or when the maximum number of files per batch has been reached. If no files are available during the scan() action, and this time is reached, the scan action returns and the batch status remains pending. If this action is not called, the default wait time of 2 seconds is used.

Example:

```
set_wait_time("60")
scan()
```

Maintenance Manager actions

The Maintenance Manager actions are divided into setup, batch processing, logging, and reporting categories.

The Maintenance Manager actions get connections to Datacap application databases and build query strings that run against these databases. These actions can also write logging information to Windows log files and send email messages that contain the log file. Maintenance Manager actions also write information to report tables in the Engine database for use by Datacap Report Viewer.

Embedded help is provided in Datacap Studio for all of the Maintenance Manager actions. To access the embedded help, select an action in the **Actions Library** tab and click **information**.

“Application setup actions”

“Query setup actions” on page 764

“Batch processing actions” on page 779

“Logging actions” on page 791

“Reporting actions” on page 796

Application setup actions

Use the Application setup actions to set up a connection from a Datacap application to Maintenance Manager.

The Application setup actions identify the name of the application and the Datacap Server to be used by Maintenance Manager. These actions also specify the login credentials, database information, and connection information that you need to connect a Datacap application to Maintenance Manager.

`"SetAdminDB"`
`"SetApplication"`
`"SetEngineDB"` on page 757
`"SetPassword"` on page 758
`"SetServer"` on page 759
`"SetStation"` on page 759
`"SetupDisconnectAll"` on page 760
`"SetupOpenApplication"` on page 761
`"SetupOpenApplicationEx"` on page 762
`"SetUser"` on page 763

SetAdminDB:

Specifies the Administration database.

Member of namespace

Maintenance Manager

Syntax

`SetAdminDB ()`

Parameters

adminDB

Type: string

Parameters

adminDB: Administration Database key in the Application Service. Smart parameters are supported.

Returns

Always **True**.

Level

Batch level.

Details

Obtains the Administrator database connection information from the application service. This is the default value used for the action `SetupOpenApplication`.

Example:

```
SetApplication("APT")
SetServer("Server 1")
SetAdminDB("*/tmadmin:cs")
SetEngineDB("*/tmengine:cs")
SetupOpenApplication("")
```

SetApplication:

Specifies the name of the Application used by Maintenance Manager.

Member of namespace

Maintenance Manager

Syntax

SetApplication (string application)

Parameters

string **application**

Parameters

application: The application name. Smart parameters are supported.

Returns

False if the application name is missing. Otherwise, **True**.

Level

Batch level.

Details

Sets the name of the application, as defined in the application service, which are used by Maintenance Manager. The application determines the rules and databases that are used by subsequent actions. This will be the default value that is used for the SetupOpenApplication action.

Example:

```
SetApplication("APT")  
SetServer("Server 1")  
SetupOpenApplication("")
```

SetEngineDB:

Specifies the Engine database.

Member of namespace

Maintenance Manager

Syntax

SetEngineDB ()

Parameters

engineDB

Type: string

Parameters

engineDB: Engine Database key in the Application Service.

Returns

Always **True**.

Level

Batch level.

Details

Obtains the Engine database connection information from the application service. This will be the default value used for the action SetupOpenApplication.

Example:

```
SetApplication("APT")
SetServer("Server 1")
SetAdminDB("*/tmadmin:cs")
SetEngineDB("*/tmengine:cs")
SetUser("Admin")
```

SetPassword:

Action to set password to connect to the Server.

Member of namespace

Maintenance Manager

Syntax

SetPassword (string password)

Parameters

String **password**

Parameters

password: Password. Smart parameters are supported.

Returns

Always **True**.

Level

Batch level.

Details

Specifies the password for the previously specified user ID. This will be the default value used for the SetupOpenApplication action.

When using LDAP or ADSI authentication, use the **SetUser**, **SetStation**, and **SetApplication** to login.

Example:

```

SetApplication("APT")
SetServer("Server 1")
SetAdminDB("*/tadmin:cs")
SetEngineDB("*/tmengine:cs")
SetUser("Admin")
SetPassword("@APPVAR(values/adv/MyNENUPassword)")
SetupOpenApplication("")

```

This example uses the Smart Parameter @APPVAR to obtain the value of the password from the value name MyNENUPassword in the **Custom Values** tab of the Application Service Manager. The value name is configurable.

SetServer:

Specifies the name of the Datacap Server.

Member of namespace

Maintenance Manager

Syntax

```
SetServer ()
```

Parameters

server Type: string

Parameters

server: Server name. Smart parameters are supported.

Returns

False if a server name is not specified. Otherwise, **True**.

Level

Batch level.

Details

The name of the Datacap server as it is defined in the application service, which will be the server used by subsequent Maintenance Manager actions. This will be the default value used for the action SetupOpenApplication.

Example:

```

SetApplication("APT")
SetServer("Server 1")
SetupOpenApplication("")

```

SetStation:

Action to set the station ID that is used to connect to the Server.

Member of namespace

Maintenance Manager

Syntax

SetStation (string station)

Parameters

string **station**

Parameters

station: Station ID. Smart parameters are supported.

Returns

Always **True**.

Level

Batch level.

Details

Specifies the station ID for login. This will be the default value that is used for the SetupOpenApplication action.

When using LDAP or ADSI authentication, use the **SetUser**, **SetStation**, and **SetApplication** to login.

Example:

```
SetApplication("APT")
SetServer("Server 1")
SetAdminDB("*/tmadmin:cs")
SetEngineDB("*/tmengine:cs")
SetStation("1")
SetupOpenApplication("")
```

SetupDisconnectAll:

Disconnect from all Datacap servers.

Member of namespace

Maintenance Manager

Syntax

SetupDisconnectAll ()

Parameters

None.

Returns

Always **True**.

Level

Any level.

Details

Closes the connections to all Datacap databases that were opened by Maintenance Manager actions. Some Maintenance Manager actions automatically close the connection to the database after a query has been performed.

Example:

```
SetupDisconnectAll()
```

SetupOpenApplication:

Creates connection to the application based on default settings.

Member of namespace

Maintenance Manager

Syntax

```
SetupOpenApplication ()
```

Parameters

None.

Returns

True if the application exists in the application service the connection was successful. Otherwise, **False**.

Level

Batch level.

Details

A simplified version of SetupOpenApplicationEx. It is identical in operation with the difference that all of the default parameters are used. The default values can be set independently by individual actions, such as SetApplication, SetServer, etc. See SetupOpenApplicationEx for more information.

When using LDAP and LLLDAP authentication the User, Password, and Station values must be blank.

Example:

```
SetApplication("APT")
SetServer("Server 1")
SetUser("user")
SetPassword("password")
SetAdminDB("*/tmadmin:cs")
SetEngineDB("*/tmengine:cs")
SetStation("1")
SetupOpenApplication()
```


SetupOpenApplicationEx:

Creates connection to the application based on the parameters provided.

Member of namespace

Maintenance Manager

Syntax

SetupOpenApplicationEx ()

Parameters

application

Type: string

server Type: string

admin Type: string

engine

Type: string

debugFlag

Type: bool

user Type: string

password

Type: string

station

Type: string

Parameters

All parameters, except for debugFlag, support Smart parameters.

- application: Application name (Optional). Default value: the name of the application that is running this action.
- server: Server name (Optional). Default value: first available server in the application.
- admin: Administration database name (Optional). Default value: first available Admin DB.
- engine: Engine database name (Optional). Default value: first available Engine DB.
- debugFlag: Debug Flag (Optional). Default value: false.
- user: User name (Optional). Default value: current user credentials. Leave Blank for LDAP authentication.
- password: Password (Optional). Default value: current user credentials. Leave Blank for LDAP authentication.
- station: Station ID (Optional). Default value: Current station name. Leave Blank for LDAP authentication.

Returns

True, if the application exists in the application service the connection was successful. Otherwise, **False**.

Level

Batch level.

Details

Similar to SetupOpenApplication, this action connects Maintenance Manager to a specific application. The difference between the two actions is that here the default parameters can be overridden in one single action. These parameters determine what application is to be monitored by Maintenance Manager. This action, or SetupOpenApplication, must be called first, then the Maintenance Manager actions can be used to run queries on the application and do required actions that are based on the results.

When you are using LDAP and LLDAP authentication the User, Password, and Station values must be blank.

Example:

```
SetupOpenApplicationEX("Survey", "", "SurveyAdm.mdb", "SurveyEng.mdb",  
"False", "", "", "1")
```

SetUser:

Action to set a user name to login to the Server.

Member of namespace

Maintenance Manager

Syntax

SetUser (string user)

Parameters

string **user**

Parameters

user: User name. Smart parameters are supported.

Returns

Always **True**.

Level

Batch level.

Details

Specifies the name of the user for the login to the Administrator and Engine databases. This will be the default value that is used for the SetupOpenApplication action.

When using LDAP or ADSI authentication, use the **SetUser**, **SetStation**, and **SetApplication** to login.

Example:

```

SetApplication("APT")
SetServer("Server 1")
SetAdminDB("*/tmadmin:cs")
SetEngineDB("*/tmengine:cs")
SetUser("Admin")
SetupOpenApplication("")

```

Query setup actions

Use the Query setup actions to build the query string that is run against the application databases that you connected to in the application setup.

Before, you can run a query against the application databases, you must build an SQL query string. When you run the actions in this category, Maintenance Manager appends the corresponding SQL code to the current query string. For example, running `QuerySetStatus("hold")` followed by `QuerySetOperator("admin")` generates the following query string.

```
Select * FROM JobMonitor WHERE queue.qu_status IN ('hold') AND qstats.qs_op IN ('admin')
```

“QueryClear”

“QuerySetAge” on page 765

“QuerySetBatchRange” on page 766

“QuerySetBranch” on page 766

“QuerySetDateFormat” on page 767

“QuerySetDateRange” on page 770

“QuerySetDateTimeFormat” on page 771

“QuerySetGeneric” on page 773

“QuerySetJobID” on page 774

“QuerySetOperator” on page 775

“QuerySetPriority” on page 775

“QuerySetSeparator” on page 776

“QuerySetStation” on page 777

“QuerySetStatus” on page 777

“QuerySetTaskID” on page 778

QueryClear:

Clears the SQL query.

Member of namespace

Maintenance Manager

Syntax

`QueryClear ()`

Parameters

None.

Returns

True if the query is cleared. Otherwise, **False**.

Level

Any level.

Details

Clears any SQL query that may be in memory. This action can be called to ensure that any Maintenance Manager query that you build is not building on any previous information.

Example:

```
QueryClear("")
```

QuerySetAge:

Selects batches based on age using a date or number of seconds.

Member of namespace

Maintenance Manager

Syntax

```
QuerySetAge ()
```

Parameters

age Type: string

queryAgeStart
Type: bool

Parameters

- **age**: All dates prior or within this date will be selected. Smart parameters are supported.
- **queryAgeStart**: If **True**, age will use the start time of the batch. If **False**, age will use the end time of the batch.

Returns

True if the query has been successfully set. It does not mean the query has been performed. Otherwise, **False**.

Level

Any level.

Details

Builds a query that selects batches based on the age of a batch. The age can be specified using a date or in seconds. The queryAgeStart parameter determines if the age is based on the start of the batch, the qs_start column, or based on the end of the batch, the qu_done column. If a date is specified, all batches will be selected based on the date. If a number is specified, all batches will be selected based on the number of seconds.

To control the age range use the exclamation point '!'. When the exclamation point is not specified, the age between now and the specified value will be selected. When the exclamation point is specified, the ages that are older than the value specified will be selected.

Example:

```
QuerySetAge("300","True")
ProcessRunSqlQuery("")
```

This example selects all batches started within last 300 seconds.

```
QuerySetAge("!300","true")
ProcessRunSqlQuery("")
```

This example selects all batches started prior to 300 seconds ago. Use the same pattern when using dates.

QuerySetBatchRange:

Sets range of batches for SQL query.

Member of namespace

Maintenance Manager

Syntax

```
QuerySetBatchRange ()
```

Parameters

start Type: string

end Type: string

Parameters

- **start:** BatchID of the first batch in the range. Default value 00000000.000".
- **end:** BatchID of the last batch in the range. Default value zzzzzzzz.zzz".

Returns

True if the query has been successfully set. It does not mean the query has been performed. Otherwise, **False**.

Level

Any level.

Details

Allows selecting batch range based on the BatchID.

Example:

```
QuerySetBatchRange("20110059.001","20110059.010")
InjectBatches("20110059.001")
```

QuerySetBranch:

Sets the minimum number of children for the SQL query.

Member of namespace

Maintenance Manager

Syntax

```
QuerySetBranch ()
```

Parameters

children

Type: int

Parameters

children: The minimum number of children for the batch.

Returns

True if the query has been successfully set. It does not mean the query has been performed. Otherwise, **False**.

Level

Any level.

Details

Selects all of the batches that have at least the minimum number of specified children.

Example:

```
QuerySetBranch("3")  
ProcessRunSqlQuery("")
```

QuerySetDateFormat:

Sets custom Date format for SQL queries.

Member of namespace

Maintenance Manager

Syntax

```
QuerySetDateFormat ()
```

Parameters

dateFormat

Type: string

Parameters

dateFormat: Date format. Smart parameters are supported.

Returns

Always **True**.

Level

Any level.

Details

Configures the Date format used by the database. If this action is not called, the value for the Date format will be "yyyy/MM/dd". Detailed description of available values are:

- **d** Represents the day of the month as a number from 1 through 31. A single-digit day is formatted without a leading zero
- **dd** Represents the day of the month as a number from 01 through 31. A single-digit day is formatted with a leading zero
- **ddd** Represents the abbreviated name of the day of the week (Mon, Tues, Wed etc.)
- **dddd** Represents the full name of the day of the week (Monday, Tuesday etc.)
- **h** 12-hour clock hour (e.g. 7)
- **hh** 12-hour clock, with a leading 0 (e.g. 07)
- **H** 24-hour clock hour (e.g. 19)
- **HH** 24-hour clock hour, with a leading 0 (e.g. 19)
- **m** Minutes
- **mm** Minutes with a leading zero
- **M** Month number
- **MM** Month number with leading zero
- **MMM** Abbreviated Month Name (e.g. Dec)
- **MMMM** Full month name (e.g. December)
- **s** Seconds
- **ss** Seconds with leading zero
- **t** Abbreviated AM / PM (e.g. A or P)
- **tt** AM / PM (e.g. AM or PM)
- **y** Year, no leading zero (e.g. 2001 would be 1)
- **yy** Year, leading zero (e.g. 2001 would be 01)
- **yyy** Year (e.g. 2001 would be 001)
- **yyyy** Year (e.g. 2001 would be 2001)
- **K** Represents the time zone information of a date and time value (e.g. +05:00)
- **z** With DateTime values, represents the signed offset of the local operating system's time zone from Coordinated Universal Time (UTC), measured in hours. (e.g. +6)
- **zz** As z but with leading zero (e.g. +06)
- **zzz** With DateTime values, represents the signed offset of the local operating system's time zone from UTC, measured in hours and minutes. (e.g. +06:00)
- **f** Represents the most significant digit of the seconds fraction; that is, it represents the tenths of a second in a date and time value.
- **ff** Represents the two most significant digits of the seconds fraction; that is, it represents the hundredths of a second in a date and time value.
- **fff** Represents the three most significant digits of the seconds fraction; that is, it represents the milliseconds in a date and time value.

- **ffff** Represents the four most significant digits of the seconds fraction; that is, it represents the ten thousandths of a second in a date and time value. While it is possible to display the ten thousandths of a second component of a time value, that value may not be meaningful. The precision of date and time values depends on the resolution of the system clock. On Windows NT 3.5 and later, and Windows Vista operating systems, the clock's resolution is approximately 10-15 milliseconds.
- **fffff** Represents the five most significant digits of the seconds fraction; that is, it represents the hundred thousandths of a second in a date and time value. While it is possible to display the hundred thousandths of a second component of a time value, that value may not be meaningful. The precision of date and time values depends on the resolution of the system clock. On Windows NT 3.5 and later, and Windows Vista operating systems, the clock's resolution is approximately 10-15 milliseconds.
- **ffffff** Represents the six most significant digits of the seconds fraction; that is, it represents the millionths of a second in a date and time value. While it is possible to display the millionths of a second component of a time value, that value may not be meaningful. The precision of date and time values depends on the resolution of the system clock. On Windows NT 3.5 and later, and Windows Vista operating systems, the clock's resolution is approximately 10-15 milliseconds.
- **fffffff** Represents the seven most significant digits of the seconds fraction; that is, it represents the ten millionths of a second in a date and time value. While it is possible to display the ten millionths of a second component of a time value, that value may not be meaningful. The precision of date and time values depends on the resolution of the system clock. On Windows NT 3.5 and later, and Windows Vista operating systems, the clock's resolution is approximately 10-15 milliseconds.
- **F** Represents the most significant digit of the seconds fraction; that is, it represents the tenths of a second in a date and time value. Nothing is displayed if the digit is zero.
- **:** Represents the time separator defined in the current `DateTimeFormatInfo.TimeSeparator` property. This separator is used to differentiate hours, minutes, and seconds.
- **/** Represents the date separator defined in the current `DateTimeFormatInfo.DateSeparator` property. This separator is used to differentiate years, months, and days.
- **"** Represents a quoted string (quotation mark). Displays the literal value of any string between two quotation marks ("). Your application should precede each quotation mark with an escape character (\)
- **'** Represents a quoted string (apostrophe). Displays the literal value of any string between two apostrophe (') characters.
- **%c** Represents the result associated with a `c` custom format specifier, when the custom date and time format string consists solely of that custom format specifier. That is, to use the `d`, `f`, `F`, `h`, `m`, `s`, `t`, `y`, `z`, `H`, or `M` custom format specifier by itself, the application should specify `%d`, `%f`, `%F`, `%h`, `%m`, `%s`, `%t`, `%y`, `%z`, `%H`, or `%M`. For more information about using a single format specifier, see [Using Single Custom Format Specifiers](#).

Example:

```
QuerySetDateFormat("dd-MMM-yy")
```


QuerySetDateRange:

Sets Date range for SQL query.

Member of namespace

Maintenance Manager

Syntax

```
QuerySetDateRange ()
```

Parameters

start Type: string

end Type: string

queryAgeStart
Type: bool

Parameters

- **start**: Date Range Start. Smart parameters are supported.
- **end**: Date Range End. Smart parameters are supported.
- **queryAgeStart**: **True** will use the batch start date. **False** will use the batch end date.

Returns

True if the query has been successfully set. It does not mean the query has been performed. Otherwise, **False**.

Level

Any level.

Details

Selects all of the batches that match the specified priority. If **queryAgeStart** is set to **True**, the query uses the batch start date, **qs_start**. If **queryAgeStart** is set to **False**, the query uses the batch end date, **qu_done**.

An exclamation point can be prefixed to the value to find all values except the ones specified.

Example:

```
QuerySetDateRange("03/16/2010","05/26/2010","True")  
ProcessRunSqlQuery("")
```

This example selects all of the batches that were created between the two specified dates.

```
QuerySetDateRange("03/16/2010","@DATE(mm/dd/yyyy)","True")  
ProcessRunSqlQuery("")
```

This example uses a Smart parameter to obtain the current date. The query selects all batches that have been created between 03/16/2010 and the current date.

QuerySetDateTimeFormat:

Sets custom DateTime format for SQL queries.

Member of namespace

Maintenance Manager

Syntax

```
QuerySetDateTimeFormat ()
```

Parameters

dateTimeFormat

Type: string

Parameters

dateTimeFormat: DateTime format. Smart parameters are supported.

Returns

Always **True**.

Level

Any level.

Details

Configures the Date and Time format used by the database. If this action is not called, the value for the Date format is yyyy/MM/dd HH:mm:ss. Detailed description of available values are:

- **d** Represents the day of the month as a number from 1 through 31. A single-digit day is formatted without a leading zero
- **dd** Represents the day of the month as a number from 01 through 31. A single-digit day is formatted with a leading zero
- **ddd** Represents the abbreviated name of the day of the week (Mon, Tues, Wed etc.)
- **dddd** Represents the full name of the day of the week (Monday, Tuesday etc.)
- **h** 12-hour clock hour (e.g. 7)
- **hh** 12-hour clock, with a leading 0 (e.g. 07)
- **H** 24-hour clock hour (e.g. 19)
- **HH** 24-hour clock hour, with a leading 0 (e.g. 19)
- **m** Minutes
- **mm** Minutes with a leading zero
- **M** Month number
- **MM** Month number with leading zero
- **MMM** Abbreviated Month Name (e.g. Dec)
- **MMMM** Full month name (e.g. December)
- **s** Seconds
- **ss** Seconds with leading zero

- **t** Abbreviated AM / PM (e.g. A or P)
- **tt** AM / PM (e.g. AM or PM)
- **y** Year, no leading zero (e.g. 2001 would be 1)
- **yy** Year, leading zero (e.g. 2001 would be 01)
- **yyy** Year (e.g. 2001 would be 001)
- **yyyy** Year (e.g. 2001 would be 2001)
- **K** Represents the time zone information of a date and time value (e.g. +05:00)
- **z** With DateTime values, represents the signed offset of the local operating system's time zone from Coordinated Universal Time (UTC), measured in hours. (e.g. +6)
- **zz** As z but with leading zero (e.g. +06)
- **zzz** With DateTime values, represents the signed offset of the local operating system's time zone from UTC, measured in hours and minutes. (e.g. +06:00)
- **f** Represents the most significant digit of the seconds fraction; that is, it represents the tenths of a second in a date and time value.
- **ff** Represents the two most significant digits of the seconds fraction; that is, it represents the hundredths of a second in a date and time value.
- **fff** Represents the three most significant digits of the seconds fraction; that is, it represents the milliseconds in a date and time value.
- **ffff** Represents the four most significant digits of the seconds fraction; that is, it represents the ten thousandths of a second in a date and time value. While it is possible to display the ten thousandths of a second component of a time value, that value may not be meaningful. The precision of date and time values depends on the resolution of the system clock. On Windows NT 3.5 and later, and Windows Vista operating systems, the clock's resolution is approximately 10-15 milliseconds.
- **fffff** Represents the five most significant digits of the seconds fraction; that is, it represents the hundred thousandths of a second in a date and time value. While it is possible to display the hundred thousandths of a second component of a time value, that value may not be meaningful. The precision of date and time values depends on the resolution of the system clock. On Windows NT 3.5 and later, and Windows Vista operating systems, the clock's resolution is approximately 10-15 milliseconds.
- **ffffff** Represents the six most significant digits of the seconds fraction; that is, it represents the millionths of a second in a date and time value. While it is possible to display the millionths of a second component of a time value, that value may not be meaningful. The precision of date and time values depends on the resolution of the system clock. On Windows NT 3.5 and later, and Windows Vista operating systems, the clock's resolution is approximately 10-15 milliseconds.
- **fffffff** Represents the seven most significant digits of the seconds fraction; that is, it represents the ten millionths of a second in a date and time value. While it is possible to display the ten millionths of a second component of a time value, that value may not be meaningful. The precision of date and time values depends on the resolution of the system clock. On Windows NT 3.5 and later, and Windows Vista operating systems, the clock's resolution is approximately 10-15 milliseconds.
- **F** Represents the most significant digit of the seconds fraction; that is, it represents the tenths of a second in a date and time value. Nothing is displayed if the digit is zero.

- `:` Represents the time separator defined in the current `DateTimeFormatInfo / TimeSeparator` property. This separator is used to differentiate hours, minutes, and seconds.
- `/` Represents the date separator defined in the current `DateTimeFormatInfo / DateSeparator` property. This separator is used to differentiate years, months, and days.
- `"` Represents a quoted string (quotation mark). Displays the literal value of any string between two quotation marks (`"`). Your application should precede each quotation mark with an escape character (`\`).
- `'` Represents a quoted string (apostrophe). Displays the literal value of any string between two apostrophe (`'`) characters.
- `%c` Represents the result associated with a `c` custom format specifier, when the custom date and time format string consists solely of that custom format specifier. That is, to use the `d`, `f`, `F`, `h`, `m`, `s`, `t`, `y`, `z`, `H`, or `M` custom format specifier by itself, the application should specify `%d`, `%f`, `%F`, `%h`, `%m`, `%s`, `%t`, `%y`, `%z`, `%H`, or `%M`. For more information about using a single format specifier, see [Using Single Custom Format Specifiers](#).

Example:

```
QuerySetDateFormat("dd-MMM-yy hh:mm:ss tt")
```

QuerySetGeneric:

Builds an SQL query using the provided column name and value.

Member of namespace

Maintenance Manager

Syntax

```
QuerySetGeneric ()
```

Parameters

column

Type: string

value Type: string

Parameters

- `column`: Column in the database table to query. Smart parameters are supported.
- `value`: Value to match within the specified column. Smart parameters are supported.

Returns

True if the query has been successfully set. It does not mean the query has been performed. Otherwise, **False**.

Level

Any level.

Details

Allows building of a query where the column and value of the tmbatch, queue or qstats table are explicitly specified.

Any wild card specified in the **value** parameter would need to be appropriate for your target database. The column value is not validated until the query is run using ProcessRunSqlQuery.

Example:

```
QuerySetGeneric("pb_userid", "admin")
ProcessRunSqlQuery("")
```

QuerySetJobID:

Sets the Job ID for the SQL query.

Member of namespace

Maintenance Manager

Syntax

QuerySetJobID ()

Parameters

jobid Type: string

Parameters

jobid: Job name. Smart parameters are supported.

Returns

True, if the query is successfully set. It does not mean that the query has been performed. Otherwise, **True**.

Level

Any level.

Details

When you are building a query, this action sets the job ID that is selected in the result set. If you want to match multiple job IDs, use commas to separate values. An exclamation point can be used to negate the query.

Example:

```
QuerySetJobID("Demo Job,Web Job")
ProcessRunSqlQuery("")
```

This example creates a result set that contains batches with job ID of Demo Job and Web Job.

```
QuerySetJobID("Demo Job")
QuerySetJobID("Web Job")
ProcessRunSqlQuery("")
```

This example creates an empty result set.

```
QuerySetJobID("!Demo Job")
ProcessRunSqlQuery("")
```

This example creates a result set that contains all batches except for the job ID of Demo Job.

QuerySetOperator:

Sets the operator for the SQL query.

Member of namespace

Maintenance Manager

Syntax

```
QuerySetOperator ()
```

Parameters

operator

Type: string

Parameters

operator: The operator ID. Smart parameters are supported.

Returns

True if the query has been successfully set. It does not mean the query has been performed. Otherwise, **False**.

Level

Any level.

Details

Selects all of the batches that match the specified operator ID.

To find multiple values, separate them with commas. An exclamation point can be prefixed to the value to find all values except the ones specified.

Example:

```
QuerySetOperator("Admin")
ProcessRunSqlQuery("")
```

QuerySetPriority:

Sets the Priority for the SQL query.

Member of namespace

Maintenance Manager

Syntax

```
QuerySetPriority ()
```

Parameters

priority

Type: string

Parameters

priority: Batch Priority.

Returns

True if the query has been successfully set. It does not mean the query has been performed. Otherwise, **False**.

Level

Any level.

Details

Selects all of the batches that match the specified priority.

To find multiple values, separate them with commas. An exclamation point can be prefixed to the value to find all values except the ones specified.

Example:

```
QuerySetPriority("1")
ProcessRunSqlQuery("")
```

QuerySetSeparator:

Sets SQL Date and Time Separator for SQL queries.

Member of namespace

Maintenance Manager

Syntax

QuerySetSeparator ()

Parameters

separator

Type: string

Parameters

separator: Date separator. Smart parameters are supported.

Returns

Always **True**.

Level

Any level.

Details

Configures the Date and Time separator that is used by the database. If this action is not called, the value for the separator is selected based on the connection string. Some database separators are MS SQL "'", Oracle '"', Access "#".

Example:

```
QuerySetSeparator("#")
```

QuerySetStation:

Sets the station for the SQL query.

Member of namespace

Maintenance Manager

Syntax

```
QuerySetStation ()
```

Parameters

station

Type: string

Parameters

station: The station ID. Smart parameters are supported.

Returns

True if the query has been successfully set. It does not mean the query has been performed. Otherwise, **False**.

Level

Any level.

Details

Selects all of the batches that match the specified station.

To find multiple values, separate them with commas. An exclamation point can be prefixed to the value to find all values except the ones specified.

Example:

```
QuerySetStation("1")  
ProcessRunSqlQuery("")
```

QuerySetStatus:

Sets the Task status for the SQL query.

Member of namespace

Maintenance Manager

Syntax

QuerySetStatus ()

Parameters

status Type: string

Parameters

status: The batch status: aborted, cancelled, finished, hold, job done, pending, running. Smart parameters are supported.

Returns

True if the query has been successfully set. It does not mean the query has been performed. Otherwise, **False**.

Level

Any level.

Details

Selects all of the batches that match the specified status.

To find multiple values, separate them with commas. An exclamation point can be prefixed to the value to find all values except the ones specified.

Example:

```
QuerySetStatus("finished")
ProcessRunSqlQuery("")
```

QuerySetTaskID:

Sets the Task ID for the SQL query.

Member of namespace

Maintenance Manager

Syntax

QuerySetTaskID ()

Parameters

taskid Type: string

Parameters

taskid: Task ID. Smart parameters are supported.

Returns

True if the query has been successfully set. It does not mean the query has been performed. Otherwise, **False**.

Level

Any level.

Details

When building a query, this action sets the task ID that is selected in the result set. The current task batches selected will match the supplied value.

To find multiple values, separate them with commas. An exclamation point can be prefixed to the value to find all values except the ones specified.

Example:

```
QuerySetTaskID("Verify")
ProcessRunSqlQuery("")
```

This example queries all batches for the survey application that are at the verify task.

```
QuerySetTaskID("@B.MyQueryTask")
ProcessRunSqlQuery("")
```

This example uses Smart parameters to retrieve the name of the task to query from a batch level variable called *MyQueryTask*.

Batch processing actions

Use the Batch processing actions to run the SQL query and complete actions on the selected database records and, optionally, the corresponding batches.

The ProcessRunSqlQuery action runs the current query string and generates a recordset that contains information about all matching batches. For example, if you build a query by using QuerySetStatus("hold") and your query locates one batch with status hold. The returned recordset contains the following information that is aggregated from the tmbatch, qstats, and queue tables.

```
<rs:data xmlns:rs="urn:schemas-microsoft-com:rowset"><z:row pb_adjustdocs="0"
pb_adjustpages="0" pb_batch="20100260.001"
pb_batchdir="C:\Datacap\APT\batches\20100260.001" pb_expectdocs="0" pb_expectpgs="8"
pb_headertable=" " pb_ndocs="0" pb_needMeet="0" pb_pagefile="rrsvscan.xml"
pb_pages="8" qs_elaps="8" qs_op="admin" qs_qid="3" qs_start="2010-09-17T07:35:26"
qs_station="2" qs_stop="2010-09-17T07:35:34" qs_taskid="VScan" qs_tsorder="0"
qu_admDB="156" qu_batch="20100260.001" qu_counter="0" qu_done="2010-09-17T07:35:34"
qu_elaps="8" qu_id="3" qu_job="Demo" qu_lock="none" qu_parent="0" qu_priority="5"
qu_spawnwtype="0" qu_start="2010-09-17T07:35:26" qu_status="hold" qu_task="VScan"
qu_tsorder="0" xmlns:z="#RowsetSchema" /></rs:data>
```

The other actions in the batch processing category can manipulate the batches that are identified in the query results record set.

“ProcessChangeBatchStatus” on page 780

“ProcessChangeBatchStatusOrder” on page 780

“ProcessChangeBatchStatusTaskOrder” on page 781

“ProcessClearAuditTable” on page 782

“ProcessClearDebugTable” on page 782

“ProcessDeleteBatches” on page 783

“ProcessDeleteBatchesEx” on page 784

“ProcessInjectBatches” on page 784

“ProcessMoveBatches” on page 785

“ProcessMoveBatchesEx” on page 786

"ProcessMoveDBRecords" on page 787

"ProcessResetPendingOrNotify" on page 788

"ProcessRunSqlQuery" on page 790

ProcessChangeBatchStatus:

Changes the status of one or more batches.

Member of namespace

Maintenance Manager

Syntax

ProcessChangeBatchStatus ()

Parameters

newStatus

Type: string

Parameters

newStatus: The new batch status: aborted, cancelled, finished, hold, job done, pending, running. Smart parameters are supported.

Returns

True if the batch status is successfully changed. Otherwise, **False**.

Level

Batch level.

Details

Using the results from a previous query performed by Maintenance Manager actions, the selected batches have their status attribute changed.

The database connection is closed by this action.

Example:

```
QuerySetStation("1")
QuerySetJobID("!Demo Job")
ProcessRunSqlQuery("")
ProcessChangeBatchStatus("hold")
```

ProcessChangeBatchStatusOrder:

Changes batch status and order.

Member of namespace

Maintenance Manager

Syntax

ProcessChangeBatchStatusOrder ()

Parameters

newStatus

Type: string

newOrder

Type: int

Parameters

- **newStatus**: The new batch status: aborted, cancelled, finished, hold, job done, pending, running. Smart parameters are supported.
- **newOrder**: New task order, zero-based index of task inside its job.

Returns

True if the batch status is successfully changed. Otherwise, **False**.

Level

Batch level.

Details

Using the results from a previous query performed by Maintenance Manager actions, the selected batches have their status and order attributes changed.

The database connection is closed by this action.

Example:

```
ProcessChangeBatchStatusOrder("hold", "1")
```

ProcessChangeBatchStatusTaskOrder:

Changes batch status, task and order.

Member of namespace

Maintenance Manager

Syntax

```
ProcessChangeBatchStatusTaskOrder ()
```

Parameters

newStatus

Type: string

newOrder

Type: int

newTask

Type: string

Parameters

- **newStatus**: The new batch status: aborted, cancelled, finished, hold, job done, pending, running. Smart parameters are supported.
- **newOrder**: The order of the new task, zero-based index of task inside its job.
- **newTask**: The name of the new task.

Returns

True if the batch status is successfully changed. Otherwise, **False**.

Level

Batch level.

Details

Using the results from a previous query performed by Maintenance Manager actions, the selected batches have their status, order and task attributes changed.

The database connection will be closed by this action.

Example:

```
QuerySetStation("1")
QuerySetJobID("!Demo Job")
ProcessRunSqlQuery("")
ProcessChangeBatchStatusTaskOrder("hold", "1", "Verify")
```

ProcessClearAuditTable:

Clears the Audit table located in the admin database.

Member of namespace

Maintenance Manager

Syntax

```
ProcessClearAuditTable ()
```

Parameters

None.

Returns

True if the table is cleared. Otherwise, **False**.

Level

Any level.

Details

Clears the Audit table located in the admin database. This action can be called to ensure that Audit table is cleared when information is moved from one database to another.

Example:

```
ProcessClearAuditTable("")
```

ProcessClearDebugTable:

Clears the Debug table located in the engine database.

Member of namespace

Maintenance Manager

Syntax

ProcessClearDebugTable ()

Parameters

None.

Returns

True if the table is cleared. Otherwise, **False**.

Level

Any level.

Details

Clears the Debug table located in the engine database. This action can be called to ensure that Debug table is cleared when information is moved from one database to another.

Example:

```
ProcessClearDebugTable("")
```

ProcessDeleteBatches:

This action should not be used, as it is scheduled to be removed in future versions. It has been replaced by ProcessDeleteBatchesEx.

Member of namespace

Maintenance Manager

Syntax

ProcessDeleteBatches ()

Parameters

None.

Returns

True if the batches are deleted. Otherwise **False**.

Level

Any level.

Details

This Action has been marked for Deprecation and will be removed in a future release. It has been replaced with the ProcessDeleteBatchesEx action.

ProcessDeleteBatchesEx:

Delete selected batches.

Member of namespace

Maintenance Manager

Syntax

ProcessDeleteBatchesEx ()

Parameters

deleteSubFolders

Type: bool

preserveEngineDBRecords

Type: bool

Parameters

- deleteSubFolders: True deletes any sub-folders within batch directories regardless of creation source.
- preserveEngineDBRecords: True retains engine database records (only batch directories are deleted).

Returns

True if the batches are deleted. Otherwise **False**.

Level

Any level.

Details

Using the results from a previous query performed by Maintenance Manager actions, the selected batches are deleted from disk.

A previous query must have been run to identify the batches to delete. The database connection is closed by this action.

Example:

```
QuerySetStation("1")
QuerySetJobID("!Demo Job")
ProcessRunSqlQuery("")
ProcessDeleteBatchesEx("false,false")
```

ProcessInjectBatches:

Injects the data from master batch to all batches selected and updates DB.

Member of namespace

Maintenance Manager

Syntax

ProcessInjectBatches ()

Parameters

masterBatchID

Type: string

Parameters

masterBatchID: Master batch to copy data from.

Returns

True if all the all the batches were successfully set. Otherwise, **False**.

Level

Any level.

Details

Allows creating large set of batches at some predefined state.

Example:

```
QuerySetBatchRange("20110059.001", "20110059.010")  
InjectBatches("20110059.001")
```

ProcessMoveBatches:

This action should not be used, as it is scheduled to be removed in future versions. It has been replaced by ProcessMoveBatchesEx.

Member of namespace

Maintenance Manager

Syntax

ProcessMoveBatches ()

Parameters

pathTo

Type: string

Parameters

pathTo: The destination directory for the batches. Smart parameters are supported.

Returns

True if the batches are successfully moved. Otherwise, **False**.

Level

Any level.

Details

This Action has been marked for Deprecation and will be removed in a future release. It has been replaced by the ProcessMoveBatchesEx action.

ProcessMoveBatchesEx:

Move selected batches to folder specified in the parameter.

Member of namespace

Maintenance Manager

Syntax

```
ProcessMoveBatchesEx ()
```

Parameters

pathTo

Type: string

moveSubFolders

Type: bool

preserveEngineDBPaths

Type: bool

continueOnError

Type: bool

Parameters

- pathTo: The destination directory for the batches. Smart parameters are supported.
- moveSubFolders: True moves any subfolders within batch directories regardless of creation source. This parameter is required if the batches contain subfolders or else operations, including database updates, will fail.
- preserveEngineDBPaths: True preserves original batch directory paths inside the engine database.
- continueOnError: True continues processing if any single batch fails to be moved with non-fatal error.

Returns

True, if the batches are successfully moved. Otherwise, **False**.

Level

Any level.

Details

Using the results from a previous query that is performed by Maintenance Manager actions, the selected batches are moved to the specified destination directory. A previous query must first be run to identify the batches to move to the new directory. The destination directory must exist. The database connection is closed by this action.

If you are moving batches from one location to another and also moving database records from one database to another, move the batches first. The movement of the database records must be done last.

Examples

The following example runs a query to select all batches from station "1" where the Job ID does not equal "Demo Job", then moves the batches to a directory called "My Old Batches".

```
QuerySetStation("1")
QuerySetJobID("!Demo Job")
ProcessRunSqlQuery("")
ProcessMoveBatchesEx("@STRING(C:\My Old Batches\),false,false,false")
```

This following example selects the same set of batches but it uses a Smart parameter to obtain the full path to directory from the Application Service. A key called BatchArchiveDirectory must exist in the Application Service. If these rules are used for multiple applications, each application can have a unique definition of this value within the Application Service, making these rules flexible.

One benefit of this approach is that in an environment where there is a test system and a production system, each system will have unique values stored in the Application Service. Each of the environments can have a different physical directory specified within the Application Service, allowing the rules in each system to remain identical.

```
QuerySetStation("1")
QuerySetJobID("!Demo Job")
ProcessRunSqlQuery("")
ProcessMoveBatchesEx("@APPPATH(BatchArchiveDirectory),false,false,false")
```

ProcessMoveDBRecords:

Creates connection to the application based on the parameters that are provided and moves selected database data to this application.

Member of namespace

Maintenance Manager

Syntax

ProcessMoveDBRecords ()

Parameters

application

Type: string

server Type: string

admin Type: string

engine

Type: string

debugFlag

Type: bool

user Type: string

password

Type: string

station

Type: string

deleteOriginal

Type: bool

targetDBSQLSeparator

Type: string

Parameters

- application: Application name (Optional). Default value: the name of the application that is running this action.
- server: Server name (Optional). Default value: first available server in the application.
- admin: Administration database name (Optional). Default value: first available Admin DB.
- engine: Engine database name (Optional). Default value: first available Engine DB.
- debugFlag: Debug Flag (Optional). Default value: false.
- user: User name (Optional). Default value: current user credentials.
- password: Password (Optional). Default value: current user credentials.
- station: Station ID (Optional). Default value: Current station name
- deleteOriginal: Delete database records in source database.
- targetDBSQLSeparator: Target database date and time separator (Optional). If blank, that action tries to detect the correct separator that is based on the connection string.

Returns

Always **True**.

Level

Any level.

Details

Creates connection to the application based on the parameters that are provided and moves selected database data to this application. When you are performing operations on a batch and you are moving the database records from one database to another, do the database move operation last. When database records are moved, the database connection is still connected to the original database. Subsequent operations are on the original database, not the records in the new database.

Example:

```
ProcessMoveDBRecords("APTBack","tms","admin","engine","false",  
"admin","admin","1","true","")
```

In this example, the source database is Oracle and the target is Access.

ProcessResetPendingOrNotify:

Resets all selected batches to Pending status.

Member of namespace

Maintenance Manager

Syntax

ProcessResetPendingOrNotify ()

Parameters

threshold

Type: int

asattachment

Type: bool

addressFrom

Type: string

addressTo

Type: string

subject

Type: string

user Type: string

password

Type: string

domain

Type: string

server Type: string

port Type: string

Parameters

All parameters, except for threshold, support Smart parameters.

- threshold: Maximum number of attempts to reset the batch to status 'Pending'.
- asattachment: Send the log as attachment to the email.
- addressFrom: Address From (Optional). Default value: currentUser@currentDomain.
- addressTo: The email recipient. If multiple recipients, separate by using a comma.
- subject: Email subject (Optional). Default value: 'NENU notification current_date_and_time'.
- user: Mail user name (Optional). Default value: current user credentials.
- password: Mail user password (Optional). Default value: current user credentials.
- domain: Mail user domain (Optional). Default value: current domain.
- server: Mail Server name (Optional). Default value: mail.current_domain.
- port: Mail Server Port number (Optional). Default value: 25.

Returns

True, if the email is sent. Otherwise, **False**.

Level

Any level.

Details

Resets the status of all batches that are selected by a previous SQL query to Pending status. The action attempts to reset the batch by the amount that is specified in the **threshold** parameter. If the maximum reset attempts are reached, the action sends an email, along with the log, to the provided list of recipients.

Example:

```
QuerySetStation("1")
QuerySetJobID("!Demo Job")
ProcessRunSqlQuery("")
ProcessResetPendingOrNotify("3", "rrodrig@somewhere.com",
"tom@somewhere.com,john@somewhere.com","", "", "", "", "", "", "")
```

ProcessRunSqlQuery:

Runs the previously defined Maintenance Manager query.

Member of namespace

Maintenance Manager

Syntax

ProcessRunSqlQuery ()

Parameters

None.

Returns

True if the query is successful. Otherwise, **False**.

Level

Any level.

Details

Performs the SQL select query using a query previously built using Maintenance Manager actions. The QuerySet actions need to have been previously called to create the query. The resulting record set is retained in memory and can be acted upon by using other actions.

The action LogWriteRecordSet can be used to write out the recordset, usually for debugging, as well as LogWriteSQLQuery which will output the constructed query. QueryClear can be used to remove any existing query settings.

Example:

```
QuerySetStation("1")
QuerySetJobID("!Demo Job")
ProcessRunSqlQuery("")
```

This example obtains results that include all batches, except for Demo Job, that have been run on station "1".

Logging actions

Use the Logging actions to write information to the Maintenance Manager and Windows log files and to send emails that contain the internal log file.

During rule execution, Maintenance Manager writes status messages to an internal log file and the Rulerunner log file.

- The internal log file is maintained in memory and is used by the **SendEmail** action.
- The Rulerunner log file is stored in the *application_name* > **batches** > **Maintenance Manager** folder.
 - “LogClear”
 - “LogConfigure”
 - “LogSendEmail” on page 793
 - “LogWriteEventLog” on page 794
 - “LogWriteRecordSet” on page 795
 - “LogWriteSQLQuery” on page 795

LogClear:

Clears current Log.

Member of namespace

Maintenance Manager

Syntax

LogClear ()

Parameters

None.

Returns

Always **True**.

Level

Any level.

Details

Clears current log.

Example:

```
LogClear()
```

LogConfigure:

Configures features of aTM logging.

Member of namespace

Maintenance Manager

Syntax

LogConfigure ()

Parameters

severity

Type: int

The log severity limit [0-9]. 0 = maximum logging. 4 or 5 = typically informational or error logging.

filePath

Type: string

Log file path name or blank to disable logging. Smart parameters are supported.

overwrite

Type: bool

True overwrites any existing file.

reflash

Type: bool

True flushes the error message buffer to disk after every write.

showTime

Type: bool

True adds the current time to each log message.

showDate

Type: bool

True adds the current date to each log message.

showSeverity

Type: bool

True adds the severity to the log.

Returns

True if logging is successfully configured. Otherwise, **False**.

Level

Any level.

Details

Configures aTM logging. The severity controls verbosity of the log; logging only severe errors will provide the best performance at expense of reduced debugging information, should a debug trace be required.

This will not change logging as configured for each of the tasks in the target application (i.e. RRS logs). If this action is not called, no aTM log file will be created. aTM logging can be evaluated to see which requests were transmitted to the Maintenance Manager Server.

Example:

```
LogConfigure("5", "@STRING(C:\ParentDir\NENU\Logs\NENU.aTM)")
```

In this example, rolling 'NENU.aTM.#.log' logs is placed into the custom directory.

```
LogConfigure("5", "@APPPATH(export)")
```

In this example, the output directory is retrieved from the application service using Smart parameters. The path is the export directory for the current application.

LogSendEmail:

Sends email with log to comma-separated list of recipients.

Member of namespace

Maintenance Manager

Syntax

```
LogSendEmail ()
```

Parameters

asattachment

Type: bool

addressFrom

Type: string

addressTo

Type: string

subject

Type: string

user

Type: string

password

Type: string

domain

Type: string

server

Type: string

port

Type: string

Parameters

All parameters support Smart parameters.

- **asattachment**: Send the log as attachment to the email.
- **addressFrom**: Address From (Optional). Default value: `currentUser@currentDomain`.
- **addressTo**: Comma-separated list of recipients.
- **subject**: email subject (Optional). Default value: 'NENU notification current_date_and_time'.
- **user**: Mail user name (Optional). Default value: current user credentials.
- **password**: Mail user password (Optional). Default value: current user credentials.
- **domain**: Mail user domain (Optional). Default value: current domain.

- server: Mail Server name (Optional). Default value: mail.current_domain.
- port: Mail Server Port number (Optional). Default value: 25.

Returns

True, if the email is sent. Otherwise, **False**.

Level

Any level.

Details

Sends an email by using the SMTP protocol. As Maintenance Manager actions run, an in-memory log tracks each of the Maintenance Manager actions that are called and their parameters. The LogSendEmail action places the action activity information into an email and send it.

Example:

```
LogSendEmail("jsmith@somewhere.com", "jdoe@somewhere.com",
mmoore@somewhere.com", "", "", "", "", "", "", "")
```

LogWriteEventLog:

Writes a message to the Event Log.

Member of namespace

Maintenance Manager

Syntax

```
LogWriteEventLog ()
```

Parameters

message

Type: string

level Type: int

eventID

Type: int

Parameters

- message: Message to write to the Event Log and the local log. Smart parameters are supported.
- level: Integer value. 0 - informational, 1 - Warning, 2 - Error.
- eventID: Integer value. Desired Event ID.

Returns

True if the event is successfully logged. Otherwise, **False**.

Level

Any level.

Details

Unconditionally writes out a message to the Windows Event Log and to the application log file for the running task. Set the level and event ID to the values that is appropriate for the message being logged.

Example:

```
LogWriteEventLog("This is an informational message.", "0", "10")
```

This example writes the message This is an informational message. to the event log.

```
LogWriteEventLog("@B.MyMessage", "0", "10")
```

This example uses Smart parameters to write out the value of the batch level variable *MyMessage*.

LogWriteRecordSet:

Outputs the results of ProcessRunSqlQuery to the error log.

Member of namespace

Maintenance Manager

Syntax

```
LogWriteRecordSet ()
```

Parameters

None.

Returns

True if the write is successful. Otherwise, **False**.

Level

Any level.

Details

Writes out the result of the last call to ProcessRunSqlQuery to the error file. This action can be useful for debugging an application.

Example:

```
QuerySetStation("1")  
QuerySetJobID("!Demo Job")  
ProcessRunSqlQuery("")  
LogWriteRecordSet("")
```

LogWriteSQLQuery:

Outputs the constructed SQL query to the error log.

Member of namespace

Maintenance Manager

Syntax

LogWriteSQLQuery ()

Parameters

None.

Returns

True if the log is successfully written. Otherwise, **False**.

Level

Any level

Details

Writes out the result of the previous calls to the "QuerySet" actions to the error file. This action can be useful for debugging an application, allowing you to view the exact SQL that was constructed and used in ProcessRunSqlQuery.

Example:

```
QuerySetStation("1")
QuerySetJobID("!Demo Job")
LogWriteSQLQuery("")
ProcessRunSqlQuery("")
```

Reporting actions

Use the Reporting actions to write information to the report tables in the Engine database for use by Datacap Report Viewer.

The Reporting actions can query the active users on an application and set the database tables that contain the reports for processed batches and users.

“ReportQueryTMUsage”

“ReportSetReportingTable” on page 797

“ReportSetUsageDBTable” on page 798

ReportQueryTMUsage:

Update the ReportUser Database with the current users.

Member of namespace

Maintenance Manager

Syntax

ReportQueryTMUsage ()

Parameters

None.

Returns

True if the database update was successful. Otherwise, **False**.

Level

Any level.

Details

This action will query the number of users current logged on and place the information into the reportUser. This information is statistical information that can later be used to generate usage reports with the Report Viewer reporting system.

Example:

```
ReportQueryTMUsage("")
```

ReportSetReportingTable:

Sets database that contains reports on all processed batches

Member of namespace

Maintenance Manager

Syntax

```
ReportSetReportingTable ()
```

Parameters

tbName

Type: string

Table name in Engine database.

batchColumn

Type: string

Optional. Name of the column that contains Batch ID.

attemptColumn

Type: string

Optional. Name of the column that contains attempts.

doneColumn

Type: string

Optional. Name of the column that contains completion result.

actionColumn

Type: string

Optional. Name of the column that contains last action that is performed.

Returns

Always **True**.

Level

Batch level.

Details

Sets database that contains reports on all processed batches. Table must exist in the engine database.

Example:

```
ReportSetReportingTable("NENU","nn_batch","nn_attempt","nn_done","nn_action")
```

ReportSetUsageDBTable:

Sets database that contains reports on users that are logged in to Datacap.

Member of namespace

Maintenance Manager

Syntax

```
ReportSetUsageDBTable ()
```

Parameters

tbName

Type: string

Table name in Engine database.

ipAddressColumn

Type: string

IP address column.

jobIDColumn

Type: string

Job ID column.

portColumn

Type: string

Port column.

processedBatchesColumn

Type: string

Processed batches column.

stationColumn

Type: string

Station ID column.

taskIDColumn

Type: string

Task ID column.

userIDColumn

Type: string

User ID column.

queryTimeColumn

Type: string

Query time column.

Returns

Always **True**.

Level

Batch level.

Details

Sets database that contains reports on users that are logged in to Datacap. Table must exist in the engine database.

Example:

```
ReportSetUsageDBTable("reportUsers","ru_ip","ru_job","ru_port","ru_bathces",  
"ru_station","ru_task","ru_user","ru_time")
```

OCR_A actions

Use the OCR_A actions to do text recognition by using the ABBYY FineReader OCR engine.

If you plan to use the NovoDynamics engine for Arabic recognition, you must separately license the engine directly from the vendor and then install the engine on the machine that is running the recognition rules. In addition, you must complete these steps:

1. Copy the contents of the NovoDyanmics bin folder into the Datacap\DCShared\OCRN folder.

Tip: The default location of the bin folder is C:\Program Files (x86)\NovoDynamics\NovoVerus\bin.

2. Use a command line to register the Datacap connector DLL:

```
C:\WINDOWS\Microsoft.NET\Framework\v2.0.50727\RegAsm.exe  
datacap.libraries.novodynamics.dll /codebase
```

You can ignore a warning that the DLL is not signed.

Arabic language recognition works optimally when the scanned image has at least a 300-DPI resolution.

"EnableEngineLogsOCR_A" on page 800

"OCRA_ConvertImage2BW" on page 800

"RecognizeBarcodeOCR_A" on page 801

"RecognizeFieldOCR_A" on page 801

"RecognizeFieldVoteOCR_A" on page 802

"RecognizePageFieldsOCR_A" on page 803

"RecognizePageOCR_A" on page 804

"RecognizeToALTOOCR_A" on page 804

"RecognizeToPDFOCR_A" on page 806

"ReleaseEngineOCR_A" on page 808

"RotateImageOCR_A" on page 808

"SetAutoRotationOCR_A" on page 809

"SetConfCalculationParamsOCR_A" on page 810

"SetFastModeOCR_A" on page 810

EnableEngineLogsOCR_A

Enables ABBYY Engine logging.

Member of namespace

ocr_a

Syntax

EnableEngineLogsOCR_A ()

Parameters

None.

Returns

True, if logging is successfully enabled. Otherwise, **False**.

Level

Any level.

Details

This action enables ABBYY Engine logging. The log file is created in the batch folder with the name `engine.log`.

Example:

```
RecognizePageOCR_A()  
EnableEngineLogsOCR_A()
```

This sequence creates a log file in the batch folder with the name `engine.log`.

OCRA_ConvertImage2BW

This action converts a Color or Grayscale image to Black and White.

Member of namespace

ocr_a

Syntax

OCRA_ConvertImage2BW (StrParam)

Parameters

The file extension that the action is to assign to the backup of the original Image file. For example: `tio`.

The extension should be 3 or 4 alphanumeric characters.

Returns

False if called at a level other than the Page. **False** if the parameter is not 3 or 4 alphanumeric characters. Otherwise, **True**.

Level

Page.

Details

This action converts a Color or Grayscale image to Black and White.

Example:

```
OCRA_ConvertImage2BW()
```

RecognizeBarcodeOCR_A

A field/page level action that retrieves barcodes.

Member of namespace

ocr_a

Syntax

```
RecognizeBarcodeOCR_A ()
```

Parameters

None.

Returns

False if the ruleset with this action is not bound to a Field object or Page object of the Document Hierarchy. Otherwise, **True**.

Level

Field and Page levels.

Details

A field/page level action that retrieves barcodes.

Example:

```
TaxpayerSSN Rule 1  
RecognizeBarcodeOCR_A
```

RecognizeFieldOCR_A

A field-level action that retrieves a zoned field's settings from the OCR/A tab of the Recognition Options Setup dialog, and uses these settings to recognize the field's value.

Member of namespace

ocr_a

Syntax

```
RecognizeFieldOCR_A ()
```

Parameters

None.

Returns

False if the ruleset with this action is not bound to a Field object of the Document Hierarchy. Otherwise, **True**.

Level

Field level.

Details

This field-level action retrieves a zoned field's settings from the **OCR/A** tab of the **Recognition Options Setup** dialog, and uses these settings to recognize the field's value.

Example:

```
TaxpayerSSN Rule 1  
RecognizeFieldOCR_A()
```

In the example, the rule uses the action to retrieve and apply settings in the **OCR/A** tab of the **Recognition Options Setup** dialog, settings that were previously assigned to a Document Hierarchy's zoned field.

RecognizeFieldVoteOCR_A

A field-level action that initiates a voting procedure that first uses specifications in the **OCR/A** tab of the **Recognition Options Setup** dialog to recognize the field's characters.

Member of namespace

ocr_a

Syntax

```
RecognizeFieldVoteOCR_A ()
```

Parameters

None.

Returns

False if the ruleset with this action is not bound to a Field object of the Document Hierarchy. Otherwise, **True**.

Level

Field level.

Details

This field-level action initiates a voting procedure that first uses specifications in the **OCR/A** tab of the **Recognition Options Setup** dialog to recognize the field's characters.

When this action stores the results of recognition, it first determines if the corresponding Field object of the Document Hierarchy contains a value. If a value is present, the action compares the field's existing value with the recognition results - character by character.

If a particular character's values match, the Confidence Rating for the character is raised to the maximum level. If the values do not match, the Confidence Rating for the character is lowered to the minimum.

Note: When using this voting procedure, the second Recognition engine is secondary and its results are never assigned. Instead, the action changes the Confidence Ratings on the basis of results provided by the first Recognition engine. If there are no recognition results previous to this action, it will act just like the RecognizeFieldOCR_A action.

Example:

```
RecognizeFieldICR_C()  
RecognizeFieldVoteOCR_A()
```

RecognizePageFieldsOCR_A

A page-level action that recognizes all fields on the page that have been configured for OCR/A recognition (see the OCR/A tab of the Recognition Options Setup dialog.)

Member of namespace

ocr_a

Syntax

```
RecognizePageFieldsOCR_A ()
```

Parameters

None.

Returns

False if the ruleset with this action is not bound to a Page object of the Document Hierarchy. Otherwise, **True**.

Level

Page level.

Details

OCR/AThis page-level action recognizes all fields on the page that have been configured for OCR/A recognition (see the **OCR/A** tab of the **Recognition Options Setup** dialog.)

Note: Individual field-level recognition actions will overwrite the results from this page-level action. The action will not recognize a zoned field if the Skip Recognition checkbox in the **OCR/A** tab of the **Recognition Options Setup** dialog has been selected.

Example:

```
ReadZones()  
RecognizePageFieldsOCR_A()
```

RecognizePageOCR_A

Refers to settings in the OCR/A tab of the Recognition Options Setup dialog to recognize all characters on a page, and populates the page's Fingerprint file (.cco) with the recognition results.

Member of namespace

ocr_a

Syntax

```
RecognizePageOCR_A ()
```

Parameters

None.

Returns

False if the ruleset with this action is not bound to a Page object of the Document Hierarchy. Otherwise, **True**.

Level

Page level.

Details

This action responds to settings in the OCR/A tab of the Recognition Options Setup dialog to recognize all characters on a page, and populates the page's CCO file with the recognition results. If a CCO file does not exist at the time this action is called, the action will create one.

Example:

```
AnalyzeImage()  
RotateImage()  
RecognizePageOCR_A()
```

This sequence creates a CCO file for the current page, and checks to see if rotation of the image is needed. Full-page recognition then takes place in response to settings in the **OCR/A** tab of the **Recognition Options Setup** dialog. The recognition results are stored in the CCO file.

RecognizeToALTOOCR_A

Converts a scanned Images (.tif) to an ALTO electronic Document Format (XML) file.

Member of namespace

ocr_a

Syntax

```
bool RecognizeToALTOOCR_A()
```

Parameters

None.

Returns

False If called at an invalid level. Otherwise, **True**.

Level

Document or Page Level.

Details

Converts a scanned Images (.tif) to an ALTO Document Format (XML) file.

When placed at Page level, the action recognizes and converts the current tif page to an ALTO file.

When placed at Document level, the action recognizes and converts all tif pages in the existing doc into one ALTO file.

Document Format

The following variables can be used to set the properties of the ALTO xml file.

y_AltoFontFormattingMode

Specifies the character attributes that are saved to the ALTO xml file.

Valid values are.

- 0 - The only saved attribute is whether characters are subscript or superscript. This value is the default.
- 1 - The following attributes are saved, whether characters are subscript, superscript, bold, italic, underlined, strikeout. Font size and font name are not saved.
- 2 - All font attributes are saved.

y_AltoWriteNondeskewedCoordinates

Specifies whether character, word, block coordinates that are written into files in ALTO format are defined on an original image. Or are defined on an image that is used for recognition to which different modifications, such as deskewing, were applied. This property is **True** by default, which means that the coordinates are defined on the original page.

If you set this property to **False**, export to ALTO is run faster because there is no need to convert the coordinates between the modified image and the original image, which takes a long time. If this property is set to the default **True** value, the baseline position is not written during export. If it is set to **False**, the baseline position is written into the resulting ALTO file because ALTO format requires the baseline position be defined by only one number. In the original coordinates, the baseline might not be strictly horizontal or vertical. In this case, it is impossible to define its position by a single number.

Document Contents

To exclude specific page types, set the variable *typesToExclude* to a comma delimited list of page types to exclude from the ALTO xml file.

To include specific page types, set the variable *typesToInclude* to a comma delimited list of page types to include in the ALTO xml file.

To exclude specific page status, set the variable *statusToExclude* to a comma delimited list of page status to exclude from the ALTO xml file.

When more than one filter is specified, the following order of precedence takes place:

- *statusToExclude* overrides *typesToInclude*
- *typesToInclude* overrides *typesToExclude*

If you are calling the action at the Document level, the types and status filters apply to both the documents and their child pages.

If you are calling the action at the Page level, the types and status filters apply to the page only.

The variables in the previous three sections must be set before you call the RecognizeToALTOOCR_A action.

Example

```
rrset("75", "@D.statusToExclude")
rrSet("Blank", "@D.typesToExclude")
RecognizeToALTOOCR_A()
```

This example creates an ALTO XML document with all of the pages that are contained in the DCO Document object except those pages with type "Blank" and status "75".

RecognizeToPDFOCR_A

Converts a scanned Images (.tif) to an Adobe Portable Document Format (PDF) file.

Member of namespace

ocr_a

Syntax

```
bool RecognizeToPDFOCR_A()
```

Parameters

None.

Returns

False If called at an invalid level. Otherwise, **True**.

Level

Document or Page Level.

Details

Converts a scanned Images (.tif) to an Adobe Portable Document Format (PDF) file. The PDF is searchable as it also includes the text as read directly by the recognition engine.

When placed at Page level, the action recognizes and converts the current tif page to a pdf file.

When placed at Document level, the action recognizes and converts all tif pages in the existing doc into one pdf file.

Document Format

To create PDF/A1A documents, set the *y_pdfA* variable to "1" before you call RecognizeToPDFOCR_A.

To create PDF/A1B documents, set the *y_pdfA* variable to "1" and the *y_pdfA1B* variable to "1" before you call RecognizeToPDFOCR_A.

To set the MRC (mixed Raster Content) Mode for conversion to PDF/A, set the *y_pdfMRCMode* variable to one of the following values:

- 0 - Engine decides whether MRC is to be used. Default.
- 1 - MRC is always used.
- 2 - MRC is never used. MRC technology uses a lossy compression algorithm. Some unimportant information from the source image (background texture, garbage, and so on) can be lost. Disable MRC if even insignificant information from the source image cannot be lost. Using a parameter of 2 helps to address issues where the text in the PDF document is too dark.

Document Contents

To exclude specific page types, set the variable *typesToExclude* to a comma delimited list of page types to exclude from the PDF.

To include specific page types, set the variable *typesToInclude* to a comma delimited list of page types to include in the PDF.

To exclude specific page status, set the variable *statusToExclude* to a comma delimited list of page status to exclude from the PDF.

When more than one filter is specified, the following order of precedence takes place:

- *statusToExclude* overrides *typesToInclude*
- *typesToInclude* overrides *typesToExclude*

If you are calling the action at the Document level, the types and status filters apply to both the documents and their child pages.

If you are calling the action at the Page level, the types and status filters apply to the page only.

Document Attributes

The following variables can be used to set the corresponding pdf document attributes:

- *y_PDFKeys*
- *y_PDFAuthor*
- *y_PDFTitle*
- *y_PDFSubject*
- *y_PDFProducer*
- *y_pdfCreator*
- *y_PDFQuality*
- *y_pdfDelTmp*

The variables in the previous three sections must be set before you call the RecognizeToPDFOCR_A action.

Example

```
rrset("IBM","@D.y_PDFProducer")
rrSet("75","@D.statusToExclude)
rrSet("Blank","@D.typesToExclude)
RecognizeToPDF_A()
```

This example creates a PDF document with all of the pages that are contained in the DCO Document object except those pages with type "Blank" and status "75".

ReleaseEngineOCR_A

This action releases the ABBYY engine.

Member of namespace

ocr_a

Syntax

ReleaseEngineOCR_A ()

Returns

Always **True**.

Level

All.

Details

This action releases the ABBYY engine.

Example:

```
ReleaseEngineOCR_A()
```

RotateImageOCR_A

Use with `Recog_Shared > RotateTIO` to update the CCO file with the correct position coordinates after image rotation.

Member of namespace

ocr_a

Syntax

RotateImageOCR_A ()

Parameters

None.

Returns

False if the ruleset with this action is not bound to a Page object of the Document Hierarchy, or if the action cannot locate the Image file representing the current page. Otherwise, **True**.

Level

Page only.

Details

This action checks if the scanned Image file needs to be rotated by 90, 180, or 270 degrees to be in the upright position. If rotation is necessary, the action saves the Image file in the new, correct position.

Example:

```
RotateImageOCR_A()  
AnalyzeImage()
```

SetAutoRotationOCR_A

This action set to False turns off automatic image orientation detection and rotation.

Member of namespace

ocr_a

Syntax

SetAutoRotationOCR_A (StrParam)

Parameters

True: Forces image orientation detection and rotation. This is the default value.

False: Image orientation and rotation will not be performed.

Returns

Always **True**.

Level

All.

Details

This action set to **True** forces image orientation detection and rotation. If this action is not called, the value will default to **True**. If used, this action must be called prior to recognition and both actions must be called at the same level.

Example:

```
SetAutoRotationOCR_A("True")  
RecognizePageOCR_A
```

SetConfCalculationParamsOCR_A

Specifies the values to use for ABBYY->Datacap confidence mapping.

Member of namespace

ocr_a

Syntax

```
SetConfCalculationParamsOCR_A (StrParam)
```

Parameters

The M and C values for the following formula:

$$\text{Datacap Confidence} = \text{MAX}(10, (M/100) * (\text{ABBY Confidence} + C))$$

The default values for M is 10. The default value for C is 60.

Returns

False if both parameters are not passed or are not numeric. Otherwise, **True**.

Level

Any level.

Details

Specifies the values to use for ABBYY->Datacap confidence mapping.

Example:

```
SetConfCalculationParamsOCR_A(0.1,70)
```

SetFastModeOCR_A

This action set to TRUE provides 2-2.5 times faster recognition speed at the cost of a moderately increased error rate (1.5-2 times more errors).

Member of namespace

ocr_a

Syntax

```
SetFastModeOCR_A (StrParam)
```

Parameters

True Enables Fast Mode which sacrifices recognition quality over speed.

False: Disables Fast Mode causing the recognition to run slower, but provides more accurate results.

If no parameter is specified, the value defaults to **False**.

Returns

Always **True**.

Level

All.

Details

This action set to TRUE provides 2-2.5 times faster recognition speed at the cost of a moderately increased error rate (1.5-2 times more errors).

It is recommended to disable fast mode if you are performing field level recognition because you will sacrifice quality yet see negligible speed increase at the field level.

If you use this action, it must be called prior to recognition.

Example:

```
SetFastModeOCR_A("True")
```

OCR_N actions

Use the OCR_N actions to do recognition by using the NovoDynamics engine.

The OCR_N actions can run recognition on a full page or on all of the field zones that are defined for the current page.

“RecognizePageFieldsOCR_N”

“RecognizePageOCR_N” on page 812

RecognizePageFieldsOCR_N

Does full page recognition and populates the fingerprint (CCO) file of the page with the results.

Member of namespace

Datacap.Libraries.NovoDynamics

Syntax

```
RecognizePageFieldsOCR_N ()
```

Parameters

None.

Returns

False if the ruleset with this action is not bound to a Page object of the Document Hierarchy. Otherwise, **True**.

Level

Page only.

Details

This page-level action recognizes all fields on the page that have been configured for OCR/N recognition (see the action library help text for available settings via runtime\setup variables).

Important: Page level recognition settings are used to recognize the fields\zones. Per zone recognition settings are not supported.

Example:

```
ReadZones()  
RecognizePageFieldsOCR_N()
```

RecognizePageOCR_N

Does recognition on all field zones that are defined for the current page and writes the results to the runtime page data file.

Member of namespace

Datacap.Libraries.NovoDynamics

Syntax

```
RecognizePageOCR_N ()
```

Parameters

None.

Returns

False if the ruleset with this action is not bound to a Page object of the Document Hierarchy. Otherwise, **True**.

Level

Page only.

Details

This action performs full page recognition.

The NormalizeCCO action from the CCO2CCO action library should be called after RecognizePageOCR_N if the application will be using the navigation and pattern match actions to find recognized text on a page or perform pattern matching.

If a CCO file does not exist at the time this action is called, the action will create one.

Example:

```
RotateImage()  
RecognizePageOCR_N()  
NormalizeCCO("")
```

This sequence creates a CCO file for the current page, and checks to see if rotation of the image is needed. Full-page recognition then takes place in response to settings (see the action library help text for available settings via runtime\setup variables).

The recognition results are stored in the CCO file. The words and lines in the CCO are then sorted for use by navigation and pattern match actions.

OCR_S actions

Use the OCR_S actions to do recognition by using the Nuance OmniPage OCR engine.

The OCR_S actions can run recognition tasks on field zones and pages, and write the results to several supported file formats.

“RecognizeDocToPDF”

“RecognizeFieldOCR_S” on page 814

“RecognizeFieldVoteOCR_S” on page 815

“RecognizePageFields2CCO_OCR_S” on page 816

“RecognizePageFieldsOCR_S” on page 817

“RecognizePageOCR_S” on page 817

“RecognizePageOCR_S_2TextFile” on page 818

“RecognizeToFile_OCR_S” on page 819

“RecognizeToPDF” on page 821

“RotateImage” on page 822

“SetEngineTimeout” on page 823

“SetFastTradeOffOCR_S” on page 824

“SetLegacyDecompositionOCR_S” on page 825

RecognizeDocToPDF

Saves all of the pages in the current document as a PDF file.

Member of namespace

OCR_S

Syntax

bool RecognizeDocToPDF (StrParam)

Parameters

A numeric value that indicates the Document Format type:

1. A PDF document with the original image in the foreground with the recognized text hidden in the background (but in the correct position). Perfect for archiving and indexing documents.
2. A general PDF document where the text in the original image is replaced by the corresponding text that is recognized by the engine.
3. A special type of PDF document, where the suspect words are covered by their images cut out from the original image.
4. A non-searchable PDF document.

Returns

False If the action is not applied at the Document level of the Document Hierarchy, or if conversion is not successful. Otherwise, **True**.

Level

Document level only.

Details

This action is Deprecated

This action was deprecated and is scheduled to be removed in a future release. It is recommended that you no longer use this action. Instead, use the RecognizeToPDFOCR_S action in the ocr_sr action library. All of the actions in the OCR_S library are deprecated and should be replaced with the corresponding actions from the ocr_sr library.

Converts all pages in a document to Adobe PDF format and places them in a searchable PDF file (.pdf)

Example:

```
RecognizeDocToPDF("1")
```

RecognizeFieldOCR_S

Does recognition on the zone of the current field and writes the result to the runtime page file.

Member of namespace

OCR_S

Syntax

```
bool RecognizeFieldOCR_S ()
```

Parameters

None.

Returns

False If the ruleset with this action is not bound to a Field object of the Document Hierarchy. Otherwise, **True**.

Level

Field only.

Details

This action is Deprecated

This action was deprecated and is scheduled to be removed in a future release. It is recommended that you no longer use this action. Instead, use the

RecognizeFieldOCR_S action in the ocr_sr action library. All of the actions in the OCR_S library are deprecated and should be replaced with the corresponding actions from the ocr_sr library.

This field-level action retrieves a zoned field's settings from the **OCR/S** tab of the **Recognition Options Setup** dialog, and uses these settings to recognize the value of the field.

Example:

```
TaxpayerSSN Rule 1  
RecognizeFieldOCR_S()
```

In the example, the rule uses the action to retrieve and apply settings in the **OCR/S** tab of the **Recognition Options Setup** dialog. These settings were previously assigned to a zoned field in the **Document Hierarchy**.

RecognizeFieldVoteOCR_S

Does recognition on the zone of the current field and compares the result to the existing field value, character by character. Raises the confidence level when the characters match and lowers it when they do not match.

Member of namespace

OCR_S

Syntax

```
bool RecognizeFieldVoteOCR_S ()
```

Parameters

None.

Returns

False If the ruleset with this action is not bound to a Field object of the Document Hierarchy. Otherwise, **True**.

Level

Field only.

Details

This action is Deprecated

This action was deprecated and is scheduled to be removed in a future release. It is recommended that you no longer use this action. Instead, use the RecognizeFieldVoteOCR_S action in the ocr_sr action library. All of the actions in the OCR_S library are deprecated and should be replaced with the corresponding actions from the ocr_sr library.

This field-level action initiates a voting procedure that first uses specifications in the **OCR/S** tab of the **Recognition Options Setup** dialog to recognize the field's characters.

When this action stores the results of recognition, it first determines whether the corresponding Field object of the Document Hierarchy contains a value. If a value is present, the action compares the field's existing value with the recognition results - character by character.

If a particular character's values match, the Confidence Rating for the character is raised to the maximum level. If the values do not match, the Confidence Rating for the character is lowered to the minimum.

Note: When you are using this voting procedure, the second Recognition engine is secondary and its results are never assigned. Instead, the action changes the Confidence Ratings based on results that are provided by the first Recognition engine. If there are no recognition results previous to this action, it acts just like the RecognizeFieldOCR_S action.

Example

```
RecognizeFieldICR_C()  
RecognizeFieldVoteOCR_S()
```

RecognizePageFields2CCO_OCR_S

Does recognition on all field zones that are defined for the current page and writes the results to the CCO file of the page.

Member of namespace

OCR_S

Syntax

```
bool RecognizePageFields2CCO_OCR_S ()
```

Parameters

None.

Returns

Always **True**.

Level

Page level.

Details

This action is Deprecated

This action was deprecated and is scheduled to be removed in a future release. It is recommended that you no longer use this action. Review the RecognizePageFieldsOCR_S action in the ocr_sr action library to determine whether it meets your needs.

Runs recognition on fields that were designated for OCR/S recognition then transfers the Zonal OCR_S recognition values to the page's CCO file.

Example

```
RecognizePageFields2CCO  
RecognizePageFields2CCO_OCR_S()  
CreateTextFile
```

RecognizePageFieldsOCR_S

Does recognition on all field zones that are defined for the current page and writes the results to the runtime page data file.

Member of namespace

OCR_S

Syntax

```
bool RecognizePageFieldsOCR_S ()
```

Parameters

None.

Returns

False If the ruleset with this action is not bound to a Page object of the Document Hierarchy. Otherwise, **True**.

Level

Page only.

Details

This action is Deprecated

This action was deprecated and is scheduled to be removed in a future release. It is recommended that you no longer use this action. Instead, use the RecognizePageFieldsOCR_S action in the ocr_sr action library. All of the actions in the OCR_S library are deprecated and should be replaced with the corresponding actions from the ocr_sr library.

This page-level action recognizes all fields on the page that were configured for OCR/S recognition (see the **OCR/S** tab of the **Recognition Options Setup** dialog.)

Note: Individual field-level recognition actions overwrite the results from this page-level action.

The action does not recognize a zoned field if the **Skip Recognition** check box in the **OCR/S** tab of the **Recognition Options Setup** dialog was selected.

Example

```
ReadZones()  
RecognizePageFieldsOCR_S()
```

RecognizePageOCR_S

Does full page recognition and populates the page's fingerprint (CCO) file with the results.

Member of namespace

OCR_S

Syntax

```
bool RecognizePageOCR_S ()
```

Parameters

None.

Returns

False If the ruleset with this action is not bound to a Page object of the Document Hierarchy. Otherwise, **True**.

Level

Page only.

Details

This action is Deprecated

This action was deprecated and is scheduled to be removed in a future release. It is recommended that you no longer use this action. Instead, use the `RecognizePageOCR_S` action in the `ocr_sr` action library. All of the actions in the `OCR_S` library are deprecated and should be replaced with the corresponding actions from the `ocr_sr` library.

This action responds to settings in the **OCR/S** tab of the **Recognition Options Setup** dialog to recognize all characters on a page, and populates the page's CCO file with the recognition results.

Important: If a CCO file does not exist at the time this action is called, the action creates one.

Example

```
AnalyzeImage()  
RotateImage()  
RecognizePageOCR_S()
```

This sequence creates a CCO file for the current page, and checks to see whether rotation of the image is needed. Full-page recognition then takes place in response to settings in the **OCR/S** tab of the **Recognition Options Setup** dialog. The recognition results are stored in the CCO file.

RecognizePageOCR_S_2TextFile

Does full page recognition and writes the recognition results to a text file in the batch folder.

Member of namespace

OCR_S

Syntax

```
bool RecognizePageOCR_S_2TextFile ()
```

Parameters

None.

Returns

False If the ruleset with this action is not bound to a Page object of the Document Hierarchy. Otherwise, **True**.

Level

Page only.

Details

This action is Deprecated

This action was deprecated and is scheduled to be removed in a future release. It is recommended that you no longer use this action. Instead, use the `RecognizeToFileOCR_S` action in the `ocr_sr` action library. All of the actions in the `OCR_S` library are deprecated and should be replaced with the corresponding actions from the `ocr_sr` library.

This action generates a Text file (.txt) that contains the raw recognition results for each page in the batch, and adds the file to the current batch.

Note: The action does not create or populate a page's Fingerprint file (.cco) file with the recognition results.

Example

```
RecognizePageOCR_S_2TextFile()
```

RecognizeToFile_OCR_S

Does full page recognition and writes the recognition results to one of several available output file types, such as .doc, .rtf, .html.

Member of namespace

OCR_S

Syntax

```
bool RecognizeToFile_OCR_S (StrParam)
```

Parameters

The action requires a Numeric parameter that ranges 1 - 22 to specify a combination of recognition targets and output formats.

Attention: Image refers to the image of the bound Page object of the Document Hierarchy. File name is the string portion of a file's name that precedes its extension.

The output for all of these parameters produce a file name that is identical to the original file name and has the extension that is specified for that parameter.

1. A PDF document with the original image in the foreground with the recognized text hidden in the background (but in the correct position). Perfect for archiving and indexing documents.
2. A general PDF document where the text in the original image is replaced by the corresponding text that is recognized by the engine.

3. A special type of PDF document, where the suspect words are covered by their images cut out from the original image.
4. A non-searchable PDF document.
5. Recognize an HTML image of the bound Page object of the Document Hierarchy. Output `.html` (HTML 140).
6. Recognize an image of the bound Page object of the Document Hierarchy in an Excel file. Output `.xls` (Excel 2000.)
7. Recognize any image of the bound Page object of the Document Hierarchy in a Word2000 file with a ".doc" extension. Output `.doc` (Word 2000).
8. Recognize any image of the bound Page object of the Document Hierarchy in a WordML file. Output `.doc` (Word ML).
9. Recognize any image of the bound Page object of the Document Hierarchy in a Word97 file. Output `.doc` (Word 97)
10. Recognize any image of the bound Page object of the Document Hierarchy in a RTF2000SWord file. Output `.rtf` (RTF 2000SWord)
11. Recognize any image of the bound Page object of the Document Hierarchy in an RTF2000 file. Output `.rtf` (RTF 2000).
12. Recognize the image of the bound Page object of the Document Hierarchy in a Text file with an ".RTF6" extension. Output `.rtf` (Rich Text).
13. Recognize the image of the Page object of the Document Hierarchy in a Text file with an "RTF6" extension. Output `.rtf` (Rich Text).
14. Recognize the image of the Page object of the Document Hierarchy in a Text file with an ".Text" extension. Output `.txt` (Text).
15. Recognize the image of the Page object of the Document Hierarchy in a Text file with an ".Csv" extension. Output `.txt` (CSV - Comma-Separated Variable).
16. Recognize the image of the Page object of the Document Hierarchy in a Text file with a ".FormattedTxt" extension. Output `.txt` (Formatted Text).
17. Recognize the image of the Page object of the Document Hierarchy in a Text file with a ".UText" extension. Output `.txt` (Text).
18. Recognize the image of the Page object of the Document Hierarchy in a Text file with a ".UCSV" extension. Output `.CSV` (Comma-Separated Variable).
19. Recognize the image of the Page object of the Document Hierarchy in a Text file with a ".UFormattedText" extension. Output `.txt` (Text).
20. Recognize the image of the Page object of the Document Hierarchy in a Text file with an ".Audio" extension. Output `.aud` (Text).
21. Recognize the image of the Page object of the Document Hierarchy in a Text file with a ".WordPad" extension. Output `.rtf` (Rich Text for WordPad).
22. Recognize the image of the Page object of the Document Hierarchy in a Text file with an ".XML" extension. Output `.xml` (XML).

Returns

False If a ruleset with this action is bound to a Field object of the Document Hierarchy, or if the parameter is not a number. Otherwise, **True**.

Level

Page or Document.

Details

This action is Deprecated

This action was deprecated and is scheduled to be removed in a future release. It is recommended that you no longer use this action. Instead, use the `RecognizeToFileOCR_S` action in the `ocr_sr` action library. All of the actions in the `OCR_S` library are deprecated and should be replaced with the corresponding actions from the `ocr_sr` library.

Runs OCR recognition on the image of a source page, and stores the output of the OCR/S recognition engine in a file. The output file is in one of 22 alternative formats. Because the files are not processed in the format you specify, this action is useful primarily for debugging the engine, or if you need raw (unverified) OCR output in that format.

Runs OCR recognition on the image of a source page, and stores the output of the OCR/S recognition engine in a file. The output file is in one of 22 alternative formats. Because the files are not processed in the format you specify, this action is useful primarily for debugging the engine, or if you need raw (unverified) OCR output in that format.

Example

```
RecognizeToFileOCR_S("21")
```

RecognizeToPDF

Does full page recognition and saves the current page as a PDF file.

Member of namespace

`OCR_S`

Syntax

```
bool RecognizeToPDF (StrParam)
```

Parameters

A numeric value that indicates the Document Format type

1. A PDF document with the original image in the foreground with the recognized text hidden in the background (but in the correct position). Perfect for archiving and indexing documents.
2. A general PDF document where the text in the original image is replaced by the corresponding text that is recognized by the engine.
3. A special type of PDF document, where the suspect words are covered by their images cut out from the original image.
4. A non-searchable PDF document.

Returns

False If the rule with this action is not applied to a page. Otherwise, **True**.

Level

Page only.

Details

This action is Deprecated

This action was deprecated and is scheduled to be removed in a future release. It is recommended that you no longer use this action. Instead, use the RecognizeToPDFOCR_S action in the ocr_sr action library. All of the actions in the OCR_S library are deprecated and should be replaced with the corresponding actions from the ocr_sr library.

This action converts a scanned Image file (.tif) to an Adobe Portable Document Format (PDF) file that is specified by the parameter.

Example

```
RecognizeToPDF("3")
```

RotatImage

Use with RotateTIO from the Recog_Shared library to update the CCO file with the correct position coordinates after image rotation.

Member of namespace

OCR_S

Syntax

```
bool RotatImage ()
```

Parameters

None

Returns

False If the ruleset with this action is not bound to a Page object of the Document Hierarchy. Or if the action cannot locate the image file that represents the current page. Otherwise, **True**.

Level

Page only.

Details

This action is Deprecated

This action was deprecated and is scheduled to be removed in a future release. It is recommended that you no longer use this action. Instead, use the RotateImageOCR_S or RotatImageExOCR_S action in the ocr_sr action library. All of the actions in the OCR_S library are deprecated and should be replaced with the corresponding actions from the ocr_sr library.

This action runs automatic rotation of black and white .TIF (or .TIFF) files. The automatic image rotation algorithm relies on, and works best with, images with good quality machine printed text.

If an image contains text with various orientations, for example vertical and horizontal, the image might be rotated undesirably.

The automatic rotation algorithm does not fully work with images that contain nine-pin dot-matrix text or other non-machine printed text.

It is recommended that you call this action in a separate ruleset after recognition due to instances where the recognition engine does not release the image until the ruleset is completed. This problem can manifest as a “cco does not exist” error in the log file.

Example

```
RotateImage()  
RecognizePageICR_C()
```

In this example, automatic image rotation is run before full page recognition by using the ICR_C actions.

SetEngineTimeout

Specifies the number of seconds to wait before it is assumed that an OCR/S recognition action is no longer running correctly.

Member of namespace

OCR_S

Syntax

```
bool SetEngineTimeout (StrParam)
```

Parameters

Numeric value that indicates the number of seconds to wait to determine that an OCR/S recognition action is stalled or exited.

Returns

Always **True**.

Level

All.

Details

This action is Deprecated

This action was deprecated and is scheduled to be removed in a future release. It is recommended that you no longer use this action. Instead, use the SetEngineTimeoutOCR_S action in the ocr_sr action library. All of the actions in the OCR_S library are deprecated and should be replaced with the corresponding actions from the ocr_sr library.

This action sets the number of seconds to wait before it assumes that an OCR/S recognition action is no longer running correctly. When the timeout is reached, the recognition process is removed from memory. SetEngineTimeout is effective only when out-of-process recognition is enabled by the use of UseOutOfProcessRecog, and if OCR/S recognition is being used.

If a recognition action does not complete within the specified number of seconds indicated by a `SetOutOfProcessRecogTimeout` action or a `SetEngineTimeout` action, it is assumed that the recognition engine encountered a severe error. It is removed from memory and recognition automatically restarts one more time. If the recognition action completes successfully within the specified time on either the first or second attempt, the recognition action is successful. If the recognition action does not complete by the specified time on the second attempt, the recognition action is set to abort, if `RecogContinueOnFailure(False)` was used.

If `SetEngineTimeout` is not called, the default value of 180 seconds is used. In normal conditions, the default value is sufficient and does not need to be changed. This value needs to be increased only if a single page consistently takes more than 3 minutes to complete, which is not a typical situation. The programmer can choose to shorten this time to reduce the time to detect failures earlier, provided there is time to run recognition in "worst case" scenarios. For best results, you can set the timeout to be the same or longer than the value specified in a `SetOutOfProcessRecogTimeout` action.

When a `SetEngineTimeout` action is called, the setting is in effect for the entire batch. You can set the value one time. Then, you can call as many recognition actions as you want.

Example

```
SetEngineTimeout("180")
RecogContinueOnFailure("True")
```

SetFastTradeOffOCR_S

Enables or disables fast OCR, which increases recognition speed but might also increase the error rate.

Member of namespace

OCR_S

Syntax

```
bool SetFastTradeOffOCR_S ()
```

Parameters

None.

Returns

Always **True**.

Level

All.

Details

This action is Deprecated

This action was deprecated and is scheduled to be removed in a future release. It is recommended that you no longer use this action.

Increases the speed of the RecognizePageOCR_S action. This action's trade-off might be accuracy for speed. This action must be called before the recognition action and it affects page and field level recognition.

Example

```
AnalyzeImage()  
Rotate Image()  
SetFastTradeOffOCR_S()  
RecognizePageOCR_S()
```

This action speeds the word recognition process of the RecognizePageOCR_S action.

SetLegacyDecompositionOCR_S

Enhances an image in preparation for recognition.

Member of namespace

OCR_S

Syntax

```
bool SetLegacyDecompositionOCR_S ()
```

Parameters

None.

Returns

Always **True**.

Level

All.

Details

This action is Deprecated

This action was deprecated and is scheduled to be removed in a future release. It is recommended that you no longer use this action.

Decomposes an image to prepare it for field or page recognition. This action intensifies gradients between and within words on the current page. The action increases recognition time and so use only as needed. This action affects both page and field level recognition.

Example

```
SetLegacyDecompositionOCR_S("1010")  
RecognizePageOCR_S()
```

This combination creates a CCO file for the current page, intensifies gradients between and within words on the page. It then uses settings in the **OCR/S** tab of the **Recognition Options Setup** dialog to complete word recognition of the page.

OCR_SR actions

Use the OCR_SR actions to do recognition by using the updated Nuance OmniPage OCR engine.

The OCR_S actions can run recognition tasks on field zones and pages, and write the results to several supported file formats.

“RecognizeFieldOCR_S”

“RecognizeFieldVoteOCR_S”

“RecognizePageFieldsOCR_S” on page 827

“RecognizePageOCR_S” on page 828

“RecognizeToFileOCR_S” on page 829

“RecognizeToPDFOCR_S” on page 831

“RotateImageOCR_S” on page 832

“SetEngineTimeoutOCR_S” on page 833

RecognizeFieldOCR_S

Does recognition on the zone of the current field and writes the result to the runtime page file.

Member of namespace

OCR_SR

Syntax

RecognizeFieldOCR_S ()

Parameters

None

Returns

False if the ruleset with this action is not bound to a Field object of the Document Hierarchy. Otherwise, **True**.

Level

Field level.

Details

This field-level action is a shortcut to zonal recognition procedures that are carried out in response to settings in the **OCR/S** tab of Datacap Studio.

Example

```
RecognizeFieldOCR_S()
```

RecognizeFieldVoteOCR_S

Does recognition on the zone of the current field and compares the result to the existing field value, character by character. Raises the confidence level when the characters match and lowers it when they do not match.

Member of namespace

OCR_SR

Syntax

RecognizeFieldVoteOCR_S ()

Parameters

None.

Returns

False if the ruleset with this action is not bound to a Field object of the Document Hierarchy. Otherwise, **True**.

Level

Field only.

Details

This field-level action initiates a voting procedure that first uses specifications in the **OCR/S** tab of the **Recognition Options Setup** dialog to recognize the characters of the field.

When this action stores the results of recognition, it first determines if the corresponding Field object of the Document Hierarchy contains a value. If a value is present, the action compares the field's existing value with the recognition results, character by character.

If a particular character's values match, the Confidence Rating for the character is raised to 9 if the original confidence is smaller than 9. Otherwise the confidence of matching characters is raised to the maximum level (10).

When using this voting procedure, the second Recognition engine is secondary and its results are never assigned. Instead, the action changes the Confidence Ratings on the basis of results provided by the first Recognition engine. If there are no recognition results previous to this action, it acts like the RecognizeFieldOCR_S action.

Example

```
RecognizeFieldICR_C()  
RecognizeFieldVoteOCR_S()
```

RecognizePageFieldsOCR_S

Does recognition on all field zones that are defined for the current page and writes the results to the runtime page data file.

Member of namespace

OCR_SR

Syntax

RecognizePageFieldsOCR_S ()

Parameters

None.

Returns

False if the ruleset with this action is not bound to a Page object of the Document Hierarchy. Otherwise, **True**.

Level

Page only.

Details

This page-level action recognizes all fields on the page that have been configured for OCR/S recognition, see the **OCR/S** tab of the **Recognition Options Setup** dialog.

Note: Individual field-level recognition actions will overwrite the results from this page-level action.

The action does not recognize a zoned field if the **Skip Recognition** checkbox in the **OCR/S** tab of the **Recognition Options Setup** dialog is selected.

Example

```
ReadZones()  
RecognizePageFieldsOCR_S()
```

RecognizePageOCR_S

Does full page recognition and populates the fingerprint (CCO) file of the page with the results.

Member of namespace

OCR_SR

Syntax

```
RecognizePageOCR_S ()
```

Parameters

None.

Returns

False if the ruleset with this action is not bound to a Page object of the Document Hierarchy. Otherwise, **True**.

Level

Page only.

Details

This action responds to settings in the **OCR/S** tab of the **Recognition Options Setup** dialog to recognize all characters on a page, and populates the page's CCO file with the recognition results.

Attention: The NormalizeCCO action from the CCO2CCO action library should be called after RecognizePageOCR_S if the application is using the navigation and pattern match actions to find recognized text on a page or perform pattern matching. If a CCO file does not exist when this action is called, the action creates one.

Example

```
AnalyzeImage()  
RotateImage()  
RecognizePageOCR_S()  
NormalizeCCO("")
```

This sequence creates a CCO file for the current page, and checks to see if rotation of the image is needed. Full-page recognition then takes place in response to settings in the **OCR/S** tab of the **Recognition Options Setup** dialog. The recognition results are stored in the CCO file. The words and lines in the CCO are then sorted for use by navigation and pattern match actions.

RecognizeToFileOCR_S

Does full page recognition and writes the recognition results to one of several available output file types, such as .doc, .rtf, .html.

Member of namespace

OCR_SR

Syntax

RecognizeToFileOCR_S ()

Parameters

FileType

Type int

Parameters

fileType - The action requires a Numeric parameter from 1-22 to specify a combination of recognition targets and output formats.

Important: Image refers to the image of the bound Page object of the Document Hierarchy. Filename is the string portion of a file's name that precedes its extension.

The output for all of these parameters will produce a file name that is identical to the original file name and will have the extension specified for that parameter.

1. A PDF document with the original image in the foreground with the recognized text hidden in the background (but in the correct position). Perfect for archiving and indexing documents.
2. A general PDF document where the text in the original image is replaced by the corresponding text recognized by the engine.

3. A special type of PDF document, where the suspect words are covered by their images cut out from the original image.
4. A non-searchable PDF document.
5. Recognize an HTML image of the bound Page object of the Document Hierarchy. Output .html (HTML 140).
6. Recognize an image of the bound Page object of the Document Hierarchy in an Excel file. Output .xls (Excel 2000.)
7. Recognize any image of the bound Page object of the Document Hierarchy in a Word2000 file with a .doc extension. Output .doc (Word 2000).
8. Recognize any image of the bound Page object of the Document Hierarchy in a WordML file. Output .doc (Word ML).
9. Recognize any image of the bound Page object of the Document Hierarchy in a Word97 file. Output .doc (Word 97)
10. Recognize any image of the bound Page object of the Document Hierarchy in a RTF2000SWord file. Output .rtf (RTF 2000SWord)
11. Recognize any image of the bound Page object of the Document Hierarchy in an RTF2000 file. Output .rtf (RTF 2000).
12. Recognize the image of the bound Page object of the Document Hierarchy in a Text file with an .RTF6 extension. Output .rtf (Rich Text).
13. Recognize the image of the Page object of the Document Hierarchy in a Text file with an .RTF6 extension. Output .rtf (Rich Text).
14. Recognize the image of the Page object of the Document Hierarchy in a Text file with an .Text extension. Output .txt (Text).
15. Recognize the image of the Page object of the Document Hierarchy in a Text file with an Csv extension. Output .txt (CSV - Comma Separated Variable).
16. Recognize the image of the Page object of the Document Hierarchy in a Text file with a .FormattedText extension. Output .txt (Formatted Text).
17. Recognize the image of the Page object of the Document Hierarchy in a Text file with a .UText extension. Output .txt (Text).
18. Recognize the image of the Page object of the Document Hierarchy in a Text file with a .UCSV extension. Output .CSV (Comma Separated Variable).
19. Recognize the image of the Page object of the Document Hierarchy in a Text file with a .UFormattedText extension. Output .txt (Text).
20. Recognize the image of the Page object of the Document Hierarchy in a Text file with an .Audio extension. Output aud (Text).
21. Recognize the image of the Page object of the Document Hierarchy in a Text file with a .WordPad extension. Output .rtf (Rich Text for WordPad).
22. Recognize the image of the Page object of the Document Hierarchy in a Text file with an .XML extension. Output .xml (XML).

Returns

False if a ruleset with this action is bound to a Field object of the Document Hierarchy, or if the parameter is not numeric. Otherwise, **True**.

Level

Page or Document.

Details

Performs OCR recognition on the image of a source page, and stores the output of the OCR/S recognition engine in a file. The output file is in one of 22 alternative formats. Because the files are not actually processed in the format you specify, this action is useful primarily for debugging the engine, or if you need raw (unverified) OCR output in that format.

Example

```
RecognizePageOCR_S_2TextFile(21)
```

RecognizeToPDFOCR_S

Does full page recognition and saves the current page as a PDF file.

Member of namespace

OCR_SR

Syntax

```
bool RecognizeToPDFOCR_S(int OutputPDFType)
```

Parameters

OutputPDFType

Type int

Parameters

A number value that indicates the PDF output type

1. A PDF document with the original image in the foreground with the recognized text hidden in the background (but in the correct position). Perfect for archiving and indexing documents.
2. A general PDF document where the text in the original image is replaced by the corresponding text that is recognized by the engine.
3. A special type of PDF document, where the suspect words are covered by their images cut out from the original image.
4. A non-searchable PDF document.

Returns

False if the rule with this action is not applied to a document or page object or if the parameters are not in the valid range. Otherwise, **True**.

Level

Document and Page only.

Details

This action converts a scanned Image file (.tif) to an Adobe Portable Document Format (PDF) file.

To exclude specific page types, set the variable *typesToExclude* to a comma delimited list of page types to exclude from the PDF.

To include specific page types, set the variable *typesToInclude* to a comma delimited list of page types to include in the PDF.

To exclude specific page status, set the variable *statusToExclude* to a comma delimited list of page status to exclude from the PDF.

When more than one filter is specified, the following order of precedence takes place:

- *statusToExclude* overrides *typesToInclude*
- *typesToInclude* overrides *typesToExclude*

If you are calling the action at the Document level, the types and status filters apply to both the documents and their child pages.

If you are calling the action at the Page level, the types and status filters apply to the page only.

These variables must be set before you call the RecognizeToPDFOCR_A action.

Example

```
rrSet("75", "@D.statusToExclude")
rrSet("Blank", "@D.typesToExclude")
RecognizeToPDF(3)
```

This example creates a PDF document with all of the pages that are contained in the DCO Document object except those pages with type "Blank" and status "75".

RotateImageOCR_S

Use with the RotateTIO action from the Recog_Shared library to update the CCO file with the correct position coordinates after image rotation.

Member of namespace

OCR_SR

Syntax

RotateImageOCR_S ()

Parameters

None

Returns

False if the ruleset with this action is not bound to a Page object of the Document Hierarchy, or if the action cannot locate the image file representing the current page. Otherwise, **True**.

Level

Page only.

Details

This action performs automatic rotation of black and white .TIF (or .TIFF) files. The automatic image rotation algorithm relies on, and works best with, images with good quality machine printed text. If an image contains text with various orientations, for example vertical and horizontal, the image might be rotated undesirably. The automatic rotation algorithm does not fully work with images containing 9-pin dot-matrix text or other non-machine printed text.

It is recommended that this action be called in a separate ruleset after recognition due to instances where the recognition engine will not release the image until the ruleset has completed. This problem can manifest as a cco does not exist error in the log file.

Example

```
RotateImageOCR_S()  
RecognizePageICR_C()
```

In this example, automatic image rotation is performed prior to full page recognition via the ICR_C actions.

SetEngineTimeoutOCR_S

Specifies the number of seconds to wait before it is determined that an OCR/S recognition action is not running properly.

Member of namespace

OCR_SR

Syntax

```
SetEngineTimeoutOCR_S ( )
```

Parameters

Seconds

Type int

Parameters

Seconds: The value that indicates the number of seconds to wait before it is determined that an OCR/S recognition action is stalled or exited.

Returns

False, if the parameter is not numeric or is less than 1. Otherwise, **True**.

Level

Page or Field.

Details

This action sets the number of seconds to wait before it is assumed that an OCR/S recognition action is no longer running correctly. When the timeout is reached, the recognition process is removed from memory.

If a recognition action does not complete within the specified number of seconds indicated by a `SetOutOfProcessRecogTimeout` action or a `SetEngineTimeout` action, it is assumed that the recognition engine encountered a severe error. It is removed from memory, and recognition is automatically restarted one more time. If the recognition action completes successfully within the specified time on either the first or second attempt, that recognition action is successful. If the recognition action does not complete by the specified time on the second attempt, the recognition action is set to abort, if the `RecogContinueOnFailure(False)` action was used.

If `SetEngineTimeout` is not called, the default value of 180 seconds is used. In normal conditions, the default value is sufficient and does not need to be changed. This value must be increased only if a single page consistently takes more than 3 minutes to complete, which is not a typical situation. The programmer can choose to shorten this time to reduce the time to detect failures earlier, provided there is time to perform recognition in worst case scenarios. For best results, this timeout can be set the same or longer than the value specified in a `SetOutOfProcessRecogTimeout` action.

Example

```
SetEngineTimeoutOCR_S(180)
RecognizeFieldOCR_S
```

OpenTextFaxServer actions

Use the `OpenTextFaxServer` actions to import faxes from an `OpenTextFaxServer`.

You can use the `OpenTextFaxServer` actions to create Datacap document batches from incoming faxes. You can also use these actions send the contents of a document to a specified fax number.

“Connect”

“ContinueOnConnectionError” on page 835

“ContinueOnFaxImportError” on page 836

“Disconnect” on page 837

“ImportFaxes” on page 837

“SendAsFax” on page 839

“SetAbortTimeout” on page 839

“SetFaxRemovalAfterImport” on page 840

“SetInputFolder” on page 841

“SetMaxNumberOfFaxes” on page 842

“SetNumberOfRetries” on page 843

“SetPollingInterval” on page 844

“SetProcessedFaxesFolder” on page 844

“SetProtocol” on page 845

“SetRetryTimeout” on page 846

“SetServerName” on page 847

“SetUserID” on page 847

“SetUserPassword” on page 848

“SetWindowsAuthentication” on page 849

Connect

Creates the connection to the Fax server.

Member of namespace

OpenTextFaxServer

Syntax

Connect ()

Returns

False if the action is not called at the batch level or if the connection to fax server cannot be established. Otherwise, **True**.

Level

Batch Level.

Details

Connects to the fax server. This action should be called after setting the server connection parameters via the following actions:

- SetServerName("myserver")
- SetUserID("myuser")
- SetUserPassword("mypassword")
- SetProtocol(4)
- SetWindowsAuthentication(True)

Example

```
SetServerName("myserver")
SetUserID("myuser")
SetUserPassword("mypassword")
SetProtocol(4)
Connect()
```

ContinueOnConnectionError

Specifies whether the batch should continue if there is an error connecting to the server.

Member of namespace

OpenTextFaxServer

Syntax

ContinueOnConnectionError ()

Parameters

Continue

Type: bool

Parameters

A boolean value specifying whether or not the batch should abort if there is an error connecting to the server.

Returns

Always **True**.

Level

Any level.

Details

When the parameter is set to **True**, the batch finishes with Pending status, avoiding the creation of more batches that will result in an aborted status.

If this action is not called, the default value of **False** is used and the batch is aborted at the end of processing.

Include this action before the Connect() action.

Example

```
ContinueOnConnectionError(true)
SetNumberOfRetries(3)
SetServerName("myserver")
SetWindowsAuthentication(True)
SetProtocol(4)
Connect()
ImportFaxes()
```

ContinueOnFaxImportError

Specifies whether the batch should abort if there is an error importing a fax.

Member of namespace

OpenTextFaxServer

Syntax

```
ContinueOnFaxImportError ( )
```

Parameters

Continue

Type: bool

Parameters

A boolean value specifying whether or not the batch should abort if there is an error importing a fax.

Returns

Always **True**.

Level

Any level.

Details

Sets a boolean value specifying whether or not the batch should abort if there is an error importing a fax. When the parameter is set to **True** the batch finishes with Pending status, and contains all faxes that were imported successfully, up to the last one that failed to be imported.

If `ContinueOnFaxImportError` is never called, the `ImportFaxes` action continues processing after an error. Call `ContinueOnFaxImportError(False)` to stop ingestion of faxes after an error.

Include this action before the `Connect()` action.

Example

```
ContinueOnFaxImportError(true)
SetNumberOfRetries(3)
SetServerName("myserver")
SetWindowsAuthentication(True)
SetProtocol(4)
Connect()
ImportFaxes()
```

Disconnect

Disconnects the connection from the Fax server.

Member of namespace

OpenTextFaxServer

Syntax

`Disconnect ()`

Returns

False if the action is not called at the batch level or if the connection to fax server cannot be closed. Otherwise, **True**.

Level

Batch Level.

Details

Disconnects to the fax server. This action should be called after the `Import()` or `Connect()` actions. Typically this action would be called at the Batch's close node, after the connection to the fax server is made and faxes are imported.

Example

```
SetServerName("myserver")
SetUserID("myuser")
SetUserPassword("mypassword")
SetProtocol(4)
Connect()
ImportPages()
Disconnect()
```

ImportFaxes

Imports the faxes from the Fax server into the document batch.

Member of namespace

OpenTextFaxServer

Syntax

ImportFaxes ()

Returns

False if the action is not called at the batch level or if an exception is encountered while importing faxes. Otherwise, **True**.

Level

Batch Level.

Details

This action imports faxes from the fax server. Each fax that is imported is stored in a document inside the Datacap batch. The following fax information will be stored in the document's variables (some of these variables can be empty):

- FaxUniqueID
- FaxStatus
- TotalPages
- LastHistoryChangeDateTime
- FromFaxNumber
- FromName
- FromVoiceNumber
- FromGeneralFaxNumber
- FromGeneralVoiceNumber
- Attachments
- ToFaxNumber
- ToVoiceNumber

Attention: Setting the batch variable *WriteFaxXMLData* to "1" causes the action to write all possible fax properties to an XML file. The XML is named based on the created document ID for a fax, for example, 20120109.000008.01.xml.

Include this action after a Connect() action.

Example

```
SetServerName("myserver")
SetUserID("myuser")
SetUserPassword("mypassword")
SetProtocol(4)
Connect()
ImportFaxes()
```

Note: If the Connect() action is not called prior to calling ImportFaxes(), ImportFaxes() automatically calls the Connect() action. However, the actions that set the connection parameters need to be called prior to ImportFaxes().

SendAsFax

Faxes the contents of the document or page to the specified Fax number.

Member of namespace

OpenTextFaxServer

Syntax

SendAsFax ()

Parameters

ToFaxNumber

Type: string

ToName

Type: string

Parameters

- ToFaxNumber: Recipient's fax number. This parameter is required. Smart parameters are supported.
- ToName: Recipient's name. This parameter is optional. If empty, the default ToName configured for the logged in user (on the server) is used. Smart parameters are supported.

Returns

False, if the action is not called at the document or page levels, or the fax number is not specified, or the document does not contain pages (attachments), or if a connection cannot be made to the fax server, or if the fax server returns an exception while attempting to send the fax. Otherwise, **True**.

Level

Document and Page levels.

Details

Faxes the document contents to the specified fax number.

Example

```
SendAsFax("123-456-8971", "John Doe")
```

A connection to the Fax server must be established via actions before you can use the SendAsFax action.

SetAbortTimeout

Sets the amount of time to wait before you stop running a batch.

Member of namespace

OpenTextFaxServer

Syntax

SetAbortTimeout ()

Parameters

Milliseconds

Type: int

Parameters

Milliseconds : The amount of time, in milliseconds, to wait before aborting a batch. The default value is 10000 ms (10 seconds).

Returns

False if the action is not called at the batch level. Otherwise, **True**.

Level

Batch Level.

Details

Sets the amount of time to wait before aborting a batch.

The action waits the specified time before returning when an abort occurs. This action can be useful to prevent a large number of aborted batches due to an abort condition. For example, if the fax server should become unavailable for a time, the abort timeout will limit the amount of aborted batches until the fax server becomes available again.

If this action is not called, the default value of 10 seconds is used.

Include this action before a ImportFaxes() action.

Example

```
SetServerName("myserver")
SetUserID("myuser")
SetUserPassword("mypassword")
SetProtocol(4)
SetAbortTimeout(5000)
Connect()
ImportFaxes()
```

SetFaxRemovalAfterImport

Sets whether to remove processed faxes from the Fax server. This action must be set to True to enable the import of new faxes.

Member of namespace

OpenTextFaxServer

Syntax

```
SetFaxRemovalAfterImport ()
```

Parameters

RemoveFaxes

Type: bool

Parameters

A boolean that sets whether or not to remove processed faxes from the server. The default value is **False**.

- **True** : Faxes will be removed from the fax server once they are imported into a Datacap batch.
- **False** : Faxes will remain in the fax server once they are imported into a Datacap batch.

The default value is **False**.

Returns

False if the action is not called at the batch level. Otherwise, **True**.

Level

Batch Level.

Details

Sets whether or not to remove processed faxes from the server after they have been imported into the Datacap batch.

If this action is not called, the default value of **False** is used.

Include this action before a ImportFaxes() action.

Example

```
SetServerName("myserver")
SetUserID("myuser")
SetUserPassword("mypassword")
SetProtocol(4)
SetFaxRemovalAfterImport(True)
Connect()
ImportFaxes()
```

SetInputFolder

Sets the name of the input folder where faxes are to be imported from.

Member of namespace

OpenTextFaxServer

Syntax

SetInputFolder ()

Parameters

FolderName

Type: string

Parameters

An string value representing the name of the user folder where faxes are to be imported from.

Returns

Always **True**.

Level

Any level.

Details

If this action is not called, the faxes are imported from the default user folder.

Include this action before a `ImportFaxes()`.

Example

```
SetNumberOfRetries(3)>
SetRetryTimeout(3000)
SetServerName("myserver")
SetWindowsAuthentication(True)
SetProtocol(4)
Connect()
SetInputFolder(INPUT)
ImportFaxes()
```

SetMaxNumberOfFaxes

Sets the maximum number of faxes that are allowed per batch.

Member of namespace

OpenTextFaxServer

Syntax

```
SetMaxNumberOfFaxes ( )
```

Parameters

MaxFaxes

Type: int

Parameters

MaxFaxes : The maximum number of faxes allowed per batch. The default value is 100.

Returns

False if the action is not called at the batch level. Otherwise, **True**.

Level

Batch Level.

Details

Sets the maximum number of faxes allowed per batch.

If this action is not called, the default value of 100 faxes per batch is used.

Include this action before a ImportFaxes() action.

Example

```
SetServerName("myserver")
SetUserID("myuser")
SetUserPassword("mypassword")
SetProtocol(4)
SetMaxNumberOfFaxes(5)
Connect()
ImportFaxes()
```

SetNumberOfRetries

Sets the number of times to attempt a connection to the Fax server after a connection error occurs.

Member of namespace

OpenTextFaxServer

Syntax

```
SetNumberOfRetries ( )
```

Parameters

NumberOfRetries

Type: int

Parameters

An integer value representing the number of times to attempt a connection to the fax server after a connection error occurs.

Returns

Always **True**.

Level

Any level.

Details

Sets the number of times to attempt a connection to the fax server after a connection error occurs.

If this action is not called, the default value of 3 is used.

Include this action before the Connect() action.

Example

```
SetNumberOfRetries(3)
SetServerName("myserver")
SetWindowsAuthentication(True)
SetProtocol(4)
Connect()
ImportFaxes()
```

SetPollingInterval

Sets the number of milliseconds to wait before the OpenTextFaxServer resumes fax polling from the Fax server.

Member of namespace

OpenTextFaxServer

Syntax

```
SetPollingInterval ()
```

Parameters

Milliseconds

Type: int

Parameters

Milliseconds : The amount of time, in milliseconds, to wait before polling the fax server again. The default value is 2000 ms (2 seconds).

Returns

False if the action is not called at the batch level. Otherwise, **True**.

Level

Batch Level.

Details

Sets the amount of time to wait before resuming fax polling from the server.

If this action is not called, the default value of 2 seconds is used.

Include this action before a ImportFaxes() action.

Example

```
SetServerName("myserver")
SetUserID("myuser")
SetUserPassword("mypassword")
SetProtocol(4)
SetPollingInterval(5000)
Connect()
ImportFaxes()
```

SetProcessedFaxesFolder

Sets the name of the folder where faxes are to be moved to after they are imported.

Member of namespace

OpenTextFaxServer

Syntax

```
SetProcessedFaxesFolder ()
```

Parameters

FolderName

Type: string

Parameters

A string value that represents the name of the user folder where faxes are to be moved to after they are imported.

Returns

Always **True**.

Level

Any level.

Details

If this action is not called, the faxes remain in the input folder.

Include this action before a ImportFaxes().

Example

```
SetNumberOfRetries(3)>
SetRetryTimeout(3000)
SetServerName("myserver")
SetWindowsAuthentication(True)
SetProtocol(4)
Connect()
SetProcessedFaxesFolder(OUTPUT)
ImportFaxes()
```

SetProtocol

Sets the protocol to use to connect to the Fax server.

Member of namespace

OpenTextFaxServer

Syntax

SetProtocol ()

Parameters

Protocol

Type: int

Parameters

The protocol to be used to connect to the fax server. The default value is 4 (TCPIP).

Valid parameter values are:

- 1 : Named Pipes
- 2 : IPXOS2
- 3 : SPX
- 4 : TCPIP

- 5 : IPX
- 6 : SecTCPIP
- 7 : SecSPX

Returns

False if the action is not called at the batch level or if the parameter is invalid. Otherwise, **True**.

Level

Batch Level.

Details

Sets the protocol to be used to connect to the fax server.

If this action is not called, the default value of 4 (TCPIP protocol) is used.

Include this action before a ImportFaxes() or Connect() action.

Example

```
SetServerName("myserver")
SetUserID("myuser")
SetUserPassword("mypassword")
SetProtocol(4)
Connect()
ImportFaxes()
```

SetRetryTimeout

Sets the number milliseconds to wait before attempting a connection to the fax server after a connection error occurs.

Member of namespace

OpenTextFaxServer

Syntax

SetRetryTimeout ()

Parameters

Milliseconds

Type: int

Parameters

An integer value representing the number milliseconds to wait before attempting a connection to the fax server after a connection error occurs.

Returns

Always **True**.

Level

Any level.

Details

If this action is not called, the default value of 3000 milliseconds is used.

Include this action before a ImportFaxes() or Connect() action.

Example

```
SetNumberOfRetries(3)>
SetRetryTimeout(3000)
SetServerName("myserver")
SetWindowsAuthentication(True)
SetProtocol(4)
Connect()
ImportFaxes()
```

SetServerName

Sets the name of the Fax server to which you can upload faxes.

Member of namespace

OpenTextFaxServer

Syntax

SetServerName ()

Parameters

ServerName

Type: string

Parameters

ServerName : The name of the fax server. Smart parameters are supported.

Returns

False if the action is not called at the batch level. Otherwise, **True**.

Level

Batch Level.

Details

Sets the name of the fax server to connect to.

Include this action before an ImportFaxes() or Connect() action

Example

```
SetServerName("myserver")
SetUserID("myuser")
SetUserPassword("mypassword")
SetProtocol(4)
Connect()
ImportFaxes()
```

SetUserID

Sets the user ID used to log in to the Fax server.

Member of namespace

OpenTextFaxServer

Syntax

SetUserID ()

Parameters

UserID

Type: string

Parameters

UserID : The user ID to be used to connect to the fax server. Smart parameters are supported.

Returns

False if the action is not called at the batch level. Otherwise, **True**.

Level

Batch Level.

Details

Sets the user ID to connect to the fax server.

Include this action before a ImportFaxes() or Connect() action.

Example

```
SetServerName("myserver")
SetUserID("myuser")
SetUserPassword("mypassword")
SetProtocol(4)
Connect()
ImportFaxes()
```

SetUserPassword

Sets the password used to log in to the Fax server.

Member of namespace

OpenTextFaxServer

Syntax

SetUserPassword ()

Parameters

UserPassword

Type: string

Parameters

UserPassword : The user ID password to connect to the fax server. Smart parameters are supported.

Returns

False if the action is not called at the batch level. Otherwise, **True**.

Level

Batch Level.

Details

Sets the user ID password to connect to the fax server.

Include this action before a ImportFaxes() or Connect() action.

It is recommended that you create an advanced value in the custom values tab in the Application Manager to encrypt your password instead of hard coding it in the action parameter. The password can be retrieved using smart parameters.

Example

```
SetServerName("myserver")
SetUserID("myuser")
SetUserPassword("mypassword")
SetProtocol(4)
Connect()
ImportFaxes()
```

SetWindowsAuthentication

Sets whether to use Windows Authentication to connect to the Fax server.

Member of namespace

OpenTextFaxServer

Syntax

SetWindowsAuthentication ()

Parameters

UseWindowsAuthentication

Type: bool

Parameters

Sets whether or not to use Windows Authentication to connect to the fax server. The default value is **False**.

- **True** : Windows Authentication will be used. The actions SetUserID() and SetUserPassword() are not required when UseWindowsAuthentication is set to **True**.
- **False** : Fax Server user authentication will be used. The actions SetUserID() and SetUserPassword() are required when UseWindowsAuthentication is set to **False**.

The default value is **False**.

Returns

False if the action is not called at the batch level. Otherwise, **True**.

Level

Batch Level.

Details

Include this action before a ImportFaxes() or Connect() action.

Example

```
SetServerName("myserver")
SetWindowsAuthentication(True)
SetProtocol(4)
Connect()
ImportFaxes()
```

PatternMatch actions

Use the PatternMatch actions for pattern-based page identification and for page registration (alignment). Page registration is important when you are working with OMR check boxes.

PatternMatch actions look for a match to specified anchor patterns, identifies the page, sets the page type, and sets the confidence level for pattern matching.

“MatchPattern”

“pat_RecogMatch_Id” on page 851

“pat_RegisterZones” on page 852

“pat_ReleasePageAnchors” on page 853

“PatternMatch_Fingerprint” on page 854

“PatternMatch_Identify” on page 855

“PatternMatch_PageType” on page 856

“SetMatchConfidence” on page 857

MatchPattern

Align the image of this field on the current page with the fingerprint

Member of namespace

PatternMatch

Syntax

MatchPattern ()

Parameters

None.

Returns

False, if any of the follow conditions occur.

- The Anchor position that is returned is not Numeric.
- No image is found.
- The accuracy of the match is below the set Confidence Value.
- An Anchor match does not occur.

Otherwise, **True**.

Level

Field level.

Details

Searches on the current image in a zone that is associated with the current field for a match to the pattern specified for this field in a fingerprint. The zoned area from the original fingerprint is matched against a larger zone in the current image. The search area is controlled by the METRIC variable. METRIC=200,100 means search from 200 pixels to the left and right, and 100 pixels above and below the expected location. If METRIC is not specified, the default is 500 pixels horizontal and vertical.

The fingerprint is determined by the current image's Fingerprint ID, or the Global Fingerprint ID if the current image is not identified.

MatchPattern can be called on any field and if matched, an offset variable is saved for that field. If called before ReadZones, then ReadZones uses the offset for that field when its position is set. Other fields are unaffected.

If the field is matched to the fingerprint with a confidence equal to or greater than the required confidence, the position of the field is set to the found location. If the field is not matched, the function returns false. If not found and the field's Required variable is non-zero, the field status is set to 1 (Error or Validation failed).

This action operates on black and white images, grayscale or color images cause the action to fail. The fingerprint image must have the same resolution (DPI) as the current page image. The geometric shape that is contained in the Anchor field must be bold and well-defined with clear edges, with crisp, black and white markings that product a distinct shape. The shape must be thick and compact, not composed of long thin lines. To avoid false positive matches, the shape must not match other shapes or black areas that might exist nearby within the same image.

Example

```
MatchPattern()
```

pat_RecogMatch_Id

Identifies the current page type by matching OCR results in any fingerprint Anchor zone with OCR results for the corresponding zone on the current page.

Member of namespace

PatternMatch

Syntax

```
pat_RecogMatch_Id ()
```

Parameters

None.

Returns

True if the ruleset is bound to a Page, and a fingerprint matching the text of at least one Anchor field is found. Otherwise, and in case of any errors, **False**. In

addition, the page variable *TemplateID* is set to the matching Fingerprint ID.

Level

Page only.

Details

pat_RecogMatch_Id identifies the page by matching text from any fingerprint Anchor field with the corresponding text on the current page. Fuzzy matching is used. Full-page OCR or ICR must be performed prior to calling this action.

If any Anchor field text in the current page matches the zonal text of any fingerprint, the page is identified by that fingerprint (first match). The Type of the current page is set to the fingerprint page type if a match is found. Full page OCR or ICR must be performed on both the fingerprints and the current image prior to calling this action. Text to be matched is extracted from each fingerprint's Anchor field, which should be defined tightly around the text in the fingerprint. The search area in the current image is the fingerprint-specific field zone in the Document Hierarchy, extended by any associated METRIC variable.

Page identification using pat_RecogMatch_ID (text matching) is mutually exclusive with identification using graphical pattern matching actions (PatternMatch_Identify, etc.). Anchor fields in the Document Hierarchy should be selected carefully so that false positive text matches do not occur.

Example

```
pat_RecogMatch_Id()
```

pat_RegisterZones

Registers and adjusts the positions of all fields on the current source page, based on the positions of the page's designated Anchor field(s).

Member of namespace

PatternMatch

Syntax

```
pat_RegisterZones ()
```

Parameters

None.

Returns

True if the ruleset with this action is bound to a Page object of the Document Hierarchy, and if the action can find all designated Anchor fields. Otherwise, **False**.

Level

Page only.

Details

pat_RegisterZones registers and adjusts the positions of all fields on the current page, based on the previously matched positions of the page's designated Anchor field(s). Anchor fields are determined by the Anchor Field setting in Datacap Studio, for each field. Prior to calling pat_RegisterZones, usually in a different task or ruleset, one of the PatternMatch actions that performs Anchor matching must be called. Then, when the pat_RegisterZones action is called, the expected positions of the Anchor fields on the image (taking into account the Fingerprint classification) are compared with the recognized positions of the fields identified as an Anchor field. The action ReadZones must be called prior to pat_RegisterZones. If any required Anchors are not matched, an operator may be required to update the Anchor position in a verify or fixup task.

All matched or manually adjusted Anchor positions are used for adjustment, Anchors that are not matched are ignored.

- If one Anchor is found, the field positions are all shifted by the same amount.
- If two or more Anchors are found, the field positions are shifted by different amounts, depending on their distance from each Anchor. This process is called Interpolation.

The expected positions of the Anchor fields on the image (taking into account the Fingerprint classification) are compared with the recognized positions of those Anchor fields - or the Anchor positions set manually by a Fixup task's operator.

Example

```
ReadZones()  
pat_RegisterZones()  
PrecognizePageFieldsOCR_S()
```

pat_ReleasePageAnchors

An action that can be called at the end of a batch to release information about the identity and location of a page's Anchor field(s).

Member of namespace

PatternMatch

Syntax

```
pat_ReleasePageAnchors ( )
```

Parameters

None.

Returns

Always **True**.

Level

Page only.

Details

This action can be optionally called to release the small amount of Anchor memory that was allocated by the action `pat_RegisterZones`. If `pat_ReleasePageAnchors` is not called, the memory will be released at the end of the batch or the next time `pat_RegisterZones` is called.

Example

```
pat_ReleasePageAnchors()
```

PatternMatch_Fingerprint

Identifies a page from a specified list of fingerprints.

Member of namespace

PatternMatch

Syntax

```
PatternMatch_Fingerprint (StrParam)
```

Parameters

A comma-separated list of one or more Fingerprint IDs.

Returns

False, if the rule that contains this action was not applied to a Page object of the Document Hierarchy; if a parameter is invalid; if a match does not occur; or if one or more of the specified fingerprints do not exist. Otherwise, **True**.

Level

Page level only.

Details

`PatternMatch_Fingerprint` identifies a page's type and fingerprint by using geometric pattern matching. The locations of unique patterns are configured as Anchor fields for each fingerprint in the Document Hierarchy. One or more Anchor fields can be used to match geometric shapes on a fingerprint to the current image. If one or more Anchor fields on the current page match a fingerprint with equal to or greater than the configured confidence level, the page is identified with that fingerprint. The action does not require all defined anchors to match - the first match is used. The action loads all Anchor field patterns from the specified fingerprints, then searches on the current image for each of the patterns in the associated zones. The search area for each zone is increased by the dimensions that are specified in the page METRIC variable. If METRIC is not specified, the default is 500 pixels horizontal and vertical. When this action finds a match, it sets the matching Fingerprint ID and Page Type. It also creates page-level fields and update the Anchor fields with Anchor-specific pattern offset values in a field-level *Image_Offset* variable. The offset can be used subsequent to matching a fingerprint. The `pat_RegisterZones` action can be used to align the zones in the fingerprint to the current image, providing more accurately positioned text in each field.

This action requires the current page image to be bi-tonal (black and white). Grayscale or color images cause the action to fail. The fingerprint image must have

the same resolution (DPI) as the current page image. The geometric shape that is contained in each Anchor field must be bold and well-defined with clear edges, with crisp, black and white markings that produce a distinct shape. The shape must be thick and compact, not composed of long thin lines. To avoid false positive matches, the shape must not match other shapes or black areas that might exist nearby within the same image.

Example

```
PatternMatch_Fingerprint(1024,1034,1035,1036)
```

This example compares the current page to the four fingerprints that are specified by their IDs.

PatternMatch_Identify

Identifies a page by using image pattern matching.

Member of namespace

PatternMatch

Syntax

```
PatternMatch_Identify ()
```

Parameters

None.

Returns

False, if the rule that contains this action was not applied to a Page object of the Document Hierarchy; if a pattern match is not found; or if fingerprints do not exist. Otherwise, **True**.

Level

Page level only.

Details

PatternMatch_Identify identifies a page's type and fingerprint by using geometric pattern matching. The locations of unique patterns are configured as Anchor fields for each fingerprint in the Document Hierarchy. One or more Anchor fields can be used to match geometric shapes on a fingerprint to the current image. If one or more Anchor fields on the current page match a fingerprint with equal to or greater than the configured confidence level, the page is identified with that fingerprint. The action does not require all defined anchors to match - the first match is used. The action loads all Anchor field patterns from the fingerprint library, then searches on the current image for each of the patterns in the associated zones. The search area for each zone is increased by the dimensions that are specified in the page METRIC variable. If METRIC is not specified, the default is 500 pixels horizontal and vertical. When this action finds a match, it sets the matching Fingerprint ID and Page Type. It also creates page-level fields and update the Anchor fields with Anchor-specific pattern offset values in a field-level *Image_Offset* variable. The offset can be used subsequent to matching a fingerprint. The pat_RegisterZones action can be used to align the zones in the fingerprint to the current image, providing more accurately positioned text in each field.

This action requires the current page image to be bi-tonal (black and white). Grayscale or color images cause the action to fail. The fingerprint image must have the same resolution (DPI) as the current page image. The geometric shape that is contained in each Anchor field must be bold and well-defined with clear edges, with crisp, black and white markings that produce a distinct shape. The shape must be thick and compact, not composed of long thin lines. To avoid false positive matches, the shape must not match other shapes or black areas that might exist nearby within the same image.

Example

```
PatternMatch_Identify()
```

PatternMatch_PageType

Identifies a page according to its Page Type.

Member of namespace

PatternMatch

Syntax

```
PatternMatch_PageType (StrParam)
```

Parameters

One or more Page Types defined in the Document Hierarchy

Returns

False if the rule containing this action was not applied to a Page object of the Document Hierarchy; if the parameter is invalid; if a match does not occur; or if fingerprints do not yet exist. Otherwise, **True**.

Level

Page level only.

Details

PatternMatch_PageType identifies a page's type and fingerprint using geometric pattern matching. The locations of unique patterns are configured as Anchor Fields for each fingerprint in the Document Hierarchy. One or more Anchor fields can be used to match geometric shapes on a fingerprint to the current image. If one or more Anchor fields on the current page match a fingerprint, at or above the configured confidence level, the page is identified with that fingerprint. The action does not require all defined anchors to match - the first match is used. The action loads all Anchor field patterns from fingerprints with the specified page types, then searches on the current image for each of the patterns in the associated zones. The search area for each zone is increased by the dimensions specified in the page METRIC variable. If METRIC is not specified, the default is 500 pixels horizontal and vertical. When this action finds a match, it sets the matching Fingerprint ID and Page Type. It will also create page-level fields and update the Anchor fields with Anchor-specific pattern offset values in a field-level *Image_Offset* variable. The offset can be used subsequent to matching a fingerprint. The pat_RegisterZones action can be used to align the zones in the fingerprint to the current image, providing more accurately positioned text in each field.

This action requires the current page image to be bitonal (black and white), grayscale or color images will cause the action to fail. The fingerprint image must have the same resolution (DPI) as the current page image. The geometric shape contained in each Anchor field should be bold and well defined with clear edges, with crisp black and white markings, producing a distinct shape. The shape should be thick and compact, not composed of long thin lines. To avoid false positive matches, the shape should not match other shapes or black areas that may exist nearby within the same image.

Example

```
PatternMatch_PageType(HCFA 1500)
```

This action looks for a match among the inventory of fingerprints that have a page type of "HCFA 1500".

SetMatchConfidence

Sets the confidence threshold for pattern matching.

Member of namespace

PatternMatch

Syntax

```
SetMatchConfidence (StrParam)
```

Parameters

The value of the confidence threshold.

The value must be between 0 (lowest confidence) and 9 (highest confidence).

Higher values require fewer differences between the compared areas to return a positive match value.

Returns

False if the parameter is not a number between 0 and 9. Otherwise, **True**.

Level

All.

Details

Sets the confidence threshold for pattern matching.

Example

```
SetMatchConfidence(9)
```

Picture actions

Use the Picture actions to do field validations by picture strings. Picture strings define the supported format of a field such as a social security number, phone number, date.

A social security number, for example, is always *<three digits>-<two digits>-<four digits>*. You can define a picture string to represent this format and then use it to make sure that social security number fields contain conforming values.

"PIC_ApplyPictureString"
"PIC_FilterFields"
"PIC_FormatFields" on page 859
"PIC_ReplaceBlankField" on page 861
"PIC_SetPictureCharacter" on page 862
"PIC_ValidateField" on page 863

PIC_ApplyPictureString

Validates the current field by using the specified picture string.

Syntax

()

Parameters

The picture string to validate the field.

Returns

False, if called at the wrong level, if the picture string is longer than the field value or if the field fails the picture string validation. Otherwise, **True**.

Level

Field level.

Details

Validates the current field using a runtime *PictureString* as an argument. See the PIC_FormatFields action for picture string details.

Using the provided picture string, this action will test that each of the characters in the current field are allowed. The provided picture string must be the same length or shorter than the data on the field. If the picture string is shorter, then the last character of the picture string will be used to validate all remaining characters in the field.

See the help for action PIC_FormatFields for an overview of picture strings. Unlike PIC_FormatFields which uses the *PictureString* variable, PIC_ApplyPictureString accepts the picture string as a variable only.

Example

```
PIC_SetPictureCharacter("0,01")  
PIC_SetPictureCharacter("1,0123")  
PIC_SetPictureCharacter("2,-./")  
PIC_ApplyPictureString("0N21N2NN")
```

This example creates custom picture strings for 0, 1 and 2. They are then used here to provide tighter control on the allowed input. "0N21N2NN" format matches a typical 6 digit date specification like "01/07/67".

PIC_FilterFields

Validates the format of the current field, when called from a field, or all fields on the current page, when called from a page. Uses the picture string that is stored in the *PictureString* variable of the field.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

All levels.

Details

Replaces a character and adjusts the confidence based on the *PictureString* defined for the field.

Lowers the Confidence Rating of any character in a field that does not satisfy the Picture String's criteria and replaces the problem character with a low confidence space. It is very similar to the FormatFields action but does not use alternative recognition characters.

This action has two roles. If a character in the field does not match the picture string format defined for that field

1. It replaces any "problem" characters with a space character and marked as low confidence.
2. It lowers the Confidence Rating of any character in a field that does not satisfy the Picture String's criteria.
3. Any alternative recognition characters are removed from the field after execution.

Note: This Action is recursive and will affect all child fields of the calling node. While not direct input to this action, this action works with picture strings that are defined for a field. See the PIC_FormatFields action for a list of all available picture string codes and information about the *PictureString* variable.

Example

```
rrSet("XxN,@F.PictureString")  
PIC_FilterFields()
```

This example expects the current field to have the first character be either a alphabetic character or a digit, the second character can be an alphabetic character, digit or punctuation character and the remaining characters must only be digits.

PIC_FormatFields

Validates the format of the current field or all fields on the current page and uses another to replace problem characters.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

All levels.

Details

Lowers the Confidence Rating of any character in a field that does not satisfy the *PictureString* criteria.

This action adjusts the character confidence of a field, and optionally replaces characters that are based on the picture string set for the field. It is similar to the *FilterFields* action.

This action has two roles, if a character in the field does not match the picture string format that is defined for that field.

1. It replaces any problem characters with an alternative character from a secondary recognition engine, if one exists. If an alternative recognition character does not exist, then the original character is unchanged.

Attention: There must be an equal number of alternative characters as the field length, and the alternative character must also be valid within the field's picture string for substitution to occur. If the alternative recognition character is also not a valid picture string character, then no substitution occurs.

2. It lowers the Confidence Rating of any character in a field that does not satisfy the picture string's criteria.

Alternative recognition characters are removed from the field after execution.

Important: This Action is recursive and affects all child fields of the calling node.

Picture Strings:

This action works with picture strings that are defined for a field. The picture string must be stored in a field variable called *PictureString*. Picture strings improve and filter recognition results, and are used to limit characters that are typed into that field during verify. The *PIC_FormatFields* and *PIC_FilterFields* actions can be called to enforce *PictureString* after recognition rules are started. The *PIC_ApplyPictureString* action is an exception that does not use the *PictureString* variable.

Recognition actions do not pay attention to this property. Individual recognition engines have their own parameters to help guide the recognition. The Web Verify task always enforces *PictureString*. Thick client Verify panels that are constructed by Batch Pilot Autoform also enforces *PictureString* specifications.

PIC_FilterFields replaces non-matching characters with low confidence spaces. *PIC_FormatFields* lowers the confidence. The *DCEdit* control enforces them during verify.

The picture values can be set in *PictureString* in two ways.

1. Use the runner action `rrSet` in a rule set. With this action, you can specify the *PictureString* variable and set it to the value that you want.
2. In the **Zones** tab of Datacap Studio, right-click on the field you want and choose **Manage Variables**.

While not direct input to this action as a standard parameter, here are the valid picture string characters that can be set in the *PictureString* field variable and are then used by this action.

- A: Alphabetic characters only or a space. Numeric and punctuation characters are not valid.
- a: Alphabetic, space and punctuation characters.
- D: Dates. The dates must be expressed with numeric characters. You can delimit months, dates, and years with hyphens, periods, and forward slashes.
- F: Float numbers, which are fractional numbers. To accommodate fractional values, you can include both numbers and a period (for the decimal separator) in this picture string. The F character allows minus signs to represent negative numbers.
- f: Numeric and punctuation characters.
- L: Lowercase alphabetic and space characters.
- l: Lowercase alphabetic, space, and punctuation characters.
- N: Numeric characters only.
- n: Uppercase alphabetic, numeric, or space characters.
- P: Punctuation and space characters.
- T: Time values. These values are expressed in numbers with a colon. In addition, the characters P, M, and A are allowed to distinguish between morning and afternoon times, and colon characters are allowed to delimit hours, minutes, and seconds.
- U: Uppercase alphabetic and space characters.
- u: Uppercase alphabetic, space, and punctuation characters.
- X: Alphabetic, space and numeric characters.
- x: Alphabetic, space, numeric, and punctuation characters.
- Z: Any character.
- #: Numeric characters and the minus sign.

`PIC_SetPictureCharacter` can be used to define up to 10 more application-specific picture strings at run time, identified as 0 through 9.

Example

```
rrSet("AN,@F.PictureString")
PIC_FormatFields()
```

This example expects the current field to contain a single alphabetic character followed by an unlimited number of digits. Here the *PictureString* variable is set at run time, but it can instead be configured at design time in the setup DCO in Datacap Studio.

PIC_ReplaceBlankField

If the current field is blank, sets the field value to the character that is specified.

Syntax

()

Parameters

A character or string that will be placed into the field if it is blank.

Returns

False if it is called at the wrong level or if the parameter is missing, otherwise **True**.

Level

Field level.

Details

- If a field is blank, it replaces it with a single character.
- If a field is empty or only contains spaces, it is replaced with the character or string that is passed in as a parameter.
- If the field is replaced with the input parameter, the confidence is changed to a low confidence of 1.

Example

```
PIC_ReplaceBlankField("~")
```

PIC_SetPictureCharacter

Defines up to 10 custom picture strings (0-9) that you can reference from the `PIC_ApplyPictureString` action.

Syntax

()

Parameters

Two comma-separated parameters

1. The picture string identifier. The value must be between 0 through 9.
2. A string of characters to associate with the picture string identifier (the first parameter).

Returns

False, if the parameter input is invalid. Otherwise, **True**.

Level

Any level.

Details

Configures application-specific picture strings.

In addition to the predefined character strings, custom picture strings can be configured. The picture string values 0 - 9 can be configured to allow validations that are not covered by the predefined settings.

It is possible to configure your verify panel edit control to restrict keyboard entry that is based on picture strings when you use the *PictureString* field variable. Only

predefined picture strings work with the edit control. Any custom picture strings that are created by `PIC_SetPictureCharacter` action do not cause the edit control to restrict user input.

Examples

This example creates custom picture strings for 0, 1 and 2. They are then used to provide tighter control on the allowed input. "0N21N2NN" format matches a typical 6-digit date specification like "01/07/67".

```
PIC_SetPictureCharacter("0,01")
PIC_SetPictureCharacter("1,0123")
PIC_SetPictureCharacter("2,-./")
PIC_ApplyPictureString("0N21N2NN")
```

This example is the same except that the picture string is set up in the *PictureString* variable in Datacap Studio, so it is not seen here.

```
PIC_SetPictureCharacter("0,01")
PIC_SetPictureCharacter("1,0123")
PIC_SetPictureCharacter("2,-./")
PIC_ValidateField()
```

PIC_ValidateField

Validates the format of the current field by using the picture string that is stored in the *PictureString* variable of the field.

Syntax

()

Parameters

None.

Returns

False if the field value does not satisfy the Picture String criteria of the field. Otherwise **True**.

Level

Field level only.

Details

Checks the value of all characters in a field against that field's *PictureString* criteria.

If a character in the field does not match the picture string format defined for that field, it lowers the Confidence Rating of any character in a field that does not satisfy the Picture String's criteria. The criteria is stored in the *PictureString* variable that is bound to the field.

Note: Fields with a status of '-1' (Hidden) are checked but this action will not return false if the value does not match the picture string criteria.

While not direct input parameters to this action, this action works with picture strings that are defined for a field. See the `PIC_FormatFields` action for a list of all available picture string codes and information about the *PictureString* variable.

Example

```
PIC_ValidateField()
```

POLR actions

Use the POLR action matches line items from your invoice image to the corresponding purchase order.

The POLR action pre-matches invoice line items with the purchase order before it runs the verification task.

“CallPOLR”

CallPOLR

Pre-matches invoice line items with the purchase order before it runs the verification task.

Syntax

()

Parameters

The ADOBDB constant number for the PO number field. When using bind variables, the data type of the PO Number is specified with this action.

Returns

Always **True**.

Level

Page level.

Details

This action is used to pre-match invoice line items with the PO prior to verify operator verification. The record set for the PO is retrieved using the information in the settings.ini. This calls the record set according to the DSN in the settings.ini. PODSN and POLookup indicate how to obtain the record set. The record set is expected to be the line items for the PO for the current document and is keyed off of the PO number. It then uses the POLR logic to perform the automatic matching.

Note: The TestPODSN and PODSN ini entries support smart parameters to allow for secure connection strings.

The settings ini file must contain values for these keys:

- [POLR]
- Qty=
- ItemID=
- Price=
- WriteUnusedPOLInes=
- PriceTolerance=
- SeparatorCharacter=

If the station name has a suffix of "-Test" then this key must exist:

- [Database]
- TestPODSN=
- TestPOLookup=

If the station name does not have a suffix of "-Test" then this key must exist:

- [Database]
- PODSN=
- POLookup=

Example:

```
CallPOLR("200")
```

Recog_Shared actions

Use the Recog_Shared actions to do various fingerprint and recognition-related functions.

The Recog_Shared actions can recognize things like check box options and write the recognition results to the page data files.

- "AnalyzeImage"
- "CCONormalization_OFF" on page 866
- "CreateTextFile" on page 867
- "IsBlankPage" on page 868
- "RecogContinueOnFailure" on page 869
- "RecogOMRThresh" on page 870
- "RecogOMRThreshold" on page 870
- "RegisterPageFields" on page 872
- "ReleaseImage" on page 873
- "RotateTio" on page 873
- "SetAdjustFieldToChars" on page 874
- "SetFingerprintRecogPriority" on page 874
- "SetFullPageRecogArea" on page 875
- "SetOutOfProcessRecogTimeout" on page 876
- "SetRecogFailureRetryDelay" on page 877
- "SnapCCOtoDCO" on page 878
- "SnapDCOtoCCO" on page 879
- "SnapFieldtoChars" on page 879
- "UseOutOfProcessRecog" on page 880

AnalyzeImage

Converts the Image file (.tif) that represents the current page to a Fingerprint file (.cco) file for the page.

Member of namespace

Recog_Shared

Syntax

AnalyzeImage ()

Parameters

None.

Returns

False if the ruleset with this action is not bound to a Page object of the Document Hierarchy. Otherwise, **True**.

Level

Page level.

Details

This action converts the Image file (.tif) that represents the current page to a CCO file for the page.

A ruleset with this action should be bound to a Page object that represents an application's source page.

The action is not required if full-page recognition takes place using actions such as RecognizePageOCR_S or RecognizePageICR_C.

Attention: Fingerprint matching accuracy can decline for images with a very large number of small dots. This condition might be due to large dotted or shaded areas on the original document, speckled noise in the scanned image, or images with text characters that are very broken up.

Example

```
AnalyzeImage()  
RotateImage()  
SetProblemValue(0.5)  
SetSearchArea(0.5)  
FindFingerprint(True)
```

This sequence generates a CCO file for the current page, then checks to see if rotation of the image is needed. Finally, the sequence attempts to match the current page with a fingerprint. For more about the matching process, see the descriptions of the AutoDoc actions.

CCONormalization_OFF

Prevents the automatic running of NormalizeCCO procedures after a full-page recognition action was run.

Member of namespace

Recog_Shared

Syntax

```
CCONormalization_OFF ()
```

Parameters

None.

Returns

False if the action does not run at the Page level. Otherwise, **True**.

Level

Page level.

Details

A full-page recognition action such as `RecognizePageICR_C` automatically calls the `NormalizeCCO` action, which is thorough but time-consuming, after recognition is complete. This action is part of the `cco2cco.rrx` file.

To bypass this procedure, place `CCONormalization_OFF` just before the recognition action.

Example

```
CCONormalization_OFF()  
RecognizePageICR_C()
```

CreateTextFile

Creates a Text file (.txt) for the current page; adds the page's recognized values to the file; and places the file in the current batch, in your application's Batches directory.

Member of namespace

Recog_Shared

Syntax

```
CreateTextFile ()
```

Parameters

None.

Returns

False if the ruleset with this action is not bound to a Page object of the Document Hierarchy, or if an Image file for the current page is not available. Otherwise, **True**.

Level

Page only.

Details

This action creates a UTF-8 encoded Text file (.txt) for the current page; adds the page's recognized values to the file; and places the file in the current batch, in your application's Batches directory.

Attention: The Text file generated by this action is handy for debugging purposes, to see what recognition is placing into the page's Fingerprint file (.cco) file. The action should follow a full-page recognition action such as `RecognizePageOCR_S`, in a rule that is applied to a Page object of the Document Hierarchy.

Example

```
RecognizePageOCR_S()  
CreateTextFile()  
SetProblemValue(0.7)  
SetSearchArea(0.5)  
FindFingerprint(True)
```

After the full-page recognition action (RecognizePageOCR_S), the CreateTextFile() action places the recognized values into a Text file that it has set up for the page, and adds the file to the current batch, in the Batches directory of the application.

The text file that is created has the same filename as the image, but is assigned a .txt filename extension.

IsBlankPage

Counts the number of words in the Fingerprint file (.cco) file of the current page and returns True if the count is less than or equal to the number you enter as the parameter.

Member of namespace

Recog_Shared

Syntax

IsBlankPage (StrParam)

Parameters

Long value indicating the maximum number of words in the Fingerprint file (.cco) of a blank source page. "50", for example, tells the action that if a CCO file has 50 words or less, its page is blank. Valid values are 0 to 2,147,483,647.

Returns

False if the action parameter is invalid or if the action is unable to locate the Image file for the current page or its CCO file. Otherwise, **True**.

Level

Page level.

Details

This action counts the number of words in the CCO file of the current page and returns **True** if the count is less than or equal to the number you enter as the parameter.

A rule containing this action should apply to a Page object; within the rule, this action should come after one of the actions that creates a fingerprint, such as AnalyzeImage, RecognizePageOCR_S, or RecognizePageICR_C.

Example

```
AnalyzeImage()  
IsBlankPage(5)  
SetPageType(Separator)
```

This sequence uses `AnalyzeImage` to create a CCO file, then checks to see if the file contains less than six words. If so, the `IsBlankPage(5)` action returns **True**. The final action, a DCO action, establishes the page as a Separator page.

RecogContinueOnFailure

Determines if a batch will abort if page or field recognition fails.

Member of namespace

Recog_Shared

Syntax

`RecogContinueOnFailure (StrParam)`

Parameters

String value **True** or **False**.

1. **True**: a recognition failure will not be automatically retried if recognition fails. The batch will continue and the application can use the value assigned to the *RecogStatus* variable to decide how to proceed on success or failure of recognition. For more information on the *RecogStatus* variable, see information about the `RecogContinueOnFailure` action.
2. **False**: causes the batch to abort if a full-page or field-level recognition action fails. If `UseOutOfProcessRecog` is enabled, the batch will abort only if the second recognition attempt fails.

Returns

Always **True**.

Level

All.

Details

This action determines if a batch will abort if page or field recognition failed.

Note: If `RecogContinueOnFailure` is not specifically called, the default **False** value is used. This means that batches will abort if recognition fails.

After a recognition operation is complete, the variable *RecogStatus* is set to indicate the success or failure of recognition. If page-level recognition is being performed, *RecogStatus* values of 0, 1 or 2 are considered successful.

The full list of values includes:

- 0 - Success
- 1 - Recognition was successful but there are no results, the page was empty.
- 2 - Recognition was successful and additional processing such as `RotateImage` was performed.
- 4 - Failure: the recognition engine cannot be instantiated.
- 5 - Failure: the recognition engine timed out (the time specified by the `SetEngineTimeout` action has expired).

- 6 - Failure: could not load image to engine.
- 7 - Failure: could not load image to engine (path not found).
- 8 - Failure: image could not be rotated.
- 10 - Failure: general failure occurred and recognition was not completed.

Example

```
RecogContinueOnFailure(True)
RecognizePageOCR_S()
```

RecogOMRThresh

This action should not be used, as it is scheduled to be removed in future versions. It has been replaced by RecogOMRThreshold.

Member of namespace

Recog_Shared

Syntax

RecogOMRThresh (StrParam)

Parameters

Two comma-separated Floating Point or Integer values that specify the count of black pixels in OMR boxes:

1. **Threshold:** the percentage of pixels in the zone - the field zone not the printed box that should be considered "checked", i.e. the lightest box that is not just noise, but should be considered a check mark.
2. **Background:** the percentage of pixels in the zone that might be due to scanner noise and/or the border of the printed box. This value also controls the range on either side of the Threshold value that is low confidence.

Details

This action was replaced by RecogOMRThreshold.

See also

RecogOMRThreshold

RecogOMRThreshold

Performs OMR check box recognition by counting black pixels within each OMR box area in a Field with one or more OMR boxes.

Member of namespace

Recog_Shared

Syntax

RecogOMRThreshold ()

Parameters

String: threshold

String: background

Parameters

Floating or integer values that specify the count of black pixels in OMR boxes:

1. **Threshold:** the percentage of pixels in the zone. The field zone that is not the printed box that must be considered checked. For example, the lightest box that is not just noise, but must be considered a check mark.
2. **Background:** the percentage of pixels in the zone that might be due to scanner noise or the border of the printed box. This value also controls the range on either side of the Threshold value that is low confidence.

The action also accepts parameters that are fractional percentages, which are needed to detect marks in large zones.

The parameters must be experimentally adjusted on real-world scanned forms. First, determine the Threshold value that correctly identifies a light mark as checked, and correctly identifies noisy zones as cleared. Second, adjust the value of the **Background** parameter to achieve an acceptable confidence interval.

Returns

Always **True**.

Level

Field level.

Details

This action performs OMR check box recognition by counting black pixels within each OMR box area in a Field with one or more OMR boxes.

- **Text Boxes:** The action sets the text value of the field to a string of 0's and 1's (one digit per OMR box). It assigns a Confidence String to the string of digits: 4 for Low Confidence up to 9 for High Confidence.
- **Density String and Confidence Value:** The action also establishes a *DensityString* variable for the Character String, indicating percentage-filled, from ASCII 48 ('0') through 148. For each possible OMR box, is a character. The ASCII value of the character minus 48 is the percentage-filled. If the Density String=0X, the first OMR field was blank, and the second was 40% filled. The ASCII value for X is 88. 88 minus 48 = 40.
- **MultiPunch and Confidence Values:** If the MultiPunch setting is set to 1 and multiple OMRs are filled beyond the threshold, the one that was filled the most is marked and set to Low Confidence.
 - If the percentage-filled is below the second parameter, the OMR box is not selected and the confidence is high.
 - If the percentage-filled is between the two parameters, the OMR box is not selected and the confidence is low.
 - If the percentage-filled is above the first parameter and below double the first parameter minus the second parameter, the OMR box is selected and the confidence is low.
 - If the percentage-filled is above double the first parameter minus the second parameter, the OMR box is selected and the confidence is high.

Note: The `RecognizeThreshold` action works best on dropout boxes, but with an appropriate background value can work effectively with boxes that are visible in the scanned image.

If you are using small visible boxes on your image, it is best to zone the area by surrounding the entire visible box with room for alignment movement. Then, factor out the black from the box by using the parameters. If you attempt to zone inside the borders of a visible box, you can get a false positive if the page does not align exactly.

This action requires `dcimage.ocx.v.6.03.22` or above.

Examples

For a small to medium size zone, 10% filled might be considered a deliberate mark. Anything below 5% (Background) is not a mark. Anything above 15% (Threshold + (Threshold – Background)) is a high confidence mark. This works with a non-dropout OMR field where the printed outline of the box takes up less than 5% of the zone area. It also works for dropout forms.

```
RecognizeThreshold("10","5")
```

The following example is for a signature line, or a large zone where the percentage-filled is much lower than for a small zone. This example assumes there is low background or noise.

```
RecognizeThreshold("2","0")
```

RegisterPageFields

Returns the field positions for all zoned fields of the current page.

Member of namespace

`Recognize_Shared`

Syntax

```
RegisterPageFields ()
```

Parameters

None.

Returns

False if a ruleset with this action is not bound to a Page object of the Document Hierarchy; or if the action cannot find the page's Fingerprint file (.cco). Otherwise, **True**.

Level

Page level.

Details

This action returns the field positions of all zoned fields of the current page. The action is similar to the `ReadZones` action of the `Zones.rxx` file.

Note: Use the `ReadZones` action when possible.

Example

```
RegisterPageFields()
```

ReleaseImage

This action has been deprecated.

Member of namespace

Recog_Shared

Syntax

```
ReleaseImage ()
```

Parameters

None.

Returns

Always **True**.

Level

Page level.

Details

This action has been deprecated.

Example

```
ReleaseImage()
```

RotateTio

Checks if an Image file processed by the ImageFix action that assigns a .tio extension to the file needs to be rotated by 90, 180, or 270 degrees. If so, the action rotates and then saves the Image file with the same .tio extension.

Member of namespace

Recog_Shared

Syntax

```
RotateTio (StrParam)
```

Parameters

A String value:

- **True** to initiate rotation
- **False** to prevent rotation

Returns

Always **True**.

Level

Page level.

Details

This action checks if an Image file processed by the ImageFix action that assigns the .tio extension needs to be rotated by 90, 180, or 270 degrees. If rotation is necessary, the action saves the Image file with the same .tio extension.

Example

```
AnalyzeImage()  
RotateTio(True)  
RotateImage()  
RecognizePageICR_C()
```

SetAdjustFieldToChars

Optional setting for SnapCCOtoDCO to adjust the field position to its character positions.

Member of namespace

Recog_Shared

Syntax

SetAdjustFieldToChars (StrParam)

Parameters

A String value:

- **True** to snap character positions
- **False** to disable snapping

Returns

True.

Level

Page or Field level.

Details

This action has SnapCCOtoDCO adjust the field position (parameter **True**) to the character positions results after snapping the character values to the field. Off by Default

Example

```
SetAdjustFieldToChars(TRUE)  
SnapCCOtoDCO()
```

SetFingerprintRecogPriority

Sets the option that controls whether a full-page recognition action is to create a Fingerprint file(.cco) - aka a CCO file - for the current page.

Member of namespace

Recog_Shared

Syntax

SetFingerprintRecogPriority (StrParam)

Parameters

String value: **True** or **False** to control the creation of the CCO.

- **True:** If a CCO already exists prior to recognition, it is replaced with a brand new one with recognition results.
- **False:** If SetFingerprintRecogPriority is not called or is set to **False** and a CCO already exists prior to recognition, the recognition results will be added to that CCO.

Returns

Always **True**.

Level

All.

Details

This action sets the option that controls whether a full-page recognition action is to create a CCO file for the current page. When the option is On, processing is faster because the call to the AnalyzeImage action is eliminated.

The difference between creating a CCO from scratch with recognition results and adding the recognition results to the existing CCO created by AnalyzeImage is that in the adding case, the recognized characters are put into the CCO in a manner that uses a different fingerprinting technique.

Note: Be sure to place this action before a full-page recognition action.

Example

```
SetFingerprintRecogPriority(True)
```

SetFullPageRecogArea

An optional action that sets the area of the current page that is to be the target of recognition procedures, when full-page recognition action is invoked.

Member of namespace

Recog_Shared

Syntax

SetFullPageRecogArea (StrParam)

Parameters

A decimal value indicating the percent of the page to be recognized in response to this action.

For example: "0.1" designates the first 10% of the page, while "1.0" calls for recognition of the entire page.

This action is helpful if you know that a page's values will always be in a particular location on the page, but recognition of the entire page is not necessary.

Returns

False if the ruleset with this action is not bound to a Page object of the Document Hierarchy, or if the action's parameter is not a decimal value. Otherwise, **True**.

Level

Page level.

Details

This optional action sets the area of the current page that will be the target of recognition procedures when full-page recognition action is called. For example: "0.1" indicates that the first 10% of the page is to be recognized; "1.00" indicates that the entire page is to be recognized.

Example

```
SetFullPageRecogArea(0.5)
```

SetOutOfProcessRecogTimeout

Sets the number of seconds to wait before it is determined that a recognition action is no longer running properly.

Member of namespace

Recog_Shared

Syntax

```
SetOutOfProcessRecogTimeout (StrParam)
```

Parameters

Numeric value that indicates the number of seconds to wait to determine that a recognition action is stalled or exited.

Returns

Always **True**.

Level

All.

Details

This action sets the number of seconds to wait before it is assumed that a recognition action is no longer running correctly. When the timeout is reached, the recognition process is removed from memory. The SetOutOfProcessRecogTimeout action is effective only when out-of-process recognition is enabled by the use of a UseOutOfProcessRecog action.

If a recognition action does not complete within the specified number of seconds indicated by a `SetOutOfProcessRecogTimeout` action or a `SetEngineTimeout` action, it is assumed that the recognition engine encountered a severe error. It is removed from memory and recognition automatically restarts one more time. If the recognition action completes successfully within the specified time on either the first or second attempt, that recognition action is successful. If the recognition action does not complete by the specified time on the second attempt, the recognition action is set to abort, if `RecogContinueOnFailure(False)` was used.

If `SetOutOfProcessRecogTimeout` is not called, the default value of 300 seconds is used. In normal conditions, the default value is sufficient and does not need to be changed. This value needs to be increased only if a single page consistently takes more than 5 minutes to complete, which is not a typical situation. The programmer can choose to shorten this time to reduce the time to detect failures earlier, provided there is time to perform recognition in worst case scenarios. For best results, you can set the timeout to be the same or longer than the value specified in a `SetEngineTimeout` action.

When a `SetOutOfProcessRecogTimeout` action is called, the setting is in effect for the entire batch so that you can set the value once, then call as many recognition actions as you want.

Example

```
SetOutOfProcessRecogTimeout(300)
UseOutOfProcessRecog(True)
RecognizePageOCR_S()
```

SetRecogFailureRetryDelay

Sets the number of seconds to wait before restarting a failed recognition action.

Member of namespace

Recog_Shared

Syntax

`SetRecogFailureRetryDelay (StrParam)`

Parameters

Numeric value indicating the number of seconds to wait before restarting a failed recognition action, and automatically reactivating recognition one more time.

Returns

Always **True**.

Level

All.

Details

This action sets the number of seconds to wait after the time specified in either a `SetOutOfProcessRecogTimeOut` action or a `SetEngineTimeout` action has expired. Once either timeout has occurred, the recognition engine is removed from memory: the action will then wait the additional time specified by the `SetRecogFailureRetryDelay` action to be sure that the engine has exited before

restarting recognition. SetRecogFailureRetryDelay only has an effect if out-of-process recognition has been enabled by a UseOutOfProcessRecog action.

If a recognition action does not complete within the number of seconds specified by a SetOutOfProcessRecog action or a SetEngineTimeout action, it is assumed that the recognition engine has encountered a severe error and that recognition will automatically be restarted one more time. If the recognition action completes successfully within the specified time on either the first or second attempt, that recognition action will be successful. If the recognition action does not complete by the specified time on the second attempt, the recognition action will be set to abort if RecogContinueOnFailure(False) has been used.

If SetRecogFailureRetryDelay is not specifically called, the default value of 10 seconds is used. Under normal conditions, the default value will be sufficient and does not need to be changed. This value needs to be increased only if a log indicates that errors are occurring when attempting to restart a failed recognition action, and the problem can be diagnosed by setting the RecogStatus to "4".

When SetRecogFailureRetryDelay is called, its setting will be in effect for the entire batch. This allows you to set the value once, and call as many recognition actions as necessary.

Example

```
SetRecogFailureRetryDelay(10)
UseOutOfProcessRecog(True)
RecognizePageOCR_S()
```

SnapCCOtoDCO

Transfers the recognition results in the current page's CCO file - its Fingerprint file - to the appropriate Field objects of the Document Hierarchy...its setup DCO.

Member of namespace

Recog_Shared

Syntax

SnapCCOtoDCO ()

Parameters

None.

Returns

False if a ruleset with this action is not bound to a Page object or Field object of the Document Hierarchy. Otherwise, **True**.

Level

Page or Field level.

Details

This action transfers the recognition results of the current page's CCO file to the appropriate Field objects of the Document Hierarchy (DCO). Note that the action only transfers values to Field objects.

SnapCCOToDCO will only clear / update field text when all of the following conditions are met:

- Field is not an OMR field, for example var *RecogType*=4).
- Field has positions assigned.
- Field does not have the variable *v_skipsnap* set to 1.
- Field has data mapping to the CCO, at least one character.
- Fixes issue that would affect processing of reserved fields. For example, fields that are used for anchor finding are followed by snapping of data.

Example

```
SnapCCOToDCO()
```

SnapDCOtoCCO

Transfers the recognition results assigned to Field objects of the Document Hierarchy (aka the setup DCO) to the current page's CCO file, also known as its Fingerprint file.

Member of namespace

Recog_Shared

Syntax

```
SnapDCOtoCCO ()
```

Parameters

None.

Returns

False if a rule with this action is not applied to a page. Otherwise, **True**.

Level

Page level.

Details

This action transfers the recognition results assigned to Field objects of the Document Hierarchy (DCO) to the current page's CCO file.

If zonal recognition is used instead of full-page recognition, the action will populate the current page's CCO file with the results of zonal recognition. Then, when the Verify task runs, a user can use the ClickNKey option to populate fields.

Example

```
SnapDCOtoCCO()
```

SnapFieldtoChars

Adjusts the zone position of the passed dco field to the field's character positions.

Member of namespace

Recog_Shared

Syntax

`SnapFieldtoChars ()`

Parameters

String: Smartparam

Parameters

A Smart Parameter value representing a valid Field location.

Returns

False if a valid DCO field is not returned from the Smart parameter value. Otherwise, **True**.

Level

Any level.

Details

This action adjusts the field position of the passed DCO to the DCO's character positions. If the field does not have a text value, no adjustment to the field zone is performed.

Example

```
SnapFieldtoChars(@F)
```

UseOutOfProcessRecog

Causes recognition to be performed in a process that is separate from the process that is running the recognition actions.

Member of namespace

Recog_Shared

Syntax

`UseOutOfProcessRecog (strParam)`

Parameters

True: Recognition actions should run in a separate process.

False: Recognition should run in the same process as the recognition actions.

Returns

Always **True**.

Level

All.

Details

This action determines in which process recognition will be performed. Using a separate process for recognition provides an additional stability and automatic recovery ability as it will automatically retry a recognition action that runs into trouble, such as recognition that has stalled or unexpectedly terminated. The action must be placed before a full-page or field-level recognition action such as `RecognizePageOCR_S`.

The action is also directly tied to the `SetRecogFailureRetryDelay` action, which determines how long (in seconds) the `UseOutOfProcessRecog` action waits to determine that recognition has stopped responding and must be retried.

If the `UseOutOfProcessRecog` action is not specifically called, its default `True` setting will be used. If the action is called specifically, the **True** or **False** setting will be in effect for the entire batch. This allows you to set the value once, and call as many recognition actions as necessary.

Example

```
UseOutOfProcessRecog(True)
SetRecogFailureRetryDelay(10)
RecognizePageOCR_S()
```

runner actions

Use the runner actions to do miscellaneous utility functions.

The runner actions can check batch integrity, manipulate the values of fields and variables, raise condition flags, and control rule execution.

- "AbortOnError" on page 882
- "CheckAllIntegrity" on page 882
- "CheckDocCount" on page 883
- "CheckPageCount" on page 883
- "DebugMode_OFF" on page 884
- "DebugMode_ON" on page 884
- "GoToNextFunction" on page 885
- "PilotMessage_Clear" on page 885
- "PilotMessage_Set" on page 886
- "ProcessChildren" on page 886
- "rr_AbortBatch" on page 887
- "rr_Get" on page 887
- "rr_WriteNode" on page 888
- "rrAppend" on page 889
- "rrCompare" on page 889
- "rrCompareCase" on page 890
- "rrCompareCaseLength" on page 891
- "rrCompareNot" on page 892
- "rrCompareNotCase" on page 893
- "rrCompareNotCaseLength" on page 894
- "rrCopy" on page 895
- "rrPrepend" on page 896
- "rrSet" on page 897

"SetBatchPriority" on page 898
"SetOperatorID" on page 898
"SetReturnValue" on page 899
"SetStationID" on page 899
"SetTaskStatus" on page 900
"SkipChildren" on page 900
"Status_Preserve_OFF" on page 901
"Status_Preserve_ON" on page 901
"Task_NumberOfSplits" on page 902
"Task_RaiseCondition" on page 903

AbortOnError

Determines whether a task that encounters an error stops or continues.

Syntax

()

Parameters

True: Abort the batch if an error occurs.

False: Do not abort the batch if an error occurs.

Returns

False if the parameter is not **True** or **False**. Otherwise, **True**.

Level

All.

Details

Determines if tasks that encounter errors are to abort, or continue processing.

Example

```
AbortOnError("Yes")
```

CheckAllIntegrity

Checks all documents in the batch to determine whether they meet the document integrity requirements that are specified in the document hierarchy (setup DCO).

Syntax

()

Parameters

None.

Returns

True if the Document Integrity of the current batch meets the requirements as defined in the setup of the Document Hierarchy. Otherwise, **False**.

Level

Batch level.

Details

Checks that the documents in the batch contain the correct type and number of pages, in line with the Document Integrity requirements of the Document Hierarchy.

Example

```
CreateDocuments()  
CheckAllIntegrity()
```

These actions are part of a rule applied to the Batch object of the Document Hierarchy. The first assembles documents from the pages in the batch; the second ensures that the makeup of each document is valid.

CheckDocCount

Determines whether the number of documents in the runtime hierarchy matches the expected document count as specified by the scan operator.

Syntax

()

Parameters

None.

Returns

True if the actual count is the same as the expected count. Otherwise, **False**.

Level

Batch level.

Details

The number of expected documents is usually provided by the operator of a job's Scan task. This very handy action can compare the actual amount to the estimate at any time after a CreateDocuments action has assembled the documents in the batch.

Example

```
CheckDocCount()
```

CheckPageCount

Determines whether the number of pages in the runtime hierarchy matches the expected page count as specified by the scan operator.

Syntax

()

Parameters

None.

Returns

True if the two counts are equal. Otherwise, **False**.

Level

Batch level.

Details

This action confirms that the number of actual images (pages) in the current task's Page file (.xml) matches the count of expected pages.

Example

```
CheckPageCount()
```

DebugMode_OFF

Disables enhanced logging.

Syntax

```
()
```

Parameters

None.

Returns

Always **True**.

Level

All.

Details

This actions turns off the enhanced logging procedures turned on by an earlier DebugMode_On action.

Enhanced logging expands the scope and depth of a processing log's information, and of the logs that a Rulerunner task generates when you are testing a rule and its actions.

This feature also increases a Log file's size significantly, and should only be used when you are testing the impact of an action and rule on the application's workflow.

Example

```
DebugMode_Off()
```

DebugMode_ON

Enables enhanced logging (disabled by default).

Syntax

```
()
```

Parameters

None.

Returns

Always **True**.

Level

All.

Details

The following example shows enhanced logging during several actions.

Example

```
DebugMode_On()  
ExportOpenConnection(@APPVAR(values/dsn/exportdb:cs))  
SetTableName(Invoice)  
ExportFieldToColumn(Number, db_Number)  
AddRecord()  
DebugMode_Off()
```

GoToNextFunction

Returns False, which causes the next function in the ruleset to run.

Syntax

()

Parameters

None.

Returns

Always **False**.

Level

All.

Details

Returns a **False** condition so that the next function in the RuleSet can run.

Example

```
IsFieldMatching("Skip")  
GoToNextFunction()
```

If the condition in the first action is met, the sequence assigns a **False** status to the second action and to the rule of which it is a part. As a result, execution continues with the next function in the Rule.

PilotMessage_Clear

Removes the MESSAGE variable from the current object.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

All.

Details

Removes the runtime *MESSAGE* variable from the bound object of the Document Hierarchy.

Example

```
PilotMessage_Clear()
```

PilotMessage_Set

Assigns a message to the *MESSAGE* variable of the current object.

Syntax

()

Parameters

The smart parameter message to be assigned to the *MESSAGE* variable. Be sure to surround the message in quotation marks.

Returns

Always **True**.

Level

All.

Details

Provides a runtime *MESSAGE* variable to the bound object of the Document Hierarchy, and assigns the Action's parameter as the variable value.

Example

```
PilotMessage_Set("Field +0F+ Value is not Valid")
```

ProcessChildren

Initiates the processing of elements that are represented by the bound object and its children.

Syntax

()

Parameters

A two-part, comma-separated specification of a Condition and a Command.

The Condition is any valid VBScript expression. The Command is the VB executable that results from the Condition.

Returns

False if the number or sequence of the arguments are invalid. Otherwise, **True**.

Level

All.

Details

A follow-up action that initiates the processing of elements represented by the bound object, and all its children.

Example

```
ProcessChildren("1,Exit")
```

rr_AbortBatch

Stops processing the current batch and sets its status to Abort.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

All.

Details

Stops processing the current batch and sets the status of the batch to Abort.

Example

```
rr_AbortBatch()
```

rr_Get

Assigns the value of the specified variable to the Text property of the current object.

Syntax

()

Parameters

A smart parameter referencing a value or which is a reference to a value that will be copied to the calling object.

Returns

False the parameter is missing. Otherwise, **True**.

Level

All.

Details

Uses the parameter's elements to locate the value of a source object's variable, and assign it to the calling object. If the calling object is a field, only the value of the field will be changed.

Example

```
rr_Get("@B.OPERATOR")
```

This example retrieves the value of the Batch object's Operator property and assigns it to the calling object's Text property, if the calling object is a field.

```
rr_Get("@DICT_WORD(..\MONTH)")
```

This example shows how Smart Parameters translates the OMR recognized value of the MONTH field to the text from a predefined dictionary. The text is then assigned to the calling object's *Text* property, if it is a field, or *Text* variable if it is not a field.

rr_WriteNode

Creates a separate XML data file for the current object.

Syntax

```
()
```

Parameters

None.

Returns

Always **True**.

Level

All.

Details

Sets up a separate XML data file element for the calling object during Rulerunner processing.

Example

```
rr_WriteNode()
```

rrAppend

Appends the value of the source object to the specified field.

Syntax

()

Parameters

Two Smart Parameters:

1. The source value.
2. A reference to the target field.

Both parameters are optional. If a parameter is not specified, it will default to the calling object. If the calling object is a field, it will use the field value.

Returns

False if the action cannot locate the target object or if the source value is empty. Otherwise, **True**.

Level

All

Details

The action retrieves the value of the source object, and appends it to the target value.

Example

```
rrAppend("@D.DocID", "@F")
```

This action inserts the current calling object's parent DocID variable value and appends it to the calling field's value.

Note: Target can not be a variable. If the source and target are the same, the action has no effect.

rrCompare

Compares the values of two variables and returns True if they are the same.

Syntax

()

Parameters

Two Smart Parameters.

1. A value or a smart parameter, which is a reference to a value.
2. A value or a smart parameter, which is a reference to a value for comparison.

Note: Either reference can specify a variable of the calling object (the bound object of the Document Hierarchy.) Alternatively, both references can identify a variable of an object that is a parent or child of the calling object.

Either parameter is optional. If a parameter is not specified, it will default to the calling object. If the calling object is a field, it will use the field value. For batch, document and page objects, it will use a variable called *Text*, creating the variable if it does not exist.

Returns

False if the compared values do not match. Otherwise, **True**.

Level

All.

Details

Uses the Smart Parameters that you enter as the parameter to locate and compare the values of two object's variables.

Example

```
rrCompare("Expected_Pages", "@B.Tot_Pages")
```

This example shows how a value is solicited from the field *Expected_Pages* off of the calling object and the Batch object. The two values are then compared: the action returns **False** if the values are not the same.

rrCompareCase

Runs a comparison of two strings or smart parameters to see whether they are identical.

Syntax

```
bool rrCompareCase(string object1, string object2, string caseSensitive)
```

Parameters

string **object1**

string **object2**

string **caseSensitive**

Parameters

Three Parameters.

1. A value or a smart parameter, which is a reference to a value.
2. A value or a smart parameter, which is a reference to a value for comparison.

Note: Either reference can specify a variable of the calling object (the bound object of the Document Hierarchy.) Alternatively, both references can identify a variable of an object that is a parent or child of the calling object.

Either parameter is optional. If a parameter is not specified, it defaults to the calling object. If the calling object is a field, it uses the field value. For batch, document and page objects, it uses a variable that is called *Text*, creating the variable if it does not exist.

3. True runs a case-sensitive compare. False runs a case insensitive compare. If not specified, the default is False.

Returns

False If the compared values do not match. Otherwise, **True**.

Level

All.

Details

Runs a comparison of two strings or smart parameters to see whether they are identical. The comparison can be run as case sensitive or case insensitive.

Example

```
rrCompareCase("Main_Job","JOBID","False")
```

This example compares the string "Main_Job" to the current Job ID. The comparison is case insensitive. If the current Job ID is "MAIN_JOB", the action returns **True**.

```
rrCompareCase("Main_Job","JOBID","True")
```

This example compares the string "Main_Job" to the current Job ID. The comparison is case-sensitive. If the current Job ID is "MAIN_JOB", the action returns **False**.

rrCompareCaseLength

Uses the smart parameters that you enter as the parameter to locate and compare the values of two object's variables.

Syntax

```
bool rrCompareCaseLength(string object1, string object2,  
string caseSensitive, string length, string fromStart)
```

Parameters

string **object1**

string **object2**

string **caseSensitive**

string **length**

string **fromStart**

Parameters

Five Parameters.

1. A value or a smart parameter, which is a reference to a value.
2. A value or a smart parameter, which is a reference to a value for comparison.
3. True runs a case-sensitive compare. False runs a case insensitive compare.
4. An integer for the number of characters to compare. If the length is 0, the entire string is compared.
5. True compares from the start of the string. False compares from the end of the string.

Returns

False If the compared values do not match. Otherwise, **True**.

Level

All.

Details

Uses the Smart Parameters that you enter as the parameter to locate and compare the values of two object's variables. The comparison can be limited to a specified number of characters from the start or the end of the string. The comparison can be run case-sensitive or case insensitive.

Example

```
rrCompareCaseLength("Main", "@JOBID", "False", 4, "True")
```

This example compares the string "Main" to the current Job ID. Only the first four letters are compared and the compare is case-sensitive. If the current Job ID is "MAIN_JOB", the action returns **True**.

```
rrCompareCaseLength("Main", "@JOBID", "True", 4, "True")
```

This example compares the string "Main" to the current Job ID. Only the first four letters are compared and the compare is case-sensitive. If the current Job ID is "MAIN_JOB", the action returns **False**.

```
rrCompareCaseLength("Main Line", "Main Job", "True", 4, "True")
```

This example compares the string "Main Line" to the string "Main Job". The compare is case-sensitive and only the first four letters are compared. The action returns **True**.

```
rrCompareCaseLength("@P.ScanSrcPath", "GOOD.BMP", "False", 8, "False")
```

This example runs a case insensitive compare of the last 8 characters of the *ScanSrcPath* variable to find the last 8 characters of the string "GOOD.BMP". If the value of *ScanSrcPath* is "c:\test\testvalidate\images\good.bmp", the action returns **True**.

rrCompareNot

Compares the values of two variables and returns False if they are the same.

Syntax

()

Parameters

Two Smart Parameters.

1. A value or a smart parameter, which is a reference to a value.
2. A value or a smart parameter, which is a reference to a value for comparison.

Note: Either reference can specify a variable of the calling object. Alternatively, both references can identify a variable of an object that is a parent or child of the calling object.

Either parameter is optional. If a parameter is not specified, it will default to the calling object. If the calling object is a field, it will use the field value.

For batch, document and page objects, it will use a variable called *Text*, creating the variable if it does not exist.

Returns

True if the compared values do not match. Otherwise, **False**.

Level

All.

Details

This action is the negation of `rrCompare`. It can be handy for when an action should be performed only when two values are different.

Example

```
rrCompareNot("Expected_Pages", "@B.Tot_Pages")
rr_AbortBatch()
```

This example shows how a value is solicited from the field `Expected_Pages` off of the calling object and the `Batch` object. The two values are then compared: the action returns **True** if the values are not the same. Here, the batch will abort if the expected pages do not match the total pages.

rrCompareNotCase

Negates the running of the `rrCompareCase` action. You can run this action in instances when two of the string or smart parameter values are different.

Syntax

```
rrCompareNotCase(string object1, string object2, string caseSensitive)
```

Parameters

string **object1**

string **object2**

string **caseSensitive**

Parameters

Three Parameters.

1. A value or a smart parameter, which is a reference to a value.
2. A value or a smart parameter, which is a reference to a value for comparison.

Note: Either reference can specify a variable of the calling object (the bound object of the Document Hierarchy.) Alternatively, both references can identify a variable of an object that is a parent or child of the calling object.

Either parameter is optional. If a parameter is not specified, it defaults to the calling object. If the calling object is a field, it uses the field value. For batch, document and page objects, it uses a variable that is called *Text*, creating the variable if it does not exist.

3. **True** runs a case-sensitive compare. **False** runs a case insensitive compare. If not specified, the default is **False**.

Returns

True if the compared values do not match. Otherwise, **False**.

Level

All.

Details

This action negates the running of the `rrCompareCase` action. You can run this action in instances when two of the string or smart parameter values are different.

Example

```
rrCompareNotCase("Main_Job","JOBID","False")
```

This example compares the string "Main_Job" to the current Job ID. The comparison is case insensitive. If the current Job ID is "MAIN_JOB", the strings match so the action returns **False**.

```
rrCompareNotCase("Main_Job","JOBID","True")
```

This example compares the string "Main_Job" to the current Job ID. The comparison is case-sensitive. If the current Job ID is "MAIN_JOB", the strings do not match so the action returns **True**.

rrCompareNotCaseLength

Negates the running of the `rrCompareCaseLength` action. You can run this action in instances when two of the values are different.

Syntax

```
bool rrCompareNotCaseLength(string object1, string object2,  
string caseSensitive, string length, string fromStart)
```

Parameters

string **object1**

string **object2**

string **caseSensitive**

string **length**

string **fromStart**

Parameters

Five Parameters.

1. A value or a smart parameter, which is a reference to a value.

2. A value or a smart parameter, which is a reference to a value for comparison.
3. True runs a case-sensitive compare. False runs a case insensitive compare.
4. An integer for the number of characters to compare. If the length is 0, the entire string is compared.
5. True compares from the start of the string. False compares from the end of the string.

Returns

True, if the compared values do not match. Otherwise, **False**.

Level

All.

Details

Uses the Smart Parameters that you enter as the parameter to locate and compare the values of two object's variables. The comparison can be limited to a specified number of characters from the start or the end of the string. The comparison can be run case-sensitive or case insensitive.

Example

```
rrCompareNotCaseLength("Main", "@JOBID", "False", 4, "True")
```

This example compares the string "Main" to the current Job ID. Only the first four letters are compared and the compare is case-sensitive. If the current Job ID is "MAIN_JOB", the action returns **False**.

```
rrCompareNotCaseLength("Main", "@JOBID", "True", 4, "True")
```

This example compares the string "Main" to the current Job ID. Only the first four letters are compared and the compare is case-sensitive. If the current Job ID is "MAIN_JOB", the comparison does not match due to case differences so the action returns **True**.

```
rrCompareNotCaseLength("Main Line", "Main Job", "True", 4, "True")
```

This example compares the string "Main Line" to the string "Main Job". The compare is case-sensitive and only the first four letters are compared. The comparison matches so the action returns **False**.

```
rrCompareNotCaseLength("@P.ScanSrcPath", "GOOD.BMP", "False", 8, "False")
```

This example runs a case insensitive compare of the last 8 characters of the *ScanSrcPath* variable to find the last 8 characters of the string "GOOD.BMP". If the value of *ScanSrcPath* is "c:\test\testvalidate\images\good.bmp", the comparison matches so the action returns **False**.

rrCopy

Copies the value, confidence levels, and positions from one field to another.

Syntax

```
()
```

Parameters

Two Smart Parameters

1. A reference to the source field
2. A reference to the target field

Either parameter is optional. If a parameter is not specified, the calling object must be a field.

Returns

False, if the action cannot retrieve the target or source object. Otherwise, **True**.

Level

Field level.

Details

The action retrieves the value, confidence, and image references (field positions) of the source field object, and copies them to the target field object. It uses the Smart Parameters that you enter as a parameter to copy the value of a source field object to a target field object. This action is unusual in that it is intended to work only on field objects.

Note: rrCopy copies more than just the value of the field. Use rrSet if only the field value is to be copied. This action is unusual because it is intended to work only on field objects.

Example

```
rrCopy("@B\OPERATOR", "@P\OPERATOR")
```

This example copies the Operator value of the Batch field to the Operator field of the bound object of the Document Hierarchy.

rrPrepend

Inserts a value at the beginning of the specified field.

Syntax

()

Parameters

Two Smart Parameters:

1. The source value.
2. A reference to the target object.

Either parameter is optional. If a parameter is not specified, it will default to the calling object. If the calling object is a field, it will use the field value.

Returns

False if the calling object and target object are the same, if the action cannot locate the target object's variable, or if the source value argument or object returns an empty string. Otherwise, **True**.

Level

All, target must be a field object.

Details

The action retrieves the value of the source object, and pre-appends it to the target field value.

Example

```
rrPrepend("@D.DocID","@F")
```

This action inserts the current calling object's parent DocID variable value and pre-appends it to the calling field's value.

Note: Target can not be a variable.

rrSet

Assigns a value to a variable or field.

Syntax

()

Parameters

Two parameters. Smart parameters are supported:

1. A smart parameter referencing a value or is a reference to a value that will be copied.
2. A smart parameter referencing a target which is receiving the value.

Either parameter is optional. If a parameter is not specified, it will default to the calling object. If the calling object is a field, it will use the field value. For batch, document and page objects, it will use a variable called *Text*, creating the variable if it does not exist.

Returns

False if the action cannot locate the target object. Otherwise, **True**.

Level

All.

Details

Uses the parameter's elements to locate the value of a source object's variable, and assign it to a specific variable of a second, receiving object. The action rrSet will set the target with the value from the source. If using a field, only the value of the field will be changed. You can use rrCopy if you wish to copy a field's value, confidence and image references (field positions) of the source field object.

Example

```
rrSet("@F.MySourceVar","@P.MyTargetVar")
```

Obtains the value from the calling field *MySourceVar* variable and assigns it to parent page of the calling object *MySourceVar* variable.


```
rrSet("@D.Tot_Pages","@B.Tot_Pages")
```

This example assumes that the calling object is a child of a Document object. It locates the value in the calling document's *Tot_Pages* variable and assigns it to the *Tot_Pages* variable of the Batch object.

```
rrSet("@DICT_VALUE(..\MONTH)","")
```

This example shows how Smart Parameters translates the OMR recognized value of the MONTH field to the text from a predefined dictionary. The text is then assigned to the calling object's Text property, if it is a field, or *Text* variable if it is not a field.

SetBatchPriority

Sets the priority of the batch at the completion of the task.

Syntax

```
()
```

Parameters

A single value to update the batch priority at the end of the Task.

Returns

False if the value of the argument is invalid. Otherwise, **True**.

Level

All.

Details

Values are typically 1-9 with 5 being the median. Batches with priority 1 are processed first, batches with priority 9 are processed last.

Example

```
SetBatchPriority("1")
```

SetOperatorID

Sets the ID of the person who is operating Rulerunner.

Syntax

```
()
```

Parameters

A Single value representing the new Operator ID value.

Returns

False if setting the value throws an error. Otherwise, **True**.

Level

All.

Details

Sets the Operator ID at the completion of the Task.

Example

```
SetOperatorID("admin")
```

SetReturnValue

Returns True or False depending on the parameter that is specified.

Syntax

()

Parameters

True: The action will return true.

False: The action will return false.

Returns

True if the action is passed the parameter true. Otherwise, **False**.

Level

All.

Details

This action will return true or false based on the input parameter.

By passing in true, the action will return true and continue with the actions in the current function. If this is the last action in a function, any following functions within the rule are skipped.

One use for this action is a quick way to disable a rule by adding a new function, that precedes all other functions in the rule, where the new function contains only this action with a parameter of true. This will cause all other functions in the rule to be skipped and the next rule will run.

Using a parameter of false, this action will return false, causing all following actions in the function to be skipped and control carries forward to the next function in the same rule. In this way, the operation is identical to the action GoToNextFunction.

Example

```
SetReturnValue("true")
```

SetStationID

Sets the ID of the station where the person is operating Rulerunner.

Syntax

()

Parameters

A Single value representing the new Station ID value.

Returns

False if setting the value throws an error. Otherwise, **True**.

Level

All.

Details

Sets the Station ID at the completion of the Task.

Example

```
SetStationID("4")
```

SetTaskStatus

Specifies the task status that is returned to an application as Abort, Canceled, Finished, Hold, or Pending when the current task completes.

Syntax

()

Parameters

Numeric value representing the status that the task is to return to User Application. The statuses include:

- 0 - Abort
- 1 - Cancelled
- 2 - Finished
- 4 - Hold
- 8 - Pending

Returns

False if the parameter is not Numeric. Otherwise, **True**.

Level

All.

Details

Sets the Task Status value that is to be returned to User Application when the current task finishes processing.

Example

```
SetTaskStatus(4)
```

SkipChildren

Prevents the running of rules on child objects of the current object.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

All.

Details

Prevents rules applied to child objects of the current parent object from being run. The action can optimize the execution of rules by eliminating the need to visit every field on every page.

Example

```
SkipChildren()
```

Status_Preserve_OFF

Allows rules to change the STATUS value of fields, for example, to assign a problem status.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

All.

Details

This action turns the Status Preserve condition of a page and its fields from On to Off.

An object's Off condition allows the actions of a Validate ruleset to assign a problem status to any Field object with an invalid captured value. The Verify task's Data Entry panel will then surround the value with a pink background, alerting the operator to the problem.

Example

```
Status_Preserve_Off()
```

Status_Preserve_ON

Prevents rules from changing the STATUS value of fields.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

All.

Details

This action changes the Status Preserve condition of a Page object and its Field objects from Off to On.

The On condition prevents a rule and its actions from assigning a "problem" status to a field, even if the field's value fails validation.

Example

```
Status_Preserve_On()
```

Task_NumberOfSplits

Specifies the number of jobs the batch is sent to when a condition is raised before it returns to the main workflow.

Syntax

()

Parameters

Integer value of the number of splits. In most cases, you will want to use "1" as the parameter.

Returns

False if the parameter you enter is not Numeric. Otherwise, **True**.

Level

All.

Details

Specifies how many times sub-batches have been created from the current batch.

Important: The action communicates but does not use the Number_of_Splits value you enter as a parameter.

Example

```
Task_NumberOfSplits(1)  
Task_RaiseCondition(0,0)
```

In this example, the User Application is alerted to create one sub-batch entry, and to raise the second child job condition for this sub-batch entry.

Task_RaiseCondition

Specifies the group index and the index of the condition to raise from the list on the Datacap Web Client Workflow tab. 0 is the first condition.

Syntax

()

Parameters

Two comma-separated Integer values:

1. The applicable value of the sub-batch index. 0 is the first sub-batch, 1 is the second, etc. The Task_NumberOfSplits action determines how many sub-batches are created.
2. The value that designates the Child Job Condition that should be assigned to the specified sub-batch. 0 is the first Child Job Condition, 1 is the second, etc.

Returns

False if either parameter is not Numeric. Otherwise, **True**.

Level

All.

Details

Assigns the correct Child Job Condition to the correct sub-batch entry created by the Task_NumberOfSplits action.

Example

```
Task_NumberOfSplits(1)
Task_RaiseCondition(0,0)
```

In this example, the User Application is alerted to create one sub-batch entry, and to raise the second child job condition for this sub-batch entry.

SPEXport actions

Use the SPEXport actions to upload documents to a Microsoft SharePoint library.

The SPEXport actions integrate Datacap applications with the SharePoint library. You run these actions to access the SharePoint server, set up document attributes and folders on the server, and upload documents to the server for storage.

“SP_CreateFolder” on page 904

“SP_Login” on page 904

“SP_SetContentType” on page 905

“SP_SetFileType” on page 905

“SP_SetProperty” on page 906

“SP_SetUploadMode” on page 907

“SP_SetUrl” on page 907

“SP_Upload” on page 908

“SP_UploadDir” on page 909

SP_CreateFolder

Creates the folder in the SharePoint where you import your documents.

Syntax

```
bool SP_CreateFolder(StrParam)
```

Parameters

The URL that specifies the folder to create in SharePoint. Smart parameters are supported. Refer to the Smart Parameter documentation for more information.

Returns

True if the folder was created successfully or if it already exists. Otherwise, **False**.

Level

All.

Details

Creates the SharePoint folder specified in the parameter string.

Note: The SP_SetUrl action optionally can define directories and subdirectories to be created during the upload.

Example

```
SP_CreateFolder("http://blue/Docs/Documents/Test")
```

SP_Login

Creates the connection to SharePoint library by using the user ID, password, optional SharePoint domain.

Syntax

```
bool SP_Login(StrParam)
```

Parameters

A string containing 3 comma separated input parameters.

1. SharePoint userID.
2. Password.
3. An optional SharePoint domain. If not included, do not include the preceding comma.

Smart Parameters are supported. Use smart parameters to prevent clear text passwords in your application by obtaining the password from the application service.

Returns

True if the login succeeded. Otherwise, **False**.

Note: If the login parameters are invalid, a failure may not occur until you call SP_Upload.

Level

All.

Details

Login to SharePoint with credentials other than the logged-in Windows User.

Example

```
SP_SetUrl("http://blue/Docs/Documents/+"@BatchID+"/"+@ID")
SP_Login("userID,password,domain")
SP_SetContentType("Invoice")
SP_SetFileType("jpg")
SP_SetProperty("Date,@Value")
SP_Upload()
```

Alternatively, you can use smart parameters to obtain information from the application service to prevent clear text passwords. Here is an example where the password is stored in a custom value called SPPassword in the application service:

```
SP_Login("userID,@APPVAR(values/adv/SPPassword),domain")
```

SP_SetContentType

Sets the type of content in the SharePoint library for the uploaded documents, such as an Invoice.

Syntax

```
bool SP_SetContentType(StrParam)
```

Parameters

A valid SharePoint content type in the selected Library. No error is raised if it is not a valid content type. Smart Parameters are supported.

Returns

True if the content type was successfully set or if the content type is not a valid content type. **False** if there is failure returned from SharePoint.

Level

All.

Details

This action sets the SharePoint Content Type for each document that is subsequently uploaded. Content Type is a SharePoint concept that defines a subset of columns (fields) within a library of documents, to be displayed and edited for a specific purpose.

Example

```
SP_SetContentType("Invoice")
```

SP_SetFileType

Sets the format in which to upload the document to the SharePoint library, for example TIF or PDF.

Syntax

```
bool SP_SetFileType(StrParam)
```

Parameters

A string indicating the type or filename extension of the images to be uploaded for each document or batch. When uploading the Batch or Document this extension is appended to the BatchID or DocumentID to select the image. The IMAGEFILE property takes precedence for Page uploads. See the description of SP_Upload for details. Valid parameters include: tif, tiff, jpg, jpeg, jpe, gif or pdf. The parameter may optionally include a period (for example .tif and .jpeg are also valid).

Returns

False if the parameter is not a three-character extension, jpeg, or tiff, with or without a leading period. Otherwise **True**.

Note: If a three-character extension is supplied that is invalid for SharePoint images, the upload may fail.

Level:

All.

Details

Note: SP_SetUploadMode takes precedence over SP_SetFileType, if SP_SetUploadMode is called prior to SP_Upload this parameter has no effect. If neither SP_SetFileType nor SP_SetUploadMode are called, tif is used as the default file type.

Example

```
SP_SetFileType("jpg")
```

SP_SetProperty

Sets the column property in SharePoint for the documents you want to upload.

Syntax

```
bool SP_SetProperty(StrParam)
```

Parameters

Two comma separated values:

1. Column name is the name or ID of the target column in SharePoint.
2. Data value is the value to be uploaded to that column. Refer to the documentation for more information about the column types.

Smart Parameters are supported.

Returns

True if the parameters are not blank. The index information is uploaded to SharePoint when a document is subsequently uploaded. Otherwise, **False**.

Level

All.

Details

Sets an index value (column in SharePoint) for the documents to follow. Can be called multiple times to set multiple index values.

Notes:

- Any spaces in column names must be replaced with “_0x02c_”.
- The real column name may be different from what is displayed in SharePoint. To determine the real Column name select the column settings and check the browser address.
- For example for the property called **Description** you may see “....3F2%7DField=Comments” at the end. This means that the real name of the Column to be used in the SP_SetProperty action is “Comments”.

Example

```
SP_SetProperty("Date,@Value")  
SP_Upload()
```

Moves the value of the current field to the SharePoint column named Date.

SP_SetUploadMode

Identifies the files to upload into the SharePoint library.

Syntax

```
bool SP_SetUploadMode(StrParam)
```

Parameters

A string or Smart Parameter identifying the page level variable where file name stored. If this action is not called the value defaults to blank and regular upload logic applied. For example SP_SetUploadMode("ParentImage") will cause uploading file with the name stored in *ParentImage* variable on the page level.

Returns

Always **True**.

Level

Batch, Document or Page level.

Details

Use this action to identify the files that will be uploaded to SharePoint.

Example

```
SP_SetUploadMode(ParentImage)  
SP_Upload()
```

SP_SetUrl

Sets the URL address of the SharePoint library.

Syntax

```
bool SP_SetUrl(StrParam)
```

Parameters

The full URL to the SharePoint repository. Smart parameters are supported. Refer to the Smart Parameter documentation for more information.

Returns

True if the action succeeded. Otherwise, **False**.

Level

All.

Details

Sets target URL of location to which image files are uploaded.

Note: /Docs/Documents/ is the default Document Library within SharePoint site.

Example:

```
SP_SetUrl("http://blue/Docs/Documents/+"@BatchID+"/"+@ID")
```

With this example, directories with names defined by /+"@BatchID+"/"+@ID are created automatically during upload.

SP_Upload

Uploads the image file and any indexes that are specified for the batch, document, or page into SharePoint.

Syntax

```
bool SP_Upload()
```

Parameters

None.

Returns

True if all documents and indexes were successfully uploaded. Otherwise, **False**.

Level

All.

Details

Uploads the image file and any indexes specified for the current page, document, or batch to SharePoint. Uses TiffMerge file naming scheme to find document level or batch level image file: DocID.TIF or BatchID.TIF by default. Pages associated with other image file types (e.g. TM000001.pdf, TM000001.jpg, etc) can be uploaded.

Note: After uploading, the variable *Upload_Folder* in the page/doc/batch will be set to the SharePoint URL where the document(s) were uploaded.

Note: If some documents in a batch are successfully uploaded and some fail, and the batch is rerun through the SharePoint Upload task, only documents that failed to upload will be re-uploaded.

Note: If any document is re-uploaded, SharePoint will replace the existing document with the newer one, or save the old version and replace it with the new version, depending on SharePoint Versioning settings.

Example

```
SP_Upload()
```

SP_UploadDir

Uploads the file into the specified folder.

Syntax

```
bool SP_UploadDir(StrParam)
```

Parameters

Two comma separated parameters:

1. The Windows folder containing only document files to upload.
2. A Boolean. **True** means delete file after upload, false means move file to the Uploaded folder in current directory.

Returns

True if the upload succeeds for all files in the directory. Otherwise, **False**.

Level

Batch or Document level.

Details

Uploads all files in specified folder.

Example

```
SP_UploadDir("C:\ParentDir\Invoice\Images\Input\",false")
```

Split actions

Use the Split action to split a batch into smaller batches so each can be processed separately.

The Split action splits batches based on the value of the specified document-level variable.

“SplitBatch”

SplitBatch

Splits a batch into smaller batches that are based on the value of the specified document-level variable.

Syntax

```
(StrParam)
```

Parameters

A smart parameter pointing to a Document or Page variable that determines if the Document or Page is to be split to a Child Batch.

Important: The action evaluates all documents and pages (including unbound* pages) in the batch.

The values of the smart parameter variable found during the document and page evaluation are grouped into buckets:

1. Pages/documents that contain the variable and the variable values are identical go into the same bucket.
2. If there are multiple buckets, all pages/documents that share the same value will split to the same child batch.
3. There can be only one child batch for each unique bucket value.

Child batches have the same name as the parent batch, but include an additional two character alpha-decimal suffix such as .01, .02, .0A, ..., up to .ZZ. This hexadecimal numbering for child batches is required by Datacap Server. Datacap Server creates the batch and queue entries for the child batches after the task is finished, when the split condition is processed. There is a maximum of 1295 child batches.

Example: **@D.Inbox**. If there is an Inbox variable in each document, this will split documents by the value of the Inbox variable.

Important: Any document or unbound* page that does not have this Inbox variable value, will remain in the parent batch.

*An unbound page is any page not inside a document.

Returns

False if an error occurs like a file could not be created, etc., and the batch will be set to abort. Otherwise **True**.

If the specified variable is not found in any documents or unbound pages, meaning there is nothing to split, the action is still considered to be successful and will return true.

Each child batch split off will generate a condition, which should be configured for Split in the workflow.

Any page or document with a blank value for the splitting value will remain in the original "parent" batch.

Level

Batch level only.

Details

This action will process all of the documents and unbound pages that are in the batch and attempt to split the identified documents and pages into child batches. The action will look in the documents or unbound pages for the variable specified

in the a parameter, group the objects that have a matching value, and split each group into a unique child batch. Only documents and unbound pages will be processed. Pages that are already placed within in a document structure will not be processed individually, the pages will be as part of their document, not as a separate page.

Additional considerations:

There is only one job routing condition raised by this action: it is the first one in the task's list of conditions.

The task's Task Setup/Task Settings screen must be configured as Job Router, and a single condition defined (by convention, call it Split).

1. Any and all child batches will be routed via this single condition.
2. If the application wants to treat the individual buckets differently, then the first step in the workflow after splitting can check the same smart parameter value and branch or re-route the child batch using that value.
3. All the structure and variables, etc. that were in the parent batch docs/pages are retained in the child batches.
4. In addition, new variables *ParentBatch* and *ParentBatchDir* are added.
5. The action can only be used once per Parent Batch.
6. The maximum number of child batches is 1295.
7. The page count and document count in child batches is not accurate after splitting. It is updated and will be accurate once the next task completes.

Example

```
SplitBatch(@D.Inbox)
```

TifMerge actions

Use the TifMerge actions to combine individual TIFF images into a multi-page TIFF file. This action is typically run at the end of the workflow so that you can upload or release the batch images as a single file.

The TifMerge actions determine the path to the Batch directory, creates the multi-page file, and lets you specify the compression to use in the final image.

"TifMerge_CheckStatus"

"TifMerge_ExportToBatchDir" on page 912

"TifMerge_MergeImages" on page 913

"TifMerge_MyImage" on page 914

"TifMerge_PreserveCompression" on page 915

"TifMerge_SetFileName" on page 915

"TifMerge_SetFilePath" on page 916

TifMerge_CheckStatus

Filters merged pages and documents that are based on their DCO status.

Syntax

```
(bool TifMerge_CheckStatus(string AcceptablePageStatuses,  
string DisregardPageStatuses, string AcceptableDocStatuses,  
string AcceptableDocStatuses)
```

Parameters

AcceptablePageStatuses

Type: string

DisregardPageStatuses

Type: string

AcceptableDocStatuses

Type: string

DisregardDocStatuses

Type: string

Parameters

1. **AcceptablePageStatuses:** a comma-separated list of the Page status values that are merged.
2. **DisregardPageStatuses:** a comma-separated list of the Page status values that are not merged.
3. **AcceptableDocStatuses:** a comma-separated list of the Document status values that are merged.
4. **DisregardDocStatuses:** a comma-separated list of the Document status values that are not merged.

Returns

Always **True**

Level

Any

Details

This action configures the acceptable statuses for documents and pages when you call the `TifMerge_MergeImages` action. If this action is not called, the status values of documents and pages is not checked.

In the following example, only the pages with an acceptable status of '0' or '49' are merged. The disregard page status of '75' is redundant because of its exclusion from the acceptable status values. By contrast, the disregard document status of '128' prevents all of the child pages of the document from being merged.

Example

```
TifMerge_CheckStatus("0,49","75","", "128")  
TifMerge_MergeImages("All")
```

TifMerge_ExportToBatchDir

Indicates that the path for the multi-Image file is to the current Batch directory.

Syntax

()

Parameters

None.

Returns

False if the path does not exist or is not accessible. Otherwise, **True**.

Level

Batch or Document usually, but Page or Field is permissible.

Details

When saving a multi-image file, this action is used to configure the current batch directory as the destination for the output file. This action must be called before the action to merge the images.

Example

```
TifMerge_ExportToBatchDir()  
TifMerge_MergeImages("All")
```

TifMerge_MergeImages

Merges the images associated with the object of the Document Hierarchy to which the action's ruleset applies into a single, multi-Image file.

Syntax

(sPageType)

Parameters

String value indicating either:

1. All if the multi-Image file is to contain images of all pages without regard to Page Type.
2. The Page Type(s) of the images to be included (comma-separated list, if the parameter includes more than one Page Type.)

Smart parameters are supported.

Returns

False if the action cannot create the multi-Image file. Otherwise, **True**.

Level

Batch or Document.

Details

This action merges the images associated with the object to which the action's rule applies into a single, multi-Image file.

At the Batch level, the action merges all Image files in the batch into one multi-Image file – or those Image files representing pages of the Page Type you specify as a parameter. At the Document level, the action assembles a new multi-Image file for each document in the batch. If you specify a Page Type, the multi-Image file for each document will include only images of pages of that type.

Actions TifMerge_SetFileName and TifMerge_SetFilePath must be called before TifMerge_MergeImages.

Example

```
TifMerge_SetFileName("@BATCHID+@DATE(dd.mm.yyyy)")
TifMerge_SetFilePath("C:\\ParentDir\\Invoice\\batches\\MultiImage")
TifMerge_MergeImages("All")

TifMerge_SetFileName("Doc_+@ID+@DATE(dd.mm.yyyy)")
TifMerge_SetFilePath("C:\\ParentDir\\Invoice\\MultiImage")
TifMerge_MergeImages("Invoice,Attachment")
```

The first example merges all images into a multi-Image file that uses the Batch ID and the processing Date for its name.

The second example applies to a Document object of the Document Hierarchy. It assembles a multi-Image file for each document in the batch; the images in a file are limited to Invoice and Attachment pages.

TifMerge_MyImage

Adds each single image to the multi-Image file.

Syntax

()

Parameters

None.

Returns

False if the action's ruleset is not applied to a Page object or if the corresponding image file for the current page cannot be found. Otherwise, **True**.

Level

Page level.

Details

Adds the current page image to the multi-Image TIF file. Actions that proceed the TifMerge_MyImage action allow you to specify under which conditions an image is to be added to the multi-Image file.

A rule with this action can only be applied to a Page object. The output image file name and destination path must have been previously set using TifMerge_SetFileName and TifMerge_SetFilePath.

Example

```
ChkDDCOStatus("0")
TifMerge_MyImage()
```

In this example, the ChkDDCOStatus action checks that the status of the current page is "0". If so, the image for the current page will be added to the multi-Image file.

If the current status of this page is not "0", the rule will fail and the TifMerge_MyImage action will not be run; therefore, the current image will not be added to the multi-Image file.

This ChkDDCOStatus action is used as an example. You can use many other actions to be sure the current image meets your merging criteria.

TifMerge_PreserveCompression

Determines the output compression type for merged images.

Syntax

()

Parameters

string PreserveCompression

Parameters

True: Preserves the original compression type of the source image.

False: Uses G4 compression for black and white images. JPEG is used for color images.

Returns

Always **True**.

Level

Any.

Details

This action will configure the output format for TifMerge_MergeImages. If this action is not called, then the default value of **False** is used, so the original image compression is not preserved.

Example

```
TifMerge_PreserveCompression("TRUE")
TifMerge_MergeImages("All")
```

TifMerge_SetFileName

Sets the name of the multi-Image file (.tif) to be created by TifMerge.

Syntax

(StrParam)

Parameters

String value of the file name to be assigned to the multi-page file. Smart parameters are supported.

Returns

Always **True**.

Level

All.

Details

This action sets the name of the multi-Image file (.tif) to be created by the TifMerge actions. The file name can be text, or a combination of text and the value of a variable you enter as a parameter. The action automatically adds the ".tif" extension to the file.

Example

The following example assumes that the rules with these actions are applied to a Document object. The names combine text values such as "Doc_" with values assigned to variables by using smart parameters.

```
TifMerge_SetFileName("Doc_+@ID+@DATE(dd.mm.yyyy)")
TifMerge_SetFilePath("c:\ParentDir\Invoice\MultiImage")
TifMerge_MergeImages("All")
```

This example combines "MultiTif_" with the ID of the Document Hierarchy object to which the ruleset is applied. Usually, a rule that contains this action applies to a Batch object or Document object, but can apply to a Page or Field object.

```
TifMerge_SetFileName("MultiTif_+@ID")
```

TifMerge_SetFilePath

Sets the path for the multi-Image file.

Syntax

(strParam)

Parameters

String value for the output path of the multi-Image TIF file. Smart parameters are supported.

Returns

False if the specified drive does not exist or the path cannot be created. Otherwise, **True**.

Level

All.

Details

Sets the path for where the multi-Image file will be created. If the folder designated in the parameter does not exist, the action will create the folder in which the TIF file will be placed.

Usually, a rule containing this action applies to a Batch object or Document object of the Document Hierarchy, but can apply to a Page or Field object.

Example

```
TifMerge_SetFileName("Doc_+@ID+@DATE(dd.mm.yyyy)")
TifMerge_SetFilePath("c:\ParentDir\Invoice\MultiImage")
TifMerge_MergeImages("All")
```

TM524 actions

The TM524 actions are for compatibility with older versions of Datacap and are no longer used

Validations actions

Use the Validations actions to check and modify the content and format of the current field value.

Other actions in the Validations library do arithmetic calculations, assign values, copy values, and check variables.

The Validations actions are described in the following table.

"AddLeadingZeros"	on page 919
"AddPaddingToEnd"	on page 919
"AddPaddingToLeft"	on page 920
"AddPaddingToRight"	on page 920
"AddPaddingToStart"	on page 921
"AddTrailingZeros"	on page 921
"AllowOnlyChars"	on page 922
"AppendFromField"	on page 922
"AppendToField"	on page 923
"AssignFieldDefault"	on page 923
"Calculate"	on page 924
"CalculateDateDifference"	on page 924
"CalculateFields"	on page 925
"CheckSubFields"	on page 926
"CompareFields"	on page 927
"ConvertFieldToCurrency"	on page 928
"ConvertToLowerCase"	on page 928
"ConvertToUpperCase"	on page 929
"CopyField"	on page 929
"CopyFieldToField"	on page 930
"DateStampField"	on page 930
"DeleteAllAlpha"	on page 931
"DeleteAllMiscChars"	on page 931
"DeleteAllNumeric"	on page 932
"DeleteAllPunct"	on page 932
"DeleteAllSysChars"	on page 933
"DeleteChildType"	on page 933
"DeleteLCSpaces"	on page 934
"DeleteParentObj"	on page 934
"DeleteSelectedChars"	on page 935
"EmptyFieldValue"	on page 935
"FailRuleSet"	on page 936
"FieldContainsValue"	on page 936
"FilterFieldSelectedChars"	on page 937

“FormatNumberToLocale” on page 937
“GetJobID” on page 938
“HasChildOfType” on page 939
“InsertChars” on page 939
“InsertDecimalPoint” on page 940
“IsFieldCurrency” on page 940
“IsFieldDate” on page 941
“IsFieldDateEqualOrAfter” on page 942
“IsFieldDateEqualOrBefore” on page 942
“IsFieldDateUpToToday” on page 942
“IsFieldDateWithinRange” on page 943
“IsFieldDateWithinXDays” on page 944
“IsFieldDateWithReformat” on page 944
“IsFieldEmpty” on page 945
“IsFieldFilled” on page 945
“IsFieldGreaterOrEqual” on page 946
“IsFieldHidden” on page 946
“IsFieldLengthMax” on page 947
“IsFieldLengthMin” on page 947
“IsFieldLessOrEqual” on page 948
“IsFieldMatching” on page 948
“IsFieldPercentAlpha” on page 949
“IsFieldPercentNonNumeric” on page 949
“IsFieldPercentNumeric” on page 950
“IsMatchingJobID” on page 951
“IsMaxOMRChecked” on page 951
“IsMinOMRChecked” on page 951
“IsPatternInField” on page 952
“IsSupportedImageFile” on page 953
“IsThisFieldEmpty” on page 953
“IsThisFieldFilled” on page 954
“IsVariableEmpty” on page 954
“IsVariableFilled” on page 955
“LeftTruncate” on page 955
“MessageBox” on page 955
“ParseMultilineAddress” on page 956
“ParseName” on page 957
“ReadCurrentObjVariable” on page 958
“ReadFieldValue” on page 958
“ReadPageVariableValue” on page 959
“ReplaceChars” on page 960
“ReplaceValueAtPosition” on page 960
“ResetField” on page 961
“RightTruncate” on page 961
“SaveAsCurrentObjVariable” on page 961

"SaveAsPageVariable" on page 962
"SetIsOverrideable" on page 962
"SplitFieldValueLeft" on page 963
"SplitFieldValuePreserveEnd" on page 963
"SplitFieldValuePreserveStart" on page 964
"SplitFieldValueRight" on page 964
"SumFields" on page 965
"TimeStampField" on page 965
"TrimSpaces" on page 966
"TruncateFromEnd" on page 966
"TruncateFromStart" on page 967

AddLeadingZeros

Inserts zeros at the beginning of a value so the character count equals the number that is specified.

Syntax

()

Parameters

A number *n* which is the maximum length of the value. Smart parameters are supported.

Returns

False if the parameter you enter is not numeric. Otherwise, **True**.

Level

Field level.

Details

Adds zeros ("0") to the beginning of the captured value of the current Field object until the total length of the value reaches the maximum *n* you specify as the parameter.

Example

```
AddLeadingZeros("10")  
2240.00 becomes 0002240.00
```

AddPaddingToEnd

Pads the captured value of the current Field object with spaces from after the last character in the string out to the number of specified characters.

Syntax

()

Parameters

A number *n* indicating the maximum permissible length of the value. If the action finds that a value's length is less than this number, it will insert spaces until the maximum length is reached. Smart parameters are supported.

Returns

Always **True**.

Level

Field level.

Details

Example

`AddPaddingToEnd("10")` uses spaces to expand a value with less than 10 characters. For example: 456.11 becomes 456.11_ _ _ _

AddPaddingToLeft

This action is deprecated and is scheduled to be removed in a future release. Use the `AddPaddingToStart` action.

Syntax

()

Parameters

A number n indicating the maximum permissible length of the value. If the action finds that a value's length is less than this number, it inserts spaces until the maximum length is reached.

Returns

Always **True**.

Level

Field level

Details

This action is replaced by the `AddPaddingToStart` action.

Example

`AddPaddingToLeft("12")` uses spaces to expand a value with less than 12 characters.

For example: RSJ-112 becomes RSJ-112_ _ _ _

AddPaddingToRight

This action is deprecated and is scheduled to be removed in a future release. Use the `AddPaddingToEnd` action.

Syntax

()

Parameters

A number n indicating the maximum permissible length of the value. If the action finds that a value's length is less than this number, it inserts spaces until the maximum length is reached.

Returns

Always **True**.

Level

Field level

Details

Pads the captured value of the current Field object with spaces from the right.

Example

`AddPaddingToRight("10")` uses spaces to expand a value with less than 10 characters.

For example: 456.111 becomes 456.111_ _ _ _

AddPaddingToStart

Pads the captured value of the current Field object with spaces from the start of the string up to the first character until the specified length is reached.

Syntax

()

Parameters

A number n indicating the maximum permissible length of the value. If the action finds that a value's length is less than this number, it inserts spaces until the maximum length is reached. Smart parameters are supported.

Returns

Always **True**.

Level

Field level.

Details

Example

`AddPaddingToStart("12")` uses spaces to expand a value with less than 12 characters.

For example: RSJ-112 becomes _ _ _ _ RSJ-112

AddTrailingZeros

Adds zeros to the end of captured value of the current Field until the length of the value reaches the maximum n you enter as the parameter.

Syntax

()

Parameters

A number n which is the maximum length of the value. Smart parameters are supported.

Returns

False if the parameter you enter is not numeric; otherwise, **True**.

Level

Field level.

Details

Example

```
AddTrailingZeros("10")  
2240.00 becomes 2240.00000
```

AllowOnlyChars

Removes all of the characters that are not specified as supported.

Syntax

()

Parameters

A Regular Expression that specifies permitted characters in the current word.

Returns

Always **True**.

Level

Field level.

Details

This action employs a Regular Expression as its parameter to identify and remove all of the characters that are not in the parameter from the value of the Field.

An empty argument removes all characters.

Example

```
AllowOnlyChars("ABCDEFG.")  
HELLO DOLLY. becomes ED.
```

AppendFromField

Appends the captured value of the specified Field object to the captured value of the current Field object.

Syntax

()

Parameters

The name of the Field object whose text value is to be appended to the value of the current field.

Returns

False if the parameter is not the name of a Field object. Otherwise, **True**.

Level

Page or Field Level.

Details

You can also apply the action at the Page level. A Text page-level variable with the appended value is added to the Data file of the page.

Example

```
AppendFromField("Number")
```

AppendToField

Appends the captured value of the current Field object to the captured value of the Field object that is specified by the parameter.

Syntax

()

Parameters

The name of the Field object to which the value is to be appended.

Returns

False if the parameter is not the name of a Field object. Otherwise, **True**.

Level

Field level.

Details

Example

```
AppendToField("FirstName")
```

If the current Field object is MiddleInitial, a rule with this action appends the value of the Middle Initial field to the FirstName Field object.

AssignFieldDefault

Assigns a default value to the current field.

Syntax

()

Parameters

The String value you're assigning to the field.

Returns

False if not called on the correct level. Otherwise, **True**.

Level

Field level.

Details

Example

```
AssignFieldDefault("Bill Paid")  
or  
AssignFieldDefault("PastDue!")
```

Calculate

This action is deprecated and is scheduled to be removed in a future release. Use the CalculateFields action.

Syntax

()

Details

This action is replaced by the CalculateFields action.

CalculateDateDifference

Calculate the differences between two dates and stores the calculation in a user defined variable.

Syntax

()

Parameters

- **startDate** : The starting date
- **endDate** : The ending date
- **targetVariable** : The variable to hold the calculated result value
- **dateProperty** : The value to calculate, 0 = days, 1 = months, 2 = quarters, 3 = years.

Smart parameters are supported.

Returns

False if the format of either date is invalid. Otherwise, **True**.

Level

Any level.

Details

Calculates the number of days, months, quarters or years between two dates. Only whole numbers are returned. Any fractional parts of the value are dropped. Quarters are calculated simply by dividing the number of months by 3. The order of the dates does not matter.

This action only supports Gregorian short dates as input and the date format must match the default format of the current locale.

Example

```
CalculateDateDifference("4/20/2012", "5/19/2012", "@P.Months", 1)
```

This example creates the page variable *Months* with a value of 0.

```
CalculateDateDifference("4/20/2012", "5/20/2012", "@P.Months", 1)
```

This example creates the page variable *Months* with a value of 1.

```
CalculateDateDifference("4/20/2012", "4/19/2013", "@P.Years", 3)
```

This example creates the page variable *Years* with a value of 0.

```
CalculateDateDifference("4/20/2012", "4/20/2013", "@P.Years", 3)
```

This example creates the page variable *Years* with a value of 1.

```
CalculateDateDifference(@P.MyDate1, @P.MyDate2, "@P.Days", 0)
```

This example creates the page variable *Days* with the number of days between the dates specified by the values in field *MyDate1* and field *MyDate2*.

```
CalculateDateDifference(@P.MyDate1, @P.MyDate2, "@P.Days", 0)
```

This example creates the page variable *Days* with the number of days between the dates specified by page variables *MyDate1* and *MyDate2*.

CalculateFields

Calculates the equation that is entered as a parameter and compares the result to the captured value of the current field object.

Syntax

()

Parameters

1. The equation that is the basis for the calculation. You can use the name of a Field object or numeric values with any arithmetic operator (+, -, *, /, ^). To use the name of a Field object, surround the field name with single quotation marks ('). A null is treated as a "0".
2. The number of decimal places to limit the logical comparison. (Optional)
3. A True/False (Default) operator to toggle Failure of all associated fields if the calculation fails. (Optional)

Returns

True if the expression is valid. **False** if the value of a field is not numeric or if the expression is not valid.

Level

Field level.

Details

If the result does not match the result of the equation, all fields involved in the equation receive a Failed status and appear pink in the applicable field of the Data Entry panel.

Example

```
CalculateFields("'SubTotal' + 'Shipping' + 'Tax' = 'Total'")
or
CalculateFields("('SubTotal' + 'Shipping' + 'Tax') - '0.05' = 'Total'")

for use with a 'tolerance' use two actions in sequence:
CalculateFields("('Wages' + 'Interest' + 'Unemployment')>=('Gross'-'0.05')")
CalculateFields("('Wages' + 'Interest' + 'Unemployment')<=('Gross'+'0.05')")
```

CheckSubFields

Determines whether the values of the specified child fields meet the specified criteria and deletes the parent field if they do not.

Syntax

()

Parameters

An expression that specifies which LINEITEM child fields are to be checked for the presence or absence of captured values. Within the expression, the name of each child Field object must be surrounded with single quotation marks ('). You can also use parentheses () in your expression. Action is placed on the parent of the LINEITEM field.

Returns

Always **True**.

Level

Field level.

Details

Validates an instance of a parent Field object by confirming the presence (or absence) of captured values for one or more of its child fields. (In the Invoices application, as an example, child fields of the LINEITEM parent include: ItemID, ItemDesc, Quantity, Price and LineTotal.)

Invalid parent Field objects are deleted if they do not meet this criteria.

This action usually runs in its own RuleSet (for example, in a Filter RuleSet) and would be applied to the DETAILS Field object (parent object to the LINEITEM field) in the Invoices example.

Example 1

```
Example 1: The captured values for a LINEITEM field are:
ItemID = 12345
ItemDesc =
Price = 12.00
LineTotal =
```

```
The action's parameter contains this expression:
CheckSubFields("('ItemID' OR 'ItemDesc') AND ('Price' OR 'LineTotal')")
```

In this example the action returns **True** and the current LINEITEM object is Valid.

Example 2

The captured values for the LINEITEM field are:
ItemID = 12345
ItemDesc = Thank you for your order
Price =
LineTotal =

The action's parameter contains this expression:
`CheckSubFields("('ItemID' OR 'ItemDesc') AND ('Price' OR 'LineTotal')")`

In this example the action's equation returns is **False** and the current LINEITEM object is Invalid. The field and the values of its child fields are deleted from the Data file.

Example 3

The captured values for the LINEITEM field are:
ItemID = 12345
LineTotal = gonetolunch

The action's parameter contains this expression:
`CheckSubFields("('ItemID') AND ('LineTotal')")`

In this example, the action returns **True** and validates the LINEITEM field, despite the presence of a nonsense entry in the LineTotal child field.

CompareFields

Compares the values of two fields by using the specified matching criteria that supports fuzzy matching.

Syntax

()

Parameters

Five comma-separated values:

1. String value of the source field's name. This is the field with a value to be compared.
2. String value of the target field's name. This is the field with a value to be compared to.
3. String value: Y or Yes; N or No. Alternatively, a Numeric value: 0 = No or 1=Yes. Yes (or Y or 1) allows the action to carry out a fuzzy rather precise comparison of the fields' values.
4. Numeric value of the percentage of required precision. Numbers less than 100 permit increasing fuzziness.
5. String value: Y or Yes; N or No. Alternatively, a Numeric value: 0 = No or 1=Yes. Yes (or Y or 1) directs the action to compare values in the fields word-by-word.

Returns

False if the designations of Field objects in the first two parameters are not valid, or if the Data Types of the values in the first two fields do not match. Otherwise, **True**.

Level

All.

Details

Locates values in two fields specified by the first two parameters. If values are present in both fields, the action compares the values according to the matching criteria of the last three parameters.

Example

```
CompareFields("Invoice_Date,Due_Date,Yes,100,Yes")
```

ConvertFieldToCurrency

Converts the value of the current field to a currency value.

Syntax

()

Parameters

None.

Returns

True if the text value is numeric and greater than one character. Otherwise, **False**.

Level

All levels, but generally applied at the Field level.

Details

Formats the text value of a field as a currency value. The following steps are performed on the field:

1. Removes existing currency symbols.
2. Replaces negative value characters such as parenthesis, 'NEG', 'CR' and trailing hyphen with a leading hyphen.
3. If a decimal exists, its position is not changed.
4. If a decimal does not exist and the field contains 2 or more characters, a decimal is inserted before the last two characters in the field.
5. If a decimal does not exist and the field contains less than 2 characters, no decimal is inserted.

Example

```
ConvertFieldToCurrency()  
A value of 1 remains 1  
A value of 12 becomes .12  
A value of 105 becomes 1.05  
A value of 104.009 remains 104.009
```

ConvertToLowerCase

Converts any upper case characters in the captured value of a Field object to lower case characters.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

Field level.

Details

Example

```
ConvertToLowerCase()
```

To ensure that the characters in all Product ID's are lower case, a Validate rule that applies to a Document Hierarchy's Item Field object would include this action.

ConvertToUpperCase

Converts the lower case characters in the captured value of a Field object to upper case characters.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

Field level.

Details

Example

```
ConvertToUpperCase()
```

A Validate rule with this action, if applied to a State Field object which accepts only abbreviations, would be sure the captured values contain Upper Case letters (AZ, AL, etc.)

CopyField

Copies the value of the current field to a specified field.

Syntax

()

Parameters

The name of the target Field object.

Returns

False if the parameter does not match the name of a Field object. Otherwise, **True**.

Level

Field level.

Details

Assigns the captured value of the current Field object to a sibling Field object you specify as the parameter.

Example

```
CopyField("Date")
```

This action places the captured value of the current field into the Date field.

CopyFieldToField

Copies the value of the current field to a specified field.

Syntax

()

Parameters

The name of the target Field object of the Document Hierarchy.

Returns

False, if the parameter does not match the name of a Field object. Otherwise, **True**.

Level

Field level.

Details

Copies the captured value of the current Field object to the Field object designated as the parameter of the action.

Example

If the current field's value is "1/1/05"

```
CopyFieldToField("Date")
```

assigns "1/1/05" (without the quotation marks) to the Document Hierarchy's Date field.

DateStampField

Updates the current Field object with today's date.

Syntax

()

Parameters

A Date format such as mm/dd/yy or dd/mm/yy. (* defaults to mm/dd/yyyy)

Smart parameters are supported.

Returns

Always **True**.

Level

Field level.

Details

Example

```
TimeStampField("*") produces  
01/20/2005
```

```
TimeStampField("dd/mm/yy") produces  
20/01/05
```

DeleteAllAlpha

Deletes all of the alphabetic characters from the captured value of the current Field object.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

Field level.

Details

Example

```
DeleteAllAlpha()  
JAN2003 becomes 2003
```

DeleteAllMiscChars

Deletes all of the UNICODE Symbol Category characters from the captured value of the current Field object.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

Field level.

Details

Example

```
DeleteAllMiscChars()
```

'100c = \$1' becomes '100 1'

DeleteAllNumeric

Deletes all of the numeric characters from the captured value of the current Field object.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

Field level.

Details

Example

```
DeleteAllNumeric()
```

JAN2003 becomes JAN

DeleteAllPunct

Deletes all of the punctuation from the value of the current field.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

Field level.

Details

Removes all characters with ASCII values 33-47,58-64,91-96, and 123-191 from the captured value of the current Field object.

Example

```
DatePUNCT()
```

DeleteAllSysChars

Deletes all of the characters with ASCII values 0 through 31 from the captured value of the current Field object.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

Field only.

Details

Example

```
DeleteAllSysChars()  
A field containing Hello Dolly becomesHelloDolly.
```

DeleteChildType

Deletes all of the child objects of the type that you designate as a parameter from the Document Hierarchy.

Syntax

()

Parameters

String value of the child object Type.

Field, for example, eliminates all Field objects. In the Invoices application, **LineItem** removes the child fields of the Details parent Field object.

Returns

False if the child objects do not exist. Otherwise, **True**.

Level

All except Batch.

Details

Example

```
DeleteChildType("Field")
```

DeleteLCSpaces

Deletes all of the low confidence spaces from the value of the current field.

Syntax

()

Parameters

None.

Returns

False if the action is unable to isolate the spaces in a field's value. Otherwise, **True**.

Level

All levels, but generally at the Field level.

Details

Uses the *ReqConf Setup dco* variable as threshold for low confidence values; defaults to 9.

Example

```
DeleteLCSpaces()
```

DeleteParentObj

Deletes the parent of the Document Hierarchy object to which a rule that contains this action is bound.

Syntax

()

Parameters

None.

Returns

False if a parent or grandparent object cannot be found, or if the deletion of the parent object fails. Otherwise, **True**.

Level

Document, Page, and Field.

Details

Example

```
DeleteParentObj()
```

DeleteSelectedChars

Deletes specified characters from the value of the current field. This action is a more flexible version of the FilterFieldSelectedChars action.

Syntax

()

Parameters

1. The character or string of characters to be deleted.
2. A number indicating the starting index within the string to start deletion. If this parameter is blank or is not a number, the starting index value defaults to the first position in the string.
3. The number of times the character or character string is to be deleted from the value. The default is 1 and * deletes all instances.

Returns

Always **True**.

Level

Field level.

Details

Example

```
DeleteSelectedChars("-",*,*)  
223-56-7669 becomes 223567669
```

```
DeleteSelectedChars("-",5,*)  
223-86-7669 becomes 223-867669
```

EmptyFieldValue

Clears the text value in the field represented by the Field object of the Document Hierarchy that is specified by the parameter.

Syntax

()

Parameters

The name of the Field object that is to be emptied.

Returns

False if the field specified by the parameter does not exist. Otherwise, **True**.

Level

Page or Field level.

Details

FailRuleSet

Causes the entire rule set to fail.

Syntax

()

Parameters

None.

Returns

Always **False**.

Level

All.

Details

Example

```
IsFieldMatching("False")
FailRuleSet()
```

FieldContainsValue

Determines whether the current field value contains some or all of the specified text but no additional text.

Syntax

()

Parameters

Text that the action is looking for in the current field. Smart parameters are supported.

Returns

False if the *CurrentObj.Text* variable of the object does not contain the parameter value. Otherwise, **True**.

Level

All, but generally at the Field level.

Details

This action determines if a field represented by the bound object of the Document Hierarchy contains some or all of the parameter's text value, without additional unspecified text.

Example

```
FieldContainsValue("NEW")
Parameter: "NEW" = TRUE
Parameter: "NEW Action" = TRUE
Parameter: "Project" = FALSE
Parameter: "NEW Project" = TRUE
Parameter: "Development" = FALSE
```

FilterFieldSelectedChars

Deletes the specified characters from the value of the current field.

Syntax

()

Parameters

A String containing the character or characters to be removed.

Every instance of the characters are removed from the captured value.

Returns

Always **True**.

Level

Field level.

Details

Removes all instances of the characters that you enter as parameters from the captured value of the current Field object.

Example

```
FilterFieldSelectedChars(0)  
11002900 becomes 1129
```

FormatNumberToLocale

Evaluates the current field value for known number patterns and if a known pattern is detected, updates the decimal and digit separators characters to match that of the current locale.

Syntax

()

Returns

True, if no errors are encountered. Otherwise, **False**.

Level

Field level.

Details

This action evaluates the current field value for known number patterns, If a known pattern is detected, the action updates the decimal and digit separators characters (if present) to match the current locale. For use when processing fields with number types formatted incorrectly for the current locale, such as US versus European locales. Known patterns are industry standard digits with or without 3-digit group separators, and 1-2 digits following a decimal separator.

Distinct Analyzed groups:

1. Numerals with decimal point

- Group Separators using comma, apostrophe, or space characters.
 - Decimal Separator using decimal character.
2. Numerals with decimal comma
 - Group Separator using decimal, apostrophe, or space characters.
 - Decimal Separator using comma character.
 3. Numerals with Arabic/Persian characters
 - Group Separator using Arabic/Persian character.
 - Decimal Separator using Arabic/Persian character.

Note: Numerals with lakhs layout and decimal mark (pattern of n,nn,nn,nnn.dd) are not supported.

Example

a) Given a US number 1,234.56 to be formatted to UK.
`FormatNumberToLocale()`
 New format will be 1.234,56

b) Given a US number 1234.5 to be formatted to UK.
`FormatNumberToLocale()`
 New format will be 1234,5

c) Given a US number 1,234.567 to be formatted to UK.
`FormatNumberToLocale()`
 Format will remain 1,234.567 since 3 digits following the decimal are not supported.

d) Given a US number 12345678.90 to be formatted to UK.
`FormatNumberToLocale()`
 New format will be 12345678.90

e) Given a UK number 1.234.567,89 to be formatted to US.
`FormatNumberToLocale()`
 New format will be 1,234,567.89

f) Given a UK number 0,5 to be formatted to US.
`FormatNumberToLocale()`
 New format will be 0.5

GetJobID

Assigns the current job ID to the Text property of the current object.

Syntax

()

Parameters

None.

Returns

False if the action cannot find a JobID value. Otherwise, **True**.

Level

All.

Details

Assigns the Job ID of the current User Application job (the `Pilot.JobID` property of the job) to the `CurrentObj.Text` variable of the bound object of the Document Hierarchy.

Example

```
GetJobID()
```

HasChildOfType

Determines whether the current object has a child of the specified type.

Syntax

()

Parameters

The name of a level of the Document Hierarchy (Batch, Document, Page, Field) or of a runtime variable.

Returns

False if the bound object does not include a child or children specified by the parameter, or a variable identified by the parameter. Otherwise, **True**.

Level

All.

Details

Determines if the bound object of the Document Hierarchy has a child or children of the type specified by the parameter. The action can also determine if a runtime variable specified as a parameter has been assigned to the bound object.

Example

```
HasChildOfType("Page")
```

This example determines if the bound object is the parent of one or more pages.

```
HasChildOfType("IGNORE")
```

In this example, the action determines if IGNORE is a runtime variable of the bound object.

InsertChars

Inserts one or more characters into the value of the current field.

Syntax

()

Parameters

1. The characters or character string to be inserted; defaults to a space.
2. A number *n* indicating the target position within the captured value; defaults to the end of the value.
3. The number of insertions; defaults to 1.

Returns

Always **True**.

Level

Field level.

Details

Inserts a character or string of characters into the captured value, one or more times.

Example

```
InsertChars("=$,1,1")
345.67 becomes  =$345.67

InsertChars("=$,1,2")
345.67 becomes  =$=$345.67
```

InsertDecimalPoint

Inserts a decimal point into the value of the current field at the specified position.

Syntax

()

Parameters

A number *n* indicating the character position at which to place the decimal. Smart parameters are supported.

Returns

Always **True**.

Level

Field level.

Details

Places a decimal character in the captured value, at the character position specified as a parameter. The parameter indicates the position of the decimal point, moving from right to left.

Example

```
InsertDecimalPoint("2")
324556 becomes 3245.56

InsertDecimalPoint("2")
355 becomes 3.55
```

IsFieldCurrency

Determines if the captured value of the Field meets the currency format of the current locale.

Syntax

()

Parameters

None.

Returns

True if the current locale's format criteria are met. Otherwise, **False**.

Level

Field level.

Details

This determination includes the number of decimal places, decimal and digit separator characters and any present currency indicators.

Example

```
IsFieldCurrency()  
$1,200 returns False  
$1,200.00 returns True
```

IsFieldDate

Checks that the value of the field has an acceptable Date format. This action uses the current locale setting to determine valid patterns.

Syntax

()

Parameters

None.

Returns

True if the specifications of the action are met. Otherwise, **False**.

Level

Field level.

Details

This action accepts any valid date from January 1st year 1 through December 31st 9999.

Example

```
IsFieldDate()  
  
In locale en-US (United States):  
April 6, 1944 returns True  
04/06/44 returns True  
30.6.44 returns False  
Feb 31,2003 returns False
```

IsFieldDateEqualOrAfter

Checks that the Date value in the current field represented by the bound Field object of the Document Hierarchy is greater than or equal to the Date value in the field that is specified as the parameter.

Syntax

()

Parameters

The name of the Field object of the Document Hierarchy to be compared with the Date value of the current field.

Returns

False if the date condition is not met, if the action is not applied at the Field level, or either field does not contain a valid date. Otherwise, **True**.

Level

Field level.

Details

Example

```
IsFieldDateEqualOrAfter("24aDtFr1")
```

IsFieldDateEqualOrBefore

Checks that the date in the current field represented by the bound Field object of the Document Hierarchy is less than or equal to the Date value in the field that is specified as the parameter.

Syntax

()

Parameters

The name of the Field object of the Document Hierarchy to be compared with Date value of the current field.

Returns

False if the date condition is not met, if the action is not applied at the Field level, or if either field does not contain a valid date value. Otherwise, **True**.

Level

Field Level.

Details

Example

```
IsFieldDateEqualOrBefore("24aDtFr1")
```

IsFieldDateUpToToday

Checks that the Date value of the current Field object is today's date or earlier.

Syntax

()

Parameters

None.

Returns

False if the value of the field is not a valid date or if the date is after Today. Otherwise, **True**.

Level

Field level.

Details

Example

```
IsFieldDate()  
IsFieldDateUpToToday()
```

This sequence confirms that a value is a date and that it is the same as, or earlier than, today's date.

IsFieldDateWithinRange

Checks that the value assigned to the Text property of the bound object is a valid Date. If yes, this action then confirms that the Date is within the range specified by the parameters.

Syntax

()

Parameters

Comma-separated Dates that define the range:

1. Start Date
2. End Date

TODAY can represent the current Date. Smart parameters are supported.

Returns

False if the value assigned to the Text property of the current object is not a valid date; if either parameters is invalid; or the Date value of the Text property is not within the range that is specified by the parameters. Otherwise, **True**.

Level

All, but generally at the Field Level.

Details

Example

```
IsFieldDateWithinRange("1/1/2006, 1/31/2006")  
IsFieldDateWithinRange("1/1/2006,TODAY")
```

IsFieldDateWithinXDays

Checks that the captured Date value of the current Field object is within n days of the number entered as a parameter.

Syntax

()

Parameters

A number n that specifies how many days make up the review period. Smart parameters are supported.

Returns

False if the value of the field is not a valid date or if the date is older than the number of days in the parameter. Otherwise, **True**.

Level

Field level.

Details

Example

```
IsFieldDate()  
IsFieldDateWithinXDays("30")
```

This sequence checks that a value is a date within 30 days of today's date.

IsFieldDateWithReformat

Confirms that the data of a field is a valid date and then formats the Date value according to the format entered as the parameter.

Syntax

()

Parameters

The Date format you want to use.

- mm/dd/yyyy
- mm/dd/yy
- dd/mm/yy
- mm.dd.yy, etc.

Defaults to system short date format if no format or single * is used. Smart parameters are supported.

Returns

False if the parameter is invalid, or the current field value is not a valid date given the specified format. Otherwise **True**.

Level

Field level.

Details

Example

```
IsFieldDateWithReformat("*")  
June 3, 2002 becomes 06/03/2002  
  
IsFieldDateWithReformat("mm.dd.yy")  
June 3, 2002 becomes 06.03.02
```

IsFieldEmpty

Checks that the Field object designated as a parameter does not have a captured value.

Syntax

()

Parameters

The name of the Field object.

Returns

False if the name of the Field object does not exist or if the field contains a captured value. Otherwise, **True**.

Level

All.

Details

Example

```
IsFieldEmpty("Shipping")  
AssignFieldDefault("NoShipping")
```

In this example, if the captured value of the Shipping Field object is \$921.11, this action return a **False** condition and terminates the rule.

If the Shipping Field object does not have a value (**True**), the AssignFieldDefault action enters the **NoShipping** parameter as the captured value of the Field object.

IsFieldFilled

Determines whether the Field object designated as a parameter contains a captured value or is empty.

Syntax

()

Parameters

The name of the Field object.

Returns

False if the name of the Field object does not exist or if the field does not contain a captured value. Otherwise, **True**.

Level

All.

Details

Example

```
IsFieldFilled("PaymentDue")
```

If the action returns **True** because the field does contain a value, the rule invokes its next action.

If the action returns **False**, the rule closes and the task applies the next rule, which might include a CopyFieldToField action.

IsFieldGreaterOrEqual

Determines if the captured value of the current Field object is greater than or equal to the value entered as a parameter.

Syntax

()

Parameters

The Numeric or Currency value which is the basis for comparison. Smart parameters are supported.

Returns

False if the parameter or the captured value of the Field object is not Numeric. Or if the result does not meet the action's requirements. Otherwise, **True**.

Level

Field level.

Details

If the value of the field is not Numeric or currency, the action returns a **False** condition.

Example

```
IsFieldGreaterOrEqual("624")  
Returns True if the Field object's value is 625.00  
Returns False if the value is 623.99.  
Returns True if the value is 624.00.
```

IsFieldHidden

Checks that the calling field is Hidden, which means the field has a variable *STATUS* that is equal to -1.

Syntax

()

Parameters

None.

Returns

True if the *STATUS* variable of the Field object = -1. Otherwise **False**.

Level

Field level.

Details

This action returns **True** if the calling field is Hidden, the corresponding Field object of the Document Hierarchy has a variable *STATUS* that is equal to -1.

Example

```
IsFieldHidden()
```

IsFieldLengthMax

Checks that the character length of the current Field object's captured value is equal to or less than the value set as a parameter.

Syntax

()

Parameters

A number *n* designating the maximum length of the value. Smart parameters are supported.

Returns

False if the parameter that you enter is not numeric, or if the number of characters exceeds the value of the parameter. Otherwise, **True**.

Level

Field level.

Details

Example

```
IsFieldLengthMax("6")  
EU2240 returns True  
EU002240 returns False
```

IsFieldLengthMin

Checks the character length of the current Field object's captured value to see if its length is equal to or longer than a number *n*.

Syntax

()

Parameters

A number *n* designating the minimum length of the value. Smart parameters are supported.

Returns

False if the parameter that you enter is not numeric, or if the number of characters is less than the parameter's value. Otherwise, **True**.

Level

Field level.

Details

Example

```
IsFieldLengthMin("6")  
EU2240 returns True  
EU22 returns False
```

IsFieldLessOrEqual

Determines if the captured value of the current Field object is less than or equal to the value entered as a parameter.

Syntax

()

Parameters

The Numeric or Currency value you want to compare against. Smart parameters are supported.

Returns

False if the parameter or captured value of the Field object is not Numeric. Or if the result does not meet the action's requirements. Otherwise, **True**.

Level

Field level.

Details

If the value of the field is not Numeric or Currency, the action returns a **False** condition.

Example

```
IsFieldLessOrEqual("625")  
Returns True if the Field object's value is 624.99  
Returns False if the value is 625.01  
Returns True if the value is 625.00
```

IsFieldMatching

Determines if the value entered as the parameter is identical to the captured value of the current Field object.

Syntax

()

Parameters

The value to be checked against the value of the Field object.

Returns

True if the requirement of the action is met. Otherwise, **False**.

Level

Field level.

Details

Example

If the Field object's value is 525.00:

```
IsFieldMatching("525.00") returns True
IsFieldMatching("525") returns False
```

IsFieldPercentAlpha

Determines if the characters in the captured value of current Field object are *n%* alphabetic.

Syntax

()

Parameters

A number (0-100) indicating the percentage necessary to return a **True** condition. The value must be numeric, without the percent sign. Smart parameters are supported.

Returns

True if the requirements of the parameter are met. Otherwise **False**, including if the Field is empty.

Level

Field level.

Details

Example

```
IsFieldPercentAlpha("50") #RPR-1421 returns False
IsFieldPercentAlpha("30") #RPR1421 returns True
```

IsFieldPercentNonNumeric

Determines if any of the characters in the captured value of the current Field object are *n%* not numeric characters.

Syntax

()

Parameters

A number (0-100) indicating the percentage that results in a **True** condition. The default percentage is 100. The value must be numeric, without the percent sign. Smart parameters are supported.

Returns

False if the parameter is non-numeric, if the field is empty, or if the value of the field exceeds the parameter's percentage of numeric characters. Otherwise, **True**.

Level

Field level.

Details

This determination includes and is not limited to valid decimal and numeric separator characters.

Example

```
Given the current value is "1,234.56US"  
(Percentage of non-numeric characters is 40%)  
IsFieldPercentNonNumeric("0") returns True  
IsFieldPercentNonNumeric("30") returns True  
IsFieldPercentNonNumeric("40") returns True  
IsFieldPercentNonNumeric("70") returns False  
IsFieldPercentNonNumeric("100") returns False
```

IsFieldPercentNumeric

Determines if the characters in the captured value of the current Field object are *n*% numeric characters.

Syntax

()

Parameters

A number (0-100) indicating the percentage that results in a **True** condition. The default percentage is 100. The value must be numeric, without the percent sign. Smart parameters are supported.

Returns

False if the parameter is non-numeric, if the field is empty, or if the value of the field exceeds the parameter's percentage of numeric characters. Otherwise, **True**.

Level

Field level.

Details

This determination does not include, and is not limited to, valid decimal and numeric separator characters.

Example

```
Given the current value is "1,234.56US"  
(Percentage of numeric characters is 60%)  
IsFieldPercentNumeric("0") returns True  
IsFieldPercentNumeric("50") returns True  
IsFieldPercentNumeric("60") returns True  
IsFieldPercentNumeric("70") returns False  
IsFieldPercentNumeric("100") returns False
```

IsMatchingJobID

Checks that the Job ID of the current User Application job matches the Job ID value of the parameter.

Syntax

()

Parameters

String value of the Job ID to be compared to the current Job ID.

Returns

True if the current Job ID matches the value of the parameter. Otherwise, **False**.

Level

All.

Details

Example

```
IsMatchingJobID("Main")
```

IsMaxOMRChecked

Indicates the maximum number of check boxes that can contain a value, such as a check.

Syntax

()

Parameters

An Integer value specifying this maximum. Smart parameters are supported.

Returns

False if the parameter you enter is not numeric, or the field is not an OMR field.

True if the number of OMR boxes checked is less than or equal to the parameter value that you entered.

Level

Field level.

Details

Example

```
IsMaxOMRChecked("1")
```

IsMinOMRChecked

Indicates the minimum number of check boxes that can contain a value, such as a check.

Syntax

()

Parameters

An Integer value specifying this minimum. Smart parameters are supported.

Returns

False if the parameter you enter is not numeric, or the field is not an OMR field.

True if the number of OMR boxes checked is greater than or equal to the parameter value that you entered.

Level

Field level.

Details

Example

```
IsMinOMRChecked("1")
```

IsPatternInField

Checks that the value of the current field contains the specified VBScript regular expression.

Syntax

```
bool IsPatternInField (StrParam)
```

Parameters

String value of the Regular Expression. The expression can include any Regular Expression characters. Smart parameters are supported.

Returns

True, if the pattern is found within the field. Otherwise, **False**.

Level

All, but generally at the Field level.

Details

Uses VBScript Regular Expression Pattern entered as parameter to search for a matching pattern in the Text value of the current object.

Attention: If you are using an input boundary, ^, it must be followed by a space and then the remainder of the search string.

Example

```
IsPatternInField("[\^\b\s\n\r]*Inv[o0][iIt1l]ce[\b\s]*")  
IsPatternInField("@STRING([\^\b\s\n\r]*Inv[o0][iIt1l]ce[\b\s]*")
```

This example searches for the word “Invoice” in the current field. To allow for recognition errors, the search allows for common recognition substitutions in “o” and “i” by matching “Invoice”, “InvOice”, “Inv0ice”, “Inv01ce”, “Involce”, and so on. The search also ignores any text that is

placed before or after the word. Encapsulating the parameter with @STRING is recommended when the value is not a Smart Parameter.

IsSupportedImageFile

Checks that the image file attached to the current page is in a supported image format.

Syntax

()

Parameters

A Boolean that determines the type of test to perform.

True: Tests the validity of the image by checking the file extension and attempting to load the image.

False: Only a file extension check is performed to determine if it is a supported file.

Returns

True if parameter is valid, the action is called at the page level and the page's *IMAGEFILE* variable (set at scan time by the scan tasks) points to a file whose format is supported (can be displayed) by the Image view control. Otherwise **False**.

Level

Page level.

Details

This action tests a file to determine if the file format is supported. The extension is checked to determine if it denotes a supported image type. If **True** is passed as a parameter, it also attempts to load the file into the Image view control.

Using **False** as a parameter improves the speed of this action. A parameter of **True** slows down the processing, especially if the images are very large, but it adds an extra confirmation that the file is not corrupted and confirm that any subformat of the file type, such as compression type, is also supported.

Example

```
IsSupportedImageFile()
```

IsThisFieldEmpty

Checks that the value of the current field is empty.

Syntax

()

Parameters

None.

Returns

False if not applied to the Field level, or if the current field has a text value. Otherwise, **True**.

Level

Field level.

Details

Confirms if the current field has no captured value.

Example

```
IsThisFieldEmpty()
```

IsThisFieldFilled

Checks that the current field has a captured value.

Syntax

()

Parameters

None.

Returns

False if not applied to the Field level, or if the current field has no text value. Otherwise, **True**.

Level

Field level.

Details

Confirms if the current field has a captured value.

Example

```
IsThisFieldFilled()
```

IsVariableEmpty

Checks that the variable specified by the parameter does not contain a value.

Syntax

()

Parameters

Name of the variable of the current object to be checked.

Returns

False if the parameter is invalid, or if the variable contains a value. Otherwise, **True**.

Level

All.

Details

This action only checks variables of the current object.

Example

```
IsVariableEmpty("TemplateID")
```

IsVariableFilled

Checks that the variable specified by the parameter contains a value.

Syntax

()

Parameters

Name of the variable of the current object to be checked.

Returns

False if the parameter is invalid, or if the variable does not contain a value. Otherwise, **True**.

Level

All.

Details

This action only checks variables of the current object.

Example

```
IsVariableFilled("TemplateID")
```

LeftTruncate

This action is deprecated and is scheduled to be removed in a future release. Use the TruncateFromEnd action.

Syntax

()

Parameters

Returns

Level

Details

This action is replaced by the TruncateFromEnd action.

MessageBox

This action is deprecated and is no longer supported.

Syntax

()

Parameters

The message you want to display to the Data Entry operator. Smart parameters are supported.

Returns

False if the current task is not interactive or if the operator clicks **No** on the **Message Box**. Otherwise, **True**.

Level

All.

Details

* This Action has been Deprecated and this functionality is no longer supported. *

Causes a **Message Box** with two buttons, labeled **Yes** and **No** to appear on the operator's screen during the Verification task. Displays a message that you enter as the parameter.

This action should only be placed in a rule that is run interactively.

If a background station runs this rule, it stops processing until a user responds to the message box. If the Data Entry operator clicks on **Yes** in the **Message Box**, the action returns **True** and the rule continues. If the operator chooses the **No** button, the action returns **False** and the rule fails.

Example

```
IsFieldEmpty("Date")
MessageBox("Date missing! Do you want to Continue?")
```

After checking to be sure the Date field does not have a value, the MessageBox action issues a warning, and asks the operator to respond by clicking **Yes** or **No**.

ParseMultilineAddress

Splits the value of the current field at each comma and saves the substrings to the specified fields. Typically used for address fields.

Syntax

()

Parameters

Comma-separated Smart Parameter String values of the names of fields that hold the parsed data, in the following order:

Name, AddressLine1, AddressLine2, City, State, Zip code or postal code, Phone

Returns

False, if the parameters are invalid or parsing cannot take place. Otherwise, **True**.

Level

Field level.

Details

Parses a multiline US address Field object's captured value.

Comma-separated String values of the names of fields that hold the parsed data, in the following order: Name, AddressLine1, AddressLine2, City, State, Zip code or postal code, and Phone. The example assumes that fields are sibling fields of the calling object. For other relationships, review smart parameter syntax.

Expected Pattern:

- Phone Number (optional)
- Name
- Address Line One
- Address Line Two (optional)
- City, State, Zip code or postal code
- Phone Number (optional)

Note: Parsing logic assumes that State and Zip or postal code are on the same address line. Only one Phone number per address field is supported. The expected pattern shows the two optional positions for this value.

Example

```
ParseMultilineAddress("VendorName,VenAddress1,VenAddress2,VenCity,  
VenState,VenZip,VenPhone")
```

ParseName

Splits the three word value of the current field and saves the substrings to the specified fields. Typically used for name fields, such as first name, middle name/initial, last name.

Syntax

()

Parameters

Three comma separated parameters:

1. The name of the Last Name Field object.
2. The name of the First Name Field object.
3. The name of the Middle Name or Middle Initial Field object.

Returns

False if not placed at the Field level; if the current field contains no data; or if the parameters are invalid. Otherwise, **True**.

Level

Field level.

Details

Parses the captured values of a name Field object. Applied to a Name field, the action parses the Last, First, and Middle names into the fields specified by the parameter.

Example

```
ParseName("LastName,FirstName,MidName")
```

Bound to a Name Field object which includes values for all three names, the action place the Last name into the LastName field, the First name into the FirstName field, and the Middle name (or middle initial) into the MiddleName field.

ReadCurrentObjVariable

Reads the Text value of the bound object of the Document Hierarchy.

Syntax

()

Parameters

The name of the bound object's property or variable.

Returns

False if the property or variable specified by the parameter is not found. Otherwise, **True**.

Level

All.

Details

For a Field object, this is the value of the object's Text property. For objects at other levels, the value is in a runtime variable of the bound object, see the following examples.

Example

```
ReadCurrentObjVariable("Text")
```

```
ReadCurrentObjVariable("TemplateID")
```

ReadFieldValue

Retrieves the captured value from a sibling Field object specified as the parameter and assigns that value to the current Field object.

Syntax

()

Parameters

The name of the source Field object.

Returns

False if the parameter does not match the name of a Field object. Otherwise, **True**.

Level

All.

Details

If the action is not bound at the Field level, then a variable called *Text* is created in the current object, and the found value is written to this variable.

Example

```
ReadFieldValue("Date")
```

This action assigns the captured value in the Date field to the current field of the working fingerprint.

ReadPageVariableValue

Assigns the value of a variable of the Page object to the selected Field object.

Syntax

()

Parameters

The name of the Page object's variable.

Returns

False if the parameter is not a valid Page-level variable. Otherwise, **True**.

Level

All.

Details

If the action is not placed at the field level, then a variable called *Text* is created in the current object, and the found value is written to this variable.

Example

```
ReadPageVariableValue("FieldCount")
```

A Validate rule with this action could transfer the number of fields in the Page object that is the parent of a selected Field object such as TotalFields.

Important: A Document Hierarchy might include multiple Page objects and certain variables that are the same for both pages. This action refers to variables of the Page object to which the selected Field object belongs.

ReplaceChars

Replaces a character or string of characters in the captured value of the current Field object with a String that you enter as one of the parameters.

Syntax

()

Parameters

1. The character or string of characters to be replaced; defaults to a space. Smart Parameter Enabled.
2. The characters of the replacement String. Smart Parameter Enabled.
3. The number of times replacement is to occur. The default is 1 and * replaces all instances.

Returns

Always **True**.

Level

Field level.

Details

Example

```
ReplaceChars(".,/*")  
01.02.2005 becomes 01/02/2005
```

ReplaceValueAtPosition

Replaces the value at the specified position within the current field with a replacement string, or deletes the value.

Syntax

()

Parameters

Two-part comma-separated value:

1. The position that contains the value to be replaced.
2. The replacement string; this parameter defaults to "" indicating a deletion.

Returns

True if the character replacement is successful. Otherwise, **False**.

Level

Field level.

Details

Example

```
ReplaceValueAtPosition("3,/")
```

ResetField

Deletes the value of the current field and sets the Position attribute of the field to 0,0,0,0.

Syntax

()

Parameters

None.

Returns

False if not called on the field level. Otherwise, **True**.

Level

Field level.

Details

The action does not clear any Alt-Text values that are associated with the bound Field object. That is the responsibility of a ClearAltText action.

Example

```
ResetField()
```

This action typically belongs to a follow-up Validate rule that deals with a **False** response to an action such as IsDate() in the previous rule. If the value of the field is not a date (in this example), the ResetField action removes the value that is there.

This action example also sets the Position attribute of the field to 0,0,0,0.

RightTruncate

This action is deprecated and is scheduled to be removed in a future release. Use the TruncateFromStart action.

Syntax

()

Parameters

Returns

Level

Details

This action is replaced by the TruncateFromStart action.

SaveAsCurrentObjVariable

This action is deprecated and is scheduled to be removed in a future release. Use the rrSet action in the runner library.

Syntax

()

Parameters

1. The name of the variable
2. The String value to be assigned.

These are comma-separated parameters, and the second is optional. If you do not include this parameter, the action automatically assigns the captured value of the selected Field object to the variable.

Returns

Always **True**.

Level

All.

Details

Use the @F smart variable to access the current field value.

SaveAsPageVariable

Assigns a value to a variable of the current Page object.

Syntax

()

Parameters

1. The name of the variable
2. The String value to be assigned.

These are comma-separated parameters, and the second is optional. If you do not include this parameter, the action automatically assigns the Text property of the calling object to the variable.

Returns

False if the parent page cannot be found. Otherwise, **True**.

Level

Page or Field Level.

Details

Example

```
SaveAsPageVariable("Amount")
```

```
SaveAsPageVariable("Amount,15000")
```

The first example assigns the captured value of the current Field object to the Amount variable of the current Page. The second example assigns 15000 to the variable.

SetIsOverrideable

Specifies if the user can or cannot override a validation that fails for the current object.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

All.

Details

Important: This status might prevent an operator from overriding validations on a field and then continuing to subsequent pages.

Example

```
SetIsOverrideable("False")  
IsFieldPercentNumeric("100")
```

In this sequence, if the captured value of the field is not 100% Numeric, an operator cannot override a Validation rule's subsequent rejection of the value.

SplitFieldValueLeft

This action is deprecated and is scheduled to be removed in a future release. Use the SplitFieldValuePreserveStart action.

Syntax

()

Parameters

Returns

Level

Details

This action is replaced by the SplitFieldValuePreserveStart action.

SplitFieldValuePreserveEnd

Splits the captured value of a Field object at the first instance of the character specified as a parameter.

Syntax

()

Parameters

String value of the separating character. Smart parameters are supported.

Returns

True if the separator character is found. Otherwise **False**.

Level

Field level.

Details

The action deletes all characters prior to the separating character, as well as the separation character. All text after the separating character remains.

Example

```
SplitFieldValuePreserveEnd("=")
```

If the value of the object is `InvNumber=A1234`, this action truncates it to `A1234`.

SplitFieldValuePreserveStart

Splits the captured value of a Field object at the first instance of the character specified as a parameter.

Syntax

()

Parameters

String value of the separating character. Smart parameters are supported.

Returns

True if the separator character is found. Otherwise **False**.

Level

SplitFieldValuePreserveEndField level.

Details

The action deletes all characters to the end of the string starting from the separating character, as well as the separation character. All text prior to the separating character remains.

Example

```
SplitFieldValuePreserveStart("c")
```

If the value of the object is `Description`, this action truncates it to `Des`.

SplitFieldValueRight

This action is deprecated and is scheduled to be removed in a future release. Use the `SplitFieldValuePreserveEnd` action.

Syntax

()

Parameters

Returns

Level

Details

This action is replaced by the `SplitFieldValuePreserveEnd` action.

SumFields

Adds the values of all child fields of the specified type and assigns the result to the current field. You can also use this action to sum the values of the specified variable for all child objects.

Syntax

()

Parameters

String value of a Field object's `Type` property or the name of a variable.

Returns

Always **True**.

Level

All.

Details

Sums captured values of any child Field if a child object's `Type` property is identical to the `Type` you specify as a parameter. Alternatively, the actions sums values assigned to a variable of the child Field objects. In this case, the variable is the same as the variable entered as a parameter.

Remember: This action must be applied to the parent object.

Example

```
SumFields("Detail")
```

```
SumFields("LineTotal")
```

The first action in the example sums the captured values of `Detail` Field objects that are children of the current Field object.

The second action in the example sums values assigned to the *LineTotal* variable of child Field objects.

In both cases, the result is a Long number.

TimeStampField

Updates the current Field object with the current time.

Syntax

()

Parameters

A time format, for example, HH:MM:SS, or HH:MM.

* defaults to HH:MM:SS. Smart parameters are supported.

Returns

Always **True**.

Level

Field level.

Details

Example

```
TimeStampField("*") produces  
09:20:02
```

```
TimeStampField("HH:MM") produces  
09:20
```

TrimSpaces

Deletes extra spaces at the beginning and end of the captured value of the current Field object.

Syntax

()

Parameters

None.

Returns

Always **RightTruncate**.

Level

Field level.

Details

Example

```
TrimSpaces()  
456.11_ _ _ _ becomes 456.11
```

TruncateFromEnd

Deletes characters from the end of the captured value of the current Field object until the length of the value equals the length indicated by the parameter.

Syntax

()

Parameters

A Number n that is the maximum length of the value. Smart parameters are supported.

Returns

False if the parameter is not Numeric. Otherwise, **True**.

Level

Field level.

Details

This action deletes characters from the end of the captured value of the current Field object until the length of the value equals the length indicated by the parameter.

Example

```
TruncateFromEnd("6")  
EU0002240 becomes EU0002
```

TruncateFromStart

Deletes characters from the start of the captured value of the current Field object until the length of the value equals the length specified by the parameter.

Syntax

()

Parameters

A number n that is the maximum length of the value. Smart parameters are supported.

Returns

Always **True**.

Level

Field level.

Details

This action deletes characters from the start of the captured value of the current Field object until the length of the value equals the length specified by the parameter.

Example

```
TruncateFromStart("6") reduces the following value:  
3,344.01 becomes 344.01
```

Vote actions

Use the Vote action when you do multi-pass data entry to check whether the first and second passes match.

The Vote action returns True if the data entered for by the first operator matches the data that is entered by the second operator.

“VoteFld”

VoteFld

Checks to see whether the data entered by the first Data Entry operator matches the data that is entered by the second Data Entry operator.

Syntax

()

Parameters

None.

Returns

False, if the values do not match. Otherwise, **True**.

Level

Field level.

Details

Checks to see whether the data entered by the first Data Entry operator matches the data entered by the second Data Entry operator.

Failed (mismatched) values set the confidence for the entire string to '1', which flags the field as Low Confidence. Positive matches set the entire strings confidence to '9' (High Confidence).

This action needs to run after the second Data Entry task is complete.

This action is used for workflows by using third pass data entry.

Example

```
VoteFld()
```

Vscan actions

Use the Vscan actions to create a batch by using existing image files.

The Vscan actions determine which documents are included in the batch and how to handle these documents as part of the scan task.

“AddDocument” on page 969

“CopyFile” on page 969

“DeleteImageFile” on page 970

“MoveImageFileToDirectory” on page 971

“Scan” on page 972

“SearchInSubdirectory” on page 973

“SetAlternateImageNames” on page 973

“SetFastMode” on page 974

“SetImageType” on page 974

"SetMailSourceFolder" on page 975

"SetMaxImageFiles" on page 976

"SetMultiPageTiff" on page 977

"SetSortOrder" on page 977

"SetSourceDirectory" on page 978

AddDocument

Adds a document node to the runtime hierarchy. All scanned pages become children of the document node.

Syntax

()

Parameters

None.

Returns

False if this action is invoked or called at the wrong level or if the setup DCO does not contain any document structure. In this case, the action will not add a document. Otherwise, **True**.

Level

Batch level.

Details

Places all scanned pages into a single, batch-wide document. This action must be called before the Scan action.

The action also adds ED (Expected Documents), AD (Actual Documents), EP (Expected Pages) and AP (Actual Pages) properties to the Page file (.xml) of a task.

This action allows a VScan task to create a batch that has all the same characteristics of a batch that is created using a standard scan task. It is not required to call this action. Use this action if your application relies on a document structure on import.

This action must precede the Scan action. If AddDocument is used and the Scan action is never subsequently called, the action will not be performed.

Example

```
AddDocument()  
SetSortOrder("Name,ASC")  
SetMaxImageFiles("100")  
Scan()
```

CopyFile

When used before the Scan action, this action tells the Scan action to also copy the files to the specified location.

Syntax

()

Parameters

1. The String value of the name of the target file system folder.
2. Optional. The file extension you wish to use for each file. If you provide an extension, it must not be preceded by a period. The action defaults to .tif if this parameter is blank.

Smart parameters are supported.

Returns

Always **True**. If the target directory does not exist or if the files cannot be created, the files are not copied but the action still returns **True**.

Level

Any level but usually the batch level.

Details

A housekeeping action that copies the current source image file to a location that you specify and specifies the copied file's extension.

The source image file will remain in the input directory while a copy is placed into the specified directory. This action is typically used when it is desired to place images into a specific location for archiving. A copy of the image will still be placed into the batch directory as well.

If the target directory already contains a file of the same name, it will give a unique name to the new file.

This action must precede the Scan action. The CopyFile sets an indicator for the Scan action to copy the file. If CopyFile is used and the Scan action is never subsequently called, the copy will not be performed. The destination directory must already exist or a message will be logged and no files will be copied.

Example

```
CopyFile("C:\ParentDirectory\Application\Images\copies.tif")
Scan()
```

This example copies the source image file to the copies folder and then adds it to the batch. It does not delete the image from the source folder.

```
CopyFile("@APPATH(vscancopydir),tif")
Scan()
```

This variant example copies the source image file to the copies folder that was specified in the application service.

DeleteImageFile

When used before the Scan action, this action tells the Scan action to delete the source files from the images folder.

Syntax

```
()
```

Parameters

None.

Returns

Always **True**.

Level

Any level but usually the batch level.

Details

A housekeeping action that deletes the current source image file. A Scan action must follow this action, which will perform the delete during the vscan process. If Scan is never called, the files are not deleted.

Example

```
DeleteImageFile()  
Scan()
```

This sequence deletes the image files from their current location after they were added to the current batch folder.

MoveImageFileToDirectory

When used before the Scan action, this action tells the Scan action to move the files from the images folder to the specified location.

Syntax

()

Parameters

The full path to the target location of the Image file. Smart parameters are supported.

Returns

False if the parameter is not a valid directory, or if permission to access/write to the directory is denied. Otherwise, **True**.

If the file cannot be written to the destination directory, the batch status will be set to abort.

Level

Any level but usually the batch level.

Details

A housekeeping action that moves the current Image file to a location you specify. The source image file will be removed from the input directory and moved to the specified directory. This action is typically used when it is desired to place images into a specific location for archiving. A copy of the image will still be placed into the batch directory.

If the target directory already contains a file of the same name, it will give a unique name to the new file.

This action must precede the Scan action. If MoveImageFileToDirectory is used and the Scan action is never subsequently called, the file move will not be performed. If the move to the new directory fails, the batch will be set to abort.

Example

```
MoveImageFiletoDirectory("C:\ParentDirectory\Application\backup")
SetMaxImageFiles("100")
Scan()
```

This sequence copies the source image files to the current batch, then moves the Image files to the specified directory.

```
MoveImageFiletoDirectory("@APPPATH(vscanmovedir)")
Scan()
```

This variant example uses a smart parameter to obtain the directory path from the application service.

Scan

Copies image files from the location that is specified by the SetSourceDirectory action to the batch folder and creates the runtime hierarchy.

Syntax

()

Parameters

None.

Returns

False if a SetSourceDirectory action does not precede this action. **False** is also be returned if fast mode is enabled and SetMultiPageTiff was called. Otherwise, **True**.

Level

Batch level.

Details

Scans a set of waiting Image files, according to the parameters set by earlier actions.

This is usually the last action in a vScan rule.

Example

```
SetImageType(".tif")
SetSourceDirectory("@APPPATH(vscanimagedir)")
SetMaxImageFiles("100")
Scan()
```

These are the elements of a sample vScan Rule. The Scan action will load the specified number of images (if available) from the specified folder into the current batch folder.

SearchInSubdirectory

When used before the Scan action, this action tells the Scan action to look in subdirectories of the images folder.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

Any level but usually the batch level.

Details

Looks for source files in sub-directories of the directory you designated with a SetSourceDirectory action. For this action to take effect, it must be called before the Scan action.

Example

```
SetSourceDirectory("@APPPATH(vscanimagedir)")  
SearchInSubdirectory()
```

In this example, if the scan directory obtained from the application service includes sub-directories, this action directs the Scan action to process the contents of the sub-directories.

SetAlternatelImageNames

Sets the file naming scheme of the Scan action.

Syntax

()

Parameters

0: Sets the naming scheme of the input files to the format used by eDocument Conversion actions.

Any other input value uses the traditional naming scheme of TM000001, TM000002, etc.

Returns

Always **True**.

Level

Any level but usually the batch level.

Details

If you are using the Convert action library, it is recommended that you enable this batch file naming scheme.

If this action is not called prior to the Scan action, then the Scan uses the traditional naming scheme of TM000001, TM000002, etc.

Example

```
SetAlternateImageNames("0")
Scan()
```

SetFastMode

Increases speed by preventing the Scan action from opening the files it is scanning. This action is provided only for compatibility with older applications, it is no longer used.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

Any level but usually the batch level.

Details

This action prevents the testing that the TIF files are valid and avoid conversion of TIF files to G4 compression for processing. Calling this action will allow TIF files that are not compatible with Datacap to be placed into a batch.

This action is provided as backwards compatibility to older applications where this action was required to properly enable fast mode for PDF files.

SetImageType will automatically enable fast mode, if it is called with a extension other than TIF or JPG. For this action to take effect, it must be called before the Scan action.

Example

```
SetImageType(.pdf)
SetFastMode()
Scan()
```

SetImageType

Specifies the extensions of image file types to use, defaults to .tif.

Syntax

()

Parameters

String value of the file's identifying extension. If you are listing multiple types, separate each one with a comma.

It is not necessary to include a period before each extension. The parameters are not case-sensitive.

Returns

True, if the parameter is one of the values that is specified above. Otherwise, **False**.

Level

Any level but usually the batch level.

Details

Uses the value of a file extension to specify the type of files the task scans.

This is an optional vScan action. The task scans .tif files by default. For this action to take effect, it must be called before the Scan action.

This action automatically enables Fast Mode scanning, if the input parameter is not a single extension of type .tiff, .tif, .jpeg, or .jpg.

Example

```
SetImageType(".tif")
Scan()
```

This sequence scans only TIF files and does not enable Fast Mode.

```
SetImageType("tiff")
Scan()
```

This sequence scans only TIFF files and does not enable Fast Mode.

```
SetImageType("bmp,jpg,jpeg,msg,tif,tiff,pdf,zip,doc,docx,xls,xlsx,eml,gif")
Scan()
```

This sequence scans all file types listed and enables Fast Mode.

SetMailSourceFolder

When used before the Scan action, this action tells the Scan action to search the images folder for emails with image file attachments. Then, to copy these emails into the batch folder.

Syntax

```
()
```

Parameters

String value of the Outlook folder you want to search.

Returns

Always **True**.

Level

Any level but usually the batch level.

Details

Searches the location you specify for e-mails that contain Image file attachments. For this action to take effect, it must be called before the Scan action.

Example

```
SetMailSourceFolder("Inbox/Images")  
Scan()
```

This example searches the Images subfolder for any e-mails with attachments, and copy these Image files into the newly created batch.

```
SetMailSourceFolder("Inbox+/@VAR(FOLDERNAME)")  
Scan()
```

This example gets the name of the subfolder from the runtime variable *FOLDERNAME*.

SetMaxImageFiles

Limits the number of files the Scan action copies.

Syntax

()

Parameters

An optional string value specifying the maximum number of files.

If you do not enter a value, the task will scan all images in the target folder (up to 32767 files) and this action returns **True**.

Returns

False if the parameter is not Numeric. Otherwise, **True**.

Level

Any level but usually the batch level.

Details

Limits the number of Image files the vScan task will add to a single batch. This action must be placed before the Scan action for the setting to take effect during Scan.

Example

```
SetMaxImageFiles("50")  
Scan()
```

Sets the maximum number of files to add to a batch at fifty.

Remember: A vScan task is a Batch Creation task: it sets up a new batch each time it scans Image files.

SetMultiPageTiff

When used before the Scan action, this action tells the Scan action to split multipage TIFF files into single page TIFF files.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

Any level but usually the batch level.

Details

Permits the use of multipage source image files. For this action to take effect, it must be called before the Scan action.

This action cannot be used if fast mode is enabled. It also requires that SetImageType be called with only one extension of TIF, TIFF, JPG or JPEG. If this action is used when fast mode is enabled, the Scan action returns **False**.

Example

```
SetImageType(".tif")
SetMultiPageTiff()
Scan()
```

If the Scan action in this sequence encounters a multipage .tif file, then it reads each one into the current batch as a separate image, thereby bursting the multipage file into individual images.

SetSortOrder

Specifies the order in which image files are imported.

Syntax

()

Parameters

Two comma-separated String values:

Parameter 1. Designation of the images' sorting field can be specified with text or numerically:

- **1** or **Name** : The input file name.
- **2** or **Type** : The file type.
- **3** or **DateCreated** : The File creation date.
- **4** or **DateLastAccessed** : The last file access date.
- **5** or **DateLastModified** : The last file modification date.
- **6** or **Size** : The file size.

Parameter 2. Optional: ASC or 1 (Ascending), DESC or 2 (Descending). If you do not include this parameter, the action defaults to ASC (1).

Returns

False if the parameters are not valid. Otherwise, **True**.

Level

Any level but usually the batch level.

Details

Sets the order in which Image files will be imported to the batch. The input files can be sorted by their file name, file type, date created, date accessed, date modified or file size. Using the optional second parameter, you can control if the values are sorted in ascending or descending order.

For this action to take effect, it must be called before the Scan action.

Example

```
SetSortOrder("Name,ASC")
Scan()
```

SetSourceDirectory

Specifies the path to the images folder. This action must precede the Scan action.

Syntax

()

Parameters

String value of the directory's name and path. Instead, you can use a Smart Parameter, such as @APPPATH, to establish the name and path of the Source Directory.

Returns

False, if the parameter is blank or the directory does not exist. Otherwise, **True**.

If the source directory does not exist, the batch status is set to abort.

Level

Any level but usually the batch level.

Details

This action indicates the name and path of the directory that contains the Image files to be scanned.

This is a required action. A task that employs a vScan rule cannot process images unless it has this key locator. This action must be called before the scan action.

Example

In the following example, the full path to the images directory is coded directly into the application rules.

```
SetSourceDirectory("c:\ParentDirectory\Application\Images")
```

In the following example, @APPPATH obtains the vscan directory from the application service. This makes the application more flexible because the same application that is installed in two different environments, such as test and production, can use two different input directories. The subdirectory Input is appended to the path.

```
SetSourceDirectory("@APPPATH(vscanimagedir)+\+Input")
```

Web Services actions

Use the Web Services actions to do utility functions for Datacap Web Client.

Datacap Web Client provides interfaces for batch creation, file transfer, queueing, monitoring, and retrieving output from Datacap processing in a service oriented architecture (SOA). The Web Services library includes actions to set the url, execute connection requests, download files, set header values, and upload files.

“WsCompare”

“WsDownloadFile” on page 980

“WsExecute” on page 980

“WsGetLineItems” on page 981

“WsGetValue” on page 982

“WsMessageLineItemPropertyAdd” on page 982

“WsMessageLineItemPropertySet” on page 983

“WsSetHeaderValue” on page 983

“WsSetMessageLineItemProperty” on page 984

“WsSetMessageProperty” on page 984

“WsSetMessageTemplate” on page 985

“WsSetResponseNamespace” on page 986

“WsUploadFile” on page 986

“WsUrlReplaceValue” on page 987

“WsUrlSet” on page 987

WsCompare

Compares the DCO value at the specified smart parameter path with the value in the response at the specified xpath location.

Member of namespace

WebServices

Syntax

```
WsCompare ()
```

Parameters

smartParameterPath

Type: string

Smart parameter path to DCO object to compare. If blank the text field of the current DCO object is used.

xPath Type: string

The xpath location in the response XML to compare. If the path is not found or the response is not XML the entire response is used.

Returns

True if the values are identical. **False** if the values are not identical.

Level

Any

Details

Example

```
WsCompare("@F, //XPath")
```

WsDownloadFile

Downloads a file from a stream to the specified file path.

Member of namespace

WebServices

Syntax

```
WsDownloadFile ()
```

Parameters

targetPath

Type: string

File path or smart parameter path to file path to use.

Returns

Always **True**.

Level

Any

Details

Uses the URI specified by WsUriSet.

Example

```
WsUriSet(GET, "http://server/downloadFile/fileName/fileExtension")  
WsUriReplaceValue(@ID, fileName)  
WsUriReplaceValue(@ID, fileExtension)  
WsDownloadFile("@PILOT(BATCHDIR)+\\+@D.DocID+,.tif")
```

WsExecute

Executes the request. The HTTP response code is saved to the calling DCO object node in a variable called HttpResponse.

Member of namespace

WebServices

Syntax

WsExecute ()

Parameters

smartParameterPath

Type: string

Optional. Smart parameter path to DCO object to save responses to. If blank the response is not be saved but can still be accessed with other actions.

xPath Type: string

Optional. xpath to the value to save to the DCO. Uses SelectSingleNode.

Returns

True if the connection was successful. **False** if the connection could not be established.

Level

Any

Details

Example

```
WsExecute("D.WebResponse")
```

WsGetLineItems

Creates DCO line item fields using the web service response.

Member of namespace

WebServices

Syntax

WsGetLineItems ()

Parameters

smartParameterPath

Type: string

Smart parameter path to details DCO object that is the parent to the line item field. If blank the text field of the current DCO object is used.

xPath Type: string

The xpath node location in response of the parent to each line item.

xmlTemplate

Type: string

Template of the response XML that maps attributes or values to DCO fields.

Returns

Always **True**.

Level

Any

Details

Example

```
WsGetLineItems("@F,//XPath")
```

WsGetValue

Populates the DCO value at the specified smart parameter path with the value in the response at the specified xpath location.

Member of namespace

WebServices

Syntax

```
WsGetValue ()
```

Parameters

smartParameterPath

Type: string

Smart parameter path to DCO object to populate. If blank the text field of the current DCO object is used.

xPath Type: string

The xpath location in the response XML to use to populate the DCO.

Returns

Always **True**

Level

Any

Details

Example

```
WsGetValue("@F,//XPath")
```

WsMessageLineItemPropertyAdd

Adds a new copy of the node specified by the xpath parameter.

Member of namespace

WebServices

Syntax

```
WsMessageLineItemPropertyAdd ()
```

Parameters

xPath Type: string

The xpath node location in template XML file to duplicate.

Returns

Always **True**.

Level

Any

Details

Example

```
WsMessageLineItemPropertyAdd("//xPath")
```

WsMessageLineItemPropertySet

Adds a new copy of the node specified by the xPath parameter.

Member of namespace

WebServices

Syntax

```
WsMessageLineItemPropertySet ()
```

Parameters

smartParameterPath

Type: string

Smart parameter path to DCO object to use to populate. If blank the text field of the current DCO object is used.

xPath Type: string

The xpath node location in the just added node. The added node is loaded as an XML document and is the root of the xpath.

Returns

Always **True**.

Level

Any

Details

Updates the node last added with WsAddMessageLineItemProperty.

Example

```
WsMessageLineItemPropertyAdd("//xPath")  
WsMessageLineItemPropertySet("@F,//xPath")
```

WsSetHeaderValue

Use to specify header properties such as content type.

Member of namespace

WebServices

Syntax

`WsSetHeaderValue ()`

Parameters

name Type: string
 Name of header key.

value Type: string
 Value of header property.

Returns

Always **True**.

Level

Any

Details

Example

```
WsSetHeaderValue(Content-type, application/xml)
```

WsSetMessageLineItemProperty

Use to set value in message body.

Member of namespace

WebServices

Syntax

`WsSetMessageLineItemProperty ()`

Parameters

value Type: string
 Value to use as the replacement. Supports smart parameters.

xPath Type: string
 XPath to locate nodes. Inner text is changed. Uses `SelectNodes`.

Returns

Always **True**.

Level

Any

Details

Example

```
WsSetMessageLineItemProperty(@F, //nodeName)
```

WsSetMessageProperty

Use to set value in message body.

Member of namespace

WebServices

Syntax

`WsSetMessageProperty ()`

Parameters

value Type: string

Value to use as the replacement. Supports smart parameters.

xPath Type: string

xPath to locate nodes. Inner text is changed. Uses `SelectNodes`.

Returns

Always **True**.

Level

Any

Details

Example

```
WsSetMessageProperty(@F, //nodeName)
```

WsSetMessageTemplate

Specify file to load as message body.

Member of namespace

WebServices

Syntax

`WsSetMessageTemplate ()`

Parameters

fileName

Type: string

File path and name to use.

Returns

Always **True**.

Level

Any

Details

Example

```
WsSetMessageTemplate(c:\file.xml)
```


WsSetResponseNameSpace

Adds the namespace to the response XML.

Member of namespace

WebServices

Syntax

```
WsSetResponseNameSpace ()
```

Parameters

key Type: string
 Name of the namespace attribute.

value Type: string
 Value of the namespace attribute.

Returns

Always **True**.

Level

Any

Details

Example

```
WsSetResponseNameSpace("@F,//XPath")
```

WsUploadFile

Uploads the specified file as a stream.

Member of namespace

WebServices

Syntax

```
WsUploadFile ()
```

Parameters

fileSource
 Type: string
 File path or smart parameter path to file path to use.

Returns

Always **True**.

Level

Any

Details

Uses the URI specified by `WsUrlSet`.

Example

```
WsUrlSet(GET, "http://server/uploadFile/fileName/fileExtension")
WsUrlReplaceValue(@ID, fileName)
WsUrlReplaceValue(@ID, fileExtension)
WsUploadFile("@PILOT(BATCHDIR)+\\+@D.DocID+.+.tif")
```

WsUrlReplaceValue

Use to specify variables in your url.

Member of namespace

WebServices

Syntax

`WsUrlReplaceValue ()`

Parameters

source Type: string

Value to use as the replacement.

target Type: string

String to be replaced in the url. All instances will be replaced

Returns

Always **True**.

Level

Any

Details

Example

```
WsUrlReplaceValue(@B.ID, batchID)
```

WsUrlSet

Set the URL to be used for following requests.

Member of namespace

WebServices

Syntax

`WsUrlSet ()`

Parameters

verb Type: string

HTTP verb to use. Default is GET. (POST, GET, PUT, DELETE)

url Type: string

Returns

Always **True**.

Level

Any

Details

Example

```
WsUrlSet(GET, "http://server/endpoint")
```

Zones actions

Use the Zones actions to work with the zones that define the position of each field on the page.

You can read zone information from a fingerprint (CCO) file, update the zone position information in the runtime hierarchy, and locate the recognition text for a specific zone. You can also assign values to fields in the runtime hierarchy, locate repeating data blocks, and more.

- "AdjustZonesToImageOffset" on page 989
- "AnchorPage" on page 989
- "CalculateLocalOffset" on page 990
- "CreateBlockCCO" on page 990
- "FindBlocks_WhiteSpace" on page 991
- "FindDataBlocks" on page 991
- "FindLineItems" on page 992
- "FindRegExBlocks" on page 992
- "FindZoneLineItems" on page 993
- "GetZoneText" on page 994
- "InheritParentPosition" on page 994
- "LoadBlockCCO" on page 995
- "LoadZones" on page 995
- "MCCOPositionAdjust" on page 996
- "MergeZones" on page 997
- "PadZone" on page 997
- "PopulateZNField" on page 998
- "PopulateZNLineItemField" on page 998
- "ReadZones" on page 999
- "RegisterPage" on page 999
- "ScanDetails" on page 1000
- "ScanDetailsByLines" on page 1001
- "ScanDetailsByVSpace" on page 1001
- "ScanLineItem" on page 1002
- "SetEOL" on page 1002
- "SetEOL_CRLF" on page 1003
- "ZoneBOTTOM_ImageBottom" on page 1003
- "ZoneBOTTOM_LowerBound" on page 1004

"ZoneBOTTOM_UpperBound" on page 1005
"ZoneImage_SaveAs" on page 1005
"ZoneLEFT_ImageLeft" on page 1006
"ZoneLEFT_LeftBound" on page 1007
"ZoneLEFT_RightBound" on page 1007
"ZoneRIGHT_ImageRight" on page 1008
"ZoneRIGHT_LeftBound" on page 1008
"ZoneRIGHT_RightBound" on page 1009
"ZoneTOP_ImageTop" on page 1009
"ZoneTOP_LowerBound" on page 1010
"ZoneTOP_UpperBound" on page 1010

AdjustZonesToImageOffset

Offsets fields on the current image in response to zone criteria in the CCO file of the source page - after the image has been offset.

Syntax

()

Parameters

None.

Returns

False if the action cannot locate the CCO file of the current page. Otherwise, **True**.

Level

Page level.

Details

Offsets fields on the current image in response to zone criteria in the CCO file of the source page - after the image has been offset.

Example

```
AdjustZonesToImageOffset()
```

AnchorPage

Finds the Anchor field on a source page, and uses the Anchor field's coordinates to locate and offset the page's other zoned fields.

Syntax

()

Parameters

None.

Returns

False if the action cannot find the Anchor field on the page. Otherwise, **True**.

Level

Page level.

Details

Finds the Anchor field on a source page, and uses the coordinates of the Anchor field to locate and offset the other zoned fields on the page.

Example

```
AnchorPage()
```

CalculateLocalOffset

Calculates the X and/or Y offset amount for the calling field.

Syntax

()

Parameters

X and/or Y

The action uses these parameters to calculate a new parent page Image_Offset value by comparing the fingerprint zone for the calling field against the current field zone.

Returns

Always **True**.

Level

Field only.

Details

Calculates the X and/or Y offset amount for the calling field.

Example

```
CalculateLocalOffset("XY")
```

CreateBlockCCO

Creates a temporary in memory CCO object, containing only the words and lines in the calling fields zone position - using the source page's CCO file.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

Field level.

Details

Useful when searching for images where the data is located asymmetrically across the page. Using this action focuses the CCO to only look at the words and lines within the defining zone in all future searches for this page.

This does not affect the run time CCO. Reloading the page reloads the CCO bound to the page, releasing this temporary CCO.

Example

```
CreateBlockCCO()
```

FindBlocks_WhiteSpace

Uses a vertical white space (pixels) to find blocks of data within the current source page. Returns each block's position assigned to a series of repeating fields based on the first child field of the calling object.

Syntax

(Strparam)

Parameters

A single parameter indicating the number of pixel between lines.

Returns

False if the action cannot divide or create any child field/zones. Otherwise, **True**.

Level

Field having 1 child field.

Details

Creates a child field with the position of each zone divided out of the CCO of the calling page.

Example

```
FindBlocks_WhiteSpace("27")
```

FindDataBlocks

Uses Start and End key words to find blocks of data within the current source page.

Syntax

()

Parameters

Key word Start Value, and its End Value.

Returns

True if the action can locate the Data block indicated by the parameter. Otherwise, **False**.

Level

Page level.

Details

Uses Start and End key words to find blocks of data within the current source page. Returns each block's position assigned to a series of repeating fields based on the first child field of the calling object. If the Fingerprint file (.cco) of the page has a Line position be sure to use action GoFirstLine() to set the Line position to the first word on the first line in the current zone.

Example

```
GoFirstLine()  
FindDataBlocks("FROM,THRU")
```

FindLineItems

This action is replaced by the FindZoneLineItems action.

Syntax

()

Parameters

5 comma separated parameters. See the help for FindZoneLineItems for details.

Details

This action should not be used because it is scheduled to be removed in future versions. It is replaced by the FindZoneLineItems action.

FindRegExBlocks

Uses a Regular Expression to find blocks of data within the current source page. Returns the position of each block that is assigned to a series of repeating fields based on the first child field of the calling object.

Syntax

()

Parameters

Regular Expression that contains the data block's Start Value, and its End Value. Parameter is a two part comma separated value of the Values to look for in the current CCO.

1. StartValue
2. EndValue

Start and End values can be adjusted up or down by xLines using the 3rd (adj top) and 4th (adj bottom) CSV positions.

Returns

True if the action can locate the Data block indicated by the parameter. Otherwise, **False**.

Level

Page level.

Details

Uses a Regular Expression to find blocks of data within the current source page.

Attention: Locate.rra is required for this action.

Example

```
FindRegExBlocks("/bFROM/b,/bTHRU/b")
```

FindZoneLineItems

Uses settings for zones to assemble a portion of the current page, limited to Line Item Detail.

Syntax

()

Parameters

1. True or False

- **True** if the action is to create Line-item fields and their sub-fields.
- **False** if the action is to create Line-item fields only.

2. True or False (OPTIONAL: defaults to True)

- **True** if the action is to analyze both the calling zone and the word and line relationships based only on the zone contents that are found in the search.
- **False** if the action is to use the existing word and line relationships (CCO for image).

3. 1 or 2 (OPTIONAL: Fill Sub fields option - defaults to 1)

- **1** - fill by CCO word; if the CCO words overlap by a certain intersection value then fill sub field with its value. Also has sub setting of a colon followed by a group words value. For example, 1:1.5 treats words 1.5 character spaces apart or closer as one word.
- **2** - fill by CCO character; if the CCO characters overlap by a certain intersection value then fill sub field with the character value.

4. 0 to 100 (OPTIONAL: Percent offset adjustment - defaults to 0) - moderates the flexibility for the left offset based on the overall length of the line. Use larger values for shorter line items.

5. .01 to .99 (OPTIONAL: Intersection Ratio - defaults to .25) - changes the intersection ratio of Words and Lines used to determine creation and value of Lineitems and their subfields.

Returns

False if there is no zone or position information for the Line Item block and its subfields, or if the parent block does not contain Line Item children. Otherwise, **True**.

Level

Field having 1 child field.

Details

Uses settings for a previously-established zone covering a block of Line Item Detail, and zones for individual Line Items, to assemble a portion of the current page, limited to Line Item Detail.

Example

```
FindLineItems("True","", "", "", "", "")
```

This action retrieves zonal information from the fingerprint about a block of Line Item Detail, individual rows, and their fields and subfields. The action applies this data to the full-length current page. It creates a file with an extension of cco (fingerprint format file) containing just Line Item Detail for use during batch processing. This file can be discarded after batch processing has finished.

GetZoneText

Retrieves the text in a zoned object of the current page.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

Field level.

Details

Retrieves the text in a zoned object of the current page.

Example

```
GetZoneText()
```

This action assumes that you have established zone parameters for this Field object of the Document Hierarchy.

InheritParentPosition

Provides the bound child object of the Document Hierarchy with the zone parameters of a parent object identified by the parameter.

Syntax

()

Parameters

Case sensitive string value of the name of the parent object. Details, for example, is the name of a parent Field object in the Invoice application. If the bound object is LINEITEM, the action provides the zone parameters of the parent to this child object.

Returns

False if the action cannot locate the parent field. Otherwise, **True**.

Level

Field level.

Details

Provides the bound child object (a subfield of another field) of the Document Hierarchy with the zone parameters of a parent object identified by the parameter.

Example

```
InheritParentPosition("Details")
```

LoadBlockCCO

Loads the CCOBlock set up by a previous CreateBlockCCO action. That action assigns the block's location to the CCOBlock variable of the current Page object.

Syntax

```
()
```

Parameters

None.

Returns

False if the block cannot be found. Otherwise, **True**.

Level

Page level.

Details

Loads the CCOBlock set up by a previous CreateBlockCCO action. That action assigns the block's location to the current Page object's CCOBlock variable.

Example

```
CreateBlockCCO()  
LoadBlock()
```

LoadZones

Same as the ReadZones action, except that it loads position information for the specified fingerprint ID.

Syntax

```
()
```

Parameters

Fingerprint ID.

Returns

Always **True**.

Level

Page or Field level.

Details

Loads position information for each node in the calling object and children. Pre-adjusts these values based on offset information stored in an *Image_Offset* variable at any node level.

Position information is based on the setup DCO position for the fingerprint ID passed as a parameter. The offset value is applied to all child objects of a node where an *Image_Offset* variable is found; unless overwritten by a child node also having an *Image_Offset* value to apply.

MCCOPositionAdjust

Combines extra pages of a multi-page document into the CCO file for the first page.

Syntax

()

Parameters

None.

Returns

False if the current document does not consist of more than one source page, or if a page to be merged did not have an associated CCO file created. Otherwise, **True**.

Level

Page level.

Details

MCCOPositionAdjust is an important action for applications using merged multiple page documents. It normalizes position coordinates populated to the DCO when the data is generated from a document using a merged CCO. The coordinates are adjusted relative to the page and also sets the DCO ImageFile property to the name of the TIFF that the data was found on.

Attention: All files to be merged must have an associated CCO file.

This action is used in conjunction with action MergeCCO_ByType from the Autodoc library in a prior ruleset.

Example

```
MCCOPositionAdjust()
```

MergeZones

Merges the zone from calling field with zone of DCO fields passed as smart parameters.

Syntax

```
()
```

Parameters

A CSV of smart parameter strings indicating the DCO fields to merge zones with.

Returns

False if the calling field does not have a valid position value. Otherwise, **True**.

Level

Field level.

Details

Merges calling field's position with each passed dco position.

Example

```
MergeZones("@P\5PAddTel,@P\5PAddZip,@P\2PatName")
```

PadZone

Pads the zone by the value passed. Number of CSV passed values varies padding value by vector.

Syntax

```
()
```

Parameters

A CSV of smart parameter strings that indicate the pixels value by which to pad the calling field.

Returns

False, if the calling field does not have a valid position that is not a zero.

False, if there are zero or more than four CSV parameters.

False, if the Smart Parameter value returns as a non-numeric.

Otherwise, **True**.

Level

Field level.

Details

Pads calling field's position with each value passed. Number of passed values varies padding vector as follows:

1. Pads Left, Top, Right, and Bottom by this value (that is, expand if positive)
2. Pads as X,Y (X: pad to Left and Right, Y: pad to Top and Bottom)
3. Pads as Left, Top, Right, and Bottom. (missing values are treated as zero).
4. Pads as Left, Top, Right, and Bottom.

Example

`PadZone("15")` Pads 15 pixels to the Left, Right, Top and Bottom position.

`PadZone("5,10")` Pads 5 pixels to the Left and Right position, and 10 to the Top and Bottom position.

`PadZone("-5,0,12,25")` Pads -5 pixels to the Left, 0 to the Right, 12 to the Top and 25 to the Bottom position.

PopulateZNField

Puts the recognition data from the CCO file that lies in the zone boundaries of the field into the current field in the page data file.

Syntax

`()`

Parameters

None.

Returns

Always **True**.

Level

Field level.

Details

This action populates the field value with the full-page recognition results (in the CCO) that fall within the boundaries of the field zone.

This action should be used with fields such as the Total or Number field in the Invoice application.

Example

`PopulateZNField()`

PopulateZNLinItemField

Populates the page data file with the recognized value in the zone for the current line item child field. Assign this action to each line item child field in the document hierarchy.

Syntax

`()`

Parameters

None.

Returns

True if the bound Field object is the child of a parent Field object, and if a fingerprint's .cco file containing block information can be found. Otherwise, **False**.

Level

Field level.

Details

Populates the fingerprint's Data file with the recognized value contained inside the zone of a child Field object of a LINEITEM parent field. This action should only be used with subfields of the LINEITEM field (ItemID, ItemDesc, Quantity, Price) in the Invoices application.

Example

```
PopulateZNLineItemField()
```

ReadZones

Loads the position information for the current object and its children from the document hierarchy (setup DCO). Adjusts the position of each object by using any *Image_Offset* value.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

Page or Field level.

Details

Loads position information for each node in the calling object and its children. Pre-adjusts these values based on offset information stored in an *Image_Offset* variable at any node level.

Position information is based on the setup DCO position for the fingerprint ID of the parent page. The offset value is applied to all child objects of a node where an *Image_Offset* variable is found; unless overwritten by a child node also having an *Image_Offset* value to apply.

RegisterPage

Locates specially marked fields and adjusts their vertical zone positions to compensate for any drift.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

Page level.

Details

RegisterPage adjusts the y positions of special fields to coordinate zone positions with the current word location of the fingerprint. Searches the calling page object for run time field names beginning with the word 'Anchor' (case sensitive). Of these fields a search is performed for variables named Word followed by the calling fingerprint ID (example: 'Word555'). The value of the variable retrieved is search for in the current pages CCO, and if found the field's zone position is 'y' axis adjusted for any word 'drift' for the given field value.

Example

```
RegisterPage()
```

ScanDetails

Searches a line item grid object for line items. You assign this action to the grid region in the document hierarchy.

Syntax

()

Parameters

None.

Returns

True if the bound Field object contains lines of data. Otherwise, **False**.

Level

Field level.

Details

Searches the DETAILS Field object's zone for instances of a LINEITEM Field object. Captures the data in the rows of a Line Item table, row by row.

This action captures all potential LINEITEM rows within the parent DETAILS field, even rows that may not fit your criteria for content.

Attention: You must run additional rulesets to delete ineligible Line Items. In the APT application, these are the Clean and Filter rulesets.

Example

```
ScanDetails()
```

ScanDetailsByLines

Searches a line item grid object for line items, where each line item consists of the specified number of rows.

Syntax

```
()
```

Parameters

Number of lines in each lineitem.

Returns

True if the bound Field object contains lines of data. Otherwise, **False**.

Level

Field level.

Details

Searches the DETAILS Field object's zone for instances of a LINEITEM Field object. Captures the data based on the number of lines (parameter) in a Line Item table, row by row.

This action captures all potential LINEITEM rows (consisting of 2 lines each) in the parent DETAILS field, even rows that may not fit your criteria for content.

Attention: You must run additional rulesets to delete ineligible Line Items. In the Invoices application, these are the Clean and Filter rulesets.

Example

```
ScanDetailsByLines("2")
```

ScanDetailsByVSpace

Searches a line item grid object for line items, where each line item is defined as the specified height in pixels.

Syntax

```
()
```

Parameters

Number of vertical pixels in each lineitem.

Returns

True if the bound Field object contains lines of data. Otherwise, **False**.

Level

Field level.

Details

Searches the DETAILS Field object's zone for instances of a LINEITEM Field object. Captures the data based on the number of vertical pixels (parameter) in a Line Item table, row by row.

Example

```
ScanDetailsByLines("45")
```

This action capture all potential LINEITEM rows (consisting of 45 pixels lines each) in the parent DETAILS field, even rows that might not fit your criteria for content.

Attention: You must run additional rulesets to delete ineligible Line Items. In the Invoices application, these are the Clean and Filter rulesets.

ScanLineItem

Searches a line item object for fields. You assign this action to each line item in the document hierarchy.

Syntax

```
()
```

Parameters

None.

Returns

False if not called from a Field. Otherwise, **True**.

Level

Field level.

Details

This action captures each subfield within the parent LINEITEM row.

Example

```
ScanLineItem()
```

SetEOL

Sets the End of Line character that will be used to separate data from a zone that contains multiple lines of text.

Syntax

```
()
```

Parameters

The End of Line separator character.

Returns

Always **True**.

Level

All.

Details

If this action is not used, the default character is a space.

Example

```
SetEOL("|")
```

This example sets the End of Line character to the '|' (pipe) character. A capture zone with two lines of text contains the captured value separated by this new character.

SetEOL_CRLF

Sets the End Of Line character that are used to separate data from a zone with multiple lines of text.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

All.

Details

Sets the End Of Line character that is used to separate data from a zone with multiple lines of text to the carriage return and Line Feed characters: ASCII values 13 and 10.

Example

```
SetEOL_CrLf()
```

This example sets the End of Line character to the Carriage Return and Line Feed characters. A capture zone with two lines of text contains the captured value separated by these characters.

ZoneBOTTOM_ImageBottom

Uses the lower boundary of the current image to specify the bottom boundary of the zone position of the current field in the page data file.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

Field level.

Details

Uses the bottom boundary of the current field's IMAGE to define the bottom edge of the field's ZONE.

Attention: An image of a field is often larger than a zone of a field , and almost always larger than the field's word.

Example

```
ZoneBOTTOM_ImageBottom()
```

ZoneBOTTOM_LowerBound

Uses the lower boundary of the current word that is located by using the actions in the Locate library to specify the bottom boundary of the zone position of the current field in the page data file.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

Field level.

Details

Uses the lower boundary of the current field's ZONE to specify the bottom of the field's WORD. This can result in a taller or shorter word, depending on the location of the zone's boundary.

Example

```
ZoneBOTTOM_LowerBound()
```

If you find that an unacceptably high percentage of values for a particular field are entered a line too high, consider the use of this action in a follow-up Locate rule to search for the misplaced value.

ZoneBOTTOM_UpperBound

Uses the upper boundary of the current word that is located by using the actions in the Locate library to specify the bottom boundary of the zone position of the current field in the page data file.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

Field level.

Details

Uses the lower boundary of the current field's ZONE to specify the top of the field's WORD. This can result in a taller or shorter word, depending on the location of the zone's lower boundary.

Example

```
ZoneBOTTOM_UpperBound()
```

This action is helpful when experience shows that a certain number of values in this field extend beyond the word's upper limit. A follow-up Locate rule with this action can find those values.

ZoneImage_SaveAs

Saves the current Objects Zone area of an image as a separate Image file.

Syntax

()

Parameters

A string that defines the file name.

You can use these options to format the file name:

- **+@BATCHID** : Adds the BatchID to the Zone Image File Name
- **+@ID** : Adds the Object ID to the Zone Image File Name
- **+@STATUS** : Adds the Object Status to the Zone Image File Name
- **+@TYPE** : Adds the Object Type to Zone Image File Name
- **+@DATE+mm/dd/yyyy** : Adds a Date Stamp to the Zone Image File Name, the required trailing date format argument shows as the default, also '+' can be used
- **+@TIME+HH:MM:SS** : Adds a Time Stamp Value to the Zone Image File Name, the required trailing time format argument shows as the default, also '+' can be used

- **+@VALUE** : Adds Object Text to Zone Image File Name
- **+#name** : Appends the value of a child name to the Image File Name

Attention: **+#name** appends the value of child name to the image name.

Returns

False if an image cannot be saved. Otherwise, **True**.

Level

Page or Field level.

Details

Saves the current Objects Zone area of an image as a separate Image file. The Zone Image file is always placed in the Batches directory; saving to the current page creates a multi-page TIFF file with the additional image as its zone.

Example

```
ZoneImage_SaveAs("SAMMY+@TYPE+@DATE+JJJ")
```

As part of a rule bound to the Details Field object of the Document Hierarchy, the action produces this Image File Name: ...\
SAMMYDETAILS243.tif.

ZoneLEFT_ImageLeft

Uses the left boundary of the current image to specify the left boundary of the zone position of the current field in the page data file.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

Field level.

Details

Uses the left boundary of the current field's image to define the left edge of the field's zone. This can extend or contract the width of the zone, depending on the placement of the left side of the image.

Attention: A field's image is often larger than a field's zone, and almost always larger than the field's word.

Example

```
ZoneLEFT_ImageLeft()
```

ZoneLEFT_LeftBound

Uses the left boundary of the current image to specify the left boundary of the zone position of the current field in the page data file.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

Field level.

Details

Uses the left boundary of the current field's word to define the left edge of the field's zone. This extends the width of the zone to accommodate the current word.

Example

```
ZoneLEFT_LeftBound()
```

If the field's recognized values reach beyond the zone's limits, you can use this action to expand the zone appropriately.

ZoneLEFT_RightBound

Uses the right boundary of the current image to specify the left boundary of the zone position of the current field in the page data file.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

Field level.

Details

Uses the right boundary of the current field's word to designate the left edge of the field's zone. This extends the width of the zone to accommodate the current word.

Example

```
ZoneLEFT_RightBound()
```

Attention: The image of a field is often larger than the field's zone, and almost always larger than the field's word.

ZoneRIGHT_ImageRight

Uses the right boundary of the current image to specify the right boundary of the zone position of the current field in the page data file.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

Field level.

Details

Uses the right boundary of the current field's image to define the right edge of the field's zone. This can extend or contract the width of the zone, depending on the placement of the right side of image.

Attention: A field's image is often larger than a field's zone, and almost always larger than the field's word.

Example

```
ZoneRight_ImageRight()
```

ZoneRIGHT_LeftBound

Uses the left boundary of the current word* to specify the right boundary of the zone position of the current field in the page data file.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

Field level.

Details

Uses the left boundary of the current field's word to designate the right edge of the field's zone. This action also excludes the word from the zone.

Example

```
ZoneRight_LeftBound()
```

This action can clean the zone by eliminating the word it contains.

ZoneRIGHT_RightBound

Uses the right boundary of the current word* to specify the right boundary of the zone position of the current field in the page data file.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

Field level.

Details

Uses the right boundary of the current field's word to designate the right edge of the field's zone.

This is a convenient action if a percentage of the field's entered values extend beyond the right edge of the zone.

Example

```
ZoneRIGHT_RightBound()
```

ZoneTOP_ImageTop

Uses the top boundary of the current image to specify the top boundary of the zone position of the current field in the page data file.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

Field level.

Details

Uses the upper boundary of the current field's image to designate the top edge of the field's zone. This can extend or contract the width of the zone, depending on the placement of the left side of the image.

Example

```
ZoneTOP_ImageTop()
```

ZoneTOP_LowerBound

Uses the lower boundary of the current word that is located by using the actions in the Locate library to specify the top boundary of zone position of the current field in the page data file.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

Field level.

Details

This action uses the upper boundary of the current field's zone to indicate the bottom of the field's word. This action can expand or contract the height of the zone, depending on the placement of the word's lower boundary within the current page.

Example

```
ZoneTOP_LowerBound()
```

If you find that a high percentage of values for a particular field are entered a line too high, you can use this action in a follow-up Locate rule to search for the misplaced value.

ZoneTOP_UpperBound

Uses the upper boundary of the current word that is located by using the actions in the Locate library to specify the top boundary of the zone position of the current field in the page data file.

Syntax

()

Parameters

None.

Returns

Always **True**.

Level

Field level.

Details

This action is helpful when you know that a certain number of values in this field will be shorter than the zone's height. A follow-up Locate rule with this action will find those values.

Example

```
ZoneTOP_UpperBound()
```

Application specific actions

The Datacap applications use actions that are specific to them.

“Medical Claims actions”

“Datacap Accounts Payable actions” on page 1014

Medical Claims actions

The following are actions specific to the Medical Claims application.

“4010Common”

“4010Institutional” on page 1012

“4010Professional” on page 1012

“5010Common” on page 1012

“5010Institutional” on page 1012

“5010Professional” on page 1012

“MC_Identify” on page 717

“MC_Validation” on page 1013

4010Common

The set of *4010Common* actions are common to both Institutional and Professional medical claim forms.

Action	Description
Load4010Settings	Load settings from the .ini file that is passed as a parameter. You can change default values and customize how the 4010 form is built. Smart parameter enabled.
Merge4010	Zips the 4010 file path, file name, and file extension settings into one file. Smart parameter enabled.

4010Institutional

This action is specific to the Institutional 4010 form.

Action	Description
Institutional_4010	Used to build a single 4010 file for each Institutional claim. Use the action on a page or a document. The action allows you to create a single export page file for each document.

4010Professional

This action is specific to the Professional 4010 form.

Action	Description
Professional_4010	Used to build a single 4010 file for each Professional claim. Use the action on a page or a document. The action allows you to create a single export page file for each document.

5010Common

The set of *5010Common* actions are common to both Institutional and Professional medical claim forms.

Action	Description
Load5010Settings	Load settings from the .ini file that is passed as a parameter. You can change default values and customize how the 5010 form is built. Smart parameter enabled.
Merge5010	Zips the 5010 file path, file name, and file extension settings into one file. Smart parameter enabled.

5010Institutional

This action is specific to the Institutional 5010 form.

Action	Description
Institutional_5010	Used to build a single 5010 file for each Institutional claim. Use the action on a page or a document. The action allows you to create a single export page file for each document.

5010Professional

This action is specific to the Professional 5010 form.

Action	Description
Professional_5010	Used to build a single 5010 file for each Professional claim. Use the action on a page or a document. The action allows you to create a single export page file for each document.

MC_Identify

Use the MC_Identify actions to identify claim forms in a batch.

The MC_Identify actions identify the medical claim forms that are processing in the batch.

“AutoField” on page 717

“FindFields” on page 718

“ReadDCOSetup” on page 719

“ReadPageSetup” on page 719

“SetFormType” on page 720

“SetMaxTolerantDistance” on page 721

MC_Validation

The actions in the *MC_Validation* library are used specifically with Professional and Institutional claim forms. The actions allow for the manipulation of batches, batch hierarchies, and data.

Action	Description
AddCenturyTo2DigitYear	Converts two-digit Year values to four-digit Year values.
AddToDetailErrorMsg	Adds the value to the existing value for the page variable ErrorMessage .
AddToErrorMsg	Adds the value to the existing value for the page variable ErrorMessage .
CalculateHCFALineCharges	Calculates charges for HCFA service lines.
CalculateResult	Determines if the calculation provided by the parameter is True or False .
CalculateUB	UB04 action that determines if the amounts in the 47ttchg fields sum correctly.
CalculateUBLLineCharges	Calculates charges for UB service lines.
CheckDocID	Checks document IDs and updates them to the proper format.
ClearErrorMsg	Clears the value of the page variable ErrorMessage .
CommonParseAddress	Parses addresses in certain fields in HCFA and UB04 forms into appropriate subfields.
CommonValAddress	Validates address values.
ConvertHyphen	Removes spaces, commas and hyphens from the current field.
FilterPID	Filters qualifier from attending physician field for UB04 claims.
FormatFieldLengths	Truncates length and sets the last character to a low confidence recognition of the field.
InheritSnippets	Assigns the snippet position information of the current Field object to the Field objects specified in the parameter.
MC_ReadZones	Adjusts autofield based OMR field zone positions on the calling page.
Parse31aPhSig	Parses field 31aPhSig of the HCFA application.

Action	Description
Parse58ainsnm	Parses field 58ainsnm of the UB04 application.
Parse58binsnm	Parses field 58binsnm of the UB04 application.
Parse58cinsnm	Parses field 58cinsnm of the UB04 application.
Parse82name	Parses field 82name of the UB92 application.
Parse83aname	Parses field 83aname of the UB92 application.
Parse83bname	Parses field 83bname of the UB92 application.
ParseLastFirstIniNames	Parses the name information in the first line of an address superfield.
ParseNDC	An action that detects and parses NDC data elements from the calling field value.
ParseUB_Eighties	Special action to parse fields 82 (Attending Physician ID) and 83 (Other Physician ID) of the UB92 application.
PopulateFromField	Copies the value from the field specified by the parameter into the current field.
SetConf	Sets confidence string for a field.
SetOriginalTIF	Replaces current working TIF file.
StripTrailingAlpha	Removes all alpha characters from the captured value, except any in the first character position.
TransformLI	Remaps the Line Item Table fields into a hierarchical structure.
UpdateCredentialList	
ValidateNPI	Validates the NPI value by evaluating the 10 digits in the value uses a modified LUHN checkdigit algorithm.
ValidateStateMil	Checks to see if the value in the field represented by the bound Field object is a valid two-character State abbreviation.
ValProcedureCode	Validates the Procedure Code fields of a HCFA-1500 form.
ValRequiredGroup	Checks that all fields in a designated group are filled with data.

Datacap Accounts Payable actions

Use the Datacap Accounts Payable actions when you work with batches of invoices on APT.

Datacap Accounts Payable actions are divided into the following categories:

Category	Description
Localization	Detects the localization and decimal separator settings of a workstation and changes the rules execution and data for the documents that you are processing if needed.
Custom	Customizes the fields in the invoice image to accommodate locale requirements and other things.
Concatenate line values	Merges line item values into page variable and page field values into document variables.
Documents	Creates and manages the documents in the batch.
FlexID	Starts the FlexID panel to manually identify the pages in the document batch.
Intellocate learning	Determines the new zones in the invoice image and adds these zones to the DCO.
Page ID	Separates and processes the page objects in the documents.
PreVerify setup	Sets the label variable that is used by Datacap Web Client and Datacap Desktop to display the label on each field.
Redaction	Redacts selected information in your invoice image for security purposes.

Embedded help is provided in Datacap Studio for all of the Datacap Accounts Payable actions. To access the embedded help, select an action in the **Actions Library** tab and click **information**. For detailed information, including information about parameters, refer to the embedded help.

“APT_Localization”

“APTCustom” on page 1016

“ConcatLineValues” on page 1017

“Documents” on page 1017

“FlexID” on page 1017

“Intellocate_Learning” on page 1017

“PageID” on page 1018

“PreVerifySetup” on page 1018

“Redaction” on page 1018

APT_Localization

Use the APT_Localization actions to work with currency values when you process documents generated from a locale that is different from the locale of your workstation.

The APT_Localization actions are described in the following table.

Action	Description
CheckAndFixLocalDecimal	Checks and fixes issues when the decimal in a currency value is not recognized and is replaced with a comma or a space. Also converts the decimal separator to use the local setting.
ConvertToLocalDecimal	Reads the computer settings and sets the decimal place.
IsFieldLocalCurrency	Queries the local decimal separator and checks to see whether the field contains a local currency value.
IsLocalDecimalSeparator	Checks to see whether the decimal separator on the workstation matches the locale of the document that you are processing.
IsOriginalEuroFormat	Checks the document level Euro Format and USCanUK Format variables for the currency values in the recognized data. Votes on the decimal format that is used in the document.
IsWorkstationLocale	Checks to see which decimal separator is used on the workstation.

APTCustom

Use the APTCustom actions to customize your APT applications.

The APTCustom actions are described in the following table.

Action	Description
AddAllTaxesToTaxField	Takes every tax detail line value and adds it to a field called Tax.
AddToDate	Specifies a time to add to the date value stored in a field.
CalculateInvoiceTotalLocalized	Calculates the total value of the localized invoice.
CalculateLineItemLocalized	Calculates the value of the localized line item.
ClearCurrentField	Erases the data from the current field.
ConvertEuroDateToUS	Converts the Date values from a European format to a US format.
ConvertUSDateToEuro	Converts the Date values from a US format to a European format.
FindTaxValue	Locates the taxes on an invoice.
IsDate_FormatEuro	Checks to see whether the Date value is in a valid European format.
IsInvoiceFromUS	Checks to see whether the invoice is from a company in the United States.
MakeFieldHighConfidence	Sets all of the character confidence field values to high confidence.
PopulateTaxType	Populates the Tax Details section with Tax Type information.

ConcatLineValues

Use the ConcatLineValues actions to merge line items and page fields in your APT applications.

The ConcatLineValues are described in the following table.

Action	Description
MergeLineItemFieldToPageField	Merges the line item values on the invoice image into a page variable
MergePageFieldToDocVar	Merges the page field values on the invoice images into a document variable

Documents

Use the Documents actions to create and manage documents on your APT applications.

The Documents actions are described in the following table.

Action	Description
CombinePreviousDoc	Copies the pages of the previous named document into the front of this document.
CountPagesToDocVar	Counts the number of page objects in a document and writes the result in a document variable.
IsFirstDocInBatch	Checks to see whether this object is the first document in the batch.
RemoveDocumentStructure	Levels the document/page hierarchy from the batch. For example, if the batch consists of multiple documents that each contain a set of pages, the document level is removed. All of the pages become a flat structure.

FlexID

Use the FlexID action to run the panel for the FlexID task.

The FlexID action is described in the following table.

Action	Description
RunFlexIDPanel	Runs the panel for the FlexID task in APT.

Intellocate_Learning

Use the Intellocate_Learning actions to allow APT to determine the new zones in your invoice image and add these zones to the DCO.

The Intellocate_Learning actions are described in the following table.

Action	Description
Learn_Zones	Learns about a new zone and adds the zone to the DCO.

Action	Description
Learn_ZonesFPX	Learns about a new zone and adds the zone to the DCO. Learn_ZonesFPX is FPXML compatible and it learns only the first line item in the zone. To write the zone to FPXML, this action must be followed by a WriteZonesFPX action.

PageID

Use the PageID actions to work with pages in your APT documents.

The PageID actions are described in the following table.

Action	Description
PageIDByBCSep	Separates the pages in the batch by using bar code separator sheets.
PageIDBySeqTypes	Applies a sequence of comma-separated page types, in order, to the pages of type Other in a batch, and repeats the sequence indefinitely.
PageIDByVariableChange	Processes all of the pages with a page type that is set according to the input parameters. For example, when pages are PageID, a variable for the page can be watched so when that variable changes, a new page type is created.

PreVerifySetup

Use the PreVerifySetup action to set the label variable before you run the Verification task on the batch of documents.

The PreVerifySetup action is described in the following table.

Action	Description
SetLabels	Reads the .ini file and sets up label variable that is used by Datacap Web Client and Datacap Desktop to display the label on each field.

Redaction

Use the Redaction actions to redact selected information in your APT invoice images for security purposes.

The Redaction actions are described in the following table.

Action	Description
EraseRect	Redacts the area of the invoice image that you specify in pixels. For example, you can configure this action as EraseRect(x1, y1, y2, bBlack) to redact the area in the image that corresponds to the specified coordinates.

Action	Description
GetAllBarcodes	Gets as many as 10 bar codes from the invoice image.
RedactByRegEx	Checks for a specified regular expression and redacts all occurrences of that regular expression.
RedactField	Redacts an area of the invoice image by using the current coordinates of the field.

Notices

This information was developed for products and services offered in the U.S.A.

IBM may not offer the products, services, or features discussed in this document in other countries. Consult your local IBM representative for information on the products and services currently available in your area. Any reference to an IBM product, program, or service is not intended to state or imply that only that IBM product, program, or service may be used. Any functionally equivalent product, program, or service that does not infringe any IBM intellectual property right may be used instead. However, it is the user's responsibility to evaluate and verify the operation of any non-IBM product, program, or service.

IBM may have patents or pending patent applications covering subject matter described in this document. The furnishing of this document does not grant you any license to these patents. You can send license inquiries, in writing, to:

IBM Director of Licensing
IBM Corporation
North Castle Drive
Armonk, NY 10504-1785
U.S.A.

For license inquiries regarding double-byte (DBCS) information, contact the IBM Intellectual Property Department in your country or send inquiries, in writing, to:

Intellectual Property Licensing
Legal and Intellectual Property Law
IBM Japan Ltd.
1623-14, Shimotsuruma, Yamato-shi
Kanagawa 242-8502 Japan

The following paragraph does not apply to the United Kingdom or any other country where such provisions are inconsistent with local law:

INTERNATIONAL BUSINESS MACHINES CORPORATION PROVIDES THIS PUBLICATION "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. Some states do not allow disclaimer of express or implied warranties in certain transactions, therefore, this statement may not apply to you.

This information could include technical inaccuracies or typographical errors. Changes are periodically made to the information herein; these changes will be incorporated in new editions of the publication. IBM may make improvements and/or changes in the product(s) and/or the program(s) described in this publication at any time without notice.

Any references in this information to non-IBM Web sites are provided for convenience only and do not in any manner serve as an endorsement of those Web sites. The materials at those Web sites are not part of the materials for this IBM product and use of those Web sites is at your own risk.

IBM may use or distribute any of the information you supply in any way it believes appropriate without incurring any obligation to you.

Licensees of this program who wish to have information about it for the purpose of enabling: (i) the exchange of information between independently created programs and other programs (including this one) and (ii) the mutual use of the information which has been exchanged, should contact:

IBM Corporation
J46A/G4
555 Bailey Avenue
San Jose, CA 95141-1003
U.S.A.

Such information may be available, subject to appropriate terms and conditions, including in some cases, payment of a fee.

The licensed program described in this document and all licensed material available for it are provided by IBM under terms of the IBM Customer Agreement, IBM International Program License Agreement or any equivalent agreement between us.

This information contains examples of data and reports used in daily business operations. To illustrate them as completely as possible, the examples include the names of individuals, companies, brands, and products. All of these names are fictitious and any similarity to the names and addresses used by an actual business enterprise is entirely coincidental.

Any performance data contained herein was determined in a controlled environment. Therefore, the results obtained in other operating environments may vary significantly. Some measurements may have been made on development-level systems and there is no guarantee that these measurements will be the same on generally available systems. Furthermore, some measurements may have been estimated through extrapolation. Actual results may vary. Users of this document should verify the applicable data for their specific environment.

Information concerning non-IBM products was obtained from the suppliers of those products, their published announcements or other publicly available sources. IBM has not tested those products and cannot confirm the accuracy of performance, compatibility or any other claims related to non-IBM products. Questions on the capabilities of non-IBM products should be addressed to the suppliers of those products.

All statements regarding IBM's future direction or intent are subject to change or withdrawal without notice, and represent goals and objectives only.

This information contains examples of data and reports used in daily business operations. To illustrate them as completely as possible, the examples include the names of individuals, companies, brands, and products. All of these names are fictitious and any similarity to the names and addresses used by an actual business enterprise is entirely coincidental.

COPYRIGHT LICENSE:

This information contains sample application programs in source language, which illustrate programming techniques on various operating platforms. You may copy,

modify, and distribute these sample programs in any form without payment to IBM, for the purposes of developing, using, marketing or distributing application programs conforming to the application programming interface for the operating platform for which the sample programs are written. These examples have not been thoroughly tested under all conditions. IBM, therefore, cannot guarantee or imply reliability, serviceability, or function of these programs.

“Trademarks”

“Privacy policy considerations”

Trademarks

The following terms are trademarks of the International Business Machines Corporation in the United States, other countries, or both: <http://www.ibm.com/legal/copytrade.shtml>

Microsoft, Windows, Windows NT, and the Windows logo are trademarks of Microsoft Corporation in the United States, other countries, or both.

Java and all Java-based trademarks and logos are trademarks or registered trademarks of Oracle and/or its affiliates.

Adobe, the Adobe logo, PostScript, and the PostScript logo are either registered trademarks or trademarks of Adobe Systems Incorporated in the United States, and/or other countries.

Linux is a registered trademark of Linus Torvalds in the United States, other countries, or both.

UNIX is a registered trademark of The Open Group in the United States and other countries.

Other product and service names might be trademarks of IBM or other companies.

Privacy policy considerations

IBM Software products, including software as a service solutions, (“Software Offerings”) may use cookies or other technologies to collect product usage information, to help improve the end user experience, to tailor interactions with the end user or for other purposes. In many cases no personally identifiable information is collected by the Software Offerings. Some of our Software Offerings can help enable you to collect personally identifiable information. If this Software Offering uses cookies to collect personally identifiable information, specific information about this offering’s use of cookies is set forth below.

The Software Offering uses session and persistent cookies that collect each user’s user name, for purposes of session management, enhanced user usability, single sign-on configuration. Session cookies cannot be disabled. Persistent cookies can be disabled, but disabling them will also eliminate the functionality they enable.

If the configurations deployed for this Software Offering provide you as customer the ability to collect personally identifiable information from end users via cookies and other technologies, you should seek your own legal advice about any laws applicable to such data collection, including any requirements for notice and consent.

For more information about the use of various technologies, including cookies, for these purposes, See IBM's Privacy Policy at <http://www.ibm.com/privacy> and IBM's Online Privacy Statement at <http://www.ibm.com/privacy/details> the section entitled "Cookies, Web Beacons and Other Technologies" and the "IBM Software Products and Software-as-a-Service Privacy Statement" at <http://www.ibm.com/software/info/product-privacy>.

Index

Special characters

.app file
 storing passwords 108
@APPPATH(<key_path>) 277
@APPVAR(<key_path>) 278
@B\<field_name>
 [.<variable_name>] 282
@BATCHID 279
@CHR(<unicode_value>) 286
@D\<field_name>
 [.<variable_name>] 282
@DATE(<format>) 286
@DCO(<property_name>) 286
@DICT_VALUE(<field>) 287
@DICT_VINDEX(<csv_string>) 287
@DICT_WINDEX(<csv_string>) 288
@DICT_WORD(<field>) 287
@EMPTY 288
@F.<variable_name> 283
@F\<field_name>
 [.<variable_name>] 282
@ID 280
@JOBID 284
@JOBNAME 284
@OPERATOR 284
@P.<variable_name> 283
@P\<field_name>
 [.<variable_name>] 281
@PATH(<key>) 288
@PILOT(<property_name>) 288
@PROCESSDIR 290
@PROJECTDIR 289
@STATION 284
@STATUS 280
@STRING(<string_value>) 290
@TASKID 285
@TASKNAME 285
@TIME(<format>) 290
@TYPE 290
@VALUE 280
@VAR(<variable_name>) 281

Numerics

4010Common 1011
4010Institutional 1012
4010Professional 1012
5010 Institutional form
 configuration 309
5010 Professional form
 configuration 320
5010Common 1012
5010Institutional 1012
5010Professional 1012

A

AbortOnError action 881, 882
about this guide xvii

access application configuration
 settings 168
access other information 175
access task information 174
access the runtime hierarchy 171, 172,
 173, 174
action details
 details 137
 viewing 137
action library summaries 333
actions 27
 application-specific 1011
 APT_Localization 1015
 APTCustom 1016
Autodoc
 BlankPagesIDBySize 335
 CalculateOffset 336
 CreateFingerprint 336
 DeleteFingerprint 337
 FindBlackFingerprint 337
 FindFingerprint 338
 FindTemplate 339
 MergeCCOs_ByType 339
 SetApplicationID 340
 SetFilter_HostName 341
 SetFilter_PageType 341
 SetFingerprint 342
 SetFingerprint
 FailureThreshold 343
 SetFingerprint
 WebServiceURL 345
 SetFingerprintSearchArea 344
 SetMaxOffset 346
 SetProblemValue 346
 SetSearchArea 347
 SetTemplateDir 347
 UpdateFingerprintStats 348
Barcode_P
 Get2DCodeBP 348
 GetAllBarcodesBP 349
 GetBarcodeBP 350
 GetDataMatrixCodeBP 352
 IdentifyByBarcodesBP 353
 MatchBarcodeBP 354
 MatchBarcodePrefixBP 354
 ReadBarcodeBP 355
 SetMinimumConfidenceBP 356
Barcode_X
 GetBarCode 357
 MatchBarcode 358
 ReadBarCode 358
CC
 FindFingerprintCC 359
 SetKnowledgeBaseCC 360
 SetLanguageCC 360
 SetListenerURLCC 361
 SetProblemValueCC 362
 UpdateKnowledgeBaseCC 363
Cco2cco
 NormalizeCCO 364
 SetMaxCharacter HeightAVG 365

actions (*continued*)
 Cco2cco (*continued*)
 SetMaxCharacter
 HeightTMM 366
CMISClient
 CMISCreateFolder 367
 CMISDeleteFile 368
 CMISDeleteFolder 369
 CMISDoesFileExist 370
 CMISDoesFolderExist 370
 CMISDownloadFile 371
 CMISLogDocumentTypes 372
 CMISLogin 373
 CMISRefreshClientCache 374
 CMISSetDocUploadProperty 375
 CMISSetDocUploadType 376
 CMISSetVersion 377
 CMISUploadFile 378
 CMISUploadPage 379
ColorToBW
 C2BW_Convert 381
 C2BW_SetAttributes 381
ConcatLineValues 1017
Connectors 105
Convert
 ExcelAutoFitColumns 387
 ExcelAutoFitRows 387
 ExcelOrientation
 ToLandscape 388
 ExcelOrientationToPortrait 389
 ExcelPrintBlankPage 389
 ExcelPrintGridlines 390
 ExcelPrintQuality 391
 ExcelScalingFactor 392
 ExcelTiffCompression 392
 ExcelWorkbookToImage 393
 ExceptionSetFileTypes 383
 ExceptionSetHandler 384
 ExceptionSetTaskCondition 385
 ExceptionSetVariableName 384
 HtmlPrintQuality 394
 HtmlTiffCompression 395
 HtmlToImage 395
 ImageDefaultDPI 397
 ImageFileTypesToConvert 397
 ImageMonoThreshold 398
 ImageMonoType 399
 ImageToTIFF 400
 OutlookMessageTo
 AttachmentOnly 401
 OutlookMessageToImage
 AndAttachment 402
 OutlookPrintQuality 403
 OutlookTiffCompression 403
 PDFBitDepth 404
 PDFCompression 405
 PDFConversionMethod 406
 PDFConversionMode 411
 PDFDocumentToImage 407, 411
 PDFGrayscale 407
 PDFHorizontalResolution 408

actions (continued)

Convert (continued)

PDFImageCompression 412
 PDFImageFileExtension 414
 PDFImageFileResolution 415
 PDFImageUse
 FastBinarization 415
 PDFQuality 409
 PDFVerticalResolution 410
 RtfPrintQuality 416
 RtfTiffCompression 417
 RtfToImage 418
 SetNamePattern 386
 SplitMultipageTiff 419
 SplitTIFFCompression 419
 TxtFontName 421
 TxtFontSize 421
 TxtPrintQuality 422
 TxtTiffCompression 422
 TxtToImage 423
 WordDocumentToImage 424
 WordPrintQuality 425
 WordTiffCompression 426
 ZipOverwrite 427
 ZipPassword 428
 ZipUnPack 428
 Datacap Accounts Payable 1014
 dci_clipfield 429
 DCImageFix
 ImageEnhance 430
 LoadSettings 431
 LoadSettings_FingerprintID 432
 DCO
 ChkConfidence 434
 ChkDCOStatus 434
 ChkDCOType 435
 ChkIntegrity 436
 ChkLastDCOType 436
 ClearAltText 437
 ClearDCO 437
 CopyPD2DD 438
 CountPagesToDocumentVar 438
 CreateDocuments 439
 CreateFields 440
 DeleteFields 440
 IsDocumentCountMoreThan 441
 IsFirstDocumentInBatch 442
 JoinPreviousDocument 442
 PropagateToAltText 443
 RemoveDocumentStructure 443
 SetDCOStatus 444
 SetDCOType 444
 SetDocStatus 445
 SetDocumentType 445
 SetFldConfidence 446
 SetPageFingerprintID 447
 SetPageStatus 448
 SetPageTemplateID 449
 SetPageType 449
 dcpdf
 dcpdf_CreateTiffFrom
 PDF_CreateDocs 451
 dcpdf_CreateTiffFromPDF 451
 dcpdf_MakePDFDoc 452
 dcpdf_MaxSizeToReconvert 454
 dcpdf_SetApplication 455
 dcpdf_SetAuthor 456

actions (continued)

dcpdf (continued)

dcpdf_SetImage Compression 457
 dcpdf_SetImageBitcount 456
 dcpdf_SetImageGrayscale 458
 dcpdf_SetImageQuality 458
 dcpdf_SetImageResolution 459
 dcpdf_SetKeywords 460
 dcpdf_SetProducer 460
 dcpdf_SetSubject 461
 dcpdf_SetTitle 462
 dcpdf_UseAltConversion
 Method 462
 Documents 1017
 Email 105
 SendEmail 463
 SetAttachment 464
 SetBlindCarbonCopyRcpts 464
 SetCarbonCopyRcpts 465
 SetEmailBody 465
 SetMailServer 466
 SetRecipients 467
 SetSender 467
 SetSubject 468
 Equalize
 EqualizeUnbalancedImage 468
 Ewsmail
 ex_abort_time 470
 ex_done_folder 470
 ex_EMLOption 471
 ex_ews_version 472
 ex_HTTP_timeout 473
 ex_load_properties_option 473
 ex_login 474
 ex_logout 475
 ex_max_docs 476
 ex_problem_folder 477
 ex_scan 478
 ex_types 479
 ex_wait_time 480
 Export
 BatchVariable_ExportValue 482
 BlankFields 482
 BlankLines 483
 BPilot 483
 CloseExportFile 484
 DCOProperty 485
 DocumentVariable_Export
 Value 486
 ExportAllFields 486
 ExportFieldValue 487
 ExportMYValue 487
 ExportSmartParameter 488
 ExportToBatchDir 488
 Filler 489
 FixedLenLJ 489
 FixedLenRJ 490
 GetDATE 490
 GetProfileString 491
 GetTime 492
 LineItem_AddElement 493
 LineItem_BlankFields 493
 LineItem_ClearElements 494
 LineItem_ExportElements 495
 LineItem_SmartParameter 495
 NewLine 496
 PageVariable_ExportValue 496

actions (continued)

Export (continued)

ResetFieldVariables 497
 SaveFilePathAsVariable 497
 SetCSV 498
 SetElementSeparator 499
 SetExportPath 499
 SetExtensionName 500
 SetFileName 501
 SetFill 501
 SetFixedLength 502
 SetIgnoreFieldStatus 502
 SetJustified 503
 SetOMR_Separator 504
 SetSpaceFill 504
 SetZeroFill 505
 Text 505
 Variable_ExportValue 506
 Variable_IsValue 506
 ExportDB
 AddRecord 507
 ExportBatchIDToColumn 508
 ExportCloseConnection 509
 ExportFieldToColumn 510
 ExportNodeXMLToColumn 511
 ExportOpenConnection 512
 ExportPropertyToColumn 513
 ExportSmartParam
 ToColumn 514
 ExportToColumn 515
 SetTableName 516
 ExportXML
 xml_CommitNode 517
 xml_NewNode 518
 xml_SaveFile 518
 xml_SetAttributeValue 519
 xml_SetExportPath 520
 xml_SetFileName 520
 xml_SetNodeValue 521
 Fax 105
 FileIO
 CheckFreeDiskSpace 522
 CopyDirectory 523
 CopyFile 524
 DeleteDirectory 525
 DeleteFile 526
 GetFileSize 527
 GetProfileString 528
 IsDirectoryPresent 529
 IsFilePresent 530
 IsFileReadOnly 530
 IsProfilePresent 531
 RenameFile 532
 SetFileReadOnly 533
 SetProfileString 534
 SplitFileName 535
 FileNet P8
 FNP8_CreateFolder 554
 FNP8_Login 555
 FNP8_MultiPageDocs 555
 FNP8_SetDestinationFolder 556
 FNP8_SetDocClassId 557
 FNP8_SetDocTitle 557
 FNP8_SetFileType 558
 FNP8_SetKeyProperty 558
 FNP8_SetLocale 559
 FNP8_SetMultiValue Property 560

actions (continued)

FileNet P8 (continued)

FNP8_SetProperty 560
 FNP8_SetPropertyEx 561
 FNP8_SetRetry 562
 FNP8_SetTargetClassID 563
 FNP8_SetTargetObjectID 563
 FNP8_SetTimeout 564
 FNP8_SetUploadMode 564
 FNP8_SetURL 565
 FNP8_UpdateProperties 565
 FNP8_Upload 566
 FNP8_UploadDir 567

FileNetIDM

AddAllImagesToDocument 537
 AddFileToDocument 537
 AddPDFImageToDocument 538
 AddTIFFImageToDocument 539
 CreateFolder 540
 FileNetDB_ADOConnect 540
 FileNETDocID_SaveAs
 SmartParameter 541
 FileNETDocID_SetValue 541
 GetDocuments 542
 GetTopFolders 542
 IndexProperty_ID_
 DateComponent 544
 IndexProperty_
 ID_Component 543
 IndexProperty_ LeftJUSTIFY 545
 IndexProperty_ RightJUSTIFY 546
 IndexProperty_
 SmartParameter 546
 IndexProperty_ID_Value 544
 Library_DMA_Initialize 547
 Library_DS_Initialize 547
 Library_IS_Initialize 548
 Library_LogIn 549
 Library_LogOff 549
 NewDocument 550
 SaveDocToFolder 551
 Upload 551
 Upload_SetNumAttempts 552
 UseIndexes_OFF 552
 UseIndexes_ON 553

FingerprintMaintenance

CloseDatabase 568
 DeleteFingerprint 569
 DeleteFingerprints 569
 OpenDatabase 570
 SetFingerprintFolder 571

FlexID 1017

FPXML

ReadZonesFPX 572
 SetDetailsAndLineitem
 PairFPX 573
 SetDirectoryFPX 574
 WriteZoneFPX 575
 WriteZonesFPX 575

global 333

Grayscale

ConvertGraytoBW 577

IBMCM

IBMCM_AddPages 578
 IBMCM_CreateFolder 579
 IBMCM_CreateItem 580
 IBMCM_DeletePages 581

actions (continued)

IBMCM (continued)

IBMCM_Logon 582
 IBMCM_ReplacePage 583
 IBMCM_SearchItem 584
 IBMCM_SetAttributeValue 585
 IBMCM_SetDestinationFolder 587
 IBMCM_SetMimeType 586
 IBMCM_StoreItemIDinDCO 588
 IBMCM_UploadDCO_DOC 589
 IBMCM_UploadDCO_Page 590

ICR_C

EnableLoggingICR_C 591
 RecognizeFieldICR_C 592
 RecognizeFieldVoteICR_C 592
 RecognizePageFields
 ICR_CEx 595
 RecognizePageFields2
 CCO_ICR_C 593
 RecognizePageFieldsICR_C 594
 RecognizePageICR_C 595
 RecognizePageToPDFICR_C 596

ICR_P

AddWord 597
 DeleteWord 598
 ImportCSF 598
 LoadFromFile 599
 NewDictionary 599
 RecognizeFieldsICR_P 600
 SaveToFile 601
 SetPostalDBPathICR_P 602

ImageConvert

AppendAllImages 603
 AppendAllImages_ByType 603
 AppendImage 604
 AppendImage_StartAsNew 605
 ConvertToJPEG 606
 ConvertToTIFF 606
 SetChrominanceFactor 607
 SetDeleteOriginal 608
 SetGrayScale 608
 SetLuminanceFactor 609
 SetTIFFCompression 610

ImageFix 610

Email

im_abort_time 611
 im_AcceptMixedAttachments 612
 im_AcceptNoAttachments 612
 im_done_folder 613
 im_login 614
 im_logout 615
 im_max_docs 615
 im_problem_folder 616
 im_scan 617
 im_SortByDate 618
 im_StoreEML 619
 im_types 620
 im_UseSSL 621
 im_wait_time 621

Imprint

AnnotateImage 622
 ImPrint 623
 Redact 624
 RedactByRegEx 625
 RedactParameters 625
 SetAdjustedWidth 626
 SetFontName 627

actions (continued)

Imprint (continued)

SetFontSize 628
 SetOpaque 628
 incorporating into applications 107
 installation
 verifying 106
 Intellocate
 iloc_AdjustZones 629
 iloc_AssignPageType 630
 iloc_SetDetailZones 631
 iloc_SetZones 631
 IsPageDataMissing 632
 Intellocate_Learning 1017
 Invoice
 AddToDetailErrorMsg 634
 AddToErrorMsg 634
 AllMixedCase 635
 AllowOnlyChars 635
 AlterDatebyDay 636
 CalculateNotesZone 636
 CaptureOpInfo 637
 CheckAndFixDecimal 637
 CheckForSticky 638
 CheckFreeDiskSpace 638
 ClearErrorMsg 639
 CreateFingerprint 640
 DetailFix 640
 DoMsgbox 641
 ExecuteSQLBind 641
 FindExportImage 642
 FPXMLUsed 642
 GenerateDetails 643
 iloc_SetDetailSimple 643
 IncrementBatchVar 644
 Is_InCharSet 650
 Is_JobName 651
 Is_JobNamePrefix 651
 IsChildFieldBlank 644
 IsChildFieldValue 645
 IsCurrentObjValue 645
 IsCurrentObjVariable 646
 IsFingerPrintClass 646
 IsInINI 647
 IsInList 647
 IsMultipageDocument 648
 IsSinglePageDocument 648
 IsStationIDSuffix 649
 IsTaskName 649
 LoadCCOFromField 652
 MovePDF 652
 OpenConnection 653
 ParseImageName 653
 PopulateZNLInItem
 FieldDynamic 654
 ReadFPXMLZones 655
 IOverlay
 Overlay 661
 SetBackgroundImage 662
 SetDitheringBackground 663
 SetHaloBackground 663
 Locate
 AddKeyList 665
 AggregateKeyList 666
 DefaultValue 667
 FilterIt 667
 FindDBList 668

actions (continued)

Locate (continued)

FindDBList_InZone 669
 FindKeyList 669
 FindKeyList_InZone 670
 FindLastKeyList 671
 FindLastKeyList_InZone 672
 FindLastRegEx 673
 FindLastRegEx_InZone 674
 FindLastRegExList 674
 FindLastRegExList_InZone 675
 FindLastWord 676
 FindLastWord_InZone 677
 FindNextDBList 677
 FindNextDBList_InZone 678
 FindNextKeyList 679
 FindNextKeyList_InZone 680
 FindNextRegExList 681
 FindNextRegExList_InZone 682
 FindRegExList 683
 FindRegExList_InZone 684
 GoAboveWord 685
 GoBelowWord 686
 GoDownLine 686
 GoFirstLine 687
 GoFirstWord 688
 GoLastLine 688
 GoLastWord 689
 GoLeftWord 689
 GoRightWord 690
 GoUpLine 691
 GroupWords 691
 GroupWordsLEFT 692
 GroupWordsRIGHT 693
 IsAlpha 693
 IsCurrency 694
 IsDateValue 695
 IsNumber 696
 IsValue 696
 IsValue_RegEx 697
 MaxLength 698
 MergeWordLF 698
 MergeWordRT 699
 MinLength 700
 RegExFind 700
 RegExFind_InZone 701
 RegExFindNext 702
 RegExFindNext_InZone 702
 ScanRT 703
 SelectSnippet 704
 SetRect 705
 UpdateDCOField 705
 UpdateField 706
 ValueInField 707
 ValueInField_Fuzzy 707
 ValueInField_RegEx 708
 WordFind 708
 WordFind_InZone 709
 WordFind_Offset 711
 WordFindNext 710
 WordFindNext_InZone 711

Lookup

ClearLookupResults 712
 CloseConnection 713
 ExecuteSQL 714
 OpenConnection 715
 PopulateWithResult 715

actions (continued)

Lookup (continued)

SmartSQL 716
 Maintenance Manager
 LogClear 791
 LogConfigure 791
 LogSendEmail 793
 LogWriteEventLog 794
 LogWriteRecordSet 795
 LogWriteSQLQuery 795
 ProcessChange BatchStatus 780
 ProcessChangeBatch
 StatusOrder 780
 ProcessChangeBatchStatus
 TaskOrder 781
 ProcessClearAuditTable 782
 ProcessClearDebugTable 783
 ProcessDeleteBatches 783
 ProcessDeleteBatchesEx 784
 ProcessInjectBatches 784
 ProcessMoveBatches 785
 ProcessMoveBatchesEx 786
 ProcessMoveDBRecords 787
 ProcessReset
 PendingOrNotify 788
 ProcessRunSqlQuery 790
 QueryClear 764
 QuerySetAge 765
 QuerySetBatchRange 766
 QuerySetBranch 767
 QuerySetDateFormat 767
 QuerySetDateRange 770
 QuerySetDateTimeFormat 771
 QuerySetGeneric 773
 QuerySetJobID 774
 QuerySetOperator 775
 QuerySetPriority 775
 QuerySetSeparator 776
 QuerySetStation 777
 QuerySetStatus 777
 QuerySetTaskID 778
 ReportQueryTMUsage 796
 ReportSetReportingTable 797
 ReportSetUsageDBTable 798
 SetAdminDB 756
 SetApplication 757
 SetEngineDB 757
 SetPassword 758
 SetServer 759
 SetStation 760
 SetupDisconnectAll 760
 SetupOpenApplication 761
 SetupOpenApplicationEx 762
 SetUser 763

MC_Identify

AutoField 717
 FindFields 718
 ReadDCOSetup 719
 ReadPageSetup 719
 SetFormType 720
 SetMaxTolerantDistance 721

MC_Validation

AddCenturyTo2YearDigit 722
 AddToDetailErrorMsg 723
 AddToErrorMsg 723
 CalculateHCFALineCharges 724
 CalculateUBLineCharges 724

actions (continued)

MC_Validation (continued)

CheckDocID 725
 ClearErrorMsg 725
 CommonParseAddress 726
 CommonValAddress 727
 ConvertHyphen 728
 FilterPID 728
 FormatFieldLengths 729
 InheritSnippets 729
 MC_ReadZones 730
 Parse31aPhSig 731
 Parse58ainsnm 731
 Parse58binsnm 732
 Parse58cinsnm 732
 ParseConditionCodes 733
 ParseEPSDT 733
 ParseLastFirstIniNames 734
 ParseNDC 735
 PopulateFromField 735
 SetConf 736
 SetOriginalTIF 736
 StripTrailingAlpha 737
 TransformLI 737
 UpdateCredentialList 739
 ValidateNPI 740
 ValProcedureCode 740
 ValRequiredCode 741
 Medical Claims 1011
 multi-pass verification 237
 myscan
 scan 742
 set_abort_time 743
 set_copy_folder 744
 set_delete_empty_folders 744
 set_folder 745
 set_image_validation 746
 set_max_docs 746
 set_metadata_types 747
 set_min_age 749
 set_move_wait_time 750
 set_multipage_burst 750
 set_problem_folder 751
 set_sort_method 752
 set_tree_mode 753
 set_types 753
 set_wait_time 754

OCR_A

EnableEngineLogsOCR_A 800
 OCRA_ConvertImage2BW 800
 RecognizeBarcodeOCR_A 801
 RecognizeFieldOCR_A 801
 RecognizeFieldVoteOCR_A 802
 RecognizePageFieldsOCR_A 803
 RecognizePageOCR_A 804
 RecognizeToALTOOCR_A 804
 RecognizeToPDFOCR_A 806
 ReleaseEngineOCR_A 808
 RotateImageOCR_A 808
 SetAutoRotationOCR_A 809
 SetConfCalculation
 ParamsOCR_A 810
 SetFastModeOCR_A 810

OCR_N

RecognizePageFieldsOCR_N 811
 RecognizePageOCR_N 812

actions (continued)

OCR_S

RecognizeDocToPDF 813
 RecognizeFieldOCR_S 814
 RecognizeFieldVoteOCR_S 815
 RecognizePageFields
 2CCO_OCR_S 816
 RecognizePageFieldsOCR_S 817
 RecognizePageOCR_S 817
 RecognizePageOCR_S_
 2TextFile 818
 RecognizeToFile_OCR_S 819
 RecognizeToPDF 821
 RotateImage 822
 SetEngineTimeout 823
 SetFastTradeOffOCR_S 824
 SetLegacyDecomposition
 OCR_S 825

OCR_SR

RecognizeFieldOCR_S 826
 RecognizeFieldVoteOCR_S 827
 RecognizePageFieldsOCR_S 827
 RecognizePageOCR_S 828
 RecognizeToFile_OCR_S 829
 RecognizeToPDFOCR_S 831
 RotateImage 832
 SetEngineTimeoutOCR_S 833

OpenTextFaxServer

Connect 835
 ContinueOn ConnectionError 835
 ContinueOn FaxImportError 836
 Disconnect 837
 ImportFaxes 838
 SendAsFax 839
 SetAbortTimeout 839
 SetFaxRemovalAfterImport 840
 SetInputFolder 841
 SetMaxNumberOfFaxes 842
 SetNumberOfRetries 843
 SetPollingInterval 844
 SetProcessedFaxesFolder 844
 SetProtocol 845
 SetRetryTimeout 846
 SetServerName 847
 SetUserID 848
 SetUserPassword 848
 SetWindowsAuthentication 849

PageID 1018

PatternMatch

MatchPattern 850
 pat_RecognizeMatch_Id 851
 pat_RegisterZones 852
 pat_ReleasePageAnchors 853
 PatternMatch_Fingerprint 854
 PatternMatch_Identify 855
 PatternMatch_PageType 856
 SetMatchConfidence 857

Picture

PIC_ApplyPictureString 858
 PIC_FilterFields 859
 PIC_FormatFields 859
 PIC_ReplaceBlankField 861
 PIC_SetPictureCharacter 862
 PIC_ValidateField 863

POLR

CallPOLR 864
 PreVerifySetup 1018

actions (continued)

Recog_Shared

AnalyzeImage 865
 CCONormalization_OFF 866
 CreateTextFile 867
 IsBlankPage 868
 RecogContinueOnFailure 869
 RecogOMRThresh 870
 RecogOMRThreshold 870
 RegisterPageFields 872
 ReleaseImage 873
 RotateTio 873
 SetAdjustFieldToChars 874
 SetFingerprintRecogPriority 875
 SetFullPageRecogArea 875
 SetOutOffProcess
 RecogTimeout 876
 SetRecogFailureRetryDelay 877
 SnapCCOtoDCO 878
 SnapDCOtoCCO 879
 SnapFieldtoChars 879
 UseOutOffProcessRecog 880

Redaction 1018

referencing connection strings 171
 referencing passwords 171

rrunner

AbortOnError 882
 CheckAllIntegrity 882
 CheckDocCount 883
 CheckPageCount 883
 DebugMode_OFF 884
 DebugMode_ON 884
 GoToNextFunction 885
 PilotMessage_Clear 886
 PilotMessage_Set 886
 ProcessChildren 886
 rr_AbortBatch 887
 rr_Get 887
 rr_WrireNode 888
 rrAppend 889
 rrCompare 889
 rrCompareCase 890
 rrCompareCaseLength 891
 rrCompareNot 892
 rrCompareNotCase 893
 rrCompareNotCaseLength 894
 rrCopy 895
 rrPrepend 896
 rrSet 897
 SetBatchPriority 898
 SetOperatorID 898
 SetReturnValue 899
 SetStationID 899
 SetTaskStatus 900
 SkipChildren 900
 Status_Preserve_OFF 901
 Status_Preserve_ON 902
 Task_NumberOfSplits 902
 Task_RaiseCondition 903
 SaveObjectVariable 655
 ScanLineItemDynamic 655
 SendOutlookNotification 656
 SetDynamicDetailZones 657
 SetPicChar 657
 SetStickyNo 657
 SetToDocIDMPTIFF 658

actions (continued)

SPExport

SP_CreateFolder 904
 SP_Login 904
 SP_SetContentType 905
 SP_SetFileType 906
 SP_SetProperty 906
 SP_SetUploadMode 907
 SP_SetUrl 908
 SP_Upload 908
 SP_UploadDir 909

Split

SplitBatch 909

SwapImages 658

SwitchMMDD 659

TifMerge

TifMerge_CheckStatus 911
 TifMerge_ExportToBatchDir 912
 TifMerge_MergeImages 913
 TifMerge_MyImage 914
 TifMerge_Preserve
 Compression 915
 TifMerge_SetFileName 915
 TifMerge_SetFilePath 916

TM524 917

TruncateFromStart 967

UpdateFPStats 659

ValidateVendor 660

Validations

AddLeadingZeros 919
 AddPaddingToEnd 919
 AddPaddingToLeft 920
 AddPaddingToRight 920
 AddPaddingToStart 921
 AddTrailingZeros 921
 AllowOnlyChars 922
 AppendFromField 922
 AppendToField 923
 AssignFieldDefault 923
 Calculate 924
 CalculateDateDifference 924
 CalculateFields 925
 CheckSubFields 926
 CompareFields 927
 ConvertFieldTo Currency 928
 ConvertToLowercase 928
 ConvertToUppercase 929
 CopyField 929
 CopyFieldToField 930
 DateStampField 931
 DeleteAllAlpha 931
 DeleteAllMiscChars 931
 DeleteAllNumeric 932
 DeleteAllPunct 932
 DeleteAllSysChars 933
 DeleteChildType 933
 DeleteLCSpaces 934
 DeleteParentObj 934
 DeleteSelectedChars 935
 EmptyFieldValue 935
 FailRuleSet 936
 FieldContainsValue 936
 FilterFieldSelectedChars 937
 FormatNumberToLocale 937
 GetJobID 938
 HasChildOfType 939
 InsertChars 939

actions (continued)

Validations (continued)

InsertDecimalPoint 940
 IsFieldCurrency 940
 IsFieldDate 941
 IsFieldDateEqualOrAfter 942
 IsFieldDateEqualOrBefore 942
 IsFieldDateUpToToday 943
 IsFieldDateWithinRange 943
 IsFieldDateWithinXDays 944
 IsFieldDateWithReformat 944
 IsFieldEmpty 945
 IsFieldFilled 945
 IsFieldGreaterOrEqual 946
 IsFieldHidden 946
 IsFieldLengthMax 947
 IsFieldLengthMin 947
 IsFieldLessOrEqual 948
 IsFieldMatching 948
 IsFieldPercent NonNumeric 949
 IsFieldPercentAlpha 949
 IsFieldPercentNumeric 950
 IsMatchingJobID 951
 IsMaxOMRChecked 951
 IsMinOMRChecked 951
 IsPatternInField 952
 IsSupportedImageFile 953
 IsThisFieldEmpty 953
 IsThisFieldFilled 954
 IsVariableEmpty 954
 IsVariableFilled 955
 LeftTruncate 955
 MessageBox 956
 ParseMultilineAddress 956
 ParseName 957
 ReadCurrentObjVariable 958
 ReadFieldValue 958
 ReadPageVariableValue 959
 ReplaceChars 960
 ReplaceValueAtPosition 960
 ResetField 961
 RightTruncate 961
 SaveAsCurrentObjVariable 961
 SaveAsPageVariable 962
 SetIsOverrideable 963
 SplitFieldValueLeft 963
 SplitFieldValuePreserveEnd 963, 964
 SplitFieldValueRight 964
 SumFields 965
 TimestampField 965
 TrimSpaces 966
 TruncateFromEnd 966

viewing details 137

Vote

VoteFld 968

Vscan

AddDocument 969
 CopyFile 969
 DeleteImageFile 970
 MoveImageFileToDirectory 971
 Scan 972
 SearchInSubdirectory 973
 SetAlternateImageNames 973
 SetFastMode 974
 SetImageType 974
 SetMailSourceFolder 975

actions (continued)

Vscan (continued)

SetMaxImageFiles 976
 SetMultiPageTiff 977
 SetSortOrder 977
 SetSourceDirectory 978

Web Services

WsCompare 979
 WsDownloadFile 980
 WsExecute 980
 WsGetLineItems 981
 WsGetValue 982
 WsMessageLineItem
 PropertyAdd 982
 WsMessageLineItem
 PropertySet 983
 WsSetHeaderValue 983
 WsSetMessageLineItem
 Property 984
 WsSetMessageProperty 985
 WsSetMessageTemplate 985
 WsSetResponseNameSpace 986
 WsUploadFile 986
 WsUrlReplaceValue 987
 WsUrlSet 987

WriteErrorMessage 660

Zones

AdjustZonesToImageOffset 989
 AnchorPage 989
 CalculateLocalOffset 990
 CreateBlockCCO 990
 FindBlocks_WhiteSpace 991
 FindDataBlocks 991
 FindLineItems 992
 FindRegExBlocks 992
 FindZoneLineItems 993
 GetZoneText 994
 InheritParentPosition 994
 LoadBlockCCO 995
 LoadZones 995
 MCCOPositionAdjust 996
 MergeZones 997
 PadZone 997
 PopulateZNField 998
 PopulateZNLinItemField 998
 ReadZones 999
 RegisterPage 1000
 ScanDetails 1000
 ScanDetailsByLines 1001
 ScanDetailsByVSpace 1001
 ScanLineItem 1002
 SetEOL 1002
 SetEOL_CRLF 1003
 ZoneBOTTOM_
 ImageBottom 1003
 ZoneBOTTOM_
 LowerBound 1004
 ZoneBOTTOM_
 UpperBound 1005
 ZoneImage_SaveAs 1005
 ZoneLEFT_ImageLeft 1006
 ZoneLEFT_LeftBound 1007
 ZoneLEFT_RightBound 1007
 ZoneRIGHT_ImageRight 1008
 ZoneRIGHT_LeftBound 1008
 ZoneRIGHT_RightBound 1009
 ZoneTOP_ImageTop 1009

actions (continued)

Zones (continued)

ZoneTOP_LowerBound 1010
 ZoneTOP_UpperBound 1010

add fingerprints

fingerprint library 262

AddAllImagesToDocument action 536, 537

AddAllTaxesToTaxField action
description 1016

AddCenturyTo2YearDigit action 722

AddDocument action 968, 969

AddFileToDocument action 536, 537

adding fingerprints 265

adding individual 40

AddKeyList action 664, 665

AddLeadingZeros action 917, 919

AddPaddingToEnd action 917, 919

AddPaddingToLeft action 917, 920

AddPaddingToRight action 917, 920

AddPaddingToStart action 917, 921

AddPDFImageToDocument action 536, 538

AddRecord action 507

AddTIFImageToDocument action 536, 539

AddToDate action

description 1016

AddToDetailErrorMsg action 632, 634, 723

AddToErrorMsg action 632, 634, 723

AddTrailingZeros action 917, 921

AddWord action 597

AdjustZonesToImageOffset action 988, 989

administering

job monitoring 248

administering the application 248

AggregateKeyList action 664, 666

AIndex web client 234

configuring 235

Done Field Statuses 256

Done Page Statuses 256

Ignored Field Statuses 256

ManualIDValidate rule 257, 259

ManualPageID settings 256

page identification 253

restructuring the batch 235

updating the application 254

Validation Statuses 256

AllMixedCase action 632, 635

AllowOnlyChars action 632, 635, 917, 922

AlterDatebyDay action 632, 636

AnalyzeImage action 865

anchor objects

pattern matching 190, 192

setting up 190

using multiple anchors 192

anchor patterns identification

pattern matching 189

AnchorPage action 988, 989

AnnotateImage action 622

AppendAllImages action 602, 603

AppendAllImages_ByType action 602, 603

AppendFromField action 917, 922

- AppendImage action 602, 604
- AppendImage_StartAsNew action 602, 605
- AppendToField action 917, 923
- application
 - administration 248
 - connecting to 16
 - debugging 143
 - updating 254
- application architecture 1
- Application Manager
 - Assigning a group to a batch
 - Defining the custom Job Monitor filter 261
- application settings
 - accessing with special variables 168
- Application setup actions
 - SetAdminDB 755, 756
 - SetApplication 755, 757
 - SetEngineDB 755, 757
 - SetPassword 755, 758
 - SetServer 755, 759
 - SetStation 755, 760
 - SetupDisconnectAll 755, 760
 - SetupOpenApplication 755, 761
 - SetupOpenApplicationEx 755, 762
 - SetUser 755, 763
- application specific actions
 - APT_Localization 1015
 - APTCustom 1016
 - ConcatLineValues 1017
 - Datacap Accounts Payable 1014
 - Documents 1017
 - FlexID 1017
 - Intellocate_Learning 1017
 - overview 333
 - PageID 1018
 - PreVerifySetup 1018
 - Redaction 1018
- Application Wizard
 - converting applications 273
 - migrating 273
- application-specific actions 1011
- application-specific variables 309
- applications
 - architecture 2
 - converting from a prior release 273
 - migrating 271
 - converting panels 275
 - migrating from a prior release 273
 - specifying data format 11
- APT_Localization action
 - description 1015
- architecture
 - Datacap applications 2
- assembling documents 51
- AssignFieldDefault action 917, 923
- Assigning a group to a batch
 - Define the custom Job Monitor filter
 - Add a rule in Datacap Studio 261
 - Add custom values in Application Manager 261
- Authentication
 - Content Engine 107
 - Email Actions 107
 - Fax Actions 107
 - FileNetIDM 107

- Authentication (*continued*)
 - IBM Content Manager 107
 - SharePoint 107
- auto registration
 - pattern matching 189
 - using the FindFingerprint action 189
- Autodoc actions
 - BlankPagesIDBySize 334, 335
 - CalculateOffset 334, 336
 - CreateFingerprint 334, 336
 - DeleteFingerprint 334, 337
 - FindBlackFingerprint 334, 337
 - FindFingerprint 334, 338
 - FindTemplate 334, 339
 - MergeCCOs_ByType 334, 339
 - SetApplicationID 334, 340
 - SetFilter_HostName 334, 341
 - SetFilter_PageType 334, 341
 - SetFingerprint 334, 342
 - SetFingerprint WebServiceURL 345
 - SetFingerprintDir 334
 - SetFingerprintFailureThreshold 334, 343
 - SetFingerprintSearchArea 334, 344
 - SetFingerprintWebServiceURL 334
 - SetMaxOffset 334, 346
 - SetProblemValue 334, 346
 - SetSearchArea 334, 347
 - SetTemplateDir 334, 347
 - UpdateFingerprintStats 348
 - UpdateFPStats 334
- Autofield 69
- AutoField action 717
- AutoFingerprint ruleset
 - adding to the Verify task profile 220
 - assigning the rule to each page type 220
 - creating 219
 - updating 267
- automate background tasks 200
- automated background processing
 - updating the TravelDocs
 - application 206
 - analyzing the Rulerunner log 209
 - defining background tasks 207
 - disabling the Rulerunner log 209
 - enabling Rulerunner logging 207
 - running a batch through the workflow 208
 - setting up background tasks 207
 - setting up Job Monitor 208
- automatic fingerprint generation 199
- updating the TravelDocs
 - application 218
 - adding the ruleset to the Verify task profile 220
 - assigning the rule to each page type 220
 - creating the AutoFingerprint ruleset 219
 - reviewing the RRS log file 221
 - running a batch through the workflow 220
- AVerify web client 242
 - creating static panels 243

B

- background tasks
 - automating 200
 - defining 207
 - processing on Rulerunner 199
 - setting up 207
 - using Rulerunner to automate 200
- Barcode_P actions
 - Get2DCodesBP 348
 - GetAllBarcodesBP 348, 349
 - GetBarcodeBP 348, 350
 - GetDataMatrixCodeBP 348, 352
 - IdentifyByBarcodesBP 353
 - MatchBarcodeBP 348, 354
 - MatchBarcodePrefixBP 348, 354
 - ReadBarcodeBP 348
 - ReadBarCodeBP 355
 - SetMinimumConfidenceBP 356
- Barcode_X actions
 - GetBarCode 356, 357
 - MatchBarCode 356, 358
 - ReadBarCode 356, 358
- Batch Pilot panels
 - converting to Datacap Desktop 275, 276
 - generating layout XML files 275
 - reviewing layout XML files 275
- Batch processing actions
 - ProcessChange BatchStatus 780
 - ProcessChangeBatch StatusOrder 779
 - ProcessChangeBatch
 - StatusTaskOrder 779
 - ProcessChangeBatchStatus 779
 - ProcessChangeBatchStatus
 - TaskOrder 781
 - ProcessClearAuditTable 779, 782
 - ProcessClearDebugTable 779, 783
 - ProcessDeleteBatches 779, 783
 - ProcessDeleteBatchesEx 779, 784
 - ProcessInjectBatches 779, 784
 - ProcessMoveBatches 779, 785
 - ProcessMoveBatchesEx 786
 - ProcessMoveBatchesEX 779
 - ProcessMoveDBRecords 779, 787
 - ProcessReset PendingOrNotify 788
 - ProcessResetPendingOrNotify 779
 - ProcessRunSqlQuery 779, 790
- batches
 - checking structural integrity 53
 - generating with VScan 30
 - preparing for verification 100
 - processing in Datacap Studio 42
 - reviewing in Datacap Desktop 101
 - running through the workflow 177, 208, 211, 217, 220, 225
 - running with document integrity problems 57
 - runtime folder 43
 - submitting 101
 - testing export tasks 166
- BatchVariable_ExportValue action 481, 482
- batches
 - validation errors 161
- BlankFields action 481, 482
- BlankLines action 481, 483
- BlankPagesIDBySize action 334, 335, 348

- BPilot action 481, 483
- branching
 - configuring 56, 216
 - defining conditions 203
 - description 202
 - raising condition flags 202
- breakpoints 145
 - clearing 146
 - disabling 146
 - full 146
 - generic 147
 - setting 146
 - types 146
- business requirements 1
 - developing 2
 - specifying field values 9
- business validation rules 10

C

- C2BW_Convert action 380, 381
- C2BW_SetAttributes action 380, 381
- Calculate action 917, 924
- calculated fields
 - validating 80
- CalculateDateDifference action 917, 924
- CalculateFields action 917, 925
- CalculateHCFALineCharges action 724
- CalculateInvoiceTotalLocalized action
 - description 1016
- CalculateLineItemLocalized action
 - description 1016
- CalculateLocalOffset action 988, 990
- CalculateNotesZone action 632, 636
- CalculateOffset action 334, 336
- CalculateUBLLineCharges action 724
- calculating cost fields
 - creating validation rules 86
- CallPOLR action 864
- CaptureOpInfo action 632, 637
- Car Type field
 - preventing overrides 99
- car type rule
 - creating validation rules 87
- Car_Type field
 - attaching a dictionary 89
- CC actions
 - FindFingerprintCC 359
 - SetKnowledgeBaseCC 359, 360
 - SetLanguageCC 359, 360
 - SetListenerURLCC 359, 361
 - SetProblemValueCC 359, 362
 - UpdateKnowledgeBaseCC 359, 363
- Cco2cco actions
 - Cco2cco 364, 365, 366
 - NormalizeCCO 364
 - SetMaxCharacter HeightAVG 365
 - SetMaxCharacter HeightTMM 366
 - SetMaxCharacterHeightAVG 364
 - SetMaxCharacterHeightTMM 364
- CCONormalization_OFF action 865, 866
- check box options
 - creating dictionaries 99
- check box recognition
 - establishing parent fields 66
 - methods 65
 - OCR/A 67

- check box recognition (*continued*)
 - pixel threshold evaluation method 68
 - setting parent field variables 67
- check mark type
 - specifying 75
- CheckAllIntegrity
 - using 53
- CheckAllIntegrity action 57, 881, 882
- CheckAndFixDecimal action 632, 637
- CheckAndFixLocalDecimal action
 - description 1015
- CheckDocCount action 881, 883
- CheckDocID action 725
- CheckForSticky action 632, 638
- CheckFreeDiskSpace action 522, 632, 638
- CheckFreeSpace action 521
- checking format validity 79
- CheckPageCount action 881, 883
- CheckSubFields action 917, 926
- Chinese
 - language codes 63
- ChkConfidence action 433, 434
- ChkDCOStatus action 433, 434
- ChkDCOType action 433, 435
- ChkIntegrity action 433, 436
- ChkLastDCOType action 433, 436
- ClearAltText action 433, 437
- ClearCurrentField action
 - description 1016
- ClearDCO action 433, 437
- ClearErrorMsg action 632, 639, 725, 726
- clearing breakpoints 146
- ClearLookupResults action 712
- CloseConnection action 712, 713
- CloseDatabase action 568
- CloseExportFile action 481, 484
- CMISClient actions
 - CMISCreateFolder 366, 367
 - CMISDeleteFile 366, 368
 - CMISDeleteFolder 366, 369
 - CMISDoesFileExist 366, 370
 - CMISDoesFolderExist 366, 370
 - CMISDownloadFile 366, 371
 - CMISLogDocumentTypes 366, 372
 - CMISLogin 366, 373
 - CMISRefreshClientCache 366, 374
 - CMISSetDocUploadProperty 366, 375
 - CMISSetDocUploadType 366, 376
 - CMISSetVersion 377
 - CMISUploadFile 366, 378
 - CMISUploadPage 366, 379
 - description 366
- CMISCreateFolder action 366, 367
- CMISDeleteFile action 366, 368
- CMISDeleteFolder action 366, 369
- CMISDoesFileExist action 366, 370
- CMISDoesFolderExist action 366, 370
- CMISDownloadFile action 366, 371
- CMISLogDocumentTypes action 366, 372
- CMISLogin action 366, 373
- CMISRefreshClientCache action 366, 374
- CMISSetDocUploadProperty action 366, 375
- CMISSetDocUploadType action 366, 376
- CMISSetVersion action 377
- CMISUploadFile action 366, 378

- CMISUploadPage action 366, 379
- code
 - single-stepping through 147
- ColorToBW actions
 - C2BW_Convert 380, 381
 - C2BW_SetAttributes 380, 381
- CombinePreviousDoc action
 - description 1017
- Common actions
 - ExceptionSetFileTypes 383
 - ExceptionSetHandler 383
 - ExceptionSetTaskCondition 383
 - ExceptionSetVariableName 383
 - SetNamePattern 383
- CommonValAddress action 727
- CompareFields action 917, 927
- condition flags
 - raining for branching 202
 - raising for splitting 202
- conditions
 - defining for branching 203
 - defining for splitting 203
- Confidence 295
- confidence level
 - setting for pattern matching 191
- confidence levels
 - description 95
 - overriding default values 96
 - runtime pages 43
- configuration
 - Email Connector actions 131
 - Fax Connector actions 135
 - FileNet Image Services Connector actions 125
 - FileNet P8 Connector 115
 - IBM Content Manager Connector actions 111
 - Rulerunner 200
 - SharePoint Connector 121
- configure remote scanning client 227
- configure the remote scanning client 250
- configuring 232
 - AIndex web client 235
 - ProtoId web client 246
 - Verifine web client 230
- configuring the export database 137
- Configuring the Upload task 250
- Connect action 834, 835
- connect to the application 16
- connection strings
 - referencing from your actions 171
 - storing in the .app file 169
- ConnectOnConnectionError 834
- ConnectOnFaxImportError 834
- Connector actions
 - installation
 - verifying 106
- log files 136
- overview 105
- Connector Actions
 - overview 108
- connectors
 - incorporating into applications 107
- Content Engine
 - Authentication 107
- ContinueOn FaxImportError action 836
- ContinueOnConnectionError action 835

- ContinueOnFaxImportError actions
 - Connect 836
- convert
 - Application Wizard 273
 - applications 273
 - creating Datacap Desktop panels 276
 - layout XML files 276
- Convert actions
 - Common
 - ExceptionSetFileTypes 383
 - ExceptionSetHandler 383
 - ExceptionSetTaskCondition 383
 - ExceptionSetVariableName 383
 - SetNamePattern 383
- Excel 382
 - ExcelAutoFitColumns 386
 - ExcelAutoFitRows 386
 - ExcelOrientationToLandscape 386
 - ExcelOrientationToPortrait 386
 - ExcelPrintBlankPage 386
 - ExcelPrintGridlines 386
 - ExcelPrintQuality 386
 - ExcelScalingFactor 386
 - ExcelTiffCompression 386
 - ExcelWorkbookToImage 386
- Html 382
 - HtmlPrintQuality 394
 - HtmlTiffCompression 394
 - HtmlToImage 394
- Images 382
 - ImageDefaultDPI 396
 - ImageFileTypesToConvert 396
 - ImageMonoThreshold 396
 - ImageMonoType 396
 - ImageToTIFF 396
- Outlook 382
 - OutlookMessageTo
 - AttachmentOnly 401
 - OutlookMessageToImage
 - AndAttachment 401
 - OutlookPrintQuality 401
 - OutlookTiffCompression 401
- Pdf 382
 - PDFBitDepth 404
 - PDFCompression 404
 - PDFConversionMethod 404
 - PDFDocumentToImage 404
 - PDFGrayscale 404
 - PDFHorizontalResolution 404
 - PDFQuality 404
 - PDFVerticalResolution 404
- PdfFRE
 - PDFConversion Mode 410
 - PDFDocumentTo Image 410
 - PDFImage Compression 410
 - PDFImageFile Extension 410
 - PDFImageFile Resolution 410
 - PDFImageUse
 - FastBinarization 410
- Rtf 382
 - RtfPrintQuality 416
 - RtfTiffCompression 416
 - RtfToImage 416
- Tiff 382
 - SplitMultipageTiff 418
 - SplitTIFFCompression 418
- Txt 382

- Convert actions (*continued*)
 - TxtPrintQuality 420
 - TxtTiffCompression 420
 - TxtToImage 420
- Word 382
 - WordDocumentToImage 424
 - WordPrintQuality 424
 - WordTiffCompression 424
- Zip 382
 - ZipOverwrite 427
 - ZipPassword 427
 - ZipUnpack 427
- ConvertEuroDateToUS action
 - description 1016
- ConvertFieldTo Currency action 928
- ConvertFieldToCurrency action 917
- ConvertGraytoBW action 576, 577
- ConvertHyphen action 728
- ConvertToJPEG action 602, 606
- ConvertToLowerCase action 917, 928
- ConvertToTIFF action 602, 606
- ConvertToUpperCase action 917, 929
- ConvertUSDateToEuro action
 - description 1016
- CopyDirectory action 521, 523
- CopyField action 917, 929
- CopyFieldToField action 917, 930
- CopyFile action 521, 524, 968, 969
- Copying the application 253
- CopyPD2DD action 433, 438
- CountPagesToDocumentVar action 438
- CountPagesToDocVar action
 - description 1017
- create document types 19
- create ExportDB ruleset 138
- create ExportXML ruleset 140
- create fingerprint classes 39
- create ManualIDValidate rule 257
- Create or copy an application
 - CMIS-based application 16
- create page fingerprints 155
- create the dictionary 89
- CreateBlockCCO action 988, 990
- CreateDocs task
 - configuring Rulerunner to run 211
 - creating 210
- CreateDocuments action 433, 439
- CreateFields action 433, 440
- CreateFingerprint action 334, 336, 632, 640
- CreateFolder action 536, 540
- CreateTextFile action 865, 867
- creating a remote scan task 249
- creating data fields 154
- creating fingerprint files 262
- creating page data files 52
- creating page types 20
- creating static panels
 - AVerify web client 243
- Creating the scan task
 - Datacap Web Client 32
- creating zones
 - other page types 71
 - TravelDocs 71
- currency fields
 - creating validation rules 85
 - validating 84

- custom pages
 - creating and using 233
- custom panels
 - specifying to use in a task 244
- customizing the panel layout 244

D

- data 79
 - exporting to a database 104
 - exporting to a text file 103
 - exporting to an XML file 105
 - recognizing on an unidentified
 - page 218
 - validating 79
- data fields
 - creating 22, 154
- data format
 - specifying for export 11
- data locating
 - text matching 179
- data recognition 58
- data validation
 - calculated fields 80
 - definition 79
 - displaying failures to operator 82
 - examples 79
 - using external sources 83
- data verification 94
 - Datacap Desktop
 - Datacap Web Client 94
 - double blind 239
 - fields 94
- Datacap
 - log files 143
 - opening sample application 13
 - workflows 24
- Datacap Accounts Payable actions
 - APT_Localization 1014, 1015
 - APTCustom 1014
 - AddAllTaxesToTaxField 1016
 - AddToDate 1016
 - CalculateInvoice
 - TotalLocalized 1016
 - CalculateLineItemLocalized 1016
 - ClearCurrentField 1016
 - ConvertEuroDateToUS 1016
 - ConvertUSDateToEuro 1016
 - FindTaxValue 1016
 - IsDate_FormatEuro 1016
 - IsInvoiceFromUS 1016
 - MakeFieldHighConfidence 1016
 - PopulateTaxType 1016
 - CheckAndFixLocalDecimal 1015
 - ConcatLineValues 1014
 - MergeLineItem
 - FieldToPageField 1017
 - MergePageFieldToDocVar 1017
 - Documents 1014
 - CombinePreviousDoc 1017
 - CountPagesToDocVar 1017
 - IsFirstDocInBatch 1017
 - RemoveDocumentStructure 1017
 - FlexID 1014
 - RunFlexIDPanel 1017
 - Intelllocate_Learning 1014
 - Learn_Zones 1017

- Datacap Accounts Payable actions
 - (continued)
 - Intellocate_Learning (continued)
 - Learn_ZonesFPX 1017
 - IsFieldLocalCurrency 1015
 - IsLocalDecimalSeparator 1015
 - IsOriginalEuroFormat 1015
 - IsWorkstationLocale 1015
 - PageID 1014
 - PageIDByBCSep 1018
 - PageIDBySeqTypes 1018
 - PageIDByVariableChange 1018
 - PreVerifySetup 1014
 - SetLabels 1018
 - Redaction 1014
 - EraseRect 1018
 - GetAllBarcodes 1018
 - RedactByRegEx 1018
 - RedactField 1018
- Datacap applications
 - architecture 2
 - Creating an application
 - Datacap Studio 16
 - specifying data format 11
- Datacap Desktop
 - Hardcopy local scans 28
 - Opening the batch 100
 - Running the scan task 33
 - scanning 29
 - Verifying pages 162
- Datacap Desktop panels
 - converting from Batch Pilot 275, 276
 - converting from DotEdit 275
 - creating Datacap Desktop panels 276
 - generating layout XML files 275
 - reviewing layout XML files 275
- Datacap Server
 - starting 12
- Datacap software
 - upgrading 271
- Datacap Studio 12
 - Application wizard
 - Convert application from previous version 16
 - Copy an application 16
 - Create an application 16
 - Assigning a group to a batch
 - Add a rule 261
 - debugging an application 145
 - description 13
 - document hierarchy 19
 - examining log files 148
 - processing a batch 42
 - Rulemanager tab 13
 - tabs 13
 - Test tab 15
 - user interface 12
 - Zones tab 14
- Datacap Web Client
 - Creating the scan task 32
 - enabling logging 143
 - Hardcopy local scans 28
 - remote scanning 226
 - Running the scan task 33
 - scanning 29
 - Verifying batches 101
- DATAFILE 295
- DataType
 - Setup DCO 298
 - Verification panel
 - Datacap Web Client 298
- DateStampField action 917, 931
- Dclip actions
 - dci_clipfield 429
- dci_clipfield action 429
- DCImageFix actions
 - ImageEnhance 430
 - LoadSettings 430, 431
 - LoadSettings_FingerprintID 430, 432
- DCO actions
 - ChkConfidence 433, 434
 - ChkDCOStatus 433, 434
 - ChkDCOType 433, 435
 - ChkIntegrity 433, 436
 - ChkLastDCOType 433, 436
 - ClearAltText 433, 437
 - ClearDCO 433, 437
 - CopyPD2DD 433, 438
 - CountPagesToDocumentVar 438
 - CreateDocuments 433, 439
 - CreateFields 433, 440
 - DeleteFields 433, 440
 - IsDocumentCountMoreThan 441
 - IsFirstDocumentInBatch 442
 - JoinPreviousDocument 442
 - PropagateToAltText 433, 443
 - RemoveDocumentStructure 443
 - SetDCOStatus 433, 444
 - SetDCOType 444
 - SetDocStatus 433, 445
 - SetDocumentType 433, 445
 - SetFldConfidence 433, 446
 - SetPageFingerprintID 433, 447
 - SetPageStatus 433, 448
 - SetPageTemplateID 433, 449
 - SetPageType 433, 449
- DCOProperty action 481, 485
- dcpdf actions
 - dcpdf_CreateTiff
 - FromPDF_CreateDocs 450
 - dcpdf_CreateTiffFrom
 - PDF_CreateDocs 451
 - dcpdf_CreateTiffFromPDF 450, 451
 - dcpdf_MakePDFDoc 450, 452
 - dcpdf_MaxSizeToReconvert 450, 454
 - dcpdf_SetApplication 450, 455
 - dcpdf_SetAuthor 450, 456
 - dcpdf_SetImage Compression 457
 - dcpdf_SetImageBitcount 450, 456
 - dcpdf_SetImageCompression 450
 - dcpdf_SetImageGrayscale 450, 458
 - dcpdf_SetImageQuality 450, 458
 - dcpdf_SetImageResolution 450, 459
 - dcpdf_SetKeywords 450, 460
 - dcpdf_SetProducer 450, 460
 - dcpdf_SetSubject 450, 461
 - dcpdf_SetTitle 450, 462
 - dcpdf_UseAltConversion
 - Method 462
 - dcpdf_UseAltConversionMethod 450
- dcpdf_CreateTiffFrom PDF_CreateDocs action 451
- dcpdf_CreateTiffFromPDF action 450, 451
- dcpdf_CreateTiffFromPDF_CreateDocs action 450
- dcpdf_MaxSizeToReconvert action 450, 454
- dcpdf_SetApplication action 450, 455
- dcpdf_SetAuthor action 450, 456
- dcpdf_SetImage Compression action 457
- dcpdf_SetImageBitcount action 450, 456
- dcpdf_SetImageCompression action 450
- dcpdf_SetImageGrayscale action 450, 458
- dcpdf_SetImageQuality action 450, 458
- dcpdf_SetImageResolution action 450, 459
- dcpdf_SetKeywords action 450, 460
- dcpdf_SetProducer action 450, 460
- dcpdf_SetSubject action 450, 461
- dcpdf_SetTitle action 450, 462
- dcpdf_UseAltConversion Method action 462
- dcpdf_UseAltConversionMethod action 450
- DD 295
- debugging an application 143
- DebugMode_OFF action 881, 884
- DebugMode_ON action 881, 884
- default panel layout
 - exporting 243
- default rules
 - assigning to new fields 72
 - assigning to new pages 72
- DefaultValue action 664, 667
- Defining group names for filtering
 - batches 260
- Defining group names for filtering
 - batchesApplication Manager
 - Application Manager 260
- defining recognition zones 155
- DeleteAllAlpha action 917, 931
- DeleteAllMiscChars action 917, 931
- DeleteAllNumeric action 917, 932
- DeleteAllPunct action 917, 932
- DeleteAllSysChars action 917, 933
- DeleteChildType action 917, 933
- DeleteDirectory action 521, 525
- DeleteFields action 433, 440
- DeleteFile action 521, 526
- DeleteFingerprint action 334, 337, 568, 569
- DeleteFingerprints action 568, 569
- DeleteImageFile action 968, 970
- DeleteLCSpaces action 917, 934
- DeleteParentObj action 917, 934
- DeleteSelectedChars action 917, 935
- DeleteWord action 597, 598
- density string values
 - interpreting 77
- DensityString 299
- DetailFix action 632, 640
- DICT
 - Setup DCO 299
 - Verification panel
 - Datacap Desktop 299
 - Datacap Web Client 299
- dictionaries
 - attaching to a field 89

- dictionaries (*continued*)
 - creating 88
 - creating for check box options 99
- dictionary
 - creating 89
- disabling breakpoints 146
- Disconnect action 834, 837
- Document assembly
 - description 49
- document creation and integrity checking
 - moving to the PageID task
 - profile 210
- document hierarchy
 - adding Flight Cost rule 86
 - adding pages 153
 - adding Validate Car Type rule 88
 - adding Validate Currency Field rule 85
 - assigning default rules 72
- Document hierarchy 17
- Document Hierarchy
 - updating 152
- document hierarchy application wizard
 - default 19
- document split from main branch
 - updating the TravelDocs application 222
 - assigning the Batch Splitting rule 223
 - routing the split document to a supervisor 223, 224
 - running a batch through the workflow 225
 - updating the Routing ruleset 222
- document structure
 - hierarchy 17
 - levels 17
- document types
 - creating 19
 - examining 4
- documents
 - assembling 51
 - checking integrity 52
 - converting 28
 - hierarchy-based 50
 - inputting into application 27
 - managing integrity problems 54
 - routing
 - using branching and splitting 201
 - specifying the structure 20
 - splitting from the main branch 222
 - assigning the Batch Splitting rule 223
 - routing the split document to a supervisor 223, 224
 - running a batch through the workflow 225
 - updating the Routing ruleset 222
 - structured 50
- DocumentVariable_ExportValue action 481
- DoMsgbox action 632, 641
- Done Field Statuses 256
- Done Page Statuses 256
- DotEdit panels
 - converting to Datacap Desktop 275
 - reviewing layout XML files 275

- double blind verification 239
- Double blind verification 236
- Dutch
 - language codes 63
- dynamic locale support
 - language settings 62
 - locale variables
 - overriding 61
 - setting 61
 - overview 60
 - recognition languages 62

E

- Eastern European
 - language codes 63
- electronic documents 27
- elements
 - smart parameters 167
- Email actions
 - SendEmail 463
 - SetAttachment 463, 464
 - SetBlindCarbonCopyRcpts 463, 464
 - SetCarbonCopyRcpts 463, 465
 - SetEmailBody 463, 465
 - SetMailServer 463, 466
 - SetRecipients 463, 467
 - SetSender 467
 - SetSubject 463, 468
- Email Connector actions
 - configuring 131
 - file upload examples 132
 - overview 127
 - parameter settings 129
 - prerequisites 128
- Email Input actions
 - description 127
- EmptyFieldValue action 917, 935
- EnableEngineLogsOCR_A action 799, 800
- EnableLoggingICR_C action 591
- enabling logging
 - Datacap Web Client 143
- Enabling manual page registration 241
- English
 - language codes 63
- Equalize actions
 - EqualizeUnbalancedImage 468
- EqualizeUnbalancedImage action 468
- EraseRect action
 - description 1018
- Ewsml actions
 - ex_abort_time 469, 470
 - ex_done_folder 469, 470
 - ex_EMLOption 469, 471
 - ex_ews_version 469, 472
 - ex_HTTP_timeout 473
 - ex_HTTP_Timeout 469
 - ex_load_properties_option 469, 473
 - ex_login 469, 474
 - ex_logout 469, 475
 - ex_max_docs 469, 476
 - ex_problem_folder 469, 477
 - ex_scan 469, 478
 - ex_types 469, 479
 - ex_wait_time 469, 480
- ex_abort_time action 469, 470

- ex_done_folder action 469, 470
- ex_EMLOption action 469, 471
- ex_ews_version action 469, 472
- ex_HTTP_timeout action 469, 473
- ex_load_properties_option action 469, 473
- ex_login action 469, 474
- ex_logout action 469, 475
- ex_max_docs action 469, 476
- ex_problem_folder action 469, 477
- ex_scan action 469, 478
- ex_types action 469, 479
- ex_wait_time action 469, 480
- examples
 - file uploads
 - Email Connector actions 132
 - FileNet Image Services Connector 126
 - FileNet P8 Connector 116
 - IBM Content Manager Connector 111
 - SharePoint Connector 121
 - import from Fax 135
- Excel actions
 - ExcelAutoFitColumns 386, 387
 - ExcelAutoFitRows 386, 387
 - ExcelOrientationToLandscape 386, 388
 - ExcelOrientationToPortrait 386, 389
 - ExcelPrintBlankPage 386, 389
 - ExcelPrintGridlines 386, 390
 - ExcelPrintQuality 386, 391
 - ExcelScalingFactor 386, 392
 - ExcelTiffCompression 386, 392
 - ExcelWorkbookToImage 386, 393
- ExcelAutoFitColumns action 386, 387
- ExcelAutoFitRows action 386, 387
- ExcelOrientationToLandscape action 386, 388
- ExcelOrientationToPortrait action 386, 389
- ExcelPrintBlankPage action 386, 389
- ExcelPrintGridlines action 386, 390
- ExcelPrintQuality action 386, 391
- ExcelScalingFactor action 386, 392
- ExcelTiffCompression action 386, 392
- ExcelWorkbookToImage action 386, 393
- ExceptionSetFileTypes action 383
- ExceptionSetHandler action 383, 384
- ExceptionSetTaskCondition action 383, 385
- ExceptionSetVariableName action 383, 384
- ExecuteSQL action 712, 714
- ExecuteSQLBind action 632, 641
- Export actions
 - BatchVariable_ExportValue 481, 482
 - BlankFields 481, 482
 - BlankLines 481, 483
 - BPilot 481, 483
 - CloseExportFile 481, 484
 - DCOProperty 481, 485
 - DocumentVariable_Export Value 486
 - DocumentVariable_ExportValue 481
 - ExportAllFields 481, 486
 - ExportFieldValue 481, 487
 - ExportMYValue 481, 487

- Export actions *(continued)*
 - ExportSmartParameter 481, 488
 - ExportToBatchDir 481, 488
 - Filler 481, 489
 - FixedLenLJ 481, 489
 - FixedLenRJ 481, 490
 - GetDATE 481, 490
 - GetProfileString 481, 491
 - GetTime 481, 492
 - LineItem_AddElement 481, 493
 - LineItem_BlankFields 481, 493
 - LineItem_ClearElements 481, 494
 - LineItem_ExportElements 481, 495
 - LineItem_SmartParameter 481, 495
 - NewLine 481, 496
 - PageVariable_ExportValue 481, 496
 - ResetFieldVariables 481, 497
 - SaveFilePathAsVariable 481, 497
 - SetCSV 481, 498
 - SetElementSeparator 481, 499
 - SetExportPath 481, 499
 - SetExtensionName 481, 500
 - SetFileName 481, 501
 - SetFill 481, 501
 - SetFixedLength 481, 502
 - SetIgnoreFieldStatus 481, 502
 - SetJustified 481, 503
 - SetOMR_Separator 481, 504
 - SetSpaceFill 481, 504
 - SetZeroFill 481, 505
 - Text 481, 505
 - Variable_ExportValue 481, 506
 - Variable_IsValue 481, 506
- export database
 - configuring 137
- export database table
 - creating 163
- Export Rental Agreement Data rule
 - attaching to the rental agreement page 139
- export task
 - testing 139, 142
- Export task profile
 - adding a ruleset
 - ExportDB ruleset 138
 - adding the ruleset
 - Export XML ruleset 141
- Export XML rules
 - attaching to the document hierarchy 141
- ExportAllFields action 481, 486
- ExportBatchIDToColumn action 507, 508
- ExportCloseConnection action 507, 509
- ExportDB actions
 - AddRecord 507
 - ExportBatchIDToColumn 507, 508
 - ExportCloseConnection 507, 509
 - ExportFieldToColumn 507, 510
 - ExportNodeXMLToColumn 507, 511
 - ExportOpenConnection 507, 512
 - ExportPropertyToColumn 507, 513
 - ExportSmartParam ToColumn 514
 - ExportSmartParamToColumn 507
 - ExportToColumn 507, 515
 - SetTableName 507, 516
- ExportDB ruleset
 - adding rules 163

- ExportDB ruleset *(continued)*
 - creating 138
- ExportFieldToColumn action 507, 510
- ExportFieldValue action 481, 487
- Exporting information from the DCO 266
- exporting position information 267
- exporting text
 - IBM Content Manager OnDemand 104
- exporting to XML
 - creating ruleset 140
- ExportMYValue action 481, 487
- ExportNodeXMLToColumn action 507, 511
- ExportOpenConnection action 507, 512
- ExportPropertyToColumn action 507, 513
- ExportSmartParameter action 481, 488
- ExportSmartParamToColumn action 507
- ExportToBatchDir action 481, 488
- ExportToColumn action 507, 515
- ExportXML actions
 - xml_CommitNode 517
 - xml_NewNode 517, 518
 - xml_SaveFile 517, 518
 - xml_SetAttributeValue 517, 519
 - xml_SetExportPath 517, 520
 - xml_SetFileName 517, 520
 - xml_SetNodeValue 517, 521
- ExportXML ruleset
 - accessing the runtime hierarchy 172, 173
 - creating 140

F

- FailRuleSet action 917, 936
- Fax Actions
 - authentication 107
- Fax Connector actions 133
 - configuring 135
 - parameter settings 134
 - prerequisites 134
- Fax Connector Actions
 - import examples 135
- field data
 - locating with text matching 181
- field definitions
 - sharing across the document hierarchy 23
- field values
 - specifying 9
- FieldContainsValue action 917, 936
- fields
 - data verification 94
 - on pages
 - specifying structure 23
 - preventing validation failure overrides 99
- FileIO actions
 - AppendAllImages 603
 - AppendAllImages_ByType 603
 - AppendImage 604
 - AppendImage_StartAsNew 605
 - CheckFreeDiskSpace 522
 - CheckFreeSpace 521

- FileIO actions *(continued)*
 - ConvertToJPEG 606
 - ConvertToTIFF 606
 - CopyDirectory 521, 523
 - CopyFile 521, 524
 - DeleteDirectory 521, 525
 - DeleteFile 521, 526
 - GetFileSize 521, 527
 - GetProfileString 521, 528
 - IsDirectoryPresent 521, 529
 - IsFilePresent 521, 530
 - IsFileReadOnly 521, 530
 - IsProfilePresent 521, 531
 - RenameFile 521, 532
 - SetChrominanceFactor 607
 - SetDeleteOriginal 608
 - SetFileReadOnly 521, 533
 - SetGrayScale 608
 - SetLuminanceFactor 609
 - SetProfileString 521, 534
 - SetTIFFCompression 610
 - SplitFileName 521, 535
- FileNet Doc ID Set Value action 536
- FileNet Image Services Connector
 - file upload examples 126
- FileNet Image Services Connector actions
 - configuring 125
 - overview 123
 - parameter settings 124
 - prerequisites 123
- FileNet P8 actions
 - FNFP8_SetDestinationFolder 554
 - FNFP8_SetDocClassId 554
 - FNFP8_SetDocTitle 554
 - FNFP8_SetFileType 554
 - FNFP8_SetKeyProperty 554
 - FNFP8_SetLocale 554
 - FNFP8_SetMimeType 554
 - FNFP8_SetMultiValueProperty 554
 - FNFP8_SetProperty 554
 - FNFP8_SetRetry 554
 - FNFP8_SetTargetClassID 554
 - FNFP8_SetTargetObjectID 554
 - FNFP8_SetTimeout 554
 - FNFP8_SetUploadMode 554
 - FNFP8_SetURL 554
 - FNFP8_CreateFolder 554
 - FNFP8_Login 554, 555
 - FNFP8_MultiPageDocs 555
 - FNFP8_SetDestinationFolder 556
 - FNFP8_SetDocClassId 557
 - FNFP8_SetDocTitle 557
 - FNFP8_SetFileType 558
 - FNFP8_SetKeyProperty 558
 - FNFP8_SetLocale 559
 - FNFP8_SetMultiValue Property 560
 - FNFP8_SetProperty 560
 - FNFP8_SetPropertyEx 561
 - FNFP8_SetRetry 562
 - FNFP8_SetTargetClassID 563
 - FNFP8_SetTargetObjectID 563
 - FNFP8_SetTimeout 564
 - FNFP8_SetUploadMode 564
 - FNFP8_SetURL 565
 - FNFP8_UpdateProperties 554, 565
 - FNFP8_Upload 554, 566
 - FNFP8_UploadDir 554, 567

- FileNet P8 Connector
 - file upload examples 116
- FileNet P8 Connector actions
 - configuring 115
 - parameter settings 114
 - prerequisites 113
- FileNetDB_ADOConnect action 536, 540
- FileNetDocID_SaveAsSmartParameter action 536
- FileNetDocID_SetValue action 536
- FileNETDocID_SetValue action 541
- FileNetIDM
 - Authentication 107
- FileNetIDM actions
 - AddAllImagesToDocument 536, 537
 - AddFileToDocument 536, 537
 - AddPDFImageToDocument 536, 538
 - AddTIFFImageToDocument 536, 539
 - CreateFolder 536, 540
 - FileNet Doc ID Set Value 536
 - FileNetDB_ADOConnect 536, 540
 - FileNetDocID_SaveAsSmartParameter 536
 - FileNETDocID_SaveAsSmartParameter 541
 - FileNetDocID_SetValue 536
 - FileNETDocID_SetValue 541
 - GetDocuments 536, 542
 - GetTopFolders 536, 542
 - IndexProperty_ID_DateComponent 544
 - IndexProperty_ID_Component 536, 543
 - IndexProperty_ID_DateComponent 536
 - IndexProperty_ID_Value 536
 - IndexProperty_LeftJUSTIFY 536, 545
 - IndexProperty_RightJUSTIFY 536, 546
 - IndexProperty_SmartParameter 536, 546
 - IndexProperty_ID_Value 544
 - Library_DMA_Initialize 536, 547
 - Library_DS_Initialize 536, 547
 - Library_IS_Initialize 536, 548
 - Library_LogIn 536, 549
 - Library_LogOff 536, 549
 - NewDocument 536, 550
 - SaveDocToFolder 536, 551
 - Upload 536, 551
 - Upload_SetNumAttempts 536, 552
 - UploadSetDelay 536
 - UseIndexes_OFF 536, 552
 - UseIndexes_ON 553
 - UseIndexes_ON 536
- Filler action 481, 489
- Filter batches by group
 - Group authentication
 - ADSI group authentication 259
 - LDAP group authentication 259
 - LLLDAP group authentication 259
 - Job Monitor 259
- FilterFieldSelectedChars action 917, 937
- FilterIt action 664, 667
- FilterPID action 728
- FindBlackFingerprint action 334, 337
- FindBlocks_WhiteSpace action 988, 991
- FindDataBlocks action 988, 991
- FindDBList action 664, 668
- FindDBList_InZone action 664, 669
- FindExportImage action 632, 642
- FindFields action 718
- FindFingerprint action 334, 338
- FindFingerprintCC action 359
- FindKeyList action 664, 669
- FindKeyList_InZone action 664, 670
- FindLastKeyList action 664, 671
- FindLastKeyList_InZone action 664, 672
- FindLastRegEx action 664, 673
- FindLastRegEx_InZone action 664, 674
- FindLastRegExList action 664, 674
- FindLastRegExList_InZone action 664, 675
- FindLastWord action 664, 676
- FindLastWord_InZone action 664, 677
- FindLineItems action 988, 992
- FindNextDBList action 664, 677
- FindNextDBList_InZone action 664, 678
- FindNextKeyList action 664, 679
- FindNextKeyList_InZone action 664, 680
- FindNextRegExList action 664, 681
- FindNextRegExList_InZone action 664, 682
- FindRegExBlocks action 988, 992
- FindRegExList action 664, 683
- FindRegExList_InZone action 664, 684
- FindTaxValue action
 - description 1016
- FindTemplate action 334, 339
- FindZoneLineItems action 988, 993
- fingerprint 40
 - applying new image processing settings 41
 - changing creation method 39
 - creating classes 39
 - enhancing images 41
- Fingerprint Created 296
- fingerprint database
 - description 263
- fingerprint files
 - creating 262
- Fingerprint functionality 262
- fingerprint generation
 - automatic 199
- fingerprint library
 - add fingerprints 262
 - creating initial for TravelDocs 38
- Fingerprint Maintenance Tool
 - setup 266
- Fingerprint Management 261
- fingerprint matching
 - creation modes 34
 - full page recognition 34
 - image analysis 34
- Fingerprint Service
 - Actions
 - SetApplicationID 340
- Fingerprint XML file
 - FPXML 264
 - Zone position 264
- fingerprint XML files 264
- FingerprintMaintenance actions
 - CloseDatabase 568
- FingerprintMaintenance actions
 - (continued)
 - DeleteFingerprint 568, 569
 - DeleteFingerprints 568, 569
 - OpenDatabase 568, 570
 - SetFingerprintFolder 568, 571
- fingerprints
 - adding
 - using actions 266
 - using the Datacap Studio Zones tab 265
 - creating 205
 - creating for page types 39
 - creating recognition zones 92
 - defining field zones 263
 - enabling FPXML 265
 - generating automatically 205, 218
- FixedLenLJ action 481, 489
- FixedLenRJ action 481, 490
- Flight Cost rule
 - adding to document hierarchy 86
- FP8_SetDestinationFolder action 554
- FP8_SetDocClassId action 554
- FP8_SetDocTitle action 554
- FP8_SetFileType action 554
- FP8_SetKeyProperty action 554
- FP8_SetLocale action 554
- FP8_SetMimeType action 554
- FP8_SetMultiValueProperty action 554
- FP8_SetProperty action 554
- FP8_SetRetry action 554, 562
- FP8_SetTargetClassID action 554
- FP8_SetTargetObjectID action 554
- FP8_SetTimeout action 554
- FP8_SetUpdateProperties action 554
- FP8_SetUploadMode action 554
- FP8_SetURL action 554
- FP8_CreateFolder action 554
- FP8_Login action 554, 555
- FP8_MultiPageDocs action 555
- FP8_SetDestinationFolder action 556
- FP8_SetDocClassId action 557
- FP8_SetDocTitle action 557
- FP8_SetFileType action 558
- FP8_SetKeyProperty action 558
- FP8_SetLocale action 559
- FP8_SetMultiValue Property action 560
- FP8_SetProperty action 560
- FP8_SetPropertyEx action 561
- FP8_SetTargetClassID action 563
- FP8_SetTargetObjectID action 563
- FP8_SetTimeout action 564
- FP8_SetUploadMode action 564
- FP8_SetURL action 565
- FP8_UpdateProperties action 565
- FP8_Upload action 554, 566
- FP8_UploadDir action 554, 567
- FormatFieldLengths action 729
- FormatNumberToLocale action 917, 937
- FPXML
 - auto fingerprinting 267
 - enabling 265
 - Fingerprint XML file
 - Zone position 264

- FPXML actions
 - ReadZonesFPX 571, 572
 - SetDetailsAndLineitemPairFPX 571, 573
 - SetDirectoryFPX 571, 574
 - WriteZoneFPX 571, 575
 - WriteZonesFPX 571, 575
- FPXMLUsed action 632, 642
- French
 - language codes 63
- full breakpoints 146

G

- generate
 - layout XML files 275
- GenerateDetails action 632, 643
- generation
 - automatic fingerprint generation 205
 - fingerprints 205
- generic breakpoints
 - setting 147
- geometric pattern matching
 - pattern matching 191
 - updating the TravelDocs application 196
 - reviewing runtime batch files 198
 - running a batch through the workflow 198
 - setting up anchor zones 197
 - updating the PageID rule 197
- using 191
- German
 - language codes 63
- Get2DCodesBP action 348
- GetAllBarcodes action
 - description 1018
- GetAllBarcodesBP action 348, 349
- GetBarcode action 356, 357
- GetBarcodeBP action 350
- GetBarcodesBP action 348
- GetDataMatrixCodeBP action 348, 352
- GetDATE action 481, 490
- GetDocuments action 536, 542
- GetFileSize action 521, 527
- GetJobID action 917, 938
- GetProfileString action 481, 491, 521, 528
- GetTime action 481, 492
- GetTopFolders action 536, 542
- GetZoneText action 988, 994
- global actions 333
 - Application setup 755
 - Autodoc 334
 - Barcode_P 348
 - Barcode_X 356
 - Batch processing 779
 - CC 359
 - Cco2cco 364
 - ColorToBW 380
 - Common 383
 - Convert 382
 - Common 383
 - Excel 386
 - Html 394
 - Images 396
 - Outlook 401
 - Pdf 404

- global actions (*continued*)
 - Convert (*continued*)
 - PdfFRE 410
 - Rtf 416
 - Tiff 418
 - Txt 420
 - Word 424
 - Zip 427
 - Dcclip 429
 - DCImageFix 430
 - DCO 433
 - dcpdf 450
 - Email 463
 - Equalize 468
 - Ewsml 469
 - Excel 386
 - Export 481
 - ExportDB 507
 - ExportXML 517
 - FileIO 521
 - FileNet P8 554
 - FileNetIDM 536
 - FingerprintMaintenance 568
 - FPXML 571
 - Grayscale 576
 - Html 394
 - IBMCM 577
 - ICR_C 591
 - ICR_P 597
 - ImageConvert 602
 - ImageFix 610
 - Images 396
 - Imail 610
 - Imprint 622
 - Intellocate 629
 - Invoice 632
 - IOOverlay 661
 - Locate 664
 - Logging 791
 - Lookup 712
 - Maintenance Manager 755
 - Application setup 755
 - Batch processing 779
 - Logging 791
 - Query setup 764
 - Reporting 796
 - MC_Identify 717, 1013
 - MC_Validation 721
 - mvscan 741
 - OCR_A 799
 - OCR_N 811
 - OCR_S 813
 - OCR_SR 826
 - OpenTextFaxServer 834
 - Outlook 401
 - overview 333
 - PatternMatch 850
 - Pdf 404
 - PdfFRE 410
 - Picture 857
 - POLR 864
 - Query setup 764
 - Recog_Shared 865
 - Reporting 796
 - rrunner 881
 - Rtf 416
 - SPEXport 903

- global actions (*continued*)
 - Split 909
 - Tiff 418
 - TifMerge 911
 - TM524 917
 - Txt 420
 - Validations 917
 - Vote 968
 - Vscan 968
 - Web Services 979
 - Word 424
 - Zip 427
 - Zones 988
- GoAboveWord action 664, 685
- GoBelowWord action 664, 686
- GoDownLine action 664, 686
- GoFirstLine action 664, 687
- GoFirstWord action 664, 688
- GoLastLine action 664, 688
- GoLastWord action 664, 689
- GoLeftWord action 664, 689
- GoRightWord action 664, 690
- GoToNextFunction action 881, 885
- GoUpLine action 664, 691
- Grayscale actions
 - ConvertGraytoBW 576, 577
- grid totals
 - validating 160, 161
- Group authentication
 - ADSI group authentication 259
 - Filter batches by group
 - Job Monitor 259
 - LDAP group authentication 259
 - LLLDAP group authentication 259
- GroupWords action 664, 691
- GroupWordsLEFT action 664, 692
- GroupWordsRIGHT action 664, 693

H

- handle document failures
 - updating the TravelDocs application
 - configuring Rulerunner to run
 - CreateDocs 211
 - creating the CreateDocs task 210
 - moving document creation and integrity checking 210
 - running a batch through the workflow 211
- handle document integrity failures
 - updating the TravelDocs application 210
- handling line item grids 148
- Hardcopy local scans
 - Datacap Desktop 28
 - Datacap Web Client 28
- HasChildOfType action 939
 - description 917
- hr_locale
 - overview 60
- hr_locale variable
 - description 294
 - language settings 62
 - overriding values 61
 - recognition languages 62
 - setting values 61

- Html actions
 - HtmlPrintQuality 394
 - HtmlTiffCompression 394, 395
 - HtmlToImage 394, 395
- HtmlPrintQuality action 394
- HtmlTiffCompression action 394, 395
- HtmlToImage action 394, 395

I

- IBM Content Manager
 - Authentication 107
- IBM Content Manager Connector
 - file upload examples 111
- IBM Content Manager Connector actions
 - configuring 111
 - overview 109
 - parameter settings 110
 - prerequisites 109
- IBMCM actions
 - IBMCM_AddPages 578
 - IBMCM_CreateFolder 577, 579
 - IBMCM_CreateItem 577, 580
 - IBMCM_DeletePages 581
 - IBMCM_Logon 577, 582
 - IBMCM_ReplacePage 583
 - IBMCM_SearchItem 584
 - IBMCM_SetAttributeValue 577, 585
 - IBMCM_SetDestinationFolder 577, 587
 - IBMCM_SetFolderAttribute 577
 - IBMCM_SetMimeType 586
 - IBMCM_Store Folder ID In DCO 577
 - IBMCM_Store Item In DCO 577
 - IBMCM_StoreItemIDInDCO 588
 - IBMCM_UploadDCO_DOC 577, 589
 - IBMCM_UploadDCO_Page 577, 590
- IBMCM_AddPages action 578
- IBMCM_CreateFolder action 577, 579
- IBMCM_CreateItem action 577, 580
- IBMCM_DeletePages action 581
- IBMCM_Logon action 577, 582
- IBMCM_ReplacePage action 583
- IBMCM_SearchItem action 584
- IBMCM_SetAttributeValue action 577, 585
- IBMCM_SetDestinationFolder action 577, 587
- IBMCM_SetFolderAttribute action 577
- IBMCM_SetMimeType action 586
- IBMCM_Store Folder ID In DCO action 577
- IBMCM_Store Item In DCO action 577
- IBMCM_StoreItemIDInDCO action 588
- IBMCM_UploadDCO_DOC action 577, 589
- IBMCM_UploadDCO_Page action 577, 590
- ICR_C actions
 - EnableLoggingICR_C 591
 - RecognizeFieldICR_C 591, 592
 - RecognizeFieldICR_CEx 591
 - RecognizeFieldVoteICR_C 591, 592
 - RecognizePageFields ICR_CEx 595
 - RecognizePageFields2CCO_ICR_C 593

- ICR_C actions (*continued*)
 - RecognizePageFields2CCO_ICR_C 591
 - RecognizePageFieldsICR_C 591, 594
 - RecognizePageICR_C 591, 595
 - RecognizePageToPDFICR_C 591, 596
- ICR_P actions
 - AddWord 597
 - DeleteWord 597, 598
 - ImportCSF 597, 598
 - LoadFromFile 597, 599
 - NewDictionary 597, 599
 - RecognizeFieldsICR_P 600
 - RecognizePageFieldsICR_P 597
 - SaveToFile 597, 601
 - SetPostalDBPathICR_P 597, 602
- identify pages manually
 - updating the TravelDocs application 212
 - adding a function 213
 - adding the conditional branch to the PageID task 215
 - configuring branching 216
 - configuring the Routing ruleset 216
 - creating the ManualPageID job and task 215
 - recognizing the data on an unidentified page 218
 - running a batch through the workflow 217
 - updating the Recognize Page ruleset 213
- IdentifyByBarcodesBP action 353
- Ignored Field Statuses 256
- iloc_AdjustZones action 629
- iloc_AssignPageType action 629, 630
- iloc_SetDetailSimple action 632, 643
- iloc_SetDetailZones action 629, 631
- iloc_SetZones action 629, 631
- im_abort_time action 610, 611
- im_AcceptMixedAttachments action 610
- im_AcceptNoAttachments action 610, 612
- im_done_folder action 610, 613
- im_login action 610, 614
- im_logout action 610, 615
- im_max_docs action 610, 615
- im_problem_folder action 610, 616
- im_scan action 610, 617
- im_SortByDate action 618
- im_StoreByDate action 610
- im_StoreEML action 610, 619
- im_types action 610, 620
- im_UseSSL action 610, 621
- im_wait_time action 610, 621
- image adjustment
 - pattern matching 187
- image enhancement
 - eliminating interference 37
 - sample fingerprints 40
 - when to complete 38
- image processing
 - determining settings 40
 - settings 40
- Image_Offset 296

- ImageConvert actions
 - AppendAllImages 602
 - AppendAllImages_ByType 602
 - AppendImage 602
 - AppendImage_StartAsNew 602
 - ConvertToJPEG 602
 - ConvertToTIFF 602
 - SetChrominanceFactor 602
 - SetDeleteOriginal 602
 - SetGrayScale 602
 - SetLuminanceFactor 602
 - SetTIFFCompression 602
- ImageDefaultDPI action 396, 397
- ImageEnhance action 430
- IMAGEFILE 296
- ImageFileTypesToConvert action 396, 397
- ImageFix actions 610
- ImageMonoThreshold action 396, 398
- ImageMonoType action 396, 399
- Images actions
 - ImageDefaultDPI 396, 397
 - ImageFileTypesToConvert 396, 397
 - ImageMonoThreshold 396, 398
 - ImageMonoType 396, 399
 - ImageToTIFF 396, 400
- ImageToTIFF action 396, 400
- Imail actions
 - im_abort_time 610, 611
 - im_AcceptMixedAttachments 610, 612
 - im_AcceptNoAttachments 610, 612
 - im_done_folder 610, 613
 - im_login 610, 614
 - im_logout 610, 615
 - im_max_docs 610, 615
 - im_problem_folder 610, 616
 - im_scan 610, 617
 - im_SortByDate 618
 - im_StoreByDate 610
 - im_StoreEML 610, 619
 - im_types 610, 620
 - im_UseSSL 610, 621
 - im_wait_time 610, 621
- ImgEnter web client 245
- ImportCSF action 597, 598
- ImportFaxes action 834, 838
- Imprint action 622
- ImPrint action 623
- Imprint actions
 - AnnotateImage 622
 - Imprint 622
 - ImPrint 623
 - Redact 622, 624
 - RedactByRegEx 622, 625
 - RedactParameters 622, 625
 - SetAdjustedWidth 622, 626
 - SetFontName 622, 627
 - SetFontSize 622, 628
 - SetOpaque 622, 628
- IncrementBatchVar action 632, 644
- Index 299
- IndexProperty_ID_Component action 536, 543
- IndexProperty_ID_DateComponent action 536
- IndexProperty_ID_Value action 536

- IndexProperty_ LeftJUSTIFY action 536, 545
- IndexProperty_ RightJUSTIFY action 536, 546
- IndexProperty_ SmartParameter action 536, 546
- IndexProperty_ID_Value action 544
- individual field positions
 - adjusting for pattern matching 196
- InheritParentPosition action 988, 994
- InheritSnippets action 729
- InsertChars action 917, 939
- InsertDecimalPoint action 917, 940
- installation
 - verifying
 - Connector actions 106
- Intellocate actions
 - iloc_AdjustZones 629
 - iloc_AssignPageType 629, 630
 - iloc_SetDetailZones 629, 631
 - iloc_SetZones 629, 631
 - IsPageDataMissing 629, 632
- Invoice actions
 - AddToDetailErrorMsg 632, 634
 - AddToErrorMsg 632, 634
 - AllMixedCase 632, 635
 - AllowOnlyChars 632, 635
 - AlterDatebyDay 632, 636
 - CalculateNotesZone 632, 636
 - CaptureOpInfo 632, 637
 - CheckAndFixDecimal 632, 637
 - CheckForSticky 632, 638
 - CheckFreeDiskSpace 632, 638
 - ClearErrorMsg 632, 639
 - CreateFingerprint 632, 640
 - DetailFix 632, 640
 - DoMsgbox 632, 641
 - ExecuteSQLBind 632, 641
 - FindExportImage 632, 642
 - FPXMLUsed 632, 642
 - GenerateDetails 632, 643
 - iloc_SetDetailSimple 632, 643
 - IncrementBatchVar 632, 644
 - Is_InCharSet 632, 650
 - Is_JobName 632, 651
 - Is_JobNamePrefix 651
 - IsChildFieldBlank 632, 644
 - IsChildFieldValue 632, 645
 - IsCurrentObjValue 632, 645
 - IsCurrentObjVariable 632, 646
 - IsFingerPrintClass 632, 646
 - IsInINI 632, 647
 - IsInList 632, 647
 - IsMultipageDocument 632, 648
 - IsSinglePageDocument 632, 648
 - IsStationIDSuffix 632, 649
 - IsTaskName 632, 649
 - JobNamePrefix 632
 - LoadCCOFFromField 632, 652
 - MovePDF 632, 652
 - OpenConnection 632, 653
 - ParseImageName 632, 653
 - PopulateZNLinItem
 - FieldDynamic 632, 654
 - ReadFPXMLZones 632, 655
 - SaveObjectVariable 632, 655
 - ScanLineItemDynamic 632, 655

- Invoice actions (*continued*)
 - SendOutlookNotification 632, 656
 - SetDynamicDetailZones 632, 657
 - SetPicChar 632, 657
 - SetStickyNo 632, 657
 - SetToDocIDMPTIFF 632, 658
 - SwapImages 632, 658
 - SwitchMMDD 632, 659
 - UpdateFPStats 632, 659
 - ValidateVendor 632, 660
 - WriteErrorMessage 632, 660
- IOOverlay actions
 - Overlay 661
 - SetBackgroundImage 661, 662
 - SetDitheringBackground 661, 663
 - SetHaloBackground 661, 663
- Is_InCharSet action 632, 650
- Is_JobName action 632, 651
- Is_JobNamePrefix action 651
- IsAlpha action 664, 693
- IsBlankPage action 865, 868
- IsChildFieldBlank action 632, 644
- IsChildFieldValue action 632, 645
- IsCurrency action 664, 694
- IsCurrentObjValue action 632, 645
- IsCurrentObjVariable action 632, 646
- IsDate_FormatEuro action
 - description 1016
- IsDateValue action 664, 695
- IsDirectoryPresent action 521, 529
- IsDocumentCountMoreThan action 441
- IsFieldCurrency action 917, 940
- IsFieldDate action 917, 941
- IsFieldDateEqualOrAfter action 917, 942
- IsFieldDateEqualOrBefore action 917, 942
- IsFieldDateUpToToday action 917, 943
- IsFieldDateWithinRange action 917, 943
- IsFieldDateWithinXDays action 917, 944
- IsFieldDateWithReformat action 917, 944
- IsFieldEmpty action 917, 945
- IsFieldFilled action 917, 945
- IsFieldGreaterOrEqual action 917, 946
- IsFieldHidden action 917, 946
- IsFieldLengthMax action 917, 947
- IsFieldLengthMin action 917, 947
- IsFieldLessOrEqual action 917, 948
- IsFieldLocalCurrency action
 - description 1015
- IsFieldMatching action 917, 948
- IsFieldPercentAlpha action 917, 949
- IsFieldPercentNonNumeric action 917, 949
- IsFieldPercentNumeric action 917, 950
- IsFilePresent action 521, 530
- IsFileReadOnly action 521, 530
- IsFingerPrintClass action 632, 646
- IsFirstDocInBatch action
 - description 1017
- IsFirstDocumentInBatch action 442
- IsInINI action 632, 647
- IsInList action 632, 647
- IsInvoiceFromUS action
 - description 1016
- IsLocalDecimalSeparator action
 - description 1015
- IsMatchingJobID action 917, 951

- IsMaxOMRChecked action 917, 951
- IsMinOMRChecked action 917, 951
- IsMultipageDocument action 632, 648
- IsNumber action 664, 696
- IsOriginalEuroFormat action
 - description 1015
- IsPageDataMissing action 629, 632
- IsPatternInField action 917, 952
- IsProfilePresent action 521, 531
- IsSinglePageDocument action 632, 648
- IsStationIDSuffix action 632, 649
- IsSupportedImageFile action 917, 953
- IsTaskName action 632, 649
- IsThisFieldEmpty action 917, 953
- IsThisFieldFilled action 917, 954
- IsValue action 664, 696
- IsValue_RegEx action 664, 697
- IsVariableEmpty action 917, 954
- IsVariableFilled action 917, 955
- IsWorkstationLocale action
 - description 1015
- Italian
 - language codes 63

J

- job information
 - accessing 174
- Job Monitor
 - setting up 208
- job monitoring 248
- JobNamePrefix action 632
- jobs
 - creating 205
 - creating to handle special conditions 204
 - description 24
- JoinPreviousDocument action 442

K

- key name
 - determining for smart parameters 169
- keyword lists
 - using with text matching 180

L

- Label
 - Setup DCO 299
 - Verification panel
 - Datacap Desktop 299
 - Datacap Web Client 299
- language codes
 - Chinese 63
 - Dutch 63
 - Eastern European 63
 - English 63
 - French 63
 - German 63
 - Italian 63
 - Portuguese 63
 - Russian 63
 - setting 294
 - Spanish 63

- language codes (*continued*)
 - Swedish 63
- language recognition
 - language settings 62
- language settings
 - dynamic locale support 62
 - hr_locale variable 62
 - recognition 62
- LAST_RR_PROFILE 294
- layout XML files
 - creating Datacap Desktop panels 276
 - generating 275
- layout XML files reviewing 275
- Learn_Zones action
 - description 1017
- Learn_ZonesFPX action
 - description 1017
- LeftTruncate action 917, 955
- Library_DMA_Initialize action 536, 547
- Library_DS_Initialize action 536, 547
- Library_IS_Initialize action 536, 548
- Library_LogIn action 536, 549
- Library_LogOff action 536, 549
- limitations
 - text matching for data
 - recognition 183
- line item grid data
 - attaching rules to document
 - hierarchy 165
 - exporting 166
 - exporting to an XML file 175
 - adding rules to the ExportXML
 - ruleset 175
 - attaching the Export Other XML
 - rules to the document
 - hierarchy 177
 - running a batch through the
 - workflow 177
- line item grid pages
 - verifying 162
- line item grids 148
 - adding existing page rules 155
 - attaching rules to the document
 - hierarchy 157
 - attaching the validation rule to the
 - DCO 160
 - creating data fields 154
 - creating page fingerprints 155
 - creating the validation rule 160
 - defining document hierarchy 149
 - defining recognition zones 155
 - exporting data 152
 - locating fields 150
 - recognition rule 157
 - creating for the grid total 157
 - recognizing data 156
 - removing non-line items 151
 - rules 149
 - rules to remove non-line items 158
 - running a batch through the
 - workflow 158
 - updating the document
 - hierarchy 152
 - validating data 159
 - validating line item totals 159
- LineItem_AddElement action 481, 493
- LineItem_BlankFields action 481, 493
- LineItem_ClearElements action 481, 494
- LineItem_ExportElements action 481, 495
- LineItem_SmartParameter action 481, 495
- LoadBlockCCO action 988, 995
- LoadCCOFromField action 632, 652
- LoadFromFile action 599
- LoadFromFileaction 597
- LoadSettings action 430, 431
- LoadSettings_FingerprintID action 430, 432
- LoadZones action 988, 995
- Local scanner setup 31
- locale variables
 - hr_locale 60
 - inheritance rules 60
 - overriding 61
 - setting 61, 294
- Locate actions
 - AddKeyList 664, 665
 - AggregateKeyList 664, 666
 - DefaultValue 664, 667
 - FilterIt 664, 667
 - FindDBList 664, 668
 - FindDBList_InZone 664, 669
 - FindKeyList 664, 669
 - FindKeyList_InZone 664, 670
 - FindLastKeyList 664, 671
 - FindLastKeyList_InZone 664, 672
 - FindLastRegex 664, 673
 - FindLastRegex_InZone 664, 674
 - FindLastRegexList 664, 674
 - FindLastRegexList_InZone 664, 675
 - FindLastWord 664, 676
 - FindLastWord_InZone 664, 677
 - FindNextDBList 664, 677
 - FindNextDBList_InZone 664, 678
 - FindNextKeyList 664, 679
 - FindNextKeyList_InZone 664, 680
 - FindNextRegexList 664, 681
 - FindNextRegexList_InZone 682
 - FindNextRegexList_InZone 664
 - FindRegexList 664, 683
 - FindRegexList_InZone 664, 684
 - GoAboveWord 664, 685
 - GoBelowWord 664, 686
 - GoDownLine 664, 686
 - GoFirstLine 664, 687
 - GoFirstWord 664, 688
 - GoLastLine 664, 688
 - GoLastWord 664, 689
 - GoLeftWord 664, 689
 - GoRightWord 664, 690
 - GoUpLine 664, 691
 - GroupWords 664, 691
 - GroupWordsLEFT 664, 692
 - GroupWordsRIGHT 664, 693
 - IsAlpha 664, 693
 - IsCurrency 664, 694
 - IsDateValue 664, 695
 - IsNumber 664, 696
 - IsValue 664, 696
 - IsValue_RegEx 664, 697
 - MaxLength 664, 698
 - MergeWordLF 664, 698
 - MergeWordRT 664, 699
- Locate actions (*continued*)
 - MinLength 664, 700
 - RegexFind 664, 700
 - RegexFind_InZone 664, 701
 - RegexFindNext 664, 702
 - RegexFindNext_InZone 702
 - ScanRT 664, 703
 - SelectSnippet 664, 704
 - SetRect 664, 705
 - UpdateDCOField 664, 705
 - UpdateField 664, 706
 - ValueInField 664, 707
 - ValueInField_Fuzzy 664, 707
 - ValueInField_RegEx 664, 708
 - WordFind 664, 708
 - WordFind_InZone 664, 709
 - WordFind_Offset 664, 711
 - WordFindNext 664, 710
 - WordFindNext_InZone 664, 711
- log files 143
 - Connector actions 136
 - examining in Datacap Studio 148
 - RRS 144
 - Rulerunner 144
 - tasks 145
- LogClear action 791
- LogConfigure action 791
- logging
 - Rulerunner 201
- Logging actions
 - LogClear 791
 - LogConfigure 791
 - LogSendEmail 791, 793
 - LogWriteEventLog 791, 794
 - LogWriteRecordSet 791, 795
 - LogWriteSQLQuery 791, 795
- LogSendEmail action 791, 793
- LogWriteEventLog action 791, 794
- LogWriteRecordSet action 791, 795
- LogWriteSQLQuery action 791, 795
- Lookup
 - Setup DCO 300
 - Verification panel
 - Datacap Desktop 300
 - Datacap Web Client 300
- Lookup actions
 - ClearLookupResults 712
 - CloseConnection 712, 713
 - ExecuteSQL 712, 714
 - OpenConnection 712, 715
 - PopulateWithResult 712, 715
 - SmartSQL 712, 716
- lookup database
 - validating car type 87
- lookup database table
 - creating 87
- LookupEx
 - Setup DCO 301
 - Verification panel
 - Datacap Desktop 301
 - Datacap Web Client 301

M

- Maintenance Manager actions
 - Application setup 755
 - SetAdminDB 755, 756

- Maintenance Manager actions *(continued)*
 - Application setup *(continued)*
 - SetApplication 755, 757
 - SetEngineDB 755, 757
 - SetPassword 755, 758
 - SetServer 755, 759
 - SetStation 755, 760
 - SetupDisconnectAll 755, 760
 - SetupOpenApplication 755, 761
 - SetupOpenApplicationEx 755, 762
 - SetUser 755, 763
 - Batch processing 755
 - ProcessChange BatchStatus 780
 - ProcessChangeBatch
 - StatusOrder 779, 780
 - ProcessChangeBatch
 - StatusTaskOrder 779
 - ProcessChangeBatchStatus 779
 - ProcessChangeBatchStatus
 - TaskOrder 781
 - ProcessClearAuditTable 779, 782
 - ProcessClearDebugTable 779, 783
 - ProcessDeleteBatches 779, 783
 - ProcessDeleteBatchesEx 779, 784
 - ProcessInjectBatches 779, 784
 - ProcessMoveBatches 779, 785
 - ProcessMoveBatchesEx 786
 - ProcessMoveBatchesEX 779
 - ProcessMoveDBRecords 779, 787
 - ProcessReset Pending
 - OrNotify 788
 - ProcessResetPendingOrNotify 779
 - ProcessRunSqlQuery 779, 790
 - Logging 755
 - LogClear 791
 - LogConfigure 791
 - LogSendEmail 791, 793
 - LogWriteEventLog 791, 794
 - LogWriteRecordSet 791, 795
 - LogWriteSQLQuery 791, 795
 - Query setup 755
 - QueryClear 764
 - QuerySetAge 764, 765
 - QuerySetBatchRange 764, 766
 - QuerySetBranch 764, 767
 - QuerySetDateFormat 764, 767
 - QuerySetDateRange 764, 770
 - QuerySetDateTimeFormat 764, 771
 - QuerySetGeneric 764, 773
 - QuerySetJobID 764, 774
 - QuerySetOperator 764, 775
 - QuerySetPriority 764, 775
 - QuerySetSeparator 764, 776
 - QuerySetStation 764, 777
 - QuerySetStatus 764, 777
 - QuerySetTaskID 764, 778
 - Reporting 755
 - ReportQueryTMUsage 796
 - ReportSetReportingTable 796, 797
 - ReportSetUsageDBTable 796, 798
 - MakeFieldHighConfidence action
 - description 1016
 - manual page identification 37
 - adding a function 213
 - adding routing to enable 212
 - manual page identification *(continued)*
 - adding the conditional branch to the PageID task 215
 - configuring branching 216
 - configuring the Routing ruleset 216
 - creating the ManualPageID job and task 215
 - recognizing the data on an unidentified page 218
 - running a batch through the workflow 217
 - updating the Recognize Page ruleset 213
 - Manual page identification and registration 240
 - ManualIDValidate rule
 - creating 257
 - testing 259
 - ManualPageID
 - updating 255
 - ManualPageID job
 - creating 215
 - ManualPageID settings
 - editing 256
 - ManualPageID task
 - creating 215
 - MatchBarCode action 356, 358
 - MatchBarcodeBP action 348, 354
 - MatchBarcodePrefixBP action 348, 354
 - MatchPattern action 850
 - MAX_TYPES 291
 - MaxLength
 - Setup DCO 301
 - Verification panel
 - Datacap Web Client 301
 - MaxLength action 664, 698
 - MC_Identify actions
 - AutoField 717, 1013
 - FindFields 717, 718, 1013
 - ReadDCOSetup 717, 719, 1013
 - ReadPageSetup 717, 719, 1013
 - SetFormType 717, 720, 1013
 - SetMaxTolerantDistance 717, 721, 1013
 - MC_ReadZones action 730
 - MC_Validation 1013
 - MC_Validation actions
 - AddCenturyTo2YearDigit 721, 722
 - AddToDetailErrorMsg 723
 - AddToDetailMsg 721
 - AddToErrorMsg 721, 723
 - CalculateHCFALineCharges 721, 724
 - CalculateUBLineCharges 721, 724
 - CheckDocID 721, 725
 - ClearErrorMsg 721, 725
 - CommonParseAddress 726
 - CommonParseAddresser 721
 - CommonValAddress 721, 727
 - ConvertHyphen 721, 728
 - FilterPID 721, 728
 - FormatFieldLengths 721, 729
 - InheritSnippets 721, 729
 - MC_ReadZones 721, 730
 - Parse31aPhSig 721, 731
 - Parse58ainsnm 721, 731
 - Parse58binsnm 732
 - Parse58cinsnm 721, 732
- MC_Validation actions *(continued)*
 - ParseConditionCodes 721, 733
 - ParseEPSDT 721, 733
 - ParseLastFirstIniNames 721, 734
 - ParseNDC 721, 735
 - PopulateFromField 721, 735
 - SetConf 721, 736
 - SetOriginalTIF 721, 736
 - StripTrailingAlpha 721, 737
 - TransformLI 721, 737
 - UpdateCredentialList 721, 739
 - ValidateNPI 721, 740
 - ValProcedureCode 721, 740
 - ValRequiredCode 721, 741
- MCCOPositionAdjust action 988, 996
- medical claim forms
 - recognizing 69
- Medical Claims
 - 5010 form configuration 309
 - actions 1011
- Medical Claims Capture
 - 5010 Institutional form configuration 309
 - 5010 Professional form configuration 320
- MergeCCOs_ByType action 334, 339
- MergeLineItem FieldToPageField action
 - description 1017
- MergePageFieldToDocVar action
 - description 1017
- MergeWordLF action 664, 698
- MergeWordRT action 664, 699
- MergeZones action 988, 997
- MESSAGE 291
- MessageBox action 917, 956
- METRIC 302
- migrate
 - Application Wizard 273
 - applications 273
 - converting panels 275, 276
 - creating Datacap Desktop panels 276
 - generating layout XML files 275
 - reviewing layout XML files 275
- migration
 - overview 271
- MIN_TYPES 292
- MinLength action 664, 700
- MoveImageFileToDirectory action 968, 971
- MovePDF action 632, 652
- Multi-pass verification
 - settings 238
- MultiLine
 - Setup DCO 302
 - Verification panel
 - Datacap Desktop 302
- MultiPunch
 - Setup DCO 302
 - Verification panel
 - Datacap Desktop 302
 - Datacap Web Client 302
- mvscan actions
 - scan 741, 742
 - set_abort_time 741, 743
 - set_copy_folder 741, 744
 - set_delete_empty_folders 741, 744
 - set_folder 741, 745

mvscan actions (continued)

- set_image_validation 741, 746
- set_max_docs 741, 746
- set_metadata_types 741, 747
- set_min_age 741, 749
- set_move_wait_time 750
- set_multipage_burst 741, 750
- set_problem_folder 741, 751
- set_sort_method 741, 752
- set_tree_mode 741, 753
- set_types 741, 753
- set_wait_time 741
- set_wait_times 754

N

navigation elements

- accessing the runtime hierarchy 174

NewDictionary action 597, 599

NewDocument action 536, 550

NewLine action 481, 496

NormalizeCCO action 364

O

objects

- associating rules 44

OCR_A actions

- EnableEngineLogsOCR_A 799, 800
- OCRA_ConvertImage2BW 799, 800
- RecognizeBarcodeOCR_A 799, 801
- RecognizeFieldOCR_A 799, 801
- RecognizeFieldVoteOCR_A 799, 802
- RecognizePageFieldsOCR_A 799, 803
- RecognizePageOCR_A 799, 804
- RecognizeToALTOOCR_A 804
- RecognizeToFileOCR_A 799
- RecognizeToPDFOCR_A 799, 806
- ReleaseEngineOCR_A 799, 808
- RotateImageOCR_A 799, 808
- SetAutoRotationOCR_A 799, 809
- SetConfCalculation
 - ParamsOCR_A 799, 810
- SetFastModeOCR_A 799, 810

OCR_N actions

- RecognizePageFieldsOCR_N 811
- RecognizePageOCR_N 811, 812

OCR_S actions

- RecognizeDocToPDF 813
- RecognizeFieldOCR_S 813, 814
- RecognizeFieldVoteOCR_S 813, 815
- RecognizeOM_OCR_S 813
- RecognizePageFields
 - 2CCO_OCR_S 813, 816
- RecognizePageFieldsOCR_S 813, 817
- RecognizePageOCR_S 813, 817
- RecognizePageOCR_S_2TextFile 818
- RecognizePageOCR_S_2TextFile 813
- RecognizePageOCR_S_Legacy 813
- RecognizeToFile_OCR_S 813, 819
- RecognizeToPDF 813, 821
- RotateImage 813, 822
- SetEngineTimeout 813, 823
- SetFastTradeOffOCR_S 813, 824
- SetLegacyDecomposition OCR_S 825
- SetLegacyDecompositionOCR_S 813

OCR_SR actions

- RecognizeFieldOCR_S 826
- RecognizeFieldVoteOCR_S 826, 827
- RecognizePageFieldsOCR_S 826, 827
- RecognizePageOCR_S 826, 828
- RecognizeToFile_OCR_S 826, 829
- RecognizeToPDFOCR_S 826, 831
- RotateImageOCR_S 826, 832
- SetEngineTimeoutOCR_S 826, 833

OCR/A

- check box recognition 67

OCRA_ConvertImage2BW action 799, 800

OMR

- methods 65

OMR fields

- creating a rule to recognize 75

OMR zones

- recognizing check boxes 71

OpenConnection action 632, 653, 712, 715

OpenDatabase action 568, 570

Opening the batch

- Datacap Desktop 100

OpenTextFaxServer actions

- Connect 834, 835
- ConnectOnConnectionError 834
- ConnectOnFaxImportError 834
- ContinueOn ConnectionError 835
- Disconnect 834, 837
- ImportFaxes 834, 838
- SendAsFax 834, 839
- SetAbortTimeout 834, 839
- SetFaxRemovalAfterImport 834, 840
- SetInputFolder 834, 841
- SetMaxNumberOfFaxes 834
- SetMaxNumberOfFaxes 842
- SetNumberOfRetries 834, 843
- SetPollingInterval 834, 844
- SetProcessedFaxesFolder 834, 844
- SetProtocol 834, 845
- SetRetryTimeout 834, 846
- SetServerName 834, 847
- SetUserID 834, 848
- SetUserPassword 834, 848
- SetWindowsAuthentication 834, 849

operation

- Rulerunner 201

optical mark recognition 65

other information

- accessing 175

Outlook actions

- OutlookMessageTo
 - AttachmentOnly 401
- OutlookMessageToImage
 - AndAttachment 402
- OutlookPrintQuality 403
- OutlookTiffCompression 403

OutlookPrintQuality action 403

OutlookTiffCompression action 403

Overlay action 661

override confidence level values 96

overview 133

- IBM Content Manager Connector
 - actions 109

P

PadZone action 988, 997

page data

- reading 59

page data files

- contents 56

- creating 52

page fields

- specifying structure 23

page fingerprints

- creating 155

page identification 33

- fingerprint matching 34

- manual 37, 240

- methods 34

pages

- identification methods 34

pattern matching 187

Protold web client 245

structure-based 36

text matching 37, 178

using AIndex web client 253

page status

- setting 96

page types

- creating 20

- creating fingerprints for 39

- examining 4

- identification 18

- identifying fields of interest 8

- image registration 189

- pattern matching 189

- versions 18

PageID task

- adding the conditional branch 215

PageIDByBCSep action

- description 1018

PageIDBySeqTypes action

- description 1018

PageIDByVariableChange action

- description 1018

pages

- identifying 33

- specifying the structure 20

PageVariable_ExportValue action 481, 496

panel layout

- customizing 244

panels

- converting 275, 276

- creating Datacap Desktop panels 276

- generating layout XML files 275

- reviewing layout XML files 275

parameter settings

- Email Connector actions 129

- Fax Connector actions 134

- FileNet Image Services Connector
 - actions 124

- FileNet P8 Connector actions 114

- IBM Content Manager Connector
 - actions 110

- SharePoint Connector actions 119

parameters

- storing in the .app file 169

parent field

- setting the required variables 67

Parse31aPhSig action 731

- Parse58binsnm action 731
- Parse58binsnm action 732
- Parse58cinsnm action 732
- ParseConditionCodes action 733
- ParseEPSDT action 733
- ParseImageName action 632, 653
- ParseLastFirstIniNames action 734
- ParseMultilineAddress action 917, 956
- ParseName action 917, 957
- ParseNDC action 735
- passwords
 - referencing from your actions 171
 - storing 108
 - storing in the .app file 169
- pat_RecogMatch_Id action 850, 851
 - description 195
 - pattern matching 195
- pat_RegisterZones action 850, 852
 - adjusting the positions of individual fields 193
 - pattern matching 193
- pat_ReleasePageAnchors action 850, 853
- pattern matching
 - adjusting images 187
 - adjusting individual field positions 196
 - adjusting the positions of individual fields 193
 - anchor patterns 189
 - auto registration
 - using the FindFingerprint action 189
 - considerations 189
 - description 188
 - determining runtime field positions 196
 - identifying pages 187
 - image registration 189
 - overview 188
 - setting the confidence level 191
 - setting up anchor objects 190
 - text-based 194
 - using geometric pattern matching 191
 - using multiple anchor objects 192
 - using the pat_RecogMatch_Id action 195
 - using the PatternMatch_Identify action 192
 - when to use 189
- PatternConfidence 296
- PatternMatch 303
- PatternMatch actions
 - MatchPattern 850
 - pat_RecogMatch_Id 850, 851
 - pat_RegisterZones 850, 852
 - pat_ReleasePageAnchors 850, 853
 - PatternMatch_Fingerprint 850, 854
 - PatternMatch_Identify 850, 855
 - PatternMatch_PageType 850, 856
 - SetMatchConfidence 850, 857
- PatternMatch_Fingerprint action 850, 854
- PatternMatch_Identify action 850, 855
 - description 192
 - pattern matching 192
- PatternMatch_PageType action 850, 856

- PD 297
- Pdf actions
 - PDFBitDepth 404
 - PDFCompression 404, 405
 - PDFConversionMethod 404, 406
 - PDFDocumentToImage 404, 407
 - PDFGrayscale 404, 407
 - PDFHorizontalResolution 404, 408
 - PDFQuality 404, 409
 - PDFVerticalResolution 404, 410
- PDFBitDepth action 404
- PDFCompression action 404, 405
- PDFConversion Mode action 410
- PDFConversionMethod action 404, 406
- PDFConversionMode action 411
- PDFDocumentTo Image action 410
- PDFDocumentToImage action 404, 407, 411
- PdffRE actions
 - PDFConversion Mode 410
 - PDFConversionMode 411
 - PDFDocumentTo Image 410
 - PDFDocumentToImage 411
 - PDFImage Compression 410
 - PDFImageCompression 412
 - PDFImageFile Extension 410
 - PDFImageFile Resolution 410
 - PDFImageFileExtension 414
 - PDFImageFileResolution 415
 - PDFImageUse FastBinarization 415
 - PDFImageUseFastBinarization 410
- PDFGrayscale action 404, 407
- PDFHorizontalResolution action 404, 408
- PDFImage Compression action 410
- PDFImageCompression action 412
- PDFImageFile Extension action 410
- PDFImageFileExtension action 414
- PDFImageFileResolution action 410
- PDFImageFileResolution action 415
- PDFImageUse FastBinarization action 410, 415
- PDFQuality action 404, 409
- PDFVerticalResolution action 404, 410
- PIC_ApplyPictureString action 857, 858
- PIC_FilterFields action 857, 859
- PIC_FormatFields 857, 859
- PIC_ReplaceBlankField action 861
- PIC_ReplaceBlankField action 857
- PIC_SetPictureCharacter action 857, 862
- PIC_ValidateField action 857, 863
- Picture actions
 - PIC_ApplyPictureString 857, 858
 - PIC_FilterFields 857, 859
 - PIC_FormatFields 857, 859
 - PIC_ReplaceBlankField 857, 861
 - PIC_SetPictureCharacter 857, 862
 - PIC_ValidateField 857, 863
- PictureString
 - Setup DCO 303
 - Verification panel
 - Datacap Web Client 303
- PilotMessage_Clear action 881, 886
- PilotMessage_Set action 881, 886
- pixel threshold evaluation method
 - check box recognition 68

- pixel threshold recognition
 - checking values 77
 - density string values 77
- POLR actions
 - CallPOLR 864
- PopulateFromField action 735
- PopulateTaxType action
 - description 1016
- PopulateWithResult action 712, 715
- PopulateZNField action 988, 998
- PopulateZNLInItemField action 988, 998
- PopulateZNLInItemFieldDynamic action 632
- Portuguese
 - language codes 63
- Pos<templateID> 304
- Position 304
- prerequisites
 - Email Connector actions 128
 - Fax Connector actions 134
 - FileNet Image Services Connector actions 123
 - FileNet P8 Connector actions 113
 - IBM Content Manager Connector actions 109
 - SharePoint Connector actions 118
- ProcessChangeBatchStatus action 779, 780
- ProcessChangeBatchStatusOrder action 779
- ProcessChangeBatchStatusTaskOrder action 779
- ProcessChildren action 881, 886
- ProcessClearAuditTable action 779, 782
- ProcessClearDebugTable action 779, 783
- ProcessDeleteBatches action 779, 783
- ProcessDeleteBatchesEx action 784
- ProcessDeleteBatchesEX action 779
- ProcessInjectBatches action 779, 784
- ProcessMoveBatches action 779, 785
- ProcessMoveBatchesEx action 786
- ProcessMoveBatchesEX action 779
- ProcessMoveDBRecords action 779, 787
- ProcessResetPendingOrNotify action 779
- ProcessRunSqlQuery action 779, 790
- PropagateToAltText action 433, 443
- Protold web client 245
 - configuring 246

Q

- Query setup action 764
- Query setup actions
 - QueryClear 764
 - QuerySetAge 764, 765
 - QuerySetBatchRange 764, 766
 - QuerySetBranch 764, 767
 - QuerySetDateFormat 764, 767
 - QuerySetDateRange 764, 770
 - QuerySetDateTimeFormat 764, 771
 - QuerySetGeneric 764, 773
 - QuerySetJobID 764, 774
 - QuerySetOperator 764, 775
 - QuerySetPriority 764, 775
 - QuerySetSeparator 764, 776
 - QuerySetStation 764, 777

- Query setup actions *(continued)*
 - QuerySetStatus 764, 777
 - QuerySetTaskID 764, 778
- QueryClear action 764
- QuerySetAge action 764, 765
- QuerySetBatchRange action 764, 766
- QuerySetBranch action 764, 767
- QuerySetDateFormat action 764, 767
- QuerySetDateRange action 764, 770
- QuerySetDateTimeFormat action 764, 771
- QuerySetGeneric action 764, 773
- QuerySetJobID action 764, 774
- QuerySetOperator action 764, 775
- QuerySetPriority action 764, 775
- QuerySetSeparator action 764, 776
- QuerySetStation action 764, 777
- QuerySetStatus action 764, 777
- QuerySetTaskID action 764, 778

R

- ReadBarCode action 356, 358
- ReadBarcodeBP action 348
- ReadBarcodeBP action 355
- ReadCurrentObjVariable action 917, 958
- ReadDCOSetup action 719
- ReadFieldValue action 917, 958
- ReadFPXMLZones action 632, 655
- ReadOnly
 - Setup DCO 305
 - Verification panel
 - Datacap Desktop 305
 - Datacap Web Client 305
- ReadPageSetup action 719
- ReadPageVariableValue action 917, 959
- ReadZones action 988, 999
- ReadZonesFPX action 571, 572
- Recog_Shared actions
 - AnalyzeImage 865
 - CCONormalization_OFF 865, 866
 - CreateTextFile 865, 867
 - IsBlankPage 865, 868
 - RecogContinueOnFailure 865, 869
 - RecogOMRThresh 865, 870
 - RecogOMRThreshold 865, 870
 - RegisterPageFields 865, 872
 - Release_Image 865
 - ReleaseImage 873
 - RotateTio 865, 873
 - SetAdjustFieldToChars 865, 874
 - SetFingerprintRecogPriority 865, 875
 - SetFullPageRecogArea 865, 875
 - SetOutOfProcessRecogTimeout 876
 - SetOutOfProcessRecogTimeout 865
 - SetRecogFailureRetryDelay 865, 877
 - SnapCCOtoDCO 865, 878
 - SnapDCOtoCCO 865, 879
 - SnapFieldtoChars 865, 879
 - UseOutOfProcessRecog 865, 880
- RecogContinueOnFailure action 865, 869
- recognition languages
 - settings 62
- recognition rule 157
 - creating for the grid total 157
- recognition zones
 - creating for fingerprints 92

- recognition zones *(continued)*
 - defining 155
 - specifying for TravelDocs 70
 - storing information 59
 - using fingerprints 58
- Recognize OMR Fields rule
 - adding to document hierarchy 75
 - updating to use
 - RecogOMRThreshold 76
- Recognize Page rule
 - updating 73, 268
- Recognize Page ruleset
 - updating for manual page
 - identification 213
- RecognizeBarcodeOCR_A action 799, 801
- RecognizeDocToPDF action 813
- RecognizeFieldICR_C action 591, 592
- RecognizeFieldICR_CEx action 591
- RecognizeFieldOCR_A action 799, 801
- RecognizeFieldOCR_S action 813, 814, 826
- RecognizeFieldOCR_S_Legacy
 - action 813
- RecognizeFieldsICR_P action 600
- RecognizeFieldVoteICR_C action 591, 592
- RecognizeFieldVoteOCR_A action 799, 802
- RecognizeFieldVoteOCR_S action 813, 815, 826, 827
- RecognizeOM_OCR action 813
- RecognizePageFields 2CCO_OCR_S
 - action 816
- RecognizePageFields2CCO_ICR_C
 - action 591
- RecognizePageFields2CCO_OCR_S
 - action 813
- RecognizePageFieldsICR_C action 591, 594
- RecognizePageFieldsICR_P action 597
- RecognizePageFieldsOCR_A action 799, 803
- RecognizePageFieldsOCR_N action 811
- RecognizePageFieldsOCR_S action 813, 817, 826, 827
- RecognizePageICR_C action 591, 595
- RecognizePageOCR_A action 799, 804
- RecognizePageOCR_N action 811, 812
- RecognizePageOCR_S action 813, 817, 826, 828
- RecognizePageOCR_S_2TextFile
 - action 813, 818
- RecognizePageToPDFICR_C action 591, 596
- RecognizeToALTOOCR_A action 804
- RecognizeToFile_OCR_S action 813, 819, 826, 829
- RecognizeToFileOCR_A action 799
- RecognizeToPDF action 813, 821
- RecognizeToPDFOCR_A action 799, 806
- RecognizeToPDFOCR_S action 826, 831
- RecogOMRThresh action 865, 870
- RecogOMRThreshold action 865, 870
- RecogStatus 305
- RecogType 305
- Redact action 622, 624

- RedactByRegEx action 622, 625
 - description 1018
- RedactField action
 - description 1018
- RedactParameter action 622
- RedactParameters action 625
- reference
 - application-specific variables 309
 - Medical Claims Capture
 - 5010 Institutional form 309
 - 5010 Professional form 320
 - smart parameter special
 - variables 277
 - standard variables 291
- RegExFind action 664, 700
- RegExFind_InZone action 664, 701
- RegExFindNext action 664, 702
- RegExFindNext_InZone action 702
- region codes
 - setting 294
- Registering a page
 - using manual anchoring 241
- RegisterPage action 988, 1000
- RegisterPageFields action 865, 872
- regular expressions
 - using with text matching 180
- ReleaseEngineOCR_A action 799, 808
- ReleaseImage action 865, 873
- remote scanning 227
 - Datacap Web Client 226
- remote scanning client
 - configuring 227, 250
 - configuring Rulerunner 252
 - creating a scan task 249
 - modifying the Verify shortcut 253
 - scan tasks 249
 - scanning batches 251
 - Upload task 250
 - uploading batches 251
 - verifying the batch 253
 - Web Job CreateDocs task 251
- Remote virtual scanning 229
- RemoveDocumentStructure action 443
 - description 1017
- RenameFile action 521, 532
- ReplaceChars action 917, 960
- ReplaceValueAtPosition action 917, 960
- Reporting actions
 - ReportQueryTMUsage 796
 - ReportSetReportingTable 796, 797
 - ReportSetUsageDBTable 796, 798
- ReportQueryTMUsage action 796
- ReportSetReportingTable action 796, 797
- ReportSetUsageDBTable action 796, 798
- ReqConf 305
- ResetField action 917, 961
- ResetFieldVariables action 481, 497
- restructuring the batch
 - using AIndex 235
 - using VeriFine 230
- review
 - layout XML files 275
- RightTruncate action 917, 961
- RotateImage action 813, 822
- RotateImageOCR_A action 799, 808
- RotateImageOCR_S action 826, 832
- RotateTio action 865, 873

- route documents
 - branching 202
 - splitting 202
 - using branching and splitting 201
- Routing ruleset
 - configuring 216
 - updating to split the branch 222
- rr_AbortBatch action 881, 887
- rr_Get action 881, 887
- rr_WriteNode action 881, 888
- rrAppend action 881, 889
- rrCompare action 881, 889
- rrCompareCase action 890
- rrCompareCaseLength action 891
- rrCompareNot action 881, 892
- rrCompareNotCase action 893
- rrCompareNotCaseLength action 894
- rrCopy action 881, 895
- rrPrepend action 881, 896
- RRS
 - log files 144
- RRS log file
 - reviewing 221
- rrSet action 881, 897
- runner actions
 - AbortOnError 881, 882
 - CheckAllIntegrity 881, 882
 - CheckDocCount 881, 883
 - CheckPageCount 881, 883
 - DebugMode_OFF 881, 884
 - DebugMode_ON 881, 884
 - GoToNextFunction 881, 885
 - PilotMessage_Clear 881, 886
 - PilotMessage_Set 881, 886
 - ProcessChildren 881, 886
 - rr_AbortBatch 881, 887
 - rr_Ger 887
 - rr_Get 881
 - rr_WriteNode 881, 888
 - rrAppend 881, 889
 - rrCompare 881, 889
 - rrCompareCase 890
 - rrCompareCaseLength 891
 - rrCompareNot 881, 892
 - rrCompareNotCase 893
 - rrCompareNotCaseLength 894
 - rrCopy 881, 895
 - rrPrepend 881, 896
 - rrSet 881, 897
 - SetBatchPriority 881, 898
 - SetOperatorID 881, 898
 - SetReturnValue 881, 899
 - SetStationID 881, 899
 - SetTaskStatus 881, 900
 - SkipChildren 881, 900
 - Status_Preserve_OFF 881, 901
 - Status_Preserve_ON 881, 902
 - Task_NumberOfSplits 881, 902
 - Task_RaiseCondition 881, 903
- Rtf actions
 - RtfPrintQuality 416
 - RtfTiffCompression 416, 417
 - RtfToImage 416, 418
- RtfPrintQuality action 416
- RtfTiffCompression action 416, 417
- RtfToImage action 416, 418

- rule execution
 - batch-level example 45
 - page identification order 48
 - page-level example 45
 - summary 48
 - validation order 48
- Rulemanager tab
 - panels 13
- Rulerunner
 - configuring 200
 - description 200
 - log files 144
 - logging 201
 - operating 201
 - overview 200
- Rulerunner log file
 - analyzing 209
 - disabling 209
- Rulerunner logging
 - enabling 207
- rules 27, 292
 - add existing to new pages 155
 - assigning default to new fields 72
 - assigning default to new pages 72
 - association with document
 - hierarchy 44
 - attaching to the document
 - hierarchy 157
 - business validation 10
 - creating to remove the non-line
 - items 158
 - order of execution 46
- rulesets 26, 27
 - modifying 30
- RunFlexIDPanel action
 - description 1017
- running a batch through the
 - workflow 158, 258, 268
- running the batch through the workflow
 - preparing 268
- Running the scan task
 - Datacap Desktop 33
 - Datacap Web Client 33
- runtime batch folder
 - batch identifier 30
 - contents 43
 - location 30
- runtime batch hierarchy
 - defined 18
 - relation to document hierarchy 18
- runtime data file
 - updating with the recognized
 - text 182
- runtime field positions
 - determining for pattern
 - matching 196
- runtime hierarchy
 - accessing 171, 172, 173, 174
- Runtime page data file 236
- runtime pages
 - checking confidence levels 43
- Russian
 - language codes 63

S

- sample applications
 - opening 13
- sample fingerprints
 - enhancing 40
- sample images 29
- SaveAsCurrentObjVariable action 917, 961
- SaveAsPageVariable action 917, 962
- SaveDocToFolder action 536, 551
- SaveFilePathAsVariable action 481, 497
- SaveObjectVariable action 632, 655
- SaveToFile action 597, 601
- scan
 - remotely 227
- Scan
 - local 31
 - remotely
 - start panel 228, 229
 - virtual 229
- scan action 741, 742
- Scan action 968, 972
- scan task
 - creating a shortcut 32
- ScanDetails action 988, 1000
- ScanDetailsByLines action 988, 1001
- ScanDetailsByVSpace action 988, 1001
- ScanLineItem action 988, 1002
- ScanLineItemDynamic action 632, 655
- scanning 27
 - electronic documents 27
 - local scanner 29
 - virtual 27
- Scanning remotely 249
- scanningDatacap Web Client
 - remote 29
- ScanRT action 664, 703
- ScanSrcPath 297
- SearchInSubdirectory action 968, 973
- SELECT
 - Setup DCO 306
 - Verification panel
 - Datacap Desktop 306
 - Datacap Web Client 306
- SelectSnippet action 664, 704
- SendAsFax action 834, 839
- SendEmail action 463
- SendOutlookNotification action 632, 656
- set generic breakpoints 147
- set_abort_time action 741, 743
- set_copy_folder action 741, 744
- set_delete_empty_folders action 741, 744
- set_folder action 741, 745
- set_image_validation action 741, 746
- set_max_docs action 741, 746
- set_metadata_types action 741, 747
- set_min_age action 741, 749
- set_move_wait_time action 750
- set_multipage_burst action 741, 750
- set_problem_folder action 741, 751
- set_sort_method action 741, 752
- set_tree_mode action 741, 753
- set_types action 741, 753
- set_wait_time action 741, 754
- SetAbortTimeout action 834, 839
- SetAdjustedWidth action 622, 626
- SetAdjustFieldToChars action 865, 874

SetAdminDB action 755, 756
 SetAlternateImageNames action 968, 973
 SetApplication action 755, 757
 SetApplicationID action 334, 340
 SetAttachment action 463, 464
 SetAutoRotationOCR_A action 799, 809
 SetBackgroundImage action 661, 662
 SetBatchPriority action 881, 898
 SetBlindCarbonCopyRcpts action 463, 464
 SetCarbonCopyRcpts action 463, 465
 SetChrominanceFactor action 602, 607
 SetConf action 736
 SetConfCalculationParamsOCR_A action 810
 SetConfCalculationParamsOCR_A action 799
 SetCSV action 481, 498
 SetDCOStatus action 433, 444
 SetDCOType action 444
 SetDeleteOriginal action 602, 608
 SetDetailsAndLineitemPairFPX action 571
 SetDirectoryFPX action 571, 574
 SetDitheringBackground action 661, 663
 SetDocStatus action 433, 445
 SetDocumentType action 433, 445
 SetDynamicDetailZones action 632, 657
 SetElementSeparator action 481, 499
 SetEmailBody action 463, 465
 SetEngineDB action 755, 757
 SetEngineTimeout action 813, 823
 SetEngineTimeoutOCR_S action 826, 833
 SetEOL action 988, 1002
 SetEOL_CRLF action 988, 1003
 SetExportPath action 481, 499
 SetExtensionName action 481, 500
 SetFastMode action 968, 974
 SetFastModeOCR_A action 799, 810
 SetFastTradeOffOCR_S action 813, 824
 SetFaxRemovalAfterImport action 834, 840
 SetFileName action 481, 501
 SetFileReadOnly action 521, 533
 SetFill action 481, 501
 SetFilter_HostName action 334, 341
 SetFilter_PageType action 334, 341
 SetFingerprint action 334, 342
 SetFingerprintDir action 334
 SetFingerprintFailureThreshold action 334, 343
 SetFingerprintFolder action 568, 571
 SetFingerprintRecogPriority action 865, 875
 SetFingerprintSearchArea action 334, 344
 SetFingerprintWebServiceURL action 334, 345
 SetFixedLength action 481, 502
 SetFldConfidence action 433, 446
 SetFontName action 622, 627
 SetFontSize action 622, 628
 SetFormType action 720
 SetFullPageRecogArea action 865, 875
 SetGrayScale action 602, 608
 SetHaloBackground action 661, 663
 SetIgnoreFieldStatus action 481, 502
 SetImageType action 968, 974
 SetInputFolder action 834, 841
 SetIsOverrideable action 917, 963
 SetJustified action 481, 503
 SetKnowledgeBaseCC action 359, 360
 SetLabels action
 description 1018
 SetLanguageCC action 359, 360
 SetLegacyDecompositionOCR_S action 825
 SetLegacyDecompositionOCR_S action 813
 SetListenerURLCC action 359, 361
 SetLuminanceFactor action 602, 609
 SetMailServer action 463, 466
 SetMailSourceFolder action 968, 975
 SetMatchConfidence action 850, 857
 SetMaxCharacterHeightAVG action 364, 365
 SetMaxCharacterHeightTMM action 364, 366
 SetMaxImageFiles action 968, 976
 SetMaxNumberOfFaxes action 834
 SetMaxNumberOfFaxes action 842
 SetMaxOffset action 334, 346
 SetMaxTolerantDistance action 721
 SetMinimumConfidenceBP action 356
 SetMultiPageTiff action 968, 977
 SetNamePattern 383
 SetNamePattern action 386
 SetNumberOfRetries action 834, 843
 SetOMR_Separator action 481, 504
 SetOpaque action 622, 628
 SetOperatorID action 881, 898
 SetOriginalTIF action 736
 SetOutOfProcessRecogTimeout action 876
 SetOutOfProcessRecogTimeout action 865
 SetPageFingerprintID action 433, 447
 SetPageStatus action 433, 448
 SetPageTemplateID action 433, 449
 SetPageType action 433, 449
 SetPassword action 755, 758
 SetPicChar action 632, 657
 SetPollingInterval action 834, 844
 SetPostalDBPathICR_P action 597, 602
 SetProblemValue action 334, 346
 SetProblemValueCC action 359, 362
 SetProcessedFaxesFolder action 834, 844
 SetProfileString action 521, 534
 SetProtocol action 834, 845
 SetRecipients action 463, 467
 SetRecogFailureRetryDelay action 865, 877
 SetRect action 664, 705
 SetRetryTimeout action 834, 846
 SetReturnValue action 881, 899
 SetSearchArea action 334, 347
 SetSender action 467
 SetServer action 755, 759
 SetServerName action 834, 847
 SetSortOrder action 968, 977
 SetSourceDirectory action 968, 978
 SetSpaceFill action 481, 504
 SetStation action 755, 760
 SetStationID action 881, 899
 SetStickyNo action 632, 657
 SetSubject action 463, 468
 SetTableName action 507, 516
 SetTaskStatus action 881, 900
 SetTemplateDir action 334, 347
 SetTIFFCompression action 602, 610
 setting breakpoints 146
 setting variables for Options and Insurance fields 74
 settings
 recognition languages 62
 SetToDocIDMPTIFF action 632, 658
 Setup DCO 17
 SetupDisconnectAll action 755, 760
 SetupOpenApplication action 755, 761
 SetupOpenApplicationEx action 755, 762
 SetUser action 755, 763
 SetUserID action 834, 848
 SetUserPassword action 834, 848
 SetWindowsAuthentication action 834, 849
 SetZeroFill action 481, 505
 Shared actions
 ExceptionSetFileTypes 383
 ExceptionSetHandler 384
 ExceptionSetTaskCondition 385
 ExceptionSetVariableName 384
 SetNamePattern 386
 SharePoint
 Authentication 107
 SharePoint Connector
 file upload examples 121
 SharePoint Connector actions
 configuring 121
 overview 117
 parameter settings 119
 prerequisites 118
 ShowChar
 Setup DCO 306
 Verification panel
 Datacap Web Client 306
 SkipChildren action 881, 900
 smart parameter special variables 277
 Smart parameter special variables
 accessing job and task information 283
 accessing runtime hierarchy 279
 accessing the application configuration file 277
 miscellaneous 285
 smart parameters
 accessing the runtime hierarchy 171
 application settings
 accessing with special variables 168
 determining the correct key name 169
 elements 167
 overview 166
 structure 167
 using 166
 SmartSQL action 712, 716
 SnapCCOtoDCO action 865, 878
 SnapDCOtoCCO action 865, 879
 SnapFieldtoChars action 865, 879
 SP_CreateFolder action 903, 904
 SP_Login action 903, 904

- SP_SetContentType action 903, 905
- SP_SetFileType action 903, 906
- SP_SetProperty action 903, 906
- SP_SetUploadMode action 903, 907
- SP_SetUrl action 903, 908
- SP_Upload action 903, 908
- SP_Upload Dir 903
- SP_UploadDir action 909
- Spanish
 - language codes 63
- special conditions
 - creating jobs for 204
- special variables
 - accessing job information 174
 - accessing other information 175
 - accessing task information 174
 - accessing the runtime hierarchy 172, 173
- SPExport actions
 - SP_CreateFolder 903, 904
 - SP_Login 903, 904
 - SP_SetContentType 903, 905
 - SP_SetFileType 903, 906
 - SP_SetProperty 903, 906
 - SP_SetUploadMode 903, 907
 - SP_SetUrl 903, 908
 - SP_Upload 903, 908
 - SP_Upload Dir 903
 - SP_UploadDir 909
- Split actions
 - SplitBatch 909
- SplitBatch action 909
- SplitFieldValueLeft action 917, 963
- SplitFieldValuePreserveEnd action 917, 963, 964
- SplitFieldValuePreserveStart action 917
- SplitFieldValueRight action 917, 964
- SplitFileName action 521, 535
- SplitMultipageTiff action 418, 419
- SplitTIFFCompression action 418, 419
- splitting
 - defining conditions 203
 - description 202
 - raising condition flags 202
- standard variables 291
 - all object types 291
 - batch variables 294
 - document variables 295
 - field variables 298
 - page variables 295
- Start panel for remote scan 228, 229
 - running validation rules 229
- STATUS 292
- status codes
 - interpreting 93
- status values
 - field 90
 - page 90
- Status_Preserve_OFF action 881, 901
- Status_Preserve_ON action 881, 902
- Sticky
 - Setup DCO 307
 - Verification panel
 - Datacap Web Client 307
- strings
 - locating with text matching 179
- StripTrailingAlpha action 737

- structured documents 50
- SumFields action 917, 965
- supported language codes
 - Chinese 63
 - Dutch 63
 - Eastern European 63
 - English 63
 - French 63
 - German 63
 - Italian 63
 - Portuguese 63
 - Russian 63
 - Spanish 63
 - Swedish 63
- SwapImages action 632, 658
- Swedish
 - language codes 63
- SwitchMMDD action 632, 659

T

- task information
 - accessing 174
- task profiles 26
- Task_NumberOfSplits action 881, 902
- Task_RaiseCondition action 881, 903
- tasks
 - creating 205
 - description 24
 - log files 145
- TEMPLATE IMAGE 297
- TemplateID 297
- Test tab 15
- Text
 - Runtime DCO 307
 - Setup DCO 307
 - Verification panel
 - Datacap Web Client 307
- Text action 481, 505
- text matching
 - identifying pages 178
 - limitations 183
 - locate fields 150
 - locating data 178, 179
 - locating field data 181
 - locating simple strings 179
 - updating the runtime data file 182
 - updating the TravelDocs
 - application 183
 - attaching rules to the document hierarchy 186
 - identifying unrecognized pages 183
 - recognizing data 184
 - running a batch through the workflow 187
 - using keyword lists 180
 - using regular expressions 180
- Text matching
 - page identification example 37
- text matching to locate fields 150
- text-based pattern matching 194
- Tiff actions
 - SplitMultipageTiff 418, 419
 - SplitTIFFCompression 418, 419
- TifMerge actions
 - TifMerge_CheckStatus 911
- TifMerge actions (*continued*)
 - TifMerge_ExportToBatchDir 911, 912
 - TifMerge_MergelImages 911, 913
 - TifMerge_MyImage 911, 914
 - TifMerge_Preserve Compression 915
 - TifMerge_PreserveCompression 911
 - TifMerge_SetFileName 911, 915
 - TifMerge_SetFilePath 911, 916
- TifMerge_CheckStatus action 911
- TifMerge_ExportToBatchDir action 911, 912
- TifMerge_MergelImages action 911, 913
- TifMerge_MyImage action 911, 914
- TifMerge_PreserveCompression
 - action 911, 915
- TifMerge_SetFileName action 911, 915
- TifMerge_SetFilePath action 911, 916
- TimeStampField action 917, 965
- TM524 actions 917
- TransformLI action 737
- travel documents
 - optional pages 7
 - required pages 7
- TravelDocs 74
 - adding new pages with line item grids 152
 - automated background processing
 - analyzing the Rulerunner log 209
 - defining background tasks 207
 - disabling the Rulerunner log 209
 - enabling Rulerunner logging 207
 - running a batch through the workflow 208
 - setting up background tasks 207
 - setting up Job Monitor 208
 - creating text zones 70
 - creating the document hierarchy 19
 - creating the initial fingerprint library 38
 - developing business requirements 3
 - enabling manual page identification 212
 - adding a function 213
 - adding the conditional branch to the PageID task 215
 - configuring branching 216
 - configuring the Routing ruleset 216
 - creating the ManualPageID job and task 215
 - recognizing the data on an unidentified page 218
 - running a batch through the workflow 217
 - updating the Recognize Page ruleset 213
 - exporting data to a database 137
 - exporting data to an XML file 140
 - exporting line item grid data to an XML file 175
 - adding rules to the ExportXML ruleset 175
 - attaching the Export Other XML rules to the document hierarchy 177
 - running a batch through the workflow 177

TravelDocs (continued)

- field status codes 93
- generating fingerprints
 - automatically 218
 - adding the ruleset to the Verify task profile 220
 - assigning the rule to each page type 220
 - creating the AutoFingerprint ruleset 219
 - reviewing the RRS log file 221
 - running a batch through the workflow 220
- interpreting density string values 77
- page status codes 93
- recognizing line item grid data 156
- routing to handle document failures 210
 - configuring Rulerunner to run CreateDocs 211
 - creating the CreateDocs task 210
 - moving document creation and integrity checking 210
 - running a batch through the workflow 211
- running a batch through the workflow 42
- running automated background processing 206
- sample images 29
- splitting a document from the main branch 222
 - assigning the Batch Splitting rule 223
 - routing the split document to a supervisor 223, 224
 - running a batch through the workflow 225
 - updating the Routing ruleset 222
- stepping through a batch 49
- testing export task 139, 142
- updating to use text matching 183
 - attaching rules to the document hierarchy 186
 - identifying unrecognized pages 183
 - recognizing data 184
 - running a batch through the workflow 187
- using geometric pattern matching 196
 - reviewing runtime batch files 198
 - running a batch through the workflow 198
 - setting up anchor zones 197
 - updating the PageID rule 197
- validating flight cost 86
- validating line item grid data 159
- TrimSpaces action 966
 - description 917
- TruncateFromEnd action 917, 966
- TruncateFromStart action 917
 - description 967
- two pass verification 238
- Two pass verification 236
- Txt actions
 - TxtFontName 421

Txt actions (continued)

- TxtFontSize 421
- TxtPrintQuality 420, 422
- TxtTiffCompression 420, 422
- TxtToImage 420, 423
- TxtFontName action 421
- TxtFontSize action 421
- TxtPrintQuality action 420, 422
- TxtTiffCompression action 420, 422
- TxtToImage action 420, 423
- TYPE 294

U

- UpdateCredentialList action 739
- UpdateDCOField action 664, 705
- UpdateField action 664, 706
- UpdateFPStats action 334, 632, 659
- UpdateKnowledgeBaseCC action 359, 363
- updating auto fingerprinting to use FPXML 267
- updating ManualPageID 255
- updating Recognize Page rule 73
- updating the application 254
- upgrades
 - overview 271
- Upload action 536, 551
- Upload_SetNumAttempts action 536, 552
- UploadSetDelay action 536
- UseIndexes_OFF action 536, 552
- UseIndexes_ON action 553
- UseIndexes_ON action 536
- UseOutOfProcessRecog action 865, 880

V

- Validate Car Type rule
 - adding to document hierarchy 88
 - creating 87
- Validate Currency Field rule
 - adding to document hierarchy 85
 - creating 85
- Validate Flight Cost rule
 - creating 86
- ValidateNPI action 740
- ValidateVendor action 632, 660
- validating car type
 - lookup database 87
- validation
 - currency fields 84
 - grid totals 160, 161
 - managing errors 84
 - overriding failures 97
 - using external data sources 83
- validation failures
 - displaying to operator 82
- validation rules
 - start panel fields 229
- Validation Statuses 256
- Validations actions
 - AddLeadingZeros 917, 919
 - AddPaddingToEnd 917, 919
 - AddPaddingToLeft 917, 920
 - AddPaddingToRight 917, 920

Validations actions (continued)

- AddPaddingToStart 917, 921
- AddTrailingZeros 917, 921
- AllowOnlyChars 917, 922
- AppendFromField 917, 922
- AppendToField 917, 923
- AssignFieldDefault 917, 923
- Calculate 917, 924
- CalculateDateDifference 917, 924
- CalculateFields 917, 925
- CheckSubFields 917, 926
- CompareFields 917, 927
- ConvertFieldTo Currency 928
- ConvertFieldToCurrency 917
- ConvertToLowerCase 917, 928
- ConvertToUpperCase 917, 929
- CopyField 917, 929
- CopyFieldToField 917, 930
- DateStampField 917, 931
- DeleteAllAlpha 917, 931
- DeleteAllMiscChars 917, 931
- DeleteAllNumeric 917, 932
- DeleteAllPunct 917, 932
- DeleteAllSysChars 917, 933
- DeleteChildType 917, 933
- DeleteLCSpaces 917, 934
- DeleteParentObj 917, 934
- DeleteSelectedChars 917, 935
- EmptyFieldValue 917, 935
- FailRuleSet 917, 936
- FieldContainsValue 917, 936
- FilterFieldSelectedChars 917, 937
- FormatNumberToLocale 917, 937
- GetJobID 917, 938
- HasChildOfType 917, 939
- InsertChars 917, 939
- InsertDecimalPoint 917, 940
- IsFieldCurrency 917, 940
- IsFieldDate 917, 941
- IsFieldDateEqualOrAfter 917, 942
- IsFieldDateEqualOrBefore 917, 942
- IsFieldDateUpToToday 917, 943
- IsFieldDateWithinRange 917, 943
- IsFieldDateWithinXDays 917, 944
- IsFieldDateWithReformat 917, 944
- IsFieldEmpty 917, 945
- IsFieldFilled 917, 945
- IsFieldGreaterOrEqual 917, 946
- IsFieldHidden 917, 946
- IsFieldLengthMax 917, 947
- IsFieldLengthMin 917, 947
- IsFieldLessOrEqual 917, 948
- IsFieldMatching 917, 948
- IsFieldPercent NonNumeric 949
- IsFieldPercentAlpha 917, 949
- IsFieldPercentNonNumeric 917
- IsFieldPercentNumeric 917, 950
- IsMatchingJobID 917, 951
- IsMaxOMRChecked 917, 951
- IsMinOMRChecked 917, 951
- IsPatternInField 917, 952
- IsSupportedImageFile 917, 953
- IsThisFieldEmpty 917, 953
- IsThisFieldFilled 917, 954
- IsVariableEmpty 917, 954
- IsVariableFilled 917, 955
- LeftTruncate 917, 955

- Validations actions (*continued*)
 - MessageBox 917, 956
 - ParseMultilineAddress 917, 956
 - ParseName 917, 957
 - ReadCurrentObjVariable 917, 958
 - ReadFieldValue 917, 958
 - ReadPageVariableValue 917, 959
 - ReplaceChars 917, 960
 - ReplaceValueAtPosition 917, 960
 - ResetField 917, 961
 - RightTruncate 917, 961
 - SaveAsCurrentObjVariable 917, 961
 - SaveAsPageVariable 917, 962
 - SetIsOverrideable 917, 963
 - SplitFieldValueLeft 917, 963
 - SplitFieldValuePreserveEnd 917, 963, 964
 - SplitFieldValuePreserveStart 917
 - SplitFieldValueRight 917, 964
 - SumFields 917, 965
 - TimeStampField 917, 965
 - TrimSpaces 917, 966
 - TruncateFromEnd 917, 966
 - TruncateFromStart 917, 967
- ValProcedureCode action 740
- ValRequiredCode action 741
- ValueInField action 664, 707
- ValueInField_Fuzzy action 664, 707
- ValueInField_RegEx action 664, 708
- Variable_ExportValue action 481, 506
- Variable_IsValue action 481, 506
- variables
 - accessing application settings 168
 - hr_locale 294
 - smart parameters 167
- verification
 - line item grid pages 162
 - skipping tasks 97
- Verification panel
 - Setup DCO
 - DataType 298
 - DICT 299
 - Label 299
 - Lookup 300
 - LookupEx 301
 - MaxLength 301
 - MultiLine 302
 - MultiPunch 302
 - PictureString 303
 - ReadOnly 305
 - SELECT 306
 - ShowChar 306
 - Sticky 307
 - Text 307
- VeriFine web client 230
 - configuring 230, 232
 - custom pages 233
 - restructuring the batch 230
- Verify shortcut
 - modifying 253
- verifying
 - AVerify web client 242
 - double blind 236
 - multi-pass 236
 - two pass 236
 - using AIndex 234
 - using the ImgEnter web client 245

- verifying (*continued*)
 - using VeriFine 230
- Verifying batches
 - Datacap Web Client 101
- Verifying pages
 - Datacap Desktop 162
- versions
 - page types 18
- Vote actions
 - VoteFld 968
- VoteFld action 968
- Vscan
 - sample images 29
- VScan
 - modifying ruleset 30
- Vscan actions
 - AddDocument 968, 969
 - CopyFile 968, 969
 - DeleteImageFile 968, 970
 - MoveImageFileToDirectory 968, 971
 - Scan 968, 972
 - SearchInSubdirectory 968, 973
 - SetAlternateImageNames 968, 973
 - SetFastMode 968, 974
 - SetImageType 968, 974
 - SetMailSourceFolder 968, 975
 - SetMaxImageFiles 968, 976
 - SetMultiPageTiff 968, 977
 - SetSortOrder 968, 977
 - SetSourceDirectory 968, 978
- VScanDatacap Studio
 - generating a batch 30

W

- Web Job CreateDocs task
 - creating 251
- Web Services actions
 - WsCompare 979
 - WsDownloadFile 980
 - WsDownloadFolder 979
 - WsExecute 979, 980
 - WsGetLineItems 979, 981
 - WsGetValue 979, 982
 - WsMessageLineItem
 - PropertyAdd 979, 982
 - WsMessageLineItem PropertySet 983
 - WsMessageLineItemPropertySet 979
 - WsSetHeaderValue 979, 983
 - WsSetMessageLineItem Property 984
 - WsSetMessageLineItemProperty 979
 - WsSetMessageProperty 979, 985
 - WsSetMessageTemplate 979, 985
 - WsSetResponseNameSpace 979, 986
 - WsUploadFile 979, 986
 - WsUrlReplaceValue 979, 987
 - WsUrlSet 979, 987
- Word actions
 - WordDocumentToImage 424
 - WordPrintQuality 424, 425
 - WordTiffCompression 424, 426
- WordDocumentToImage action 424
- WordFind action 664, 708
- WordFind_InZone action 664, 709
- WordFind_Offset action 664, 711
- WordFindNext action 664, 710
- WordFindNext_InZone action 664, 711

- WordPrintQuality action 424, 425
- WordTiffCompression action 424, 426
- workflows 24
 - automatic processing 199
 - creating jobs 205
 - creating tasks 205
 - Datacap Web Client Operations
 - tab 258
 - description 24
 - processing on Rulerunner 199
 - routing automatically 199
 - shortcuts 258
- WriteErrorMessage action 632, 660
- WriteZoneFPX action 571, 575
- WriteZonesFPX action 571, 575
- WsCompare action 979
- WsDownloadFile action 980
- WsDownloadFolder action 979
- WsExecute action 979, 980
- WsGetLineItems action 979, 981
- WsGetValue action 982
- WsGetValueaction 979
- WsMessageLineItemPropertyAdd
 - action 979
- WsMessageLineItemPropertySet
 - action 979
- WsSetHeaderValue action 979, 983
- WsSetMessageLineItemProperty
 - action 979
- WsSetMessageProperty action 979, 985
- WsSetMessageTemplate action 979, 985
- WsSetResponseNameSpace action 979, 986
- WsUploadFile action 979, 986
- WsUrlReplaceValue action 979, 987
- WsUrlSet action 979, 987

X

- xml_CommitNode action 517
- xml_NewNode action 517, 518
- xml_SaveFile action 517, 518
- xml_SetAttributeValue action 517, 519
- xml_SetExportPath action 517, 520
- xml_SetFileName action 517, 520
- xml_SetNodeValue action 517, 521

Z

- Zip actions
 - ZipOverwrite 427
 - ZipPassword 427, 428
 - ZipUnpack 427
 - ZipUnPack 428
- ZipOverwrite action 427
- ZipPassword action 427, 428
- ZipUnpack action 427
- ZipUnPack action 428
- Zone_Offset 307
- ZoneBOTTOM_ImageBottom action 988
- ZoneBOTTOM_LowerBound action 988
- ZoneBOTTOM_UpperBound action 988
- ZoneImage_SaveAs action 988, 1005
- ZoneLEFT_ImageLeft action 988, 1006
- ZoneLEFT_LeftBound action 988
- ZoneLEFT_RightBound action 988

- ZoneRIGHT_ImageRight action 988, 1008
- ZoneRIGHT_LeftBound action 988
- ZoneRIGHT_RightBound action 988
- Zones actions
 - AdjustZonesToImageOffset 988, 989
 - AnchorPage 988, 989
 - CalculateLocalOffset 988, 990
 - CreateBlockCCO 988, 990
 - FindBlocks_WhiteSpace 988, 991
 - FindDataBlocks 988, 991
 - FindLineItems 988, 992
 - FindRegExBlocks 988, 992
 - FindZoneLineItems 988, 993
 - GetZoneText 988, 994
 - InheritParentPosition 988, 994
 - LoadBlockCCO 988, 995
 - LoadZones 988, 995
 - MCCOPositionAdjust 988, 996
 - MergeZones 988, 997
 - PadZone 988, 997
 - PopulateZNField 988, 998
 - PopulateZNLineItemField 988, 998
 - ReadZones 988, 999
 - RegisterPage 988, 1000
 - ScanDetails 988, 1000
 - ScanDetailsByLines 988, 1001
 - ScanDetailsByVSpace 988, 1001
 - ScanLineItem 988, 1002
 - SetEOL 988, 1002
 - SetEOL_CRLF 988, 1003
 - ZoneBOTTOM_ImageBottom 1003
 - ZoneBOTTOM_LowerBound 1004
 - ZoneBOTTOM_UpperBound 1005
 - ZoneBOTTOM_ImageBottom 988
 - ZoneBOTTOM_LowerBound 988
 - ZoneBOTTOM_UpperBound 988
 - ZoneImage_SaveAs 988, 1005
 - ZoneLEFT_ImageLeft 988, 1006
 - ZoneLEFT_LeftBound 988, 1007
 - ZoneLEFT_RightBound 988, 1007
 - ZoneRIGHT_ImageRight 988, 1008
 - ZoneRIGHT_LeftBound 988, 1008
 - ZoneRIGHT_RightBound 988, 1009
 - ZoneTOP_ImageTop 988, 1009
 - ZoneTOP_LowerBound 988, 1010
 - ZoneTOP_UpperBound 988, 1010
- Zones tab 14
- ZoneTOP_ImageTop action 988, 1009
- ZoneTOP_LowerBound action 988, 1010
- ZoneTOP_UpperBound action 988, 1010
- ZZoneLEFT_LeftBound action 1007
- ZZoneLEFT_RightBound action 1007
- ZZoneRIGHT_LeftBound action 1008
- ZZoneRIGHT_RightBound action 1009



Product Number: 5725-C15

SC27-6375-00

